차 례

Proven C 라이브러리 완전 매뉴얼 (v0.92.0) 6
목차 6
저작권 6
따라 하며 익히기: 아홉 개의 짧은 프로그램 7
이 글이 있는 이유 7
목차 7
1과 — 출력, 그리고 되는 빌드 7
2과 — 자기 길이를 아는 텍스트 8
3과 — 실패할 수 있는 호출은 그렇다고 말한다 9
4과 — 값과 오류는 함께 온다 10
5과 — 기억을 누가 주는가는 인자다 11
6과 — 0장의 프로그램을 한 줄씩 12
7과 — 한꺼번에 풀려나는 기억 13
8과 — 컨테이너는 자기 할당자를 기억한다 14
9과 — 바깥 세상도 실패하고, 같은 방식으로 말한다 15
다음에 읽을 것 17
Chapter 0: 여기서부터 시작 18
목차 18
1. 이 매뉴얼은 누구를 위한 것인가 18
2. 이 라이브러리가 존재하는 이유 18
3. 첫 프로그램 21
4. 빌드와 include 22
5. 모든 페이지에서 만나게 될 다섯 가지 계약 23
6. 의도와 설계 철학 25
7. 빌드와 include 모델 25
8. 전역 계약(Global contracts) 26
9. 소유권과 파괴 매트릭스 27
10. 연산 동작 클래스 31
11. 매뉴얼 챕터 32
12. 플랫폼 지원과 검증 33
13. 부록 B: 용어집 34
14. 부록 C: 공개 헤더 맵 38
15. 부록 D: libc 대응표 41
16. 다음에 읽을 것 42
1장: 기초 — 에러, 타입, 메모리 view 43
목차 43
1. 에러는 값이다 43
2. 타입, 그리고 왜 표기가 다른가 48
3. 메모리 view: 포인터와 길이를 하나로 49
4. 감길 수 없는 크기 산술 51
5. 정렬 52
6. Panic: 에러를 돌려줄 상대가 남지 않았을 때 56
7. 버전 매크로 57
8. 예제와 오용 사례 57
2장: 할당 — 힙, 아레나, 풀, 버퍼 59
목차 59
1. 할당이 매개변수인 이유, 그리고 heap allocator 59
2. Arena: 여러 객체, 하나의 수명 60

3. Pool: 여러 객체, 하나의 크기 63
4. Allocator 트레잇 65
5. 원시 바이트 버퍼 66
6. 예제와 오용 사례 67
7. 잘못된 할당자 잡기: `alloc_check.h` 75
3장: 문자열, 포매팅, 스캐닝 79
목차 79
1. U8 문자열과 view 79
2. U16 문자열과 view 87
3. 포매팅 92
4. 스캐닝 95
5. 예제와 오용 사례 98
4장: 컨테이너와 알고리즘 108
목차 108
1. 동적 배열 108
2. 침습적(intrusive) 리스트 111
3. 링 버퍼 114
4. 해시 맵 116
5. 알고리즘 120
6. 용도별 해싱 121
7. 바이트를 텍스트로: hex와 Base64 129
8. 예제와 오용 사례 133
Chapter 8: Formatting and Scanning (v0.92.0) 144
목차 144
1. 설계 모델 144
2. 포매터 데이터 모델 145
3. 포매터 생성자와 선택자 147
4. 형식 문자열 문법 151
5. 형식화 API 153
6. 콘솔 출력 헬퍼 157
7. 스캐너 데이터 모델 157
8. 스캐너 기본 API 158
9. 스캔 인자 모델 159
10. 구조적 스캔 문법 159
11. 스캔 형식화 API 160
12. 예제와 오용 사례 161
13. 프리스탠딩과 빌드 모드 참고 166
Chapter 5: Hosted Services 175
목차 175
1. 파일시스템 API 175
2. 시스템 입출력과 환경변수 181
3. 메모리 매핑 183
4. 시간 API 184
5. 예제와 오용 사례 188
디렉터리를 한 번에 한 엔트리씩 읽기 192
트리 순회 193
스트림: writer와 reader 196
표준 스트림 200
UTF-16 텍스트 입출력과 윈도 콘솔 202

난수, 용도별 206
9장: 네트워킹 226
목차 226
1. 세 가지 규칙 226
2. 주소 227
3. 기한 229
4. TCP 230
5. UDP 233
6. 소켓 여럿, 스레드 하나 235
7. 전송 237
8. 짝, 그리고 루프 깨우기 240
9. selector: 소켓 수천 개 242
10. 플랫폼마다 다른 것 245
11. 여기에 없는 것 246
10장: URL과 HTTP 메시지 247
목차 247
1. 서버가 아니라 코덱 247
2. URL 248
3. 써도 안전한 경로 251
4. 헤드 파싱 252
5. 본문은 얼마나 긴가 253
6. 본문 디코딩 254
7. 메시지 쓰기 256
8. 날짜 259
9. 참조와 폼 259
10. 범위와 multipart 본문 260
11. 인증 261
12. 쿠키 264
13. 서버 전송 이벤트 265
14. 여기에 없는 것 267
11장: HTTP 클라이언트와 서버 268
목차 268
1. 이것이 무엇이고, 무엇 하나가 아닌가 268
2. 서버 268
3. 핸들러 안에서 271
4. 서버가 묻지 않고 하는 일 272
5. 핸들러를 돌리는 두 가지 방법 273
6. 클라이언트 276
7. 요청 하나, 응답 하나 278
8. 리다이렉트, 인증 요구, 쿠키 279
9. 연결과 프록시 280
10. TLS가 붙을 자리 281
11. 프로토콜 바꾸기 285
12. 여기에 없는 것 287
12장: WebSocket 289
목차 289
1. WebSocket이 무엇인지, 한 쪽으로 289
2. 연결 290
3. 보내기와 받기 291

4. 닫기, 그리고 닫히기 292
5. 서버의 연결은 어디에 사는가 293
6. 코덱: 핸드셰이크 296
7. 코덱: 프레임 297
8. 코덱: 디코더 298
9. 압축된 메시지 302
10. 여기에 없는 것 311
13장: 인증서와 신뢰 312
목차 312
1. 인증서가 무엇인지, 한 쪽으로 312
2. 하나 읽기 313
3. PEM 313
4. 어느 이름을 위한 것인가 314
5. 신뢰 앵커 314
6. 체인 검증 315
7. 피닝 318
8. 무엇을 검사하고, 무엇을 받아들이고, 무엇이 없는가 318
14장: TLS 320
목차 320
1. TLS가 하는 일, 한 쪽으로 320
2. 설정 321
3. HTTPS와 wss: 두 줄 323
4. 어떤 연결이든: 전송 래퍼 326
5. 엔진 326
6. 재개 331
7. 클라이언트 인증서와 핀 334
8. 실패할 때 335
9. 무엇을 협상하고, 무엇을 붙들고, 무엇이 없는가 336
10. 레거시 묶음: 설정 하나하나가 무엇에 노출시키는가 339
15장: 이벤트 루프와 이벤트 구동 서버 342
목차 342
1. 서비스하는 두 가지 방식 342
2. 루프 343
3. 타이머 344
4. 다른 곳에서 하는 일 344
5. 여러분 자신의 소켓 346
6. 이벤트 구동 서버 352
7. 답하기 353
8. 요청 본문 358
9. 한도, 시간, 거절 359
10. 스레드, TLS, 종료 360
11. 루프 위의 WebSocket 361
12. 이벤트 구동 클라이언트 373
13. 여러 루프 382
14. 연결 하나가 붙드는 것, 측정한 것, 그리고 여기 없는 것 385
16장: 압축 389
목차 389
1. DEFLATE가 하는 일, 한 쪽으로 389
2. 버퍼 전체 389

3. 스트림 391
4. 레벨, 윈도, 그리고 그 비용 394
5. 남이 보낸 데이터 395
6. HTTP 위에서 395
7. Zstandard 404
8. Brotli 410
9. 무엇을 받아들이고, 무엇이 없으며, 어떻게 시험했는가 413
17장: HTTP/2 - 프레임과 헤더 압축 416
목차 416
1. HTTP/2가 바꾸는 것, 한 쪽으로 416
2. 헤더 압축 417
3. 프레임 421
4. 메시지로서의 헤더 목록 426
5. 연결 426
6. 이벤트 구동 서버의 HTTP/2 435
7. 이벤트 구동 클라이언트의 HTTP/2 441
8. 무엇을 받아들이고, 무엇이 없으며, 어떻게 시험했는가 445
9. 이벤트 구동 서버의 HTTP/3 447
10. 이벤트 구동 클라이언트의 HTTP/3 449
6장: 실행, 별칭, PAL, Freestanding, 크로스 빌드 451
목차 451
1. 스택리스 코루틴 451
2. Job 시스템 452
3. 스레드, allocator, 포인터 프로버넌스 455
4. 별칭 계층 457
5. PAL 계약 458
6. Freestanding 부분집합 459
7. 크로스 컴파일 460
8. 예제와 오용 사례 461
Proven Freestanding 모드 (v0.92.0) 466
0. 이 모드가 존재하는 이유, 그리고 라이브러리 전체가 이 제약에 의해 빚어진 이유 466
1. 빌드 프로파일 467
2. 사용 가능한 소스 파일 467
3. 모듈 사용 가능성 468
4. 최소 정적 arena 설정 473
5. Panic 핸들러 오버라이드 474
6. Cortex-M 컴파일 명령 예제 475
7. RISC-V ELF 컴파일 명령 예제 475
8. Freestanding 모드에서의 포매팅 475
8a. Float 큰 정수 용량 조정 476
9. 호스티드 API 피하기 476
10. 수명 규칙은 여전히 적용된다 477
11. 검증 477
부록 A: `alias_xcv.h` Alias 인덱스 (7장) 479
사용 규칙 479
Alias 표 479

Proven C 라이브러리 완전 매뉴얼 (v0.92.0)

C23 시스템 라이브러리 `proven`의 매뉴얼입니다. 각 장은 `manual-ko/` 아래 파일 하나이며, 영문판(`manual/`)의 미러입니다. 코드와 API 계약을 빌드가 검증하는 정본은 영문판이고, 두 문서가 어긋나면 영문본이 기준입니다.

목차

- 따라 하며 익히기 — 짧은 프로그램 아홉 개
- 0장 — 여기서부터
- 1장 — 기초
- 2장 — 할당
- 3장 — 문자열과 텍스트
- 4장 — 컨테이너와 알고리즘
- 8장 — 형식화와 파싱
- 5장 — Hosted 서비스
- 9장 — 네트워킹
- 10장 — URL과 HTTP 메시지
- 11장 — HTTP 클라이언트와 서버
- 12장 — WebSocket
- 13장 — 인증서와 신뢰
- 14장 — TLS
- 15장 — 이벤트 루프와 이벤트 구동 서버
- 16장 — 압축
- 17장 — HTTP/2: 프레임과 헤더 압축
- 6장 — 실행과 플랫폼
- 프리스탠딩
- 부록 A — Alias 인덱스

저작권

- 지은이: rubidus — rubidus@gmail.com
- 저장소: github.com/rubidus-api/proven_c_lib
- 판: v0.92.0
- 라이선스: MIT — Copyright (c) 2026 rubidus-api. 라이브러리와 이 매뉴얼은 같은 라이선스를 따르며, 전문은 저장소 루트의 `LICENSE`에 있습니다.
- 이 매뉴얼의 모든 코드 예제는 빌드가 컴파일하고, 인용된 예제는 실행까지 합니다.

따라 하며 익히기: 아홉 개의 짧은 프로그램

Part I — 여기서 시작합니다. 입문용 C 책 한 권 외에 사전 지식은 필요 없습니다. **6과를 마치면** 0장의 인사말 프로그램을 한 줄씩 읽어 낼 수 있고, 레퍼런스 장들이 낯선 낱말의 벽으로 보이지 않게 됩니다. **9과를 마치면** 아레나, 자라는 컨테이너, 파일을 한 번씩 써 본 것이 됩니다 --- 텍스트를 다룰 줄 알게 된 프로그램이 다음으로 찾는 것들입니다.

이 글이 있는 이유

0장은 첫 쪽부터 도는 프로그램을 보여 줍니다. 그런데 그 서른 줄에는 새 개념이 다섯 개나 한꺼번에 들어 있습니다 — 인자로 건네는 할당자, 반드시 확인해야 하는 result, 자기 길이를 아는 view, 자르는 대신 거부하는 append, 만든 할당자로 짝지어 부르는 destroy. C 책 한 권을 막 뗐다면 다섯은 넷이 많습니다.

그래서 이 실습은 하나씩만 더합니다. 프로그램 아홉 개, 새로 나오는 것은 매번 딱 하나입니다. 앞의 여섯은 0장의 프로그램까지 이끌고, 뒤의 셋은 그 너머의 아레나·컨테이너·파일로 갑니다. 아홉 모두 `manual/examples/` 아래의 진짜 파일이고 빌드가 컴파일해 **실제로 돌립니다.** 그러니 여기 실린 것은 정말로 돌아간 코드입니다.

빌드는 여느 C 프로그램과 같습니다. 이 라이브러리는 여러분의 코드와 함께 컴파일하는 소스라서 설치할 것이 없습니다.

```
cc -std=c23 -Iinclude your_program.c src/proven/*.c platform/*.c -o your_program
```

이 줄이 낯설다면 0장 §4를 먼저 읽고 오세요.

목차

1. 1과 — 출력, 그리고 되는 빌드
2. 2과 — 자기 길이를 아는 텍스트
3. 3과 — 실패할 수 있는 호출은 그렇다고 말한다
4. 4과 — 값과 오류는 함께 온다
5. 5과 — 기억을 누가 주는가는 인자다
6. 6과 — 0장의 프로그램을 한 줄씩
7. 7과 — 한꺼번에 풀려나는 기억
8. 8과 — 컨테이너는 자기 할당자를 기억한다
9. 9과 — 바깥 세상도 실패하고, 같은 방식으로 말한다
10. 다음에 읽을 것

1과 — 출력, 그리고 되는 빌드

새로 나오는 것 하나: `proven_println`, 그리고 그 인자가 검사된다는 사실.

`printf("%d", 3.5)`는 컴파일됩니다. 하지만 틀렸고, 운이 나쁘면 쓰레기를 찍거나 죽습니다. 서식 문자열은 그냥 *문자열*이라, 뒤에 오는 인자와 이어 주는 것이 아무것도 없기 때문입니다. `proven_println`은 자리표로 `{}`를 쓰고 인자마다 `PROVEN_ARG`로 감쌉니다. 이 매크로가 인자의 타입을 함께 실어 보내므로, 서식과 인자가 어긋날 수가 없습니다.

```
/*
 * 1과 - 한 줄을 빌드해서 돌려 본다.
 *
 * 아직 이 라이브러리의 생각에 대한 것은 하나도 없다. 이 프로그램이 답하는 물음은
 * "내 빌드 명령이 되는가" 하나뿐이고, 그것만 따로 물어 볼 값어치가 있다. 뒤의
 * 모든 과가 그것을 전제하기 때문이다.
 *
 * printf(fmt, ...) 는 검사되지 않는다. "%d" 자리에 double 을 건넸다고 컴파일러가
 * 알려 주지 못한다. proven_println 은 검사한다. 인자마다 PROVEN_ARG 로 감싸
 * 자기 타입을 함께 들고 가기 때문이다.
 */

int main(void) {
```

```
    proven_println("hello from proven");

    /* {} 가 자리표다. 값은 PROVEN_ARG 를 거치면서 어떤 타입인지를 함께 싣는다.
     * 그래서 서식 문자열과 인자가 어긋날 수가 없다. */
    proven_println("one number: {}", PROVEN_ARG(42));
    proven_println("two of them: {} and {}", PROVEN_ARG(1), PROVEN_ARG(2));

    return EXAMPLE_OK();
}
```

돌리면 세 줄이 찍힙니다. 빌드되고 돌았다면 include 경로와 소스 목록이 맞다는 뜻이고, 이다음부터는 빌드가 아니라 라이브러리 이야기입니다.

처음에 잘 걸리는 것. `{}`는 `%s`가 아닙니다. 틀릴 글자가 아예 없고, `long`이냐 `int`냐를 기억할 필요도 없습니다 — `PROVEN_ARG`가 압니다.

2과 — 자기 길이를 아는 텍스트

새로 나오는 것 하나: 뷰(view) --- 포인터와 크기가 함께 다니는 것.

여러분이 아는 C에서 문자열은 포인터 하나이고, 길이는 「처음 0 바이트가 나오는 자리」입니다. 그것을 만지는 함수마다 얼마나 되는지 알아내려고 바이트를 훑어야 하고, 0이 없으면 끝을 넘어 훑습니다. 이 설계 하나가 이 언어의 보안 권고 상당수의 뿌리입니다.

view는 그 빠진 반쪽을 드러냅니다. 바이트를 소유하지 않고 빌려 쓰며, 아무것도 해제하지 않습니다.

```
/*
 * 2과 - 자기 길이를 아는 텍스트.
 *
 * 여러분이 아는 C 에서 문자열은 포인터 하나이고, 길이는 "처음 0 바이트가 나오는
 * 자리"다. 함수마다 얼마나 되는지 알아내려고 바이트를 훑어야 하고, 0 이 없으면
 * 끝을 넘어 훑는다.
 *
 * 뷰(view)는 C 가 암묵으로 남겨 둔 그 짝이다. 포인터와 크기가 함께 다닌다.
 * 빌려 쓸 뿐이라 바이트를 소유하지 않고 해제하지도 않는다.
 */

int main(void) {
    /* PROVEN_LIT 은 리터럴에서 뷰를 만든다. 크기는 컴파일 때 정해지므로 여기서
     * 훑는 일은 없다 - strlen 과 다른 점이다. */
    proven_u8str_view_t hello = PROVEN_LIT("hello");

    EXAMPLE_REQUIRE(hello.size == 5, "the view already knows its own length");
    EXAMPLE_REQUIRE(hello.ptr != NULL, "and it points at the literal's bytes");

    /* 길이가 포인터와 함께 다니므로, 비교는 크기 검사에 memcmp 하나다.
     * 훑을 일도 없고 끝을 넘어갈 길도 없다. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(hello, PROVEN_LIT("hello")), "same text");
    EXAMPLE_REQUIRE(!proven_u8str_view_eq(hello, PROVEN_LIT("hell")), "shorter text differs");

    /* 뷰는 무언가의 *일부*를 복사 없이 가리킬 수 있다. 아래는 리터럴의 가운데다 -
     * 여전히 빌린 것이고, 할당은 없다. */
    proven_u8str_view_t ell = { .ptr = hello.ptr + 1, .size = 3 };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(ell, PROVEN_LIT("ell")), "a window onto the same bytes");

    proven_println("whole: {} (size {})", PROVEN_ARG(hello), PROVEN_ARG(hello.size));
    proven_println("part : {} (size {})", PROVEN_ARG(ell), PROVEN_ARG(ell.size));

    return EXAMPLE_OK();
}
```

눈여겨볼 것. `PROVEN_LIT`은 크기를 컴파일 때 정하므로 `strlen`이 아닙니다. 그리고 세 번째 view는 리터럴의 *가운데*를 복사 없이 가리킵니다 --- view가 무언가의 일부를 가리킬 수 있다는 것, 이것이 이 라이브러리의 자르기와 파싱이 할당을 하지 않는 이유입니다.

처음에 잘 걸리는 것. view는 빌린 것입니다. 가리키던 바이트가 사라지면 --- 스코프를 벗어난 지역 버퍼, 이미 destroy 한 문자열 --- 그 view는 여느 C 포인터와 똑같이 매달린 포인터가 됩니다. 이 라이브러리는 몰래 수명을 늘려 주지 않습니다.

3과 — 실패할 수 있는 호출은 그렇다고 말한다

새로 나오는 것 하나: `proven_err_t`, 그리고 자르는 대신 거부하기.

C는 실패를 서로 무관한 세 방식으로 알립니다. 특별한 반환값, 널 포인터, 그리고 다음 호출이 덮어 쓰는 전역 `errno`. 셋 다 안 보기 쉽고, 안 봐도 아무도 뭐라 하지 않습니다. 여기서는 실패할 수 있는 호출이 오류를 *값*으로 돌려주고, 그 함수들에는 `[[nodiscard]]`가 붙어 있어 버리면 습관이 아니라 컴파일러 경고가 됩니다.

```
/*
 * 3과 - 실패할 수 있는 호출은 반환값으로 그렇다고 말한다.
 *
 * 여러분이 아는 C 는 실패를 세 가지 방식으로 알린다. 특별한 반환값(-1), 널
 * 포인터, 그리고 다음 호출이 덮어써 버리는 errno 라는 전역. 셋 다 안 보기 쉽고,
 * 안 봐도 아무도 뭐라 하지 않는다.
 *
 * 여기서는 실패할 수 있는 호출이 proven_err_t 를 돌려준다. 그냥 값이라서 담아
 * 둘 수도, 비교할 수도, 위로 넘길 수도 있다. 그리고 이 함수들에 붙은
 * [[nodiscard]] 덕분에 무시하는 것은 습관이 아니라 컴파일러 경고가 된다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 딱 8 바이트짜리 자리. (그 자리가 어디서 오는지는 2장의 주제다. 지금은
     * 우리가 "얼마나"를 말했다는 것만 보면 된다.) */
    proven_result_u8str_t s = proven_u8str_create(alloc, 8);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "8 bytes should be available");

    /* 이건 들어간다. */
    proven_err_t err = proven_u8str_append(&s.value, PROVEN_LIT("12345678"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "eight bytes into eight bytes fits exactly");

    /* 이건 안 들어간다 - 그리고 라이브러리는 *거부*한다. 들어갈 만큼만 붙이지
     * 않는다. 잘린 낱말은 짧아진 낱말이 아니라 다른 낱말이기 때문이다. */
    proven_err_t too_much = proven_u8str_append(&s.value, PROVEN_LIT("9"));
    EXAMPLE_REQUIRE(!proven_is_ok(too_much), "one byte more than capacity must fail");
    EXAMPLE_REQUIRE(too_much == PROVEN_ERR_OUT_OF_BOUNDS, "and it says why: out of bounds");

    /* 거부는 아무것도 바꾸지 않았다. 문자열은 그대로다. 이것이 "실패 원자성"의
     * 뜻이다 - 실패한 호출은 반쯤 된 상태를 남기지 않는다. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("12345678")),
                    "the refused append must not have written anything");

    proven_println("after the refusal, the string is still: {}",
                   PROVEN_ARG(proven_u8str_as_view(&s.value)));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

눈여겨볼 것. 들어가지 못한 append는 아무것도 바꾸지 않습니다. 들어갈 만큼만 넣지도 않습니다. 이것을 실패 원자성이라고 하는데, 이유는 잘린 문자열이 짧아진 메시지가 아니라 다른 메시지이기 때문입니다 --- 잘린 경로는 다른 파일을 가리키고, 잘린 명령은 다른 명령입니다.

처음에 잘 걸리는 것. 검사는 `proven_is_ok(err)`이고 `err == PROVEN_OK`도 같은 말입니다. 하면 안 되는 것은 그것을 무시하고 값을 읽는 일입니다.

4과 — 값과 오류는 함께 온다

새로 나오는 것 하나: result 구조체 --- `.err`와 `.value`가 한 번에 돌아온다.

돌려줄 값이 없는 호출에는 오류 코드만으로 충분합니다. 돌려줄 값이 있으면 라이브러리는 둘을 작은 구조체 하나에 담아 줍니다. 규칙은 한 문장입니다. **`err`를 보기 전에는 `value`는 아무 뜻도 없다.**

```
/*
 * 4과 - 돌려줄 값이 있는 호출은, 값과 오류가 함께 온다.
 *
 * 돌려줄 것이 없을 때는 proven_err_t 하나로 충분하다. 돌려줄 것이 *있으면*
 * 필드 둘짜리 작은 구조체를 받는다. `err` 와 `value` 다. 규칙은 한 문장이다.
 * `err` 를 보기 전에는 `value` 는 아무 뜻도 없다.
 *
 * malloc 이 NULL 인지 확인하던 것과 같은 규율인데, 확인해야 한다는 사실이
 * 기억해야 할 관습이 아니라 타입의 일부라는 점이 다르다.
 */

/* 여러분의 함수도 이것을 돌려줄 수 있다. 라이브러리에 마법은 없다. */
static proven_result_size_t safe_div(proven_size_t a, proven_size_t b) {
    proven_result_size_t res = {0};
    if (b == 0) {
        res.err = PROVEN_ERR_INVALID_ARG;    /* value 는 0 그대로 - 아무 뜻도 없다 */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = a / b;
    return res;
}

int main(void) {
    proven_result_size_t ok = safe_div(10, 2);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "dividing by 2 is fine");
    EXAMPLE_REQUIRE(ok.value == 5, "and only now may we read the value");

    proven_result_size_t bad = safe_div(10, 0);
    EXAMPLE_REQUIRE(!proven_is_ok(bad.err), "dividing by zero must fail");
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "and say which rule was broken");
    /* bad.value 는 0 이지만 그것은 답이 아니다. 답이 없다는 뜻이다. */

    /* 라이브러리 자신의 호출도 같은 모양이다. proven_u8str_create 는 만든
     * 문자열과 그것을 지키는 오류를 함께 돌려준다. */
    proven_allocator_t alloc = proven_heap_allocator();
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    if (!proven_is_ok(s.err)) return 1;      /* 만들어진 것이 없으니 지울 것도 없다 */

    proven_println("10 / 2 = {}", PROVEN_ARG(ok.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

눈여겨볼 것. `safe_div`는 라이브러리 함수가 아니라 여러분의 함수입니다. 이 방식은 그냥 구조체를 돌려주는 것일 뿐, 밑에 깔린 장치가 없습니다. 그래서 이 쪽을 읽은 순간부터 여러분 코드에 그대로 쓸 수 있습니다.

처음에 잘 걸리는 것. 실패했을 때 `value` 자리에는 보통 0이 들어 있습니다. 그 0은 답이 아니라 *답이 없다는 것*입니다. 둘을 가르는 것이 `err` 검사입니다.

5과 — 기억을 누가 주는가는 인자다

새로 나오는 것 하나: `proven_allocator_t` --- 가정하지 않고 건네받는다.

`malloc`은 여러분 대신 내려진 결정입니다. 힙 하나, 전략 하나, 그리고 호출 자리에는 그런 티가 나지 않습니다. 이 라이브러리에서는 기억이 필요한 것이면 무엇이든 할당자를 인자로 받고 그것만 씁니다. 따라 나오는 둘이 요점입니다 --- 「이건 누가 할당했지?」에 호출을 읽어 답할 수 있고, 같은 코드에 다른 할당자를 건네도 코드는 그대로입니다.

```c
/*
 * 5과 - 기억을 누가 주는가는 전역이 아니라 인자다.
 *
 * malloc 은 여러분 대신 내려진 전역 결정이다. 힙 하나, 전략 하나, 그리고 호출
 * 자리에는 보이지 않는다. 여기서는 기억이 필요한 것이면 무엇이든
 * proven_allocator_t 를 받아 그것만 쓴다. 따라 나오는 둘이 요점이다.
 *
 *   - "이건 누가 할당했지?" 에 호출을 보고 언제나 답할 수 있다.
 *   - 같은 코드에 다른 할당자를 건네도 코드는 그대로다.
 */

/* 이 함수는 기억이 어디서 오는지 알지도, 신경 쓰지도 않는다. */
static proven_result_u8str_t make_greeting(proven_allocator_t alloc,
                                           proven_u8str_view_t name) {
    proven_result_u8str_t out = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(out.err)) return out;

    proven_err_t err = proven_u8str_append(&out.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&out.value, name);
    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &out.value);   /* 만든 것을 되돌린다 */
        out.err = err;
        out.value = (proven_u8str_t){0};
    }
    return out;
}

int main(void) {
    /* (a) 보통의 힙 - 밑에서 malloc 과 free 가 돈다 */
    proven_allocator_t heap = proven_heap_allocator();

    proven_result_u8str_t a = make_greeting(heap, PROVEN_LIT("world"));
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "the heap should be able to give 64 bytes");
    proven_println("from the heap : {}", PROVEN_ARG(proven_u8str_as_view(&a.value)));
    /* 만든 것과 *같은* 할당자로 지운다. 그 짝이 이 라이브러리 소유 규칙의
     * 전부다. */
    proven_u8str_destroy(heap, &a.value);

    /* (b) 아레나 - 기억 덩이 하나를 차례로 나눠 주고 한꺼번에 되돌린다.
     *     위의 make_greeting 은 조금도 바뀌지 않았다는 데 주목할 것. */
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[512];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t from_arena = proven_arena_as_allocator(&arena);
```

```
    proven_result_u8str_t b = make_greeting(from_arena, PROVEN_LIT("arena"));
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "the arena has room for this too");
    proven_println("from an arena : {}", PROVEN_ARG(proven_u8str_as_view(&b.value)));

    /* 여기엔 지우는 반복문이 없다. 아레나는 reset 으로 전부 되돌린다. 그 이야기는
     * 2장의 몫이고, 지금의 요점은 *부르는 쪽이 골랐다*는 것뿐이다. */
    proven_arena_reset(&arena);

    return EXAMPLE_OK();
}
```

눈여겨볼 것. `make_greeting`은 한 번 쓰였고 힙에서도, 아레나에서도 돕니다. 아레나는 여러분이 가진 기억 덩이 하나를 포인터를 밀어 가며 나눠 주고 한꺼번에 되돌리는 방식입니다. 함수는 바뀌지 않았고, 전역으로 설정한 것도 없습니다.

소유 규칙 한 줄: 만든 쪽이 지운다, 그것도 같은 할당자로.

처음에 잘 걸리는 것. 아레나에서는 `destroy`가 아무것도 되돌리지 않습니다 --- 아레나는 reset 으로 해제합니다. 그래도 부르세요. 그 짝이 있어야 나중에 할당자를 바꿔도 코드가 옳습니다.

6과 — 0장의 프로그램을 한 줄씩

새로 나오는 것 하나: 없습니다. 그것이 이 과제입니다.

```
/*
 * 6과 - 0장의 프로그램을 한 줄씩 읽는다.
 *
 * 새로 나오는 것은 없다. 매뉴얼이 첫 쪽에 싣는 그 인사말 프로그램 그대로이고,
 * 이 과의 요점은 이제 그 모든 부분에 이름을 댈 수 있다는 것이다. 건네받는
 * 할당자(5과), 반드시 확인해야 하는 result(4과), 거부가 돌려주는 오류(3과),
 * 자기 길이를 들고 다니는 뷰(2과), 그리고 인자를 검사하는 출력(1과).
 *
 * 이것이 이제 평범하게 읽힌다면 이 실습은 할 일을 다한 것이고, 레퍼런스 장들이
 * 여러분에게 열린 것이다.
 */

int main(void) {
    /* 5과 - 기억이 어디서 오는지는 부르는 쪽이 정한다 */
    proven_allocator_t alloc = proven_heap_allocator();

    /* 4과 - 문자열과 그것을 지키는 오류가 함께 온다 */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
    if (!proven_is_ok(greeting.err)) return 1;

    /* 2과 - 자기 크기를 아는, 빌려 쓰는 텍스트 */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* 3과 - 각 append 는 들어가거나 거부한다. 자르는 것은 하나도 없다 */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
                                         PROVEN_LIT("hello, world!")),
```

```
                "the three appends should have built the whole greeting");

    /* 1과 - 서식 문자열과 인자는 어긋날 수 없다 */
    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

    /* 다시 5과 - 만든 할당자로 지운다 */
    proven_u8str_destroy(alloc, &greeting.value);
    return EXAMPLE_OK();
}
```

저 주석들이 새 정보가 아니라 *아는 것에 붙은 이름표*로 읽힌다면, 이제 레퍼런스 장을 펼칠 준비가 된 것입니다. 다음 세 과는 그 장들로 들어가는 첫걸음 --- 아레나, 컨테이너, 파일 --- 이고, 모양은 같습니다. 과마다 새 생각 하나.

7과 — 한꺼번에 풀려나는 기억

새로 나오는 것 하나: `proven_arena_reset` --- 아레나(arena)가 나눠 준 것을 호출 하나로 전부 돌려받는다.

5과는 `make_greeting`에 아레나를 건네고 끝에서 reset 했지만, 그 reset 이 무엇을 하는지는 말하지 않았습니다. 생각은 이것이 전부입니다. 아레나는 여러분이 가진 기억 덩어리를 포인터를 앞으로 밀며 조각조각 나눠 줍니다. 조각 하나를 돌려받는 방법은 없습니다 --- 아레나에 `destroy`를 불러도 돌아오는 것이 없습니다. reset 하면 *전부*가 한꺼번에 돌아옵니다.

이것은 대부분의 프로그램이 가진 모양에 맞습니다. 바퀴마다 임시 글을 좀 만들어 쓰고, 쓰고 나면 끝나는 루프입니다. 바퀴 안에서는 마음껏 할당하고 끝에서 reset 하면, 따라다닐 것도 샐 것도 없습니다.

```
/*
 * 7과 - 한꺼번에 풀려나는 기억.
 *
 * 5과는 make_greeting 에 아레나를 건넨 뒤 reset 했지만, 그 reset 이 무엇인지는
 * 말하지 않았다. 이번 과는 그 호출 하나에 대한 것이다.
 *
 * 아레나는 여러분이 가진 기억 덩어리 위에서 포인터를 앞으로 밀어 가며 나눠 준다.
 * 객체 하나하나를 푸는 free 는 없다. 아레나에 destroy 를 불러도 돌아오는 것은
 * 없다. 모든 것이 proven_arena_reset 한 번에 한꺼번에 돌아온다. 제약처럼 들리지만
 * 그것이 아레나를 쓰는 이유다. 일감마다 임시 글을 만드는 루프는 문자열을 몇 개를
 * 만들었든 문장 하나로 전부 버릴 수 있다.
 */

int main(void) {
    /* 이 배열의 주인은 아레나가 아니라 우리다. 아레나는 조각을 나눠 줄 뿐이다. */
    alignas(PROVEN_MAX_ALIGN) proven_byte_t backing[256];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = backing, .size = sizeof backing });
    proven_allocator_t scratch = proven_arena_as_allocator(&arena);

    static const proven_u8str_view_t names[] = {
        PROVEN_LIT("ada"), PROVEN_LIT("grace"), PROVEN_LIT("barbara"),
    };

    for (proven_size_t i = 0; i < sizeof names / sizeof names[0]; ++i) {
        /* 한 바퀴에 문자열 둘, 둘 다 아레나에서 나온다. */
        proven_result_u8str_t line = proven_u8str_create(scratch, 32);
        proven_result_u8str_t note = proven_u8str_create(scratch, 32);
        EXAMPLE_REQUIRE(proven_is_ok(line.err) && proven_is_ok(note.err),
                        "each round fits easily in 256 bytes");
        if (!proven_is_ok(line.err) || !proven_is_ok(note.err)) return 1;

        proven_err_t err = proven_u8str_append(&line.value, PROVEN_LIT("hello, "));
        if (proven_is_ok(err)) err = proven_u8str_append(&line.value, names[i]);
```

```
        if (proven_is_ok(err)) err = proven_u8str_append(&note.value, PROVEN_LIT("(scratch)"));
        EXAMPLE_REQUIRE(proven_is_ok(err), "the appends fit their capacity");

        /* 바퀴마다 첫 할당은 `backing` 의 맨 앞에 떨어진다. 앞 바퀴 끝의
         * reset 이 전부 돌려주었기 때문이다. */
        EXAMPLE_REQUIRE((const void *)proven_u8str_as_view(&line.value).ptr == (const void *)backing,
                        "each round starts again at the beginning of the block");

        proven_println("round {}: {} {} ({} bytes in use)",
                       PROVEN_ARG(i), PROVEN_ARG(proven_u8str_as_view(&line.value)),
                       PROVEN_ARG(proven_u8str_as_view(&note.value)),
                       PROVEN_ARG(arena.offset));

        /* 문장 하나로 두 문자열이 사라진다. destroy 루프도, 잊을 것도 없다.
         * 여기서부터 `line` 과 `note` 는 회수된 기억을 가리킨다. */
        proven_arena_reset(&arena);
        EXAMPLE_REQUIRE(arena.offset == 0, "reset reclaims everything the round allocated");
    }

    /* 아레나는 자라지 않는다. 덩어리보다 많이 달라고 하면 3과에서 본 것과 같은
     * 오류 값이 온다. 죽지도 않고, 몰래 malloc 으로 넘어가지도 않는다. */
    proven_result_u8str_t too_big = proven_u8str_create(scratch, 1024);
    EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "a 256-byte arena refuses 1 KiB");

    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

눈여겨볼 것. 바퀴마다 쓰는 바이트 수가 같고, 바퀴마다 첫 문자열이 `backing`의 맨 앞에 떨어집니다 --- 프로그램이 그것을 확인합니다. reset 은 문자열 두 개를 푼 것이 아니라 숫자 하나를 0으로 되돌렸을 뿐이고, 그래서 한 바퀴에 문자열을 둘 만들든 이천 개 만들든 값이 같습니다. 그리고 아레나가 너무 작으면 답은 `PROVEN_ERR_NOMEM`, 3과와 같은 오류 값이지 죽는 것이 아닙니다.

처음에 잘 걸리는 것. reset 뒤에도 `line`과 `note`는 옛 포인터를 그대로 들고 있고, 그 포인터는 이제 다음 바퀴가 덮어쓸 기억을 가리킵니다. reset 은 아레나가 내준 모든 포인터를 한꺼번에 허공을 가리키게 만듭니다. 아레나 밖으로 복사해 두지 않은 것은 바퀴 너머로 들고 가지 마세요.

8과 — 컨테이너는 자기 할당자를 기억한다

새로 나오는 것 하나: 컨테이너는 할당자(allocator)를 만들 때 한 번 받아서 간직한다.

3과의 문자열은 자리가 정해져 있었고, 할당자를 `create`에도 `destroy`에도 건넸습니다. 자라는 배열은 그렇게 할 수 없습니다. 자리를 넘치는 push 는 호출자가 계획하지 않은 순간에 다시 할당해야 하기 때문입니다. 그래서 배열은 만들 때 받은 할당자를 간직합니다. 한 번 건네고 나면 push, get, destroy 는 배열만 받습니다.

```
/*
 * 8과 - 컨테이너는 자기 할당자를 기억한다.
 *
 * 문자열은 할당자로 만들고 같은 할당자로 없앤다. 두 번 다 여러분이 건넨다.
 * 자라는 배열은 push 가 저장 공간을 넘칠 때마다 나중에 다시 할당해야 하므로,
 * 만들 때 받은 할당자를 간직한다. 새로 나오는 것은 그 하나다. 할당자는 만들 때
 * 한 번 건네고, 그 뒤의 push, get, destroy 는 배열 말고는 아무것도 받지 않는다.
 *
 * PROVEN_ARRAY_* 매크로는 원소 타입을 인자로 받아 값을 그 타입으로 만든다.
 * 그래서 그 타입으로 바뀔 수 없는 값 - int 배열에 push 하는 구조체 - 은
 * 컴파일되지 않는다. 적어 준 타입이 배열을 만들 때의 타입과 같은지는 알지
 * 못한다. 매번 같은 타입을 적는 것은 여러분 몫이다.
 */

int main(void) {
```

```
    proven_allocator_t alloc = proven_heap_allocator();

    /* 2 는 처음 용량이지 한도가 아니다. */
    proven_result_array_t made = PROVEN_ARRAY_INIT(alloc, int, 2);
    EXAMPLE_REQUIRE(proven_is_ok(made.err), "creating an empty array must succeed");
    if (!proven_is_ok(made.err)) return 1;
    proven_array_t squares = made.value;

    /* 둘 들어갈 자리에 열 번 push 한다. 들어가지 않는 push 마다 배열이 기억한
     * 할당자로 저장 공간을 늘린다. 여기에는 할당자가 없다. */
    for (int i = 1; i <= 10; ++i) {
        proven_err_t err = PROVEN_ARRAY_PUSH(&squares, int, i * i);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the heap can grow a ten-int array");
        if (!proven_is_ok(err)) {
            PROVEN_ARRAY_DESTROY(&squares);
            return 1;
        }
    }
    EXAMPLE_REQUIRE(squares.len == 10, "ten pushes, ten elements");

    /* get 은 원소를 가리키는 포인터를 돌려주고, 인덱스가 끝을 넘으면 NULL 을
     * 돌려준다. 범위를 벗어난 인덱스는 엉뚱한 곳 읽기가 아니라 검사할 수 있는 답이다. */
    const int *third = PROVEN_ARRAY_GET(&squares, int, 2);
    EXAMPLE_REQUIRE(third && *third == 9, "element 2 is 3 * 3");
    EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&squares, int, 10) == NULL, "index 10 is past the end");

    int sum = 0;
    for (proven_size_t i = 0; i < squares.len; ++i) {
        sum += *PROVEN_ARRAY_GET(&squares, int, i);
    }
    EXAMPLE_REQUIRE(sum == 385, "1 + 4 + 9 + ... + 100");
    proven_println("{} squares, sum {}", PROVEN_ARG(squares.len), PROVEN_ARG(sum));

    /* destroy 는 배열만 받는다. 간직해 둔 할당자로 풀어 준다. */
    PROVEN_ARRAY_DESTROY(&squares);
    return EXAMPLE_OK();
}
```

눈여겨볼 것. 배열은 둘 들어갈 자리로 만들어져 push 를 열 번 받았는데, 그 어디에도 할당자가 나오지 않습니다. 끝을 넘는 `PROVEN_ARRAY_GET`은 `NULL`입니다 --- 저장 공간 너머에 무엇이 있든 읽어 버리는 것이 아니라, 확인할 수 있는 답입니다. 매크로가 원소 타입(`int`)을 인자로 받으므로 그 타입으로 바뀔 수 없는 값(구조체 같은)은 컴파일되지 않습니다. 배열을 만들 때와 같은 타입을 적는 것은 여전히 여러분 몫입니다.

처음에 잘 걸리는 것. `PROVEN_ARRAY_GET`이 돌려주는 포인터는 배열의 저장 공간 *안*을 가리킵니다. 다음 push 가 저장 공간을 더 큰 덩어리로 옮길 수 있고, 그러면 옛 포인터는 허공을 가리킵니다. 꺼내서 쓰고, push 너머로 들고 가지 마세요. 4장에 이 실수가 통째로 나옵니다. 7과의 걸림돌이 같은 이유로 다시 나온 것입니다 --- 그 기억이 언제 옮겨질지는 여러분의 포인터가 아닌 다른 것이 정합니다.

9과 — 바깥 세상도 실패하고, 같은 방식으로 말한다

새로 나오는 것 하나: 여러분 프로그램 탓이 아닌 실패도 같은 오류 값으로 온다.

지금까지의 실패는 모두 프로그램 안에 원인이 있었습니다 --- 자리가 모자라거나 인자가 틀렸거나. 파일은 바깥의 실패를 데려옵니다. 없는 이름, 쓸 권한이 없는 디렉터리, 가득 찬 디스크. 어느 것도 버그가 아니고, 모두 멀쩡한 프로그램에게 일어납니다. 라이브러리는 이것들도 앞에서와 같은 `proven_err_t` 값으로 알리므로, 새로 배울 두 번째 오류 체계가 없습니다.

쓰는 호출은 통파일 호출입니다. `proven_fs_write_file_atomic`은 호출 하나로 파일을 쓰고, `proven_fs_read_all_u8str`은 파일을 소유하는 문자열로 되읽습니다. open 도, 읽기 루프도, 잊어버릴 close 도 없습니다.

```
/*
 * 9과 - 바깥 세상도 실패하고, 같은 방식으로 그렇다고 말한다.
 *
 * 지금까지는 프로그램 안의 이유로만 실패할 수 있었다. 자리가 모자라거나 인자가
 * 틀렸거나. 파일은 없을 수도, 읽을 수 없을 수도, 디스크가 가득 찼을 수도 있고,
 * 그 어느 것도 여러분 코드의 버그가 아니다. 라이브러리는 그런 실패도 앞에서와
 * 같은 오류 값으로 알린다. 그러니 이미 배운 검사가 검사의 전부다.
 *
 * 여기 쓰는 것은 통파일 호출이다. 호출 하나로 파일을 쓰고, 하나로 되읽는다.
 * open 도, 읽기 루프도, 잊어버릴 close 도 없다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 현재 디렉터리의 이름 하나. 프로그램은 끝나기 전에 이것을 지운다. */
    proven_u8str_view_t path = PROVEN_LIT("proven_tutorial_notes.tmp");
    proven_u8str_view_t text = PROVEN_LIT("buy milk\ncall home\n");

    /* 원자적이다. 읽는 쪽은 옛 파일이나 새 파일을 보지, 반쪽을 보지 않는다.
     * 할당자는 호출이 경로를 다루며 쓸 수 있는 임시 작업 공간이다. */
    proven_err_t err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing a small file in the current directory should work");
    if (!proven_is_ok(err)) return 1;

    /* 읽기는 소유하는 문자열을 돌려준다. 4과의 모양, 5과의 규칙 그대로다.
     * `alloc` 으로 만들어졌으니 `alloc` 으로 없앤다. */
    proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(back.err), "the file we just wrote can be read");
    if (proven_is_ok(back.err)) {
        EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&back.value), text),
                        "the bytes come back exactly as written");
        proven_println("read {} bytes back", PROVEN_ARG(proven_u8str_as_view(&back.value).size));
        proven_u8str_destroy(alloc, &back.value);
    }

    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the file we created should work");

    /* 이제 파일은 없고, 그것을 읽는 것은 프로그램 바깥에서 온 실패다.
     * 그 실패도 보통 값으로 오고, 어떤 실패인지 말해 준다. */
    proven_result_u8str_t missing = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(missing.err == PROVEN_ERR_NOT_FOUND, "a removed file reads as NOT_FOUND");
    if (proven_is_ok(missing.err)) proven_u8str_destroy(alloc, &missing.value);
    proven_println("reading it again: not found, as expected");

    return EXAMPLE_OK();
}
```

눈여겨볼 것. `proven_fs_read_all_u8str`이 돌려주는 문자열은 소유하는 것입니다. `alloc`으로 만들고 `alloc`으로 없앱니다 --- 5과의 규칙 그대로입니다. 지운 파일을 다시 읽으면 `PROVEN_ERR_NOT_FOUND`가 옵니다. 3과의 거부와 같은 종류의 값이고, 무엇이 잘못됐는지 이름을 댑니다. 쓰기 이름의 *atomic(원자적)*은 읽는 쪽이 옛 파일이나 새 파일을 보지 둘이 섞인 것은 보지 않는다는 뜻입니다. 무엇을 약속하지 않는지(예컨대 정전을 견디는 것)는 5장에 있습니다.

처음에 잘 걸리는 것. 쓰기와 `proven_fs_remove`에 건네는 할당자는 경로를 다루는 데 쓰는 임시 작업 공간이지 무언가의 주인이 아닙니다. 그 호출들은 가져간 것을 돌아오기 전에 다 돌려줍니다. destroy 할 것이 생기는 것은 문자열을 건네주는 읽기뿐입니다.

다음에 읽을 것

알고 싶은 것	읽을 곳
위 내용의 완전한 판	0장
타입·오류·계약의 자세한 것	1장
아레나·풀·버퍼 제대로	2장
소유하는 문자열, 자르기, 인코딩	3장
배열·맵·리스트·해시	4장
파일·스트림·시간·난수	5장
모르는 낱말	용어집

Chapter 0: 여기서부터 시작

Part I — 여기서 시작합니다. 입문용 C 책 한 권 외에 사전 지식은 필요 없습니다. **이 장을 마치면** `proven`을 링크한 프로그램을 빌드할 수 있고, 다른 어떤 장이든 읽을 수 있으며, 모르는 용어를 찾아볼 수 있습니다.

> **문체에 대해.** 이 장만 경어체(합니다체)로 쓰였고, 1장부터의 레퍼런스 장들은 평서체(해라체)를 씁니다. 의도한 것입니다 — 0장은 독자에게 직접 말을 거는 입문 장이고, 나머지는 찾아보는 문서이기 때문입니다.

목차

1. 이 매뉴얼은 누구를 위한 것인가
2. 이 라이브러리가 존재하는 이유
3. 첫 프로그램
4. 빌드와 include
5. 모든 페이지에서 만나게 될 다섯 가지 계약
6. 의도와 설계 철학
7. 빌드와 include 모델
8. 전역 계약(Global contracts)
9. 소유권과 파괴 매트릭스
10. 연산 동작 클래스
11. 매뉴얼 챕터
12. 플랫폼 지원과 검증
13. 부록 B: 용어집
14. 부록 C: 공개 헤더 맵
15. 부록 D: libc 대응표
16. 다음에 읽을 것

---

1. 이 매뉴얼은 누구를 위한 것인가

이 매뉴얼은 여러분이 입문용 C 책을 한 권 끝냈다고 가정합니다. 구체적으로는 변수와 제어 흐름, 함수, 배열, `struct`, 포인터와 `*`/`&`, `malloc`과 `free`, `printf`, `strlen`을 쓰는 `char *` 문자열, 그리고 `gcc main.c -o main`으로 프로그램을 컴파일하는 것에 익숙하다고 봅니다. 이것들이 모두 익숙하다면 충분합니다.

반대로 다음은 **가정하지 않습니다**: 규율로서의 ownership(소유), 빌려 쓰는(borrowed) 데이터와 소유(owned) 데이터의 차이, 아레나(arena)나 풀(pool), 인터페이스로 쓰이는 함수 포인터 테이블, C23 어트리뷰트, 컴파일러가 적극적으로 이용해 먹는 대상으로서의 미정의 동작(undefined behaviour), "그냥 되던데" 수준을 넘어선 정렬 (alignment), atomic, 그리고 라이브러리가 최선을 다하는 대신 연산을 *거부*할 수도 있다는 발상. 이 하나하나는 처음 등장하는 자리에서 설명하며, 전부 §13의 용어집에도 있습니다.

이 문서는 C 튜토리얼이 아닙니다. 포인터가 무엇인지는 설명하지 않습니다. 대신 이 라이브러리가 왜 `errno`를 설정하는 대신 에러가 담긴 `struct`를 돌려주는지는 길게 설명합니다. 그것은 여러분이 동의하지 않을 권리가 있는 설계 결정이고, 아무도 설명해 주지 않은 결정에는 반대할 수조차 없기 때문입니다.

---

2. 이 라이브러리가 존재하는 이유

C는 여러분에게 거의 아무것도 주지 않으면서 여러분을 완전히 믿습니다. 그것이 C의 큰 장점입니다 — 런타임도 없고, 숨은 할당도 없고, 여러분이 쓰지 않은 비용도 없습니다 — 그리고 그래서 C는 여전히 운영체제와 임베디드 장치, 작고 예측 가능해야 하는 모든 것의 언어입니다.

그리고 같은 다섯 가지 버그가 50년째 출하되고 있는 이유이기도 합니다. 이 라이브러리는 그 다섯 가지 버그에 대한 답의 모음입니다. 각 답에는 대가가 따르고, 이 절은 그 대가가 무엇인지 말합니다.

문자열 함수는 무엇이 얼마나 큰지 모릅니다

```
char buf[64];
strcpy(buf, name);            /* wrong: how long is name? strcpy never asks */
strcat(buf, ", welcome!");    /* wrong: and how much room is left now? */
```

`strcpy`는 목적지 포인터와 원본 포인터를 받습니다. 그 시그니처 어디에도 목적지의 크기가 실려 있지 않으므로, 아무것도 그것을 검사할 수 없습니다. 이 함수는 64바이트 버퍼의 200번째 바이트에도 기꺼이 쓰고, 그렇게 망가뜨리는 것은 컴파일러가 마침 그 뒤에 배치한 무엇이든입니다 — 흔히는 반환 주소입니다. 이것은 부주의한 사람들이 가끔 저지르는 실수가 아닙니다. 이 언어의 역사에서 가장 많이 악용된 버그 유형이며, API가 그것을 기본 동작으로 만들어 두었습니다.

`strncpy`는 전통적인 해답이지만 그 자체가 또 하나의 함정입니다. 항상 NUL로 종단하지는 않기 때문에, "안전한" 버전이 조용히 문자열이 아닌 문자열을 만들어 냅니다.

이 라이브러리는 대신 이렇게 합니다. 문자열이 자기 길이를 지니고 다닙니다. `proven_u8str_view_t`는 포인터와 크기가 항상 함께 있는 것입니다. append는 목적지의 용량을 알기 때문에 그 용량을 검사하고, 텍스트가 들어가지 않으면 **에러를 반환하고 아무것도 쓰지 않습니다** — 자르지 않습니다. 잘린 경로는 틀린 경로이고, 잘린 명령은 다른 명령이기 때문입니다. 챕터 3을 참조하세요.

대가. 모든 문자열이 한 워드가 아니라 두 워드이고, NUL 종단 형태를 따로 요청하지 않으면 `proven` 문자열을 `printf("%s")`에 바로 넘길 수 없습니다.

실패를 확인하게 만드는 장치가 없습니다

```
char *p = malloc(n);
p[0] = 'x';                   /* wrong: malloc returns NULL when it fails */
```

`malloc`은 `NULL`을 반환해서 실패를 알리는데, 여러분이 끝내 들여다보지 않아도 C는 한마디도 하지 않습니다. `fopen`도, `realloc`도, 센티널 값을 반환하는 모든 함수가 마찬가지입니다. 에러는 제공되어 있고, 알아차리는 것은 선택 사항입니다.

`errno`는 더 나쁩니다. 호출 이후까지 살아남는 전역이기 때문입니다. 다른 라이브러리 호출이 덮어쓸 기회를 갖기 전에, 정확히 알맞은 순간에 확인해야 하고, 성공한 호출도 거기에 쓰레기 값을 넣을 수 있다는 사실을 기억해야 합니다.

이 라이브러리는 대신 이렇게 합니다. 실패할 수 있는 함수는 에러를 값으로 반환하고, 돌려줄 결과까지 있을 때는 둘이 하나의 `struct`에 담겨 함께 돌아옵니다:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return 1;      /* s.value means nothing until you check */
```

하는 일 자체가 실패할 수 있는 함수에는 `[[nodiscard]]`가 붙어 있어서, 에러를 버리는 코드는 컴파일러가 빌드를 거부합니다. 물론 의도적으로 무시할 수는 있습니다 — 호출 앞에 `(void)`를 붙이면 됩니다 — 그리고 그것을 타이핑해야 한다는 점이 바로 핵심입니다. 챕터 1을 참조하세요.

대가. `if`가 늘어납니다. 깊은 곳의 실패에서 한 번에 빠져나오게 해 주는 예외 메커니즘이 없으므로, 에러 경로가 코드의 모양에 그대로 드러납니다. 그 드러남이 바로 기능입니다.

`printf`는 여러분이 말하는 것을 그대로 믿습니다

```
printf("%d\n", 3.0);          /* wrong: %d with a double. This compiles. */
printf("%s\n", 42);           /* wrong: and this one crashes */
```

포맷 문자열은 런타임에 아무도 검사하지 않습니다. 요즘 컴파일러는 리터럴 포맷에 대해 경고해 주는데, 포맷이 변수가 되는 순간까지만 도움이 됩니다. 그다음부터는 varargs 스택에 마침 들어 있는 바이트를 포맷 문자열이 요구한 모양대로 읽어 대는 함수로 되돌아갑니다.

이 라이브러리는 대신 이렇게 합니다. `{}`는 타입이 들어 있지 않은 자리표시자이고, 타입은 인자에서 오며 `_Generic`으로 컴파일 타임에 결정됩니다:

```
proven_println("{} is {}", PROVEN_ARG(name), PROVEN_ARG(count));
```

타입을 두 번 쓰지 않았으므로 `%d`와 `double`이 어긋날 여지 자체가 없습니다. 입문용 설명은 챕터 3 §3을, 전체 문법은 챕터 8을 보세요.

대가. 인자마다 `PROVEN_ARG`로 감싸야 하고, 손에 익은 것과는 다른 포맷 언어를 써야 합니다.

이건 누가 free하나요?

```
char *s = build_message();    /* do I free this? the type does not say */
```

함수가 반환한 `char *`는 갓 할당된 것일 수도, 호출자의 버퍼를 가리킬 수도, 읽기 전용 메모리에 있는 문자열 리터럴일 수도, 다음 호출이 덮어쓸 정적 버퍼를 가리킬 수도 있습니다. 네 경우 모두 타입은 똑같습니다. 답은 문서에 있고, 문서는 코드와 어긋나기 마련입니다.

이 라이브러리는 대신 이렇게 합니다. ownership이 타입 이름과 시그니처에 드러나 있습니다. `proven_u8str_t`는 **owned**입니다 — `_create`에서 받았으니 같은 할당자(allocator)로 `_destroy`해야 합니다. `proven_u8str_view_t`는 **borrowed**입니다 — 다른 누군가가 소유한 바이트를 가리키므로 절대 destroy하지 않고, 소유자가 사라지는 순간 유효하지 않게 됩니다. 할당할 수 있는 함수는 모두 allocator를 파라미터로 받으므로, allocator가 없는 시그니처는 할당할 수 없습니다.

대가. C에 하나뿐이던 타입이 둘이 되고, 모든 경계에서 "이건 누가 소유하지?"를 물어야 하는 규율이 생깁니다 — 어차피 치르던 비용을, 더 늦게 디버거 안에서 치르는 대신 지금 치르는 것입니다.

아무도 타입 검사를 해 줄 수 없는 비교 함수

```
qsort(a, n, sizeof *a, cmp);  /* cmp takes const void*; get it wrong and it is UB */
```

`qsort`는 비교자를 `void *` 인터페이스로 받으므로, 파라미터 타입이 틀린 비교자도 그대로 컴파일됩니다. 이 버그의 고전적인 형태는 가리키는 대상 대신 포인터 자체를 비교하는 것인데, 그러면 잘 실행되고 아무것도 제대로 정렬하지 않으면서 절대 죽지 않는 프로그램이 나옵니다.

이 라이브러리는 대신 이렇게 합니다. 모양은 똑같은 `void *`입니다 — 여기는 C이고, 다른 방법은 없습니다 — 대신 라이브러리가 계약을 정확히 문서화하고, 복사해 쓸 수 있는 동작하는 비교자를 제공하며, `proven_array_sort`는 공격자가 고른 입력에서 *O(n²)*으로 퇴화하는 퀵소트가 아니라 *O(n log n)*을 보장하는 introsort입니다. 챕터 4를 참조하세요.

바이트에는, 여러분이 정해 주지 않아도, 타입이 있습니다

여러분은 메모리를 바이트로 생각합니다. C의 추상 기계는 그렇지 않습니다. 메모리를 타입이 있는 것으로 다루며, 같은 바이트를 서로 다른 두 타입의 포인터로 읽는 것은 미정의 동작입니다 — 그리고 컴파일러는 그것을, 최적화가 켜졌을 때만, 조용히 이용해도 됩니다. 이것을 **strict aliasing**이라 부르며, 바이트 버퍼를 폭이 다른 포인터로 읽는 손수 짠 모든 파서 아래에 깔린 함정입니다:

```
void *buf = malloc(8);
uint32_t *w = buf;      /* 같은 메모리를 32비트로 봄 – 캐스트도, 경고도 없음 */
uint16_t *h = buf;      /* 같은 메모리를 16비트로 봄 */
*w = 0xAAAAAAAAu;
*h = 0x1234;            /* 하위 절반을 바꿈 */
printf("%08x\n", *w);   /* aaaa1234를 기대하면 틀림: -O2에서는 aaaaaaaa가 찍힘 */
```

`-O0`으로 컴파일하면 `aaaa1234`가, `-O2`에서는 `aaaaaaaa`가 찍힙니다. 컴파일러가 `uint16_t` 쓰기와 `uint32_t` 읽기는 같은 메모리를 건드릴 수 없다고 가정하고 그 쓰기를 지워 버렸기 때문입니다. 경고는 없고, 프로그램은 디버그 빌드에서 돌린 모든 테스트를 통과했습니다. 이것이 리눅스 커널이 `-fno-strict-aliasing`으로 컴파일해 피하는 부류의 버그입니다 — 규칙 하나 때문에 플래그 하나를 통째로.

이 라이브러리는 대신 이렇게 합니다. 원시 메모리는 `unsigned char`의 별칭인 `proven_byte_t`입니다 — 이 규칙이 명시적으로 면제하는 유일한 타입입니다. 표준이 어떤 객체의 바이트든 그 타입으로 들여다보는 것을 허용하기 때문입니다. 평범한 API는 여러분의 바이트를 몰래 더 넓은 타입으로 재해석하지 않으므로, 위 버그는 그것을 통해서는 쓸 수가 없습니다. (strict aliasing에는 더 미묘한 형제 *provenance*가 있고, 라이브러리 이름이 거기서 왔습니다. 챕터 6 §3과 프로젝트 README가 그것을 다룹니다.)

이 라이브러리가 아닌 것

이것은 프레임워크가 아니며, 여러분의 `main`을 차지할 생각이 없습니다. 초기화해야 할 전역 상태가 없고, 스레드를 띄우지 않으며, `atexit` 핸들러를 등록하지 않고, 여러분이 allocator를 건네주지 않은 것은 아무것도 할당하지 않습니다. 모든 모듈은 단독으로 쓸 수 있습니다. 대부분은 운영체제가 전혀 없어도 돌아갑니다 — 프리스탠딩(freestanding) 모드를 보세요.

문제	C가 주는 것	`proven`이 주는 것	대가
버퍼 오버런	`strcpy`, `strcat` — 어디에도 크기가 없음	view가 길이를 지님; 쓰기는 자르는 대신 거부	문자열당 두 워드
확인되지 않은 실패	`NULL` 반환과 `errno`	에러를 값으로 반환, 버려서는 안 되는 것에는 `[[nodiscard]]`	`if`가 늘어남
포맷 불일치	`printf`는 포맷 문자열을 믿음	타입을 인자에서 가져오는 `{}`	호출마다 `PROVEN_ARG`
불분명한 ownership	`char *`가 네 가지 서로 다른 것을 의미	owned와 borrowed가 서로 다른 타입	타입이 하나에서 둘로
숨은 할당	무엇이든 `malloc`을 부를 수 있음	allocator를 받는 함수만 할당 가능 (한정된 예외 하나: 과도하게 긴 줄에서의 `proven_println`)	시그니처에 파라미터가 하나 더
숨은 타입을 가진 바이트	메모리를 더 넓은 포인터로 재해석하면 최적화기가 이용하는 UB	원시 바이트는 규칙이 면제하는 타입 `proven_byte_t`를 거침	—

---

3. 첫 프로그램

이것이 전부입니다. 모든 줄이 §5의 계약 중 하나이고, 빌드가 바로 이 파일을 컴파일하고 실행하므로 조용히 사실이 아니게 될 수 없습니다.

```
/*
 * 첫 프로그램. 일부러 작게 만들었고, 여기 있는 줄 하나하나가 이 매뉴얼의 모든
 * 쪽에서 만나게 될 다섯 계약 가운데 하나다.
 *
 * 여러분이 이미 아는 C 와 견주어 보라.
 *
 *     char buf[64];
 *     strcpy(buf, name);          <- name 은 얼마나 긴가? strcpy 는 묻지 않는다.
 *     strcat(buf, ", welcome!");  <- 이제 남은 자리는? strcat 도 묻지 않는다.
 *     printf("%s\n", buf);
 *
 * 저 프로그램은 `name` 이 여러분의 짐작보다 길어지는 날까지만 옳고, 그날부터는
 * 보안 권고문이 된다. 아래 판은 그럴 수 없다. 모든 쓰기가 목적지의 크기를 알고,
 * 실패할 수 있는 모든 연산이 조용히 무시할 수 없는 오류를 돌려준다.
 */

int main(void) {
    /* (1) 할당자를 여러분이 건넨다. 라이브러리가 등 뒤에서 전역 malloc 에 손을
     *     뻗는 일이 없으므로, 무엇을 누가 할당했는지 언제나 알 수 있다. */
    proven_allocator_t alloc = proven_heap_allocator();

    /* (2) 실패할 수 있는 것은 오류를 값과 *함께* 돌려준다. 확인해야 한다고 기억할
     *     errno 같은 것이 없고, `greeting.err` 를 보기 전에는 `greeting.value` 는
     *     아무 뜻도 없다. */
    proven_result_u8str_t greeting = proven_u8str_create(alloc, 64);
```

```
    if (!proven_is_ok(greeting.err)) return 1;

    /* (3) 뷰는 자기 길이를 아는, 빌려 쓰는 텍스트다. PROVEN_LIT 은 리터럴에서
     *     컴파일 때 그것을 만든다 - 여기서 strlen 처럼 훑는 일은 없다. */
    proven_u8str_view_t name = PROVEN_LIT("world");

    /* (4) append 는 자르는 대신 거부한다. "hello, " 와 이름이 위에서 요청한 64
     *     바이트에 들어가지 못하면 PROVEN_ERR_OUT_OF_BOUNDS 를 돌려주고 아무것도
     *     쓰지 않는다 - 낱말의 반쪽을 조용히 담아 두고 넘어가는 일이 없다. */
    proven_err_t err = proven_u8str_append(&greeting.value, PROVEN_LIT("hello, "));
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, name);
    if (proven_is_ok(err)) err = proven_u8str_append(&greeting.value, PROVEN_LIT("!"));

    if (!proven_is_ok(err)) {
        proven_u8str_destroy(alloc, &greeting.value);
        return 1;
    }

    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&greeting.value),
                                         PROVEN_LIT("hello, world!")),
                    "the three appends should have built the whole greeting");

    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&greeting.value)));

    /* (5) `alloc` 으로 만들었으니 *같은* `alloc` 으로 지운다. 소유하는 것은 정확히
     *     한 번 지우고, 빌린 것은 - 위의 `name` 같은 - 아예 지우지 않는다. */
    proven_u8str_destroy(alloc, &greeting.value);

    return EXAMPLE_OK();
}
```

`EXAMPLE_REQUIRE`와 `EXAMPLE_OK`는 라이브러리의 일부가 아닙니다 — `manual/examples/example.h`에서 오며, 이 매뉴얼의 모든 예제가 본문이 말하는 대로 여전히 동작하는지 빌드가 확인할 수 있게 하려고 존재합니다. 여러분의 프로그램에서는 둘 다 쓰지 않습니다.

예제가 직접 정의하는 타입에는 `ex_` 접두사가 붙습니다(`ex_app_t`, `ex_task_t`). 여러분의 프로그램도 자기 접두사를 붙이는 것이 좋습니다. POSIX는 `_t`로 끝나는 이름을 예약하고, 접두사 없는 `task_t`나 `node_t`는 시스템이나 SDK 헤더에 이미 있을 수 있는 이름입니다(macOS의 Mach 헤더에 `task_t`가 있습니다). 이 라이브러리는 접미사를 유지하고, 정의하는 모든 타입에 `proven_`을 붙입니다.

여기저기서 계속 나오므로, 잠시 멈춰서 볼 만한 세부 사항 셋:

- **`proven_u8str_create(alloc, 64)`는 64바이트의 용량을 요청합니다.** 그리고 그 용량에는 NUL 종단 문자가 포함됩니다. 이 문자열은 스스로 커지지 않습니다. `proven_u8str_append`는 고정 용량 형태라서, 텍스트가 들어가지 않으면 실패합니다. 성장을 원할 때는 `proven_u8str_append_grow`를 부르면서 allocator를 넘기며, 어느 쪽을 쓰고 있는지는 시그니처만 봐도 한눈에 드러납니다.
- **`PROVEN_LIT("hello, ")`는 런타임 비용이 0입니다.** 문자열 리터럴에 대한 컴파일 타임 `sizeof`일 뿐입니다. 텍스트가 다른 곳에서 오는 경우를 위해 `proven_u8str_view_from_cstr(s)`가 있고, 그쪽은 실제로 NUL을 찾아 스캔합니다 — 하나는 공짜이고 다른 하나는 *O(n)*이기 때문에 이름을 다르게 지은 것입니다.
- **`destroy`는 allocator를 다시 받습니다.** 문자열은 자신을 만든 allocator를 기억하지 않으므로, 같은 것을 넘겨야 합니다. 덕분에 문자열이 작게 유지되고 의존 관계가 눈에 보이지만, 실제 위험 요소이기도 합니다 — §5를 보세요.

---

4. 빌드와 include

전부를 위한 헤더는 하나입니다:

```
#include "proven.h"
```

이것이 공개 API 전체를 끌어옵니다. 쓰는 것만 include하고 싶다면 모든 모듈이 `include/proven/` 아래에 자기 헤더를 가지고 있고 — `#include "proven/u8str.h"`, `#include "proven/fs.h"` 하는 식입니다 — 아래 §14가 모든 헤더를 그것을 문서화한 챕터로 연결해 줍니다.

이 라이브러리에 필요한 전부인, 손으로 컴파일하기:

```
gcc -std=c2x -Iinclude -Iplatform your_program.c src/proven/*.c platform/*.c -o your_program
```

`src/proven/`은 이식 가능한 라이브러리 본체입니다. `platform/`은 운영체제와 이야기하는 얇은 계층으로, **PAL**(platform abstraction layer, 플랫폼 추상화 계층)입니다. 시스템 호출을 하는 것은 전부 거기에 있고 다른 어디에도 없으며, 그래서 나머지 라이브러리를 운영체제가 전혀 없는 대상용으로 컴파일할 수 있습니다.

이 저장소는 빌드 시스템 대신 C 프로그램으로 스스로를 빌드합니다:

```
cc -std=c2x -o nob nob.c     # build the build driver, once
./nob build                  # compile everything and run the whole test suite
./nob release                # the same, optimised (NDEBUG: misuse checks compiled out)
./nob hardened               # optimised, misuse checks kept in
```

인자 없이 `./nob`을 실행하면 나머지 모드를 보여줍니다 — 새니타이저, freestanding, 크로스 컴파일, 벤치마크입니다. `make`도 CMake도 없고, 설치할 것도 없습니다.

C23 컴파일러가 필요합니다. 이 라이브러리는 C23 기능을 의도적으로 사용하며, 가장 눈에 띄는 것이 `[[nodiscard]]`입니다. GCC 13+와 Clang 16+, 그리고 Windows 에서는 mingw-w64 GCC(x86-64, i686; 전체 테스트를 네이티브로 실행)가 검증되어 있습니다. MSVC와 clang-cl은 C23 지원이 불완전해 지원하지 않으며, 차후에 지원할 수 있습니다. 빌드 드라이버는 `-std=c23`을 먼저 시험해 보고, 조금 오래된 컴파일러를 위해 `-std=c2x`로 물러섭니다.

---

5. 모든 페이지에서 만나게 될 다섯 가지 계약

이 다섯 규칙이 이 라이브러리의 생김새 대부분을 설명합니다. 각각은 여기서 한 번 진술되고, 다른 모든 곳에서는 전제로 깔립니다. 형식적인 버전은 아래 §8에 있고, 여기 있는 것은 일상 언어판입니다.

#	계약	한 줄 요약	챕터
1	**에러는 값이다**	실패 가능한 호출은 `proven_err_t`나 `{ err, value }` struct를 반환한다	1
2	**view는 borrowed다**	view는 다른 누군가가 소유한 메모리를 가리키는 포인터 + 길이다	3
3	**할당은 파라미터다**	할당할 수 있는 함수는 allocator를 받고, 할 수 없는 함수는 받지 않는다	2
4	**호출자 소유 상태는 복사하면 안 된다**	어떤 struct는 자기 자신을 가리킨다; 복사하면 dangling 포인터가 생긴다	5
5	**거부하되 절대 자르지 않는다**	들어가지 않는 연산은 실패하고 아무것도 쓰지 않는다	3

1. 에러는 값이다

실패 가능한 함수는 모두 에러를 돌려줍니다. 그 외에 돌려줄 것이 없으면 맨(bare) `proven_err_t`이고, 값이 있으면 값과 에러가 함께 이동하며 에러를 확인하기 전까지 값은 무의미합니다. `PROVEN_OK`은 0이며, `proven_is_ok(err)`가 `err == 0`보다 잘 읽힙니다.

잘못된 예 — 에러보다 값을 먼저 읽기:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
proven_u8str_append(&s.value, text);   /* wrong: s.value is garbage if create failed */
```

둘을 짝지어 놓은 이유가 바로 값이 성공 경로에서만 유효하다는 데 있습니다. 매번, 먼저 확인하세요.

2. view는 borrowed다

`proven_u8str_view_t`는 포인터와 크기입니다. 아무것도 소유하지 않고, 아무것도 할당하지 않으며, 마음대로 복사해도 됩니다 — 하지만 가리키는 대상이 파괴되거나 이동하는 순간 유효하지 않게 됩니다.

잘못된 예 — 가리키는 대상보다 오래 사는 view:

```
proven_u8str_view_t name;
{
    proven_result_u8str_t owned = proven_u8str_create(alloc, 16);
    (void)proven_u8str_append(&owned.value, PROVEN_LIT("temp"));
    name = proven_u8str_as_view(&owned.value);
    proven_u8str_destroy(alloc, &owned.value);   /* the bytes are gone */
}
use(name);                                       /* wrong: dangling view */
```

이것은 dangling 포인터와 똑같은 수명 버그이지만, view가 값*처럼 보이기* 때문에 따로 말해 둘 가치가 있습니다. view는 값이 아닙니다. struct를 뒤집어쓴 포인터입니다.

3. 할당은 파라미터다

시그니처를 읽으세요. `proven_u8str_append(str, data)`는 할당할 수 없으므로 텍스트가 들어가지 않으면 실패합니다. `proven_u8str_append_grow(alloc, str, data)`는 allocator를 받으므로 성장할 수 있습니다. 이 라이브러리의 어떤 것도 여러분 몰래 `malloc`을 부르지 않으며, 그래서 arena에서, pool에서, 또는 힙이 없는 장치에서도 쓸 수 있습니다.

여기서 따라 나오는 것이 바로 위험 요소입니다: **생성할 때 쓴 allocator로 파괴해야 합니다.** 객체는 그것을 기억하지 않습니다.

잘못된 예 — 짝이 맞지 않는 allocator:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value);   /* wrong: heap free on arena memory */
```

현재로서는 아무것도 이것을 검사하지 않습니다. 나중에, 다른 어딘가에서 드러나는 힙 손상입니다.

4. 호출자 소유 상태는 복사하면 안 된다

어떤 객체는 여러분이 소유하고 포인터로 넘기는 struct입니다 — 버퍼드 쓰기 스트림(writer), line 읽기 스트림(reader), 디렉터리 이터레이터 같은 것들입니다. 그중 여럿은 자기 자신의 필드를 가리키는 포인터를 품고 있어서, struct를 복사하면 여전히 *원본*을 가리키는 포인터가 함께 복사됩니다.

잘못된 예 — state struct 복사:

```
proven_writer_buf_t a = ...;
proven_writer_buf_t b = a;          /* wrong: b's internals still point into a */
```

아래 §9.2에 이런 타입 스물여덟 개가 모두 나열되어 있습니다. 규칙은 간단합니다. 살아 있을 자리에서 만들고, `&it`을 넘기고, 대입하지 마세요.

5. 거부하되 절대 자르지 않는다

결과가 들어가지 않을 때, 이 라이브러리는 연산을 실패시키고 목적지를 그대로 둡니다. "들어가는 만큼"을 쓰지 않습니다. 잘린 경로는 엉뚱한 파일을 열고, 잘린 명령은 엉뚱한 명령을 실행하며, 잘린 숫자는 다른 숫자입니다 — 그리고 그 하나하나가 눈에 보이는 에러보다 나쁩니다.

정말로 자르는 것이 원하는 동작인 경우를 위해서는, 얼마나 썼는지 알려 주는 별도의, 이름이 다른 함수가 있습니다:

```
proven_result_size_t r = proven_u8str_append_partial(&s, huge);
/* r.value is how many bytes were actually appended. Reading it is the point. */
```

잘못된 예 — 둘이 똑같이 동작한다고 가정하기:

```
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the count WAS the result */
```

여기까지가 일상 언어로 쓴 입문입니다. 이 장의 나머지는 다른 장들이 기대는 레퍼런스입니다: 라이브러리의 의도와 빌드 모델을 형식적으로 적은 것, 전역 계약, 소유권 매트릭스, 연산 동작 클래스, 읽는 순서, 플랫폼 지원, 그리고 부록들. 예전에는 매뉴얼의 첫 페이지인 별도 문서였는데, 첫 페이지를 차례만으로 두기 위해 이곳으로 옮겼습니다.

6. 의도와 설계 철학

`proven`은 컴팩트한 C23 시스템 기반 라이브러리입니다. 메모리 ownership, 에러 제어 흐름, 플랫폼 접근을 전역 상태 뒤에 숨기지 않으면서 실용적인 인프라를 원하는 C 프로그램을 위한 것입니다.

libc 대체품이 아닙니다. 할당자(allocator) 기반 메모리 도구, 바이트 뷰(view), 컨테이너, 문자열, 형식화(formatting), 파싱(scanning), 해싱(FNV, SipHash, CRC-32, SHA-256), hex/Base64 인코딩, OS 강도 난수, 파일시스템 헬퍼, 버퍼드 스트림, 시간 헬퍼, 메모리 매핑, 스택리스 코루틴 매크로, bounded 작업(job) system을 집중적으로 제공합니다.

핵심 설계 원칙:

- C23 우선: 빌드 드라이버는 `-std=c23`를 사용합니다.
- 명시적 에러: 실패 가능한 함수는 `proven_err_t` 또는 `proven_result_*_t`를 반환합니다.
- 명시적 ownership: 소유(owned) 객체는 분명한 destroy 함수와 allocator 규칙을 가집니다.
- Failure atomicity(실패 원자성): grow/realloc 계열 API는 문서에 달리 적지 않는 한 할당 실패 시 기존 객체를 보존합니다.
- 포인터 provenance 규율: 원시 객체 접근은 `proven_byte_t`와 경계 있는 view를 사용합니다.
- PAL 격리: 호스티드(hosted) OS 서비스는 `platform/` 아래에 있고 공개 wrapper를 통해 호출됩니다.
- 핵심 컨테이너는 숨겨진 락을 추가하지 않습니다. 공유 변경(shared mutation)은 호출자의 동기화가 필요합니다.
- 빌드 시스템은 저장소에 체크인된 단일 `nob.c` 하나이며, 테스트는 평범한 C 실행 파일입니다.

7. 빌드와 include 모델

설치할 것이 없습니다

이 라이브러리에는 `configure`도, CMake도, 받아올 패키지도, 어딘가에 놓아야 할 공유 오브젝트도 없습니다. 그냥 C 소스입니다. 여러분의 파일과 나란히 놓고 프로그램에 함께 컴파일하면 됩니다.

이는 의도한 선택이고, 대가가 있습니다. 시스템 패키지를 얻지 못하고, 업데이트는 버전 제약을 올리는 것이 아니라 새 소스를 받아오는 일이 됩니다. 대신 얻는 것은 이렇습니다. 라이브러리가 지금 보고 있는 것과 다른 버전일 수 없고, 여러분이 고르지 않은 빌드 플래그를 주워올 수 없으며, 배포판이 다른 옵션으로 빌드했다는 이유로 링크에 실패할 수 없습니다. hosted 시스템 과 베어메탈 양쪽에서 돌아가야 하는 라이브러리에게 "프로그램과 함께 컴파일한다"는 양쪽 모두에서 통하는 유일한 모델입니다.

중요한 디렉터리는 둘입니다. `src/proven/`은 이식 가능한 라이브러리 본체로, 어디에서도 OS를 호출하지 않습니다. `platform/`은 시스템 호출을 하는 얇은 계층이며, 새 타깃이 교체해야 하는 유일한 부분입니다. 프리스탠딩(freestanding) 빌드는 hosted 파일들을 그냥 빼고 빌드합니다. freestanding 가이드를 보십시오.

빌드 드라이버 `nob.c`는 빌드 시스템이 아니라 C 프로그램이며, 여기의 다른 모든 것과 똑같은 방식으로 컴파일합니다. 저장소에 함께 들어 있으므로 부트스트랩 단계도, 따로 설치할 버전도 없습니다.

C23 컴파일러가 필요합니다 — GCC 13+ 또는 Clang 16+ (Windows 에서는 mingw-w64 GCC; MSVC는 지원하지 않음). 드라이버가 `-std=c23`을 시험해 보고, 아직 과도기 철자를 쓰는 컴파일러에는 `-std=c2x`로 물러납니다.

hosted 테스트 스위트를 빌드하고 실행:

```
cc nob.c -o nob
./nob build
```

자주 쓰는 검증 명령:

```
./nob release
./nob strict
./nob strict-error
./nob asan
./nob ubsan
./nob tsan
./nob regression
./nob regression-asan
./nob regression-ubsan
./nob freestanding
./nob cross -build-root build-out/proven_c_lib
```

전체 hosted API가 필요하면 우산(umbrella) 헤더를 사용합니다:

```
#include "proven.h"
```

더 좁은 translation unit을 원하면 작은 include를 사용합니다:

```
#include "proven/heap.h"
#include "proven/u8str.h"
#include "proven/fmt.h"
```

직접 hosted 애플리케이션 빌드는 `nob.c`와 같은 소스 레이아웃을 따르면 됩니다:

```
cc -std=c23 -D_DEFAULT_SOURCE -D_POSIX_C_SOURCE=200809L \
  -Iinclude -Iplatform \
  app.c src/proven/*.c platform/proven_sys_*.c \
  -pthread -o app
```

8. 전역 계약(Global contracts)

8.1 Result 계약

result 값은 `err == PROVEN_OK`일 때만 사용할 수 있습니다.

올바른 예:

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
if (proven_is_ok(r.err)) {
    /* only now does r.value mean anything */
    proven_mem_mut_t mem = r.value;
    (void)proven_mem_copy(mem.ptr, mem.size, proven_mem_view_from_u8(PROVEN_LIT("hi")));
    alloc.free_fn(alloc.ctx, mem.ptr);
}
```

잘못된 예 (보이는 그대로는 컴파일되지 않음 — 에러를 확인하기 전에 값을 읽음):

```
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 128, PROVEN_DEFAULT_ALIGNMENT);
use_bytes(r.value.ptr, r.value.size); /* wrong: r.err was not checked */
```

8.2 공개 struct 계약

많은 공개 struct가 C 사용성을 위해 레이아웃을 노출합니다. 그렇다고 임의 필드 변경이 지원된다는 뜻은 아닙니다. 헤더가 명시적으로 허용하지 않는 한, 내부 필드를 직접 변경하는 것은 호출자 오용으로 취급하세요.

잘못된 예:

```
arr.len = 999;      /* wrong: breaks array invariants */
str.internal.cap=0; /* wrong: breaks string invariants */
```

불변식(invariant)을 유지하는 함수와 매크로를 사용하세요.

8.3 Borrowed view 계약

view는 메모리를 소유하지 않습니다. `proven_u8str_view_t`, `proven_u16str_view_t`, `proven_mem_view_t`, `proven_mem_mut_t`는 참조된 저장소가 살아 있고 이동되지 않은 동안에만 유효합니다.

잘못된 예:

```
proven_u8str_view_t v = proven_u8str_as_view(&s);
proven_u8str_append_grow(alloc, &s, PROVEN_LIT("more"));
use_view(v); /* wrong: growth may reallocate s */
```

8.4 Allocator 계약

`proven_allocator_t`는 세 함수 포인터가 모두 있을 때만 유효합니다. Reallocation은 failure-atomic이어야 합니다: 실패하면 기존 할당이 그대로 유효해야 합니다.

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* no usable allocator here: e.g. the freestanding heap stub */
    proven_panic("no heap allocator on this target");
}
```

8.5 PAL 경계 계약

`src/proven/` 아래 코드는 OS 헤더에 직접 의존하면 안 됩니다. hosted 서비스는 `platform/proven_sys_*.[ch]`와 `proven_fs_*`, `proven_sysio_*`, `proven_time_*`, `proven_mmap_*` 같은 공개 wrapper를 통해야 합니다.

애플리케이션 코드는 공개 API를 우선하세요. 직접 PAL 호출은 포팅과 플랫폼 통합용입니다.

9. 소유권과 파괴 매트릭스

이 라이브러리에는 **두 종류의 객체**가 있고, 그 차이가 여러분이 무엇을 해야 하는지를 결정합니다.

Owning 객체는 자신이 할당한 저장소를 쥐고 있으며, 반드시 destroy해야 합니다. 바로 아래 표가 그것입니다.

Caller-owned state 객체(호출자 소유 상태)는 아무것도 할당하지 않습니다. 여러분이 — 보통 스택에 — 선언해 생성자에 넘기는 임시 작업용(scratch) struct이고, 생성자는 그 안을 가리키는 작은 값 핸들을 돌려줍니다. **destroy 함수가 없습니다.** 해제할 것이 없기 때문입니다. 대신 owning 객체에는 없는 규칙 하나가 있고, 그게 바로 발목을 잡습니다:

> **핸들이 만들어진 뒤에는 caller-owned state 객체를 복사하거나 이동해서는 안 됩니다.** 핸들은 그 struct 내부를 가리키는 포인터를 쥐고 있습니다. struct를 복사하면 핸들은 여전히 원본을 가리키고, 원본이 스코프를 벗어나면 핸들은 죽은 메모리를 가리킵니다.

이들은 § 4.2에 정리되어 있습니다.

9.1 Owning 객체 — 반드시 destroy해야 함

객체	저장소 소유	allocator 보관	Destroy 함수	비고
`proven_arena_t`	아니오 — 호출자가 backing 슬라이스(slice) 소유	아니오	`proven_arena_destroy(&arena)` (no-op)	호출자 메모리 위의 bump pointer: `alloc`은 오프셋을 전진시키고, `free`는 no-op, `reset`은 빈 상태로 되감음. backing 블록은 호출자가 소유·해제.
`proven_pool_t`	예 — 아이템 + recycle bin	예 (`base_alloc`)	`proven_pool_destroy(&pool)`	고정 아이템 크기. **allocator 트레잇을 통해** (`proven_pool_as_allocator`) 사용합니다: 트레잇의 `free_fn`은 슬롯을 해제하는 대신

				재사용을 위해 recycle bin으로 되돌립니다. `proven_pool_free`는 없습니다 — 다른 allocator와 마찬가지로 해제는 트레잇을 거칩니다.
`proven_buf_t`	예	아니오	`proven_buf_destroy(alloc, &buf)`	호출자가 일치하는 allocator를 넘겨야 함.
`proven_u8str_t`	예	아니오	`proven_u8str_destroy(alloc, &str)`	유효할 때 항상 NUL 종단.
`proven_u16str_t`	예	아니오	`proven_u16str_destroy(alloc, &str)`	내부 길이는 바이트로 추적; API 길이는 `proven_u16` 단위.
`proven_array_t`	예	예	`proven_array_destroy(&arr)` 또는 `PROVEN_ARRAY_DESTROY(&arr)`	원소를 가리키는 포인터는 성장으로 무효화될 수 있음.
`proven_ring_t`	예	예	`proven_ring_destroy(&ring)` 또는 `PROVEN_RING_DESTROY(&ring)`	고정 용량; 성장 없음.
`proven_map_t`	예	예	`proven_map_destroy(&map)` 또는 `PROVEN_MAP_DESTROY(&map)`	Borrowed U8 키는 복사되지 않음.
`proven_fs_list()`가 반환한 `proven_array_t`	예	예 + owned 엔트리 이름	`proven_fs_list_destroy(&list)`	일반 array destroy를 쓰지 말 것; 엔트리 이름 정리가 필요.
`proven_mmap_t`	OS 매핑	OS 핸들 상태	`proven_mmap_destroy(&map)`	매핑을 가리키는 view는 매핑과 함께 죽음.
`proven_job_sys_t *`	예	내부	`proven_job_system_close(sys)` 후 `proven_job_system_destroy(sys)`	destroy는 producer와 경쟁(race)하면 안 됨.

9.2 Caller-owned state — destroy 없음, 복사 금지

이들은 아무것도 할당하지 않고 아무것도 해제하지 않습니다. 하나 선언해 그 주소를 생성자에 넘기고, 돌려받은 작은 핸들을 사용합니다. struct는 핸들보다 **오래 살아야** 하며, 핸들이 살아 있는 동안 **복사·이동해서는 안 됩니다**.

State 객체	생성 함수	뒷받침하는 핸들	비고
`proven_sha256_t`	`proven_sha256_init`	(직접 사용)	해싱 컨텍스트. 시작 *전*에는 복사해도 안전하고, 스트림 도중 복사는 해시를 fork하려는 의도가 아닌 한 무의미.

`proven_sha1_t`, `proven_md5_t`	`proven_sha1_init`, `proven_md5_init`	(직접 사용)	레거시 다이제스트 컨텍스트. `proven_sha256_t`와 같은 규칙이다.
`proven_sha512_t`	`proven_sha512_init` / `proven_sha384_init`	(직접 사용)	SHA-512와 SHA-384의 컨텍스트. 같은 규칙입니다. 시작한 계열의 함수로 끝내십시오.
`proven_hmac_t`	`proven_hmac_init`	(직접 사용)	진행 중인 MAC. **키 재료를 담고 있습니다.** `proven_hmac_final`이 그것을 지웁니다. 복사본은 키의 두 번째 사본입니다.
`proven_xoshiro256ss_t`	`proven_xoshiro256ss_seed`	`proven_rng_t`	복사하면 **수열이 복제됩니다** — 재현(replay)에는 의도적이고 유용하지만, 그 외에는 버그.
`proven_chacha_rng_t`	`proven_chacha_rng_seed` / `_seed_from_entropy`	`proven_rng_t`	복사하면 키스트림이 복제됩니다: "독립적인" 두 토큰이 같은 토큰이 됨.
`proven_writer_buf_t`	`proven_writer_from_buffer`	`proven_writer_t`	**복사 금지.**
`proven_writer_u8str_t`	`proven_writer_from_u8str`	`proven_writer_t`	**복사 금지.** 문자열과 allocator도 그보다 오래 살아야 함.
`proven_writer_buffered_t`	`proven_writer_buffered`	`proven_writer_t`	**복사 금지.** 그것이나 그 버퍼가 죽기 전에 반드시 `proven_writer_flush` 해야 함.
`proven_reader_view_t`	`proven_reader_from_view`	`proven_reader_t`	**복사 금지.**
`proven_reader_buffered_t`	`proven_reader_buffered`	`proven_reader_t`	**복사 금지.** `proven_reader_read_line`이 반환한 view는 이 버퍼 안을 가리킴.
`proven_sysio_std_t`	`proven_sysio_stdout_writer` / `_stderr_writer` / `_stdin_reader`	`proven_writer_t` / `proven_reader_t`	**복사 금지.** 쓰기 스트림(writer)가 가리키는 표준 핸들을 보관.
`proven_sysio_out_t`	`proven_sysio_stdout_buffered` / `_file_buffered`	`proven_writer_t`	**복사 금지.** 반드시 flush 필요.
`proven_sysio_lines_t`	`proven_sysio_lines_open` / `_stdin_lines`	(`proven_sysio_read_line`으로 사용)	유일한 예외: `proven_sysio_read_line`이 매 호출마다 재바인딩하므로, 이것은 **이동해도** 됨.
`proven_sysio_scanner_t`	`proven_sysio_scanner_init`	(직접 사용)	반대 방향의 예외: 이것은 버퍼를 **소유**하므로

			`proven_sysio_scanner_deinit`을 호출해야 함.
`proven_transport_stream_t`	`proven_transport_reader` / `proven_transport_writer`	`proven_reader_t` / `proven_writer_t`	**복사 금지.** 감싼 전송도 이것보다 오래 살아야 합니다.
`proven_http_body_t`	`proven_http_body_init`	(직접 사용)	디코딩 중인 본문. 도중에 복사하면 디코더가 갈라집니다. 하지 마세요.
`proven_url_query_iter_t`	`proven_url_query_iter`	(직접 사용)	쿼리 문자열 안의 위치. 복사본은 독립된 위치입니다.
`proven_net_waker_t`	`proven_net_waker_open`	(직접 사용)	열려 있는 동안 **복사하지 마십시오.** `proven_net_waker_close`가 필요합니다.
`proven_http_cookie_jar_t`	`proven_http_cookie_jar_init`	(직접 사용)	**복사하지 마십시오:** 메모리를 소유합니다. `proven_http_cookie_jar_destroy`가 필요합니다.
`proven_sse_t`	`proven_sse_init`	(직접 사용)	여러분이 준 작업 메모리 안을 가리키며, 그 메모리는 이것보다 오래 살아야 합니다.
`proven_http_client_response_t`	`proven_http_client_send` / `_get`	(직접 사용)	살아 있는 동안 **복사하지 마십시오.** send가 실패한 뒤에도 `proven_http_client_finish`가 필요합니다.
`proven_ws_decoder_t`	`proven_ws_decoder_init`	(직접 사용)	디코딩 중인 스트림. 도중에 복사하면 디코더가 갈라집니다. 복사하지 마십시오.

잘못된 예 — 이 복사는 무해해 보이지만 use-after-free입니다:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);

proven_sysio_out_t saved = out;   /* wrong: `w` still points into `out`, not `saved` */
use_elsewhere(&saved);            /* and if `out` goes out of scope, `w` is dangling */
```

잘못된 예 — 팩토리 함수에서 state를 값으로 반환해도 같은 일이 벌어집니다:

```
proven_sysio_out_t make_logger(void) {
    proven_sysio_out_t out;
    proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);
    (void)w;
    return out;   /* wrong: any writer made from `out` addresses this dead frame */
}
```

올바른 예 — state는 제자리에 두고, 핸들이 이동합니다:

```
proven_byte_t buf[512];
proven_sysio_out_t out;                                  /* lives as long as `w` */
```

```
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });

(void)proven_fprintln(w, "one syscall, not {}", PROVEN_ARG(100));
(void)proven_writer_flush(w);                          /* or the bytes never happened */
```

10. 연산 동작 클래스

한 연산, 세 가지 정직한 대답

"이 텍스트를 저 문자열에 덧붙여라"는 텍스트가 들어가지 않을 때 옹호할 만한 동작이 셋 있고, 대부분의 라이브러리는 그중 하나를 골라 그 선택을 숨깁니다. 이 라이브러리는 셋을 모두 드러내고, 각각에 다른 이름을 주며, 그 차이를 시그니처에 둡니다 — 어느 것을 원하는지는 그 텍스트가 *무엇인지*에 달려 있고, 그것은 호출자만 알기 때문입니다.

파일 경로에 덧붙이는 경우를 생각해 보십시오.

- 경로가 들어가지 않는데 **자르는 것은 치명적입니다.** `documents/report.pdf`가 `documents/rep`가 되는데, 이는 다른 파일이고 실제로 존재할 수도 있는 파일입니다. 이 연산은 실패하고 아무것도 바꾸지 않아야 합니다.
- 이제 로그 한 줄에 덧붙이는 경우를 생각해 보십시오. 들어가지 않으면 **자르는 것이 괜찮습니다** — 메시지 전체를 잃느니 대부분이라도 남는 편이 낫습니다 — 얼마나 썼는지 알려주기만 한다면요.
- 그리고 쌓아 올리는 중인 버퍼에 덧붙이는 경우, 그냥 **자라기를** 원할 것입니다.

아래 세 클래스가 그 세 대답입니다. 어느 것을 받을지는 함수 이름과 allocator 매개변수의 유무가 결정하며, 플래그나 전역 변수가 결정하는 일은 결코 없습니다.

- `proven_u8str_append(str, data)` — allocator가 없으므로 자랄 수 없습니다: **거부합니다**.
- `proven_u8str_append_partial(str, data)` — 개수를 반환합니다: **자르고 알려줍니다**.
- `proven_u8str_append_grow(alloc, str, data)` — allocator를 받습니다: **자랍니다**.

라이브러리 전반의 기본값은 첫 번째이며, 챕터 0 §5가 그 이유를 설명합니다. 잘린 경로는 엉뚱한 파일을 열고, 잘린 명령은 엉뚱한 명령을 실행하며, 잘린 숫자는 다른 숫자입니다.

잘못된 예 — truncating 형태가 돌려준 개수를 무시:

```
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the count WAS the answer */
```

여러 API가 의도적으로 세 가지 동작 클래스를 드러냅니다:

클래스	예	용량 부족 시 동작
Atomic 고정 용량	`proven_u8str_append`, `proven_u16str_append`, `proven_u8str_append_fmt`	에러를 반환하고 기존 객체를 그대로 둠.
Best-effort truncating	`proven_u8str_append_partial`, `proven_u16str_append_partial`, `proven_u8str_append_fmt_trunc`	들어가는 만큼 쓰고, 유효한 객체를 보존하며, 얼마를 썼는지 보고.
Atomic growable	`proven_u8str_append_grow`, `proven_u16str_append_grow`, `proven_u8str_append_fmt_grow`	allocator로 성장; 할당 실패 시 기존 객체를 그대로 둠.
거부하되 절대 자르지 않음	`proven_hex_encode`, `proven_base64_encode`, `proven_base64_decode`, `proven_reader_read_line`	**아무것도 쓰지 않고** `PROVEN_ERR_OUT_OF_BOUNDS`를 반환. 반쯤 인코딩된 문자열이나 짧아진 줄은 옳아 보이는 오답이므로, 이들은 그런 것을 만들지 않음. 버퍼 크기는 눈대중이 아니라 모듈의 크기 함수(`proven_base64_encoded_size` 등)로 정할 것.

클래스를 의도적으로 고르세요. truncating 함수를 all-or-nothing 함수처럼 다루지 마세요.

잘못된 예 — truncating과 atomic 형태가 똑같이 동작한다고 가정:

```
/* `_partial` wrote what fit and told you so; the error you did not read is the
   difference between "all of it" and "some of it". */
(void)proven_u8str_append_partial(&s, huge);   /* wrong: the result was the point */
```

11. 매뉴얼 챕터

상세 레퍼런스는 읽기 쉽고 소스에 근거를 두도록 챕터별로 나눕니다.

입문용 C 책을 막 뗐다면 따라 하며 익히기부터 보세요. 짧은 프로그램 아홉 개로 개념을 하나씩만 더해 갑니다. 여섯 번째에서 이 장의 인사말 프로그램을 한 줄씩 읽고, 마지막 셋은 아레나·컨테이너·파일로 나아갑니다.

그 밖의 경우 이 장부터 읽으십시오. 아무것도 전제하지 않는 유일한 장입니다: 이 라이브러리가 왜 존재하는지를 이 라이브러리가 답하려는 C의 버그들로부터 논증하고, hello world 프로그램, 빌드 방법, 나머지 장들이 당연하게 여기는 다섯 가지 계약, 그리고 용어집과 libc 대응표를 담고 있습니다.

읽는 순서

챕터들은 부(Part)로 묶여 있고, 각 부는 그 앞의 부들만 필요로 하도록 배열되어 있습니다. 이 순서는 헤더 의존 그래프와 같지 않으며, 임의로 정한 것도 아닙니다. 문자열은 allocator가 필요하고, 컨테이너도 allocator가 필요하며, hosted 서비스는 문자열이 필요합니다. 아래 순서는 자료 자체가 요구하는 순서입니다.

챕터 *번호*는 안정적인 식별자이지 읽는 순서가 아닙니다. 그중 둘은 일부러 순서를 벗어나 있습니다. alias 인덱스는 찾아보는 부록이고, 8장은 3장이 주제를 소개한 뒤에 읽는 레퍼런스입니다.

부	읽을 것	선행 조건	그러면 할 수 있는 것
I — 여기서부터	따라 하며 익히기 → 0	입문용 C 책 한 권	라이브러리를 붙여 빌드하고, 아래 무엇이든 읽기
II — 모든 프로그램이 쓰는 어휘	1 → 2 → 3	0장	에러를 값으로 다루고, 메모리를 의도적으로 소유하고, 텍스트를 안전하게 담기
III — 자료구조	4	II부	array, map, list, ring, 정렬, 검색, 해싱, 인코딩
IV — 텍스트 입출력	8	3장 §3–§4	무엇이든 형식화하고 파싱하며, 포매터에 내 타입 가르치기
V — 운영체제와 대화하기	5 → 9 → 10 → 11 → 12 → 13 → 14 → 15	II부	파일, 디렉터리, 스트림, 표준 I/O, 시간, 난수, 매핑. 기한이 있는 소켓. URL과 HTTP 메시지. HTTP 클라이언트와 서버. WebSocket. 인증서와 신뢰. TLS. 이벤트 루프와 그 위의 서버
VI — 더 나아가기	6 → freestanding	II–V부	코루틴, job, 스레드 안전성, 베어메탈, 크로스 빌드
부록	A, B, C, D	—	찾아보기

챕터 목록

- **따라 하며 익히기**: 짧은 프로그램 아홉 개, 한 번에 개념 하나 — Part I, 선택 입구

1. **여기서부터 시작**: 왜 존재하는가, hello world, 다섯 계약, 용어집, libc 대응표 — I부
2. **Foundation**: 타입, 에러, 메모리 view, 정렬, 버전, 패닉(panic) — II부
3. **Allocation**: 힙(heap), 아레나(arena), 풀(pool), byte buffer, 그리고 allocator 트레잇 — II부
4. **문자열과 텍스트**: U8, U16, 그리고 형식화와 파싱 입문 — II부; 텍스트 자료의 튜토리얼 절반
5. **컨테이너와 알고리즘**: array, list, ring, map, 정렬/검색, 해싱, 인코딩 — III부
6. **Hosted 서비스**: 파일시스템, 트리 순회, 스트림, sysio, 환경변수, 난수, mmap, 시간 — V부
7. **실행과 플랫폼**: 코루틴, job, 스레드 안전성, alias, PAL, 크로스 빌드 — VI부
8. **부록 A — Alias 인덱스**: `alias_xcv.h`의 모든 철자 — 참조 전용; 읽는 자료가 아님
9. **형식화와 파싱**: 전체 `fmt.h`와 `scan.h` 레퍼런스 — IV부; 텍스트 자료의 레퍼런스 절반
10. **네트워킹**: 주소, TCP, UDP, 기한, 준비 상태, 전송 — V부; 5장 다음
11. **URL과 HTTP 메시지**: URL 파싱과 안전한 경로, HTTP/1.1 헤드 파서, 본문 프레이밍, 쓰기 함수, 날짜, 범위, multipart, 인증, 쿠키, 이벤트 스트림 — V부; 순수 텍스트 처리, 프리스탠딩에서 사용 가능
12. **HTTP 클라이언트와 서버**: 모든 기다림에 한도가 있는 핸들러와 루프, 리다이렉트·인증 요구·쿠키·프록시를 다루는 클라이언트 — V부; 9장과 10장 다음
13. **WebSocket**: 어느 쪽에서든 여는 연결, 메시지, 닫기. 그리고 프레임 코덱만 따로 — V부; 11장 다음; 코덱은 프리스탠딩에서 사용 가능
14. **인증서와 신뢰**: X.509 인증서 읽기, 인증서가 위하는 이름, 신뢰 앵커, 체인 검증, 피닝 — V부; 4장 다음; 시스템 루트 읽기 외에는 프리스탠딩에서 사용 가능
15. **TLS**: 두 줄로 하는 HTTPS와 wss, 전송 래퍼, 엔진, 재개, 클라이언트 인증서와 핀, 실패의 뜻 — V부; 11장과 13장 다음; 엔진은 프리스탠딩에서 사용 가능
16. **이벤트 루프와 이벤트 구동 서버**: 타이머와 게시된 일을 가진 루프, 아무것도 기다리지 않는 HTTP 서버, 나중에 답하기, backpressure, 연결마다 스레드를 두지 않는 WebSocket, 여러 요청을 동시에 진행하는 클라이언트, 여러 루프, 한가한 연결이 붙드는 것과 측정한 것 — V부; 9장과 11장 다음; 호스티드 전용

3장과 8장은 둘 다 포매터와 스캐너를 다루며, 이 분담은 의도적입니다. 3장은 문자열과 나란히 이들을 소개하며, 일상적인 경우와 생산성을 내기에 충분한 만큼을 담습니다. 8장은 완전한 레퍼런스입니다. 전체 형식 문법, 모든 인자 생성자, 스캐너의 에러 코드와 복구 규칙, 그리고 포매터에 내 타입을 가르치는 방법을 담습니다. 3장을 먼저 읽고, 어떤 지정자나 실패의 정확한 동작이 필요할 때 8장을 펴십시오.

12. 플랫폼 지원과 검증

주요 검증 hosted 대상:

- C23 모드의 GCC 또는 Clang을 쓰는 Linux x86_64.

해당 toolchain이 설치되어 있으면 compile-only 크로스 커버리지가 존재합니다:

- Linux AArch64.
- Linux ARM hard-float.
- `i686-linux-gnu-gcc` 또는 `gcc -m32` multilib을 통한 Linux i686.
- MinGW/WinAPI 경로를 통한 Windows x86_64와 i686.
- ARM Cortex-M freestanding.
- RISC-V ELF freestanding.

크로스 매트릭스는 컴파일, 공개 헤더 가시성, 대상 ABI 가정을 확인합니다. 대상 플랫폼에서의 런타임 검증을 대체하지는 않습니다.

Freestanding 모드는 OS 기반 서비스를 제거한 축소된 서브셋을 빌드합니다. 자세한 내용은 `manual-freestanding-ko.md`와 챕터 6을 참조하세요.

13. 부록 B: 용어집

이 매뉴얼이 평범한 단어인 양 쓰는 용어들입니다. 평범한 C 단어가 아니며, 하나하나가 무게를 지고 있습니다.

본문이 지키는 규칙이 셋 있습니다. 덕분에 여기 나오는 어떤 말도 짐작할 필요가 없습니다.

- **저희끼리만 쓰는 은어를 만들지 않습니다.** 어떤 생각에 널리 쓰이는 말이 이미 있으면 그 말을 씁니다. 이 라이브러리가 새로 만든 것만 여기에서 정의합니다.
- **영어 용어는 각 장에서 처음 나올 때 괄호로 원문을 함께 적습니다** — 뷰(view)처럼. 영어로 개념을 익힌 분은 원래 이름을 찾을 수 있고, 그렇지 않은 분은 번역되지 않은 낱말을 마주치지 않습니다.
- **줄임말은 처음 나올 때 풀어 씁니다.** 예를 들어 PAL은 플랫폼 추상화 계층(Platform Abstraction Layer, PAL)으로 적습니다. 모든 줄임말은 아래 표에도 들어 있습니다.

용어	여기서의 뜻
owned	파괴할 책임이 여러분에게 있습니다. `_create`에서 와서 `_destroy`로 가며, 정확히 한 번입니다.
borrowed	다른 누군가가 소유한 메모리를 가리킵니다. 절대 파괴하지 않습니다. 소유자가 살아 있는 동안에만 유효합니다.
view	borrowed 포인터 + 길이 쌍, 예를 들어 `proven_u8str_view_t`. 복사 가능하고, 소유하지 않으며, 할당하지 않습니다.
allocator	네 가지를 담은 값: 컨텍스트 포인터 하나와 함수 포인터 셋(alloc, realloc, free). 할당할 수 있는 모든 것에 값으로 전달됩니다.
arena	하나의 블록 안에서 포인터를 밀어 가며 메모리를 나눠 주는 allocator. 개별 free는 아무 일도 하지 않고, arena 전체를 한 번에 reset하거나 파괴합니다. 빠르며, "수명이 같은 작은 것 여럿"에 딱 맞습니다.
pool	고정 크기 하나짜리 객체를 여럿 다루는 allocator로, free 리스트를 두어 해제가 실제로 슬롯을 재활용합니다.
trait(트레잇)	인터페이스로 쓰이는 함수 포인터 struct — 가상 함수 테이블에 대한 C의 답입니다. `proven_allocator_t`, `proven_writer_t`, `proven_rng_t`가 트레잇입니다. C 키워드가 아니라 빌려 온 용어입니다.
PAL	플랫폼 추상화 계층(platform abstraction layer): 실제 시스템 호출을 하는 `platform/` 아래의 코드. OS에 의존하는 유일한 부분입니다.
freestanding	운영체제도 libc도 없는 빌드 — 베어메탈입니다. `PROVEN_FREESTANDING`으로 선택합니다.
failure atomicity(실패 원자성)	연산이 실패하면 아무것도 바꾸지 않습니다. 실패한 grow는 기존 데이터를 온전하고 유효하게 남겨 둡니다.
provenance	포인터가 어느 할당에서 왔는가. C의 최적화기는 서로 다른 할당에서 온 포인터가 절대 겹치지 않는다고

	가정하며, 이를 어기는 것은 단지 놀라운 일이 아니라 미정의 동작입니다. 챕터 6에서 다룹니다.
UB(undefined behaviour, 미정의 동작)	"예측할 수 없는 출력"이 아닙니다 — 표준이 아무 요구도 하지 않으며, 최적화기는 그런 일이 결코 일어나지 않는다고 가정해도 됩니다. UB가 여러분의 `if`를 지워 버릴 수 있는 이유입니다.
`[[nodiscard]]`	C23 어트리뷰트. 반환값을 버리면 컴파일러가 에러를 냅니다. 에러를 놓쳐서는 안 되는 모든 함수에 붙어 있습니다.
fixed-capacity(고정 용량)	성장하지 않습니다. 가득 차면 실패합니다. allocator를 받지 않습니다.
growable(성장 가능)	가득 차면 재할당합니다. allocator를 받습니다. 이름에 항상 `_grow`가 붙습니다.
CSPRNG	암호학적으로 안전한 의사난수 생성기: 앞선 출력을 본 뒤에도 공격자가 예측할 수 없는 출력.
intrusive(침입형)	리스트의 링크가 따로 할당된 노드가 아니라 여러분의 struct 안에 들어 있습니다. 원소마다 할당이 발생하지 않습니다.
code unit(코드 유닛)	인코딩의 원소 하나: UTF-8에서는 한 바이트, UTF-16에서는 16비트 값. 문자가 아닙니다 — 한 문자가 여러 개를 차지할 수 있습니다.
code point(코드 포인트)	유니코드에서 한 글자에 매겨진 번호입니다. 한 코드 포인트는 UTF-8에서 1~4바이트, UTF-16에서 코드 유닛 1~2개를 차지합니다. 둘 중 무엇을 세어도 글자 수가 되지 않는 이유입니다.
API(Application Programming Interface, 응용 프로그래밍 인터페이스)	라이브러리가 다른 프로그램이 부르라고 내놓은 함수와 타입의 집합입니다. 이 라이브러리에서는 `include/proven/`에 선언된 모든 것입니다.
hosted(호스티드)	밑에 운영체제와 C 표준 라이브러리가 있는 빌드입니다. *freestanding*의 반대입니다.
heap(힙)	`malloc`이 메모리를 나눠 주는 범용 메모리 영역입니다. `proven_heap_allocator()`가 이것을 쓰는 allocator입니다.
slice(슬라이스)	**쓸 수 있는** borrowed 포인터 + 길이 쌍입니다. 예를 들어 `proven_mem_mut_t`. 같은 생각의 읽기 전용 형태가 *view*입니다.
result(결과 구조체)	에러 코드와 값을 함께 담은 작은 struct입니다. 예를 들어 `proven_result_u8str_t`. 옆에 있는 에러를 확인하기 전까지 값은 아무 뜻도 없습니다.
panic(패닉)	실패를 돌려줄 호출자가 없을 때 택하는 의도적인 정지입니다. `proven_set_panic_handler()`로 그때 할 일을 고르며, 기본값은 프로그램을 멈춥니다.
reserve(예약)	컨테이너의 용량을 **지금** 올려서 이후의 증가가 재할당하지 않게 하는 일입니다. 힙에서는 복사를, arena에서는 죽은 저장 공간을 아낍니다.

dangling(허공을 가리키는)	이미 해제되었거나 옮겨진 메모리를 가리키는 포인터입니다. 그것을 쓰는 것은 미정의 동작이며, 여기서 흔한 원인은 재할당할 수 있는 호출을 사이에 두고 포인터를 붙들고 있는 것입니다.
use-after-free(해제 후 사용)	dangling 포인터로 읽거나 쓰는 일입니다. 아래의 새니타이저가 잡아냅니다.
sanitizer(새니타이저)	실행 시 검사를 덧붙이는 컴파일러 모드입니다. **ASan**(AddressSanitizer)은 메모리 오류를, **UBSan**(UndefinedBehaviorSanitizer)은 미정의 동작을, **TSan**(ThreadSanitizer)은 데이터 경쟁을 찾습니다. `./nob asan`, `./nob ubsan`, `./nob tsan`.
partial write / short read(부분 쓰기 / 짧은 읽기)	요청한 것보다 적은 바이트를 옮긴 한 번의 쓰기나 읽기입니다. 에러가 아니라 정상이며, 짧은 읽기를 입력의 끝으로 착각하는 것이 파일 뒤끝을 잃는 고전적인 방법입니다.
EOF(End Of File, 파일의 끝)	더 읽을 입력이 없다는 뜻입니다. 0바이트 성공이 아니라 `PROVEN_ERR_EOF`로 보고되므로 "아직 아무것도 안 왔다"와 혼동될 수 없습니다.
flush(비우기)	버퍼드 writer에 쌓인 바이트를 다음 단계로 밀어 보내는 일입니다. 이 라이브러리의 어떤 것도 종료 시에 대신 flush해 주지 않습니다.
back-pressure(배압)	소비자가 따라오지 못해 생산자를 늦추는 일입니다. `proven_writer_write_partial()`이 그것을 알아채고 대응하게 해 주는 호출입니다.
durability(지속성)	정전을 겪고도 데이터가 남는다는 보장입니다. 운영체제에 도달한 것만으로는 모자랍니다. `proven_fs_sync()`가 파일의 바이트를 저장 장치에 올리고, `proven_fs_sync_dir()`이 이름 바꾸기에 대해 같은 일을 합니다.
atomic rename(원자적 이름 바꾸기)	완성된 임시 파일을 원래 이름 위로 rename해 파일을 교체하는 방식입니다. 읽는 쪽은 옛 파일 전체나 새 파일 전체를 보며, 반쯤 쓰인 혼합물은 보지 않습니다.
advisory lock(권고 잠금)	마찬가지로 잠금을 요청하는 프로세스만 막는 잠금입니다(`proven_fs_lock()`). 접근 권한 통제가 아니라 협력하는 프로그램들 사이의 약속입니다.
hard link(하드 링크)	같은 파일에 대한 두 번째 **이름**입니다. 원본이라는 것이 없고, 마지막 이름이 지워질 때까지 데이터가 삽니다. 같은 파일 시스템 안에서만 됩니다.
symbolic link(심볼릭 링크)	경로를 담은 작은 파일입니다. 파일 시스템을 건너뛸 수 있고, 아무것도 가리키지 않을 수도 있습니다. 그때는 따라가기가 실패합니다.
memory mapping(메모리 매핑)	파일 내용을 어떤 주소에 나타나게 해서 프로세서가 메모리처럼 읽게 하는 일입니다(`mmap.h`). **SHARED** 매핑의 쓰기는 파일로 돌아가고, **PRIVATE** 매핑은

	*기록 시 복사*라서 쓰기가 이 프로세스에만 남습니다.
copy-on-write(기록 시 복사)	누군가 쓰기 전까지 메모리를 공유하다가, 쓰는 쪽에게만 사적인 사본을 주는 방식입니다. `PROVEN_MMAP_PRIVATE`가 하는 일입니다.
cursor(커서)	스캐너가 읽고 있는 텍스트에서의 현재 위치입니다(`proven_scan_t.cursor`). 스캔은 이것을 앞으로 밀고, `proven_scan_skip_*`은 의도적으로 옮깁니다.
locale(로케일)	소수점이 .인지 ,인지를 포함한, 지역 관습에 대한 시스템의 설정입니다. 이 라이브러리의 수 파싱은 **로케일에서 자유롭습니다**. 어떤 기계에서도 쉼표는 소수점이 아닙니다.
entropy(엔트로피)	운영체제나 하드웨어에서 오는, 진짜로 예측할 수 없는 비트입니다. 생성기는 엔트로피로 씨앗을 받고, 시계나 계수기는 아무리 무작위처럼 보여도 엔트로피가 아닙니다.
seed(씨앗)	생성기의 시작값입니다. 같은 씨앗은 같은 수열을 다시 냅니다. 재현 가능한 시험에는 꼭 필요하고, 키에는 치명적입니다.
PRNG / CSPRNG	의사난수 생성기(Pseudo-Random Number Generator)는 씨앗에서 수열을 계산합니다. **암호학적으로 안전한**(Cryptographically Secure) 것은 앞선 출력을 본 공격자에게도 예측 불가능하다는 성질이 더 있습니다.
HashDoS(해시 서비스 거부 공격)	충돌하도록 고른 키를 해시 테이블에 먹여 O(1) 조회를 *$O(n^2)$*으로 만드는 공격입니다. `proven_map_create()`는 키가 있는 해시로 이를 막고, `proven_map_create_trusted()`는 내가 고른 키에 한해 그 방어를 포기합니다.
open addressing(개방 주소법)	map의 배치 방식입니다. 항목들이 평평한 버킷 배열 하나에 살고, 충돌하면 따로 할당된 노드 사슬을 따라가는 대신 다음 칸으로 옮겨 갑니다.
tombstone(묘비)	map에서 항목이 지워진 자리에 남기는 표시입니다. 조회가 그 자리를 지나 계속 탐색하게 해 줍니다. 묘비도 적재율에 셈되므로, 삭제가 잦으면 여전히 리해시가 일어납니다.
rehash(리해시)	버킷 배열을 새 크기로 다시 만드는 일입니다. 앞서 `get_mut`이 돌려준 모든 포인터를 무효로 만들며, 그래서 포인터가 아니라 키를 붙들어야 합니다.
checksum(검사합)	우연한 손상을 알아내는 짧은 값입니다 — `proven_crc32()`. 사고는 잡아내지만 변조는 결코 잡지 못합니다.
digest / hash(다이제스트 / 해시)	데이터에서 계산한 고정 크기 값입니다. **SHA-256**은 변조를 드러내는 암호학적 다이제스트이고, **FNV-1a**와 **SipHash-2-4**는 테이블용 해시이며 이 중 키를 쓰는 것은 SipHash뿐입니다.

hex / Base64 / Base64URL	바이트를 텍스트로 적는 방법들입니다. hex는 바이트당 두 글자, Base64는 3바이트를 `+` / `=`를 쓰는 네 글자로 묶고, Base64URL은 `-` `_`를 쓰고 패딩이 없어 URL이나 파일 이름에 안전합니다.
padding(패딩)	출력 길이를 4의 배수로 맞추려고 Base64가 붙이는 `=` 문자입니다. Base64URL은 붙이지 않습니다.
BMP(Basic Multilingual Plane, 기본 다국어 평면)	유니코드의 첫 65,536개 코드 포인트입니다. 그 밖의 글자 — 이모지, 드문 한중일 한자 다수 — 는 UTF-16 코드 유닛 두 개가 필요합니다.
NUL terminator(NUL 종결자)	C 문자열의 끝을 표시하는 0 바이트입니다. *view*에는 이것이 없으며, 그래서 대신 길이를 지니고 다닙니다.
shortest round trip(최단 왕복 표기)	다시 읽었을 때 정확히 같은 부동소수점 값이 되는 가장 짧은 자릿수로 찍는 것입니다. `PROVEN_FLOAT_FORMAT_MODE_SHORTEST`가 이것을 요청하며, 직렬화기가 원하는 성질입니다.
dispatch macro(디스패치 매크로)	인자의 타입에서 함수를 골라 주는, C11 `_Generic` 위에 만든 매크로입니다 — `PROVEN_ARG(x)`와 `PROVEN_SCAN_ARG(&x)`. 이름 있는 생성자들 중에서 고를 뿐, 그 자신이 생성자는 아닙니다.
identity constructor(항등 생성자)	`proven_arg_identity()` / `proven_scan_arg_identity()`. 이미 만들어진 인자를 받아 그대로 통과시켜, 매크로로 굴러가는 코드가 그것을 받아들일 수 있게 합니다.
scratch(임시 작업용) allocator	결과를 소유하는 allocator와 따로, 임시 작업 메모리만을 위해 넘기는 allocator입니다. 예를 들어 `proven_map_set_with_scratch()`의 `scratch` 매개변수.
stackless coroutine(스택 없는 코루틴)	자기 스택 없이 멈췄다 이어 갈 수 있는 함수입니다. 상태가 여러분이 들고 있는 struct 안에 삽니다. `coro.h`는 이것을 switch로 구현하므로 스레드도 할당도 들지 않습니다.
bounded queue(상한 있는 큐)	용량이 정해져 있어 가득 차면 늘어나는 대신 거절하는 큐입니다. job 시스템이 이것을 쓰므로, 일꾼보다 빠른 생산자는 메모리를 소진하는 대신 거절을 듣습니다.
monotonic clock(단조 시계)	앞으로만 가는 시계로, 무언가에 걸린 시간을 재는 데 씁니다. 시스템 시각이 교정될 때 뛸 수 있는 벽시계(wall clock)와 다릅니다. 벽시계로 소요 시간을 재면 음수가 나오는 일이 그래서 생깁니다.

---

14. 부록 C: 공개 헤더 맵

찾는 방법

공개 헤더는 35개이고 우산 헤더가 하나 있습니다. `#include "proven.h"`는 전부를 끌어오며, 이 매뉴얼의 예제들이 하는 방식입니다. 개별 헤더를 include하는 것은 컴파일 시간이 신경 쓰이거나 의존 관계를 파일에 드러내고 싶을 때 씁니다.

이 표가 파일 이름만으로는 알 수 없는 두 가지를 알려줍니다.

- **어느 챕터가 그것을 문서화하는가.** 모든 헤더에는 그것을 설명하는 챕터가 정확히 하나 있고, 빌드는 모든 공개 함수가 `manual/` 어딘가에 이름을 올리도록 강제합니다 — 그러니 어떤 심벌이 기대한 챕터에 없더라도 매뉴얼 어딘가에는 있고, 이 맵이 그 위치를 알려줍니다.
- **freestanding 빌드에서 살아남는가.** 챕터 5에 배정된 헤더들이 hosted 쪽입니다. 파일시스템, 표준 스트림, 시계, 가상 메모리 또는 스레드가 필요합니다. 그 밖의 모든 것은 운영체제 없이 컴파일됩니다. 모듈별 권위 있는 표는 freestanding 가이드에 있습니다.

헤더	주요 용도	챕터
`proven.h`	우산 include	이 파일
`types.h`	고정 폭 별칭, 검사된 산술, 에러 enum	챕터 1
`error.h`	에러 술어(predicate) 헬퍼	챕터 1
`memory.h`	바이트 view, 슬라이싱, 범위 검사, memcmp	챕터 1
`align.h`	정렬 상수와 align-up 헬퍼	챕터 1
`version.h`	버전 매크로	챕터 1
`panic.h`	등록 가능한 panic 핸들러	챕터 1
`config.h`	컴파일 타임 기능 토글 (`PROVEN_FREESTANDING`, `PROVEN_FMT_NO_FLOAT`, `PROVEN_NO_U16STR` 등)	챕터 1, 6
`allocator.h`	Allocator 트레잇	챕터 2
`heap.h`	PAL 기반 heap allocator	챕터 2
`alloc_check.h`	자기 블록을 아는 할당자 래퍼: 남의 블록 해제, 이중 해제, 잘못된 realloc을 그 호출에서 잡는다(테스트와 디버깅용; `PROVEN_ALLOC_CHECK` 없으면 꺼짐)	챕터 2
`arena.h`	Bump allocator	챕터 2
`pool.h`	고정 크기 recycler allocator	챕터 2
`buffer.h`	고정 용량 byte buffer	챕터 2
`u8str.h`	Owned U8 문자열과 빌려 쓰는(borrowed) U8 view	챕터 3
`u16str.h`	Owned U16 문자열과 borrowed U16 view	챕터 3
`utf.h`	엄격한 UTF-8 <-> UTF-16 변환: 재기, 변환, 조각 단위 변환, 늘리기	챕터 3
`fmt.h`	구조적 formatter와 format 인자	챕터 3
`scan.h`	구조적 scanner와 타입 있는 scan 목적지	챕터 3
`float_parse.h`	로케일 없는 십진수 → `double`/`float` 파서 (`proven_strtod`, `proven_parse_double_ascii`)	챕터 8
`float_format.h`	`double`/`float` → 십진수 formatter (고정 `%f`/`%e`, shortest)	챕터 8
`float_config.h`	Float 엔진 튜닝 (`PROVEN_FLOAT_BIGINT_LIMBS`, 정밀도 상한)	챕터 6, 8
`array.h`	제네릭 growable 벡터	챕터 4
`list.h`	침입형(intrusive) 이중 연결 리스트	챕터 4
`ring.h`	고정 용량 FIFO ring	챕터 4
`map.h`	Open-addressing map	챕터 4
`algorithm.h`	Array 정렬·검색 헬퍼	챕터 4
`hash.h`	FNV-1a, SipHash-2-4, CRC-32, SHA-256, 용도별, SHA-384, SHA-512	챕터 4
`hmac.h`	SHA-2 계열 위의 HMAC과 HKDF	챕터 4

deflate.h	DEFLATE 압축과 풀기, 그대로 또는 zlib·gzip으로. 스트리밍이며 출력은 호출자가 한정한다	챕터 16
zstd.h	Zstandard 풀기와 압축(레벨 아홉). 스트리밍이며 출력과 창은 호출자가 한정한다	챕터 16
brotli.h	Brotli 풀기. 스트리밍이며 출력과 창은 호출자가 한정한다. PROVEN_NO_BROTLI로 뺄 수 있다	챕터 16
hash_legacy.h	SHA-1과 MD5, 그것을 못 박은 형식용	챕터 4
encode.h	Hex와 Base64 (표준 + URL-safe), 바이트↔텍스트	챕터 4
fs.h	파일, 디렉터리, 메타데이터, 링크, 락, read-all, 트리 순회	챕터 5
stream.h	버퍼드 writer·읽기 스트림(reader)와 line reader — 그리고 sysio.h를 통해 표준 스트림 (hosted 전용)	챕터 5
net.h	주소, TCP, UDP, 유닉스 도메인 스트림, 기한, 준비 상태, 전송 (hosted 전용)	챕터 9
url.h	URL 파싱, 퍼센트 코딩, 루트 안에 머물도록 해소한 요청 경로	챕터 10
http.h	HTTP/1.1 코덱: 헤드 파서, 본문 프레이밍과 디코더, 쓰기 함수, 날짜, 범위, multipart 본문, 콘텐츠 코딩 헤더	챕터 10
hpack.h	HPACK, HTTP/2의 헤더 압축: 인코더와 디코더와 그 테이블. 한도는 호출자가 정한다	챕터 17
h2.h	HTTP/2 프레임, 종류별 규칙, 헤더 목록을 요청이나 응답으로 만드는 검사, 그리고 바이트를 받고 바이트를 돌려주는, 스트림과 흐름 제어를 가진 연결: 소켓 없는 프로토콜이다. (소켓은 http_event.h다.)	챕터 17
http_auth.h	Basic과 Digest 인증 값	챕터 10
http_cookie.h	호스트 전용 쿠키 보관함	챕터 10
sse.h	서버 전송 이벤트 파서	챕터 10
http_client.h	HTTP/1.1 클라이언트: 리다이렉트, 인증 요구, 쿠키, 프록시, 연결 재사용, 요청하면 gzip 응답을 풀어 준다 (호스티드 전용)	챕터 11, 16
http_server.h	HTTP/1.1 서버: 핸들러와 한도가 있는 루프, 스레드 하나 또는 job system. 핸들러가 요청하면 응답을 압축한다 (호스티드 전용)	챕터 11, 16
ws.h	WebSocket 코덱: 핸드셰이크 값과 확인, 프레임, 메시지 디코더, 압축 확장의 핸드셰이크와 프레임 비트	챕터 12
ws_conn.h	전송 위의 WebSocket 연결, 클라이언트와 서버. 요청하면 메시지를 압축한다 (호스티드 전용)	챕터 12
cert.h	X.509 인증서: 엄격한 읽기, 호스트 이름 대조, PEM, 신뢰 앵커 저장소, 체인 검증. 시스템 루트 읽기만 호스티드 전용.	챕터 13
tls.h	TLS 1.3과 줄인 TLS 1.2, 클라이언트와 서버: 설정, I/O 없는 엔진, 그리고 (호스티드 전용) 전송 래퍼와 HTTP 랩	챕터 14
loop.h	이벤트 루프: 소켓 준비 상태, 타이머, 다른 스레드가 게시한 일, 함께 쓰는 임시 작업용 버퍼 (호스티드 전용)	챕터 15
http_event.h	루프 위에서 이벤트로 구동되는 HTTP 서버 - HTTP/1.1, 그리고 요청하면 그 옆에 HTTP/2: 콜백, backpressure가 있는 조각 응답, 선택적 TLS, 요청하면 응답을 압축한다 (호스티드 전용)	챕터 15, 16, 17
ws_event.h	이벤트 구동 서버 위의 WebSocket: 조각으로 오는 메시지, 거절되고 재개되는 보내기, 생존 확인과 close 처리, 요청하면 메시지 압축 (호스티드 전용)	챕터 15, 12

http_event_client.h	루프 위에서 이벤트로 구동되는 HTTP 클라이언트: 한 스레드에서 동시에 진행되는 여러 요청. 요청하면 HTTP/2 - 오리진마다 간직하는 연결 하나에서. 좁다 - 리다이렉트와 이름 해석 없음, HTTP/1.1의 재사용 없음. 요청하면 gzip 응답을 풀어 준다 (호스티드 전용)	챕터 15, 16, 17
sysio.h	표준 스트림을 writer/reader로, stdin 줄 입력, 버퍼드 출력, 출력, 파싱, 환경변수 접근	챕터 5
random.h	용도별 난수: xoshiro256** (재현 가능), ChaCha20 (암호학적), OS CSPRNG, 무편향 range/shuffle 헬퍼. 생성기는 freestanding에서 동작하고 OS 소스만 hosted.	챕터 5
mmap.h	메모리 매핑 파일 영역	챕터 5
time.h	타임스탬프, datetime, sleep, datetime 형식화	챕터 5
coro.h	스택리스 코루틴 매크로	챕터 6
job.h	Bounded worker-thread job system	챕터 6
alias_xcv.h	선택적 짧은 alias 계층과 생성된 철자 맵	챕터 6, 7

15. 부록 D: libc 대응표

이미 C를 쓰고 있다면 이것이 가장 빠른 입구입니다. 각 행은 그 맞바꿈을 설명하는 챕터로 연결됩니다.

원래 쓰던 것	대신 쓸 것	무엇이 다른가
malloc / free	proven_heap_allocator() + _create / _destroy	allocator가 파라미터라서, 같은 코드가 arena나 pool 위에서도 돌아갑니다. 챕터 2
strcpy, strcat	proven_u8str_append, _append_grow	크기를 알고 있으므로, 오버런을 저지르는 대신 거부합니다. 챕터 3
strlen	view.size	길이가 이미 거기 있습니다. 아무것도 스캔하지 않습니다. 챕터 3
동등 비교용 strcmp	proven_u8str_view_eq	중간에 NUL이 박힌 텍스트에서도 동작하고, 끝을 넘어가지 않습니다. 챕터 3
strstr	proven_u8str_view_find	인덱스 또는 PROVEN_INDEX_NOT_FOUND를 반환하며, 탐색이 단순 무식하지 않습니다. 챕터 3
strtok	(아직 대응물 없음)	strtok은 입력을 변경하고 중첩해서 쓸 수 없습니다. view 기반 분할기가 RFC-0004에 설계되어 있습니다.
printf	proven_println("{}", PROVEN_ARG(x))	타입이 포맷 문자열이 아니라 인자에서 옵니다. 챕터 8
sprintf	proven_u8str_append_fmt	크기가 정해진 목적지에 쓰고, 그것을 넘기기를 거부합니다. 챕터 8
sscanf	proven_scan_*, proven_scan_fmt	어느 필드가 실패했고 커서가 어디서 멈췄는지 알려 줍니다. 챕터 8
strtod	proven_parse_f64_ascii	올바르게 반올림하고, 로케일에 영향받지 않으며, errno를 쓰지 않습니다. 챕터 8

`fopen` / `fread` / `fclose`	`proven_fs_open`, `_read`, `_close`, 또는 `proven_fs_read_all_u8str`	명시적인 에러, 숨은 버퍼링 없음, 파일 전체를 한 번의 호출로. 챕터 5
`fgets`	`proven_sysio_read_line`, `proven_reader_read_line`	버퍼를 정확히 꽉 채우는 줄도 잃어버리지 않고 반환합니다. 챕터 5
`qsort`	`proven_array_sort`	introsort입니다: 퀵소트의 최악 경우가 아니라 O(n log n) 보장. 챕터 4
`bsearch`	`proven_array_binary_search`	같은 모양, 같은 비교자 계약. 챕터 4
`rand`	`proven_xoshiro256ss_*` 또는 `proven_random_bytes`	재현 가능하고 빠르거나, 예측 불가능하고 안전하거나 — 여러분이 의도적으로 고릅니다. 챕터 5
`time` / `clock`	`proven_time_now`, `proven_time_monotonic_now`, `proven_time_breakdown`	나노초 단위이고, 벽시계와 단조 시계가 서로 다른 두 호출입니다. 챕터 5
`socket` / `connect` / `send` / `recv` / `poll`	`proven_net_connect`, `_read`, `_write_all`, `proven_net_poll`	기다리는 모든 호출이 기한을 받고, 거절·리셋·입력의 끝이 서로 다른 답입니다. 챕터 9
`assert`	`proven_panic` + panic 훅	freestanding 빌드에서도 동작하고, 교체할 수 있습니다. 챕터 1

16. 다음에 읽을 것

각 장이 그 앞의 장들만 필요로 하도록 순서를 잡았습니다.

파트	읽을 것	무엇을 위해
I	이 장	계약과 어휘
II	1 → 2 → 3	에러, 메모리, 텍스트: 모든 프로그램이 쓰는 것
III	4	배열, 맵, 리스트, 링, 정렬, 해싱, 인코딩
IV	8	챕터 3이 소개한 뒤의, 형식화와 파싱 전체
V	5 → 9 → 10 → 11 → 12 → 13 → 14 → 15	파일, 스트림, 표준 I/O, 시간, 난수, 매핑. 그다음 소켓. 그다음 URL과 HTTP. 그다음 클라이언트와 서버. 그다음 WebSocket. 그다음 인증서. 그다음 TLS. 그다음 이벤트 루프
VI	6 → freestanding	코루틴, job, 스레드 안전성, 베어메탈, 크로스 빌드
부록	A: alias 인덱스, 위의 B와 D	찾아보기

숙련된 C 프로그래머이고 시간이 없다면, 위의 §15를 읽고, 그다음 챕터 1을 읽고, 그다음 필요한 것을 다루는 챕터를 읽으세요. 더 초심자라면 파트 I과 II를 순서대로 읽으세요 — 짧고, 뒤의 모든 것이 그것을 전제로 합니다.

1장: 기초 — 에러, 타입, 메모리 view

2부 — 모든 프로그램이 사용하는 어휘. 선행 조건: 0장. 이 장을 마치면 이 라이브러리가 보고하는 모든 실패를 처리할 수 있고, 크기를 잃어버리지 않으면서 메모리 영역을 기술할 수 있으며, 조용히 감기지 않는 크기 산술을 할 수 있다.

이 장은 `types.h`, `error.h`, `memory.h`, `align.h`, `version.h`, `panic.h`를 다룬다 — 다른 모든 장이 그 위에 세워지는 조각들이다. 여기에는 할당하는 것이 아무것도 없고, 운영체제와 이야기하는 것도 아무것도 없다. 그러면서도 진짜로 반드시 읽어야 하는, 가장 짧은 장이다.

목차

1. 에러는 값이다
2. 타입, 그리고 왜 표기가 다른가
3. 메모리 view: 포인터와 길이를 하나로
4. 감길 수 없는 크기 산술
5. 정렬
6. Panic: 에러를 돌려줄 상대가 남지 않았을 때
7. 버전 매크로
8. 예제와 오용 사례

1. 에러는 값이다

왜 이것이 맨 앞에 오는가

이 매뉴얼의 나머지 모든 페이지는 여기서 실패가 어떻게 보고되는지 여러분이 안다고 가정한다. 그러니 이 절은 건너뛸 수 없는 유일한 절이다.

C는 함수가 실패를 어떻게 보고해야 하는지에 대해 스스로와 합의한 적이 한 번도 없다. `malloc`은 `NULL`을 반환한다. `fopen`도 `NULL`을 반환한다. `read`는 `-1`을 반환한다. `strtol`은 `0`을 반환하고 어쩌면 `errno`를 설정하는데, 그것도 여러분이 먼저 `errno`를 지웠을 때에만 그렇다. 성공한 호출도 거기에 쓰레기를 남겨 두는 것이 허용되기 때문이다. `printf`는 거의 아무도 확인하지 않는 음수를 반환한다. 네 가지 관례가 있고, 이들이 공유하는 유일한 성질은 **무시해도 조용하고 합법이라는 것**이다:

```
char *p = malloc(n);
p[0] = 'x';                    /* wrong: malloc returns NULL on failure */

FILE *f = fopen(path, "r");
fread(buf, 1, n, f);           /* wrong: f may be NULL */

long v = strtol(s, NULL, 10);  /* wrong: 0 could be the value or the failure */
```

더 깊은 문제는 이것들이 틀리기 쉽다는 데 있지 않다. **타입이 무언가 일어날 수 있다는 사실을 전혀 말해 주지 않는다는 것**이 문제다. `char *`는 `NULL`이 될 수 있든 없든 똑같은 타입이므로, 그 무엇도 — 컴파일러도, 대충 훑어보는 리뷰어도, 새벽 두 시의 여러분도 — 확인하라는 신호를 받지 못한다.

`errno`는 에러를 전역이면서 일시적인 것으로 만들어 상황을 더 나쁘게 한다. 정확히 알맞은 순간에 읽어야 하고, 중간에 끼어든 어떤 라이브러리 호출이든 그것을 덮어쓸 수 있으며, 스레드마다 따로 존재하므로 한 부류의 버그를 고치려다 또 다른 부류를 열어젖혔다.

이 라이브러리가 대신 하는 일

에러는 반환값이며, 자기만의 타입을 가진다.

함수의 결과가 성공 아니면 실패뿐일 때에는 `proven_err_t`를 반환한다 — `PROVEN_OK`가 `0`인 평범한 열거형이다:

```
proven_err_t err = proven_u8str_append(&s, PROVEN_LIT("hello"));
if (!proven_is_ok(err)) { /* handle it */ }
```

함수가 돌려줄 값이 있을 때에는 값과 에러가 하나의 구조체 안에 함께 실려 오며, **에러를 확인하기 전까지 그 값은 아무 의미도 없다**:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 64);
if (!proven_is_ok(s.err)) return;      /* s.value is not a string; do not touch it */
```

이는 Rust의 `Result`나 C++23의 `std::expected`와 같은 발상을, C가 허용하는 유일한 방식 — 값으로 반환되는 구조체 — 으로 표현한 것이다. 할당도, 간접 참조도, 숨은 제어 흐름도 없다. 여러분이 포기하는 것은 에러 처리를 조용히 건너뛸 수 있는 능력인데, 그것이 바로 요점이다.

라이브러리의 나머지가 이 세 가지 성질에 의존하므로 명시해 둘 가치가 있다:

- **`PROVEN_OK`은 0이다.** 그래서 `{0}`으로 영초기화한 result 구조체는 "성공, 비어 있음"을 뜻하는 상태로 시작한다.
- **실패 가능한 함수는 `[[nodiscard]]`다** — 반환값을 버리는 코드를 컴파일러가 거부하게 만드는 C23 속성이다. 여전히 에러를 무시할 수는 있지만, 호출 앞에 `(void)`를 써서 그것이 의도였음을 밝혀야 한다.
- 함수가 달리 문서화하지 않는 한 **실패는 failure-atomic이다**: 연산이 실패했다면 아무것도 바꾸지 않았다. 실패한 append는 여러분의 문자열을 있던 그대로 남겨 둔다.

레퍼런스

```
typedef enum {
    PROVEN_OK = 0,
    PROVEN_ERR_NOMEM,
    PROVEN_ERR_OUT_OF_BOUNDS,
    PROVEN_ERR_INVALID_ENCODING,
    PROVEN_ERR_INVALID_ARG,
    PROVEN_ERR_IO,
    PROVEN_ERR_NOT_FOUND,
    PROVEN_ERR_INVALID_STATE,
    PROVEN_ERR_NEED_MORE,
    PROVEN_ERR_OVERFLOW,
    PROVEN_ERR_UNSUPPORTED,
    PROVEN_ERR_AGAIN,
    PROVEN_ERR_EOF,
    PROVEN_ERR_BUSY,
    PROVEN_ERR_PERMISSION,
    PROVEN_ERR_INVALID_FORMAT,
    PROVEN_ERR_EXISTS,
    PROVEN_ERR_TIMEOUT,
    PROVEN_ERR_REFUSED,
    PROVEN_ERR_RESET,
    PROVEN_ERR_UNREACHABLE,
    PROVEN_ERR_UNTRUSTED,
    PROVEN_ERR_EXPIRED,
    PROVEN_ERR_NOT_YET_VALID,
    PROVEN_ERR_NAME_MISMATCH,
    PROVEN_ERR_PROTOCOL,
    PROVEN_ERR_CANCELLED
} proven_err_t;

#define PROVEN_ERR_LAST         PROVEN_ERR_CANCELLED        /* moves when a code is added */
#define PROVEN_ERR_RESERVED_END 0x1000                      /* never reached, in any version */
```

에러	일반적인 의미	보통의 대응
`PROVEN_OK`	성공.	계속 진행한다.
`PROVEN_ERR_NOMEM`	allocator가 메모리를 제공하지 못했다.	이 연산을 포기한다. 객체는 바뀌지 않았다.
`PROVEN_ERR_OUT_OF_BOUNDS`	인덱스, 크기, 용량, 또는 범위가 유효하지 않다 — "들어가지 않는다"도 포함한다.	대상을 키우거나, 입력을 거부한다.

PROVEN_ERR_INVALID_ENCODING	인코딩된 텍스트가 검증에 실패했다 (hex, Base64).	입력을 잘못된 형식으로 보고 거부한다.
PROVEN_ERR_INVALID_ARG	널 포인터, 유효하지 않은 allocator, 불가능한 모드, 잘못된 인자 개수.	호출을 고친다 — 이것은 보통 잘못된 데이터가 아니라 여러분 코드의 버그를 뜻한다.
PROVEN_ERR_IO	OS 또는 장치가 연산을 실패시켰다.	재시도하거나, 사용자에게 보고한다.
PROVEN_ERR_NOT_FOUND	파일, 키, 부분 문자열, 또는 리소스가 존재하지 않는다.	"없음" 분기를 탄다. 흔히 에러조차 아니다.
PROVEN_ERR_INVALID_STATE	객체의 상태가 이 연산을 허용하지 않는다.	호출 순서를 고친다.
PROVEN_ERR_NEED_MORE	입력이 더 들어오면 완성될 무언가의 한가운데서 끝났다 - 잘린 UTF-8 문자, 읽기에 잘린 숫자.	가진 것을 두고 더 읽는다. 완전한 텍스트라면 잘못된 입력이다.
PROVEN_ERR_OVERFLOW	정수 변환이나 크기 산술이 오버플로했다.	그 크기를 거부한다. §4를 보라.
PROVEN_ERR_UNSUPPORTED	이 플랫폼이나 빌드 프로파일에서는 사용할 수 없다.	다른 경로를 택한다(예: 파이프는 seek할 수 없다).
PROVEN_ERR_AGAIN	지금은 안 된다. 나중에 다시 시도하라.	보통 기다린 뒤 재시도한다.
PROVEN_ERR_EOF	입력의 끝.	읽기 루프를 멈춘다 — 예외가 아니라 예상된 일이다.
PROVEN_ERR_BUSY	큐, 락, 또는 리소스가 사용 중이다.	물러나서 기다린다.
PROVEN_ERR_PERMISSION	접근이 거부되었다.	보고한다. 재시도해도 소용없다.
PROVEN_ERR_INVALID_FORMAT	포맷이나 스캔 템플릿 자체가 잘못되었다.	포맷 문자열을 고친다 — 잘못된 데이터가 아니라 버그다.
PROVEN_ERR_EXISTS	이름을 만들라는 호출이 이미 있는 이름을 만났다. 배타적 생성 (PROVEN_FS_CREATE_NEW), proven_fs_mkdir, proven_fs_link.	다른 이름을 고르거나, 있는 것을 쓴다. 다만 그것이 기대한 것인지 확인한 뒤에.
PROVEN_ERR_TIMEOUT	준 기한이 연산이 끝나기 전에 지났다.	재시도하거나, 포기하거나, 닫는다 — 객체는 여전히 쓸 수 있다. 망가진 것은 없다.
PROVEN_ERR_REFUSED	상대가 응답했고 거절했다: 그 주소에서 듣고 있는 것이 없다.	다른 주소를 시도하거나 보고한다. 곧바로 재시도하면 같은 답이 온다.
PROVEN_ERR_RESET	상대가 연결을 갑자기 끝냈거나, 도중에 중단됐다.	이쪽 끝을 닫는다. 전송 중이던 것은 도착하지 않았을 수 있다.
PROVEN_ERR_UNREACHABLE	그 네트워크나 호스트로 가는 경로가 없다.	보고하거나 나중에 재시도한다 — 원인은 프로그램 밖에 있다.
PROVEN_ERR_UNTRUSTED	상대의 신원을 확인할 수 없었다: 상대의 인증서에서 여러분이 믿는 무엇까지도 받아들일 만한 체인이 없다.	계속하기를 거부한다.
PROVEN_ERR_EXPIRED	인증서가 유효 기간의 끝을 지났다.	거부하고, 만료되었다고 사용자에게 알린다. 모든 것이 이렇게 실패하면 시계를 의심한다.

PROVEN_ERR_NOT_YET_VALID	인증서의 유효 기간이 아직 시작되지 않았다.	거부한다. 대개 시계가 범인이다.
PROVEN_ERR_NAME_MISMATCH	인증서는 유효하지만, 닿으려던 이름이 아닌 다른 이름을 위한 것이다.	거부한다. 잘못 이끌렸거나 가로채인 연결이 이렇게 보인다.
PROVEN_ERR_PROTOCOL	상대가 보안 프로토콜의 규칙을 어겼거나, 양쪽이 함께 쓸 수 있는 것이 없다. TLS 단위가, 상대가 잘못한 것 가운데 인증서에 관한 것이 아닌 모든 것에 이것을 돌려준다.	연결을 닫는다.
PROVEN_ERR_CANCELLED	이벤트 루프에서 시작한 작업이 끝나기 전에 취소되었다: proven_loop_cancel, 또는 그 소켓에 대한 proven_loop_close(15장). 작업 자신의 콜백으로 한 번 온다.	작업이 쥐고 있던 것을 해제한다 - 그때부터 루프는 그것을 건드리지 않는다.

여러분 자신의 코드를 위한 자리. 모든 proven_err_t 값은 PROVEN_ERR_RESERVED_END(0x1000)보다 작다. 이것은 이번 판의 모습이 아니라 이후 모든 판에 대한 약속이다: proven은 그 이상의 코드를 결코 정의하지 않는다. PROVEN_ERR_LAST는 지금 가장 큰 코드를 가리키며 코드가 추가되면 함께 움직인다 - 마지막이라고 알고 있는 코드가 아니라 이것과 비교하라. 그러므로 자기 실패를 가진 프로그램은 proven의 코드를 더 넓은 정수에 그대로 실어 나르고 자기 코드는 0x1000부터 매길 수 있다: 다시 매기는 표가 필요 없고, 넘겨받은 proven 코드는 위 표가 말하는 뜻 그대로다.

```
/* An application's own codes, above everything proven will ever use. */
enum { APP_ERR_FIRST = PROVEN_ERR_RESERVED_END, APP_ERR_BAD_GLYPH = APP_ERR_FIRST, APP_ERR_TOO_DEEP };
_Static_assert(PROVEN_ERR_LAST < APP_ERR_FIRST, "proven's codes stay below ours");
int app_err = APP_ERR_TOO_DEEP;   /* a proven_err_t converts into the same int unchanged */
(void)app_err;
```

```
static inline int proven_is_ok(proven_err_t err);
#define PROVEN_IS_OK(err) proven_is_ok(err)
```

proven_is_ok은 err == PROVEN_OK일 때 0이 아닌 값을 반환한다. PROVEN_IS_OK은 매크로처럼 보이기를 원하는 자리를 위한 매크로 표기다. 둘은 같은 일을 한다.

올바른 예:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (!proven_is_ok(s.err)) {
    return;   /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(alloc, &s.value);
```

반례 — 에러를 맨 진리값처럼 검사하기:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
if (s.err) {
    /* wrong: works today only because PROVEN_OK is 0. It reads as "if there is
       an error" to someone who already knows that, and as nothing at all to
       everyone else. Say what you mean: !proven_is_ok(s.err). */
    return s.err;
}
```

반례 — 이 설계 전체가 막으려고 존재하는 바로 그 실수:

```
proven_result_u8str_t s = proven_u8str_create(alloc, 32);
(void)proven_u8str_append(&s.value, text);   /* wrong: s.value may be garbage */
proven_u8str_destroy(alloc, &s.value);       /* wrong: destroying what was never created */
```

.err를 확인하기 전에 .value를 읽는 것은 이 관례가 여러분 대신 잡아 줄 수 없는 유일한 에러다. 구조체는 어느 쪽이든 존재하므로 컴파일러는 불평하지 않는다 — 그러니 호출 바로 다음 줄에 확인을 적는 습관을 들일 가치가 있다.

실전 예제: 값으로서의 에러

이 프로그램은 테스트 스위트가 컴파일하고 실행하므로 낡을 수 없다. 실패 가능한 호출이 취하는 두 가지 형태 — 돌려줄 것이 없을 때의 맨 proven_err_t와, 돌려줄 것이 있을 때의 proven_result_*_t — 을 보여주고, result 안의 값이 그 옆의 에러를 확인하기 전까지는 아무 의미도 없는 이유를 보여준다.

```
/*
 * proven 에서 오류는 값이다. 실패할 수 있는 호출은 오류나 result 를 돌려주고,
 * 컴파일러가 그것을 보게 만든다. 풀어 되돌릴 것도 없고, 찾아볼 전역도 없다.
 */

/* 돌려줄 값이 없는 실패 가능 연산은 proven_err_t 를 돌려준다. */
static proven_err_t write_greeting(proven_u8str_t *out) {
    return proven_u8str_append(out, PROVEN_LIT("hello"));
}

/* 돌려줄 값이 *있으면* 그것을 지키는 오류와 함께 싸여 온다. 값은 `err` 를 확인한
 * 뒤에야 뜻을 갖는다. */
static proven_result_size_t half(proven_size_t n) {
    proven_result_size_t res = {0};
    if (n % 2 != 0) {
        res.err = PROVEN_ERR_INVALID_ARG;   /* res.value 는 0 그대로 둔다: 아무 뜻도 없다 */
        return res;
    }
    res.err = PROVEN_OK;
    res.value = n / 2;
    return res;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 맨 proven_err_t 를 확인하기 ------------------------------------ */
    proven_result_u8str_t s = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a 32-byte string should succeed");

    proven_err_t err = write_greeting(&s.value);
    if (!proven_is_ok(err)) {
        /* 아무것도 붙지 않았고 문자열은 여전히 온전하다. proven 의 늘리는 연산은
         * 실패 원자적이다. */
        proven_u8str_destroy(alloc, &s.value);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), PROVEN_LIT("hello")),
                    "the greeting should have been appended");

    /* --- result 구조체를 확인하기 --------------------------------------- */
    proven_result_size_t ok = half(10);
    EXAMPLE_REQUIRE(proven_is_ok(ok.err), "10 is even, so halving it must succeed");
    EXAMPLE_REQUIRE(ok.value == 5, "half of 10 is 5");

    proven_result_size_t bad = half(7);
    EXAMPLE_REQUIRE(bad.err == PROVEN_ERR_INVALID_ARG, "7 is odd, so halving it must fail");
    /* bad.value 는 읽으면 안 된다. 여기서는 0 이지만 그것은 이 함수의 구현 사정일
     * 뿐, result 타입의 약속이 아니다. */
```

```
    /* --- 오류를 실수로 흘릴 수는 없다 ---------------------------------- */
    /* proven_u8str_append 에는 [[nodiscard]] 가 붙어 있어 이렇게 쓰면 컴파일 오류다.
     *
     *     proven_u8str_append(&s.value, PROVEN_LIT("!"));
     *
     * 실패를 정말로 무시하겠다면, 무시한다고 적어야 한다. */
    (void)proven_u8str_append(&s.value, PROVEN_LIT("!"));

    printf("greeting: %s\n", proven_u8str_as_cstr(&s.value));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

2. 타입, 그리고 왜 표기가 다른가

그냥 `int`와 `char`를 쓰면 안 되는가?
C의 내장 타입은 모호하기로 유명하다. `int`는 최소 16비트, `long`은 최소 32비트이고, `char`는 컴파일러와 대상 플랫폼에 따라 부호가 있을 수도 없을 수도 있다. 그렇지 않다고 가정한 코드는 여러분의 기계에서는 동작하고 남의 기계에서는 깨지며, 그 깨짐은 대개 조용하다.

C99는 `<stdint.h>`(`int32_t`, `uint64_t` 등)로 폭 문제를 해결했다. 이 라이브러리는 그 아래에서 그것들을 사용하되 자기만의 이름을 붙인다. 이유는 하나다: **라이브러리가 지원하는 모든 대상에서 같은 것을 뜻하는 단 하나의 표기**가 필요하기 때문이다. `<stdint.h>`가 유일하게 쓸 수 있는 헤더일지도 모르는 프리스탠딩(freestanding) 환경까지 포함해서 말이다.

두 번째의, 더 미묘한 이유가 있고 실전에서 중요한 것은 그쪽이다: `proven_u8`과 `proven_byte_t`는 둘 다 8비트지만 서로 다른 것을 뜻한다.

- `proven_u8`은 0에서 255 사이의 **수**다. 더하고, 비교하고, 출력하라.
- `proven_byte_t`는 **어떤 객체 표현의 바이트**다. 이는 `unsigned char`의 별칭이며, 무엇이든 그 원시 바이트를 미정의 동작 없이 들여다볼 수 있도록 C가 허용하는 유일한 타입이다.

원시 메모리에 `proven_byte_t`를 쓰는 것은 취향의 문제가 아니다. C의 strict-aliasing 규칙은 잘못된 타입의 포인터로 객체를 읽는 것이 미정의라고 말하며, `unsigned char`가 명시적인 예외다. 이 라이브러리의 모든 버퍼, view, 해시 입력이 `proven_byte_t`인 것은 정확히 그 이유 때문이다.

레퍼런스

이름	의미	비고
`proven_i8`, `proven_i16`, `proven_i32`, `proven_i64`	부호 있는 고정 폭 정수	표준 고정 폭 타입의 별칭.
`proven_u8`, `proven_u32`, `proven_u64`	부호 없는 고정 폭 정수	`proven_u8`은 **수치값**이다. 원시 객체 바이트에는 `proven_byte_t`를 쓰라.
`proven_u16`	16비트 코드 유닛 타입	`<uchar.h>`가 있으면 `char16_t`를, 없으면 `uint_least16_t`를 쓴다. U16 문자열 API가 UTF-16 코드 유닛에 이 타입을 쓴다.
`proven_byte_t`	바이트 단위 객체 표현	`unsigned char`의 별칭. 어떤 객체의 바이트든 합법적으로 들여다볼 수 있는 타입이다.
`proven_size_t`	크기 및 인덱스 타입	`size_t`의 별칭. 부호 없음 — 둘을 빼기 전에 §4를 보라.
`proven_ptrdiff_t`	포인터 차이, 부호 있는 오프셋	`ptrdiff_t`의 별칭.

`proven_intptr_t`, `proven_uintptr_t`	포인터 크기 정수	arena의 범위 검사처럼 명시적인 포인터-정수 작업에만 쓴다.

```
typedef struct {
    proven_err_t  err;
    proven_size_t value;
} proven_result_size_t;
```

`proven_result_size_t`는 크기 또는 에러를 반환한다. `value`는 `err == PROVEN_OK`일 때만 유효하다. 부분 append, 파일 읽기와 쓰기, 크기 조회에서 돌려받는 것이 이것이다 — 답이 "몇 개인가"이면서 연산이 실패할 수 있는 모든 자리에서 쓰인다.

반례 — 바이트를 작은 수의 저장소처럼 다루기:

```
proven_u8 *raw = (proven_u8 *)&some_struct;   /* wrong type for raw inspection */
for (size_t i = 0; i < sizeof some_struct; i++) hash ^= raw[i];
```

주류 컴파일러 전부에서 우연히 동작하지만, 그래도 이것은 잘못 쓴 타입이다. aliasing 보장이 뒤에 있는 쪽은 `proven_byte_t`이고, 그것을 쓰는 것은 여러분이 수가 아니라 표현을 들여다보고 있음을 문서화한다.

3. 메모리 view: 포인터와 길이를 하나로

문제: 포인터는 잊어버린다
포인터는 무언가가 어디서 시작하는지만 말하고 그 외에는 아무것도 말하지 않는다. "어디서 끝나는가"라는 나머지 절반에 대해 C가 지금까지 해 온 모든 것은 그 위에 얹은 관례였다:

- 문자열을 위한 **NUL 종료자** — 길이를 알고 싶을 때마다 O(n) 스캔이 들고, 0 바이트를 포함한 텍스트를 표현할 수 없으며, 종료자가 없어지는 순간 버퍼 오버런을 낸다.
- 그 밖의 모든 것을 위한 **별도의 길이 매개변수** — 누군가 잘못된 값을 넘기기 전까지만 동작한다. 두 인자를 서로 묶어 주는 것이 아무것도 없기 때문이다.

```
void process(const unsigned char *data, size_t len);
process(buf, sizeof buf);        /* fine */
process(buf, sizeof(buf) - 1);   /* also compiles; now the last byte is invisible */
process(other, sizeof buf);      /* also compiles; now it reads past the end */
```

저것 하나하나가 모두 합법적인 호출이다. 포인터와 길이 사이의 관계는 오직 프로그래머의 머릿속에만 존재한다.

이 라이브러리가 대신 하는 일
view는 포인터와 크기를 한 구조체에 담아 값으로 전달한 것이다. 둘은 하나이기 때문에 떨어질 수 없다.

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_mem_view_t;  /* read-only */
typedef struct {       proven_byte_t *ptr; proven_size_t size; } proven_mem_mut_t;   /* writable  */
typedef struct {       proven_byte_t *ptr; proven_size_t size; } proven_mem_t;       /* owned     */
```

이들은 두 워드다. 복사는 공짜이고, 근처 어디에도 할당이 없다. view가 하지 않는 일은 소유다. view는 남의 것인 바이트를 가리키며, 그 바이트가 무효가 되는 순간 함께 무효가 된다. 이 규칙이 0장의 계약 2다.

세 가지 표기는 의도만 다르며, 그 의도가 핵심이다:

- `proven_mem_view_t` — **빌려 쓰는(borrowed), 읽기 전용.** `const`가 가리켜지는 바이트에 붙어 있으므로 컴파일러가 강제한다.
- `proven_mem_mut_t` — **borrowed, 쓰기 가능.** 이를 통해 쓸 수 있지만, 여전히 소유하지는 않는다.
- `proven_mem_t` — **owned.** 누군가는 이것을 만들어 낸 할당자(allocator)로 해제해야 한다. 타입이 그것을 강제하지는 않는다. 알릴 뿐이다.

그리고 실패할 수 있는 연산을 위한 result 래퍼 두 개가 있다:

```
typedef struct { proven_err_t err; proven_mem_mut_t  value; } proven_result_mem_mut_t;
typedef struct { proven_err_t err; proven_mem_view_t value; } proven_result_mem_view_t;
```

proven_result_mem_mut_t는 allocator가 돌려주는 것이다. proven_result_mem_view_t는 검사된 슬라이스가 돌려주는 것이다.

슬라이싱, 검사된 것과 검사되지 않은 것
view의 부분 범위를 취하는 것은 view로 하게 될 가장 흔한 일이며, 의도적으로 두 가지 형태로 제공된다:

API	목적	반환
proven_mem_view_from_owned(mem)	소유(owned) 블록에 대한 읽기 전용 view.	proven_mem_view_t.
proven_mem_mut_from_owned(mem)	owned 블록에 대한 쓰기 가능 view.	proven_mem_mut_t.
proven_mem_view_slice_checked(view, offset, size)	서브뷰, 검증함.	proven_result_mem_view_t.
proven_mem_view_slice_unchecked(view, offset, size)	서브뷰, 검증하지 **않음**.	proven_mem_view_t.
proven_mem_mut_slice_checked(mut, offset, size)	쓰기 가능 서브슬라이스, 검증함.	proven_result_mem_mut_t.
proven_mem_mut_slice_unchecked(mut, offset, size)	쓰기 가능 서브슬라이스, 검증하지 **않음**.	proven_mem_mut_t.
proven_range_contains_ptr(base, cap, ptr, size, out_offset)	이 포인터 범위가 저 할당 안에 있는가? 정수 주소 비교이며, 무관한 포인터끼리 포인터 산술을 하지 않는다.	_Bool.
proven_memcmp(s1, s2, size)	원시 메모리를 비교한다.	같으면 0. 부호는 바이트 순서(부호 없음)에 따른다.
proven_mem_copy(dst, dst_cap, src)	view를 dst로 경계 검사하며 복사한다.	PROVEN_OK. 들어가지 않으면 PROVEN_ERR_OUT_OF_BOUNDS이며 — **아무것도 기록되지 않는다**. 크기가 0이 아닌데 널 포인터이면 PROVEN_ERR_INVALID_ARG. 영역은 겹쳐서는 안 된다.
proven_mem_move(dst, dst_cap, src)	proven_mem_copy와 같지만 영역이 겹쳐도 된다.	위와 같다.
proven_mem_equal_ct(a, b)	두 범위가 같은지를, 어디가 다른지에 좌우되지 않는 시간에 알려준다. 비밀 - MAC, 토큰 - 과 비교할 때 쓴다. proven_memcmp는 앞쪽 몇 바이트가 맞았는지를 흘린다.	bool. 크기가 다른 범위는 같지 않다.
proven_mem_wipe(mem)	컴파일러가 지워 버리지 못하는 방식으로 0을 덮어쓴다. 다 쓴 키나 비밀번호에 쓴다: 버퍼가 범위를 벗어나기 직전의 평범한 지우기 루프는 죽은 코드로 삭제된다.	없음.

검사된 형태의 정확한 동작:

- view.size > 0 && view.ptr == NULL → PROVEN_ERR_INVALID_ARG(뒤에 메모리가 없는 크기는 빈 view가 아니라 버그다).
- 요청한 범위가 view 밖으로 벗어남 → PROVEN_ERR_OUT_OF_BOUNDS.
- size == 0 → PROVEN_OK와 함께 빈 view(ptr == NULL, size == 0).

기본적으로 검사된 형태를 쓰라. 검사되지 않은 형태는 바로 윗줄들에서 이미 범위를 증명한 경우 — 예컨대 여러분이 직접 계산한 경계를 가진 루프 안 — 를 위해 존재하며, 그때 두 번째 검사는 순수 비용일 뿐이다. 그런 경우는 실제로 있고, 느낌보다 좁다.

반례 — 바깥에서 온 수에 검사 없는 슬라이싱을 쓰기:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were never validated. This is the buffer
   overrun from the top of section 3, reintroduced through a safer-looking API. */
```

반례 — 빈 view를 거부하기:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG;   /* wrong: {NULL, 0} is legal */
```

빈 view는 정상적인 값이다. 길이 0인 슬라이스, 내용이 없는 파일, 방금 초기화된 문자열이 그렇다. 잘못된 view를 가려내는 검사는 크기가 0이 아닌데 포인터가 널인 경우다:

```
proven_mem_view_t view = {0};   /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG;   /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

매크로에 view 넘기기

여러 진입점이 함수형 매크로다: `PROVEN_ARG`, `proven_scan_fmt`, print와 `append_fmt` 계열, `PROVEN_MAP_*_U8_*` 래퍼. 전처리기는 괄호 안에 있지 않은 모든 쉼표에서 매크로 인수를 나누는데, 복합 리터럴의 중괄호 안 쉼표는 괄호 안이 아니다. 그래서 `(proven_u8str_view_t){ p, n }`을 넘기면 매크로는 인수 두 개를 받고, 컴파일러 오류는 호출한 적도 없는 내부 함수 이름을 댄다. 변수나 `PROVEN_LIT`을 넘기거나, 복합 리터럴을 괄호로 감싼다:

```
const proven_byte_t bytes[] = { '4', '2' };
proven_u8str_view_t digits = { bytes, sizeof bytes };
proven_println("{}", PROVEN_ARG(digits));                                  /* a variable */
proven_println("{}", PROVEN_ARG(((proven_u8str_view_t){ bytes, 1 })));     /* parentheses */
```

4. 감길 수 없는 크기 산술

왜 곱셈 하나에 함수 호출이 필요한가

`proven_size_t`는 부호가 없고, C의 부호 없는 산술은 오버플로하지 않는다 — 조용하고 합법적으로 2^64를 법으로 감긴다. 그 규칙 하나가 업계 할당 버그의 상당 부분 뒤에 있다:

```
proven_size_t total = count * sizeof(ex_item_t);   /* wrong: wraps for large count */
void *p = malloc(total);                           /* succeeds, and is far too small */
items[count - 1] = x;                              /* writes way past the end */
```

`count = 2^61`이고 항목이 16바이트라면 `total`은 `0`이다. `malloc(0)`은 성공한다. 그 뒤의 모든 쓰기는 범위 밖이고, 어디에서도 에러를 보고하지 않았다. 크기가 프로그램 바깥에서 온 데이터로부터 계산될 때마다 — 파일 헤더, 네트워크 패킷, 사용자가 준 개수 — 같은 모양이 나타난다.

뺄셈에는 거울상 문제가 있고, 이쪽이 더 자주 걸린다:

```
proven_size_t remaining = view.size - offset;   /* wrong when offset > size: a huge number */
```

이 라이브러리가 대신 하는 일

산술을 수행하고 그 결과가 들어맞았는지 알려 주는 매크로 세 개다.

```
#define PROVEN_CKD_ADD(res, a, b)   /* true on overflow */
#define PROVEN_CKD_SUB(res, a, b)
#define PROVEN_CKD_MUL(res, a, b)
```

매크로	계산	반환	*res 기록

PROVEN_CKD_ADD(res, a, b)	a + b	**오버플로 시 true**, 성공 시 false	들어맞을 때만
PROVEN_CKD_SUB(res, a, b)	a - b	**오버플로 시 true**(부호 없는 언더플로 포함)	들어맞을 때만
PROVEN_CKD_MUL(res, a, b)	a * b	**오버플로 시 true**	들어맞을 때만
PROVEN_SIZE_MAX	—	proven_size_t가 담을 수 있는 가장 큰 값	—

- res는 답이 들어갈 곳을 가리키는 **포인터**다.
- 이들은 **오버플로 시 true**, 성공 시 false를 반환한다. 처음에는 거꾸로 읽힌다. 이 관례는 C23의 <stdckdint.h>에서 왔고(컴파일러가 제공하면 이 매크로들이 그것을 사용한다), 그렇지 않으면 GCC/Clang의 __builtin_*_overflow 계열에서 왔다. 호출을 *"이거 오버플로했나?"*로 읽으면 말이 맞아떨어진다.
- PROVEN_SIZE_MAX는 계산한 뒤 검사하는 대신 한계와 직접 비교하고 싶을 때를 위해 있다.

올바른 예:

```
typedef struct { int id; double weight; } ex_item_t;

proven_size_t count = 1024;
proven_size_t total = 0;
proven_err_t err = PROVEN_OK;

if (PROVEN_CKD_MUL(&total, count, sizeof(ex_item_t))) {
    err = PROVEN_ERR_OVERFLOW;   /* refuse to allocate a size that wrapped */
} else {
    proven_result_mem_mut_t block = alloc.alloc_fn(alloc.ctx, total, alignof(ex_item_t));
    err = block.err;
    if (proven_is_ok(err)) {
        alloc.free_fn(alloc.ctx, block.value.ptr);
    }
}
(void)err;
```

반례:

```
proven_size_t total = count * sizeof(ex_item_t);   /* wrong: may wrap silently */
```

프로그램의 모든 산술 식에 이것들이 필요하지는 않다. 필요한 곳은 크기가 **여러분이 직접 고르지 않은 값으로부터 계산되는** 자리이며, 버그가 있는 곳이 정확히 거기다.

5. 정렬

정렬이란 무엇이고, 라이브러리가 왜 이를 언급하는가

C의 모든 타입에는 정렬이 있다. 하드웨어가 효율적으로, 또는 아예 로드할 수 있으려면 그 값이 시작해야 하는 주소다. double은 보통 8로 나누어떨어지는 주소를 원한다. 어떤 아키텍처는 정렬되지 않은 로드에 폴트를 내고, x86은 그저 느려질 뿐이다. 어느 쪽이든 C는 정렬되지 않은 포인터로 객체에 접근하는 것을 미정의 동작이라고 말한다.

여러분이 이것을 생각할 필요가 없었던 이유는 malloc이 무엇에나 알맞게 정렬된 메모리를 반환하기 때문이다. 그 보장은 여러분이 직접 메모리를 나눠 주기 시작하는 순간 정확히 사라진다 — 그리고 직접 메모리를 나눠 주는 일이 바로 2장의 아레나(arena)와 풀(pool)이 하는 일이다. 포인터를 13바이트 밀고 그 결과를 건네주는 arena는 방금 여러분에게 정렬되지 않은 double을 준 것이다.

그래서 여기의 헬퍼들은 allocator를 위해, 그리고 allocator를 직접 쓰는 여러분을 위해 존재한다.

레퍼런스

매크로	의미
PROVEN_DEFAULT_ALIGNMENT	여러분이 달리 말하지 않을 때 라이브러리가 쓰는 기본값. 현재 8이다.
PROVEN_MAX_ALIGN	alignof(max_align_t) — 어떤 내장 타입에도 충분하다.

함수	목적	반환
`proven_is_pow2(x)`	x가 0이 아닌 2의 거듭제곱인가? 정렬값은 그래야 한다.	`bool`.
`proven_mem_align_up(addr, align)`	크기나 주소를 `align`의 다음 배수로 올림한다.	정렬된 값. `align`이 유효하지 않거나 올림이 오버플로하면 0.
`proven_uintptr_align_up(addr, align)`	같은 일을 `proven_uintptr_t` 주소에 대해 한다.	위와 같다.

에러 관례에 주의하라: 이들은 에러 코드를 담을 자리가 없으므로 **실패 시 0**을 반환한다. 0은 결코 유효한 정렬 주소가 아니므로, 0인지 검사하는 것은 모호하지 않다.

올바른 예:

```
typedef struct { int id; double weight; } ex_item_t;

proven_size_t size = 100;
proven_size_t aligned = proven_mem_align_up(size, alignof(ex_item_t));
proven_err_t err = (aligned == 0) ? PROVEN_ERR_OVERFLOW : PROVEN_OK;
(void)err;
```

반례 — 고전적인 비트 조작 버전:

```
proven_size_t aligned = (size + align - 1) & ~(align - 1);   /* wrong: may overflow */
```

저 줄은 수많은 프로덕션 코드에 들어 있고, 표현 범위 꼭대기 근처의 입력을 빼면 모든 입력에 대해 옳다. 그 근처에서는 `size + align - 1`이 작은 수로 감기고, 결과는 시작한 곳보다 낮은 주소가 된다.

실전 예제: view·슬라이싱·정렬을 한자리에서
이 장의 세 가지 생각 — 길이를 스스로 지니고 다니는 뷰(view), 잘라 내기 전에 범위를 검사하는 슬라이스(slice), 2의 거듭제곱이어야만 하는 정렬 경계(alignment boundary) — 은 프로그램이 이들을 한꺼번에 쓸 때 비로소 요점이 드러난다. 아래 프로그램은 고정 크기 레코드(record, 같은 크기의 자료 한 칸) 표를 자기가 소유한 메모리 한 덩이에 담고, 그런 프로그램이라면 반드시 해야 하는 네 가지 일을 한다. 표를 고쳐 쓰면 안 되는 코드에 넘기기, 한 행(row)만 고칠 수 있게 넘기기, 뒤쪽 행들을 앞으로 당겨 한 행을 지우기, 그리고 다른 데서 건네받은 포인터가 애초에 이 표를 가리키기는 하는지 판단하기.

이 중 둘은 C가 조용히 틀리기 가장 쉬운 자리다. 겹치는(overlapping) 영역을 옮기는 것은 `proven_mem_copy`가 약속한 일이 아니어서 `proven_mem_move`라는 별도의 호출이 있다. 그리고 서로 다른 객체에 속할 수도 있는 두 포인터를 `<`로 비교하는 것은 미정의 동작(undefined behaviour, 컴파일러가 "그런 일은 절대 없다"고 가정해도 되는 상황)이므로, "이 포인터가 내 버퍼 안에 있는가"는 `>=`와 `<`가 아니라 `proven_range_contains_ptr`로 묻는다.

이 프로그램은 테스트 스위트가 컴파일하고 실행하므로 낡을 수 없다.

```
#include <string.h>

/*
 * 소유한 버퍼 하나를, 세 가지 방식으로 이야기하기.
 *
 * 이 프로그램은 고정 크기 레코드로 된 작은 표를 자기가 소유한 기억 덩이 하나에 담고,
 * 그런 프로그램이면 반드시 하게 되는 네 가지를 한다.
 *
 *   1. 고치면 안 되는 읽는 쪽에 표를 건네고(읽기 전용 뷰),
 *   2. 한 행만 고쳐도 되는 쪽에 그 행을 건네고(슬라이스),
 *   3. 뒤 행들을 앞으로 덮어 옮겨 한 행을 지우고(겹치는 복사 - 보통의 복사에는
 *      허용되지 않는 일이다),
 *   4. 남이 돌려준 포인터를 받아, 행 번호로 쓰기 전에 애초에 이 표를 가리키기는
 *      하는지 가린다.
 *
```

```
 * 네 걸음 모두 맨 `char *` 가 길이를 잃어버리고 프로그램이 나중에야 그것을 알게 되는
 * 자리다. 여기 쓰인 타입들은 길이를 포인터와 함께 들고 다니므로, 경계에 대한 물음이
 * 실수가 벌어질 바로 그 자리에서 답해진다.
 */

#define ROW_SIZE  8u
#define ROW_COUNT 4u

/* 읽는 쪽은 뷰를 받는다. 모든 바이트를 읽을 수 있고 하나도 쓸 수 없다. 타입에 붙은
 * const 는 권고가 아니다 - 그것을 통해 대입하면 컴파일되지 않는다. */
static proven_size_t count_nonzero(proven_mem_view_t table) {
    proven_size_t n = 0;
    for (proven_size_t i = 0; i < table.size; ++i) {
        if (table.ptr[i] != 0) {
            ++n;
        }
    }
    return n;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 표를 할당한다. 할당자는 읽고 쓸 수 있는 슬라이스를 돌려준다. 소유한 쪽이 쥐는
     * 것이 그것이다 - 포인터와 그에 딸린 크기. */
    proven_result_mem_mut_t got = alloc.alloc_fn(alloc.ctx, ROW_SIZE * ROW_COUNT, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "allocating the record table must succeed");
    if (!proven_is_ok(got.err)) {
        return 1;
    }

    /* proven_mem_t 가 "내가 이것을 소유한다" 는 타입이다. 소유한 덩이를 변수 하나에
     * 두고 거기서 파생한 뷰와 슬라이스를 나눠 주는 것, 그것이 이 라이브러리가 요구하는
     * 소유 규율의 전부다. */
    proven_mem_t owned = { .ptr = got.value.ptr, .size = got.value.size };
    proven_mem_mut_t table = proven_mem_mut_from_owned(owned);

    /* i 번 행을 i + 1 값의 바이트로 채운다. 그래야 옮겨진 행을 알아볼 수 있다. */
    for (proven_u32 row = 0; row < ROW_COUNT; ++row) {
        proven_result_mem_mut_t slot = proven_mem_mut_slice_checked(table, row * ROW_SIZE, ROW_SIZE);
        EXAMPLE_REQUIRE(proven_is_ok(slot.err), "every row of the table must be in bounds");
        if (!proven_is_ok(slot.err)) {
            alloc.free_fn(alloc.ctx, owned.ptr);
            return 1;
        }
        memset(slot.value.ptr, (int)(row + 1), slot.value.size);
    }

    /* 1. 읽기 전용으로 쓰기. 읽는 쪽은 표를 고칠 수도, 끝을 넘어갈 수도 없다.
     *    두 사실을 한꺼번에 건네받았기 때문이다. */
    proven_mem_view_t read_only = proven_mem_view_from_owned(owned);
    EXAMPLE_REQUIRE(count_nonzero(read_only) == ROW_SIZE * ROW_COUNT,
                    "every byte of every row was written");

    /* 2. 없는 행을 달라고 하는 것은 디버깅할 사고가 아니라 다룰 수 있는 오류다.
     *    행이 넷인 표의 4번 행은 끝에서 한 행 지난 자리에서 시작한다. */
    proven_result_mem_mut_t past_end = proven_mem_mut_slice_checked(table, ROW_COUNT * ROW_SIZE, ROW_SIZE);
    EXAMPLE_REQUIRE(past_end.err == PROVEN_ERR_OUT_OF_BOUNDS,
                    "slicing past the end must report OUT_OF_BOUNDS, not return a bad slice");
```

```
/* 3. 2번과 3번 행을 한 행씩 앞으로 옮겨 1번 행을 지운다. 원본과 목적지가
 *    겹치므로, 보통의 복사가 다루면 안 되는 바로 그 경우다. proven_mem_copy 는
 *    겹치지 않는 구역을 계약으로 적고 있고, 겹쳐도 받는 것은 proven_mem_move 다. */
proven_mem_view_t tail = {
    .ptr  = table.ptr + 2 * ROW_SIZE,
    .size = 2 * ROW_SIZE
};
proven_err_t moved = proven_mem_move(table.ptr + 1 * ROW_SIZE, table.size - 1 * ROW_SIZE, tail);
EXAMPLE_REQUIRE(proven_is_ok(moved), "moving the tail of the table down must succeed");
EXAMPLE_REQUIRE(table.ptr[1 * ROW_SIZE] == 3, "row 2 moved into row 1's place");
EXAMPLE_REQUIRE(table.ptr[2 * ROW_SIZE] == 4, "row 3 moved into row 2's place");

/* 옮기기도 다른 모든 것처럼 경계가 있다. 원본을 담기에 목적지가 작으면 거부되고,
 * 아무것도 쓰이지 않는다. */
proven_err_t refused = proven_mem_move(table.ptr, ROW_SIZE, tail);
EXAMPLE_REQUIRE(refused == PROVEN_ERR_OUT_OF_BOUNDS,
                "a move that would not fit must be refused before it writes");

/* 4. 다른 데서 온 포인터. 서로 무관한 포인터를 < 로 비교하는 것은 C 에서 미정의
 *    동작이고 - 컴파일러는 그런 일이 없다고 가정해도 된다 - 그래서 "이 포인터가
 *    내 버퍼를 가리키는가" 는 비교를 옳게 해 주는 함수로 물어야 한다. */
proven_size_t offset = 0;
const proven_byte_t *from_elsewhere = table.ptr + 2 * ROW_SIZE;
bool inside = proven_range_contains_ptr(table.ptr, table.size, from_elsewhere, ROW_SIZE, &offset);
EXAMPLE_REQUIRE(inside, "a pointer to row 2 is inside the table");
EXAMPLE_REQUIRE(offset / ROW_SIZE == 2, "and its offset recovers the row index");

proven_u8 unrelated[ROW_SIZE] = { 0 };
EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size, unrelated, ROW_SIZE, NULL),
                "a pointer into a different object is not inside the table");

/* 안에서 시작하지만 밖에서 끝나는 행도 밖이다. 검사는 시작 자리가 아니라 구간
 * 전체에 대한 것이다. */
EXAMPLE_REQUIRE(!proven_range_contains_ptr(table.ptr, table.size,
                                           table.ptr + table.size - 1, ROW_SIZE, NULL),
                "a range that starts inside but runs past the end is refused");

/* 5. 구간 검사로 그 행이 경계 안임을 증명했다면, 검사 없는 슬라이스가 옳은
 *    호출이다. 하는 일이 없고, 증명은 주석이 아니라 바로 윗줄이다. 그 증명 없이는
 *    절대 쓰지 말 것. */
proven_mem_mut_t row2 = proven_mem_mut_slice_unchecked(table, offset, ROW_SIZE);
EXAMPLE_REQUIRE(row2.size == ROW_SIZE && row2.ptr[0] == 4,
                "the unchecked slice names the row the checked test just proved");

/* 6. 정렬 - 부르는 쪽이 그것을 만나는 유일한 자리다. 정렬이 다른 두 가지를 한
 *    덩이 안에 나란히 놓는 일. 둘째 것의 주소는 올림해야 하고, 올림은 2의 거듭제곱
 *    경계에서만 정의된다 - 그래서 경계를 먼저 검사한다. */
proven_size_t want_align = alignof(proven_u32);
EXAMPLE_REQUIRE(proven_is_pow2(want_align), "an alignment boundary must be a power of two");
EXAMPLE_REQUIRE(!proven_is_pow2(24u), "24 is not a power of two, so it is not a valid boundary");

proven_uintptr_t raw     = (proven_uintptr_t)(table.ptr + 1);   /* 일부러 홀수 */
proven_uintptr_t aligned = proven_uintptr_align_up(raw, want_align);
EXAMPLE_REQUIRE(aligned >= raw, "aligning up never moves backwards");
EXAMPLE_REQUIRE(aligned % want_align == 0, "and the result is on the boundary asked for");
EXAMPLE_REQUIRE(aligned - raw < want_align, "the padding is never more than one boundary");

/* 2의 거듭제곱이 아닌 경계는 그럴듯한 틀린 주소 대신 0 을 돌려준다. 그래서 실수가
 * 아래로 흘러가지 않고 여기서 멈춘다. */
EXAMPLE_REQUIRE(proven_uintptr_align_up(raw, 24u) == 0,
```

```
                "aligning to a non-power-of-two returns 0, not a wrong address");

    alloc.free_fn(alloc.ctx, owned.ptr);
    return EXAMPLE_OK();
}
```

반례 — 겹침을 금지하는 복사로 같은 삭제를 쓴 경우:

```
/* rows 2..3 moved down onto row 1: source and destination overlap */
proven_mem_copy(table.ptr + ROW_SIZE, table.size - ROW_SIZE, tail);   /* wrong */
```

`proven_mem_copy`는 겹치지 않는 영역을 문서로 약속한다. 이 호출은 그 위반을 보고하지 않는다. 그저 가장 빠른 순서로 복사할 자유를 가질 뿐이고, 그 순서가 바뀌는 날 표에는 같은 행이 조용히 두 벌 들어간다.

반례 — 범위 질문을 평범한 포인터 비교로 물은 경우:

```
if (ptr >= table.ptr && ptr + size <= table.ptr + table.size) { /* wrong */ }
```

`ptr`이 다른 할당에서 왔다면 이 비교는 "답이 틀린 비교"가 아니라 미정의 동작이다.

6. Panic: 에러를 돌려줄 상대가 남지 않았을 때

에러를 반환하는 라이브러리에 panic이 왜 있는가
이 장의 모든 내용은 실패가 들여다볼 수 있는 값이어야 한다고 주장해 왔다. 패닉(panic)은 그것이 항상 가능하지는 않다는 인정이다.

어떤 API는 구조상 에러 채널이 없다. `proven_arena_alloc_or_panic()`은 result가 아니라 메모리 블록을 반환하는데, 이는 할당 실패가 프로그램이 아예 실행될 수 없다는 뜻이고 돌려줄 만한 것이 없는, 프로그램의 초기 설정 단계를 위해 존재하기 때문이다. freestanding 빌드에서는 `abort()`도, `stderr`도, 되돌아갈 곳도 없을 수 있다.

그래서 라이브러리는 `proven_panic()`을 호출해 panic을 일으키며, 이는 여러분이 교체할 수 있는 핸들러로 디스패치한다. `exit`를 호출하지도, `stderr`에 출력하지도, 운영체제를 가정하지도 않는다.

```
typedef void (*proven_panic_handler_t)(const char *msg);

void proven_panic(const char *msg);
void proven_set_panic_handler(proven_panic_handler_t handler);
```

API	목적	비고
`proven_panic(msg)`	panic을 일으킨다. 설치된 핸들러로 디스패치한다.	라이브러리의 `_or_panic` API가 호출한다. 여러분이 직접 호출해도 된다.
`proven_set_panic_handler(handler)`	핸들러를 설치한다.	기본값으로 되돌리려면 `NULL`을 넘겨라. 스레드 안전하지 않다 — 다른 스레드가 생기기 전, 시작 단계에서 설치하라.
`proven_panic_handler_t`	`void (*)(const char *msg)`	핸들러는 반환해서는 안 된다.

기본 핸들러는 trap한다: GCC와 Clang에서는 `__builtin_trap()`이고, 그 밖의 컴파일러에서는 무한 루프다. 디스패치는 weak symbol이 아니라 함수 포인터를 거치므로, ELF와 PE/COFF(Windows, mingw-w64) 툴체인에서 같은 방식으로 링크된다.

직접 설치하기(파일 스코프 함수이므로, 이는 다른 함수 안에 붙여넣는 것이 아니라 리스팅이다):

```
static void my_panic(const char *msg) {
    log_critical(msg);            /* your logger */
    for (;;) {
        /* reset, halt, or wait for a debugger */
    }
```

```
}

proven_set_panic_handler(my_panic);   /* pass NULL to restore the default */
```

panic 핸들러는 반환해서는 안 된다. 반환하면 `_or_panic` 계열은 돌려줄 것이 없고, 그 result의 유효성은 보장되지 않는다.

반례 — 반환하는 핸들러:

```
static void my_panic(const char *msg) {
    fprintf(stderr, "%s\n", msg);   /* wrong: this returns, and then the caller
                                       proceeds with a block that was never allocated */
}
```

반례 — 평범한 코드에서 `_or_panic`에 손을 뻗기:

```
proven_mem_mut_t block = proven_arena_alloc_or_panic(&arena, n);
/* wrong for anything that handles input: an arena that is merely full has just
   killed the process. Use proven_arena_alloc and read the error. */
```

7. 버전 매크로

진단용으로, 그리고 라이브러리 버전에 맞춰 적응해야 하는 코드를 위한 컴파일 타임 식별자다.

```
#define PROVEN_VERSION_MAJOR  0
#define PROVEN_VERSION_MINOR  92
#define PROVEN_VERSION_PATCH  0
#define PROVEN_VERSION_STRING "proven_c_lib-v0.92.0"
#define PROVEN_VERSION_ENCODE(major, minor, patch) ((major) * 1000000L + (minor) * 1000L + (patch))
#define PROVEN_VERSION_NUM    PROVEN_VERSION_ENCODE(PROVEN_VERSION_MAJOR, PROVEN_VERSION_MINOR,
PROVEN_VERSION_PATCH)
```

버전은 시맨틱 버전 — `MAJOR.MINOR.PATCH` — 이고 v0.0.1부터 시작한다. PATCH 릴리스는 고치거나 문서화할 뿐 호출자가 바꿔야 할 것이 없다. MINOR 릴리스는 기존 호출을 깨지 않고 더한다. MAJOR 릴리스는 기존 호출을 깰 수 있다. MAJOR가 0인 동안은 MINOR 사이에서도 API가 움직일 수 있다.

`PROVEN_VERSION_STRING`은 빌드 보고서나 `--version` 플래그에서 출력하는 것이다. 숫자 셋은 라이브러리 버전에 맞춰 적응해야 하는 코드를 위한 것이고, `PROVEN_VERSION_NUM`은 그 셋을 `#if`용 정수 하나로 접은 것이다 — 리터럴이 아니라 `PROVEN_VERSION_ENCODE`와 비교한다: `#if PROVEN_VERSION_NUM >= PROVEN_VERSION_ENCODE(0, 1, 0)`.

v0.0.1 이전의 릴리스는 날짜로 번호를 붙였고(`v26.MM.DDx`), `PROVEN_VERSION_NUM`은 그 날짜였다. 번호는 체계와 함께 처음부터 다시 시작했으므로, 옛 날짜 값과의 비교는 지금의 모든 릴리스에서 틀린다. `PROVEN_VERSION_ENCODE`로 다시 쓰라.

숫자와 문자열은 빌드가 `include/proven/version.h`와 대조해 검사하고, 문자열은 숫자와도 대조하므로, 매뉴얼이 헤더로부터 어긋날 수 없다. (한 번 어긋난 적이 있고, 이 문장이 다시는 그러지 않을 이유다: 문자열에는 게이트가 걸려 있었고 나머지에는 없었다.)

8. 예제와 오용 사례

안전한 result 처리

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };

proven_result_mem_view_t part = proven_mem_view_slice_checked(view, 4, 8);
if (!proven_is_ok(part.err)) {
    return;   /* the range was not inside the view */
}

proven_byte_t payload[8];
proven_err_t err = proven_mem_copy(payload, sizeof payload, part.value);
(void)err;
```

증명한 뒤에만 검사 없는 슬라이싱

호출자가 이미 범위를 증명했을 때 올바른 예:

```
proven_byte_t record[16] = {0};
proven_mem_view_t view = { .ptr = record, .size = sizeof record };
proven_size_t offset = 4;
proven_size_t size = 8;

if (offset <= view.size && size <= view.size - offset) {
    proven_mem_view_t part = proven_mem_view_slice_unchecked(view, offset, size);
    (void)part;   /* proved in range: safe to read */
}
```

저 검사의 모양에 주목하라. 더 자연스러운 `offset + size <= view.size` 대신 `size <= view.size - offset`으로 쓴 것은 정확히 §4 때문이다. 앞서 나온 `offset <= view.size` 검사가 있으면 합은 감길 수 있지만 차는 감길 수 없다.

반례:

```
proven_mem_view_t part = proven_mem_view_slice_unchecked(view, user_offset, user_size);
/* wrong: user_offset and user_size were not validated */
```

빈 view는 허용된다

크기가 0인 view는 널 포인터를 가질 수 있다. 그것을 무턱대고 거부하지 마라.

반례:

```
if (!view.ptr) return PROVEN_ERR_INVALID_ARG; /* wrong for empty views */
```

더 나은 예:

```
proven_mem_view_t view = {0};   /* size 0, ptr NULL: a legal empty view */
proven_err_t err = PROVEN_OK;

if (view.size > 0 && !view.ptr) {
    err = PROVEN_ERR_INVALID_ARG;   /* only a null pointer with bytes behind it is a bug */
}
(void)err;
```

다음으로 갈 곳

2장은 이 타입들이 일을 시작하는 곳이다. allocator는 `proven_mem_mut_t`를 만들어 내고, arena는 §5의 정렬 헬퍼를 사용하며, 라이브러리의 모든 확장 가능 컨테이너는 allocator를 매개변수로 받는다. §1이 에러를 반환값으로 받는 이유로 든 것과 같은 이유 때문이다 — 비용은 시그니처에 보여야 한다.

2장: 할당 — 힙, 아레나, 풀, 버퍼

Part II — 모든 프로그램이 쓰는 어휘. 선행 장: 1장. 이 장을 마치면 반사적으로 `malloc`을 집어드는 대신 할당 전략을 의도적으로 고를 수 있고, 세 가지 비용 중 어느 것을 치르고 있는지 알게 된다.

이 장은 `heap.h`, `arena.h`, `pool.h`, `allocator.h`, `buffer.h`를 다룬다. 예전에 이 장의 7절이었던 스레드 안전성과 포인터 provenance 자료는 이제 나머지 동시성 주제와 함께 6장에 있다 — 책에서 가장 어려운 자료가 맨 앞쪽 장 중 하나에 들어앉아 있었기 때문이다.

목차

1. 할당이 매개변수인 이유, 그리고 heap allocator
2. Arena: 여러 객체, 하나의 수명
3. Pool: 여러 객체, 하나의 크기
4. Allocator trait
5. 원시 바이트 버퍼
6. 예제와 오용 사례
7. 잘못된 할당자 잡기: `alloc_check.h`

1. 할당이 매개변수인 이유, 그리고 heap allocator

`malloc`의 문제

`malloc`은 전역이다. 어떤 함수든 호출할 수 있고, 시그니처의 그 무엇도 함수가 호출하는지 아닌지를 말해 주지 않으며, 메모리가 무엇에 쓰이든 모든 호출은 똑같은 범용 allocator로 간다.

여기서 따라 나오는 결과가 넷이고, 아마 이미 겪어 봤을 것이다:

- **시그니처만 보고는 함수가 할당하는지 알 수 없다.** `char *build_message(int n)`은 할당할 수도, 정적 버퍼 안의 포인터를 반환할 수도, 리터럴을 반환할 수도 있다. 세 경우 모두 타입이 같으므로 호출자는 해제해야 하는지 알 수 없고, 답은 틀렸을지도 모르는 주석 속에 있다.
- **프로그램의 한 부분만 전략을 바꿀 수 없다.** 파서가 파싱이 끝날 때 전부 죽는 작은 할당을 만 번 한다면, `malloc`과 `free`는 만 번의 범용 할당과 만 번의 해제를 한다. bump allocator라면 한 번이면 된다. 모든 호출 지점을 다시 쓰지 않고는 그렇게 하겠다고 말할 방법이 없다.
- **실패 경로를 테스트할 수 없다.** `malloc`을 원할 때 실패시키려면 전역으로 가로채야 한다 — `LD_PRELOAD`, 링커 트릭, 라이브러리 내부 호출까지 잡아채는 `#define malloc my_malloc`. 그런데 정작 가장 테스트하고 싶은 분기는 메모리가 바닥났을 때 실행되는 그 분기다.
- **힙이 없는 곳에서는 아예 쓸 수 없다.** 펌웨어, 커널, 부트로더: `malloc`이 없고, 그것을 전제하는 모든 라이브러리는 쓸 수 없다.

`malloc`은 잘못 설계된 것이 아니다. 그것은 고정된 정책이며, 다르게 하라고 말할 매개변수를 받지 않는다는 사실 때문에 모든 호출 지점에 하드코딩되어 있는 것이다.

이 라이브러리가 대신 하는 것

allocator는 값이고, 매개변수다. 할당할 수 있는 함수는 `proven_allocator_t`를 받고, 할당할 수 없는 함수는 받지 않는다. 발상은 그게 전부이고, 모든 결과가 여기서 따라 나온다:

```
proven_err_t proven_u8str_append(proven_u8str_t *str, proven_u8str_view_t data);
proven_err_t proven_u8str_append_grow(proven_allocator_t alloc, proven_u8str_t *str, proven_u8str_view_t
data);
```

시그니처를 읽어 보라. 첫 번째는 문자열을 키울 수 없으므로 텍스트가 들어가지 않으면 실패한다. 두 번째는 키울 수 있고, allocator를 받음으로써 그렇다고 말한다. 어느 쪽을 호출했는지 궁금해할 일이 결코 없다.

이제 같은 코드가 호출자가 고르는 세 가지 서로 다른 전략과 함께 동작한다:

Allocator	얻는 방법	개별 해제?	쓸 때

Heap	`proven_heap_allocator()`	예	일반적인 경우. 서로 무관한 수명을 가진 객체들.
Arena	`proven_arena_create(backing)` 후 `proven_arena_as_allocator(&a)`	**아니오** — free는 no-op이다; 전체를 reset하거나 destroy한다	전부 같은 순간에 죽는 많은 할당: 요청 하나, 프레임 하나, 파싱 하나.
Pool	`proven_pool_init(&p, base, size, align, bin_cap)` 후 `proven_pool_as_allocator(&p)`	예, free list로 들어간다	**하나의 고정 크기**인 객체를 반복해서 할당하고 해제할 때.

힙(heap)에서 시작하라. 이유가 있을 때 나머지 둘로 손을 뻗되, 그 이유는 대개 측정이다.

Heap allocator(힙 할당자)

```
proven_allocator_t proven_heap_allocator(void);
```

이것은 라이브러리의 인터페이스를 입은 `malloc`, `realloc`, `free`다. 그러지 않을 구체적인 이유가 없다면 이것을 써야 하며, 여기에 영리한 구석은 전혀 없다 — 그게 요점이다. 이 매뉴얼에서 달리 할 이유가 없는 모든 예제는 이것을 쓴다:

```
proven_allocator_t heap = proven_heap_allocator();

proven_result_u8str_t s = proven_u8str_create(heap, 64);
if (!proven_is_ok(s.err)) {
    return;   /* nothing was created, so there is nothing to destroy */
}
(void)proven_u8str_append(&s.value, PROVEN_LIT("ready"));
proven_u8str_destroy(heap, &s.value);   /* the SAME allocator */
```

값은 네 워드 — 컨텍스트 포인터 하나와 함수 포인터 셋 — 이고 값으로 전달된다. 복사는 공짜이고, 자신의 struct에 저장해도 되며, 파괴할 필요도 없다.

프리스탠딩(freestanding) 빌드에서는 `proven_heap_allocator`가 존재하지 않는다. 감쌀 `malloc`이 없기 때문이다. 이는 우회해야 할 제약이 아니라, 라이브러리 전체가 allocator를 매개변수로 받는 바로 그 이유이며, 정적 배열 위의 아레나(arena)가 같은 코드를 마이크로컨트롤러에서 돌아가게 하는 이유다. freestanding 모드를 참고하라.

잘못된 예 — 생성할 때와 다른 allocator로 파괴하기:

```
proven_result_u8str_t s = proven_u8str_create(arena_alloc, 64);
proven_u8str_destroy(heap_alloc, &s.value);   /* wrong: heap free on arena memory */
```

객체는 자신을 만든 allocator를 기억하지 않는다 — 그것이 객체를 작게 유지하는 요인이다. 오늘날 이것을 검사하는 것은 없으며, 실패는 다른 어딘가에서 나중에 드러나는 힙 손상이다. 구조적으로 짝을 지어라: allocator를 객체 옆에 두거나, 둘을 함께 전달하라.

2. Arena: 여러 객체, 하나의 수명

문제: 사망 시점이 같은 수많은 작은 할당

설정 파일을 파싱한다고 하자. 키마다 문자열 하나, 값마다 문자열 하나, 섹션마다 노드 하나를 할당한다 — 작은 할당 수천 개다. 그리고 파싱이 끝나면 결과를 만들고, 그 할당 하나하나는 모두 같은 순간에 쓰레기가 된다.

`malloc`으로는 그 대가를 두 번 치른다. 할당마다 free list를 검색하고, 장부를 갱신하고, 헤더가 붙은 메모리를 반환한다; `free`마다 메모리를 돌려주고 어쩌면 이웃과 병합한다. 그리고 하나도 빠뜨려선 안 된다. 잊어버린 `free` 하나면 누수이기 때문이다.

arena가 하는 관찰은 **그 객체들이 전부 같은 수명을 가진다**는 것이며, 따라서 개별 해제는 무의미한 작업이라는 것이다. 함께 죽는다면, 함께 해제될 수 있다.

Arena가 동작하는 방식

arena는 메모리 블록 하나와 오프셋 하나다. 할당한다는 것은 요청받은 정렬에 맞게 오프셋을 올림하고, 그 주소를 반환하고, 오프셋을 앞으로 옮기는 것이다. 알고리즘 전체가 이게 전부다:

```
[####used####|                    free                    ]
             ^ offset
```

- **할당은 명령어 몇 개다.** 검색도, free list도, 할당마다 붙는 헤더도 없다.
- **`free`는 아무 일도 하지 않는다.** 의도적으로 no-op이다.
- **회수는 arena 전체를 reset하거나 destroy해서 한다.** 오프셋을 0으로 되돌리는 것이다. 연산 하나가 객체 만 개를 해제한다.

메모리는 여러분이 준다. `proven_arena_create`는 `proven_mem_mut_t`를 받는다 — 힙에서 받아 온 블록이든, `static` 배열이든, 스택 위의 영역이든 상관없다. arena는 스스로 할당하지 않으며, 그것이 밑에 힙이 없어도 동작하게 하는 요인이다.

포기하는 것

arena는 범용 allocator가 아니며, 그 맞바꿈은 실제적이다:

- **객체 하나를 해제할 수 없다.** 수명이 실제로 공유되지 않는다면, arena는 설계상 누수한다 — 메모리는 reset에서만 회수된다.
- **바닥나는 것은 단단한 한계다.** backing 블록은 고정되어 있다. 여기서의 `PROVEN_ERR_NOMEM`은 기계의 메모리가 바닥났다는 뜻이 아니라, 이 arena가 바닥났다는 뜻이다.
- **arena를 가리키는 모든 포인터는 reset에서 죽는다.** 한꺼번에, 아무 경고도 없이. reset보다 오래 사는 뷰(view)는 dangling view다. 0장의 계약 2를 보라.

수명 공유가 프로그램에 대한 사실일 때 arena를 쓰라. 희망 사항일 때가 아니라.

```
proven_arena_t
typedef struct {
    proven_mem_mut_t backing;
    proven_size_t offset;
} proven_arena_t;
```

필드:

- `backing`: 호출자가 소유하는 가변 메모리 범위.
- `offset`: `backing` 내에서 다음 할당 위치.

Arena 함수와 헬퍼

API	의도	매개변수	반환
`proven_arena_create(backing)`	backing 슬라이스 위에 arena를 초기화한다.	`backing`: 호출자 소유 메모리.	`proven_arena_t`.
`proven_arena_reset(arena)`	arena의 모든 할당을 버린다.	`arena`.	void.
`proven_arena_destroy(arena)`	형식적 정리.	`arena`.	void; 호출자 backing arena에는 no-op.
`proven_arena_alloc_aligned(arena, size, align)`	명시적 정렬로 할당한다.	`arena`, 바이트 `size`, 2의 거듭제곱 `align`.	`proven_result_mem_mut_t`.
`proven_arena_realloc_aligned(arena, old_ptr, old_size, new_size, align)`	재할당; 꼬리(tail) 할당을 제자리에서 확장/축소할 수 있다.	이전 할당 세부 정보와 새 크기.	`proven_result_mem_mut_t`.
`proven_arena_alloc(arena, size)`	`PROVEN_DEFAULT_ALIGNMENT`로 할당한다.	`arena`, 바이트 `size`.	`proven_result_mem_mut_t`.

`proven_arena_alloc_aligned_or_panic(arena, size, align)`	할당하거나 패닉(panic)을 호출한다.	`arena`, `size`, `align`.	`proven_mem_mut_t`.
`proven_arena_alloc_or_panic(arena, size)`	기본 정렬 panic 할당.	`arena`, `size`.	`proven_mem_mut_t`.
`proven_arena_as_allocator(arena)`	arena를 `proven_allocator_t`로 노출한다.	`arena`.	allocator trait, 또는 널 arena에 대해 제로 allocator.

Trait 어댑터 헬퍼:

- `proven_arena_alloc_trait`
- `proven_arena_realloc_trait`
- `proven_arena_free_trait`

이들은 allocator 트레잇(trait)가 함수 포인터를 필요로 하기 때문에 노출된다. 애플리케이션 코드는 보통 대신 `proven_arena_as_allocator()`를 호출한다.

예제:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 64);
if (!proven_is_ok(r.err)) {
    return;   /* the arena cannot grow: it reports NOMEM instead */
}

proven_arena_reset(&arena);   /* reclaims r and everything else at once */
proven_arena_destroy(&arena);
```

성장 가능한 컨테이너와 함께 쓰는 arena
성장 가능한 컨테이너는 arena allocator를 사용할 수 있지만, arena의 `free`가 no-op이므로 성장이 이전 블록을 버릴 수 있다. 가능하면 용량을 미리 확보(reserve)하라.

올바른 예:

```
alignas(max_align_t) proven_byte_t storage[4096];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

/* Ask for the capacity up front, so growth never has to abandon a block. */
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 128);
if (!proven_is_ok(ar.err)) {
    return;
}
(void)PROVEN_ARRAY_PUSH(&ar.value, int, 10);
PROVEN_ARRAY_DESTROY(&ar.value);   /* correct, but arena free reclaims nothing */
proven_arena_reset(&arena);        /* this is what gives the bytes back */
```

잘못된 예 — 아주 작은 초기 용량에서 arena 안으로 성장시키기:

```
proven_result_array_t ar = PROVEN_ARRAY_INIT(a, int, 1);
for (int i = 0; i < 10000; ++i) {
```

```
    PROVEN_ARRAY_PUSH(&ar.value, int, i); /* wrong: every regrow abandons the old block */
}
```

재성장할 때마다 arena에 더 큰 블록을 요청하고 이전 것을 "해제"하는데 — arena에서 그것은 아무 일도 하지 않는다. 배열 자체는 올바르게 끝나지만, arena는 지금까지 할당한 모든 중간 크기를 그대로 쥔 채 끝난다: 원하던 버퍼 하나 위에 1, 2, 4, 8 … 8192개 원소어치의 버려진 공간이 얹힌다. 위의 올바른 예처럼 용량을 미리 확보하라.

잘못된 예 — reset보다 오래 사는 view:

```
proven_u8str_view_t name = /* ... built in the arena ... */;
proven_arena_reset(&arena);
use(name);                 /* wrong: those bytes are now free space */
```

이것이 arena의 가장 날카로운 모서리다. reset은 회수하는 메모리를 건드리지 않으므로 바이트는 보통 아직 그대로 있고, 버그는 보통 테스트에서 드러나지 않는다 — 다음 할당이 그 위에 덮어쓰기 직전까지는.

3. Pool: 여러 객체, 하나의 크기

arena가 풀지 못하는 문제

arena는 수명이 공유된다고 가정한다. 정반대 모양을 한 작업 부하도 많다: 한 타입의 객체가 특별한 순서 없이 계속 만들어지고 파괴되는 경우다. 이벤트의 연결 리스트. 커졌다 작아졌다 하는 트리의 노드들. 생겼다 사라지는 연결 레코드들.

arena는 이것을 할 수 없다 — 객체 하나를 결코 회수하지 않으므로, 오래 도는 프로그램은 한없이 커질 것이다. 힙은 할 수 있고, 그것이 정확히 그 비용이다: 할당마다 검색하고, 해제마다 장부를 갱신하며, 범용 allocator는 이미 천 번이나 처리한 요청에 대해 범용적인 일을 한다.

풀(pool)이 하는 관찰은 **이 객체들이 전부 같은 크기**라는 것이다. 크기가 같다면, 해제된 것 하나가 앞으로의 요청에 정확히 들어맞으므로, 아무런 검색 없이 곧바로 다시 건네줄 수 있다.

Pool이 동작하는 방식

pool은 해제된 블록들의 작은 스택 — bin — 을 유지하고, 뻔한 일을 한다:

- **할당**: bin에 뭔가 있으면 하나를 pop해서 반환한다. 포인터 읽기 하나와 감소 하나다. bin이 비어 있을 때만 하위 allocator로 간다.
- **해제**: bin에 자리가 있으면 재사용을 위해 블록을 push한다. bin이 꽉 찼으면 pool이 메모리를 영원히 세워 두지 않도록 하위 allocator로 되돌려준다.

따라서 `bin_cap`은 다이얼이다: 이 pool이 예비로 쥐고 있어도 되는 메모리의 양이다.

포기하는 것

- **하나의 pool은 정확히 하나의 크기와 정렬만 처리한다.** 그 밖의 요청은 다른 데서 처리되는 것이 아니라 `PROVEN_ERR_INVALID_ARG`로 거부된다. pool이 만들어질 때보다 더 엄격한 정렬도 거부되고, 더 느슨한 것은 블록이 이미 그것을 충족하므로 괜찮다.
- **pool은 살아 있는 객체를 추적하지 않는다.** `proven_pool_destroy`는 bin에 있는 것을 해제하지, 여러분이 아직 쥐고 있는 것을 해제하지 않는다. 할당한 것을 먼저 전부 해제하라. 그러지 않으면 누수인 데다가 이제 dangling이다.

arena 대 pool을 한 줄씩으로: arena는 *많이 할당하고 한 번에 전부 해제*를 위한 것이고, pool은 *같은 크기를 몇 번이고 저렴하게 할당하고 해제*하기 위한 것이다.

```
proven_pool_t
typedef struct {
    proven_allocator_t base_alloc;
    proven_size_t item_size;
    proven_size_t item_align;
    void **bin;
    proven_size_t bin_cap;
```

```
    proven_size_t bin_len;
} proven_pool_t;
```

필드:

- `base_alloc`: 새 블록과 재활용 bin에 사용되는 allocator.
- `item_size`: pool이 받아들이는 정확한 객체 크기. 그 외 크기는 `PROVEN_ERR_INVALID_ARG`로 거부된다.
- `item_align`: 모든 블록이 할당되는 정렬. 더 엄격한 정렬 요청은 `PROVEN_ERR_INVALID_ARG`로 거부되고, 더 느슨한 요청은 블록이 이미 그것을 충족하므로 처리된다.
- `bin`: 캐시된 free-list 배열.
- `bin_cap`: 최대 캐시 블록 개수.
- `bin_len`: 현재 캐시 블록 개수.

재활용이 작동하는 방식

pool은 하나의 큰 슬랩(slab)을 소유하지 않는다. 각 항목을 `base_alloc`에서 개별적으로 할당하고, 해제된 블록들의 작은 **스택**(`bin`, 최대 `bin_cap`개의 포인터로 이루어진 배열)을 유지한다:

- **할당.** `bin_len > 0`이면 맨 위 포인터를 pop한다(O(1), `base_alloc` 호출 없음) — 이것이 pool의 전체 요점이다. 그렇지 않으면 `base_alloc`로 넘어간다.
- **해제.** `bin_len < bin_cap`이면 재사용을 위해 포인터를 bin에 push한다; 그렇지 않으면(bin이 꽉 참) `base_alloc`로 곧바로 되돌려 해제한다. 따라서 bin은 pool이 재사용을 위해 세워둘 수 있는 메모리의 양을 제한한다.
- **파괴.** `proven_pool_destroy`는 bin에 아직 남아 있는 모든 포인터와 bin 배열 자체를 해제한다 — 하지만 살아 있는(넘겨준) 항목은 추적하지 **않으므로**, pool을 파괴하기 전에 할당한 모든 것을 해제해야 한다.

이는 pool을 지역(region) allocator가 아니라 수명이 짧은 동일 타입 객체(노드, 이벤트)를 위한 churn 최적화기(optimizer)로 만든다. "많이 할당하고 한 번에 전부 해제"하고 싶으면 `arena`를 사용하라; "같은 크기를 몇 번이고 저렴하게 할당/해제"하고 싶으면 `pool`을 사용하라.

반례

```
/* WRONG: the pool only handles its configured size/alignment. */
proven_allocator_t a = proven_pool_as_allocator(&pool);   /* item_size == sizeof(Node) */
a.alloc_fn(a.ctx, sizeof(BigThing), alignof(BigThing));   /* not the pool's item -> rejected */

/* WRONG: destroying while items are still live leaks (and dangles) them. */
void *p = a.alloc_fn(a.ctx, sizeof(Node), alignof(Node)).value.ptr;
proven_pool_destroy(&pool);   /* `p` is NOT freed and is now dangling */
/* RIGHT: free every handed-out item first, then destroy. */
```

Pool 함수

API	의도	반환
`proven_pool_init(pool, base_alloc, item_size, item_align, bin_cap)`	고정 크기 pool을 초기화한다.	`PROVEN_OK` 또는 에러.
`proven_pool_as_allocator(pool)`	pool이 뒷받침하는 allocator trait를 반환한다.	`proven_allocator_t`.
`proven_pool_destroy(pool)`	캐시된 블록과 bin 저장소를 해제한다.	void.

예제:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
proven_err_t e = proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 64);
if (!proven_is_ok(e)) {
    return;
}
```

```
proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    /* Hand it back before destroy: the pool does not track live items. */
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

proven_pool_destroy(&pool);
```

잘못된 예:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(LargerObject), alignof(LargerObject));
/* wrong: one pool is for one fixed object size and alignment */
```

4. Allocator 트레잇

이 절이 첫 번째가 아니라 네 번째인 이유

여러분은 세 가지 allocator를 그들이 공유하는 인터페이스를 보지 않고 이미 써 보았고, 그것은 의도적이었다. 인터페이스는 이 발상에서 가장 재미없는 부분이고 아무 맥락 없이 읽기에 가장 어려운 것이다: 함수 포인터 typedef 셋과 정렬 계약은, 그 뒤에 있는 heap과 arena와 pool을 보기 전까지는 거의 아무 의미도 없다.

이 절이 필요한 이유는 둘이다: 자신만의 allocator를 작성하는 것, 그리고 이 라이브러리의 모든 allocator가 지키기로 약속한 규칙을 이해하는 것. 위의 셋을 쓰기만 할 생각이라면 `proven_heap_allocator()`, `proven_arena_as_allocator()`, `proven_pool_as_allocator()`가 API 전부이니 건너뛰어도 된다.

여기서 trait란 인터페이스로 쓰이는 함수 포인터 struct를 말한다 — 손으로 써 낸, C 버전의 가상 테이블이다. `proven_allocator_t`는 라이브러리에서 가장 중요한 trait이고, `stream.h`와 `random.h`도 같은 모양을 쓴다.

함수 포인터 타입

```
typedef proven_result_mem_mut_t (*proven_alloc_fn_t)(
    void *ctx,
    proven_size_t size,
    proven_size_t align
);

typedef proven_result_mem_mut_t (*proven_realloc_fn_t)(
    void *ctx,
    void *old_ptr,
    proven_size_t old_size,
    proven_size_t new_size,
    proven_size_t align
);

typedef void (*proven_free_fn_t)(void *ctx, void *ptr);
```

의도:

- `alloc_fn`은 새로운 바이트 슬라이스를 할당한다.
- `realloc_fn`은 할당 크기를 변경하며 failure-atomic이어야 한다: 실패 시 이전 블록은 여전히 유효하고 변경되지 않으므로, 호출자는 아무것도 잃지 않는다.
- 블록은 할당될 때와 **같은** `align`으로 재할당되고 해제되어야 한다. allocator는 과도하게 정렬된(over-aligned) 요청에 대해 일반 요청과는 다른 하위 메커니즘을 선택할 수 있으며, heap allocator가 실제로 그렇게 한다: `align <= alignof(max_align_t)` — 이 라이브러리의 모든 문자열, 버퍼, 바이트 배열이 여기에 해당한다 — 는 제자리(in place)에서 성장이 일어날 수 있도록 `malloc`/`realloc`을 거치고, 그보다 더 엄격하게 정렬된 것은 정렬된 allocator를 거친다. 블록을 다른 정렬 클래스로 되돌려주는 것은 정의되지 않은 동작이다.
- `free_fn`은 메모리를 해제한다. allocator가 크기 메타데이터를 필요로 한다면, 내부적으로 그것을 추적해야 한다.

`proven_allocator_t`

```
typedef struct {
    void *ctx;
    proven_alloc_fn_t alloc_fn;
    proven_realloc_fn_t realloc_fn;
    proven_free_fn_t free_fn;
} proven_allocator_t;
```

필드:

- `ctx`: allocator별 상태.
- `alloc_fn`: 할당 함수.
- `realloc_fn`: 재할당 함수.
- `free_fn`: 해제 함수.

레퍼런스

멤버 / API	형태	계약
`ctx`	`void *`	allocator 자신의 상태. 호출자에게는 불투명하며, 첫 번째 인자로 되돌아온다.
`alloc_fn`	`proven_alloc_fn_t`	`align` 정렬로 `size` 바이트를 할당한다. `proven_result_mem_mut_t`를 반환한다.
`realloc_fn`	`proven_realloc_fn_t`	크기를 바꾼다. **failure-atomic**: 실패 시 이전 블록은 손대지 않은 채 여전히 유효하다.
`free_fn`	`proven_free_fn_t`	해제한다. 블록이 할당될 때와 같은 `align` 클래스를 주어야 한다.
`proven_alloc_is_valid(alloc)`	`static inline bool`	모든 함수 포인터가 널이 아닐 때, 즉 trait를 호출할 수 있을 때 true.

이 라이브러리의 모든 allocator가 지키며, 여러분의 것도 지켜야 하는 세 가지 규칙:

1. **같은 allocator, 전체 수명 동안.** 한 블록의 할당, 재할당, 해제는 하나의 allocator로 하라.
2. **같은 정렬 클래스.** 어떤 정렬로 할당된 블록은 그 정렬로 재할당되고 해제되어야 한다. allocator는 과도하게 정렬된 요청에 다른 메커니즘을 쓸 수 있다.
3. **실패는 아무것도 바꾸지 않는다.** 실패한 `realloc_fn`은 이전 블록을 유효하고 변경되지 않은 채로 남긴다.

```
static inline bool proven_alloc_is_valid(proven_allocator_t alloc);
```

목적: 모든 함수 포인터가 널이 아닌지 확인한다.

반환: trait를 호출할 수 있으면 true.

올바른 예:

```
proven_allocator_t heap = proven_heap_allocator();
proven_err_t err = proven_alloc_is_valid(heap) ? PROVEN_OK : PROVEN_ERR_UNSUPPORTED;
(void)err;   /* a freestanding build hands back a zero allocator, not a valid one */
```

잘못된 예:

```
proven_allocator_t alloc = {0};
proven_result_mem_mut_t r = alloc.alloc_fn(alloc.ctx, 64, 8); /* wrong: null call */
```

5. 원시 바이트 버퍼

무엇을 위한 것인가

`proven_buf_t`는 이 장에서 가장 단순한 것이다: 포인터 하나, 길이 하나, 용량 하나. 자기 바이트를 소유하고, 결코 성장하지 않으며, `proven_u8str_t`와 `proven_u16str_t`가 이것으로 만들어져 있다.

*텍스트*가 아니라 *바이트*를 원할 때 — 조립 중인 레코드, 소켓에 곧 써 넣을 프레임 — 그리고 최대 크기를 미리 알 때 직접 손을 뻗어라. 내용이 텍스트라면 대신 3장의 문자열 타입을 쓰라. 같은 저장소에 더해 NUL 종료, view, 검색, 포매팅까지 준다.

여기 있는 다른 모든 것과 마찬가지로 allocator를 저장하지 **않으므로**, `proven_buf_destroy`는 `proven_buf_create`에 주었던 것과 같은 것을 받는다.

proven_buf_t

```
typedef struct {
    proven_byte_t *ptr;
    proven_size_t len;
    proven_size_t cap;
} proven_buf_t;
```

필드:

- `ptr`: 저장소 포인터.
- `len`: 현재 사용 중인 바이트.
- `cap`: 할당된 바이트.

proven_result_buf_t

```
typedef struct {
    proven_err_t err;
    proven_buf_t value;
} proven_result_buf_t;
```

버퍼 함수

API	의도	반환
`proven_buf_create(alloc, cap)`	용량이 0이 아닌 버퍼를 할당한다.	`proven_result_buf_t`; 유효하지 않은 allocator 또는 용량 0은 `PROVEN_ERR_INVALID_ARG`를 반환한다.
`proven_buf_append(buf, data)`	원시 바이트가 들어맞으면 추가한다.	`PROVEN_OK` 또는 에러.
`proven_buf_destroy(alloc, buf)`	일치하는 allocator로 버퍼 저장소를 해제한다.	void.

예제:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abc",
    .size = 3,
});
(void)e;   /* PROVEN_ERR_OUT_OF_BOUNDS if it would not fit: the buffer never grows */

proven_buf_destroy(alloc, &buf);
```

6. 예제와 오용 사례

파괴 시 allocator 일치시키기

올바른 예:

```
proven_allocator_t heap = proven_heap_allocator();
```

```
proven_result_buf_t r = proven_buf_create(heap, 128);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_destroy(heap, &r.value);   /* the same allocator that created it */
```

잘못된 예:

```
proven_result_buf_t r = proven_buf_create(heap, 128);
proven_buf_destroy(other_alloc, &r.value); /* wrong: allocator mismatch */
```

이것은 그 호출에서 아무것도 알리지 않는다. 테스트에서 잡으려면 할당자들을 `alloc_check.h` 뒤에 두라 (7절).

Arena의 free는 no-op이다

```
alignas(max_align_t) proven_byte_t storage[1024];
proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t a = proven_arena_as_allocator(&arena);

proven_result_mem_mut_t r = proven_arena_alloc(&arena, 32);
if (proven_is_ok(r.err)) {
    /* Legal, and correct to write - but it intentionally reclaims nothing. */
    a.free_fn(a.ctx, r.value.ptr);
}
proven_arena_reset(&arena);   /* only this gives the 32 bytes back */
```

Pool 크기 계약

올바른 예:

```
typedef struct Node { int value; } Node;

proven_pool_t pool = {0};
if (!proven_is_ok(proven_pool_init(&pool, alloc, sizeof(Node), alignof(Node), 8))) {
    return;
}
proven_allocator_t node_alloc = proven_pool_as_allocator(&pool);

/* The pool serves exactly the size and alignment it was configured with. */
proven_result_mem_mut_t n = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node), alignof(Node));
if (proven_is_ok(n.err)) {
    node_alloc.free_fn(node_alloc.ctx, n.value.ptr);
}

/* Anything else is refused with PROVEN_ERR_INVALID_ARG rather than served. */
proven_result_mem_mut_t wrong = node_alloc.alloc_fn(node_alloc.ctx, sizeof(Node) * 2, alignof(Node));
(void)wrong;

proven_pool_destroy(&pool);
```

잘못된 예:

```
node_alloc.alloc_fn(node_alloc.ctx, sizeof(Other), alignof(Other));
/* wrong: PROVEN_ERR_INVALID_ARG - a pool is for one size and one alignment */
```

버퍼 append는 성장하지 않는다

잘못된 예:

```
proven_buf_append(&buf, huge_data); /* returns out-of-bounds; no automatic growth */
```

성장이 필요하면 `proven_u8str_append_grow()`나 배열을 사용하라.

버퍼 append는 겹침을 허용한다

올바른 예:

```
proven_result_buf_t r = proven_buf_create(alloc, 64);
if (!proven_is_ok(r.err)) {
    return;
}
proven_buf_t buf = r.value;

proven_err_t e = proven_buf_append(&buf, (proven_mem_view_t){
    .ptr = (const proven_byte_t *)"abcdefgh",
    .size = 8,
});
if (proven_is_ok(e)) {
    /* The source view points back into the buffer's own bytes. That is allowed:
     * append moves rather than copies, so the overlap is well defined. */
    e = proven_buf_append(&buf, (proven_mem_view_t){
        .ptr = buf.ptr + 2,
        .size = 4,
    });
}
(void)e;

proven_buf_destroy(alloc, &buf);
```

잘못된 예:

```
proven_sys_mem_copy(buf.ptr + buf.len, buf.ptr + 2, 4); /* copy is not the public contract here */
```

실전 예제: 호출자가 제공한 메모리 위의 arena

테스트 스위트에서 컴파일되고 실행된다. arena를 손 뻗어 쓸 가치가 있게 만드는 bump-and-drop 수명을 보여준다: 할당은 거의 공짜이고, 개별적으로 해제되는 것은 없으며, `proven_arena_reset`이 전체 영역을 한 번에 회수한다.

```
/*
 * 아레나는 기억을 소유하지 않는다. *여러분이* 가진 기억 위로 포인터를 밀고 갈 뿐이다.
 * 거래는 그것이 전부다. 할당은 덧셈 한 번이고, 낱개 해제는 아예 없으며, reset 한 번에
 * 전부 되돌려 받는다.
 *
 * 쓸 값어치가 나는 모양은 "밀고 나서 버리기"다. 한 단계 동안 마음껏 할당하고, 단계가
 * 끝나면 reset 하나로 전부 회수한다. 물건마다 장부를 적을 일이 없으니 틀릴 일도 없고,
 * 샐 것도 없다 - 아래의 뒷받침 저장소는 자동 수명을 가진 평범한 배열이다.
 */

int main(void) {
    /* 뒷받침 저장소는 부르는 쪽의 것이다. 넉넉히 정렬해 두어, 첫 바이트부터도 어떤
     * 정렬 요구든 아레나가 맞춰 줄 수 있게 한다. */
    alignas(max_align_t) static proven_byte_t storage[4096];

    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
        .ptr = storage,
        .size = sizeof storage,
    });

    /* --- 밀기 ------------------------------------------------------------- */
    proven_result_mem_mut_t a = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(a.err), "64 bytes must fit in a 4 KiB arena");
    EXAMPLE_REQUIRE(a.value.ptr == storage, "the first allocation starts at the backing store");

    /* 타입이 PROVEN_DEFAULT_ALIGNMENT 보다 큰 정렬을 요구하면 그것을 명시한다.
     * 아레나는 거기까지 채워 넘어가므로, 건너뛴 바이트는 reset 전까지 그냥 사라진다. */
```

```
    proven_result_mem_mut_t b = proven_arena_alloc_aligned(&arena, 32, 64);
    EXAMPLE_REQUIRE(proven_is_ok(b.err), "an over-aligned block must still fit");
    EXAMPLE_REQUIRE(((uintptr_t)b.value.ptr % 64) == 0, "the block must honour the requested alignment");

    /* --- 다른 API 에 건네는 할당자로서의 아레나 --------------------------- */
    /* proven 에서 proven_allocator_t 를 받는 것이면 무엇이든 아레나로 굴릴 수 있다.
     * 그래서 아래 문자열은 `storage` 안에 산다. */
    proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);
    EXAMPLE_REQUIRE(proven_alloc_is_valid(arena_alloc), "the arena must expose a usable allocator");

    proven_result_u8str_t s = proven_u8str_create(arena_alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "the arena should be able to back a 32-byte string");

    proven_err_t err = proven_u8str_append_grow(arena_alloc, &s.value, PROVEN_LIT("scratch line"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending into an arena-backed string must succeed");

    /* 지우는 것은 여전히 옳고 소유 규칙이 요구하는 일이다 - 다만 아레나의 free 는
     * 아무 일도 하지 않으므로 되돌리는 것이 없다. 이것은 누수가 아니다. 바이트는
     * `storage` 의 것이고, 아래의 reset 이 그것을 돌려준다. */
    proven_u8str_destroy(arena_alloc, &s.value);

    proven_size_t used = arena.offset;
    EXAMPLE_REQUIRE(used > 64, "every allocation above came out of the same backing store");

    /* --- 버리기 ------------------------------------------------------------ */
    /* 한 문장이 64바이트 블록과 정렬된 블록과 문자열을 모두 되돌린다. reset 의 값은
     * 열 개를 할당했든 만 개를 할당했든 같다. */
    proven_arena_reset(&arena);
    EXAMPLE_REQUIRE(arena.offset == 0, "reset must reclaim every allocation at once");

    /* 저장소가 정말 다시 쓰인다는 증거다. 다음 할당이 처음 자리로 돌아온다. reset
     * 이전에 나눠 준 포인터는 이제 전부 매달린 포인터다 - reset 이 공짜인 값이다. */
    proven_result_mem_mut_t c = proven_arena_alloc(&arena, 64);
    EXAMPLE_REQUIRE(proven_is_ok(c.err), "allocation after reset must succeed");
    EXAMPLE_REQUIRE(c.value.ptr == storage, "after reset the arena bumps from the beginning again");

    /* --- 바닥나면 죽는 것이 아니라 오류다 ---------------------------------- */
    proven_result_mem_mut_t too_big = proven_arena_alloc(&arena, sizeof storage);
    EXAMPLE_REQUIRE(too_big.err == PROVEN_ERR_NOMEM, "an arena cannot grow: it reports NOMEM instead");

    printf("arena: %zu bytes used before reset, %zu in use now\n",
           (size_t)used, (size_t)arena.offset);

    /* 형식상의 뒷정리. 부르는 쪽이 뒷받침하는 아레나에서는 하는 일이 없지만, 나중에
     * 뒷받침 저장소가 힙이 되더라도 수명이 눈에 보이게 남는다. */
    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

실전 예제: 고정 크기 블록을 재활용하는 pool

테스트 스위트에서 컴파일되고 실행된다. 해제된 블록이 재활용 bin에서 곧장 다시 나오는 것과, pool이 무엇을 거부하는지를 보여준다.

```
/*
 * 풀(pool)은 구역이 아니라 잦은 교체를 위한 최적화다. 타입 하나를 위한 것으로,
 * 같은 크기의 블록을 몇 번이고 할당하고 해제하는 자리 - 리스트 노드, 이벤트, 파티클 -
 * 에서 매번 malloc 값을 치르지 않게 한다.
 *
 * 풀은 해제된 블록을 담아 두는 작은 스택("bin")을 갖는다. 해제하면 바탕 할당자로
 * 돌려보내는 대신 bin 에 얹고, 할당하면 거기서 하나를 꺼낸다. 둘 다 O(1) 이고 힙을
```

```
 * 건드리지 않는다. 그 재활용이 존재 이유 전부이고, 아래 검사가 그것이 실제로
 * 일어남을 보인다.
 *
 * 소유: 풀은 해제된 블록을 캐시하지만, 자기가 *내준* 블록은 추적하지 않는다. 가져간
 * 블록은 destroy 전에 모두 돌려주어야 한다 - 풀은 자기가 모르는 것을 해제할 수 없다.
 */

typedef struct {
    int id;
    int score;
} ex_node_t;

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    EXAMPLE_REQUIRE(proven_alloc_is_valid(heap), "hosted builds have a heap allocator");

    /* 풀은 bin 으로 감당 못 하는 블록을 위한 바탕 할당자를 받고, 자기가 다루는 그 한
     * 타입의 정확한 크기와 정렬을 받는다. 마지막 인자는 재사용을 위해 세워 둘 해제된
     * 블록의 최대 개수다. */
    proven_pool_t pool = {0};
    proven_err_t err = proven_pool_init(&pool, heap, sizeof(ex_node_t), alignof(ex_node_t), 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "initializing a pool of ex_node_t must succeed");
    if (!proven_is_ok(err)) {
        return 1;
    }

    proven_allocator_t nodes = proven_pool_as_allocator(&pool);

    /* --- 첫 블록: bin 이 비어 있으니 힙에서 온다 --------------------------- */
    proven_result_mem_mut_t first = nodes.alloc_fn(nodes.ctx, sizeof(ex_node_t), alignof(ex_node_t));
    EXAMPLE_REQUIRE(proven_is_ok(first.err), "the pool must be able to serve its own item type");
    if (!proven_is_ok(first.err)) {
        proven_pool_destroy(&pool);
        return 1;
    }

    ex_node_t *n = (ex_node_t *)first.value.ptr;
    *n = (ex_node_t){ .id = 1, .score = 100 };
    void *first_addr = n;

    /* --- 돌려주기: 힙이 아니라 bin 으로 들어간다 --------------------------- */
    nodes.free_fn(nodes.ctx, n);
    EXAMPLE_REQUIRE(pool.bin_len == 1, "a freed block is cached for reuse, not returned to the heap");
    /* 여기서부터 `n` 은 매달린 포인터다. 그 바이트는 다시 풀의 것이다. */

    /* --- 두 번째 블록: 방금 해제한 것이 곧바로 돌아온다 -------------------- */
    proven_result_mem_mut_t second = nodes.alloc_fn(nodes.ctx, sizeof(ex_node_t), alignof(ex_node_t));
    EXAMPLE_REQUIRE(proven_is_ok(second.err), "allocating from a non-empty bin must succeed");
    EXAMPLE_REQUIRE(second.value.ptr == first_addr, "the recycled block is the one that was freed");
    EXAMPLE_REQUIRE(pool.bin_len == 0, "taking it back out empties the bin");

    /* 재활용된 기억은 0 으로 채워지지 *않는다* - 풀이 남겨 둔 그대로다.
     * 갓 받은 malloc 에 하듯 모든 필드를 초기화할 것. */
    ex_node_t *m = (ex_node_t *)second.value.ptr;
    *m = (ex_node_t){ .id = 2, .score = 50 };
    EXAMPLE_REQUIRE(m->id == 2, "the recycled block is ours to overwrite");

    /* --- 풀 하나는 크기 하나와 정렬 하나만 감당한다 ------------------------ */
    /* 그 밖의 요청은 거부된다. 이것은 범용 할당자가 아니고, 크기가 다른 블록을 조용히
     * 내주지도 않는다. 코드는 PROVEN_ERR_UNSUPPORTED - "내 일이 아니다" - 이지 INVALID_ARG
```

```
     * 가 아니다. 후자였다면 "쓰레기를 건넸다"로 읽혀 여러분이 자기 코드에서 버그를 찾아
     * 헤매게 만들었을 것이다. */
    proven_result_mem_mut_t wrong = nodes.alloc_fn(nodes.ctx, sizeof(ex_node_t) * 2, alignof(ex_node_t));
    EXAMPLE_REQUIRE(wrong.err == PROVEN_ERR_UNSUPPORTED, "the pool only serves its configured item size");

    /* --- 지우기 전에 살아 있는 블록을 모두 돌려준다 ------------------------ */
    /* proven_pool_destroy 는 bin 에 있는 것과 bin 자체를 해제한다. `m` 은 아직 나가
     * 있으므로, 이 free 를 건너뛰면 그대로 누수가 된다. */
    nodes.free_fn(nodes.ctx, m);

    printf("pool: %zu block(s) cached for reuse at teardown\n", (size_t)pool.bin_len);

    proven_pool_destroy(&pool);
    return EXAMPLE_OK();
}
```

실전 예제: 시동 시 할당, 제자리에서 늘리기, 할당자 감싸기

앞의 두 예제는 대부분의 코드가 쓰는 방식으로 arena와 pool을 쓴다. 이 예제는 프로그램이 실제 규모가 되었을 때 손이 가는 세 가지를 다룬다.

1. **시동(start-up) 단계의 할당.** 어떤 할당은 실패했을 때 갈 길이 없다. 프로그램이 자기 설정 표조차 얻지 못했다면 남은 할 일이 없다. `proven_arena_alloc_or_panic()`과 `proven_arena_alloc_aligned_or_panic()`은 result가 아니라 메모리 블록 자체를 돌려주고, 실패는 패닉 핸들러(panic handler, 회복 불가능한 실패가 났을 때 불리는 함수)로 넘긴다 (1장 6절). 그때 무슨 일이 일어날지는 `proven_set_panic_handler()`로 고른다. 예제는 메시지를 기록하는 핸들러를 설치해 실패를 보여 준 뒤, 기본 핸들러를 되돌린다.
2. **마지막으로 받은 블록 늘리기.** `proven_arena_realloc_aligned()`는 그 블록을 있던 자리에서 늘린다. 가장 최근 블록이 곧 사용 영역의 끝에 있는 블록이기 때문이다. 그보다 앞선 블록은 끝으로 복사된다 — 여전히 옳지만, 더 이상 공짜는 아니다.
3. **할당자(allocator) 감싸기.** 할당자는 함수 포인터 셋과 문맥 포인터 하나이므로, 호출을 세거나 기록하거나 일부러 열 번째 호출을 실패시키는 래퍼(wrapper, 감싸는 코드)는 전달 함수 세 개면 된다. arena 자신의 세 함수 — `proven_arena_alloc_trait()`, `proven_arena_realloc_trait()`, `proven_arena_free_trait()` — 가 공개되어 있는 이유가 바로 이것이다. 여러분의 래퍼는 arena를 다시 구현하는 대신 이들을 부르면 된다. 그러면 `proven_allocator_t`를 받는 라이브러리의 모든 것이, 그 사실을 모른 채 여러분의 래퍼를 지나간다.

```
/*
 * 자기 기억을 소유하는 프로그램이 언젠가 반드시 하게 되는 세 가지, 그리고 그것을 하는
 * 호출들.
 *
 *   - 시작할 때 할당하기. 이때 기억이 바닥나는 것은 프로그램이 이어서 갈 수 있는
 *     상황이 아니다. `_or_panic` 계열이 그것을 위한 것이고, 패닉 처리기를 다는 것이
 *     "이어서 갈 수 없다" 가 이 프로그램에서 무슨 뜻인지 정하는 방법이다 - 덫에
 *     걸리는 대신 로그 한 줄과 종료 같은 것으로.
 *
 *   - 마지막에 할당한 블록을, 복사 없이 늘리기. 아레나는 그것을 제자리에서 할 수
 *     있다. 마지막에 할당된 블록이 곧 쓰인 구역의 끝에 앉아 있는 그 블록이기 때문이다.
 *
 *   - 할당자를 계측하기 - 할당 횟수를 세거나, 시험에서 열 번째를 일부러 실패시키거나 -
 *     재는 대상 코드는 건드리지 않고서. 여기서 할당자는 함수 포인터 셋과 문맥 포인터
 *     하나이므로, 감싸는 일은 전달 함수 셋을 쓰는 일이다. 아레나 자신의 셋이 공개되어
 *     있는 이유가 바로 이것이다 - 여러분의 껍데기가 그것으로 전달한다.
 */

/* --- 패닉 처리기는 무엇을 위한 것인가 ---------------------------------- */

static int g_panics = 0;
static char g_last_panic[128];

/* 패닉 처리기는 메시지를 받아 프로그램의 운명을 정한다. 기본 처리기는 즉시 덫에
```

```
 * 걸리는데, 그것이 운영에서는 옳고 시험에서는 쓸모없다 - 그래서 이 처리기는 메시지를
 * 적어 두고 돌아온다. 돌아오는 것은 패닉 경로를 *일부러 시험할 때만* 허용되고, 그때
 * 패닉한 호출이 돌려준 기억 블록은 써서는 안 된다. */
static void record_panic(const char *msg) {
    ++g_panics;
    snprintf(g_last_panic, sizeof g_last_panic, "%s", msg);
}

/* --- 자기를 지나가는 것을 세는 할당자 ------------------------------------ */

typedef struct {
    proven_arena_t *arena;
    proven_size_t   live_bytes;
    proven_size_t   alloc_calls;
    proven_size_t   free_calls;
} ex_counting_ctx_t;

/* 셋 각각이 proven_allocator_t 의 필드 하나에 대응한다. 각자 제 장부를 적은 뒤 아레나의
 * 공개된 특성 함수로 전달하므로, 재는 대상이 되는 동작은 아레나를 다시 구현한 것이
 * 아니라 정확히 아레나의 동작이다. */
static proven_result_mem_mut_t counting_alloc(void *ctx, proven_size_t size, proven_size_t align) {
    ex_counting_ctx_t *c = (ex_counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_alloc_trait(c->arena, size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes += size;
        ++c->alloc_calls;
    }
    return r;
}

static proven_result_mem_mut_t counting_realloc(void *ctx, void *old_ptr, proven_size_t old_size,
                                                proven_size_t new_size, proven_size_t align) {
    ex_counting_ctx_t *c = (ex_counting_ctx_t *)ctx;
    proven_result_mem_mut_t r = proven_arena_realloc_trait(c->arena, old_ptr, old_size, new_size, align);
    if (proven_is_ok(r.err)) {
        c->live_bytes = c->live_bytes - old_size + new_size;
    }
    return r;
}

static void counting_free(void *ctx, void *ptr) {
    ex_counting_ctx_t *c = (ex_counting_ctx_t *)ctx;
    ++c->free_calls;
    proven_arena_free_trait(c->arena, ptr);   /* 아레나의 free 는 아무 일도 하지 않는다. 세는 것이 요점이다 */
}

int main(void) {
    alignas(max_align_t) static proven_byte_t storage[1024];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){ .ptr = storage, .size = sizeof storage });

    /* --- 1. 실패하면 안 되는 시작 시점 할당 ------------------------------- */

    proven_set_panic_handler(record_panic);

    /* 풀어 볼 result 가 없다. 이들은 블록을 곧바로 돌려준다. 부르는 쪽이 손쓸 수 있는
     * 오류가 없기 때문이다. 차이는 그것이 전부다. */
    proven_mem_mut_t table = proven_arena_alloc_or_panic(&arena, 256);
    EXAMPLE_REQUIRE(table.ptr != NULL, "a 256-byte start-up allocation must succeed");
    EXAMPLE_REQUIRE(g_panics == 0, "a successful allocation must not panic");
```

```
    /* 기본 경계보다 큰 정렬이 필요한 타입을 위한 정렬 지정 꼴 - 여기서는 64바이트
     * 캐시 줄이고, 그것이 흔한 이유다. */
    proven_mem_mut_t cache_line = proven_arena_alloc_aligned_or_panic(&arena, 64, 64);
    EXAMPLE_REQUIRE(((proven_uintptr_t)cache_line.ptr % 64) == 0,
                    "the block must start on the boundary that was asked for");
    EXAMPLE_REQUIRE(g_panics == 0, "an over-aligned allocation that fits must not panic either");

    /* 이제 일부러 실패하는 경우다. 아레나가 결코 담을 수 없는 크기를 청한다. 기록하는
     * 처리기를 달아 두었으니 그것을 지켜볼 수 있다. 기본 처리기였다면 프로그램은 여기서
     * 멈췄을 것이고, 그것이 기본 처리기의 목적이다. */
    proven_mem_mut_t impossible = proven_arena_alloc_or_panic(&arena, sizeof storage * 2);
    (void)impossible;   /* 처리기가 돌아온 뒤, 이 블록은 아무 뜻도 없다 */
    EXAMPLE_REQUIRE(g_panics == 1, "exhausting the arena through _or_panic must panic");
    EXAMPLE_REQUIRE(g_last_panic[0] != '\0', "the handler receives a message naming the call");
    printf("panic handler saw: %s\n", g_last_panic);

    /* NULL 을 건네면 덫에 거는 기본 처리기가 되돌아온다. 시험용 처리기를 그대로 두면
     * 진짜 실패가 조용한 오염으로 바뀐다. */
    proven_set_panic_handler(NULL);

    /* --- 2. 가장 최근 블록을 제자리에서 늘리기 ---------------------------- */

    /* 머리말을 읽고 나서 본문이 짐작보다 길다는 것을 알게 된 파서는, 방금 받은 버퍼를
     * 복사하는 것이 아니라 늘리고 싶어 한다. */
    proven_result_mem_mut_t buf = proven_arena_alloc_aligned(&arena, 32, alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(buf.err), "the initial 32-byte buffer must fit");

    proven_size_t before = arena.offset;
    proven_result_mem_mut_t grown = proven_arena_realloc_aligned(&arena, buf.value.ptr, 32, 96,
alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(grown.err), "growing the most recent block must succeed");
    EXAMPLE_REQUIRE(grown.value.ptr == buf.value.ptr,
                    "the most recent block grows in place: same address, no copy");
    EXAMPLE_REQUIRE(arena.offset == before + 64, "only the extra 64 bytes were taken");

    /* 제자리 경로는 *마지막에* 할당된 블록에만 열려 있다. 다른 블록을 하나 더 받고 나면
     * 앞의 것은 있던 자리에서 더 늘릴 수 없다 - 아레나는 대신 그것을 끝으로 복사하고,
     * 옛 바이트는 다음 reset 까지 죽은 자리가 된다. 어느 쪽이든 옳다. 다만 공짜가 아닐
     * 뿐이다. */
    proven_result_mem_mut_t other = proven_arena_alloc(&arena, 16);
    EXAMPLE_REQUIRE(proven_is_ok(other.err), "a second block must fit");
    proven_result_mem_mut_t moved = proven_arena_realloc_aligned(&arena, grown.value.ptr, 96, 128,
alignof(proven_u32));
    EXAMPLE_REQUIRE(proven_is_ok(moved.err), "growing an older block must still succeed");
    EXAMPLE_REQUIRE(moved.value.ptr != grown.value.ptr, "but it is relocated, not extended");

    /* --- 3. 세는 껍데기 뒤의 아레나 ---------------------------------------- */

    proven_arena_reset(&arena);

    ex_counting_ctx_t counted = { .arena = &arena };
    proven_allocator_t alloc = {
        .ctx        = &counted,
        .alloc_fn   = counting_alloc,
        .realloc_fn = counting_realloc,
        .free_fn    = counting_free,
    };
    EXAMPLE_REQUIRE(proven_alloc_is_valid(alloc), "all three function pointers must be present");

    /* 이제 할당자를 받는 라이브러리의 어느 부분이든 그 껍데기를 지나 돈다. 그것을 알지도
```

```
     * 못한 채, 바뀐 것도 없이. */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "creating a string through the wrapper must succeed");

    proven_err_t err = proven_u8str_append_grow(alloc, &s.value, PROVEN_LIT("a line long enough to need more
room"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending past the initial capacity must succeed");

    proven_u8str_destroy(alloc, &s.value);

    EXAMPLE_REQUIRE(counted.alloc_calls >= 1, "the wrapper saw the string being created");
    EXAMPLE_REQUIRE(counted.free_calls >= 1, "and saw it being destroyed");
    printf("counting allocator: %zu alloc call(s), %zu free call(s), %zu bytes handed out\n",
           (size_t)counted.alloc_calls, (size_t)counted.free_calls, (size_t)counted.live_bytes);

    proven_arena_destroy(&arena);
    return EXAMPLE_OK();
}
```

반례 — 시험용 패닉 핸들러를 설치한 채 두는 경우:

```
proven_set_panic_handler(record_panic);   /* returns instead of stopping */
/* ... the rest of the program ... */     /* wrong: no restore */
```

돌아오는 핸들러는 이후의 모든 메모리 부족 패닉을, 널 블록을 돌려주고 계속 진행하는 호출로 바꾼다. 시험할 코드 주위에서만 설치하고, `proven_set_panic_handler(NULL)`로 기본값을 되돌린다.

반례 — 돌아온 핸들러 뒤에서 그 블록을 쓰는 경우:

```
proven_mem_mut_t m = proven_arena_alloc_or_panic(&arena, huge);
memset(m.ptr, 0, huge);   /* wrong: m.ptr is null if the handler returned */
```

반례 — 셋 중 하나를 빠뜨린 래퍼:

```
proven_allocator_t alloc = { .ctx = &counted, .alloc_fn = counting_alloc };  /* wrong */
```

`proven_alloc_is_valid()`는 이것에 false를 돌려주고, 이 할당자를 통한 첫 `realloc`이나 `destroy`는 널 포인터를 호출한다. 둘은 그저 전달만 하더라도 셋을 모두 채운다.

7. 잘못된 할당자 잡기: `alloc_check.h`

문제: 다른 곳에서 터지는 실수

이 라이브러리의 모든 소유(owned) 객체 - 문자열, 버퍼, 배열 - 는 호출자가 넘기는 할당자(allocator)로 해제되고, 메모리가 어느 할당자에서 왔는지는 아무것도 기록하지 않는다. 아레나(arena) 문자열을 힙(heap)으로 파괴하거나, 블록을 두 번 해제하거나, 할당 때와 다른 이전 크기로 블록을 늘려도 그 자리에서 실패하지 않는다. 힙은 자기가 준 적 없는 메모리를 받아 장부가 망가지고, 충돌은 나중에, 잘못한 것이 없는 코드에서 일어난다. 모든 소유자에 할당자를 담으면 대부분의 프로그램이 쓰지도 않을 검사를 위해 모든 문자열이 두 배가 되므로 라이브러리는 그렇게 하지 않는다(B-023). 검사는 반대편에 있어야 한다: 자기 블록이 무엇인지 정확히 알 수 있는 할당자 쪽에.

`alloc_check.h`가 하는 일

`proven_alloc_check_wrap`은 어떤 할당자 - 힙, 아레나, 풀(pool), 또는 당신의 것 - 앞에든 검사기를 두고, 대신 쓸 할당자를 돌려준다. 검사기는 자기가 내준 모든 블록을 당신이 준 메모리에 기록하고, 그 밖의 것을 거부한다: 기록에 없는 블록(다른 할당자의 것, 또는 이미 해제된 것)의 해제나 realloc, 그리고 할당 때와 다른 이전 크기나 정렬로 하는 realloc. 거부는 실수를 한 그 호출에서의 `proven_panic`이고, 거부된 포인터는 절대 넘겨지지 않으므로 테스트의 panic 핸들러가 돌아와도 밑의 할당자는 망가지지 않는다. 테스트 끝에서 `proven_alloc_check_live`는 누수 검사다.

테스트와 버그 추적용이지, 운영용이 아니다. 모든 해제와 realloc이 기록을 선형으로 찾는다. 할당자를 만드는 자리에 `proven_alloc_checked`를 쓰고 그대로 두라: `PROVEN_ALLOC_CHECK`가 정의되었을 때만 감싸고, 아니면 받

은 할당자를 비용 없이 그대로 돌려준다. 테스트와 디버그 실행에서는 빌드에 매크로를 정의하라(-DPROVEN_ALLOC_CHECK). 파일이 포함하는 첫 proven 헤더보다 먼저 정의되어야 한다 - 스위치는 헤더와 함께 읽히므로, #include "proven.h" 뒤의 #define은 검사를 조용히 꺼 둔 채로 둔다.

```
proven_alloc_check_t heap_chk;
proven_alloc_check_entry_t heap_rec[256];   /* up to 256 live blocks */
proven_allocator_t checked = proven_alloc_checked(&heap_chk, proven_heap_allocator(), heap_rec, 256);
/* ... the whole program or test uses `checked` ... */
(void)checked;
if (proven_alloc_check_live(&heap_chk) != 0) { /* a leak, when the checks are on */ }
```

API	의도	반환
proven_alloc_check_wrap(&st, inner, entries, cap)	inner를 감싸고 살아 있는 블록을 cap개까지 기록. 늘 검사한다.	proven_allocator_t (잘못된 인자면 invalid)
proven_alloc_checked(&st, inner, entries, cap)	PROVEN_ALLOC_CHECK가 정의되면 같은 일; 아니면 inner 그대로.	proven_allocator_t
proven_alloc_check_owns(&st, ptr)	ptr이 이 검사기의 살아 있는 블록인지.	bool
proven_alloc_check_live(&st)	살아 있는 블록 수: 테스트 끝에 0이면 누수 없음.	proven_size_t

당신이 들고 있는 구조체

```
typedef struct { void *ptr; proven_size_t size; proven_size_t align; } proven_alloc_check_entry_t;

typedef struct {
    proven_allocator_t          inner;
    proven_alloc_check_entry_t *entries;   /* your record storage */
    proven_size_t               cap, live, live_bytes, peak_live;
    proven_size_t               faults;    /* refused operations, each also a panic */
} proven_alloc_check_t;   /* caller-owned; outlives the allocator; do not copy once wrapped */
```

주의할 점, 그리고 무엇이 잘못되는가

감싼 할당자만 검사한다. 감싸지 않은 힙으로 아레나 문자열을 파괴하는 것은 여전히 검사되지 않는다: 힙에게는 묻지 않으니까. 테스트가 쓰는 모든 할당자를 감싸라.

기록이 가득 차면 통과가 아니라 panic이다. 조용히 기록을 멈춘 기록은 기록되지 않은 블록의 다음 해제를 남의 것으로 보고할 것이다. 테스트가 한꺼번에 들고 있는 최대 블록 수에 맞게 cap을 잡아라; peak_live가 그 값을 알려 준다. 반례(wrong) - 테스트에 비해 너무 작은 기록:

```
proven_alloc_check_entry_t rec[4];
proven_allocator_t a = proven_alloc_check_wrap(&st, heap, rec, 4);
/* wrong: a fifth live block panics "the record is full" - enlarge rec, do not ignore it */
```

반례 - include 뒤에서 켜는 경우. 스위치는 헤더와 함께 읽히므로, 이 파일은 검사가 꺼진 채 돌고 아무것도 그렇다고 말하지 않는다:

```
#include "proven.h"
#define PROVEN_ALLOC_CHECK        /* wrong: too late - use -DPROVEN_ALLOC_CHECK */
```

뜨거운 경로에서 켜 두는 것은 스스로 고른 속도 저하다. 검색은 살아 있는 블록 수에 선형이다.

예제: 잘못된 할당자로 파괴된 문자열

```
#include <string.h>

/*
 * 잘못된 할당자를, 실수가 일어난 자리에서 잡기.
 *
 * 이 라이브러리의 어떤 것도 블록이 어느 할당자에서 왔는지 기억하지 않는다: 호출자가
```

```
 * 만들 때 하나, 파괴할 때 하나를 넘기고, 둘이 다르면 할당자의 장부가 망가져 프로그램은
 * 나중에 엉뚱한 곳에서 깨진다. alloc_check 래퍼는 어떤 할당자 앞에든 놓여, 자기가 내준
 * 블록을 기록하고, 그 밖의 것은 - panic 으로, 그 호출에서 - 거부한다.
 *
 * 테스트와 버그 추적용이다. 당신의 코드에서는 proven_alloc_checked() 를 쓰고
 * -DPROVEN_ALLOC_CHECK 로 켜고 끈다: 매크로가 없으면 받은 할당자를 그대로 돌려주어 비용이
 * 없다. (첫 proven #include 뒤의 #define 은 늦다 - 스위치는 헤더와 함께 읽힌다.) 이 예제는
 * 무조건 검사하는 proven_alloc_check_wrap() 을 불러, 빌드 플래그와 상관없이 검사를 보여 준다.
 */

/* 돌아오는 panic 핸들러 - 이 예제가 거부를 보여 주고 계속 가도록. 실제 프로그램은 실수에서
 * 멈추는 기본 핸들러를 그대로 둔다. */
static int g_refused;
static void note_panic(const char *msg) {
    ++g_refused;
    printf("refused: %s\n", msg);
}

int main(void) {
    proven_set_panic_handler(note_panic);

    /* 할당자 둘, 각자 블록 16개를 기록할 자리가 있는 검사기 뒤에. */
    static proven_byte_t arena_mem[2048];
    proven_arena_t arena = proven_arena_create((proven_mem_mut_t){ arena_mem, sizeof arena_mem });
    proven_alloc_check_t arena_chk, heap_chk;
    proven_alloc_check_entry_t arena_rec[16], heap_rec[16];
    proven_allocator_t scratch = proven_alloc_check_wrap(&arena_chk, proven_arena_as_allocator(&arena),
arena_rec, 16);
    proven_allocator_t heap = proven_alloc_check_wrap(&heap_chk, proven_heap_allocator(), heap_rec, 16);

    /* --- 올바른 사용은 그대로 통과한다 ---------------------------------------- */

    proven_result_u8str_t name = proven_u8str_create_from_view(heap, PROVEN_LIT("report"));
    EXAMPLE_REQUIRE(proven_is_ok(name.err), "a string from the checked heap");
    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_append_grow(heap, &name.value, PROVEN_LIT(".txt"))), "grown by
the same allocator");
    EXAMPLE_REQUIRE(proven_alloc_check_owns(&heap_chk, name.value.internal.ptr), "the heap checker owns its
block");

    /* --- 실수를 그 호출에서 잡는다 -------------------------------------------- */

    /* 임시 작업용 아레나에서 만든 임시 문자열을... */
    proven_result_u8str_t tmp = proven_u8str_create(scratch, 64);
    EXAMPLE_REQUIRE(proven_is_ok(tmp.err), "a temporary in the arena");
    proven_u8str_t t = tmp.value;
    /* ...힙으로 파괴한다. 검사가 없으면 힙이 아레나 메모리를 해제할 것이다. */
    proven_u8str_destroy(heap, &t);
    EXAMPLE_REQUIRE(g_refused == 1 && heap_chk.faults == 1, "the heap refuses a block it never gave out");
    EXAMPLE_REQUIRE(proven_alloc_check_live(&arena_chk) == 1, "and the arena block is untouched");
    t = tmp.value;
    proven_u8str_destroy(scratch, &t);   /* 올바른 쪽 */

    /* --- 테스트 끝의 누수 검사 ------------------------------------------------ */

    proven_u8str_destroy(heap, &name.value);
    EXAMPLE_REQUIRE(proven_alloc_check_live(&heap_chk) == 0 && proven_alloc_check_live(&arena_chk) == 0,
                    "nothing is left live");
    printf("peak %zu live block(s) on the heap, %zu refusal(s)\n", (size_t)heap_chk.peak_live,
(size_t)heap_chk.faults);
```

```
    proven_set_panic_handler(NULL);
    return EXAMPLE_OK();
}
```

3장: 문자열, 포매팅, 스캐닝

Part II — 모든 프로그램이 쓰는 어휘. 선행 장: 1장과 2장. 이 장을 마치면 NUL 종료자가 프로그램의 운명을 결정하지 않는 방식으로 텍스트를 담을 수 있고, 오버플로를 거부하는 문자열을 만들 수 있으며, 일상적인 경우의 포매팅과 파싱을 할 수 있다.

이 장은 `u8str.h`, `u16str.h`, `utf.h`, `fmt.h`, `scan.h`를 다룬다. 텍스트 자료의 **튜토리얼 절반**으로서, 매일 만나는 사례로 포매터와 스캐너를 소개한다. 8장이 레퍼런스 절반이다 — 전체 문법, 모든 인자 생성자, 스캐너의 에러와 복구 규칙을 담는다. 이 장을 먼저 읽으라.

목차

1. U8 문자열과 view
2. U16 문자열과 view
3. 포매팅
4. 스캐닝
5. 예제와 오용 사례

1. U8 문자열과 view

문제: C 문자열은 자기 길이를 모른다

C 문자열은 포인터이고, 어디서 끝나는지는 메모리 어딘가의 0 바이트가 결정한다. 문자열당 1바이트를 아끼려고 1972년에 내린 그 결정 하나가 놀랄 만큼 많은 피해의 배후에 있다:

- **길이 구하기가 검색이다.** `strlen`은 문자열을 훑는다. 반복마다 `strlen(s)`를 검사하는 루프는 제곱 시간이고, 겉보기에는 평범한 코드처럼 보인다.
- **텍스트가 0 바이트를 담을 수 없다.** 그래서 C 문자열은 UTF-16 버퍼도, 프로토콜 프레임도, 파일의 일부도, 어떤 이진 데이터도 담을 수 없다 — 문자열 타입과 바이트 타입은 다른 것인데도 언어는 아닌 척한다.
- **종료자가 없는 것은 탐지할 수 없다.** 자리가 없는 버퍼로 `strcpy`하면 스택 프레임 어딘가에서 0을 찾을 때까지 써 나간다. 아무것도 그것을 보고하지 않는다. 이는 이 언어 역사상 가장 많이 악용된 버그 부류다.
- **문자열의 일부를 저렴하게 가리킬 수 없다.** "이 줄의 세 번째 필드"라는 말은 그 바이트를 복사해 내거나, 원본 위에 0을 써서 원본을 망가뜨리거나 둘 중 하나를 뜻한다. `strtok`은 두 번째를 골랐고, 그래서 입력을 변형시키며 중첩할 수 없다.

전통적인 땜질인 `strncpy`는 항상 NUL로 종료해 주지는 않는다 — 그러니 그 "안전한" 함수가 문자열이 아닌 문자열을 만들어 낼 수 있다.

이 라이브러리가 대신 하는 것

두 개의 타입이 있고, 그 둘의 차이는 소유권이다:

- `proven_u8str_view_t` — **borrowed.** 포인터와 크기가 함께, 다른 누군가가 소유한 바이트를 가리킨다. 복사는 공짜다. 아무것도 할당하지 않고 아무것도 파괴하지 않으며, 소유자가 사라지면 함께 유효하지 않게 된다. 함수에 전달하는 것이 이것이다.
- `proven_u8str_t` — **owned.** 자기 저장소와 용량을 가지고, NUL 종료자를 대신 유지해 주므로 `proven_u8str_as_cstr`이 여전히 NUL을 원하는 libc 함수에 바이트를 건네줄 수 있다. 할당자(allocator)로 만들었으니, 같은 것으로 파괴한다.

소유 문자열은 그 할당자를 기억하지 않으며, 아무것도 그것을 확인하지 않는다. 할당하거나 해제할 수 있는 모든 호출 - `_create`, `_reserve`, `_grow` 계열, `_destroy` - 은 할당자를 인자로 받고, 그것은 문자열을 만든 할당자여야 한다. 다른 것을 넘기면 돌려줄 오류가 없다: 그 할당자는 자기가 준 적 없는 메모리를 받아 장부가 망가지고, 충돌은 나중에 엉뚱한 곳에서 일어난다. 모든 문자열에 할당자를 담으면 크기가 두 배가 되고 빌려 쓰는(borrowed) 문자열에는 의미도 없으므로 라이브러리는 그렇게 하지 않는다. 짝은 당신의 코드에서 지켜라 - 소유자마다 할당자 하나, 같은 자리에서 넘긴다.

반례 - 아레나(arena)에서 만든 문자열을 힙(heap)으로 파괴하는 경우:

```
proven_u8str_t s = proven_u8str_create(arena, 64).value;
proven_u8str_destroy(proven_heap_allocator(), &s);   /* wrong: the heap frees arena memory */
```

둘 다 문자가 아니라 **바이트**를 센다. "한"은 UTF-8에서 3바이트이고 1문자인데, 이 라이브러리는 3이라고 말할 것이다. 그것이 라이브러리가 아는 것이기 때문이다. 여기서 텍스트는 바이트의 나열이며, 그 바이트를 문자로 해석하는 일은 이 라이브러리에 없는 Unicode 계층의 몫이다.

뷰(view)는 자기 길이를 지니고 다니므로, NUL 종료자가 어렵게 만들었던 모든 것이 평범해진다: 길이는 필드이고, 텍스트가 0 바이트를 담을 수 있으며, 부분 범위는 복사 없이 같은 메모리를 가리키는 view이고, 들어가지 않을 쓰기는 수행되는 대신 거부된다.

잘못된 예 — view를 C 문자열처럼 다루기:

```
proven_u8str_view_t v = proven_u8str_view_slice(line, 4, 8);   /* a field inside a line */
printf("%s\n", (const char *)v.ptr);   /* wrong: no terminator, prints until it finds a zero */
```

view는 의도적으로 NUL 종료되지 않는다: 보통 다른 누군가의 버퍼 한가운데를 가리키며, 거기에 종료자를 쓰면 다음 필드를 손상시킬 것이다. 크기를 받는 `proven_println("{}", PROVEN_ARG(v))`를 쓰거나, 먼저 소유(owned) 문자열로 복사하라.

구조체

```
typedef struct {
    const proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_view_t;

typedef struct {
    proven_byte_t *ptr;
    proven_size_t size;
} proven_u8str_mut_t;

typedef struct {
    proven_buf_t internal;
    bool         borrowed;
} proven_u8str_t;

typedef struct {
    proven_err_t err;
    proven_u8str_t value;
} proven_result_u8str_t;

typedef struct {
    proven_err_t err;
    const char *value;
} proven_result_cstr_t;
```

의도:

- `proven_u8str_view_t`: 빌려 쓰는(borrowed) 읽기 전용 바이트 문자열 view.
- `proven_u8str_mut_t`: borrowed 가변 바이트 문자열 view.
- `proven_u8str_t`: NUL로 종료되는 owned 문자열.
- `proven_result_u8str_t`: owned 문자열용 result 래퍼.
- `proven_result_cstr_t`: 할당된 NUL 종료 C 문자열용 result 래퍼.

내부 레이아웃

view는 보이는 그대로다 — 포인터 하나와 바이트 개수 하나:

```
typedef struct { const proven_byte_t *ptr; proven_size_t size; } proven_u8str_view_t;
typedef struct { proven_byte_t *ptr;       proven_size_t size; } proven_u8str_mut_t;
```

owned 문자열은 `proven_buf_t`(2장의 고정 용량 바이트 버퍼)에 플래그 하나를 더해 감싼다:

```
typedef struct {
    proven_buf_t internal;   /* the bytes + length + capacity; always keeps room for a NUL */
```

```
    bool         borrowed;   /* false = allocator-owned (default); true = wraps caller memory */
} proven_u8str_t;
```

- `internal.len`이 바이트 길이다(`proven_buf_t`는 `ptr` / `len` / `cap`이며 — `size` 멤버는 없다). 문자열이 유효한 동안 `internal.len` 위치의 바이트는 항상 NUL 종료자이므로, `proven_u8str_as_cstr()`은 O(1)이다.
- `borrowed`는 0으로 초기화된 핸들에서 `false`이므로, allocator가 소유하는 문자열이 안전한 기본값이다. 이 값은 여러분이 소유한 `[buf, buf+cap)`을 감싸는 `proven_u8str_borrow`에 의해서만 `true`가 된다: 그때는 성장 연산이 재할당을 거부하고(`PROVEN_ERR_OUT_OF_BOUNDS`), `proven_u8str_destroy`는 no-op이 된다.
- 문자열을 바꾸려고 이 필드들을 직접 읽거나 쓰지 **말라**. 아래의 함수를 사용하라. 길이를 얻으려고 `internal.len`을 읽는 것은 괜찮지만, `proven_u8str_as_view()`를 선호하라.

반례 — borrowed 문자열을 owned 문자열처럼 다루기:

```
proven_byte_t stack[8];
proven_u8str_t s = proven_u8str_borrow(stack, sizeof stack);   /* borrowed */

/* This would have to reallocate caller memory, so it refuses: the call returns
   PROVEN_ERR_OUT_OF_BOUNDS and `stack` is left exactly as it was. A borrowed
   string never silently escapes to the heap. */
proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT("far too long for eight bytes"));
(void)e;   /* == PROVEN_ERR_OUT_OF_BOUNDS */

/* destroy is a no-op here: `stack` is yours, and the library will not free it.
   Writing it anyway is correct, and it keeps teardown code uniform. */
proven_u8str_destroy(alloc, &s);
```

매크로

매크로	의도	비고
`PROVEN_LIT(s)`	문자열 리터럴로부터 `proven_u8str_view_t`를 만든다.	문자열 리터럴에만 사용하라.
`PROVEN_LIT_INIT(s)`	리터럴 view의 초기화자 형태.	집합체 초기화에 사용하라.
`PROVEN_INDEX_NOT_FOUND`	find 함수가 반환하는 센티널.	`(proven_size_t)-1`과 같다.

U8 함수

API	의도	반환
`proven_u8str_create(alloc, limit)`	`limit` 바이트에 NUL을 더한 용량을 가진 빈 owned 문자열을 만든다.	`proven_result_u8str_t`.
`proven_u8str_create_from_view(alloc, view)`	view를 새 owned 문자열로 복사한다.	`proven_result_u8str_t`.
`proven_u8str_borrow(buf, cap)`	호출자가 소유한 `[buf, buf+cap)`을 고정 용량 문자열로 감싼다(할당 없음). 고정 용량 연산과 `append_fmt`는 동작하고, 성장 연산은 호출자 메모리 재할당을 거부하며, destroy는 no-op이다. `cap`은 NUL을 포함한다.	`proven_u8str_t`.
`proven_u8str_reset(str)`	버퍼/용량은 재사용을 위해 유지한 채 빈 상태로 자른다(owned든 borrowed든).	`proven_err_t`.
`proven_u8str_is_valid(str)`	공개 문자열 불변식을 검증한다.	`bool`.

proven_u8str_reserve(alloc, str, new_cap)	내부 용량이 최소 new_cap 바이트가 되도록 보장한다. (borrowed 문자열은 성장할 수 없다.)	proven_err_t.
proven_u8str_append(str, data)	원자적 고정 용량 append.	PROVEN_OK 또는 PROVEN_ERR_OUT_OF_BOUNDS.
proven_u8str_append_partial(str, data)	잘라내는 append.	proven_result_size_t; value는 쓰인 바이트 수.
proven_u8str_append_grow(alloc, str, data)	원자적 성장 가능 append.	proven_err_t.
proven_u8str_append_byte(alloc, str, b)	바이트 하나를 append하며, 필요하면 성장한다.	proven_err_t.
proven_u8str_replace_at(str, index, old_len, data)	정확한 바이트 범위를 고정 용량 의미로 치환한다.	proven_err_t.
proven_u8str_replace_at_grow(alloc, str, index, old_len, data)	replace_at과 같지만, 편집이 들어가지 않을 때 실패하는 대신 버퍼를 성장시킨다(2배씩).	proven_err_t; 할당 실패 시 문자열은 변경되지 않는다.
proven_u8str_insert(str, index, data)	고정 용량 의미로 index에 바이트를 삽입한다.	proven_err_t.
proven_u8str_insert_grow(alloc, str, index, data)	insert와 같지만 필요할 때 버퍼를 성장시킨다. 수동 reserve가 필요 없다.	proven_err_t; 할당 실패 시 문자열은 변경되지 않는다.
proven_u8str_remove(str, index, len)	바이트 범위를 제거한다.	proven_err_t.
proven_u8str_replace_first(str, start_offset, target, replacement)	start 이후 처음 일치하는 target을 치환한다.	찾지 못해도 PROVEN_OK.
proven_u8str_view_find(haystack, start_offset, needle)	바이트 부분 문자열의 첫 등장을 찾는다(어떤 바이트 값이든 올바르게 처리하며, NUL 종료를 요구하지 않는다).	인덱스 또는 PROVEN_INDEX_NOT_FOUND.
proven_u8str_view_starts_with(str, prefix)	접두사 검사.	int 진리값.
proven_u8str_view_starts_with_ascii_ci(str, prefix)	ASCII 글자의 대소문자를 무시한 접두사 검사(로캘도 유니코드 대소문자 접기도 없음).	int 진리값.
proven_u8str_view_ends_with(str, suffix)	접미사 검사.	int 진리값.
proven_u8str_view_slice(str, index, len)	잘라 맞춘(clamped) 부분 view를 반환한다.	proven_u8str_view_t.
proven_u8str_as_cstr(str)	내부의 NUL 종료 포인터를 반환한다.	const char *; 성장/파괴에 의해 무효화된다.
proven_mem_view_from_u8(view)	U8 view를 바이트 view로 변환한다.	proven_mem_view_t.
proven_u8str_view_to_cstr(view, alloc)	임의의 view로부터 NUL 종료 C 문자열을 할당한다.	proven_result_cstr_t; 호출자가 allocator로 해제한다.
proven_cstr_len(s)	NUL까지의 바이트를 센다.	proven_size_t.

`proven_u8str_view_from_cstr(s)`	신뢰할 수 있는 NUL 종료 문자열로부터 view를 만든다.	널 포인터에는 빈 view.
`proven_u8str_view_from_bytes(ptr, size)`	어떤 객체 포인터에서든 `size` 바이트의 view를 캐스트 없이 만든다.	`proven_u8str_view_t`; 복사하지 않음.
`proven_u8str_view_eq(a, b)`	바이트 동등성 검사.	int 진리값.
`proven_u8str_view_eq_ascii_ci(a, b)`	ASCII 글자의 대소문자를 무시한 동등성: HTTP 필드 이름, URL 스킴, MIME 타입.	int 진리값.
`proven_u8str_destroy(alloc, str)`	일치하는 allocator로 owned 저장소를 해제한다.	void.
`proven_u8str_as_view(str)`	현재 내용을 borrow한다.	`proven_u8str_view_t`.

기본 U8 예제

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("log"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

proven_err_t e = proven_u8str_append_grow(alloc, &s, PROVEN_LIT(": ready"));
if (!proven_is_ok(e)) {
    proven_u8str_destroy(alloc, &s);
    return;
}

/* Valid until the next growing call: as_cstr points into the string's storage. */
const char *cstr = proven_u8str_as_cstr(&s);
(void)cstr;

proven_u8str_destroy(alloc, &s);
```

뷰 나누기, 다듬기, 끝에서 찾기, 정렬하기

뷰(view)는 그 안에서 찾고, 슬라이스(slice)로 잘라 내고, 같은지 비교할 수 있었고, 그게 전부였다. 프로그램이 텍스트 한 줄로 하는 나머지 일 - 필드로 자르고, 필드를 다듬고, 마지막 점을 찾고, 결과를 정렬하는 일 - 은 모든 호출자가 손으로 썼고, 손으로 쓴 판본들은 같은 곳에서 틀렸다. 나누기가 가장 분명한 경우다. 유능한 사람이 처음 쓰는 루프는 *"구분자가 발견되는 동안 계속"*이다:

```
proven_size_t off = 0, at;
while ((at = proven_u8str_view_find(s, off, sep)) != PROVEN_INDEX_NOT_FOUND) {
    emit(proven_u8str_view_slice(s, off, at - off));   /* wrong: the tail is never emitted */
    off = at + sep.size;
}
```

이 루프는 마지막 구분자 뒤의 텍스트를 잃고, 그것도 흔한 경우에서 틀린다: `"a,b,c"`는 필드 두 개, `"a"`는 없음, `""`도 없음. 바로잡으려면 마지막 필드를 루프 밖으로 끌어내야 하는데, 마감에 쫓길 때 빠뜨리는 게 바로 그 단계다. 호출자들이 실제로 택한 대안 - 필드마다 소유(owned) 문자열 하나 - 는 3.4배 느리고 필드마다 할당 한 번이었다(RFC-0004 2.2절).

그래서 뷰 어휘에 나머지 낱말이 생겼다. 모두 뷰 위의 순수 함수이고 할당하지 않는다. 두 규칙이 전부에 적용된다. **잘못 만들어진 뷰** - `ptr == NULL`이면서 `size > 0` - **는 빈 뷰로 취급한다**, 함수마다 있는 가드로. 그리고 **빈 결과는 언제나 `{NULL, 0}`**이다. `proven_u8str_view_slice`가 이미 쓰던 표기라서, 빈 결과는 위치를 담지 않는다: 뷰는 포인터가 아니라 크기로 검사하라.

나누기는 영구적인 계약을 지킨다: 구분자 n개는 필드 n + 1개가 된다. 왼쪽부터 겹치지 않게 센다. `"a,"`는 `"a"`와 빈 필드, `",a"`는 빈 필드와 `"a"`, `"a,,b"`는 가운데 빈 필드를 지키며(`strtok`와 다르다), `""`는 필드 0개가 아

니라 빈 필드 하나다. 빈 구분자는 입력 전체를 한 번 내준다 - 빈 필드를 끝없이 내지 않는다. 뻔한 구현은 그렇게 되는데, 빈 needle 검색이 시작한 자리에서 일치하기 때문이다. 반복자는 자기 자신이 아니라 원본을 가리키므로 복사할 수 있고, 복사본은 따로 계속된다.

다듬기는 정확히 여섯 바이트만 안다: 공백, `\t`, `\n`, `\v`, `\f`, `\r`. 로케일도 유니코드도 아니다 - 줄 바꿈 없는 공백(no-break space)은 여기서 공백이 아니다. 접두사·접미사 제거는 그것이 없으면 뷰를 그대로 둔다. 오류가 아니며, 적용됐는지 알아야 하면 `proven_u8str_view_starts_with`로 먼저 물어라.

`proven_u8str_view_find_last`는 위치를 돌려준다: 마지막 출현의 시작이고, 출현은 겹칠 수 있다. `find_last("aaa", "aa")`는 1이다. 빈 needle이면 위치는 `size`다 - `_find`와 같은 답 - 그리고 이것이 올바른 바이트 인덱스가 아닌 유일한 결과다. 앞방향 검색과 같은 표본으로 같은 선택을 한다: 보통 텍스트에서는 needle의 가장 드문 바이트를 끝에서부터 찾아 그 주변을 검증하고, 엔트로피가 낮은 haystack - 긴 반복, 작은 알파벳 - 에서는 입력이 무엇이든 선형인 알고리즘으로 바꾼다 (64바이트까지는 역방향 Shift-Or, 그 너머는 역방향 Two-Way).

`proven_u8str_view_cmp`는 바이트를 부호 없이 비교하고 접두사를 먼저 둔다. 그래서 `"\xFF"`는 `"a"` 뒤에, `"app"`은 `"apple"` 앞에 온다. 뷰 안의 NUL은 데이터다. 답은 부호로 한다 - 반드시 -1, 0, 1은 아니다. `proven_u8str_view_cmp_ptr`는 같은 비교를 `proven_array_sort`에 맞춘 모양으로, 원소를 가리키는 포인터를 받는다.

API	의도	반환
`proven_u8str_view_split(src, sep)`	나누기 시작; 필드는 `src` 안을 가리키는 뷰.	`proven_u8str_view_split_t`
`proven_u8str_view_split_next(&it, &field)`	다음 필드, 더 없으면 `false`.	`bool`
`proven_u8str_view_trim(s)`	양 끝에서 공백 여섯 바이트를 뗀다.	`proven_u8str_view_t`
`proven_u8str_view_trim_start(s)`, `proven_u8str_view_trim_end(s)`	한쪽 끝만.	`proven_u8str_view_t`
`proven_u8str_view_remove_prefix(s, prefix)`	`prefix`를 뗀 `s`, 아니면 그대로의 `s`.	`proven_u8str_view_t`
`proven_u8str_view_remove_suffix(s, suffix)`	`suffix`를 뗀 `s`, 아니면 그대로의 `s`.	`proven_u8str_view_t`
`proven_u8str_view_find_last(haystack, needle)`	마지막 출현의 시작, 또는 `PROVEN_INDEX_NOT_FOUND`.	`proven_size_t`
`proven_u8str_view_contains(haystack, needle)`	0부터의 `find`가 `NOT_FOUND`가 아님.	`bool`
`proven_u8str_view_cmp(a, b)`	바이트 단위 부호 없는 순서, 접두사 먼저; 부호만.	`int`
`proven_u8str_view_cmp_ptr(pa, pb)`	`proven_array_sort`용 `cmp`: 뷰를 가리키는 포인터.	`int`
`proven_u8str_view_is_well_formed(s)`	`{NULL, n > 0}`에서만 거짓.	`bool`

```
typedef struct {
    proven_u8str_view_t rest;  /* not yet yielded; points into the caller's source bytes */
    proven_u8str_view_t sep;
    bool                done;  /* set once the final field has been yielded */
} proven_u8str_view_split_t;   /* copyable: a copy continues independently */
```

반례 - 필드나 `_is_well_formed`로 루프를 끝내는 경우. 빈 필드는 `{NULL, 0}`이고, 텍스트 끝을 넘은 슬라이스도 그렇다. 둘 다 well formed다. `_is_well_formed`는 뷰를 읽어도 안전하다는 것만 말한다. 나누기는 `split_next`의 반환값으로, 검색은 `PROVEN_INDEX_NOT_FOUND`로 끝내라:

```
while (proven_u8str_view_split_next(&it, &f) && f.ptr != NULL) { ... }  /* wrong: stops at the first empty
field */
while (proven_u8str_view_is_well_formed(rest)) { ... }                /* wrong: never false for a slice */
```

반례 - needle을 확인하지 않고 `find_last`의 답 자리의 바이트를 읽는 경우. 빈 needle이면 답은 `size`다:

```
proven_size_t at = proven_u8str_view_find_last(s, needle);
char c = (char)s.ptr[at];   /* wrong when needle is empty: at == s.size, one past the end */
```

예제는 끝과 가운데에 구분자가 겹친 레코드를 나누고, 필드마다 다듬고, 마지막 점을 찾고, 이중 접미사를 떼고, 이름들을 정렬한다:

```
#include <string.h>

/*
 * 매일 하는 텍스트 일을 뷰 위에서: 한 줄을 필드로 나누고, 다듬고, 알려진 접두사나
 * 접미사를 떼고, 마지막 점을 찾고, 얻은 것을 정렬한다. 정렬에 쓰는 배열 말고는 아무것도
 * 할당하지 않는다. 모든 결과는 처음 받은 텍스트 안을 가리킨다.
 *
 * 두 규칙이 내내 지켜진다. 빈 결과는 {NULL, 0} 이므로, 뷰는 포인터가 아니라 크기로
 * 검사한다. 그리고 분할 루프는 split_next 의 반환값으로 끝낸다 - 구분자 n 개는 빈 필드를
 * 포함해 언제나 n + 1 개의 필드가 된다.
 */

static bool is(proven_u8str_view_t v, const char *want) {
    return v.size == strlen(want) && (v.size == 0 || memcmp(v.ptr, want, v.size) == 0);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 나누고, 필드마다 다듬기 ------------------------------------------- */

    /* 끝에 구분자가 있고 빈 필드가 있는 레코드: 구분자 다섯 개, 그래서 필드 여섯 개 -
     * 마지막은 빈 필드다. 사람이 처음 쓰는 루프("구분자가 발견되는 동안")는 다섯 개를
     * 내고 꼬리를 잃는다. */
    proven_u8str_view_t record = PROVEN_LIT(" report.tar.gz , draft.txt,,notes.md ,\tREADME ,");
    proven_u8str_view_t fields[8];
    int n = 0;
    proven_u8str_view_split_t it = proven_u8str_view_split(record, PROVEN_LIT(","));
    proven_u8str_view_t f;
    while (n < 8 && proven_u8str_view_split_next(&it, &f)) {
        fields[n++] = proven_u8str_view_trim(f);   /* ' ', \t, \n, \v, \f, \r - 그 밖에는 없다 */
    }
    EXAMPLE_REQUIRE(n == 6, "five separators give six fields");
    EXAMPLE_REQUIRE(is(fields[0], "report.tar.gz") && is(fields[3], "notes.md") && is(fields[4], "README"),
                    "each field is trimmed on both ends");
    EXAMPLE_REQUIRE(fields[2].size == 0 && fields[5].size == 0, "the empty fields are kept, and tested by
size");

    /* 한쪽 끝이 의미 있는 텍스트를 위한 한쪽 다듬기. */
    EXAMPLE_REQUIRE(is(proven_u8str_view_trim_start(PROVEN_LIT("  indented  ")), "indented  "), "trim_start");
    EXAMPLE_REQUIRE(is(proven_u8str_view_trim_end(PROVEN_LIT("  indented  ")), "  indented"), "trim_end");

    /* --- 접두사, 접미사, 그리고 마지막 점 ---------------------------------- */

    proven_u8str_view_t name = fields[0];
    /* 확장자는 마지막 점 뒤에 있다: find 가 아니라 find_last. */
    proven_size_t dot = proven_u8str_view_find_last(name, PROVEN_LIT("."));
    EXAMPLE_REQUIRE(dot == 10, "the last dot in report.tar.gz is at 10");
    EXAMPLE_REQUIRE(is(proven_u8str_view_slice(name, dot + 1, name.size), "gz"), "so the extension is gz");
```

```
    /* remove_suffix 는 접미사가 없으면 뷰를 그대로 둔다 - 오류가 아니므로, 알아야 할
     * 때는 starts_with/ends_with 로 먼저 물어라. */
    EXAMPLE_REQUIRE(is(proven_u8str_view_remove_suffix(name, PROVEN_LIT(".tar.gz")), "report"), "the double
suffix goes");
    EXAMPLE_REQUIRE(is(proven_u8str_view_remove_suffix(name, PROVEN_LIT(".zip")), "report.tar.gz"),
                    "an absent suffix changes nothing");
    EXAMPLE_REQUIRE(is(proven_u8str_view_remove_prefix(PROVEN_LIT("# heading"), PROVEN_LIT("# ")), "heading"),
                    "a known prefix goes");
    EXAMPLE_REQUIRE(proven_u8str_view_contains(fields[1], PROVEN_LIT("draft")), "contains is find !=
NOT_FOUND");

    /* --- 비어 있지 않은 필드 정렬 ---------------------------------------- */

    proven_result_array_t ra = proven_array_create(alloc, 8, sizeof(proven_u8str_view_t),
alignof(proven_u8str_view_t));
    EXAMPLE_REQUIRE(proven_is_ok(ra.err), "creating the array must succeed");
    if (!proven_is_ok(ra.err)) return EXAMPLE_OK();
    proven_array_t names = ra.value;
    for (int i = 0; i < n; ++i) {
        if (fields[i].size > 0) (void)proven_array_push(&names, &fields[i]);
    }
    /* cmp_ptr 는 정렬용 모양의 cmp 다: 뷰를 가리키는 포인터를 받는다. 바이트는 부호 없이
     * 비교되고, 접두사가 먼저 온다. ASCII 에서 'R' 이 'd' 보다 작으므로 "README" 가
     * 소문자 이름들보다 앞선다. */
    proven_array_sort(&names, proven_u8str_view_cmp_ptr);
    const proven_u8str_view_t *first = proven_array_get(&names, 0);
    const proven_u8str_view_t *last = proven_array_get(&names, 3);
    EXAMPLE_REQUIRE(first && is(*first, "README") && last && is(*last, "report.tar.gz"), "sorted bytewise");
    EXAMPLE_REQUIRE(proven_u8str_view_cmp(PROVEN_LIT("app"), PROVEN_LIT("apple")) < 0,
                    "cmp answers by sign - never compare it with -1");

    /* --- well-formed 는 "찾았다" 도 "비어 있지 않다" 도 아니다 ---------------- */

    /* 끝을 넘은 슬라이스는 {NULL, 0} 이고, 이는 well formed 다 - 그래서 이 술어로는
     * 루프를 끝낼 수도, "비었다" 와 "끝을 넘었다" 를 가를 수도 없다. 뷰를 읽어도
     * 안전하다는 것만 말한다. */
    proven_u8str_view_t past = proven_u8str_view_slice(name, 100, 5);
    EXAMPLE_REQUIRE(proven_u8str_view_is_well_formed(past) && past.size == 0, "past the end: empty and well
formed");
    EXAMPLE_REQUIRE(!proven_u8str_view_is_well_formed((proven_u8str_view_t){ NULL, 3 }), "only {NULL, n > 0}
is not");

    /* --- 대소문자, ASCII 방식 ------------------------------------------------ */

    /* HTTP 필드 이름, URL 스킴, MIME 타입은 ASCII 글자의 대소문자만 무시한다:
     * 로캘도, 유니코드 대소문자 접기도 없다. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq_ascii_ci(PROVEN_LIT("Content-Type"), PROVEN_LIT("content-TYPE")),
"field names ignore ASCII case");
    EXAMPLE_REQUIRE(!proven_u8str_view_eq_ascii_ci(PROVEN_LIT("@"), PROVEN_LIT("`")), "a byte next to the
letters is only itself");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with_ascii_ci(PROVEN_LIT("HTTPS://example.org"),
PROVEN_LIT("https:")), "and so does a prefix test");

    /* 어떤 버퍼든 부르는 자리에서 캐스트 없이 view로. */
    const unsigned char raw[] = { 'G', 'E', 'T' };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_view_from_bytes(raw, sizeof raw), PROVEN_LIT("GET")), "a
view of raw bytes");

    printf("%d fields, %zu names sorted\n", n, (size_t)names.len);
```

```
    proven_array_destroy(&names);
    return EXAMPLE_OK();
}
```

2. U16 문자열과 view

두 번째 문자열 타입이 아예 존재하는 이유

UTF-8이 옳은 기본값이고 이 라이브러리는 거기에 전념한다. `proven_u16str_t`는 단 하나의 이유로 존재한다: **Windows API가 UTF-16이기 때문이다.** 모든 "와이드" 진입점 — `CreateFileW`, `GetEnvironmentVariableW`, `W` 계열 전체 — 은 `wchar_t *`를 받고, Windows에서 그것은 16비트다. 그 API들과 대화하는 라이브러리에는 그들의 코드 유닛을 바이트인 척하지 않고 담을 타입이 필요하다.

그러니 이것은 경계 타입이다. UTF-16 API에 닿는 곳에서 이것을 쓰고, 그 밖의 모든 곳에서는 `proven_u8str_t`를 쓴다. 이 타입은 의도적으로 작다 — create, destroy, append, 길이 — 프로그램이 이 타입으로 사고하도록 만든 것이 아니기 때문이다.

붙들어 둘 것이 둘 있다. 모든 UTF-16 버그의 근원이기 때문이다:

- **코드 유닛은 문자가 아니다.** UTF-16은 Basic Multilingual Plane 밖의 것을 *서로게이트 쌍*으로 인코딩한다 — 한 문자를 뜻하는 두 개의 `proven_u16` 값이다. 이모지는 코드 유닛 두 개다. 그 사이를 잘라내면 짝 없는 서로게이트가 나오고, 그것은 유효한 UTF-16이 아니다.
- **여기서 `size`는 바이트가 아니라 코드 유닛을 센다.** `proven_u16str_view_t.size`는 `proven_u16` 값의 개수다. 밑에 깔린 `proven_buf_t`는 바이트를 추적하며, 그래서 `proven_u16str_len`이 나눗셈을 한다. 이 두 단위를 섞는 것이 이 타입에서 가장 흔한 실수다.

`proven_u16str_t`는 `u"..."` 리터럴이나 이미 가지고 있는 코드 유닛으로부터 만들거나 - 흔한 경우로 - 프로그램이 들고 있는 UTF-8을 `utf.h`로 변환해서 만든다(다음 소절).

`PROVEN_NO_U16STR`이 정의되면 U16 API는 제외되며, 이는 프리스탠딩(freestanding) 빌드의 기본값이다 — 베어메탈 타깃에는 대화할 Windows API가 없다.

잘못된 예 — 코드 유닛 하나가 문자 하나라고 가정하기:

```
proven_u16str_view_t v = ...;                /* text containing an emoji */
proven_size_t chars = proven_u16str_len(&s);  /* wrong: that is code units */
```

구조체

```
typedef struct {
    const proven_u16 *ptr;
    proven_size_t size;
} proven_u16str_view_t;

typedef struct {
    proven_buf_t internal;
} proven_u16str_t;

typedef struct {
    proven_err_t err;
    proven_u16str_t value;
} proven_result_u16str_t;
```

매크로

매크로	의도
`PROVEN_U16_LIT(s)`	UTF-16 리터럴 표현식 `u"..."`로부터 `proven_u16str_view_t`를 만든다.

U16 함수

API	의도	반환
`proven_u16str_create(alloc, unit_limit)`	빈 owned U16 문자열을 만든다.	`proven_result_u16str_t.`

`proven_u16str_create_from_view(alloc, view)`	U16 view를 owned 문자열로 복사한다.	`proven_result_u16str_t.`
`proven_u16str_destroy(alloc, str)`	owned U16 저장소를 해제한다.	void.
`proven_u16str_append(str, data)`	원자적 고정 용량 append.	`proven_err_t.`
`proven_u16str_append_partial(str, data)`	잘라내는 append.	`proven_result_size_t.`
`proven_u16str_append_grow(alloc, str, data)`	원자적 성장 가능 append.	`proven_err_t.`
`proven_u16str_as_ptr(str)`	내부 `proven_u16` 포인터를 반환한다.	`const proven_u16 *.`
`proven_u16str_len(str)`	코드 유닛 단위 길이를 반환한다.	`proven_size_t.`

예제:

```
#ifndef PROVEN_NO_U16STR
proven_result_u16str_t r =
    proven_u16str_create_from_view(alloc, PROVEN_U16_LIT("hello"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u16str_t s = r.value;

(void)proven_u16str_append_grow(alloc, &s, PROVEN_U16_LIT(" world"));

/* Length is in code units, not characters: "hello world" is 11 units. */
proven_size_t units = proven_u16str_len(&s);
(void)units;

proven_u16str_destroy(alloc, &s);
#endif
```

참고: `proven_u16`은 코드 유닛이지, 반드시 하나의 완전한 Unicode 문자인 것은 아니다. UTF-16 서로게이트 쌍은 코드 유닛 두 개를 쓴다.

UTF-8과 UTF-16 사이의 변환 (`utf.h`)
프로그램은 UTF-8을 들고 있고, 윈도 와이드 API는 UTF-16을 원하며, 그 API가 돌려주는 이름과 텍스트는 UTF-16인데 프로그램의 나머지는 그것을 UTF-8로 원한다. 이 경계를 넘는 프로그램은 모두 변환기가 필요하고, 변환기는 올바르지 않은 텍스트를 어떻게 할지 정해야 한다: 홀로 남은 연속 바이트, 과장 인코딩(overlong encoding), 인코딩된 서로게이트, UTF-16 쪽의 짝 없는 서로게이트. 고쳐 주고 싶은 유혹이 있다 - U+FFFD로 바꾸고 계속 가는 것 - 그러나 운영체제에 파일 이름을 건네는 라이브러리에게는 틀린 답이다. 고쳐진 이름은 다른 이름이고, 프로그램은 열라고 한 적 없는 파일을 연다.

그래서 `utf.h`는 어디서나 엄격하다. 잘못된 입력은 `PROVEN_ERR_INVALID_ENCODING`이고, 내 메모리에 쓰는 형태들은 거부할 때 아무것도 쓰지 않는다. 손실 모드는 없다.

잘못된 것과 덜 온 것은 다른 답이다. 조각으로 읽는 텍스트 - 파이프, 버퍼 쓰기 스트림(writer), 콘솔에서 오는 - 는 다음 조각이 완성할 문자의 한가운데서 끝날 수 있다. 부분 변환 형태는 거기서 `PROVEN_ERR_NEED_MORE`로 멈추고 입력을 얼마나 썼는지 알려 주므로, 꼬리를 간직했다가 더 받아 다시 시도하면 된다. 텍스트 전체를 받는 형태에는 기다릴 "다음"이 없으므로, 그쪽에서 문자 중간에 끝나는 텍스트는 잘못된 텍스트다.

세 가지 모양은 라이브러리의 나머지가 쓰는 그대로다: 재고 나서 내 버퍼로 변환(전부 아니면 전무), 소유(owned) 문자열을 늘리며 덧붙이기(전부 아니면 전무), 그리고 입력이나 출력이나 올바른 텍스트가 다할 때까지 온전한 문자만 변환하는 부분 변환. 부분 변환은 서로게이트 쌍의 반쪽이나 UTF-8 시퀀스의 일부를 절대 쓰지 않는다.

API	의도	반환

`proven_utf8_to_utf16_size(src)`	`src`가 변환될 코드 유닛 수; 검증을 겸한다.	`proven_result_size_t`
`proven_utf16_to_utf8_size(src, n)`	코드 유닛 `n`개가 변환될 UTF-8 바이트 수; 검증을 겸한다.	`proven_result_size_t`
`proven_utf8_to_utf16(src, out, out_cap, &written)`	`src` 전부를 `out`으로, 아니면 아무것도. 종결자 없음.	`proven_err_t`
`proven_utf16_to_utf8(src, n, out, out_cap, &written)`	반대 방향으로 같은 일.	`proven_err_t`
`proven_utf8_to_utf16_partial(src, out, out_cap)`	무언가가 멈출 때까지 온전한 문자만.	`proven_utf_step_t`
`proven_utf16_to_utf8_partial(src, n, out, out_cap)`	반대 방향; 끝의 상위 서로게이트는 `NEED_MORE`.	`proven_utf_step_t`
`proven_utf8_append_to_u16str(alloc, dst, src)`	`src`의 UTF-16을 덧붙이며 `dst`를 늘린다; 전부 아니면 전무.	`proven_err_t`
`proven_utf16_append_to_u8str(alloc, dst, src, n)`	코드 유닛 `n`개의 UTF-8을 덧붙이며 `dst`를 늘린다; 전부 아니면 전무.	`proven_err_t`
`proven_utf8_decode_next(s, pos)`	바이트 `pos`에서 시작하는 문자 하나, 또는 그렇지 못한 이유; `len`은 언제나 나아갈 거리다.	`proven_utf8_char_t`
`proven_utf8_encode_one(cp, out)`	코드 포인트 하나를 UTF-8로 `out[0..3]`에.	길이 1~4; 서로게이트나 U+10FFFF 초과면 0.
`proven_utf8_validate(s, bad_at)`	`s` 전체가 올바른 UTF-8인지(끝에서 잘린 문자는 아님).	`PROVEN_OK`, 또는 `PROVEN_ERR_INVALID_ENCODING`과 `*bad_at`의 첫 잘못된 위치.

```
typedef struct {
    proven_err_t  err;       /* OK, OUT_OF_BOUNDS (output full), NEED_MORE (input ends
                                mid-character), INVALID_ENCODING (malformed at `consumed`) */
    proven_size_t consumed;  /* input units used: bytes for UTF-8, code units for UTF-16 */
    proven_size_t written;   /* output units written */
} proven_utf_step_t;

typedef enum {
    PROVEN_TEXT_UTF8, PROVEN_TEXT_UTF16LE, PROVEN_TEXT_UTF16BE,
    PROVEN_TEXT_AUTO         /* readers only: decided by a byte order mark */
} proven_text_encoding_t;    /* for the u16 writers and readers in stream.h (chapter 5) */

typedef struct {
    proven_err_t  err;       /* OK, INVALID_ENCODING, NEED_MORE, EOF, OUT_OF_BOUNDS, INVALID_ARG */
    proven_u32    cp;        /* the scalar value when err is OK; 0 otherwise */
    proven_size_t len;       /* bytes this step covers: the character, the maximal subpart of
                                malformed input, or the bytes left when the text ends mid-character */
} proven_utf8_char_t;
```

반례 - 바이트 하나를 코드 유닛 하나로 넓히는 경우. ASCII에서는 맞기 때문에 테스트를 통과해 살아남고, 다른 모든 문자는 의미 없는 유닛 두세 개가 된다. 라이브러리 자신의 `proven_time_u16_fmt`가 `utf.h`가 생기기 전까지 호출자의 비 ASCII 요일 이름에 정확히 이렇게 했다:

```
for (size_t i = 0; i < text.size; ++i) {
    proven_u16 unit = (proven_u16)text.ptr[i];        /* wrong: a byte is not a code unit */
    (void)proven_u16str_append_grow(alloc, &wide, (proven_u16str_view_t){ &unit, 1 });
}
```

반례 - 조각을 전체 텍스트 형태로 변환하는 경우. 문자 한가운데서 끝나는 조각은 텍스트가 멀쩡한데도 잘못된 것으로 거부된다. 부분 변환 형태를 쓰고 꼬리를 이어 가져가라:

```
proven_utf8_to_utf16(piece, out, cap, &n);   /* wrong for pieces: a split character is INVALID */
```

한 번에 문자 하나. 분할기, 셰이퍼, 렉서는 텍스트를 문자 하나씩 걷는다. 그리고 텍스트 전체를 거부하기보다 잘못된 바이트를 그 자리에서 알려 주기를 원한다. `proven_utf8_decode_next`는 바이트 오프셋 하나에서 시작하는 문자 하나를 같은 엄격한 규칙으로 디코드하고, 그 `len`은 언제나 나아갈 거리다: 문자의 길이, 또는 잘못된 입력이면 최대 부분 시퀀스(maximal subpart) - 아직 올바른 시작이었던 가장 긴 앞부분 - 이며, 이것이 유니코드가 다시 맞추라고 말하는 자리다. 아무것도 대신 넣지 않는다. 텍스트를 그리는 프로그램은 잘못된 걸음마다 U+FFFD 하나를 넣고(유니코드 권장 방식), 검증하는 프로그램은 처음에서 멈춘다.

```
proven_u8str_view_t text = PROVEN_LIT("a\xC3\xA9\xFF" "z\xE2\x82");
proven_size_t chars = 0, malformed = 0;
for (proven_size_t pos = 0; pos < text.size;) {
    proven_utf8_char_t c = proven_utf8_decode_next(text, pos);
    if (c.err == PROVEN_OK) ++chars;                              /* c.cp: 'a', U+00E9, 'z' */
    else if (c.err == PROVEN_ERR_INVALID_ENCODING) ++malformed;  /* the FF: one step */
    /* PROVEN_ERR_NEED_MORE: E2 82 start a character this text never finished */
    pos += c.len;                                                 /* at least 1 inside the text */
}
(void)chars; (void)malformed;
```

예제는 한국어 파일 이름을 와이드 API용으로 변환했다가 되돌리고, 아무것도 쓰지 않는 거부를 보이고, 소유 문자열을 양방향으로 늘리며, 두 조각 사이에서 잘린 문자를 이어 붙인다:

```
#include <string.h>

/*
 * 프로그램은 UTF-8 로 생각하고, 윈도의 "와이드" 호출은 UTF-16 을 원한다. utf.h 가
 * 양방향의 건널목이고, 엄격하다 - 올바른 UTF-8 이나 UTF-16 이 아닌 텍스트는 고쳐지지
 * 않고 거부된다. 잘못된 바이트를 슬쩍 물음표로 바꾸는 변환기는, 막 열려던 파일 이름을
 * 바꿔 버린 것이다.
 *
 * 이 라이브러리 어디서나처럼 세 가지 모양이 있다: 재고 나서 내 버퍼로 변환(전부 아니면
 * 전무), 소유 문자열을 늘리며 덧붙이기(전부 아니면 전무), 조각으로 도착하는 텍스트를
 * 위한 부분 변환.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* "보고서.txt" - 프로그램의 나머지가 들고 있는 모양 그대로의 파일 이름. */
    proven_u8str_view_t name = PROVEN_LIT("\xEB\xB3\xB4\xEA\xB3\xA0\xEC\x84\x9C.txt");

    /* --- 재고 나서, 내 버퍼로 변환 ---------------------------------------- */

    /* 먼저 묻는다: 답은 코드 단위이고, 검증도 겸한다. 한글 아홉 바이트는 여기서
     * 코드 단위 아홉 개가 아니라 세 개다. */
    proven_result_size_t need = proven_utf8_to_utf16_size(name);
    EXAMPLE_REQUIRE(proven_is_ok(need.err) && need.value == 7, "3 syllables + \".txt\" are 7 code units");

    /* 와이드 API 가 기대하는 NUL 자리로 하나 더; 변환기는 NUL 을 쓰지 않는다. */
    proven_u16 wide[16];
    proven_size_t units = 0;
    proven_err_t err = proven_utf8_to_utf16(name, wide, 15, &units);
    EXAMPLE_REQUIRE(proven_is_ok(err) && units == 7, "the name converts whole");
    wide[units] = 0;
    EXAMPLE_REQUIRE(wide[0] == 0xBCF4, "the first code unit is the syllable, not its first byte");
    /* 윈도라면: CreateFileW((LPCWSTR)wide, ...). 어디서나 돌도록 여기서는 부르지 않는다. */
```

```
    /* 그리고 되돌리기: 와이드 API 가 돌려준 이름을, 나머지 모두를 위해 UTF-8 로. */
    proven_result_size_t back = proven_utf16_to_utf8_size(wide, units);
    EXAMPLE_REQUIRE(proven_is_ok(back.err) && back.value == name.size, "the reverse size is the original byte
count");
    proven_byte_t bytes[32];
    proven_size_t nbytes = 0;
    err = proven_utf16_to_utf8(wide, units, bytes, sizeof bytes, &nbytes);
    EXAMPLE_REQUIRE(proven_is_ok(err) && nbytes == name.size && memcmp(bytes, name.ptr, nbytes) == 0,
                    "the round trip gives the same bytes");

    /* --- 엄격: 잘못된 텍스트는 거부되고, 아무것도 쓰이지 않는다 -------- */

    /* 잘렸거나 잘못 해독된 이름에 있을 법한, 홀로 남은 연속 바이트. */
    proven_u8str_view_t broken = PROVEN_LIT("bad\x80.txt");
    wide[0] = 0x1234;
    err = proven_utf8_to_utf16(broken, wide, 15, &units);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_ENCODING, "a stray continuation byte is refused");
    EXAMPLE_REQUIRE(units == 0 && wide[0] == 0x1234, "and the output is untouched");

    /* --- 소유 문자열 늘리기 ---------------------------------------------- */

    proven_result_u16str_t r = proven_u16str_create(alloc, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating the wide path must succeed");
    proven_u16str_t path = r.value;
    err = proven_utf8_append_to_u16str(alloc, &path, PROVEN_LIT("C:\\reports\\"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the directory appends");
    err = proven_utf8_append_to_u16str(alloc, &path, name);
    EXAMPLE_REQUIRE(proven_is_ok(err) && proven_u16str_len(&path) == 11 + 7, "and the name, growing the
string");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&path)[18] == 0, "still NUL-terminated for the system call");

    /* 실패한 덧붙이기는 아무것도 바꾸지 않는다: 반쯤 변환된 꼬리가 없다. */
    err = proven_utf8_append_to_u16str(alloc, &path, broken);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_ENCODING && proven_u16str_len(&path) == 18, "a refused append
leaves the path as it was");

    proven_result_u8str_t r8 = proven_u8str_create(alloc, 8);
    EXAMPLE_REQUIRE(proven_is_ok(r8.err), "creating the UTF-8 copy must succeed");
    proven_u8str_t copy = r8.value;
    err = proven_utf16_append_to_u8str(alloc, &copy, proven_u16str_as_ptr(&path), proven_u16str_len(&path));
    EXAMPLE_REQUIRE(proven_is_ok(err) && copy.internal.len == 11 + name.size, "the whole path comes back as
UTF-8");

    /* --- 부분 변환: 조각으로 도착하는 텍스트 ------------------------------ */

    /* 읽기 스트림이 4 바이트씩 넘겨 주고, "고" 는 첫 바이트 뒤에서 잘린다. 부분 변환은
     * 온전한 것만 변환하고 NEED_MORE 로 멈추며, 얼마나 썼는지 알려 준다. 호출자는
     * 나머지를 간직했다가 다음 조각을 붙인다. */
    proven_u16 out[8];
    proven_utf_step_t st = proven_utf8_to_utf16_partial((proven_u8str_view_t){ name.ptr, 4 }, out, 8);
    EXAMPLE_REQUIRE(st.err == PROVEN_ERR_NEED_MORE && st.consumed == 3 && st.written == 1,
                    "one whole syllable, and the start of the next kept back");
    st = proven_utf8_to_utf16_partial((proven_u8str_view_t){ name.ptr + st.consumed, 6 }, out + 1, 7);
    EXAMPLE_REQUIRE(proven_is_ok(st.err) && st.written == 2 && out[1] == 0xACE0, "with the next piece it
completes");

    /* 반대 방향에서도 같은 구분이다: 끝의 상위 서로게이트는, 나머지 반쪽이 다음
     * 조각에 있는 쌍의 반쪽일 수 있다. */
    const proven_u16 emoji_half[] = { 'o', 'k', 0xD83D };
```

```
    proven_byte_t u8out[16];
    st = proven_utf16_to_utf8_partial(emoji_half, 3, u8out, sizeof u8out);
    EXAMPLE_REQUIRE(st.err == PROVEN_ERR_NEED_MORE && st.consumed == 2, "a trailing high surrogate waits");

    /* --- 문자 하나 내보내기, 텍스트 전체 검사하기 ----------------------------- */

    /* 인코딩은 문자가 아닌 것을 거절한다: 홀로 선 서로게이트, U+10FFFF 너머. */
    proven_byte_t one[4];
    EXAMPLE_REQUIRE(proven_utf8_encode_one(0xBCF4, one) == 3 && one[0] == 0xEB, "a Hangul syllable is three
bytes");
    EXAMPLE_REQUIRE(proven_utf8_encode_one(0xD83D, one) == 0, "a surrogate is not a character");

    /* 검증은 텍스트가 어디서부터 UTF-8이 아닌지 알려 준다. */
    proven_size_t bad = 0;
    EXAMPLE_REQUIRE(proven_utf8_validate(name, NULL) == PROVEN_OK, "the name is valid UTF-8");
    EXAMPLE_REQUIRE(proven_utf8_validate(PROVEN_LIT("ok\xC3"), &bad) == PROVEN_ERR_INVALID_ENCODING && bad ==
2, "a cut-off character is found at its first byte");

    printf("converted a %zu-byte name to %zu code units and back\n", (size_t)name.size, (size_t)need.value);

    proven_u8str_destroy(alloc, &copy);
    proven_u16str_destroy(alloc, &path);
    return EXAMPLE_OK();
}
```

3. 포매팅

문제: `printf`는 타입을 두 번 듣는다

`printf("%d", x)`는 `x`의 타입을 두 번 말한다 — 한 번은 포맷 문자열에서, 한 번은 `x`를 전달하면서 — 그리고 그 둘이 일치하는지는 아무것도 검사하지 않는다. 가변 인자는 타입을 지우므로, 함수는 호출 규약이 남긴 바이트가 무엇이든 포맷이 요구한 모양으로 읽는다:

```
printf("%d\n", 3.0);      /* wrong: reads a double's bytes as an int */
printf("%s\n", 42);       /* wrong: dereferences 42 as a pointer */
printf("%d %d\n", 1);     /* wrong: reads an argument that was never passed */
```

셋 다 컴파일된다. 현대 컴파일러는 포맷이 리터럴일 때 경고해 주는데, 그건 포맷이 변수가 되기 전까지의 이야기다 — 그리고 그때 여러분에게는 요청만 하면 임의의 스택 메모리를 읽어 주는 함수가 생기며, 이는 자기 이름까지 붙은 취약점 부류다.

두 번째 문제는 출력이 어디로 가느냐다. `sprintf`는 자기가 크기를 모르는 버퍼에 쓴다. `snprintf`는 크기를 받고 나서 **잘라내며**, 썼을 뻔한 길이를 반환한다 — 그러니 비교하는 것을 잊은 호출자는 조용히 짧아진 경로나 명령이나 식별자를 얻는다.

이 라이브러리가 대신 하는 것

자리표시자 안에는 타입이 없다. `{}`는 "여기에 값이 온다"고 말하고, 타입은 인자에서 오며 컴파일 시간에 결정된다:

```
proven_println("{} scored {}", PROVEN_ARG(name), PROVEN_ARG(score));
```

`PROVEN_ARG`는 `_Generic` 디스패치다 — 컴파일러가 인자의 정적 타입에 맞는 생성자를 고른다. 포맷은 결코 타입을 말하지 않으므로 포맷과 인자가 어긋나는 일은 애초에 불가능하다. `:` 뒤의 스펙이 제어하는 것은 표현 — 폭, 채움, 정렬, 정밀도, 진법 — 이지 결코 해석이 아니다.

목적지는 크기를 아는 객체이며, 고정 용량 형태는 잘라내는 대신 거부한다: `proven_u8str_append_fmt`은 `PROVEN_ERR_OUT_OF_BOUNDS`로 실패하고 아무것도 쓰지 않으며, `proven_u8str_append_fmt_grow`는 allocator를 받아 성장한다. 어느 쪽을 호출했는지는 호출 그 자체에 드러난다.

여러분의 타입도 여기에 참여할 수 있다. `PROVEN_ARG_OF(&obj, render_fn)`은 여러분이 정의한 타입이 다른 모든 것처럼 `{}`로 출력되게 해 준다 — 확장 지점은 이름 레지스트리가 아니라 컴파일 시간의 타입 있는 지점이다. 8장 §5.1이 방법을 보여준다.

비용을 그대로 말하자면: 인자마다 `PROVEN_ARG`를 두르는 것은 `%d`보다 타이핑이 많고, 이 포맷 언어는 손에 익은 그 언어가 아니다. 그 대가로 사는 것은 이 절 첫머리의 버그 부류를 아예 쓸 수 없게 된다는 것이다.

포매터는 `proven_u8str_t`나 PAL 기반 스트림에 쓴다. `{}` 자리표시자, `{1}` 같은 명시적 인덱스, 이스케이프된 중괄호 `{{`와 `}}`, 그리고 `{:0>5}`, `{:*^10}`, `{:.<10}` 같은 폭/정렬 스펙을 갖춘 작은 구조적 포맷 언어를 사용한다.

잘못된 예 — 스펙이 타입을 고른다고 가정하기:

```
proven_println("{:d}", PROVEN_ARG(3.5));   /* wrong: the spec formats, it does not convert */
```

구조체와 열거형

```
typedef struct {
    proven_err_t err;
    proven_size_t written;
    proven_size_t required;
} proven_fmt_result_t;
```

필드:

- `err`: 결과 코드.
- `written`: 실제로 쓰인 바이트.
- `required`: 전체 출력에 필요한 바이트.

`proven_arg_type_t` 변종:

- `PROVEN_ARG_NONE`
- `PROVEN_ARG_I32`
- `PROVEN_ARG_U32`
- `PROVEN_ARG_I64`
- `PROVEN_ARG_U64`
- `PROVEN_ARG_F64` — `PROVEN_FMT_NO_FLOAT`가 정의되지 않은 경우
- `PROVEN_ARG_CSTR`
- `PROVEN_ARG_STR_VIEW`
- `PROVEN_ARG_DATETIME`
- `PROVEN_ARG_PTR`
- `PROVEN_ARG_FN`

```
typedef struct {
    proven_arg_type_t type;
    union {
        proven_i32 i32;
        proven_u32 u32;
        proven_i64 i64;
        proven_u64 u64;
        double f64;
        const char *cstr;
        proven_u8str_view_t str_view;
        proven_datetime_t datetime;
        const void *ptr;
        void (*fn)(void);
    } value;
} proven_arg_t;
```

포맷 인자 생성자

API	의도

`proven_arg_none()`	내부 센티널 값.
`proven_arg_i32(v)`, `proven_arg_u32(v)`	정수 인자.
`proven_arg_i64(v)`, `proven_arg_u64(v)`	넓은 정수 인자.
`proven_arg_f64(v)`	부동소수점 인자 — float 포매팅이 비활성화되지 않은 경우.
`proven_arg_cstr(v)`	신뢰할 수 있고 살아 있는 NUL 종료 C 문자열.
`proven_arg_cstr_n(v, max_len)`	경계가 있는 C 문자열 인자; `max_len`까지만 NUL을 탐색한다.
`proven_arg_str_view(v)`	borrowed 문자열 view 인자.
`proven_arg_u16(v)`	borrowed UTF-16 view, UTF-8로 렌더링된다. 엄격: 짝 없는 서로게이트는 포맷 전체를 실패시킨다. 너비는 u8 view와 같이 UTF-8 바이트로 센다. `PROVEN_ARG`는 `proven_u16str_view_t`에 이것을 고른다.
`proven_arg_datetime(v)`	datetime 인자.
`proven_arg_ptr(v)`	객체 포인터 인자.
`proven_arg_fn(v)`	함수 포인터 인자.
`proven_arg_ucstr(v)`	unsigned char C 문자열 헬퍼.
`proven_arg_identity(v)`	기존 `proven_arg_t`를 그대로 통과시킨다.

포맷 매크로

매크로	의도
`PROVEN_ARG(x)`	지원되는 인자 타입에 대한 `_Generic` 선택자.
`PROVEN_ARG_FN(f)`	함수 포인터 포매팅 헬퍼.
`PROVEN_ARG_CSTR_N(v, max_len)`	경계가 있는 C 문자열 헬퍼.
`proven_u8str_append_fmt(str, fmt, ...)`	원자적 고정 용량 포매팅.
`proven_u8str_append_fmt_trunc(str, fmt, ...)`	최선을 다하는 잘라내기 포매팅.
`proven_u8str_append_fmt_grow(alloc, str, fmt, ...)`	원자적 성장 가능 포매팅.
`proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)`	임시 패치 할당에 임시 작업용(scratch) allocator를 쓰는 성장 가능 포매팅.
`PROVEN_FMT_IS_OK(res)`	`proven_fmt_result_t`를 검사한다.

포매팅 엔진

```
proven_fmt_result_t proven_u8str_fmt_internal(
    proven_allocator_t alloc,
    proven_u8str_t *str,
    bool trunc,
    const char *fmt,
    proven_allocator_t scratch,
    const proven_arg_t *args,
    proven_size_t args_count
);
```

일반 사용자 코드는 이 내부 엔진 대신 매크로를 호출해야 한다. 엔진은 인덱스 0에 선두 `proven_arg_none()` 센티널이 있을 것을 기대한다.

사용되지 않는 여분의 포맷 인자는 `PROVEN_ERR_INVALID_ARG`를 반환한다.

예제:

```
proven_result_u8str_t r = proven_u8str_create(alloc, 8);
if (!proven_is_ok(r.err)) {
    return;
}
```

```
proven_u8str_t s = r.value;

/* The target is only 8 bytes; the _grow form reallocates rather than truncate. */
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(
    alloc,
    &s,
    "name={} score={:0>4}",
    PROVEN_ARG(PROVEN_LIT("ada")),
    PROVEN_ARG(42)
);
if (!PROVEN_FMT_IS_OK(fr)) {
    proven_u8str_destroy(alloc, &s);
    return;
}
/* s == "name=ada score=0042" */

proven_u8str_destroy(alloc, &s);
```

4. 스캐닝

문제: `scanf`는 어디서 멈췄는지 말해 주지 않는다
입력을 파싱하는 것은 포매팅의 거울이고, libc의 답은 더 나쁘다. `sscanf`는 *몇 개의 필드를 채웠는지*만 반환하고 그 외에는 아무것도 주지 않는다 — 어느 것이 실패했는지도, 어디까지 갔는지도, 왜인지도:

```
int n = sscanf(line, "%d %d %d", &a, &b, &c);
if (n != 3) { /* wrong: which field? at what offset? was it malformed or missing? */ }
```

세 번째 필드가 잘못된 형식이라면, 여러분이 아는 것은 둘을 얻었다는 것뿐이다. 사용자에게 위치를 보고할 수 없고, 잘못된 레코드를 건너뛰고 재개할 수 없으며, "입력이 떨어졌다"와 "숫자가 아닌 무언가를 찾았다"를 구별할 수 없다. 그리고 `char *`로의 `%s`는 `strcpy`와 똑같은 무경계 쓰기 문제를 가지며, 이번에는 입력이 프로그램 바깥에서 온다.

그리고 `strtol`이 있는데, 그 계약은 먼저 `errno`를 지우고, 나중에 검사하고, "숫자가 하나도 없음"을 탐지하려고 `endptr`을 입력과 비교할 것을 요구한다 — 변환 하나에 제대로 해내야 할 것이 셋이다.

이 라이브러리가 대신 하는 것
커서는 여러분의 것이다. `proven_scan_t`는 파싱 중인 view와 그 안의 오프셋을 담는다. 각 스캔은 커서에서 읽고 성공하면 커서를 앞으로 옮긴다. 커서는 여러분이 읽을 수 있는 필드이므로, 파싱이 정확히 어디서 멈췄는지 항상 알 수 있다 — 그것이 사용자에게 보여줄 위치다.

각 스캔은 result를 반환한다. `proven_scan_i64`는 `{err, val}`을 돌려주므로, "숫자가 아님", "범위를 벗어남", "입력의 끝"이 하나의 빠진 필드가 아니라 서로 다른 에러가 된다.

실패는 커서를 되돌린다 — 원시 스캐너들에 한해서. 실패한 `proven_scan_i64`는 커서를 있던 자리에 남기므로, 같은 위치에서 다른 것을 시도할 수 있다. 그것이 짐작 대신 복구를 가능하게 하는 것이다.

이것에 기대기 전에 알아 둘 것 하나: 구조적 스캔(`proven_scan_fmt`, `{}` 형태)은 필드 전체에 걸쳐 트랜잭션적이지 **않다**. 세 번째 자리표시자가 실패하면 앞의 두 목적지에는 이미 쓰여 있다. 8장 §11.1이 에러 코드와 복구 패턴을 온전히 다룬다.

스캐너는 borrow된 `proven_u8str_view_t`로부터 파싱한다. 커서가 진행 상황을 추적한다.

잘못된 예 — 부분적인 구조적 스캔을 아무 일도 없었던 것처럼 다루기:

```
proven_err_t e = proven_scan_fmt(&sc, "{} {} {}", ...);
if (!proven_is_ok(e)) {
    /* wrong: destinations for the fields that DID parse have already been written */
}
```

Result 구조체

```
typedef struct { proven_err_t err; proven_i64 val; } proven_result_i64_t;
typedef struct { proven_err_t err; proven_u64 val; } proven_result_u64_t;
typedef struct { proven_err_t err; double val; } proven_result_f64_t;
typedef struct { proven_err_t err; proven_u8str_view_t val; } proven_result_u8str_view_t;
```

proven_scan_t

```
typedef struct {
    proven_u8str_view_t view;
    proven_size_t cursor;
} proven_scan_t;
```

목적: 입력과 현재 파싱 위치를 담는다. `proven_scan_init()`은 유효하지 않은 비어 있지 않은 널 view를 빈 view로 정규화한다.

스캐너 함수

API	의도	반환
`proven_scan_init(view)`	view로부터 스캐너를 만든다.	`proven_scan_t`.
`proven_scan_skip_whitespace(scan)`	공백을 지나 진행한다.	void.
`proven_scan_i64(scan)`	부호 있는 64비트 정수를 파싱한다.	`proven_result_i64_t`.
`proven_scan_u64(scan)`	부호 없는 64비트 정수를 파싱한다.	`proven_result_u64_t`.
`proven_scan_f64(scan)`	부동소수점 값을 파싱한다.	`proven_result_f64_t`.
`proven_parse_double_ascii(view)`	공백을 건너뛰지 않고 로케일 독립적인 ASCII float 토큰 하나를 파싱한다.	`proven_parse_double_result_t`.
`proven_parse_f64_ascii(view)`	같은 로케일 독립 binary64 파서에 대한 호환 별칭.	`proven_parse_f64_result_t`.
`proven_strtod(nptr, endptr)`	공백 건너뛰기와 `endptr` 보고를 갖춘 `strtod` 스타일 토큰 하나를 파싱한다.	`double`.
`proven_scan_str(scan)`	공백으로 구분된 토큰을 입력에 대한 view로 파싱한다.	`proven_result_u8str_view_t`.
`proven_scan_skip_until(scan, target)`	target을 찾으면 커서를 그쪽으로 옮긴다.	`proven_err_t`.
`proven_scan_skip_until_number(scan)`	다음으로 숫자처럼 보이는 위치로 커서를 옮긴다.	void.

스캔 인자 타입

`proven_scan_arg_type_t`는 `proven_scan_arg_t`에 저장된 목적지 종류를 식별한다. `PROVEN_SCAN_ARG(&x)`는 다음에 대한 포인터를 지원한다:

- `short`, `unsigned short`
- `int`, `unsigned int`
- `long`, `unsigned long`
- `long long`, `unsigned long long`
- `double`
- `proven_u8str_view_t`

네이티브 목적지 생성자:

- `proven_scan_arg_short`, `proven_scan_arg_ushort`
- `proven_scan_arg_int`, `proven_scan_arg_uint`
- `proven_scan_arg_long`, `proven_scan_arg_ulong`
- `proven_scan_arg_llong`, `proven_scan_arg_ullong`

명시적 고정 폭 및 유틸리티 헬퍼:

- `proven_scan_arg_i32`, `proven_scan_arg_u32`
- `proven_scan_arg_i64`, `proven_scan_arg_u64`
- `proven_scan_arg_f64`
- `proven_scan_arg_str_view`
- `proven_scan_arg_none`
- `proven_scan_arg_identity`

Long 별칭:

```
#define PROVEN_SCAN_ARG_LONG(ptr)  proven_scan_arg_long(ptr)
#define PROVEN_SCAN_ARG_ULONG(ptr) proven_scan_arg_ulong(ptr)
```

포맷 스캐닝 매크로

매크로	의도
`PROVEN_SCAN_ARG(x)`	`_Generic` 목적지 선택자.
`proven_scan_fmt_cursor(scan_ptr, fmt, ...)`	기존 커서에서 스캔한다.
`proven_scan_fmt(view, fmt, ...)`	임시 커서로 view에서 스캔한다.

스캔 엔진

```
proven_err_t proven_scan_fmt_internal(
    proven_scan_t *scan,
    const char *fmt,
    const proven_scan_arg_t *args,
    proven_size_t args_count
);
```

API	의도	반환
`proven_scan_fmt_internal(scan, fmt, args, args_count)`	기존 커서 위에서 동작하는 구조적 스캐너 엔진.	`proven_err_t.`
`proven_scan_fmt_internal_view(view, fmt, args, count)`	view 위에 임시 스캐너를 만드는 편의 래퍼.	`proven_err_t.`

일반 사용자 코드는 매크로를 호출해야 한다. 엔진은 선두 센티널 인자를 기대한다.

중요한 동작: `proven_scan_fmt_internal()`은 뒤쪽에서 리터럴 불일치가 일어나면 에러를 반환하기 전에 커서를 진행시키고 앞선 목적지에 써 넣었을 수 있다. 트랜잭션 같은 파싱이 필요하다면 커서와 목적지 값을 먼저 저장하라.

예제:

```
proven_scan_t scan = proven_scan_init(PROVEN_LIT("ID: 402 SCORE: 99.5 ada"));
int id = 0;
double score = 0.0;
proven_u8str_view_t user = {0};

proven_err_t e = proven_scan_fmt_cursor(
    &scan,
    "ID: {} SCORE: {} {}",
    PROVEN_SCAN_ARG(&id),
    PROVEN_SCAN_ARG(&score),
    PROVEN_SCAN_ARG(&user)
);
if (!proven_is_ok(e)) {
    return;
}
```

```
/* id == 402, score == 99.5, user borrows "ada" out of the input - it is not a
 * copy, so it is only valid while the scanned bytes are. */
```

Float 파싱 관련 사항

- `proven_scan_f64()`와 `proven_parse_double_ascii()`는 공유된 decimal-to-binary64 백엔드를 거친다.
- 유한한 십진 입력은 round-to-nearest, ties-to-even 동작으로 IEEE-754 binary64로 반올림된다.
- 현재 변환 스택은 `Clinger fast path -> direct Eisel-Lemire -> staged Eisel-Lemire layer -> exact bigint fallback`이며, 대표적인 입력이 어느 경로를 탔는지 테스트가 확인할 수 있도록 내부 카운터를 둔다.
- direct Eisel-Lemire 계층은 유효숫자에 128비트로 자른 5의 거듭제곱을 곱하고, 곱의 하위 비트로 반올림을 증명한다. 증명할 수 없는 것(동률일 수 있는 값, 비정규 또는 오버플로 결과)은 넘긴다. 19자리보다 긴 유효숫자는 앞 19자리와 거기에 1을 더한 값으로 위아래를 묶고, 두 경계가 같게 반올림될 때 그 값을 쓴다.
- staged Eisel-Lemire 계층은 현재 생성된 `5^q` 양수 지수 경우와, 십진 유효숫자가 요구되는 `5^q`를 깔끔하게 상쇄하는 정확한 음수 지수 경우를 받아들인다.
- 모든 대상에서(128비트 정수 타입이 필요 없다) 같은 계층이 정상 범위 경우에 대해 보수적으로 반올림된 음수 지수 비율 부분집합도 받아들이며, 원래의 좁은 프로토타입이 허용하던 것보다 더 넓은 좌시프트 정규화를 포함한다.
- 넓혀진 캐시된 거듭제곱 곱 경로는 이제 `5e-324`를 포함한 일부 비정규(subnormal) 경우도 단계적으로 처리하지만, true-min에 대한 절반 임계값 아래의 값들은 여전히 정확한 bigint fallback으로 넘긴다.
- 체크인된 캐시 `5^q` 상수들은 `scripts/generate_float_decimal_tables.py`로 로컬에서 생성된다.
- `proven_parse_double_ascii()`는 선행 공백을 건너뛰지 않는다.
- `proven_strtod()`는 선행 ASCII 공백을 건너뛰고, `endptr`을 갱신하며, 오버플로 시 부호 있는 무한대를 반환하고, 언더플로 시 부호 있는 0을 보존한다.

5. 예제와 오용 사례

view는 C 문자열이 아니다

잘못된 예:

```
proven_u8str_view_t view = get_view();
printf("%s\n", (const char *)view.ptr); /* wrong: view may not be NUL-terminated */
```

올바른 예:

```
proven_u8str_view_t view = proven_u8str_view_slice(PROVEN_LIT("/etc/hosts"), 5, 5);

/* A view is a pointer and a length into somebody else's bytes. To hand it to a
 * C API that wants a NUL, allocate a real C string from it. */
proven_result_cstr_t c = proven_u8str_view_to_cstr(view, alloc);
if (!proven_is_ok(c.err)) {
    return;
}
/* c.value is "hosts", NUL-terminated. It is yours, so free it. */
alloc.free_fn(alloc.ctx, (void *)c.value);
```

`PROVEN_LIT`은 리터럴을 위한 것이다

올바른 예:

```
proven_u8str_view_t a = PROVEN_LIT("abc");
(void)a;
```

잘못된 예:

```
const char *runtime = getenv("NAME");
proven_u8str_view_t a = PROVEN_LIT(runtime); /* wrong: macro requires literal syntax */
```

이렇게 하라:

```
const char *runtime = "NAME=value";   /* any trusted NUL-terminated string */
proven_u8str_view_t a = proven_u8str_view_from_cstr(runtime);
(void)a;
```

찾지 못한 것과 치환된 것을 구별하라

`proven_u8str_replace_first()`는 target을 찾지 못했을 때 `PROVEN_OK`를 반환한다. 그 차이가 중요하다면 먼저 검색하라.

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("the old way"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&s), 0, PROVEN_LIT("old"));
if (at != PROVEN_INDEX_NOT_FOUND) {
    (void)proven_u8str_replace_first(&s, 0, PROVEN_LIT("old"), PROVEN_LIT("new"));
}

proven_u8str_destroy(alloc, &s);
```

경계가 있는 포맷 입력

잘못된 예:

```
char *untrusted = get_untrusted_pointer();
proven_println("{}", PROVEN_ARG(untrusted));
/* wrong: C-string formatting scans until NUL */
```

올바른 예:

```
char untrusted[16] = { 'n', 'o', ' ', 'n', 'u', 'l', ' ', 'h', 'e', 'r', 'e', '!', '!', '!', '!', '!' };
proven_size_t max_len = sizeof untrusted;

/* The bounded form stops looking for a NUL after max_len bytes. */
proven_println("{}", PROVEN_ARG_CSTR_N(untrusted, max_len));
```

Borrow된 고정 용량 문자열

`proven_u8str_borrow`를 사용하면 아무 할당 없이 스택이나 정적 버퍼에 포매팅할 수 있다 — allocator가 없는 코드와 뜨거운 경로에서 유용하다. 고정 용량 연산(그리고 `proven_u8str_append_fmt`)을 사용하고, `proven_u8str_reset`으로 재사용하며, 성장 연산이나 `proven_u8str_destroy`는 호출하지 말라(메모리는 호출자가 소유한다).

```
int cur = 3;
int total = 10;

proven_byte_t line[64];
proven_u8str_t s = proven_u8str_borrow(line, sizeof line);   /* cap includes NUL */

proven_fmt_result_t fr = proven_u8str_append_fmt(&s, "L{}/{}", PROVEN_ARG(cur), PROVEN_ARG(total));
if (PROVEN_FMT_IS_OK(fr)) {
    proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
}

(void)proven_u8str_reset(&s);   /* reuse next frame, no allocation */
```

오용: borrow된 용량을 넘길 성장 호출은 호출자 메모리를 재할당하는 대신 `PROVEN_ERR_OUT_OF_BOUNDS`를 반환한다.

```
proven_byte_t small[4];
proven_u8str_t t = proven_u8str_borrow(small, sizeof small);
proven_err_t e = proven_u8str_append_grow(alloc, &t, PROVEN_LIT("toolong"));
(void)e;   /* e == PROVEN_ERR_OUT_OF_BOUNDS; small[] is untouched */
```

자기 참조 포매팅

잘못된 예:

```
const char *inside = proven_u8str_as_cstr(&s);
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(inside));
/* wrong: self-aliasing C-string arguments are rejected */
```

올바른 예:

```
proven_result_u8str_t r = proven_u8str_create_from_view(alloc, PROVEN_LIT("ab"));
if (!proven_is_ok(r.err)) {
    return;
}
proven_u8str_t s = r.value;

/* A view carries a length, so the formatter knows exactly which bytes to snapshot. */
proven_u8str_view_t before = proven_u8str_as_view(&s);
proven_fmt_result_t fr = proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(before));
(void)fr;   /* s == "abab" */

proven_u8str_destroy(alloc, &s);
```

트랜잭션적 스캐닝

잘못된 가정:

```
proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
/* if suffix mismatches, x and scan.cursor may already have changed */
```

올바른 패턴:

```
proven_scan_t scan = proven_scan_init(PROVEN_LIT("42 prefix"));
int x = -1;

proven_size_t old_cursor = scan.cursor;
int old_x = x;

proven_err_t e = proven_scan_fmt_cursor(&scan, "{} suffix", PROVEN_SCAN_ARG(&x));
if (!proven_is_ok(e)) {
    /* The literal "suffix" did not match, but 42 was already written into x and
     * the cursor already moved. Put both back yourself. */
    scan.cursor = old_cursor;
    x = old_x;
}
```

실전 예제: owned 문자열과 borrow된 문자열

테스트 스위트에서 컴파일되고 실행된다. 중요한 구분은 이것이다: owned 문자열은 재할당될 수 있고 반드시 파괴해야 한다; borrow된 문자열은 호출자 메모리를 감싸고, 결코 재할당하지 않으며, 여러분이 준 버퍼를 넘어 성장하는 대신 그것을 조용히 옮기지 않고 거부한다.

```
/*
 * 여기 문자열 손잡이가 둘 있고, 둘을 가르는 것은 크기가 아니라 소유다.
 *
 *   proven_u8str_t      - 고칠 수 있는 바이트 문자열. 할당을 소유하거나(create)
 *                         여러분의 것을 빌린다(borrow).
 *   proven_u8str_view_t - 남의 바이트를 가리키는 포인터와 길이. 아무것도 소유하지
 *                         않고, NUL 로 끝나지 않으며, 그 바이트가 살아 있는 동안에만
 *                         쓸 수 있다.
 *
 * 읽기만 하는 함수에 건네는 것이 뷰이고, 손에 쥐고 있는 것이 u8str 다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- *소유하는* 문자열: 할당자의 기억, 여러분이 지울 것 ---------------- */
```

```
/* 용량 인자는 내용 바이트 수다. NUL 은 그 밖에 따로 있어서, as_cstr 은 언제나
 * O(1) 이고 언제나 안전하다. */
proven_result_u8str_t r = proven_u8str_create(alloc, 16);
EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 16-byte string must succeed");
if (!proven_is_ok(r.err)) {
    return 1;
}
proven_u8str_t path = r.value;

/* append 는 용량이 고정이다. 들어가거나 실패하고, 실패했다면 문자열에 손도 대지
 * 않았다. 재할당하지 않으므로 할당자도 필요 없다. */
proven_err_t err = proven_u8str_append(&path, PROVEN_LIT("/etc/hosts"));
EXAMPLE_REQUIRE(proven_is_ok(err), "10 bytes fit in a 16-byte string");

/* append_grow 는 늘어나는 쌍둥이다. 문자열을 만들 때 쓴 할당자를 주면 필요할 때
 * 재할당한다. 여전히 실패 원자적이다 - 할당이 실패하면 문자열은 그대로다. */
err = proven_u8str_append_grow(alloc, &path, PROVEN_LIT(".backup.original"));
EXAMPLE_REQUIRE(proven_is_ok(err), "append_grow must reallocate rather than fail");

/* 가운데를 고치기. insert 는 꼬리를 오른쪽으로, remove 는 왼쪽으로 민다. */
err = proven_u8str_insert_grow(alloc, &path, 0, PROVEN_LIT("/srv"));
EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a prefix must succeed");

err = proven_u8str_remove(&path, proven_u8str_as_view(&path).size - 9, 9);  /* ".original" 을 떼어 낸다 */
EXAMPLE_REQUIRE(proven_is_ok(err), "removing the trailing suffix must succeed");

/* 찾는 것이 없으면 replace_first 는 PROVEN_OK 를 돌려준다 - "할 일이 없다" 는
 * 오류가 아니다. 그 차이가 중요하면 먼저 찾아볼 것. */
err = proven_u8str_replace_first(&path, 0, PROVEN_LIT("hosts"), PROVEN_LIT("fstab"));
EXAMPLE_REQUIRE(proven_is_ok(err), "replacing an existing substring must succeed");

/* --- 읽기: 복사하지 말고 뷰를 빌릴 것 ---------------------------------- */
/* as_view 는 공짜다. 그 뷰는 다음 편집 전까지만 쓸 수 있다. 늘리는 호출은 재할당할
 * 수 있고, 그러면 뷰(그리고 cstr)는 매달린 포인터가 된다. */
proven_u8str_view_t v = proven_u8str_as_view(&path);

EXAMPLE_REQUIRE(proven_u8str_view_eq(v, PROVEN_LIT("/srv/etc/fstab.backup")),
                "the edits above should have produced /srv/etc/fstab.backup");
EXAMPLE_REQUIRE(proven_u8str_view_starts_with(v, PROVEN_LIT("/srv")),
                "the inserted prefix is at the front");

proven_size_t dot = proven_u8str_view_find(v, 0, PROVEN_LIT(".backup"));
EXAMPLE_REQUIRE(dot != PROVEN_INDEX_NOT_FOUND, "the suffix must be found");

/* 슬라이스는 *같은* 바이트를 가리키는 뷰다 - 할당도 복사도 없다. */
proven_u8str_view_t stem = proven_u8str_view_slice(v, 0, dot);
EXAMPLE_REQUIRE(proven_u8str_view_eq(stem, PROVEN_LIT("/srv/etc/fstab")),
                "slicing at the suffix leaves the stem");

/* as_cstr 은 C API 로 나가는 비상구이고, 소유한 문자열이 길이 뒤에 NUL 을 지키기
 * 때문에만 옳다. 뷰로는 이렇게 하지 *말 것*. `stem.ptr` 은 NUL 로 끝나지 않는다 -
 * 그냥 `path` 안을 가리킬 뿐이다. */
printf("owned:  %s\n", proven_u8str_as_cstr(&path));

/* --- *빌린* 문자열: 여러분의 기억, 할당은 전혀 없다 -------------------- */
/* 타입도 연산도 같다 - 다만 바이트가 이 스택 버퍼다. `cap` 은 NUL 을 포함하므로
 * 내용은 31 바이트까지 담긴다. */
proven_byte_t line[32];
proven_u8str_t status = proven_u8str_borrow(line, sizeof line);
```

```
    err = proven_u8str_append(&status, PROVEN_LIT("mounted "));
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending into a borrowed buffer needs no allocator");
    err = proven_u8str_append(&status, stem);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a view can be appended just like a literal");

    /* 빌린 문자열에도 늘리는 호출은 있지만, 자기 것이 아닌 기억을 재할당하지는
     * 않는다. 자료가 넘치면 OUT_OF_BOUNDS 이고 `line` 은 손대지 않은 채로 남는다.
     * 빌린 문자열이 등 뒤에서 조용히 힙으로 달아나는 일은 없다. */
    err = proven_u8str_append_grow(alloc, &status,
                                   PROVEN_LIT(" ...and a great deal more text than fits"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a borrowed string reports overflow instead of reallocating caller memory");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&status), PROVEN_LIT("mounted /srv/etc/fstab")),
                    "the failed append must have left the string unchanged");

    printf("borrowed: %s\n", proven_u8str_as_cstr(&status));

    /* reset 은 비어 있는 상태로 자르고 버퍼는 그대로 둔다. 그래서 다음 프레임이 같은
     * 32 바이트를 할당 없이 다시 쓴다. */
    err = proven_u8str_reset(&status);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reset must succeed on a borrowed string");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&status).size == 0, "reset empties the string");

    /* --- destroy: 소유 규칙을 그대로 적어 보면 ------------------------------ */
    /* 빌린 문자열에 대한 destroy 는 아무 일도 하지 않는다 - `line` 은 라이브러리가
     * 해제할 것이 아니다. 그래도 부르는 것이 옳고 값도 들지 않으며, 덕분에 뒷정리 코드는
     * 자기가 쥔 문자열이 어느 쪽인지 몰라도 된다. */
    proven_u8str_destroy(alloc, &status);

    /* 소유한 문자열에 대한 destroy 는 할당을 해제한다. 그리고 반드시 그 문자열을 만들
     * 때 쓴 할당자를 주어야 한다. */
    proven_u8str_destroy(alloc, &path);
    return EXAMPLE_OK();
}
```

실전 예제: 버퍼는 고정인데 데이터는 그렇지 않을 때

앞의 예제는 자리가 모자라면 문자열을 늘린다. 실제 코드의 상당 부분은 그럴 수 없다. 레코드에는 고정 폭 필드가 있고, 로그 한 줄에는 상한이 있으며, 아레나(arena) 위에 놓인 문자열은 재할당할 때마다 옛 사본을 다음 reset까지 남긴다. 그런 호출자는 데이터가 들어가지 않을 때 호출이 주는 세 가지 답 중 어느 것인지를 알아야 한다.

호출의 종류	데이터가 들어가지 않을 때 하는 일	해당 호출
원자적(atomic), 고정 용량	`PROVEN_ERR_OUT_OF_BOUNDS`로 거절하고 아무것도 바꾸지 않는다.	`proven_u8str_append`, `proven_u8str_insert`, `proven_u8str_replace_at`
최선 노력(best-effort, 잘라 쓰기)	들어가는 만큼 쓰고, `PROVEN_ERR_OUT_OF_BOUNDS`와 **함께** 실제로 쓴 바이트 수를 돌려준다.	`proven_u8str_append_partial`
원자적, 확장 가능	allocator를 통해 재할당한다. 할당이 실패하면 아무것도 바꾸지 않는다.	`proven_u8str_append_grow`, `proven_u8str_replace_at_grow`, `proven_u8str_append_byte`

보조 호출 둘도 여기 나온다. `proven_u8str_reserve()`는 용량을 미리 한 번에 올려 두어 이후의 증가가 재할당을 부르지 않게 한다. 힙(heap)에서는 복사를 아끼고, arena에서는 재할당이 남기는 죽은 저장 공간을 아낀다.

`proven_u8str_is_valid()`는 문자열 핸들 자체의 필드가 서로 모순되지 않는지 본다. 문자열이 다른 코드에서 넘어오는 경계에서 한 번 확인할 값어치가 있고, 편집할 때마다 되풀이할 검사는 아니다.

아래 프로그램은 상한이 있는 로그 한 줄을 쓰고, 넘치는 append를 거절당하고, 최선 노력 호출로 일부러 잘라 내고, `proven_u8str_replace_at()`으로 경로 가운데를 고친다 — 먼저 짧아지는 방향(언제나 들어간다), 다음에 길어지는 방향(들어가지 않아 거절된다), 그리고 같은 편집을 `proven_u8str_replace_at_grow()`로 다시 한다. 마지막은 `proven_u8str_view_ends_with()`로 파일 확장자를 확인하는 것이다.

```
/*
 * 앞의 예제는 자리가 모자라면 문자열을 늘렸다. 이번 것은 늘리는 것이 허용되지 않는
 * 경우 - 크기가 고정된 레코드, 길이 상한이 단단한 로그 줄, 재할당하면 안 되는 아레나
 * 속 버퍼 - 그리고 자료가 들어가지 않을 때 호출이 줄 수 있는 두 가지 정직한 답에 대한
 * 것이다.
 *
 *   "안 됩니다, 그리고 아무것도 바꾸지 않았습니다" - 원자적 호출들: append, insert,
 *                                   replace_at. 먼저 용량을 검사하므로, 거부는
 *                                   문자열을 그대로 남긴다.
 *   "일부만 했고, 얼마나 했는지 알려 드립니다" - 최선 노력 호출: append_partial.
 *                                   담을 수 있는 만큼 담고 몇 바이트를 썼는지
 *                                   알려 준다.
 *
 * 둘 다 쓸모가 있다. 잘못 고르면 레코드를 조용히 자르거나 조용히 버린다. `_grow` 짝은
 * 세 번째 답 - "네, 자리를 더 찾았습니다" - 이고, 견주어 보라고 끝에 실었다.
 */

#define FIELD_CAP 32u

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t r = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating the field buffer must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u8str_t field = r.value;

    /* is_valid 는 손잡이 자신의 구조를 검사한다 - 포인터와, 서로 어긋나지 않는 용량과
     * 길이. 문자열이 다른 데서 넘어올 때 여러분 코드의 경계에서 한 번 단언해 둘 값어치가
     * 있다. 편집할 때마다 할 검사는 아니다. 모든 편집이 그 성질을 지키기 때문이다. */
    EXAMPLE_REQUIRE(proven_u8str_is_valid(&field), "a freshly created string must be structurally valid");

    /* --- 미리 자리를 잡아 두기 ------------------------------------------- */

    /* reserve 는 용량을 지금 올려 두어 나중의 성장이 재할당하지 않게 한다. 힙에서는
     * 복사를 아끼고, 아레나에서는 더 나쁜 것을 아낀다. 거기서는 재할당마다 옛 블록이
     * 다음 reset 까지 새기 때문이다. 필요할 만큼을, 한 번에 청할 것. */
    proven_err_t err = proven_u8str_reserve(alloc, &field, 64);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reserving 64 bytes must succeed");
    EXAMPLE_REQUIRE(field.internal.cap >= 64, "the capacity must actually be at least what was asked for");

    /* --- 원자적 호출: 들어가거나, 아무것도 바꾸지 않거나 ------------------- */

    err = proven_u8str_append(&field, PROVEN_LIT("2026-01-01 level=info "));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the prefix fits in the reserved capacity");

    /* append_byte 는 한 바이트를 더한다. 구분자, 종결자, 이스케이프 문자가 그런
     * 것들이다. 늘리는 호출이라 할당자를 받는다 - 한 바이트야말로 생각지 못한 경계에서
     * 용량 검사가 걸리는 바로 그 경우다. */
    err = proven_u8str_append_byte(alloc, &field, (proven_u8)'[');
```

```
    EXAMPLE_REQUIRE(proven_is_ok(err), "appending a single separator byte must succeed");

    err = proven_u8str_append(&field, PROVEN_LIT("disk full"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the message fits");

    err = proven_u8str_append_byte(alloc, &field, (proven_u8)']');
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing the bracket must succeed");

    /* 이제 용량이 담을 수 있는 것보다 많이 청해 본다. 원자적 append 는 거부하고 -
     * 이것이 믿고 기댈 값어치가 있는 성질이다 - 문자열은 호출 전에 담고 있던 것을
     * 그대로 담고 있다. */
    proven_size_t before = proven_u8str_as_view(&field).size;
    err = proven_u8str_append(&field, PROVEN_LIT(" and a very long trailing explanation that certainly does
not fit"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized atomic append must be refused");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before, "and must leave the string untouched");

    /* --- 최선 노력 호출: 들어가는 만큼, 그리고 그 개수 -------------------- */

    /* 보고서의 폭 고정 열이 이것을 쓰는 경우다. 들어가는 만큼 쓰고, 얼마나 썼는지 알아
     * 두어 부르는 쪽이 그 값을 온전한 척하는 대신 잘렸다고 표시할 수 있게 한다. */
    proven_result_size_t part = proven_u8str_append_partial(&field, PROVEN_LIT(" ...more text than there is
room for"));
    EXAMPLE_REQUIRE(part.err == PROVEN_ERR_OUT_OF_BOUNDS, "a partial append that truncates still reports the
truncation");
    EXAMPLE_REQUIRE(part.value > 0, "but it wrote what it could");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&field).size == before + part.value,
                    "and the string grew by exactly the number of bytes it reports");
    printf("partial append wrote %zu byte(s) before the buffer was full\n", (size_t)part.value);

    /* --- 늘리지 않고 가운데를 고치기 ---------------------------------------- */

    proven_result_u8str_t r2 = proven_u8str_create(alloc, FIELD_CAP);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the second buffer must succeed");
    proven_u8str_t path = r2.value;
    err = proven_u8str_append(&path, PROVEN_LIT("var/log/service.log"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the path fits");

    /* insert 는 꼬리를 오른쪽으로 민다. 용량 고정이다 - 들어가거나 거부한다. */
    err = proven_u8str_insert(&path, 0, PROVEN_LIT("/"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a leading slash must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/service.log")),
                    "the insert lands at index 0");

    /* replace_at 은 어떤 자리의 old_len 바이트를 길이에 상관없는 자료로 바꾼다.
     * 결과가 여전히 들어가기만 하면 된다. "service"(7)를 "daemon"(6)으로 바꾸면 문자열이
     * 짧아지므로 용량 때문에 실패할 수 없다. */
    proven_size_t at = proven_u8str_view_find(proven_u8str_as_view(&path), 0, PROVEN_LIT("service"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the substring must be found before it can be replaced");
    err = proven_u8str_replace_at(&path, at, 7, PROVEN_LIT("daemon"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "a shortening replacement must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&path), PROVEN_LIT("/var/log/daemon.log")),
                    "and produce the expected path");

    /* 같은 편집을 거꾸로 하면 32바이트 버퍼를 넘치고, 용량 고정 호출은 경로를 자르는
     * 대신 거부한다 - 자르는 쪽이 바로 엉뚱한 파일에 조용히 쓰게 되는 그 실패다. */
    before = proven_u8str_as_view(&path).size;
    err = proven_u8str_replace_at(&path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-this-
buffer"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "a replacement that does not fit must be refused");
```

```
    EXAMPLE_REQUIRE(proven_u8str_as_view(&path).size == before, "and must leave the path unchanged");

    /* replace_at_grow 는 재할당을 허락받은 같은 편집이다. 버퍼가 힙에 있고 길이가 정말
     * 한정되지 않았을 때 쓸 것. 한계가 형식의 일부라면 용량 고정 호출을 고를 것. */
    err = proven_u8str_replace_at_grow(alloc, &path, at, 6, PROVEN_LIT("a-replacement-name-far-too-long-for-
this-buffer"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the growing variant makes room instead of refusing");
    EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".log")),
                    "the extension is still at the end after the edit");

    /* ends_with 는 확장자 검사가 실제로 던지는 물음에 답한다. 손으로 셈한 인덱스로 하면
     * off-by-one 이 사는 자리가 되고, 포인터에 strcmp 로 하면 뷰에는 없는 NUL 이
     * 필요해진다. */
    EXAMPLE_REQUIRE(!proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT(".txt")),
                    "and it is not a .txt file");

    /* 빈 접미사는 무엇의 접미사이기도 하다. 접미사 목록을 도는 반복문이 특별한 경우를
     * 두지 않아도 되게 해 주는 답이다. */
    EXAMPLE_REQUIRE(proven_u8str_view_ends_with(proven_u8str_as_view(&path), PROVEN_LIT("")),
                    "every string ends with the empty suffix");

    printf("log line: %s\n", proven_u8str_as_cstr(&field));
    printf("path:     %s\n", proven_u8str_as_cstr(&path));

    proven_u8str_destroy(alloc, &path);
    proven_u8str_destroy(alloc, &field);
    return EXAMPLE_OK();
}
```

반례 — 최선 노력 호출을 원자적 호출처럼 다루는 경우:

```
(void)proven_u8str_append_partial(&line, field);   /* wrong: ignores the count */
```

잘렸다는 사실이 보고되는 곳은 반환값뿐이다. 그것을 버리면 "레코드가 잘렸다"가 "레코드가 멀쩡해 보인다"로 바뀐다.

반례 — 바이트 수를 글자 수로 읽는 경우:

```
if (part.value == field.size) { /* all of it was written */ }
```

이 비교는 옳고, **바이트** 단위로 옳다. UTF-8 한 글자는 최대 4바이트이므로 최선 노력 append는 글자 중간에서 멈출 수 있다. 뒤끝이 유효한 UTF-8이어야 한다면 자를 지점을 용량이 아니라 여러분이 정한다.

실전 예제: 시스템 호출에 넘길 UTF-16 문자열 조립하기

`proven_u16str_t`가 제 몫을 하는 곳은 운영체제 호출이 UTF-16을 요구하는 경계뿐이다. 보통은 윈도의 와이드(wide) API가 그 이유다. 방식은 언제나 같다. 코드 단위(code unit)를 모은 다음 `proven_u16str_as_ptr()`를 그 호출에 넘긴다.

이 코드가 옳은지는 세 가지가 정한다.

- **단위는 코드 단위이지 바이트도 글자도 아니다.** 용량 32는 코드 단위 32개, 곧 64바이트다. 기본 다국어 평면(Basic Multilingual Plane, BMP) 밖의 글자 — 이모지, 드문 한중일 한자 다수 — 는 두 개를 차지한다.
- **`proven_u16str_as_ptr()`는 복사하지 않는다.** 그 포인터는 문자열을 늘리는 다음 append 전까지만 유효하다.
- **결과는 NUL로 끝난다.** 일부러 잘라 낸 뒤에도 그렇기 때문에, 종결자를 기대하는 시스템 호출에 그대로 넘겨도 안전하다.

```
/*
 * UTF-16 이 이 라이브러리에 있는 이유는 하나다. 어떤 운영체제 호출은 그것만 받는다.
 * 윈도의 "와이드" API 가 늘 그 경우다 - CreateFileW 에 건네는 파일 이름은 바이트가
 * 아니라 NUL 로 끝나는 16비트 코드 단위의 줄이다.
 *
```

```
 * 그래서 이 타입이 하는 일은 좁다. 코드 단위를 모으고, 개수를 맞게 지키고, 시스템
 * 호출이 원하는 포인터를 내주는 것. 프로그램의 나머지는 UTF-8 로 두어야 한다.
 *
 * 하나만 헷갈리지 말 것 - 단위다. 여기서 용량 32 는 코드 단위 32개, 곧 64 바이트이고,
 * 기본 다국어 평면 밖의 문자는 - 이모지, 드문 한자 대부분 - 두 개를 쓴다. 코드 단위의
 * 개수는 문자의 개수가 아니고, 한 번도 그랬던 적이 없다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 인자는 바이트 한계가 아니라 코드 단위 한계다. */
    proven_result_u16str_t r = proven_u16str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 32-code-unit string must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_u16str_t name = r.value;
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 0, "a new string is empty");

    /* PROVEN_U16_LIT 은 u"..." 리터럴에서 뷰를 만들고 단위 개수를 리터럴 자체에서
     * 계산한다. 그래서 개수가 글과 어긋날 수 없다. */
    proven_err_t err = proven_u16str_append(&name, PROVEN_U16_LIT("C:\\logs\\"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the directory prefix fits in 32 code units");

    err = proven_u16str_append(&name, PROVEN_U16_LIT("service.log"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the file name fits too");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == 8 + 11, "the length is a count of code units");

    /* 바이트 문자열 쌍둥이처럼 원자적이다. 자료가 넘치면 거부되고 문자열은 그대로
     * 남는다. 그래서 경로가 반쯤 적히는 일이 없다. */
    proven_size_t before = proven_u16str_len(&name);
    err = proven_u16str_append(&name, PROVEN_U16_LIT(".a-suffix-long-enough-to-overflow-the-capacity"));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_OUT_OF_BOUNDS, "an oversized append must be refused");
    EXAMPLE_REQUIRE(proven_u16str_len(&name) == before, "and must not truncate the path");

    /* --- 시스템 호출이 원하는 포인터 ------------------------------------- */

    /* as_ptr 은 내부의 코드 단위를 NUL 로 끝난 채로, 복사 없이 돌려준다. 호출 직전의
     * 마지막 걸음이고, 그 포인터는 다음 append 전까지만 쓸 수 있다. 늘리는 append 는
     * 저장소를 옮길 수 있다. */
    const proven_u16 *wide = proven_u16str_as_ptr(&name);
    EXAMPLE_REQUIRE(wide != NULL, "an assembled string must yield a pointer");
    EXAMPLE_REQUIRE(wide[0] == (proven_u16)'C', "the first code unit is the drive letter");
    EXAMPLE_REQUIRE(wide[proven_u16str_len(&name)] == 0, "the sequence is NUL-terminated for the system
call");
    /* 윈도에서는 이것이 이 타입의 존재 이유 전부다.
     *     HANDLE h = CreateFileW((LPCWSTR)wide, ...);
     * 여기서는 부르지 않는다. 이 예제는 다른 모든 곳에서도 돌아야 하기 때문이다. */

    /* --- 자르는 것이 옳은 답일 때 ---------------------------------------- */

    /* 어떤 시스템 구조체에는 폭이 고정된 필드가 있다 - 이를테면 16 단위짜리 이름표 -
     * 거기서는 들어가지 않는 이름을 거부하는 것이 아니라 자르는 것이 뜻이다. 부분
     * append 가 그것을 위한 것이다. 담을 수 있는 만큼 담고 몇 단위를 썼는지 알려 주어,
     * 부르는 쪽이 그 값을 잘렸다고 표시할 수 있게 한다. */
    proven_result_u16str_t r2 = proven_u16str_create(alloc, 16);
    EXAMPLE_REQUIRE(proven_is_ok(r2.err), "creating the fixed-width label must succeed");
    proven_u16str_t label = r2.value;
```

```
    proven_result_size_t wrote = proven_u16str_append_partial(&label, PROVEN_U16_LIT("a-label-that-is-longer-
than-the-field"));
    EXAMPLE_REQUIRE(wrote.err == PROVEN_ERR_OUT_OF_BOUNDS, "the truncation is reported, not hidden");
    EXAMPLE_REQUIRE(wrote.value == 16, "it filled the field exactly");
    EXAMPLE_REQUIRE(proven_u16str_len(&label) == wrote.value, "and the length matches what it says it wrote");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&label)[wrote.value] == 0,
                    "a truncated string is still NUL-terminated, so it is still safe to pass on");

    printf("assembled %zu code unit(s); label truncated to %zu\n",
           (size_t)proven_u16str_len(&name), (size_t)wrote.value);

    proven_u16str_destroy(alloc, &label);
    proven_u16str_destroy(alloc, &name);
    return EXAMPLE_OK();
}
```

반례 — UTF-16 버퍼 크기를 바이트로 잡는 경우:

```
proven_result_u16str_t r = proven_u16str_create(alloc, sizeof(buf));   /* wrong */
```

인자는 코드 단위의 개수다. 바이트 수를 넘기면 의도한 것의 두 배를 요구하게 되고, 그 바이트 수가 UTF-8 문자열에서 왔다면 변환 결과를 담지도 못하는 버퍼를 요구하게 된다.

반례 — append를 사이에 두고 포인터를 붙들고 있는 경우:

```
const proven_u16 *w = proven_u16str_as_ptr(&name);
proven_err_t e = proven_u16str_append_grow(alloc, &name, more);   /* may reallocate */
use_wide(w);   /* wrong: w may dangle */
```

포인터는 그것을 소비하는 호출 바로 앞에서 얻는다.

4장: 컨테이너와 알고리즘

3부 — 자료구조. 선행 조건: 2부 (1, 2, 3). 이 장을 마치면 작업에 맞는 컨테이너를 고르고, 보장된 한계 안에서 정렬하고 검색하며, 올바른 이유로 해싱하고, 바이트를 텍스트로 또 그 반대로 바꿀 수 있다.

이 장은 `array.h`, `list.h`, `ring.h`, `map.h`, `algorithm.h`, `hash.h`, `encode.h`를 다룬다. 여기 있는 모든 컨테이너는 할당자(allocator)를 받는다. 2장이 먼저 오는 이유가 그것이다.

목차

1. 동적 배열
2. 침습적(intrusive) 리스트
3. 링 버퍼
4. 해시 맵
5. 알고리즘
6. 용도별 해싱
7. 바이트를 텍스트로: hex와 Base64
8. 예제와 오용 사례

1. 동적 배열

문제: 매번 손으로 다시 쓰는 그 배열

확장 가능한 배열은 모든 C 프로그램이 결국 한 번은 작성하는 자료구조이며, 매번 조금씩 다르게 작성된다:

```
int *items = malloc(cap * sizeof(int));
/* ... later ... */
if (n == cap) {
    cap *= 2;
    items = realloc(items, cap * sizeof(int));   /* wrong on two counts */
    items[n++] = x;
}
```

한 줄에 버그가 둘 있고, 둘 다 고전이다. `cap * sizeof(int)`는 큰 `cap`에 대해 **랩어라운드**할 수 있고, 그러면 거대한 개수에 대해 작은 할당이 만들어진다 — 1장 §4를 보라. 그리고 `realloc`의 결과를 곧바로 `items`에 대입하면 **NULL을 반환할 때 옛 블록이 누수된다.** 원래 포인터가 사라졌는데 그것이 가리키던 메모리는 여전히 할당된 채로 남기 때문이다.

버그를 빼고 보더라도, 손으로 쓴 버전은 원소 타입마다 다시 써야 하거나, `void *`로 제네릭하게 만들어 타입 검사를 잃어야 한다.

이 라이브러리는 대신 무엇을 하는가

`proven_array_t`는 생성될 때의 원소 크기와 정렬을 유지하는 확장 벡터다. 그래서 템플릿 없이도, 호출 지점에서 `void *`를 쓰지 않고도 어떤 타입에나 동작한다 — `PROVEN_ARRAY_*` 매크로가 타입을 받아, 컴파일러가 아직 검사할 수 있는 자리에서 캐스팅을 수행한다.

- 확장은 검사된 산술을 사용하므로, 위의 오버플로는 작은 할당이 아니라 `PROVEN_ERR_OVERFLOW`가 된다.
- 확장은 **실패-원자적(failure-atomic)**이다: 확장할 수 없으면 기존 원소는 손대지 않은 채로 여전히 유효하다. 아무것도 누수되지 않고 아무것도 잃지 않는다.
- 문자열 타입과 달리 **allocator를 내부에 저장한다** — `PROVEN_ARRAY_DESTROY`가 배열만 받는 이유가 그것이다.

`proven_array_t`는 연속된 원소 저장소를 소유하므로, `proven_array_sort`와 §5의 검색들에 넘기는 것도 바로 이것이다.

잘못된 예 — push를 건너 포인터를 붙잡고 있기:

```
int *first = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
(void)PROVEN_ARRAY_PUSH(&arr, int, 42);   /* may reallocate */
*first = 7;                               /* wrong: `first` may point at freed memory */
```

이것은 타입이 제거해 줄 수 없는 유일한 위험이다. 확장은 저장소를 이동시키므로, **배열을 가리키는 모든 포인터나 뷰(view)는 확장을 유발할 수 있는 모든 연산에 의해 무효화된다.** 가리키는 대신 인덱스를 쓰거나, push 이후에 포인터를 다시 가져오라.

구조체

```
typedef struct {
    proven_allocator_t alloc;
    proven_byte_t *data;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_array_t;

typedef struct {
    proven_err_t err;
    proven_array_t value;
} proven_result_array_t;
```

필드:

- `alloc`: 성장과 파괴에 쓰이는 allocator.
- `data`: 원소를 위한 바이트 저장소.
- `len`: 현재 원소 개수.
- `cap`: 원소 단위 용량.
- `elem_size`: 각 원소의 크기.
- `align`: 각 원소의 정렬.

함수

API	의도	반환
`proven_array_create(alloc, init_cap, elem_size, align)`	제네릭 배열을 만든다.	`proven_result_array_t`.
`proven_array_is_valid(arr)`	공개 배열 불변식을 검증한다.	`bool`.
`proven_array_reserve(arr, new_cap)`	최소 `new_cap`개의 원소를 보장한다.	`proven_err_t`.
`proven_array_push(arr, element)`	포인터로부터 복사하여 원소 하나를 추가한다.	`proven_err_t`.
`proven_array_pop(arr, out_element)`	마지막 원소를 꺼낸다; `out_element`가 null이면 버린다.	`proven_err_t`.
`proven_array_get_mut(arr, index)`	가변 원소 포인터를 얻는다.	포인터 또는 null.
`proven_array_get(arr, index)`	const 원소 포인터를 얻는다.	포인터 또는 null.
`proven_array_destroy(arr)`	저장소를 해제하고 상태를 비운다.	void.
`proven_array_clear(arr)`	모든 원소를 지운다. 용량은 남는다.	void.
`proven_array_truncate(arr, new_len)`	`new_len`으로 줄인다.	`proven_err_t`. 더 길면 `OUT_OF_BOUNDS`.
`proven_array_insert(arr, index, element)`	`index`(0..len)에 넣고 나머지를 뒤로 민다. `element`는 배열 안을 가리켜도 된다.	`proven_err_t`.

proven_array_remove_at(arr, index, out_element)	index의 원소를 순서를 지키며 뺀다.	proven_err_t.
proven_array_swap_remove(arr, index, out_element)	마지막 원소를 그 자리로 옮겨 O(1)에 뺀다. 순서는 지키지 않는다.	proven_err_t.
proven_array_extend(arr, elements, count)	원소 count개를 덧붙인다. 재할당은 많아야 한 번. 배열 자신의 원소여도 된다.	proven_err_t.

편집 연산은 실패하면 아무것도 바꾸지 않고, 끝을 넘는 index는 PROVEN_ERR_OUT_OF_BOUNDS로 거부하며, 배열을 키울 수 있는 두 연산은 push처럼 배열 안을 가리키던 포인터를 무효로 만든다.

```
/* A to-do list: insert at the front, take one out from the middle keeping order, drop
 * finished items in O(1) when order does not matter, and append a batch at once. */
proven_result_array_t todo = PROVEN_ARRAY_INIT(alloc, int, 4);
if (proven_is_ok(todo.err)) {
    const int batch[] = { 10, 20, 30, 40 };
    int first = 5, taken = 0;
    proven_err_t e = proven_array_extend(&todo.value, batch, 4);               /* 10 20 30 40 */
    if (proven_is_ok(e)) e = proven_array_insert(&todo.value, 0, &first);      /* 5 10 20 30 40 */
    if (proven_is_ok(e)) e = proven_array_remove_at(&todo.value, 2, &taken);   /* 5 10 30 40, took 20 */
    if (proven_is_ok(e)) e = proven_array_swap_remove(&todo.value, 0, NULL);   /* 40 10 30 */
    if (proven_is_ok(e)) e = proven_array_truncate(&todo.value, 2);            /* 40 10 */
    proven_println("{} items, took {}", PROVEN_ARG(todo.value.len), PROVEN_ARG(taken));
    proven_array_clear(&todo.value);                                           /* empty, capacity kept */
    proven_array_destroy(&todo.value);
}
```

매크로

매크로	의도
PROVEN_ARRAY_INIT(alloc, type, init_cap)	sizeof(type)와 alignof(type)를 사용하는 타입 안전 생성.
PROVEN_ARRAY_PUSH(arr_ptr, type, value)	임시 복합 리터럴로 rvalue나 lvalue를 push한다.
PROVEN_ARRAY_POP(arr_ptr, type, out_ptr)	출력 포인터로 pop하거나 null로 버린다.
PROVEN_ARRAY_GET(arr_ptr, type, index)	타입이 지정된 const 원소 포인터.
PROVEN_ARRAY_GET_MUT(arr_ptr, type, index)	타입이 지정된 가변 원소 포인터.
PROVEN_ARRAY_DESTROY(arr_ptr)	배열을 파괴한다.

예제:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;

(void)PROVEN_ARRAY_PUSH(&nums, int, 10);
(void)PROVEN_ARRAY_PUSH(&nums, int, 20);

/* GET points into the array's own storage, and returns NULL out of range. */
const int *first = PROVEN_ARRAY_GET(&nums, int, 0);
if (first) {
    proven_println("first={}", PROVEN_ARG(*first));
}
```

```
/* The array stores its allocator, so destroy needs nothing but the array. */
PROVEN_ARRAY_DESTROY(&nums);
```

2. 침습적(intrusive) 리스트

문제: 배운 대로의 리스트는 원소마다 노드를 할당한다

```
struct node { struct node *next; void *data; };
```

이것이 교과서의 연결 리스트이며, 교과서가 언급하지 않는 두 가지 비용이 있다. 삽입할 때마다 **할당한다** — 천 개의 항목은 곧 포인터를 담기 위해서만 존재하는 천 번의 할당이고, 각각은 실패할 수 있으며 각각은 짝이 되는 해제를 필요로 한다. 그리고 `data`는 `void *`이므로 리스트는 자신이 무엇을 담고 있는지 전혀 모른다: 다시 읽어낼 때마다 컴파일러가 검사할 수 없는 캐스트가 된다.

침습적 리스트는 대신 무엇을 하는가

관계를 뒤집는다: **메모리는 당신이 소유하고, 링크가 그 안에 산다.**

```
typedef struct {
    int                id;
    proven_list_node_t  link;   /* the list's hook, inside your struct */
} ex_task_t;
```

이제 삽입은 포인터 두 개를 쓰는 일이다. 아무것도 할당하지 않으므로 **실패할 수 없다** — `proven_list_push_back`이 `void`를 반환한다는 점에 주목하라. 이 라이브러리에서는 이례적인 일이고, 바로 그것이 요점이다. 제거도 마찬가지다. 객체는 스택에도, 배열 안에도, 아레나 안에도, 어디에나 살 수 있다. 리스트는 이미 그 안에 들어 있는 포인터들을 재배치할 뿐이다.

링크에서 다시 당신의 객체로 돌아가는 것은 `PROVEN_LIST_ENTRY(node, ex_task_t, link)`이며, 노드의 주소에서 멤버의 오프셋을 뺀다. 이것은 맞기를 바라는 캐스트가 아니라, 컴파일러가 타입을 검사하는 산술이다.

포기하는 것: **객체는 자신이 가진 링크 멤버 수만큼의 리스트에만 속할 수 있고**, 그 수는 구조체를 선언할 때 당신이 정한다. 어떤 항목이 큐와 인덱스에 동시에 있어야 한다면, 링크가 둘 필요하다.

`proven_list_t`는 침습적 이중 연결 **원형** 리스트다 — 헤드가 sentinel 노드이므로, 삽입과 제거가 양 끝을 특별히 취급하는 일이 없다. 노드를 할당하지 않는다.

잘못된 예 — 평범한 반복자로 순회하면서 제거하기:

```
PROVEN_LIST_FOR_EACH(it, &queue) {
    if (should_drop(it)) proven_list_remove(it);   /* wrong: it->next is read after unlink */
}
```

`proven_list_remove`는 노드 자신의 포인터를 통해 쓰기를 하므로, 루프는 방금 떼어낸 노드에서 `next`를 읽게 된다. `PROVEN_LIST_FOR_EACH_SAFE`는 두 번째 변수를 받아 본문이 실행되기 전에 다음 포인터를 읽는다. 그것이 바로 이 매크로가 존재하는 이유다.

완성 예제: 링크가 호출자의 구조체 안에 사는 큐

테스트 스위트가 컴파일하고 실행한다. 스택에 할당된 task들의 큐를 만들고, 순회하고, 안전한 순회 안에서 하나를 제거하고, 중간에 삽입한다 — 프로그램 어디에도 allocator가 없다.

```
/*
 * *침입형*(intrusive) 리스트는 여러분의 자료를 담을 노드를 따로 할당하는 대신, 연결
 * 고리를 여러분의 구조체 *안에* 둔다.
 *
 * 배울 때 본 리스트는 이렇게 생겼다.
 *
 *     struct node { struct node *next; void *data; };
 *
 * 넣을 때마다 노드를 할당하므로 항목 천 개짜리 리스트는 포인터를 담자고 존재하는 할당
 * 천 번이 되고, 하나하나가 실패할 기회이자 해제할 거리다. 더 나쁜 것은 `data` 가
 * void* 라는 점이다 - 리스트는 자기가 무엇을 담았는지 모르므로 읽을 때마다 컴파일러가
 * 검사할 수 없는 형변환을 한다.
 *
```

```
 * 침입형 리스트는 그것을 뒤집는다. 기억은 *여러분*이 소유하고 고리는 그 안에 산다.
 * 넣는 데 할당이 없으니 실패할 수 없다. 빼는 데도 할당이 없으니 실패할 수 없다. 그리고
 * 고리가 알려진 타입의 멤버이므로, 고리에서 물건으로 돌아오는 것은 형변환이 아니라
 * 컴파일러가 대신 해 주는 산술이다.
 *
 * 대가는 물건 하나가 자기가 가진 고리 멤버 수만큼의 리스트에만 들어갈 수 있다는 것이고,
 * 그것은 구조체를 선언할 때 여러분이 내리는 결정이다.
 */

/* 고리가 멤버다. 이 작업은 한 번에 정확히 한 리스트에만 들어갈 수 있다. */
typedef struct {
    int                id;
    proven_list_node_t  link;
} ex_task_t;

int main(void) {
    /* 이 프로그램에는 할당자가 아예 없다. 작업들은 스택에 있고, 리스트는 그 안에 사는
     * 포인터들을 다시 이어 줄 뿐이다. */
    ex_task_t a = { .id = 1 };
    ex_task_t b = { .id = 2 };
    ex_task_t c = { .id = 3 };

    proven_list_t queue;
    proven_list_init(&queue);
    EXAMPLE_REQUIRE(proven_list_is_empty(&queue), "a freshly initialised list is empty");

    /* --- 넣기는 실패할 수 없다. 할당하는 것이 없으므로 ----------------- */
    proven_list_push_back(&queue, &a.link);
    proven_list_push_back(&queue, &b.link);
    proven_list_push_back(&queue, &c.link);
    EXAMPLE_REQUIRE(!proven_list_is_empty(&queue), "three tasks are queued");

    /* --- 훑기: PROVEN_LIST_ENTRY 가 고리에서 물건을 되찾아 준다 -------- */
    proven_list_node_t *it = NULL;
    int seen[3] = {0}, n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 2 && seen[2] == 3,
                    "the walk visits every task, in insertion order");

    /* --- 훑으면서 빼려면 *안전한* 꼴이 필요하다 ------------------------ */
    /* proven_list_remove 는 노드 자신의 next/prev 포인터를 통해 쓴다. 그래서 그냥
     * FOR_EACH 를 쓰면 방금 떼어 낸 노드에서 `it->next` 를 읽게 된다. _SAFE 꼴은 본문이
     * 돌기 *전에* 다음 포인터를 읽어 둔다. */
    proven_list_node_t *safe = NULL;
    PROVEN_LIST_FOR_EACH_SAFE(it, safe, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (t->id == 2) proven_list_remove(it);
    }

    n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 2 && seen[0] == 1 && seen[1] == 3, "task 2 was unlinked");

    /* --- 가운데에 끼워 넣기는 포인터 바꿔 끼우기다 --------------------- */
```

```
    ex_task_t d = { .id = 4 };
    proven_list_insert_after(&a.link, &d.link);

    n = 0;
    PROVEN_LIST_FOR_EACH(it, &queue) {
        ex_task_t *t = PROVEN_LIST_ENTRY(it, ex_task_t, link);
        if (n < 3) seen[n++] = t->id;
    }
    EXAMPLE_REQUIRE(n == 3 && seen[0] == 1 && seen[1] == 4 && seen[2] == 3,
                    "task 4 sits directly after task 1");

    /* 지울 것이 없다. 리스트는 아무것도 소유한 적이 없다. `a`, `c`, `d` 는 여전히
     * 멀쩡한 지역 변수이고, 그 수명은 리스트가 없었을 때와 똑같이 이 함수의 것이다. */
    return EXAMPLE_OK();
}
```

구조체

```
typedef struct proven_list_node_t {
    struct proven_list_node_t *next;
    struct proven_list_node_t *prev;
} proven_list_node_t;

typedef struct {
    proven_list_node_t head;
} proven_list_t;
```

함수와 매크로

API	의도	반환
`proven_list_init(list)`	원형 센티넬을 초기화한다.	void.
`proven_list_insert_after(target, node)`	`target` 뒤에 `node`를 삽입한다.	void.
`proven_list_push_back(list, node)`	꼬리에 삽입한다.	void.
`proven_list_remove(node)`	노드를 떼어내고 링크를 null로 오염(poison)시킨다.	void.
`proven_list_is_empty(list)`	빈 리스트 또는 null 리스트인지 검사한다.	int 진리값.
`PROVEN_CONTAINER_OF(ptr, type, member)`	멤버 포인터를 부모 객체 포인터로 변환한다.	포인터.
`PROVEN_LIST_FOR_EACH(iter, list)`	노드를 순회한다.	루프 매크로.
`PROVEN_LIST_FOR_EACH_SAFE(iter, safe_next, list)`	현재 노드를 제거하면서 안전하게 순회한다.	루프 매크로.
`PROVEN_LIST_ENTRY(ptr, type, member)`	리스트 노드를 담고 있는 객체로 변환한다.	포인터.

예제:

```
typedef struct Item {
    int value;
    proven_list_node_t link;   /* the list lives inside the object; it allocates nothing */
} Item;

proven_list_t list;
proven_list_init(&list);

Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

int total = 0;
```

```
proven_list_node_t *it = NULL;
PROVEN_LIST_FOR_EACH(it, &list) {
    /* The iterator walks nodes; ENTRY converts a node back to its owner. */
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    total += item->value;
}
proven_println("total={}", PROVEN_ARG(total));   /* 3 */
```

3. 링 버퍼

문제: 커져서는 안 되는 큐

빠른 생산자와 느린 소비자 사이의 큐는 한 가지 질문에 답해야 한다: 소비자가 뒤처지면 무슨 일이 일어나는가? 확장 가능한 큐는 "더 할당한다"고 답하는데, 그러면 일시적인 지연이 무한한 메모리 증가로, 그리고 결국에는 작업을 버리는 것보다 더 나쁜 무언가로 바뀐다.

손으로 쓰는 대안은 배열 하나에 head 인덱스, tail 인덱스, 그리고 모듈로 산술을 더한 것이다 — 그리고 여기에는 유명한 버그가 하나 있다. `head == tail`일 때 버퍼는 비어 있는가, 가득 찼는가? 별도의 개수를 유지하거나 슬롯 하나를 일부러 낭비하지 않는 한 두 상태는 똑같아 보이고, 프로그래머 세대마다 이것을 다시 발견한다.

이 라이브러리는 대신 무엇을 하는가

`proven_ring_t`는 개수를 유지하는 고정 용량 FIFO다. 그래서 비어 있음과 가득 참이 구별되며, 커지는 대신 **거부한다**:

- 가득 찬 링에 대한 `proven_ring_push`는 `PROVEN_ERR_OUT_OF_BOUNDS`를 반환한다. 가장 오래된 항목을 덮어쓰지 않고, 재할당도 하지 않는다. 호출자가 결정한다 — 기다리거나, 새 항목을 버리거나, 배압(backpressure)을 보고하거나 — 무엇이 옳은지는 호출자만 알기 때문이다.
- 비어 있는 링에 대한 `proven_ring_pop`은 낡은 슬롯을 되돌려주는 대신 실패한다.

용량은 생성 시점에 한 번 정해지며, 그 숫자가 곧 정책이다: 생산자가 얼마나 앞서 나가도 되는지를 말한다. 이벤트 큐, 최근 항목의 로그, 오디오나 센서 버퍼 — 한계가 제약이 아니라 설계의 일부인 곳이라면 링을 꺼내라.

이 라이브러리의 대부분의 타입과 달리, `proven_ring_t`는 **allocator를 내부에 저장한다.** `PROVEN_RING_DESTROY`가 링만 받는 이유가 그것이다.

잘못된 예 — 가득 찬 링을 즉시 재시도할 오류로 취급하기:

```
while (PROVEN_RING_PUSH(&ring, ex_event_t, e) != PROVEN_OK) { }   /* wrong: spins forever */
```

그 루프 안의 무엇도 소비하지 않으므로 링은 계속 가득 차 있다. 한계가 있는 큐의 거부는 *소비자*에 대한 정보이며, 그것을 무시하는 루프는 곧 멈춤(hang)이다.

완성 예제: 가득 찰 때까지 push하기, 그리고 거부가 어떤 모습인지

테스트 스위트가 컴파일하고 실행한다.

```
/*
 * 고리 버퍼는 내용을 옮기지도, 늘어나지도 않는 고정 크기 큐다. 용량을 한 번 정해 주면
 * push 는 꼬리에 넣고 pop 은 머리에서 빼며, 가득 차면 push 는 *거부한다*.
 *
 * 이것이 없으면 여러분이 쓰게 될 C 는 배열 하나에 인덱스 둘, 그리고 그것을 감아 주는
 * 나머지 연산이고, 버그는 언제나 같다 - "지금 가득 찬 것인가 빈 것인가?" 개수를 따로
 * 세거나 칸 하나를 버리지 않는 한 두 상태 모두 head == tail 이다. 이 고리는 개수를
 * 세므로 그 물음이 생기지 않는다.
 *
 * 만드는 쪽과 쓰는 쪽의 속도가 다르고 만드는 쪽이 얼마나 앞서 나갈지에 단단한 한계를
 * 두고 싶을 때 쓴다 - 이벤트 큐, 로그 고리, 오디오 버퍼. 가득 찼을 때의 거부가 바로
 * 그 기능이다 - 역압(backpressure)이다. 늘어나는 큐는 몰려드는 입력에 기억을 먹어
 * 치우는 것으로 답하다가 더 나쁜 일을 맞는다.
 */
```

```
int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 이벤트 4개짜리 용량. 5가 되는 일은 없다. */
    proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 4);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating a 4-slot ring should succeed");
    proven_ring_t ring = r.value;

    /* --- 가득 찰 때까지 push ---------------------------------------- */
    for (int i = 1; i <= 4; ++i) {
        proven_err_t err = PROVEN_RING_PUSH(&ring, int, i);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the first four pushes fit");
    }

    /* 다섯 번째 push 는 거부된다. 가장 오래된 항목을 덮어쓰지도, 늘어나지도 않는다.
     * 가득 찬 고리는 가득 찬 고리이고, 어떻게 할지는 부르는 쪽이 정한다 - 기다리든,
     * 새 이벤트를 버리든, 역압을 알리든. */
    proven_err_t full = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(full == PROVEN_ERR_OUT_OF_BOUNDS,
                    "pushing into a full ring must be refused, not silently absorbed");

    /* --- 넣은 차례대로 pop ------------------------------------------ */
    int out = 0;
    proven_err_t err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(proven_is_ok(err) && out == 1, "pop returns the oldest entry first");

    /* 이제 자리가 다시 생겼고, 고리는 아무것도 옮기지 않은 채 저장소를 감아 돈다. */
    err = PROVEN_RING_PUSH(&ring, int, 5);
    EXAMPLE_REQUIRE(proven_is_ok(err), "after a pop there is room for one more");

    int expected[] = { 2, 3, 4, 5 };
    for (int i = 0; i < 4; ++i) {
        err = PROVEN_RING_POP(&ring, int, &out);
        EXAMPLE_REQUIRE(proven_is_ok(err) && out == expected[i],
                        "entries come out in the order they went in");
    }

    /* --- 빈 상태도 가득 찬 상태처럼 군다: 거부할 뿐 자료를 지어내지 않는다 --- */
    err = PROVEN_RING_POP(&ring, int, &out);
    EXAMPLE_REQUIRE(!proven_is_ok(err), "popping an empty ring must fail rather than return junk");

    PROVEN_RING_DESTROY(&ring);
    return EXAMPLE_OK();
}
```

구조체

```
typedef struct {
    proven_allocator_t alloc;
    proven_mem_mut_t internal;
    proven_size_t head;
    proven_size_t tail;
    proven_size_t len;
    proven_size_t cap;
    proven_size_t elem_size;
    proven_size_t align;
} proven_ring_t;

typedef struct {
    proven_err_t err;
```

```
    proven_ring_t value;
} proven_result_ring_t;
```

함수와 매크로

API	의도	반환
`proven_ring_create(alloc, cap, elem_size, align)`	고정 용량 링을 만든다.	`proven_result_ring_t`.
`proven_ring_is_valid(ring)`	링 불변식을 검증한다.	`bool`.
`proven_ring_push(ring, element)`	원소 하나를 push한다; 가득 차면 실패한다.	`proven_err_t`.
`proven_ring_pop(ring, out_element)`	원소 하나를 pop한다; null 출력이면 버린다.	`proven_err_t`.
`proven_ring_destroy(ring)`	저장소를 해제한다.	void.
`PROVEN_RING_INIT(alloc, type, cap)`	타입 안전 생성.	`proven_result_ring_t`.
`PROVEN_RING_PUSH(ring_ptr, type, value)`	타입 안전 push.	`proven_err_t`.
`PROVEN_RING_POP(ring_ptr, type, out_ptr)`	타입 안전 pop.	`proven_err_t`.
`PROVEN_RING_DESTROY(ring_ptr)`	링을 파괴한다.	void.

예제:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_ring_t q = r.value;

proven_err_t e = PROVEN_RING_PUSH(&q, int, 7);
if (!proven_is_ok(e)) {
    /* The ring is fixed-capacity: a full ring is PROVEN_ERR_OUT_OF_BOUNDS,
     * never a silent grow. */
    PROVEN_RING_DESTROY(&q);
    return;
}

int out = 0;
e = PROVEN_RING_POP(&q, int, &out);   /* FIFO: out == 7. Empty ring is OUT_OF_BOUNDS. */
(void)e;

PROVEN_RING_DESTROY(&q);
```

4. 해시 맵

`proven_map_t`는 tombstone을 갖는 open-addressing 해시 맵이다. 정수 키와 빌려 쓰는(borrowed) 또는 소유(owned) U8 문자열 키를 지원한다. allocator를 내부에 저장한다.

구조체

```
typedef enum {
    PROVEN_KEY_TYPE_INT,          /* keys are proven_size_t integers */
    PROVEN_KEY_TYPE_U8_BORROWED,  /* keys are u8 views; caller keeps the bytes alive */
    PROVEN_KEY_TYPE_U8_OWNED      /* keys are u8 views; the map copies and frees the bytes */
} proven_key_type_t;

typedef union {
    proven_size_t id;             /* used when key_type == PROVEN_KEY_TYPE_INT */
    proven_u8str_view_t str;      /* used for the two U8 key modes */
} proven_map_key_t;
```

```
typedef struct {
    proven_allocator_t alloc;     /* allocator for the bucket array and owned keys */
    proven_mem_mut_t internal;    /* the single contiguous bucket array (ptr + size) */
    proven_size_t len;            /* live entries */
    proven_size_t used;           /* live entries + tombstones (drives the load factor) */
    proven_size_t cap;            /* number of buckets, always a power of two */
    proven_size_t elem_size;      /* size of one stored value */
    proven_size_t align;          /* alignment requested for the value */
    proven_size_t bucket_stride;  /* bytes per bucket: align_up(header + elem_size) */
    proven_size_t payload_offset; /* bytes from a bucket start to its value payload */
    proven_key_type_t key_type;   /* key mode chosen at create time */
} proven_map_t;

typedef struct {
    proven_err_t err;
    proven_map_t value;
} proven_result_map_t;
```

내부 동작 원리

맵은 cap개의 버킷으로 이루어진 단일 평면 배열이다 — 값에 대해 항목별 할당이 없다(그리고 INT/BORROWED 모드에서는 키에 대해서도 없다). 그래서 조회가 캐시 친화적으로 유지된다. 각 버킷은 다음과 같이 배치된다:

```
[ header: state + key ][ padding to the value's alignment ][ value payload ]
^ bucket start                                              ^ payload_offset
¦<------------------------ bucket_stride ------------------------------------>¦
```

헤더의 state는 **EMPTY**(한 번도 사용 안 함), **OCCUPIED**(살아 있는 키+값을 보유), **TOMBSTONE**(제거된 항목) 중 하나다. payload_offset과 bucket_stride는 생성 시점에 elem_size/align으로부터 한 번 계산되므로, 버킷 i의 값 주소는 그냥 internal.ptr + i*bucket_stride + payload_offset이다.

- **해싱, 그리고 기본값이 왜 안전한 쪽인가. 키는 OS CSPRNG에서 한 번 뽑은 프로세스별 비밀 아래에서 해싱된다** — 왜냐하면 신뢰할 수 없는 키를 예측 가능한 함수로 해싱하는 맵은 터지기를 기다리는 서비스 거부(DoS)이기 때문이다: 키를 통제하는 공격자는 오프라인에서 충돌을 계산해 그 전부를 한 버킷에 몰아넣고, 모든 조회를 선형 스캔으로 바꾼다. 그들이 볼 수 없는 비밀로 해시를 키잉하는 것이 그것을 막으며, 이는 Python, Rust, Linux 커널이 자기네 테이블에 대해 내린 것과 같은 선택이다. 문자열 키는 SipHash-2-4를, 정수 키는 키의 8바이트에 대한 SipHash-1-3을 쓴다 - 정수라고 더 안전하지 않다. 요청에서 온 사용자 id나 레코드 번호는 그것을 보낸 쪽이 고른다. 키가 전부 여러분 자신의 프로그램에서 온다면, proven_map_create_trusted가 빠른 unkeyed 함수를 대신 선택한다 - 문자열에는 FNV-1a, 정수에는 SplitMix 스타일 비트 혼합 finaliser(작은 테이블에서 조회당 약 7 ns 적다). proven_map_hash는 주어진 맵이 실제로 어느 함수를 쓰는지 알려준다. (CSPRNG도 공격자 모델도 없는 프리스탠딩(freestanding) 타깃에서는 키가 unkeyed 함수로 후퇴한다.)
- **탐사(Probing).** 선형 open addressing: 시작 버킷은 hash & (cap - 1)이며(cap이 2의 거듭제곱이라 저렴하다), 그다음 검색은 한 번에 한 버킷씩 앞으로 걸어가며 감싸고, 키(같은 키를 가진 OCCUPIED)나 EMPTY 버킷(키가 없음을 증명함)을 찾을 때까지 계속한다. 선형 탐사는 걸어가는 버킷들을 메모리 상에서 연속되게 유지한다.
- **제거와 tombstone.** proven_map_remove는 버킷을 그냥 비울 수 없다 — 그러면 탐사 체인이 끊겨 나중 키들이 숨겨진다. 대신 버킷을 TOMBSTONE으로 표시한다. Tombstone은 조회에서 건너뛰어지지만 여전히 슬롯을 차지하므로, len이 아니라 used(살아 있는 것 + tombstone)가 성장을 견인한다.
- **부하율(load factor)과 리사이즈.** used >= cap * 3/4이 되면, 맵은 다음 2의 거듭제곱 크기의 새 버킷 배열을 할당하고 모든 OCCUPIED 항목을 그리로 **재해싱**하며, 그 과정에서 모든 tombstone을 버린다. 용량은 커지기만 하고, 결코 줄어들지 않는다. proven_map_reserve로 미리 예약하라(특히 아레나 allocator에서는, 뒤에 죽은 배열을 남기지 않기 위해).

키 모드 — 하나 고르기

- `PROVEN_KEY_TYPE_INT`: 키는 `proven_size_t`다. 키 저장소 없음.
- `PROVEN_KEY_TYPE_U8_BORROWED`: 버킷이 view(포인터 + 길이)를 저장한다. **맵은 바이트를 절대 복사하지 않으므로**, 호출자는 항목이 존재하는 동안 내내 그 정확한 바이트를 살아 있고 옮겨지지 않은 채로 유지해야 한다. 이미 안정된 어딘가에 사는 키(문자열 리터럴, interned 문자열)에 가장 저렴하다.
- `PROVEN_KEY_TYPE_U8_OWNED`: 삽입 시(`proven_map_set_u8_owned` / `PROVEN_MAP_SET_U8_OWNED`) 맵이 키 바이트를 자기 저장소로 **복제**하고 remove/destroy 시 해제하므로, 호출 직후 원본 버퍼를 해제하거나 재사용해도 된다.

`set_with_scratch` / 별칭(alias) 사례

`proven_map_set`에 넘기는 `element`가 *맵 자신의 버킷 배열 안*을 가리킨다면(예를 들어 `proven_map_get`이 반환한 포인터를 통해 한 키에서 다른 키로 값을 복사하는 경우), 바로 그 삽입이 유발하는 재해싱이 여러분이 읽고 있는 바이트를 옮기거나 해제할 수 있다. `proven_map_set_with_scratch`(그리고 `*_WITH_SCRATCH_*` 매크로)는 원본 바이트를 먼저 임시 작업용(scratch) allocator의 임시 버퍼로 담아두므로 삽입이 별칭 안전(alias-safe)해진다. 영구 저장소는 여전히 `map->alloc`을 쓴다. scratch allocator는 그 일시적 복사에만 쓰인다.

함수

API	의도	반환
`proven_map_create(alloc, init_cap, key_type, elem_size, align)`	맵을 만든다.	`proven_result_map_t`.
`proven_map_is_valid(map)`	맵 불변식을 검증한다.	`bool`.
`proven_map_reserve(map, new_cap)`	용량을 보장한다.	`proven_err_t`.
`proven_map_set_with_scratch(map, key, element, scratch)`	임시 별칭 안전 복사에 scratch를 써서 삽입/갱신한다.	`proven_err_t`.
`proven_map_set(map, key, element)`	삽입/갱신한다.	`proven_err_t`.
`proven_map_set_u8_owned(map, key, element)`	맵이 소유하는 U8 키 저장소로 삽입/갱신한다.	`proven_err_t`.
`proven_map_get_mut(map, key)`	가변 값을 조회한다.	포인터 또는 null.
`proven_map_get(map, key)`	const 값을 조회한다.	포인터 또는 null.
`proven_map_remove(map, key)`	키가 있으면 제거한다.	`proven_err_t`.
`proven_map_destroy(map)`	맵 저장소를 해제한다.	void.
`proven_map_len(map)`	항목 수. NULL이면 0.	`proven_size_t`.
`proven_map_iter_init(map)`	모든 항목을 버킷 순서로 도는 순회를 시작한다.	`proven_map_iter_t`.
`proven_map_iter_next(&it, &key, &value)`	다음 항목. 마지막 뒤에는 `PROVEN_ERR_EOF`, 순회 중 새 키나 reserve로 맵이 다시 해싱되면 `PROVEN_ERR_INVALID_STATE`. 항목 제거와 값 변경은 허용된다.	`proven_err_t`.

모든 항목 방문하기. 맵에는 호출자가 기댈 수 있는 순서가 없다 - 버킷 순서이고, 키 있는 맵은 프로세스마다 다르다 - 하지만 담긴 것을 나열하거나, 직렬화하거나, `proven_map_destroy` 전에 값이 가진 자원을 풀기 위해 순회할 수는 있다. 방금 돌려받은 항목을 제거해도 된다:

```
/* Count words, then drop the rare ones while walking the map. */
proven_result_map_t words = proven_map_create(alloc, 16, PROVEN_KEY_TYPE_U8_BORROWED, sizeof(int),
_Alignof(int));
if (proven_is_ok(words.err)) {
    int one = 1, three = 3;
    (void)proven_map_set(&words.value, (proven_map_key_t){ .str = PROVEN_LIT("rare") }, &one);
    (void)proven_map_set(&words.value, (proven_map_key_t){ .str = PROVEN_LIT("common") }, &three);
    proven_map_iter_t it = proven_map_iter_init(&words.value);
```

```
    proven_map_key_t key;
    void *value;
    while (proven_map_iter_next(&it, &key, &value) == PROVEN_OK) {
        if (*(int *)value < 2) (void)proven_map_remove(&words.value, key);
    }
    proven_println("{} word(s) left", PROVEN_ARG(proven_map_len(&words.value)));
    proven_map_destroy(&words.value);
}
```

매크로

매크로	의도
`proven_map_create_with_capacity(...)`	용량 할당을 강조하는 별칭.
`PROVEN_MAP_INIT_INT(alloc, type, init_cap)`	정수 키 타입 맵을 만든다.
`PROVEN_MAP_INIT_U8_BORROWED(alloc, type, init_cap)`	borrowed 문자열 키 타입 맵을 만든다.
`PROVEN_MAP_INIT_U8_OWNED(alloc, type, init_cap)`	owned 문자열 키 타입 맵을 만든다.
`PROVEN_MAP_SET_INT(map_ptr, int_key, type, value)`	정수 키를 설정한다.
`PROVEN_MAP_SET_WITH_SCRATCH_INT(map_ptr, int_key, type, value, scratch)`	scratch allocator를 써서 정수 키를 설정한다.
`PROVEN_MAP_SET_U8_BORROWED(map_ptr, u8_view, type, value)`	borrowed U8 키를 설정한다.
`PROVEN_MAP_SET_U8_OWNED(map_ptr, u8_view, type, value)`	owned U8 키를 설정한다.
`PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED(map_ptr, u8_view, type, value, scratch)`	scratch allocator를 써서 borrowed U8 키를 설정한다.
`PROVEN_MAP_GET_INT(map_ptr, type, int_key)`	정수 키로 const 값을 얻는다.
`PROVEN_MAP_GET_U8_BORROWED(map_ptr, type, u8_view)`	borrowed U8 키로 const 값을 얻는다.
`PROVEN_MAP_GET_U8_OWNED(map_ptr, type, u8_view)`	owned U8 키로 const 값을 얻는다.
`PROVEN_MAP_GET_MUT_INT(map_ptr, type, int_key)`	정수 키로 가변 값을 얻는다.
`PROVEN_MAP_GET_MUT_U8_BORROWED(map_ptr, type, u8_view)`	borrowed U8 키로 가변 값을 얻는다.
`PROVEN_MAP_GET_MUT_U8_OWNED(map_ptr, type, u8_view)`	owned U8 키로 가변 값을 얻는다.
`PROVEN_MAP_REMOVE_INT(map_ptr, int_key)`	정수 키를 제거한다.
`PROVEN_MAP_REMOVE_U8_BORROWED(map_ptr, u8_view)`	U8 키를 제거한다.
`PROVEN_MAP_REMOVE_U8_OWNED(map_ptr, u8_view)`	owned U8 키를 제거한다.
`PROVEN_MAP_DESTROY(map_ptr)`	맵을 파괴한다.

owned 키 경로:

`PROVEN_KEY_TYPE_U8_OWNED`는 키 바이트의 복제본을 맵 내부에 저장한다. 원본 버퍼가 삽입 후 옮겨지거나 해제될 수 있을 때 사용하라. owned 바이트는 remove와 destroy 시 해제되며, 재해싱 동안 항목과 함께 이동한다.

`PROVEN_HARDENED`와 디버그 검증은 맵 자신의 내부 저장소 안에 들어가는 일부 borrowed 키 포인터를 거부할 수 있다. 그 검사는 방어적일 뿐이며, borrowed 키를 자기 소유(self-owning)로 만들지는 않는다.

예제:

```
typedef struct UserInfo {
    int level;
    double budget;
} UserInfo;
```

```
proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, UserInfo, 8);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t users = r.value;

UserInfo u = { .level = 3, .budget = 99.0 };
(void)PROVEN_MAP_SET_INT(&users, 404, UserInfo, u);

/* GET returns a pointer into the bucket array, or NULL when absent. Any insert
 * that rehashes invalidates it: look it up, use it, drop it. */
const UserInfo *found = PROVEN_MAP_GET_INT(&users, UserInfo, 404);
if (found) {
    proven_println("level={}", PROVEN_ARG(found->level));
}

PROVEN_MAP_DESTROY(&users);
```

5. 알고리즘

배열 알고리즘은 `proven_array_t`와 비교 콜백에 대해 동작한다.

비교 함수

```
typedef int (*proven_compare_fn_t)(const void *a, const void *b);
```

`a < b`이면 음수, 같으면 0, `a > b`이면 양수를 반환한다.

함수

API	의도	반환
`proven_array_sort(arr, cmp)`	배열을 제자리(in place) 정렬한다.	void.
`proven_array_binary_search(arr, key, cmp)`	정렬된 배열을 검색한다.	원소 포인터 또는 null.
`proven_array_linear_search(arr, key, cmp)`	임의의 배열을 스캔하며 검색한다.	원소 포인터 또는 null.

`proven_array_sort`는 introsort다: Bentley-McIlroy 3방향 분할, 작은 범위에 대한 insertion-sort 컷오프, 그리고 재귀가 `2*log2(n)` 레벨을 넘으면 쓰이는 heapsort fallback으로 이루어진다.

명시할 만한 성질이 둘 있다. 실제로 발목을 잡는 것들이기 때문이다:

- **O(n log n)은 전형적인 경우가 아니라 보장이다.** heapsort fallback이 그것을 보장으로 만든다. 적대적 순서로 최악의 경우에 도달할 수 있는 정렬은, 자기가 작성하지 않은 데이터를 정렬하는 어떤 프로그램에서도 서비스 거부다.
- **중복 키는 느린 경우가 아니라 빠른 경우다.** 피벗과 같은 원소들은 최종적이며 결코 재귀되지 않는 런(run)으로 모이므로, 전부-같은 입력은 한 번의 패스로 끝난다. 이것이 중요한 이유는 저기수성(low-cardinality) 키 — 상태 열, enum, 버킷 id — 가 실제로 호출자들이 정렬 기준으로 삼는 것이고, 그것들이 바로 순진한 2방향 분할이 퇴화하는 대상이기 때문이다.

정렬은 안정(stable)하지 않다: 같은 원소들은 재배열될 수 있다.

비교자는 평범한 파일 스코프 함수이므로, 형태는 이렇다:

```
static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;
    return (x > y) - (x < y);   /* not (x - y): that overflows */
}

proven_array_sort(&nums, cmp_int);
int key = 20;
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
```

이 장 끝의 완성 예제(manual/examples/ex_04_array.c)가 바로 이것을 컴파일하고 실행한 버전이다.

6. 용도별 해싱

단 하나의 "해시"란 없다. 어느 함수가 올바른지는 그 결과로 무엇을 하려는지에 전적으로 달려 있으며, 잘못된 것을 집으면 쓸데없이 느린 프로그램이나 조용히 안전하지 않은 프로그램을 얻는다. proven/hash.h는 작업당 정확히 하나의 primitive를 제공하므로, 작업 이름을 대는 순간 선택이 끝난다:

여러분이…	**사용**	**그리고 결정적으로**
여러분 자신의 테이블에 키를 해싱, **신뢰된** 입력	proven_hash_bytes (FNV-1a)	빠르다; 여기서 crypto 해시는 이득 없이 ~50배 느리다
신뢰할 수 없는 입력의 키를 테이블에 해싱	proven_hash_keyed (SipHash-2-4)	FNV는 공격자가 모든 키를 한 버킷에 충돌시켜 O(1) 테이블을 $O(n^2)$로 바꾸게 한다
데이터가 전송/디스크에서 **손상**되지 않았는지 확인	proven_crc32	체크섬; gzip/zlib/PNG와 상호운용된다
콘텐츠 **지문 채취**: 중복 제거, 콘텐츠 주소 지정, "같은 파일?"	proven_sha256	고의로 위조된 일치에 대해 안전한 유일한 것

기억할 한 줄: **CRC-32와 FNV는 사고를 탐지하지 공격을 탐지하지 않는다.** 누군가 여러분을 속여서 이득을 볼 수 있을 때 두 가지가 "같은지"를 결정하는 데 이것들을 쓰지 말라 — 그것이 proven_sha256이 있는 이유다. 그리고 keyed 해시는 키가 진짜 무작위성에서 한 번 고른 진짜 비밀일 때에만 안전하다; 고정 키는 키가 아닌 것이나 마찬가지다.

모든 함수는 바이트 단위로 정확하며 엔디안에 독립적이다: 같은 입력은 어떤 타깃에서든 같은 출력을 준다. 왜냐하면 기계에 따라 바뀌는 지문은 콘텐츠의 지문이 아니라 기계의 지문일 것이기 때문이다. 넷 모두 로열티 없는 알고리즘이며(FNV, CRC-32: public domain; SipHash: CC0; SHA-256: FIPS 180-4, 특허 없음), 각 명세로부터 구현되었고 각각의 공식 알려진 답(known-answer) 벡터에 대해 검증되었다.

참조

API	**의도**	**반환**
proven_hash_bytes(view)	FNV-1a 64. 여러분이 직접 고른 키를 위한 해시 테이블 해시.	proven_u64.
proven_hash_keyed(view, key[16])	16바이트 비밀 아래의 SipHash-2-4. 같은 작업이되, 공격자가 공급하는 키를 위한 것.	proven_u64.
proven_crc32(view)	한 번에 하는 CRC-32(IEEE, reflected) — gzip/zlib/PNG가 지니는 그것.	proven_u32.
proven_crc32_update(crc, view)	청크 스트림에 대한 같은 CRC. 0에서 시작하라; 호출 사이에 여러분이 쥐고 있는 값이 진짜 CRC이므로, 저장하고, 기록하고, 재개할 수 있다.	proven_u32.
proven_sha256(view, out[32])	한 번에 하는 SHA-256.	void; PROVEN_SHA256_SIZE 바이트를 쓴다.
proven_sha256_init/_update/_final	한 번에 메모리에 담을 수 없는 콘텐츠에 대한 같은 다이제스트. 다이제스트는 오직 바이트에만 의존	void.

	하며, 어떻게 청크되었는지에는 결코 의존하지 않는다.	
`proven_sha256_to_hex(digest, out[65])`	`sha256sum`과 `git`이 출력하는 64자 소문자 표기. NUL 종료.	void.

여러분이 쥐는 유일한 구조체:

```
typedef struct {
    /* Opaque. A running SHA-256: the 8-word chain value, a 64-byte block being
       filled, and the message length. Declare one, init it, update it, finalise it. */
} proven_sha256_t;

#define PROVEN_SHA256_SIZE 32   /* the digest; size your output buffer with this */
```

`proven_sha256_t`는 아무것도 할당하지 않는다 — 호출자 소유 상태이므로 파괴할 것이 없다.

주의사항, 그리고 무엇이 잘못되는가

고정 키를 가진 keyed 해시는 keyed 해시가 아니다. `proven_hash_keyed`의 보안은 전적으로 키가 진짜 무작위성에서 한 번 뽑은 비밀이라는 데 달려 있다. 이것이 `proven_map_create`가 그것을 대신 해주는 이유이자, 테이블을 위해 여러분이 직접 `proven_hash_keyed`를 호출하는 일이 거의 없어야 하는 이유다.

잘못된 예:

```
proven_byte_t key[16] = { 0 };            /* wrong: a "secret" everyone knows */
proven_u64 h = proven_hash_keyed(user_input, key);
```

```
proven_byte_t key[16];
memcpy(key, &timestamp, sizeof timestamp); /* wrong: guessable, and mostly zero */
```

올바른 예 — 시작 시점에 OS에서 한 번 뽑아라:

```
proven_byte_t key[16];
if (proven_random_bytes(key, sizeof key)) {
    proven_u64 h = proven_hash_keyed(
        proven_mem_view_from_u8(PROVEN_LIT("session-token")), key);
    (void)h;
}
```

두 가지가 "같다"고 결정하는 데 CRC-32나 FNV를 쓰지 말라 — 누군가 여러분을 속여서 이득을 볼 수 있을 때는. 둘 다 사소하게 위조할 수 있다: 같은 CRC-32를 갖는 두 번째 입력을 만드는 것은 교과서 수준의 산수다.

잘못된 예 — 공격자가 오염시킬 수 있는 콘텐츠 주소 지정 저장소:

```
if (proven_crc32(incoming) == stored_crc) {
    accept_as_identical(incoming);   /* wrong: a forged collision costs seconds */
}
```

올바른 예 — 속일 수 없어야 하는 지문은 `proven_sha256`이며, 32바이트 전체에 걸쳐 비교한다.

`PROVEN_SHA256_SIZE`가 아닌 다이제스트 버퍼는 버퍼 오버플로다. `proven_sha256`은 정확히 32바이트를 쓰며, 여러분의 배열이 얼마나 큰지 알지 못한다.

```
proven_byte_t digest[16];        /* wrong: SHA-256 is 32 bytes */
proven_sha256(data, digest);     /* writes 32 into a 16-byte buffer */
```

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 쓰임새로 나눈 해시. 이 모듈은 할 일마다 함수를 정확히 하나씩 준다. 그러니 고를 것은
 * "내가 할 일이 무엇인가" 하나뿐이고, *그것*을 틀리는 것이 위험의 전부다.
 */

int main(void) {
```

```
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* 할 일 1: 믿을 수 있는 입력을 내 표의 자리로. 빠르고, 암호용이 아니다. */
    proven_u64 table_hash = proven_hash_bytes(data);
    EXAMPLE_REQUIRE(table_hash != 0, "FNV-1a produces a spread-out 64-bit value");

    /* 할 일 2: *믿을 수 없는* 입력을 표의 자리로. 목적은 같지만, 입력을 고르는 공격자도
     * 모두를 충돌시킬 수 없다. 키를 모르기 때문이다. 키는 시작할 때 진짜 난수로 한 번
     * 고른다. 고정된 키를 쓰면 이 방법의 뜻이 사라진다. */
    proven_byte_t key[16] = { 0 };   /* 실제 코드에서는 난수원에서 한 번 채운다 */
    proven_u64 safe_hash = proven_hash_keyed(data, key);
    EXAMPLE_REQUIRE(safe_hash != table_hash, "a keyed hash is a different function");

    /* 할 일 3: 이 바이트들이 상했는가? 해시가 아니라 검사합이다. gzip/zlib/PNG 와
     * 그대로 맞물린다 - 셋 다 정확히 이 CRC-32 를 쓴다. */
    proven_u32 checksum = proven_crc32(data);
    /* 널리 쓰이는 CRC-32 확인값이다. 진짜 그 함수라는 것을 눈으로 볼 수 있다. */
    EXAMPLE_REQUIRE(proven_crc32(proven_mem_view_from_u8(PROVEN_LIT("123456789"))) == 0xcbf43926u,
                    "CRC-32 of \"123456789\" is the shared check value");
    (void)checksum;

    /* 할 일 4: 내용의 지문 - 중복 제거, 내용 주소화, "이 둘이 같은 파일인가". 일치를
     * 위조하려는 상대 앞에서도 안전하게 답한다. 답이 속아 넘어가면 안 되는 자리에서
     * 집는 것이 이것이다. */
    proven_byte_t digest[PROVEN_SHA256_SIZE];
    proven_sha256(data, digest);

    char hex[65];
    proven_sha256_to_hex(digest, hex);
    /* sha256sum 과 git 이 찍는 것과 같은 표기라, 그대로 맞물린다. */
    EXAMPLE_REQUIRE(hex[64] == '\0' && proven_cstr_len(hex) == 64,
                    "a SHA-256 fingerprint is 64 lowercase hex characters");

    /* SHA-256 은 흘려 넣을 수도 있다. 한 번에 기억에 담을 수 없는 내용을 위해서다 -
     * 다이제스트는 바이트에만 달렸지, 몇 토막으로 나눠 넣었는지에는 달리지 않는다. */
    proven_sha256_t ctx;
    proven_sha256_init(&ctx);
    proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("the quick ")));
    proven_sha256_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("brown fox")));
    proven_byte_t streamed[PROVEN_SHA256_SIZE];
    proven_sha256_final(&ctx, streamed);

    bool same = true;
    for (proven_size_t i = 0; i < PROVEN_SHA256_SIZE; ++i) {
        if (streamed[i] != digest[i]) same = false;
    }
    EXAMPLE_REQUIRE(same, "two updates of the halves equal one hash of the whole");

    return EXAMPLE_OK();
}
```

레거시 다이제스트: SHA-1과 MD5

proven/hash_legacy.h에는 깨졌지만 남의 형식이 여전히 이름을 부르는 다이제스트 둘이 있다. 입력을 고를 수 있는 사람은 MD5가 같은 서로 다른 메시지 둘을 몇 초 만에 만들고, SHA-1이 같은 둘도 알려진, 감당할 만한 비용으로 만든다. 누군가 속여서 이득을 볼 수 있는 곳에서 두 가지가 "같은가"를 정하는 데에는 둘 다 쓸 수 없다. 그 일은 여전히 proven_sha256의 것이다.

이 둘은 선택권이 없는 경우를 위해 있다.

형식이 요구하는 것	쓸 것	만나는 곳
SHA-1	`proven_sha1`	WebSocket accept 키(RFC 6455), `sha1sum` 파일, Git 객체 이름
MD5	`proven_md5`	오래된 서버에 대한 HTTP Digest 인증, `md5sum` 파일, `Content-MD5`

API	의도	반환
`proven_sha1(view, out[20])`	한 번에 계산하는 SHA-1.	void. `PROVEN_SHA1_SIZE`바이트를 쓴다.
`proven_sha1_init/_update/_final`	같은 다이제스트를 청크의 흐름 위에서.	void.
`proven_sha1_to_hex(digest, out[41])`	`sha1sum`과 `git`이 찍는 40자 소문자 표기. NUL로 끝난다.	void.
`proven_md5(view, out[16])`	한 번에 계산하는 MD5.	void. `PROVEN_MD5_SIZE`바이트를 쓴다.
`proven_md5_init/_update/_final`	같은 다이제스트를 청크의 흐름 위에서.	void.
`proven_md5_to_hex(digest, out[33])`	`md5sum`이 찍는 32자 소문자 표기. NUL로 끝난다.	void.

`proven_sha1_t`와 `proven_md5_t`는 `proven_sha256_t`처럼 호출자가 소유하는 상태다. 할당하는 것이 없고 destroy할 것도 없다. 두 다이제스트 모두 규격(FIPS 180-4, RFC 1321)에서 구현했고 그 문서가 싣고 있는 벡터로 검사했다. 이 헤더는 프리스탠딩 빌드에서도 쓸 수 있다.

잘못된 예 — 더 짧다는 이유로 깨진 다이제스트를 고르는 새 설계:

```
proven_byte_t id[PROVEN_MD5_SIZE];
proven_md5(upload, id);              /* wrong: two different uploads can share this id */
store_under(id, upload);
```

올바른 예 — 선택권이 있다면 `proven_sha256`이다. 32바이트가 들어갈 자리에 비해 길면 그 출력을 잘라 쓴다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * SHA-1과 MD5. 둘 다 상대가 입력에 손댈 수 있는 곳에서는 깨진 해시이고, 둘 다 아무도 바꿀 수
 * 없는 형식에 여전히 적혀 있다. 부를 이유는 그것 하나뿐이다.
 */

int main(void) {
    /* WebSocket 핸드셰이크(RFC 6455): 서버는 클라이언트의 키를 읽었다는 증거로
     * SHA-1(키 + 고정 GUID)을 Base64로 돌려준다. 형식이 SHA-1이라고 하니 SHA-1이다. */
    proven_sha1_t ctx;
    proven_sha1_init(&ctx);
    proven_sha1_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ==")));
    proven_sha1_update(&ctx, proven_mem_view_from_u8(PROVEN_LIT("258EAFA5-E914-47DA-95CA-C5AB0DC85B11")));
    proven_byte_t sha1[PROVEN_SHA1_SIZE];
    proven_sha1_final(&ctx, sha1);

    proven_byte_t accept[32];
    proven_size_t accept_len = 0;
    proven_err_t err = proven_base64_encode((proven_mem_view_t){ .ptr = sha1, .size = sizeof sha1 },
                                            accept, sizeof accept, &accept_len);
    EXAMPLE_REQUIRE(proven_is_ok(err), "20 bytes encode to 28 Base64 characters");
    EXAMPLE_REQUIRE(proven_u8str_view_eq((proven_u8str_view_t){ .ptr = accept, .size = accept_len },
                                         PROVEN_LIT("s3pPLMBiTxaQ9kYGzzhZRbK+xOo=")),
                    "the accept key printed in RFC 6455 section 1.3");
```

```
    /* 한 번에 계산하기, 그리고 sha1sum과 git이 찍는 hex 표기. */
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("abc"));
    proven_sha1(data, sha1);
    char sha1_hex[41];
    proven_sha1_to_hex(sha1, sha1_hex);
    EXAMPLE_REQUIRE(proven_cstr_len(sha1_hex) == 40, "a SHA-1 digest is 40 hex characters");

    /* MD5: 내려받은 파일을 md5sum 출력으로 공개된 체크섬 파일과 맞춰 본다. 손상된 다운로드는
     * 잡는다. 위조된 것은 못 잡는다 - 같은 MD5를 가진 두 번째 파일은 누구나 만들 수 있다. */
    proven_byte_t md5[PROVEN_MD5_SIZE];
    proven_md5(data, md5);
    char md5_hex[33];
    proven_md5_to_hex(md5, md5_hex);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_view_from_cstr(md5_hex),
                                         PROVEN_LIT("900150983cd24fb0d6963f7d28e17f72")),
                    "the digest md5sum prints for \"abc\"");

    /* MD5도 스트림으로 먹일 수 있다. 다이제스트는 바이트에만 달려 있다. */
    proven_md5_t mctx;
    proven_md5_init(&mctx);
    proven_md5_update(&mctx, proven_mem_view_from_u8(PROVEN_LIT("a")));
    proven_md5_update(&mctx, proven_mem_view_from_u8(PROVEN_LIT("bc")));
    proven_byte_t streamed[PROVEN_MD5_SIZE];
    proven_md5_final(&mctx, streamed);

    bool same = true;
    for (proven_size_t i = 0; i < PROVEN_MD5_SIZE; ++i) {
        if (streamed[i] != md5[i]) same = false;
    }
    EXAMPLE_REQUIRE(same, "two updates of the pieces equal one hash of the whole");

    return EXAMPLE_OK();
}
```

인증과 키 유도: HMAC과 HKDF

다이제스트는 바이트가 무엇인지 말해 준다. 누가 만들었는지는 말해 주지 않는다. 누구든 무엇이든 해시할 수 있으므로, SHA-256을 옆에 달고 온 메시지는 보낸 이에 대해 아무것도 증명하지 않는다. `hmac.h`에는 거기에 비밀 키를 끌어들이는 두 가지 구성이 있다.

필요한 것	쓸 것	이유
메시지가 키를 가진 누군가에게서 왔음을 아는 것 - 서명된 쿠키, 웹훅, 토큰	`proven_hmac`	키를 가진 쪽만 MAC을 계산할 수 있고, 메시지의 한 비트만 바뀌어도 MAC이 바뀐다
비밀 하나에서 여러 키 - 공유 비밀에서 암호화 키와 MAC 키	`proven_hkdf`	키마다 레이블에 묶인다. 하나를 알아도 다른 것에 대해 알 수 있는 것이 없다
비밀번호를 저장하거나 확인하는 것	어느 것도 아니다	둘 다 빠르고, 비밀번호는 추측할 수 있다: 공격자는 초당 수십억 번 시도한다. 여기에는 일부러 느리게 만든 함수(PBKDF2, scrypt, Argon2)가 필요한데, 이 라이브러리에는 없다

API	의도	반환
`proven_sha512(view, out[64])`, `proven_sha384(view, out[48])`	한 번에 하는 SHA-512와 SHA-384.	void.

`proven_sha512_init/_update/_final`, `proven_sha384_init/_update/_final`	조각으로 오는 내용에 대한 같은 것. 다이제스트는 시작한 계열의 함수로 끝낸다.	void.
`proven_hmac(hash, key, data, out)`	한 번에 하는 HMAC. `hash`는 `PROVEN_HMAC_SHA256`, `_SHA384` 또는 `_SHA512`.	`proven_err_t`: 모르는 해시이거나 크기가 있는 null 포인터면 `INVALID_ARG`.
`proven_hmac_init(&h, hash, key)`, `proven_hmac_update(&h, data)`, `proven_hmac_final(&h, out)`	조각으로 오는 메시지에 대한 같은 것. `final`이 상태에서 키를 지운다.	`init`: `proven_err_t`. 나머지는 void.
`proven_hmac_size(hash)`	MAC의 바이트 수: 32, 48 또는 64.	`proven_size_t`. 모르는 해시면 0.
`proven_hkdf(hash, salt, ikm, info, out)`	입력 키 재료에서 `info`에 묶인 키 `out.size`바이트를 유도한다.	`proven_err_t`: 해시 크기의 255배를 넘으면 `OUT_OF_BOUNDS`. `INVALID_ARG`. 오류일 때에는 아무것도 쓰지 않는다.
`proven_hkdf_extract(hash, salt, ikm, prk)`, `proven_hkdf_expand(hash, prk, info, out)`	두 단계를 따로: 한 번 추출하고, 키마다 확장한다.	`proven_err_t`, 위와 같다. `prk`가 해시보다 짧으면 `INVALID_ARG`.

MAC 버퍼는 해시가 무엇이든 `PROVEN_HMAC_MAX_SIZE`(64)바이트이고, 그 가운데 `proven_hmac_size(hash)`바이트가 쓰인다: 버퍼는 상수로 크기를 잡고 비교는 그 크기로 하라.

MAC은 `proven_mem_equal_ct`로 비교하라. `memcmp`로는 절대 안 된다. 평범한 비교는 처음 다른 바이트에서 멈추고, 걸린 시간이 앞쪽 몇 바이트가 맞았는지를 말해 준다. 추측을 많이 내고 응답 시간을 잴 수 있는 위조자는 MAC을 한 바이트씩 알아낸다 - 전체에 2^256번이 아니라 바이트마다 256번의 시도로. `proven_mem_equal_ct`는 무엇을 발견하든 모든 바이트를 본다(1장).

잘못된 예:

```
if (memcmp(received_mac, expected_mac, 32) == 0) accept();       /* wrong: leaks how many bytes matched */
```

올바른 예 — `proven_mem_equal_ct((proven_mem_view_t){ received, 32 }, (proven_mem_view_t){ expected, 32 })`.

키가 전부다. `proven_random_bytes`에서, 적어도 MAC만큼 길게 얻어라. 단어이거나, 제품 이름이거나, MAC보다 짧은 키는 가장 약한 고리이고, 구성의 어떤 부분도 그것을 메워 주지 않는다.

HMAC은 메시지가 진짜라고 말할 뿐 새것이라고 말하지 않는다. 유효한 서명 토큰은 제시될 때마다 유효하다. 재전송이 문제라면 - 어떤 행동을 일으키는 것이라면 문제다 - 서명되는 내용 안에 만료 시각이나 카운터를 넣고 확인하라.

`info`가 유도된 키들을 서로 떼어 놓는다. 같은 비밀과 같은 `info`로 두 번 부르면 같은 키가 나온다 - 그것이 요점이다. 양쪽이 같은 키를 유도해야 하니까. 그러므로 두 가지 용도에는 두 가지 레이블을 써야 한다. 그러지 않으면 키 하나가 두 가지 일을 하게 되고, 프로토콜은 그렇게 깨진다.

잘라 낸 SHA-512는 SHA-384가 아니다. 둘은 일부러 다른 값에서 시작한다. SHA-512 다이제스트의 앞 48바이트는 같은 입력의 SHA-384와 다른 수다.

SHA-384, SHA-512, HMAC, HKDF가 여기 있는 까닭은 TLS가 그것들로 지어지기 때문이고, 공개인 까닭은 프로그램이 자기 목적에 그것들을 필요로 하기 때문이다. FIPS 180-4, RFC 2104, RFC 5869에서 구현했고 그 문서들의 벡터로 시험했다.

테스트 스위트가 컴파일하고 실행한다:

```
#include <string.h>

/*
 * 다이제스트는 바이트가 무엇인지 말한다. MAC은 누가 그것을 보증했는지 말하고, KDF는 비밀 하나를
```

```
 * 프로토콜에 필요한 여러 키로 바꾼다. 그리고 어떤 비밀에든 따라붙는 호출 둘: 새지 않는 비교와,
 * 최적화로 지워지지 않는 지우기.
 */

static bool hex_is(const proven_byte_t *got, proven_size_t n, const char *want) {
    char text[129];
    proven_size_t len = 0;
    return proven_hex_encode((proven_mem_view_t){ got, n }, (proven_byte_t *)text, sizeof text, &len) ==
PROVEN_OK &&
           len == strlen(want) && memcmp(text, want, len) == 0;
}

int main(void) {
    // ---- SHA-512와 SHA-384 ---------------------------------------------------
    /* SHA-256과 같은 모양이다: 한 번에, 또는 메모리에 다 들어가지 않는 것에는 init / update / final.
     * SHA-384는 시작 값이 다른 SHA-512를 48바이트로 자른 것이다. */
    proven_byte_t d512[PROVEN_SHA512_SIZE], d384[PROVEN_SHA384_SIZE], again[PROVEN_SHA512_SIZE];
    proven_sha512(proven_mem_view_from_u8(PROVEN_LIT("abc")), d512);
    EXAMPLE_REQUIRE(hex_is(d512, 8, "ddaf35a193617aba"), "SHA-512 of abc begins as FIPS 180-4 prints it");
    proven_sha384(proven_mem_view_from_u8(PROVEN_LIT("abc")), d384);
    EXAMPLE_REQUIRE(hex_is(d384, 8, "cb00753f45a35e8b"), "and SHA-384");

    proven_sha512_t running;
    proven_sha512_init(&running);
    proven_sha512_update(&running, proven_mem_view_from_u8(PROVEN_LIT("a")));
    proven_sha512_update(&running, proven_mem_view_from_u8(PROVEN_LIT("bc")));
    proven_sha512_final(&running, again);
    EXAMPLE_REQUIRE(memcmp(d512, again, sizeof d512) == 0, "in pieces, the same digest");

    proven_sha384_t running384;
    proven_sha384_init(&running384);
    proven_sha384_update(&running384, proven_mem_view_from_u8(PROVEN_LIT("abc")));
    proven_sha384_final(&running384, again);
    EXAMPLE_REQUIRE(memcmp(d384, again, sizeof d384) == 0, "and for SHA-384");

    // ---- HMAC: 키를 가진 누군가가 보증한 메시지 ------------------------------
    /* 서버가 토큰에 서명하고, 나중에 돌아온 것이 자기가 발급한 그것인지 확인한다. */
    proven_byte_t key[32];
    EXAMPLE_REQUIRE(proven_random_bytes(key, sizeof key), "a key from the system: 32 bytes for a 32-byte
MAC");
    proven_mem_view_t token = proven_mem_view_from_u8(PROVEN_LIT("user=ada;expires=1767225600"));
    proven_byte_t mac[PROVEN_HMAC_MAX_SIZE];
    EXAMPLE_REQUIRE(proven_hmac(PROVEN_HMAC_SHA256, (proven_mem_view_t){ key, sizeof key }, token, mac) ==
PROVEN_OK, "the token's MAC");
    EXAMPLE_REQUIRE(proven_hmac_size(PROVEN_HMAC_SHA256) == 32, "32 bytes of it; the buffer is sized for the
largest");

    /* 확인하기: 다시 계산해서, 둘이 어디서 다른지 흘리지 "않고" 비교한다. memcmp는 처음 틀린
     * 바이트에서 멈추고, 걸린 시간이 위조자에게 어디까지 맞았는지 알려 준다. */
    proven_byte_t check[PROVEN_HMAC_MAX_SIZE];
    proven_hmac_t h;
    EXAMPLE_REQUIRE(proven_hmac_init(&h, PROVEN_HMAC_SHA256, (proven_mem_view_t){ key, sizeof key }) ==
PROVEN_OK, "the streaming form, for a message in pieces");
    proven_hmac_update(&h, proven_mem_view_from_u8(PROVEN_LIT("user=ada;")));
    proven_hmac_update(&h, proven_mem_view_from_u8(PROVEN_LIT("expires=1767225600")));
    proven_hmac_final(&h, check);                  /* `h`에서 키도 지운다 */
    EXAMPLE_REQUIRE(proven_mem_equal_ct((proven_mem_view_t){ mac, 32 }, (proven_mem_view_t){ check, 32 }),
"the token is genuine");

    /* 누군가 토큰을 바꾼다. 그에 맞는 MAC은 만들 수 없다. */
```

```
    proven_mem_view_t forged = proven_mem_view_from_u8(PROVEN_LIT("user=eve;expires=1767225600"));
    EXAMPLE_REQUIRE(proven_hmac(PROVEN_HMAC_SHA256, (proven_mem_view_t){ key, sizeof key }, forged, check) ==
PROVEN_OK &&
                    !proven_mem_equal_ct((proven_mem_view_t){ mac, 32 }, (proven_mem_view_t){ check, 32 }), "a
changed token does not carry the old MAC");

    /* 공개된 벡터 하나. 이것이 남들이 계산하는 그 HMAC임을 보이려고: RFC 4231, 2번 사례. */
    EXAMPLE_REQUIRE(proven_hmac(PROVEN_HMAC_SHA256, proven_mem_view_from_u8(PROVEN_LIT("Jefe")),
                                proven_mem_view_from_u8(PROVEN_LIT("what do ya want for nothing?")), check) ==
PROVEN_OK &&
                    hex_is(check, 32, "5bdcc146bf60754e6a042426089575c75a003f089d2739839dec58b964ec3843"),
"RFC 4231, test case 2");

    // ---- HKDF: 비밀 하나, 키 여럿 ------------------------------------------------
    /* 양쪽이 공유 비밀에 합의했다. 그것은 아직 키가 아니다: 용도마다 레이블에 묶인 키를 하나씩
     * 유도해서, 하나를 알아도 다른 것에 대해 알 수 있는 것이 없게 한다. */
    proven_byte_t shared[32];
    EXAMPLE_REQUIRE(proven_random_bytes(shared, sizeof shared), "stands for the output of a key exchange");
    proven_mem_view_t secret = { shared, sizeof shared };
    proven_mem_view_t salt = proven_mem_view_from_u8(PROVEN_LIT("example-protocol v1"));       /* 비밀이 아니
다. 비어도 된다 */

    proven_byte_t enc_key[32], mac_key[32];
    EXAMPLE_REQUIRE(proven_hkdf(PROVEN_HMAC_SHA256, salt, secret,
proven_mem_view_from_u8(PROVEN_LIT("encryption")), (proven_mem_mut_t){ enc_key, sizeof enc_key }) == PROVEN_OK
&&
                    proven_hkdf(PROVEN_HMAC_SHA256, salt, secret,
proven_mem_view_from_u8(PROVEN_LIT("authentication")), (proven_mem_mut_t){ mac_key, sizeof mac_key }) ==
PROVEN_OK,
                    "two keys from one secret");
    EXAMPLE_REQUIRE(!proven_mem_equal_ct((proven_mem_view_t){ enc_key, 32 }, (proven_mem_view_t){ mac_key,
32 }), "different labels, unrelated keys");

    /* 두 단계를 따로: 한 번 추출하고, 필요한 만큼 확장한다. */
    proven_byte_t prk[PROVEN_HMAC_MAX_SIZE], enc_again[32];
    EXAMPLE_REQUIRE(proven_hkdf_extract(PROVEN_HMAC_SHA256, salt, secret, prk) == PROVEN_OK &&
                    proven_hkdf_expand(PROVEN_HMAC_SHA256, (proven_mem_view_t){ prk, 32 },
proven_mem_view_from_u8(PROVEN_LIT("encryption")),
                                       (proven_mem_mut_t){ enc_again, sizeof enc_again }) == PROVEN_OK &&
                    proven_mem_equal_ct((proven_mem_view_t){ enc_key, 32 }, (proven_mem_view_t){ enc_again,
32 }), "extract then expand is the same derivation");

    /* HKDF는 해시의 255블록까지만 준다. 더 달라고 하면 잘라 주는 것이 아니라 거절한다. */
    static proven_byte_t too_much[255 * 32 + 1];
    EXAMPLE_REQUIRE(proven_hkdf_expand(PROVEN_HMAC_SHA256, (proven_mem_view_t){ prk, 32 },
proven_mem_view_from_u8(PROVEN_LIT("")),
                                       (proven_mem_mut_t){ too_much, sizeof too_much }) ==
PROVEN_ERR_OUT_OF_BOUNDS, "more than 8160 bytes from SHA-256: refused");

    // ---- 비밀을 다 썼을 때 ------------------------------------------------------
    /* 지운다. 여기에 평범한 루프를 쓰면 최적화기가 삭제한다 - 그 뒤로 버퍼를 읽는 것이 없으니까 - .
     * 그러면 키가 메모리에 남는다. 이 쓰기는 지워지지 않는다. */
    proven_mem_wipe((proven_mem_mut_t){ key, sizeof key });
    proven_mem_wipe((proven_mem_mut_t){ shared, sizeof shared });
    proven_mem_wipe((proven_mem_mut_t){ prk, sizeof prk });
    proven_mem_wipe((proven_mem_mut_t){ enc_key, sizeof enc_key });
    proven_mem_wipe((proven_mem_mut_t){ mac_key, sizeof mac_key });
    proven_byte_t zeros[32] = {0};
    EXAMPLE_REQUIRE(proven_mem_equal_ct((proven_mem_view_t){ key, 32 }, (proven_mem_view_t){ zeros, 32 }),
"the key is gone from its buffer");
```

```
    return EXAMPLE_OK();
}
```

7. 바이트를 텍스트로: hex와 Base64

일단 어떤 것을 해싱할 수 있고(위) 무작위 토큰을 뽑을 수 있으면(`random.h`), 그 바이트를 텍스트만 담는 곳 — URL, HTTP 헤더, 로그 줄, JSON 문자열 — 에 써야 한다. 그것이 `encode.h`다. 암호도, 압축도 없다; 이미 모두가 합의한 두 인코딩을, 숨은 할당 없이, 그리고 보통 잘못되는 두 가지 방식 없이 해낸다.

원하는 것	사용	알파벳
사람이 읽는 다이제스트나 몇 바이트	`proven_hex_encode`	소문자 hex, `sha256sum`과 `git`이 출력하는 것
URL, 쿠키, 파일명 안의 바이트	`proven_base64url_encode`	`-` `_`, 패딩 **없음** — 이스케이프할 것도, 망가뜨릴 `=`도 없음
HTTP 헤더, MIME, JSON 안의 바이트	`proven_base64_encode`	표준 `+` `/`, `=` 패딩

핵심은 두 가지 거부다:

- **디코더는 한 바이트를 쓰기 전에 입력 전체를 검증한다.** 프로그램 바깥에서 온 텍스트는 유효하다고 보장되지 않는다; 엉뚱한 문자, 잘못된 길이, 잘못된 패딩, 끼어든 공백은 아무것도 커밋하지 않은 채 `PROVEN_ERR_INVALID_ENCODING`이다 — 끝을 넘어선 읽기도 아니고, 호출자가 한 바이트 들어가서 손상을 발견하게 되는 조용히 짧은 결과도 아니다. `proven_base64_decode`는 알파벳 **둘 다**와 패딩 있음/없음을 받아들인다. 자기가 방출하는 것만 받는 디코더는 세상의 Base64의 절반을 거부하기 때문이다.
- **출력 크기는 추측이 아니라 호출이다** — `proven_hex_encoded_size`, `proven_base64_encoded_size`, 그리고 그들의 디코드 짝. 한 바이트 모자란 버퍼는 아무것도 쓰지 않은 채 `PROVEN_ERR_OUT_OF_BOUNDS`이며, 결코 잘린 접두사가 아니다.

이것은 순수 계산이다 — 할당도, OS도 없다 — 그리고 freestanding에서 이용 가능하다.

참조

모든 호출은 입력, 호출자 소유 출력 버퍼와 그 용량, 그리고 선택적 `written_out`을 받는다. 그 어느 것도 할당하지 않는다.

API	의도	반환
`proven_hex_encoded_size(n)`	`n` 바이트에 대해 `proven_hex_encode`가 쓸 문자 수: `n * 2`, NUL 없음.	`proven_size_t`. `n * 2`가 들어가지 않으면 `PROVEN_SIZE_MAX`.
`proven_hex_decoded_size(n)`	`n` 문자에 대해 `proven_hex_decode`가 쓸 바이트 수: `n / 2`.	`proven_size_t`.
`proven_hex_encode(data, out, cap, &w)`	소문자 hex.	`PROVEN_OK`; `cap`이 모자라면 `OUT_OF_BOUNDS`(**아무것도 쓰지 않음**); NULL out이나 `{NULL, >0}` view에는 `INVALID_ARG`.
`proven_hex_decode(text, out, cap, &w)`	hex를 디코드; 대문자와 소문자 둘 다 허용.	홀수 길이나 hex가 아닌 바이트에는 `INVALID_ENCODING`(**아무것도 커밋하지 않음**).
`proven_base64_encoded_size(n)`	두 Base64 형식 모두에 대한 상한: `4 * ceil(n/3)`.	`proven_size_t`. 그 값이 들어가지 않으면 `PROVEN_SIZE_MAX`.
`proven_base64_decoded_size(n)`	**패딩 있는 것과 없는** 텍스트에 대한 상한: `3 * ceil(n/4)`.	`proven_size_t`.

`proven_base64_encode(data, out, cap, &w)`	표준 알파벳(`+ /`), `=` 패딩.	위와 같음.
`proven_base64url_encode(data, out, cap, &w)`	URL-안전 알파벳(`- _`), 패딩 **없음**.	위와 같음.
`proven_base64_decode(text, out, cap, &w)`	디코드. 알파벳 **둘 다**, 패딩 있음/없음을 받아들인다.	엉뚱한 바이트, 잘못된 길이, 잘못된 패딩에는 `INVALID_ENCODING`.

`written_out`은 NULL이어도 된다. 진입 시와 모든 오류 경로에서 `0`으로 설정되므로, 결코 낡은 값이 아니다.

주의사항, 그리고 무엇이 잘못되는가

출력 크기는 눈대중이 아니라 크기 함수로 잡아라. 인코더는 잘라내는 대신 모자란 버퍼를 거부한다 — 즉 손으로 계산한 크기에서 얻는 실패는 조용한 손상이 아니라 여러분이 처리해야 하는 오류다. 그것이 좋은 결과다; 요점은 여러분이 여전히 그것을 처리해야 한다는 것이다.

잘못된 예 — 고전적인 하나 차이(off-by-one) 실수이며, 이제 오버플로 대신 *실패*한다:

```
proven_byte_t out[16];                      /* wrong: 12 bytes of hex needs 24 chars */
proven_hex_encode(twelve_bytes, out, sizeof out, &w);   /* OUT_OF_BOUNDS, nothing written */
```

올바른 예:

```
proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("twelve bytes"));
proven_byte_t out[64];
proven_size_t w = 0;
if (proven_hex_encoded_size(data.size) <= sizeof out &&
    proven_is_ok(proven_hex_encode(data, out, sizeof out, &w))) {
    /* `w` characters of lowercase hex in `out` - not NUL-terminated */
}
```

Base64URL은 패딩을 방출하지 않으며, 디코더는 그것을 받아들인다. `proven_base64_decoded_size`는 *올림*하므로 패딩 없는 텍스트에 대해서도 올바른 상한이다. 그것을 `3 * (n / 4)`로 "개선"하지 말라 — 그것은 패딩 없는 꼬리가 실어 나르는 1~2바이트를 버림하며, 여러분은 이 라이브러리 자신의 URL-안전 출력을 디코드하는 데 실패할 것이다. (실제로 한 번 그런 일이 있었고, 감사(audit)가 그것을 잡아냈다.)

```
proven_size_t cap = 3 * (text_len / 4);   /* wrong: 0 for "QQ", which decodes to 1 byte */
```

디코더를 직접 손으로 만들지 말라. 이것들이 존재하는 이유는 바로, 디코드가 여러분의 프로그램 바깥에서 텍스트를 읽고, 모두가 작성하는 두 줄짜리 루프가 그것을 신뢰하기 때문이다.

잘못된 예 — 홀수 입력에서 끝을 넘어 읽고, 쓰레기를 데이터로 받아들인다:

```
for (size_t i = 0; i < len; i += 2)              /* wrong: no length check */
    out[i/2] = (hexval(text[i]) << 4) | hexval(text[i+1]);   /* wrong: no validation */
```

`proven_hex_decode`는 한 바이트를 쓰기 전에 입력 **전체**를 검증하므로, 끝 근처의 엉뚱한 문자 하나가 여러분이 완전하다고 믿는 절반만 디코드된 접두사를 쥐게 만들 수 없다.

공백은 일부러 건너뛰지 않는다. 붙여넣은, 줄바꿈된 Base64 덩어리는 `INVALID_ENCODING`이지 조용히 다른 결과가 아니다. 줄바꿈된 입력을 받아들이고 싶다면, 공백을 직접 제거하라 — 의도적으로, 여러분이 볼 수 있는 곳에서.

목적지 크기는 손으로 짐작하지 말고 크기 함수로 구한다. 인코딩마다 짝이 하나씩 있고 — `proven_hex_encoded_size()` / `proven_hex_decoded_size()`, 그리고 `proven_base64_encoded_size()` / `proven_base64_decoded_size()` — 둘이 약속하는 것이 다르다.

방향	그 수의 의미	쓰는 법
인코딩	hex와 표준 Base64에서는 **정확한 값**. Base64URL은 패딩을 빼므로 결과가 더 짧을 수 있으나, 이 값은 여전히 안전한 상한이다.	이만큼 할당한다. 호출이 실제로 쓴 양을 알려준다.

디코딩	**상한**이다. 패딩과 쓰인 알파벳은 텍스트를 읽기 전에는 알 수 없다.	상한만큼 할당한 뒤, 길이로는 상한이 아니라 보고된 개수를 쓴다.

아래 예제는 그것으로 끝맺는다. `proven_base64_encoded_size()`로 크기를 잡아 목적지를 할당하고, 거기에 인코딩하고, `proven_base64_decoded_size()`로 역방향 버퍼 크기를 잡아 디코딩하고, 같은 바이트를 `proven_hex_decode()`로도 왕복시킨다 — 실제 도구들이 내놓는 대문자 입력과, 온전한 바이트 수가 될 수 없는 홀수 길이 입력까지 포함해서다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 바이트를 글자로, 쓰임새별로. 규칙은 해시와 같다. 할 일마다 함수 하나, 그리고 위험은
 * 할 일을 잘못 고르는 것. 사람이 읽을 것에는 hex, URL 에 들어갈 것에는 Base64URL,
 * 선로로 나갈 것에는 표준 Base64.
 */

int main(void) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("the quick brown fox"));

    /* 할 일 1: 사람이 읽거나 붙여 넣을 다이제스트 - hex, sha256sum 과 git 이 쓰는 표기. */
    proven_byte_t hex[64];   /* proven_hex_encoded_size(19) = 38 */
    proven_size_t hn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hn)),
                    "hex encode into a buffer sized by proven_hex_encoded_size");
    EXAMPLE_REQUIRE(hn == proven_hex_encoded_size(data.size), "two hex chars per byte");

    /* 할 일 2: URL 에 들어갈 토큰 - Base64URL. 퍼센트 이스케이프가 필요 없고, 파서가
     * 걸려 넘어질 '=' 채움도 없다. */
    proven_byte_t token_bytes[16] = { 0 };   /* 실제 코드에서는: proven_random_bytes(token_bytes, 16) */
    proven_byte_t url[32];
    proven_size_t un = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64url_encode(
                        (proven_mem_view_t){ token_bytes, sizeof token_bytes }, url, sizeof url, &un)),
                    "base64url encode a token");
    /* URL 안전 토큰에는 '=' 가 없다. */
    bool has_pad = false;
    for (proven_size_t i = 0; i < un; ++i) if (url[i] == '=') has_pad = true;
    EXAMPLE_REQUIRE(!has_pad, "the URL form emits no padding");

    /* 할 일 3: 선로로 나가는 바이트 - 표준 Base64, HTTP 와 MIME 이 기대하는 +/= 문자표. */
    proven_byte_t b64[64];
    proven_size_t bn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, b64, sizeof b64, &bn)),
                    "standard base64 encode");

    /* 그리고 왕복한다. 디코드는 바이트를 그대로 되돌려 준다. 두 문자표와 채움이 있든
     * 없든 받아 주는 디코더는 일부러 그렇게 만든 것이다 - 실제 입력은 온갖 모양으로 온다. */
    proven_byte_t back[32];
    proven_size_t dn = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
                        (proven_mem_view_t){ b64, bn }, back, sizeof back, &dn)),
                    "decode the base64 back");
    EXAMPLE_REQUIRE(dn == data.size && proven_memcmp(back, data.ptr, dn) == 0,
                    "what comes back is exactly what went in");

    /* 검증하는 디코더의 요점: 쓰레기는 추측하지 않고 거부한다. 이것을 두 줄짜리 반복문에
     * 먹였다면 끝을 넘어 읽거나 조용히 짧은 결과를 얻었을 것이다. */
    proven_err_t bad = proven_base64_decode(
        proven_mem_view_from_u8(PROVEN_LIT("not valid base64!!")), back, sizeof back, &dn);
```

```
    EXAMPLE_REQUIRE(bad == PROVEN_ERR_INVALID_ENCODING,
                    "a stray character is INVALID_ENCODING, with nothing committed");

    /* 그리고 한 바이트 모자란 버퍼는 거부된다. 잘리는 일은 없다. */
    proven_byte_t tiny[4];
    EXAMPLE_REQUIRE(proven_hex_encode(data, tiny, sizeof tiny, &hn) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "a too-small output buffer is OUT_OF_BOUNDS, not a truncated prefix");

    /* --- 짐작하지 말고 목적지 버퍼의 크기를 셈하기 ----------------------- */

    /* 위의 인코드와 디코드는 모두 넉넉한 것이 뻔한 고정 배열에 썼다. 실제 코드는
     * 목적지를 할당하고, 그러면 크기는 기억해 내는 것이 아니라 셈해야 한다. 크기 함수
     * 넷이 그것을 위한 것이다 - 인코딩마다, 방향마다 하나씩. */
    proven_allocator_t alloc = proven_heap_allocator();

    proven_size_t need = proven_base64_encoded_size(data.size);
    EXAMPLE_REQUIRE(need > 0, "an encoded size must be computed, not guessed");

    proven_result_mem_mut_t out = alloc.alloc_fn(alloc.ctx, need, 1);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "allocating exactly the encoded size must succeed");

    proven_size_t wrote = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_encode(data, out.value.ptr, out.value.size, &wrote)),
                    "encoding into a buffer sized by the size function must fit exactly");
    EXAMPLE_REQUIRE(wrote == need, "and use all of it: this size is exact for standard base64");

    /* 디코드 방향은 정확한 개수가 아니라 *상한*이다. base64 글에는 채움이 있을 수
     * 있으므로, 디코더가 실제로 몇 바이트를 썼는지 알려 준다. 버퍼는 상한으로 잡고,
     * 그다음에는 알려 준 개수를 쓸 것. */
    proven_size_t bound = proven_base64_decoded_size(wrote);
    EXAMPLE_REQUIRE(bound >= data.size, "the decoded bound must cover the real output");

    proven_result_mem_mut_t plain = alloc.alloc_fn(alloc.ctx, bound, 1);
    EXAMPLE_REQUIRE(proven_is_ok(plain.err), "allocating the decode bound must succeed");

    proven_size_t got = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_base64_decode(
                        (proven_mem_view_t){ out.value.ptr, wrote }, plain.value.ptr, plain.value.size,
&got)),
                    "decoding back must succeed");
    EXAMPLE_REQUIRE(got == data.size && proven_memcmp(plain.value.ptr, data.ptr, got) == 0,
                    "and reproduce the original bytes exactly");

    /* hex 도 같은 짝을 갖는다. 입력 길이가 짝수일 때 디코드 상한은 정확하고, 올바른
     * hex 는 언제나 짝수다 - 홀수 길이는 잘못된 입력이고, 디코더는 마지막 자리를
     * 버리는 대신 그렇다고 말한다. */
    proven_size_t hex_len = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_encode(data, hex, sizeof hex, &hex_len)),
                    "re-encode to hex, since the refused call above left its count unusable");

    proven_size_t hex_bound = proven_hex_decoded_size(hex_len);
    EXAMPLE_REQUIRE(hex_bound == data.size, "two hex characters decode to one byte");

    proven_byte_t from_hex[32];
    proven_size_t hex_got = 0;
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
                        (proven_mem_view_t){ hex, hex_len }, from_hex, sizeof from_hex, &hex_got)),
                    "hex decodes back to the original bytes");
    EXAMPLE_REQUIRE(hex_got == data.size && proven_memcmp(from_hex, data.ptr, hex_got) == 0,
                    "and the round trip is exact");
```

```
    /* hex 는 들어올 때 대소문자를 가리지 않는다. 도구마다 같은 다이제스트를 다른
     * 대소문자로 찍기 때문에 중요하다. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_hex_decode(
                        proven_mem_view_from_u8(PROVEN_LIT("DEADBEEF")), from_hex, sizeof from_hex,
&hex_got)),
                    "uppercase hex decodes too");
    EXAMPLE_REQUIRE(hex_got == 4, "and yields one byte per two characters");

    /* 글자 수가 홀수면 온전한 바이트 수가 될 수 없다. */
    EXAMPLE_REQUIRE(proven_hex_decode(proven_mem_view_from_u8(PROVEN_LIT("abc")),
                                      from_hex, sizeof from_hex, &hex_got) == PROVEN_ERR_INVALID_ENCODING,
                    "an odd-length hex string is malformed, not silently truncated");

    alloc.free_fn(alloc.ctx, plain.value.ptr);
    alloc.free_fn(alloc.ctx, out.value.ptr);

    (void)url; (void)un;
    return EXAMPLE_OK();
}
```

8. 예제와 오용 사례

배열 안을 가리키는 포인터는 낡을 수 있다

잘못된 예:

```
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
PROVEN_ARRAY_PUSH(&nums, int, 30);
*p = 99; /* wrong: push may have reallocated the array */
```

올바른 예:

```
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_array_t nums = r.value;
(void)PROVEN_ARRAY_PUSH(&nums, int, 10);

/* Push first, then take the pointer. A pointer fetched before a push may point
 * at a block the array has already reallocated away. */
(void)PROVEN_ARRAY_PUSH(&nums, int, 30);
int *p = PROVEN_ARRAY_GET_MUT(&nums, int, 0);
if (p) {
    *p = 99;
}

PROVEN_ARRAY_DESTROY(&nums);
```

하나의 침습적 노드는 한 번에 하나의 리스트에 속한다

잘못된 예:

```
proven_list_push_back(&list_a, &item.link);
proven_list_push_back(&list_b, &item.link); /* wrong */
```

멤버십마다 내장 노드를 하나씩 써라.

안전 매크로로 순회 중 제거하기

올바른 예:

```
typedef struct Item {
    int value;
    proven_list_node_t link;
```

```
} Item;

proven_list_t list;
proven_list_init(&list);
Item a = { .value = 1 };
Item b = { .value = 2 };
proven_list_push_back(&list, &a.link);
proven_list_push_back(&list, &b.link);

/* The SAFE form reads `it->next` into `next` before the body runs, so removing
 * `it` (which nulls its links) cannot strand the loop. */
proven_list_node_t *it = NULL;
proven_list_node_t *next = NULL;
PROVEN_LIST_FOR_EACH_SAFE(it, next, &list) {
    Item *item = PROVEN_LIST_ENTRY(it, Item, link);
    if (item->value % 2 == 0) {
        proven_list_remove(it);
    }
}
```

링 push는 커지지 않는다

잘못된 예:

```
PROVEN_RING_PUSH(&q, int, value); /* wrong if you ignore full-ring errors */
```

올바른 예:

```
proven_result_ring_t r = PROVEN_RING_INIT(alloc, int, 2);
if (!proven_is_ok(r.err)) {
    return;
}
proven_ring_t q = r.value;

int value = 7;
for (int i = 0; i < 3; ++i) {
    proven_err_t e = PROVEN_RING_PUSH(&q, int, value);
    if (e == PROVEN_ERR_OUT_OF_BOUNDS) {
        /* The ring is full. Drop the item, or drain one first - but decide. */
        int drop = 0;
        (void)PROVEN_RING_POP(&q, int, &drop);
        e = PROVEN_RING_PUSH(&q, int, value);
    }
    (void)e;
}

PROVEN_RING_DESTROY(&q);
```

borrowed 맵 키는 항목보다 오래 살아야 한다

잘못된 예:

```
proven_u8str_t key = make_key(alloc);
PROVEN_MAP_SET_U8_BORROWED(&m, proven_u8str_as_view(&key), int, 1);
proven_u8str_destroy(alloc, &key);
/* wrong: map still points at freed key bytes */
```

올바른 선택지:

- borrowed 키에는 문자열 리터럴이나 다른 안정된 저장소를 사용하라.
- owned 키 객체를 다른 곳에 저장하고, 맵 항목을 제거한 뒤 그것들을 파괴하라.
- 맵이 키 저장소를 소유해야 한다면 `PROVEN_KEY_TYPE_U8_OWNED`와 `proven_map_set_u8_owned()`를 사용하라.

같은 맵에서 borrow한 값을 삽입할 때는 scratch를 써라

`element`가 맵 저장소 안을 가리키고 삽입이 재해싱할 수 있다면, `proven_map_set_with_scratch()`나 scratch 매크로를 사용하라.

```
proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, int, 4);
if (!proven_is_ok(r.err)) {
    return;
}
proven_map_t m = r.value;
(void)PROVEN_MAP_SET_INT(&m, 1, int, 42);

/* `src` points into the map's own bucket array. Passing it straight to
 * proven_map_set would be an alias: a rehash triggered by that same insert can
 * free the bytes it is still reading from. The scratch form captures the source
 * bytes into a temporary from `scratch` first, so the insert is alias-safe. */
const int *src = PROVEN_MAP_GET_INT(&m, int, 1);
if (src) {
    (void)proven_map_set_with_scratch(&m, (proven_map_key_t){ .id = 2 }, src, scratch);
}

PROVEN_MAP_DESTROY(&m);
```

이진 검색은 정렬된 입력을 요구한다

잘못된 예:

```
int *hit = proven_array_binary_search(&nums, &key, cmp_int);
/* wrong if nums was not sorted by cmp_int */
```

완성 예제: 배열, 정렬, 이진 검색

테스트 스위트가 컴파일하고 실행한다. 중복 키에 대한 정렬의 동작에 주목하라: 그것들은 빠른 경우이지, 이차(quadratic)인 경우가 아니다.

```
/*
 * proven_array_t 는 원소 타입 하나짜리 늘어나는 벡터다. 만들 때 쓴 할당자를 스스로
 * 담아 두므로 push 는 늘릴 수 있고 destroy 는 할당자를 다시 받지 않고도 해제할 수
 * 있다 - 뒤집어 말하면 배열은 *자기 자신만으로* 지워져야 한다는 뜻이다.
 *
 * PROVEN_ARRAY_* 매크로는 void* 로 된 속을 타입 안전하게 감싼 얼굴이다. 타입에서
 * sizeof/alignof 를 뽑고 원소를 포인터로 건네므로, 레코드 배열에 int 를 밀어 넣는
 * 코드는 컴파일되지 않는다.
 */

/* 우선순위를 키로 하는 작업 큐. 실제 자료는 값의 가짓수가 적다 - 우선순위 셋에 작업은
 * 여럿. 그 사실이 정렬에서 중요하다. 아래를 볼 것. */
typedef struct {
    int priority;
    int id;
} ex_task_t;

/* 비교자가 곧 배열의 순서다. 정렬과 이진 탐색에는 *반드시* 같은 것을 써야 한다 -
 * 다른 순서로 찾는 것은 컴파일러가 볼 수 없는 버그다.
 *
 * (x > y) - (x < y) 꼴은 일부러 쓴 것이다. 두 int 를 빼면 큰 값에서 넘쳐 엉뚱한
 * 부호를 돌려준다. */
static int cmp_priority(const void *a, const void *b) {
    int x = ((const ex_task_t *)a)->priority;
    int y = ((const ex_task_t *)b)->priority;
    return (x > y) - (x < y);
}

int main(void) {
```

```
proven_allocator_t alloc = proven_heap_allocator();

/* --- push / get / pop ------------------------------------------------- */
/* init_cap 은 한계가 아니라 힌트다. push 는 그것을 넘어 늘어난다. 잘 잡아 두면
 * 재할당을 피할 뿐이다. */
proven_result_array_t r = PROVEN_ARRAY_INIT(alloc, ex_task_t, 4);
EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an array of ex_task_t must succeed");
if (!proven_is_ok(r.err)) {
    return 1;
}
proven_array_t tasks = r.value;

/* 일부러 중복이 많게 했다. 실제 키가 그렇게 생겼기 때문이다 - 상태 열, enum,
 * 버킷 번호. */
static const ex_task_t seed[] = {
    { .priority = 2, .id = 10 }, { .priority = 0, .id = 11 },
    { .priority = 2, .id = 12 }, { .priority = 1, .id = 13 },
    { .priority = 2, .id = 14 }, { .priority = 0, .id = 15 },
    { .priority = 1, .id = 16 }, { .priority = 2, .id = 17 },
};

for (proven_size_t i = 0; i < sizeof seed / sizeof seed[0]; ++i) {
    proven_err_t err = PROVEN_ARRAY_PUSH(&tasks, ex_task_t, seed[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing into a growable array must succeed");
    if (!proven_is_ok(err)) {
        PROVEN_ARRAY_DESTROY(&tasks);
        return 1;
    }
}
EXAMPLE_REQUIRE(tasks.len == 8, "eight pushes give eight elements");

/* get 은 배열 저장소 *안*을 가리키는 포인터를 돌려준다 - 사본이 아니고, 다음
 * push 가 재할당하면 매달린 포인터가 된다. push 를 다 한 뒤에 얻어 쓰고, 담아
 * 두지는 말 것. */
const ex_task_t *front = PROVEN_ARRAY_GET(&tasks, ex_task_t, 0);
EXAMPLE_REQUIRE(front && front->id == 10, "element 0 is the first one pushed");

/* 범위를 벗어나면 널 포인터다. 죽는 것도, 미정의 동작도 아니다. */
EXAMPLE_REQUIRE(PROVEN_ARRAY_GET(&tasks, ex_task_t, 99) == NULL, "an out-of-range index yields NULL");

/* pop 은 마지막 원소를 복사해 내보낸다(그냥 버리려면 NULL 을 건넨다). */
ex_task_t last = {0};
proven_err_t err = PROVEN_ARRAY_POP(&tasks, ex_task_t, &last);
EXAMPLE_REQUIRE(proven_is_ok(err), "popping a non-empty array must succeed");
EXAMPLE_REQUIRE(last.id == 17 && tasks.len == 7, "pop returns the last element and shrinks the array");

/* 도로 넣는다. 이 예제의 나머지가 여덟 개를 모두 필요로 한다. */
err = PROVEN_ARRAY_PUSH(&tasks, ex_task_t, last);
EXAMPLE_REQUIRE(proven_is_ok(err), "pushing the popped element back must succeed");

/* --- 정렬 ------------------------------------------------------------- */
/* 인트로소트다. O(n log n) 은 평균이 아니라 보장이다 - 힙소트로 물러나는 길이
 * 있어, 상대가 유도할 수 있는 제곱 시간 경우를 배제한다. 그리고 여기서는 같은 키가
 * *빠른* 쪽이다. 피벗과 같은 것은 모두 한 구간으로 갈라 두고 다시 들어가지 않으므로,
 * 위의 중복된 우선순위는 서로 다른 값보다 값이 덜 든다.
 *
 * 안정 정렬은 아니다. 작업 10 과 12 는 어느 쪽이 먼저 나올지 정해져 있지 않다. */
proven_array_sort(&tasks, cmp_priority);

for (proven_size_t i = 1; i < tasks.len; ++i) {
```

```
        const ex_task_t *prev = PROVEN_ARRAY_GET(&tasks, ex_task_t, i - 1);
        const ex_task_t *cur  = PROVEN_ARRAY_GET(&tasks, ex_task_t, i);
        EXAMPLE_REQUIRE(prev && cur && prev->priority <= cur->priority,
                        "after sorting, priorities must be non-decreasing");
    }

    /* --- 이진 탐색 ------------------------------------------------------- */
    /* 바로 이 비교자로 정렬해 두었기 때문에만 옳다. 키는 원소 하나 통째지만, 비교자가
     * 읽는 필드만 뜻이 있다. */
    ex_task_t key = { .priority = 1, .id = 0 };
    const ex_task_t *hit = (const ex_task_t *)proven_array_binary_search(&tasks, &key, cmp_priority);
    EXAMPLE_REQUIRE(hit != NULL, "priority 1 is present, so the search must find it");
    EXAMPLE_REQUIRE(hit && hit->priority == 1, "the hit must be an element with the searched key");
    /* 키가 중복되면 그 키를 가진 *어떤* 원소를 찾지, 첫 번째를 찾지는 않는다. 첫
     * 번째가 필요하면 찾은 자리에서 뒤로 훑을 것. */

    ex_task_t absent = { .priority = 9, .id = 0 };
    EXAMPLE_REQUIRE(proven_array_binary_search(&tasks, &absent, cmp_priority) == NULL,
                    "a key that is not there must return NULL");

    printf("tasks: %zu sorted, found priority %d (id %d)\n",
           (size_t)tasks.len, hit ? hit->priority : -1, hit ? hit->id : -1);

    /* 배열은 원소 저장소를 소유한다. destroy 는 배열을 만들 때 쓴 할당자로 그것을
     * 돌려준다. 여기 원소는 그냥 자료다 - 원소가 무언가를 소유했다면 하나씩 먼저
     * 지워야 했을 것이다. */
    PROVEN_ARRAY_DESTROY(&tasks);
    return EXAMPLE_OK();
}
```

완성 예제: 정수 키 맵과 owned 문자열 키 맵

테스트 스위트가 컴파일하고 실행한다. owned 키 절반이 잘못 이해하기 쉬운 요점을 증명한다: 맵이 키를 복사하므로, 그것을 만든 버퍼는 즉시 재사용해도 되는 여러분의 것이다.

```
/*
 * proven_map_t 는 납작한 개방 주소법 해시 맵이다. 값 타입은 만들 때 정해지고 버킷
 * 배열 안에 그대로 담긴다 - 값을 위한 항목별 할당이 없고, get 은 그 배열 속을 곧장
 * 가리키는 포인터를 돌려준다.
 *
 * 재미있는 결정은 *키* 쪽이다.
 *
 *   PROVEN_KEY_TYPE_INT          - 키가 proven_size_t 다. 소유할 것이 없다.
 *   PROVEN_KEY_TYPE_U8_BORROWED  - 버킷이 여러분의 포인터와 길이를 담는다. 맵은
 *                                  바이트를 복사하지 않으므로, 항목이 살아 있는
 *                                  동안 *여러분*이 그 바이트를 살아 있고 움직이지
 *                                  않게 지켜야 한다. 문자열 리터럴에 알맞다.
 *   PROVEN_KEY_TYPE_U8_OWNED     - 맵이 키 바이트를 제 저장소로 복사하고 나중에
 *                                  해제한다. 실행 중에 만든 키에 알맞고, 대개의
 *                                  키가 그렇다.
 *
 * 이 예제의 뒷부분이 OWNED 가 존재하는 이유다.
 */

typedef struct {
    int  level;
    long score;
} ex_player_t;

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();
```

```
    /* --- 정수 키 ------------------------------------------------------------ */
    proven_result_map_t r = PROVEN_MAP_INIT_INT(alloc, ex_player_t, 8);
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "creating an int-keyed map must succeed");
    if (!proven_is_ok(r.err)) {
        return 1;
    }
    proven_map_t by_id = r.value;

    proven_err_t err = PROVEN_MAP_SET_INT(&by_id, 404, ex_player_t, ((ex_player_t){ .level = 3, .score =
990 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting into the map must succeed");
    err = PROVEN_MAP_SET_INT(&by_id, 7, ex_player_t, ((ex_player_t){ .level = 1, .score = 10 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a second key must succeed");

    /* 이미 있는 키에 set 하면 값을 바꾼다. 항목을 더하지 않는다. */
    err = PROVEN_MAP_SET_INT(&by_id, 7, ex_player_t, ((ex_player_t){ .level = 2, .score = 40 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "re-setting an existing key must succeed");
    EXAMPLE_REQUIRE(by_id.len == 2, "re-setting a key replaces its value rather than adding an entry");

    /* get 은 버킷 배열 속을 가리키는 포인터를, 없으면 NULL 을 돌려준다. 다시 해싱하는
     * 삽입이 일어나면 무효가 된다 - 찾고, 쓰고, 버릴 것. */
    const ex_player_t *p = PROVEN_MAP_GET_INT(&by_id, ex_player_t, 7);
    EXAMPLE_REQUIRE(p && p->level == 2 && p->score == 40, "get must see the replaced value");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, ex_player_t, 999) == NULL, "a missing key yields NULL");

    err = PROVEN_MAP_REMOVE_INT(&by_id, 7);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&by_id, ex_player_t, 7) == NULL, "a removed key is gone");
    EXAMPLE_REQUIRE(by_id.len == 1, "removal decrements the live entry count");

    PROVEN_MAP_DESTROY(&by_id);

    /* --- 소유하는 문자열 키 ------------------------------------------------- */
    /* 같은 맵인데, 실행 중에 지어낸 이름을 키로 쓴다 - 빌린 키였다면 매달린 포인터가
     * 되기를 기다리는 자리다. */
    proven_result_map_t rm = PROVEN_MAP_INIT_U8_OWNED(alloc, ex_player_t, 8);
    EXAMPLE_REQUIRE(proven_is_ok(rm.err), "creating an owned-string-keyed map must succeed");
    if (!proven_is_ok(rm.err)) {
        return 1;
    }
    proven_map_t by_name = rm.value;

    /* 키마다 다시 쓸 작정인 임시 버퍼. BORROWED 맵에서 그 작정은 치명적이다. 모든
     * 항목이 바로 이 같은 바이트를 가리키게 된다. */
    proven_byte_t scratch[32];
    proven_u8str_t name = proven_u8str_borrow(scratch, sizeof scratch);

    err = proven_u8str_append(&name, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "building the first key must succeed");

    /* set_u8_owned 는 키 바이트를 맵 저장소로 *복사한다*. 돌아온 뒤로 맵의 키는
     * `scratch` 와 아무 상관이 없다. */
    err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), ex_player_t,
                                  ((ex_player_t){ .level = 9, .score = 5000 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting with an owned key must succeed");

    /* 그래서 버퍼는 곧바로 다음 키에 다시 쓸 수 있고... */
    err = proven_u8str_reset(&name);
    EXAMPLE_REQUIRE(proven_is_ok(err), "the key buffer is ours again the moment set returns");
```

```
    err = proven_u8str_append(&name, PROVEN_LIT("grace"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "overwriting the buffer with the next key must succeed");

    err = PROVEN_MAP_SET_U8_OWNED(&by_name, proven_u8str_as_view(&name), ex_player_t,
                                  ((ex_player_t){ .level = 4, .score = 700 }));
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting the second owned key must succeed");

    /* ...첫 항목은 그 덮어쓰기에 조금도 다치지 않는다. 이것이 요점 전부다. 맵은
     * 지금 "grace" 라고 적힌 버퍼의 뷰가 아니라 "ada" 의 제 사본을 쥐고 있다. BORROWED
     * 맵이었다면 둘 다 "grace" 로 키가 잡힌 항목 둘을 보고했을 것이다 - 아니면 더 나쁘게,
     * 해제된 기억이 키인 항목 하나를. */
    const ex_player_t *ada = PROVEN_MAP_GET_U8_OWNED(&by_name, ex_player_t, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(ada && ada->score == 5000, "the copied key survives the caller reusing its buffer");

    const ex_player_t *grace = PROVEN_MAP_GET_U8_OWNED(&by_name, ex_player_t, PROVEN_LIT("grace"));
    EXAMPLE_REQUIRE(grace && grace->score == 700, "the second key is a separate entry");
    EXAMPLE_REQUIRE(by_name.len == 2, "two distinct keys means two entries");

    /* remove 는 맵이 만든 키 사본을 해제한다 - 여러분이 직접 해제하는 일은 없다. */
    err = PROVEN_MAP_REMOVE_U8_OWNED(&by_name, PROVEN_LIT("ada"));
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing an owned key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_OWNED(&by_name, ex_player_t, PROVEN_LIT("ada")) == NULL,
                    "the removed entry is gone");

    printf("map: %zu name(s) left, grace at level %d\n",
           (size_t)by_name.len, grace ? grace->level : -1);

    /* destroy 는 버킷 배열과 그 안에 남은 키 사본을 모두 해제한다(여기서는 "grace").
     * `scratch` 는 우리 것이고 맵보다 오래 살며, 빌린 `name` 손잡이는 해제할 것이 없다. */
    PROVEN_MAP_DESTROY(&by_name);
    proven_u8str_destroy(alloc, &name);
    return EXAMPLE_OK();
}
```

실전 예제: 컨테이너 크기 잡기, 정렬되지 않은 데이터 찾기, 흘러가는 데이터 검사합 구하기

앞의 실전 예제들은 컨테이너를 하나씩 다룬다. 이 예제는 그 조각들이 들어가 있는 프로그램 — 작은 이벤트 수집기 — 이고, 컨테이너가 여럿 얽히고 나서야 생기는 질문들에 답한다.

- **채우는 도중에 컨테이너가 재할당하지 않게 한다.** `proven_array_reserve()`와 `proven_map_reserve()`는 용량을 한 번에 정한다. 힙에서는 복사를 아끼고, 아레나(arena) 뒤에서는 재할당이 다음 reset까지 남기는 죽은 저장 공간을 아낀다. map에서는 포인터의 정확성 문제이기도 하다. 리해시(rehash, 버킷 배열을 다시 만드는 일)는 앞서 `get_mut`이 돌려준 모든 포인터를 무효로 만든다.
- **내가 만들지 않은 핸들을 확인한다.** `proven_array_is_valid()`, `proven_ring_is_valid()`, `proven_map_is_valid()`는 핸들 자체의 필드가 서로 맞는지 묻는다. 0으로만 초기화된, 혹은 아예 만든 적 없는 컨테이너를 첫 push가 아니라 내 코드의 경계에서 잡아내는 검사다.
- **정렬되지 않은 데이터를 찾는다.** 이벤트 로그는 도착 순서로 둔다. 그 순서 자체가 기록하려는 대상이기 때문이다. 그래서 이진 탐색(binary search)은 쓸 수 없다 — 정렬되지 않은 입력에서 그것은 "없음"을 돌려주지 않고 틀린 답을 돌려준다. 올바른 호출은 `proven_array_linear_search()`이고, 순서상 **첫 번째** 일치를 돌려준다.
- **map에 이미 있는 값을 한 번의 조회로 고친다.** `proven_map_get_mut()`은 map 자신의 저장 공간을 가리키는 포인터를 돌려주므로, 카운터는 읽고 고쳐 다시 쓰는 대신 있는 자리에서 증가한다.
- **키에 어떤 해시가 어울리는지 고른다.** `proven_map_create()`는 문자열 키를 키가 있는 SipHash로 해싱하므로, 키를 고르는 공격자가 모든 키를 한 버킷에 몰아넣을 수 없다. `proven_map_create_trusted()`는 빠른 FNV-1a를 쓰며, 모든 키를 내 코드가 고를 때만 옳다. `proven_map_hash()`는 map이 실제로 키를 놓는 데 쓴 값을 돌려주므로, 둘의 차이는 주장이 아니라 관찰할 수 있는 사실이 된다.

- **조각으로 도착하는 데이터의 검사합을 구한다.** `proven_crc32_update()`는 각 조각을 진행 중인 값에 접어 넣는다. 조각 경계를 어떻게 나누었든 결과는 전체에 대한 `proven_crc32()`와 같다.

```
/*
 * 컨테이너 장들은 구조를 하나씩 따로 보인다. 실제 프로그램은 여럿을 한꺼번에 쓰고, 그때
 * 떠오르는 물음들은 어느 한 컨테이너에 대한 것이 아니다.
 *
 *   - 채우는 동안 컨테이너가 재할당하지 않게 하려면?  reserve.
 *   - 남이 건넨 컨테이너가 쓸 만한지 확인하려면?  is_valid.
 *   - 자료가 정렬돼 있지 *않을* 때 찾으려면?  선형 탐색 - 그리고 여기서 이진 탐색이 왜
 *     틀린 답을 주는지 아는 것.
 *   - 맵에 이미 있는 값을 두 번 찾지 않고 고치려면?  get_mut.
 *   - 공격에 견디는 해시가 필요한가, 빠른 쪽인가?  키를 누가 고르느냐에 달렸고,
 *     map_hash 가 그 차이를 눈으로 보게 해 준다.
 *   - 토막토막 도착하는 자료의 검사합을 구하려면?  crc32_update.
 *
 * 이 프로그램은 작은 이벤트 수집기다. 이벤트가 도착하고, 클라이언트별로 세어지고, 최근
 * 몇 개는 진단 덤프용으로 남고, 지나가는 동안 묶음의 검사합이 구해진다.
 */

typedef struct {
    proven_u32 client;
    proven_u32 code;      /* 이벤트 코드. 0 은 "연결이 닫혔다" 는 뜻이다 */
} ex_event_t;

/* 클라이언트 번호로 찾기. 이것은 이벤트를 *찾기* 위한 비교이지 정렬하기 위한 것이
 * 아니다. 아래 배열은 *도착한 차례*로 있고 그대로 둔다. */
static int by_client(const void *a, const void *b) {
    const ex_event_t *x = (const ex_event_t *)a;
    const ex_event_t *y = (const ex_event_t *)b;
    return (x->client > y->client) - (x->client < y->client);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. reserve: 용량을 한 번에 정한다 ------------------------------- */

    proven_result_array_t ar = PROVEN_ARRAY_INIT(alloc, ex_event_t, 4);
    EXAMPLE_REQUIRE(proven_is_ok(ar.err), "creating the event log must succeed");
    if (!proven_is_ok(ar.err)) {
        return 1;
    }
    proven_array_t events = ar.value;

    /* 시작하기 전에 묶음 크기를 알고 있으니 자리를 한 번에 청한다. 이것이 없으면 배열은
     * 채워지는 동안 두 배씩 늘며 그때마다 내용을 복사하고, 아레나 할당자 뒤에서라면 그
     * 복사마다 옛 블록이 아레나가 reset 될 때까지 남는다. */
    proven_err_t err = proven_array_reserve(&events, 16);
    EXAMPLE_REQUIRE(proven_is_ok(err), "reserving room for the whole batch must succeed");
    EXAMPLE_REQUIRE(events.cap >= 16, "the capacity is now at least what was asked for");

    /* is_valid 는 손잡이 자체가 구조적으로 온전한지 묻는다 - 서로 맞아떨어지는 포인터와
     * 길이와 용량. 컨테이너가 다른 코드에서 넘어오는 자리나, 0 으로 초기화된 손잡이가
     * 빠져나갔을 수 있는 자리에서 단언할 것. push 마다 되풀이할 것은 아니다. */
    EXAMPLE_REQUIRE(proven_array_is_valid(&events), "a created array must be structurally valid");

    proven_array_t never_created = {0};
    EXAMPLE_REQUIRE(!proven_array_is_valid(&never_created),
                    "a zero-initialised handle is not a usable array");
```

```
static const ex_event_t batch[] = {
    { .client = 7, .code = 200 }, { .client = 3, .code = 200 },
    { .client = 7, .code = 404 }, { .client = 9, .code = 200 },
    { .client = 3, .code = 500 }, { .client = 7, .code = 0   },
};
proven_size_t batch_len = sizeof batch / sizeof batch[0];

proven_size_t cap_before = events.cap;
for (proven_size_t i = 0; i < batch_len; ++i) {
    err = PROVEN_ARRAY_PUSH(&events, ex_event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing an event must succeed");
}
EXAMPLE_REQUIRE(events.cap == cap_before, "the reserve was enough: nothing reallocated while filling");

/* --- 2. 정렬되지 않은 자료에서 찾기 ---------------------------------- */

/* 로그는 도착한 차례로 있고, 그 차례를 지키고 싶다. 그것이 기록하려는 대상이기
 * 때문이다. 여기서 이진 탐색은 더 빠르고 *틀리다*. 정렬된 구간에서만 써야 하는데,
 * 정렬되지 않은 자료에서는 "없다" 를 돌려주는 것이 아니라 헛소리를 돌려준다. 옳은
 * 도구는 선형 탐색이고, 이 크기의 묶음에서 O(n) 은 아무것도 아니다. */
ex_event_t key = { .client = 9, .code = 0 };
const ex_event_t *found = (const ex_event_t *)proven_array_linear_search(&events, &key, by_client);
EXAMPLE_REQUIRE(found != NULL, "client 9 appears in the batch");
EXAMPLE_REQUIRE(found->code == 200, "and linear search returns the FIRST match in order");

ex_event_t absent = { .client = 42, .code = 0 };
EXAMPLE_REQUIRE(proven_array_linear_search(&events, &absent, by_client) == NULL,
                "a client that never appeared is reported as not found");

/* --- 3. 가장 최근 이벤트를 담는 고리 ---------------------------------- */

proven_result_ring_t rr = PROVEN_RING_INIT(alloc, ex_event_t, 4);
EXAMPLE_REQUIRE(proven_is_ok(rr.err), "creating the recent-events ring must succeed");
proven_ring_t recent = rr.value;
EXAMPLE_REQUIRE(proven_ring_is_valid(&recent), "a created ring must be structurally valid");

proven_ring_t unset = {0};
EXAMPLE_REQUIRE(!proven_ring_is_valid(&unset), "a zero-initialised ring handle is not usable");

/* 이 고리는 가득 차면 덮어쓰는 대신 거부한다. 그래서 "최근 넷을 남긴다" 는 것은
 * 넣기 전에 가장 오래된 것을 우리가 직접 버린다는 뜻이다. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (recent.len == recent.cap) {
        ex_event_t dropped;
        err = PROVEN_RING_POP(&recent, ex_event_t, &dropped);
        EXAMPLE_REQUIRE(proven_is_ok(err), "a full ring must yield its oldest element");
    }
    err = PROVEN_RING_PUSH(&recent, ex_event_t, batch[i]);
    EXAMPLE_REQUIRE(proven_is_ok(err), "pushing after making room must succeed");
}
EXAMPLE_REQUIRE(recent.len == 4, "the ring holds the four most recent events");

/* --- 4. 클라이언트별로 세기, 고칠 때마다 찾기는 한 번 ----------------- */

/* create_with_capacity 는 proven_map_create 인데, 용량 인자가 왜 거기 있는지 말해
 * 주는 이름을 달았다. 지금 크기를 잡아 두면 나중의 재해싱을 피한다. 재해싱은 모든
 * 버킷을 복사하고, 전에 get_mut 이 돌려준 모든 포인터를 무효로 만든다. */
proven_result_map_t mr = proven_map_create_with_capacity(alloc, 8, PROVEN_KEY_TYPE_INT,
                                                         sizeof(proven_u32), alignof(proven_u32));
```

```
EXAMPLE_REQUIRE(proven_is_ok(mr.err), "creating the counter map must succeed");
proven_map_t counts = mr.value;
EXAMPLE_REQUIRE(proven_map_is_valid(&counts), "a created map must be structurally valid");

err = proven_map_reserve(&counts, 32);
EXAMPLE_REQUIRE(proven_is_ok(err), "reserving map capacity must succeed");

for (proven_size_t i = 0; i < batch_len; ++i) {
    proven_map_key_t k = { .id = batch[i].client };

    /* get_mut 은 맵 저장소 *안*을 가리키는 포인터를 돌려준다. 그래서 계수기는 그것이
     * 사는 자리에서 올라간다 - 찾기 한 번, 되쓰는 복사 없음. 그 포인터는 다음 삽입
     * 전까지만 쓸 수 있고, 위에서 용량을 미리 잡아 둔 또 하나의 이유가 그것이다. */
    proven_u32 *seen = (proven_u32 *)proven_map_get_mut(&counts, k);
    if (seen) {
        *seen += 1;
    } else {
        proven_u32 one = 1;
        err = proven_map_set(&counts, k, &one);
        EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a new client must succeed");
    }
}

const proven_u32 *seven = PROVEN_MAP_GET_INT(&counts, proven_u32, 7);
EXAMPLE_REQUIRE(seven != NULL && *seven == 3, "client 7 sent three events");

/* 닫힘 이벤트(코드 0)는 그 클라이언트가 갔다는 뜻이다. 계수기를 버린다. */
for (proven_size_t i = 0; i < batch_len; ++i) {
    if (batch[i].code == 0) {
        err = proven_map_remove(&counts, (proven_map_key_t){ .id = batch[i].client });
        EXAMPLE_REQUIRE(proven_is_ok(err), "removing a present key must succeed");
    }
}
EXAMPLE_REQUIRE(PROVEN_MAP_GET_INT(&counts, proven_u32, 7) == NULL, "the closed client is gone");

/* --- 5. 어느 해시인가, 그리고 그 차이를 보는 법 ---------------------- */

/* 같은 키를 담은 문자열 키 맵 둘. 기본 쪽은 키가 있는 SipHash 로 해싱하므로, 키를
 * 고르는 공격자도 그것들을 한 버킷으로 몰아넣을 수 없다. 믿는 쪽은 빠른 FNV-1a 를
 * 쓰고, 모든 키를 여러분 코드가 고를 때*만* 옳은 선택이다. */
proven_result_map_t untrusted = PROVEN_MAP_INIT_U8_BORROWED(alloc, proven_u32, 8);
EXAMPLE_REQUIRE(proven_is_ok(untrusted.err), "creating the default string-key map must succeed");
proven_map_t from_network = untrusted.value;

proven_result_map_t tr = proven_map_create_trusted(alloc, 8, PROVEN_KEY_TYPE_U8_BORROWED,
                                                   sizeof(proven_u32), alignof(proven_u32));
EXAMPLE_REQUIRE(proven_is_ok(tr.err), "creating the trusted-key map must succeed");
proven_map_t internal = tr.value;

EXAMPLE_REQUIRE(from_network.trusted_keys == false, "the default map defends against chosen keys");
EXAMPLE_REQUIRE(internal.trusted_keys == true, "the trusted map opts out of that defence");

/* map_hash 는 맵이 실제로 키를 놓을 때 쓰는 값을 드러낸다. 그래서 이 선택이 믿고
 * 받아들이는 것이 아니라 눈으로 볼 수 있는 것이 된다. */
proven_map_key_t name = { .str = PROVEN_LIT("user-agent") };
proven_u64 keyed = proven_map_hash(&from_network, name);
proven_u64 fast  = proven_map_hash(&internal, name);
EXAMPLE_REQUIRE(keyed != fast, "the same key hashes differently under the two functions");
EXAMPLE_REQUIRE(proven_map_hash(&internal, name) == fast, "and each function is deterministic");
```

```
    /* 맵이 소유하지 않은 기억에 사는 키는 *빌린* 것이다. 그 바이트가 맵보다 오래 살아야
     * 한다. 여기서는 문자열 리터럴이니 그렇다. 곧 다시 쓸 버퍼에서 키가 온다면 대신
     * 소유하는 키 맵(PROVEN_MAP_INIT_U8_OWNED / proven_map_set_u8_owned)을 쓸 것.
     * 그쪽은 복사한다. */
    proven_u32 hits = 1;
    err = proven_map_set(&from_network, name, &hits);
    EXAMPLE_REQUIRE(proven_is_ok(err), "inserting a borrowed string key must succeed");
    EXAMPLE_REQUIRE(PROVEN_MAP_GET_U8_BORROWED(&from_network, proven_u32, PROVEN_LIT("user-agent")) != NULL,
                    "and it can be looked up by an equal view, not the same pointer");

    /* --- 6. 흘러가는 자료를 토막으로 검사합 ------------------------------ */

    /* 묶음은 지나가는 동안 검사합이 구해진다. 파일이나 소켓을 읽는 프로그램이 해야 하는
     * 일이 그것이다 - 전체를 한꺼번에 쥐지 않는다. 흐르는 값을 0 에서 시작해 토막을 하나씩
     * 먹이면, 마지막 값은 이어 붙인 것을 한 번에 부른 결과와 같다. */
    proven_u32 running = 0;
    for (proven_size_t i = 0; i < batch_len; ++i) {
        proven_mem_view_t chunk = { .ptr = (const proven_byte_t *)&batch[i], .size = sizeof batch[i] };
        running = proven_crc32_update(running, chunk);
    }
    proven_mem_view_t whole = { .ptr = (const proven_byte_t *)batch, .size = sizeof batch };
    EXAMPLE_REQUIRE(running == proven_crc32(whole),
                    "chunked and whole-buffer CRC-32 must agree, whatever the chunking");

    printf("events=%zu recent=%zu crc32=%08x\n",
           (size_t)events.len, (size_t)recent.len, (unsigned)running);

    proven_map_destroy(&internal);
    proven_map_destroy(&from_network);
    proven_map_destroy(&counts);
    PROVEN_RING_DESTROY(&recent);
    PROVEN_ARRAY_DESTROY(&events);
    return EXAMPLE_OK();
}
```

반례 — 도착 순서 로그에 이진 탐색을 쓰는 경우:

```
const ex_event_t *e = proven_array_binary_search(&events, &key, by_client);   /* wrong */
```

`proven_array_binary_search()`는 같은 비교 함수로 정렬된 구간에서만 쓸 수 있다. 정렬되지 않은 입력에서 이 함수는 실패하지 않는다. 반씩 좁혀 가다 그 자리에 있던 원소를 답이라고 내놓거나, 찾는 원소가 두 칸 옆에 있는데도 없다고 말한다.

반례 — 삽입을 사이에 두고 `get_mut` 포인터를 붙들고 있는 경우:

```
proven_u32 *seen = proven_map_get_mut(&counts, k);
proven_err_t e = proven_map_set(&counts, other_key, &one);   /* may rehash */
*seen += 1;                                                  /* wrong: may dangle */
```

삽입할 수 있는 무언가를 사이에 두고 붙들 것은 포인터가 아니라 키다.

반례 — 프로그램 밖에서 온 키에 빠른 해시를 쓰는 경우:

```
proven_result_map_t m = proven_map_create_trusted(alloc, 64, PROVEN_KEY_TYPE_U8_BORROWED,
                                                  sizeof(session_t), alignof(session_t));
/* keys are HTTP header names taken from the request */   /* wrong */
```

그것이 바로 키가 있는 해시가 막으려는 경우다. 키를 나 아닌 누군가가 고른다면 `proven_map_create()`를 쓴다.

Chapter 8: Formatting and Scanning (v0.92.0)

이 장은 `fmt.h`와 `scan.h`에 대한 상세 레퍼런스다. Chapter 3은 더 짧은 개요와 일상적인 예제를 제공한다. 이 장은 정확한 문법, 파라미터 형태, 반환값, 그리고 호출자가 흔히 실수하는 지점에 초점을 맞춘다.

목차

1. Design model
2. Formatter data model
3. Formatter constructors and selectors
4. Format string grammar
5. Formatting APIs

5.1. Formatting a type of your own

1. Console print helpers
2. Scanner data model
3. Scanner primitive APIs
4. Scan argument model
5. Structural scan grammar
6. Scan formatting APIs

11.1. Scan error code guide and recovery

1. Examples and misuse cases
2. Freestanding and build-mode notes

1. 설계 모델

타입을 두 번 쓰지 않는 이유

이 장의 두 절반은 모두 한 가지를 없애기 위해 존재한다. 프로그래머가 타입을 말하는데 컴파일러가 그것을 값과 대조해 볼 수 없는 자리다.

`printf("%d", x)`는 타입을 두 번 말한다 — 한 번은 `%d`로, 한 번은 `x`를 넘기면서 — 그리고 varargs가 두 번째를 지워버리므로 아무것도 둘을 비교할 수 없다. `scanf("%d", &x)`는 더 나쁘다. 포맷이 어떻게 파싱할지 그리고 포인터를 통해 무엇을 쓸지를 둘 다 결정하므로, 불일치는 헛소리를 출력하는 대신 메모리를 망가뜨린다. 둘은 반대 방향에서 온 같은 결함이고, 둘 다 조용히 컴파일된다.

여기서 플레이스홀더는 **타입을 전혀 담지 않는다**. `{}`는 위치를 표시할 뿐이고, 타입은 포매팅 쪽에서는 `PROVEN_ARG(x)`, 스캐닝 쪽에서는 `PROVEN_SCAN_ARG(&x)`에서 오며, 둘 다 인자의 정적 타입에 대해 컴파일 시점에 `_Generic`으로 결정된다. `:` 뒤의 spec은 표현만 제어한다 — width, fill, 정렬, 정밀도, 진법 — 결코 해석을 제어하지 않는다. 문자열에 타입을 쓴 적이 없으므로 `%d`냐 `double`이냐를 틀릴 여지가 없다.

두 번째 설계 결정은 **어느 쪽도 버퍼를 소유하지 않는다**는 것이다. 포매팅은 여러분이 제공한 목적지에 덧붙이고 들어가지 않으면 거부한다. 스캐닝은 여러분이 제공한 뷰(view)에서 읽고 여러분이 읽을 수 있는 커서를 옮긴다. 할당자(allocator)를 건네지 않는 한 여기서는 아무것도 할당하지 않으며, 바로 그 점이 이 장 전체가 프리스탠딩(freestanding) 빌드에서 동작하게 만든다(§13).

포매팅 측면과 스캐닝 측면은 정반대의 문제를 해결한다.

- 포매팅은 타입이 있는 값을 받아 텍스트로 렌더링한다.
- 스캐닝은 텍스트를 받아 타입이 있는 값을 기록한다.

이 프로젝트는 양쪽 모두 의도적으로 작게 유지한다.

- 포매팅은 간결한 플레이스홀더 언어, positional 재사용, 단순한 정렬(alignment), width, 그리고 숫자 값에 대한 hex 렌더링을 지원한다.
- 스캐닝은 타입이 있는 목적지 포인터, 엄격한 플레이스홀더 개수 검사, 16진 정수를 위한 `{:x}`, 그리고 공백 축약(whitespace collapsing)을 포함한 리터럴 매칭을 지원한다.

- 어느 쪽도 완전한 printf나 scanf 복제를 목표로 하지 않는다.

실용적인 결과로, 이 API들은 대규모 범용 포맷 엔진보다 추론하기 쉬우면서도, 문법은 일반적인 시스템 코드 작업에 충분히 표현력이 있다.

2. 포매터 데이터 모델

proven_fmt_result_t

```
typedef struct {
proven_err_t  err;
proven_size_t written;
proven_size_t required;
} proven_fmt_result_t;
```

의미:

- err: 연산의 상태 코드.
- written: 목적지에 실제로 기록된 바이트 수.
- required: 전체 포맷 출력에 필요한 총 바이트 수.

먼저 err를 사용하라. 나머지 필드는 잘림(truncating)이나 부분적으로 성공한 연산에서 가장 유용하다.

성공한 result는 다음과 같은 모습이다.

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "hello {}",
        PROVEN_ARG("world")
    );
    if (proven_is_ok(r.err)) {
        proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}
```

잘림이 발생한 result도 얼마나 더 많은 공간이 필요했는지는 여전히 알려줄 수 있다. 여기서는 목적지를 일부러 너무 작게 만들었으므로 written이 required에 못 미치게 되며, 그럼에도 문자열은 여전히 유효하다.

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 8);   /* on purpose: too small */
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_trunc(
        &s,
        "name={} score={}",
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    /* r.written is what fit; r.required is what the whole output needed. */
    proven_println("wrote {} of {} bytes",
                   PROVEN_ARG(r.written), PROVEN_ARG(r.required));

    proven_u8str_destroy(alloc, &s);
}
```

proven_arg_type_t

```
typedef enum {
PROVEN_ARG_NONE,
```

```
PROVEN_ARG_I32,
PROVEN_ARG_U32,
PROVEN_ARG_I64,
PROVEN_ARG_U64,
#ifndef PROVEN_FMT_NO_FLOAT
PROVEN_ARG_F64,
#endif
PROVEN_ARG_CSTR,
PROVEN_ARG_STR_VIEW,
PROVEN_ARG_DATETIME,
PROVEN_ARG_PTR,
PROVEN_ARG_FN,
} proven_arg_type_t;
```

포매터는 현재 다음 값 클래스들을 인식한다.

- signed 32비트 정수
- unsigned 32비트 정수
- signed 64비트 정수
- unsigned 64비트 정수
- 부동소수점 값 (`PROVEN_FMT_NO_FLOAT`가 정의되지 않은 경우)
- 신뢰할 수 있는 C 문자열
- 빌려 쓰는(borrowed) U8 문자열 뷰
- datetime
- 객체 포인터
- 함수 포인터

`proven_arg_t`

```
typedef struct {
proven_arg_type_t type;
union {
proven_i32 i32;
proven_u32 u32;
proven_i64 i64;
proven_u64 u64;
double f64;
const char *cstr;
proven_u8str_view_t str_view;
proven_datetime_t datetime;
const void *ptr;
void (*fn)(void);
} value;
} proven_arg_t;
```

union 필드는 선택된 `type`과 일치해야 한다. union 필드를 잘못 채워 놓고 포매터가 알아서 추측해 주기를 바라며 `proven_arg_t`를 억지로 만들지 마라.

잘못된 예:

```
proven_arg_t arg = {0};
arg.type = PROVEN_ARG_I64;
arg.value.u64 = 123; /* wrong: type and union field do not match */
```

올바른 예:

```
proven_arg_t arg = proven_arg_i64(123);
(void)arg;   /* pass it to a formatting macro; PROVEN_ARG(arg) accepts it as-is */
```

3. 포매터 생성자와 선택자

생성자 요약

API	파라미터	반환	의도
`proven_arg_none(void)`	없음	`proven_arg_t`	내부 sentinel 값.
`proven_arg_i32(int v)`	signed 정수	`proven_arg_t`	32비트 signed 정수로 렌더링.
`proven_arg_u32(unsigned int v)`	unsigned 정수	`proven_arg_t`	32비트 unsigned 정수로 렌더링.
`proven_arg_i64(long long v)`	넓은 signed 정수	`proven_arg_t`	64비트 signed 정수로 렌더링.
`proven_arg_u64(unsigned long long v)`	넓은 unsigned 정수	`proven_arg_t`	64비트 unsigned 정수로 렌더링.
`proven_arg_f64(double v)`	부동소수점 값	`proven_arg_t`	float 포매팅이 비활성화되지 않은 한, 부동소수점 텍스트로 렌더링.
`proven_arg_cstr(const char *v)`	신뢰할 수 있는 live C 문자열	`proven_arg_t`	NUL로 종료되는 C 문자열을 렌더링.
`proven_arg_cstr_n(const char *v, proven_size_t max_len)`	경계가 있을 수 있는 C 문자열	`proven_arg_t`	NUL을 찾되 `max_len`까지만 렌더링.
`proven_arg_str_view(proven_u8str_view v)`	빌려온 U8 뷰	`proven_arg_t`	NUL 종료를 가정하지 않고 빌려온 뷰를 렌더링.
`proven_arg_datetime(proven_datetime_t v)`	datetime 값	`proven_arg_t`	포매터의 datetime 규칙으로 datetime을 렌더링.
`proven_arg_ptr(const void *v)`	객체 포인터	`proven_arg_t`	포인터 값을 렌더링.
`proven_arg_fn(void (*v)(void))`	함수 포인터	`proven_arg_t`	원시 함수 포인터 표현을 렌더링.
`proven_arg_ucstr(const unsigned char *v)`	unsigned-char 문자열	`proven_arg_t`	`proven_arg_cstr`에 대한 편의 래퍼.
`proven_arg_identity(proven_arg_t v)`	기존 argument 객체	`proven_arg_t`	통과(pass-through) 헬퍼.
`proven_arg_bool(bool v)`	boolean	`proven_arg_t`	`1` / `0`이 아니라 `true` / `false`를 단어로 렌더링.
`proven_arg_char(char v)`	문자 하나	`proven_arg_t`	**문자**를 렌더링. 이것이 바로 `char` 변수는 문자로 렌더링되는데 리터럴 `PROVEN_ARG('Z')`는 여전히 `90`으로 렌더링되는 이유다. C에서 `'Z'`는 `int` 타입이며, 어떤 `_Generic`으로도 이것을 숫자 90과 구별할 수 없다.
`proven_arg_custom(const void *v, proven_fmt_custom_fn fn)`	어떤 타입이든	`proven_arg_t`	라이브러리가 전혀 알지 못하는 타입을, 당신이 제공하는 함수를 통해 렌더링. Formatting a user-defined type 참조.

PROVEN_ARG(x)

PROVEN_ARG(x)는 통상적인 진입점이다. _Generic을 사용하여 컴파일러가 x의 타입으로부터 생성자를 고른다.

현재 매핑은 다음과 같다.

- _Bool, char, signed char, short, int -> proven_arg_i32
- unsigned char, unsigned short, unsigned int -> proven_arg_u32
- long, long long -> proven_arg_i64
- unsigned long, unsigned long long -> proven_arg_u64
- double, float -> proven_arg_f64 (PROVEN_FMT_NO_FLOAT가 정의되지 않은 경우)
- const char *, char * -> proven_arg_cstr
- unsigned char *, const unsigned char * -> proven_arg_ucstr
- void *, const void * -> proven_arg_ptr
- proven_u8str_view_t -> proven_arg_str_view
- proven_datetime_t -> proven_arg_datetime
- proven_arg_t -> proven_arg_identity

중요한 결과:

- PROVEN_ARG는 함수 포인터를 선택하지 않는다.
- 함수 포인터에는 PROVEN_ARG_FN(f)를 사용하라.

잘못된 예:

```
void helper(void) {}
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(helper)); /* wrong */
```

올바른 예:

```
void helper(void);   /* whatever function you want to print the address of */

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &rs.value, "{}",
                                                         PROVEN_ARG_FN(helper));
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

PROVEN_ARG_FN(f)

이 매크로는 호출자가 함수 포인터를 void *로 캐스팅하지 않고도 넘길 수 있도록 존재한다. proven_arg_fn을 감싼 작은 안전 래퍼다.

예:

```
void helper(void);

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "callback = {}",
        PROVEN_ARG_FN(helper)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        proven_eprintln("formatting the callback failed");
    }
```

```
    proven_u8str_destroy(alloc, &s);
}
```

PROVEN_ARG_CSTR_N(v, max_len)
이 매크로는 경계가 있는 문자열 헬퍼다. 소스가 완전히 신뢰할 수 있는 C 문자열이 아닐 수 있지만 여전히 C 문자열과 유사한 입력 처리를 원할 때 사용하라.

이 매크로는 max_len까지만 NUL을 탐색한 뒤 경계 지어진 접두부를 뷰로서 포매팅한다.

좋은 사용 사례 - buf가 통신선(wire)에서 들어왔으므로 NUL 종료가 전혀 보장되지 않는 경우:

```
char buf[128];                      /* filled from an untrusted source */
(void)proven_mem_copy(buf, sizeof buf, proven_mem_view_from_u8(PROVEN_LIT("payload")));

proven_result_u8str_t rs = proven_u8str_create(alloc, 64);
if (proven_is_ok(rs.err)) {
    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &rs.value,
        "payload={}",
        PROVEN_ARG_CSTR_N(buf, sizeof buf)   /* looks for NUL only within 128 bytes */
    );
    (void)r;
    proven_u8str_destroy(alloc, &rs.value);
}
```

나쁜 사용 사례:

```
const char *buf = get_network_buffer();
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(buf)); /* wrong if buf is not trusted */
```

부동소수점 형식화 참고
PROVEN_FMT_NO_FLOAT가 정의되면 float 지원은 제네릭 셀렉터에서 제거되고 float 생성자는 사용할 수 없다. 이는 런타임 토글이 아니라 컴파일 시점의 구성 선택이다.

double의 기본 {} 렌더링은 유한 값에 대해 소수점 이하 6자리 고정 형식을 만들며, 크기가 간결한 형식에 비해 너무 크거나 너무 작으면 과학적 표기(scientific)로 전환된다. 이전 버전과 달리, 이 출력은 이제 **정확하고 정수만 사용하는** 엔진으로 계산된다(double/long double 근사 없음). 자릿수는 정확히 반올림되므로(round-half-to-even), {}는 같은 값에 대해 printf("%.6f") / %.6e가 출력하는 것과 일치한다. 최단(shortest) 라운드트립 형식이나 6이 아닌 다른 precision이 필요하면 아래에 설명하는 proven_float_format_* 정책 API를 사용하라.

정확도와 한계
- 기본 {} float 출력은 소수점 이하 6자리를 사용하며, 가장 가까운 값으로 정확히 반올림하되 동점(tie)은 짝수로 처리한다(glibc %.6f/%.6e와 일치). 어떤 크기에서도 정확하며, 구성 가능한 큰 정수(big-integer) 용량을 넘어서는 precision/크기 한계는 없다.
- 라운드트립 직렬화에는 **최단(shortest)** 정책 (proven_float_format_options_shortest())을 사용하라. 이는 다시 파싱하면 정확히 같은 값이 되는 가장 짧은 십진 문자열을 방출한다.
- 십진수-double 스캐닝은 IEEE-754 binary64로, 가장 가까운 값으로 반올림하되 동점은 짝수로 처리하도록 정확히 반올림되며, 호스트의 strtod와 비트 단위로 일치한다. 가장 작은 subnormal에 대한 중간 임계값 아래의 값들은 입력 부호를 보존한 채 부호 있는 0으로 반올림된다.
- 파서는 Clinger fast path -> Eisel-Lemire -> 정확한 big-integer 폴백으로 계층화되어 있다. 모든 계층이 정확하고 폴백이 최종 판정자이므로, 결과는 모든 입력에 대해 정확히 반올림된다. 캐시된 거듭제곱 테이블은 생성된 소스다 (scripts/generate_float_decimal_tables.py).
- 정확 폴백 big-integer 용량은 임베디드 타겟을 위해 PROVEN_FLOAT_BIGINT_LIMBS(include/proven/float_config.h 참조)로 조정할 수 있다. fast path는 big integer를 절대 건드리지 않는다.

- 검증: 포매터와 파서는 40.8억 개의 모든 유한 `binary32` 값과 25.6억 개의 무작위 `binary64` 값에 대해 호스트 C 라이브러리와 남김없이 대조 검사되었으며, 불일치가 하나도 없었다. 알고리즘, 방법론, glibc와의 벤치마크는 `docs/float-correctness-and-performance.md`를 참조하라.

엔진 내부(개념적으로)

API를 사용하는 데 이 내용이 전혀 필요하지 않다. 이는 출력이 왜 신뢰할 수 있는지 알고 싶은 독자를 위한 것이다. 전체 내용은 `docs/float-correctness-and-performance.md`에 있다.

파싱(십진수 → binary64), 세 계층, 빠른 것 우선. 결과는 항상 정확히 반올림된다. 계층은 순전히 속도의 계단일 뿐이며, 각 계층은 정확한 답을 보장할 수 있을 때만 선택된다.

1. Clinger fast path. 값의 유효 자릿수가 적고 지수가 작을 때, 유효숫자(significand)와 `10^exp` 모두 `double`로 정확히 표현 가능하므로, 단 한 번의 반올림된 곱셈/나눗셈이 증명 가능하게 올바르다. 일상적인 대부분의 숫자를 커버한다.
2. Eisel-Lemire. 캐시된 10의 거듭제곱과의 64×128비트 고정소수점 곱셈이며, 결과가 반올림 경계로부터 충분히 멀어 확실한지를 검사한다. 검사가 불확실하면(값이 중간의 동점 지점에 놓이면) 다음 계층으로 넘어간다.
3. 정확한 big-integer 폴백. 값을 큰 정수들의 비(`significand`와 `5^q/2^q`)로 구성하고, 후보 `double` 및 그 이웃과 정확히 비교한다. 이것이 동점과 subnormal을 올바르게 만드는 최종 판정자다. 시드된 ±16-ULP 윈도우가 탐색을 몇 번의 비교로 유지한다. big-integer 용량은 `PROVEN_FLOAT_BIGINT_LIMBS`로 제한된다. 이 계층만이 limb를 (스택에) 할당하며, fast path는 여기에 절대 도달하지 않는다.

포매팅(binary64/32 → 십진수). 두 엔진, 그 어디에도 `long double` 없음:

- 최단(shortest) (`proven_float_format_options_shortest()`): **Grisu3** fast path가 거의 모든 입력에 대해 최소한의 라운드트립 자릿수를 ~90 ns에 만든다. Grisu3가 최소성을 증명할 수 없는 드문 경우에는 정확한 **Dragon4**(Burger–Dybvig, round-to-even) 코어로 폴백한다. 결과는 같은 비트로 다시 파싱되는 유일한 최단 십진수다.
- 고정 `%f` / 과학적 `%e` (기본 `{}`와 고정 옵션): 정확한 정수 엔진이 값을 big-integer `mul_pow5`/시프트로 `10^precision`만큼 스케일링하고, 정수 `divmod`를 수행한 뒤, round-half-to-even으로 반올림한다. 따라서 어떤 precision과 크기에서도 glibc와 일치하며, `2^64`/precision 한계가 없다. 극단적인 지수는 실제 임의정밀도(arbitrary-precision) 연산을 하며 그만큼 느리다(실무에서는 드물다).

공개 부동소수점 파싱 API

세 진입점이 하나의 정확히 반올림되는 백엔드를 공유한다.

- `proven_scan_f64(scan)` — `proven_scan_t` 커서로부터 파싱하며, 실패 시 커서를 복원한다. 네이티브하고 길이로 경계 지어진 경로(NUL 종료 불필요).
- `proven_parse_double_ascii(view)` — `proven_u8str_view_t`로부터 로케일 독립적인 ASCII 토큰 하나를 파싱하고 소비한 길이를 보고한다.
- `proven_strtod(nptr, endptr)` — C 문자열 위의 `strtod` 스타일 편의 래퍼로, 선행 ASCII 공백을 건너뛰고 `endptr`을 보고한다.

실전 예제: 파싱

```c
#include "proven/scan.h"
#include "proven/float_parse.h"

/* (1) Native, view-based parsing through a scanner cursor. */
proven_scan_t sc = proven_scan_init(proven_u8str_view_from_cstr("3.14159e2 rest"));
proven_result_f64_t r = proven_scan_f64(&sc);
if (r.err == PROVEN_OK) {
    /* r.val == 314.159; the cursor now sits at " rest". */
    proven_println("parsed {}", PROVEN_ARG(r.val));
}

/* (2) strtod-style wrapper for C strings. endptr reports where parsing stopped. */
char *end = NULL;
```

```c
double v = proven_strtod("  -0.5\t", &end);   /* v == -0.5, *end == '\t' */
(void)v;

/* A trailing exponent marker with no digits stops like strtod: "1e" parses 1,
   leaving endptr at 'e'. Inputs with hundreds of significant digits and extreme
   exponents are still rounded correctly via the exact fallback. */
```

실전 예제: 정책 API로 형식화하기

`proven_float_format_f64_policy` / `_f32_policy`는 호출자 버퍼에 직접 기록하고 기록한 바이트 수를 보고한다. 이들은 절대 할당하지 않는다.

```c
#include "proven/float_format.h"

char buf[64];
proven_size_t n = 0;

/* Shortest round-trippable form: 0.1 -> "0.1" (not "0.10000000000000001"). */
(void)proven_float_format_f64_policy(buf, sizeof buf, 0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(), &n);
/* buf == "0.1", n == 3 */

/* Fixed precision (correctly rounded, round-half-to-even). */
proven_float_format_options_t opt = proven_float_format_options_fixed_default();
opt.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 3.14159,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, opt, &n);
/* buf == "3.14" */

/* Always scientific - this is what {:e} selects. Six fractional digits by default,
   a signed two-digit-minimum exponent: exactly what printf's %e prints. */
proven_float_format_options_t sci = proven_float_format_options_scientific();
sci.precision = 2;
(void)proven_float_format_f64_policy(buf, sizeof buf, 42.0,
    PROVEN_FLOAT_FORMAT_POLICY_DEFAULT, sci, &n);
/* buf == "4.20e+01" - where fixed would give "42.00" and shortest "42" */
```

- `PROVEN_FLOAT_FORMAT_POLICY_RYU`는 최단 출력을 선택한다. `DEFAULT`/`SIMPLE`은 정확한 고정 precision 경로(`%f`, 아주 크거나 작은 크기에서는 `%e`로 전환)를 선택한다.
- 버퍼가 너무 작으면 `PROVEN_ERR_OUT_OF_BOUNDS`를 반환하며(값은 절대 소리 없이 잘리지 않는다), 지원되지 않는 정책에는 `PROVEN_ERR_INVALID_ARG`를 반환한다.
- 제네릭 `{}` 포매터(`proven_u8str_append_fmt*`)는 내부적으로 `DEFAULT` 정책을 사용하므로, 일상적인 로깅에서는 이 API를 직접 호출할 필요가 없다.

4. 형식 문자열 문법

포매터는 의도적으로 작은 문법을 받아들인다.

치환 필드

지원되는 형식:

- `{}`: 다음 positional argument
- `{0}`: 첫 번째 사용자 argument
- `{1}`: 두 번째 사용자 argument
- `{2}`: 세 번째 사용자 argument
- 이하 계속

번호 매김은 사용자 대상이며 0부터 시작한다. 구현은 인덱스 0에 숨겨진 sentinel을 저장하고, 사용자 인덱스 `0`을 내부 argument 슬롯 `1`로 매핑한다.

중괄호 이스케이프

- {{는 리터럴 {가 된다
- }}는 리터럴 }가 된다

레이아웃 명세

```
{:[[fill]align][sign][#][0][width][.precision][type]}
```

모든 부분은 선택적이며, 순서는 나머지 세계가 사용하는 것과 같다. 따라서 Python이나 Rust에서 복사한 스펙은 여기서도 그곳과 같은 의미를 가진다.

부분	값	하는 일
`align`	`< > ^`	왼쪽, 오른쪽, 가운데. 기본값 `>`.
`fill`	align 앞에 오는 임의의 문자	패딩 문자. 기본값은 공백.
`sign`	`+` 또는 공백	음수가 아닌 수에 부호를 강제하거나, 부호 자리를 하나 예약한다.
`#`		대체 형식(alternate form): `0x`, `0X`, `0o`, `0b` 접두어. 정수 전용.
`0`		0 채움. `42`에 대한 `{:08}`은 `00000042`.
`width`	숫자, 최대 10000	최소 필드 width.
`.precision`	`.N`, 최대 60	소수 자릿수. **float 전용.**
`type`	`x X o b d`(int), `f g e`(float)	기수(base)와 대소문자. `f` 고정, `g` 최단 라운드트립, `e` 과학적 표기(printf `%e`).

과거에 거짓이었던 만큼 알아둘 가치가 있는 두 가지가 있다.

- **선행 `0`은 0 채움이지 width의 첫 자리가 아니다.** `{:08}`은 v26.07.12f 전까지 `"      42"`를 만들었다(공백 패딩, 오류 없음). 명시적 fill은 여전히 우선한다: `{:*>08}`은 `*`로 채운다.
- **0 채움은 부호와 자릿수 사이에 들어간다.** `42`에 대한 `{:+08}`은 `+0000042`이며, `0000+42`가 아니다. 패딩은 숫자의 일부이고, 숫자의 부호가 먼저 온다.

부동소수점

`{}`는 정확히 반올림된 소수점 이하 6자리를 준다. `{:.3}`은 3자리, `{:.0}`은 0자리를 주고, `{:e}`는 과학적 표기를 강제한다 - 가수(mantissa), `precision` 소수 자릿수(기본 6자리), 부호 있는 최소 두 자리 지수, 정확히 반올림, 정확히 printf의 `%e`와 같다 - 이는 `{:f}`와 `{:g}`가 도달하지 못하는 형식이다: `{:f}`는 지수를 절대 보여주지 않고, `{:g}`는 더 짧을 때만 지수를 사용한다. `{:f}`는 고정 형식을 강제하고, `{:g}`는 라운드트립되는 가장 짧은 표현을 준다.

v26.07.12i 전까지 **이 중 어느 것도 존재하지 않았다**: 모든 float은 영원히 정확히 6자리 소수로만 나왔다. 정확 엔진은 언제나 이 모든 것을 할 수 있었으나, `{}` 문법이 그것에 도달하지 못했을 뿐이다. 눈에 보이는 대가는 float 컬럼을 정렬할 수 없다는 것이었다. `12.5`는 9자 폭으로, `100.0`은 10자 폭으로 렌더링되었기 때문이다.

```
proven_byte_t buf[64];
proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);
(void)proven_u8str_append_fmt(&line, "{:>9.2}", PROVEN_ARG(12.5));    /* "    12.50" */
```

인자가 지킬 수 없는 명세는 에러다

double에 대한 `{:x}`, 정수에 대한 `{:.2}`, 정수에 대한 `{:f}`, 문자열에 대한 `{:#}`: 모두 `PROVEN_ERR_INVALID_FORMAT`.

이들은 과거에 *무시*되었다 - double에 대한 `{:x}`는 `3.500000`을 출력하고 성공을 보고했다. 호출자는 무언가를 요청하고, 다른 것을 받았으며, 그것이 제대로 되었다는 말을 들었다. 그것은 있을 수 있는 최악의 결과이며, 거부하는 것보다 나쁘다.

`char` and `bool`

`PROVEN_ARG('Z')`는 `Z`로 렌더링되고, `bool`은 `true` 또는 `false`로 렌더링된다. 둘 다 과거에는 정수 경로를 거쳤으므로, 문자가 `90`으로 출력되었고 문자 하나를 방출할 방법이 아예 없었다 - hex 덤프의 ASCII 컬럼은 별도 버퍼에서 손으로 만들어 문자열로 넘겨야 했다. 이제 대문자 hex 덤프는 한 개의 루프다.

```
proven_byte_t hexbuf[64];
proven_u8str_t hexline = proven_u8str_borrow(hexbuf, sizeof hexbuf);
unsigned char byte = 0xde;
(void)proven_u8str_append_fmt(&hexline, "{:02X} ", PROVEN_ARG((unsigned)byte));
(void)proven_u8str_append_fmt(&hexline, "{}", PROVEN_ARG((char)'.'));
```

5. 형식화 API

`proven_u8str_fmt_internal(...)`

```
proven_fmt_result_t proven_u8str_fmt_internal(
proven_allocator_t alloc,
proven_u8str_t *str,
bool trunc,
const char *fmt,
proven_allocator_t scratch,
const proven_arg_t *args,
proven_size_t args_count
);
```

이것은 내부 포매팅 엔진이다. 사용자 코드는 보통 대신 공개 매크로를 호출해야 한다.

파라미터:

- `alloc`: 문자열이 커져야 할 때 사용되는 할당자
- `str`: 목적지 U8 문자열
- `trunc`: true이면 best-effort 잘림을 허용하고, false이면 atomic(원자적) 동작을 유지
- `fmt`: 포맷 텍스트
- `scratch`: 필요할 때 임시 alias 패칭에 사용되는 할당자
- `args`: 인덱스 0의 숨겨진 sentinel을 포함한 포맷 argument 배열
- `args_count`: sentinel을 포함한 `args`의 총 길이

반환값:

- `proven_fmt_result_t`

중요한 규칙:

- `args_count`는 플레이스홀더 개수에 숨겨진 sentinel을 더한 값과 일치해야 한다
- 사용되지 않는 여분의 argument는 오류다
- 누락된 argument는 오류다
- 엔진이 목적지 문자열과 빌려온 뷰 argument 사이의 aliasing을 감지하면, 실패 atomicity를 보존하기 위해 임시 작업용(scratch) 할당자를 사용할 수 있다

`proven_u8str_append_fmt(str, fmt, ...)`

고정 용량 문자열로의 atomic(원자적) 포매팅. 결과가 들어맞지 않으면, 함수는 실패를 보고하고 목적지를 변경하지 않는다.

전부 아니면 전무(all-or-nothing) 동작을 원할 때 사용하라.

`proven_u8str_append_fmt_trunc(str, fmt, ...)`

Best-effort 포매팅. 들어맞는 만큼 기록하고, 얼마나 기록되었는지와 얼마나 필요했는지를 보고한다.

부분 출력이 허용될 때 사용하라.

proven_u8str_append_fmt_grow(alloc, str, fmt, ...)
성장 가능한(growable) 포매팅. 제공된 할당자를 통해 목적지 문자열을 재할당할 수 있다. 할당 실패 시, 기존 문자열은 유효하게 유지된다.

수동 용량 계획 없이 출력을 들어맞게 하고 싶을 때 사용하라.

proven_u8str_append_fmt_with_scratch(alloc, str, fmt, scratch, ...)
별도의 scratch 할당자를 사용하는 성장 가능한 포매팅. argument 목록에 목적지 버퍼와 alias할 수 있는 문자열 뷰가 있어 임시 패칭이 필요할 때 유용하다.

alloc과 scratch 모두에 실제 할당자를 사용하라. 호출에 충분할 만큼 수명이 길지 않은 한, 죽은 아레나(arena)나 일회용 임시 버퍼를 넘기지 마라.

부동소수점 형식화 정책 이음매
공개 float 정책 헤더는 float 포매팅을 위한 명시적 정책 레이어를 제공한다. 의도적으로 작으며, 정확한 고정 precision 포매터를 기본 경로로 유지한다.

주요 진입점은 다음과 같다.

- proven_float_format_f64_policy(...)
- proven_float_format_f32_policy(...)
- proven_float_format_options_fixed_default()
- proven_float_format_options_shortest()

정책 참고 사항:

- PROVEN_FLOAT_FORMAT_POLICY_DEFAULT와 PROVEN_FLOAT_FORMAT_POLICY_SIMPLE은 정확한 고정 precision 출력(정확히 반올림, round-half-to-even)을 선택한다.
- PROVEN_FLOAT_FORMAT_POLICY_RYU는 최단 출력 정책 분기다.
- 정책 API는 지원되지 않는 enum 값에 대해 PROVEN_ERR_INVALID_ARG를 반환한다.
- 정책 API는 호출자가 제공한 버퍼가 너무 작을 때 PROVEN_ERR_OUT_OF_BOUNDS를 반환한다.

예:

```
char buf[128];
proven_size_t written = 0;
proven_err_t err = proven_float_format_f64_policy(
    buf,
    sizeof buf,
    0.1,
    PROVEN_FLOAT_FORMAT_POLICY_RYU,
    proven_float_format_options_shortest(),
    &written
);
if (proven_is_ok(err)) {
    /* buf holds the shortest form that parses back to exactly 0.1 */
    proven_println("{}", PROVEN_ARG_CSTR_N(buf, written));
}
```

PROVEN_FMT_IS_OK(res)
proven_fmt_result_t를 검사하는 작은 헬퍼 매크로. 의도를 간결하게 유지하고 싶을 때 사용하라.

예:

```
proven_result_u8str_t rs = proven_u8str_create(alloc, 32);
if (proven_is_ok(rs.err)) {
    proven_u8str_t s = rs.value;

    proven_fmt_result_t r = proven_u8str_append_fmt_grow(
        alloc,
        &s,
        "name={} score={:0>4}",
```

```
        PROVEN_ARG("ada"),
        PROVEN_ARG(42)
    );
    if (!PROVEN_FMT_IS_OK(r)) {
        /* the string is untouched: grow-mode formatting is failure-atomic */
        proven_eprintln("formatting failed");
    }

    proven_u8str_destroy(alloc, &s);
}
```

콘솔식 헬퍼

`sysio` 레이어는 같은 포매터 기계를 사용하는 print 헬퍼들을 제공한다.

- `proven_print(fmt, ...)`
- `proven_println(fmt, ...)`
- `proven_eprint(fmt, ...)`
- `proven_eprintln(fmt, ...)`

이들은 stdout이나 stderr로 직접 포맷 출력을 원할 때 편리하다. 여전히 `proven_err_t`를 반환하므로, 출력이 중요할 때는 결과를 검사하라.

예:

```
if (!proven_is_ok(proven_println("hello {}", PROVEN_ARG("world")))) {
    /* the write to stdout failed - a closed pipe, a full disk */
    proven_eprintln("stdout is not writable");
}
```

5.1. 내가 만든 타입 형식화하기

`PROVEN_ARG`는 `_Generic` 위에 세워졌고, `_Generic`은 컴파일 시점에 알려준 타입에 대해서만 디스패치할 수 있다. 당신의 타입은 알려줄 수 없다. 그래서 포매터의 argument 집합 — 정수, float, 문자열, 포인터, datetime — 은 **닫힌** 집합이었다: `ex_rect_t`, `uuid_t`, `vec3_t`는 `{}`에 전혀 넘길 수 없었다.

이를 우회하는 두 방법은 모두 나빴다. 값을 scratch 문자열로 미리 포매팅해서 *그것*을 넘기는 방법: 값마다 할당과 복사가 로깅 경로에서 발생하는데, 로깅 경로야말로 할당이 바로 실패한 상황에서도 계속 동작해야 하는 유일한 경로다. 아니면 필드를 하나씩 출력하고 컬럼 정렬은 포기하는 방법.

`PROVEN_ARG_OF(&obj, render)`가 그 문이다.

```
proven_err_t render(proven_fmt_sink_t out, const void *obj);
```

그 시그니처의 형태로부터 세 가지가 따라 나온다.

- **렌더러는 버퍼가 아니라 sink를 받는다.** 렌더러는 공간이 얼마나 있는지 알 필요가 없고, 무엇도 오버플로할 수 없다. `proven_fmt_put`으로 방출한다.
- **합성(compose)된다.** 렌더러는 포매터를 다시 호출할 수 있으며 — 스택 버퍼로, 할당자 없이 — 그 결과를 sink에 넘길 수 있다. 필드 자체가 사용자 타입인 타입도 자연스럽게 중첩된다.
- **Width, fill, alignment가 동작한다.** 포매터는 렌더러를 **두 번** 실행한다: 먼저 counting sink에 대해 출력 폭을 알아내고, 그다음 실제로, 그 주위에 패딩을 적용하여 실행한다. 이것이 `{:>10}`이 당신의 타입 컬럼을 int 컬럼을 정렬하는 것과 똑같이 정렬하는 이유이며, 렌더러가 **결정적(deterministic)**이어야 하고 `obj`를 변경해서는 안 되는 이유다. 두 패스가 불일치하면, 포매터는 정렬된 컬럼에 잘못된 폭의 필드를 방출하고 나중에 알아차리게 하는 대신 `PROVEN_ERR_INVALID_ARG`를 반환한다.

포매터가 **하지 않을** 일은 추측이다. 사각형에 대한 `{:x}`, UUID에 대한 `{:.2}`, 행렬에 대한 `{:+}` — 라이브러리는 그것들이 당신의 타입에 무엇을 의미하는지 전혀 모르므로, `PROVEN_ERR_INVALID_FORMAT`으로 거부한다. 그럴듯한 답을 지어내고 성공을 보고하는 것이야말로 포매터가 거짓말을 시작하는 방식이다. 당신이 요청하지 않은 타입 글자는 오류보다 낫지 않다.

테스트 스위트가 컴파일하고 실행함:

```c
/*
 * 라이브러리가 들어 본 적 없는 타입을 형식화하기.
 *
 * `PROVEN_ARG` 는 `_Generic` 위에 서 있고, 그것은 미리 알려 준 타입에만 갈래를 태울 수
 * 있다 - 그리고 여러분의 타입을 알려 줄 방법이 없다. 그래서 `PROVEN_ARG_OF` 가 생기기
 * 전에는 `ex_rect_t` 를 찍을 방법이 아예 없었다. 임시 문자열에 미리 형식화해서 그것을
 * 건네거나(값마다 할당 한 번과 복사 한 번, 그것도 할당하면 안 되는 그 로그 경로에서),
 * 필드를 하나씩 찍고 열 맞추기는 포기하거나 둘 중 하나였다.
 *
 * 렌더러는 버퍼가 아니라 *그릇*을 받는다. 그것이 이 방식을 조립 가능하게 만든다. 렌더러는
 * 형식화기를 다시 부를 수 있고, 그 출력은 그저 어딘가로 가는 바이트다. 그리고 형식화기가
 * 내보내기 전에 렌더러의 출력을 재기 때문에 - 세는 그릇에 대고 한 번 돌려 본다 - 폭과
 * 채움과 정렬이 사용자 타입에서도 int 에서와 똑같이 먹는다.
 */

typedef struct { int w, h; } ex_rect_t;

static proven_err_t render_rect(proven_fmt_sink_t out, const void *obj) {
    const ex_rect_t *r = (const ex_rect_t *)obj;

    /* 조립: 형식화기를 스택 버퍼로. 할당자는 어디에도 없다. */
    proven_byte_t tmp[64];
    proven_u8str_t s = proven_u8str_borrow(tmp, sizeof tmp);
    proven_fmt_result_t f = proven_u8str_append_fmt(&s, "{}x{}",
                                                    PROVEN_ARG(r->w), PROVEN_ARG(r->h));
    if (!PROVEN_FMT_IS_OK(f)) return f.err;

    return proven_fmt_put(out, proven_u8str_as_view(&s));
}

int main(void) {
    ex_rect_t a = { .w = 1920, .h = 1080 };
    ex_rect_t b = { .w = 640,  .h = 480  };

    proven_byte_t buf[128];
    proven_u8str_t line = proven_u8str_borrow(buf, sizeof buf);

    /* 다른 인자와 똑같다. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&line, "mode={}", PROVEN_ARG_OF(&a, render_rect));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a user type should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line), PROVEN_LIT("mode=1920x1080")),
                    "the renderer's bytes should be what came out");

    /* 그리고 정렬된다. 형식화기가 먼저 재는 이유가 바로 이것이다 - 사용자 정의 값의 열이
     * 다른 무엇의 열과 똑같이 줄을 맞춘다. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "[{:>10}]\n[{:>10}]",
                                PROVEN_ARG_OF(&a, render_rect),
                                PROVEN_ARG_OF(&b, render_rect));
    EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "two user types should format");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&line),
                                         PROVEN_LIT("[ 1920x1080]\n[   640x480]")),
                    "both rows should be right-aligned to the same width");

    /* 여러분의 타입에 대해 라이브러리가 해석할 수 없는 지정자는 추측하지 않고 거부된다.
     * 사각형에 `{:x}` 는 뜻이 없고, 성공했다고 알리면서 그럴듯한 무언가로 답하는 것이
     * 형식화기가 여러분에게 거짓말을 시작하는 방식이다. */
    (void)proven_u8str_reset(&line);
    r = proven_u8str_append_fmt(&line, "{:x}", PROVEN_ARG_OF(&a, render_rect));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_INVALID_FORMAT,
```

```
                "a type letter on a user type should be an error");

    return EXAMPLE_OK();
}
```

6. 콘솔 출력 헬퍼

`proven_println`은 무엇이고, 비용은 얼마인가

`proven_println("{}", PROVEN_ARG(x))`는 프로그램에서 텍스트를 내보내는 가장 짧은 방법이며, 진단 출력이나 일회성 도구, 출력이 몇 줄뿐인 프로그램에는 알맞은 도구다.

루프에는 잘못된 도구인데, 그 이유는 호출 지점에서 보이지 않으므로 명시할 가치가 있다. **호출 하나가 각각 하나의 write 시스템 콜이다.** `proven_println` 만 번 호출은 시스템 콜 만 번이고, 이는 포매팅 자체보다 대략 두 자릿수만큼 비싸다. `printf`는 libc가 대신 flush해 주는 버퍼 뒤에 이것을 숨긴다. 이 라이브러리는 여러분 모르게 버퍼링하지 않는데, 요청하지 않은 버퍼는 종료 시점에, 크래시 때, 또는 두 기록자가 뒤섞일 때 여러분을 놀라게 하는 버퍼이기 때문이다.

할당에 관한 주의 하나. 줄은 512바이트 스택 버퍼에 포매팅되므로 일반적인 줄은 할당 비용이 0이다. 들어가지 않는 줄은 거부되는 대신 그 호출에 한해 전역 힙(heap)으로 물러난다 — 길다는 이유로 출력을 거부하는 편이 더 나쁘기 때문이다. 이는 allocator 매개변수가 없는 호출이 그래도 할당할 수 있는 라이브러리 내 유일한 지점이며, 과도하게 긴 경우로 한정된다. 아무것도 할당하지 않는다는 보장이 필요하다면 직접 버퍼에 포매팅한 뒤 그것을 기록할 것.

출력량이 문제가 될 때는 `proven_sysio_stdout_buffered`에서 버퍼드 쓰기 스트림(writer)를 받아 거기에 포매팅한다 — argument 규칙은 동일하고, 줄마다 한 번이 아니라 flush마다 한 번의 시스템 콜이 된다. Chapter 5가 스트림 계층을 다룬다.

잘못된 예 — 출력하면서 스캐닝용 argument 생성자를 집어들기:

```
proven_println("{}", PROVEN_SCAN_ARG(&x));   /* wrong: that builds a scan destination */
```

이 섹션은 상세 I/O API가 Chapter 5에 있기 때문에 의도적으로 짧다. 포매터 사용자에게 중요한 점은 콘솔 헬퍼들이 문자열 append API와 같은 argument 규칙을 공유한다는 것이다.

흔한 실수:

- `proven_println`에 `PROVEN_SCAN_ARG`를 사용하기
- 모든 포맷 문자열에 `PROVEN_LIT`이 필요하다고 가정하기
- 출력 함수도 여전히 실패할 수 있다는 것을 잊기

7. 스캐너 데이터 모델

스캐너는 당신이 이미 가지고 있는 바이트 위의 커서다. 텍스트를 소유하지 않고, 복사하지 않으며, 어디에서도 읽지 않는다 - `proven_scan_t`는 뷰에 오프셋을 더한 것이며, 그것이 전부다.

```
typedef struct {
    proven_u8str_view_t view;    /* the bytes being read; not owned */
    proven_size_t       cursor;  /* how far in we are */
} proven_scan_t;
```

이것이 `scanf`와 다른 이유이므로 언급할 가치가 있는 두 가지 결과:

- **스캐너는 절대 할당하지 않고 당신의 입력에 절대 기록하지 않는다.** 스캔된 단어는 원본 바이트 *안*을 가리키는 `proven_u8str_view_t`로 반환된다. 그것은 정확히 그 바이트가 유효한 동안만 유효하며, 그 이상은 아니다. 그것들보다 오래 살아야 한다면, `proven_u8str_create_from_view()`로 복사하라.
- **커서는 당신의 것이다.** 그것은 평범한 필드다. 저장하거나, 복원하거나, 손으로 진행시킬 수 있다(§12는 `proven_scan_skip_until` 이후 정확히 그렇게 한다). 스캐너의 어떤 것도 당신에게 숨겨져 있지 않으므로, 당신을 위해 되돌려야 할 것도 없다.

`proven_scan_init()`은 잘못된 형식의 뷰(`size > 0`인데 null 포인터)를 신뢰하는 대신 빈 뷰로 정규화한다. 따라서 쓰레기로부터 만들어진 스캐너는 역참조 대신 입력 끝(EOF)으로 읽힌다.

각 primitive는 값을 그것을 지키는 오류와 짝지은 result 구조체를 반환한다: `proven_result_i64_t`, `proven_result_u64_t`, `proven_result_f64_t`, `proven_result_u8str_view_t`. **오류가 `PROVEN_OK`가 아닌 한 값은 무의미하다** - 스캐너는 실패를 뜻하는 sentinel 값을 사용하지 않는데, 모든 sentinel은 동시에 정당한 입력이기도 하기 때문이다.

8. 스캐너 기본 API

```
void                     proven_scan_skip_whitespace(proven_scan_t *scan);
proven_result_i64_t      proven_scan_i64(proven_scan_t *scan);
proven_result_u64_t      proven_scan_u64(proven_scan_t *scan);
proven_result_i64_t      proven_scan_i64_hex(proven_scan_t *scan);
proven_result_u64_t      proven_scan_u64_hex(proven_scan_t *scan);
proven_result_f64_t      proven_scan_f64(proven_scan_t *scan);
proven_result_u8str_view_t proven_scan_str(proven_scan_t *scan);
proven_err_t             proven_scan_skip_until(proven_scan_t *scan, proven_u8str_view_t target);
void                     proven_scan_skip_until_number(proven_scan_t *scan);
```

값을 반환하는 것들은 `[[nodiscard]]`다: 결과를 버리는 스캔은 애초에 할 필요가 없던 스캔이다.

공통 동작

- **선행 공백은 건너뛴다** — 모든 값 스캐너가 그렇게 한다. 먼저 `proven_scan_skip_whitespace()`를 호출할 필요가 없다. 그 함수는 커서를 직접 위치시키고 싶을 때를 위해 존재한다.
- **스캐닝은 값에 속할 수 없는 첫 바이트에서 멈춘다.** `"12abc"`는 `12`를 내고 커서를 `a`에 남긴다. 그것은 오류가 아니다 - 스캐너는 당신이 물은 질문에 답했고 나머지는 다음에 묻는 이를 위해 남겼다.
- **실패 시 커서는 복원된다.** 따라서 실패한 스캔은 비사건(non-event)이다: 돌아서서 같은 위치를 다른 것으로 파싱할 수 있다. §12가 이렇게 한다.

정수 스캐너

입력	`proven_scan_i64`	이유
`"42"`, `"+42"`, `"-42"`	`OK` - 42, 42, -42	부호는 숫자의 일부다
`"9223372036854775808"`	`PROVEN_ERR_OVERFLOW`	`INT64_MAX`보다 하나 큼. **랩(wrap)하지 않는다**
`"abc"`, `""`	`PROVEN_ERR_INVALID_ARG`	여기에 숫자가 없다
`"0x10"`	`OK` - **0**, 커서는 1에	**십진수만**: 0, 그 뒤에 텍스트

마지막 행이 사람들을 놀라게 하는 것이다. `proven_scan_i64`와 `proven_scan_u64`는 십진수를 읽는다: 8진수도, 기수 접두어도 없다. `0x10`은 정수 0이고, `x10`은 여전히 입력에 남아 있다.

16진수는 따로, 명시적으로 요청한다. `proven_scan_u64_hex`는 대소문자 16진 숫자를 읽으며, `0x`는 뒤에 숫자가 올 때만 받아들인다(`strtoul`과 같아서 `"0xg"`는 0 다음에 `"xg"`). `proven_scan_i64_hex`는 부호를 더한다. 포맷 문자열에서는 `{:x}`(또는 `{:X}`)가 어떤 정수 인자든 같은 방식으로 읽는다 - 16진 출력을 위한 포매터의 spec을 거꾸로 읽은 것이다. 오버플로, 커서 복원, 스트림 신호는 십진수와 같다.

```
/* A UnicodeData-style range, "0041..005A", read as two hexadecimal numbers. */
proven_u32 first = 0, last = 0;
proven_err_t e = proven_scan_fmt(PROVEN_LIT("0041..005A"), "{:x}..{:x}", PROVEN_SCAN_ARG(&first),
PROVEN_SCAN_ARG(&last));
/* e == PROVEN_OK, first == 0x41, last == 0x5A */

/* The primitive, for a hand-written loop: */
proven_scan_t s = proven_scan_init(PROVEN_LIT("0x1F rest"));
proven_result_u64_t h = proven_scan_u64_hex(&s);   /* h.val == 0x1F, cursor on " rest" */
proven_scan_t t = proven_scan_init(PROVEN_LIT("-7f"));
proven_result_i64_t g = proven_scan_i64_hex(&t);   /* g.val == -0x7F: the signed form takes a sign */
(void)e; (void)h; (void)g;
```

`proven_scan_u64`는 unsigned를 뜻한다: `"-1"`은 `18446744073709551615`로 랩하는 것이 아니라 `PROVEN_ERR_INVALID_ARG`다. 음수를 조용히 거대한 양수로 바꾸는 스캐너는 경계 검사가 무력화되는 방식이다.

부동소수점 스캐너
proven_scan_f64는 라이브러리의 나머지와 같은 정확히 반올림되는 십진 엔진을 거친다: 가장 가까운 값으로 반올림, 동점은 짝수로, 그 어디에도 long double 없음. nan과 inf를 받아들인다.

두 경계 동작은 **의도적으로 비대칭**이며, 그 비대칭이 요점이다.

- "1e309"는 PROVEN_ERR_OVERFLOW를 준다. 올바른 유한 답이 없으므로, 요청하지 않은 무한대를 건네는 대신 거부한다.
- "1e-400"은 PROVEN_OK와 0.0을 주며, 부호는 보존된다. 0으로의 언더플로는 정확히 반올림된 답 *그 자체*다. 그것을 오류로 보고하는 것은 올바른 산술을 실패로 보고하는 것이 될 것이다.

낱말과 커서 이동
proven_scan_str은 다음 공백으로 구분된 구간을 입력 안을 가리키는 뷰로 반환한다. 공백 외에 아무것도 남지 않은 경우는 PROVEN_ERR_INVALID_ARG다.

완전한 뷰에서는 "입력이 떨어졌다"와 "입력이 틀렸다"가 같은 사실이다 - 더 이상 입력이 없으므로, 끝에서 잘린 숫자는 정말로 잘못된 형식이다. **스트림** 위에서는 이 둘이 반대되는 사실이며, 그 차이가 기다릴지 오류를 보고할지를 결정한다. 파싱이 가진 것의 끝을 넘어 달렸을 때 스캐너는 proven_scan_t::needs_more를 설정하며, 버퍼드 스캐너는 정확히 그것을 사용해 다시 채우고 재시도한다: -를 전달한 뒤 잠시 후 12를 전달하는 파이프는 -12로 스캔된다. 이전에는 잘못된 형식의 숫자였다. 실제로 존재하는 잘못된 바이트 - 숫자가 와야 할 자리의 글자 - 는 여전히 오류이며, 이후 어떤 입력도 그것을 바꾸지 못한다. 스캐너는 그것을 기다리지 않는다.

proven_scan_skip_until(scan, target)은 커서를 target을 **지나서**가 아니라 target으로 옮긴다 - 그것을 얼마나 소비할지는 당신이 결정한다. target이 거기 없으면 결과는 PROVEN_ERR_NOT_FOUND이고 **커서는 움직이지 않는다**: 스캐너는 탐색에 실패한 입력을 소비하지 않는다.

proven_scan_skip_until_number는 첫 숫자에서, 또는 바로 뒤에 숫자가 따라오는 부호에서 멈춘다. 숫자가 없으면 커서를 입력의 끝까지 몰아간다 - 따라서 읽을 것이 있다고 가정하기 전에 커서(cursor), 곧 스캐너가 지금까지 읽어 온 위치를 scan.cursor < scan.view.size로 검사하라.

9. 스캔 인자 모델
스캔 argument는 컴파일러가 선택하는 **당신의 목적지를 가리키는 타입 있는 포인터**다.

```
PROVEN_SCAN_ARG(&x)     /* _Generic on the pointer type */
```

이것이 스캐너가 scanf와 가장 첨예하게 다른 지점이다. 잘못 쓸 포맷 글자가 없는데, 포맷 글자가 아예 없기 때문이다. long에 대한 %d나, 너무 작은 버퍼에 대한 %s는 여기서 가능한 실수가 아니다: 목적지의 타입 *자체*가 명세이며, 불일치는 손상된 스택이 아니라 컴파일 오류다.

지원되는 목적지: short, unsigned short, int, unsigned int, long, unsigned long, long long, unsigned long long, double, 그리고 proven_u8str_view_t.

PROVEN_SCAN_ARG_LONG(&x)와 PROVEN_SCAN_ARG_ULONG(&x)는 호출 지점에서 명시적이길 원하는 호출자를 위해 존재한다. PROVEN_SCAN_ARG는 이미 long*와 unsigned long*를 처리한다.

좁은 목적지는 범위 검사된다. "70000"을 short로 스캔하는 것은 잘린 4464가 아니라 PROVEN_ERR_OVERFLOW다. 값은 64비트로 파싱되어, 무언가를 저장하기 전에 목적지의 범위와 대조 검사된다.

proven_scan_arg_* 생성자들은 argument 배열을 손으로 만들어야 할 때를 위해 공개되어 있지만, 호출자가 사용하는 것은 매크로다.

10. 구조적 스캔 문법
포맷 문자열로 파싱하는 것이 출력하는 것보다 위험한 이유
잘못된 포맷 문자열로 포매팅하면 잘못된 출력이 나온다. **그것으로 파싱하면 메모리가 망가진다.** 포맷이 여러분이 넘긴 포인터를 통해 무엇을 쓸지 결정하기 때문이다. c가 char인데 sscanf("%d", &c)를 하면 1바이트 객체에 4바이트를 쓰는데, 호출문 어디에도 그런 말이 없다.

이 비대칭이 이 섹션의 모양을 결정한다. 구조적 스캔은 포매터와 같은 {} 문법을 쓰지만, 목적지 타입은 PROVEN_SCAN_ARG(&x)에서 온다 — 같은 _Generic 디스패치이므로 기록되는 폭은 객체의 폭이고, 거기 담기에 너무 큰 값은 이웃한 세 바이트가 아니라 PROVEN_ERR_OVERFLOW가 된다.

쓰기 전에 반드시 체득해야 할 성질 하나. **구조적 스캔은 플레이스홀더 전체에 걸쳐 트랜잭션이 아니다.** 세 번째 {}가 실패하면 앞의 두 목적지는 이미 기록된 뒤다. 이는 의도한 절충이다 — 줄 전체가 파싱될 때까지 모든 목적지를 버퍼링하려면 할당이 필요한데, 이 스캐너는 아무것도 할당하지 않는다 — 그러나 그 결과로 실패한 proven_scan_fmt는 변수들을 일부만 갱신된 상태로 남긴다. 실패 시 그것들을 쓰레기로 취급하거나, 지역 변수에 스캔한 뒤 성공했을 때만 복사해 낼 것. §11.1이 두 패턴을 모두 보여준다.

패턴 속 리터럴은 문법의 나머지 절반이며 사람들이 잘 활용하지 않는 부분이다. 플레이스홀더가 아닌 모든 것은 입력과 정확히 일치해야 하므로, "12-34"에 대한 "{}:{}"는 한 필드를 조용히 돌려주는 대신 리터럴에서 실패한다.

스캔 포맷 문자열은 포매터의 것을 거꾸로 읽은 것이다.

- 플레이스홀더는 argument 하나를 순서대로 소비한다.
- 그 외의 모든 것은 **입력과 정확히 일치해야 하는 리터럴**이다.

스캐닝 측의 스펙은 하나뿐이다: {:x}(또는 {:X})는 정수 인자를 16진수로 읽는다(8절). 실수나 문자열 인자에 붙으면 포매터와 마찬가지로 PROVEN_ERR_INVALID_FORMAT이다. Width, fill, alignment는 포매팅의 관심사이며, 스캐너는 거기 있는 것을 읽는다.

포맷 안의 공백은 특별하지 않다. 값 스캐너들이 스스로 선행 공백을 건너뛰므로, 두 플레이스홀더 사이에 공백이 있는 포맷과 없는 포맷은 "7 8"을 동일하게 파싱한다 - 포맷의 공백은 입력의 공백과 일치하고, 그것이 없었더라도 두 번째 스캐너가 어차피 건너뛰었을 것이다.

플레이스홀더의 수는 argument의 수와 같아야 한다. 입력의 값이 너무 적은 것은 오류다. **너무 많은 것은 오류가 아니다**(§11.1).

11. 스캔 형식화 API

```
proven_scan_fmt(view, fmt, ...)           /* scan a view from the beginning */
proven_scan_fmt_cursor(&scan, fmt, ...)   /* continue from an existing cursor */
proven_err_t proven_scan_fmt_internal(...) /* what the macros expand to */
```

자기완결적인 한 줄에는 proven_scan_fmt를 사용하라. 스캔이 같은 입력 위를 걷는 더 긴 여정의 한 단계일 때는 proven_scan_fmt_cursor를 사용하라: 이것은 당신이 소유한 커서를 진행시키므로, §8의 primitive들과 자유롭게 섞인다.

11.1. 스캔 에러 코드 안내와 복구

코드	실제로 일어난 일	할 일
PROVEN_OK	모든 플레이스홀더가 채워졌고 모든 리터럴이 일치했다.	값들을 발행(publish)하라.
PROVEN_ERR_INVALID_ARG	입력이 당신이 요청한 형태가 아니다 - 플레이스홀더가 읽을 값이 없었거나, 입력이 떨어졌다.	그 줄은 일치하지 않는다. 보고하되, 같은 형태로 재시도하지 마라.
PROVEN_ERR_NEED_MORE	**버퍼드 스캐너 전용.** 토큰이 읽기 경계에서 반으로 잘렸다: 그 나머지가 아직 도착하지 않았다. 보통은 이것을 보지 않는다 - proven_sysio_scanner_scan이 당신을 위해 다시 채우고 재시도한다 - 이는 스캐너가 스스로에게 하는 말이다.	아무것도. 이미 처리되었다.
PROVEN_ERR_NOT_FOUND	포맷 안의 **리터럴**이 일치하지 않았다.	그 줄은 예상과 다른 형태다. 저장된 커서에서 다른 포맷을 시도하라.

PROVEN_ERR_OVERFLOW	숫자는 제대로 된 형식이었지만 목적지에 들어맞지 않는다.	입력이 유효한데 목적지가 너무 좁은 것일 수도, 입력이 적대적인 것일 수도 있다. 그 둘은 매우 다른 상황이다 - 타입을 넓히기 전에 구별하라.
PROVEN_ERR_INVALID_FORMAT	포맷 문자열 자체가 잘못된 형식이다.	입력이 아니라 당신 코드의 버그다.

구조적 스캐너는 트랜잭션이 아니며, 이것이 물어뜯는 그 지점이다.

리터럴이 일치에 실패했을 때, 불일치 *이전*의 플레이스홀더들은 이미 기록되어 있다. 호출은 PROVEN_ERR_NOT_FOUND를 반환하는데 당신의 목적지는 그래도 값을 담고 있다 - 아래의 id는 실패한 호출에서 온 7이다.

```
int id = -1;
double ratio = -1.0;
proven_err_t err = proven_scan_fmt(line, "id={} XXX={}",
                                   PROVEN_SCAN_ARG(&id), PROVEN_SCAN_ARG(&ratio));
/* err == PROVEN_ERR_NOT_FOUND, and id == 7: it was written before the failure. */
```

그러므로: **실패 시 모든 목적지를 오염된(clobbered) 것으로 취급하라.** 전부 아니면 전무가 필요하면, 지역 변수로 스캔하고 호출이 성공한 뒤에만 발행하라 - §12의 예제가 그 형태를 보여준다. 대안으로, 호출 전에 scan.cursor를 저장하고 이후에 복원하라. 커서는 평범한 필드이며, 그것은 의도적이다.

후행 입력은 오류가 아니다. "7 8"에 대해 플레이스홀더 하나를 스캔하면 값 7로 성공하고 8은 소비되지 않은 채 남는다. 스캐너는 당신이 요청한 것을 일치시키고 멈췄다. 당신이 묻지 않은 것을 단속하지 않는다. 줄 전체가 소비되어야 한다면, 커서를 직접 검사하라.

```
if (scan.cursor != scan.view.size) { /* there is unparsed input left */ }
```

12. 예제와 오용 사례

실전 예제: 한 줄을 형식화하고 다시 읽어 들이기
테스트 스위트가 컴파일하고 실행함. 스택에서 빌린 문자열(오버플로 시 atomic)과 할당자 기반 문자열(성장함)로 포매팅하고, 신뢰할 수 없는 바이트에 경계 argument를 사용한 뒤, 그 줄을 다시 파싱한다 - float은 정확히 라운드트립된다.

```
/*
 * 형식화와 파싱은 같은 생각의 두 반쪽이다. `{}` 는 타입 있는 값을 글로 그려 내고,
 * 스캐너는 글을 타입 있는 목적지로 되읽는다. 둘 다 호출 자리에서 타입이 검사되므로
 * (_Generic 이 생성자를 고른다), 실행 중에 어긋날 서식 문자열/인자 짝이 없다.
 *
 * 중요한 선택은 *바이트가 어디로 가는가* 다.
 *
 *   append_fmt       - 용량 고정, 원자적. 너무 길면? 아무것도 쓰이지 않고
 *                      PROVEN_ERR_OUT_OF_BOUNDS 를 받는다. 할당자가 끼지 않으므로
 *                      스택 버퍼에서도 돈다.
 *   append_fmt_grow  - 할당자를 등에 업는다. 들어가도록 늘리고, 할당이 실패하면
 *                      문자열은 있던 그대로 남는다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 용량 고정: 할당자도, 할당도 없다 ---------------------------------- */
    /* borrow 는 부르는 쪽의 기억을 감싼다. 그래서 이 문자열은 통째로 스택에 산다. `cap`
     * 은 NUL 을 포함하므로 32 바이트는 내용 31 바이트를 담는다. 지울 것도 없다. */
    proven_byte_t stack_buf[32];
    proven_u8str_t fixed = proven_u8str_borrow(stack_buf, sizeof stack_buf);

    proven_fmt_result_t r = proven_u8str_append_fmt(&fixed, "port={}", PROVEN_ARG(8080));
```

```
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "a short line should fit in 32 bytes");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
                "the fixed-capacity append should have rendered the port");

/* 원자적이라는 말은 원자적이라는 뜻이다. 들어가지 않는 append 는 아무것도 바꾸지
 * 않는다. 문자열은 여전히 온전하고 전에 담고 있던 것을 그대로 담고 있다 - 치울 잘린
 * 꼬리가 없다. (잘린 꼬리가 원하는 것이라면 append_fmt_trunc 를 쓸 것.) */
proven_fmt_result_t too_long = proven_u8str_append_fmt(
    &fixed, " and a great deal more text than will ever fit here {}", PROVEN_ARG(1));
EXAMPLE_REQUIRE(too_long.err == PROVEN_ERR_OUT_OF_BOUNDS, "the overlong append must fail");
EXAMPLE_REQUIRE(too_long.required > too_long.written, "it reports what it would have needed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&fixed), PROVEN_LIT("port=8080")),
                "a failed atomic append must leave the string untouched");

/* --- 지정자: 채움, 정렬, 폭, 16진 ------------------------------------ */
proven_result_u8str_t created = proven_u8str_create(alloc, 8);   /* 일부러 작게 */
EXAMPLE_REQUIRE(proven_is_ok(created.err), "creating the output string should succeed");
if (!proven_is_ok(created.err)) return 1;
proven_u8str_t out = created.value;

/* grow 는 필요한 만큼 재할당하므로, 처음 용량은 한계가 아니라 힌트다.
 * `{:0>4}` = 채움 '0', 오른쪽 정렬, 폭 4. `{:x}` = 소문자 16진, 0x 없음. */
r = proven_u8str_append_fmt_grow(alloc, &out, "id={:0>4} tag={:*^9} addr=0x{:x}",
                                 PROVEN_ARG(7),
                                 PROVEN_ARG(PROVEN_LIT("ok")),
                                 PROVEN_ARG(48879));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the growing append should succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
                                     PROVEN_LIT("id=0007 tag=***ok**** addr=0xbeef")),
                "fill/align/width/hex should render exactly this");
printf("%s\n", proven_u8str_as_cstr(&out));

/* --- 믿을 수 없는 글은 경계가 있고, NUL 로 끝난다고 믿지 않는다 -------- */
/* char* 에 PROVEN_ARG 를 쓰면 "NUL 이 나올 때까지 걸어라" 는 뜻이다 - 리터럴에는
 * 괜찮지만 소켓에서 온 것에는 버퍼 넘어 읽기다. 이 버퍼에는 NUL 이 아예 없다.
 * PROVEN_ARG_CSTR_N 은 대신 길이에서 멈추므로 실제로 있는 것만 읽는다. 여러분이 직접
 * 만들지 않은 것에는 이것을 쓸 것. */
const char untrusted[4] = {'a', 'b', 'c', 'd'};   /* 일부러 종결자를 두지 않았다 */
EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
r = proven_u8str_append_fmt_grow(alloc, &out, "payload={}",
                                 PROVEN_ARG_CSTR_N(untrusted, sizeof untrusted));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "the bounded append should succeed");
EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("payload=abcd")),
                "the bounded argument should render its whole 4 bytes and stop");

/* --- 레코드를 형식화하고, 도로 파싱하기 ------------------------------- */
proven_i64 sensor_id = 42;
double reading = 3.14159;

EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "reset should keep the buffer");
r = proven_u8str_append_fmt_grow(alloc, &out, "{} {} {}",
                                 PROVEN_ARG(sensor_id),
                                 PROVEN_ARG(PROVEN_LIT("boiler")),
                                 PROVEN_ARG(reading));
EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "formatting the record should succeed");
printf("record: %s\n", proven_u8str_as_cstr(&out));

/* 뷰 하나 위의 스캐너 하나. 호출마다 커서를 자기가 삼킨 만큼 앞으로 옮기므로 호출이
 * 왼쪽에서 오른쪽으로 이어진다 - 그리고 하나하나가 따로 실패할 수 있는데, 그것이
 * 파서와 짐작을 가르는 차이다. */
```

```
    proven_scan_t sc = proven_scan_init(proven_u8str_as_view(&out));

    proven_result_i64_t id = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(id.err), "the first field should parse as an integer");
    EXAMPLE_REQUIRE(id.val == sensor_id, "the integer should round-trip");

    /* scan_str 은 *파싱 중인 문자열 안*을 가리키는 뷰를 돌려준다 - 복사도 소유도 하지
     * 않으므로 `out` 이 살아 있는 동안에만 쓸 수 있다. */
    proven_result_u8str_view_t name = proven_scan_str(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(name.err), "the second field should parse as a word");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(name.val, PROVEN_LIT("boiler")), "the name should round-trip");

    proven_result_f64_t temp = proven_scan_f64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(temp.err), "the third field should parse as a float");

    /* 근사하게가 아니라 정확히 같다. 스캐너는 올바르게 반올림하므로 글자에 가장 가까운
     * double 을 돌려주고 - 형식화기가 내놓은 글자(소수 여섯 자리)는 이 값을 모호함 없이
     * 지목한다. 비트 하나까지 우리가 시작한 그 double 이다. 소수 여섯 자리보다 더 필요한
     * 값이라면 가장 짧은 정책(proven_float_format_options_shortest)으로 형식화하면 같은
     * 왕복이 성립한다. */
    EXAMPLE_REQUIRE(temp.val == reading, "the float must round-trip exactly, not approximately");

    /* 입력을 남김없이 삼켰다. 조용히 남겨 둔 것이 없다. */
    proven_result_i64_t extra = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(!proven_is_ok(extra.err), "there should be nothing left to scan");

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}
```

실전 예제: 스캐너의 에러 코드와 그로부터의 복구

테스트 스위트가 컴파일하고 실행함. 위 표의 모든 코드를 여기서 일부러 유발한다. 비-트랜잭션 실패까지 포함하는데 - 읽기만 한 계약은 배우지 못한 계약이기 때문이다.

```
/*
 * 스캐너의 오류 코드들, 그리고 거기서 되살아나는 법.
 *
 * 이 스캐너는 scanf 가 아니다. 받지 않은 포인터를 통해 쓰는 일이 없고, 폭을 짐작하지
 * 않으며, 여러 가지 중 무엇이 잘못됐는지를 말해 준다. 마지막 것은 그 코드들이 무슨
 * 뜻인지 알아야 도움이 되므로, 이 프로그램은 하나하나를 일부러 일으켜 본다.
 */

static proven_u8str_view_t v(const char *s) {
    return proven_u8str_view_from_cstr(s);
}

int main(void) {
    /* --- 기본 호출들은 실패하면 커서를 되돌린다 ---------------------------- */
    /* 실패한 파싱은 없던 일이다. 커서는 있던 자리에 있으므로, 같은 자리를 다른 것으로
     * 읽어 볼 수 있다. */
    {
        proven_scan_t sc = proven_scan_init(v("abc"));
        proven_result_i64_t n = proven_scan_i64(&sc);
        EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG, "'abc' is not an integer");
        EXAMPLE_REQUIRE(sc.cursor == 0, "a failed integer scan leaves the cursor alone");

        /* 그래서 같은 자리를 낱말로 읽을 수 있다. */
        proven_result_u8str_view_t w = proven_scan_str(&sc);
        EXAMPLE_REQUIRE(proven_is_ok(w.err) && proven_u8str_view_eq(w.val, PROVEN_LIT("abc")),
                        "the same bytes parse fine as a word");
```

```
}

/* --- 들어가지 않는 수는 감긴 값이 아니라 OVERFLOW 다 ------------------ */
{
    proven_scan_t sc = proven_scan_init(v("9223372036854775808"));   /* INT64_MAX + 1 */
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(n.err == PROVEN_ERR_OVERFLOW, "one past INT64_MAX must not wrap");
    EXAMPLE_REQUIRE(sc.cursor == 0, "the cursor is restored on overflow too");
}

/* --- 그러나 아래로 넘치는 실수는 오류가 *아니다* ---------------------- */
/* 너무 크면 OVERFLOW 이고, 너무 작으면 부호를 지킨 0 이다. 그 비대칭은 일부러다.
 * 0 으로 내려앉는 것은 올바르게 반올림한 답이지만, 위로 넘치는 데는 올바른 유한한
 * 답이 아예 없다. */
{
    proven_scan_t big = proven_scan_init(v("1e309"));
    proven_result_f64_t b = proven_scan_f64(&big);
    EXAMPLE_REQUIRE(b.err == PROVEN_ERR_OVERFLOW, "1e309 does not fit a double");

    proven_scan_t tiny = proven_scan_init(v("-1e-400"));
    proven_result_f64_t t = proven_scan_f64(&tiny);
    EXAMPLE_REQUIRE(proven_is_ok(t.err), "1e-400 underflows, which is not an error");
    EXAMPLE_REQUIRE(t.val == 0.0, "it rounds to zero");
}

/* --- 정수 스캐너는 십진만 읽는다 ------------------------------------- */
/* "0x10" 은 열여섯이 아니다. 0 하나이고, 그 뒤는 스캐너가 보라고 하지 않은 글이다.
 * 사람들이 놀라는 자리라 알아 둘 값어치가 있다. */
{
    proven_scan_t sc = proven_scan_init(v("0x10"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 0, "0x10 scans as the integer 0");
    EXAMPLE_REQUIRE(sc.cursor == 1, "and the cursor stops before the 'x'");
}

/* --- 파싱은 그 값에 속할 수 없는 첫 바이트에서 멈춘다 ----------------- */
{
    proven_scan_t sc = proven_scan_init(v("12abc"));
    proven_result_i64_t n = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(n.err) && n.val == 12, "12abc yields 12");
    EXAMPLE_REQUIRE(sc.cursor == 2, "and leaves 'abc' for whoever asks next");
}

/* --- 부호 없음은 부호 없음이라는 뜻이다 ------------------------------- */
{
    proven_scan_t sc = proven_scan_init(v("-1"));
    proven_result_u64_t n = proven_scan_u64(&sc);
    EXAMPLE_REQUIRE(n.err == PROVEN_ERR_INVALID_ARG,
                    "-1 is rejected rather than wrapping to a huge unsigned value");
}

/* --- 값까지 찾아가기: skip_until ------------------------------------- */
/* skip_until 은 커서를 목표 *위*에 두지, 지나쳐 두지 않는다. 그것을 얼마나 삼킬지는
 * 여러분이 정한다. */
{
    proven_scan_t sc = proven_scan_init(v("port=8080"));
    proven_err_t err = proven_scan_skip_until(&sc, PROVEN_LIT("="));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the '=' is there");
    EXAMPLE_REQUIRE(sc.cursor == 4, "the cursor sits on the '=' itself");
```

```
    ++sc.cursor;                                /* 그것을 넘어선다 */
    proven_result_i64_t port = proven_scan_i64(&sc);
    EXAMPLE_REQUIRE(proven_is_ok(port.err) && port.val == 8080, "the port parses");

    /* 못 찾으면 NOT_FOUND 이고 커서는 움직이지 않는다 - 스캐너는 찾아가지 못한
     * 입력을 삼키지 않는다. */
    proven_scan_t sc2 = proven_scan_init(v("port=8080"));
    proven_err_t missing = proven_scan_skip_until(&sc2, PROVEN_LIT("#"));
    EXAMPLE_REQUIRE(missing == PROVEN_ERR_NOT_FOUND, "there is no '#'");
    EXAMPLE_REQUIRE(sc2.cursor == 0, "and the cursor stayed put");
}

/* --- 구조를 읽는 스캐너 ------------------------------------------------ */
{
    int id = 0;
    double ratio = 0.0;
    proven_u8str_view_t name = {0};

    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5 name=ada"),
                                       "id={} ratio={} name={}",
                                       PROVEN_SCAN_ARG(&id),
                                       PROVEN_SCAN_ARG(&ratio),
                                       PROVEN_SCAN_ARG(&name));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the line matches the shape");
    EXAMPLE_REQUIRE(id == 7 && ratio == 0.5, "the values land in the right places");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(name, PROVEN_LIT("ada")), "including the word");
}

/* --- 구조를 읽는 스캐너는 트랜잭션이 *아니다* ------------------------- */
/*
 * 이것이 사람을 무는 자리다. 리터럴이 맞지 않으면 파싱은 오류를 돌려주는데 - 그
 * 어긋남 *앞*의 자리표들은 이미 목적지에 쓰여 버렸다. 호출이 실패했는데도 `id` 는
 * 7 이다.
 *
 * 그러니 실패했을 때는 모든 목적지가 더럽혀졌다고 여길 것. 전부 아니면 전무가
 * 필요하면 지역 변수로 파싱하고 호출이 성공한 뒤에만 공표할 것. 아래 코드가 하는
 * 일이 그것이다.
 */
{
    int id = -1;
    double ratio = -1.0;
    proven_err_t err = proven_scan_fmt(v("id=7 ratio=0.5"),
                                       "id={} XXX={}",       /* 리터럴이 틀렸다 */
                                       PROVEN_SCAN_ARG(&id),
                                       PROVEN_SCAN_ARG(&ratio));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_NOT_FOUND, "the literal 'XXX=' is not in the input");
    EXAMPLE_REQUIRE(id == 7, "and yet id was already written: the scan is not atomic");

    /* 안전한 모양: 지역 변수로 파싱하고 성공했을 때 공표한다. */
    int good_id = 0;
    double good_ratio = 0.0;
    int published_id = -1;
    proven_err_t ok = proven_scan_fmt(v("id=7 ratio=0.5"), "id={} ratio={}",
                                      PROVEN_SCAN_ARG(&good_id), PROVEN_SCAN_ARG(&good_ratio));
    if (proven_is_ok(ok)) published_id = good_id;
    EXAMPLE_REQUIRE(published_id == 7, "publish only what a successful scan produced");
}

/* --- 입력이 모자랄 때, 그리고 남을 때 ---------------------------------- */
{
```

```
        int a = 0, b = 0;
        proven_err_t short_input = proven_scan_fmt(v("5"), "{} {}",
                                                   PROVEN_SCAN_ARG(&a), PROVEN_SCAN_ARG(&b));
        EXAMPLE_REQUIRE(!proven_is_ok(short_input), "two placeholders, one value: that fails");

        /* 뒤에 남은 입력은 오류가 *아니다*. 스캐너는 청한 것을 맞추고 멈췄다. 청하지
         * 않은 것까지 단속하지는 않는다. 줄 전체를 삼켜야 한다면 그것은 여러분이
         * 확인할 일이다. */
        int only = 0;
        proven_scan_t sc = proven_scan_init(v("7 8"));
        proven_err_t err = proven_scan_fmt_cursor(&sc, "{}", PROVEN_SCAN_ARG(&only));
        EXAMPLE_REQUIRE(proven_is_ok(err) && only == 7, "the first value scans");
        EXAMPLE_REQUIRE(sc.cursor < sc.view.size, "and '8' is still sitting there, unconsumed");
    }

    /* --- 좁은 목적지는 범위가 검사된다 ------------------------------------ */
    {
        short small = 0;
        proven_err_t err = proven_scan_fmt(v("70000"), "{}", PROVEN_SCAN_ARG(&small));
        EXAMPLE_REQUIRE(err == PROVEN_ERR_OVERFLOW,
                        "70000 does not fit a short, and the scanner says so rather than truncating");
    }

    return EXAMPLE_OK();
}
```

오용: `0x10`이 16이라고 가정하기
그것은 0이다. 십진 스캐너들은 십진수 전용이며, `x10`은 여전히 입력에 남아 있다. 입력이 16진이면 그렇다고 말하라: `proven_scan_u64_hex`, 또는 포맷의 `{:x}`.

오용: 남은 입력을 에러로 다루기
그것은 오류가 아니다. `"7 8"`에 대한 플레이스홀더 하나는 성공한다. 신경 쓰인다면 커서를 검사하라.

오용: 실패한 스캔 뒤의 목적지를 믿기
그것들은 오염되었다. §11.1을 참조하라.

오용: 입력이 사라진 뒤에도 스캔한 낱말을 들고 있기
`proven_scan_str`은 복사본이 아니라 **입력 안을 가리키는 뷰**를 반환한다. 버퍼가 사라지면, 그 단어도 사라진다. 그것이 온 바이트보다 오래 살아야 한다면 `proven_u8str_create_from_view()`로 복사하라.

13. 프리스탠딩과 빌드 모드 참고

float만 컴파일에서 빠지는 이유
이 장의 모든 것은 이식 가능한 계산이지만 한 부분만 예외이고, 그 부분은 유난히 비싸다.

`double`을 올바르게 포매팅하는 것 — 왕복(round-trip)하는 가장 짧은 십진 표현을, subnormal을 포함한 모든 입력에 대해 얻는 것 — 에는 큰 정수 연산과 조회 테이블이 필요하다. 포매터에서 단일 코드 조각으로는 가장 크며, 대부분의 펌웨어는 `double`을 아예 출력하지 않는다. 그래서 freestanding 프로파일은 `PROVEN_FMT_NO_FLOAT`를 설정해 그것을 덜어내고, 마이크로컨트롤러용 빌드는 결코 호출하지 않을 수 킬로바이트의 십진 변환 테이블을 지고 다니지 않는다.

이것이 런타임 결정이 아니라 빌드 타임 결정인 것은 의도적이다. 코드가 단지 도달 불가능한 것이 아니라 *부재*하므로, 링커를 설득해 남겨둘 수 없다. 작은 타깃에서도 float를 원하는 빌드를 위한 관련 손잡이 — 큰 정수 용량 — 는 freestanding 가이드 §8a가 다룬다.

스캐너는 코어다: I/O를 하지 않고, 아무것도 할당하지 않으며, 어떤 플랫폼 레이어도 건드리지 않는다. 따라서 freestanding 빌드에서도 호스티드(hosted) 빌드에서와 정확히 똑같이 사용할 수 있다.

유일한 빌드 모드 의존성은 float다. Freestanding 빌드는 `PROVEN_FMT_NO_FLOAT`(float *포매터*를 컴파일에서 제외)와 `PROVEN_NO_U16STR`를 설정한다. `proven_scan_f64`는 십진 파싱 엔진(`float_parse.c`와 `float_decimal.c`)을 끌어오는데, 이는 정수 전용이며

- `long double` 없음, libm 없음, 소프트 float 헬퍼 호출 없음 - 코드 크기 면에서

공짜는 아니다. 그것이 중요한 타겟이면서 float을 읽을 필요가 없다면, 그저 호출하지 마라: 스캐너의 다른 어떤 것도 float 경로를 참조하지 않는다.

`proven_sysio_scanner_*`(Chapter 5)는 **다른** 것이다: 파일 위의 버퍼드 스캐너로, I/O를 하기 때문에 hosted 전용이다. 이 장에서 설명한 스캐너는 당신이 이미 가지고 있는 바이트를 읽는다.

실전 예제: 더 넓은 수, 자유로운 텍스트, 부동소수점
앞의 예제들은 정해진 서식을 `proven_i32`로 읽어 들인다. 입력이 실제 데이터가 되는 순간 세 가지가 따라오며, 이 예제는 그것들을 한꺼번에 다룬다.

목적지 타입이 곧 계약이다. `proven_scan_arg_i64()`, `proven_scan_arg_u32()`, `proven_scan_arg_u64()`는 값이 들어갈 타입을 이름으로 말하고, 스캐너는 감기는 대신 넘침을 보고한다. 32비트로는 모자란 바이트 수와, 결코 음수일 수 없는 값 — 이 둘이 목적지 타입을 잘못 고른 것이 숫자가 커지기 전까지 아무도 모르는 버그가 되는 경우다. 일반 매크로 `PROVEN_SCAN_ARG()`는 넘겨받은 포인터로 이들 중 하나를 골라 주며, 이름 있는 형태는 그 매크로가 고르는 것을 손으로 적은 것이다.

모든 입력이 정해진 서식인 것은 아니다. `proven_scan_skip_whitespace()`는 공백·탭·줄바꿈을 지나가고, `proven_scan_skip_until_number()`는 다음 숫자 자리까지, 또는 바로 뒤에 숫자가 붙은 부호 자리까지 커서를 옮긴다. 음수가 부호를 잃지 않는 이유다. 둘 다 실패할 수 없다. "건너뛸 것이 없었다"는 에러가 아니며, 입력의 끝에서 커서는 넘어가지 않고 멈춘다.

부동소수점에는 로케일(locale) 문제가 있고, 이 라이브러리에는 없다. `strtod()`는 `"3,5"`를 어떤 로케일에서는 `3.5`로, 다른 로케일에서는 `3`으로 읽으므로 같은 프로그램이 기계마다 다른 말을 한다.
`proven_parse_double_ascii()`는 구조적으로 로케일에서 자유롭다. 쉼표는 결코 소수점이 아니다. 이 함수는 숫자가 쓴 바이트 수를 알려 주어 호출자가 그 지점부터 이어 갈 수 있게 하고, 해석할 수 없는 텍스트를 진짜 `0`과 구분되지 않는 `0`이 아니라 에러로 보고한다. `proven_parse_f64_ascii()`는 같은 함수의 예전 이름이며, 기존 호출 자리가 그대로 읽히도록 남아 있다.

내보내는 쪽에서는 `proven_arg_f64()`가 부동소수점 값이 형식화기로 들어가는 통로다. `float`도 `double`도 이것을 지나므로, 출력이 그 값을 담아 둔 변수의 폭에 좌우되지 않는다. 정확한 표기가 중요할 때는 `proven_float_format_f32_policy()`와 그 `f64` 짝이 정책과 옵션을 명시적으로 받는다. 최단(shortest) 모드는 다시 읽었을 때 정확히 같은 값이 되는 가장 짧은 자릿수를 요구한다. 직렬화기가 원하는 성질이 이것이다. 필요 없는 수에까지 열일곱 자리를 찍지 않으면서 왕복을 정확하게 만든다.

```
#include <string.h>

/*
 * 글에서 수를 읽어 내고, 다시 써 넣기.
 *
 * 여기 쓰인 글은 프로그램이 실제로 만나는 종류다. 폭과 부호가 뒤섞인 로그 줄, 그리고
 * 관심 있는 수가 산문 속에 묻혀 있는 측정값 파일. 그래서 앞의 8장 예제들이 다루지 않는
 * 세 가지 일이 나온다.
 *
 *   - int 보다 *넓은* 타입, 그리고 부호 없는 타입으로 파싱하기. 여기서는 목적지 타입이
 *     물음의 전부다 - 32비트에 들어가지 않는 바이트 수야말로 파일이 5 GB 가 될 때까지
 *     아무도 알아채지 못하는 고전적인 넘침이다.
 *   - 입력이 빡빡한 형식이 아닐 때 커서를 손으로 옮기기 - 공백을 건너뛰거나, 다음 수가
 *     시작하는 자리까지 건너뛰기.
 *   - 십진 문자열을 double 로, 그리고 다시 글로, C 라이브러리의 로케일에 매인 변환을
 *     거치지 않고 정확하게 옮기기.
 *
 * 부동소수점 쪽은 들리는 것보다 중요하다. strtod 는 어느 로케일에서는 "3,5" 를 3.5 로,
 * 다른 로케일에서는 3 으로 읽고, 그러면 같은 프로그램이 기계마다 서로 다른 소리를 한다.
```

```
 * 여기 쓰인 파서는 만들어질 때부터 로케일이 없다. 환경이 무어라 하든 쉼표가 소수점이
 * 되는 일은 없다.
 */

int main(void) {
    /* --- 1. 값이 필요로 하는 타입으로 파싱하기 --------------------------- */

    /* 로그 줄 하나: 64비트가 필요한 요청 번호, 결코 음수가 아니고 4 GB 를 넘을 수 있는
     * 바이트 수, 작은 상태 코드, 그리고 음수 오프셋. */
    proven_scan_t scan = proven_scan_init(
        PROVEN_LIT("id=9007199254740993 bytes=5368709120 status=404 delta=-17"));

    proven_i64 id = 0;
    proven_u64 bytes = 0;
    proven_u32 status = 0;
    proven_i32 delta = 0;

    /* 인자 생성자마다 목적지 타입을 이름으로 밝히므로, 스캐너는 옳은 폭으로 쓰고 감기는
     * 대신 넘침을 알린다. 일반 매크로 PROVEN_SCAN_ARG() 는 포인터의 타입에서 이것들을
     * 대신 골라 준다. 이름 붙은 꼴은 그것이 고르는 것 자체이고, 그 선택을 눈에 보이게
     * 하고 싶을 때 여러분이 적는 것이다. */
    proven_err_t err = proven_scan_fmt_cursor(&scan, "id={} bytes={} status={} delta={}",
                                      proven_scan_arg_i64(&id),
                                      proven_scan_arg_u64(&bytes),
                                      proven_scan_arg_u32(&status),
                                      proven_scan_arg_i32(&delta));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the log line must succeed");
    EXAMPLE_REQUIRE(id == 9007199254740993LL, "a 64-bit id survives, which a 32-bit destination could not");
    EXAMPLE_REQUIRE(bytes == 5368709120ULL, "and so does a byte count larger than 4 GiB");
    EXAMPLE_REQUIRE(status == 404u, "the small unsigned value reads normally");
    EXAMPLE_REQUIRE(delta == -17, "and a signed destination accepts the minus sign");

    /* 목적지 타입은 힌트가 아니라 계약이다. 음수에는 부호 없는 표현이 없고, 스캐너는
     * 그것을 어마어마한 값으로 감아 버리는 대신 거부한다. */
    proven_scan_t neg = proven_scan_init(PROVEN_LIT("-17"));
    proven_u32 nowhere = 12345;
    err = proven_scan_fmt_cursor(&neg, "{}", proven_scan_arg_u32(&nowhere));
    EXAMPLE_REQUIRE(err != PROVEN_OK, "a negative value cannot be scanned into an unsigned destination");
    EXAMPLE_REQUIRE(nowhere == 12345, "and the destination is left as it was");

    /* --- 2. 입력이 빡빡하지 않을 때 커서 옮기기 -------------------------- */

    /* 자유로운 글: 수가 중요하고 그 사이의 낱말은 아니다. */
    proven_scan_t notes = proven_scan_init(
        PROVEN_LIT("   sample A measured 42 units; sample B measured -8 units"));

    /* skip_whitespace 는 공백과 탭과 줄바꿈을 지나 나아간다. 손으로 몰고 가는 형식의 필드
     * 사이에서 부르는 것이고, 실패하는 일이 없다. "공백이 없었다" 는 오류가 아니기
     * 때문이다. */
    proven_scan_skip_whitespace(&notes);
    EXAMPLE_REQUIRE(notes.cursor == 3, "the three leading spaces are consumed");

    /* skip_until_number 는 커서를 첫 숫자, 또는 곧바로 숫자가 따라오는 부호까지 몰고
     * 간다. "이 줄에서 수를 찾아라" 를 손으로 쓴 반복문에서 한 문장으로 바꿔 주는
     * 호출이다. */
    proven_scan_skip_until_number(&notes);
    proven_i32 first = 0;
    err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&first));
    EXAMPLE_REQUIRE(proven_is_ok(err) && first == 42, "the first number in the line is 42");
```

```
proven_scan_skip_until_number(&notes);
proven_i32 second = 0;
err = proven_scan_fmt_cursor(&notes, "{}", proven_scan_arg_i32(&second));
EXAMPLE_REQUIRE(proven_is_ok(err) && second == -8,
                "and the next one keeps its sign, because the sign is part of the number");

/* 끝에 이르면 더 찾을 것이 없고, 커서는 달려 나가는 대신 멈춘다. 파싱이 망가진 것이
 * 아니라 끝난 것이다. */
proven_scan_skip_until_number(&notes);
EXAMPLE_REQUIRE(notes.cursor == notes.view.size, "with no number left, the cursor lands at the end");

/* --- 3. 십진 글을 double 로, 로케일 없이 ---------------------------- */

proven_parse_double_result_t d = proven_parse_double_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(proven_is_ok(d.err), "parsing a decimal number must succeed");
EXAMPLE_REQUIRE(d.val > 3.14158 && d.val < 3.14160, "and produce the value the digits spell");

/* consumed 는 수가 어디서 끝났는지 알려 주므로 부르는 쪽이 거기서 이어 갈 수 있다 -
 * 목록을 토막으로 복사하지 않고 파싱하는 방법이 그것이다. */
EXAMPLE_REQUIRE(d.consumed == 7, "consumed reports exactly the bytes the number used");

/* 여기서 쉼표는 소수점이 아니고 앞으로도 아니다. 프로그램이 어느 로케일에서 돌든
 * 마찬가지다. 수는 쉼표에서 끝난다. */
proven_parse_double_result_t comma = proven_parse_double_ascii(PROVEN_LIT("3,5"));
EXAMPLE_REQUIRE(proven_is_ok(comma.err) && comma.consumed == 1 && comma.val == 3.0,
                "a comma ends the number: the parser is locale-free by construction");

/* proven_parse_f64_ascii 는 예전 이름을 단 같은 함수다. 지금 이름이 생기기 전에 쓰인
 * 코드를 위해 남겨 두었다. 새 코드는 proven_parse_double_ascii 를 쓸 것. 옛 호출
 * 자리도 그대로 읽히도록 둘 다 여기 있다. */
proven_parse_f64_result_t same = proven_parse_f64_ascii(PROVEN_LIT("3.14159 rest"));
EXAMPLE_REQUIRE(same.val == d.val && same.consumed == d.consumed,
                "the compatibility name is the same parser");

/* 아예 수가 아닌 글은 조용한 0 이 아니라 오류다 - atof() 였다면 진짜 "0" 과 구별할
 * 방법 없이 그것을 돌려주었을 것이다. */
proven_parse_double_result_t junk = proven_parse_double_ascii(PROVEN_LIT("not a number"));
EXAMPLE_REQUIRE(!proven_is_ok(junk.err), "unparsable text is reported, not turned into 0");
EXAMPLE_REQUIRE(junk.consumed == 0, "and nothing was consumed");

/* --- 4. float 를 다시 글로 ------------------------------------------ */

/* proven_arg_f64 가 부동소수점 값이 형식화기로 들어가는 길이다. float 도 double 도
 * 그것을 지나므로, 프로그램이 찍는 자릿수가 그 값이 어쩌다 담겨 있던 변수의 폭에
 * 좌우되지 않는다. */
proven_allocator_t alloc = proven_heap_allocator();
proven_result_u8str_t line = proven_u8str_create(alloc, 64);
EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");

proven_fmt_result_t out = proven_u8str_append_fmt(&line.value, "measured {} units",
                                                  proven_arg_f64(d.val));
EXAMPLE_REQUIRE(proven_is_ok(out.err), "formatting the parsed value must succeed");
EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&line.value),
                                              PROVEN_LIT("measured 3.14159")),
                "and spell the number it was given");

/* 정책을 주는 꼴이 명시적인 쪽이다. 기본값이 아니라 특정한 표기가 필요할 때 쓴다.
 * SHORTEST 는 다시 읽었을 때 정확히 이 값이 되는 가장 적은 자릿수를 청한다 -
 * 직렬화기가 원하는 성질이 그것이다. 필요 없는 수까지 열일곱 자리로 찍지 않고도
 * 왕복을 정확하게 만들어 주기 때문이다. */
```

```
    char shortest[64];
    proven_size_t wrote = 0;
    float measured = 0.1f;
    proven_err_t ferr = proven_float_format_f32_policy(shortest, sizeof shortest, measured,
                                                       PROVEN_FLOAT_FORMAT_POLICY_RYU,
                                                       proven_float_format_options_shortest(),
                                                       &wrote);
    EXAMPLE_REQUIRE(proven_is_ok(ferr), "formatting a float in shortest mode must succeed");
    EXAMPLE_REQUIRE(wrote > 0 && shortest[wrote] == '\0', "the result is written and terminated");
    EXAMPLE_REQUIRE(strcmp(shortest, "0.1") == 0,
                    "0.1f prints as 0.1: the shortest text that reads back as the same float");

    /* 그리고 왕복한다. 그 글은 자기가 나온 값으로 되읽힌다. 가장 짧은 방식이 존재하는
     * 이유가 그 보장이다. */
    proven_parse_double_result_t roundtrip = proven_parse_double_ascii(
        (proven_u8str_view_t){ .ptr = (const proven_byte_t *)shortest, .size = wrote });
    EXAMPLE_REQUIRE(proven_is_ok(roundtrip.err), "the shortest form parses back");
    EXAMPLE_REQUIRE((float)roundtrip.val == measured, "as exactly the float it was printed from");

    printf("scanned id=%lld bytes=%llu; formatted %s and %s\n",
           (long long)id, (unsigned long long)bytes, proven_u8str_as_cstr(&line.value), shortest);

    proven_u8str_destroy(alloc, &line.value);
    return EXAMPLE_OK();
}
```

반례 — 큰 값을 32비트 목적지로 읽는 경우:

```
proven_i32 bytes = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "bytes={}", proven_scan_arg_i32(&bytes));
/* input says 5368709120 */                                    /* wrong type */
```

값이 들어가지 않는다. 필드가 담을 수 있는 범위가 아니라 손에 익은 것으로 목적지 타입을 고르는 일은, 파일 오프셋을 `int`로 선언하는 것과 같은 실수다.

반례 — 기계를 건너다니는 데이터에 `atof`나 `strtod`를 쓰는 경우:

```
double v = strtod(text, NULL);   /* wrong: the decimal point depends on the locale */
```

게다가 `atof`는 실패를 보고할 방법이 아예 없다. 해석할 수 없는 텍스트가 `0`이 되고, 그것은 데이터 안의 진짜 0과 구분되지 않는다.

반례 — `skip_until_number`가 "없음"을 알려 준다고 가정하는 경우:

```
proven_scan_skip_until_number(&scan);
proven_i32 n = 0;
proven_err_t e = proven_scan_fmt_cursor(&scan, "{}", proven_scan_arg_i32(&n));  /* may fail */
```

이 함수는 커서를 옮길 뿐 아무것도 돌려주지 않는다. 남은 숫자가 없으면 커서는 입력의 끝에 놓이고, 그 사실을 알려 주는 것은 뒤따르는 스캔이다. 숫자를 찾았다고 가정하는 대신 그 에러를 확인한다.

실전 예제: 인자 타입을 손으로 이름 붙이기

`PROVEN_ARG(x)`와 `PROVEN_SCAN_ARG(&x)`는 넘겨받은 것의 타입에서 인자 생성자를 골라 주며, 대부분의 코드는 무엇이 골라졌는지 알 필요가 없다. 그 선택을 도로 가져와야 하는 상황이 둘 있다.

1. **매크로가 판단할 타입이 내 의도를 담고 있지 않을 때.** `proven_u8str_view_t`, 분해된 날짜, 진단용으로 찍는 주소 — `proven_arg_str_view()`, `proven_arg_datetime()`, `proven_arg_ptr()`가 있는 이유는 이들이 얼떨결에 빠지는 기본값이 아니라 의도적인 선택이기 때문이다.
2. **인자 목록을 실행 중에 만들 때.** 매개변수가 이미 `proven_arg_t`인 로그 헬퍼는 그것을 다시 감쌀 수 없다. 값을 인자로 바꾸는 매크로는 이미 인자인 것과는 상관이 없다. `proven_arg_identity()`와 `proven_scan_arg_identity()`가 바로 그 자리에서, 매크로로 굴러가는 같은 코드 경로가 미리 만들어진 인자를 받아들이게 해 준다.

예제는 양쪽을 이름으로 적는다. 형식화 쪽은 문자 하나, 불(bool) 값, 부호 없는 32·64비트 값, 텍스트를 넘기는 세 가지 방법(`proven_arg_str_view()`, `proven_arg_cstr()`, `proven_arg_ucstr()`), 날짜와 포인터. 스캔 쪽은 C의 모든 기본 정수 폭(`short`, `int`, `long`, `long long`과 그 부호 없는 형태), `double`, 그리고 복사 없이 붙잡는 문자열 view.

텍스트 생성자 셋은 호출자를 얼마나 믿는가에서 갈리며, 그 순서를 기억해 둘 값어치가 있다.

생성자	무엇을 받는가	무엇이 잘못될 수 있는가
`proven_arg_str_view`	포인터 **와** 길이	종결자를 찾아 훑는 일이 없으니 빠질 종결자도 없다 — 기본으로 이것을 쓴다
`proven_arg_cstr`	NUL로 끝나는 C 문자열	종결자가 있어야 하고 메모리가 살아 있어야 한다. 아니면 형식화기가 끝을 넘어 읽는다
`proven_arg_ucstr`	같은 것을 `unsigned char *`로	바이트 버퍼에 진짜 경고를 지우는 캐스트를 붙이지 않아도 되도록 존재한다

```
/*
 * 형식화기에 건네는 모든 값은 proven_arg_t 로 도착하고, 스캐너가 써 넣는 모든 목적지는
 * proven_scan_arg_t 로 도착한다. 대개는 둘 다 볼 일이 없다. PROVEN_ARG(x) 와
 * PROVEN_SCAN_ARG(&x) 가 건넨 것의 타입에서 알맞은 생성자를 골라 주고, 그것이 그
 * 매크로들의 존재 이유다.
 *
 * 두 상황에서는 그 선택을 매크로에게서 도로 가져오게 되는데, 둘 다 평범한 일이다.
 *
 *   1. 매크로가 알아볼 타입이 없을 때. `proven_u8str_view_t`, 쪼개 놓은 날짜, 진단용으로
 *      찍는 날주소 - 이런 것들은 생성자를 이름으로 불러야 한다. 여러분이 뜻하는 바를
 *      뜻하는, 매크로가 갈래를 태울 수 있는 평범한 C 타입이 없기 때문이다.
 *
 *   2. 인자 목록을 *실행 중에* 짓고 있을 때. "부르는 쪽이 이미 모아 둔 무엇이든" 을 받는
 *      로그 도우미는 int 나 문자열이 아니라 proven_arg_t 값을 받는다 - 그리고 값을
 *      인자로 바꾸는 매크로에 이미 인자인 것을 건넬 수는 없다. 항등 생성자가 그것을 위한
 *      것이다. 매크로로 굴러가는 같은 코드 길이, 앞서 만들어진 인자도 받아들이게 해 준다.
 *
 * 그래서 이 예제는 양쪽을 다 이름으로 적는다. 목적지의 폭과 부호 유무는 장식이 아니다 -
 * 70000 이 70000 으로 도착할지 4464 로 도착할지를 정하는 것이 그것이다.
 */

/* 부르는 쪽이 이미 만들어 둔 인자를 받는 로그 도우미. 매개변수가 proven_arg_t 이므로 그
 * 안에서 PROVEN_ARG 는 틀린 도구다 - 그 값은 이미 인자이고, 그렇다고 말해 주는 것이
 * proven_arg_identity 다. */
static proven_fmt_result_t log_pair(proven_u8str_t *out, const char *fmt,
                                    proven_arg_t a, proven_arg_t b) {
    return proven_u8str_append_fmt(out, fmt, proven_arg_identity(a), proven_arg_identity(b));
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_result_u8str_t line = proven_u8str_create(alloc, 256);
    EXAMPLE_REQUIRE(proven_is_ok(line.err), "creating the output string must succeed");
    if (!proven_is_ok(line.err)) {
        return 1;
    }
    proven_u8str_t out = line.value;

    /* --- 형식화 인자를 직접 이름으로 부르기 ------------------------------ */

    /* 문자 하나와 플래그 하나. bool 이 1/0 이 아니라 true/false 로 찍힌다는 것이 눈에
```

```
     * 보이도록 여기서는 이름으로 적었다. */
    proven_fmt_result_t r = proven_u8str_append_fmt(&out, "flag={} mark={}",
                                                    proven_arg_bool(true),
                                                    proven_arg_char('!'));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a bool and a char must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("flag=true mark=!")),
                    "a bool renders as a word, not as a digit");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* 부호 없는 폭들. 생성자가 그 값이 무엇인지에 대한 선언이다. 부호 없는 32비트 개수와
     * 부호 없는 64비트 바이트 합계는 서로 다른 사실이고, 그것을 적어 두면 자기가 가진
     * 것이 어느 쪽인지 말하게 된다. */
    r = proven_u8str_append_fmt(&out, "files={} bytes={}",
                                proven_arg_u32(1200u),
                                proven_arg_u64(5368709120ULL));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting unsigned values must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("files=1200
bytes=5368709120")),
                    "a 64-bit total is printed in full, not truncated to 32 bits");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* 형식화기에 글을 주는 세 가지 방법. 부르는 쪽을 얼마나 믿는지 순서대로.
     *
     *   proven_arg_str_view  - 포인터 *와* 길이. 종결자를 찾아 훑는 일이 없으므로,
     *                          없어서 문제가 될 종결자도 없다. 이것을 고를 것.
     *   proven_arg_cstr      - NUL 로 끝나는 C 문자열. 형식화기가 종결자를 찾아 걸으므로
     *                          그것이 반드시 있어야 하고, 그 기억도 살아 있어야 한다.
     *   proven_arg_ucstr     - 같은 것인데 `unsigned char *` 용이다. 바이트 버퍼는 보통
     *                          그 타입으로 적힌다. 부르는 쪽이 진짜 경고를 입막음하는
     *                          형변환을 쓰지 않아도 되도록 있는 것이다.
     */
    const char *name = "report.txt";
    const unsigned char *tag = (const unsigned char *)"draft";
    proven_u8str_view_t note = PROVEN_LIT("first pass");

    r = proven_u8str_append_fmt(&out, "{} [{}] {}",
                                proven_arg_cstr(name),
                                proven_arg_ucstr(tag),
                                proven_arg_str_view(note));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting the three text forms must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out),
                                         PROVEN_LIT("report.txt [draft] first pass")),
                    "all three produce the same kind of output from different kinds of pointer");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* 날짜 하나와 주소 하나. 둘 다 자기 뜻을 뜻하는 평범한 C 타입이 없다. 날짜는
     * 구조체이고, 진단용으로 찍는 주소는 우연히 빠지는 것이 아니라 일부러 하는 일이다. */
    proven_datetime_t when = proven_time_breakdown(0);   /* 기점: 고정된, 확인할 수 있는 값 */
    int local = 0;

    r = proven_u8str_append_fmt(&out, "at {} object {}",
                                proven_arg_datetime(when),
                                proven_arg_ptr(&local));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting a date and a pointer must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(proven_u8str_as_view(&out), PROVEN_LIT("at 1970-01-01")),
                    "the epoch breaks down to the first of January 1970");
    EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&out), 0, PROVEN_LIT("0x")) !=
```

```
PROVEN_SIZE_MAX,
                    "and an address is rendered in hexadecimal");

    EXAMPLE_REQUIRE(proven_is_ok(proven_u8str_reset(&out)), "clearing the line must succeed");

    /* 여기서 지은 인자를, 넘겨, 저기서 형식화한다. */
    r = log_pair(&out, "status={} retries={}", proven_arg_cstr("ok"), proven_arg_u32(3u));
    EXAMPLE_REQUIRE(proven_is_ok(r.err), "formatting pre-built arguments must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out), PROVEN_LIT("status=ok retries=3")),
                    "an argument built by the caller formats exactly as one built in place");

    /* --- 파싱 목적지를 직접 이름으로 부르기 ------------------------------ */

    /* 들어오는 쪽에서는 생성자가 목적지를 이름으로 밝히고, 목적지가 "너무 크다" 의 뜻을
     * 정한다. 이것들은 평범한 C 타입들 - short, int, long, long long 과 그 부호 없는 꼴 -
     * 이고, 이미 가진 변수가 고정 폭 타입이 아니라 그중 하나인 아주 흔한 경우를 위한
     * 것이다. */
    proven_scan_t scan = proven_scan_init(
        PROVEN_LIT("h=-32000 uh=65000 i=-2000000 ui=4000000000 l=-9000000 ul=9000000 "
                   "ll=-9007199254740993 ull=18446744073709551615 f=2.5 word=alpha"));

    short h = 0;
    unsigned short uh = 0;
    int i = 0;
    unsigned int ui = 0;
    long l = 0;
    unsigned long ul = 0;
    long long ll = 0;
    unsigned long long ull = 0;
    double f = 0.0;
    proven_u8str_view_t word = {0};

    proven_err_t err = proven_scan_fmt_cursor(
        &scan, "h={} uh={} i={} ui={} l={} ul={} ll={} ull={} f={} word={}",
        proven_scan_arg_short(&h),
        proven_scan_arg_ushort(&uh),
        proven_scan_arg_int(&i),
        proven_scan_arg_uint(&ui),
        proven_scan_arg_long(&l),
        proven_scan_arg_ulong(&ul),
        proven_scan_arg_llong(&ll),
        proven_scan_arg_ullong(&ull),
        proven_scan_arg_f64(&f),
        proven_scan_arg_str_view(&word));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning every plain integer width must succeed");
    EXAMPLE_REQUIRE(h == -32000 && uh == 65000u, "the short forms hold their values");
    EXAMPLE_REQUIRE(i == -2000000 && ui == 4000000000u, "and so do the int forms");
    EXAMPLE_REQUIRE(l == -9000000L && ul == 9000000UL, "and the long forms");
    EXAMPLE_REQUIRE(ll == -9007199254740993LL, "a value needing 64 bits arrives whole");
    EXAMPLE_REQUIRE(ull == 18446744073709551615ULL, "including the largest unsigned 64-bit value");
    EXAMPLE_REQUIRE(f > 2.4999 && f < 2.5001, "the floating-point destination reads a decimal");

    /* 파싱된 문자열 뷰는 파싱 중인 글 *안*을 가리킨다. 복사한 것도 할당한 것도 없으므로,
     * 그 글이 사는 동안 정확히 그만큼만 쓸 수 있다 - 파싱보다 오래 살아야 한다면
     * proven_u8str_t 로 복사할 것. */
    EXAMPLE_REQUIRE(proven_u8str_view_eq(word, PROVEN_LIT("alpha")), "the word is captured as a view");

    /* 목적지의 폭은 스캐너가 지키는 약속이다. 70000 은 short 에 들어가지 않으므로 4464 로
     * 감기는 대신 거부된다 - 형변환이었다면 아무도 다시 보지 않을 파일 안에서 조용히
     * 그렇게 만들어 냈을 값이다. */
```

```
    proven_scan_t narrow = proven_scan_init(PROVEN_LIT("70000"));
    short too_small = 7;
    err = proven_scan_fmt_cursor(&narrow, "{}", proven_scan_arg_short(&too_small));
    EXAMPLE_REQUIRE(err != PROVEN_OK, "a value that does not fit the destination is refused");
    EXAMPLE_REQUIRE(too_small == 7, "and the destination keeps the value it had");

    /* 파싱 쪽 항등 생성자. 이유는 형식화 쪽과 같다. 이미 만들어진 파싱 인자를 받는
     * 도우미는 그것을 다시 감쌀 수 없다. */
    proven_scan_t again = proven_scan_init(PROVEN_LIT("41"));
    proven_i32 answer = 0;
    proven_scan_arg_t prebuilt = proven_scan_arg_i32(&answer);
    err = proven_scan_fmt_cursor(&again, "{}", proven_scan_arg_identity(prebuilt));
    EXAMPLE_REQUIRE(proven_is_ok(err) && answer == 41, "a pre-built scan argument works unchanged");

    printf("arguments: %s\n", proven_u8str_as_cstr(&out));

    proven_u8str_destroy(alloc, &out);
    return EXAMPLE_OK();
}
```

반례 — 이미 인자인 것을 다시 감싸는 경우:

```
static proven_fmt_result_t log_one(proven_u8str_t *out, proven_arg_t a) {
    return proven_u8str_append_fmt(out, "{}", PROVEN_ARG(a));   /* wrong */
}
```

`proven_arg_identity(a)`를 쓴다. `PROVEN_ARG`는 값을 위한 것이고, `a`는 이미 인자다.

반례 — 텍스트가 사라진 뒤에도 스캔한 view를 들고 있는 경우:

```
proven_u8str_view_t word = {0};
{
    proven_u8str_t line = read_a_line(alloc);
    proven_scan_t s = proven_scan_init(proven_u8str_as_view(&line));
    proven_err_t e = proven_scan_fmt_cursor(&s, "word={}", proven_scan_arg_str_view(&word));
    proven_u8str_destroy(alloc, &line);     /* wrong: `word` pointed into `line` */
}
use(word);
```

스캔한 view는 스캔 대상 텍스트 안쪽을 가리킨다. 그 텍스트보다 오래 살아야 한다면 `proven_u8str_t`로 복사한다.

Chapter 5: Hosted Services

5부 — 운영체제와 대화하기. 선행 조건: 2부 (1, 2, 3). 이 장을 마치면 충돌 시에도 데이터를 잃지 않고 파일을 읽고 쓰며, 입력을 한 줄씩 읽고, 작업에 맞는 무작위성을 생성하고, 벽시계 시간과 경과 시간을 구별할 수 있다.

이 장에서는 `fs.h`, `sysio.h`, `mmap.h`, `time.h`, `stream.h`, `random.h`를 다룬다. 이 안의 모든 것은 운영체제를 필요로 한다: 이 장은 프리스탠딩(freestanding) 빌드에 적용되지 않는 매뉴얼의 유일한 부분이다.

이 API들은 호스티드 플랫폼 지원이 필요하며, 현재의 freestanding 하위 집합에서는 제외된다.

목차

1. 파일시스템 API
2. 시스템 I/O와 환경
3. 메모리 매핑
4. Time API
5. 예제와 오용 사례
6. 트리 순회
7. 스트림: writer와 reader
8. 표준 스트림
9. UTF-16 텍스트 입출력과 윈도 콘솔
10. 용도별 무작위성

1. 파일시스템 API

파일시스템 계층은 플랫폼 파일 핸들, 경로, 디렉터리 목록, 메타데이터, 권한, 링크, 잠금을 감싼다.

원시 파일시스템 헬퍼는 신뢰할 수 없는 경로를 정제하지 않고, 루트 감금(root confinement)을 강제하지 않으며, symlink-race TOCTOU를 방어하지 않는다. 신뢰할 수 없는 경로를 받아들이는 호출자는 API를 사용하기 전에 그것을 검증해야 한다.

구조체와 열거형

```
typedef struct {
    union {
        void *ptr;
        int fd;
    } internal;
} proven_fs_handle_t;

typedef proven_fs_handle_t proven_file_t;
```

의도: POSIX나 Win32 세부사항을 상위 계층에 노출하지 않으면서 플랫폼 파일 핸들을 표현한다.

```
typedef enum {
    PROVEN_FS_READ   = 1 << 0,
    PROVEN_FS_WRITE  = 1 << 1,
    PROVEN_FS_APPEND = 1 << 2,
    PROVEN_FS_CREATE = 1 << 3,
    PROVEN_FS_TRUNC  = 1 << 4,
    PROVEN_FS_CREATE_NEW = 1 << 5
} proven_fs_mode_t;
```

파일 모드 플래그는 조합할 수 있다. 예:

- `PROVEN_FS_READ`
- `PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC`
- `PROVEN_FS_WRITE | PROVEN_FS_APPEND | PROVEN_FS_CREATE`

```
typedef enum {
    PROVEN_FS_TYPE_FILE,
```

```
    PROVEN_FS_TYPE_DIR,
    PROVEN_FS_TYPE_OTHER
} proven_fs_type_t;

typedef struct {
    proven_u8str_t name;
    proven_fs_type_t type;
    proven_size_t size;
} proven_fs_entry_t;
```

디렉터리 엔트리는 `name`을 소유(owned)로 소유한다. `proven_fs_list()`가 반환한 배열에는 `proven_fs_list_destroy()`를 사용하라.

```
typedef struct {
    proven_err_t err;
    proven_file_t value;
} proven_result_file_t;

typedef enum {
    PROVEN_FS_PERM_OWNER_R = 1 << 8,
    PROVEN_FS_PERM_OWNER_W = 1 << 7,
    PROVEN_FS_PERM_OWNER_X = 1 << 6,
    PROVEN_FS_PERM_GROUP_R = 1 << 5,
    PROVEN_FS_PERM_GROUP_W = 1 << 4,
    PROVEN_FS_PERM_GROUP_X = 1 << 3,
    PROVEN_FS_PERM_OTHER_R = 1 << 2,
    PROVEN_FS_PERM_OTHER_W = 1 << 1,
    PROVEN_FS_PERM_OTHER_X = 1 << 0,
    PROVEN_FS_PERM_DEFAULT = PROVEN_FS_PERM_OWNER_R ¦ PROVEN_FS_PERM_OWNER_W ¦
                             PROVEN_FS_PERM_GROUP_R ¦ PROVEN_FS_PERM_OTHER_R
} proven_fs_perms_t;

typedef enum {
    PROVEN_FS_LOCK_SHARED,
    PROVEN_FS_LOCK_EXCLUSIVE,
    PROVEN_FS_LOCK_UNLOCK
} proven_fs_lock_type_t;

typedef struct {
    proven_size_t size;          /* size in bytes (0 for directories on some hosts) */
    proven_fs_type_t type;       /* PROVEN_FS_TYPE_FILE / _DIR - there is no symlink type */
    proven_fs_perms_t perms;     /* the nine permission bits only; the file type is in `type` */
    proven_i64 created_at;       /* always 0: no birth time is queried - see below */
    proven_i64 modified_at;      /* last-modification time, seconds since the Unix epoch */
    unsigned long long dev;      /* device id (POSIX st_dev; 0 where unavailable) */
    unsigned long long ino;      /* inode number (POSIX st_ino; 0 where unavailable) */
    unsigned long long uid;      /* owner id  (POSIX st_uid; 0 on Windows — no uid/gid) */
    unsigned long long gid;      /* group id  (POSIX st_gid; 0 on Windows) */
} proven_fs_stat_t;
```

`uid`/`gid`(v26.06.22a에서 추가됨)는 POSIX 소유자 및 그룹 id를 담는다. 둘 다 Windows에서는 `0`인데, Windows에는 `uid`/`gid` 개념이 없기 때문이다. `ls -l` 스타일의 소유자/그룹 열을 표시할 때는 호스트의 `getpwuid`/`getgrgid`로 이들을 이름으로 해석하라.

한 필드는 보이는 것보다 좁고, 다른 한 필드에는 날카로운 모서리가 있다:

- `created_at`은 **항상 0**이다. 평범한 `stat()`에는 이식 가능한 birth time이 없으며, PAL은 그것을 요청하지 않는다. `modified_at`만이 실제 타임스탬프를 담는다.
- `type`은 일반 파일이면 `FILE`, 디렉터리면 `DIR`, 그 밖의 모든 것—FIFO, 소켓, 디바이스, 끊어진(dangling) symlink—에 대해서는 **`PROVEN_FS_TYPE_OTHER`**이다. (v26.07.13a 이전까지는 이 모든 것이 `FILE`로 stat되어, 호출자에게 그것들을 열어 바이트를 읽을 수 있다고 알려주었다. 끊어진 symlink는 애초에 열 수 없다.)

type은 여기서도, 그리고 디렉터리 순회에서도 **symlink를 따라간다**. 그것이 두 값을 일치시키는 원리다: 일반 파일을 가리키는 symlink는 FILE이고, 디렉터리를 가리키는 symlink는 DIR이다. 이어지는 모서리는 실재한다—**재귀 순회기는 루프에 빠질 수 있다**. 조상을 가리키는 symlink는 사이클이고 그 type이 DIR이라고 말하기 때문이다. 깊이 제한을 두거나, (dev, ino) 쌍을 기억해 두고 이미 본 것으로는 내려가기를 거부하라.

```
proven_fs_stat_t st;
if (proven_is_ok(proven_fs_stat(scratch, PROVEN_LIT("/etc/hosts"), &st))) {
    proven_println("size={} uid={} gid={}",
        PROVEN_ARG(st.size), PROVEN_ARG(st.uid), PROVEN_ARG(st.gid));
}
```

매크로

매크로	의도
PROVEN_FS_PATH_SEP	라이브러리 수준에서 선호되는 구분 문자, 현재는 '/'.

파일 함수

API	의도	반환
proven_fs_open(scratch, path, mode)	파일 경로를 연다. scratch는 경로 변환에 사용된다.	proven_result_file_t.
proven_fs_close(file)	파일 핸들을 닫는다.	**에러를 반환한다.** 쓰기를 한 파일에서는 close *자체*가 쓰기의 일부다: NFS, CIFS 및 쿼터를 강제하는 파일시스템은 write-back 실패를 오직 이곳에서만 보고한다. 읽기만 한 파일에서는 (void)로 무시해도 괜찮다.
proven_fs_read(file, dest)	가변 슬라이스로의 단일 읽기 시도.	proven_result_size_t.
proven_fs_write(file, src)	바이트 view로부터의 단일 쓰기 시도.	proven_result_size_t.
proven_fs_write_all(file, src)	모든 바이트가 쓰이거나 에러가 발생할 때까지 재시도.	proven_err_t.
proven_fs_size(file)	열린 파일의 크기를 조회.	proven_result_size_t.
proven_fs_rename(scratch, src, dest)	경로 이름 변경 또는 이동. dest를 대체한다. src가 없으면 PROVEN_ERR_NOT_FOUND.	proven_err_t.
proven_fs_remove(scratch, path)	파일, 또는 빈 디렉터리 제거 (POSIX remove() 처럼, Windows에서도). 비어 있지 않은 디렉터리는 PROVEN_ERR_INVALID_STATE.	proven_err_t.
proven_fs_copy(temp_alloc, src, dest)	임시 버퍼 할당을 사용해 파일 복사.	proven_err_t.
proven_fs_mkdir(scratch, path)	디렉터리 하나 생성. 부모가 있어야 한다. 이름이 이미 쓰이고 있으면 PROVEN_ERR_EXISTS, 부모가 없으면 PROVEN_ERR_NOT_FOUND.	proven_err_t.
proven_fs_mkdir_all(scratch, path)	디렉터리와 그 위의 빠진 디렉터리를 모두 생성. 이미 디렉터리인	proven_err_t.

	단계는 오류가 아니고, 길을 막은 파일은 PROVEN_ERR_EXISTS.	
proven_fs_rmdir(scratch, path)	빈 디렉터리 제거. 없으면 PROVEN_ERR_NOT_FOUND, 비어 있지 않으면 PROVEN_ERR_INVALID_STATE.	proven_err_t.
proven_fs_list(alloc, path)	디렉터리를 proven_fs_entry_t의 proven_array_t로 나열.	proven_result_array_t.
proven_fs_list_destroy(alloc, list)	디렉터리 목록과 엔트리 이름을 파괴.	void.
proven_fs_chmod(scratch, path, perms)	권한 설정. 이름이 없으면 PROVEN_ERR_NOT_FOUND.	proven_err_t.
proven_fs_lock(file, type, wait)	파일 잠금 획득/해제. wait가 false일 때 남이 쥔 잠금은 PROVEN_ERR_BUSY.	proven_err_t.
proven_fs_stat(scratch, path, out_stat)	메타데이터 채우기.	proven_err_t.

proven_fs_stat()은 perms에 아홉 개의 권한 비트만 보고하므로, stat의 perms를 그대로 proven_fs_chmod()에 다시 넘길 수 있다. 예전에는 원시 POSIX st_mode를 담았는데, chmod가 거부하는 파일 타입 비트가 포함되어 있어서—이 필드의 자명한 용도인 그 왕복(round-trip)이 실제 파일마다 PROVEN_ERR_INVALID_ARG로 실패했다. 파일 타입은 type에서 읽어라.

위치, 그리고 atomic과 durable의 차이

seek할 수 없는 핸들은 그렇다고 말한다. 파이프, FIFO, 터미널은 proven_fs_seek에서 PROVEN_ERR_IO가 아니라 PROVEN_ERR_UNSUPPORTED를 반환한다. seek 불가능은 호출의 실패가 아니라 그 대상의 속성이며, 그것에 적응하는 코드—스캐너가 그렇다—는 둘을 구별할 수 있어야 한다.

pread와 pwrite는 위치를 이동하지 않는다. 그것이 바로 그들의 목적이다: 하나의 핸들을 공유하는 두 읽기 스트림(reader)는 어느 쪽도 이동시키지 않는 커서를 두고 경합할 수 없다.

atomic과 durable은 서로 다른 약속이며, 이 둘을 혼동하는 것이 데이터가 손실되는 방식이다.

- proven_fs_write_file_atomic은 *reader*가 절반만 쓰인 파일을 결코 보지 않도록 보장한다. 전원 차단에 대해서는 아무것도 말하지 않는다: 커널이 여전히 당신의 바이트를 붙들고 있을 수 있고, rename이 그것이 가리키는 데이터보다 먼저 디스크에 도달할 수 있다.
- proven_fs_write_file_durable은 그 창(window)을 닫는다. 오직 작동하는 하나의 순서로: 임시 파일을 fsync하고, **그 다음** rename하고, **그 다음** 디렉터리를 fsync한다. 파일은 sync하되 디렉터리는 하지 않으면, 바이트는 안전하지만 그것을 가리키는 이름은 안전하지 않은 크래시 창이 남는다—그것이 바로 atomic 쓰기가 막고자 존재하는 그 손상이다.

durable 형태는 저장 장치를 두 번 기다린다. 쓰기를 잃는 것이 기다림보다 나쁠 때는 이것을, 그렇지 않을 때는 atomic 형태를 사용하라.

예전에는 여기에 proven_sysio_flush가 있었는데, 그것은 이 어느 것도 아니었다: 존재하지 않는 버퍼를 flush한다고 주장했다. 그것은 **삭제되었다**. 버퍼드 쓰기 스트림(writer)의 바이트를 OS로 밀어 넣는 것은 proven_writer_flush이고, OS의 바이트를 디스크로 밀어 넣는 것은 proven_fs_sync이다. 이 둘은 서로 다른 연산이며, 한 단어가 정직하게 둘 모두를 뜻할 수는 없었다.

중요한 동작:

- proven_fs_read()와 proven_fs_write()는 단일 연산 API이며 요청된 것보다 적은 바이트를 처리할 수 있다.
- **파일 끝에서의 읽기는 0바이트 성공이 아니라 PROVEN_ERR_EOF를 반환한다.** 자명한 방식으로 작성된 루프—`if (r.value == 0) break;`—는 결코 그 분기를 타지 않으며, 파일의 끝을 I/O 실패로 취급한다. PROVEN_ERR_EOF를 명시적으로 검사하라. 이 장 끝의 완성 예제가 그 형태를 보여준다.

- 모든 바이트를 반드시 써야 할 때는 `proven_fs_write_all()`을 사용하라.
- 크기 0의 읽기/쓰기 요청은 non-null 버퍼를 요구하지 않고 0바이트를 처리한 채로 성공해야 한다.
- `proven_fs_is_absolute()`는 POSIX 절대 경로, Windows 드라이브-루트 경로, UNC 경로, 확장된 Windows 경로 형식을 인식한다.
- `proven_fs_read_all()`은 EOF까지 읽는다. 미리 측정된 크기까지 읽는 것이 아니다. 파일이 보고한 크기는 초기 용량의 씨앗(seed)으로만 쓰이므로, 일반 파일은 여전히 한 번의 할당과 한 번의 패스로 읽힌다. 이것이 중요한 이유는 `proven_fs_size()`가 일반 파일이 아닌 모든 것에 대해 0을 보고하기 때문이다: FIFO, 캐릭터 디바이스, `/proc` 엔트리에는 미리 알 수 있는 크기가 없으며, EOF까지 읽는 것만이 그 내용을 얻는 유일한 방법이다. 또한 이는 읽히는 도중 커지는 파일이 조용히 절단되지 않음을 뜻한다. `value.size`는 언제나 실제 바이트 수이며, 빈 소스는 `PROVEN_OK`와 함께 `{ .ptr = NULL, .size = 0 }`을 낳는다.
- `proven_fs_read_all()`과 `proven_fs_read_all_u8str()`은 소스가 보고된 크기를 초과해 커질 경우 `realloc_fn`을 가진 할당자(allocator)를 필요로 한다. 그렇지 않은(non-growing) allocator는 그 경우 `PROVEN_ERR_UNSUPPORTED`를 반환한다.
- `proven_fs_read_all()`은 소스가 끝날 때까지 상한 없이 읽는다 - 믿을 수 있는 경로에만 쓴다. 프로그램 밖에서 온 경로에는 `proven_fs_read_all_bounded()`를 쓴다. `max_bytes` 바이트를 넘는 소스를 `PROVEN_ERR_OUT_OF_BOUNDS`로 거부한다. 크기가 그렇다고 말하는 파일은 아무것도 할당하기 전에, 크기를 알 수 없는 소스(FIFO, `/proc`, `/dev/zero`)는 상한 다음 바이트가 도착하는 즉시 거부한다.

```
/* An upload path comes from a request, so it may name /dev/zero, a FIFO that never
 * ends, or a 40 GB file. proven_fs_read_all would read until the allocator gives up;
 * the bounded form stops at the limit and allocates no more than it. */
proven_result_mem_mut_t body = proven_fs_read_all_bounded(alloc, PROVEN_LIT("upload.bin"), 1u << 20);
if (body.err == PROVEN_ERR_OUT_OF_BOUNDS) {
    proven_eprintln("upload.bin is larger than 1 MiB; refused");
} else if (proven_is_ok(body.err)) {
    proven_println("read {} bytes", PROVEN_ARG(body.value.size));
    if (body.value.ptr) alloc.free_fn(alloc.ctx, body.value.ptr);
}
```

- `proven_fs_read_all_u8str()`은 대부분의 호출자가 원하는 파일 전체 읽기다: 결과가 NUL로 종료되므로 `proven_u8str_as_view()`와 `proven_u8str_as_cstr()`가 두 번째 복사 없이 그 위에서 동작한다. 종료 슬롯은 미리 예약되므로 추가 할당 비용이 들지 않는다. 내용은 UTF-8로 검증되지 않는다. `proven_u8str_destroy()`로 해제하라.
- `proven_fs_write_file()`은 atomic이 아니다: reader가 부분적으로 쓰인 파일을 관찰할 수 있고, 쓰기 도중의 실패는 파일을 절단된 채로 남긴다. `proven_fs_write_file_atomic()`은 형제 임시 파일을 쓰고 그것을 대상 위로 rename하므로, 동시 reader는 전체 이전 파일 또는 전체 새 파일 둘 중 하나를 본다. 이는 reader에 대해 atomic이지만 전원 손실에 대해 durable하지는 않다: rename이 데이터보다 먼저 디스크에 도달할 수 있다. durability가 필요할 때는 위에서 설명한 `proven_fs_write_file_durable`(또는 `proven_fs_sync`)로 명시적으로 요청하라.
- 임시 파일 이름은 `<path>.pvtmp`에 무작위 문자 13개를 붙인 것이라 예측할 수 없다. 남은 파일이나 다른 사용자가 미리 만든 이름이 쓰기를 막지 못하고, 이미 있는 이름은 건너뛴다(`PROVEN_ERR_EXISTS`는 열여섯 번 연속으로 이름이 있을 때만). 만든 뒤 rename하기 전에 죽은 writer는 임시 파일을 남기며, proven은 방금 자신이 만든 것이 아닌 임시 파일을 지우지 않는다 - 버려진 파일과 진행 중인 쓰기를 구별할 수 없기 때문이다. 오래 도는 서비스는 디렉터리를 나열해 `proven_fs_is_staging_name()`이 참인 오래된 항목을 지울 수 있다.

```
/* A cleanup pass a service might run at startup: count the staging files the atomic
 * writes of an earlier, killed run left in its state directory. Removing them is the
 * caller's decision - only the caller knows no writer is still filling one. */
proven_result_dir_t dir = proven_fs_dir_open(scratch, PROVEN_LIT("."));
if (proven_is_ok(dir.err)) {
    proven_size_t leftovers = 0;
    proven_fs_dir_entry_t entry;
    proven_err_t e;
    while ((e = proven_fs_dir_next(&dir.value, &entry)) != PROVEN_ERR_EOF) {
        if (proven_is_ok(e) && proven_fs_is_staging_name(entry.name)) ++leftovers;
```

```
        else if (e != PROVEN_OK && e != PROVEN_ERR_INVALID_ENCODING) break;
    }
    proven_fs_dir_close(&dir.value);
    proven_println("staging leftovers: {}", PROVEN_ARG(leftovers));
}
```

읽기 전용 대상은 넷 다 거절한다. `proven_fs_write_file`, `proven_fs_write_file_atomic`, `proven_fs_write_file_durable`, `proven_fs_copy`는 대상의 소유자 쓰기 비트가 꺼져 있으면 `PROVEN_ERR_PERMISSION`을 돌려주고 파일을 그대로 둔다. 그 비트가 두 플랫폼이 "이 파일에 쓰지 말라"를 기록하는 유일한 자리다 — POSIX에서는 모드 `0200`, Windows에서는 READONLY 속성이 같은 비트로 보고된다.

하나의 규칙인 이유는, 한 플랫폼 안에서 같은 질문에 세 가지 답이 나왔기 때문이다. `write_file`은 거절했다. 대상을 쓰기로 열기 때문이다. `write_file_atomic`은 성공했다. `rename`은 파일이 아니라 *디렉터리*에 권한을 묻기 때문에 파일의 모드를 아예 보지 않는다. `copy`는 성공했고 **그 뒤 파일을 쓰기 가능한 상태로 남겼다**. 원본의 모드를 옮겨 오기 때문이다. 호출자가 걸어 둔 보호가 사라졌는데 아무것도 그 사실을 말하지 않았다. 어느 함수를 골랐는지는 권한에 대한 결정이 아니다.

보호된 파일을 정말로 바꾸려는 호출자는 표시를 먼저 푼다 — 한 줄이고, 눈에 보인다. 거절은 되돌릴 수 있는 쪽이고, 실수로 덮어쓴 것은 되돌릴 수 없다. 보안 경계가 아니라 사고 방지 장치다: 모드는 작업 전에 읽고 작업 후에 반영되며, `chmod`를 할 수 있는 사람은 표시를 풀 수 있다.

`proven_fs_rename`도 그 목록에 있고, 있어야 한다. 대상을 대체하는 함수이고, atomic 쓰기가 바로 그 위에 지어져 있다 — 이것이 빠지면 한 함수에서 거절당한 호출자가 다른 함수로 같은 결과를 얻는다.
`proven_fs_remove`는 일부러 **빠져 있다**: 이름을 지우는 것은 디렉터리 연산이고, POSIX는 파일 자신의 모드에 발언권을 준 적이 없다. 이 지점에서 두 플랫폼은 실제로 다르다 — Windows는 읽기 전용 파일을 지우지 않는다 — 그래서 `proven_fs_remove`는 그 차이를 입출력 오류 뒤에 숨기지 않고 `PROVEN_ERR_PERMISSION`으로 보고한다.

거절은 이제 어떤 거절인지 말한다. `proven_fs_open`, `proven_fs_rename`, `proven_fs_remove`는 예전에 전부 `PROVEN_ERR_IO`로 답하던 자리에서 `PROVEN_ERR_NOT_FOUND`, `PROVEN_ERR_PERMISSION`, `PROVEN_ERR_BUSY`를 답한다. 사용자에게 묻기, 다시 시도하기, 포기하기는 서로 다른 세 가지 답이고, 오류 코드 하나로는 그중 어느 것도 할 수 없다.

디렉터리, 모드, 링크, 잠금 호출도 같은 규칙을 따른다. `proven_fs_mkdir`와 `proven_fs_link`는 만들라고 받은 이름이 이미 쓰이고 있으면 `PROVEN_ERR_EXISTS`를 답한다. `proven_fs_mkdir`, `proven_fs_rmdir`, `proven_fs_chmod`, `proven_fs_link`, `proven_fs_rename`은 이름이나 부모 디렉터리가 없으면 `PROVEN_ERR_NOT_FOUND`를, 플랫폼이 부르는 쪽을 거절하면 `PROVEN_ERR_PERMISSION`을 답한다. 기다리지 말라고 한 `proven_fs_lock`은 남이 쥔 잠금에 `PROVEN_ERR_BUSY`를 답한다. 두 가지 거절에는 이미 뜻이 맞는 코드가 붙는다. 비어 있지 않은 디렉터리는 `proven_fs_rmdir`와 `proven_fs_remove`에 `PROVEN_ERR_INVALID_STATE`를 답한다. 실패한 것은 없고, 고칠 것은 호출 순서다 — 먼저 비워라. 파일 시스템을 건너야 하는 하드 링크나 이름 변경은 `PROVEN_ERR_UNSUPPORTED`를 답한다. 그 길은 없으니 다른 길을 택한다 — 복사한 뒤 지운다.

`proven_fs_mkdir`는 한 단계를 만들고, 이름이 이미 쓰이고 있으면 그렇다고 보고한다. 한 단계에서는 그것이 정보이기 때문이다. 경로 전체가 있기만 하면 되고 누가 만들었는지는 상관없는 호출자는 `proven_fs_mkdir_all`을 부른다. 빠진 단계를 모두 만들고, 이미 디렉터리인 단계는 끝난 것으로 치며, 어떤 단계가 디렉터리가 아닌 다른 것이면 여전히 `PROVEN_ERR_EXISTS`로 거절한다. 실패하기 전에 만든 디렉터리는 그대로 남는다.

잘못된 예 — `proven_fs_mkdir`의 모든 거절을 실패로 읽기:

```
proven_err_t e = proven_fs_mkdir(alloc, dir);
if (!proven_is_ok(e)) return e;                    /* wrong: stops when dir is already there */
```

두 번 실행한 프로그램은 두 번째에, 자기가 만든 디렉터리에서 실패한다. 뜻하는 바를 그대로 요청하거나 — `proven_fs_mkdir_all`, "돌아올 때 이 경로는 디렉터리다" — 아니면 `PROVEN_ERR_EXISTS`를 의도적으로 받아들여라.

잘못된 예 — 무엇이 있는지 묻지 않고 `PROVEN_ERR_EXISTS`를 받아들이기:

```
proven_err_t e = proven_fs_mkdir(alloc, dir);
if (e == PROVEN_ERR_EXISTS) e = PROVEN_OK;         /* wrong: a FILE named dir also answers this */
```

proven_fs_mkdir는 그 이름을 무엇이 차지하고 있는지 보지 않는다. 같은 이름의 파일도 같은 답을 주고, 뒤따르는 dir/... 쓰기는 덜 분명한 곳에서 실패한다. proven_fs_mkdir_all은 물어보고, 디렉터리일 때만 PROVEN_OK를 돌려준다.

누가 읽고 있는 파일도 바꿔 넣는다. 다른 프로세스가 대상을 열어 두고 있어도 그 프로세스가 삭제 공유(delete sharing)를 허용했다면 — proven_fs_open이 그렇게 연다 — atomic 쓰기는 성공하고, 읽던 쪽은 열어 둔 핸들로 옛 내용을 끝까지 읽는다. POSIX가 원래 그렇고, Windows 10 1809 이후에서는 POSIX 방식 rename으로 같은 동작을 한다. 그 rename을 모르는 곳 — 1809 이전 Windows, FAT32·exFAT, 네트워크 드라이브 — 에서는 MoveFileExW로 물러서며, 거기서는 누가 열어 두기만 해도 교체가 거절되고 PROVEN_ERR_BUSY가 온다. 삭제 공유를 허용하지 않고 연 파일은 어느 Windows에서든 PROVEN_ERR_BUSY다. 보호가 아니라 사용 중이므로, 다시 시도하면 될 수 있다.

예:

```
proven_result_file_t of = proven_fs_open(
    alloc,
    PROVEN_LIT("out.txt"),
    PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC
);
if (proven_is_ok(of.err)) {
    proven_err_t e = proven_fs_write_all(
        of.value,
        proven_mem_view_from_u8(PROVEN_LIT("hello\n"))
    );
    /* The close is part of the write. Close on the failure path too - the handle is
     * ours either way - but do not throw the answer away: on a network filesystem or
     * over quota, close() is the only place the failure appears. */
    proven_err_t ce = proven_fs_close(of.value);
    if (proven_is_ok(e)) e = ce;
    if (!proven_is_ok(e)) {
        proven_eprintln("writing out.txt failed");
    }
}
```

2. 시스템 입출력과 환경변수

표준 스트림

```
proven_file_t proven_sysio_stdin(void);
proven_file_t proven_sysio_stdout(void);
proven_file_t proven_sysio_stderr(void);
```

목적: 표준 스트림을 proven_file_t 핸들로 노출한다. 이들은 또한 writer와 reader이기도 하다— 아래 표준 스트림을 보라. 그것이 stdin을 한 줄씩 읽고, stdout을 버퍼링하며, 둘 중 어느 쪽으로든 직접 포매팅할 수 있게 해준다.

proven_sysio_scanner_t

```
typedef struct {
    proven_file_t file;
    proven_allocator_t alloc;
    proven_u8 *buffer;
    proven_size_t capacity;
    proven_size_t cursor;
    proven_size_t length;
    bool eof;
} proven_sysio_scanner_t;
```

목적: 경계가 있는 스트림 입력을 위한 버퍼드 스캐너로, seek 가능한 파일뿐 아니라 파이프와 stdin에도 안전하다. 토큰이 EOF 전에 현재 로드된 조각의 끝에 도달하면, 스캐너는 버퍼를 다시 채우고 재시도한다. 다시

채운 후에도 버퍼 전체에 들어갈 수 없는 토큰만이 절단되어 받아들여지는 대신 PROVEN_ERR_OUT_OF_BOUNDS로 거부된다.

Sysio 함수와 매크로

API	의도	반환
proven_sysio_scanner_init(scanner, file, alloc, buffer_capacity)	스캐너 버퍼를 할당하고 스트림을 바인딩.	proven_err_t.
proven_sysio_scanner_deinit(scanner)	스캐너 버퍼를 해제.	void.
proven_sysio_scanner_scan_impl(scanner, fmt, args, args_count)	내부 스캐너 엔진.	proven_err_t.
proven_sysio_scanner_scan(scanner, fmt, ...)	타입 안전 버퍼드 스캔 매크로.	proven_err_t.
proven_sysio_print_impl(handle, fmt, args, args_count)	내부 출력 엔진.	proven_err_t.
proven_sysio_scan_chunk_impl(handle, fmt, args, args_count)	한-청크 스캔 엔진.	proven_err_t.
proven_print(fmt, ...)	stdout으로 출력.	proven_err_t.
proven_println(fmt, ...)	개행과 함께 stdout으로 출력.	proven_err_t.
proven_eprint(fmt, ...)	stderr로 출력.	proven_err_t.
proven_eprintln(fmt, ...)	개행과 함께 stderr로 출력.	proven_err_t.
proven_scan_fmt_from_file(file, fmt, ...)	파일의 고정 크기 청크 하나에서 스캔.	proven_err_t.
proven_scan_fmt_from_stdin(fmt, ...)	stdin에서 고정 크기 청크 하나를 스캔.	proven_err_t.
proven_env_get(alloc, key)	환경 변수를 owned U8 문자열로 읽음.	proven_result_u8str_t.

proven_sysio_scan_chunk_impl()은 seek 가능한 파일 입력을 위한 것이다. 최대 하나의 고정 크기 청크를 읽는다. 핸들을 되감을 수 없으면, 읽기 전에 PROVEN_ERR_UNSUPPORTED를 반환한다. 완전한 토큰이 준비되기 전에 청크가 가득 차면, PROVEN_ERR_OUT_OF_BOUNDS를 반환하고 파일 커서를 청크의 시작으로 되감는다. 문자열 뷰(view) 목적지도 읽기 전에 PROVEN_ERR_UNSUPPORTED로 거부한다. 그런 view는 이 함수의 지역 청크 버퍼를 빌리므로 반환 즉시 무효가 되기 때문이다. 반복적인 버퍼드 스캔이나 빌린 문자열 결과에는 호출자가 버퍼를 소유하는 proven_sysio_scanner_t를 사용하라. 그 스캐너가 반환한 문자열 view는 다음 scan 호출이나 스캐너 해제 전까지만 유효하다. 어느 쪽이든 공유 버퍼를 다시 채우거나 압축하거나 해제할 수 있기 때문이다.

예:

```
proven_println("answer={}", PROVEN_ARG(42));
proven_eprintln("warning: {}", PROVEN_ARG(PROVEN_LIT("low memory")));
```

환경 변수 예. proven_env_get은 owned 문자열을 돌려주므로 반드시 파괴해야 한다. 빈 문자열로 설정된 변수는 오류가 아니라 빈 문자열이고, 올바른 텍스트가 아닌 값(POSIX의 UTF-8이 아닌 바이트, Windows의 짝 없는 서로게이트)은 PROVEN_ERR_INVALID_ENCODING이다 - 무엇도 대신 넣지 않는다. 디렉터리 엔트리 이름도 같은 규칙을 따른다:

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t path = proven_u8str_as_view(&env.value);
    proven_println("PATH is {} bytes", PROVEN_ARG(path.size));
    proven_u8str_destroy(alloc, &env.value);
}
```

3. 메모리 매핑

문제: 복사하고 싶지 않은 파일 읽기

`proven_fs_read_all_u8str`는 파일을 당신이 소유하는 메모리로 읽어 들인다. 설정 파일이라면 그것이 정확히 옳다. 하지만 4 GB짜리 데이터베이스, 메모리 매핑된 인덱스, 또는 두 프로세스가 동시에 봐야 하는 파일이라면 세 가지 면에서 틀렸다: 4 GB의 RAM이 필요하고, 읽든 읽지 않든 모든 바이트를 복사하며, 그 복사본은 오직 당신만의 것이다.

메모리 매핑은 대신 운영체제에게 파일의 내용이 어떤 주소에 나타나게 해 달라고 요청한다. 미리 복사되는 것은 없다. 당신이 건드리는 페이지만 필요할 때 디스크에서 읽히고, 결코 건드리지 않는 페이지는 결코 읽히지 않는다. 두 프로세스가 같은 파일을 `PROVEN_MMAP_SHARED`로 매핑하면 하나의 페이지 집합을 보게 되므로, 한쪽에서의 쓰기가 다른 쪽에서 보인다.

이것의 비용, 그리고 쓰지 말아야 할 때

실패 양상이 오류처럼 보이지 않기 때문에, 이 절은 매뉴얼에서 득실이 가장 첨예한 부분이다:

- **읽기가 fault를 일으킬 수 있다.** 파일이 메모리에 있으면 I/O 오류는 반환값이다. 매핑에서는, 디스크 읽기가 실패하는 페이지를 건드리면 오류 코드가 아니라 **시그널**(`SIGBUS`)이 전달된다. 이를 우회하도록 작성할 수 있는 `if`는 없다.
- **잘림(truncation)은 지뢰다.** 매핑한 뒤에 다른 프로세스가 파일을 줄이면, 새 끝을 넘어선 페이지를 건드리는 것 역시 `SIGBUS`다. 다른 무언가가 잘라낼 수 있는 파일을 매핑하는 것은 어떤 API도 대신 고쳐 줄 수 없는 방식으로 안전하지 않다.
- **공짜가 아니다.** 매핑을 설정하는 것은 syscall이자 페이지 테이블 작업이며, 첫 접촉마다 페이지 fault가 난다. 작은 파일이라면 `proven_fs_read_all_u8str`가 그냥 더 빠르다.
- **`proven_mmap_sync`가 지속성의 지점이다.** 공유 매핑에 대한 쓰기는 결국 파일에 도달한다. 지금 디스크에 있어야 한다면, 요청하라.

큰 파일을 드문드문 읽을 때, 프로세스 간에 공유하는 읽기 전용 데이터에, 그리고 큰 파일에 대한 임의 접근에 매핑을 쓰라. 그 밖의 모든 것에는 평범한 읽기를 쓰라.

매핑은 호출자가 소유하는 상태다: `proven_mmap_as_view`는 **매핑 안을 가리키는** view를 건네주므로, 그 view는 `proven_mmap_destroy`가 실행되는 순간 죽는다.

잘못된 예 — 매핑을 파괴한 뒤에 view를 사용하기:

```
proven_u8str_view_t data = proven_mmap_as_view(m);
proven_err_t e = proven_mmap_destroy(&m);
(void)e;
parse(data);                /* wrong: those addresses are no longer mapped - SIGSEGV */
```

잘못된 예 — 매핑을 키울 수 있는 버퍼처럼 취급하기:

```
/* wrong: a mapping is a window onto a file of a fixed size at map time.
   Appending means changing the file and mapping it again. */
```

구조체와 열거형

```
typedef enum {
    PROVEN_MMAP_READ  = 0x01,
    PROVEN_MMAP_WRITE = 0x02,
    PROVEN_MMAP_EXEC  = 0x04
} proven_mmap_prot_t;

typedef enum {
    PROVEN_MMAP_PRIVATE = 0x01,
    PROVEN_MMAP_SHARED  = 0x02
} proven_mmap_flags_t;

typedef struct {
    void *ptr;
    proven_size_t size;
```

```
    proven_fs_handle_t file;
    void *internal_handle;
} proven_mmap_t;

typedef struct {
    proven_err_t err;
    proven_mmap_t value;
} proven_result_mmap_t;
```

함수

API	의도	반환
`proven_mmap_create(file, offset, size, prot, flags)`	파일 영역을 매핑. `size == 0`은 EOF까지 매핑한다. 오프셋은 플랫폼의 매핑 세분성(granularity)과 일치해야 한다.	`proven_result_mmap_t`.
`proven_mmap_destroy(mmap)`	영역을 언매핑하고 상태를 정리.	`proven_err_t`.
`proven_mmap_sync(mmap)`	공유된 쓰기 가능 변경분을 저장소로 flush.	`proven_err_t`.
`proven_mmap_as_view(mmap)`	매핑된 메모리를 U8 view로 빌림(borrow).	`proven_u8str_view_t`.

예:

```
proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("data.bin"), PROVEN_FS_READ);
if (proven_is_ok(f.err)) {
    proven_result_mmap_t mr = proven_mmap_create(
        f.value,
        0,
        0,
        PROVEN_MMAP_READ,
        PROVEN_MMAP_PRIVATE
    );
    if (proven_is_ok(mr.err)) {
        /* The view borrows the mapping: it dies when the mapping is destroyed. */
        proven_u8str_view_t bytes = proven_mmap_as_view(mr.value);
        proven_println("mapped {} bytes", PROVEN_ARG(bytes.size));
        (void)proven_mmap_destroy(&mr.value);
    }
    (void)proven_fs_close(f.value);
}
```

4. 시간 API

문제: 서로 다른 두 질문, 하나의 단어
"시간"은 서로 닮았지만 하는 짓은 전혀 다른 두 가지를 뜻한다.

벽시계(wall clock)는 *지금 몇 시인가?*에 답한다 — 사용자가 읽는 그것이다. 이 시계는 튀어도 된다: NTP가 보정하고, 서머타임이 옮기고, 관리자가 설정한다. 이것으로 경과 시간을 재면 음수가 나올 수 있는데, 이는 윤초가 있는 날에 나타나고 테스트 중인 누구의 노트북에서도 나타나지 않는 버그다.

단조(monotonic) 시계는 *언제부터 얼마나 지났는가?*에 답한다. 앞으로만 움직이며 어떤 달력과도 관계가 없다. 연산의 시간을 잴 때 쓰는 것이 이것이다.

libc는 이 구분을 흐린다. `time()`은 벽시계의 초 단위 정수를 준다 — 무언가를 재기에는 너무 거칠다. `clock()`은 **CPU** 시간을 재므로, 소켓을 기다리는 프로그램은 시간이 전혀 걸리지 않은 것처럼 보인다. 어느 이름도 자신이 어느 질문에 답하는지 말해 주지 않으며, 둘 다 서로의 용도로 흔히 쓰인다.

이 라이브러리는 대신 무엇을 하는가

시계는 둘이고, 각각 할 일이 하나다.

`proven_time_now()`는 **벽시계**다: Unix epoch 이후의 나노초를 부호 있는 64비트 값으로 준다. 날짜가 있고, NTP나 관리자가 맞추는 시계이므로 어느 방향으로든 건너뛸 수 있다. 분해하거나 포맷하는 데 쓴다.

`proven_time_monotonic_now()`는 **단조 시계**다: 프로세스가 사는 동안 고정된 기점부터의 나노초다. 뒤로 가지 않고 아무도 맞추지 않으므로, 두 번 읽은 값의 차이가 경과 시간이다. 날짜가 없고, 읽은 값은 다른 프로세스나 다음 실행에서는 아무 뜻도 없다. 경과 시간, 타임아웃, 기한은 이것으로 잰다.

`proven_time_breakdown()`은 그 숫자를 `proven_datetime_t`로 바꾸며, 그 필드는 사람이 쓰는 것들이다: `month`는 libc의 0-11이 아니라 **1-12**이고, `year`는 1900년 이후의 햇수가 아니라 실제 연도다. `struct tm`의 그 두 가지 off-by-N 관례는 의도적으로 갈라설 만큼 충분히 많은 버그를 낳았다.

포맷팅은 3장과 같은 `{}` 자리표시자를 쓴다: 이름이 필드를 고르고 스펙이 자리를 채우므로, `{month:0>2}`는 너비 2로 0을 채운다. 월과 요일 이름은 `proven_time_locale_t`에서 오므로, 다른 언어로 렌더링하는 것은 전역을 설정하는 것이 아니라 다른 locale을 넘기는 일이다.

잘못된 예 — 벽시계로 시간을 재고 부호를 믿기:

```
proven_time_t t0 = proven_time_now();
do_work();
proven_time_t t1 = proven_time_now();
proven_u64 ns = (proven_u64)(t1 - t0);   /* wrong: an NTP step back makes this enormous */
```

올바른 예 — 경과 시간은 건너뛸 수 없는 단조 시계에서 온다:

```
proven_time_t t0 = proven_time_monotonic_now();
do_work();
proven_i64 elapsed_ns = proven_time_monotonic_now() - t0;   /* never negative */
```

가진 것이 벽시계 값 둘뿐이라면 - 로그의 타임스탬프 둘 - 차이를 부호 있는 값으로 유지하고, 음수는 지속 시간이 아니라 "시계가 움직였다"로 취급하라.

잘못된 예 — sleep이 정확하다고 가정하기:

```
proven_time_sleep(15);
/* wrong to assume exactly 15ms have passed: sleep guarantees AT LEAST that long,
   and the scheduler decides when you actually run again. */
```

완성 예제: 경과 시간, 날짜, 그리고 포맷된 타임스탬프

테스트 스위트가 컴파일하고 실행한다. 이 예제가 무엇을 단언하지 *않는지*에 주목하라 — sleep의 상한이다. 그것을 단언하면 바쁜 기계에서 실패하는 테스트가 되기 때문이다.

```
/*
 * 시간에는 똑같이 생겼지만 같지 않은 두 갈래가 있고, 잘못 고르는 것이 시간 재기의
 * 고전적 버그다.
 *
 *   - *벽시계*는 "지금 몇 시인가" 에 답한다. 사용자가 보고 싶어 하는 것이고, 뛰는 것이
 *     허용된다 - NTP 가 고치고, 서머타임이 옮기고, 관리자가 맞춘다. 그것으로 기간을
 *     재면 음수 경과 시간이 나올 수 있고, 윤초가 있던 날에 실제로 유명하게 그랬다.
 *
 *   - *단조* 시계는 "그때로부터 얼마나" 에 답한다. 앞으로만, 고른 속도로 가고, 어떤
 *     달력과도 관계가 없다. 어떤 작업의 시간을 잴 때 쓰는 것이 이것이다.
 *
 * libc 는 이 구분을 흐린다. time() 은 초 단위 벽시계라 재는 데는 쓸모없다. clock() 은
 * 경과 시간이 아니라 CPU 시간을 재므로, 잠든 프로그램은 즉시 끝난 것처럼 보인다. 어느
 * 이름도 자기가 둘 중 어느 물음에 답하는지 말해 주지 않는다.
 *
 * 여기서는 둘이 각자의 호출을 갖는다. proven_time_now() 는 벽시계, 곧 유닉스 기점부터의
 * 나노초이고 날짜로 형식화된다. proven_time_monotonic_now() 는 단조 시계, 곧 이 프로세스
 * 밖에서는 아무 뜻도 없는 기점부터의 나노초이고, 두 번 읽은 값을 빼면 기간이 된다.
 */
```

```
int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 기간으로 쓰기 ------------------------------------------------- */
    proven_time_t start = proven_time_now();            /* 벽시계: 아래의 날짜에 쓴다 */
    proven_time_t t0 = proven_time_monotonic_now();     /* 단조 시계: 기간에 쓴다 */
    proven_time_sleep(15);                 /* 밀리초 */
    proven_i64 elapsed_ns = proven_time_monotonic_now() - t0;

    EXAMPLE_REQUIRE(elapsed_ns > 0, "the monotonic clock moves forward across a sleep");
    /* sleep 은 청한 시간 *이상*을 보장하지, 이하를 보장하지 않는다. 언제 다시 돌지는
     * 스케줄러가 정한다. 여기서 상한을 단언하면 바쁜 기계에서 실패하는 시험이 되고,
     * 그래서 이 예제는 그러지 않는다. */
    EXAMPLE_REQUIRE(elapsed_ns >= 10 * 1000 * 1000,
                    "sleeping 15ms must take at least ~10ms");

    /* --- 날짜로 쓰기 --------------------------------------------------- */
    proven_datetime_t dt = proven_time_breakdown(start);
    EXAMPLE_REQUIRE(dt.year >= 2020 && dt.year < 3000, "the epoch breakdown gives a plausible year");
    EXAMPLE_REQUIRE(dt.month >= 1 && dt.month <= 12, "month is 1-12, not 0-11 as in libc's tm");
    EXAMPLE_REQUIRE(dt.day >= 1 && dt.day <= 31, "day is 1-31");
    EXAMPLE_REQUIRE(dt.hour <= 23 && dt.min <= 59 && dt.sec <= 60, "sec allows 60 for leap seconds");
    EXAMPLE_REQUIRE(dt.weekday <= 6, "weekday is 0-6 with 0 = Sunday");

    /* proven_time_now_datetime() 은 위의 두 호출을 하나로 합친 것이다. 달력 꼴만
     * 필요할 때 쓴다. */
    proven_datetime_t now = proven_time_now_datetime();
    EXAMPLE_REQUIRE(now.year == dt.year, "both routes read the same clock");

    /* --- 시각을 글자로 ------------------------------------------------- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "a 64-byte string is enough for a timestamp");

    /* 달 이름과 요일 이름은 로케일이 준다. proven_time_locale_en 이 내장된 영어
     * 로케일이다. 다른 언어로 찍으려면 여러분의 것을 건넨다. */
    proven_err_t err = proven_time_u8_fmt(alloc, &s.value, dt, &proven_time_locale_en,
                                          "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}");
    EXAMPLE_REQUIRE(proven_is_ok(err), "formatting a datetime should succeed");

    proven_u8str_view_t out = proven_u8str_as_view(&s.value);
    EXAMPLE_REQUIRE(out.size == 19, "year-month-day hour:min:sec is exactly 19 characters");
    EXAMPLE_REQUIRE(out.ptr[4] == '-' && out.ptr[7] == '-' && out.ptr[13] == ':',
                    "the separators land where the pattern put them");

    proven_println("formatted: {}", PROVEN_ARG(out));

    proven_u8str_destroy(alloc, &s.value);
    return EXAMPLE_OK();
}
```

타입

```
typedef proven_i64 proven_time_t;
```

UNIX epoch 이후 나노초.

```
typedef struct {
    proven_i32 year;
    proven_u8 month;
    proven_u8 day;
    proven_u8 hour;
```

```
    proven_u8 min;
    proven_u8 sec;
    proven_u32 ms;
    proven_u8 weekday;
} proven_datetime_t;
```

필드 범위:

- `month`: 1 ~ 12.
- `day`: 1 ~ 31.
- `hour`: 0 ~ 23.
- `min`: 0 ~ 59.
- `sec`: 0 ~ 60.
- `ms`: 0 ~ 999.
- `weekday`: 0 ~ 6, 일요일이 0.

```
typedef struct {
    const proven_u8str_view_t *month_names;
    const proven_u8str_view_t *month_short_names;
    const proven_u8str_view_t *weekday_names;
    const proven_u8str_view_t *weekday_short_names;
} proven_time_locale_t;
```

`proven_time_locale_en`은 기본 영어 로케일이다.

함수

API	의도	반환
`proven_time_u8_fmt(alloc, str, dt, locale, fmt)`	포맷된 날짜시간을 U8 문자열에 덧붙임.	`proven_err_t`.
`proven_time_u16_fmt(alloc, str, dt, locale, fmt)`	U16이 비활성화되지 않은 한 포맷된 날짜시간을 U16 문자열에 덧붙임.	`proven_err_t`.
`proven_time_now()`	현재 타임스탬프(나노초).	`proven_time_t`.
`proven_time_breakdown(time_ns)`	epoch 나노초를 분해된 UTC 시간으로 변환.	`proven_datetime_t`.
`proven_time_monotonic_now()`	단조 시계: 프로세스별 기점부터의 나노초. 두 값을 빼서 경과 시간을 얻는다. 날짜가 아니다.	`proven_time_t`.
`proven_time_now_datetime()`	현재 로컬 분해 시간.	`proven_datetime_t`.
`proven_time_sleep(ms)`	밀리초 동안 sleep.	void.

날짜시간 포맷 키:

- `{year}`, `{month}`, `{day}`, `{hour}`, `{min}`, `{sec}`, `{ms}`, `{wday_num}`
- 로케일을 사용하는 `{Month}`, `{mon}`
- 로케일을 사용하는 `{Weekday}`, `{wday}`

예:

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_u8str_t s = r.value;

    proven_datetime_t now = proven_time_now_datetime();
    proven_err_t e = proven_time_u8_fmt(
        alloc,
        &s,
        now,
```

```
        &proven_time_locale_en,
        "{year}-{month:0>2}-{day:0>2} {hour:0>2}:{min:0>2}:{sec:0>2}"
    );
    if (proven_is_ok(e)) {
        proven_println("{}", PROVEN_ARG(proven_u8str_as_view(&s)));
    }

    proven_u8str_destroy(alloc, &s);
}
```

5. 예제와 오용 사례

단일 쓰기는 부분적일 수 있다

잘못됨(Wrong):

```
proven_result_size_t w = proven_fs_write(file, data);
/* wrong: one write may be partial, and w.value is never looked at */
```

올바름(Correct):

```
proven_result_file_t f = proven_fs_open(alloc, PROVEN_LIT("out.txt"),
                                        PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
if (proven_is_ok(f.err)) {
    proven_mem_view_t data = proven_mem_view_from_u8(PROVEN_LIT("payload"));
    proven_err_t e = proven_fs_write_all(f.value, data);   /* loops until done */
    proven_err_t ce = proven_fs_close(f.value);            /* and the close can fail too */
    if (proven_is_ok(e)) e = ce;
    (void)e;
}
```

디렉터리 목록에는 특별한 파괴가 필요하다

잘못됨(Wrong):

```
proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
PROVEN_ARRAY_DESTROY(&r.value); /* wrong: entry names leak */
```

올바름(Correct):

```
proven_result_array_t r = proven_fs_list(alloc, PROVEN_LIT("."));
if (proven_is_ok(r.err)) {
    proven_fs_list_destroy(alloc, &r.value);
}
```

destroy 이후에 매핑을 사용하지 말라

잘못됨(Wrong):

```
proven_mmap_destroy(&map);
use_bytes(map.ptr, map.size); /* wrong: mapping has been released */
```

스트림에는 버퍼드 sysio 스캔을 사용하라

stdin이나 파이프에서 반복적으로 읽을 때는 다음을 선호하라:

```
proven_sysio_scanner_t scanner = {0};
proven_err_t e = proven_sysio_scanner_init(&scanner, proven_sysio_stdin(), alloc, 4096);
if (proven_is_ok(e)) {
    int value = 0;
    e = proven_sysio_scanner_scan(&scanner, "{}", PROVEN_SCAN_ARG(&value));
    proven_sysio_scanner_deinit(&scanner);
}
```

환경 값은 owned 문자열이다

잘못됨(Wrong):

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("PATH"));
use_path(proven_u8str_as_view(&env.value)); /* wrong if env.err was not checked, and it leaks */
```

올바름(Correct):

```
proven_result_u8str_t env = proven_env_get(alloc, PROVEN_LIT("HOME"));
if (proven_is_ok(env.err)) {
    proven_u8str_view_t home = proven_u8str_as_view(&env.value);
    proven_println("HOME={}", PROVEN_ARG(home));
    proven_u8str_destroy(alloc, &env.value);
}
```

완성 예제: 파일 전체를 읽고 쓰기

테스트 스위트에 의해 컴파일되고 실행됨. 이것은 대부분의 호출자가 실제로 원하는 파일 전체 API이며, 대상의 권한을 보존하는 atomic 재작성을 포함한다.

```
/*
 * 파일 통째 API: 한 번 부르면 들어가고, 한 번 부르면 나온다. 이것이 있는 이유는 열고 -
 * 읽는 반복문 - 닫는 그 춤사위가 파일 다루기 버그 대부분이 사는 자리이기 때문이다.
 * 잊은 close, EOF 로 오해한 부분 읽기, 실패한 쓰기가 남긴 잘린 파일. 파일을 통째로 읽거나
 * 쓰는 것이라면 이것이 그 API 다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 지금 디렉터리의 상대 경로다. 이 예제가 쓸 수 있는 /tmp 에 기대면 안 되고, 만든
     * 것은 돌아가기 전에 지운다. */
    proven_u8str_view_t path = PROVEN_LIT("proven_example_wholefile.tmp");
    proven_u8str_view_t text = PROVEN_LIT("first line\nsecond line\n");

    /* --- 한 번의 호출로 쓰기 ------------------------------------------------ */
    /* 원자적이지 않다. 동시에 읽는 쪽은 이 파일이 반쯤 쓰인 것을 볼 수 있다. 여기서는
     * 괜찮다. 아직 아무도 이 파일을 보고 있지 않기 때문이다. */
    proven_err_t err = proven_fs_write_file(alloc, path, proven_mem_view_from_u8(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole file should succeed");
    if (!proven_is_ok(err)) return 1;

    /* --- 날바이트로 되읽기 -------------------------------------------------- */
    /* proven_fs_read_all 은 미리 잰 크기까지가 아니라 EOF 까지 읽는다. 그래서 크기를
     * 미리 알 수 없는 파이프나 /proc 항목에서도 돈다. */
    proven_result_mem_mut_t raw = proven_fs_read_all(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(raw.err), "reading the whole file should succeed");
    if (proven_is_ok(raw.err)) {
        EXAMPLE_REQUIRE(raw.value.size == text.size, "read_all should return every byte written");
        /* 그 덩이는 그냥 할당자의 기억이다 - 그것을 내준 할당자에게 돌려줄 것.
         * proven_fs_read_all_destroy 같은 것은 없다. */
        alloc.free_fn(alloc.ctx, raw.value.ptr);
    }

    /* --- 문자열로 되읽기 ---------------------------------------------------- */
    /* 대개의 부르는 쪽이 원하는 것이 이것이다. 결과가 NUL 로 끝나므로 뷰에도, as_cstr
     * 에도, 스캐너에도 두 번째 복사 없이 건넬 수 있다. 종결자 자리는 미리 잡아 두므로
     * 할당이 더 들지 않는다. */
    proven_result_u8str_t s = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(s.err), "reading the whole file as a string should succeed");
    if (!proven_is_ok(s.err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&s.value), text),
```

```
                "the file's contents should come back unchanged");
    printf("read back %zu bytes: %s", (size_t)proven_u8str_as_view(&s.value).size,
           proven_u8str_as_cstr(&s.value));
    proven_u8str_destroy(alloc, &s.value);

    /* --- stat, 그리고 권한의 왕복 ------------------------------------------ */
    proven_fs_stat_t st = {0};
    err = proven_fs_stat(alloc, path, &st);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat on a file we just wrote should succeed");
    EXAMPLE_REQUIRE(st.type == PROVEN_FS_TYPE_FILE, "a regular file should stat as a FILE");
    EXAMPLE_REQUIRE(st.size == text.size, "stat should report the size we wrote");

    /* `perms` 는 권한 비트 아홉 개만 담고 그 밖의 것은 담지 않는다. 그래서 그대로
     * chmod 에 도로 먹일 수 있다. 이 필드의 요점이 그것이다 - 파일의 모드를 읽어 두었다가
     * 나중에 되돌리는 것. (예전에는 날 POSIX st_mode 를 담았는데, 그 파일 종류 비트를
     * chmod 가 거부해서 이 뻔한 왕복이 실패했다.) */
    err = proven_fs_chmod(alloc, path, st.perms);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a stat's perms must be accepted back by chmod");

    /* 이제 파일을 소유자 전용으로 바꾼다. 그래야 다음 검사가 증명할 것이 생긴다. */
    proven_fs_perms_t private_perms = PROVEN_FS_PERM_OWNER_R | PROVEN_FS_PERM_OWNER_W;
    err = proven_fs_chmod(alloc, path, private_perms);
    EXAMPLE_REQUIRE(proven_is_ok(err), "restricting the file to its owner should succeed");

    /* 플랫폼이 그중 무엇을 간직하는가: POSIX 에서는 아홉 비트 모두, Windows 에서는
     * 소유자 쓰기 비트(READONLY 속성)뿐이다. 다시 쓰기는 바로 이것을 지켜야 한다. */
    proven_fs_stat_t st1 = {0};
    err = proven_fs_stat(alloc, path, &st1);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat after chmod should succeed");

    /* --- 원자적으로 다시 쓰기 ---------------------------------------------- */
    /* 형제 임시 파일 하나에 rename 하나. 동시에 읽는 쪽은 옛 파일 전체이거나 새 파일
     * 전체를 보지, 반쯤 섞인 것을 보지 않는다. 읽는 쪽에게 원자적이지, 전원이 나가도
     * 견디는 것은 아니다. 그래야 할 때는 proven_fs_write_file_durable 이 그것을 청한다. */
    proven_u8str_view_t text2 = PROVEN_LIT("replacement\n");
    err = proven_fs_write_file_atomic(alloc, path, proven_mem_view_from_u8(text2));
    EXAMPLE_REQUIRE(proven_is_ok(err), "the atomic rewrite should succeed");

    proven_fs_stat_t st2 = {0};
    err = proven_fs_stat(alloc, path, &st2);
    EXAMPLE_REQUIRE(proven_is_ok(err), "stat after the atomic rewrite should succeed");
    EXAMPLE_REQUIRE(st2.size == text2.size, "the file should now hold the replacement text");
    /* rename 은 옛 이름 위에 *새* 아이노드를 씌우므로, 권한을 옮겨 주지 않으면 잃는다.
     * 옮겨 준다 - 0600 파일을 다시 써도 0644 로 다시 공개되지 않는다. */
    EXAMPLE_REQUIRE(st2.perms == st1.perms,
                "the atomic rewrite must preserve the target's permissions");

    /* --- 뒷정리 ------------------------------------------------------------ */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}
```

완성 예제: open, read, write, close

테스트 스위트에 의해 컴파일되고 실행됨. read 루프에 주목하라: 파일 끝에서의 읽기는 0바이트 성공이 아니라 `PROVEN_ERR_EOF`를 반환하므로, 0바이트만 검사하는 루프는 작성자가 기대한 방식으로 결코 종료되지 않는다.

```c
/*
 * 열고-읽고-쓰고-닫는 길. 파일 통째 API(ex_05_fs_wholefile)로는 모자랄 때 쓴다.
 * 흘려 보내고 있거나, 버퍼를 여러분이 소유하고 싶을 때다.
 *
 * 여기서 반드시 맞춰야 할 하나: 읽기나 쓰기 한 번은 청한 바이트 수 *만큼까지*를 옮기지,
 * 정확히 그만큼을 옮기지 않는다. 짧은 읽기 한 번을 파일 끝으로 여기는 것이 파일의 꼬리를
 * 조용히 잃는 고전적인 방법이다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t path = PROVEN_LIT("proven_example_stream.tmp");
    proven_u8str_view_t text = PROVEN_LIT("streamed bytes, read back in chunks\n");

    /* --- 쓰기 ------------------------------------------------------------- */
    /* CREATE 는 없으면 만들고, TRUNC 는 있으면 비운다. 할당자는 플랫폼 호출을 위해 경로를
     * 변환하는 데만 쓰인다. */
    proven_result_file_t out = proven_fs_open(alloc, path,
                                              PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "opening the file for writing should succeed");
    if (!proven_is_ok(out.err)) return 1;

    /* write_all 이 우리 대신 반복해 준다. proven_fs_write 는 한 번만 시도하고 더 적게 쓸
     * 수도 있는데, 그것은 부르는 쪽이 뜻한 바가 거의 아니다. */
    proven_err_t err = proven_fs_write_all(out.value, proven_mem_view_from_u8(text));

    /* close 는 쓰기의 일부이고, 네트워크나 할당량이 걸린 파일 시스템에서는 실패가 드러나는
     * *유일한* 자리다. 바이트는 버퍼에 담겼고 write() 는 그렇다고 했으며, 디스크가 마침내
     * 아니라고 말하는 자리가 close() 다. 실패 경로에서도 닫을 것 - 손잡이는 어느 쪽이든
     * 우리 것이다 - 다만 그 답을 버리지는 말 것. */
    proven_err_t cerr = proven_fs_close(out.value);
    if (proven_is_ok(err)) err = cerr;

    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the whole buffer should succeed");
    if (!proven_is_ok(err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }

    /* --- 읽기 ------------------------------------------------------------- */
    proven_result_file_t in = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(in.err), "opening the file for reading should succeed");
    if (!proven_is_ok(in.err)) {
        (void)proven_fs_remove(alloc, path);
        return 1;
    }

    /* size 는 버퍼 크기를 잡는 힌트이지, 읽기 한 번이 몇 바이트를 건넬지에 대한 약속이
     * 아니다 - 그리고 보통 파일이 아닌 것(파이프, 장치, /proc 항목)에서는 0 이다. 아래
     * 반복문은 그것에 기대지 않는다. */
    proven_result_size_t sz = proven_fs_size(in.value);
    EXAMPLE_REQUIRE(proven_is_ok(sz.err), "querying the size of an open file should succeed");
    EXAMPLE_REQUIRE(sz.value == text.size, "the file should be as long as what we wrote");

    proven_byte_t buf[128];
    proven_size_t total = 0;

    /* 부분 읽기 반복문. 한 바퀴마다 버퍼에 남은 만큼을 청하고 실제로 온 만큼 나아간다.
     * 짧은 읽기는 파일의 끝이 아니라 정상이다. 파일의 끝은 따로 있는 상태 -
```

```
     * 바이트 0 과 함께 오는 PROVEN_ERR_EOF - 이므로 반복문은 그것으로만 끝난다. 원본이
     * 버퍼보다 커져도 반복문은 멈추는데, 그것을 알아채는 일은 부르는 쪽의 몫이다(여기서는
     * 일어날 수 없지만, 자라나는 파일이라면 그럴 수 있다). */
    for (;;) {
        if (total == sizeof buf) break;   /* 버퍼가 가득 찼다: 어떻게 할지는 부르는 쪽이 정한다 */

        proven_mem_mut_t dest = { .ptr = buf + total, .size = sizeof buf - total };
        proven_result_size_t r = proven_fs_read(in.value, dest);
        if (r.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(r.err)) {
            (void)proven_fs_close(in.value);
            (void)proven_fs_remove(alloc, path);
            EXAMPLE_REQUIRE(false, "reading from the open file should not fail");
            return 1;
        }
        total += r.value;
    }

    (void)proven_fs_close(in.value);

    EXAMPLE_REQUIRE(total == text.size, "the loop should have read every byte in the file");
    proven_u8str_view_t got = { .ptr = buf, .size = total };
    EXAMPLE_REQUIRE(proven_u8str_view_eq(got, text), "the bytes should come back unchanged");

    printf("read %zu bytes in chunks: %.*s", (size_t)total, (int)total, (const char *)buf);

    /* --- 뒷정리 ------------------------------------------------------------ */
    err = proven_fs_remove(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(err), "removing the temp file should succeed");

    return EXAMPLE_OK();
}
```

디렉터리를 한 번에 한 엔트리씩 읽기

`proven_fs_list`는 당신이 그 중 어느 것을 보기 전에 디렉터리 **전체**를 읽으며, 모든 이름마다 문자열을 할당한다. 50,000개 엔트리로 측정: **189 ms, +4.2 MB 상주(resident), 50,008회 할당**, 마지막 엔트리가 읽힐 때까지 아무것도 보이지 않는다. 이는 설정 디렉터리에는 괜찮고 메일 스풀에는 무용하다.

`proven_fs_dir_open` / `_next` / `_close`는 같은 디렉터리를 한 번에 한 엔트리씩 순회하며 **엔트리당 아무것도 할당하지 않는다**.

API	의도	반환
`proven_fs_dir_open(scratch, path)`	스트리밍 반복자를 연다. `scratch`는 경로 변환에만 쓰인다.	`proven_result_dir_t`.
`proven_fs_dir_next(&dir, &entry)`	다음 엔트리. 더 이상 없으면 `PROVEN_ERR_EOF`. 이름이 올바른 텍스트가 아닌 엔트리 하나에는 `PROVEN_ERR_INVALID_ENCODING`이며, 다음 호출은 이어서 읽는다.	`proven_err_t`.
`proven_fs_dir_close(&dir)`	반복자를 해제.	void.

```
typedef struct {
    proven_u8str_view_t name;   /* BORROWED: points into the iterator's own storage,
                                   valid only until the next proven_fs_dir_next */
    proven_fs_type_t    type;   /* FILE / DIR / OTHER - and it follows symlinks */
} proven_fs_dir_entry_t;
```

이렇게 사용하라:

```
proven_result_dir_t d = proven_fs_dir_open(alloc, PROVEN_LIT("."));
if (proven_is_ok(d.err)) {
    proven_fs_dir_t dir = d.value;
    proven_fs_dir_entry_t entry;
    for (;;) {
        proven_err_t e = proven_fs_dir_next(&dir, &entry);
        if (e == PROVEN_ERR_EOF) break;
        if (e == PROVEN_ERR_INVALID_ENCODING) continue;   /* a name that is not text: skip it */
        if (!proven_is_ok(e)) break;              /* a real error: report it */
        proven_println("{}", PROVEN_ARG(entry.name));
    }
    proven_fs_dir_close(&dir);
}
```

이름은 빌려 쓰는(borrowed)이며, 다음 호출에서 소멸한다. 이것이 전체를 무할당으로 만드는 것—그리고 당신이 그것을 붙들어 두는 순간 dangling 포인터로 만드는 것이다.

잘못됨(Wrong):

```
proven_u8str_view_t names[100];
int n = 0;
while (proven_is_ok(proven_fs_dir_next(&dir, &entry)))
    names[n++] = entry.name;    /* wrong: every entry aliases the same storage, and the
                                   next _next overwrites it. All 100 end up equal - to
                                   whatever the last entry happened to be. */
```

올바름(Correct): 보관해야 하는 것들에 대해 바이트를 복사하라(`proven_u8str_create_from_view`)—또는 `proven_fs_list`를 사용하라. 그것은 모든 엔트리에 대해 정확히 그렇게 하고 그 대가를 청구한다.

`PROVEN_ERR_EOF`가 끝이며, 그 밖의 것은 실패다. "OK 아님"에서 멈추는 루프는 권한 에러를 완전한 목록으로 취급한다.

```
while (proven_is_ok(proven_fs_dir_next(&dir, &entry))) { ... }
/* wrong: an I/O error ends the loop exactly like the end of the directory does,
   and you cannot tell whether you saw everything. */
```

트리 순회

`proven_fs_dir_*`는 디렉터리 하나를 순회한다. `proven_fs_walk`는 트리를 순회한다—그리고 그것이 하기를 거부하는 것을 정확히 말할 가치가 있다. 그 거부들이 곧 기능이기 때문이다:

루프에 빠질 수 없다	결코 symlink를 통과해 내려가지 않는다.
탈출할 수 없다	같은 규칙: 트리 밖으로 나가는 링크는 보고되되 따라가지 않는다.
거짓말할 수 없다	읽을 수 없는 디렉터리는 그 디렉터리를 명명하는 *에러*로 돌아오고, 순회는 계속된다. 읽을 수 없는 하위 트리를 조용히 건너뛰는 트리 순회기는 백업이 파일을 놓치고도 성공을 보고하는 방식이다.
비대해질 수 없다	현재 경로의 *레벨*당 하나의 열린 핸들, 그리고 재사용되는 하나의 경로 버퍼. 메모리는 파일 수가 아니라 깊이의 함수다.

symlink된 디렉터리는 여전히 **보고된다**—그것은 존재하고, `type`은 `DIR`이며, `is_symlink`는 true다—단지 진입되지 않을 뿐이다. 그것을 숨기는 것은 그 자체로 일종의 거짓말이 될 것이다. 그것을 따라가고 싶다면, 당신에게는 경로가 있다: 그것에 대해 두 번째 순회를 열면, 사이클 문제는 당신의 소유가 된다.

당신이 받는 구조체

```
typedef struct {
    proven_u8str_view_t path;   /* the whole path, from the root you passed in.
                                   BORROWED: it points into the walk's one reused
                                   buffer and is valid only until the next call. */
    proven_u8str_view_t name;   /* the last component of `path`. Same lifetime. */
```

```
    proven_fs_type_t    type;   /* FILE / DIR / OTHER - and it FOLLOWS symlinks,
                                   exactly as proven_fs_stat does. */
    proven_size_t       size;   /* bytes, for a regular file; 0 otherwise. */
    proven_size_t       depth;  /* 0 for an entry directly inside the root. */
    bool                is_symlink;  /* reached through a symlink. `type` describes
                                        the TARGET; the walk does not enter it. */
} proven_fs_walk_entry_t;
```

`entry.path`와 `entry.name`은 순회의 재사용되는 하나의 버퍼에서 빌린 것이며 **다음 호출 전까지만** 유효하다. 이 단계보다 오래 살아남게 하려면 복사하라. 그것이 백만 엔트리의 순회가 백만 번이 아니라 한 번의 할당으로 드는 것의 대가다.

잘못됨(Wrong)—디렉터리 반복자가 가진 것과 같은 함정, 같은 이유로:

```
proven_u8str_view_t found[100];
int n = 0;
while (proven_is_ok(proven_fs_walk_next(&walk, &entry)))
    found[n++] = entry.path;   /* wrong: every entry aliases ONE buffer. When the loop
                                  ends, all 100 point at whatever the last path was. */
```

두 개의 제한, 둘 다 조용해지는 대신 그렇다고 말한다:

- `max_depth`—얼마나 깊이 내려갈지. 제한 에 있는 디렉터리는 여전히 보고된다(그것은 하나의 엔트리다). 진입되지 않을 뿐이다.
- `PROVEN_FS_WALK_DEPTH_LIMIT`(256)—순회 자체의 스택이 결코 넘지 않는 깊이. 그것을 지나친 디렉터리는 그 디렉터리를 명명하며 `PROVEN_ERR_OUT_OF_BOUNDS`로 돌아온다.

테스트 스위트에 의해 컴파일되고 실행됨:

```
/*
 * 나무 훑기.
 *
 * 재귀 훑기가 틀리는 세 가지, 그리고 이것이 대신 하는 일.
 *
 *   맴돈다.       조상을 가리키는 심링크는 순환이다. 이 훑기는 심링크를 *지나* 내려가지
 *                않는다 - 심링크된 디렉터리도 보고는 되고, 다만 들어가지 않을 뿐이다 -
 *                그래서 순환이 불가능하고, 물어본 나무를 벗어나 나머지 파일 시스템으로
 *                걸어 나가는 일도 없다.
 *
 *   거짓말한다.   읽을 수 없는 디렉터리를 건너뛰고는 성공했다고 보고한다. 백업이 하위
 *                나무 하나를 통째로 놓치는 방식이 그것이다. 여기서는 오류가
 *                proven_fs_walk_next 에서 돌아오고, 그 항목이 어느 디렉터리인지 말해
 *                주며, 훑기는 다음 형제부터 이어 간다. 어떻게 할지는 여러분이 정한다.
 *
 *   부푼다.       하나라도 내주기 전에 디렉터리 전체를 기억에 읽어 들이면, 큰 나무를
 *                훑는 데 큰 할당이 든다. 이것은 지금 경로의 *층*마다 열린 손잡이 하나와
 *                다시 쓰는 경로 버퍼 하나만 쥔다 - 그래서 기억 사용량은 파일이 몇
 *                개인가가 아니라 깊이의 함수다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 훑어 볼 작은 나무: 파일 하나, 디렉터리 하나, 그 안의 파일 하나. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk"))), "mkdir");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_mkdir(alloc, PROVEN_LIT("ex_walk/inner"))), "mkdir inner");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/top.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("top")))), "write top.txt");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_file(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"),
        proven_mem_view_from_u8(PROVEN_LIT("deep")))), "write deep.txt");

    proven_result_walk_t walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"),
```

```
                                                PROVEN_FS_WALK_UNLIMITED);
EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the walk should open");

proven_size_t files = 0;
proven_size_t dirs = 0;
proven_size_t unreadable = 0;
proven_size_t total_bytes = 0;

for (;;) {
    proven_fs_walk_entry_t entry = {0};
    proven_err_t err = proven_fs_walk_next(&walk.value, &entry);

    if (err == PROVEN_ERR_EOF) break;

    if (!proven_is_ok(err)) {
        /* 읽을 수 없었던 디렉터리이거나, 훑기의 스택보다 깊은 것이다. 건너뛰는 것이
         * 아니라 *보고된다* - `entry.path` 가 어느 것인지 말해 준다 - 그리고 훑기는
         * 계속된다. 나무를 복사하는 도구라면 여기서 실패해야 하고, 나무를 보고하는
         * 도구라면 세어서 알려야 한다. 하지 말아야 할 것은 없던 일로 하는 것이다. */
        unreadable++;
        continue;
    }

    /* `entry.path` 와 `entry.name` 은 빌린 것이다. 훑기가 다시 쓰는 버퍼 하나를
     * 가리키고 다음 호출 전까지만 쓸 수 있다. 더 오래 필요하면 복사할 것. */
    if (entry.type == PROVEN_FS_TYPE_DIR) {
        dirs++;
    } else if (entry.type == PROVEN_FS_TYPE_FILE) {
        files++;
        total_bytes += entry.size;
    }
}

proven_fs_walk_close(&walk.value);

EXAMPLE_REQUIRE(files == 2, "two files: top.txt and inner/deep.txt");
EXAMPLE_REQUIRE(dirs == 1, "one directory: inner");
EXAMPLE_REQUIRE(unreadable == 0, "and nothing in this tree is unreadable");
EXAMPLE_REQUIRE(total_bytes == 7, "three bytes plus four");

/* 깊이 제한: max_depth 0 은 뿌리 바로 안에 있는 것을 보고하고 아무 데도 내려가지
 * 않는다. 한계에 걸린 디렉터리도 여전히 항목이므로 여전히 보고된다. */
walk = proven_fs_walk_open(alloc, PROVEN_LIT("ex_walk"), 0);
EXAMPLE_REQUIRE(proven_is_ok(walk.err), "the shallow walk should open");

proven_size_t shallow = 0;
for (;;) {
    proven_fs_walk_entry_t entry = {0};
    proven_err_t err = proven_fs_walk_next(&walk.value, &entry);
    if (err == PROVEN_ERR_EOF) break;
    if (proven_is_ok(err)) shallow++;
}
proven_fs_walk_close(&walk.value);

EXAMPLE_REQUIRE(shallow == 2, "top.txt and inner - but nothing inside inner");

(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner/deep.txt"));
(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/top.txt"));
(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk/inner"));
(void)proven_fs_remove(alloc, PROVEN_LIT("ex_walk"));
```

```
    return EXAMPLE_OK();
}
```

스트림: writer와 reader

포매터의 유일한 sink는 예전에 `proven_u8str_t`였다. 파일은 `proven_file_t`였다. 인메모리 스캐너는 view를 읽었고, 파일 스캐너는 또 다른 무언가를 읽었다. **네 개의 타입, 네 개의 함수 계열, 공통 인터페이스 없음**—그래서 메모리와 파일 양쪽에서 동작하는 하나의 `serialize(sink, value)`를 쓸 수 없었고, 파일로 포매팅하는 것 자체가 불가능했으며, 파일을 한 줄씩 읽을 방법이 전혀 없었다.

writer는 바이트 sink다. **reader**는 바이트 source다. 둘 다 값으로 전달되는 작은 vtable로, 정확히 `proven_allocator_t`처럼, 같은 이유로 그렇다: 호출자가 바이트가 어디로 가는지 결정하며, 아무것도 숨겨지지 않는다.

API	의도
`proven_writer_from_file(&file)`	열린 파일 위의 언버퍼드 sink.
`proven_writer_from_u8str(&state, &str, alloc)`	owned 문자열에 덧붙인다.
`proven_writer_from_buffer(&state)`	고정된 호출자 메모리. 절대로 아무것도 할당하지 않는다.
`proven_writer_buffered(&state, inner, buf)`	작은 쓰기들을 누적한다. 버퍼는 **당신이** 공급한다.
`proven_fprint(w, fmt, ...)` / `proven_fprintln`	writer로 직접 포매팅. 무할당.
`proven_reader_from_file(&file)` / `_from_view(&state, view)`	바이트 source.
`proven_reader_buffered(&state, inner, buf)`	버퍼드 source; 라인 읽기에 필수.
`proven_reader_read_line(&state)`	개행 없는 한 줄.
`proven_writer_is_valid(w)` / `proven_reader_is_valid(r)`	생성자가 성공했는가? 0으로 채워진 핸들은 invalid이고, 모든 생성자는 잘못된 인자에 대해 그런 것을 반환한다—따라서 NULL 검사가 아니라 이것이 그 검사다.
`proven_fwrite_fmt(w, scratch, fmt, ...)`	**당신이** 크기를 정한 임시 작업용(scratch) 버퍼를 쓰는 `proven_fprint`. `proven_fprint`는 512바이트 스택 버퍼를 사용하고 더 긴 줄에 대해 `OUT_OF_BOUNDS`를 반환한다. 이것이 그런 줄을 포매팅하는 방법이다.
`proven_fmt_to_writer_impl(w, scratch, fmt, args, n)`	위의 두 매크로가 확장되는 함수. 자신만의 가변 인자 래퍼를 만드는 경우에만 직접 호출하라. 매크로는 당신이 인자를 세지 않아도 되도록 존재한다.

명확히 언급할 가치가 있는 네 가지 규칙. 각각은 이것이 잘못 설계될 수 있었던 방식이기 때문이다:

- **버퍼링은 당신이 공급하는 메모리를 사용한다.** `proven_writer_buffered`는 `proven_arena_create`가 그러하듯 `proven_mem_mut_t`를 받는다. 숨겨진 전역 버퍼는 없으며, 이는 곧 당신을 위해 그것을 flush해 줄 소멸자도 없다는 뜻이다—**버퍼가 스코프를 벗어나기 전에 당신이 flush해야 한다.** 그 대가로, 당신의 로깅 경로는 결코 할당하지 않으며, out-of-memory 상태에서 빠져나오며 로그를 남기는 프로그램이 여전히 로그를 남길 수 있다.
- **가득 찬 sink는 거부한다. 절단하지 않는다.** 가득 찬 고정 버퍼는 `PROVEN_ERR_OUT_OF_BOUNDS`를 반환하고 `overflowed`를 기록한다. 당신의 데이터 끝을 조용히 버리는 sink는 받을 수 없다고 말하는 sink보다 나쁘다.
- **reader의 버퍼에 비해 너무 긴 줄은 에러다**, 절단된 줄이 아니다. 온전한 것처럼 돌려받은 절단된 줄은 호출자가 탐지할 방법이 없는 손상이다. 버퍼는 당신 것이다; 예상하는 입력에 맞춰 크기를 정하라.

- **부분 쓰기는 얼버무릴 실패가 아니라 하나의 사실이다.** `write_fn`은 `proven_result_size_t`를 반환한다: sink가 얼마나 받았는지, 그리고 무엇이 잘못되었는지. 파이프, 소켓, 또는 차오르는 디스크는 정말로 당신의 6000바이트 중 4096바이트를 받아들이고 나서 실패한다. 그리고 "전부 소비하거나 실패한다"고 말하는 트레이트는 그런 sink를 올바르게 작성하는 것을 그저 불가능하게 만든다. `proven_writer_write`는 여전히 전부-아니면-전무를 뜻한다(루프를 돈다); `proven_writer_write_partial`은 당신이 얼마나 진행했는지 봐야 할 때 있다. 버퍼드 writer는 sink가 받지 **않은** 꼬리만 보관한다—첫 버전은 버퍼 전체를 보관하고 다시 보냈으며, 그래서 실패하는 sink가 받아들인 접두부를 두 번 받았다.
- **한 번 실패한 writer는 실패한 채로 남는다.** writer가 바이트를 잃은 뒤에는—sink가 죽은 버퍼드 writer, 오버플로한 고정 버퍼—그것이 만들어내던 스트림에 수신자가 볼 수 없는 구멍이 생기므로, 이후의 모든 쓰기와 flush는 에러를 반환한다. 들어갈 수 있는 더 짧은 청크조차 거부된다: 그것을 쓰면 구멍 뒤에 놓여, 결과가 완전한 출력처럼 보일 것이기 때문이다. `clear()`는 없다: 복구 스토리가 있다면 그것은 이 writer가 괜찮은 척하는 것이 아니라 새 sink 위의 새 writer를 수반한다. (이전에는 실패한 쓰기 이후 `flush`가 `PROVEN_OK`를 답했다—버퍼가 비어 있어서 실패할 것이 남지 않았기 때문이다—그리고 "쓰고, 쓰고, 쓰고, flush를 검사한다"는, 거의 모두가 버퍼드 writer를 쓰는 방식이 가득 찬 디스크에서 성공을 보고했다.)

reader의 규칙은 그 거울상이다: **실패한 읽기는 에러이지, 결코 파일의 끝이 아니다.** `proven_reader_read`는 source가 깨질 때 깨끗한 0바이트 EOF가 아니라 `PROVEN_ERR_IO`를 반환한다— 디스크 에러로 잘려나간 파일과 그냥 끝난 파일은, 둘을 구별할 수 없는 호출자에게는 같은 것이며, 그 중 하나만 안전하게 처리할 수 있기 때문이다.

10,000줄을 stdout으로 내보내며 측정한 비용:

	`write()` 시스템 콜	`malloc()`
`proven_println`	10,000	**0** (이전엔 10,000)
버퍼드 writer, 호출자 메모리 8 KiB	**24**	**0**

`malloc()` 열은 512바이트 스택 버퍼에 들어가는 줄, 즉 일반적인 경우에 대한 값이다. 그보다 긴 줄은 거부되는 대신 그 한 번의 호출에 한해 힙(heap)으로 물러난다.

`proven_println`은 의도적으로 여전히 줄당 하나의 시스템 콜이다: 그것을 버퍼링하려면 숨겨진 전역 상태가 필요할 것이다. 24를 원하는 호출자는 버퍼드 writer를 만들고 그렇게 하겠다고 말한다.

완성 예제: 하나의 직렬화기, 세 개의 목적지, 그리고 되읽기
테스트 스위트에 의해 컴파일되고 실행됨. `render_row`가 자신의 바이트가 어디로 가는지 모른다는 점에 주목하라—그것이 바로 요점 전부다.

```
/*
 * 쓰는 쪽과 읽는 쪽. "바이트가 어디로 가는가" 를 위한 인터페이스 하나와 "바이트가
 * 어디서 오는가" 를 위한 인터페이스 하나.
 *
 * 요점은 아래 코드 - render_row - 가 자기가 문자열로 쓰는지, 고정 버퍼로 쓰는지, 파일로
 * 쓰는지 모르고 신경도 쓰지 않는다는 것이다. 예전에는 그럴 수 없었다. 형식화기가 받는
 * 그릇은 proven_u8str_t 하나뿐이었다.
 */

/* 직렬화기 하나. 목적지가 아니라 그릇을 받는다. */
static proven_err_t render_row(proven_writer_t w, int id, const char *name) {
    proven_fmt_result_t r = proven_fprintln(w, "{:>4} | {}", PROVEN_ARG(id), PROVEN_ARG(name));
    return r.err;
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 같은 코드로, 늘어나는 문자열에 ----------------------------------- */
    proven_result_u8str_t s = proven_u8str_create(alloc, 16);
```

```
EXAMPLE_REQUIRE(proven_is_ok(s.err), "string create");

proven_writer_u8str_t s_state;
proven_writer_t to_string = proven_writer_from_u8str(&s_state, &s.value, alloc);
EXAMPLE_REQUIRE(proven_is_ok(render_row(to_string, 7, "ada")), "render into a string");

printf("into a string:\n%s", proven_u8str_as_cstr(&s.value));
proven_u8str_destroy(alloc, &s.value);

/* --- 같은 코드로, 여러분이 소유한 기억에: 할당은 0 -------------------- */
proven_byte_t fixed[64];
proven_writer_buf_t b_state = { .buf = { .ptr = fixed, .size = sizeof fixed } };
proven_writer_t to_buffer = proven_writer_from_buffer(&b_state);
EXAMPLE_REQUIRE(proven_is_ok(render_row(to_buffer, 8, "grace")), "render into a buffer");
EXAMPLE_REQUIRE(b_state.len > 0, "the buffer received the row");

/* 가득 찬 버퍼는 *거부한다*. 자르지 않는다. 여러분 자료의 끝을 조용히 버리는 그릇은
 * 받을 수 없다고 말하는 그릇보다 나쁘다. */
proven_byte_t tiny[4];
proven_writer_buf_t t_state = { .buf = { .ptr = tiny, .size = sizeof tiny } };
proven_writer_t to_tiny = proven_writer_from_buffer(&t_state);
EXAMPLE_REQUIRE(proven_writer_write_str(to_tiny, PROVEN_LIT("far too long")) == PROVEN_ERR_OUT_OF_BOUNDS,
                "a full buffer refuses rather than truncating");
EXAMPLE_REQUIRE(t_state.overflowed, "and it records that it did");

/* --- 같은 코드로, 파일에, 버퍼를 두고 ---------------------------------- */
/*
 * 버퍼는 아레나에서와 똑같이 *여러분이* 대는 기억이다. 이 라이브러리에는 숨은 전역
 * 상태가 없으므로 종료 시점에 대신 흘려 줄 수 없다 - 그래서 버퍼가 스코프를 벗어나기
 * 전에 반드시 flush 해야 한다. 그 대가로 로그 경로에서는 할당이 전혀 일어나지 않는다.
 * 여기서 만 줄은 malloc 0 번에 write 시스템 호출 스물 몇 번이지만, proven_println 을
 * 만 번 부르면 시스템 호출 10,000 번이다.
 */
proven_u8str_view_t path = PROVEN_LIT("example_stream_rows.txt");
proven_result_file_t f = proven_fs_open(alloc, path,
    (proven_fs_mode_t)(PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC));
EXAMPLE_REQUIRE(proven_is_ok(f.err), "open the output file");
proven_file_t file = f.value;

proven_byte_t out_buf[4096];
proven_writer_buffered_t w_state;
proven_writer_t to_file = proven_writer_buffered(&w_state,
    proven_writer_from_file(&file),
    (proven_mem_mut_t){ .ptr = out_buf, .size = sizeof out_buf });

for (int i = 0; i < 3; ++i) {
    EXAMPLE_REQUIRE(proven_is_ok(render_row(to_file, i, "row")), "render into the file");
}
EXAMPLE_REQUIRE(proven_is_ok(proven_writer_flush(to_file)),
                "flush: nothing is written until you say so");
/* 그리고 close - 쓰기가 도착하지 못했다고 알려 줄 수 있는 마지막 자리다. */
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(file)), "closing the written file");

/* --- 한 줄씩 되읽기 ---------------------------------------------------- */
/* 파일을 한 줄씩 읽는 것은 예전에는 아예 되지 않았다. 길은 파일 전체를 기억에 올려
 * 손으로 자르는 것뿐이었다. */
proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopen for reading");
proven_file_t rfile = rf.value;
```

```
    proven_byte_t in_buf[128];
    proven_reader_buffered_t r_state;
    (void)proven_reader_buffered(&r_state, proven_reader_from_file(&rfile),
                                 (proven_mem_mut_t){ .ptr = in_buf, .size = sizeof in_buf });

    int lines = 0;
    for (;;) {
        proven_result_u8str_view_t line = proven_reader_read_line(&r_state);
        if (line.err == PROVEN_ERR_EOF) break;
        EXAMPLE_REQUIRE(proven_is_ok(line.err), "read a line");
        /* 그 뷰는 읽는 쪽의 버퍼 *안*을 가리키고, 다음 호출 전까지만 쓸 수 있다. 그보다
         * 오래 살아야 한다면 복사할 것. */
        printf("line %d: %.*s\n", lines, (int)line.val.size, (const char *)line.val.ptr);
        ++lines;
    }
    EXAMPLE_REQUIRE(lines == 3, "three rows in, three lines out");

    (void)proven_fs_close(rfile);
    (void)proven_fs_remove(alloc, path);

    return EXAMPLE_OK();
}
```

당신이 보유하는 구조체

두 **핸들**은 값으로 전달되며 저렴하다:

```
typedef struct {
    void *ctx;
    proven_result_size_t (*write_fn)(void *ctx, proven_mem_view_t chunk);
    proven_err_t (*flush_fn)(void *ctx);   /* may be NULL: this sink holds nothing back */
} proven_writer_t;

typedef struct {
    void *ctx;
    proven_result_size_t (*read_fn)(void *ctx, proven_mem_mut_t dest);
} proven_reader_t;
```

`write_fn`은 **그 뒤에 실패하더라도 얼마나 나갔는지 보고한다**. 그리고 그것은 사소한 배려가 아니다: 파이프나 가득 찬 디스크로의 쓰기는 정말로 일부 바이트를 내보내고 나서 실패한다. 더 깔끔한 "전부 아니면 전무"라는 거짓말 위에 세워진 버퍼드 writer는 실패 시 버퍼 전체를 보관하고 다음 flush에 다시 보냈다—6000바이트 페이로드가 처음 4096바이트가 **중복된** 채로 10,096바이트로 도착했다. 데이터를 잃는 것은 나쁘다; 조용히 두 배로 만드는 것은 더 나쁘다. 수신자가 알 수 없기 때문이다.

그 밖의 모든 것은 호출자 소유 상태다— `proven_writer_buf_t`, `proven_writer_u8str_t`, `proven_writer_buffered_t`, `proven_reader_view_t`, `proven_reader_buffered_t`. 이들은 아무것도 할당하지 않고, destroy가 없으며, 핸들이 그 안을 가리키고 있는 동안 **복사되거나 이동되어서는 안 된다**.

주의사항, 그리고 무엇이 잘못되는가

결코 flush되지 않는 버퍼드 writer는 결코 일어나지 않은 출력이다. 숨겨진 상태가 없으므로, 당신을 위해 그것을 flush해 줄 소멸자도 없다—그리고 여기 있는 어떤 것도 `atexit` 핸들러를 등록하지 않는다. 당신의 프로세스를 소유하는 라이브러리는 당신이 추론할 수 없는 라이브러리이기 때문이다.

잘못됨(Wrong):

```
proven_writer_buffered_t st;
proven_writer_t w = proven_writer_buffered(&st, inner, buf);
(void)proven_fprintln(w, "the important line");
return;                       /* wrong: the buffer dies with the frame, and so does the line */
```

올바름(Correct)—버퍼나 내부 sink가 사라지기 전에 flush하라:

```
proven_byte_t buf[256];
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out,
    (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf });
(void)proven_fprintln(w, "the important line");
(void)proven_writer_flush(w);          /* now it has happened */
```

reader가 당신에게 건네는 줄은 그 버퍼 안을 가리킨다. 그것은 다음 호출 전까지만 유효하다. 그것이 백만 줄을 읽는 데 백만 번의 할당이 아니라 하나의 버퍼가 들게 하는 원리다—그리고 당신이 그것을 붙들어 두는 순간 dangling 포인터다.

잘못됨(Wrong):

```
proven_u8str_view_t lines[100];
for (int i = 0; i < 100; ++i) {
    proven_result_u8str_view_t ln = proven_reader_read_line(&st);
    lines[i] = ln.val;        /* wrong: every entry aliases the SAME buffer, and the
                                 next read_line overwrites what the last one returned */
}
```

올바름(Correct): 다시 호출하기 전에, 보관해야 하는 바이트를 복사하라(`proven_u8str_t`로, 아레나로, 어디로든).

flush는 durability 장벽이 아니다. `proven_writer_flush`는 버퍼드 writer의 바이트를 그 뒤의 것으로 밀어 넣는다; 파일의 바이트를 디스크에 올리는 것은 `proven_fs_sync`다. 이 둘은 서로 다른 연산이며, 한 단어가 정직하게 둘 모두를 뜻할 수는 없었다—그것이 바로 둘 다라고 주장하면서 어느 쪽도 아니었던 옛 `proven_sysio_flush`가 사라진 이유다.

버퍼보다 긴 줄은 절단이 아니라 거부된다. `PROVEN_ERR_OUT_OF_BOUNDS`—그리고 reader는 그 줄에서 멈춘 채로 남는다: 재동기화(resync)는 없다. 담을 수 없는 줄은 그 바이트를 어떻게 할지 결정하지 않고서는 건너뛸 수 있는 줄이 아니기 때문이다. 예상하는 입력에 맞춰 버퍼의 크기를 정하라. (버퍼를 정확히 채우는 줄은 괜찮다: 개행이 함께 들어가지 않아도 되고, 개행이 전혀 없는 마지막 줄도 마찬가지다.)

표준 스트림

`stream.h`에는 writer, reader, 버퍼드 writer, 그리고 라인 reader가 있다. `sysio.h`에는 stdin, stdout, stderr가 있다. 이 둘이 서로 소개되기 전까지는, 두 가지가 그저 불가능했다—그리고 한 호출은 거짓말이었다.

stdin을 한 줄씩 읽는 방법이 없었다. 프로그램이 stdin으로 하는 가장 흔한 일인데, 선택지는 토큰 스캐너이거나, 결코 끝나지 않을 수도 있는 스트림 전체를 읽는 것이었다. 그 다리(bridge)가 그것을 고친다: 표준 핸들이 호출자 소유 저장소에 주차되므로, 라인 reader에게 가리킬 안정적인 무언가가 생긴다.

```
proven_byte_t buf[4096];
proven_sysio_lines_t lines;
if (proven_is_ok(proven_sysio_stdin_lines(&lines, (proven_mem_mut_t){ .ptr = buf, .size = sizeof buf }))) {
    for (;;) {
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(line.err)) break;   /* OUT_OF_BOUNDS: a line longer than `buf` */
        /* `line.val` points INTO `buf` and is valid only until the next call. */
        proven_println("{}", PROVEN_ARG(line.val));
    }
}
```

이것은 라인 reader의 속성을 물려받는데, 이는 두 번째 것을 쓰지 않는 것의 요점이다: view는 무할당, `"\r\n"`이 처리되고, 후행 개행이 없는 마지막 줄도 여전히 반환되며, 당신의 버퍼보다 긴 줄은 `PROVEN_ERR_OUT_OF_BOUNDS`다—결코 조용히 절단된 줄이 아니다.

포매터를 표준 스트림으로 겨눌 수 없었다. `proven_fprintln`은 writer를 받는데; stdout은 그것이 아니었다. 이제는 그렇고, 버퍼드 writer일 수도 있다—그래서 천 개의 작은 줄이 천 번이 아니라 한 번의 시스템 콜로 든다.

`proven_sysio_stdout_writer(&st)`	stdout 위의 언버퍼드 writer. 모든 쓰기가 하나의 write 시스템 콜이다.
`proven_sysio_stderr_writer(&st)`	stderr에 대해 같은 것—에러에 대해 당신이 원하는 것이다: 다음 코드 줄이 실행되기 전에 그것이 나간다.
`proven_sysio_stdin_reader(&st)`	stdin 위의 reader.
`proven_sysio_stdout_buffered(&out, buf)`	당신이 소유하는 버퍼 위의 버퍼드 writer 뒤에 놓인 stdout.
`proven_sysio_file_buffered(&out, file, buf)`	임의의 열린 파일에 대해 같은 것.

그리고 이제 `flush`는 무언가를 뜻한다. `proven_sysio_flush`는 예전에 존재하지 않는 버퍼를 flush한다고 주장했다: POSIX에서는 no-op, Windows에서는 디스크 sync. 그것은 **삭제되었다**. 버퍼드 writer의 바이트를 OS로 밀어 넣는 것은 `proven_writer_flush`이고, OS의 바이트를 디스크로 밀어 넣는 것은 `proven_fs_sync`다. 이 둘은 서로 다른 연산이며 이제는 그렇다고 말한다.

> **당신은 버퍼드 writer를 flush해야 한다.** 버퍼가 차거나 당신이 flush하기 전까지는 아무것도 터미널에 도달하지 않으며, 이 라이브러리의 어떤 것도 당신 몰래 그것을 하기 위해 `atexit` 핸들러를 등록하지 않는다—당신의 프로세스를 소유하는 라이브러리는 당신이 추론할 수 없는 라이브러리이기 때문이다. 결코 flush되지 않는 버퍼드 출력은 결코 일어나지 않은 출력이다. 직접 호출들 (`proven_print`, `proven_println`, `proven_eprint`)은 바로 이 이유로 언버퍼드로 남는다: 그것들이 쓰는 것은 반환되기 전에 이미 나가는 중이다.

당신이 보유하는 구조체

```
typedef struct {
    proven_file_t        file;
    bool                 console;   /* a Windows console: see the next section */
    proven_sysio_carry_t carry;     /* text in flight at the console edge; not yours to touch */
} proven_sysio_std_t;
    /* Storage for a standard handle, so a writer or reader has something stable to
       point at. proven_writer_from_file takes a proven_file_t * and the file must
       outlive the writer - so it cannot be a temporary. This is that storage. */

typedef struct {
    proven_sysio_std_t       std;
    proven_writer_buffered_t buffered;
} proven_sysio_out_t;        /* a buffered writer over a standard stream or a file */

typedef struct {
    proven_sysio_std_t       std;
    proven_reader_buffered_t buffered;
} proven_sysio_lines_t;      /* a line reader over a standard stream or a file */
```

셋 모두 호출자 소유 상태다.

주의사항, 그리고 무엇이 잘못되는가

이 상태 구조체들은 자기 자신을 가리키는 포인터를 담는다. 당신이 돌려받는 writer는 당신이 넘긴 구조체 안의 `&st->std.file`을 주소로 삼는다. 구조체를 복사하거나 값으로 반환하면, writer는 여전히 원본을 가리킨다—그것은 죽은 프레임일 수 있다.

잘못됨(Wrong)—그리고 한 감사에서 정확히 이것이 heap-use-after-free로 재현되었다:

```
proven_sysio_out_t out;
proven_writer_t w = proven_sysio_stdout_buffered(&out, buf);
```

```
proven_sysio_out_t copy = out;      /* wrong: `w` still points into `out` */
/* ... `out` goes out of scope ... */
(void)proven_writer_write_str(w, PROVEN_LIT("boom"));   /* writes through dead storage */
```

유일한 예외는 proven_sysio_lines_t다: proven_sysio_read_line은 매 호출마다 그것을 다시 바인딩하므로, 라인 reader는 이동**될 수 있다**. 그것은 의도된 배려인데, 그것이 상태를 포인터로 받기 때문이다—"재배치 가능"을 뜻하는 형태—그리고 라이브러리는 약속의 형태를 한 함정을 놓아서는 안 되기 때문이다.

0으로 초기화된 proven_file_t는 invalid 핸들이 아니다—POSIX에서 그것은 fd 0, 즉 stdin이다. 라이브러리는 당신이 채워 넣기를 잊은 핸들과 정당하게 fd 0을 가리키는 핸들을 구별할 수 없다.

```
proven_sysio_out_t out;
proven_file_t f = {0};                                  /* wrong: this is stdin */
proven_writer_t w = proven_sysio_file_buffered(&out, f, buf);   /* writes to fd 0 */
```

버퍼드 stdout과 언버퍼드 stderr는 당신이 쓴 순서대로 뒤섞이지 않는다. 당신이 버퍼링하는 것은 당신의 버퍼에 앉아 있는 동안 stderr는 곧장 나간다. 당신의 출력 뒤에 나타나야 하는 에러를 출력하기 전에 flush하라.

UTF-16 텍스트 입출력과 윈도 콘솔

두 문제가 이 절을 함께 쓰는 것은 원인이 같아서다: 텍스트가 어디서나 바이트인 것은 아니다.

UTF-16 텍스트는 들어오고 나갈 길이 없었다. proven_u16str_t - 윈도 와이드 API가 돌려주는 이름, UTF-16 파일에서 온 텍스트 - 를 출력할 수도, 파일에 쓸 수도, 한 줄씩 읽을 수도 없었다. 이제 u16 텍스트는 u8 텍스트가 다니는 모든 통로로 다닌다. 출력은 포매터를 거치므로 proven_u16str_view_t에 대한 PROVEN_ARG가 proven_println, proven_eprintln, 파일이나 다른 어떤 쓰기 스트림(writer)으로의 proven_fprintln, proven_u8str_append_fmt에서 동작하고, UTF-8로 쓴다. 받는 쪽이 UTF-16 *바이트*를 원하면 - UTF-16LE 파일을 읽는 윈도 도구 같은 - proven_writer_write_u16이 지정한 바이트 순서로 쓰고, 바이트 순서 표시(BOM)는 proven_writer_write_bom으로 요청할 때만 쓴다.

입력은 proven_u16_reader_t다: 어떤 바이트 원본 위에서든 UTF-8, UTF-16LE, UTF-16BE를 내 코드 유닛 버퍼로 해독하는 읽기 스트림(reader)으로, 한 줄씩 또는 한 덩어리씩 읽는다. PROVEN_TEXT_AUTO는 앞머리의 바이트 순서 표시가 정하게 하고 그것을 소비한다. 표시가 없으면 UTF-8이다. 원본을 두 번 읽는 사이에 잘린 문자를 이어 붙이고, 바이트 줄 리더의 규칙을 지킨다: "\r\n"은 '\r'을 잃고, 마지막 줄은 개행이 없어도 되며, 버퍼보다 긴 줄은 PROVEN_ERR_OUT_OF_BOUNDS다. utf.h처럼 엄격하다: 잘못된 입력은 그 앞의 올바른 텍스트를 넘겨준 뒤 PROVEN_ERR_INVALID_ENCODING으로 리더를 멈추고, 문자 한가운데서 끝나는 원본도 마찬가지다.

윈도 콘솔은 파일이 아니다. 콘솔은 WriteFile이 건넨 바이트를 자기 코드 페이지 - 한국어 윈도에서는 949 - 로 해독하므로, 누군가 chcp 65001을 실행해 두지 않은 한 UTF-8을 쓰면 글자가 깨졌고, 콘솔에 대한 ReadFile은 UTF-8이 아니라 코드 페이지 바이트를 돌려주었다. 라이브러리 안에서 콘솔의 코드 페이지를 바꾸면, 이 프로그램이 끝난 뒤에도 그 콘솔을 함께 쓰는 다른 모든 프로그램의 코드 페이지가 바뀐다. 그래서 sysio 계층은 다른 길을 택한다: 표준 핸들이 콘솔이면(GetConsoleMode가 받아들이면) 그 쓰기·읽기 스트림은 WriteConsoleW와 ReadConsoleW로 UTF-16을 주고받으며 경계에서 변환한다. proven_print와 proven_eprint, stdout·stderr 쓰기 스트림, 버퍼 쓰기 스트림, u8·u16 줄 리더, 토큰 스캐너가 모두 여기에 해당한다. 두 번의 쓰기에 걸쳐 잘린 UTF-8 문자 - 버퍼 쓰기 스트림은 문자 경계가 아니라 버퍼 크기에서 플러시한다 - 는 상태 구조체에 담아 두었다가 다음 쓰기로 완성한다. 콘솔 줄 맨 앞의 Ctrl+Z는 모든 윈도 콘솔 프로그램에서처럼 입력의 끝이다. 파일, 파이프, 리디렉션된 스트림은 콘솔이 아니므로 여전히 정확한 바이트를 받고, POSIX에서는 어떤 것도 콘솔이 아니다.

stdin을 UTF-16으로, 한 줄씩 읽기:

```
#ifndef PROVEN_NO_U16STR
proven_u16 wbuf[512];
proven_sysio_u16_lines_t in;
if (proven_is_ok(proven_sysio_stdin_u16_lines(&in, wbuf, 512))) {   /* stdin is read as UTF-8 */
    for (;;) {
        proven_result_u16str_view_t line = proven_sysio_read_u16_line(&in);
```

```
        if (line.err == PROVEN_ERR_EOF) break;
        if (!proven_is_ok(line.err)) break;   /* OUT_OF_BOUNDS, or INVALID_ENCODING */
        /* `line.val` points INTO `wbuf` until the next call - e.g. hand it to a wide API. */
        (void)proven_eprintln("{} code units", PROVEN_ARG(line.val.size));
    }
}
#endif
```

`proven_arg_u16(view)` / `PROVEN_ARG(view)`	포매터 인자로서의 u16 텍스트, UTF-8로 쓰인다.
`proven_writer_write_u16(w, text, enc)`	u16 텍스트를 어떤 쓰기 스트림으로든 UTF-8, UTF-16LE, UTF-16BE로. 먼저 검증한다: 잘못된 텍스트는 아무것도 쓰지 않는다.
`proven_writer_write_bom(w, enc)`	`enc`의 바이트 순서 표시, 요청할 때만.
`proven_u16_reader_init(&st, reader, enc, buf, cap)`	어떤 읽기 스트림 위의 u16 리더, 내 코드 유닛 `cap`개 (최소 2) 버퍼를 거친다.
`proven_u16_reader_read_line(&st)`	다음 줄을, 내 버퍼 안을 가리키는 u16 뷰로.
`proven_u16_reader_read(&st, dest, cap)`	코드 유닛 최대 `cap`개, 서로게이트 쌍의 반쪽은 절대 없이.
`proven_sysio_u16_lines_open(&st, file, enc, buf, cap)`	같은 리더를 파일이나 표준 스트림 위에.
`proven_sysio_stdin_u16_lines(&st, buf, cap)`	stdin 위에, UTF-8로 읽는다(콘솔은 UTF-16으로 읽는다).
`proven_sysio_read_u16_line(&st)`	다음 줄; 호출 사이에 구조체를 옮겨도 된다.

당신이 들고 있는 구조체

```
typedef struct {
    proven_reader_t inner;  proven_text_encoding_t enc;   /* AUTO resolved from the BOM */
    proven_u16 *buf;  proven_size_t cap, len, cursor;     /* your buffer, in code units */
    proven_byte_t raw[1024];  proven_size_t raw_pos, raw_len;   /* bytes read; [raw_pos, raw_len) not yet
decoded */
    proven_u16 peek[2];  proven_size_t peek_len;          /* one character of lookahead */
    bool bom_checked, eof;  proven_err_t err;             /* err is sticky */
} proven_u16_reader_t;

typedef struct {
    proven_sysio_std_t  std;
    proven_u16_reader_t reader;
} proven_sysio_u16_lines_t;   /* a u16 line reader over a standard stream or a file */
```

둘 다 호출자 소유 상태다. 아무것도 할당하지 않는다: 원시 바이트는 구조체 안에, 해독된 텍스트는 내 버퍼에 놓인다.

주의할 점, 그리고 무엇이 잘못되는가

콘솔 핸들에 대한 `proven_writer_from_file`은 변환하지 않는다. 그것은 파일 쓰기 스트림이라 바이트를 있는 그대로 쓴다 - UTF-8이 아닌 코드 페이지의 윈도 콘솔에서는 깨진 글자다. 콘솔을 감지하는 `proven_sysio_*` 호출로 콘솔 쓰기·읽기 스트림을 만들어라.

```
proven_file_t out = proven_sysio_stdout();
proven_writer_t w = proven_writer_from_file(&out);   /* wrong for a console: bytes, not text */
proven_sysio_std_t st;
proven_writer_t ok = proven_sysio_stdout_writer(&st); /* right: converts on a console */
```

엄격은 콘솔에서도 엄격이다. 콘솔로 보낸 잘못된 UTF-8은 그 앞의 올바른 텍스트를 보여 준 뒤 `PROVEN_ERR_INVALID_ENCODING`으로 거부된다. 무엇도 물음표로 바뀌지 않는다. 플러시는 텍스트의 끝이 아니다 - 버퍼 쓰기 스트림은 버퍼 크기에서 비우고, 당신은 어느 두 바이트 사이에서든 플러시할 수 있다 - 그래서 플

러시 때 아직 열려 있는 문자의 바이트는 간직되고, 다음 쓰기가 그것을 완성하거나, 완성할 수 없으면 거부된다. `proven_print`와 `proven_eprint`는 포맷된 텍스트 전체를 쓰므로, 이들에게 문자 한가운데서 끝나는 텍스트는 그대로 거부된다.

명시한 인코딩은 바이트 순서 표시를 벗기지 않는다. `PROVEN_TEXT_UTF16LE`로 연 파일이 `FF FE`로 시작하면 첫 줄의 첫 유닛으로 문자 U+FEFF가 전달된다. 누가 썼는지 통제할 수 없는 파일은 `PROVEN_TEXT_AUTO`로 열어라.

서로게이트 쌍은 한 문자다. `proven_writer_write_u16`은 쌍의 상위 반쪽으로 끝나는 텍스트를, 다음 호출이 하위 반쪽을 줄 예정이었더라도 거부한다 - 쌍은 한 호출 안에 두어라. 같은 이유로 `proven_u16_reader_read`는 유닛 두 개의 자리가 필요하다.

너비는 UTF-8 바이트로 센다. u16 인자를 감싼 `{:>10}`은 u8 뷰(view)에서와 똑같이 UTF-8 열 바이트로 채운다 - 열 문자가 아니고, 터미널 열 칸도 아니다. 한글 한 음절은 세 바이트이고 두 칸이다.

예제: u16 텍스트를 표준 스트림으로, UTF-16LE 파일로, 그리고 다시 안으로

```
#include <string.h>

/*
 * UTF-16 텍스트의 입출력: 표준 스트림으로, 읽을 쪽이 기대하는 인코딩의 파일로, 그리고
 * 한 줄씩 다시 안으로.
 *
 * 출력은 포매터를 거치므로, u8 텍스트를 받는 모든 곳이 u16 텍스트도 받는다: PROVEN_ARG 는
 * proven_u16str_view_t 에 u16 렌더러를 골라 UTF-8 로 쓴다. 받는 쪽이 UTF-16 바이트를
 * 원하면 - UTF-16LE 파일을 읽는 윈도 도구 같은 - 쓰기 스트림이 지정한 바이트 순서로 쓴다.
 *
 * 입력은 세 인코딩 중 어느 것이든 내 u16 버퍼로 해독하는 읽기 스트림이다. 원본을 두 번
 * 읽는 사이에 잘린 문자를 이어 붙이고, 바이트 줄 리더의 규칙을 그대로 지킨다: "\r\n" 은
 * '\r' 을 잃고, 마지막 줄은 개행이 없어도 되며, 버퍼보다 긴 줄은 잘린 줄이 아니라 오류다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* "안녕 🙂" - 한글과, 서로게이트 쌍인 이모지. */
    static const proven_u16 hello[] = { 0xC548, 0xB155, ' ', 0xD83D, 0xDE42 };
    proven_u16str_view_t text = { hello, 5 };

    /* --- 표준 스트림으로 ------------------------------------------- */

    /* stdout 과 stderr 에는 UTF-8. 윈도 콘솔이면 sysio 계층이 UTF-16 으로 넘기므로,
     * 코드 페이지가 무엇이든 올바르게 보인다. */
    proven_err_t err = proven_println("stdout: {}", PROVEN_ARG(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "u16 text prints like u8 text");
    err = proven_eprintln("stderr: {}", proven_arg_u16(text));
    EXAMPLE_REQUIRE(proven_is_ok(err), "and to stderr");

    /* 버퍼 쓰기 스트림도 받는다. 플러시하라, 안 하면 없었던 일이다. */
    proven_sysio_out_t out;
    proven_byte_t buf[256];
    proven_writer_t w = proven_sysio_stdout_buffered(&out, (proven_mem_mut_t){ buf, sizeof buf });
    EXAMPLE_REQUIRE(proven_is_ok(proven_fprintln(w, "buffered: {}", PROVEN_ARG(text)).err), "fprintln takes
u16");
    EXAMPLE_REQUIRE(proven_is_ok(proven_writer_flush(w)), "and the flush sends it");

    /* 엄격: 짝 없는 서로게이트는 쓰레기를 찍는 대신 줄 전체를 실패시킨다. */
    static const proven_u16 torn_units[] = { 'o', 'k', 0xD83D };
    proven_u16str_view_t torn = { torn_units, 3 };
    err = proven_println("{}", PROVEN_ARG(torn));
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_ENCODING, "half a surrogate pair is refused");
```

```
    /* --- 파일로, BOM 을 붙인 UTF-16LE 로 ---------------------------------- */

    proven_u8str_view_t path = PROVEN_LIT("proven_example_u16.txt");
    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_WRITE ¦ PROVEN_FS_CREATE ¦
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the file must succeed");
    if (!proven_is_ok(f.err)) return EXAMPLE_OK();

    proven_sysio_out_t fout;
    proven_writer_t fw = proven_sysio_file_buffered(&fout, f.value, (proven_mem_mut_t){ buf, sizeof buf });
    /* BOM 은 요청했기 때문에만 쓰인다. */
    err = proven_writer_write_bom(fw, PROVEN_TEXT_UTF16LE);
    for (int i = 0; i < 2 && proven_is_ok(err); ++i) {
        err = proven_writer_write_u16(fw, text, PROVEN_TEXT_UTF16LE);
        if (proven_is_ok(err)) err = proven_writer_write_u16(fw, (proven_u16str_view_t){ u"\r\n", 2 },
PROVEN_TEXT_UTF16LE);
    }
    if (proven_is_ok(err)) err = proven_writer_flush(fw);
    EXAMPLE_REQUIRE(proven_is_ok(err), "two lines of UTF-16LE are written");
    (void)proven_fs_close(f.value);

    /* --- 다시 안으로, 한 줄씩 ---------------------------------------------- */

    f = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "opening the file again must succeed");
    if (proven_is_ok(f.err)) {
        /* AUTO: BOM 이 UTF-16LE 라고 알려 주고, 소비된다. 버퍼는 코드 단위이고
         * 가장 긴 줄을 담아야 한다. */
        proven_u16 line_buf[64];
        proven_sysio_u16_lines_t lines;
        err = proven_sysio_u16_lines_open(&lines, f.value, PROVEN_TEXT_AUTO, line_buf, 64);
        EXAMPLE_REQUIRE(proven_is_ok(err), "the line reader opens");
        int n = 0;
        for (;;) {
            proven_result_u16str_view_t line = proven_sysio_read_u16_line(&lines);
            if (line.err == PROVEN_ERR_EOF) break;
            EXAMPLE_REQUIRE(proven_is_ok(line.err), "each line reads");
            if (!proven_is_ok(line.err)) break;
            /* 뷰는 다음 호출 전까지 line_buf 안을 가리킨다. */
            EXAMPLE_REQUIRE(line.val.size == 5 && memcmp(line.val.ptr, hello, sizeof hello) == 0,
                            "each line is the text, without its CR LF");
            ++n;
        }
        EXAMPLE_REQUIRE(n == 2 && lines.reader.enc == PROVEN_TEXT_UTF16LE, "two lines, read as UTF-16LE");
        (void)proven_fs_close(f.value);
    }
    (void)proven_fs_remove(alloc, path);

    /* --- 어떤 읽기 스트림이든, 어떤 인코딩이든 ------------------------------ */

    /* 이미 메모리에 있는 바이트 - 여기서는 UTF-8 - 위의 같은 리더. read() 는 어느 줄에
     * 속하든 코드 단위를 내주고, 쌍의 반쪽은 절대 내주지 않는다. */
    proven_reader_view_t src;
    proven_u16 rbuf[16];
    proven_u16_reader_t rd;
    err = proven_u16_reader_init(&rd, proven_reader_from_view(&src, PROVEN_LIT("A\xF0\x9F\x99\x82\nB")),
                                 PROVEN_TEXT_UTF8, rbuf, 16);
    EXAMPLE_REQUIRE(proven_is_ok(err), "a u16 reader over UTF-8 bytes");
    proven_result_u16str_view_t first = proven_u16_reader_read_line(&rd);
    EXAMPLE_REQUIRE(proven_is_ok(first.err) && first.val.size == 3, "'A' and the emoji's two units");
```

```
    proven_u16 rest[4];
    proven_result_size_t got = proven_u16_reader_read(&rd, rest, 4);
    EXAMPLE_REQUIRE(proven_is_ok(got.err) && got.value == 1 && rest[0] == 'B', "the last line, without a
newline");
    EXAMPLE_REQUIRE(proven_u16_reader_read(&rd, rest, 4).err == PROVEN_ERR_EOF, "then the end");

    return EXAMPLE_OK();
}
```

콘솔 경로는 리눅스에서도 도는 프로그램으로는 보여 줄 수 없다. 그것은 윈도 11에서 `b039-console-check.c`로 확인했다. 이 프로그램은 코드 페이지 949의 자기 콘솔을 만들고, 각 sysio 경로로 쓴 뒤 화면 버퍼를 다시 읽으며, 콘솔 입력 버퍼에 키 이벤트를 넣어 줄 리더와 스캐너로 읽는다.

난수, 용도별

단일한 "random"은 없다. 동일해 보이지만 그렇지 않은 두 가지 작업이 있다:

당신의 작업	사용	이유
키, 토큰, nonce—공격자가 추측해서는 안 되는 모든 것.	`proven_random_bytes`, 또는 그것으로 시드된 `proven_chacha_rng_t`.	오직 암호학적 source만이 추측 불가능하다.
같지만, OS가 없는 타깃에서.	보드 자체의 엔트로피로 시드된 `proven_chacha_rng_t`.	ChaCha20은 순수 산술이다; OS가 필요 없다. 그것은 오직 그 시드만큼만 추측 불가능하다.
시뮬레이션, 테스트, 게임, 표본추출.	`proven_xoshiro256ss_t`.	빠르고, **재현 가능하다**: 같은 시드가 같은 실행을 재생하며, 그것이 실패하는 테스트를 디버깅 가능하게 만든다.
범위 내의 수, 셔플, [0,1)의 float.	`proven_rng_below`, `proven_rng_range`, `proven_rng_f64`, `proven_rng_shuffle`—어떤 source 위에서든.	`% n`은 편향되어 있고, 모두가 그럼에도 그것을 쓴다. 이것들은 그렇지 않다.

두 요구사항은 정면으로 대립한다. 재현 가능이란 예측 가능을 뜻하고, 예측 가능은 정확히 토큰이 그래서는 안 되는 것이다: `proven_xoshiro256ss_t`의 몇 개 출력이 그 전체 상태를, 따라서 그것이 앞으로 낳을 모든 수를 드러낸다. 그것은 재생해야 하는 시뮬레이션에는 기능이고 세션 토큰에는 재앙이다—그래서 둘은 혼동될 수 없는 이름을 지니며, 선택은 무언가가 어떻게 시드되었는가에 묻히는 대신 호출 지점에서 가시적이다.

트레이트는 실패하지 않는다(infallible). `proven_rng_t`는 무작위 바이트의 source이며, valid한 것에서 뽑는 것은 실패할 수 없다. 그것은 단순화가 아니라, 실패가 어디로 갔는지다. 운영체제에 엔트로피를 요청하는 것은 실패할 수 있으므로, 그 실패는 정확히 한 곳—시딩—에 국한되며, 당신은 그것을 시작 시점에 한 번 검사한다. 하류의 모든 draw는 전역적(total)이다.

`proven_random_bytes(buf, len)`	OS CSPRNG. 실패 시 `false`를 반환한다; 그러면 `buf`를 사용하지 말라. `len == 0`은 성공적인 no-op이다.
`proven_random_u64()`	OS로부터의 강한 한 워드, 실패 시 `0`.
`proven_random_u64_checked(&out)`	강한 한 워드, 그리고 없으면 `false`. 토큰이나 키에 쓰는 형태.
`proven_chacha_rng_seed_from_entropy(&g)`	엔트로피 source로부터 암호학적 생성기를 시드. **이것이 실패할 수 있는 호출이다.**
`proven_random_set_source(fn, ctx)`	엔트로피 source를 설치. 호스티드 타깃에는 OS가 이미 설치되어 있다; 베어메탈 타깃은 보드의 TRNG를 설치한다.
`proven_chacha_rng_seed(&g, seed32)`	당신이 공급하는 32바이트로부터 시드—보드의 하드웨어 엔트로피 source. 결코 시계로부터는 아니다.

`proven_xoshiro256ss_seed(&g, seed)`	재현 가능 생성기를 시드. 시드 0도 괜찮다: SplitMix64를 통해 확장된다.

엔트로피는 어디서 오는가

위의 모든 것은 순수 산술이다—생성기들, 헬퍼들, 그리고 `proven_random_bytes` 자체까지. 플랫폼에 따라 다른 것은 그 뒤의 **엔트로피 source**인데, 그것이 프로그램이 스스로 계산할 수 없는 유일한 것이기 때문이다.

- **호스티드:** OS CSPRNG가 당신을 위해 설치되어 있다—Linux의 `getrandom`, BSD와 macOS의 `getentropy`, Windows의 `BCryptGenRandom`, 그리고 이들 중 어느 것도 없는 곳의 `/dev/urandom`. 당신은 아무것도 호출하지 않는다.
- **베어메탈:** 당신이 하나를 설치하기 전까지는 source가 없다. 보드는 실제 엔트로피를 가지고 있다—온칩 TRNG, 링 오실레이터, ADC의 노이즈 플로어—그리고 라이브러리는 그것이 어디 있는지 알 수 없다.

```
/* On a board: hand the library its hardware entropy, once, at startup.
 * (A listing, not a fragment: it defines a function, and `hardware_rng_read` is
 * whatever your SoC calls its entropy register.) */
static bool board_trng(void *ctx, void *buf, proven_size_t len) {
    (void)ctx;
    /* read the SoC's entropy register into buf; return false if it is not ready */
    return hardware_rng_read(buf, len);
}

proven_random_set_source(board_trng, NULL);

/* From here everything above works unchanged - including the one call that turns a few
 * hundred bytes of hardware entropy into an endless cryptographic stream. */
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* the TRNG was not ready. The generator is INERT - it yields zeros and an invalid
     * trait - so ignoring this does not get you plausible-looking bytes. */
}
```

설치된 source가 없으면 `proven_random_bytes`는 **false**를 반환한다. 그것은 시계로 시드된 PRNG로 폴백하지 않는다. 그것은 성공처럼 보이지만 아무것도 보고하지 않는 보안 구멍이기 때문이다—거부는 호출자가 처리할 수 있는 하나의 사실이다.

의도적으로 내장된 **`RDRAND` / `RNDR` 백엔드는 없다.** 호스티드 타깃에서는 OS가 이미 CPU의 명령을 자신의 풀에 섞어 넣으므로, 그것을 직접 호출하는 것은 아무것도 얻지 못하면서 그 섞임을 잃게 하고; 유일한 source로 쓰이는 원시 하드웨어 명령은 사람들이 10년간 논쟁해 온 바로 그 방식이다. 원한다면, 그것은 이 훅 뒤로 네 줄이다—그러면 그 선택은 가시적으로 당신 것이다.

당신이 보유하는 구조체

셋 모두 호출자 소유 상태다: 아무것도 할당하지 않고, 파괴할 것이 없으며, **하나를 복사하면 그 시퀀스를 복제한다**.

```
typedef struct { const proven_rng_vtable_t *vt; void *ctx; } proven_rng_t;
    /* The trait: two pointers, held by value. `ctx` points at one of the generators
       below, which must outlive it. Drawing from a VALID one cannot fail - that is
       the whole design: the failure lives in seeding, not in drawing. */

typedef struct { proven_u64 s[4]; } proven_xoshiro256ss_t;
    /* 256 bits of state. Reproducible, and NOT secret-grade. */

typedef struct {
    proven_u32    state[16];   /* the ChaCha state: constants, key, counter, nonce */
    proven_byte_t block[64];   /* the keystream block currently being handed out */
    proven_size_t used;        /* how much of it is spent */
    proven_u32    seeded;      /* set only by seeding. A zero-initialised struct is
```

```
                                the shape of "never seeded", and must stay inert. */
} proven_chacha_rng_t;
```

레퍼런스

API	의도	반환
`proven_random_bytes(buf, len)`	엔트로피 source로부터 채움(기본은 OS). **실패할 수 있는 유일한 호출.**	`bool`. `false`이면 `buf`는 미지정이며 사용해서는 안 된다. `len == 0`은 성공한다.
`proven_random_u64()`	같은 source로부터의 강한 한 워드.	`proven_u64`, 실패 시 `0`—그것도 valid한 draw이므로, 둘을 구별해야 할 때는 `proven_random_bytes`를 사용하라.
`proven_random_u64_checked(&out)`	비밀에 쓰는 강한 한 워드.	`bool`. `false`이면 `out`은 0이며 쓰면 안 된다: 0인 토큰은 이 함수가 막으려는 예측 가능한 비밀이다.
`proven_random_set_source(fn, ctx)`	엔트로피 source를 설치. 호스티드 타깃에는 불필요; 이것이 보드가 자신의 TRNG를 넘겨주는 방법이다.	void.
`proven_xoshiro256ss_seed(&g, seed)`	재현 가능 생성기를 시드. 어떤 시드든 괜찮다—0조차; SplitMix64를 통해 확장된다.	void.
`proven_xoshiro256ss_next(&g)`	다음 워드. 핫 패스: 트레이트를 통해서가 아니라 직접 호출하라.	`proven_u64`.
`proven_xoshiro256ss_rng(&g)`	헬퍼를 위해 `proven_rng_t`로 본다.	`proven_rng_t`.
`proven_chacha_rng_seed(&g, seed32)`	당신이 공급하는 32바이트의 **실제 엔트로피**로부터 암호학적 생성기를 시드.	void.
`proven_chacha_rng_seed_from_entropy(&g)`	설치된 source로부터 시드. **이것을 검사하라.**	`bool`. `false`이면 생성기는 INERT 상태로 남는다—0을 낳고 invalid한 트레이트를 낳는다.
`proven_chacha_rng_next/_fill`	draw. 일단 시드되면 실패할 수 없다.	`proven_u64` / void.
`proven_chacha_rng(&g)`	`proven_rng_t`로 본다.	`proven_rng_t`—생성기가 성공적으로 시드된 적이 없으면 **invalid**.
`proven_rng_u64(rng)` / `proven_rng_fill(rng, buf, len)`	어느 생성기로부터든 트레이트를 통해 draw.	`proven_u64` / void. invalid source에는 `0` / no-op.
`proven_rng_below(rng, bound)`	`[0, bound)`에서 균일, **무편향**.	`proven_u64`; `bound == 0`일 때 `0`.
`proven_rng_range(rng, lo, hi)`	`[lo, hi]`에서 균일, 포함적. 전체 `INT64_MIN..INT64_MAX` 범위도 오버플로하지 않는다.	`proven_i64`; `hi < lo`이면 `lo`.
`proven_rng_f64(rng)`	`[0, 1)`에서 균일. 53비트; 결코 `1.0`을 반환하지 않는다.	`double`.
`proven_rng_shuffle(rng, base, count, elem_size)`	무편향 Fisher-Yates 순열, 제자리(in place).	void.

```
/* A session token must not be guessable, so it may not be the 0 that proven_random_u64
 * returns when the entropy source fails. The checked form says so instead. */
proven_u64 token;
if (!proven_random_u64_checked(&token)) {
    proven_eprintln("no entropy source: refusing to issue a session token");
} else {
    proven_println("token {:x}", PROVEN_ARG(token));
}
```

주의사항, 그리고 무엇이 잘못되는가

`proven_xoshiro256ss_t`로 결코 비밀을 생성하지 말라. 그것은 예측 가능하기 때문에 빠르다: 그 출력 몇 개가 256비트 상태 전체를, 그리고 그 상태로부터 그것이 앞으로 낳을 모든 수를 드러낸다. 두 생성기는 정확히 이 이유로 혼동될 수 없는 이름을 지닌다.

잘못됨(Wrong)—공격자가 몇 개를 지켜본 뒤 계산할 수 있는 세션 토큰:

```
proven_xoshiro256ss_t g;
proven_xoshiro256ss_seed(&g, 12345);
proven_u64 session_token = proven_xoshiro256ss_next(&g);   /* wrong: predictable */
```

암호학적 생성기를 결코 시계, 카운터, 또는 시리얼 번호로 시드하지 말라. ChaCha20은 정확히 그 시드만큼만 추측 불가능하다. 시계에서 유도된 시드는 완벽하게 무작위처럼 보이지만 그렇지 않은 스트림을 낳는다—그것은 명백한 실패보다 나쁘다. 아무것도 그것을 보고하지 않기 때문이다.

```
proven_byte_t seed[32] = { 0 };
memcpy(seed, &now_ns, sizeof now_ns);      /* wrong: ~20 bits of real entropy, and guessable */
proven_chacha_rng_seed(&g, seed);
```

시딩을 검사하라. 그것은 여기서 실패할 수 있는 유일한 것이며, 바로 그래서 그것을 무시하는 것이 솔깃하다. 무시하면 생성기는 inert 상태가 되어 당신에게 0을 건넨다—설계상 그럴듯한 값이 아니라 가시적으로 죽은 값이다.

잘못됨(Wrong):

```
proven_chacha_rng_t g;
proven_chacha_rng_seed_from_entropy(&g);   /* wrong: the bool was the point */
proven_chacha_rng_fill(&g, key, 32);       /* key is now 32 zero bytes */
```

올바름(Correct):

```
proven_chacha_rng_t g;
if (!proven_chacha_rng_seed_from_entropy(&g)) {
    /* No entropy. There is nothing safe to do here except refuse to continue. */
} else {
    proven_byte_t key[32];
    proven_chacha_rng_fill(&g, key, sizeof key);   /* cannot fail: it is seeded */
}
```

`% n`은 편향되어 있고, 모두가 그럼에도 그것을 쓴다. `n`이 2^64를 나누어떨어지게 하지 않는 한 낮은 값들이 더 자주 나온다—어쩌다 하는 점검에서는 보이지 않지만, 셔플이나 표본추출에서는 실재한다.

```
proven_u64 die = proven_rng_u64(rng) % 6 + 1;   /* wrong: 1 and 2 are slightly likelier */
```

올바름(Correct): `proven_rng_below(rng, 6) + 1`.

시드된 생성기를 복사하지 말라—그 스트림을 복제하려는 의도가 아니라면. 하나에서 복사된 두 개의 "독립적인" 생성기는 동일한 출력을 낳는다—그것은 시뮬레이션 재생에는 기능이고 두 개의 토큰을 발급하는 데는 재앙이다.

테스트 스위트에 의해 컴파일되고 실행됨:

```
/*
 * 쓰임새로 나눈 난수. "난수" 하나는 없다. 똑같이 생겼지만 같지 않은 할 일이 둘 있고,
 * 잘못 고르는 것이 위험의 전부다.
```

```
 *
 *   키, 토큰, 논스 - 공격자가 알아맞히면 안 되는 것 - 에는 *암호용* 난수원이 필요하다.
 *   시뮬레이션, 시험, 게임에는 *재현 가능한* 것이 필요하다. 다시 돌려 볼 수 없는 실패한
 *   실행은 디버깅할 수 없는 실패한 실행이기 때문이다. 두 요구는 정면으로 부딪친다.
 *   재현 가능하다는 것은 예측 가능하다는 뜻이고, 예측 가능한 것이야말로 토큰이 되어서는
 *   안 되는 것이다. 그래서 라이브러리는 둘에 다른 이름을 주고, 선택은 씨앗을 어떻게
 *   뿌렸는지에 묻히는 대신 여기 호출 자리에서 눈에 보인다.
 */

int main(void) {
    /* ---- 할 일 1: 비밀. OS 의 CSPRNG - 그리고 난수가 실패할 수 있는 유일한 자리. ---- */
    proven_byte_t key[32];
    EXAMPLE_REQUIRE(proven_random_bytes(key, sizeof key),
                    "the OS must give us strong bytes on a hosted platform");

    /* ---- 할 일 2: 암호용 바이트를 아주 많이, 또는 OS 가 없는 보드에서 조금이라도.
     * ChaCha20 은 순수한 산술이다. 진짜 엔트로피로 한 번 씨를 뿌리면 그 뒤로는 운영체제가
     * 필요 없다 - 뽑을 때마다 시스템 호출을 하지 않고, 베어메탈에서도 돈다.
     * 씨 뿌리기가 실패할 수 있는 *유일한* 걸음이므로, 확인해야 할 것도 그것 하나다. ---- */
    proven_chacha_rng_t crypto;
    EXAMPLE_REQUIRE(proven_chacha_rng_seed_from_entropy(&crypto), "seed the CSPRNG from the OS, once");

    proven_byte_t token[16];
    proven_chacha_rng_fill(&crypto, token, sizeof token);   /* 실패할 수 없다: 이미 씨가 뿌려져 있다 */

    /* ---- 할 일 3: *재현되는* 실행. xoshiro256** 은 빠르고 씨앗에서 똑같이 다시
     * 재생된다. 그것이 실패한 시뮬레이션을 디버깅할 수 있게 만든다. 비밀급이 *아니다* -
     * 출력 몇 개면 상태 전체가 드러난다. 토큰을 만들라고 건네는 일은 절대 없어야 한다. ---- */
    proven_xoshiro256ss_t sim;
    proven_xoshiro256ss_seed(&sim, 12345);

    proven_xoshiro256ss_t replay;
    proven_xoshiro256ss_seed(&replay, 12345);
    EXAMPLE_REQUIRE(proven_xoshiro256ss_next(&sim) == proven_xoshiro256ss_next(&replay),
                    "the same seed replays the same run - that is the whole point");

    /* ---- 도우미들은 proven_rng_t 특성을 통해 *어떤* 난수원 위에서도 돈다. ---- */
    proven_rng_t rng = proven_xoshiro256ss_rng(&sim);

    /* 범위 안의 수. `rng_u64() % 6` 은 모두가 쓰는 것이고, 상한이 2^64 를 나누어떨어지게
     * 하지 않는 한 *치우쳐* 있다 - 작은 값이 더 자주 나온다. 이것은 그렇지 않다. */
    for (int i = 0; i < 100; ++i) {
        proven_u64 die = proven_rng_below(rng, 6) + 1;
        EXAMPLE_REQUIRE(die >= 1 && die <= 6, "a die roll is 1..6, uniformly");
    }

    proven_i64 temperature = proven_rng_range(rng, -40, 85);
    EXAMPLE_REQUIRE(temperature >= -40 && temperature <= 85, "an inclusive range, both ends");

    double p = proven_rng_f64(rng);
    EXAMPLE_REQUIRE(p >= 0.0 && p < 1.0, "a double in [0, 1) - never 1.0");

    /* 치우치지 않은 섞기: 위의 치우치지 않은 인덱스 위에서 도는 피셔-예이츠. 이 반복문의
     * `% n` 판은 어떤 순서를 눈에 띄게 더 좋아한다. */
    int deck[10];
    for (int i = 0; i < 10; ++i) deck[i] = i;
    proven_rng_shuffle(rng, deck, 10, sizeof deck[0]);

    int sum = 0;
    for (int i = 0; i < 10; ++i) sum += deck[i];
```

```
    EXAMPLE_REQUIRE(sum == 45, "a shuffle is a permutation: every card is still there, once");

    /* 암호용 생성기도 같은 특성을 만족하므로, 고르기만 하면 되는 것이 아니라 알아맞힐 수
     * 없어야 할 때 같은 도우미를 그대로 쓸 수 있다. */
    proven_rng_t secure = proven_chacha_rng(&crypto);
    proven_u64 unguessable_index = proven_rng_below(secure, 1000);
    EXAMPLE_REQUIRE(unguessable_index < 1000, "the helpers do not care which source they draw from");

    (void)token;
    (void)key;
    return EXAMPLE_OK();
}
```

실전 예제: 정전이 나도 파일이 깨지지 않게 교체하기
파일을 제자리에서 덮어쓰는 방식에는, 시험에서는 결코 보이지 않고 운영에서는 언젠가 반드시 보이는 실패 방식이 있다. 쓰는 도중에 전원이 나가고, 남은 파일은 예전 것도 새 것도 아니게 된다. 해법은 오래된 네 단계 절차이며, 네 단계 모두가 제 몫을 한다.

1. 새 내용을 진짜 파일 옆의 **임시 파일**에 쓴다.
2. `proven_fs_sync()` — 새 파일의 바이트가 운영체제 캐시가 아니라 저장 장치에 도달한다.
3. `proven_fs_rename()` — 이름이 한 번의 나눌 수 없는 단계로 새 파일로 넘어간다. 읽는 쪽은 옛 파일 전체나 새 파일 전체를 보며, 섞인 것은 결코 보지 않는다.
4. `proven_fs_sync_dir()` — 이름 바꾸기 자체가 저장 장치에 도달한다.

2단계를 건너뛰면 내용이 도착한 적 없는 파일을 이름이 가리킬 수 있다. 4단계를 건너뛰면 내용은 안전한데 그것을 가리키는 이름이 안전하지 않다.

같은 예제가 함께 쓰이는 레코드 단위 호출들도 다룬다.

호출	무엇을 위한 것인가
`proven_fs_pread` / `proven_fs_pwrite`	절대 오프셋에서 읽고 쓰되 **파일 위치를 옮기지 않는다**. 하나의 핸들을 여러 스레드가 함께 써도 안전한 이유다.
`proven_fs_seek` / `proven_fs_tell`	위치를 옮기고, 지금 어디인지 묻는다. 끝에서 음수 오프셋으로 seek 하면 길이를 몰라도 마지막 레코드를 찾는다.
`proven_fs_truncate`	길이를 직접 정한다. 한 번의 호출로 파일 시스템이 숫자 하나를 고치며, 대안은 남길 부분을 복사하는 것이다.
`proven_fs_lock`	**권고(advisory) 잠금**이다. 마찬가지로 잠금을 요청하는 다른 프로세스만 막고, 아예 요청하지 않는 프로그램에는 아무 영향이 없다.
`proven_fs_copy`	바이트를 복제해 독립된 두 번째 파일을 만든다.
`proven_fs_link`	같은 파일에 대한 두 번째 **이름**(하드 링크). 원본이라는 것이 없고, 마지막 이름이 사라질 때까지 데이터가 산다. 같은 파일 시스템 안에서만 된다.
`proven_fs_symlink`	경로를 담은 작은 파일(심볼릭 링크). 파일 시스템을 건너뛸 수 있고, 아무것도 가리키지 않을 수도 있다.
`proven_fs_is_absolute`	이 경로가 루트에서 시작하는가? 규칙이 플랫폼마다 달라서 함수인 것이다.
`proven_fs_mkdir` / `proven_fs_mkdir_all`	디렉터리 하나, 또는 그 위의 빠진 단계까지 모두 만든다. 앞의 것은 이미 쓰이는 이름에 `PROVEN_ERR_EXISTS`를 답하고, 뒤의 것은 이미 디렉터리인 단계를 받아들인다.
`proven_fs_rmdir`	**빈** 디렉터리를 지운다. 비어 있지 않으면 `PROVEN_ERR_INVALID_STATE`로 거절하므로, 재귀 삭제는 명시적인 결정으로 남는다.

```
/*
 * 정전이 파일을 반쯤 쓰인 채로 남겨 두지 못하게 하며 파일을 고치기.
 *
 * 조리법은 오래되었고, 그 모든 걸음이 하중을 받는다.
 *
 *   1. 새 내용을 진짜 파일 옆의 *임시* 파일에 쓴다,
 *   2. proven_fs_sync   - 이제 새 파일의 바이트가 장치에 있다,
 *   3. proven_fs_rename - 이름이 한 걸음에 새 파일로 넘어간다. 읽는 쪽은 옛 파일 전체나
 *                         새 파일 전체를 보지, 그 사이를 보지 않는다,
 *   4. proven_fs_sync_dir - 이제 그 이름 바꾸기 자체가 장치에 있다.
 *
 * 2를 건너뛰면 내용이 도착한 적 없는 파일을 이름이 공표할 수 있다. 4를 건너뛰면 내용은
 * 안전한데 그 이름이 안전하지 않을 수 있다. 둘 다 시험에서는 드러나지 않는다. 둘 다
 * 운영에서, 한 번, 드러난다.
 *
 * 이 프로그램은 레코드 단위 호출들 - seek/tell, pread/pwrite, truncate - 과, 이 모든
 * 일을 두 벌의 프로그램이 동시에 하지 못하게 막는 권고 잠금도 함께 보인다.
 */

typedef struct {
    proven_u32 id;
    proven_u32 score;
} ex_record_t;

static proven_err_t write_records(proven_file_t f, const ex_record_t *recs, proven_size_t n) {
    proven_mem_view_t view = { .ptr = (const proven_byte_t *)recs, .size = n * sizeof recs[0] };
    return proven_fs_write_all(f, view);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_u8str_view_t live = PROVEN_LIT("proven_example_durable.dat");
    proven_u8str_view_t temp = PROVEN_LIT("proven_example_durable.dat.new");
    proven_u8str_view_t here = PROVEN_LIT(".");

    /* is_absolute 는 경로를 잇거나 기준 디렉터리에 대고 풀기 전에 물어 둘 값어치가 있는
     * 물음에 답한다 - 이 경로가 이미 뿌리에서 시작하는가? 규칙은 플랫폼마다 다르다 -
     * 여기서는 앞의 '/', 윈도에서는 드라이브 문자나 UNC 접두사 - 그리고 바로 그 때문에
     * 이것은 '/' 와의 비교가 아니라 호출이다. */
    EXAMPLE_REQUIRE(!proven_fs_is_absolute(live), "the working paths in this example are relative");
    EXAMPLE_REQUIRE(proven_fs_is_absolute(PROVEN_LIT("/etc/hosts")), "a leading slash is absolute on POSIX");

    static const ex_record_t initial[] = {
        { .id = 1, .score = 10 }, { .id = 2, .score = 20 },
        { .id = 3, .score = 30 }, { .id = 4, .score = 40 },
    };

    /* --- 평범한 첫 쓰기 ------------------------------------------------- */

    proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the data file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    proven_err_t err = write_records(f.value, initial, 4);
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing the initial records must succeed");
    err = proven_fs_close(f.value);
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing after a write must succeed");
```

```
/* --- 두 벌이 서로 끼어들지 못하게 하는 권고 잠금 ---------------------- */

proven_result_file_t rw = proven_fs_open(alloc, live, PROVEN_FS_READ | PROVEN_FS_WRITE);
EXAMPLE_REQUIRE(proven_is_ok(rw.err), "reopening for update must succeed");
if (!proven_is_ok(rw.err)) return 1;

/* *배타* 잠금은 같은 것을 청하는 다른 모든 프로세스를 밖에 세운다. "권고" 는 말
 * 그대로다. 협조하는 프로그램을 막을 뿐, 잠금을 아예 청하지 않는 프로그램에는 아무
 * 영향이 없다. `wait = false` 는 막지 않고 곧바로 돌아온다 - 달리 할 일이 있을 때
 * 옳은 선택이고, 잠금을 쥔 쪽이 여러분을 기다리고 있을 수 있을 때는 유일하게 안전한
 * 선택이다. */
err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_EXCLUSIVE, false);
EXAMPLE_REQUIRE(proven_is_ok(err), "taking the exclusive lock must succeed when nobody holds it");

/* --- 레코드 하나를, 위치를 지정해 읽고 쓰기 --------------------------- */

/* pread 는 절대 위치에서 읽고 파일 위치를 옮기지 *않는다*. 그래서 손잡이 하나를
 * 나눠 쓰는 두 스레드에서도 안전하다 - 그들이 다툴 공유 커서가 없다. */
ex_record_t third = {0};
proven_mem_mut_t into = { .ptr = (proven_byte_t *)&third, .size = sizeof third };
proven_result_size_t got = proven_fs_pread(rw.value, into, 2 * sizeof(ex_record_t));
EXAMPLE_REQUIRE(proven_is_ok(got.err) && got.value == sizeof third, "reading record 2 must succeed");
EXAMPLE_REQUIRE(third.id == 3 && third.score == 30, "and yield the record that was written there");

/* tell 은 위치를 알려 준다. pread 뒤에도 그것은 움직이지 않았다. */
proven_result_u64_t pos = proven_fs_tell(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == 0, "pread must not move the file position");

/* pwrite 는 그 레코드를 제자리에서 고친다. 이번에도 커서는 건드리지 않는다. */
third.score = 99;
proven_mem_view_t out_view = { .ptr = (const proven_byte_t *)&third, .size = sizeof third };
proven_result_size_t put = proven_fs_pwrite(rw.value, out_view, 2 * sizeof(ex_record_t));
EXAMPLE_REQUIRE(proven_is_ok(put.err) && put.value == sizeof third, "writing record 2 back must succeed");

/* seek 은 커서를 옮기는 쪽이고, 도착한 위치를 돌려준다. 음수 오프셋으로 *끝*에서
 * seek 하는 것이 파일 길이를 먼저 알지 않고 마지막 레코드를 찾는 방법이다. */
proven_result_u64_t last = proven_fs_seek(rw.value, -(proven_i64)sizeof(ex_record_t), PROVEN_FS_SEEK_END);
EXAMPLE_REQUIRE(proven_is_ok(last.err), "seeking to the last record must succeed");
EXAMPLE_REQUIRE(last.val == 3 * sizeof(ex_record_t), "which is three records in");

pos = proven_fs_tell(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(pos.err) && pos.val == last.val, "tell agrees with the seek result");

/* truncate 는 길이를 곧장 정한다. 마지막 레코드를 버리는 것이 호출 하나에 O(1) 이다.
 * 옛날 방식 - 전부 읽고 남길 부분을 다시 쓰기 - 은 파일 시스템이 수 하나를 고쳐서
 * 하는 일에 O(n) 복사를 치르는 것이었다. */
err = proven_fs_truncate(rw.value, 3 * sizeof(ex_record_t));
EXAMPLE_REQUIRE(proven_is_ok(err), "truncating to three records must succeed");
proven_result_size_t size = proven_fs_size(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 3 * sizeof(ex_record_t),
                "the file is now exactly three records long");

/* 잠금을 명시적으로 푼다. 손잡이를 닫아도 풀리지만, 그렇다고 적어 두면 임계 구역이
 * 코드에 눈에 보인 채로 남는다. */
err = proven_fs_lock(rw.value, PROVEN_FS_LOCK_UNLOCK, false);
EXAMPLE_REQUIRE(proven_is_ok(err), "releasing the lock must succeed");
err = proven_fs_close(rw.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the update handle must succeed");

/* --- 견디는 바꿔치기 ------------------------------------------------- */
```

```
static const ex_record_t replacement[] = {
    { .id = 1, .score = 11 }, { .id = 2, .score = 22 },
};

/* 1. 새 내용을 옛 파일 옆에 쓴다. CREATE_NEW 는 임시 이름이 이미 있으면 거부하고,
 *    그것이 죽어 버린 실행이 남긴 찌꺼기를 조용히 재사용하지 않고 알아채는 방법이다. */
proven_result_file_t tmp = proven_fs_open(alloc, temp,
                                          PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
EXAMPLE_REQUIRE(proven_is_ok(tmp.err), "creating the temporary file must succeed");
if (!proven_is_ok(tmp.err)) return 1;

err = write_records(tmp.value, replacement, 2);
EXAMPLE_REQUIRE(proven_is_ok(err), "writing the new contents must succeed");

/* 2. 그 바이트를 저장 장치까지 끝까지 밀어 준다. 값이 비싸고, 비싸야 마땅하다.
 *    정전 뒤에도 자료가 있다는 보장을 사는 것이고, 그 값은 장치까지 실제로 다녀오는
 *    일이다. */
err = proven_fs_sync(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "syncing the new file's data must succeed");

err = proven_fs_close(tmp.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the temporary file must succeed");

/* 3. 이름을 넘긴다. 한 디렉터리 안의 rename 은 원자적이다. 읽는 쪽은 옛 파일이나 새
 *    파일을 보지, 반쯤 쓰인 것을 보지 않는다. */
err = proven_fs_rename(alloc, temp, live);
EXAMPLE_REQUIRE(proven_is_ok(err), "renaming the temporary file over the live one must succeed");

/* 4. 그 이름 바꾸기 자체를 견디게 만든다. 디렉터리가 장치에 닿기 전까지는, 새 내용이
 *    안전하지 않을 수도 있는 이름 아래에 안전하게 있는 것이다. */
err = proven_fs_sync_dir(alloc, here);
/* Windows 에는 flush 할 디렉터리 핸들이 없고, 그렇다고 말한다: 거짓 OK 가 아니라
 * UNSUPPORTED. 거기서는 rename 의 지속성을 파일 시스템이 정한다. */
EXAMPLE_REQUIRE(proven_is_ok(err) || err == PROVEN_ERR_UNSUPPORTED,
                "syncing the directory must succeed, or say it cannot");

proven_result_file_t check = proven_fs_open(alloc, live, PROVEN_FS_READ);
EXAMPLE_REQUIRE(proven_is_ok(check.err), "the live path must now open");
size = proven_fs_size(check.value);
EXAMPLE_REQUIRE(proven_is_ok(size.err) && size.value == 2 * sizeof(ex_record_t),
                "and hold exactly the replacement records");
err = proven_fs_close(check.value);
EXAMPLE_REQUIRE(proven_is_ok(err), "closing the verification handle must succeed");

/* --- 복사, 그리고 두 가지 링크 ---------------------------------------- */

/* copy 는 바이트를 복제한다. 이제부터는 서로 독립인 파일 둘이다. 할당자는 복사가
 * 자료를 옮겨 가는 임시 버퍼를 위한 것이다. */
proven_u8str_view_t backup = PROVEN_LIT("proven_example_durable.bak");
err = proven_fs_copy(alloc, live, backup);
EXAMPLE_REQUIRE(proven_is_ok(err), "copying the file must succeed");

/* *하드* 링크는 같은 파일의 두 번째 이름이다. 원본이라는 것이 없다. 자료는 마지막
 * 이름이 지워질 때까지 산다. 두 이름은 같은 파일 시스템에 있어야 한다. 이름과 그
 * 자료가 둘에 걸칠 수는 없기 때문이다. */
proven_u8str_view_t hard = PROVEN_LIT("proven_example_durable.hard");
err = proven_fs_link(alloc, live, hard);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a hard link must succeed");
```

```
proven_fs_stat_t st_live = {0}, st_hard = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, live, &st_live)), "stat of the live name");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, hard, &st_hard)), "stat of the hard link");
EXAMPLE_REQUIRE(st_live.ino == st_hard.ino && st_live.dev == st_hard.dev,
                "both names refer to the same file, which is what a hard link means");

/* *심볼릭* 링크는 경로를 담은 작은 파일이다. 다른 파일 시스템의 무언가를 가리킬 수도
 * 있고, 아무것도 아닌 것을 가리킬 수도 있다 - 그때 따라가기는 실패하는데, 하드 링크는
 * 결코 그럴 수 없다. */
proven_u8str_view_t soft = PROVEN_LIT("proven_example_durable.link");
err = proven_fs_symlink(alloc, live, soft);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a symbolic link must succeed");

proven_fs_stat_t st_soft = {0};
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_stat(alloc, soft, &st_soft)),
                "stat follows the symbolic link to its target");
EXAMPLE_REQUIRE(st_soft.size == st_live.size, "so it reports the target's size");

/* --- 디렉터리, 그리고 뒷정리 ---------------------------------------- */

proven_u8str_view_t dir = PROVEN_LIT("proven_example_durable_dir");
err = proven_fs_mkdir(alloc, dir);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating a directory must succeed");

/* 이미 있는 디렉터리를 만들라는 요청은 거절되고, 그 거절이 이유를 말한다:
 * PROVEN_ERR_EXISTS 다. 부모가 없으면 PROVEN_ERR_NOT_FOUND 다. 부르는 쪽은 디스크를
 * 다시 들여다보지 않고도 "할 일이 없다" 와 "무언가 잘못됐다" 를 가를 수 있다. */
err = proven_fs_mkdir(alloc, dir);
EXAMPLE_REQUIRE(err == PROVEN_ERR_EXISTS, "a second mkdir of the same name must say it exists");

/* proven_fs_mkdir_all 은 경로에서 빠진 단계를 모두 만들고, 이미 디렉터리인 단계는
 * 오류가 아니다. 그래서 트리 안에 쓰기 전에 그냥 부르면 된다. 길을 막은 *파일* 은
 * 여전히 PROVEN_ERR_EXISTS 다. */
proven_u8str_view_t deep = PROVEN_LIT("proven_example_durable_dir/a/b");
err = proven_fs_mkdir_all(alloc, deep);
EXAMPLE_REQUIRE(proven_is_ok(err), "creating two missing levels at once must succeed");
err = proven_fs_mkdir_all(alloc, deep);
EXAMPLE_REQUIRE(proven_is_ok(err), "and asking again must succeed too");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_rmdir(alloc, deep)), "removing the inner level must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_rmdir(alloc, PROVEN_LIT("proven_example_durable_dir/a"))),
                "removing the level above it must succeed");

/* rmdir 은 *빈* 디렉터리만 지운다. 그 거부가 기능이다. 재귀 삭제는 부르는 쪽이
 * 명시적으로 내려야 하는 결정이지, 잘못 들어온 경로 인자 하나가 일으킬 수 있는 일이
 * 아니다. 이 거부에는 자기 코드가 있다. PROVEN_ERR_INVALID_STATE 다. 실패한 것은 없고,
 * 디렉터리를 먼저 비워야 한다는 뜻이다. */
proven_u8str_view_t inside = PROVEN_LIT("proven_example_durable_dir/file.txt");
proven_result_file_t child = proven_fs_open(alloc, inside, PROVEN_FS_WRITE | PROVEN_FS_CREATE);
EXAMPLE_REQUIRE(proven_is_ok(child.err), "creating a file inside it must succeed");
EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(child.value)), "closing it must succeed");

err = proven_fs_rmdir(alloc, dir);
EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_STATE, "removing a non-empty directory must be refused");

EXAMPLE_REQUIRE(proven_is_ok(proven_fs_remove(alloc, inside)), "removing the file must succeed");
err = proven_fs_rmdir(alloc, dir);
EXAMPLE_REQUIRE(proven_is_ok(err), "and then the empty directory can be removed");

printf("durable replace complete: %zu byte(s) live\n", (size_t)size.value);
```

```
    (void)proven_fs_remove(alloc, soft);
    (void)proven_fs_remove(alloc, hard);
    (void)proven_fs_remove(alloc, backup);
    (void)proven_fs_remove(alloc, live);
    return EXAMPLE_OK();
}
```

반례 — 이 절차가 대신하려는 제자리 덮어쓰기:

```
proven_result_file_t f = proven_fs_open(alloc, live, PROVEN_FS_WRITE | PROVEN_FS_TRUNC);
proven_err_t e = proven_fs_write_all(f.value, new_contents);   /* wrong */
```

자르기와 마지막 바이트 쓰기 사이에서 디스크 위의 파일은 불완전하다. 그 사이의 크래시는 갱신을 잃는 것이 아니라 이미 있던 데이터를 잃는다.

반례 — sync 없이 이름부터 바꾸는 경우:

```
proven_err_t e = proven_fs_close(tmp.value);        /* no proven_fs_sync */
e = proven_fs_rename(alloc, temp, live);            /* wrong */
```

파일을 닫는 것은 그 바이트를 장치에 올려놓는 일이 아니다. 이름 바꾸기는 지속되는데 그것이 가리키는 내용은 그렇지 않을 수 있다.

반례 — 잠금이 모두를 막는다고 가정하는 경우:

```
proven_err_t e = proven_fs_lock(f.value, PROVEN_FS_LOCK_EXCLUSIVE, true);
/* another program writes the file without ever calling proven_fs_lock */
```

권고 잠금은 그것을 모두가 쓰기로 한 프로그램들 사이의 약속이다. 접근 권한 통제가 아니다.

실전 예제: reader와 writer, 그리고 표준 스트림

writer는 "바이트가 가는 곳", **reader**는 "바이트가 오는 곳"이다. 각각은 포인터 둘이다. 작은 함수 표 하나와, 그 함수들이 다루는 상태 하나. 이 구조의 값어치는 전부 여기서 나온다. writer를 상대로 쓴 코드는 그 바이트가 파일로 가는지, 문자열로 가는지, 터미널로 가는지, 시험용 버퍼로 가는지 알지 못하고 — reader를 상대로 쓴 코드는 디스크의 파일 대신 메모리의 문자열을 상대로 시험할 수 있다.

예제는 그 조각들을 한데 놓는다.

- `proven_reader_from_view()`는 이미 손에 있는 바이트 위에 reader를 만든다. 파서를 파일 없이 시험하는 방법이다. `proven_reader_read()`는 버퍼 크기까지 채우고 실제로 얻은 양을 알려 준다. 짧은 읽기는 정상이며, 입력의 끝은 0바이트 성공이 아니라 `PROVEN_ERR_EOF`다.
- `proven_sysio_stdout_writer()`와 `proven_sysio_stderr_writer()`는 버퍼링 없는 표준 스트림이고, `proven_sysio_stdin_reader()`는 표준 입력이다. `proven_writer_write()`는 "전부, 아니면 에러"를 뜻하고, `proven_writer_write_partial()`은 옮길 수 있는 만큼 옮기고 그 개수를 알려 준다 — 재시도나 배압(back-pressure)을 직접 다루는 호출자를 위한 것이다.
- `proven_reader_is_valid()`와 `proven_writer_is_valid()`는 만들어진 적 없는 핸들을 첫 읽기·쓰기가 아니라 경계에서 잡는다.
- `proven_sysio_file_buffered()`는 열린 파일을, **여러분이** 준 버퍼 위의 버퍼드 writer로 감싼다. 버퍼링의 메모리 비용이 여러분이 고른 숫자로 남는다는 뜻이다. 반드시 flush해야 한다. 여기서는 아무것도 종료 시에 대신 flush해 주지 않는다.
- `proven_sysio_lines_open()`은 같은 방식의 호출자 제공 버퍼로 그 파일을 한 줄씩 읽는다. 예상되는 가장 긴 **줄**에 맞춰 크기를 잡는다. 더 긴 줄은 조용히 잘리는 대신 `PROVEN_ERR_OUT_OF_BOUNDS`가 된다.
- `proven_scan_fmt_from_file()`(내부적으로 `proven_sysio_scan_chunk_impl()`을 부른다)은 입력이 자유 서식이 아니라 정해진 모양일 때 파일 핸들에서 타입 있는 값을 바로 뽑는다.

```
/*
 * 쓰는 쪽은 "바이트가 가는 어딘가" 이고 읽는 쪽은 "바이트가 오는 어딘가" 이며, 각각
 * 포인터 둘이다. 작은 함수 표 하나와 그 함수들이 다룰 상태 하나. 그것이 전부다. 이
 * 얼개의 값어치는, 쓰는 쪽에 기대어 쓴 코드가 그 바이트가 파일로 가는지 문자열로 가는지
 * 터미널로 가는지 시험용 버퍼로 가는지 알지도 못하고 신경 쓰지도 않는다는 것이다 -
 * 그리고 읽는 쪽에 기대어 쓴 코드는 디스크의 파일 대신 기억 속 문자열에 대고 시험할 수
```

```
 * 있다.
 *
 * 이 프로그램은 둘 다, 그리고 거기 매달린 표준 스트림과 파일 배관을 보인다.
 *
 *   - 뷰(기억 속 문자열) 위의 읽는 쪽. 파일 시스템을 건드리지 않고 파서를 시험하는
 *     방법이다,
 *   - 버퍼 없는 stdout 과 stderr 쓰는 쪽, 그리고 "이걸 다 써라" 와 "쓸 수 있는 만큼
 *     써라" 의 차이,
 *   - 파일 위의 버퍼 둔 쓰는 쪽. 작은 쓰기 여럿을 큰 쓰기 몇으로 바꿔 준다,
 *   - 그 같은 파일 위의 줄 단위 읽는 쪽,
 *   - 파일 손잡이에서 곧장 형식화된 값 읽기.
 *
 * 사람들이 반나절을 잃게 만드는 주의 하나: 아래의 상태 구조체들은 선언된 자리에 그대로
 * 있어야 한다. 쓰는 쪽은 자기 상태 구조체 *안*을 가리키는 포인터를 쥐므로, 구조체를
 * 복사하면 사본은 죽은 것이 되고 주소는 원본을 가리킨 채로 남는다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* --- 1. 이미 가진 바이트 위의 읽는 쪽 --------------------------------- */

    /* 아래 파서는 이것이 문자열이라는 것을 모른다. 파일이든 파이프든 소켓이든 똑같이
     * 돌 것이다 - 그래서 시험이 값싼 쪽을 쓸 수 있다. */
    proven_reader_view_t src_state;
    proven_reader_t src = proven_reader_from_view(&src_state, PROVEN_LIT("id=41\nid=42\n"));

    /* is_valid 는 그 읽는 쪽이 실제로 만들어졌는지 묻는다 - 0 으로 초기화된 손잡이는
     * 읽는 쪽이 아니고, 그것을 첫 읽기가 아니라 경계에서 말해 주는 검사가 이것이다. */
    EXAMPLE_REQUIRE(proven_reader_is_valid(src), "a reader made from a view must be usable");
    proven_reader_t never_made = {0};
    EXAMPLE_REQUIRE(!proven_reader_is_valid(never_made), "a zero-initialised reader handle is not");

    /* read 는 dest.size 바이트까지 채우고 실제로 얼마나 받았는지 알려 준다. 짧은 읽기는
     * 정상이다 - 파일의 끝이 아니고, 그것을 끝으로 여기는 것이 입력의 꼬리를 잃는
     * 고전적인 방법이다. 파일의 끝에는 제 코드가 따로 있다, PROVEN_ERR_EOF. */
    proven_byte_t chunk[5];
    proven_mem_mut_t into = { .ptr = chunk, .size = sizeof chunk };
    proven_result_size_t got = proven_reader_read(src, into);
    EXAMPLE_REQUIRE(proven_is_ok(got.err), "the first read must succeed");
    EXAMPLE_REQUIRE(got.value == 5, "and fill the buffer from the view");

    proven_size_t total = got.value;
    for (;;) {
        got = proven_reader_read(src, into);
        if (got.err == PROVEN_ERR_EOF) {
            break;
        }
        EXAMPLE_REQUIRE(proven_is_ok(got.err), "reads before end of file must succeed");
        total += got.value;
    }
    EXAMPLE_REQUIRE(total == 12, "the loop must consume the whole view, however it was chunked");

    /* --- 2. 표준 스트림 ---------------------------------------------------- */

    /* 상태 구조체가 쓰는 쪽이 가리키는 손잡이를 쥐고 있으므로, 쓰는 쪽보다 오래 살아야
     * 한다. 둘을 나란히 선언하는 습관이 그것을 지켜 준다. */
    proven_sysio_std_t out_state;
    proven_writer_t out = proven_sysio_stdout_writer(&out_state);
    EXAMPLE_REQUIRE(proven_writer_is_valid(out), "the stdout writer must be usable");
```

```
    /* write 는 "전부, 아니면 오류" 라는 뜻이다. 안에서 반복한다. 시스템 수준의 쓰기 한
     * 번은 청한 것보다 적은 바이트를 옮길 수 있기 때문이다. */
    proven_err_t err = proven_writer_write(out, proven_mem_view_from_u8(PROVEN_LIT("stream example:
start\n")));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing a whole line to stdout must succeed");

    /* write_partial 은 정직한 저수준 쌍둥이다. 옮길 수 있는 만큼 옮기고 그 개수를 알려
     * 준다. 재시도나 역압을 여러분이 다룰 때 쓰고, 그냥 바이트를 내보내고 싶으면 write 를
     * 쓸 것. */
    proven_result_size_t part = proven_writer_write_partial(out, proven_mem_view_from_u8(PROVEN_LIT("partial
write\n")));
    EXAMPLE_REQUIRE(proven_is_ok(part.err), "a partial write to a terminal or pipe must succeed");
    EXAMPLE_REQUIRE(part.value > 0, "and report how many bytes it moved");

    /* stderr 에 버퍼를 두지 않은 것은 일부러다. 진단은 다음 줄의 코드가 돌기 전에 나가야
     * 하고, 그 다음 줄이 바로 죽는 줄일 때 그것이 필요하다. */
    proven_sysio_std_t err_state;
    proven_writer_t diag = proven_sysio_stderr_writer(&err_state);
    EXAMPLE_REQUIRE(proven_writer_is_valid(diag), "the stderr writer must be usable");
    err = proven_writer_write(diag, proven_mem_view_from_u8(PROVEN_LIT("stream example: diagnostics go
here\n")));
    EXAMPLE_REQUIRE(proven_is_ok(err), "writing to stderr must succeed");

    /* stdin 위의 읽는 쪽도 같은 방식으로 만든다. 이 예제는 거기서 읽지 않는다 - 시험
     * 실행에는 타이핑하는 사람이 없다 - 다만 만드는 모양을 보이고, 필터 프로그램이 돌
     * 손잡이가 이것이다. */
    proven_sysio_std_t in_state;
    proven_reader_t stdin_reader = proven_sysio_stdin_reader(&in_state);
    EXAMPLE_REQUIRE(proven_reader_is_valid(stdin_reader), "the stdin reader must be usable");

    /* --- 3. 파일로 나가는 버퍼 둔 출력 ---------------------------------- */

    proven_u8str_view_t path = PROVEN_LIT("proven_example_streams.txt");
    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_WRITE | PROVEN_FS_CREATE |
PROVEN_FS_TRUNC);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "creating the output file must succeed");
    if (!proven_is_ok(f.err)) return 1;

    /* 버퍼는 여러분의 것이다. 라이브러리가 등 뒤에서 하나 할당하지 않으므로, 버퍼링의
     * 기억 비용은 여러분이 고른, 눈에 보이는 수다. 256바이트 버퍼를 지나는 예순 줄은
     * 예순 번이 아니라 몇 번의 쓰기다. */
    proven_byte_t outbuf[256];
    proven_sysio_out_t file_out;
    proven_writer_t w = proven_sysio_file_buffered(&file_out, f.value,
                                                   (proven_mem_mut_t){ .ptr = outbuf, .size = sizeof
outbuf });
    EXAMPLE_REQUIRE(proven_writer_is_valid(w), "the buffered file writer must be usable");

    for (int i = 0; i < 20; ++i) {
        proven_fmt_result_t line_out = proven_fprintln(w, "reading {} = {}",
                                                       proven_arg_i32(i), proven_arg_i32(i * i));
        EXAMPLE_REQUIRE(proven_is_ok(line_out.err), "writing a formatted line must succeed");
    }

    /* 흘려 보내지 않은 버퍼 출력은 일어나지 않은 출력이다. 여기 어느 것도 여러분 대신
     * 종료 시점에 흘려 주지 않는다. */
    err = proven_writer_flush(w);
    EXAMPLE_REQUIRE(proven_is_ok(err), "the flush is what actually writes the file");
    err = proven_fs_close(f.value);
```

```
    EXAMPLE_REQUIRE(proven_is_ok(err), "closing the file must succeed");

    /* --- 4. 한 줄씩 되읽기 ------------------------------------------------ */

    proven_result_file_t rf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(rf.err), "reopening for reading must succeed");
    if (!proven_is_ok(rf.err)) return 1;

    /* 버퍼는 예상되는 가장 긴 *줄*에 맞춰 잡을 것. 더 긴 줄은 OUT_OF_BOUNDS 로 보고된다 -
     * 조용히 반으로 잘리는 일은 없다. 그 자르기가 나쁜 레코드 하나를 그럴듯한 레코드
     * 둘로 만드는 실패다. */
    proven_byte_t linebuf[128];
    proven_sysio_lines_t lines;
    err = proven_sysio_lines_open(&lines, rf.value, (proven_mem_mut_t){ .ptr = linebuf, .size = sizeof
linebuf });
    EXAMPLE_REQUIRE(proven_is_ok(err), "opening a line reader over the file must succeed");

    proven_size_t count = 0;
    for (;;) {
        proven_result_u8str_view_t line = proven_sysio_read_line(&lines);
        if (line.err == PROVEN_ERR_EOF) {
            break;
        }
        EXAMPLE_REQUIRE(proven_is_ok(line.err), "reading a line must succeed until end of file");
        /* 그 뷰는 linebuf 안을 가리키고 다음 호출 전까지만 쓸 수 있다. 그래서 백만 줄이
         * 백만 번의 할당이 아니라 버퍼 하나만큼의 값이 든다 - 그리고 남겨 둘 줄은 복사해야
         * 하는 이유이기도 하다. */
        if (count == 0) {
            EXAMPLE_REQUIRE(proven_u8str_view_eq(line.val, PROVEN_LIT("reading 0 = 0")),
                            "the first line reads back exactly as it was written");
        }
        ++count;
    }
    EXAMPLE_REQUIRE(count == 20, "every line written must be read back");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(rf.value)), "closing the read handle must succeed");

    /* --- 5. 파일에서 줄이 아니라 값을 읽기 -------------------------------- */

    /* 입력이 자유로운 글이 아니라 정해진 모양일 때는, 곧장 파싱하는 편이 자르고 변환하는
     * 반복문을 손으로 쓰는 수고를 던다. 이것은 파일 손잡이에서 한 토막을 읽어 그 안에서
     * 수 둘을 뽑아낸다. */
    proven_result_file_t sf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(sf.err), "opening the file for scanning must succeed");

    proven_i32 index = -1, square = -1;
    err = proven_scan_fmt_from_file(sf.value, "reading {} = {}",
                                    proven_scan_arg_i32(&index), proven_scan_arg_i32(&square));
    EXAMPLE_REQUIRE(proven_is_ok(err), "scanning the first record must succeed");
    EXAMPLE_REQUIRE(index == 0 && square == 0, "and produce the values that were written");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(sf.value)), "closing the scan handle must succeed");

    printf("streams: %zu byte(s) read from the view, %zu line(s) round-tripped\n",
           (size_t)total, (size_t)count);

    (void)proven_fs_remove(alloc, path);
    return EXAMPLE_OK();
}
```

반례 — writer를 만든 뒤 상태 구조체를 복사하는 경우:

```
proven_sysio_out_t state;
proven_writer_t w = proven_sysio_file_buffered(&state, file, buf);
proven_sysio_out_t moved = state;    /* wrong: w still points into `state` */
```

writer는 그 구조체 **안쪽**을 가리키는 포인터를 들고 있다. writer가 사는 동안 상태는 선언한 자리에 둔다.

반례 — flush를 잊는 경우:

```
proven_fmt_result_t r = proven_fprintln(w, "done");
return 0;                            /* wrong: the buffer never reached the file */
```

실전 예제: 파일을 메모리에 매핑하고, 그 변경을 지속시키기
메모리 매핑된 파일은 프로세서가 메모리처럼 읽는 파일이다. 운영체제가 그 페이지들을 어떤 주소에 나타나게 해 주고, 읽기는 read 호출 없이 일어난다. 큰 파일에 무작위로 접근할 때, 특히 여러 프로세스가 함께 볼 때 어울린다. 스트리밍에는 어울리지 않는다. 거기서는 버퍼드 reader가 더 단순하고, 파일 크기를 주소 공간에 묶지도 않는다.

쓰기가 살아남는지를 정하는 구분은 이것이다.

매핑	쓰기가 가는 곳	proven_mmap_sync()
PROVEN_MMAP_SHARED	파일로 간다	저장 장치까지 밀어 넣는다
PROVEN_MMAP_PRIVATE	사적인 사본으로 간다(기록 시 복사, copy-on-write) — 이 프로세스에만	되돌려 쓸 것이 없으므로 PROVEN_ERR_UNSUPPORTED를 돌려준다

그 거절은 일부러 그런 것이다. 공유 매핑이라고 믿었던 호출자가, 데이터가 없어진 뒤가 아니라 sync 자리에서 그 사실을 알게 된다.

예제는 달력 형식화로 끝난다. 기록하는 레코드에 날짜가 붙기 때문이다. UTF-8 형태는 proven_time_u8_fmt(), UTF-16 형태는 proven_time_u16_fmt()이며, 후자가 옳은 상황은 딱 하나 — 와이드 문자열을 받는 시스템 호출에 그 텍스트를 바로 넘길 때다.

```
#include <string.h>

/*
 * 기억에 사상된 파일은 프로세서가 기억처럼 읽는 파일이다. 운영체제가 그 쪽들을 어떤
 * 주소에 나타나게 해 주고, 읽기는 read 호출 없이 일어난다. 한 모양의 문제에는 옳은
 * 도구다 - 여러 프로세스가 함께 보는 큰 파일에 무작위로 접근하는 일 - 그리고 흘려 읽기에는
 * 틀린 도구다. 거기서는 버퍼를 둔 읽기 쪽이 더 단순하고, 파일 크기를 여러분의 주소 공간에
 * 묶지도 않는다.
 *
 * 틀리기 쉬운 자리는 지속성이고, 이 예제가 있는 이유가 그것이다.
 *
 *   PROVEN_MMAP_SHARED  - 쓰기가 파일로 간다. proven_mmap_sync 가 그것을 저장 장치까지
 *                         밀어 준다.
 *   PROVEN_MMAP_PRIVATE - 쓰기가 복사-후-쓰기다. 이 프로세스 안에만 있고 다른 어디에도
 *                         없다. 되쓸 것이 없으므로, sync 를 청하면 조용히 아무 일도 하지
 *                         않는 대신 그렇다고 말한다.
 *
 * 이 예제는 달력 형식화기도 나란히 보인다. 여기서 쓰는 레코드가 시각을 싣기 때문이고,
 * 윈도 API 를 위해 쓰는 시각이야말로 UTF-16 형식화기가 옳은 호출인 유일한 자리다.
 */

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();
    proven_u8str_view_t path = PROVEN_LIT("proven_example_mmap.dat");

    /* 사상은 파일을 늘릴 수 없다. 그러니 파일은 사상하기 전에 사상하려는 크기여야 한다. */
    static const char initial[] = "record 0: pending    \n";
    proven_result_file_t create = proven_fs_open(alloc, path,
                                                 PROVEN_FS_WRITE | PROVEN_FS_CREATE | PROVEN_FS_TRUNC);
```

```
    EXAMPLE_REQUIRE(proven_is_ok(create.err), "creating the backing file must succeed");
    if (!proven_is_ok(create.err)) return 1;

    proven_mem_view_t seed = { .ptr = (const proven_byte_t *)initial, .size = sizeof initial - 1 };
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_write_all(create.value, seed)), "writing the record must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(create.value)), "closing it must succeed");

    /* --- 공유 사상: 쓰기가 파일에 닿는다 ---------------------------------- */

    proven_result_file_t f = proven_fs_open(alloc, path, PROVEN_FS_READ | PROVEN_FS_WRITE);
    EXAMPLE_REQUIRE(proven_is_ok(f.err), "opening the file for mapping must succeed");
    if (!proven_is_ok(f.err)) return 1;

    /* size 0 은 "파일 끝까지" 라는 뜻이다. */
    proven_result_mmap_t m = proven_mmap_create(f.value, 0, 0,
                                                PROVEN_MMAP_READ | PROVEN_MMAP_WRITE,
                                                PROVEN_MMAP_SHARED);
    EXAMPLE_REQUIRE(proven_is_ok(m.err), "mapping the file must succeed");
    if (!proven_is_ok(m.err)) {
        (void)proven_fs_close(f.value);
        return 1;
    }
    proven_mmap_t map = m.value;

    /* 읽기는 그냥 기억을 읽는 것이다 - 호출도 복사도 없다. as_view 는 사상 전체를 바이트
     * 뷰로 돌려주므로, 보통의 뷰 도우미들이 그대로 먹는다. */
    proven_u8str_view_t contents = proven_mmap_as_view(map);
    EXAMPLE_REQUIRE(contents.size == sizeof initial - 1, "the mapping covers the whole file");
    EXAMPLE_REQUIRE(proven_u8str_view_starts_with(contents, PROVEN_LIT("record 0:")),
                    "and shows the bytes that were written");

    /* 쓰기는 기억에 쓰는 것이다. 상태 필드를 일부러 폭 고정으로 두었다. 사상은 파일을
     * 길게 만들 수 없으므로, 제자리 편집은 이미 있는 자리에 들어가야 한다. */
    proven_size_t at = proven_u8str_view_find(contents, 0, PROVEN_LIT("pending"));
    EXAMPLE_REQUIRE(at != PROVEN_SIZE_MAX, "the status field must be found");
    memcpy((proven_byte_t *)map.ptr + at, "done   ", 7);

    /* sync 가 지속성의 걸음이다. 그것 없이는 바뀐 내용이 페이지 캐시에 있고, 거기서는 이
     * 프로그램이 끝나는 것은 견디지만 기계가 전원을 잃는 것은 견디지 못한다. */
    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_sync(&map)), "syncing a shared mapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&map)), "unmapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(f.value)), "closing the mapped file must succeed");

    /* 편집이 이 프로세스의 기억에만이 아니라 파일에 들어가 있다. */
    proven_result_u8str_t back = proven_fs_read_all_u8str(alloc, path);
    EXAMPLE_REQUIRE(proven_is_ok(back.err), "reading the file back must succeed");
    EXAMPLE_REQUIRE(proven_u8str_view_find(proven_u8str_as_view(&back.value), 0, PROVEN_LIT("done")) !=
PROVEN_SIZE_MAX,
                    "the mapped write reached the file");
    proven_u8str_destroy(alloc, &back.value);

    /* --- 사적 사상: 쓰기가 아무 데도 가지 않는다 -------------------------- */

    proven_result_file_t pf = proven_fs_open(alloc, path, PROVEN_FS_READ);
    EXAMPLE_REQUIRE(proven_is_ok(pf.err), "reopening for a private mapping must succeed");

    proven_result_mmap_t pm = proven_mmap_create(pf.value, 0, 0, PROVEN_MMAP_READ, PROVEN_MMAP_PRIVATE);
    EXAMPLE_REQUIRE(proven_is_ok(pm.err), "a private read mapping must succeed");
    proven_mmap_t priv = pm.value;
```

```
    /* 사적 사상에 sync 를 청하면 받아 놓고 무시하는 대신 거부된다. 그 거부가 쓸모 있는
     * 동작이다. 사상이 공유라고 믿고 있던 쪽이 자료가 사라진 뒤가 아니라 여기서 알게 된다. */
    EXAMPLE_REQUIRE(proven_mmap_sync(&priv) == PROVEN_ERR_UNSUPPORTED,
                    "a private mapping has nothing to write back, and says so");

    EXAMPLE_REQUIRE(proven_is_ok(proven_mmap_destroy(&priv)), "unmapping the private mapping must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_fs_close(pf.value)), "closing it must succeed");

    /* --- 시각을, 두 문자열 타입 모두로 ----------------------------------- */

    proven_datetime_t now = proven_time_now_datetime();

    proven_result_u8str_t stamp = proven_u8str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(stamp.err), "creating the timestamp string must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_time_u8_fmt(alloc, &stamp.value, now, &proven_time_locale_en,
                                                    "{year}-{month:0>2}-{day:0>2}")),
                    "formatting the date as UTF-8 must succeed");
    EXAMPLE_REQUIRE(proven_u8str_as_view(&stamp.value).size == 10, "a YYYY-MM-DD date is ten characters");

    /* 같은 호출의 UTF-16 꼴이다. 그것이 옳은 유일한 자리를 위한 것 - 와이드 문자열을 받는
     * 시스템 호출에 글을 곧장 건네는 자리다. */
    proven_result_u16str_t wide = proven_u16str_create(alloc, 64);
    EXAMPLE_REQUIRE(proven_is_ok(wide.err), "creating the wide timestamp string must succeed");
    EXAMPLE_REQUIRE(proven_is_ok(proven_time_u16_fmt(alloc, &wide.value, now, &proven_time_locale_en,
                                                     "{year}-{month:0>2}-{day:0>2}")),
                    "formatting the same date as UTF-16 must succeed");
    EXAMPLE_REQUIRE(proven_u16str_len(&wide.value) == 10, "ten code units, one per character here");
    EXAMPLE_REQUIRE(proven_u16str_as_ptr(&wide.value)[4] == (proven_u16)'-',
                    "and the same layout as the UTF-8 form");

    printf("mapped record updated; stamped %s\n", proven_u8str_as_cstr(&stamp.value));

    proven_u16str_destroy(alloc, &wide.value);
    proven_u8str_destroy(alloc, &stamp.value);
    (void)proven_fs_remove(alloc, path);
    return EXAMPLE_OK();
}
```

반례 — 매핑이 파일을 늘려 줄 것이라고 기대하는 경우:

```
proven_result_mmap_t m = proven_mmap_create(f.value, 0, 1 << 20, ...);  /* file is 22 bytes */
memcpy((char *)m.value.ptr + 4096, data, n);                            /* wrong */
```

파일 길이를 먼저 정하고 — `proven_fs_truncate()`가 한 번에 해 준다 — 존재하는 만큼을 매핑한다.

실전 예제: 난수는 어디에서 오는가

위의 예제는 어떤 생성기를 고를지를 다룬다. 이것은 그 아래 계층, 곧 바이트의 출처와, 어느 출처를 받았는지 신경 쓰지 않는 코드를 쓰는 법이다.

- **`proven_rng_t`가 그 인터페이스다.** 이것을 받는 함수는 운영체제 생성기로도, ChaCha20으로도, xoshiro로도, 시험용으로 여러분이 만든 가짜로도 고치지 않고 동작한다. `proven_rng_is_valid()`는 출처가 실제로 만들어졌는지 말해 준다. 만들어지지 않은 것에서 뽑으면 숫자를 지어내는 대신 0을 돌려주므로, 뽑을 때마다 믿는 대신 경계에서 한 번 확인한다. `proven_rng_u64()`는 64비트 낱말 하나를, `proven_rng_fill()`은 버퍼 전체를 한 번에 채운다.
- **고정된 씨앗(seed)은 시험을 재현 가능하게 만든다.** `proven_chacha_rng_seed()`는 씨앗 바이트를 직접 받으므로 `proven_chacha_rng_next()`는 알려진 수열을 걷는다. "일주일에 한 번 실패한다"를 다시 돌려볼 수 있는 실패로 바꾸는 것이 바로 이것이다. 운영에서는 씨앗이 진짜 엔트로피에서 와야 한다. 그것을 아는 사람은 이후의 모든 바이트를 알기 때문이다.

- `proven_random_u64()`는 엔트로피 출처에서 강한 낱말 하나를 바로 뽑는다. 시동 시의 해시 키나 식별자 같은 일회성에는 알맞고, 반복문 안에서는 그르다. 호출마다 운영체제까지 다녀오기 때문이다. 출처가 실패하면 0을 돌려주므로, 토큰이나 키에는 실패를 알려 주는 `proven_random_u64_checked()`를 쓴다.
- `proven_random_set_source()`는 엔트로피 출처 자체를 설치한다. 호스티드(hosted) 프로그램에는 운영체제의 것이 이미 있으니 그대로 두면 되고, 베어메탈 프로그램에는 없으니 그 보드의 하드웨어 출처가 여기로 들어간다.

```
#include <string.h>

/*
 * 다른 예제는 어느 생성기를 고를지를 보인다. 이것은 그 밑의 층에 대한 것이다. 난수가
 * *어디서* 오는가, 그리고 그것을 신경 쓰지 않는 코드를 어떻게 쓰는가.
 *
 *   proven_rng_t 는 난수 바이트의 원천을 포인터 한 쌍으로 나타낸 것이다 - 작은 함수
 *   표와 그 함수들이 다룰 생성기 상태. proven_rng_t 를 받는 코드는 OS 생성기에서도,
 *   ChaCha20 에서도, xoshiro 에서도, 시험용으로 여러분이 지어낸 가짜에서도 한 줄도
 *   고치지 않고 돈다.
 *
 *   proven_random_set_source 는 *그보다* 아래 층이다. 생성기가 씨를 받는 날 엔트로피가
 *   어디서 오는가. 호스트가 있는 프로그램에는 이미 하나가 있고 - 운영체제의 것 - 그대로
 *   두어야 한다. 베어메탈 프로그램에는 없고, 그 기계의 하드웨어 원천을 다는 자리가
 *   이 고리다.
 *
 * 고정된 씨앗 부분은 보기보다 중요하다. *알려진* 씨앗으로 씨를 뿌린 암호용 생성기는
 * 알려진 수열을 내놓고, 그것이 난수가 끼는 시험을 "일주일에 한 번 실패" 가 아니라
 * 재현 가능한 것으로 만든다.
 */

/* 전혀 무작위가 아닌 "엔트로피" 원천이다. 그냥 센다. 이런 것이 실제 프로그램에 들어갈
 * 자리는 없고 - 장의 반례를 볼 것 - 다만 이 고리가 어떻게 도는지 보이기에, 그리고 돌
 * 때마다 같은 바이트를 내야 하는 시험에 딱 맞는 모양이다. */
static bool counting_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_u8 *next = (proven_u8 *)ctx;
    proven_u8 *out = (proven_u8 *)buf;
    for (proven_size_t i = 0; i < len; ++i) {
        out[i] = (*next)++;
    }
    return true;
}

/* 특성에 기대어 쓴 함수. 자기가 어느 생성기를 받았는지 끝내 알지 못한다. */
static proven_u64 roll_total(proven_rng_t rng, int rolls) {
    proven_u64 sum = 0;
    for (int i = 0; i < rolls; ++i) {
        sum += proven_rng_below(rng, 6) + 1;
    }
    return sum;
}

int main(void) {
    /* --- 1. 쓰기 전에 확인할 수 있는 원천 ---------------------------------- */

    proven_rng_t nothing = {0};
    EXAMPLE_REQUIRE(!proven_rng_is_valid(nothing), "a zero-initialised source is not a generator");

    /* 잘못된 원천에서 뽑아도 죽지 않고 수를 지어내지도 않는다. 0 을 돌려준다. 정의된,
     * 심심한 답이다 - 그런데 0 의 연속은 난수가 아니므로, 뽑을 때마다 믿는 대신 받을 때
     * 한 번 원천을 확인할 것. */
    EXAMPLE_REQUIRE(proven_rng_u64(nothing) == 0, "an invalid source yields 0, not a fabricated value");
```

```
/* --- 2. *알려진* 씨앗에서 나온 암호용 생성기 --------------------------- */

/* proven_chacha_rng_seed 는 씨앗 바이트를 곧장 받으므로 수열이 재현된다. 시험에서는
 * 그것이 원하는 바이고 운영에서는 결코 아니다. 씨앗을 알아낸 사람은 그 생성기가
 * 내놓을 모든 바이트를 안다. */
proven_byte_t seed[PROVEN_CHACHA_SEED_SIZE];
memset(seed, 0xA5, sizeof seed);

proven_chacha_rng_t a, b;
proven_chacha_rng_seed(&a, seed);
proven_chacha_rng_seed(&b, seed);

/* next 는 한 번에 64비트 낱말 하나를 돌려준다. 같은 씨앗을 받은 생성기 둘은 같은
 * 수열을 걷는다 - 시험이 기대는 성질이 그것이다. */
proven_u64 first = proven_chacha_rng_next(&a);
EXAMPLE_REQUIRE(first == proven_chacha_rng_next(&b), "the same seed replays the same sequence");
EXAMPLE_REQUIRE(proven_chacha_rng_next(&a) == proven_chacha_rng_next(&b), "and keeps replaying it");

/* --- 3. 특성을 통해 쓰기 ------------------------------------------------ */

proven_rng_t rng = proven_chacha_rng(&a);
EXAMPLE_REQUIRE(proven_rng_is_valid(rng), "a seeded generator makes a valid source");

proven_u64 word = proven_rng_u64(rng);
(void)word;   /* 어떤 64비트 값이든 옳은 답이다. 단언할 것이 없다 */

/* fill 은 무더기로 하는 꼴이다. 나머지를 스스로 처리해야 하는 64비트 낱말 반복문
 * 대신, 버퍼 하나를 한 번의 호출로 채운다. */
proven_byte_t nonce[12] = {0};
proven_rng_fill(rng, nonce, sizeof nonce);

bool all_zero = true;
for (proven_size_t i = 0; i < sizeof nonce; ++i) {
    if (nonce[i] != 0) all_zero = false;
}
EXAMPLE_REQUIRE(!all_zero, "filling from a seeded generator must produce something");

/* 같은 함수를, 서로 다른 생성기 둘이 굴린다. 이 특성이 존재하는 이유는 이것 하나다. */
proven_chacha_rng_t c;
proven_chacha_rng_seed(&c, seed);
proven_u64 crypto_total = roll_total(proven_chacha_rng(&c), 50);

proven_xoshiro256ss_t fast;
proven_xoshiro256ss_seed(&fast, 7);
proven_u64 fast_total = roll_total(proven_xoshiro256ss_rng(&fast), 50);

EXAMPLE_REQUIRE(crypto_total >= 50 && crypto_total <= 300, "50 dice must total between 50 and 300");
EXAMPLE_REQUIRE(fast_total >= 50 && fast_total <= 300, "whichever generator produced them");

/* --- 4. 생성기를 쥐지 않고 강한 낱말 하나 ------------------------------- */

/* proven_random_u64 는 엔트로피 원천에서 곧장 뽑는다. 한 번뿐인 일에는 편하고 -
 * 시작할 때 잡는 표의 해시 키, 요청 번호 - 반복문에는 틀린 도구다. 호출마다 운영체제로
 * 다녀오는 값이 들기 때문이다. 무더기로 뽑을 때는 생성기에 한 번 씨를 뿌리고 거기서
 * 뽑을 것. */
proven_u64 one_off = proven_random_u64();
proven_u64 another = proven_random_u64();
EXAMPLE_REQUIRE(one_off != another || one_off != 0,
                "two draws from the OS source are essentially never the same value");
```

```
    /* --- 5. 엔트로피 원천 달기 ------------------------------------------- */

    /* 호스트가 있는 대상에서는 운영체제의 원천이 이미 달려 있고 그대로 두어야 한다. 이
     * 고리는 베어메탈을 위한 것이다. 거기서는 그 보드의 엔트로피가 어느 하드웨어
     * 레지스터에 사는지 라이브러리가 알 길이 없다. 여기서는 일부러 가짜 원천을 달았다.
     * 오직 그 장치를 보이고 갈아 끼우기가 먹혔음을 증명하기 위해서다 - 진짜는 진짜
     * 하드웨어 엔트로피여야 한다. */
    proven_u8 counter = 0;
    proven_random_set_source(counting_entropy, &counter);

    proven_byte_t drawn[4] = {0};
    EXAMPLE_REQUIRE(proven_random_bytes(drawn, sizeof drawn), "the installed source must answer");
    EXAMPLE_REQUIRE(drawn[0] == 0 && drawn[1] == 1 && drawn[2] == 2 && drawn[3] == 3,
                    "and it is the source we installed that answered");

    /* 플랫폼 기본값을 되돌려 놓는다. 시험용 원천을 그대로 둔 채로 두는 것이, 프로그램이
     * 운영에서 예측 가능한 키를 만들어 내게 되는 방법이다. */
    proven_random_set_source(NULL, NULL);

    proven_byte_t real[8] = {0};
    EXAMPLE_REQUIRE(proven_random_bytes(real, sizeof real), "the OS source is back and working");

    printf("random: first word %llu, dice totals %llu and %llu\n",
           (unsigned long long)first, (unsigned long long)crypto_total, (unsigned long long)fast_total);

    return EXAMPLE_OK();
}
```

반례 — 무작위처럼 보이기만 하는 것을 설치하는 경우:

```
static bool clock_entropy(void *ctx, void *buf, proven_size_t len) {
    proven_time_t t = proven_time_now();       /* wrong: predictable */
    memcpy(buf, &t, len < sizeof t ? len : sizeof t);
    return true;
}
proven_random_set_source(clock_entropy, NULL);
```

시계, 일련번호, 초기화되지 않은 버퍼, 의사 난수 생성기는 모두 얼핏 보아 넘어가지만 추측할 수 있는 것을 만든다. 보드에 진짜 엔트로피가 없다면 아무것도 설치하지 않는 것으로 그 사실을 말한다. 거절은 호출자가 대응할 수 있는 사실이고, 조용한 예측 가능성은 그렇지 않다.

반례 — 시험용 출처를 설치한 채 두는 경우:

```
proven_random_set_source(counting_entropy, &counter);
/* ... the rest of the program, now generating predictable keys ... */
```

시험이 끝나는 즉시 `proven_random_set_source(NULL, NULL)`로 플랫폼 기본값을 되돌린다.

9장: 네트워킹

5부 — 운영체제와 대화하기. 선행 조건: 2부 (1, 2, 3), 그리고 5장의 §4(두 시계)와 스트림 절. 이 장을 마치면 TCP나 UDP 소켓을 열고, 기다리는 모든 것에 시간 제한을 걸고, 거절된 연결과 끊어진 연결을 구별하고, 스레드 하나로 여러 소켓을 처리하고, 무엇이 실어 나르는지 상관하지 않는 프로토콜을 쓸 수 있다.

이 장은 `net.h`를 다룬다. 5장처럼 운영체제가 필요하며 프리스탠딩 빌드에는 들어가지 않는다. 호스티드 빌드에서는 `PROVEN_NO_NET`으로 뺄 수 있다.

목차

1. 세 가지 규칙
2. 주소
3. 기한
4. TCP
5. UDP
6. 소켓 여럿, 스레드 하나
7. 전송
8. 짝, 그리고 루프 깨우기
9. selector: 소켓 수천 개
10. 플랫폼마다 다른 것
11. 여기에 없는 것

1. 세 가지 규칙

BSD 소켓은 마흔 살이고, 그 날카로운 모서리마다 이름이 붙어 있다. `net.h`는 그중 셋을 치우는 것이 일의 전부인 얇은 계층이다.

기다릴 수 있는 모든 호출은 기한을 받는다. 맨 소켓의 `recv`는 상대가 기다리게 하는 만큼 기다린다. 그렇게 지은 프로그램은 책상 위에서는 돌고, 운영 환경에서는 클라이언트 하나가 연결만 열고 아무 말도 하지 않는 첫 순간에 멈춘다. 연결을 하나씩 처리하는 서버라면 그런 클라이언트 하나로 충분하다. 여기서는 그런 프로그램을 쓸 방법이 없다. read, write, accept, connect 모두 기한 인자를 받고, 무엇이든 말해야 한다.

실패는 어느 실패인지 말한다. "호출이 실패했다"만으로는 다음에 무엇을 할지 정할 수 없다. 상대가 거절했는지, 사라졌는지, 제대로 끝냈는지는 서로 다른 세 상황이고 대응도 셋이다.

일어난 일	코드	합당한 대응
상대가 보내기를 끝냈다	`PROVEN_ERR_EOF`	읽기를 멈춘다. 정상적인 끝이다
기한이 지났다	`PROVEN_ERR_TIMEOUT`	재시도하거나, 포기하거나, 닫는다 — 망가진 것은 없다
그 주소에서 듣는 것이 없다	`PROVEN_ERR_REFUSED`	다른 주소를 시도하거나 보고한다. 재시도하면 같은 답이 온다
상대가 사라졌거나 연결이 중단됐다	`PROVEN_ERR_RESET`	닫는다. 전송 중이던 것은 사라졌을 수 있다
호스트나 네트워크로 가는 경로가 없다	`PROVEN_ERR_UNREACHABLE`	보고하거나 나중에 재시도한다
바인드할 주소가 이미 쓰이고 있거나, 시스템에 소켓이 바닥났다	`PROVEN_ERR_BUSY`	물러나거나 다른 포트를 고른다
그 주소는 바인드할 수 없다	`PROVEN_ERR_PERMISSION`	보고한다. 재시도해도 소용없다
호스트 이름에 주소가 없다. 유닉스 도메인 경로가 없다	`PROVEN_ERR_NOT_FOUND`	보고한다
데이터그램이 버퍼에 들어가지 않았거나, 보내기에 너무 크다	`PROVEN_ERR_OUT_OF_BOUNDS`	버퍼를 키운다. 덜 보낸다

소켓이 열려 있지 않다	PROVEN_ERR_INVALID_STATE	호출 순서를 고친다

이 중 어느 것도 맞지 않을 때 남는 것이 PROVEN_ERR_IO다.

숨기는 것이 없다. 이 헤더의 어떤 호출도 할당하지 않는다. 소켓은 여러분이 소유하고 닫아야 하는 작은 값이다. 소켓 위의 읽기 스트림(reader)이나 쓰기 스트림(writer)은 여러분이 선언한 상태를 쓴다. 프로세스 전체에 걸친 상태는 하나뿐이고 - Windows에서 Winsock을 시작하는 일 - 첫 사용 때 한 번 이뤄지며 여러분이 관리할 것이 아니다.

2. 주소

proven_net_addr_t는 소켓이 어디 있는지, 또는 어디로 가야 하는지를 말한다. 평범한 값이다. 복사하고, 비교하고, 표에 넣어도 된다.

```
typedef struct {
    proven_net_family_t family;   /* IPV4, IPV6 or UNIX; NONE when unset */
    proven_u16 port;              /* host byte order: the number you would write down */
    proven_u32 scope_id;          /* IPv6 zone for a link-local address; 0 otherwise */
    proven_byte_t ip[16];         /* network order: 4 bytes for IPv4, 16 for IPv6 */
    proven_u8 path_len;           /* Unix-domain only */
    char path[104];               /* Unix-domain only */
} proven_net_addr_t;
```

참조

API	의도	반환
proven_net_addr_ipv4(a, b, c, d, port)	숫자 넷으로 만드는 IPv4 주소.	proven_net_addr_t.
proven_net_addr_ipv6(ip[16], port, scope_id)	네트워크 순서의 바이트 열여섯 개로 만드는 IPv6 주소.	proven_net_addr_t.
proven_net_addr_loopback(family, port)	이 기계만: 127.0.0.1 또는 ::1.	proven_net_addr_t. family에 루프백이 없으면 family가 NONE.
proven_net_addr_any(family, port)	모든 인터페이스: 0.0.0.0 또는 ::.	proven_net_addr_t.
proven_net_addr_unix(path, &out)	파일 시스템 경로로 만드는 유닉스 도메인 주소.	proven_err_t: 빈 경로나 NUL이 든 경로는 INVALID_ARG, PROVEN_NET_UNIX_PATH_MAX를 넘으면 OUT_OF_BOUNDS.
proven_net_addr_parse(text, port, &out)	리터럴을 파싱한다: 192.0.2.1, 2001:db8::1, [2001:db8::1], fe80::1%3. 네트워크를 건드리지 않는다.	proven_err_t: text가 주소가 아니면 INVALID_FORMAT. 그때 out은 그대로다.
proven_net_addr_format(&addr, out, &written)	192.0.2.1:80, [2001:db8::1]:80, 또는 경로를 쓴다. NUL은 쓰지 않는다. PROVEN_NET_ADDR_TEXT_MAX바이트면 언제나 충분하다.	proven_err_t: out이 작으면 OUT_OF_BOUNDS.
proven_net_addr_eq(&a, &b)	family, 주소, 포트, zone이 같은가 - 또는 경로가 같은가.	bool.
proven_net_resolve(host, port, out, cap, &count)	시스템 리졸버로 이름을 찾는다. 리터럴은 질의 없이 답한다.	proven_err_t: NOT_FOUND, TIMEOUT(네임 서버가 답하지 않음), 빈 이름이나 너무 긴 이름은 INVALID_ARG.

파서의 두 가지 결정은 의도한 것이다. IPv4 리터럴은 정확히 십진수 넷이다. 오래된 파서들이 받아 주는 127.1과 2130706433은 거절하고, 그 파서들이 팔진수 *8*로 읽는 010.0.0.1도 거절한다. 그리고 IPv6는 들어올

때의 표기는 여럿이지만 나갈 때는 하나다 - 소문자, 앞자리 0 없음, 가장 긴 0 그룹의 연속을 ::로(RFC 5952). 그래서 같은 주소 둘은 같은 텍스트로 찍히고, 그 텍스트를 map 키나 검색할 로그 필드로 쓸 수 있다.

주의사항, 그리고 무엇이 잘못되는가

`proven_net_resolve`는 블록하고 기한이 없다. 시스템 리졸버이고, 리졸버가 기한을 제공하지 않으며, 망가진 네트워크에서는 수십 초가 걸릴 수 있다. 이 헤더에서 한도 없이 기다리는 호출은 이것 하나이고, 헤더는 아닌 척하지 않고 그렇게 적는다.

잘못된 예 — 다른 모든 연결을 처리하는 루프 안에서 이름을 해석하기:

```
for (;;) {
    wait_for_events();
    if (client_wants_upstream)
        proven_net_resolve(name, 443, addrs, 4, &n);   /* wrong: every client waits on DNS */
}
```

올바른 예 — 기다려도 되는 자리에서 해석한다. 시작할 때, 또는 job에서 (6장). 그리고 주소를 루프에 넘긴다.

"any"에 바인드하면 서비스를 공개하는 것이다. `proven_net_addr_any`는 네트워크를 향한 것까지 포함해 기계의 모든 인터페이스에서 듣는다. 자기 기계에서만 쓸 도구는 `proven_net_addr_loopback`에 바인드한다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 주소는 평범한 값이다. 만들고, 텍스트에서 파싱하고, 찍고, 둘을 비교한다.
 * 이 프로그램의 어느 줄도 소켓을 열거나 네트워크에 묻지 않는다.
 */

static bool prints_as(const proven_net_addr_t *addr, proven_u8str_view_t want) {
    proven_byte_t text[PROVEN_NET_ADDR_TEXT_MAX];
    proven_size_t n = 0;
    if (proven_net_addr_format(addr, (proven_mem_mut_t){ text, sizeof text }, &n) != PROVEN_OK) return false;
    return proven_u8str_view_eq((proven_u8str_view_t){ text, n }, want);
}

int main(void) {
    /* 부품으로 만든다. 포트는 종이에 적는 그 숫자다 - 호스트 바이트 순서. */
    proven_net_addr_t web = proven_net_addr_ipv4(192, 0, 2, 10, 80);
    EXAMPLE_REQUIRE(prints_as(&web, PROVEN_LIT("192.0.2.10:80")), "an IPv4 address prints as address:port");

    /* 프로그램에 가장 자주 필요한 고정 주소 둘: 이 기계만, 그리고 모든 인터페이스. */
    proven_net_addr_t local = proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 8080);
    proven_net_addr_t everywhere = proven_net_addr_any(PROVEN_NET_FAMILY_IPV6, 8080);
    EXAMPLE_REQUIRE(prints_as(&local, PROVEN_LIT("127.0.0.1:8080")), "loopback is 127.0.0.1");
    EXAMPLE_REQUIRE(prints_as(&everywhere, PROVEN_LIT("[::]:8080")), "any is ::, and IPv6 is bracketed before
a port");

    /* 텍스트에서 파싱한다. IPv6 주소 하나의 여러 표기가 같은 바이트로 파싱되고, 나갈 때의
     * 표기는 정확히 하나다. 그래서 텍스트를 비교하고 키로 쓸 수 있다. */
    proven_net_addr_t a, b;
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("2001:DB8:0:0:0:0:0:1"), 443, &a) == PROVEN_OK, "the long
form parses");
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("[2001:db8::1]"), 443, &b) == PROVEN_OK, "the short,
bracketed form parses");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&a, &b), "and they are the same address");
    EXAMPLE_REQUIRE(prints_as(&a, PROVEN_LIT("[2001:db8::1]:443")), "printed in the one canonical form");

    /* 주소가 아닌 것은 거절한다 - 다른 파서들이 조용히 받아 주는 것까지. */
    proven_net_addr_t bad;
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("010.0.0.1"), 80, &bad) == PROVEN_ERR_INVALID_FORMAT,
```

```
                  "a leading zero is refused, not read as octal");
    EXAMPLE_REQUIRE(proven_net_addr_parse(PROVEN_LIT("example.com"), 80, &bad) == PROVEN_ERR_INVALID_FORMAT,
                  "a name is not a literal: that is proven_net_resolve's job");

    /* 주소가 패킷이나 파일에서 올 때는 네트워크 순서의 원시 바이트 열여섯 개로. */
    proven_byte_t raw[16] = { 0xfe, 0x80, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 };
    proven_net_addr_t link_local = proven_net_addr_ipv6(raw, 22, 3);
    EXAMPLE_REQUIRE(prints_as(&link_local, PROVEN_LIT("[fe80::1%3]:22")), "a link-local address carries its
zone");

    /* 유닉스 도메인 주소는 파일 시스템 경로다. */
    proven_net_addr_t sock;
    EXAMPLE_REQUIRE(proven_net_addr_unix(PROVEN_LIT("/run/app.sock"), &sock) == PROVEN_OK, "a path makes an
address");
    EXAMPLE_REQUIRE(prints_as(&sock, PROVEN_LIT("/run/app.sock")), "and prints as the path");

    /* 리터럴의 해석은 질의 없이 답하므로, 이 줄은 네트워크가 없어도 동작한다.
     * 진짜 이름은 시스템 리졸버로 가고, 블록하며, 기한이 없다. 그래도 되는 자리에서
     * 불러라. */
    proven_net_addr_t found[4];
    proven_size_t count = 0;
    EXAMPLE_REQUIRE(proven_net_resolve(PROVEN_LIT("192.0.2.10"), 80, found, 4, &count) == PROVEN_OK && count
== 1,
                  "a literal resolves to itself");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&found[0], &web), "the same address as the one built by hand");

    return EXAMPLE_OK();
}
```

3. 기한

기한은 길이가 아니라 시점이다. 호출이 포기해야 하는, 단조 시계 (5장 §4)의 한 값이다.

	뜻
`proven_net_deadline_in(ms)`	지금부터 `ms`밀리초 뒤의 기한.
`PROVEN_NET_NO_DEADLINE`	걸리는 만큼 기다린다. 그러면 말 없는 상대와 함께 기다리게 된다.
`PROVEN_NET_DONT_WAIT`	지금 당장 할 수 있는 것만 한다. 나머지는 `PROVEN_ERR_TIMEOUT`이다.

절대 시점인 이유는 기한 하나로 교환 전체를 묶기 위해서다. "이 요청에 5초"는 connect에도, 각 write에도, 각 read에도 넘기는 값 하나다. 호출별 타임아웃이었다면 걸음마다 같은 5초를 다시 계산해야 하고, 대개는 그러지 않는다.

단조 시계 위에 있는 이유는 시스템 시간을 맞춰도 기한이 움직이지 않게 하기 위해서다.

그리고 지나간 기한은 아무것도 망가뜨리지 않는다. `PROVEN_ERR_TIMEOUT`은 "아직"이라는 뜻이고, 소켓은 전과 똑같이 쓸 수 있다. 다시 시도할지, 포기할지, 닫을지는 여러분이 정한다.

잘못된 예 — 타임아웃을 죽은 연결로 취급하기:

```
proven_result_size_t r = proven_net_read(&conn, buf, proven_net_deadline_in(100));
if (r.err != PROVEN_OK) proven_net_close(&conn);   /* wrong: TIMEOUT is not a failure of the connection */
```

올바른 예 — 이 프로토콜에서 침묵이 무엇을 뜻하는지 정한다:

```
if (r.err == PROVEN_ERR_TIMEOUT) { send_keepalive(&conn); continue; }
if (r.err == PROVEN_ERR_EOF)     { finish(&conn); break; }
if (r.err != PROVEN_OK)          { proven_net_close(&conn); break; }
```

4. TCP

TCP 연결은 방향마다 하나씩의 바이트 스트림이다. 메시지 경계가 없다. 한쪽이 write 한 번으로 보낸 것을 다른 쪽은 read 세 번으로 받을 수 있고, write 둘을 read 하나로 받을 수도 있다.

참조

API	의도	반환
`proven_net_listen(at, backlog, &listener, &bound)`	`at`에서 듣는다. 포트 0은 OS가 고른다. `bound`(선택)는 실제로 바인드된 주소다.	`proven_err_t`: `BUSY`(사용 중), `PERMISSION`, `INVALID_ARG`(이 기계의 주소가 아님), `UNSUPPORTED`.
`proven_net_accept(&listener, until, &conn, &peer)`	다음 연결을 받는다. `peer`는 선택이다.	`proven_err_t`: 오지 않았으면 `TIMEOUT`.
`proven_net_listener_addr(&listener, &out)`	리스너가 바인드된 곳.	`proven_err_t`.
`proven_net_listener_close(&listener)`	듣기를 멈춘다. 이미 받은 연결에는 영향이 없다.	`proven_err_t`.
`proven_net_connect(to, until, &conn)`	연결한다. 실패하면 닫을 것이 없다.	`proven_err_t`: `REFUSED`, `TIMEOUT`, `UNREACHABLE`, `NOT_FOUND`(유닉스 도메인 경로).
`proven_net_connect_start(to, &conn)`	연결을 시작하고 곧바로 돌아온다. 여러 소켓을 함께 기다리는 프로그램을 위한 것이다(9절, 또는 15장의 루프).	`proven_err_t`: 이미 연결되었으면 `PROVEN_OK`. 진행 중이면 `AGAIN` - 그때 `conn`은 쓸 수 있다고 보고될 때까지 지켜보고 닫는 것만 할 수 있다. 그 밖에는 실패이고 열린 것이 없다.
`proven_net_connect_finish(&conn)`	진행 중이던 연결이, 그 소켓이 쓸 수 있다거나 실패했다고 보고된 뒤 어떻게 끝났는지.	`proven_err_t`: 연결되었으면 `PROVEN_OK`. `AGAIN`이면 다시 기다린다. 그 밖에는 실패이고 `conn`은 닫혔다. 열려 있지 않으면 `INVALID_STATE`.
`proven_net_read(&conn, dest, until)`	도착한 것을 `dest.size`까지 읽는다.	`proven_result_size_t`: 끝에서는 `EOF` - 0바이트 성공은 없다. `TIMEOUT`. `RESET`.
`proven_net_write(&conn, src, until)`	지금 들어가는 만큼 쓴다. `.value`는 실패했을 때도 나간 양이다.	`proven_result_size_t`.
`proven_net_write_all(&conn, src, until)`	`src` 전부를 쓰거나, 어디까지 갔는지 말하며 실패한다.	`proven_result_size_t`: `.err`가 OK일 때에만 `.value == src.size`.
`proven_net_shutdown_write(&conn)`	"더 보낼 것이 없다." 상대는 `EOF`를 읽고, 이쪽은 계속 읽을 수 있다.	`proven_err_t`.
`proven_net_close(&conn)`	닫는다. 열려 있지 않은 연결에 불러도 안전하다.	`proven_err_t`.
`proven_net_conn_local_addr(&conn, &out)`, `proven_net_conn_peer_addr(&conn, &out)`	연결의 두 끝.	`proven_err_t`.
`proven_net_conn_set_nodelay(&conn, on)`	작은 write를 모으지 않고 바로 보낸다.	`proven_err_t`: 유닉스 도메인 연결에서는 `UNSUPPORTED`.

`proven_net_listener_is_open(&l)`, `proven_net_conn_is_open(&c)`	값이 열린 소켓을 들고 있는가.	`bool`.

`proven_net_listener_t`와 `proven_net_conn_t`는 여러분이 소유하는 작은 값이다. 연 것은 하나하나 닫고, 열려 있는 것을 복사하지 마라. 복사본 둘은 소켓 하나의 주인 둘이다.

두 번에 나눠 하는 connect. `proven_net_connect`는 기다린다. 많은 소켓을 한꺼번에 기다리는 프로그램은 그럴 수 없으므로, 같은 connect가 호출 둘로도 제공된다:

```
proven_err_t proven_net_connect_start(proven_net_addr_t to, proven_net_conn_t *out);
proven_err_t proven_net_connect_finish(proven_net_conn_t *conn);
```

`proven_net_connect_start`는 곧바로 돌아온다: 연결이 이미 되었으면 `PROVEN_OK`(루프백 인터페이스에서는 흔히 그렇다), 진행 중이면 `PROVEN_ERR_AGAIN`. 뒤의 경우 그 소켓은 지켜보고 닫는 것만 할 수 있다: `proven_net_poll`이나 셀렉터나 15장의 루프로 쓸 수 있다고 보고될 때까지 기다린 다음, `proven_net_connect_finish`에게 어떻게 끝났는지 묻는다. 어느 호출에서든 실패하면 열린 것이 남지 않는다. 15장의 이벤트 구동 클라이언트가 이 쌍 위에 만들어져 있다.

같은 호출들이 **유닉스 도메인** 스트림 소켓에도 쓰인다. `proven_net_addr_unix`로 만든 주소를 넘기면 된다. listen은 경로를 만들고, 리스너를 닫아도 그 경로는 지워지지 않는다. 그래서 서버는 듣기 전에 남아 있는 경로를 지운다.

주의사항, 그리고 무엇이 잘못되는가

read 한 번이 메시지 하나가 아니다. 스트림은 바이트를 순서대로, 네트워크가 나눈 조각대로 전한다.

잘못된 예 — 요청이 read 한 번에 다 온다고 가정하기:

```
proven_result_size_t r = proven_net_read(&conn, buf, until);
handle_request(buf, r.value);     /* wrong: this may be half a request, or one and a half */
```

올바른 예 — 프로토콜이 메시지가 완성됐다고 말할 때까지 읽는다. 알려 준 길이, 약속된 구분자, 또는 `PROVEN_ERR_EOF`. 아래 예제는 끝까지 읽는다.

write 한 번이 전부를 보내지 않을 수 있다. `proven_net_write`는 얼마나 나갔는지 보고한다. 그 개수를 무시하면 버퍼가 마침 차 있을 때마다 - 곧, 부하가 걸린 운영 환경에서 - 메시지가 조용히 잘린다. 끝까지 보내는 호출은 `proven_net_write_all`이고, 그것이 실패했을 때 다시 보내면 데이터가 중복되는지를 알려 주는 것이 그 개수다.

reset은 끝이 아니다. `PROVEN_ERR_EOF`는 상대가 끝났다고 말했고 보낸 것을 모두 읽었다는 뜻이다. `PROVEN_ERR_RESET`은 상대가 말없이 사라졌고, 전송 중이던 것의 일부가 - 어느 방향이든 - 없어졌을 수 있다는 뜻이다. `EOF`로 끝난 파일 전송은 완전하다. `RESET`으로 끝난 것은 몇 바이트가 도착했든 완전하지 않다.

닫는 것과 끝내는 것은 다르다. 상대가 전부 받았는지 확실히 하려면, `proven_net_shutdown_write`로 끝났다고 말하고, `EOF`까지 읽고, 그다음에 닫는다. 읽지 않은 데이터가 남은 연결을 닫는 것이 바로 *상대편*이 `PROVEN_ERR_RESET`을 받게 되는 경로다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 처음부터 끝까지의 TCP 교환. 기다릴 수 있는 모든 호출에 기한이 붙는다.
 *
 * 양쪽 끝이 이 프로그램 하나에 산다. 커널은 리스너의 백로그가 연결을 받는 순간 - accept가
 * 불리기 전에 - 연결을 완성하므로 이것이 가능하다. 실제 클라이언트와 서버는 같은 호출들을
 * 두 프로세스에 나눠 놓은 것이다.
 */

int main(void) {
    /* 포트 0: OS가 빈 포트를 고르고, `at`이 어느 것인지 알려 준다. 루프백: 이 기계만. */
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 16, &listener,
```

```
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_net_listener_is_open(&listener), "the listener opens");
    EXAMPLE_REQUIRE(at.port != 0, "and reports the port it was given");

    /* 교환 전체에 기한 하나: 호출이 몇 번이 되든 지금부터 5초. */
    proven_net_deadline_t until = proven_net_deadline_in(5000);

    proven_net_conn_t client, server;
    proven_net_addr_t peer;
    EXAMPLE_REQUIRE(proven_net_connect(at, until, &client) == PROVEN_OK, "the client connects");
    EXAMPLE_REQUIRE(proven_net_accept(&listener, until, &server, &peer) == PROVEN_OK, "the server accepts");
    EXAMPLE_REQUIRE(proven_net_conn_is_open(&client) && proven_net_conn_is_open(&server), "two open ends");

    /* 양쪽 끝이 두 주소를 다 안다. accept가 준 상대 주소는 클라이언트 자신의 로컬 주소다. */
    proven_net_addr_t client_local, client_peer, listening;
    EXAMPLE_REQUIRE(proven_net_conn_local_addr(&client, &client_local) == PROVEN_OK &&
                    proven_net_conn_peer_addr(&client, &client_peer) == PROVEN_OK, "the client's two
addresses");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&client_local, &peer), "the server saw the client's address");
    EXAMPLE_REQUIRE(proven_net_listener_addr(&listener, &listening) == PROVEN_OK &&
                    proven_net_addr_eq(&client_peer, &listening), "the client's peer is where the server
listens");

    /* 작은 조각 둘로 쓰는 요청: 이것이 없으면 둘이 함께 나가려고 기다릴 수 있다. */
    EXAMPLE_REQUIRE(proven_net_conn_set_nodelay(&client, true) == PROVEN_OK, "small writes go at once");

    /* write_all은 전부 보내거나, 어디까지 갔는지 말한다. */
    proven_mem_view_t request = proven_mem_view_from_u8(PROVEN_LIT("what time is it?"));
    proven_result_size_t sent = proven_net_write_all(&client, request, until);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == request.size, "the whole request went");

    /* "더 보낼 것이 없다." 서버는 요청을 읽고, 그다음 입력의 끝을 읽는다. */
    EXAMPLE_REQUIRE(proven_net_shutdown_write(&client) == PROVEN_OK, "the client half-closes");

    /* 서버는 끝까지 읽는다. read는 도착한 만큼을 돌려준다 - 스트림에는 메시지 경계가
     * 없다 - 그러니 "전부"를 읽는 올바른 방법은 루프뿐이다. */
    proven_byte_t buf[64];
    proven_size_t got = 0;
    for (;;) {
        proven_result_size_t r = proven_net_read(&server, (proven_mem_mut_t){ buf + got, sizeof buf - got },
until);
        if (r.err == PROVEN_ERR_EOF) break;                 /* 클라이언트가 끝냈다 */
        EXAMPLE_REQUIRE(r.err == PROVEN_OK, "a read that is not the end succeeds");
        if (r.err != PROVEN_OK) break;
        got += r.value;
    }
    EXAMPLE_REQUIRE(got == request.size, "the server received the whole request");

    /* write 한 번은 일부만 보낼 수 있고, 개수가 얼마인지 말한다. 여기서는 전부 들어간다. */
    proven_mem_view_t reply = proven_mem_view_from_u8(PROVEN_LIT("time to close"));
    proven_result_size_t w = proven_net_write(&server, reply, until);
    EXAMPLE_REQUIRE(w.err == PROVEN_OK && w.value == reply.size, "a short reply goes in one write");

    proven_result_size_t r = proven_net_read(&client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == reply.size, "the client reads the reply after its half-
close");
```

```
    /* 더 오는 것이 없고, 클라이언트는 그것을 알아내려고 영원히 기다리지 않는다. 타임아웃은
     * 손상이 아니다. 연결은 전과 똑같이 쓸 수 있다. */
    r = proven_net_read(&client, (proven_mem_mut_t){ buf, sizeof buf }, proven_net_deadline_in(50));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_TIMEOUT, "silence is a timeout, not a hang");

    /* 모든 소켓은 그 주인이 닫는다. */
    EXAMPLE_REQUIRE(proven_net_close(&server) == PROVEN_OK, "the server closes its end");
    r = proven_net_read(&client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_EOF, "which the client reads as the end");
    EXAMPLE_REQUIRE(proven_net_close(&client) == PROVEN_OK, "the client closes");
    EXAMPLE_REQUIRE(proven_net_listener_close(&listener) == PROVEN_OK, "the listener closes");

    /* 리스너가 사라지면 같은 주소가 거절한다 - 타임아웃과는 다른 답이다. */
    proven_net_conn_t nobody;
    EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(15000), &nobody) == PROVEN_ERR_REFUSED,
                    "nothing listening is PROVEN_ERR_REFUSED");

    return EXAMPLE_OK();
}
```

5. UDP

UDP는 메시지 - 데이터그램 - 를 하나씩, 각각 따로 보낸다. 데이터그램은 온전히 도착하거나 아예 도착하지 않는다. 순서가 바뀌거나, 두 번 오거나, 영영 오지 않을 수 있고, 어느 쪽인지 보낸 쪽에 알려 주는 것은 없다.

참조

API	의도	반환
`proven_net_udp_open(at, &udp, &bound)`	`at`에 바인드된 소켓을 연다. 클라이언트는 `proven_net_addr_any(family, 0)`에 바인드한다.	`proven_err_t`: `BUSY`, `PERMISSION`, `UNSUPPORTED`(유닉스 도메인 주소).
`proven_net_udp_send_to(&udp, to, data, until)`	데이터그램 하나를 보낸다: `data` 전부 아니면 아무것도.	`proven_err_t`: `data`가 데이터그램이 될 수 있는 크기를 넘으면 `OUT_OF_BOUNDS`.
`proven_net_udp_recv_from(&udp, dest, &from, until)`	데이터그램 하나를 받는다. `from`은 선택이다.	`proven_result_size_t`: 잘렸으면 `OUT_OF_BOUNDS`와 `.value == dest.size`. `TIMEOUT`.
`proven_net_udp_addr(&udp, &out)`	소켓이 바인드된 곳.	`proven_err_t`.
`proven_net_udp_close(&udp)`	닫는다. 열려 있지 않은 소켓에 불러도 안전하다.	`proven_err_t`.
`proven_net_udp_is_open(&udp)`	값이 열린 소켓을 들고 있는가.	`bool`.

주의사항, 그리고 무엇이 잘못되는가

"보냈다"는 떠났다는 뜻이지 도착했다는 뜻이 아니다. `proven_net_udp_send_to`의 성공은 데이터그램을 네트워크에 넘겼다는 것이다. 답이 필요한 프로토콜은 기한을 두고 답을 기다리고, 오지 않으면 다시 보낸다.

길이 0인 데이터그램도 메시지다. 스트림에서 0바이트는 끝을 뜻할 것이고, 그래서 `proven_net_read`는 그것을 돌려주지 않는다. 데이터그램은 정말로 비어 있을 수 있으므로, `proven_net_udp_recv_from`이 `.value` 0으로 성공하는 것은 끝도 아니고 오류도 아니다.

너무 작은 버퍼는 나머지를 잃는다. 데이터그램에서 들어가지 않은 부분은 다음 받기를 위해 남겨지지 않는다. 사라진다.

잘못된 예 — 어림으로 잡은 버퍼, 그리고 무시한 오류:

```
proven_byte_t buf[64];
proven_result_size_t r = proven_net_udp_recv_from(&udp, buf_mem, &from, until);
parse(buf, r.value);      /* wrong: on OUT_OF_BOUNDS this is the first 64 bytes of something longer */
```

올바른 예 — 프로토콜이 허용하는 가장 큰 메시지에 맞춰 버퍼를 잡고, `PROVEN_ERR_OUT_OF_BOUNDS`는 잘못된 패킷으로 취급한다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * UDP: 스트림이 아니라 메시지다. 보내기 한 번이 데이터그램 하나이고 받기 한 번이 데이터그램
 * 하나이며, 보낸 쪽 주소가 붙어 온다 - 그리고 도착한다는 약속은 없다.
 */

int main(void) {
    proven_net_udp_t server, client;
    proven_net_addr_t server_at, client_at;
    proven_err_t err = proven_net_udp_open(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &server,
&server_at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_net_udp_is_open(&server), "the server's socket opens");
    EXAMPLE_REQUIRE(proven_net_udp_open(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &client, NULL) ==
PROVEN_OK,
                    "and the client's");
    EXAMPLE_REQUIRE(proven_net_udp_addr(&client, &client_at) == PROVEN_OK && client_at.port != 0,
                    "a socket can be asked where it is bound");

    proven_net_deadline_t until = proven_net_deadline_in(2000);

    /* 보내기 둘은 데이터그램 둘이다. 성공은 각각이 이 기계를 떠났다는 뜻이다. */
    EXAMPLE_REQUIRE(proven_net_udp_send_to(&client, server_at, proven_mem_view_from_u8(PROVEN_LIT("first")),
until) == PROVEN_OK &&
                    proven_net_udp_send_to(&client, server_at, proven_mem_view_from_u8(PROVEN_LIT("second
message")), until) == PROVEN_OK,
                    "two datagrams leave");

    /* 받기 한 번은 온전한 데이터그램 하나이고, 누가 보냈는지 말해 준다. */
    proven_byte_t buf[32];
    proven_net_addr_t from;
    proven_result_size_t r = proven_net_udp_recv_from(&server, (proven_mem_mut_t){ buf, sizeof buf }, &from,
until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == 5, "the first datagram, alone: five bytes");
    EXAMPLE_REQUIRE(proven_net_addr_eq(&from, &client_at), "from the client's address");

    /* 너무 작은 버퍼는 데이터그램을 자르고, 잘랐다고 말한다. 나머지는 사라진다 - 다음
     * 받기는 다음 데이터그램이다 - 그러니 버퍼는 가장 큰 메시지에 맞춰라. */
    proven_byte_t small[6];
    r = proven_net_udp_recv_from(&server, (proven_mem_mut_t){ small, sizeof small }, &from, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_OUT_OF_BOUNDS && r.value == sizeof small,
                    "fourteen bytes into six: PROVEN_ERR_OUT_OF_BOUNDS, six kept");

    /* 답장은 데이터그램이 온 주소로 간다. */
    EXAMPLE_REQUIRE(proven_net_udp_send_to(&server, from, proven_mem_view_from_u8(PROVEN_LIT("ack")), until)
== PROVEN_OK,
                    "the server answers the sender");
    r = proven_net_udp_recv_from(&client, (proven_mem_mut_t){ buf, sizeof buf }, NULL, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == 3, "and the client receives it");
```

```
    /* 더 오는 것이 없다. 잃어버린 데이터그램도 정확히 이렇게 보인다. 모든 받기에 기한이
     * 있고 모든 UDP 프로토콜에 재시도가 있는 이유다. */
    r = proven_net_udp_recv_from(&client, (proven_mem_mut_t){ buf, sizeof buf }, NULL,
proven_net_deadline_in(50));
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_TIMEOUT, "silence is a timeout");

    EXAMPLE_REQUIRE(proven_net_udp_close(&client) == PROVEN_OK && proven_net_udp_close(&server) == PROVEN_OK,
                    "both sockets are closed by their owner");
    return EXAMPLE_OK();
}
```

6. 소켓 여럿, 스레드 하나

기한은 소켓 하나에 대한 기다림을 묶는다. 서버는 자기 소켓 전부를 한꺼번에 기다리다가 손이 필요한 것을 처리한다. 그것이 `proven_net_poll`이다.

참조

API	의도	반환
`proven_net_listener_handle(&l)`, `proven_net_conn_handle(&c)`, `proven_net_udp_handle(&u)`	종류가 무엇이든, poll이 보는 소켓.	`proven_net_handle_t`. 열려 있지 않은 소켓이면 `valid`가 아니다.
`proven_net_poll(items, count, until, &ready)`	적어도 하나가 준비될 때까지 기다린다. `PROVEN_NET_POLL_INLINE_MAX`(64)개까지.	`proven_err_t`: 아무것도 준비되지 않았으면 `TIMEOUT`(`ready`는 0). 항목이 너무 많으면 `OUT_OF_BOUNDS`. 핸들이 유효하지 않으면 `INVALID_ARG`.
`proven_net_poll_with(scratch, items, count, until, &ready)`	여러분이 준 메모리로, 개수 제한 없이 같은 일을 한다.	`proven_err_t`. `scratch`가 작으면 `OUT_OF_BOUNDS`.
`proven_net_poll_scratch_size(count)`	항목 `count`개에 필요한 임시 작업용(scratch) 메모리 크기.	`proven_size_t`. 표현할 수 없으면 `SIZE_MAX`.

```
typedef struct {
    proven_net_handle_t handle;
    proven_u8 want;   /* PROVEN_NET_READABLE and/or PROVEN_NET_WRITABLE */
    proven_u8 got;    /* filled in: zero, or what is ready - which may include PROVEN_NET_FAILED */
} proven_net_poll_item_t;
```

준비됨은 "기다리지 않는다"는 뜻이지 "성공한다"는 뜻이 아니다. 읽을 수 있는 연결에는 데이터가, 또는 끝이, 또는 오류가 있을 수 있다. 읽을 수 있는 리스너에는 받을 연결이 있다. 그래서 준비 보고 다음의 호출은 `PROVEN_NET_DONT_WAIT`를 받고, 그 결과에 따라 행동한다. `PROVEN_NET_FAILED`로 보고된 항목은 오류 상태이거나 끊긴 것이고, 다음 read나 write가 어느 쪽인지 말해 준다.

쓸 것이 있을 때에만 쓰기 가능을 물어라. 한가한 연결은 거의 언제나 쓸 수 있으므로, 언제나 그것을 묻는 루프는 매번 즉시 깨어나 아무 일도 하지 않으며 프로세서를 돌린다.

잘못된 예:

```
items[i].want = PROVEN_NET_READABLE | PROVEN_NET_WRITABLE;   /* wrong when nothing is queued: never sleeps */
```

올바른 예 — `PROVEN_NET_WRITABLE`은 마지막 write가 요청보다 적게 보냈거나 타임아웃된 연결에만, 그리고 그 데이터가 나갈 때까지만 건다.

`proven_net_poll_with`가 있는 이유는 운영체제가 자기 배열을 원하고 이 라이브러리는 여러분 몰래 그것을 할당하지 않기 때문이다. `proven_net_poll_scratch_size`로 크기를 잰 작업 메모리를 주고, 다음 호출에 다시 쓴다. 여기서의 준비 상태는 POSIX의 `poll`과 Windows의 `WSAPoll`이다. 소켓 수십 개나 수백 개에 맞는 도구다. 한 번에 바쁜 것이 적은 수천 개에는 §9의 selector를 써라.

테스트 스위트가 컴파일하고 실행한다:

```c
/*
 * 스레드 하나, 소켓 여럿: 그중 하나라도 손이 필요해질 때까지 기다렸다가, 필요한 것들을
 * 처리한다. 모든 단일 스레드 서버 안에 있는 루프다.
 *
 * "준비됨"은 그 종류의 다음 호출이 기다리지 않는다는 뜻이다. 그래서 준비됐다고 보고된
 * 소켓은 PROVEN_NET_DONT_WAIT로 다루고, 루프는 poll 말고는 어디서도 블록하지 않는다.
 */

int main(void) {
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 16, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the listener opens");

    /* 클라이언트 둘이 연결하고 각자 메시지 하나를 보낸다. */
    proven_net_conn_t clients[2];
    for (int i = 0; i < 2; ++i) {
        EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(5000), &clients[i]) == PROVEN_OK, "a
client connects");
    }
    EXAMPLE_REQUIRE(proven_net_write_all(&clients[0], proven_mem_view_from_u8(PROVEN_LIT("from the first")),
proven_net_deadline_in(5000)).err == PROVEN_OK &&
                    proven_net_write_all(&clients[1], proven_mem_view_from_u8(PROVEN_LIT("from the second")),
proven_net_deadline_in(5000)).err == PROVEN_OK,
                    "each sends a message");

    /* 서버: 리스너 하나와 연결 최대 둘을 목록 하나에. */
    proven_net_conn_t served[2] = {0};
    proven_size_t served_count = 0;
    proven_size_t bytes_seen = 0;

    proven_net_deadline_t until = proven_net_deadline_in(5000);
    while (bytes_seen < 14 + 15) {
        proven_net_poll_item_t items[3];
        proven_size_t count = 0;
        items[count++] = (proven_net_poll_item_t){ .handle = proven_net_listener_handle(&listener), .want =
PROVEN_NET_READABLE };
        for (proven_size_t i = 0; i < served_count; ++i) {
            items[count++] = (proven_net_poll_item_t){ .handle = proven_net_conn_handle(&served[i]), .want =
PROVEN_NET_READABLE };
        }

        proven_size_t ready = 0;
        err = proven_net_poll(items, count, until, &ready);
        EXAMPLE_REQUIRE(err == PROVEN_OK && ready > 0, "something becomes ready before the deadline");
        if (err != PROVEN_OK) break;

        /* 읽을 수 있는 리스너에는 기다리는 연결이 있다. */
        if (items[0].got & PROVEN_NET_READABLE) {
            EXAMPLE_REQUIRE(served_count < 2 &&
                            proven_net_accept(&listener, PROVEN_NET_DONT_WAIT, &served[served_count], NULL) ==
PROVEN_OK,
                            "the accept that poll predicted does not wait");
            served_count++;
        }
```

```
        /* 읽을 수 있는 연결에는 데이터가, 아니면 끝이 있다. `items[1 + i]`는 `served[i]`에서 만들었다. */
        for (proven_size_t i = 1; i < count; ++i) {
            if (!items[i].got) continue;
            proven_byte_t buf[64];
            proven_result_size_t r = proven_net_read(&served[i - 1], (proven_mem_mut_t){ buf, sizeof buf },
PROVEN_NET_DONT_WAIT);
            EXAMPLE_REQUIRE(r.err == PROVEN_OK, "the read that poll predicted does not wait");
            bytes_seen += r.value;
        }
    }
    EXAMPLE_REQUIRE(served_count == 2 && bytes_seen == 29, "both clients were accepted and both messages read,
in one thread");

    /* proven_net_poll은 PROVEN_NET_POLL_INLINE_MAX개까지 받는다. 그보다 많은 서버는 자기
     * 작업 메모리를 넘긴다. OS는 배열을 원하고, 이 라이브러리는 그것을 할당하지 않는다. */
    proven_net_poll_item_t one = { .handle = proven_net_conn_handle(&served[0]), .want =
PROVEN_NET_WRITABLE };
    proven_byte_t scratch[256];
    proven_size_t ready = 0;
    EXAMPLE_REQUIRE(proven_net_poll_scratch_size(1) <= sizeof scratch, "the scratch for one item is small");
    EXAMPLE_REQUIRE(proven_net_poll_with((proven_mem_mut_t){ scratch, sizeof scratch }, &one, 1,
PROVEN_NET_DONT_WAIT, &ready) == PROVEN_OK &&
                    (one.got & PROVEN_NET_WRITABLE), "an idle connection is writable at once");

    /* UDP 소켓도 다른 것들과 같은 목록에서 기다린다. */
    proven_net_udp_t udp;
    EXAMPLE_REQUIRE(proven_net_udp_open(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &udp, NULL) ==
PROVEN_OK, "a UDP socket");
    proven_net_poll_item_t quiet = { .handle = proven_net_udp_handle(&udp), .want = PROVEN_NET_READABLE };
    EXAMPLE_REQUIRE(proven_net_poll(&quiet, 1, proven_net_deadline_in(30), &ready) == PROVEN_ERR_TIMEOUT &&
ready == 0,
                    "nothing arrives on it: PROVEN_ERR_TIMEOUT, the idle tick of an event loop");
    (void)proven_net_udp_close(&udp);

    for (int i = 0; i < 2; ++i) {
        (void)proven_net_close(&clients[i]);
        (void)proven_net_close(&served[i]);
    }
    (void)proven_net_listener_close(&listener);
    return EXAMPLE_OK();
}
```

7. 전송

프로토콜 - 줄 단위 명령 집합, HTTP 교환 - 은 한 번만 쓰여야 하고, 자기 바이트를 무엇이 실어 나르는지 몰라야 한다. 오늘 그것은 TCP 연결이다. TLS 아래에서는 그 위의 암호화된 세션이다. 테스트에서는 메모리 버퍼 둘이고 네트워크는 없다. `proven_transport_t`가 그 이음매다.

```
typedef struct {
    void *ctx;
    proven_result_size_t (*read_fn)(void *ctx, proven_mem_mut_t dest, proven_net_deadline_t until);
    proven_result_size_t (*write_fn)(void *ctx, proven_mem_view_t src, proven_net_deadline_t until);
    proven_err_t (*shutdown_fn)(void *ctx);   /* may be NULL */
    proven_err_t (*close_fn)(void *ctx);      /* may be NULL */
} proven_transport_t;
```

값으로 넘기는 작은 vtable로, `proven_allocator_t`(2장)와 `proven_writer_t`와 같은 모양이고 이유도 같다. 호출자가 정하고, 숨기는 것이 없다. 직접 만들려면 구조체를 채우면 된다. 계약은 TCP 호출들의 것이다. read는 0바이트로 성공하지 않고, write는 실패하더라도 나간 바이트를 보고한다.

참조

API	의도	반환
`proven_net_conn_transport(&conn)`	전송으로서의 연결. 전송을 닫으면 연결이 닫힌다.	`proven_transport_t`.
`proven_transport_is_valid(t)`	read와 write 함수가 있는가.	`bool`.
`proven_transport_read(t, dest, until)`, `proven_transport_write(t, src, until)`	`proven_net_read`, `proven_net_write`와 같다.	`proven_result_size_t`. 유효하지 않은 전송이면 `INVALID_ARG`.
`proven_transport_write_all(t, src, until)`	전부 쓰거나, 개수와 함께 실패한다. 아무것도 받지 않으면서 오류도 내지 않는 전송은 끝없는 루프가 아니라 `PROVEN_ERR_IO`다.	`proven_result_size_t`.
`proven_transport_shutdown(t)`, `proven_transport_close(t)`	보내는 쪽을 끝낸다. 닫는다. 전송에 할 일이 없으면 `PROVEN_OK`.	`proven_err_t`.
`proven_transport_reader(&state, t, timeout_ms)`	전송 위의 `proven_reader_t`. 줄 reader를 비롯해 스트림을 읽는 모든 것에 쓴다.	`proven_reader_t`. `state`가 NULL이거나 `t`가 유효하지 않으면 유효하지 않다.
`proven_transport_writer(&state, t, timeout_ms)`	전송 위의 `proven_writer_t`. `proven_fprint`가 보낼 수 있게 된다.	`proven_writer_t`.

reader와 writer 어댑터는 소켓을 5장의 스트림 절에 있는 모든 것과 잇는다. reader에는 기한을 넘길 자리가 없으므로, 어댑터를 거치는 read나 write는 각자의 기한을 받는다. 시작한 순간부터 `timeout_ms`, 그 값이 0이면 기한 없음이다. `proven_transport_stream_t` 상태는 호출자 소유다. reader나 writer보다 오래 살아야 하고 움직여서는 안 된다.

writer는 조각을 받는 대로 보낸다. 메시지를 여러 작은 조각으로 포맷한다면 `proven_writer_buffered`로 감싸서 적은 수의 패킷으로 나가게 하고, 답을 기다리기 전에 flush하라.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * proven_transport_t에 대고 한 번 쓴 프로토콜을 TCP 연결 위에서 돌린다.
 *
 * 아래 함수는 자기 바이트를 무엇이 실어 나르는지 모른다. 오늘은 소켓이고, TLS 아래에서는
 * 소켓 위의 암호화된 세션이 되며, 테스트에서는 메모리 버퍼 둘일 수 있다. 이 인터페이스가
 * 있는 이유가 그것이다.
 */

/* 요청 한 줄을 보내고 답 한 줄을 읽는다. 걸음마다 5초 제한. */
static proven_err_t ask(proven_transport_t t, const char *question, proven_u8str_view_t expected) {
    if (!proven_transport_is_valid(t)) return PROVEN_ERR_INVALID_ARG;

    /* 포매터가 전송에 곧바로 쓴다. */
    proven_transport_stream_t out_state;
    proven_writer_t out = proven_transport_writer(&out_state, t, 5000);
    proven_err_t err = proven_fprintln(out, "ASK {}", PROVEN_ARG(question)).err;
    if (err != PROVEN_OK) return err;

    /* 줄 reader가 거기서 읽어, 이 함수가 가진 버퍼에 담는다. */
    proven_transport_stream_t in_state;
    proven_byte_t line_buf[128];
    proven_reader_buffered_t lines;
    (void)proven_reader_buffered(&lines, proven_transport_reader(&in_state, t, 5000),
```

```
                                (proven_mem_mut_t){ line_buf, sizeof line_buf });
    proven_result_u8str_view_t reply = proven_reader_read_line(&lines);
    if (reply.err != PROVEN_OK) return reply.err;
    return proven_u8str_view_eq(reply.val, expected) ? PROVEN_OK : PROVEN_ERR_INVALID_FORMAT;
}

int main(void) {
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 4, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the listener opens");

    proven_net_conn_t client_conn, server_conn;
    EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(5000), &client_conn) == PROVEN_OK &&
                    proven_net_accept(&listener, proven_net_deadline_in(5000), &server_conn, NULL) ==
PROVEN_OK,
                    "a connected pair");

    /* 전송으로서의 연결. 여기서부터는 어느 쪽도 소켓을 입에 올리지 않는다. */
    proven_transport_t client = proven_net_conn_transport(&client_conn);
    proven_transport_t server = proven_net_conn_transport(&server_conn);
    proven_net_deadline_t until = proven_net_deadline_in(5000);

    /* 서버 쪽 절반을 손으로 한다. 답을 클라이언트가 묻기 전에 써 둔다. 스레드 하나에서는
     * 괜찮다. 바이트가 연결 안에서 기다릴 뿐이다. */
    proven_result_size_t w = proven_transport_write_all(server, proven_mem_view_from_u8(PROVEN_LIT("ANSWER
42\n")), until);
    EXAMPLE_REQUIRE(w.err == PROVEN_OK && w.value == 10, "the server's reply is on its way");

    EXAMPLE_REQUIRE(ask(client, "the question", PROVEN_LIT("ANSWER 42")) == PROVEN_OK,
                    "the protocol function sends its line and reads the reply");

    /* 그리고 서버는 클라이언트가 쓴 것을 원시 호출로 읽는다. */
    proven_byte_t buf[64];
    proven_result_size_t r = proven_transport_read(server, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK &&
                    proven_u8str_view_eq((proven_u8str_view_t){ buf, r.value }, PROVEN_LIT("ASK the
question\n")),
                    "the request line arrived as written");

    /* write 한 번은 얼마나 갔는지 보고한다. 여기서는 전부다. */
    w = proven_transport_write(server, proven_mem_view_from_u8(PROVEN_LIT("BYE\n")), until);
    EXAMPLE_REQUIRE(w.err == PROVEN_OK && w.value == 4, "one more line");

    /* 끝내기와 닫기도 같은 인터페이스를 거친다. */
    EXAMPLE_REQUIRE(proven_transport_shutdown(server) == PROVEN_OK, "the server ends its sending side");
    r = proven_transport_read(client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_OK && r.value == 4, "the client reads the last line");
    r = proven_transport_read(client, (proven_mem_mut_t){ buf, sizeof buf }, until);
    EXAMPLE_REQUIRE(r.err == PROVEN_ERR_EOF, "and then the end");

    EXAMPLE_REQUIRE(proven_transport_close(client) == PROVEN_OK && proven_transport_close(server) ==
PROVEN_OK,
                    "closing a transport closes what carries it");
    EXAMPLE_REQUIRE(!proven_net_conn_is_open(&client_conn), "the connection underneath is closed");
```

```
    (void)proven_net_listener_close(&listener);
    return EXAMPLE_OK();
}
```

8. 짝, 그리고 루프 깨우기

`proven_net_poll`에 막혀 있는 루프에게는 소켓만 보이고 다른 것은 보이지 않는다. 그 루프를 위해 어떤 작업을 끝낸 스레드나 그 루프를 멈추고 싶은 스레드는 변수로는 루프에 닿을 수 없다. 루프는 운영체제 안에서 잠들어 있기 때문이다. 필요한 것은 다른 스레드가 말할 때 읽을 수 있게 되는 소켓이다. 그것이 깨우개(waker)다.

참조

API	의도	반환
`proven_net_pair(&a, &b)`	주소 없이 서로 이어진 두 연결. 한 쪽에 쓴 것이 다른 쪽에서 읽힌다.	`proven_err_t`: 시스템에 소켓이 모자라면 `BUSY`. 둘 다 닫아야 한다.
`proven_net_waker_open(&waker)`	깨우개를 연다.	`proven_err_t`: `BUSY`.
`proven_net_waker_handle(&waker)`	poll 목록에 `PROVEN_NET_READABLE`을 원한다고 넣을 핸들.	`proven_net_handle_t`.
`proven_net_waker_wake(&waker)`	읽을 수 있게 만든다. 어느 스레드에서 불러도, 여러 스레드가 동시에 불러도 안전하다.	없음. 호출자가 대응할 수 있는 방식으로는 실패하지 않는다.
`proven_net_waker_drain(&waker)`	지금까지의 깨우기를 읽어 없앤다. poll하는 스레드가 깨어난 뒤에 부른다.	없음.
`proven_net_waker_close(&waker)`	닫는다.	없음.

깨어난 뒤에는 비워라. 깨우개는 비울 때까지 읽을 수 있는 상태로 남는다. 이것을 잊은 루프는 poll하고, 곧장 돌아오고, poll하고, 곧장 돌아오며 프로세서를 헛돌린다.

깨우기는 합쳐진다. 비우기 한 번 전에 `proven_net_waker_wake`를 열 번 불러도 깨우기는 한 번이다. 깨우개는 "보라"고 말할 뿐 몇 번인지는 말하지 않는다. 그러니 루프가 봐야 할 것은 다른 곳 - 큐, 플래그 - 에 두고, 루프는 깨어날 때마다 그것을 확인한다.

플래그를 세운 다음 깨워라. 그리고 비운 다음에 보라. 비우기 전이 아니다. 순서가 반대면 보는 것과 비우는 것 사이에 깨우기가 끼어들어, 보이지 않은 채로 비워질 수 있다.

잘못된 예:

```
if (work_pending()) do_work();        /* wrong: work queued after this line... */
proven_net_waker_drain(&waker);       /* ...has its wake drained here, and waits for the next poll to time out
*/
```

올바른 예:

```
proven_net_waker_drain(&waker);
if (work_pending()) do_work();
```

맨 나중에 닫아라. 다른 스레드가 아직 `proven_net_waker_wake`를 부를 수 있는 동안에는 `proven_net_waker_close`가 안전하지 않다. 그 스레드들을 먼저 멈추거나 join하라.

`proven_net_pair`는 깨우개를 이루는 재료이고, 테스트나 프로그램이 네트워크 없이 연결 하나를 원하는 곳이면 어디서나 혼자서도 쓸모가 있다. POSIX에서는 유닉스 도메인 짝이고, Windows에서는 loopback TCP 연결이다. 지원하는 모든 Windows 버전에 있는 것이 그것뿐이기 때문이다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * proven_net_poll에 막혀 있는 루프에게는 소켓만 보인다. 다른 스레드에서 그 루프에게
 * 무언가를 알리려면 - "할 일이 생겼다", "멈춰라" - 여러분이 말할 때 읽을 수 있게 되는
```

```
 * 소켓이 필요하다. 그것이 깨우개(waker)다.
 *
 * proven_net_pair는 깨우개를 이루는 재료이고 그 자체로도 쓸모가 있다: 주소 없이 서로
 * 이어진 두 끝으로, 프로그램의 두 부분 사이에 바이트를 건넬 때 쓴다.
 */

static void finish_work(void *arg) {
    proven_net_waker_t *waker = arg;
    proven_time_sleep(20);              /* 어떤 작업을 대신하는 자리 */
    proven_net_waker_wake(waker);       /* 어느 스레드에서 불러도 된다 */
}

int main(void) {
    /* 짝: 한쪽 끝에 넣은 것이 다른 쪽 끝에서 나온다. */
    proven_net_conn_t a, b;
    proven_err_t err = proven_net_pair(&a, &b);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "a pair opens");
    EXAMPLE_REQUIRE(proven_net_write_all(&a, proven_mem_view_from_u8(PROVEN_LIT("across")),
proven_net_deadline_in(1000)).err == PROVEN_OK, "write to one end");
    proven_byte_t buf[16];
    proven_result_size_t got = proven_net_read(&b, (proven_mem_mut_t){ buf, sizeof buf },
proven_net_deadline_in(1000));
    EXAMPLE_REQUIRE(got.err == PROVEN_OK && got.value == 6, "read from the other");
    (void)proven_net_close(&a);
    (void)proven_net_close(&b);

    /* poll 목록에 든 깨우개. 읽을 것이 없으니 poll은 1초를 꼬박 기다릴 터인데... */
    proven_net_waker_t waker;
    EXAMPLE_REQUIRE(proven_net_waker_open(&waker) == PROVEN_OK, "a waker opens");
    proven_job_sys_t *jobs = NULL;
    EXAMPLE_REQUIRE(proven_job_system_init(proven_heap_allocator(), 1, 4, &jobs) == PROVEN_OK, "a worker
thread");
    EXAMPLE_REQUIRE(proven_job_submit_ex(jobs, finish_work, &waker) == PROVEN_OK, "work is handed to it");

    proven_net_poll_item_t item = { .handle = proven_net_waker_handle(&waker), .want = PROVEN_NET_READABLE };
    proven_size_t ready = 0;
    proven_time_t start = proven_time_monotonic_now();
    err = proven_net_poll(&item, 1, proven_net_deadline_in(1000), &ready);
    proven_i64 waited_ms = (proven_time_monotonic_now() - start) / 1000000;
    /* ...작업 스레드가 깨웠다. */
    EXAMPLE_REQUIRE(err == PROVEN_OK && ready == 1 && waited_ms < 900, "the poll returns when the worker says
so, not at its deadline");

    /* 비워야 한다. 그러지 않으면 계속 읽을 수 있는 상태로 남아 다음 poll이 아무 일 없이 곧장 돌아온다. */
    proven_net_waker_drain(&waker);
    EXAMPLE_REQUIRE(proven_net_poll(&item, 1, proven_net_deadline_in(20), &ready) == PROVEN_ERR_TIMEOUT,
"drained: quiet again");

    /* 비우기 전의 여러 번 깨우기는 한 번의 깨우기다. */
    proven_net_waker_wake(&waker);
    proven_net_waker_wake(&waker);
    proven_net_waker_drain(&waker);
    EXAMPLE_REQUIRE(proven_net_poll(&item, 1, proven_net_deadline_in(20), &ready) == PROVEN_ERR_TIMEOUT, "two
wakes, one drain: quiet");

    /* 깨우개는 어떤 스레드도 더는 깨울 수 없을 때에만 닫는다: 작업 스레드를 먼저 멈춘다. */
```

```
    proven_job_system_close(jobs);
    proven_job_system_destroy(jobs);
    proven_net_waker_close(&waker);
    return EXAMPLE_OK();
}
```

9. selector: 소켓 수천 개

`proven_net_poll`은 호출할 때마다 목록 전체를 건네받고, 시스템은 호출할 때마다 모든 항목을 살핀다. 소켓 수십 개라면 아무것도 아니다. 연결 만 개 가운데 다섯 개가 할 말이 있을 때에는, 그 다섯을 찾으려고 만 개 항목을 복사하고 살핀다 - 루프가 한 바퀴 돌 때마다.

selector는 같은 질문을 거꾸로 묻는다. 소켓을 한 번 등록하면 시스템이 기억하고, 기다리면 준비된 소켓만 돌아오며 나머지는 들여다보지 않는다.

참조

API	의도	반환
`proven_net_selector_create(alloc, &selector)`	selector를 만든다.	`proven_err_t`: `INVALID_ARG`. `NOMEM`. 디스크립터가 모자라면 `BUSY`.
`proven_net_selector_create_poll(alloc, &selector)`	시스템이 무엇을 제공하든 이식형 종류로 만든다.	같다.
`proven_net_selector_kind(selector)`	무엇 위에 지어졌는지.	`PROVEN_NET_SELECTOR_EPOLL`, `_KQUEUE`, `_IOCP`(Windows) 또는 `_POLL`.
`proven_net_selector_add(selector, handle, want, tag)`	소켓을 등록한다: 무엇을 지켜볼지, 그리고 이벤트와 함께 돌아올 포인터.	`proven_err_t`: 이미 등록되어 있으면 `EXISTS`. `INVALID_ARG`. `NOMEM`. `BUSY`.
`proven_net_selector_modify(selector, handle, want, tag)`	그 소켓에 묻는 것과 tag를 바꾼다.	`proven_err_t`: `NOT_FOUND`.
`proven_net_selector_remove(selector, handle)`	꺼낸다.	`proven_err_t`: `NOT_FOUND`.
`proven_net_selector_wait(selector, events, cap, until, &count)`	무언가 준비되거나 `until`이 될 때까지 기다리고, `cap`개까지 보고한다.	`proven_err_t`: 아무것도 없었으면 `TIMEOUT`(`count`는 0).
`proven_net_selector_count(selector)`	등록된 소켓 수.	`proven_size_t`.
`proven_net_selector_destroy(selector)`	해제한다. 소켓은 닫지 않는다.	없음.

```
typedef struct {
    void *tag;        /* what you registered with the socket */
    proven_u8 got;    /* PROVEN_NET_READABLE, PROVEN_NET_WRITABLE and/or PROVEN_NET_FAILED */
} proven_net_ready_t;
```

아무것도 묻지 않은 소켓도 연결을 잃으면 알려 준다 - 한 번. 시스템은 소켓에 대해 묻든 말든 오류와 끊김을 보고하고, 계속 보고한다. `want`를 0으로 등록한 소켓(멈춰 둔 연결)은 `PROVEN_NET_FAILED`로(끊김이면 읽기·쓰기 가능으로) 한 번 보고되고, `proven_net_selector_modify`로 묻는 것을 바꿀 때까지 그대로 둔다. 바꾸면 다시 보고된다. v0.38.0 전에는 기다릴 때마다 보고되어, 그런 소켓이 하나 있는 루프는 잠들지 못했다. (Linux와 이식형 종류에서 그렇다. `kqueue` 종류는 아무것도 묻지 않은 소켓에는 아무것도 등록하지 않으므로 무언가를 물을 때까지 보고하지 않는다. 그 종류는 실행해 보지 못했다.)

무엇을 얻는지는 시스템에 달려 있고, selector가 어느 쪽인지 말해 준다. Linux에서는 `epoll`이고 BSD와 macOS에서는 `kqueue`다. 거기서는 기다림의 비용이 준비된 소켓 수에 비례한다. Windows에서는 I/O 완료 포트다(`PROVEN_NET_SELECTOR_IOCP`): 연결의 읽기는 0바이트 수신으로 지켜보고, 쓸 자리, 받아들일 연결, 데이터그램, 아무것도 묻지 않은 동안 끊긴 연결은 시스템 스레드 풀의 알림으로 지켜본다. 알림은 묻는 동안 소켓마다 약 500바이트와 핸들 두 개가 들므로, 그 비용은 리스너, 데이터그램 소켓, 쓰기를 기다리는 연결, 아무것

도 묻지 않은 연결이 치르고 읽기를 기다리는 연결은 치르지 않는다. 기다릴 때마다 그렇게 보고된 것을 `proven_net_poll`이 쓰는 호출로 확인하므로, 이식형 종류가 보고할 것을 보고한다. 읽기를 지켜본 연결은 열려 있는 동안 그 포트에 묶여 있다. 그 밖의 곳에서는 이식형 종류다: 같은 인터페이스를 `proven_net_poll`이 쓰는 호출 위에 얹은 것으로, 기다릴 때마다 여전히 모든 소켓을 살핀다. 동작은 같고 확장성은 같지 않다. `proven_net_selector_kind`가 알려 준다. 수만 연결을 쥘 작정인 서버는 확인해야 한다.

소켓을 닫기 전에 꺼내라. 닫힌 소켓의 번호는 다음에 열리는 소켓에게 주어진다. 옛 등록을 아직 쥐고 있는 selector는 여러분이 등록한 것이 아닌 소켓의 이벤트를 - 옛 tag와 함께 - 보고하거나, 이식형 종류에서는 기다릴 때마다 실패를 보고한다.

잘못된 예:

```
proven_net_close(&conn);                                  /* wrong: the selector still holds its number */
proven_net_selector_remove(selector, proven_net_conn_handle(&conn));   /* too late: the handle is no longer
valid */
```

올바른 예 — `proven_net_selector_remove`를 먼저, 그다음 `proven_net_close`.

준비 상태는 레벨 트리거다. 소켓은 준비되어 있는 동안 계속 보고된다. `proven_net_poll`과 똑같다: 처리하지 않은 이벤트는 다음 기다림에 다시 온다. 그래서 §6의 규칙이 그대로 통한다 - 이벤트 뒤에는 `PROVEN_NET_DONT_WAIT`를 받는 호출을 하고, 쓸 것이 있는 동안에만 쓰기 가능을 물어라.

tag는 여러분 것이고, 그 수명도 그렇다. selector는 받은 포인터를 돌려줄 뿐 그 안을 들여다보지 않는다. 그것이 가리키는 기록이, 그에 대한 이벤트가 이미 selector에서 꺼내졌을 수 있는 동안에 해제되면, 루프의 다음 반복이 해제된 메모리를 읽는다. 이것을 피하는 방식: 이벤트 묶음을 처리하는 도중에 연결을 닫을 때에는 그 기록에 표시만 하고, 묶음이 끝난 뒤에 해제한다.

스레드 하나. selector는 여러 스레드가 쓰기에 안전하지 않다. 기다리는 루프가 다른 스레드가 한 일에 반응하게 하려면 깨우개(§8)를 등록하고 깨워라.

양쪽으로 준비된 소켓은 이벤트 하나일 수도 둘일 수도 있다. `kqueue`는 읽기 가능과 쓰기 가능을 따로 보고한다. 소켓마다 하나를 기대하지 말고 이벤트 하나하나를 그것이 말하는 대로 처리하라.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * selector: 소켓을 한 번씩 등록해 두고, 준비된 것에 대해서만 통지받는다.
 *
 * 루프는 poll 예제의 그것이다 - 기다리고, 준비된 것을 기다리지 않고 처리한다 - . 다만 기다림과
 * 기다림 사이에 다시 짓는 것이 없고, 한가한 연결 천 개가 기다림에 아무 비용도 지우지 않는다.
 */

typedef struct {
    proven_net_conn_t conn;
    int number;
    bool open;
} ex_client_t;

int main(void) {
    proven_net_listener_t listener;
    proven_net_addr_t at;
    proven_err_t err = proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 16, &listener,
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the listener opens");

    proven_net_selector_t *selector = NULL;
    EXAMPLE_REQUIRE(proven_net_selector_create(proven_heap_allocator(), &selector) == PROVEN_OK, "a
```

```
selector");
    /* Linux에서는 epoll, BSD와 macOS에서는 kqueue. 그 밖에서는 poll 위의 같은 인터페이스. */
    proven_net_selector_kind_t kind = proven_net_selector_kind(selector);
    printf("selector kind: %s\n", kind == PROVEN_NET_SELECTOR_EPOLL ? "epoll" : kind ==
PROVEN_NET_SELECTOR_KQUEUE ? "kqueue" : kind == PROVEN_NET_SELECTOR_IOCP ? "iocp" : "poll");

    /* tag는 이벤트와 함께 돌아오는 것이다: 그 소켓에 대한 여러분 자신의 기록을 가리키는 포인터.
     * 여기서 리스너의 tag는 리스너 자신이다. */
    EXAMPLE_REQUIRE(proven_net_selector_add(selector, proven_net_listener_handle(&listener),
PROVEN_NET_READABLE, &listener) == PROVEN_OK,
                    "the listener is registered, once");

    /* 클라이언트 셋이 연결하고, 둘째와 셋째가 무언가를 말한다. */
    proven_net_conn_t far[3];
    for (int i = 0; i < 3; ++i) EXAMPLE_REQUIRE(proven_net_connect(at, proven_net_deadline_in(5000), &far[i])
== PROVEN_OK, "a client connects");
    EXAMPLE_REQUIRE(proven_net_write_all(&far[1], proven_mem_view_from_u8(PROVEN_LIT("from one")),
proven_net_deadline_in(5000)).err == PROVEN_OK &&
                    proven_net_write_all(&far[2], proven_mem_view_from_u8(PROVEN_LIT("from two")),
proven_net_deadline_in(5000)).err == PROVEN_OK, "two of them send");

    ex_client_t clients[3] = {0};
    int accepted = 0, messages = 0;
    for (int round = 0; round < 20 && messages < 2; ++round) {
        proven_net_ready_t ready[8];
        proven_size_t count = 0;
        err = proven_net_selector_wait(selector, ready, 8, proven_net_deadline_in(1000), &count);
        if (err == PROVEN_ERR_TIMEOUT) continue;        /* 루프의 한가한 한 박자 */
        EXAMPLE_REQUIRE(err == PROVEN_OK, "something is ready");
        for (proven_size_t i = 0; i < count; ++i) {
            if (ready[i].tag == &listener) {
                /* 준비됨은 "기다리지 않는다"는 뜻이다: 대기 중인 연결을 기다리지 않고 모두 받는다. */
                while (accepted < 3 && proven_net_accept(&listener, PROVEN_NET_DONT_WAIT,
&clients[accepted].conn, NULL) == PROVEN_OK) {
                    ex_client_t *c = &clients[accepted];
                    c->number = accepted++;
                    c->open = true;
                    EXAMPLE_REQUIRE(proven_net_selector_add(selector, proven_net_conn_handle(&c->conn),
PROVEN_NET_READABLE, c) == PROVEN_OK,
                                    "each accepted connection is registered, with its record as the tag");
                }
            } else {
                ex_client_t *c = ready[i].tag;
                proven_byte_t buf[32];
                proven_result_size_t got = proven_net_read(&c->conn, (proven_mem_mut_t){ buf, sizeof buf },
PROVEN_NET_DONT_WAIT);
                if (got.err == PROVEN_OK) messages++;
            }
        }
    }
    EXAMPLE_REQUIRE(accepted == 3 && messages == 2, "three connections accepted, two messages read");
    EXAMPLE_REQUIRE(proven_net_selector_count(selector) == 4, "four sockets registered: the listener and three
connections");

    /* 첫 연결은 말한 적이 없어서 보고된 적도 없다: 해 줄 일이 없었다. 이제 그 연결에 다른 것을
     * 묻는다 - 써도 되는가? - 그러면 곧장 보고된다. */
    proven_net_ready_t one[4];
    proven_size_t count = 0;
    EXAMPLE_REQUIRE(proven_net_selector_wait(selector, one, 4, proven_net_deadline_in(30), &count) ==
PROVEN_ERR_TIMEOUT, "all quiet: nothing is reported");
```

```
    EXAMPLE_REQUIRE(proven_net_selector_modify(selector, proven_net_conn_handle(&clients[0].conn),
PROVEN_NET_WRITABLE, &clients[0]) == PROVEN_OK, "ask about writing instead");
    EXAMPLE_REQUIRE(proven_net_selector_wait(selector, one, 4, proven_net_deadline_in(1000), &count) ==
PROVEN_OK && count == 1 &&
                    one[0].tag == &clients[0] && (one[0].got & PROVEN_NET_WRITABLE), "an idle connection is
writable");
    /* 쓸 것이 있는 동안에만 쓰기 가능을 물어라. 그러지 않으면 모든 기다림이 곧장 돌아온다. */
    EXAMPLE_REQUIRE(proven_net_selector_modify(selector, proven_net_conn_handle(&clients[0].conn),
PROVEN_NET_READABLE, &clients[0]) == PROVEN_OK, "back to reading");

    /* 소켓은 닫기 "전에" 꺼낸다. */
    for (int i = 0; i < 3; ++i) {
        EXAMPLE_REQUIRE(proven_net_selector_remove(selector, proven_net_conn_handle(&clients[i].conn)) ==
PROVEN_OK, "removed");
        (void)proven_net_close(&clients[i].conn);
        (void)proven_net_close(&far[i]);
    }
    EXAMPLE_REQUIRE(proven_net_selector_remove(selector, proven_net_listener_handle(&listener)) == PROVEN_OK
&&
                    proven_net_selector_count(selector) == 0, "and the listener: the selector is empty");
    (void)proven_net_listener_close(&listener);

    /* 이식형 종류를 이름으로 요청할 수 있다 - 둘을 비교하거나, 더 나은 것이 없는 시스템의
     * 코드 경로를 돌려 보려고. */
    proven_net_selector_t *portable = NULL;
    EXAMPLE_REQUIRE(proven_net_selector_create_poll(proven_heap_allocator(), &portable) == PROVEN_OK &&
                    proven_net_selector_kind(portable) == PROVEN_NET_SELECTOR_POLL, "a selector of the poll
kind");
    proven_net_selector_destroy(portable);
    proven_net_selector_destroy(selector);
    return EXAMPLE_OK();
}
```

10. 플랫폼마다 다른 것

이 호출들은 Linux, BSD, macOS, Windows에서 똑같이 동작한다. 아래는 라이브러리가 흡수하는 차이들이다. 소켓을 직접 쓸 때 발목을 잡는 것들이라 적어 둔다.

함정	net.h가 하는 일
닫힌 연결에 쓰면 SIGPIPE가 발생해 프로세스가 죽는다(POSIX).	모든 send가 그 시그널을 억제한다. write는 PROVEN_ERR_RESET을 돌려준다. 여러분 대신 시그널 핸들러를 설치하지 않는다.
Winsock은 이름 해석을 포함해 어떤 호출보다도 먼저 시작돼야 한다.	첫 사용 때 한 번 시작한다.
Windows에서 SO_REUSEADDR은 다른 프로세스가 여러분의 포트에 바인드해 연결을 가져가게 한다.	리스너는 POSIX에서 SO_REUSEADDR을, Windows에서 SO_EXCLUSIVEADDRUSE를 쓴다. 한 주소의 두 번째 리스너는 어디서나 PROVEN_ERR_BUSY다.
IPv6 리스너가 어떤 시스템에서는 IPv4도 받고 어떤 시스템에서는 받지 않는다.	IPv6 소켓은 어디서나 IPv6 전용이다. 두 프로토콜을 다 받는 서버는 리스너를 둘 연다.
Windows의 논블로킹 send는 어떤 크기의 버퍼든 받아서 전부 큐에 넣는다.	send는 한정된 조각으로 내놓는다. 그래서 가득 찬 버퍼가 다른 곳에서처럼 밀어내고, 기한이 작동할 수 있다.
Windows는 상대의 정상 종료를 WSAPoll에 읽기 가능이 아니라 끊김으로 보고한다.	POSIX처럼 읽기 가능으로 보고한다. 뒤따르는 read는 PROVEN_ERR_EOF를 돌려준다.

닫힌 포트로 보낸 UDP가 같은 Windows 소켓의 나중 받기를 실패하게 만든다.	그 보고를 끈다. 받기는 자기가 받은 데이터그램에 대해서만 답한다.
자식 프로세스가 열린 소켓을 물려받는다.	모든 소켓은 상속되지 않게(close-on-exec) 만든다.
Winsock은 없는 경로에 대한 유닉스 도메인 connect를 "거절"로 답한다.	POSIX처럼 `PROVEN_ERR_NOT_FOUND`다.

두 가지는 다르고, 다른 채로 남는다. Windows는 거절된 루프백 연결을 보고하기까지 약 2초를 기다리고, POSIX는 즉시 답한다 - 기한에 그만큼을 감안하라. 그리고 유닉스 도메인 소켓에는 Windows 10 버전 1803 이상이 필요하다. 그 family가 없는 곳에서 `proven_net_listen`과 `proven_net_connect`는 `PROVEN_ERR_UNSUPPORTED`를 돌려준다.

11. 여기에 없는 것

- **암호화.** 여기서 만든 연결은 암호화되지 않는다. TLS는 전송 인터페이스 위의 한 층이다: 14장.
- **이름 해석의 기한.** §2를 보라.
- **기계 한 대를 넘는 Windows의 규모.** 거기서 selector와 15장의 이벤트 루프는 I/O 완료 포트 위에 지어졌다. Windows 11 가상 머신 한 대에서 루프백으로 한 번 쟀다: 연결 10,000개를 유지하고 열 개가 요청할 때, 이 selector 위의 11장 블로킹 서버는 초당 약 52,000 요청에 왕복 중앙값 0.15 ms로 답했다 - 1,000개를 유지할 때와 같다. 이벤트 루프의 수치는 15장 14절에 있다. 거기서 재지 않은 것: 다른 기계의 클라이언트, selector에 등록한 데이터그램 소켓, 그리고 약 16,000개가 넘는 연결(양쪽 끝을 한 기계에 두니 포트가 바닥났다).
- **유닉스 도메인 데이터그램, raw 소켓, 멀티캐스트, `TCP_NODELAY` 외의 소켓 옵션.**
- **HTTP.** 메시지 코덱은 10장에 있다. 그것을 이 소켓 위에서 구동하는 클라이언트와 서버는 11장에 있다.

10장: URL과 HTTP 메시지

5부 — 운영체제와 대화하기. 선행 조건: 2부 (1, 2, 3). 바이트가 오는 곳은 9장이지만, 여기 있는 것은 9장 없이도 쓸 수 있다. 이 장을 마치면 URL을 나누고, 요청 경로를 열어도 안전한 경로로 바꾸고, HTTP/1.1 요청이나 응답을 도착하는 대로 파싱하고, 본문이 얼마나 긴지 알아내고, 그 본문을 디코딩하고, 남의 입력이 망가뜨릴 수 없는 메시지를 쓸 수 있다. 그리고 메시지 둘레에 놓이는 조각들 - 리다이렉트 풀기, 폼, 바이트 범위, 파일 올리기, 인증 응답, 쿠키 보관함, 이벤트 스트림 - 을 갖게 된다.

이 장은 `url.h`, `http.h`, `http_auth.h`, `http_cookie.h`, `sse.h`를 다룬다. 모두 소켓도 파일도 없는 텍스트 처리다. 할당하는 것은 쿠키 보관함뿐이고, 그것도 여러분이 준 할당자(allocator)에서 한다. 9장과 달리 이 헤더들은 프리스탠딩(freestanding) 빌드에서 쓸 수 있고, `PROVEN_NO_NET`으로 빠지지 않는다. 이것들을 소켓 위에서 일하게 하는 클라이언트와 서버는 11장에 있다.

목차

1. 서버가 아니라 코덱
2. URL
3. 써도 안전한 경로
4. 헤드 파싱
5. 본문은 얼마나 긴가
6. 본문 디코딩
7. 메시지 쓰기
8. 날짜
9. 참조와 폼
10. 범위와 multipart 본문
11. 인증
12. 쿠키
13. 서버 전송 이벤트
14. 여기에 없는 것

1. 서버가 아니라 코덱

`http.h`는 어디에서도 읽지 않고 어디에도 쓰지 않는다. 이미 가진 바이트를 건네면 그것이 무슨 말인지 알려주고, 메모리를 건네면 거기에 메시지를 쓴다. 그 바이트를 무엇이 실어 나르는지 - 블로킹 소켓, 이벤트 루프, TLS 세션, 테스트 - 는 이 헤더의 일이 아니고, 그것이 요점이다. HTTP에서 제대로 하기 어려운 부분은 그 모두에서 같고, 여기서 한 번만 쓰였다.

두 가지 성질이 따라 나오고, 이 장의 나머지가 둘에 기댄다.

접두사는 결코 오류가 아니다. 바이트는 네트워크가 나눈 조각대로 도착한다. 지금까지 온 것이 올바른 메시지의 앞부분이라면 파서는 `PROVEN_ERR_NEED_MORE`를 답한다. 실패도 아니고 섣부른 성공도 아니다. 그래서 그 둘레의 루프는 언제나 같다. 파싱하고, 더 필요하다면 더 읽고, 아니면 행동한다.

모호함은 해소하지 않고 거절한다. HTTP 메시지는 흔히 두 프로그램이 읽는다 - 앞단의 프록시와 뒷단의 서버. 둘이 같은 바이트를 다르게 읽는 곳마다, 하나에게는 다른 하나가 보지 못하는 요청을 보여 줄 수 있다. 그것이 *요청 밀반입(request smuggling)*이고, 너그러운 파서가 조용히 해석 하나를 고르는 자리에만 산다. 이 파서는 고르지 않는다.

거절하는 것	왜 중요한가
LF만, 또는 CR만 있는 줄 끝	한 파서는 한 줄로, 다른 파서는 두 줄로 본다
헤더 이름과 콜론 사이의 공백	`Content-Length : 5`는 한 파서에게는 헤더이고 다른 파서에게는 아니다
공백이나 탭으로 시작하는 헤더 줄(폐기된 줄 접기)	한 파서에게는 이어지는 줄, 다른 파서에게는 새 필드

둘 이상의 `Content-Length`, 또는 순수한 숫자가 아닌 값	길이 둘은 본문이 끝나는 곳에 대한 의견 둘이다
`Transfer-Encoding`과 `Content-Length`가 함께	고전적인 경우: 파서마다 다른 쪽을 믿는다
단일 `chunked`가 아닌 모든 전송 코딩	모르는 코딩을 건너뛰는 파서는 본문을 다르게 자른다
공백, 부호, `0x`가 붙거나 16자리를 넘는 청크 크기	청크 크기 파싱이 또 하나의 고전이다

각 행은 배포된 소프트웨어가 받아 주고, 공개된 공격이 쓰는 것이다. 거절은 평범한 오류 값이다. 400으로 답하고 닫는다.

2. URL

`proven_url_parse`는 절대 URL을 여러분이 준 텍스트를 가리키는 뷰(view)들로 나눈다.

```
https://user:pw@example.com:8443/docs/a%20b?q=1#top
\___/   \_____/ \_________/ \__/\_________/ \_/ \_/
scheme  userinfo    host    port   path    query fragment
```

```
typedef struct {
    proven_u8str_view_t scheme, userinfo, host, path, query, fragment;
    proven_u16 port;
    bool has_userinfo, has_port, has_query, has_fragment;
    bool host_is_ipv6;      /* written in brackets; `host` is without them */
} proven_url_t;
```

파싱은 디코딩하지 않는다. 모든 구성 요소는 퍼센트 기호까지 적힌 그대로 돌아온다. 디코딩은 지금 쓰려는 조각 하나에 대해 따로 하는 걸음이다. 통째로 디코딩한 URL은 다시 나눌 수 없기 때문이다. 경로 세그먼트 안의 데이터였던 `%2F`가 세그먼트를 나누는 `/`가 되고, 쿼리 값 안의 `%26`이 그 값을 끝내는 `&`가 된다.

참조

API	의도	반환
`proven_url_parse(text, &url)`	`scheme://authority/path?query#fragment`를 나눈다. ASCII만, 공백 없이, 모든 `%` 뒤에 hex 두 자리.	`proven_err_t`: 그 밖의 것은 `INVALID_FORMAT`. 그때 `url`은 그대로다.
`proven_url_default_port(scheme)`	`http`/`ws`는 80, `https`/`wss`는 443, `ftp`는 21.	`proven_u16`. 모르는 스킴은 0.
`proven_url_effective_port(&url)`	연결할 포트: 적힌 것, 없으면 스킴의 것.	`proven_u16`.
`proven_url_scheme_is(&url, scheme)`	스킴을 대소문자 구분 없이 비교한다.	`bool`.
`proven_url_split_target(target, &path, &query, &has_query)`	HTTP 요청 타깃을 나눈다: `/a?b`, `http://h/a?b`, 또는 `*`.	`proven_err_t`: 그 밖에는 `INVALID_FORMAT`.
`proven_url_query_iter(query)`, `proven_url_query_next(&it, &name, &value)`	`name=value&...`를 걸으며 인코딩된 뷰를 낸다. 빈 쌍은 건너뛴다.	반복자. `bool`(끝에서 false).
`proven_url_percent_decode(in, out, &written)`	`%XX`를 바이트로. `out`이 `in`이어도 된다. `+`는 `+`로 남는다.	`proven_err_t`: 잘못된 이스케이프는 `INVALID_FORMAT`. `OUT_OF_BOUNDS`.
`proven_url_form_decode(in, out, &written)`	같은 일을 하고, `+`를 공백으로 읽는다: 쿼리의 이름과 값에 쓴다.	`proven_err_t`.
`proven_url_encode_component(in, out, &written)`	구성 요소 *하나*를 위해 인코딩한다: 글자, 숫자, `-._~` 외에는 모두 `%XX`가 된다.	`proven_err_t`: `OUT_OF_BOUNDS`. `OVERFLOW`.

`proven_url_encode_path(in, out, &written)`	같은 일을 하되 `/`는 둔다.	`proven_err_t`.
`proven_url_form_encode(in, out, &written)`	같은 일을 하되 공백을 `+`로 쓴다.	`proven_err_t`.
`proven_url_encoded_size_max(n)`	`n`바이트를 인코딩하기에 언제나 충분한 크기.	`proven_size_t`. 표현할 수 없으면 `SIZE_MAX`.

이것은 RFC 3986이고, 브라우저가 구현하는 URL 알고리즘이 아니다. 브라우저는 받은 것을 고쳐 준다 - 역슬래시를 슬래시로, 공백을 인코딩하고, 없는 스킴을 추측한다. 여기서는 아무것도 고치지 않는다. 고쳐야 할 텍스트는 `PROVEN_ERR_INVALID_FORMAT`이다. 사람이 타이핑한 URL을 받는 프로그램은 추측하는 대신 그렇다고 알려 줘야 한다.

주의사항, 그리고 무엇이 잘못되는가
구성 요소를 디코딩하라. URL 전체는 절대 아니다.

잘못된 예:

```
proven_url_percent_decode(whole_url, buf, &n);      /* wrong: "a%2Fb" and "a/b" are now the same path */
proven_url_parse(decoded, &url);
```

올바른 예 — 먼저 파싱하고, 필요한 구성 요소 하나를 그것에 맞는 디코더로 디코딩한다. 쿼리의 이름이나 값에는 `proven_url_form_decode`, 경로에는 `proven_url_path_resolve`.

디코딩된 텍스트는 바이트다. `proven_url_percent_decode`는 인코딩돼 있던 것을 그대로 돌려준다. NUL, 줄바꿈, UTF-8이 아닌 바이트. 보낸 쪽의 데이터다. 그것으로 하려는 일에 맞게 검증하라.

userinfo는 호스트가 아니다. `http://example.com@evil.test/`의 호스트는 `evil.test`다. 파서는 이것을 옳게 처리한다 - 호스트는 마지막 `@` 뒤에서 시작한다 - 하지만 URL을 사람에게 보여 주거나 허용 목록과 대조하는 프로그램은 텍스트의 앞부분이 아니라 `url.host`를 비교해야 한다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * URL: 나누고, 지금 쓰려는 조각만 디코딩하고, 네트워크에서 온 경로는 해소하기 전에는
 * 파일 시스템에 닿게 하지 않는다.
 */

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    /* 파싱은 적힌 그대로를 가리키는 뷰를 준다. 디코딩하는 것도 복사하는 것도 없다. */
    proven_url_t url;
    proven_err_t err = proven_url_parse(PROVEN_LIT("https://example.com:8443/docs/a%20b?q=caf%C3%A9+au+lait&
page=2#top"), &url);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "an absolute URL parses");
    EXAMPLE_REQUIRE(view_is(url.host, "example.com") && url.has_port && url.port == 8443, "host and port");
    EXAMPLE_REQUIRE(view_is(url.path, "/docs/a%20b"), "the path is still encoded");
    EXAMPLE_REQUIRE(url.has_query && url.has_fragment && view_is(url.fragment, "top"), "query and fragment are
there");

    /* 스킴 이름은 대소문자 구분 없이 비교한다. 포트가 없는 URL은 스킴의 기본 포트를 얻는다. */
    proven_url_t plain;
    EXAMPLE_REQUIRE(proven_url_parse(PROVEN_LIT("HTTP://example.com/"), &plain) == PROVEN_OK, "another URL");
    EXAMPLE_REQUIRE(proven_url_scheme_is(&plain, PROVEN_LIT("http")), "HTTP is http");
    EXAMPLE_REQUIRE(proven_url_effective_port(&plain) == 80 && proven_url_effective_port(&url) == 8443,
                    "the port to connect to: the default, or the one written");
    EXAMPLE_REQUIRE(proven_url_default_port(PROVEN_LIT("wss")) == 443, "wss is 443");
```

```
    /* 절대 URL이 아닌 것은 고쳐 주지 않고 거절한다. */
    proven_url_t bad;
    EXAMPLE_REQUIRE(proven_url_parse(PROVEN_LIT("example.com/path"), &bad) == PROVEN_ERR_INVALID_FORMAT, "no
scheme: refused");
    EXAMPLE_REQUIRE(proven_url_parse(PROVEN_LIT("http://example.com/a b"), &bad) == PROVEN_ERR_INVALID_FORMAT,
"a raw space: refused");

    /* 쿼리는 쌍 단위로 걷는다. 이름과 값을 각각 따로, '+'가 공백을 뜻하는 폼 데이터로
     * 디코딩한다. 쿼리 전체를 먼저 디코딩하면 값 안의 인코딩된 '&'가 구분자로
     * 바뀌어 버린다. */
    proven_url_query_iter_t it = proven_url_query_iter(url.query);
    proven_u8str_view_t name, value;
    proven_byte_t text[64];
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_url_query_next(&it, &name, &value) && view_is(name, "q"), "the first pair is q");
    EXAMPLE_REQUIRE(proven_url_form_decode(value, (proven_mem_mut_t){ text, sizeof text }, &n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ text, n }, "caf\xc3\xa9 au lait"), "its value, decoded");
    EXAMPLE_REQUIRE(proven_url_query_next(&it, &name, &value) && view_is(name, "page") && view_is(value, "2"),
"the second pair");
    EXAMPLE_REQUIRE(!proven_url_query_next(&it, &name, &value), "and no third");

    /* 일반 퍼센트 디코딩은 '+'를 그대로 두고, *바이트*를 낸다 - 어떤 바이트든. */
    EXAMPLE_REQUIRE(proven_url_percent_decode(PROVEN_LIT("a%2Fb+c"), (proven_mem_mut_t){ text, sizeof text },
&n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ text, n }, "a/b+c"), "%2F becomes a slash; '+' stays");

    /* 반대 방향: 값이 자기가 들어갈 구성 요소를 끝낼 수 없도록 인코딩한다. */
    proven_mem_view_t file_name = proven_mem_view_from_u8(PROVEN_LIT("report 100%/final?.txt"));
    proven_byte_t enc[96];
    EXAMPLE_REQUIRE(proven_url_encoded_size_max(file_name.size) <= sizeof enc, "three bytes per byte is always
enough");
    EXAMPLE_REQUIRE(proven_url_encode_component(file_name, (proven_mem_mut_t){ enc, sizeof enc }, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ enc, n }, "report%20100%25%2Ffinal%3F.txt"), "one path
segment: even the slash is encoded");
    EXAMPLE_REQUIRE(proven_url_encode_path(proven_mem_view_from_u8(PROVEN_LIT("/my docs/a b.txt")),
(proven_mem_mut_t){ enc, sizeof enc }, &n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ enc, n }, "/my%20docs/a%20b.txt"), "a whole path: the
slashes stay");
    EXAMPLE_REQUIRE(proven_url_form_encode(proven_mem_view_from_u8(PROVEN_LIT("a b&c")), (proven_mem_mut_t)
{ enc, sizeof enc }, &n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ enc, n }, "a+b%26c"), "form data: a space is '+'");

    /* 서버 쪽에서 보면. 요청 타깃을 먼저 나누고... */
    proven_u8str_view_t path, query;
    bool has_query = false;
    EXAMPLE_REQUIRE(proven_url_split_target(PROVEN_LIT("/static/css/../img/logo%20v2.png?v=3"), &path, &query,
&has_query) == PROVEN_OK &&
                    has_query && view_is(query, "v=3"), "target into path and query");

    /* ...경로는 다른 무엇이 보기 전에 해소한다: 한 번 디코딩하고, 점 세그먼트를
     * 없애고, 루트를 벗어나려 하면 거절한다. */
    proven_byte_t clean[128];
    EXAMPLE_REQUIRE(proven_url_path_resolve(path, (proven_mem_mut_t){ clean, sizeof clean }, &n) == PROVEN_OK
&&
                    view_is((proven_u8str_view_t){ clean, n }, "/static/img/logo v2.png"), "a clean path under
the root");

    /* 파일 서버가 거절하려고 존재하는 요청들. */
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/../etc/passwd"), (proven_mem_mut_t){ clean, sizeof
```

```
clean }, &n) == PROVEN_ERR_PERMISSION,
                    "climbing above the root");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/%2e%2e/etc/passwd"), (proven_mem_mut_t){ clean,
sizeof clean }, &n) == PROVEN_ERR_PERMISSION,
                    "the same climb with the dots encoded");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/..%2f..%2fetc/passwd"), (proven_mem_mut_t){ clean,
sizeof clean }, &n) == PROVEN_ERR_INVALID_FORMAT,
                    "an encoded slash");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/file%00.png"), (proven_mem_mut_t){ clean, sizeof
clean }, &n) == PROVEN_ERR_INVALID_FORMAT,
                    "an encoded NUL");
    EXAMPLE_REQUIRE(proven_url_path_resolve(PROVEN_LIT("/%c0%ae%c0%ae/"), (proven_mem_mut_t){ clean, sizeof
clean }, &n) == PROVEN_ERR_INVALID_FORMAT,
                    "an overlong encoding of a dot");

    return EXAMPLE_OK();
}
```

3. 써도 안전한 경로

요청 경로는 파일을 가리키고, 그것을 고르는 쪽은 보낸 쪽이다. `GET /../../etc/passwd`는 파일 서버에 대한 가장 오래된 공격이고 표기가 여럿이다. `proven_url_path_resolve`는 요청 경로와 파일을 여는 모든 것 사이에 둘 단 하나의 함수다.

API	의도	반환
`proven_url_path_resolve(raw_path, out, &written)`	한 번 디코딩하고, 파일 이름에 있어서는 안 될 것을 거절하고, `.`과 `..`을 해소한다. 결과는 `/`로 시작하고, 점 세그먼트와 빈 세그먼트가 없으며, 끝의 `/`는 유지된다. `out`은 많아야 `raw_path.size`바이트면 된다.	`proven_err_t`: `..`이 루트 위로 올라가려 하면 `PERMISSION`. 아래의 거절들은 `INVALID_FORMAT`. `out`이 작으면 `OUT_OF_BOUNDS`.

하는 일을 순서대로 적는다. 그 순서가 곧 보안 성질이다.

1. **한 번 디코딩한다.** `%2e%2e`는 `..`이 되고, 그다음 `..`으로 취급된다. 그리고 `%252e%252e`는 `%2e%2e`라는 텍스트가 되며, 이상한 파일 이름일 뿐 그 이상이 아니다 - 한 번 더 디코딩하면 경로 이탈이 되는데, 그것이 이중 디코딩 버그다.
2. **거절한다.** 인코딩된 슬래시(`%2F` - 나중에 세그먼트 하나를 둘로 쪼갠다), 적혔든 인코딩됐든 역슬래시(Windows에서는 구분자다), NUL을 포함한 모든 제어 문자(`file%00.png`는 C에서 이름을 자른다), 그리고 디코딩한 뒤 올바른 UTF-8이 아닌 것 - 일부 디코더가 받아 주는 점의 과장(overlong) 인코딩 `%c0%ae`가 여기서 걸린다.
3. **해소한다.** 디코딩된 텍스트에서 `.`과 `..`을. 더 지울 것이 없는데 나온 `..`은 `PROVEN_ERR_PERMISSION`이다. 경로는 형식에 맞고, 가져서는 안 될 것을 요구한다.

잘못된 예 — 디코딩 전에 `..`을 검사하기:

```
if (contains(raw_path, "..")) return 400;         /* wrong: "%2e%2e" sails through */
proven_url_percent_decode(raw_path, buf, &n);
open_under_root(buf, n);
```

잘못된 예 — "안전하게" 두 번 디코딩하기:

```
proven_url_percent_decode(raw, a, &n);
proven_url_percent_decode(a_view, b, &m);          /* wrong: "%252e%252e" has just become ".." */
```

올바른 예:

```
proven_err_t err = proven_url_path_resolve(raw_path, clean_mem, &n);
if (err == PROVEN_ERR_PERMISSION)     return 403;   /* or 400 */
```

```
if (err != PROVEN_OK)                return 400;
open_under_root(clean, n);
```

하지 않는 일. 경로 *문법*으로 일어나는 이탈을 막는다. 여러분의 파일 시스템은 모른다. 루트 안의 심볼릭 링크는 여전히 밖을 가리킬 수 있고, Windows는 일부 파일 이름에 고유한 뜻을 준다(`CON`, 끝의 점, `name:stream`). 그것들은 파일을 여는 자리에서 검사하라. 위 예제는 해소 함수를 보여 주고, 파일을 여는 절반은 5장의 것이다.

4. 헤드 파싱

*헤드*는 본문 앞의 모든 것이다. 요청 줄이나 상태 줄, 헤더 필드들, 그리고 그것을 끝내는 빈 줄.

```
typedef struct { proven_u8str_view_t name, value; } proven_http_header_t;

typedef struct {
    proven_http_method_t method;       /* GET, POST, ...; OTHER for a token the enum has no name for */
    proven_u8str_view_t method_text;
    proven_u8str_view_t target;        /* "/a?b", "http://h/a", "*", or "host:port" */
    proven_u8 version_minor;           /* 1 or 0 */
    proven_http_header_t *headers;     /* the array you supplied */
    proven_size_t header_count;
} proven_http_request_t;

typedef struct {
    proven_u16 status;
    proven_u8str_view_t reason;        /* may be empty; means nothing to a program */
    proven_u8 version_minor;
    proven_http_header_t *headers;
    proven_size_t header_count;
} proven_http_response_t;
```

모든 뷰는 여러분이 파싱한 버퍼 안을 가리킨다. 복사되는 것이 없으므로, 버퍼는 파싱된 헤드보다 오래 살아야 하고 움직여서는 안 된다. 필드를 담을 배열도 여러분의 것이다.

참조

API	의도	반환
`proven_http_parse_request(data, headers, cap, max_head, &req, &head_size)`	`data` 앞쪽에서 요청 헤드를 파싱한다. `head_size`는 본문이 시작하는 자리다. `max_head`가 0이면 `PROVEN_HTTP_DEFAULT_MAX_HEAD`(16 KiB).	`proven_err_t`: `NEED_MORE`(더 읽어라). `OUT_OF_BOUNDS`(헤드가 한도를 넘거나 필드가 `cap`보다 많음: 431). `INVALID_FORMAT`(400). `UNSUPPORTED`(HTTP/1.0도 1.1도 아님: 505).
`proven_http_parse_response(data, headers, cap, max_head, &res, &head_size)`	응답에 대해 같은 일을 한다.	위와 같다.
`proven_http_header_find(headers, count, name, &value)`	그 이름의 첫 필드. 대소문자 구분 없음. `value`는 NULL이어도 된다.	`bool`.
`proven_http_header_count(headers, count, name)`	그 이름의 필드가 몇 개인가.	`proven_size_t`.
`proven_http_header_has_token(headers, count, name, token)`	그런 필드 중 쉼표로 나뉜 값에 `token`이 든 것이 있는가: `Connection: keep-alive, Upgrade`에는 `upgrade`가 있다.	`bool`.

`proven_http_method_from_text(text)`, `proven_http_method_text(method)`	메서드 토큰과 enum 사이를 오간다. 대소문자를 구분한다: `get`은 `GET`이 아니다.	enum. 뷰(`OTHER`면 비어 있음).
`proven_http_reason_phrase(status)`	표준 문구: `Not Found`.	뷰. 문구가 없는 코드는 `Unknown`.
`proven_http_request_keep_alive(&req)`, `proven_http_response_keep_alive(&res)`	연결이 다음 메시지를 실을 수 있는가: HTTP/1.1은 `Connection: close`가 없는 한, HTTP/1.0은 `Connection: keep-alive`가 있을 때만.	`bool`.

한도는 헤드가 아직 미완성일 때 검사한다. `max_head`바이트가 도착했는데 그 안에 헤드의 끝이 없으면 답은 `NEED_MORE`가 아니라 즉시 `PROVEN_ERR_OUT_OF_BOUNDS`다. 헤더 줄을 끝없이 보내는 클라이언트는 `max_head`바이트만큼만 비용을 치르게 한다.

요청 줄 앞의 빈 줄은 건너뛰고(클라이언트는 오래전부터 본문 뒤에 남는 CRLF를 보내 왔다) `head_size`에 센다.

주의사항, 그리고 무엇이 잘못되는가

`NEED_MORE`는 실패가 아니다.

잘못된 예:

```
if (proven_http_parse_request(data, h, 32, 0, &req, &n) != PROVEN_OK) return send_400();   /* wrong: a slow client gets a 400 */
```

올바른 예 — 결과 셋에 대응 셋. `NEED_MORE`면 더 읽는다(9장의 기한과 함께). `OUT_OF_BOUNDS`, `INVALID_FORMAT`, `UNSUPPORTED`면 431, 400, 505로 답하고 닫는다. `PROVEN_OK`면 계속한다.

필드는 반복될 수 있다. `proven_http_header_find`는 첫 번째를 돌려준다. 반복이 뜻을 바꾸는 필드라면 `proven_http_header_count`에 물어라 - 이 라이브러리가 두 번째 `Content-Length`를 거절하는 방법이 바로 그것이다.

뷰는 버퍼와 함께 죽는다. 다음 요청을 같은 버퍼에 읽어 들이면, 앞 헤드의 모든 뷰는 이제 다른 것을 가리킨다. 남겨 둘 것은 복사하라.

5. 본문은 얼마나 긴가

헤드 다음으로 가장 중요한 질문은 메시지가 어디서 끝나는가다. 다음 메시지가 거기서 시작하기 때문이다. 답은 헤드가 준다.

```
typedef struct {
    proven_http_body_kind_t kind;   /* NONE, LENGTH, CHUNKED, or UNTIL_CLOSE (responses only) */
    proven_u64 length;              /* for LENGTH */
} proven_http_framing_t;
```

API	의도	반환
`proven_http_request_framing(&req, &framing)`	청크, 길이, 또는 본문 없음.	`proven_err_t`: 1절의 모호한 경우는 `INVALID_FORMAT`(400). `chunked`가 아닌 전송 코딩은 `UNSUPPORTED`(501).
`proven_http_response_framing(&res, request_method, &framing)`	같은 일을 하되 무엇을 물었는지 안다: `HEAD`에 대한 응답, 1xx, 204, 304, 성공한 `CONNECT`에는 본문이 없다. 두 헤더가 다 없으면 본문은 상대가 닫을 때까지다.	위와 같다.

응답에 요청의 메서드가 필요한 이유는 같은 헤더가 다른 뜻을 갖기 때문이다. `HEAD`에 대한 응답은 `GET`이었다면 가졌을 `Content-Length`를 싣고, 본문은 아예 없다.

6. 본문 디코딩

```
proven_http_body_t body;                        /* caller-owned; nothing to destroy */
proven_http_body_init(&body, framing, max_body_bytes);
proven_http_body_feed(&body, in, &consumed, &payload, &done);
```

`proven_http_body_t`는 호출자 소유(owned) 상태다. 할당하는 것이 없고 destroy할 것도 없다.

API	의도	반환
`proven_http_body_init(&body, framing, max_body)`	시작한다. `max_body`는 받아들일 양이다 - 기본값이 없다.	`proven_err_t`: `Content-Length`가 이미 그것을 넘으면 `OUT_OF_BOUNDS`(413).
`proven_http_body_feed(&body, in, &consumed, &payload, &done)`	`in`의 앞부분을 소비한다. `payload`는 그 안의 본문 바이트이고, `in` 안을 가리키는 뷰다.	`proven_err_t`: `INVALID_FORMAT`(잘못된 청크 프레이밍). `OUT_OF_BOUNDS`(한도 초과). 오류 뒤에는 `INVALID_STATE`.
`proven_http_body_end(&body)`	상대가 닫았다: 본문은 완전했는가?	`PROVEN_OK`, 또는 메시지가 잘렸으면 `PROVEN_ERR_NEED_MORE`.
`proven_http_body_received(&body)`	지금까지 전달된 본문 바이트.	`proven_u64`.

`feed`는 아무것도 복사하지 않는다. 청크 본문이라면 청크 프레이밍을 건너뛰고 그 사이의 데이터를, 호출마다 연속된 한 조각씩 돌려준다. 그래서 루프는 이렇다. 가진 것을 먹이고, `payload`를 쓰고, `consumed`만큼 전진하고, 전부 소비했는데 `done`이 아직 false면 더 읽는다. `done`이 true가 되면, `in`에서 `consumed` 뒤의 바이트는 **다음** 메시지의 것이다.

한도를 밝혀라. `max_body`는 "낯선 이가 나에게 얼마나 들고 있게 할 수 있는가"에 대한 답이다. 그것을 넘는 `Content-Length`는 본문을 한 바이트도 읽기 전에 거절되고, 크기가 그것을 넘길 청크는 데이터가 도착한 뒤가 아니라 크기를 읽을 때 거절된다.

닫힘이 언제나 끝은 아니다. 닫을 때까지 이어지는 본문은 상대가 닫을 때 제대로 끝난다. 그 밖의 본문이 닫힘으로 중단되면 잘린 것이다 - `proven_http_body_end`가 `PROVEN_ERR_NEED_MORE`를 돌려준다 - 그리고 온전한 것으로 취급해서는 안 된다. 온전한 파일처럼 보이는 반쪽 파일은 다운로드의 최악의 결과다.

오류 뒤에는 연결이 끝난다. 프레이밍이 틀리면 다음 메시지가 어디서 시작하는지 알 수 없다. 할 수 있으면 답하고, 닫는다.

테스트 스위트가 컴파일하고 실행한다 - 어색하게 잘린 네 조각으로 도착하는 요청:

```
/*
 * 서버가 보는 요청 하나: 바이트는 조각으로 도착하고, 헤드는 다 모였을 때 파싱하고,
 * 프레이밍이 본문 길이를 알려 주고, 본문은 오는 대로 디코딩한다.
 *
 * 여기에 소켓은 나오지 않는다. 코덱은 가진 바이트를 받을 뿐이고 그것이 어디서 왔는지는
 * 여러분의 일이다. 같은 코드가 블로킹 read 아래에서도, 이벤트 루프에서도, 이런 테스트에서도
 * 도는 이유다.
 */

/* 네트워크가 전해 줄 법한 모양의 요청: 어색한 자리에서 잘려 있다. */
static const char *const pieces[] = {
    "POST /upload/notes.txt?overwrite=1 HTT",
    "P/1.1\r\nHost: example.com\r\nTransfer-Encoding: chun",
    "ked\r\nConnection: keep-alive, TE\r\nX-Tag: a\r\nX-Tag: b\r\n\r\n5\r\nhel",
    "lo\r\n7;note=x\r\n, world\r\n0\r\n\r\nGET /next HTTP/1.1\r\n\r\n",
};

int main(void) {
    proven_byte_t buf[512];                 /* 읽기 버퍼: 헤드가 여기에 들어가야 한다 */
    proven_size_t have = 0;
```

```
    proven_size_t next_piece = 0;

    proven_http_header_t fields[16];
    proven_http_request_t req;
    proven_size_t head_size = 0;

    /* 헤드가 완성될 때까지 읽는다. NEED_MORE는 오류가 아니다. 말 그대로의 뜻이다. */
    for (;;) {
        proven_err_t err = proven_http_parse_request((proven_mem_view_t){ buf, have }, fields, 16, sizeof buf,
&req, &head_size);
        if (err == PROVEN_OK) break;
        EXAMPLE_REQUIRE(err == PROVEN_ERR_NEED_MORE, "an incomplete head asks for more; anything else is a
400, 431 or 505");
        if (err != PROVEN_ERR_NEED_MORE) return EXAMPLE_OK();
        /* "read": 다음 조각을 덧붙인다. */
        proven_size_t len = proven_cstr_len(pieces[next_piece]);
        for (proven_size_t i = 0; i < len; ++i) buf[have + i] = (proven_byte_t)pieces[next_piece][i];
        have += len;
        next_piece++;
    }

    /* 헤드. `buf` 안을 가리키는 뷰들이다. */
    EXAMPLE_REQUIRE(req.method == PROVEN_HTTP_POST && req.version_minor == 1, "POST, HTTP/1.1");
    EXAMPLE_REQUIRE(proven_http_method_from_text(req.method_text) == PROVEN_HTTP_POST &&
                    proven_u8str_view_eq(proven_http_method_text(req.method), PROVEN_LIT("POST")), "the
method, both ways");

    proven_u8str_view_t host;
    EXAMPLE_REQUIRE(proven_http_header_find(fields, req.header_count, PROVEN_LIT("host"), &host) &&
                    proven_u8str_view_eq(host, PROVEN_LIT("example.com")), "header names compare without
case");
    EXAMPLE_REQUIRE(proven_http_header_count(fields, req.header_count, PROVEN_LIT("X-Tag")) == 2, "a field may
repeat");
    EXAMPLE_REQUIRE(proven_http_header_has_token(fields, req.header_count, PROVEN_LIT("Connection"),
PROVEN_LIT("te")),
                    "a token inside a comma-separated list");
    EXAMPLE_REQUIRE(proven_http_request_keep_alive(&req), "the connection stays open for another request");

    /* 요청이 가리키는 곳. 나누고, 해소한다 - 경로를 어디에든 쓰기 전에. */
    proven_u8str_view_t path, query;
    bool has_query = false;
    proven_byte_t clean[128];
    proven_size_t clean_len = 0;
    EXAMPLE_REQUIRE(proven_url_split_target(req.target, &path, &query, &has_query) == PROVEN_OK &&
                    proven_url_path_resolve(path, (proven_mem_mut_t){ clean, sizeof clean }, &clean_len) ==
PROVEN_OK &&
                    proven_u8str_view_eq((proven_u8str_view_t){ clean, clean_len }, PROVEN_LIT("/upload/
notes.txt")),
                    "the target's path, resolved");

    /* 본문은 얼마나 긴가? 헤드가 정하고, 모호한 헤드는 추측이 아니라 오류다. */
    proven_http_framing_t framing;
    EXAMPLE_REQUIRE(proven_http_request_framing(&req, &framing) == PROVEN_OK && framing.kind ==
PROVEN_HTTP_BODY_CHUNKED,
                    "this body is chunked");

    /* 디코딩한다. 한도는 상대가 얼마나 보내도 되는지에 대한 호출자의 선언이다. */
    proven_http_body_t body;
    EXAMPLE_REQUIRE(proven_http_body_init(&body, framing, 1024) == PROVEN_OK, "at most 1 KiB of body");
```

```
    proven_byte_t content[64];
    proven_size_t content_len = 0;
    proven_size_t pos = head_size;          /* 본문은 헤드가 끝난 자리에서 시작한다 */
    bool done = false;
    while (!done) {
        if (pos == have) {
            /* 지금까지 읽은 것을 다 썼다: 더 "read"한다. 실제 서버라면 상대가 닫았을 때
             * 여기서 proven_http_body_end를 부르고 멈춘다. */
            EXAMPLE_REQUIRE(next_piece < sizeof pieces / sizeof pieces[0], "more input exists");
            proven_size_t len = proven_cstr_len(pieces[next_piece]);
            for (proven_size_t i = 0; i < len; ++i) buf[have + i] = (proven_byte_t)pieces[next_piece][i];
            have += len;
            next_piece++;
        }
        proven_size_t used = 0;
        proven_mem_view_t payload;
        proven_err_t err = proven_http_body_feed(&body, (proven_mem_view_t){ buf + pos, have - pos }, &used,
&payload, &done);
        EXAMPLE_REQUIRE(err == PROVEN_OK, "the chunk framing is well formed");
        if (err != PROVEN_OK) break;
        /* `payload`는 `buf` 안을 가리키는 뷰다: 청크 프레이밍을 건너뛴 본문 바이트. */
        for (proven_size_t i = 0; i < payload.size; ++i) content[content_len + i] = payload.ptr[i];
        content_len += payload.size;
        pos += used;
    }
    EXAMPLE_REQUIRE(proven_u8str_view_eq((proven_u8str_view_t){ content, content_len }, PROVEN_LIT("hello,
world")),
                    "two chunks, joined");
    EXAMPLE_REQUIRE(proven_http_body_received(&body) == 12 && proven_http_body_end(&body) == PROVEN_OK,
"twelve bytes, complete");

    /* 버퍼에 남은 것은 다음 요청이고, 손대지 않았다. */
    proven_http_request_t next;
    EXAMPLE_REQUIRE(proven_http_parse_request((proven_mem_view_t){ buf + pos, have - pos }, fields, 16, 0,
&next, &head_size) == PROVEN_OK &&
                    proven_u8str_view_eq(next.target, PROVEN_LIT("/next")), "the decoder stopped exactly where
the next request begins");

    return EXAMPLE_OK();
}
```

7. 메시지 쓰기

쓰기 함수들은 여러분이 준 메모리에 덧붙인다. 각각 버퍼와 길이를 받아, 그 길이의 자리에 쓰고, 길이를 전진시킨다. 실패하면 길이는 움직이지 않고 그 앞의 것은 건드리지 않는다.

API	덧붙이는 것
`proven_http_write_request_line(out, &len, method, target)`	`GET /path HTTP/1.1` CRLF
`proven_http_write_status_line(out, &len, status, reason)`	`HTTP/1.1 404 Not Found` CRLF. `reason`이 비어 있으면 표준 문구를 보낸다
`proven_http_write_header(out, &len, name, value)`	`Name: value` CRLF
`proven_http_write_header_u64(out, &len, name, number)`	`Content-Length: 1234` CRLF
`proven_http_write_head_end(out, &len)`	헤드를 끝내는 빈 줄
`proven_http_write_chunk_begin(out, &len, size)`	`size`바이트(0이 아님) 청크를 시작하는 줄
`proven_http_write_chunk_end(out, &len)`	청크 데이터 뒤의 CRLF

`proven_http_write_last_chunk(out, &len)`	청크 본문을 끝내는 0 청크와 빈 트레일러

모두 `proven_err_t`를 돌려준다. 들어가지 않으면 `PROVEN_ERR_OUT_OF_BOUNDS`, 쓰라고 한 것이 그 이름의 물건이 되지 못하면 `PROVEN_ERR_INVALID_ARG`다.

중요한 것은 두 번째 오류다. 헤더 값은 거의 언제나 다른 곳에서 온다 - 파일 이름, 리다이렉트 대상, 클라이언트가 보낸 무언가. 거기에 CR과 LF가 들어 있으면, 그대로 쓰는 순간 필드가 끝나고 *보낸 쪽*이 고른 새 필드가 시작된다.

```
Location: /home
Set-Cookie: session=attacker        <- this line came from inside the "value"
```

그것이 *응답 분할(response splitting)*이고, 요청 밀반입의 쓰기 쪽 쌍둥이다. `proven_http_write_header`는 CR, LF, NUL, 그 밖의 제어 문자가 든 값과 토큰이 아닌 이름을 거절하고, `proven_http_write_request_line`은 공백이나 줄바꿈이 든 타깃을 거절한다. 이 검사는 선택 사항이 아니고 "raw" 변종도 없다.

잘못된 예 — 마지막에 한 번만 검사하기:

```
proven_http_write_status_line(out, &len, 200, PROVEN_LIT(""));
proven_http_write_header(out, &len, name, user_value);      /* refused: len did not move */
err = proven_http_write_head_end(out, &len);                /* OK - and the header is silently missing */
```

올바른 예 — 호출마다 검사하거나, 예제처럼 첫 실패가 나머지를 멈추도록 잇는다. 쓰기 함수들은 `[[nodiscard]]`라서 첫 번째 형태에는 컴파일러가 이의를 제기한다.

길이를 아는 본문은 `Content-Length`를 쓰고 바이트를 쓴다. 바이트는 여러분이 직접 쓴다. 길이를 모르는 본문은 `Transfer-Encoding: chunked`를 쓰고, 조각마다 `chunk_begin`, 데이터, `chunk_end`를 쓰고, 끝에 `last_chunk`를 쓴다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 응답을 쓰고, 그것을 다시 읽는다.
 *
 * 쓰기 함수는 `len` 자리에 덧붙이고 `len`을 옮긴다. 메시지를 깨뜨릴 수 있는 값 - 헤더 안의
 * 줄바꿈 - 은 쓰지 않고 거절한다. 그래서 요청에서 가져온 문자열로 보낸 쪽이 고른 헤더를
 * 끼워 넣을 수 없다.
 */

int main(void) {
    proven_byte_t msg[512];
    proven_mem_mut_t out = { msg, sizeof msg };
    proven_size_t len = 0;

    /* Date 헤더: 벽시계를, HTTP가 쓰는 단 하나의 형식으로. */
    proven_byte_t date[PROVEN_HTTP_DATE_SIZE];
    proven_time_t now = proven_time_now();
    EXAMPLE_REQUIRE(proven_http_date_format(now, date) == PROVEN_OK, "now, as an HTTP date");

    /* 길이를 미리 알 수 없는 응답은 청크로 나간다. */
    proven_err_t err = proven_http_write_status_line(out, &len, 200, PROVEN_LIT(""));     /* "" : 표준 문구 */
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Date"), (proven_u8str_view_t)
{ date, sizeof date });
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Content-Type"),
PROVEN_LIT("text/plain; charset=utf-8"));
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Transfer-Encoding"),
PROVEN_LIT("chunked"));
    if (err == PROVEN_OK) err = proven_http_write_header_u64(out, &len, PROVEN_LIT("X-Request-Id"), 4711);
    if (err == PROVEN_OK) err = proven_http_write_head_end(out, &len);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the head is written");
    proven_size_t head_len = len;

    static const char *const parts[] = { "written in ", "two chunks" };
```

```
    for (proven_size_t i = 0; i < 2 && err == PROVEN_OK; ++i) {
        proven_size_t n = proven_cstr_len(parts[i]);
        err = proven_http_write_chunk_begin(out, &len, n);
        if (err != PROVEN_OK || n > sizeof msg - len) { err = PROVEN_ERR_OUT_OF_BOUNDS; break; }
        for (proven_size_t k = 0; k < n; ++k) msg[len + k] = (proven_byte_t)parts[i][k];   /* 청크의 데이터 */
        len += n;
        err = proven_http_write_chunk_end(out, &len);
    }
    if (err == PROVEN_OK) err = proven_http_write_last_chunk(out, &len);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "the body is written");

    /* 줄바꿈이 든 헤더 값은 아예 쓰이지 않는다. */
    proven_size_t before = len;
    EXAMPLE_REQUIRE(proven_http_write_header(out, &len, PROVEN_LIT("Location"), PROVEN_LIT("/home\r\nSet-
Cookie: session=stolen")) == PROVEN_ERR_INVALID_ARG &&
                    len == before, "response splitting is refused, and nothing was appended");

    /* --- 클라이언트 쪽: 같은 바이트를 다시 읽는다 ---------------------------------- */

    proven_http_header_t fields[16];
    proven_http_response_t res;
    proven_size_t head_size = 0;
    EXAMPLE_REQUIRE(proven_http_parse_response((proven_mem_view_t){ msg, len }, fields, 16, 0, &res,
&head_size) == PROVEN_OK &&
                    head_size == head_len, "the head parses, and ends where it was written to end");
    EXAMPLE_REQUIRE(res.status == 200 && proven_u8str_view_eq(res.reason, proven_http_reason_phrase(200)),
"200 OK");
    EXAMPLE_REQUIRE(proven_http_response_keep_alive(&res), "an HTTP/1.1 response leaves the connection open");

    /* 본문의 프레이밍은 그것이 답하는 요청에 달려 있다: HEAD에 대한 응답에는 본문이 없다. */
    proven_http_framing_t framing;
    EXAMPLE_REQUIRE(proven_http_response_framing(&res, PROVEN_HTTP_HEAD, &framing) == PROVEN_OK &&
framing.kind == PROVEN_HTTP_BODY_NONE,
                    "had this answered HEAD, no body would follow");
    EXAMPLE_REQUIRE(proven_http_response_framing(&res, PROVEN_HTTP_GET, &framing) == PROVEN_OK && framing.kind
== PROVEN_HTTP_BODY_CHUNKED,
                    "it answers GET, and the body is chunked");

    proven_http_body_t body;
    proven_byte_t text[64];
    proven_size_t text_len = 0;
    proven_size_t pos = head_size;
    bool done = false;
    EXAMPLE_REQUIRE(proven_http_body_init(&body, framing, sizeof text) == PROVEN_OK, "accept at most what the
buffer holds");
    while (!done && pos < len) {
        proven_size_t used = 0;
        proven_mem_view_t payload;
        if (proven_http_body_feed(&body, (proven_mem_view_t){ msg + pos, len - pos }, &used, &payload,
&done) != PROVEN_OK) break;
        for (proven_size_t i = 0; i < payload.size; ++i) text[text_len + i] = payload.ptr[i];
        text_len += payload.size;
        pos += used;
    }
    EXAMPLE_REQUIRE(done && pos == len && proven_u8str_view_eq((proven_u8str_view_t){ text, text_len },
PROVEN_LIT("written in two chunks")),
                    "the body, decoded");

    /* 날짜는 그것을 만든 시각으로, 초 단위까지 되돌아온다. */
    proven_u8str_view_t date_text;
```

```
    proven_time_t parsed = 0;
    EXAMPLE_REQUIRE(proven_http_header_find(fields, res.header_count, PROVEN_LIT("Date"), &date_text) &&
                    proven_http_date_parse(date_text, now, &parsed) == PROVEN_OK && parsed == now - now %
1000000000,
                    "the Date header parses back to the same second");

    /* 요청도 같은 방식으로 쓴다. */
    len = 0;
    err = proven_http_write_request_line(out, &len, PROVEN_LIT("GET"), PROVEN_LIT("/search?q=a%20b"));
    if (err == PROVEN_OK) err = proven_http_write_header(out, &len, PROVEN_LIT("Host"),
PROVEN_LIT("example.com"));
    if (err == PROVEN_OK) err = proven_http_write_head_end(out, &len);
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_u8str_view_eq((proven_u8str_view_t){ msg, len },
                    PROVEN_LIT("GET /search?q=a%20b HTTP/1.1\r\nHost: example.com\r\n\r\n")), "a request head,
byte for byte");

    return EXAMPLE_OK();
}
```

8. 날짜

HTTP는 날짜 형식 하나를 쓰고, 셋을 읽어야 한다.

API	의도	반환
`proven_http_date_format(wall_ns, out[29])`	`Sun, 06 Nov 1994 08:49:37 GMT`. 언제나 29바이트 (`PROVEN_HTTP_DATE_SIZE`), 언제나 GMT, NUL 없음.	`proven_err_t`: 1970년 이전은 `INVALID_ARG`.
`proven_http_date_parse(text, now_ns, &wall_ns)`	그 형식과, 폐기된 `Sunday, 06-Nov-94 08:49:37 GMT`, asctime의 `Sun Nov  6 08:49:37 1994`를 읽는다.	`proven_err_t`: `INVALID_FORMAT`. `OVERFLOW`.

벽시계(`proven_time_now`)를 넘겨라. 날짜가 없는 단조 시계는 안 된다 (5장 § 4).

파서는 그 날짜가 존재하는지 검사한다 - 2월 30일은 거절하고, 날짜와 맞지 않는 요일도 거절한다. `now_ns`는 규칙 하나에만 쓰인다. 폐기된 형식에 있는 두 자리 연도는, 그 두 자리를 가진 해 중 50년 넘게 미래가 아닌 가장 최근의 해로 읽는다.

`PROVEN_ERR_OVERFLOW`는 `proven_time_t`가 담을 수 없는 올바른 날짜다. 그 나노초는 2262년 4월에 끝난다. 서버는 그런 날짜를 일부러 보낸다 - 9999년의 `Expires`는 "영원히"라는 뜻이다 - 그러니 그 답은 잘못된 헤더가 아니라 "먼 미래"로 취급하라.

9. 참조와 폼

`Location` 헤더, 페이지 안의 링크, 폼의 `action`. 이것들은 모두 *참조*이고, 그것이 어디로 이어지는지는 그것이 발견된 URL에 달려 있다. `proven_url_resolve`가 그 단계이며, RFC 3986 5.2절이 정한 대로 한다.

`http://example.com/docs/guide/intro.html?v=2`에서 발견된 참조	해소 결과
`chapter2.html`	`http://example.com/docs/guide/chapter2.html`
`../img/logo.png`	`http://example.com/docs/img/logo.png`
`/login?next=%2F`	`http://example.com/login?next=%2F`
`?v=3`	`http://example.com/docs/guide/intro.html?v=3`
`//cdn.example.net/x`	`http://cdn.example.net/x`
`https://other.example/`	`https://other.example/`

API	의도	반환
`proven_url_resolve(base, reference, out, &written)`	`reference`를 절대 URL `base`에 대해 해소한다. 점 세그먼트는 제거된다. 그 자체로 절대 URL인 참조는 그대로 돌려준다.	`proven_err_t`: `base`가 절대 URL이 아니거나 결과에 호스트가 없게 되면 `INVALID_FORMAT`. `OUT_OF_BOUNDS`.
`proven_url_form_append(out, &len, name, value)`	폼 본문의 `name=value` 한 쌍을 인코딩해서 덧붙인다. 첫 쌍이 아니면 앞에 `&`를 붙인다.	`proven_err_t`: `OUT_OF_BOUNDS`. 그때 `len`과 그 앞의 바이트는 그대로다.

해소된 URL은 남이 고른 목적지다. 그 `Location`을 보낸 서버가 골랐다. 해소는 그것이 어디로 이어지는지 알려 줄 뿐이다. 거기로 갈지 말지 - 다른 호스트로, `https`에서 `http`로 내려가서, 여러분의 내부 네트워크 안의 주소로 - 는 해소 함수가 내리는 결정이 아니다. 앞의 둘은 11장의 클라이언트가 결정한다. 셋째는 여러분 몫이다.

`proven_url_form_append`는 `application/x-www-form-urlencoded`를 쓴다. 쿼리 문자열과 단순한 폼 본문의 형식이다. 공백은 `+`가 되고, 글자와 숫자와 `-._~`가 아닌 것은 모두 `%XX`가 된다. 이름과 값은 바이트이므로 주어진 그대로 인코딩된다.

10. 범위와 multipart 본문

범위 요청은 리소스의 일부를 요청한다. 내려받기를 이어 가거나 로그의 끝을 읽을 때 쓴다. 요청은 바이트를 지목하고, 답은 자기가 어느 바이트를 싣고 있는지 말하는 `206`이다.

API	의도	반환
`proven_http_write_range(out, &len, first, last)`	`Range: bytes=first-last`를 덧붙인다. "끝까지"는 `last`에 `PROVEN_HTTP_RANGE_TO_END`를 준다.	`proven_err_t`: `last < first`이면 `INVALID_ARG`.
`proven_http_range_parse(value, size, &first, &last)`	서버 쪽: `Range` 값을 `size`바이트짜리 리소스에 견주어 읽는다. `bytes=a-b`, `bytes=a-`, `bytes=-n`을 받고 리소스 크기에 맞춰 자른다.	`proven_err_t`: `OUT_OF_BOUNDS` - 416으로 답하라. 범위가 여럿이거나 다른 단위면 `UNSUPPORTED` - 리소스 전체를 보내라. `INVALID_FORMAT` - 헤더를 무시하라.
`proven_http_write_content_range(out, &len, first, last, total)`	206을 위한 `Content-Range: bytes first-last/total`을 덧붙인다.	`proven_err_t`: `first <= last < total`이 아니면 `INVALID_ARG`.
`proven_http_content_range_parse(value, &first, &last, &total, &has_total)`	클라이언트 쪽: 그것을 다시 읽는다. `/*`이면 `has_total`이 false다.	`proven_err_t`: `INVALID_FORMAT`.

`proven_http_range_parse`의 세 오류는 서로 다른 세 가지 답이며, RFC 9110이 하나하나 이름을 붙였다. 만족시킬 수 없는 범위는 `Content-Range: bytes */size`를 단 `416`이다. 이 서버가 구현하지 않는 범위나 읽을 수 없는 헤더는 *무시*한다 - 응답은 리소스 전체를 실은 평범한 `200`이다. `Range` 헤더에 `400`으로 답하는 것은 틀렸다.

이어받는 클라이언트는 무엇이 돌아왔는지 확인해야 한다. 서버는 `Range`를 무시해도 된다. 5000번 바이트부터 요청했는데 답이 `200`이라면 본문은 0번 바이트에서 시작한다. 그것을 이미 가진 5000바이트 뒤에 덧붙이면 소리 없이 망가진 파일이 된다. `Content-Range`가 요청한 자리에서 시작하는 `206`에만 덧붙여라.

`multipart/form-data` 본문은 폼이 파일을 올리는 방식이다. 필드 하나하나가 파트이고, 파트들은 어느 파트 안에도 나타나서는 안 되는 경계 줄로 나뉜다.

API	의도	반환
`proven_http_multipart_boundary(random, out)`	여러분이 준 무작위 16바이트로 경계 -	없음.

	PROVEN_HTTP_BOUNDARY_SIZE(40) 글자 - 를 만든다.	
proven_http_multipart_write_content_type(out, &len, boundary)	Content-Type: multipart/form-data; boundary=...를 덧붙인다.	proven_err_t: 경계가 비었거나, 70자를 넘거나, 따옴표가 필요하면 INVALID_ARG.
proven_http_multipart_write_part(out, &len, boundary, name, filename, content_type)	파트의 여는 부분을 덧붙인다. filename이 비면 평범한 필드다. content_type이 비면 쓰지 않는다. 파트의 바이트는 이어서 여러분이 쓴다.	proven_err_t: 이름이 비었거나 content_type에 제어 문자가 있으면 INVALID_ARG.
proven_http_multipart_write_part_end(out, &len)	파트의 바이트를 끝내는 줄바꿈을 덧붙인다.	proven_err_t.
proven_http_multipart_write_end(out, &len, boundary)	닫는 경계를 덧붙인다.	proven_err_t.

경계는 예측할 수 없어야 한다. 올린 파일의 바이트 안에 경계 줄이 들어갈 수 있다면, 그 파일은 자기 파트를 끝내고 다른 파트 - 폼에 없던 필드 - 를 시작할 수 있다. proven_random_bytes에서 얻은 16바이트는 그 가능성을 키를 맞히는 것만큼 낮춘다. 카운터나 시계로 만든 경계는 그렇지 않다.

필드 이름이나 파일 이름 안의 큰따옴표와 줄바꿈은 브라우저가 하듯 퍼센트 인코딩해서 쓴다. 그래서 a".png라는 파일이 자기가 적힌 따옴표 문자열을 닫을 수 없다.

이 함수들은 틀만 만든다. 파트의 바이트는 _write_part와 _write_part_end 사이에 여러분이 쓰므로, 큰 파일도 통째로 쥐지 않고 조각조각 보낼 수 있다. 그리고 남이 보낸 multipart 본문을 파싱하는 것은 여기에 없다.

11. 인증

HTTP 자체의 인증은 요구와 응답이다. 401 응답이 WWW-Authenticate를 싣고 오고, 요청을 Authorization과 함께 다시 보낸다.

API	의도	반환
proven_http_basic_auth(user, password, out, &written)	Basic <base64(user:password)> 값을 쓴다.	proven_err_t: user에 콜론이 있거나 어느 쪽에든 제어 문자가 있으면 INVALID_ARG. OUT_OF_BOUNDS.
proven_http_auth_offers(header_value, scheme)	WWW-Authenticate 값이 scheme을 내놓는지(대소문자 무시).	bool.
proven_http_digest_challenge_parse(header_value, &challenge)	WWW-Authenticate 값에서 Digest 요구를 찾는다. 여럿이면 이 라이브러리가 가진 가장 강한 알고리즘의 것을 고른다.	proven_err_t: NOT_FOUND - Digest 요구가 없다. UNSUPPORTED - 없는 알고리즘뿐이거나 qop=auth가 없다. INVALID_FORMAT.
proven_http_digest_auth(&challenge, user, password, method, uri, nonce_count, cnonce, out, &written)	그에 답하는 Authorization 값을 쓴다. uri는 보낸 그대로의 요청 타깃이고, cnonce는 여러분이 주는 무작위 텍스트이며, nonce_count는 새 nonce마다 1에서 시작한다.	proven_err_t: INVALID_ARG. OUT_OF_BOUNDS.

Basic은 비밀번호를 헤더에 실을 수 있게 인코딩한 것이지 숨긴 것이 아니다. 요청을 읽을 수 있는 사람은 누구나 비밀번호를 읽을 수 있다. 암호화되지 않은 연결에서는 경로 위의 모든 사람이 그렇다. TLS 위에서 쓰거나 아예 쓰지 마라.

Digest는 비밀번호를 보내지 않는다. 하는 일은 그것뿐이다. 응답은 비밀번호와 서버의 nonce와 요청 줄에 대한 해시라서, 엿듣는 사람이 나중에 로그인하는 데 쓸 것을 얻지 못한다. 그래도 요청과 응답은 읽을 수 있

는 채로 네트워크를 건너고, 교환 한 번을 붙잡은 사람은 그것을 놓고 느긋하게 비밀번호를 추측해 볼 수 있다. 평문 HTTP 위의 Basic보다는 낫다. TLS를 대신하지는 못한다.

`MD5`, `MD5-sess`, `SHA-256`, `SHA-256-sess`가 `qop=auth`와 함께 구현되어 있다. MD5가 있는 까닭은 현장의 서버들이 아직 그것을 요구하기 때문이다 - 레거시 다이제스트는 4장을 보라. 둘 다 내놓는 요구에는 SHA-256으로 답한다. `qop=auth-int`, `SHA-512-256`, `userhash` 옵션은 없다.

예상하지 않은 요구에는 답하지 마라. 요구하는 서버 아무에게나 자격 증명을 보내는 클라이언트는 엉뚱한 서버에도 보낸다 - 이를테면 리다이렉트 뒤에. 그 자격 증명이 어느 호스트를 위한 것인지 정하고, 그 호스트에만 답하라.

테스트 스위트가 컴파일하고 실행한다:

```
#include <string.h>

/*
 * 클라이언트와 서버가 메시지 코덱 둘레에서 필요로 하는 작은 조각들: 리다이렉트 풀기,
 * 폼 만들기, 리소스의 일부 요청하기, 파일 올리기, 인증 요구에 답하기. 모두 텍스트를 받아
 * 텍스트를 낸다 - 소켓도 할당도 없다.
 */

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    proven_byte_t buf[512];
    proven_mem_mut_t out = (proven_mem_mut_t){ buf, sizeof buf };
    proven_size_t len = 0;

    /* Location 헤더는 참조다. 그것이 어디로 이어지는지는 지금 어디에 있는지에 달려 있다. */
    proven_u8str_view_t base = PROVEN_LIT("http://example.com/docs/guide/intro.html?v=2");
    EXAMPLE_REQUIRE(proven_url_resolve(base, PROVEN_LIT("../img/logo.png"), out, &len) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "http://example.com/docs/img/logo.png"), "a
relative reference");
    EXAMPLE_REQUIRE(proven_url_resolve(base, PROVEN_LIT("/login?next=%2F"), out, &len) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "http://example.com/login?next=%2F"), "an
absolute path");
    EXAMPLE_REQUIRE(proven_url_resolve(base, PROVEN_LIT("//cdn.example.net/x"), out, &len) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "http://cdn.example.net/x"), "another host,
same scheme");

    /* 폼 본문을 한 쌍씩. 구분자와 인코딩은 알아서 처리된다. */
    len = 0;
    EXAMPLE_REQUIRE(proven_url_form_append(out, &len, proven_mem_view_from_u8(PROVEN_LIT("q")),
proven_mem_view_from_u8(PROVEN_LIT("tea & cake"))) == PROVEN_OK &&
                    proven_url_form_append(out, &len, proven_mem_view_from_u8(PROVEN_LIT("page")),
proven_mem_view_from_u8(PROVEN_LIT("2"))) == PROVEN_OK,
                    "two fields");
    EXAMPLE_REQUIRE(view_is((proven_u8str_view_t){ buf, len }, "q=tea+%26+cake&page=2"), "application/x-www-
form-urlencoded");

    /* 범위. 클라이언트가 요청하고, 서버는 그 요청을 자기가 가진 것의 크기에 견주어 읽는다. */
    len = 0;
    EXAMPLE_REQUIRE(proven_http_write_range(out, &len, 100, PROVEN_HTTP_RANGE_TO_END) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "Range: bytes=100-\r\n"), "from byte 100 to the
end");
    proven_u64 first = 0, last = 0, total = 0;
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=100-"), 1000, &first, &last) == PROVEN_OK &&
first == 100 && last == 999, "of 1000 bytes: 100 through 999");
```

```
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=-50"), 1000, &first, &last) == PROVEN_OK &&
first == 950 && last == 999, "the last fifty");
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=2000-"), 1000, &first, &last) ==
PROVEN_ERR_OUT_OF_BOUNDS, "past the end: answer 416");
    EXAMPLE_REQUIRE(proven_http_range_parse(PROVEN_LIT("bytes=0-9,20-29"), 1000, &first, &last) ==
PROVEN_ERR_UNSUPPORTED, "several ranges: send the whole thing");
    len = 0;
    EXAMPLE_REQUIRE(proven_http_write_content_range(out, &len, 950, 999, 1000) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, len }, "Content-Range: bytes 950-999/1000\r\n"), "the
206 says which part it is");
    bool has_total = false;
    EXAMPLE_REQUIRE(proven_http_content_range_parse(PROVEN_LIT("bytes 950-999/1000"), &first, &last, &total,
&has_total) == PROVEN_OK &&
                    first == 950 && last == 999 && has_total && total == 1000, "and the client reads it
back");

    /* multipart/form-data: 텍스트 필드 하나와 파일 하나. 경계는 여러분이 준 무작위 바이트에서
     * 나오므로, 미리 짐작해 파트 안에 심어 둘 수 없다. */
    proven_byte_t random[16];
    EXAMPLE_REQUIRE(proven_random_bytes(random, sizeof random), "sixteen random bytes");
    proven_byte_t boundary_bytes[PROVEN_HTTP_BOUNDARY_SIZE];
    proven_http_multipart_boundary(random, boundary_bytes);
    proven_u8str_view_t boundary = (proven_u8str_view_t){ boundary_bytes, sizeof boundary_bytes };

    proven_byte_t head[128];
    proven_size_t head_len = 0;
    EXAMPLE_REQUIRE(proven_http_multipart_write_content_type((proven_mem_mut_t){ head, sizeof head },
&head_len, boundary) == PROVEN_OK, "the header that announces it");

    len = 0;
    proven_err_t err = proven_http_multipart_write_part(out, &len, boundary, PROVEN_LIT("title"),
PROVEN_LIT(""), PROVEN_LIT(""));
    if (err == PROVEN_OK) { memcpy(buf + len, "Holiday", 7); len += 7; }      /* 파트의 바이트는 여러분이 쓴다
*/
    if (err == PROVEN_OK) err = proven_http_multipart_write_part_end(out, &len);
    if (err == PROVEN_OK) err = proven_http_multipart_write_part(out, &len, boundary, PROVEN_LIT("photo"),
PROVEN_LIT("a \"b\".png"), PROVEN_LIT("image/png"));
    if (err == PROVEN_OK) { memcpy(buf + len, "\x89PNG", 4); len += 4; }
    if (err == PROVEN_OK) err = proven_http_multipart_write_part_end(out, &len);
    if (err == PROVEN_OK) err = proven_http_multipart_write_end(out, &len, boundary);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "a body of two parts");
    buf[len] = '\0';
    EXAMPLE_REQUIRE(strstr((const char *)buf, "filename=\"a %22b%22.png\"") != NULL, "a quote in a file name
cannot end the quoted string");

    /* 인증. Basic은 두 문자열을 인코딩한 것이다 - 암호화한 것이 아니다. */
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_http_basic_auth(PROVEN_LIT("Aladdin"), PROVEN_LIT("open sesame"), out, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ=="), "the value
of an Authorization header");

    /* Digest는 요구에 답하는 방식이라 비밀번호 자체는 보내지 않는다. */
    proven_u8str_view_t www = PROVEN_LIT("Digest realm=\"http-auth@example.org\", qop=\"auth, auth-int\",
algorithm=SHA-256, "
                                         "nonce=\"7ypf/xlj9XXwfDPEoM4URrv/xwf94BcCAzFZH4GiTo0v\",
opaque=\"FQhe/qaU925kfnzjCev0ciny7QMkPqMAFRtzCUYo5tdS\"");
    EXAMPLE_REQUIRE(proven_http_auth_offers(www, PROVEN_LIT("digest")) && !proven_http_auth_offers(www,
PROVEN_LIT("Basic")), "what the server offers");
    proven_http_digest_challenge_t challenge;
```

```
    EXAMPLE_REQUIRE(proven_http_digest_challenge_parse(www, &challenge) == PROVEN_OK && challenge.algorithm ==
PROVEN_HTTP_DIGEST_SHA256 && challenge.qop_auth, "the challenge");
    EXAMPLE_REQUIRE(proven_http_digest_auth(&challenge, PROVEN_LIT("Mufasa"), PROVEN_LIT("Circle of Life"),
PROVEN_LIT("GET"), PROVEN_LIT("/dir/index.html"),
                                            1, PROVEN_LIT("f2/wE4q74E6zIJEtWaHKaf5wv/H5QzzpXusqGemxURZJ"),
out, &n) == PROVEN_OK, "the answer");
    buf[n] = '\0';
    /* RFC 7616 3.9.1절이 바로 이 입력으로 계산해 보인 response 값. */
    EXAMPLE_REQUIRE(strstr((const char *)buf,
"response=\"753927fa0e85d155564e2e272a28d1802ca10daf4496794697cf8db5856cb6c1\"") != NULL, "is the one in the
RFC");

    return EXAMPLE_OK();
}
```

12. 쿠키

쿠키는 서버가 되돌려 보내 달라고 요청하는 이름과 값이다. `proven_http_cookie_jar_t`가 요청과 요청 사이에 그것을 보관한다. `Set-Cookie` 헤더가 심은 것을 저장하고, 요청에 실을 `Cookie` 헤더를 써 준다.

API	의도	반환
`proven_http_cookie_jar_init(&jar, alloc, max_cookies)`	`max_cookies`개까지 담는 보관함을 만든다.	`proven_err_t`: 한도가 0이면 `INVALID_ARG`. `NOMEM`.
`proven_http_cookie_jar_store(&jar, host, path, secure, set_cookie, now)`	`host`와 `path`로 보낸 요청의 응답에 있던 `Set-Cookie` 값 하나를 처리한다. `now`는 `proven_time_now()`다.	`proven_err_t`: `INVALID_FORMAT` - 쿠키가 아니다, 무시하라. `PERMISSION` - 이 호스트가 심을 수 있는 것이 아니다. `OUT_OF_BOUNDS` - `PROVEN_HTTP_COOKIE_MAX_SIZE`보다 크다. `NOMEM`. 어느 경우든 보관함은 그대로다.
`proven_http_cookie_jar_header(&jar, host, path, secure, now, out, &written)`	요청에 실을 `Cookie` 값을 쓴다: `a=1; b=2`. 보낼 것이 없으면 `written`이 0이다.	`proven_err_t`: `OUT_OF_BOUNDS`.
`proven_http_cookie_jar_count(&jar)`	쿠키를 몇 개 갖고 있는지.	`proven_size_t`.
`proven_http_cookie_jar_clear(&jar)`	모두 잊는다.	없음.
`proven_http_cookie_jar_destroy(&jar)`	해제한다.	없음.

이 보관함은 일부러 호스트 전용이다. 브라우저는 `www.example.com`이 `example.com` 전체를 위한 쿠키를 심게 해 주는데, 그것을 안전하게 하려면 공개 접미사 목록 - `co.uk`는 사이트가 아니고 `example.co.uk`는 사이트라고 말해 주는 표 - 이 필요하다. 이 라이브러리에는 그런 표가 없고, 그 표 없이 `Domain`을 따르는 보관함은 `evil.co.uk`가 `co.uk` 아래 모든 사이트를 위한 쿠키를 심게 내버려 둔다. 그래서 `Domain` 속성은 검사한 다음 - 그것을 보낸 호스트를 덮지 않으면 `PROVEN_ERR_PERMISSION`이다 - 옆으로 치운다. 모든 쿠키는 그것을 심은 바로 그 호스트에게만 돌아가고 다른 곳으로는 가지 않는다. 하위 도메인끼리 쿠키를 공유하는 데 기대는 사이트는 이 보관함으로는 동작하지 않는다. 받아서는 안 될 것이 그 사이트에 닿는 일도 없다.

보관함이 RFC 6265 가운데 구현한 것: `Path`와 그 일치 규칙, `Max-Age`와 `Expires`(`Max-Age`가 이긴다), `Secure` - 암호화되지 않은 연결로는 보내지 않고, 그런 연결에서 심으려 하면 거절한다 - , 이름과 경로가 같은 쿠키의 교체, 만료를 통한 삭제, 보관함이 찼을 때 가장 오래된 것 내보내기. `HttpOnly`와 `SameSite`는 브라우저의 스크립트와 페이지가 무엇을 해도 되는지를 말한다. 프로그램에는 둘 다 없으므로 받아들이고 무시한다.

보관함은 스레드 안전하지 않고, 메모리 안에만 산다. destroy하면 쿠키는 사라진다.

13. 서버 전송 이벤트

이벤트 스트림은 끝나지 않는 응답이다. `Content-Type: text/event-stream`이 오고, 그다음 연결이 살아 있는 동안 줄이 계속 온다.

```
id: 41
event: price
data: {"sym":"X","p":12.5}

data: a second event, of the default type
```

빈 줄이 이벤트를 끝낸다. `proven_sse_t`는 스트림을 도착한 조각 그대로 받아 이벤트를 돌려준다.

API	의도	반환
`proven_sse_init(&sse, work)`	시작한다. `work`는 줄과 조립 중인 이벤트를 위한 메모리이고 - 절반씩 - , 따라서 받아들일 가장 큰 이벤트의 한도가 된다. 최소 64바이트.	`proven_err_t`: `INVALID_ARG`.
`proven_sse_feed(&sse, in, &consumed, &event, &have_event)`	이벤트 하나가 완성되거나 `in`을 다 쓸 때까지 `in`의 바이트를 소비한다. `have_event`가 true면 `event`를 쓰고 나머지로 다시 부른다.	`proven_err_t`: 줄이나 이벤트가 `work`보다 커지면 `OUT_OF_BOUNDS`. 그 뒤로는 `INVALID_STATE`.
`proven_sse_last_id(&sse)`	마지막으로 본 `id`. 다시 연결할 때 `Last-Event-ID`로 보낸다.	`proven_u8str_view_t`. 없었으면 비어 있다.

```
typedef struct {
    proven_u8str_view_t event;   /* the type; empty means the default, "message" */
    proven_u8str_view_t data;    /* the data lines joined with LF */
    proven_u8str_view_t id;      /* the last id so far, from this event or an earlier one */
    proven_u32 retry_ms;         /* valid when has_retry */
    bool has_retry;
} proven_sse_event_t;
```

이벤트의 뷰(view)는 다음 feed까지만 유효하다. `work` 안을 가리키기 때문이다. 간직할 것은 복사하라.

이벤트 중간에서 끝난 스트림은 그 이벤트를 전달하지 않은 것이다. 표준은 이벤트가 빈 줄로 내보내진다고 정한다. 그 전에 끊긴 이벤트는 절반만 전달되는 것이 아니라 버려진다. 연결이 끝나면 `Last-Event-ID`를 달고 다시 연결하라. 서버가 다시 보내 줄 수 있다.

파서는 WHATWG 규칙을 따른다 - 세 가지 줄 끝, 주석 줄, 맨 앞의 바이트 순서 표시, 콜론 없는 필드, 콜론 뒤의 공백 하나 - . 다시 연결하지는 않는다. 그것은 클라이언트의 일이다. 이것은 파서이고, 연결은 11장의 것이다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 클라이언트가 메시지와 메시지 사이에 간직하는 두 가지 상태: 서버가 심은 쿠키와,
 * 서버 전송 이벤트 스트림에서의 자기 위치. 어느 쪽도 소켓을 건드리지 않는다.
 */

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    // ---- 쿠키 보관함 ------------------------------------------------------------
    proven_http_cookie_jar_t jar;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_init(&jar, proven_heap_allocator(), 32) == PROVEN_OK, "a jar for up
```

```
to 32 cookies");
    proven_time_t now = proven_time_now();

    /* 응답의 Set-Cookie 헤더 하나하나를, 그것이 어디서 왔는지와 함께 저장한다. */
    proven_u8str_view_t host = PROVEN_LIT("shop.example.com");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/"), true, PROVEN_LIT("sid=abc123;
Path=/; Secure; HttpOnly"), now) == PROVEN_OK, "a session cookie");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/cart/view"), true,
PROVEN_LIT("items=3; Max-Age=3600"), now) == PROVEN_OK,
                    "one with no Path: it belongs to /cart");
    /* 남의 도메인을 위한 쿠키는 거절된다. 이 보관함은 브라우저보다 더 엄격해서, 모든 쿠키를
     * 그것을 심은 바로 그 호스트에만 돌려준다. */
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/"), true, PROVEN_LIT("track=1;
Domain=ads.example.net"), now) == PROVEN_ERR_PERMISSION,
                    "not this host's to set");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_count(&jar) == 2, "two cookies");

    /* 요청을 보낼 때 보관함이 Cookie 헤더의 값을 써 준다. */
    proven_byte_t buf[256];
    proven_mem_mut_t out = { buf, sizeof buf };
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/cart/checkout"), true, now, out,
&n) == PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "items=3; sid=abc123"), "under /cart: both, the
longer path first");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/account"), true, now, out, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "sid=abc123"), "elsewhere: only the one for /");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/account"), false, now, out, &n) ==
PROVEN_OK && n == 0,
                    "over plain HTTP: nothing - sid is Secure. n == 0 means send no header");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, PROVEN_LIT("example.com"), PROVEN_LIT("/"), true, now,
out, &n) == PROVEN_OK && n == 0,
                    "another host: nothing");

    /* 시간이 흐르면 Max-Age가 다한다. 서버는 쿠키를 만료시켜서 지운다. */
    proven_time_t later = now + (proven_time_t)7200 * 1000000000;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_header(&jar, host, PROVEN_LIT("/cart"), true, later, out, &n) ==
PROVEN_OK &&
                    view_is((proven_u8str_view_t){ buf, n }, "sid=abc123"), "two hours on, items has
expired");
    EXAMPLE_REQUIRE(proven_http_cookie_jar_store(&jar, host, PROVEN_LIT("/"), true, PROVEN_LIT("sid=; Path=/;
Max-Age=0"), later) == PROVEN_OK &&
                    proven_http_cookie_jar_count(&jar) == 0, "logged out: the jar is empty");

    proven_http_cookie_jar_clear(&jar);       /* 또는 직접 비운다 */
    proven_http_cookie_jar_destroy(&jar);

    // ---- 서버 전송 이벤트 ------------------------------------------------
    /* 파서에는 도착한 것을 도착한 조각 그대로 넣고, 파서는 온전한 이벤트를 돌려준다. */
    proven_byte_t work[512];
    proven_sse_t sse;
    EXAMPLE_REQUIRE(proven_sse_init(&sse, (proven_mem_mut_t){ work, sizeof work }) == PROVEN_OK, "a parser
with room for 256-byte events");

    static const char *const arrived[] = {
        ": keep-alive\n\n",                          /* 주석 줄: 이벤트가 아니다 */
        "id: 41\nevent: price\ndata: {\"sym\":\"X\",",   /* 중간에서 끊긴 이벤트 */
        "\ndata:  \"p\":12.5}\n\n",                  /* ...그리고 그 끝 */
        "data: plain\n\n",
```

```
    };
    int events = 0;
    for (proven_size_t i = 0; i < 4; ++i) {
        proven_mem_view_t in = proven_mem_view_from_u8(proven_u8str_view_from_cstr(arrived[i]));
        while (in.size > 0) {
            proven_size_t used = 0;
            proven_sse_event_t ev;
            bool have = false;
            EXAMPLE_REQUIRE(proven_sse_feed(&sse, in, &used, &ev, &have) == PROVEN_OK, "the stream parses");
            in.ptr += used;
            in.size -= used;
            if (!have) continue;          /* 전부 소비했고 아직 이벤트는 없다 */
            events++;
            if (events == 1) {
                EXAMPLE_REQUIRE(view_is(ev.event, "price") && view_is(ev.id, "41"), "a typed event with an
id");
                EXAMPLE_REQUIRE(view_is(ev.data, "{\"sym\":\"X\",\n \"p\":12.5}"), "two data lines, joined by
a line feed");
            } else {
                EXAMPLE_REQUIRE(ev.event.size == 0 && view_is(ev.data, "plain"), "no type given: a
\"message\"");
                EXAMPLE_REQUIRE(view_is(ev.id, "41"), "the id stays until another replaces it");
            }
        }
    }
    EXAMPLE_REQUIRE(events == 2, "two events from four pieces");
    /* 연결이 끊긴 뒤 이것을 Last-Event-ID로 보내면 서버가 거기서부터 이어 줄 수 있다. */
    EXAMPLE_REQUIRE(view_is(proven_sse_last_id(&sse), "41"), "where to resume");

    return EXAMPLE_OK();
}
```

14. 여기에 없는 것

- **클라이언트나 서버.** 11장에 있다.
- **HTTP/2와 HTTP/3.** `HTTP/2.0`이라고 적힌 헤드는 `PROVEN_ERR_UNSUPPORTED`다.
- **코덱 안의 콘텐츠 코딩.** 파서와 본문 디코더는 본문을 보낸 그대로 건넨다. 관련 헤더를 읽는 헬퍼가 둘 있고 (`proven_http_accepts_coding`, `proven_http_content_coding`), 이 코덱 위에 지은 서버와 클라이언트는 `gzip`을 압축하고 풀 수 있다: 16장 6절.
- **트레일러.** 청크 본문 뒤의 트레일러는 검사하고 건너뛸 뿐, 돌려주지 않는다.
- **multipart 본문 파싱, 그리고 다중 범위(`multipart/byteranges`) 응답.**
- **공개 접미사 목록**, 따라서 하위 도메인 사이에 공유되는 쿠키 - §12를 보라.
- **`Negotiate`, `NTLM`, bearer 토큰 흐름.** bearer 토큰은 여러분이 직접 쓰는 헤더다.

11장: HTTP 클라이언트와 서버

5부 — 운영체제와 대화하기. 선행 조건: 9장의 주소·기한·전송과 10장의 요청·응답·헤더. 핸들러를 job system에서 돌리려면 6장의 §1. 이 장을 마치면 여러분의 함수 하나로 HTTP를 서비스할 수 있다. 모든 기다림에는 한도가 있고, 형식이 어긋난 요청은 여러분에게 닿기 전에 거절된다. 그리고 URL을 가져올 수 있다 - 리다이렉트와 로그인과 프록시를 거쳐서, 본문은 여러분의 속도로 읽으면서.

이 장은 `http_server.h`와 `http_client.h`를 다룬다. 둘 다 소켓이 필요하다. 프리스탠딩(freestanding) 빌드에는 들어가지 않고, 호스티드(hosted) 빌드에서는 `PROVEN_NO_NET`으로 빠진다.

목차

1. 이것이 무엇이고, 무엇 하나가 아닌가
2. 서버
3. 핸들러 안에서
4. 서버가 묻지 않고 하는 일
5. 핸들러를 돌리는 두 가지 방법
6. 클라이언트
7. 요청 하나, 응답 하나
8. 리다이렉트, 인증 요구, 쿠키
9. 연결과 프록시
10. TLS가 붙을 자리
11. 프로토콜 바꾸기
12. 여기에 없는 것

1. 이것이 무엇이고, 무엇 하나가 아닌가

10장은 코덱이다. 바이트를 메시지로, 메시지를 바이트로 바꾸며 소켓은 건드리지 않는다. 9장은 소켓이고 HTTP를 모른다. 이 두 헤더는 그 사이의 부분이다 - 연결을 받고, 읽고, 파싱하고, 시간 제한을 걸고, 연결을 열어 두는 루프와, 클라이언트 쪽에서 그것을 거울처럼 뒤집은 것.

이것은 드라이버이고 그 위의 것은 아무것도 아니다. 서버에는 라우터도, 정적 파일 핸들러도, 세션도, 템플릿도 없다. 요청 하나로 함수 하나를 부른다. 클라이언트에는 재시도 정책도, 캐시도, JSON도 없다. 요청 하나를 보내고 응답 하나를 준다. 그 위에 프로그램이 짓는 것은 프로그램의 것이다.

TLS는 여러분이 고르는 것이고, 기본값은 없음이다. 서버는 설정에 TLS 설정을 주지 않으면 평문 HTTP를 말하고, 클라이언트는 TLS 랩을 주지 않으면 `https` URL을 `PROVEN_ERR_UNSUPPORTED`로 거절한다 - 슬그머니 `http`로 대신 가져오지 않는다. 둘 다 필드 하나씩이며, 방법은 14장에 있다. 그것 없이는 이 둘이 보내고 받는 모든 것을 경로 위의 누구든 읽을 수 있고 바꿀 수 있다.

- 서버에 TLS 설정을 주거나, TLS를 끝내 주는 것 뒤에 두거나, loopback 인터페이스나 믿을 수 있는 네트워크 안에 두어라.
- 클라이언트로 `http://`를 통해 다른 기계에 비밀번호나 토큰이나 세션 쿠키를 보내지 마라.

이것은 코드가 하는 일을 적은 것이지, 따져 보고 받아들일 권고가 아니다.

2. 서버

```
static void handle(void *ctx, proven_http_exchange_t *x) {
    (void)proven_http_exchange_respond(x, 200, NULL, 0, body);
}

proven_http_server_config_t config = {0};
config.alloc = proven_heap_allocator();
config.handler = handle;
proven_http_server_create(&config, &server);
proven_http_server_listen(server, proven_net_addr_any(PROVEN_NET_FAMILY_IPV4, 8080), NULL);
proven_http_server_run(server);          /* until proven_http_server_stop */
proven_http_server_destroy(server);
```

참조

API	의도	반환
`proven_http_server_create(&config, &server)`	서버를 만든다. 아직 listen하지 않는다.	`proven_err_t`: 할당자(allocator)나 핸들러가 없으면 `INVALID_ARG`. `NOMEM`. 소켓이 모자라면 `BUSY`.
`proven_http_server_listen(server, at, &bound)`	주소 하나에서 listen한다. 넷까지 - 이를테면 IPv4 하나와 IPv6 하나. `bound`(선택)는 실제로 묶인 주소를 받으므로 포트 0을 쓸 수 있다.	`proven_net_listen`의 오류들. 다섯 번째에는 `OUT_OF_BOUNDS`.
`proven_http_server_run(server)`	멈출 때까지, 또는 종료가 끝날 때까지 서비스한다.	`proven_err_t`: stop 뒤, 또는 모두 끝낸 종료 뒤에는 `PROVEN_OK`; 기한이 무언가를 끊은 종료 뒤에는 `TIMEOUT`.
`proven_http_server_poll(server, until)`	한 바퀴: 무언가 준비되거나 `until`이 될 때까지 기다리고, 준비된 것을 처리한다. 자기 루프가 있는 프로그램을 위한 것이다.	`proven_err_t`: 처리한 것이 있으면 `PROVEN_OK`, 없으면 `TIMEOUT`.
`proven_http_server_stop(server)`	`run`이 돌아오게 한다. 어느 스레드에서든, 핸들러 안에서든 안전하다.	없음.
`proven_http_server_shutdown(server, grace_ms)`	부드럽게 끝낸다(아래). 어느 스레드에서든, 핸들러 안에서든 안전하고, 곧바로 돌아온다.	`proven_err_t`: 이미 요청되었으면 `INVALID_STATE`.
`proven_http_server_shutting_down(server)`	종료가 요청되었는지. 어느 스레드에서든: 넘겨받은 연결을 쥔 핸들러를 위한 것.	`bool`.
`proven_http_server_drained(server, &why)`	`poll`을 스스로 부르는 프로그램을 위해: 종료가 끝났는지, 어떻게 끝났는지.	`bool`; `why`는 `run`이 돌려주는 것과 같다.
`proven_http_server_destroy(server)`	모든 연결과 리스너를 닫고 서버를 해제한다. 핸들러 안에서는 부르지 않는다.	없음.
`proven_http_server_connection_count(server)`	지금 열려 있는 연결 수. 루프의 스레드에서 쓴다.	`proven_size_t`.

```
typedef struct {
    proven_allocator_t alloc;          /* required */
    proven_http_handler_fn handler;    /* required: void (*)(void *ctx, proven_http_exchange_t *) */
    void *handler_ctx;
    proven_job_sys_t *jobs;            /* NULL: handlers run on the loop's thread (section 5) */
    proven_size_t max_connections;     /* 0: 64 */
    proven_size_t max_head_bytes;      /* 0: 16 KiB - request head, and response head */
    proven_size_t max_headers;         /* 0: 64 fields */
    proven_u64 max_body_bytes;         /* 0: 1 MiB */
    proven_u32 head_timeout_ms;        /* 0: 10 s */
    proven_u32 body_timeout_ms;        /* 0: 30 s */
    proven_u32 write_timeout_ms;       /* 0: 30 s */
    proven_u32 idle_timeout_ms;        /* 0: 60 s */
    proven_u32 linger_ms;              /* 0: 1 s */
```

```
    const proven_tls_config_t *tls;    /* NULL: plain HTTP */
} proven_http_server_config_t;
```

구조체를 0으로 채우고 필요한 것만 정한다. 모든 한도에는 기본값이 있고, 어느 것도 끌 수 없다. "무제한"은 없다.

각 한도가 막는 것

한도	막는 것
`head_timeout_ms`	연결을 열고 요청을 1분에 한 바이트씩 보내는 클라이언트. 첫 바이트부터 헤드의 끝까지 이만큼의 시간이 주어지고, 넘기면 `408`을 받고 닫힌다. 아무것도 보내지 않는 새 연결에도 같은 시간이 주어진다.
`body_timeout_ms`	같은 수법을 본문으로 쓰는 것. 본문 읽기 한 번이 이만큼 기다릴 수 있다.
`write_timeout_ms`	큰 응답을 요청해 놓고 읽지 않는 클라이언트. 쓰기 한 번이 이만큼 기다릴 수 있다.
`idle_timeout_ms`	열어 둔 채 쓰지 않는 연결. HTTP가 허락하는 대로 말없이 닫는다.
`linger_ms`	클라이언트에 대한 한도가 아니라, 닫는 중인 연결을 서버가 얼마나 더 참아 주느냐다. 연결을 끝내는 응답 - 업로드 도중의 거절, `Connection: close` - 을 보낸 뒤 서버는 할 말을 다 했지만, 이 시간 동안은 클라이언트가 보내는 것을 계속 읽어서 버린다. 곧바로 닫으면 클라이언트 쪽에서 그 응답이 리셋으로 바뀌기 때문이다. 그 상태의 연결도 `max_connections`에 들어간다: 한꺼번에 많은 요청을 거절하는 곳에서는 짧게 잡아야 자리가 빨리 돌아오고, 느린 회선의 클라이언트에게는 길게 잡는 편이 맞다.
`tls`	인증서를 가진 TLS 설정(14장): 그러면 모든 연결이 HTTPS다. 서버보다 오래 살아야 한다. 핸드셰이크는 `head_timeout_ms`를 첫 요청의 헤드와 함께 쓴다.
`max_head_bytes`, `max_headers`	끝나지 않는 헤드, 필드가 만 개인 헤드: `431`.
`max_body_bytes`	받을 준비가 된 것보다 큰 본문: `413`. 길이를 미리 알렸으면 읽기 전에, 청크였으면 읽는 도중에.
`max_connections`	메모리가 감당할 수 있는 것보다 많은 연결. 다음 클라이언트는 listen 대기열에서 기다린다. 거절되지 않는다.

연결 하나의 메모리는 받아들일 때 정해진다. `2 * max_head_bytes`에 4 KiB와 헤더 배열을 더한 것으로, 기본값이면 약 38 KiB이고 연결 64개면 약 2.4 MiB다. 클라이언트가 무엇을 보내든 연결 하나의 비용은 늘지 않는다.

주의사항, 그리고 무엇이 잘못되는가

`proven_net_addr_any`는 모든 인터페이스다. 거기서 listen하는 서버는 그 기계가 닿는 곳이면 어디서든 평문으로 접근할 수 있다. 같은 기계를 위한 서비스라면 `proven_net_addr_loopback`에서 listen하라.

IPv6 리스너는 IPv4를 받지 않는다. 9장이 말하듯 여기서 IPv6 소켓은 모든 플랫폼에서 IPv6 전용이다. 둘 다 받으려면 `proven_http_server_listen`을 두 번 불러라.

`destroy`는 `stop`이 아니다. `stop`은 `run`이 돌아오게 하고, 서버는 멈춘 채로 남는다. `run`을 다시 부르면 곧장 돌아온다. `destroy`는 서버를 해제한다. 핸들러 안에서는 `stop`을 부르고 `destroy`는 절대 부르지 마라.

부드럽게 끝내기: `proven_http_server_shutdown(server, grace_ms)`. `stop`은 연결을 모두 그대로 두고 `destroy`가 끊게 한다. 종료는 그것들을 차례대로 끝낸다. 다음 바퀴부터 리스너가 닫히고 - 새 연결은 거절된다 - 요청이 하나도 오지 않은 연결은 닫힌다. 요청이 오기 시작했거나 핸들러에 가 있는 연결은 그것을 끝내고, 응답에 `Connection: close`가 붙는다. `run`은 남은 것이 없으면 `PROVEN_OK`를, `grace_ms`가 지나면 `PROVEN_ERR_TIMEOUT`을 돌려준다 - 그때 루프가 쥔 것을 닫는다. **기한은 연결을 제한하지 핸들러를 제한하지 않는다**: 워커에서 아직 도는 핸들러는 돌아올 때까지 연결을 쥐고, `destroy`는 언제나처럼 그것을 기다린다. 업그레이드로 넘긴 연결은 프로그램의 것이다: 그런 연결을 쥔 핸들러는 `proven_http_server_shutting_down`을 보고 그것을 닫는다(WebSocket

이면 1001, "going away"로). 노는 연결이 닫히는 바로 그 순간 클라이언트가 요청을 보낼 수 있다. 다시 보낼 수 있으면 다시 보내고, 이 라이브러리의 클라이언트는 한 번 그렇게 한다. 아래 예제가 그렇게 끝난다.

열린 연결이 루프에 치르게 하는 값. 루프는 selector(9장 §9)에서 기다리고 타임아웃을 타이머 휠에 둔다. 그래서 루프 한 바퀴의 비용은 열려 있는 연결 수가 아니라 할 일이 있는 연결 수에 비례한다: 한가한 keep-alive 연결 이만 개를 쥔 채로도, 하나 더 있는 연결의 요청은 아무것도 없을 때와 같은 시간이 걸린다(selector가 `epoll`인 Linux에서 측정했다). Windows에서는 selector가 I/O 완료 포트 위에 지어져 있다. Windows 11 가상 머신에서, 10,000개를 유지한 가운데 요청하는 연결 열 개는 초당 약 52,000번, 중앙값 0.15 ms로 답을 받았다 - 1,000개 가운데서와 같다(15장 14절).

이것으로 한계 하나가 없어지고 둘이 남는다. 연결은 여전히 받아들여진 순간부터 자기 버퍼를 쥐고(아래), 요청은 여전히 핸들러가 도는 동안 스레드 하나를 차지한다(§5). 바쁜 연결 수만 개를 위한 서버에는 그 둘 다 참이 아니어야 하는데, 이 서버는 아직 그것을 제공하지 않는다. 타임아웃은 약 60분의 1초 단위로 지켜진다: 연결은 한도보다 그만큼 더 살 수 있지만 더 짧게 살지는 않는다.

3. 핸들러 안에서

핸들러는 헤드를 읽고 파싱하고 검사한 요청과 함께 불린다. 본문은 읽지 않은 상태다. 읽을지는 핸들러가 정한다.

참조

API	의도	반환
`proven_http_exchange_request(x)`	요청: 메서드, 타깃, 헤더(10장의 `proven_http_request_t`).	포인터. 뷰(view)는 핸들러가 돌아갈 때까지 유효하다.
`proven_http_exchange_peer(x)`	클라이언트의 주소.	`proven_net_addr_t`.
`proven_http_exchange_early(x)`	요청이 핸드셰이크가 끝나기 전에 TLS 조기 데이터(0-RTT)로 왔다 - `early_data`가 있는 `tls` 설정에서, 본문 없는 GET, HEAD, OPTIONS에만: 그 밖의 것이 조기에 오면 서버가 스스로 425로 답한다(14장 6절). GET이 무언가를 바꾸는 핸들러는 조기에 온 것에 425로 답하고, 클라이언트는 그것을 다시 보낸다.	`bool`.
`proven_http_exchange_read(x, dest)`	요청 본문을 더 읽는다.	`proven_result_size_t`: 본문이 끝나면 `.err`가 `EOF`. `TIMEOUT`. `max_body_bytes`를 넘으면 `OUT_OF_BOUNDS`. 청크 틀이 잘못되면 `INVALID_FORMAT`. 클라이언트가 떠났으면 `RESET`.
`proven_http_exchange_respond(x, status, headers, count, body)`	메모리에 있는 본문과 함께 응답 전체를 보낸다.	`proven_err_t`: 응답을 이미 시작했으면 `INVALID_STATE`. 서버가 맡은 헤더나 쓰기 함수가 거절하는 값이면 `INVALID_ARG`. 헤드가 들어가지 않으면 `OUT_OF_BOUNDS`. `TIMEOUT`, `RESET`.
`proven_http_exchange_begin(x, status, headers, count, content_length)`	본문이 조각조각 뒤따르는 응답을 시작한다. 길이를 모르면 `PROVEN_HTTP_LENGTH_UNKNOWN`: 본문은 청크로 나간다.	`respond`와 같다.

`proven_http_exchange_write(x, data)`	그 본문을 더 쓴다.	`proven_err_t`: 알린 길이를 넘으면 `OUT_OF_BOUNDS`. `INVALID_STATE`. `TIMEOUT`, `RESET`.
`proven_http_exchange_end(x)`	끝낸다. 선택 사항이다: 핸들러에서 돌아가도 같은 일이 일어난다.	`proven_err_t`.
`proven_http_exchange_flush(x)`	압축 중인 응답이 붙들고 있는 것을 내보낸다(16장 5절). 다른 응답에는 아무 일도 하지 않는다.	`proven_err_t`: `begin` 전이나 `end` 후면 `INVALID_STATE`. 그 밖에는 쓰기와 같다.
`proven_http_exchange_compress(x)`	클라이언트가 gzip을 받아들이면 이 응답을 압축해 달라고 요청한다. 응답을 시작하기 전에. 16장 6절 - 쓰기 전에 그 절의 주의할 점을 읽을 것.	없음.

주의사항, 그리고 무엇이 잘못되는가

타깃은 클라이언트가 쓴 텍스트다. `request->target`은 보낸 그대로다 - `/a/b?x=1`이거나 절대 URL이고, 디코딩되지 않았다. 비교하기 전에 `proven_url_split_target`으로 나누고, `proven_url_path_resolve` (10장 §3) 없이 그것으로 파일 이름을 만들지 마라.

잘못된 예:

```
snprintf(file, sizeof file, "/var/www%.*s", (int)req->target.size, req->target.ptr);   /* wrong: "/../../etc/passwd" */
```

올바른 예 — 경로는 `proven_url_split_target`으로 얻고, 거기에 `proven_url_path_resolve`를 적용하고, `PROVEN_OK` 결과만 루트에 잇는다.

`Content-Length`, `Transfer-Encoding`, `Connection`, `Date`를 직접 쓰지 마라. 서버가 여러분이 실제로 보내는 것에 맞춰 쓴다. 하나라도 주면 `PROVEN_ERR_INVALID_ARG`이고 아무것도 보내지지 않는다. 서로 어긋나는 `Content-Length` 필드 두 개가 바로 요청 스머글링의 재료가 되는 모호함이고, 여기서는 그것을 만들어 낼 수 없다.

요청에서 가져온 헤더 값이 응답을 쪼갤 수 없다. 줄바꿈이 든 값은 쓰기 함수가 거절하므로, `Location:` 뒤에 클라이언트가 고른 텍스트를 붙여도 헤더나 두 번째 응답이 끼어들 수 없다. 그렇게 거절된 뒤에는 아무것도 보내지지 않았으니 다른 응답 - 오류 페이지 - 을 대신 보낼 수 있다.

쓰기의 결과를 확인하고, 실패하면 멈춰라. `PROVEN_ERR_RESET`과 `PROVEN_ERR_TIMEOUT`은 클라이언트가 사라졌거나 읽지 않는다는 뜻이다. 아무도 받지 않는 큰 본문을 계속 만들어 내는 핸들러는 연결 하나를 - 스레드 하나짜리 모델에서는 서버 전체를 - 아무 보람 없이 붙잡는다.

아무것도 보내지 않고 돌아가면 클라이언트는 `500`을 받는다. 이것은 모든 요청이 답을 받는다는 서버의 약속을 지키는 것이지, 오류를 보내는 방법이 아니다. 본문 읽기가 실패했다면 상태 코드는 그 실패에 맞는 것 - `408`, `413`, `400` - 이 된다.

exchange에서 나온 모든 것은 핸들러가 돌아갈 때 끝난다 - 요청, 그 뷰, exchange 자체. exchange를 다른 스레드에 넘기고 돌아가는 핸들러는 허공을 가리키는 포인터를 넘긴 것이다. 나중에 답하려면 돌아가지 마라. 핸들러 안에서 기다려라(핸들러를 job system에서 돌리면(§5) 작업 스레드 하나의 비용이 든다).

쓸 바이트 수만큼만 길이를 알려라. 더 쓰면 거절된다. 덜 쓰면 고칠 수 없다 - 클라이언트가 나머지를 기다리고 있다 - . 그래서 그 지점에서 연결을 닫고, 클라이언트는 잘린 응답을 본다. 그것이 사실이다.

4. 서버가 묻지 않고 하는 일

형식이 어긋나거나, 모호하거나, 너무 큰 요청은 핸들러에 닿지 않는다.

요청	답
HTTP/1.x로 파싱되지 않는다: LF만 있는 줄 끝, 콜론 앞의 공백, 제어 바이트, 접힌 헤더	`400`

`Host`가 정확히 하나가 아닌 HTTP/1.1	`400`
`Content-Length`와 `Transfer-Encoding`이 함께 있다. 서로 다른 길이 둘. 숫자가 아닌 길이	`400`
`max_head_bytes`보다 큰 헤드, `max_headers`보다 많은 필드	`431`
다른 HTTP 버전	`505`
`chunked`가 아닌 전송 코딩	`501`
`max_body_bytes`보다 크다고 알린 본문	`413`
`head_timeout_ms` 안에 끝나지 않은 헤드	`408`
job system이 요청을 더 받을 수 없다(§5)	`503`

이들은 모두 이 절의 끝에 적은 방식으로 연결을 닫는다. 일부러 엄격하다. 두 서버가 요청 하나를 다르게 읽는 곳에서 공격자는 요청 안에 요청을 숨긴다. 치료법은 한 가지 읽기를 고르는 것이 아니라 모호한 것을 거절하는 것이다.

그리고 핸들러에 닿은 요청의 둘레에서 하는 일:

- **Keep-alive.** 연결은 다음 요청을 위해 열려 있다. 클라이언트가 닫아 달라고 했거나, 요청이 `keep-alive` 없는 HTTP/1.0이었거나, 응답의 틀을 잡을 수 없었던 경우는 예외다.
- **파이프라이닝.** 잇달아 보낸 요청에는 순서대로 답한다.
- **읽지 않은 본문.** 핸들러가 요청 본문을 읽지 않았다면 서버가 읽어서 버린다 - 64 KiB까지 - . 그래야 다음 요청이 제자리에서 시작한다. 그보다 크면 닫는다.
- **`Expect: 100-continue`.** `100 Continue`는 핸들러가 본문을 처음 읽을 때 보낸다. 헤더만 보고 거절하는 핸들러는 본문을 청하지 않으며, 그때는 연결을 닫는다. 본문이 뒤따를지는 클라이언트만 알기 때문이다.
- **`HEAD`.** 핸들러는 `GET`처럼 답한다. 서버는 헤드를 - `Content-Length`를 포함해서 - 보내고 본문은 보내지 않는다.
- **`204`와 `304`**는 본문도 길이도 없이 보낸다.
- **HTTP/1.0.** 길이를 모르는 응답은 1.0 클라이언트에게 청크로 보낼 수 없다. 연결이 닫힐 때까지 보낸다.
- 모든 응답의 **`Date`**.
- **답을 망가뜨리지 않고 닫기.** 서버는 연결을 닫을 때 먼저 그렇다고 알리고 - 보내기를 멈춘다 - , 그다음 클라이언트도 닫을 때까지 최대 1초 동안 클라이언트가 보내는 것을 계속 읽어서 버린다. 읽지 않은 입력을 남긴 채 닫힌 소켓은 reset으로 답하고, reset은 클라이언트의 시스템이 아직 읽지 않은 응답을 버리게 만든다. 그러면 올리는 도중에 `413`을 받은 클라이언트가 그 대신 끊어진 연결을 보게 된다. 그 1초는 고정이며 설정 항목이 아니다.

5. 핸들러를 돌리는 두 가지 방법

설정의 `jobs`가 고른다.

`jobs == NULL`: 루프의 스레드에서. 스레드 하나가 모든 것을 한다. 핸들러가 도는 동안 - 또는 느린 클라이언트가 본문을 보내기를, 응답을 읽기를 기다리는 동안 - 다른 연결은 서비스받지 못한다. 계산하고 답하는 핸들러에는 괜찮다. 단순하고, 잠금이 필요한 것이 없다. 느린 무언가를 기다리는 핸들러에는 틀렸다.

`jobs` 지정: job system의 작업 스레드에서. 요청 하나하나가 6장의 job system에 넘겨지고, 루프는 곧장 연결을 받고 읽는 일로 돌아간다. 핸들러는 자기 작업 스레드에서 본문을 읽고 응답을 쓴다. 핸들러가 돌아갈 때까지 연결은 그 핸들러의 것이다. 핸들러들이 동시에 돌므로:

- 핸들러들이 공유하는 것에는, 공유 데이터에 언제나 필요한 보호가 필요하다.
- 설정의 `alloc`은 여러 스레드에서 불리므로 스레드 안전해야 한다 - 힙(heap) 할당자는 그렇고, 아레나(arena)는 그렇지 않다.
- job 큐가 차면 그 요청에는 `503`으로 답하고 연결을 닫는다. 감당하려는 부하에 맞춰 큐 크기를 정하고, `503`은 서버가 이만하면 충분하다고 말하는 것으로 여겨라.

핸들러는 두 경우에 똑같이 쓴다. 어느 쪽에서도 해서는 안 되는 것은 영원히 막히는 것이다. 돌아가지 않는 핸들러는 연결 하나를 영원히 붙잡고, 그와 함께 작업 스레드 하나 - 또는 서버 - 를 붙잡는다.

느린 클라이언트가 치르게 하는 값은 루프가 아니라 작업 스레드다. 그래도 작업 스레드 하나는 든다. `jobs`를 지정하면, 본문을 느리게 보내는 클라이언트는 그 핸들러를 돌리는 작업 스레드를 `body_timeout_ms`까지 차지한다. 그런 클라이언트가 충분히 많으면 모든 작업 스레드를 차지한다. 타임아웃이 그것을 한정할 뿐 공짜로 만들어 주지는 않는다. 그것이 문제라면 핸들러에서 요청 본문은 꼭 필요할 때에만 읽고, 타임아웃을 짧게 잡아라.

`proven_http_server_destroy`는 작업 스레드에서 돌고 있는 핸들러를 기다린다. job system은 서버를 destroy한 뒤에 닫아라. 앞이 아니다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 온전한 HTTP 서버: 요청에 답하는 함수 하나와, 나머지를 모두 맡는 루프.
 *
 * 핸들러가 보는 요청은 이미 읽고 검사하고 한도를 적용한 것이다. 핸들러는 원하면 본문을
 * 읽고, 응답 하나를 보낸다 - 통째로 또는 조각조각.
 */

typedef struct {
    proven_http_server_t *server;
    int served;
    proven_err_t ended;                 /* run이 돌려준 것 */
} ex_app_t;

static void handle(void *ctx, proven_http_exchange_t *x) {
    ex_app_t *app = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    app->served++;

    /* 타깃은 경로이고 쿼리가 붙을 수도 있다. 비교하기 전에 나눈다. */
    proven_u8str_view_t path, query;
    bool has_query;
    if (proven_url_split_target(req->target, &path, &query, &has_query) != PROVEN_OK) {
        (void)proven_http_exchange_respond(x, 400, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bad
target\n")));
        return;
    }

    if (req->method == PROVEN_HTTP_GET && proven_u8str_view_eq(path, PROVEN_LIT("/"))) {
        /* 가장 단순한 답: 상태 코드, 여러분의 헤더, 메모리에 있는 본문. Content-Length와
         * Date와 Connection은 대신 써 준다. */
        proven_http_header_t type = { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain; charset=utf-8") };
        (void)proven_http_exchange_respond(x, 200, &type, 1, proven_mem_view_from_u8(PROVEN_LIT("hello\n")));

    } else if (req->method == PROVEN_HTTP_POST && proven_u8str_view_eq(path, PROVEN_LIT("/shout"))) {
        /* 본문을 오는 대로 읽고, 길이를 미리 알지 못한 채 조각조각 답한다:
         * 응답은 청크로 나간다. 여기서는 어디에도 본문 전체를 쥐고 있지 않다. */
        if (proven_http_exchange_begin(x, 200, NULL, 0, PROVEN_HTTP_LENGTH_UNKNOWN) != PROVEN_OK) return;
        proven_byte_t piece[256];
        for (;;) {
            proven_result_size_t got = proven_http_exchange_read(x, (proven_mem_mut_t){ piece, sizeof
piece });
            if (got.err != PROVEN_OK) break;          /* PROVEN_ERR_EOF: 본문이 끝났다 */
            for (proven_size_t i = 0; i < got.value; ++i) {
                if (piece[i] >= 'a' && piece[i] <= 'z') piece[i] = (proven_byte_t)(piece[i] - 32);
            }
            if (proven_http_exchange_write(x, (proven_mem_view_t){ piece, got.value }) != PROVEN_OK)
return;   /* 클라이언트가 떠났다 */
        }
        (void)proven_http_exchange_end(x);
```

```
    } else if (proven_u8str_view_eq(path, PROVEN_LIT("/whoami"))) {
        proven_net_addr_t peer = proven_http_exchange_peer(x);
        proven_byte_t text[PROVEN_NET_ADDR_TEXT_MAX];
        proven_size_t n = 0;
        if (proven_net_addr_format(&peer, (proven_mem_mut_t){ text, sizeof text }, &n) != PROVEN_OK)
return;   /* 아무것도 보내지 않고 돌아가면: 500 */
        (void)proven_http_exchange_respond(x, 200, NULL, 0, (proven_mem_view_t){ text, n });

    } else if (proven_u8str_view_eq(path, PROVEN_LIT("/quit"))) {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        /* 부드럽게 끝낸다: 열린 것은 - 이 응답도 - 끝나고, 그다음 run이 돌아온다. */
        if (proven_http_server_shutdown(app->server, 5000) != PROVEN_OK) proven_http_server_stop(app->server);

    } else {
        (void)proven_http_exchange_respond(x, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("no such
page\n")));
    }
}

static void serve(void *arg) {
    ex_app_t *app = arg;
    app->ended = proven_http_server_run(app->server);   /* 종료가 끝날 때까지 */
}

/* `path`를 GET 또는 POST하고 본문 전체를 비교한다. */
static bool fetch_is(proven_http_client_t *client, proven_u16 port, const char *method, const char *path,
const char *body, proven_u16 status, const char *expect) {
    char url[128];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    proven_http_client_request_t req = {
        .method = proven_u8str_view_from_cstr(method),
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .body = proven_mem_view_from_u8(proven_u8str_view_from_cstr(body)),
    };
    proven_http_client_response_t resp;
    bool ok = proven_http_client_send(client, &req, &resp) == PROVEN_OK && resp.status == status;
    proven_u8str_t text = {0};
    ok = ok && proven_http_client_read_all(&resp, proven_heap_allocator(), &text, 4096) == PROVEN_OK;
    ok = ok && (expect == NULL || proven_u8str_view_eq(proven_u8str_as_view(&text),
proven_u8str_view_from_cstr(expect)));
    proven_u8str_destroy(proven_heap_allocator(), &text);
    proven_http_client_finish(&resp);
    return ok;
}

int main(void) {
    static ex_app_t app;
    proven_allocator_t heap = proven_heap_allocator();

    /* 설정을 0으로 채우고 필요한 것만 정한다. 0인 필드에는 모두 기본값이 있다. */
    proven_http_server_config_t config = {0};
    config.alloc = heap;
    config.handler = handle;
    config.handler_ctx = &app;
    config.max_body_bytes = 64 * 1024;        /* 이보다 큰 요청 본문에는 413으로 답한다 */
    config.head_timeout_ms = 5000;            /* 클라이언트가 요청 헤드를 보내는 데 허락된 시간 */
    proven_err_t err = proven_http_server_create(&config, &app.server);
    EXAMPLE_REQUIRE(err == PROVEN_OK, "a server");
```

```
    /* 포트 0은 시스템이 고르게 한다. 고른 값은 `at`에 담긴다. */
    proven_net_addr_t at;
    err = proven_http_server_listen(app.server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(app.server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    /* 자기 루프가 있는 프로그램은 poll을 부른다: 마감 시각으로 묶인 한 바퀴. */
    EXAMPLE_REQUIRE(proven_http_server_poll(app.server, proven_net_deadline_in(10)) == PROVEN_ERR_TIMEOUT,
"nobody has connected yet");

    /* 대부분의 프로그램은 run을 부른다. 여기서는 다른 스레드에서 돌려서, 이 스레드가 클라이언트 노릇을 한다.
*/
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &app) == PROVEN_OK, "the server loop is
started");

    proven_http_client_config_t client_config = {0};
    client_config.alloc = heap;
    client_config.max_idle_connections = 2;
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "a client");

    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/", "", 200, "hello\n"), "GET /");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "POST", "/shout", "a body, read and answered in pieces", 200, "A
BODY, READ AND ANSWERED IN PIECES"), "POST /shout");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/whoami?verbose=1", "", 200, NULL), "GET /whoami");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/nothing-here", "", 404, "no such page\n"), "a 404 is
the handler's to send");
    EXAMPLE_REQUIRE(fetch_is(client, at.port, "GET", "/quit", "", 200, "bye\n"), "GET /quit");

    /* 핸들러가 종료를 요청했다: 열린 것이 남지 않았을 때 run이 돌아왔다. */
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.served == 5, "five requests reached the handler");
    EXAMPLE_REQUIRE(app.ended == PROVEN_OK && proven_http_server_connection_count(app.server) == 0, "run
returned PROVEN_OK: everything open had finished");
    /* run 대신 poll을 스스로 부르는 프로그램은 같은 것을 proven_http_server_drained에 묻는다. */
    proven_err_t how = PROVEN_ERR_INVALID_STATE;
    EXAMPLE_REQUIRE(proven_http_server_shutting_down(app.server) && proven_http_server_drained(app.server,
&how) && how == PROVEN_OK, "the shutdown is over, and nothing was cut");

    proven_http_client_destroy(client);
    proven_http_server_destroy(app.server);   /* 남은 것을 닫는다 */
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

6. 클라이언트

```
proven_http_client_config_t config = {0};
config.alloc = proven_heap_allocator();
proven_http_client_create(&config, &client);
```

```
proven_http_client_response_t resp;
if (proven_http_client_get(client, PROVEN_LIT("http://example.com/"), &resp) == PROVEN_OK) {
    /* resp.status, resp.headers; then proven_http_client_read until PROVEN_ERR_EOF */
}
proven_http_client_finish(&resp);
proven_http_client_destroy(client);
```

참조

API	의도	반환
`proven_http_client_create(&config, &client)`	클라이언트를 만든다. 설정의 문자열은 복사된다.	`proven_err_t`: 할당자가 유효하지 않거나, 프록시가 `http`나 `socks5`가 아니거나, 자격 증명에 제어 문자가 있으면 `INVALID_ARG`. `NOMEM`.
`proven_http_client_destroy(client)`	한가한 연결을 닫고 해제한다. 모든 응답을 먼저 finish하라.	없음.
`proven_http_client_send(client, &request, &response)`	요청을 보내고 응답의 헤드를 읽는다.	`proven_err_t`. §7을 보라.
`proven_http_client_get(client, url, &response)`	URL만으로 하는 `send`.	같다.
`proven_http_client_read(&response, dest)`	본문을 더 읽는다.	`proven_result_size_t`: 끝에서 `.err`가 `EOF`. 본문이 중간에 끊겼으면 `RESET`. `TIMEOUT`. `INVALID_FORMAT`.
`proven_http_client_read_all(&response, alloc, &out, max_bytes)`	본문의 나머지를 읽어 문자열에 덧붙인다.	`proven_err_t`: 본문이 `max_bytes`보다 길면 `OUT_OF_BOUNDS` - 그때 `out`에는 그만큼이 들어 있다.
`proven_http_client_finish(&response)`	응답을 놓아준다. 연결은 남겨 두거나 닫는다. send가 실패한 뒤에도 안전하다.	없음.

```
typedef struct {
    proven_allocator_t alloc;               /* required */
    proven_u32 connect_timeout_ms;          /* 0: 30 s */
    proven_u32 io_timeout_ms;               /* 0: 30 s, for each read and each write */
    proven_size_t max_head_bytes;           /* 0: 16 KiB */
    proven_u32 max_redirects;               /* 0: none are followed */
    proven_size_t max_idle_connections;     /* 0: every connection is closed after use */
    proven_u8str_view_t proxy;              /* "http://host:port" or "socks5://host:port" */
    proven_u8str_view_t no_proxy;           /* hosts reached without it: "localhost, .corp.example" */
    proven_u8str_view_t username, password; /* offered when a server answers 401 */
    proven_u8str_view_t user_agent;         /* empty: none is sent */
    proven_http_cookie_jar_t *cookies;      /* NULL: cookies are ignored */
    proven_http_tls_wrap_fn tls_wrap;       /* NULL: an https URL is PROVEN_ERR_UNSUPPORTED */
    void *tls_ctx;
    proven_u32 expect_timeout_ms;           /* 0: 1 s - see expect_continue in section 7 */
} proven_http_client_config_t;
```

0으로 채운 설정은 가장 적게 한다. `alloc`만 정하면 클라이언트는 리다이렉트를 따르지 않고, 연결을 남겨 두지 않고, 자격 증명을 내놓지 않고, 쿠키를 저장하지 않고, `User-Agent`를 보내지 않는다. 그 하나하나는 여러분이 알고서 켜는 것이다.

클라이언트 하나에 스레드 하나. 클라이언트는 여러 스레드가 동시에 쓰기에 안전하지 않다. 스레드마다 하나씩 주어라. 값싸다.

7. 요청 하나, 응답 하나

```
typedef struct {
    proven_u8str_view_t method;             /* empty: GET */
    proven_u8str_view_t url;                /* absolute: http://host[:port]/path?query */
    const proven_http_header_t *headers;    /* extra header fields */
    proven_size_t header_count;
    proven_mem_view_t body;                 /* a body in memory: sent with Content-Length */
    proven_reader_t body_stream;            /* or a stream: sent chunked */
    bool expect_continue;                   /* ask the server before sending the body */
} proven_http_client_request_t;

typedef struct {
    proven_u16 status;
    proven_u8str_view_t reason;
    const proven_http_header_t *headers;
    proven_size_t header_count;
    proven_u8str_view_t url;                /* where this response came from, after redirects */
    proven_u32 redirects;
    void *internal;
} proven_http_client_response_t;
```

`proven_http_client_send`는 상태 코드와 헤더가 도착하면 돌아온다. 본문은 아직 연결 위에 있고, 여러분이 자기 버퍼에 조각조각 읽는다. 그래서 응답이 얼마나 크든 드는 메모리는 여러분이 고른 만큼이고, 내려받은 것을 곧장 파일로 보낼 수 있다. 스트림에서 오는 요청 본문은 `proven_reader_t`, 곧 5장의 읽기 스트림(reader)이다.

오류 상태 코드는 오류가 아니다. `404`나 `500`은 응답이다. `send`는 `PROVEN_OK`를 돌려주고 여러분이 `status`를 본다. `send`가 실패하는 것은 들여다볼 응답이 없을 때다.

`send`의 반환	이유
`PROVEN_ERR_INVALID_FORMAT`	URL이 절대 URL이 아니거나, 응답이 HTTP가 아니었다 - 또는 최종 응답 없이 중간(`1xx`) 응답만 여덟 개를 넘었다.
`PROVEN_ERR_UNSUPPORTED`	`http`가 아닌 스킴 - 또는 `tls_wrap` 없는 `https`.
`PROVEN_ERR_INVALID_ARG`	클라이언트가 직접 쓰는 헤더를 주었다: `Host`, `Content-Length`, `Transfer-Encoding`, `Connection`. 또는 본문과 스트림을 함께 주었다.
`PROVEN_ERR_NOT_FOUND`	호스트 이름이 해석되지 않는다.
`PROVEN_ERR_REFUSED`, `PROVEN_ERR_UNREACHABLE`, `PROVEN_ERR_TIMEOUT`, `PROVEN_ERR_RESET`	연결: 9장에서의 뜻 그대로.
`PROVEN_ERR_OUT_OF_BOUNDS`	요청이나 응답 헤드가 `max_head_bytes`보다 크다.
`PROVEN_ERR_PERMISSION`	프록시가 거절했다.
`PROVEN_ERR_UNTRUSTED`, `PROVEN_ERR_EXPIRED`, `PROVEN_ERR_NOT_YET_VALID`, `PROVEN_ERR_NAME_MISMATCH`, `PROVEN_ERR_PROTOCOL`	`tls_wrap`에서: 서버를 검증할 수 없었거나 TLS 핸드셰이크가 실패했다(14장 8절). 요청의 어떤 부분도 보내지지 않았다.

주의사항, 그리고 무엇이 잘못되는가

언제나 `finish`를 불러라. 응답은 finish할 때까지 연결과 메모리를 붙잡는다. `finish`는 `send`가 실패한 응답에도 안전하므로, 단순한 모양이 옳은 모양이다: `send`하고, 성공했으면 쓰고, 어느 쪽이든 `finish`한다.

`PROVEN_ERR_EOF`는 본문의 끝이다. `PROVEN_ERR_RESET`은 아니다. 알린 길이보다 먼저 멈춘 본문 - 서버가 죽었거나 네트워크가 끊겼다 - 은 `PROVEN_ERR_RESET`으로 보고되고 결코 `EOF`로 보고되지 않는다. `EOF`만을 온전한 본문으로 다뤄라.

잘못된 예:

```
while (proven_http_client_read(&resp, buf).err == PROVEN_OK) save(buf);   /* wrong: stops on RESET and calls
the file done */
commit_download();
```

올바른 예 — 결과를 간직하고, 그것이 `PROVEN_ERR_EOF`일 때에만 확정한다.

`read_all`에는 한도가 필요하고, 그것은 여러분이 고른다. 얼마나 보낼지는 서버가 정한다. `max_bytes`는 여러분이 쥘 최대치다. 그보다 긴 본문은 `PROVEN_ERR_OUT_OF_BOUNDS`이지, 기계에 메모리가 남지 않을 때까지 자라는 할당이 아니다.

응답의 뷰는 응답과 함께 죽는다. `headers`, `reason`, `url`은 `finish`가 해제하는 메모리를 가리킨다. 간직할 것은 복사하라.

스트림 본문은 한 번만 보낸다. `body_stream`에서 읽은 본문은 다시 읽을 수 없으므로, 그런 요청은 되풀이되지 않는다 - 본문이 필요한 리다이렉트에도, 인증 요구에도, 남겨 둔 연결이 죽은 것으로 드러났을 때에도. 되풀이를 일으켰을 그 응답을 여러분이 받고, 여러분이 정한다.

큰 본문을 보내기 전에 묻기. `expect_continue`를 켜면 클라이언트는 헤드를 `Expect: 100-continue`와 함께 보내고, 서버가 말할 때까지 본문을 붙들어 둔다.

서버가	클라이언트는
`100 Continue`로 답하면	본문을 보내고, 호출은 그 뒤에 오는 응답을 돌려준다.
최종 응답 - `401`, `413`, `417` - 으로 답하면	본문을 한 바이트도 보내지 않는다. 그 답이 응답이다. 그 뒤 연결은 답이 뭐라고 하든 간직하지 않는다. 스트림 본문은 읽히지 않은 채다.
`expect_timeout_ms`(정하지 않으면 1초) 동안 아무 말이 없으면	본문을 그냥 보낸다: 이 규칙을 모르는 서버는 본문을 기다리고 있다.

왕복 한 번이 더 드니, 보내지 않을 만한 값어치가 있는 본문에 쓴다: 크기나 빠진 자격 증명 때문에 서버가 거절할 수 있는 업로드다. 본문이 없는 요청에는 아무 일도 하지 않는다. 플래그를 켜고, 필드를 직접 쓰지는 마라: 여러분의 헤더에 들어 있는 `Expect`는 그대로 보내지고, 본문은 곧바로 뒤따른다. 이 라이브러리의 서버들은 여기에 답한다(4절).

업로드 도중에 온 답. `expect_continue`가 없어도 서버는 본문을 다 받기 전에 답하고 읽기를 멈출 수 있다. 그러면 클라이언트의 쓰기가 실패한다 - 그리고 클라이언트는 무엇을 보고하기 전에 서버가 한 말을 읽는다: 도착해 있던 `413`이 호출의 결과이지, `PROVEN_ERR_RESET`이 아니다. 답이 없을 때에만 실패한 쓰기가 오류다. 한계가 둘 있고, 둘 다 이 라이브러리가 아니라 시스템의 것이다: 시간이 다 된 쓰기는 두 번째로 기다리지 않고 곧바로 보고한다. 그리고 Windows에서는 리셋이, 도착했지만 아직 읽지 않은 것을 버리므로 거기서는 답을 잃을 수 있다 - 그것에 기대지 않는 방법이 `expect_continue`다.

`io_timeout_ms`는 기다림 하나하나를 한정하지, 요청 전체를 한정하지 않는다. 10초마다 한 바이트를 보내는 서버는 30초 타임아웃에 걸리지 않는다. 교환 전체에 한도를 두려면 `send` 전에 시각을 적어 두고, 충분히 기다렸으면 읽기를 멈춰라.

이름 해석에는 9장에서처럼 기한이 없다. `connect_timeout_ms`는 이름이 해석된 뒤부터 센다.

8. 리다이렉트, 인증 요구, 쿠키

리다이렉트(`301`, `302`, `303`, `307`, `308`)는 `max_redirects`까지 따르고, 각 `Location`은 그것을 보낸 URL에 대해 해소한다. `response.url`과 `response.redirects`가 어디에 어떻게 도착했는지 말해 준다.

상태 코드	다음 요청
`303`	본문 없는 `GET` - 메서드가 무엇이었든
`301`, `302`	메서드가 `POST`였으면 본문 없는 `GET`. 아니면 같은 요청을 다시
`307`, `308`	본문을 포함해 같은 요청을 다시

다음 경우에는 리다이렉트를 따르지 않고 그 3xx 응답을 여러분에게 넘긴다.

- 한도에 닿았다.
- https에서 http로 이어진다 - 클라이언트는 서버가 시켰다고 암호화된 연결에서 내려오지 않는다.
- http와 https가 아닌 스킴으로 이어진다.
- 요청을 다시 보내야 하는데 본문이 스트림이었다.

자격 증명은 그것이 주어진 호스트에 남는다. 리다이렉트가 다른 호스트나 포트나 스킴으로 이어지면, 여러분이 준 Authorization, Cookie, Proxy-Authorization 헤더는 두고 가고, 새 곳에서 온 401에는 설정된 사용자 이름과 비밀번호로 답하지 않는다. 토큰을 맡길 만큼 믿는 서버라도, 그 토큰을 자기가 고른 서버로 보내게 할 수는 없어야 한다.

클라이언트가 확인하지 않는 것은 리다이렉트가 어디로 이어지는가다. 서버는 http://127.0.0.1/이나 여러분의 네트워크 안쪽 주소로 리다이렉트할 수 있다. URL이 믿지 못할 사람에게서 온다면 - "이 링크를 가져와" 기능 - 모든 홉의 목적지를 확인하는 것은 여러분 몫이다. max_redirects를 0으로 두고, Location을 직접 읽고, 정하라.

인증 요구. username을 정해 두면 Digest나 Basic 요구가 담긴 401에 한 번 답한다. Digest가 있으면 Digest로, 아니면 Basic으로. stale이라고 보고된 Digest nonce에는 한 번 더 답한다. 그래서 틀린 비밀번호의 값은 요청 두 번이지 무한 반복이 아니다. 자격 증명은 서버가 요구하기 전에는 결코 보내지 않는다 - 그리고 평문 HTTP에서 Basic으로 요구받으면 읽을 수 있는 채로 보낸다. §1이 그대로 적용된다.

쿠키. cookies에 보관함을 주면 모든 응답의 Set-Cookie 헤더가 저장되고 모든 요청이 일치하는 쿠키를 싣는다. 요청에 여러분이 직접 넣은 Cookie 헤더는 그 요청에서 보관함의 것을 대신한다. 보관함은 10장 §12의 호스트 전용 보관함이고, 거기 적힌 것이 모두 따라온다.

9. 연결과 프록시

재사용. max_idle_connections가 0보다 크면, 응답을 끝까지 읽은 연결은 클라이언트에게 돌아가 같은 스킴·호스트·포트로 가는 다음 요청을 실어 나른다. 클라이언트가 한가한 연결을 모아 두는 이곳을 흔히 연결 풀(pool)이라 부른다. 본문을 읽기 전에 finish한 응답의 연결은 재사용할 수 없고 - 본문의 나머지가 아직 그 위에 있다 - 닫힌다. 연결을 남기려면 finish 전에 PROVEN_ERR_EOF까지 읽어라.

남겨 둔 연결은 죽어 있을 수 있다. 서버는 한가한 연결을 예고 없이 닫는다. 재사용한 연결에서 답의 바이트가 하나도 오기 전에 요청이 실패하면 클라이언트는 새 연결로 한 번 다시 보낸다. 아무것도 돌아오지 않았기 때문에 안전한 것이다 - 그리고 두 번 보낼 수 없는 스트림 본문에는 하지 않는다.

프록시. proxy는 http://[user:password@]host:port 또는 socks5://[user:password@]host:port다.

프록시	http URL일 때	https URL일 때
HTTP	프록시에 URL 전체를 보내고, 프록시가 가져온다.	프록시에 host:port로의 CONNECT를 요청한다. 요청은 그 터널을 지나간다.
SOCKS5	host:port로의 터널, 그다음 평범한 요청.	같다. 그 안에 TLS가 들어간다.

- 대상의 이름은 여기서 해석하지 않고 프록시에 넘긴다 - SOCKS5에는 이름으로 넘기며, 다른 곳에서 socks5h라 부르는 것이 이것이다. 클라이언트를 돌리는 기계는 그 이름을 조회하지 않는다.
- 프록시 URL의 자격 증명은 프록시에게 - HTTP 프록시에는 Proxy-Authorization: Basic으로, SOCKS5에는 RFC 1929의 사용자/비밀번호 방식으로 - 가고 다른 누구에게도 가지 않는다. 터널 안으로 들어가 원 서버까지 가지 않는다. 두 형태 모두 프록시까지는 읽을 수 있는 채로 네트워크를 건넌다.
- 거절하는 프록시(407, 403, SOCKS5의 "허용되지 않음")는 PROVEN_ERR_PERMISSION이다.
- HTTP 프록시는 평문 http 교환의 모든 것을 보고, 바꿀 수 있다. 프록시란 그런 것이다.

프록시 없이 닿는 호스트. no_proxy는 쉼표로 나눈 목록이다.

항목	면제되는 것

`example.org` 또는 `.example.org`	그 이름과 그 아래의 모든 이름 - `www.example.org`는 되고 `notexample.org`는 아니다. 대소문자는 가리지 않는다.
`10.0.0.5`, `::1` 또는 `[::1]`	그 주소, URL에 쓴 것과 같은 표기로. 주소는 통째로 비교한다: 범위도 `10.0.0.0/8`도 없다.
`*`	모든 호스트: 프록시를 쓰지 않는다.

포트는 항목의 일부가 아니다. 목록에 있는 호스트에는 직접 연결한다 - 따라서 그 이름은 이 기계에서 찾게 되는데, 그것이 프록시가 다른 때에는 막아 주는 바로 그 일이다.

프록시 설정을 환경에서 읽지 않는다. `HTTP_PROXY`와 그 친척들은 뜻밖의 일을 많이 일으켜 온 관례다. 여러분의 프로그램이 그것을 따른다면, 프로그램이 읽어서 `proxy`와 `no_proxy`를 직접 정한다.

10. TLS가 붙을 자리

```
typedef proven_err_t (*proven_http_tls_wrap_fn)(void *ctx, proven_transport_t plain,
                                                proven_u8str_view_t host,
                                                proven_net_deadline_t until,
                                                proven_transport_t *out);
```

`https` URL이면 클라이언트는 연결하고 - 프록시 터널이 있으면 그것을 거쳐서 - , 연결된 전송과 호스트 이름으로 `tls_wrap`을 부른다. 그 함수가 핸드셰이크를 하고 암호화하는 전송을 돌려준다. 클라이언트는 그 위에서 HTTP를 말하고 차이를 알지 못한다. 9장의 전송 인터페이스가 그것이 만들어진 목적대로 일하는 것이다.

라이브러리가 하나를 제공한다: `proven_tls_http_wrap`, 컨텍스트는 `proven_tls_config_t *`다 (14장 3절). 이 이음매가 인터페이스로 남아 있는 까닭은, 그 둘레의 클라이언트 동작 - 리다이렉트에서 내려가지 않기, 자격 증명을 원래 자리에 두기, 핸드셰이크보다 터널이 먼저 - 이 뒤에 어떤 TLS가 있는지에 기대지 않게 하고, 다른 TLS 구현을 가진 프로그램이 대신 그것을 붙일 수 있게 하기 위해서다.

조기 데이터. 그 설정에 `early_data`가 있으면 wrap은 서버 이름의 가장 새 세션으로 재개하고, 티켓이 허락하면 아무것도 보내지 않고 돌아온다: 이 클라이언트가 다음에 쓰는 요청 헤드가 ClientHello와 함께 조기 데이터로 가서, 제 몫의 왕복 없이 답을 받는다. 본문 없는 GET, HEAD, OPTIONS만 조기에 간다 - 그 밖의 것은 클라이언트가 핸드셰이크를 먼저 끝낸다 - 그리고 425 Too Early로 답한 서버에는 요청을 새 연결에서 조기가 아니게 한 번 더 보낸다(14장 6절).

직접 쓴다면 반드시 서버를 검증해야 한다. `host`를 넘기는 것은 인증서를 그것에 견주어 확인하라는 뜻이다. 검증 없이 암호화만 하는 wrap은, 그 연결 한가운데 있는 누군가만 빼고 모두에게서 비밀인 연결을 준다 - 그리고 `PROVEN_OK`를 돌려주므로 그 뒤의 어느 것도 알아챌 수 없다. 상대를 검증할 수 없으면 `PROVEN_ERR_UNTRUSTED`를 돌려줘라. 클라이언트는 그것을 위로 넘기고 아무것도 보내지 않는다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * HTTP 클라이언트: URL에서 바이트까지. 그 사이에 놓인 것 - 리다이렉트, 로그인, 쿠키 - 은
 * 클라이언트가 처리하고 응답에 드러난다.
 *
 * 상대 서버는 같은 프로그램 안, 다른 스레드에 있어서 이 예제에는 네트워크가 필요 없다.
 * 파일의 앞 절반이 그 서버의 핸들러이고, 클라이언트는 main에 있다.
 */

static proven_http_server_t *g_server;

static void site(void *ctx, proven_http_exchange_t *x) {
    (void)ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    proven_u8str_view_t value;

    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/old-home"))) {
        proven_http_header_t h = { PROVEN_LIT("Location"), PROVEN_LIT("/home") };
        (void)proven_http_exchange_respond(x, 301, &h, 1, (proven_mem_view_t){0});
```

```
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/home"))) {
        proven_http_header_t h = { PROVEN_LIT("Set-Cookie"), PROVEN_LIT("visited=yes; Path=/") };
        (void)proven_http_exchange_respond(x, 200, &h, 1, proven_mem_view_from_u8(PROVEN_LIT("welcome
home")));
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/cookie"))) {
        bool has = proven_http_header_find(req->headers, req->header_count, PROVEN_LIT("Cookie"), &value);
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(has ? value :
PROVEN_LIT("none")));
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/private"))) {
        /* "Basic" + base64("ada:lovelace") */
        if (proven_http_header_find(req->headers, req->header_count, PROVEN_LIT("Authorization"), &value) &&
            proven_u8str_view_eq(value, PROVEN_LIT("Basic YWRhOmxvdmVsYWNl"))) {
            (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("the
private page")));
        } else {
            proven_http_header_t h = { PROVEN_LIT("WWW-Authenticate"), PROVEN_LIT("Basic
realm=\"example\"") };
            (void)proven_http_exchange_respond(x, 401, &h, 1, proven_mem_view_from_u8(PROVEN_LIT("who are
you?")));
        }
    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/count"))) {
        /* 요청 본문이 몇 바이트였는지로 답한다. */
        proven_byte_t piece[512];
        proven_u64 total = 0;
        for (;;) {
            proven_result_size_t got = proven_http_exchange_read(x, (proven_mem_mut_t){ piece, sizeof
piece });
            if (got.err != PROVEN_OK) break;
            total += got.value;
        }
        char text[32];
        int n = snprintf(text, sizeof text, "%llu bytes", (unsigned long long)total);
        (void)proven_http_exchange_respond(x, 200, NULL, 0, (proven_mem_view_t){ (const proven_byte_t *)text,
(proven_size_t)n });
    } else {
        (void)proven_http_exchange_respond(x, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("not found")));
    }
}

static void serve(void *arg) {
    (void)arg;
    (void)proven_http_server_run(g_server);
}

static char g_url[128];
static proven_u8str_view_t url_for(proven_u16 port, const char *path) {
    int n = snprintf(g_url, sizeof g_url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    return (proven_u8str_view_t){ (const proven_byte_t *)g_url, (proven_size_t)n };
}

static bool view_is(proven_u8str_view_t v, const char *text) {
    return proven_u8str_view_eq(v, proven_u8str_view_from_cstr(text));
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- 말을 걸 상대가 되어 줄 서버 ----------------------------------------
    proven_http_server_config_t server_config = {0};
    server_config.alloc = heap;
```

```
    server_config.handler = site;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &g_server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(g_server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(g_server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, NULL) == PROVEN_OK, "its loop runs on
another thread");

    // ---- 클라이언트 ------------------------------------------------------------
    proven_http_cookie_jar_t jar;
    EXAMPLE_REQUIRE(proven_http_cookie_jar_init(&jar, heap, 32) == PROVEN_OK, "a cookie jar");

    /* 0으로 채우고 필요한 것만 정한다. 할당자 말고 모든 필드가 0이면 클라이언트는
     * 리다이렉트를 따르지 않고, 연결을 남겨 두지 않고, 자격 증명을 보내지 않고, 쿠키를 갖지 않는다. */
    proven_http_client_config_t config = {0};
    config.alloc = heap;
    config.max_redirects = 5;
    config.max_idle_connections = 4;          /* 다음 요청을 위해 연결을 남겨 둔다 */
    config.io_timeout_ms = 5000;              /* 어떤 읽기도 쓰기도 이보다 오래 기다리지 않는다 */
    config.user_agent = PROVEN_LIT("manual-example/1.0");
    config.username = PROVEN_LIT("ada");      /* 서버가 요구할 때에만 내놓는다 */
    config.password = PROVEN_LIT("lovelace");
    config.cookies = &jar;
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&config, &client) == PROVEN_OK, "a client");

    /* GET. 상태 코드와 헤더가 도착하면 호출이 돌아온다. 본문은 아직 회선 위에 있고,
     * 여러분의 속도로 여러분의 버퍼에 읽는다. */
    proven_http_client_response_t resp;
    err = proven_http_client_get(client, url_for(at.port, "/old-home"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && resp.status == 200, "the 301 was followed to a 200");
    EXAMPLE_REQUIRE(resp.redirects == 1 && resp.url.size > 5 && view_is((proven_u8str_view_t){ resp.url.ptr +
resp.url.size - 5, 5 }, "/home"),
                    "and the response says where it ended up");
    proven_byte_t buf[64];
    proven_size_t have = 0;
    for (;;) {
        proven_result_size_t got = proven_http_client_read(&resp, (proven_mem_mut_t){ buf + have, sizeof buf -
have });
        if (got.err == PROVEN_ERR_EOF) break;             /* 본문을 끝까지 읽었다 */
        EXAMPLE_REQUIRE(got.err == PROVEN_OK, "a piece of the body");
        if (got.err != PROVEN_OK) break;                  /* PROVEN_ERR_RESET: 중간에 끊겼다 */
        have += got.value;
    }
    EXAMPLE_REQUIRE(view_is((proven_u8str_view_t){ buf, have }, "welcome home"), "the body");
    /* 반드시 finish한다. 끝까지 읽었다면 연결은 다시 쓰이도록 클라이언트에게 돌아간다. */
    proven_http_client_finish(&resp);

    /* 그 응답이 쿠키를 심었다. 보관함이 그것을 갖고 있고 다음 요청이 실어 보낸다. */
    EXAMPLE_REQUIRE(proven_http_cookie_jar_count(&jar) == 1, "one cookie in the jar");
```

```
    proven_u8str_t text = {0};
    err = proven_http_client_get(client, url_for(at.port, "/cookie"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_http_client_read_all(&resp, heap, &text, 1024) == PROVEN_OK,
"read_all: the whole body, up to a limit you name");
    EXAMPLE_REQUIRE(view_is(proven_u8str_as_view(&text), "visited=yes"), "the server saw the cookie");
    proven_http_client_finish(&resp);

    /* 로그인 뒤에 있는 페이지. 첫 답은 인증 요구가 담긴 401이다. 클라이언트가 설정된
     * 자격 증명으로 그에 답하고, 여러분에게는 결과만 보인다. */
    EXAMPLE_REQUIRE(proven_u8str_reset(&text) == PROVEN_OK, "reuse the string");
    err = proven_http_client_get(client, url_for(at.port, "/private"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && resp.status == 200 && proven_http_client_read_all(&resp, heap, &text,
1024) == PROVEN_OK, "let in");
    EXAMPLE_REQUIRE(view_is(proven_u8str_as_view(&text), "the private page"), "the page");
    proven_http_client_finish(&resp);

    /* 모든 것을 정한 요청: 메서드, 추가 헤더, 그리고 스트림에서 오는 본문. 스트림 본문은
     * 청크로 나간다 - 그래서 크기를 모르는 파일이나 파이프도 올릴 수 있다. */
    proven_reader_view_t source;
    proven_http_header_t headers[1] = { { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain") } };
    proven_http_client_request_t req = {
        .method = PROVEN_LIT("POST"),
        .url = url_for(at.port, "/count"),
        .headers = headers,
        .header_count = 1,
        .body_stream = proven_reader_from_view(&source, PROVEN_LIT("nineteen bytes here")),
    };
    EXAMPLE_REQUIRE(proven_u8str_reset(&text) == PROVEN_OK, "reuse the string");
    err = proven_http_client_send(client, &req, &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && proven_http_client_read_all(&resp, heap, &text, 1024) == PROVEN_OK, "a
POST from a stream");
    EXAMPLE_REQUIRE(view_is(proven_u8str_as_view(&text), "19 bytes"), "the server counted them");
    proven_http_client_finish(&resp);

    /* 오류 상태 코드는 응답이지 오류가 아니다: 호출은 성공하고 여러분이 status를 본다. */
    err = proven_http_client_get(client, url_for(at.port, "/missing"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_OK && resp.status == 404, "a 404 arrives as a response");
    proven_http_client_finish(&resp);

    /* 오류란 들여다볼 응답이 없을 때다. */
    err = proven_http_client_get(client, PROVEN_LIT("https://127.0.0.1/"), &resp);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_UNSUPPORTED, "https without a tls_wrap: the client says so rather than
sending in the clear");
    proven_http_client_finish(&resp);         /* 실패한 뒤에 불러도 해롭지 않다 */

    proven_u8str_destroy(heap, &text);
    proven_http_client_destroy(client);
    proven_http_cookie_jar_destroy(&jar);

    proven_http_server_stop(g_server);
    proven_job_group_wait(threads, &running);
    proven_http_server_destroy(g_server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

11. 프로토콜 바꾸기

요청은 연결이 HTTP를 그만 실어 나르게 해 달라고 청할 수 있다: `Connection: Upgrade`와 `Upgrade: <protocol>`. 서버가 동의하면 `101 Switching Protocols`로 답하고, 그 뒤의 모든 바이트는 다른 프로토콜의 것이다. 두 드라이버 모두 그 시점에 연결을 넘겨줄 수 있다.

API	의도	반환
`proven_http_exchange_upgrade(x, protocol, headers, count, &transport, &early)`	핸들러 안에서: `Upgrade: protocol`과 `Connection: Upgrade`를 단 `101`로 답하고, 연결을 서버에서 꺼낸다.	`proven_err_t`: 응답을 이미 시작했거나, 요청에 읽지 않은 본문이 있거나, HTTP/1.0이면 `INVALID_STATE`. 서버가 맡은 헤더(`Upgrade` 포함)면 `INVALID_ARG`. `NOMEM`. `OUT_OF_BOUNDS`. `TIMEOUT`, `RESET`.
`request.upgrade`	클라이언트 요청에서: 요청할 프로토콜. `Connection: Upgrade`와 `Upgrade`는 클라이언트가 직접 쓴다.	-
`proven_http_client_upgrade(&response, &transport, &early)`	`101`을 받은 뒤: 연결을 클라이언트에서 꺼낸다.	`proven_err_t`: 응답이 `101`이 아니거나 이미 꺼냈으면 `INVALID_STATE`.

어느 쪽이든 연결을 **소유(owned)**하는 `proven_transport_t`를 받는다. 그것은 더 이상 서버나 클라이언트의 것이 아니다. `max_connections`에 세어지지 않고, 그들의 타임아웃이 적용되지 않고, 재사용되지 않으며, 여러분이 `proven_transport_close`를 부를 때까지 열려 있다. 서버 핸들러는 돌아간 뒤에도 그것을 갖고 있어도 된다.

`early`는 다른 프로토콜의 시작이다. 상대는 핸드셰이크가 끝나기를 기다렸다가 보낼 의무가 없으므로, 새 프로토콜의 첫 바이트들이 HTTP 헤드와 같은 읽기에 도착할 수 있다. 그것이 `early`에 있다 - exchange와 함께, 또는 `proven_http_client_finish`에서 죽는 뷰다 - . 이것을 무시하는 프로그램은 대화의 첫머리를 잃는데, 어떤 연결에서는 잃고 어떤 연결에서는 잃지 않는다.

동의하기 전에 확인하라. `proven_http_exchange_upgrade`는 `101`을 쓴다. 요청이 여러분이 뜻하는 프로토콜에 대한 올바른 요청이었는지는 판단하지 않는다. WebSocket이라면 그 확인과 거기 필요한 응답 헤더가 `proven_ws_check_request`와 `proven_ws_accept_key`다 - 아니면 이 모두를 해 주는 12장의 `proven_ws_conn_accept`를 써라.

넘겨받은 뒤에는 한도가 여러분 몫이다. 모든 기다림에 한도가 있다는 서버의 약속은 `101`에서 끝난다. 그 전송에 `PROVEN_NET_NO_DEADLINE`으로 읽으면 영원히 기다린다.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * 프로토콜 바꾸기: 요청이 이 연결에서 HTTP를 그만 말하자고 청하고, 서버가 101로 동의하면,
 * 그때부터 같은 연결이 다른 것을 실어 나른다.
 *
 * 여기서 "다른 것"은 장난감이다 - 텍스트 줄을 받아 대문자로 돌려보낸다 - . 요점은 넘겨받는
 * 과정이기 때문이다. 사람들이 보통 뜻하는 프로토콜은 WebSocket이고, 12장의 연결이 이 단계들을
 * 대신 해 준다.
 */

static proven_http_server_t *g_server;

static void handle(void *ctx, proven_http_exchange_t *x) {
    (void)ctx;
    const proven_http_request_t *request = proven_http_exchange_request(x);
    /* 이것이 "우리" 프로토콜로 바꾸자는 요청인가? 확인은 핸들러의 일이다. */
    if (!proven_http_header_has_token(request->headers, request->header_count, PROVEN_LIT("Upgrade"),
PROVEN_LIT("shout")) ||
```

```
        !proven_http_header_has_token(request->headers, request->header_count, PROVEN_LIT("Connection"),
PROVEN_LIT("Upgrade"))) {
        (void)proven_http_exchange_respond(x, 426, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("this address
speaks only shout\n")));
        return;
    }

    /* 서버가 101을 쓰고, 연결은 우리 것이 된다: 소켓을 소유한 전송. */
    proven_transport_t t;
    proven_mem_view_t early;                /* 클라이언트가 101을 기다리지 않고 보낸 바이트 */
    if (proven_http_exchange_upgrade(x, PROVEN_LIT("shout"), NULL, 0, &t, &early) != PROVEN_OK) return;

    /* 여기서부터는 HTTP가 아니고 서버의 한도는 하나도 적용되지 않는다: 모든 기다림에 우리의
     * 기한이 필요하다. `early`는 핸들러가 돌아가면 죽으므로 먼저 쓴다. */
    proven_byte_t line[64];
    proven_size_t n = early.size < sizeof line ? early.size : sizeof line;
    for (proven_size_t i = 0; i < n; ++i) line[i] = early.ptr[i];
    while (n < sizeof line && (n == 0 || line[n - 1] != '\n')) {
        proven_result_size_t got = proven_transport_read(t, (proven_mem_mut_t){ line + n, sizeof line - n },
proven_net_deadline_in(5000));
        if (got.err != PROVEN_OK) break;
        n += got.value;
    }
    for (proven_size_t i = 0; i < n; ++i) if (line[i] >= 'a' && line[i] <= 'z') line[i] = (proven_byte_t)
(line[i] - 32);
    (void)proven_transport_write_all(t, (proven_mem_view_t){ line, n }, proven_net_deadline_in(5000));
    /* 붙들어 두었다가 핸들러가 돌아간 뒤에 써도 된다. 어느 쪽이든 닫는 것은 우리 몫이다. */
    (void)proven_transport_close(t);
}

static void serve(void *arg) {
    (void)arg;
    (void)proven_http_server_run(g_server);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_http_server_config_t server_config = {0};
    server_config.alloc = heap;
    server_config.handler = handle;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &g_server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(g_server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(g_server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, NULL) == PROVEN_OK, "its loop runs on
another thread");

    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 2 };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "a client");
```

```
    char url[64];
    int url_len = snprintf(url, sizeof url, "http://127.0.0.1:%u/", (unsigned)at.port);

    /* `upgrade`가 프로토콜의 이름을 정한다. `Connection: Upgrade`와 `Upgrade`는 클라이언트가 쓴다. */
    proven_http_client_request_t request = { .url = { (const proven_byte_t *)url,
(proven_size_t)url_len }, .upgrade = PROVEN_LIT("shout") };
    proven_http_client_response_t response;
    err = proven_http_client_send(client, &request, &response);
    EXAMPLE_REQUIRE(err == PROVEN_OK && response.status == 101, "the server agreed: 101 Switching Protocols");

    /* 101은 그 연결을 가져가기 전까지는 다른 응답과 다를 바 없는 응답이다. */
    proven_transport_t t;
    proven_mem_view_t early;
    EXAMPLE_REQUIRE(proven_http_client_upgrade(&response, &t, &early) == PROVEN_OK, "the connection is ours
now");
    proven_http_client_finish(&response);       /* 여전히 필요하다. 이제 연결은 건드리지 않는다 */

    proven_byte_t reply[16];
    proven_size_t have = 0;
    EXAMPLE_REQUIRE(proven_transport_write_all(t, proven_mem_view_from_u8(PROVEN_LIT("hello\n")),
proven_net_deadline_in(5000)).err == PROVEN_OK, "a line, in the new protocol");
    while (have < 6) {
        proven_result_size_t got = proven_transport_read(t, (proven_mem_mut_t){ reply + have, sizeof reply -
have }, proven_net_deadline_in(5000));
        if (got.err != PROVEN_OK) break;
        have += got.value;
    }
    EXAMPLE_REQUIRE(have == 6 && proven_u8str_view_eq((proven_u8str_view_t){ reply, 6 },
PROVEN_LIT("HELLO\n")), "and the answer, in capitals");
    (void)proven_transport_close(t);            /* 소켓을 닫고 전송을 해제한다 */

    /* `upgrade`가 없으면 같은 주소가 HTTP로 답하고, 가져갈 것이 없다. */
    err = proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)url_len }, &response);
    EXAMPLE_REQUIRE(err == PROVEN_OK && response.status == 426, "an ordinary request is told what this address
speaks");
    EXAMPLE_REQUIRE(proven_http_client_upgrade(&response, &t, &early) == PROVEN_ERR_INVALID_STATE, "a response
that is not a 101 has no connection to give");
    proven_http_client_finish(&response);

    proven_http_client_destroy(client);
    proven_http_server_stop(g_server);
    proven_job_group_wait(threads, &running);
    proven_http_server_destroy(g_server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

12. 여기에 없는 것

- 양쪽 방향의 **TLS**. §1.
- **HTTP/2와 HTTP/3.**
- **요청하지 않은 압축.** 아무것도 설정하지 않으면 `Accept-Encoding`을 보내지 않고, 콘텐츠 코딩을 풀지 않으며, 응답을 압축하지 않는다. 클라이언트 설정의 `decompress`와 핸들러에서 부르는 `proven_http_exchange_compress`가 각 방향의 gzip을 켠다(Zstandard는 클라이언트의 `decompress_zstd`와 서버의 `compress_zstd`가, Brotli는 클라이언트에서만 `decompress_brotli`가 켠다): 16장 6절.
- **WebSocket**은 이 두 헤더에 없다. §11 위에 지은 12장에 있다.
- 서버의 **라우터, 정적 파일, 세션, 미들웨어**. 클라이언트의 **재시도, 캐시, 호스트별 연결 한도**.

- 양쪽 방향의 **트레일러**.
- **이벤트 구동 클라이언트의** `Expect: 100-continue`. `expect_continue`는 블로킹 클라이언트의 것이다. 15장의 클라이언트는 묻지 않고 본문을 보낸다.
- **프록시 자동 설정,** `no_proxy`**의 주소 범위, 프록시에 대한 Digest 인증.**

12장: WebSocket

5부 — 운영체제와 대화하기. 선행 조건: 11장의 HTTP 클라이언트와 서버, 그리고 그 장의 프로토콜 바꾸기 절. 코덱 절반만 볼 것이면 10장. 이 장을 마치면 어느 쪽에서든 WebSocket 연결을 열고, 텍스트와 바이너리 메시지를 주고받고, 제대로 닫고, 상대가 규칙을 어겼을 때 여러분의 프로그램이 무엇을 하는지 말할 수 있다 - 또는 소켓 없이 프레임을 직접 뜯어볼 수 있다.

이 장은 `ws.h`와 `ws_conn.h`를 다루고, 9절에서는 그 둘과 15장의 이벤트 구동 서버에서 쓰는 압축된 메시지를 다룬다. `ws.h`는 순수한 텍스트·바이트 처리로, 프리스탠딩(freestanding) 빌드에서 쓸 수 있고 `PROVEN_NO_NET`의 영향을 받지 않는다. `ws_conn.h`는 소켓이 필요하다. 호스티드(hosted) 전용이고 `PROVEN_NO_NET`으로 빠진다.

목차

1. WebSocket이 무엇인지, 한 쪽으로
2. 연결
3. 보내기와 받기
4. 닫기, 그리고 닫히기
5. 서버의 연결은 어디에 사는가
6. 코덱: 핸드셰이크
7. 코덱: 프레임
8. 코덱: 디코더
9. 압축된 메시지
10. 여기에 없는 것

1. WebSocket이 무엇인지, 한 쪽으로

HTTP는 질문 하나에 답 하나다. WebSocket(RFC 6455)은 양쪽이 할 말이 생길 때마다 말하고 싶을 때 연결이 되는 것이다. HTTP 요청 한 번과 `101` 응답 뒤에, 같은 TCP 연결이 한쪽이 닫을 때까지 양방향으로 *메시지*를 실어 나른다.

```
client                                          server
  ¦ GET /chat HTTP/1.1                            ¦
  ¦ Upgrade: websocket                            ¦
  ¦ Sec-WebSocket-Key: <16 random bytes>          ¦
  ¦-----------------------------------------------¦
  ¦            HTTP/1.1 101 Switching Protocols   ¦
  ¦            Sec-WebSocket-Accept: <the answer>¦
  ¦<----------------------------------------------¦
  ¦  message  --->         <---  message          ¦      any number, either way, any time
  ¦  ping     --->         <---  pong             ¦
  ¦  close    --->         <---  close            ¦      then the connection ends
```

메시지는 **텍스트**(UTF-8이고, 검사된다)이거나 **바이너리**(아무 바이트)다. 회선 위에서 메시지는 하나 이상의 *프레임*이다. 길거나 흘려보내는 메시지는 프래그먼트로 나뉘고 받는 쪽이 이어 붙인다. 그 사이로 작은 제어 프레임 세 가지가 다닌다. 반대쪽이 아직 있는지 알아보는 **ping**과 **pong**, 그리고 코드와 이유를 싣는 **close**다.

두 가지 규칙은 까닭을 알기 전에는 이상해 보인다. 클라이언트는 모든 프레임을 **마스킹**한다 - 페이로드를 무작위 4바이트 키와 XOR한다 - . 서버는 결코 하지 않는다. 마스크는 아무것도 숨기지 않는다. 웹 페이지의 스크립트가 네트워크를 건너는 바이트를 정확히 고르지 못하게 막을 뿐이고, 한때는 그것만으로 캐싱 프록시를 오염시킬 수 있었다. 그리고 핸드셰이크의 키와 답은 인증이 아니다. 서버가 정말 WebSocket을 말하며, 받은 것을 되돌려 보내는 HTTP 서버가 아니라는 것만 증명한다.

`ws://`는 보호되지 않는다. `ws://` 연결은 11장이 `http://`에 대해 말한 그대로, 경로 위의 누구든 읽고 바꿀 수 있다. `wss://`는 TLS 위의 같은 프로토콜이다. HTTP 클라이언트에 TLS 랩을, 또는 HTTP 서버에 TLS 설정을 주면(14장) 이 장의 나머지는 아무것도 바뀌지 않는다.

2. 연결

```
/* a client */
proven_ws_conn_config_t config = { .alloc = proven_heap_allocator() };
proven_ws_conn_t *ws;
if (proven_ws_conn_connect(http_client, PROVEN_LIT("ws://example.com/feed"), NULL, 0,
                           PROVEN_LIT(""), &config, &ws, NULL) == PROVEN_OK) {
    proven_ws_conn_send_text(ws, PROVEN_LIT("hello"));
    ...
    proven_ws_conn_close(ws, PROVEN_WS_CLOSE_NORMAL, PROVEN_LIT(""));
    proven_ws_conn_destroy(ws);
}
```

참조

API	의도	반환
`proven_ws_conn_connect(client, url, headers, count, protocols, &config, &ws, &http_status)`	HTTP 클라이언트를 거쳐 클라이언트로 연결한다 - 그래서 그 클라이언트의 프록시, 타임아웃, `tls_wrap`이 적용된다. `url`은 `ws://`나 `wss://`다. `http_status`(선택)는 서버가 답한 상태 코드를 받는다.	`proven_err_t`: 답이 `101`이 아니면 `REFUSED`(`http_status`를 보라). `101`이 이 핸드셰이크에 대한 답이 아니면 `INVALID_FORMAT`. `tls_wrap` 없는 `wss`는 `UNSUPPORTED`. 무작위 바이트를 얻지 못하면 `IO`. 그리고 `proven_http_client_send`의 오류들.
`proven_ws_conn_accept(x, protocol, &config, &ws)`	HTTP 핸들러 안에서 서버로 받아들인다: 요청을 확인하고, `101`로 답하고, 연결을 가져온다. `protocol`은 고를 서브프로토콜이거나 비어 있다.	`proven_err_t`: `NOT_FOUND` - 업그레이드가 아니다, 아무것도 보내지 않았다, HTTP로 답하라. `UNSUPPORTED` - 다른 버전이다, `426`을 보냈다. `INVALID_FORMAT` - 형식이 어긋난 업그레이드다, `400`을 보냈다. `INVALID_ARG` - 클라이언트가 내놓지 않은 `protocol`이다, 아무것도 보내지 않았다.
`proven_ws_conn_open(transport, is_server, early, &config, &ws)`	핸드셰이크를 이미 마친 전송 위에 연결을 만든다. 성공하면 연결이 전송을 소유한다.	`proven_err_t`: `INVALID_ARG`. `NOMEM`. `early`가 16 KiB를 넘으면 `OUT_OF_BOUNDS`. `IO`.
`proven_ws_conn_destroy(ws)`	전송을 닫고 연결을 해제한다.	없음.
`proven_ws_conn_protocol(ws)`	합의된 서브프로토콜. 없으면 비어 있다.	`proven_u8str_view_t`.

```
typedef struct {
    proven_allocator_t alloc;           /* required */
    proven_size_t max_message_bytes;    /* 0: 1 MiB - the largest message accepted */
    proven_u32 io_timeout_ms;           /* 0: 30 s - for each write */
    proven_u32 close_timeout_ms;        /* 0: 5 s - how long close waits for the peer */
} proven_ws_conn_config_t;
```

연결 하나는 만들 때 약 21 KiB가 든다 - 16 KiB 읽기 버퍼와 4 KiB 보내기 버퍼 - . 여기에 지금까지 조립해야 했던 가장 큰 메시지만큼이 더해지는데, 다음 메시지를 위해 간직되며 `max_message_bytes`를 넘지 않는다. 메모리는 모두 `alloc`에 준 할당자(allocator)에서 온다.

주의사항, 그리고 무엇이 잘못되는가

연결은 한 번에 스레드 하나의 것이다. "한 스레드가 보내는 동안 다른 스레드가 받는다"도 안 된다. 받기는 ping에 답하고, 그것은 보내기다. 한 연결을 두 스레드가 쓰면 두 프레임의 바이트가 섞이고, 상대는 원인과 동떨어진 곳을 가리키는 프로토콜 오류로 닫는다. 연결을 한 스레드에 맡겨라. 모든 호출을 잠금 하나로 감싸면 바이트는 섞이지 않지만, `proven_ws_conn_receive`는 기다리는 내내 그 잠금을 쥐고 있고 모든 보내기가 함께

기다린다 - 그래서 한 스레드가 메시지를 기다리는 동안 다른 스레드가 보내는 프로그램(채팅 클라이언트)에는 루프의 연결이 맞다: 15장 11절의 "두 스레드, 연결 하나"에 그 프로그램이 있다.

`max_message_bytes`는 상대가 여러분에게 쥐게 할 수 있는 양이다. 메시지는 여러분이 보기 전에 메모리 안에서 조립된다. 이 한도는 무엇이 오든 연결 하나가 그 일에 할당할 최대치다. 넘는 메시지는 코드 1009로 연결을 끝낸다. 상상할 수 있는 가장 큰 메시지가 아니라 기대하는 메시지에 맞춰 정하라.

서브프로토콜은 장식이 아니라 합의다. 클라이언트가 이름들을 내놓고, 서버가 그중 하나를 고르거나 아무것도 고르지 않는다. `proven_ws_conn_accept`는 클라이언트가 내놓지 않은 이름을 거절하고, `proven_ws_conn_connect`는 그런 이름을 고른 서버를 거절한다. 여러분의 프로토콜에 버전이 있다면, 양쪽이 어느 버전을 공유하는지 알게 되는 곳이 여기다. `proven_ws_conn_protocol`을 확인하고, 짐작하지 마라.

`Origin` 헤더를 확인하는 것은 서버 몫이다. 브라우저는 모든 WebSocket 핸드셰이크에 `Origin`을 실어 보내고, 거기에 동일 출처 정책을 적용하지 않는다. 어떤 웹 페이지든 여러분의 서버에 WebSocket을 열 수 있고, 브라우저는 사용자의 쿠키를 붙여 준다. 서버가 브라우저에서 닿을 수 있고 쿠키를 쓴다면, 받아들이기 전에 `Origin`을 예상하는 사이트와 비교하라. 그러지 않으면 사용자가 방문한 아무 페이지나 그 사용자 행세를 할 수 있다.

`proven_ws_check_origin(&request, allowed, count)`가 그 비교다. 출처는 브라우저가 쓰는 대로 쓴다 - `https://app.example`, 포트는 스킴의 기본 포트가 아닐 때에만, 경로도 끝의 슬래시도 없이 - 그리고 대소문자만 빼고 통째로 비교한다.

결과	뜻	서버가 할 일
`PROVEN_OK`	`Origin`이 하나이고 목록에 있다.	받아들인다.
`PROVEN_ERR_NOT_FOUND`	`Origin`이 없다: 브라우저가 아니다.	정한다 - 브라우저가 아닌 프로그램은 사용자의 쿠키를 싣고 오지 않으며, 어떤 `Origin`이든 마음대로 쓸 수 있었다.
`PROVEN_ERR_PERMISSION`	목록에 없는 출처(`null` 포함), 또는 `Origin` 필드가 둘.	`403`으로 답한다.

파싱된 요청을 받으므로, 블로킹 서버의 핸들러에서도, 이벤트 구동 서버의 `on_request`에서도, 코덱만 쓸 때에도 같은 호출이다.

3. 보내기와 받기

API	의도	반환
`proven_ws_conn_send_text(ws, text)`	텍스트 메시지를 보낸다.	`proven_err_t`: UTF-8이 아니면 `INVALID_ENCODING` - 아무것도 보내지 않는다. close 뒤이거나 `_send_part`로 시작한 메시지 안이면 `INVALID_STATE`. `TIMEOUT`, `RESET`.
`proven_ws_conn_send_binary(ws, data)`	바이너리 메시지를 보낸다.	같다. 인코딩 검사만 없다.
`proven_ws_conn_send_part(ws, text, data, last)`	메시지의 프래그먼트 하나를 보낸다. `last`가 메시지를 끝낸다. `text`는 첫 호출에서만 읽는다.	같다. 이렇게 보내는 텍스트는 검사하지 **않는다**.
`proven_ws_conn_ping(ws, data)`	최대 125바이트를 실은 ping을 보낸다.	`proven_err_t`: 더 길면 `INVALID_ARG`.
`proven_ws_conn_receive(ws, until, &message)`	다음 온전한 메시지를, 또는 `until`까지 기다린다.	`proven_err_t`: `TIMEOUT` - 잃은 것은 없다, 다시 불러라. `EOF` - 상대가 닫았다. `RESET` - 연결이 끊겼다. `INVALID_FORMAT`, `INVALID_ENCODING`,

		`OUT_OF_BOUNDS` - 상대가 규칙을 어겼고, 그렇다고 알렸다.
`proven_ws_conn_pong_count(ws)`	도착한 pong의 수.	`proven_u64`.

```
typedef struct {
    bool text;                  /* text (UTF-8, checked) rather than binary */
    proven_mem_view_t data;     /* owned by the connection; good until the next receive */
} proven_ws_message_t;
```

`proven_ws_conn_receive`는 기다리는 동안 프로토콜의 살림을 한다. **ping에는 pong으로 답하고** 여러분은 그것을 보지 못한다. **pong은 센다. 프래그먼트는 이어 붙이고**, 그 사이에 낀 제어 프레임은 도착한 자리에서 처리한다. 돌려주는 것은 언제나 온전한 메시지다.

주의사항, 그리고 무엇이 잘못되는가

거절에는 최대 1초가 걸린다. 상대가 규칙을 어기면 연결은 close를 보낸 다음, `receive`가 돌아오기 전에 상대가 아직 보내고 있는 것을 최대 1초 동안 읽어서 버린다 - 그러지 않으면 소켓을 닫는 순간 연결이 reset되어, 상대가 읽기도 전에 그 close가 사라진다.

`PROVEN_ERR_TIMEOUT`은 그 뒤에도 받을 수 있는 유일한 오류다. 아직 온전한 메시지가 도착하지 않았다는 뜻이다. 도착한 일부는 간직되고, 다음 호출이 이어 간다. 다른 결과는 모두 연결을 끝낸다. 그 뒤의 모든 호출에서 같은 오류가 돌아오고, 남은 일은 `proven_ws_conn_destroy`다.

메시지의 바이트는 다음 receive까지만 유효하다. `message.data`는 연결이 다시 쓰는 메모리를 가리키는 뷰(view)다. 간직할 것은 복사하라.

잘못된 예:

```
proven_ws_conn_receive(ws, until, &first);
proven_ws_conn_receive(ws, until, &second);
use(first.data);                    /* wrong: first.data now holds the second message, or freed memory */
```

올바른 예 — 다시 받기 전에 `first.data`를 쓰거나 복사한다.

ping에는 받는 동안에만 답한다. 연결에는 자기 스레드가 없다. `proven_ws_conn_receive`를 부르지 않고 오랫동안 보내기만 하는 프로그램은 ping에 답하지 않고, ping으로 죽은 연결을 찾는 상대는 이 연결이 죽었다고 결론짓는다. 많이 보낸다면 이따금 받아라 - 아무것도 없을 수 있으면 `PROVEN_NET_DONT_WAIT`로.

ping을 대신 보내 주는 것도 없다. 말없이 죽은 연결 - 덮인 노트북, 잊어버린 NAT - 을 찾으려면 `proven_ws_conn_ping`을 보내고, 기다릴 만큼 받아 보고, `proven_ws_conn_pong_count`가 움직였는지 보라. 반대쪽이 한마디 없이 사라진 TCP 연결은 그러지 않으면 몇 시간이고 열려 있는 것처럼 보인다.

`_send_part`는 텍스트를 검사하지 않는다. `proven_ws_conn_send_text`는 UTF-8이 아닌 텍스트를 거절한다. 그런 텍스트를 받으면 상대는 연결을 끊어야 하기 때문이다. 프래그먼트는 하나씩 검사할 수 없다 - 문자 하나가 두 프래그먼트 사이에서 잘릴 수 있다 - . 그래서 거기서는 조각들이 합쳐서 UTF-8이어야 하고, 그것을 확실히 하는 것은 여러분 몫이다.

타임아웃된 쓰기는 연결을 끝낸다. `io_timeout_ms`는 쓰기 하나하나를 한정한다. 그것이 다하면 프레임의 일부가 나갔을 수 있고, 프레임 중간에서 이어 갈 방법은 없다. 연결은 끝난 것이다.

4. 닫기, 그리고 닫히기

API	의도	반환
`proven_ws_conn_close(ws, code, reason)`	close 프레임을 보내고 상대의 close를 기다린다. 그 사이에 도착한 메시지는 버린다.	`proven_err_t`: 양쪽이 모두 닫았으면 `PROVEN_OK`. 상대가 `close_timeout_ms` 안에 답하지 않으면 `TIMEOUT`. 보낼 수 없는 코드나 이유면 `INVALID_ARG`.

`proven_ws_conn_close_code(ws)`	상대의 close 코드. close 없이 연결이 끝났으면 1006. 열려 있는 동안은 0.	`proven_u16.`
`proven_ws_conn_close_reason(ws)`	함께 온 이유.	`proven_u8str_view_t.`

닫기는 작은 핸드셰이크다. 한쪽이 close 프레임을 보내고, 다른 쪽이 close로 답하고, 그다음 TCP 연결이 끝난다. 이것을 건너뛰어도 - 닫지 않은 연결을 destroy해도 - 동작은 한다. 상대는 코드 1006 "비정상 종료"를 보는데, 네트워크가 고장 났을 때 보는 것과 같다. 1006에 로그를 남기거나 재시도하는 상대는 그렇게 할 것이다. 먼저 닫아라.

코드	뜻	보내는 쪽
1000	정상: 목적을 이뤘다.	여러분
1001	떠남: 서버가 내려가거나 페이지를 떠난다.	여러분
1002	프로토콜 오류.	연결 - 프로토콜이 금지한 프레임에
1003	지원하지 않는 데이터: 바이너리를 원했는데 텍스트, 또는 그 반대.	여러분
1007	잘못된 데이터: UTF-8이 아닌 텍스트.	연결
1008	정책 위반.	여러분
1009	메시지가 너무 크다.	연결 - `max_message_bytes`를 넘으면
1011	내부 오류.	여러분
3000-4999	여러분과 여러분의 응용이 정의한다.	여러분
1005, 1006	결코 보내지 않는다: "코드가 없었다"와 "close 프레임이 오지 않았다".	아무도. 읽기만 한다

상대가 닫으면 `receive`는 `PROVEN_ERR_EOF`를 돌려주고, 그 close에는 이미 답했다. 원하면 코드와 이유를 읽고, destroy하라. 더는 아무것도 보낼 수 없다.

이유는 로그를 읽는 사람을 위한 것이다. UTF-8 최대 123바이트. 프로그램이 파싱해야 하는 것을 넣지 말고, 읽을 수 있는 채로 네트워크를 건넌다는 것을 기억하라.

close는 반대 방향을 비워 주지 않는다. 상대가 여러분의 close를 보기 전에 보낸 메시지는 `proven_ws_conn_close`가 버린다. 그것이 필요하면 먼저 닫지 말고, 보내기를 멈춘 채 상대가 닫을 때까지 계속 받아라.

5. 서버의 연결은 어디에 사는가

`proven_ws_conn_accept`는 연결을 HTTP 서버에서 꺼낸다(11장 §11). 그 순간부터 서버의 루프, 연결 수 한도, 타임아웃은 그 연결과 아무 상관이 없고, 남는 질문은 어느 스레드가 `proven_ws_conn_receive`를 부를 것인가다.

핸들러가 넘겨주고 돌아간다. 핸들러를 어떻게 돌리든 통하는 모양이고, 예제가 쓰는 모양이다. 연결을 여러분의 스레드나 job에 주고 돌아간다. 서버는 계속 서비스한다.

핸들러가 남아서 대화한다. 쓰기는 간단하다. 핸들러를 job system에서 돌리면 연결이 이어지는 동안 작업 스레드 하나를 차지한다 - 그래서 작업 스레드 수가 곧 가질 수 있는 WebSocket 연결 수이고, 다음 클라이언트의 핸드셰이크는 하나가 닫힐 때까지 큐에서 기다린다. 핸들러를 루프의 스레드에서 돌리면 그 시간 동안 서버 전체가 멈추는데, 그것이 의도인 경우는 거의 없다.

어느 쪽이든 **여기서 WebSocket 연결 하나는 스레드 하나의 값이 든다.** `proven_ws_conn_receive`는 연결 하나를 기다린다. 여럿을 기다리는 호출은 없다. 연결 백 개는 receive에 막힌 스레드 백 개다. 연결 수십 개나 수백 개에는 쓸 만한 설계이고 수만 개에는 틀린 설계다 - 그런 경우에는 §6-§8의 코덱을 여러분의 준비 상태 루프(9장의 `proven_net_poll`)에서, 연결마다 디코더 하나씩 두고 구동해야 한다.

대신 걸어 주는 한도가 없다. HTTP 서버의 `max_connections`는 연결이 업그레이드되면 그것을 세지 않는다. 클라이언트가 WebSocket을 끝없이 열 수 있다면, 여러분이 세어서 충분할 때 핸드셰이크에 `503`으로 답하라.

테스트 스위트가 컴파일하고 실행한다:

```
/*
 * WebSocket 서버와 클라이언트를 한 프로그램에.
 *
 * 서버는 핸들러가 업그레이드를 받아들이는 평범한 HTTP 서버다. 핸들러는 남아서 대화하지
 * 않는다: 연결을 작업 스레드에 넘기고 돌아간다. 그래서 서버의 루프는 다음 요청을 받을 수 있다.
 * 클라이언트는 HTTP 클라이언트를 거쳐 연결하고, 그때부터 두 끝은 같은 종류의 것이다.
 */

static proven_http_server_t *g_server;
static proven_job_sys_t *g_workers;

/* 작업 스레드에서 보내는 연결 하나의 일생: 상대가 닫을 때까지 메시지를 되돌려 보낸다. */
static void talk(void *arg) {
    proven_ws_conn_t *ws = arg;
    for (;;) {
        proven_ws_message_t msg;
        /* ping에는 receive 안에서 답한다. 30초 동안 말이 없는 상대는 포기한다. */
        proven_err_t err = proven_ws_conn_receive(ws, proven_net_deadline_in(30000), &msg);
        if (err != PROVEN_OK) break;        /* PROVEN_ERR_EOF: 상대가 닫았고, 그에 답했다 */
        err = msg.text ? proven_ws_conn_send_text(ws, (proven_u8str_view_t){ msg.data.ptr, msg.data.size })
                       : proven_ws_conn_send_binary(ws, msg.data);
        if (err != PROVEN_OK) break;
    }
    proven_ws_conn_destroy(ws);             /* 반드시: 이제 이것이 소켓을 소유한다 */
}

static void handle(void *ctx, proven_http_exchange_t *x) {
    (void)ctx;
    proven_ws_conn_config_t config = { .alloc = proven_heap_allocator(), .max_message_bytes = 64 * 1024 };
    proven_ws_conn_t *ws = NULL;
    /* 클라이언트가 내놓았을 때에만 서브프로토콜을 고른다. */
    const proven_http_request_t *request = proven_http_exchange_request(x);
    proven_u8str_view_t protocol = proven_ws_request_offers(request, PROVEN_LIT("echo.v1")) ?
PROVEN_LIT("echo.v1") : PROVEN_LIT("");

    proven_err_t err = proven_ws_conn_accept(x, protocol, &config, &ws);
    if (err == PROVEN_ERR_NOT_FOUND) {
        /* 업그레이드가 아예 아니다 - 이 주소로 온 평범한 요청이다. 아무것도 보내지 않았다. */
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("this is a
WebSocket endpoint\n")));
        return;
    }
    if (err != PROVEN_OK) return;           /* 잘못된 핸드셰이크에는 이미 400이나 426으로 답했다 */

    /* 연결은 더 이상 서버의 것이 아니다. 스레드를 주고 돌아간다. */
    if (proven_job_submit_ex(g_workers, talk, ws) != PROVEN_OK) proven_ws_conn_destroy(ws);
}

static void serve(void *arg) {
    (void)arg;
    (void)proven_http_server_run(g_server);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
```

```
    // ---- 서버 ------------------------------------------------------------
    proven_http_server_config_t server_config = {0};
    server_config.alloc = heap;
    server_config.handler = handle;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &g_server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(g_server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(g_server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    /* 서버의 루프에 스레드 하나, 그리고 연결을 맡을 작업 스레드들: 열려 있는 WebSocket
     * 하나가 그것이 이어지는 동안 작업 스레드 하나를 차지한다. */
    proven_job_sys_t *loop_thread = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &loop_thread) == PROVEN_OK &&
                    proven_job_system_init(heap, 4, 16, &g_workers) == PROVEN_OK &&
                    proven_job_group_submit(loop_thread, &running, serve, NULL) == PROVEN_OK, "the loop and
the workers are started");

    // ---- 클라이언트 ------------------------------------------------------
    proven_http_client_config_t http_config = { .alloc = heap };
    proven_http_client_t *http = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&http_config, &http) == PROVEN_OK, "an HTTP client to connect
through");

    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/talk", (unsigned)at.port);
    proven_ws_conn_config_t config = { .alloc = heap };
    proven_ws_conn_t *ws = NULL;
    proven_u16 status = 0;
    err = proven_ws_conn_connect(http, (proven_u8str_view_t){ (const proven_byte_t *)url, (proven_size_t)n },
NULL, 0,
                                 PROVEN_LIT("echo.v1, echo.v0"), &config, &ws, &status);
    EXAMPLE_REQUIRE(err == PROVEN_OK && status == 101, "connected: the server answered 101 and the answer
checked out");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_ws_conn_protocol(ws), PROVEN_LIT("echo.v1")), "the subprotocol
the server chose");
    /* 연결은 이제 독립된 것이다. 쓰는 데 HTTP 클라이언트는 필요 없다. */
    proven_http_client_destroy(http);

    proven_ws_message_t msg;
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(ws, PROVEN_LIT("hello")) == PROVEN_OK, "a text message");
    EXAMPLE_REQUIRE(proven_ws_conn_receive(ws, proven_net_deadline_in(5000), &msg) == PROVEN_OK && msg.text &&
                    proven_u8str_view_eq((proven_u8str_view_t){ msg.data.ptr, msg.data.size },
PROVEN_LIT("hello")), "comes back, whole");

    proven_byte_t bytes[4] = { 0x00, 0xff, 0x10, 0x80 };
    EXAMPLE_REQUIRE(proven_ws_conn_send_binary(ws, (proven_mem_view_t){ bytes, sizeof bytes }) == PROVEN_OK &&
                    proven_ws_conn_receive(ws, proven_net_deadline_in(5000), &msg) == PROVEN_OK && !msg.text
&& msg.data.size == 4, "binary is any bytes");

    /* 텍스트는 보내기 전에 검사한다: 이것을 받으면 상대는 연결을 끊어야 한다. */
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(ws, PROVEN_LIT("not \xff text")) == PROVEN_ERR_INVALID_ENCODING,
"not UTF-8: refused here, nothing sent");
```

```
    /* 조각조각 만들어지는 메시지는 프래그먼트로 나간다. 반대쪽에는 여전히 메시지 하나로 보인다. */
    EXAMPLE_REQUIRE(proven_ws_conn_send_part(ws, true, proven_mem_view_from_u8(PROVEN_LIT("one, ")), false) ==
PROVEN_OK &&
                    proven_ws_conn_send_part(ws, true, proven_mem_view_from_u8(PROVEN_LIT("two, ")), false) ==
PROVEN_OK &&
                    proven_ws_conn_send_part(ws, true, proven_mem_view_from_u8(PROVEN_LIT("three")), true) ==
PROVEN_OK, "three fragments");
    EXAMPLE_REQUIRE(proven_ws_conn_receive(ws, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
msg.data.size == 15, "one message of fifteen bytes");

    /* 반대쪽이 살아 있는가? ping을 보내고 잠시 receive한다: pong은 그 사이에 세어진다. */
    EXAMPLE_REQUIRE(proven_ws_conn_ping(ws, proven_mem_view_from_u8(PROVEN_LIT("still there?"))) == PROVEN_OK,
"a ping");
    err = proven_ws_conn_receive(ws, proven_net_deadline_in(200), &msg);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_TIMEOUT && proven_ws_conn_pong_count(ws) == 1, "no message came, but the
pong did: the peer is alive");

    /* 닫기는 핸드셰이크다: 우리의 close, 그다음 상대의 close. */
    EXAMPLE_REQUIRE(proven_ws_conn_close(ws, PROVEN_WS_CLOSE_NORMAL, PROVEN_LIT("done")) == PROVEN_OK,
"closed, both ways");
    EXAMPLE_REQUIRE(proven_ws_conn_close_code(ws) == PROVEN_WS_CLOSE_NORMAL &&
proven_ws_conn_close_reason(ws).size == 0, "the peer echoed the code, with no reason of its own");
    proven_ws_conn_destroy(ws);

    // ---- 어떤 전송 위에서든 되는 연결 ----------------------------------------
    /* 핸드셰이크는 HTTP의 일이고, 그 뒤는 전송만 있으면 된다. 여기서는 소켓 짝이다. */
    proven_net_conn_t a, b;
    proven_ws_conn_t *left = NULL, *right = NULL;
    EXAMPLE_REQUIRE(proven_net_pair(&a, &b) == PROVEN_OK, "two joined sockets");
    EXAMPLE_REQUIRE(proven_ws_conn_open(proven_net_conn_transport(&a), false, (proven_mem_view_t){0}, &config,
&left) == PROVEN_OK &&
                    proven_ws_conn_open(proven_net_conn_transport(&b), true, (proven_mem_view_t){0}, &config,
&right) == PROVEN_OK, "a client end and a server end");
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(left, PROVEN_LIT("no handshake needed")) == PROVEN_OK &&
                    proven_ws_conn_receive(right, proven_net_deadline_in(1000), &msg) == PROVEN_OK &&
msg.data.size == 19, "a message across");
    proven_ws_conn_destroy(left);
    proven_ws_conn_destroy(right);

    proven_http_server_stop(g_server);
    proven_job_group_wait(loop_thread, &running);
    proven_http_server_destroy(g_server);
    proven_job_system_close(g_workers);         /* 대화를 마무리하고 있는 작업 스레드를 기다린다 */
    proven_job_system_destroy(g_workers);
    proven_job_system_close(loop_thread);
    proven_job_system_destroy(loop_thread);
    return EXAMPLE_OK();
}
```

6. 코덱: 핸드셰이크

ws.h는 연결 없는 프로토콜이다. 바이트를 주면 그것이 무슨 뜻인지 말해 준다. 전송이 이 라이브러리가 연 소켓이 아닐 때, 한 루프에서 많은 연결을 서비스할 때, 소켓이 아예 없는 타깃에서 써라.

API	의도	반환
proven_ws_make_key(random, out)	여러분이 준 무작위 16바이트로 클라이언트의 Sec-WebSocket-Key(24글자)를 만든다.	없음.

`proven_ws_accept_key(key, out)`	키에 답하는 `Sec-WebSocket-Accept` 값(28글자).	`proven_err_t`: 키가 16바이트를 나타내는 Base64 24글자가 아니면 `INVALID_FORMAT`.
`proven_ws_check_request(&request, &key)`	파싱된 요청에 대한 서버의 확인 - 이벤트 구동 서버가 보여 주는 HTTP/2 확장 CONNECT도(15장 11절), 그때 `key`는 비어 있다.	`proven_err_t`: `PROVEN_OK` - 101로 답하라(HTTP/2에서는 200: `proven_ws_event_accept`가 어느 쪽이든 한다). `NOT_FOUND` - WebSocket 업그레이드가 아니다. `UNSUPPORTED` - 다른 버전이다, `Sec-WebSocket-Version: 13`을 단 426으로 답하라. `INVALID_FORMAT` - 400으로 답하라.
`proven_ws_request_offers(&request, name)`	요청이 서브프로토콜을 내놓는지. 정확히 비교한다.	`bool`.
`proven_ws_check_origin(&request, allowed, count)`	요청의 `Origin`이 서비스하는 출처 가운데 하나인지(2절).	`proven_err_t`: `PROVEN_OK`; `NOT_FOUND` - `Origin` 없음; `PERMISSION` - 다른 출처, 또는 필드가 둘; `INVALID_ARG`.
`proven_ws_check_response(&response, key, offered, &protocol)`	답에 대한 클라이언트의 확인: 101, 맞는 accept 값, 내놓았던 서브프로토콜, 확장 없음.	`proven_err_t`: 아니면 `INVALID_FORMAT`.

accept 값은 키 뒤에 RFC가 정한 고정 문자열을 붙인 것의 SHA-1을 Base64로 적은 것이다. SHA-1은 서명 용도로는 깨졌지만 이 용도는 그것을 개의치 않는다. 여기에 비밀은 없고, 이 값은 WebSocket 서버가 아닌 HTTP 서버가 반사적으로 만들어 내지 않을 무언가이기만 하면 된다. 증명하는 것도 그것이 전부다. 서버를 인증하지 않고 클라이언트를 인증하지 않는다. 누구든 계산할 수 있다.

확장의 이름을 댄 응답은 `proven_ws_check_response`가 **거절한다**: 아무것도 제안하지 않은 클라이언트가 요청하지 않은 프레임을 예고받는 것이기 때문이다. 압축을 제안한 클라이언트는 그 대신 `proven_ws_check_response_deflate`(9절)로 확인한다. 이 함수는 그 확장 하나를 제안한 형태 그대로만 받아들인다.

7. 코덱: 프레임

```
typedef struct {
    bool fin;                   /* the last frame of its message */
    proven_u8 opcode;           /* PROVEN_WS_TEXT, _BINARY, _CONTINUATION, _CLOSE, _PING, _PONG */
    bool masked;
    proven_byte_t mask[4];
    proven_u64 length;          /* payload bytes that follow the header */
} proven_ws_frame_t;
```

API	의도	반환
`proven_ws_frame_parse(data, &frame, &header_size)`	`data` 앞에서 프레임 헤더 - 2~14바이트 - 를 파싱한다.	`proven_err_t`: `NEED_MORE`. 예약 비트(첫째 비트는 메시지가 시작하는 곳에서는 `rsv1`로 알려 준다: 9절), 정의되지 않은 opcode, 가장 짧은 형태가 아닌 길이, 프래그먼트로 나뉘었거나 125바이트를 넘는 제어 프레임은 `INVALID_FORMAT`.
`proven_ws_frame_write(out, &len, &frame)`	프레임 헤더를 덧붙인다. 페이로드는 그 뒤에 여러분이 쓴다.	`proven_err_t`: 파서가 거절할 프레임이면 `INVALID_ARG`. `OUT_OF_BOUNDS`.

`proven_ws_mask(data, mask, offset)`	페이로드 바이트를 제자리에서 마스킹하거나 푼다. `offset`은 `data`가 페이로드 안에서 시작하는 위치다.	없음.
`proven_ws_close_write(out, &len, code, reason)`	close 프레임의 페이로드를 덧붙인다.	`proven_err_t`: 보낼 수 없는 코드나 123바이트를 넘는 이유는 `INVALID_ARG`. `INVALID_ENCODING`. `OUT_OF_BOUNDS`.
`proven_ws_close_parse(payload, &code, &reason)`	그것을 읽는다. 빈 페이로드는 코드 1005다.	`proven_err_t`: `INVALID_FORMAT`. `INVALID_ENCODING`.
`proven_ws_close_code_is_valid(code)`	코드가 close 프레임에 실릴 수 있는지.	`bool`.
`proven_ws_close_code_for(err)`	디코더 오류를 알리는 close 코드: 1002, 1007, 1009, 또는 1011.	`proven_u16`.

길이의 표기는 하나뿐이다. 125까지는 둘째 바이트에, 65535까지는 두 바이트를 더, 그 너머는 여덟 바이트에 적는다. `proven_ws_frame_write`는 언제나 가장 짧은 것을 쓰고, `proven_ws_frame_parse`는 그 밖의 것을 거절한다 - 한 프레임에 두 가지 인코딩이 있으면 필터와 그 뒤의 프로그램이 프레임이 어디서 끝나는지를 두고 어긋나게 된다.

클라이언트는 응용이 예측할 수 없는 키로 마스킹해야 한다. 4바이트는 `proven_random_bytes`나 그것으로 시드한 생성기에서 얻어라 - 카운터나 상수는 목적을 무너뜨린다. 서버는 `masked`를 false로 두고 페이로드를 그대로 쓴다.

코덱에는 무작위 원천이 없다. 키와 마스크는 10장의 multipart 경계처럼 인자다. 그래서 같은 코드가 물어볼 운영체제가 없는 곳에서도 돈다.

8. 코덱: 디코더

헤더 파싱은 작은 부분이다. 받는 쪽이 제대로 해야 하는 것은 스트림이다. 제멋대로인 조각으로 도착하는 프레임, 이어 붙일 프래그먼트, 프래그먼트 사이의 제어 프레임, 프래그먼트 경계를 넘어 검사해야 하는 텍스트, 그리고 거절할 것들의 목록. `proven_ws_decoder_t`가 그것이다.

API	의도	반환
`proven_ws_decoder_init(&decoder, from_client, max_message_bytes)`	시작한다. `from_client`가 true면 모든 프레임이 마스킹되어 있어야 한다(서버의 디코더). false면 어느 것도 마스킹되어 있으면 안 된다.	없음.
`proven_ws_decoder_feed(&decoder, in, &consumed, &event)`	보고할 것이 생기거나 `in`을 다 쓸 때까지 `in`의 바이트를 소비한다.	`proven_err_t`: `INVALID_FORMAT`(close 1002). `INVALID_ENCODING`(1007). `OUT_OF_BOUNDS`(1009). close 프레임 뒤의 바이트는 `EOF`.

```
typedef struct {
    proven_ws_event_kind_t kind;   /* PROVEN_WS_EVENT_NONE, _DATA, _PING, _PONG, _CLOSE */
    bool text, first, last;        /* DATA: text or binary; the ends of the message */
    proven_mem_view_t data;        /* DATA: a piece, unmasked, inside `in`. PING, PONG: the payload. CLOSE:
the reason. */
    proven_u16 close_code;         /* CLOSE */
} proven_ws_event_t;
```

루프는 10장의 본문 디코더에서 본 그것이다. 읽은 것을 넣고, 이벤트에 따라 행동하고, 나머지를 넣고, 이벤트가 NONE이면 더 읽는다.

메시지는 붙잡히지 않고 지나간다. DATA 이벤트는 메시지의 *조각*이고, 여러분이 넣은 버퍼 안에, 있던 자리에서 마스크가 풀린 채로 있다. first와 last가 메시지의 양 끝을 표시한다. 기가바이트짜리 메시지는 그만큼 많은 이벤트일 뿐 할당은 없다 - 조각을 잇고 싶다면 잇는 것은 여러분 몫이고, 한도도 여러분이 고른다(ws_conn.h가 max_message_bytes까지 대신 이어 준다).

in에는 쓰기가 일어난다. 마스크 풀기가 제자리에서 일어나므로 in은 const가 아니다. 바뀌면 안 되는 메모리를 넣지 마라.

디코더가 거절하는 것, 각각 그 close 코드를 알려 주는 오류와 함께:

스트림	오류
클라이언트가 보냈는데 마스킹되지 않은 프레임, 서버가 보냈는데 마스킹된 프레임	INVALID_FORMAT
예약 비트(첫째 비트는 있어서는 안 되는 곳에 있을 때, 또는 압축에 합의하지 않았을 때만: 9절), 정의되지 않은 opcode, 가장 짧은 형태가 아닌 길이	INVALID_FORMAT
125바이트를 넘거나 프래그먼트로 나뉜 제어 프레임	INVALID_FORMAT
열린 메시지가 없는데 온 continuation. 열린 메시지 안에서 시작한 새 메시지	INVALID_FORMAT
페이로드가 1바이트이거나, 보낼 수 없는 코드를 실은 close 프레임	INVALID_FORMAT
UTF-8이 아닌 텍스트(프래그먼트를 넘어 검사한다). UTF-8이 아닌 close 이유	INVALID_ENCODING
프래그먼트를 합쳐 한도를 넘는 메시지	OUT_OF_BOUNDS

이들 가운데 어느 것이든 일어난 뒤에는 디코더가 같은 오류를 되풀이한다. 의도한 것이다. 프레임 하나가 틀리면 다음 프레임이 어디서 시작하는지 알 수 없고, 짐작하다가는 틀린 프레임 하나가 다른 대화가 된다.

답하는 것은 여러분 몫이다. 디코더는 ping을 보고할 뿐 pong을 보내지 않는다. close를 보고할 뿐 답하는 close를 보내지 않는다. 디코더에는 무엇이든 보낼 방법이 없다 - 그래서 어디서나 돌 수 있다.

테스트 스위트가 컴파일하고 실행한다:

```
#include <string.h>

/*
 * WebSocket 코덱만으로: 핸드셰이크의 두 값, 프레임 하나를 쓰고 읽기, 그리고 어색한 조각으로
 * 스트림을 받는 디코더. 소켓은 어디에도 없다 - "네트워크"는 배열이다.
 */

int main(void) {
    // ---- 여는 핸드셰이크 ----------------------------------------------------
    /* 클라이언트는 무작위 16바이트로 키를 만들고, 서버는 WebSocket을 말하는 것만이 계산할
     * 값으로 답한다. 이것은 RFC 6455에 실린 값이다. */
    proven_byte_t accept[PROVEN_WS_ACCEPT_SIZE];
    EXAMPLE_REQUIRE(proven_ws_accept_key(PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="), accept) == PROVEN_OK &&
                    memcmp(accept, "s3pPLMBiTxaQ9kYGzzhZRbK+xOo=", sizeof accept) == 0, "the RFC's key and its
answer");

    proven_byte_t random[16], key[PROVEN_WS_KEY_SIZE];
    EXAMPLE_REQUIRE(proven_random_bytes(random, sizeof random), "sixteen random bytes");
    proven_ws_make_key(random, key);
    EXAMPLE_REQUIRE(proven_ws_accept_key((proven_u8str_view_t){ key, sizeof key }, accept) == PROVEN_OK, "a
key of our own, and its answer");

    /* 서버는 파싱된 요청을 보고 그것이 네 가지 중 무엇인지 안다. */
    static const char upgrade[] = "GET /chat HTTP/1.1\r\nHost: example.com\r\nUpgrade:
websocket\r\nConnection: Upgrade\r\n"
                                  "Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==\r\nSec-WebSocket-Protocol:
```

```
chat, superchat\r\n"
                                  "Origin: http://example.com\r\nSec-WebSocket-Version: 13\r\n\r\n";
    proven_http_header_t fields[16];
    proven_http_request_t request;
    proven_size_t head = 0;
    proven_u8str_view_t client_key;
    EXAMPLE_REQUIRE(proven_http_parse_request((proven_mem_view_t){ (const proven_byte_t *)upgrade, sizeof
upgrade - 1 }, fields, 16, 0, &request, &head) == PROVEN_OK, "the request parses");
    EXAMPLE_REQUIRE(proven_ws_check_request(&request, &client_key) == PROVEN_OK, "it is a WebSocket upgrade:
answer 101");
    EXAMPLE_REQUIRE(proven_ws_request_offers(&request, PROVEN_LIT("superchat")) && !
proven_ws_request_offers(&request, PROVEN_LIT("mqtt")), "and it offers these subprotocols");
    /* 쿠키로 사용자를 알아보는 서버는 페이지가 어디서 왔는지 확인한다. */
    const proven_u8str_view_t served[] = { PROVEN_LIT("https://app.example"), PROVEN_LIT("http://example.
com") };
    EXAMPLE_REQUIRE(proven_ws_check_origin(&request, served, 2) == PROVEN_OK &&
proven_ws_check_origin(&request, served, 1) == PROVEN_ERR_PERMISSION,
                    "its Origin is one the server serves; were it not, the answer would be 403");

    /* 클라이언트는 답을 확인한다: 맞는 accept 값, 자기가 내놓은 서브프로토콜, 그 밖에는 없음. */
    static const char answer[] = "HTTP/1.1 101 Switching Protocols\r\nUpgrade: websocket\r\nConnection:
Upgrade\r\n"
                                 "Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=\r\nSec-WebSocket-
Protocol: chat\r\n\r\n";
    proven_http_response_t response;
    proven_u8str_view_t chosen;
    EXAMPLE_REQUIRE(proven_http_parse_response((proven_mem_view_t){ (const proven_byte_t *)answer, sizeof
answer - 1 }, fields, 16, 0, &response, &head) == PROVEN_OK, "the response parses");
    EXAMPLE_REQUIRE(proven_ws_check_response(&response, client_key, PROVEN_LIT("chat, superchat"), &chosen) ==
PROVEN_OK &&
                    proven_u8str_view_eq(chosen, PROVEN_LIT("chat")), "it answers this handshake, and chose
chat");

    // ---- 프레임 하나 ------------------------------------------------------
    /* 클라이언트의 텍스트 프레임: 헤더, 그다음 4바이트 키와 XOR한 페이로드. */
    proven_byte_t wire[256];
    proven_mem_mut_t out = { wire, sizeof wire };
    proven_size_t len = 0;
    proven_ws_frame_t frame = { .fin = true, .opcode = PROVEN_WS_TEXT, .masked = true, .mask = { 0x37, 0xfa,
0x21, 0x3d }, .length = 5 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK && len == 6, "a six-byte header");
    memcpy(wire + len, "Hello", 5);
    proven_ws_mask((proven_mem_mut_t){ wire + len, 5 }, frame.mask, 0);
    len += 5;
    EXAMPLE_REQUIRE(memcmp(wire, "\x81\x85\x37\xfa\x21\x3d\x7f\x9f\x4d\x51\x58", 11) == 0, "byte for byte the
masked Hello of RFC 6455");

    proven_ws_frame_t parsed;
    proven_size_t header_size = 0;
    EXAMPLE_REQUIRE(proven_ws_frame_parse((proven_mem_view_t){ wire, 3 }, &parsed, &header_size) ==
PROVEN_ERR_NEED_MORE, "half a header: read more");
    EXAMPLE_REQUIRE(proven_ws_frame_parse((proven_mem_view_t){ wire, len }, &parsed, &header_size) ==
PROVEN_OK &&
                    parsed.opcode == PROVEN_WS_TEXT && parsed.masked && parsed.length == 5 && header_size ==
6, "the header, read back");

    // ---- 스트림을 디코딩하기 ----------------------------------------------
    /* 사이에 ping이 낀 두 프래그먼트짜리 텍스트 메시지, 그다음 close. */
    len = 0;
    frame = (proven_ws_frame_t){ .fin = false, .opcode = PROVEN_WS_TEXT, .length = 3 };
```

```
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "first fragment");
    memcpy(wire + len, "Hel", 3); len += 3;
    frame = (proven_ws_frame_t){ .fin = true, .opcode = PROVEN_WS_PING, .length = 0 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "a ping, in the middle of the
message");
    frame = (proven_ws_frame_t){ .fin = true, .opcode = PROVEN_WS_CONTINUATION, .length = 2 };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "last fragment");
    memcpy(wire + len, "lo", 2); len += 2;
    proven_byte_t close_payload[PROVEN_WS_MAX_CONTROL];
    proven_size_t close_len = 0;
    EXAMPLE_REQUIRE(proven_ws_close_write((proven_mem_mut_t){ close_payload, sizeof close_payload },
&close_len, PROVEN_WS_CLOSE_NORMAL, PROVEN_LIT("bye")) == PROVEN_OK, "a close payload");
    frame = (proven_ws_frame_t){ .fin = true, .opcode = PROVEN_WS_CLOSE, .length = close_len };
    EXAMPLE_REQUIRE(proven_ws_frame_write(out, &len, &frame) == PROVEN_OK, "the close frame");
    memcpy(wire + len, close_payload, close_len); len += close_len;

    /* 클라이언트가 서버가 보낸 것을 디코딩한다: `false` - 이 프레임들은 마스킹되어 있으면
     * 안 된다. 스트림은 한 번에 4바이트씩 도착하지만 디코더는 개의치 않는다. */
    proven_ws_decoder_t decoder;
    proven_ws_decoder_init(&decoder, false, 1024);
    char message[32];
    proven_size_t message_len = 0;
    int pings = 0;
    proven_u16 close_code = 0;
    for (proven_size_t pos = 0; pos < len;) {
        proven_size_t n = len - pos < 4 ? len - pos : 4;
        proven_mem_mut_t piece = { wire + pos, n };
        while (piece.size > 0) {
            proven_size_t used = 0;
            proven_ws_event_t ev;
            EXAMPLE_REQUIRE(proven_ws_decoder_feed(&decoder, piece, &used, &ev) == PROVEN_OK, "the stream
decodes");
            piece.ptr += used;
            piece.size -= used;
            if (ev.kind == PROVEN_WS_EVENT_DATA) {
                memcpy(message + message_len, ev.data.ptr, ev.data.size);      /* 메시지의 한 조각 */
                message_len += ev.data.size;
            } else if (ev.kind == PROVEN_WS_EVENT_PING) {
                pings++;                                                       /* 실제 프로그램은 pong으로 답
한다 */
            } else if (ev.kind == PROVEN_WS_EVENT_CLOSE) {
                close_code = ev.close_code;
            }
        }
        pos += n;
    }
    EXAMPLE_REQUIRE(message_len == 5 && memcmp(message, "Hello", 5) == 0, "two fragments, one message");
    EXAMPLE_REQUIRE(pings == 1 && close_code == PROVEN_WS_CLOSE_NORMAL, "the ping between them, and the
close");

    // ---- 받는 쪽이 거절하는 것 ------------------------------------------
    /* 서버가 보낸 마스킹된 프레임은 프로토콜 위반이다. 오류가 보낼 close 코드를 알려 준다. */
    proven_byte_t bad[] = { 0x81, 0x81, 1, 2, 3, 4, 'x' ^ 1 };
    proven_ws_event_t ev;
    proven_size_t used = 0;
    proven_ws_decoder_init(&decoder, false, 1024);
    proven_err_t err = proven_ws_decoder_feed(&decoder, (proven_mem_mut_t){ bad, sizeof bad }, &used, &ev);
    EXAMPLE_REQUIRE(err == PROVEN_ERR_INVALID_FORMAT && proven_ws_close_code_for(err) ==
PROVEN_WS_CLOSE_PROTOCOL_ERROR, "refused: close with 1002");
```

```
    /* close 프레임의 페이로드만 따로 읽기. 코드 1005는 "코드가 없었다"는 뜻이고 그 자체는
     * 보낼 수 없다. */
    proven_u16 code = 0;
    proven_u8str_view_t reason;
    EXAMPLE_REQUIRE(proven_ws_close_parse((proven_mem_view_t){ close_payload, close_len }, &code, &reason) ==
PROVEN_OK &&
                    code == 1000 && proven_u8str_view_eq(reason, PROVEN_LIT("bye")), "code and reason");
    EXAMPLE_REQUIRE(proven_ws_close_code_is_valid(4000) && !
proven_ws_close_code_is_valid(PROVEN_WS_CLOSE_NO_STATUS), "which codes may travel");

    return EXAMPLE_OK();
}
```

9. 압축된 메시지

```
bool proven_ws_conn_compressed(const proven_ws_conn_t *conn);            /* ws_conn.h */
bool proven_ws_stream_compressed(const proven_ws_stream_t *ws);          /* ws_event.h, 15장 */

#define PROVEN_WS_DEFLATE_OFFER "permessage-deflate; client_no_context_takeover; server_no_context_takeover"
proven_err_t proven_ws_deflate_accept(const proven_http_request_t *request, proven_mem_mut_t out,
proven_size_t *len, int *window_bits);
proven_err_t proven_ws_check_response_deflate(const proven_http_response_t *response, proven_u8str_view_t key,
                                              proven_u8str_view_t offered, proven_u8str_view_t *protocol, bool
*deflate);

#define PROVEN_WS_DEFLATE_OFFER_TAKEOVER "permessage-deflate; client_max_window_bits"
/* proven_ws_deflate_terms_t: bool server_takeover, client_takeover; int server_window_bits,
client_window_bits; */
proven_err_t proven_ws_deflate_accept_takeover(const proven_http_request_t *request, proven_mem_mut_t out,
proven_size_t *len,
                                               proven_ws_deflate_terms_t *terms);
proven_err_t proven_ws_check_response_takeover(const proven_http_response_t *response, proven_u8str_view_t
key,
                                               proven_u8str_view_t offered, proven_u8str_view_t *protocol,
bool *deflate,
                                               proven_ws_deflate_terms_t *terms);

#define PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER "permessage-deflate; client_max_window_bits;
server_no_context_takeover"
proven_err_t proven_ws_deflate_accept_peer_no_takeover(const proven_http_request_t *request, proven_mem_mut_t
out, proven_size_t *len,
                                                       proven_ws_deflate_terms_t *terms);
proven_err_t proven_ws_check_response_peer_no_takeover(const proven_http_response_t *response,
proven_u8str_view_t key,
                                                       proven_u8str_view_t offered, proven_u8str_view_t
*protocol, bool *deflate,
                                                       proven_ws_deflate_terms_t *terms);
void proven_inflate_restart(proven_inflate_t *z);                        /* deflate.h, 16장 */
void proven_ws_decoder_allow_compressed(proven_ws_decoder_t *decoder);
```

WebSocket 연결은 핸드셰이크에서 메시지를 압축하기로 합의할 수 있다: 이 확장은 `permessage-deflate`(RFC 7692)라고 부르고, 모든 브라우저가 제안하며, 16장의 DEFLATE다. JSON 한 쪽이나 로그 여러 줄은 3분의 1 이하로 줄어든다. 각 쪽이 메시지에서 다음 메시지로 윈도를 이어 가므로, 앞선 메시지와 닮은 짧은 메시지도 그만큼 줄어든다.

요청하기 전에는 꺼져 있다. 연결 설정 - `proven_ws_conn_config_t`나 15장의 `proven_ws_event_config_t` - 에서 `compress`를 켠다. 그러면 클라이언트는 확장을 제안하고 서버는 제안을 받으면 수락한다. 상대가 동의했는지는 `proven_ws_conn_compressed`와 `proven_ws_stream_compressed`가 말해 준다. **그 밖에는 아무것도 달라지지 않는다**: 같은 호출로 보내고 받으며, 상대가 동의하지 않은 연결은 그저 압축 없이 이어진다.

	압축에 합의했을 때
보내는 메시지	8바이트 이상이면 압축된다(`compress_no_takeover`에서는 64바이트). 더 짧으면 그대로 가고, 윈도에도 들어가지 않는다
받는 메시지	보기 전에 풀린다: `proven_ws_conn_receive`에서는 통째로, 이벤트 구동 서버에서는 최대 16 KiB 조각으로
`max_message_bytes`	**풀린** 크기를 센다. 몇백 바이트가 그것을 넘겨 펼쳐지면, 너무 큰 평문 메시지와 마찬가지로 1009로 연결이 끝난다
텍스트	푼 다음 UTF-8인지 검사한다. 아니면 1007
풀리지 않는 바이트	1007
합의하지 않은 연결에서 압축 표시가 된 프레임	1002

각 쪽이 메시지에서 다음 메시지로 윈도를 이어 간다("context takeover"): 메시지가 앞선 메시지들을 참조한다. 작고 비슷한 메시지가 이득을 보는 곳이 거기다 - 이 라이브러리의 압축기로, 종류마다 메시지 2,000개를 잰 값이다:

메시지	**하나하나 따로 압축**	**takeover (기본 윈도 11)**	**takeover, 윈도 15**
채팅 JSON, 약 140바이트	원래 크기의 80.7%	22.1%	17.3%
시세, 약 60바이트	100% (너무 짧아 압축 안 함)	39.3%	35.5%
ping, 약 25바이트	100% (너무 짧아 압축 안 함)	26.1%	26.1%
게임 상태, 약 750바이트	30.4%	17.4%	16.7%

대가는 첫 압축 메시지부터 **연결이 끝날 때까지** 드는 메모리다: 압축기(기본 윈도 11비트에서 약 28 KiB, 15에서 324 KiB)와 해제기(약 35 KiB) - 한 번씩 보내고 받은 연결에서 약 64 KiB로 쟀다 (`test_unit_ws_event`). 메시지 하나의 시간은 짧은 메시지에서 거의 같고, 750바이트짜리는 따로 압축할 때 24 us에 비해 38 us였다.

설정의 `compress_no_takeover`는 다른 형태("no context takeover", 양방향)를 요청한다. v0.80.0 전에는 이것뿐이었다: 압축기는 한 번의 전송 동안만, 해제기는 압축된 메시지의 첫 프레임부터 마지막 프레임까지만 존재하고, **유휴 연결은 압축 상태를 전혀 쥐지 않는다** - 조용한 연결 만 개는 압축이 없을 때와 같은 비용이 든다. 어느 쪽이든 이것을 요청할 수 있다: takeover를 쓰는 서버도 takeover를 막는 제안에는 그대로 답하고, takeover를 쓰는 클라이언트도 takeover를 막는 답을 받아들인다. 이쪽의 윈도를 제한하는 상대(제안의 `server_max_window_bits`, 답의 `client_max_window_bits`)는 따른다. 이 압축기의 최소보다 작은 256바이트로 제한하는 상대에게는 이쪽 메시지를 압축 없이 보낸다.

`compress_peer_no_takeover`는 둘 사이의 형태다: **이쪽은 윈도를 이어 가고, 상대는 그러지 못한다.** 이쪽이 상대에게 보내는 메시지는 이득을 그대로 얻고, 메시지 사이에 해제기가 필요 없으므로 압축 메시지를 하나씩 주고받은 유휴 연결은 양방향 takeover일 때 약 64 KiB를 쥐는 대신 약 30 KiB를 쥔다(측정, `test_unit_ws_event`). 대신 상대의 짧고 비슷한 메시지는 따로 압축할 때만큼만 줄어든다. 서버는 제안에 있든 없든 `client_no_context_takeover`로 답하고(RFC 7692가 허용한다), 클라이언트는 `server_no_context_takeover`를 제안한다. 그것을 따르지 않을 서버는 확장을 거절하므로 연결은 압축 없이 이어진다. 그래도 takeover하는 클라이언트는 1007로 닫힌다: 그 메시지는 따로는 풀리지 않는다.

`proven_ws_conn_open`은 핸드셰이크를 하지 않는다: 양 끝이 같게 말해야 한다 - takeover, 또는 `compress_no_takeover`(`compress_peer_no_takeover`는 거기서 `PROVEN_ERR_INVALID_ARG`로 거부된다: 핸드셰이크만 맺을 수 있는 합의다). v0.80.0 전에 빌드한 끝은 언제나 takeover 없음을 뜻한다. 다른 끝에 `compress_no_takeover`를 주어라. 그러지 않으면 그 끝의 두 번째 압축 메시지는 1007로 거부된다.

예제: 필드 하나, 그리고 그것이 바꾸는 것

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <string.h>

/*
 * 프로그램이 쓰는 방식 그대로의 압축된 WebSocket 메시지: 설정의 필드 하나.
 *
 * 여기서 서버는 이벤트 구동 서버이고 클라이언트는 블로킹 클라이언트다. 둘 다 `compress`를
 * 켠다. 핸드셰이크가 합의한다. 그 뒤로 모든 호출은 전과 똑같이 쓰고, 네트워크를 건너는
 * 것은 말한 것의 일부분이다.
 */

typedef struct {
    proven_loop_t *loop;
    bool compressed;                  /* 서버가 합의됐다고 본 것 */
    proven_size_t received, largest, pieces;
} ex_app_t;

/* 압축된 메시지는 풀린 채로, 최대 16 KiB 조각으로 도착한다. 여러분이 쥐지 않는 한 전체를
 * 쥐는 것은 없다. 이 핸들러는 세기만 하고, 메시지가 끝나면 답한다. */
static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    ex_app_t *app = ctx;
    (void)text;
    if (first) { app->received = 0; app->pieces = 0; app->largest = 0; }
    app->received += piece.size;
    app->pieces++;
    if (piece.size > app->largest) app->largest = piece.size;
    if (!last) return;
    char answer[64];
    int n = snprintf(answer, sizeof answer, "%u bytes", (unsigned)app->received);
    /* 여느 메시지처럼 보낸다. 짧은 것은 그대로 간다: 8바이트 미만에서는 얻을 것이 없다
     * (compress_no_takeover에서는 64바이트 미만. 메시지가 자기 자신만 참조할 수 있기 때문이다). */
    (void)proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)answer,
(proven_size_t)n });
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    proven_ws_event_config_t config = {
        .on = { .on_message = on_message },
        .ctx = app,
        .compress = true,             /* 클라이언트가 제안하면 확장을 받아들인다 */
        .max_message_bytes = 256 * 1024,   /* 풀린 뒤의 메시지에 걸리는 한도: 몇 바이트가 기가바이트가 될 수는
없다 */
    };
    proven_ws_stream_t *ws = NULL;
    if (proven_ws_event_accept(stream, head, &config, &ws) != PROVEN_OK) {
        (void)proven_http_stream_respond(stream, 400, NULL, 0, (proven_mem_view_t){ 0 });
        return;
    }
    /* 합의됐는지는 클라이언트에도 달렸다. 어느 쪽이든 달리 할 일은 없다. */
    app->compressed = proven_ws_stream_compressed(ws);
    /* 이쪽에도 같은 스위치가 있다. 답을 압축하면 안 되는 서버라면
     * proven_ws_stream_set_compress(ws, false). 여기서는 압축해도 된다. */
    proven_ws_stream_set_compress(ws, true);
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
```

```
    ex_app_t app = { 0 };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request }, .ctx = &app };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "the loop runs on
another thread");

    // ---- 압축을 요청하는 클라이언트 ------------------------------------------
    proven_http_client_config_t client_config = { .alloc = heap };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "an HTTP client");
    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/", (unsigned)at.port);
    proven_ws_conn_config_t ws_config = { .alloc = heap, .compress = true };       /* 확장을 제안한다 */
    proven_ws_conn_t *conn = NULL;
    EXAMPLE_REQUIRE(proven_ws_conn_connect(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, NULL, 0, PROVEN_LIT(""),
                                           &ws_config, &conn, NULL) == PROVEN_OK, "connected");
    EXAMPLE_REQUIRE(proven_ws_conn_compressed(conn), "and the server agreed to compress");

    /* 보고서 십만 바이트. 전송되는 것은 몇백 바이트다. */
    static char report[100000];
    for (proven_size_t i = 0; i < sizeof report; ++i) report[i] = "temperature 21.5, humidity 40\n"[i % 30];
    proven_ws_message_t msg;
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, (proven_u8str_view_t){ (const proven_byte_t *)report,
sizeof report }) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.data.size == 12 && memcmp(msg.data.ptr, "100000 bytes", 12) == 0, "the server received
all of it");

    /* 한도는 풀린 크기에 걸린다. 한 글자 삼십만 개는 압축하면 몇백 바이트다 - 그래도
     * 너무 많다: 서버는 1009로 연결을 끝낸다. */
    /* 압축기를 거치지 않는 메시지 하나 - 상대가 고른 텍스트 옆에 비밀을 실은 메시지에 필요한 것.
     * 스위치는 메시지가 시작될 때 읽힌다. */
    proven_ws_conn_set_compress(conn, false);
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, (proven_u8str_view_t){ (const proven_byte_t *)report,
1000 }) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.data.size == 10 && memcmp(msg.data.ptr, "1000 bytes", 10) == 0, "a thousand bytes,
sent as they are");
    proven_ws_conn_set_compress(conn, true);

    static char flood[300000];
    memset(flood, 'x', sizeof flood);
```

```
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, (proven_u8str_view_t){ (const proven_byte_t *)flood, sizeof
flood }) == PROVEN_OK, "a message that inflates past the server's limit");
    EXAMPLE_REQUIRE(proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_ERR_EOF &&
proven_ws_conn_close_code(conn) == 1009, "is answered with 1009, however small it was on the wire");
    proven_ws_conn_destroy(conn);
    proven_http_client_destroy(client);

    proven_loop_stop(app.loop);
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.compressed, "the server found the extension agreed as well");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

예제: 코덱만으로 다루는 확장

코덱을 직접 구동하는 프로그램(6절부터 8절)을 위한 것: 핸드셰이크 함수, 프레임의 비트, 그리고 네 바이트.

```
#include <stdio.h>
#include <string.h>

/*
 * 코덱만으로 다루는 압축된 WebSocket 메시지: 소켓도 드라이버도 없다.
 *
 * permessage-deflate라는 확장은 세 가지로 이루어진다. 핸드셰이크가 그것에 합의한다.
 * 프레임 헤더의 비트 하나가 "이 메시지는 압축되었다"고 말한다. 그리고 페이로드는 마지막
 * 네 바이트를 뗀 DEFLATE다. 앞의 둘은 코덱이 하고, 셋째는 deflate.h가 한다.
 *
 * ws_conn.h나 ws_event.h를 쓰는 프로그램은 필드 하나를 정할 뿐 이것을 볼 일이 없다.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- 핸드셰이크 ------------------------------------------------------------
    /* 클라이언트는 헤더 하나로 요청한다. PROVEN_WS_DEFLATE_OFFER는 이 라이브러리가 말하는 두
     * 형태 가운데 단순한 쪽이다: 양쪽 방향 모두, 메시지 하나하나를 따로 압축한다. 다른 하나인
     * context takeover는 맨 끝에 있다. */
    proven_http_header_t asked[] = {
        { PROVEN_LIT("Sec-WebSocket-Extensions"), PROVEN_LIT(PROVEN_WS_DEFLATE_OFFER) },
    };
    proven_http_request_t request = { .headers = asked, .header_count = 1 };

    /* 서버는 제안된 것을 보고 답을 쓴다 - 또는 맞춰 줄 수 있는 것이 없으면
     * (PROVEN_ERR_NOT_FOUND) 연결은 압축 없이 이어진다. */
    proven_byte_t answer[PROVEN_WS_DEFLATE_ANSWER_MAX];
    proven_size_t answer_len = 0;
    int window_bits = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept(&request, (proven_mem_mut_t){ answer, sizeof answer },
&answer_len, &window_bits) == PROVEN_OK, "the offer can be met");
    printf("the server answers: %.*s\n", (int)answer_len, (const char *)answer);
    EXAMPLE_REQUIRE(window_bits == 15, "and may compress with the full window");

    /* 브라우저의 제안은 모양이 다르지만, 그것도 맞춰 준다. */
    proven_http_header_t browser[] = { { PROVEN_LIT("Sec-WebSocket-Extensions"), PROVEN_LIT("permessage-
deflate; client_max_window_bits") } };
    proven_http_request_t from_browser = { .headers = browser, .header_count = 1 };
    proven_size_t len2 = 0;
```

```
    EXAMPLE_REQUIRE(proven_ws_deflate_accept(&from_browser, (proven_mem_mut_t){ answer, sizeof answer },
&len2, &window_bits) == PROVEN_OK, "a browser's offer too");

    /* 클라이언트는 답 전체를 검사한다: 늘 보던 것들, 그리고 확장이 요청한 형태로 돌아왔는지.
     * (키와 accept 값은 RFC 6455에 실린 한 쌍이다.) */
    proven_http_header_t answered[] = {
        { PROVEN_LIT("Upgrade"), PROVEN_LIT("websocket") },
        { PROVEN_LIT("Connection"), PROVEN_LIT("Upgrade") },
        { PROVEN_LIT("Sec-WebSocket-Accept"), PROVEN_LIT("s3pPLMBiTxaQ9kYGzzhZRbK+xOo=") },
        { PROVEN_LIT("Sec-WebSocket-Extensions"), { answer, answer_len } },
    };
    proven_http_response_t response = { .status = 101, .headers = answered, .header_count = 4 };
    proven_u8str_view_t protocol;
    bool compressed = false;
    EXAMPLE_REQUIRE(proven_ws_check_response_deflate(&response, PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="),
PROVEN_LIT(""), &protocol, &compressed) == PROVEN_OK && compressed,
                    "the client finds the extension agreed");

    // ---- 압축된 메시지가 도착한다 ------------------------------------------------------
    /* 이 프레임은 RFC 7692에 있다: 서버가 보낸, 압축된 텍스트 "Hello". 압축하지 않은 텍스트
     * 프레임이라면 81일 첫 바이트가 c1이다: 더해진 비트가 RSV1이다. */
    proven_byte_t wire[] = { 0xc1, 0x07, 0xf2, 0x48, 0xcd, 0xc9, 0xc9, 0x07, 0x00 };

    proven_ws_decoder_t decoder;
    proven_ws_event_t event;
    proven_size_t used = 0;
    /* 확장에 대해 듣지 못한 디코더는 그 비트를 거부한다. 그래야 한다. */
    proven_byte_t copy[sizeof wire];
    memcpy(copy, wire, sizeof wire);
    proven_ws_decoder_init(&decoder, false, 0);
    EXAMPLE_REQUIRE(proven_ws_decoder_feed(&decoder, (proven_mem_mut_t){ copy, sizeof copy }, &used, &event)
== PROVEN_ERR_INVALID_FORMAT, "without the extension, the bit is an error");

    proven_ws_decoder_init(&decoder, false, 0);
    proven_ws_decoder_allow_compressed(&decoder);
    EXAMPLE_REQUIRE(proven_ws_decoder_feed(&decoder, (proven_mem_mut_t){ wire, sizeof wire }, &used, &event)
== PROVEN_OK &&
                    event.kind == PROVEN_WS_EVENT_DATA && event.compressed && event.last, "with it, the
message is handed over, marked compressed");

    /* 디코더는 압축을 풀지 않는다: 아무것도 할당하지 않는다. 건네주는 바이트는 raw DEFLATE이고,
     * 보낸 쪽이 마지막 네 바이트 - 언제나 00 00 ff ff - 를 떼어 냈다. 그것을 다시 붙이고
     * 푼다. 나오는 양의 한도는 여러분 버퍼의 크기다. */
    proven_inflate_t *z = NULL;
    EXAMPLE_REQUIRE(proven_inflate_create(heap, PROVEN_DEFLATE_RAW, &z) == PROVEN_OK, "a decompressor for raw
DEFLATE");
    static const proven_byte_t tail[4] = { 0x00, 0x00, 0xff, 0xff };
    proven_byte_t text[64];
    proven_size_t taken = 0, made = 0, total = 0;
    bool ended = false;
    EXAMPLE_REQUIRE(proven_inflate(z, event.data, &taken, (proven_mem_mut_t){ text, sizeof text }, &made,
&ended) == PROVEN_OK && taken == event.data.size, "the payload");
    total = made;
    EXAMPLE_REQUIRE(proven_inflate(z, (proven_mem_view_t){ tail, 4 }, &taken, (proven_mem_mut_t){ text +
total, sizeof text - total }, &made, &ended) == PROVEN_OK && taken == 4, "then the four bytes");
    total += made;
    EXAMPLE_REQUIRE(total == 5 && memcmp(text, "Hello", 5) == 0, "and the message is Hello");
    printf("the message: %.*s\n", (int)total, (const char *)text);
    /* 이제야 - 그 전이 아니라 - 텍스트이고, 검사는 여러분 몫이다: 디코더는 할 수 없었다. */
    proven_inflate_destroy(z);
```

```
    // ---- 압축된 메시지를 보낸다 ----------------------------------------------------
    /* 반대 방향: sync flush로 압축하면 같은 네 바이트로 끝나는데, 그것을 떼어 낸다.
     * 이것을 싣는 프레임에는 `rsv1`이 켜져 있다. */
    static const char said[] = "to be, or not to be, that is the question: to be, or not to be";
    proven_deflate_options_t options = { .alloc = heap, .format = PROVEN_DEFLATE_RAW };
    proven_deflate_t *c = NULL;
    EXAMPLE_REQUIRE(proven_deflate_create(&options, &c) == PROVEN_OK, "a compressor");
    proven_byte_t packed[128];
    EXAMPLE_REQUIRE(proven_deflate(c, (proven_mem_view_t){ (const proven_byte_t *)said, sizeof said - 1 },
&taken, (proven_mem_mut_t){ packed, sizeof packed }, &made,
                                   PROVEN_DEFLATE_FLUSH_SYNC, &ended) == PROVEN_OK && taken == sizeof said - 1
&& made > 4, "compressed, with a sync flush");
    EXAMPLE_REQUIRE(memcmp(packed + made - 4, tail, 4) == 0, "which ends in 00 00 ff ff");
    proven_size_t payload = made - 4;
    proven_deflate_destroy(c);

    proven_ws_frame_t frame = { .fin = true, .opcode = PROVEN_WS_TEXT, .rsv1 = true, .length = payload };
    proven_byte_t header[PROVEN_WS_MAX_FRAME_HEADER];
    proven_size_t header_len = 0;
    EXAMPLE_REQUIRE(proven_ws_frame_write((proven_mem_mut_t){ header, sizeof header }, &header_len, &frame) ==
PROVEN_OK && header[0] == 0xc1, "a frame header with the bit");
    printf("%u bytes of text go out as %u\n", (unsigned)(sizeof said - 1), (unsigned)(header_len + payload));
    EXAMPLE_REQUIRE(header_len + payload < sizeof said - 1, "fewer than were said");

    // ---- context takeover와 함께 ---------------------------------------------------
    /* 다른 형태: 양쪽이 메시지에서 다음 메시지로 창을 이어 가므로, 메시지가 앞선 메시지들을
     * 가리킬 수 있다. 클라이언트는 PROVEN_WS_DEFLATE_OFFER_TAKEOVER를 제안하고, 서버는
     * proven_ws_deflate_accept_takeover로 답한다. 이 함수는 제안이 요청한 곳에서만 takeover를
     * 막고, 무엇이 합의되었는지 알려 준다. */
    proven_http_header_t asked_take[] = { { PROVEN_LIT("Sec-WebSocket-Extensions"),
PROVEN_LIT(PROVEN_WS_DEFLATE_OFFER_TAKEOVER) } };
    proven_http_request_t request_take = { .headers = asked_take, .header_count = 1 };
    proven_ws_deflate_terms_t terms;
    answer_len = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept_takeover(&request_take, (proven_mem_mut_t){ answer, sizeof
answer }, &answer_len, &terms) == PROVEN_OK &&
                    terms.server_takeover && terms.client_takeover, "takeover is agreed both ways");
    printf("the server answers: %.*s\n", (int)answer_len, (const char *)answer);
    answered[3].value = (proven_u8str_view_t){ answer, answer_len };
    proven_ws_deflate_terms_t seen;
    EXAMPLE_REQUIRE(proven_ws_check_response_takeover(&response, PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="),
PROVEN_LIT(""), &protocol, &compressed, &seen) == PROVEN_OK &&
                    compressed && seen.server_takeover && seen.client_takeover, "and the client sees the same
terms");

    /* 한쪽이 자기 창은 유지하고 상대의 것은 막을 수 있다: 그러면 메시지 사이에 해제기가 필요
     * 없다. 서버는 proven_ws_deflate_accept_peer_no_takeover로, 클라이언트는
     * PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER를 제안하고 proven_ws_check_response_peer_no_takeover로
     * 답을 확인한다. 여기서는 서버가 막고, 이어서 클라이언트가 막는다. */
    answer_len = 0;
    EXAMPLE_REQUIRE(proven_ws_deflate_accept_peer_no_takeover(&request_take, (proven_mem_mut_t){ answer,
sizeof answer }, &answer_len, &terms) == PROVEN_OK &&
                    terms.server_takeover && !terms.client_takeover, "the server keeps its window and forbids
the client's");
    printf("a server that forbids the client's: %.*s\n", (int)answer_len, (const char *)answer);
    proven_http_header_t asked_own[] = { { PROVEN_LIT("Sec-WebSocket-Extensions"),
PROVEN_LIT(PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER) } };
    proven_http_request_t request_own = { .headers = asked_own, .header_count = 1 };
    answer_len = 0;
```

```
    EXAMPLE_REQUIRE(proven_ws_deflate_accept_takeover(&request_own, (proven_mem_mut_t){ answer, sizeof
answer }, &answer_len, &terms) == PROVEN_OK &&
                    !terms.server_takeover && terms.client_takeover, "a server honours a client that forbids
its takeover");
    answered[3].value = (proven_u8str_view_t){ answer, answer_len };
    EXAMPLE_REQUIRE(proven_ws_check_response_peer_no_takeover(&response,
PROVEN_LIT("dGhlIHNhbXBsZSBub25jZQ=="), PROVEN_LIT(""), &protocol, &compressed, &seen) == PROVEN_OK &&
                    compressed && !seen.server_takeover && seen.client_takeover, "and the client checks that
it did");

    /* takeover하는 쪽은 연결에 압축기 하나를 두고 메시지마다 sync flush로 끝낸다. 읽는 쪽은
     * 해제기 하나를 둔다. 같은 줄을 두 번: 두 번째는 첫 번째를 가리키는 참조로 나간다. */
    EXAMPLE_REQUIRE(proven_deflate_create(&options, &c) == PROVEN_OK && proven_inflate_create(heap,
PROVEN_DEFLATE_RAW, &z) == PROVEN_OK, "one of each, kept");
    for (int round = 0; round < 2; ++round) {
        EXAMPLE_REQUIRE(proven_deflate(c, (proven_mem_view_t){ (const proven_byte_t *)said, sizeof said - 1 },
&taken, (proven_mem_mut_t){ packed, sizeof packed }, &made,
                                       PROVEN_DEFLATE_FLUSH_SYNC, &ended) == PROVEN_OK && made > 4, "a message
through the kept compressor");
        printf("message %d goes out as %u bytes\n", round + 1, (unsigned)(made - 4));
        if (round == 1) EXAMPLE_REQUIRE(made - 4 < 10, "the second is a few bytes: a reference into the
first");
        /* 읽는 쪽: 도착한 그대로의 payload - 빠진 네 바이트를 다시 붙인 것. */
        proven_byte_t back[128];
        proven_size_t got = 0;
        EXAMPLE_REQUIRE(proven_inflate(z, (proven_mem_view_t){ packed, made }, &taken, (proven_mem_mut_t)
{ back, sizeof back }, &got, &ended) == PROVEN_OK &&
                        got == sizeof said - 1 && memcmp(back, said, got) == 0, "and the kept decompressor
reads it");
        /* 보내는 쪽은 메시지를 마지막 블록으로 끝낼 수도 있다(RFC 7692, 7.2.3.4). 그러면 해제기가
         * `ended`를 알리고, 읽는 쪽은 창을 지키는 proven_inflate_restart로 이어 간다 - 창을 잊는
         * reset이 아니다. */
        if (ended) proven_inflate_restart(z);
    }
    proven_deflate_destroy(c);
    proven_inflate_destroy(z);
    return EXAMPLE_OK();
}
```

참조

API	의도	반환
`compress` (두 연결 설정)	확장을 제안한다(클라이언트) 또는 제안받으면 수락한다(서버).	-
`compress_level`, `compress_window_bits` (둘 다)	16장 4절과 같다: 1부터 9 또는 -1. 9부터 15. 0이면 6, 그리고 takeover에서 11, 아니면 15.	범위를 벗어나면 `INVALID_ARG`.
`compress_no_takeover` (둘 다)	메시지를 하나하나 따로 압축하고 메시지 사이에 아무것도 남기지 않는다 - v0.80.0 전의 형태.	-
`proven_ws_conn_compressed(conn)`, `proven_ws_stream_compressed(ws)`	상대가 동의했는가.	`bool`.
`proven_ws_conn_set_compress(conn, on)`, `proven_ws_stream_set_compress(ws, on)`	이쪽이 이제부터 보내는 메시지를 압축할지. 압축하면 안 되는 한 메시지 - 상대가 고른 텍스트 옆의 비밀, 또는 이미 압축된 바이트 -	없음.

	를 위한 것: 끄고, 보내고, 켠다. 압축에 합의하지 않은 연결에서는 아무 일도 하지 않는다.	
PROVEN_WS_DEFLATE_OFFER	클라이언트가 보내는 Sec-WebSocket-Extensions 값.	문자열 리터럴.
proven_ws_deflate_accept(request, out, &len, &window_bits)	요청의 제안 가운데 서버가 고른다. 답을 쓴다.	proven_err_t: 맞춰 줄 제안이 없으면 NOT_FOUND - 압축 없이 간다. OUT_OF_BOUNDS(PROVEN_WS_DEFLATE_ANSWER_MAX 바이트면 충분하다).
proven_ws_check_response_deflate(response, key, offered, &protocol, &deflate)	제안을 보낸 클라이언트를 위한 proven_ws_check_response.	같고, 제안하지 않은 형태의 답에는 INVALID_FORMAT.
PROVEN_WS_DEFLATE_OFFER_TAKEOVER	takeover를 요청하는 클라이언트가 보내는 값.	문자열 리터럴.
proven_ws_deflate_accept_takeover(request, out, &len, &terms)	윈도를 이어 가는 서버의 proven_ws_deflate_accept: 제안이 요청한 곳에서만 takeover를 막는다. terms는 합의된 것을 말한다.	proven_ws_deflate_accept와 같다.
proven_ws_check_response_takeover(response, key, offered, &protocol, &deflate, &terms)	PROVEN_WS_DEFLATE_OFFER_TAKEOVER를 보낸 클라이언트의 검사: 네 매개변수 무엇이든 한 번씩. terms는 합의된 것을 말한다.	같고, 매개변수가 두 번이거나 모르는 것이거나 범위 밖이면 INVALID_FORMAT.
compress_peer_no_takeover (둘 다)	이쪽은 윈도를 이어 가고, 상대는 메시지를 하나씩 따로 압축해야 한다. 서버는 답에서 막고, 클라이언트는 제안에서 요청한다.	proven_ws_conn_open에서 INVALID_ARG.
PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER	상대 윈도는 이어 가고 서버에게는 이어 가지 말라고 요청하는 클라이언트가 보내는 값.	문자열 리터럴.
proven_ws_deflate_accept_peer_no_takeover(request, out, &len, &terms)	답에 (연재), client_no_context_takeover가 붙는 proven_ws_deflate_accept_takeover.	proven_ws_deflate_accept와 같다.
proven_ws_check_response_peer_no_takeover(response, key, offered, &protocol, &deflate, &terms)	PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER를 보낸 클라이언트의 검사.	proven_ws_check_response_takeover와 같고, server_no_context_takeover가 없는 답에는 INVALID_FORMAT.
proven_ws_deflate_terms_t	누가 윈도를 이어 가도 되는지 (server_takeover, client_takeover), 각 쪽 윈도가 얼마까지인지 (server_window_bits, client_window_bits. 8이면 이쪽은 압축할 수 없어 압축 없이 보낸다).	-
proven_inflate_restart(z)	ended 뒤에 윈도를 지키며 이어 간다. takeover를 쓰는 상대가 메시지를 마지막 블록으로 끝냈을 때.	없음.

proven_ws_decoder_allow_compressed(decoder)	디코더가 압축된 메시지를 compressed 표시와 함께 통과시키게 한다. 풀지는 않는다.	없음.
proven_ws_frame_t의 rsv1	압축된 메시지를 표시하는 비트. 첫 프레임에 있다.	-

주의사항, 그리고 무엇이 잘못되는가

- **비밀과 상대가 고른 글자가 함께 실린 메시지는 압축하지 말 것.** 압축된 메시지의 길이는 연결을 지켜보는 이에게 그 둘이 얼마나 겹치는지를 알려 준다. TLS가 있든 없든 그렇다 - 16장 5절에서 설명한 누출이다. takeover(기본)에서는 비밀과 상대가 고른 글자가 같은 메시지에 있지 않아도 된다: 앞선 메시지가 뒤 메시지의 길이도 바꾼다. compress_no_takeover에서는 메시지를 하나씩 따로 압축하므로 한 메시지가 다른 메시지를 새게 하지는 못한다. 한 메시지 안의 것은 여전히 샐 수 있다. 여기서 압축은 연결 단위다: 그런 메시지가 오가는 연결은 압축을 요청하지 말아야 한다.
- **한도는 풀린 크기에 걸리며, 정해 두어야 한다.** 킬로바이트가 기가바이트가 되는 것을 막는 것이 max_message_bytes다. 기본값은 1 MiB.
- **raw DEFLATE에는 체크섬이 없다.** 전송 중 손상된 압축 메시지는 오류 없이 다른 바이트로 풀릴 수 있다. TLS가 그것을 막는다. TLS가 없다면 중요한 것은 직접 검사한다.
- **맞춰 줄 수 없는 것만 제안하는 상대는 거절하지 않고 압축 없이 응대한다.** 그런 제안은 하나뿐이다: 256바이트 압축 윈도(server_max_window_bits=8)인데, 이 압축기에는 그 크기가 없다. 클라이언트 자신의 윈도 크기(client_max_window_bits)는 받아들이고 무시한다: 어떤 윈도든 읽을 수 있다.
- **compress_no_takeover인 클라이언트는 다른 형태로 답하는 서버를 거부한다.** 그 제안을 server_no_context_takeover 없이 받아들이는 서버는 요청과 달리 메시지에 걸쳐 압축할 것이다. 연결은 PROVEN_ERR_INVALID_FORMAT으로 실패한다. RFC 7692를 따르는 서버는 제안을 그대로 수락하거나 거절한다.
- **메모리.** takeover에서는: 약 28 KiB(윈도 11)와 35 KiB를 첫 압축 메시지부터 끝까지 쥐고, 이벤트 구동 서버는 메시지가 도착하는 동안 16 KiB를 더 쓴다. compress_no_takeover에서는: 메시지 하나를 푸는 데 약 35 KiB(이벤트 구동 서버에서는 51 KiB), 하나를 압축하는 데 윈도에 따라 11 KiB에서 325 KiB까지이며 메시지가 필요로 하는 것보다 크지 않고, 메시지를 위해 할당해 메시지와 함께 해제한다 - 그래서 바쁜 연결은 takeover 연결이 하지 않는 할당을 메시지마다 한다. compress_peer_no_takeover에서는: 압축기는 쥐고, 해제기는 받는 메시지마다.
- **코덱만 쓸 때 텍스트 검사는 여러분 몫이다.** 디코더는 아직 텍스트가 아닌 바이트를 검사할 수 없다. 푼 다음에 검사한다. 그리고 나오는 양을 센다: 디코더의 한도는 압축된 바이트를 센다.

10. 여기에 없는 것

- **permessage-deflate 말고 다른 확장.** 9절.
- **자기 takeover는 지키면서 클라이언트의 takeover만 막는 서버**(약 64 KiB 대신 28 KiB를 쥘 것이다): 서버는 각 제안이 허락하는 만큼만 정확히 takeover한다.
- **여러 연결을 한꺼번에 기다리는 호출.** §5를 보라.
- **한 연결에서 동시에 보내고 받기.**
- **자동 ping, 재연결, 백오프.** 정책이고, 여러분의 것이다.
- **ws_conn.h로 메모리보다 큰 메시지 다루기.** 메시지는 통째로 조립된다. 흘려 받으려면 디코더를 써라.
- **Origin을 대신 확인해 주기.** 여기의 어떤 서버도 스스로 출처를 거절하지 않는다: 출처 목록은 여러분의 것이고, 호출도 그렇다(proven_ws_check_origin, §2).
- **HTTP/2 WebSocket(RFC 8441).**

13장: 인증서와 신뢰

5부 — 운영체제와 대화하기. 선행 조건: 다이제스트는 4장, 저장소가 받는 할당자(allocator) - 예제에서는 힙(heap) 할당자 - 는 2장. 이 장을 마치면 X.509 인증서를 읽고, 어느 이름을 위한 것인지 말하고, 믿는 인증서를 불러오고, 상대의 인증서 체인을 믿을지 결정할 수 있다 - 그리고 어떤 검사를 했고 어떤 검사를 하지 않았는지 정확히 안다.

이 장은 `cert.h`를 다룬다. 읽기와 검증은 아무것도 할당하지 않고 운영체제를 부르지 않는다. 프리스탠딩(freestanding) 빌드에서 쓸 수 있다. 신뢰 앵커 저장소는 여러분이 준 할당자로 할당한다. 호출 하나, `proven_cert_store_add_system`은 운영체제의 루트를 읽으며 호스티드(hosted) 전용이다.

이것이 무엇인가. 이것은 TLS의 인증서 절반이며 따로도 쓸 수 있다. 파일, 메시지, 또는 다른 라이브러리의 핸드셰이크에서 건네받은 인증서를 검사하는 일이다. 프로토콜 자체 - 핸드셰이크와 암호화된 레코드 - 는 14장이고, 여기 있는 모든 것을 쓴다.

그리고 이 장에 무엇이 더해지든 남아 있을 경고. 이것은 외부 감사를 받지 않은 새 구현이다. 다른 구현과 대조하고, 공개된 적대적 벡터로 시험했으며(어느 것인지는 8절), 그것은 증거이지 증명이 아니다. 능력 있는 공격자를 상대로 사용자가 의존하는 프로그램이라면 감사받은 것으로 인증서를 검사해야 한다.

목차

1. 인증서가 무엇인지, 한 쪽으로
2. 하나 읽기
3. PEM
4. 어느 이름을 위한 것인가
5. 신뢰 앵커
6. 체인 검증
7. 피닝
8. 무엇을 검사하고, 무엇을 받아들이고, 무엇이 없는가

1. 인증서가 무엇인지, 한 쪽으로

공개 키가 있으면 어떤 메시지가 짝이 되는 개인 키를 가진 쪽에서 왔는지 확인할 수 있다. 그러나 그쪽이 *누구인지*는 알려 주지 않는다. **인증서**는 그 빠진 문장이다. "이 공개 키는 `example.com`의 것이다", 그리고 **발급자**의 서명. 발급자의 공개 키가 있으면 그 서명을 확인할 수 있는데, 그 키는 발급자 자신의 인증서에 들어 있고, 그 인증서는 그것의 발급자가 서명했다. 문장들은 **체인**을 이루고, 체인은 어디선가 끝나야 한다. 누가 서명해서가 아니라 이미 여러분의 기계에 있었기 때문에 믿는 인증서에서 끝난다. 그것이 **신뢰 앵커**, 곧 루트다.

그러니 상대를 믿는다는 것은 질문 넷이고, `proven_cert_verify`는 넷을 모두 묻는다.

질문	건너뛰면 잘못되는 것
이 인증서에서 내가 가진 앵커까지 유효한 서명의 체인이 있는가?	누구나 어떤 이름으로든 인증서를 만들 수 있다. 체인만이 여러분이 믿는 누군가가 보증했다고 말해 준다.
체인의 모든 인증서가 이 순간에 유효했는가?	몇 해 전에 유출된 키도 여전히 서명한다. 날짜는 유출이 문제가 되는 기간을 제한한다.
체인의 모든 발급자가 발급할 *자격*이 있었는가?	평범한 웹 사이트의 인증서도 진짜 기관이 서명한 것이다. 그것이 다른 인증서에 서명할 수 있다면 사이트 주인 누구나 어떤 이름의 인증서든 찍어 낼 수 있다.
인증서의 이름이 내가 닿으려던 이름인가?	`attacker.example`의 완벽하게 유효한 인증서는 `bank.example`에 대해 아무것도 증명하지 않는다.

형식: 인증서는 **DER** - 압축된 이진 인코딩 - 이고, 보통 **PEM**으로 다닌다. PEM은 그 DER를 Base64로 바꿔 `-----BEGIN CERTIFICATE-----` 줄과 `END` 줄 사이에 넣은 것으로, 텍스트 파일에 붙여 넣어도 살아남는다.

2. 하나 읽기

```
proven_err_t proven_cert_parse(proven_mem_view_t der, proven_cert_t *out);
```

`proven_cert_parse`는 DER 인증서 하나를 `proven_cert_t`로 읽는다. **아무것도 복사하지 않는다.** 구조체의 모든 필드는 여러분이 넘긴 바이트를 가리키는 뷰(view)이며, 구조체를 쓰는 동안 그 바이트가 살아 있어야 한다. 해제할 것은 없다.

읽기는 일부러 엄격하다. 인증서는 낯선 이가 보낸 입력이고, 두 프로그램이 다르게 읽을 수 있는 인코딩은 둘 중 하나가 속는 길이다. 그래서: 확정 길이만, 각각 가장 짧은 형태로. 쓸데없는 앞자리 0이 없는 정수. 정확히 `FF`인 `TRUE`. 서명된 부분의 안과 밖에 똑같이 적힌 서명 알고리즘. 두 번 나오지 않는 확장. 그리고 끝 뒤에는 한 바이트도 없다. 그 밖의 것은 모두 `PROVEN_ERR_INVALID_FORMAT`이다.

가장 자주 쓰게 될 필드:

필드	무엇인가
`der`, `tbs`	인증서 전체와, 발급자가 서명한 부분
`subject`, `issuer`	두 이름, 각각 DER. 바이트로 비교하라. 호스트 이름을 찾으려고 파싱하지 마라(4절)
`not_before`, `not_after`	유효 기간, 1970-01-01 UTC부터의 초
`public_key_info`	알고리즘이 붙은 공개 키, DER - 핀이 해시하는 것(7절)
`key_kind`, `key`, `rsa_n`, `rsa_e`	키 자체: RSA, P-256, P-384 또는 Ed25519. 그 밖은 `PROVEN_CERT_KEY_UNKNOWN`
`sig_kind`, `sig_hash`, `signature`	*발급자*가 이 인증서에 어떻게 서명했는가
`is_ca`, `has_path_len`, `path_len`	인증서를 발급할 수 있는지, 그 아래 체인이 얼마나 깊어도 되는지
`key_usage`, `ext_key_usage`	키를 무엇에 써도 되는지, `PROVEN_CERT_KU_*`와 `PROVEN_CERT_EKU_*` 플래그
`alt_names`	이 인증서가 위하는 이름들. `proven_cert_alt_name_next`로 훑는다
`unknown_critical`	"이해하지 못하면 받아들이지 말라"고 표시된 확장을 담고 있는데, 이 라이브러리가 이해하지 못한다

이 라이브러리가 모르는 알고리즘 - 다른 곡선의 키, SHA-1 서명 - 은 파싱 에러가 **아니다**. 인증서는 종류가 `UNKNOWN`인 채로 파싱된다. 시스템 루트 저장소에는 그런 인증서가 몇 개 들어 있고, 그 하나 때문에 나머지를 잃어서는 안 되기 때문이다. 체인이 실제로 그것을 필요로 하면 나중에 검증에서 실패한다.

```
bool proven_cert_alt_name_next(const proven_cert_t *cert, proven_size_t *pos,
                               proven_cert_name_kind_t *kind, proven_mem_view_t *value);
```

`proven_cert_alt_name_next`는 subjectAltName 항목을 훑는다. `*pos = 0`으로 시작해 `false`를 돌려줄 때까지 부른다. `PROVEN_CERT_NAME_DNS` 값은 이름의 ASCII 텍스트이고, `PROVEN_CERT_NAME_IP` 값은 주소 바이트 4개 또는 16개이며, 그 밖의 것(이메일 주소, URI)은 원래 내용 그대로 `PROVEN_CERT_NAME_OTHER`로 나온다.

3. PEM

```
proven_err_t proven_pem_next(proven_mem_view_t pem, proven_size_t *pos, proven_u8str_view_t *label,
                             proven_mem_mut_t out, proven_size_t *written);
```

`proven_pem_next`는 `*pos` 이후의 다음 블록을 찾아 그 Base64를 `out`에 디코딩하고, `label`을 `BEGIN` 뒤의 단어(인증서라면 `CERTIFICATE`)로 맞추고, `*pos`를 블록 뒤로 옮긴다. 여러 개가 든 파일은 반복해서 부른다. 블록 사이의 텍스트 - 주석, 어떤 도구가 덧붙이는 사람이 읽는 덤프 - 는 건너뛴다.

반환	언제
`PROVEN_OK`	블록 하나를 디코딩했다. `out`에 `*written`바이트가 있다
`PROVEN_ERR_NOT_FOUND`	블록이 더 없다
`PROVEN_ERR_OUT_OF_BOUNDS`	`out`이 작다. `*written`은 필요한 크기이고 `*pos`는 움직이지 않았다. 키워서 다시 부른다

`PROVEN_ERR_INVALID_ENCODING`	짝이 맞는 `END` 줄이 없는 블록, 또는 Base64가 아닌 본문

인증서만을 위한 것이 아니다. 키 파일도 라벨이 다른 PEM이다. 이 호출은 안에 무엇이 들었는지 상관하지 않는다.

4. 어느 이름을 위한 것인가

```
bool proven_cert_matches_host(const proven_cert_t *cert, proven_u8str_view_t host);
```

`proven_cert_matches_host`는 인증서가 `host`를 위한 것인지 답한다. `host`는 DNS 이름이거나 텍스트로 적은 IP 주소다. 규칙은 브라우저들이 정착시킨 것이고, 하나하나가 한때 열려 있던 구멍을 막는다.

- **subjectAltName만 본다.** subject의 "common name"은 인증서들이 한때 호스트 이름을 적어 넣던 자유 텍스트다. 호스트 이름이 아니며, 들여다보지 않는다.
- **와일드카드는 맨 왼쪽 라벨 전체이고 정확히 라벨 하나를 뜻한다.** `*.example.com`은 `www.example.com`과 맞는다. `example.com`과도, `a.b.example.com`과도 맞지 않으며, `w*.example.com`은 아무 뜻도 없다. 별표 뒤에 라벨이 둘보다 적은 패턴(`*.com`)은 아무것과도 맞지 않는다.
- **IP 주소는 주소로서, IP 항목과만 비교한다.** `192.0.2.1`은 우연히 `192.0.2.1`이라고 적힌 DNS 항목과 절대 맞지 않고, `2001:db8::1`은 주소를 어떻게 적었든 맞는다.
- **대소문자는 상관없고, 호스트 끝의 점 하나는 무시한다.**

이름은 ASCII로 비교한다. 국제화된 이름은 인증서에 나타나는 형태인 `xn--` 꼴로 줘야 한다.

5. 신뢰 앵커

```
proven_err_t proven_cert_store_create(proven_allocator_t alloc, proven_cert_store_t **out);
void proven_cert_store_destroy(proven_cert_store_t *store);
proven_err_t proven_cert_store_add_der(proven_cert_store_t *store, proven_mem_view_t der);
proven_err_t proven_cert_store_add_pem(proven_cert_store_t *store, proven_mem_view_t pem, proven_size_t
*added);
proven_err_t proven_cert_store_add_system(proven_cert_store_t *store, proven_size_t *added);   /* hosted */
proven_size_t proven_cert_store_count(const proven_cert_store_t *store);
```

`proven_cert_store_t`는 여러분이 믿는 인증서의 집합이다. 더한 것의 **사본을 직접 갖는다.** 그래서 더할 때 쓴 바이트는 곧바로 해제해도 된다. `proven_cert_store_destroy`가 사본과 저장소를 해제한다.

라이브러리는 루트를 싣고 다니지 않는다. 라이브러리에 컴파일해 넣은 신뢰 기관 목록은 나가는 날 낡고, 사용자 절반에게는 틀리다. 여러분이 고른다.

- `proven_cert_store_add_system`은 운영체제의 루트를 가져온다 - 공개 인터넷과 대화할 때 맞는 집합이다. Windows에서는 `ROOT` 시스템 저장소를 읽는다. 그 밖에서는 환경 변수 `SSL_CERT_FILE`의 경로와 흔한 번들 위치 가운데 읽을 수 있는 첫 파일을 읽는다. 기계에 읽을 수 있는 것이 없으면 - 최소한의 컨테이너에는 흔히 없다 - `PROVEN_ERR_NOT_FOUND`를 돌려준다. 그것은 보고할 일이지, 모든 것을 믿는 것으로 피해 갈 일이 아니다. **Windows에서는 답이 기대보다 작을 수 있다:** Windows는 지금까지 필요했던 루트만 갖고 있다가, 자신의 TLS가 다른 루트를 만나면 그때 받아 온다. 이 호출은 설치된 것을 읽을 뿐 받아 오기를 일으키지 않으므로, 그 기계가 한 번도 쓴 적 없는 루트로 가는 체인은 거기서 `PROVEN_ERR_UNTRUSTED`다. Windows에서 임의의 서버에 닿아야 하는 프로그램은 번들을 함께 싣고 `proven_cert_store_add_pem`을 써야 한다.
- `proven_cert_store_add_pem`은 여러분이 공급하는 번들을 받는다. 조직의 기관, 또는 프로그램이 대화하는 서버 하나의 인증서 하나. 인증서가 아닌 블록과 파싱되지 않는 인증서는 건너뛰고, `*added`가 받아들인 수를 센다. 인증서가 하나도 없는 번들은 `PROVEN_ERR_NOT_FOUND`다.
- `proven_cert_store_add_der`는 하나를 더한다.

저장소는 검증으로 바뀌지 않으므로, 검증만 하는 스레드라면 몇 개든 저장소 하나를 함께 써도 된다. 다른 스레드가 검증하는 동안 저장소에 더하는 것은 경쟁이다.

6. 체인 검증

```
proven_err_t proven_cert_verify(const proven_mem_view_t *chain, proven_size_t count,
                                const proven_cert_verify_options_t *options,
                                proven_cert_verify_result_t *result);
```

`chain[0]`은 상대 자신의 인증서다. 나머지는 상대가 보낸 그 밖의 것들이며 **순서는 아무래도 좋다** - 서버는 순서에 무심하고, 필요 없는 인증서를 보내기도 한다. `proven_cert_verify`는 나머지를 재료로 삼아 `chain[0]`에서 앵커까지의 경로를 찾는다. 그 가운데 파싱되지 않는 인증서는 그냥 쓰이지 않는다.

옵션:

- `anchors` - 저장소. 필수.
- `host` - 닿으려던 이름. 비워 두면 이름 검사를 건너뛴다. 신원을 다른 방법(핀, 계정에 대응시킨 클라이언트 인증서)으로 확인할 때에만 옳다.
- `now` - 시각, 1970-01-01 UTC부터의 초. **여러분이 공급한다.** 스스로 시계를 읽는 라이브러리는 만료된 인증서를 기다리지 않고는 시험할 수 없고, 시계가 없는 곳에서는 돌 수 없다. 호스티드 시스템에서는 벽시계 시각을 넘겨라. 배터리로 유지되는 시계가 없는 장치라면 어떤 시각을 가정할지 정하고, 날짜 검사가 그 가정만큼만 믿을 만하다는 것을 알아 둬라.
- `use` - 상대가 서버이면 `PROVEN_CERT_USE_SERVER`, 여러분에게 인증서를 내미는 클라이언트이면 `PROVEN_CERT_USE_CLIENT`.

답은 두 수준으로 온다. **에러 코드**는 프로그램이 분기하는 기준이다.

반환	뜻	합리적인 대응
`PROVEN_OK`	경로가 있고 모든 검사를 통과했다	진행
`PROVEN_ERR_UNTRUSTED`	앵커까지 받아들일 만한 경로가 없다	거부. 이 인증서를 보증하는 것이 없다
`PROVEN_ERR_EXPIRED`	경로는 있지만 그 위의 인증서가 `not_after`를 지났다	거부. 만료되었다고 사용자에게 알린다. 모든 사이트가 이렇게 실패하면 시계를 의심하라
`PROVEN_ERR_NOT_YET_VALID`	경로는 있지만 그 위의 인증서가 `not_before` 이전이다	거부. 대개 시계가 범인이다
`PROVEN_ERR_NAME_MISMATCH`	체인은 좋은데 인증서가 다른 이름을 위한 것이다	거부. 잘못 이끌렸거나 가로채인 연결이 이렇게 보인다
`PROVEN_ERR_INVALID_FORMAT`	`chain[0]`이 인증서가 아니다	거부
`PROVEN_ERR_INVALID_ARG`	체인, 옵션 또는 저장소가 없다	호출을 고친다

`result`의 **fault**(선택 - 원하지 않으면 널 포인터를 넘긴다)는 로그 한 줄이나 에러 메시지를 위한 정확한 이유다. `PROVEN_CERT_FAULT_NO_ISSUER`, `_BAD_SIGNATURE`, `_ALGORITHM`, `_NOT_A_CA`, `_PATH_LENGTH`, `_NAME_CONSTRAINT`, `_CRITICAL_EXTENSION`, `_USAGE`, `_EXPIRED`, `_NOT_YET_VALID`, `_NAME_MISMATCH`, `_TOO_DEEP`, `_MALFORMED`. `result->depth`는 받아들인 경로의 인증서 수이며 앵커를 포함한다.

여러 가지가 한꺼번에 틀렸을 때 답은 성공에 가장 가까운 것이다. 앵커까지 서명된 경로가 있지만 만료되었다면, 아무것도 찾지 못했다는 답이 아니라 만료되었다는 답을 받는다. 그것이 "인증서를 갱신하라"와 "당신은 누구인가?"의 차이다.

테스트 스위트가 컴파일하고 실행한다:

```
#include <string.h>

/*
 * 인증서는 어떤 키가 어떤 이름의 것이라는 서명된 진술이다. 여기서는 하나를 읽고, 어느 이름을 위한
 * 것인지 묻고, 믿을지 결정한다: 신뢰 앵커까지의 체인, 날짜, 그리고 이름 - 무엇이 실패했는지
 * 말해 주는 에러와 함께.
 */
```

```
/* 이 예제를 위해 만든 루트와, 그 루트가 example.test에 발급한 인증서. Ed25519라서 짧다.
 * 2026년부터 2036년까지 유효하다. */
static const char ROOT_PEM[] =
    "-----BEGIN CERTIFICATE-----\n"
    "MIIBAjCBtaADAgECAhQxTiZoH2JgH7bPWBktoxUyd0d3ZzAFBgMrZXAwFzEVMBMG\n"
    "A1UEAwwMRXhhbXBsZSBSb290MB4XDTI2MDEwMTAwMDAwMFoXDTM2MDEwMTAwMDAw\n"
    "MFowFzEVMBMGA1UEAwwMRXhhbXBsZSBSb290MCowBQYDK2VwAyEAJcpfTb5GYzM+\n"
    "bgKssEvuWEC6urqf1Znqz9I8CY2RdLujEzARMA8GA1UdEwEB/wQFMAMBAf8wBQYD\n"
    "K2VwA0EARcXC/GhPDj4gAU3zpeSAhJ033zALpz2P91wtEqbf2R4Z7drW+RSRQGp2\n"
    "Ky/CDeEyx//WjylXEoHVNvixwKrxAw==\n"
    "-----END CERTIFICATE-----\n"
    ;

static const char LEAF_PEM[] =
    "-----BEGIN CERTIFICATE-----\n"
    "MIIBPTCB8KADAgECAhRBU6wAvGvpDo28++Tq/BfZ/8DkNzAFBgMrZXAwFzEVMBMG\n"
    "A1UEAwwMRXhhbXBsZSBSb290MB4XDTI2MDEwMTAwMDAwMFoXDTM2MDEwMTAwMDAw\n"
    "MFowFzEVMBMGA1UEAwwMZXhhbXBsZS50ZXN0MCowBQYDK2VwAyEA3dq7G1ndiCbb\n"
    "IAS6O7cRIlxdACAdLSsPSRROKyglnTOjTjBMMAwGA1UdEwEB/wQCMAAwJwYDVR0R\n"
    "BCAwHoIMZXhhbXBsZS50ZXN0gg4qLmV4YW1wbGUudGVzdDATBgNVHSUEDDAKBggr\n"
    "BgEFBQcDATAFBgMrZXADQQBVDpl9ZAqf4j2R3NTG8JuKfPJuh+PD99QVdpjnqPAR\n"
    "dgg3sGIps+UTADlifcSOVlqJF6T9ya5lHpO5FwIpLpMI\n"
    "-----END CERTIFICATE-----\n"
    ;

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- PEM: 바이트를 감싼 텍스트 ------------------------------------------------
    /* 인증서는 PEM으로 다닌다: BEGIN과 END 줄 사이의 Base64. proven_pem_next는 다음 블록을 찾아
     * 디코딩한다. 안의 바이트는 DER이고, 나머지 모든 호출이 받는 것이 그것이다. */
    proven_byte_t leaf_der[1024];
    proven_size_t pos = 0, leaf_len = 0;
    proven_u8str_view_t label;
    proven_mem_view_t leaf_pem = { (const proven_byte_t *)LEAF_PEM, sizeof LEAF_PEM - 1 };
    EXAMPLE_REQUIRE(proven_pem_next(leaf_pem, &pos, &label, (proven_mem_mut_t){ leaf_der, sizeof leaf_der },
&leaf_len) == PROVEN_OK,
                    "블록이 디코딩된다");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(label, PROVEN_LIT("CERTIFICATE")), "그리고 무엇인지 말한다");
    EXAMPLE_REQUIRE(proven_pem_next(leaf_pem, &pos, &label, (proven_mem_mut_t){ leaf_der, 0 },
&(proven_size_t){ 0 }) == PROVEN_ERR_NOT_FOUND,
                    "두 번째 블록은 없다");

    // ---- 인증서 하나 읽기 ---------------------------------------------------------
    /* 파싱은 아무것도 복사하지 않는다: proven_cert_t의 모든 필드는 leaf_der를 가리키는 뷰이고,
     * 구조체를 쓰는 동안 leaf_der가 살아 있어야 한다. */
    proven_cert_t leaf;
    EXAMPLE_REQUIRE(proven_cert_parse((proven_mem_view_t){ leaf_der, leaf_len }, &leaf) == PROVEN_OK, "파싱된
다");
    EXAMPLE_REQUIRE(leaf.version == 3 && leaf.key_kind == PROVEN_CERT_KEY_ED25519 && leaf.key.size == 32 && !
leaf.is_ca,
                    "버전 3, Ed25519 키, CA 아님");
    EXAMPLE_REQUIRE(leaf.not_before < 1811808000 && leaf.not_after > 1811808000, "2027년 6월에 유효");

    proven_size_t at = 0, names = 0;
    proven_cert_name_kind_t kind;
    proven_mem_view_t name;
    while (proven_cert_alt_name_next(&leaf, &at, &kind, &name)) {
        if (kind == PROVEN_CERT_NAME_DNS) names++;
    }
```

```
    EXAMPLE_REQUIRE(names == 2, "subjectAltName에 DNS 이름 둘");

    /* 어느 호스트를 위한 것인가? subjectAltName만 센다. 와일드카드는 정확히 라벨 하나를 뜻한다. */
    EXAMPLE_REQUIRE(proven_cert_matches_host(&leaf, PROVEN_LIT("example.test")), "이름 그 자체");
    EXAMPLE_REQUIRE(proven_cert_matches_host(&leaf, PROVEN_LIT("www.example.test")), "와일드카드 아래 라벨 하나");
    EXAMPLE_REQUIRE(!proven_cert_matches_host(&leaf, PROVEN_LIT("a.b.example.test")), "둘은 아니다");
    EXAMPLE_REQUIRE(!proven_cert_matches_host(&leaf, PROVEN_LIT("example.org")), "다른 이름도 아니다");

    // ---- 신뢰 앵커 ------------------------------------------------------------
    /* 저장소는 이미 믿는 인증서를 담는다. 받은 것을 복사하므로 ROOT_PEM은 이 뒤에 해제해도 된다.
     * 라이브러리는 루트를 싣고 다니지 않는다: 직접 공급한다. */
    proven_cert_store_t *anchors = NULL;
    proven_size_t added = 0;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK, "저장소");
    EXAMPLE_REQUIRE(proven_cert_store_add_pem(anchors, (proven_mem_view_t){ (const proven_byte_t *)ROOT_PEM, sizeof ROOT_PEM - 1 }, &added) == PROVEN_OK &&
                    added == 1 && proven_cert_store_count(anchors) == 1, "번들에서 앵커 하나");

    // ---- 검증 ----------------------------------------------------------------
    /* chain[0]은 상대의 인증서이고, 상대가 보낸 나머지가 순서 없이 뒤따른다. 시각은 호출자가
     * 공급한다: 스스로 시계를 읽는 라이브러리는 테스트할 수 없고, 시계가 없는 곳에서는 돌 수
     * 없다. */
    proven_mem_view_t chain[1] = { { leaf_der, leaf_len } };
    proven_cert_verify_options_t options = {
        .anchors = anchors,
        .host = PROVEN_LIT("www.example.test"),
        .now = 1811808000,                      /* 2027-06-01 */
        .use = PROVEN_CERT_USE_SERVER,
    };
    proven_cert_verify_result_t result;
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_OK && result.depth == 2, "길이 2의 경로: 리프와 앵커");

    /* 틀리는 방식마다 답이 따로 있다. 에러 코드는 거친 답이고, 결과의 fault는 로그 한 줄을
     * 위한 것이다. */
    options.host = PROVEN_LIT("example.org");
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_NAME_MISMATCH &&
                    result.fault == PROVEN_CERT_FAULT_NAME_MISMATCH, "체인은 좋고 이름은 아니다");
    options.host = PROVEN_LIT("www.example.test");
    options.now = 2127427200;                   /* 2037-06-01 */
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_EXPIRED, "십 년 뒤에는 만료되었다");
    options.now = 1811808000;
    leaf_der[leaf_len - 1] ^= 1;
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_UNTRUSTED &&
                    result.fault == PROVEN_CERT_FAULT_BAD_SIGNATURE, "비트 하나가 바뀌면 서명이 더는 검증되지 않는다");
    leaf_der[leaf_len - 1] ^= 1;

    proven_cert_store_t *empty = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &empty) == PROVEN_OK, "저장소");
    options.anchors = empty;
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_ERR_UNTRUSTED &&
                    result.fault == PROVEN_CERT_FAULT_NO_ISSUER, "앵커가 없으면 아무것도 믿지 않는다");

    // ---- 핀 ------------------------------------------------------------------
    /* 핀은 인증서 공개 키(SubjectPublicKeyInfo)의 SHA-256이다. 어떤 키를 기대하는지 아는
     * 프로그램은 체인에 더해, 또는 체인 대신 이것을 비교한다. */
    proven_byte_t pin[32], again[32];
```

```
    EXAMPLE_REQUIRE(proven_cert_key_sha256(&leaf, pin) == PROVEN_OK, "핀");
    proven_sha256(leaf.public_key_info, again);
    EXAMPLE_REQUIRE(proven_mem_equal_ct((proven_mem_view_t){ pin, 32 }, (proven_mem_view_t){ again, 32 }), "키
인코딩의 해시다");

    // ---- 시스템의 루트 ------------------------------------------------------------
    /* 공개 인터넷과 대화할 때 앵커는 운영체제의 것이다. 기계에 그것이 있는지는 기계의 사정이므로
     * 이 예제는 두 답을 모두 받아들인다. */
    proven_cert_store_t *system_roots = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &system_roots) == PROVEN_OK, "저장소");
    proven_err_t sys = proven_cert_store_add_system(system_roots, &added);
    EXAMPLE_REQUIRE(sys == PROVEN_OK || sys == PROVEN_ERR_NOT_FOUND, "시스템 저장소를 읽었거나, 없다");
    EXAMPLE_REQUIRE((sys == PROVEN_OK) == (proven_cert_store_count(system_roots) > 0), "읽었다면 비어 있지 않
다");

    /* DER 인증서 하나도 더할 수 있다. */
    proven_byte_t root_der[1024];
    proven_size_t root_len = 0;
    pos = 0;
    EXAMPLE_REQUIRE(proven_pem_next((proven_mem_view_t){ (const proven_byte_t *)ROOT_PEM, sizeof ROOT_PEM -
1 }, &pos, &label,
                                    (proven_mem_mut_t){ root_der, sizeof root_der }, &root_len) == PROVEN_OK,
"블록이 디코딩된다");
    EXAMPLE_REQUIRE(proven_cert_store_add_der(empty, (proven_mem_view_t){ root_der, root_len }) == PROVEN_OK,
"DER로 된 루트");
    EXAMPLE_REQUIRE(proven_cert_verify(chain, 1, &options, &result) == PROVEN_OK, "이제 비어 있던 저장소가 리
프를 믿는다");

    proven_cert_store_destroy(system_roots);
    proven_cert_store_destroy(empty);
    proven_cert_store_destroy(anchors);
    return EXAMPLE_OK();
}
```

7. 피닝

```
proven_err_t proven_cert_key_sha256(const proven_cert_t *cert, proven_byte_t out[32]);
```

`proven_cert_key_sha256`은 인증서 공개 키의 SHA-256을, SubjectPublicKeyInfo 인코딩에 대해 준다. 그것이 **핀**이다. 서버가 어떤 키를 갖고 있는지 미리 아는 프로그램은 이 값을 비교해, 어떤 기관이 뭐라고 하든 나머지를 모두 거부할 수 있다.

인증서가 아니라 키를 핀으로 삼아라. 인증서는 몇 달마다 다시 발급되고 그때마다 해시가 바뀐다. 그 밑의 키는 그대로 둘 수 있다. `proven_mem_equal_ct`로 비교하고, 핀을 적어도 둘 - 쓰고 있는 키와 오프라인에 둔 예비 키 - 실어 보내라. 핀으로 삼은 유일한 키를 잃은 프로그램은 업데이트 없이는 고칠 수 없다.

핀은 검증에 **더해서**(체인도 좋아야 하고 키도 기대한 것이어야 한다) 쓸 수도 있고 검증 **대신**(상대의 인증서를 파싱하고, 핀을 비교하고, 체인은 만들지 않는다) 쓸 수도 있다. 뒤쪽은 직접 운영하는 자체 서명 서버에 맞다. 그때는 인증서의 날짜와 이름을 전혀 검사하지 않으므로 핀이 보안의 전부다.

8. 무엇을 검사하고, 무엇을 받아들이고, 무엇이 없는가

경로 위의 모든 인증서에 대해 검사하는 것:

- 그 위에 놓인 발급자의 서명.
- 여러분이 공급한 시각에 대한 유효 기간. 앵커도 포함한다.
- 발급자가 CA라는 것: `basicConstraints`가 그렇게 말하고, `keyUsage`가 있다면 인증서 서명을 허용한다. 확장이 전혀 없는(버전 1) 앵커는 여러분이 저장소에 넣었으므로 CA로 받아들인다. CA라고 말하지 않는 버전 3 인증서는 받아들이지 않는다.
- 발급자가 자기 아래에 허용하는 경로 길이.

- 발급자가 거는 이름 제약, DNS 이름과 IP 주소에 대해 - 이 라이브러리가 주장하는 이름은 그것뿐이다. 다른 이름 형태에 대한 제약은 적용하지 않는다.
- 확장 키 용도, 있다면: 리프와 모든 중간 인증서가 여러분이 요청한 용도를 허용해야 한다.
- 읽지 못한 채 남은 critical 확장이 없다는 것.

받아들이는 알고리즘. 서명: 2048~8192비트 키에 대한 SHA-256, SHA-384 또는 SHA-512의 RSA PKCS #1 v1.5와 RSA-PSS. 같은 세 해시와 P-256, P-384의 ECDSA. Ed25519. SHA-1이나 MD5 서명, 2048비트 미만의 RSA 키, 그 밖의 곡선은 fault `PROVEN_CERT_FAULT_ALGORITHM` 또는 `_BAD_SIGNATURE`로 거부한다. 앵커 자신의 서명은 검사하지 않는다 - 앵커는 저장소에 있어서 믿는 것이다 - 그래서 SHA-1로 서명된 오래된 루트도 여전히 체인의 닻이 된다.

한계. 경로는 최대 `PROVEN_CERT_MAX_DEPTH`(10)개의 인증서다. 리프 외에 최대 16개의 인증서를 고려하고, 서명 검사 64번 뒤에는 탐색을 멈춘다. 상대가 뒤엉킨 묶음을 보내 검증을 비싸게 만들 수 없다.

여기에 없으며, 감안해서 계획해야 하는 것:

- **폐기.** CRL도, OCSP도, 스테이플된 응답의 검사도 없다. 유효했다가 그 뒤 폐기된 인증서는 검증된다. 수명이 짧은 인증서가 실질적인 방어다.
- **Certificate Transparency.** 인증서가 공개 로그에 기록되었는지 검사하지 않는다.
- **인증서 정책.** 정책 확장은 그것을 담은 인증서가 거부되지 않도록 인식하되, 강제하지 않는다.
- **국제화된 이름.** ASCII로 비교한다. `xn--` 꼴로 줘라.
- **개인 키.** 여기 있는 어떤 것도 개인 키를 읽거나 갖고 있지 않다.

어떻게 시험했는가. 다른 구현이 만든 인증서 체인으로, 위에 적은 수락과 거부 하나하나에 대해. 공개 서버에서 저장한 체인을 시스템의 루트에 대해. 그리고 그 밑의 서명 알고리즘은 Project Wycheproof로 - 구현이 깨진다고 알려진 방식으로 구현을 깨도록 만든 입력 모음이다. `TEST.md`의 카탈로그가 등록된 테스트 각각이 무엇을 덮는지 말한다.

14장: TLS

5부 — 운영체제와 대화하기. 선행 조건: 인증서와 신뢰 앵커를 다루는 13장. HTTPS는 11장, wss는 12장. 이 장을 마치면 HTTPS로 서비스하고 가져오고, `wss` 연결을 열고, 여러분 자신의 어떤 연결이든 TLS로 감싸고, 소켓이 없는 곳에서는 프로토콜을 직접 몰 수 있다 - 그리고 프로그램이 상대에 대해 무엇을 믿는지, 왜 믿는지 말할 수 있다.

이 장은 `tls.h`를 다룬다. 그 안의 엔진은 운영체제를 부르지 않으며 프리스탠딩(freestanding) 빌드에서 쓸 수 있다. 전송 래퍼와 소켓에 닿는 모든 것은 호스티드(hosted) 전용이고 `PROVEN_NO_NET`으로 빠진다.

먼저 읽을 것. 이것은 외부 감사를 받지 않은 새 TLS 구현이다. 프로토콜의 공표된 예시 핸드셰이크와, 다른 구현 둘과 양쪽 역할로, 그리고 적대적 입력 모음과 대조해 시험했다(정확히 무엇을 했고 무엇이 불가능했는지는 9절). 그것은 증거이지 증명이 아니다. 능력 있는 공격자를 상대로 사용자가 TLS에 의존하는 프로그램이라면 감사받은 것에서 TLS를 끝내고, 이 라이브러리는 그 뒤에서 써야 한다.

목차

1. TLS가 하는 일, 한 쪽으로
2. 설정
3. HTTPS와 wss: 두 줄
4. 어떤 연결이든: 전송 래퍼
5. 엔진
6. 재개
7. 클라이언트 인증서와 핀
8. 실패할 때
9. 무엇을 협상하고, 무엇을 붙들고, 무엇이 없는가
10. 레거시 묶음: 설정 하나하나가 무엇에 노출시키는가

1. TLS가 하는 일, 한 쪽으로

TCP 연결은 파이프 하나를 준다. 두 끝 사이의 누구나 - 카페의 무선 공유기, 인터넷 사업자, 라우터를 장악한 누군가 - 그 안을 지나는 것을 읽고 바꿀 수 있으며, 어느 끝도 그것을 알아챌 수 없다. TLS(Transport Layer Security. 이 라이브러리는 버전 1.3(RFC 8446)과, 줄여 낸 버전 1.2(RFC 5246)를 말한다 - 어느 부분인지는 9절에 있다. 더 오래된 부분은 설정이 이름을 들어 요구할 때에만 있다, 10절)는 그 파이프 위에 세 가지 성질을 얹는다.

성질	뜻	무엇으로 만드는가
인증	반대쪽 끝이 닿으려던 그 상대다	13장대로 검사한 상대의 인증서와, 인증서의 키를 갖고 있음을 증명하는 서명
기밀성	중간의 누구도 데이터를 읽을 수 없다	중간에서 지켜보고도 계산할 수 없는 키 교환으로 합의한 키
무결성	중간의 누구도 데이터를 들키지 않고 바꾸거나 버리거나 순서를 바꿀 수 없다	모든 레코드에는 키를 가진 쪽만 만들 수 있는 태그가 붙는다. 깨끗한 끝도 그 자체가 메시지다

두 단계로 일어난다. **핸드셰이크**는 처음의 메시지 몇 개다. 양쪽이 알고리즘에 합의하고, 키 몫을 교환하고, 서버가(선택적으로 클라이언트도) 인증서를 내밀어 그 키의 주인임을 증명하고, 둘이 같은 비밀 키를 유도한다. 그 뒤로는 **레코드**가 암호화되고 태그가 붙은 채 여러분의 데이터를 나른다.

한 줄을 쓰기 전에 알아 둘 귀결 셋:

- **이름이 절반이다.** 검증하지 않은 상대에게 하는 암호화는 대답한 아무에게나 하는 암호화다. 클라이언트는 어느 이름에 닿으려 했는지 알아야 하고, 인증서는 그 이름을 위한 것이어야 한다.
- **깨끗한 닫기가 중요하다.** 연결이 그냥 멈추면 TLS는 "상대가 끝냈다"와 "누가 나머지를 숨기려고 선을 끊었다"를 구별할 수 없다. 그래서 닫기는 메시지이고, 그것 없이 끝난 연결은 데이터의 끝이 아니라 에러로 보고된다.

- **비용이 든다.** 핸드셰이크는 공개 키 연산이다. 이 구현의 숫자는 9절에 있고, 작지 않다.

2. 설정

```
proven_err_t proven_tls_config_create(const proven_tls_options_t *options, proven_tls_config_t **out);
void proven_tls_config_destroy(proven_tls_config_t *config);
```

`proven_tls_config_t`는 모든 연결에 똑같은 것을 담는다. 누구를 믿을지, 여러분이 누구인지, 어떤 애플리케이션 프로토콜을 말하는지. **한 번 만들면 바뀌지 않으므로** 설정 하나를 모든 연결과 모든 스레드가 함께 쓴다. `proven_tls_config_create`는 받은 것을 복사하고 - 앵커 저장소만 예외로, 설정보다 오래 살아야 한다 - `proven_tls_config_destroy`는 해제하면서 개인 키를 지운다.

`proven_tls_options_t`를 0으로 초기화하고 `alloc`을 채운 뒤 해당하는 것을 채운다.

필드	누구에게	뜻
`alloc`	양쪽	필수. 이 설정으로 만든 연결도 이 할당자(allocator)로 할당한다(예제에서는 힙(heap) 할당자)
`anchors`	검증하는 쪽	신뢰 앵커(13장). 클라이언트에는 필요하다. 서버는 클라이언트 인증서를 검사할 때만 필요하다
`verify`	검증하는 쪽	`PROVEN_TLS_VERIFY_CHAIN`(기본): 앵커까지의 체인, 지금 유효, 기대한 이름. `PROVEN_TLS_VERIFY_PIN_ONLY`: 7절
`pins`, `pin_count`	검증하는 쪽	공개 키 핀(7절)
`certificate_pem`, `private_key_pem`	서버. 인증서를 가진 클라이언트	이쪽의 인증서들(자기 것이 먼저)과 키. 키는 P-256, Ed25519, 또는 2048~4096비트의 RSA이고, PKCS #8 블록(라벨 `PRIVATE KEY`)이나 `EC PRIVATE KEY` 블록이나 `RSA PRIVATE KEY` 블록, 암호화되지 않은 것
`client_auth`	서버	클라이언트에게 인증서를 요구할지(7절)
`alpn`, `alpn_count`	양쪽	애플리케이션 프로토콜 이름, 선호하는 순서(`http/1.1`). 서버가 클라이언트가 내민 것 가운데 자기 첫 선택을 고른다. 양쪽에 목록이 있고 겹치는 것이 없으면 핸드셰이크는 실패한다
`random`, `now`	양쪽	예측할 수 없는 바이트와 시각의 출처. 호스티드 시스템에서는 널로 둔다: 운영체제의 원천과 벽시계를 쓴다. 프리스탠딩 빌드는 둘 다 공급해야 한다
`no_resumption`, `ticket_lifetime_s`	양쪽, 서버	6절
`early_data`	양쪽: HTTP/3, 그리고 TCP 위의 HTTP/1.1과 HTTP/2	6절 "조기 데이터"

keep_peer_certificate	검증하는 쪽	핸드셰이크 뒤에도 상대의 인증서를 보관한다 (proven_tls_peer_certificate용)
max_handshake_bytes	양쪽	받아들이는 가장 큰 핸드셰이크 메시지. 기본 65,536
min_version, max_version	양쪽	이쪽이 동의하는 가장 오래된, 가장 새로운 프로토콜 버전: PROVEN_TLS_VERSION_1_2 또는 PROVEN_TLS_VERSION_1_3. 0이 기본값이다 - 최소는 1.2, 최대는 1.3. **TLS 1.2를 거부하려면 min_version = PROVEN_TLS_VERSION_1_3으로 둔다**(9절). PROVEN_TLS_VERSION_1_1과 PROVEN_TLS_VERSION_1_0은 레거시다(10절)
legacy	양쪽	PROVEN_TLS_LEGACY_* 비트: 여기서 이름을 들지 않으면 빠져 있는 TLS의 부분들. 기본값 0은 그 어느 것도 아니다. **하나라도 켜기 전에 10절을 읽을 것**

anchors, verify, pins는 어느 역할에서든 상대를 어떻게 검증하는지를 말한다. 아무것도 검증하지 않는 설정은 없다. 체인을 검사할 클라이언트에 앵커가 없으면 설정을 만들 때 거부된다. 공인 기관을 건너뛰고 싶은 테스트는 자기 루트를 만들거나 키를 핀으로 삼는다.

옵션에서 틀릴 수 있는 것은 모두 여기서 에러가 된다 - 남의 기계에서 새벽 세 시에 일어나는 첫 핸드셰이크가 아니라.

반환	언제
PROVEN_ERR_INVALID_ARG	할당자가 없다. 이쪽이 체인을 검증하는데 앵커가 없다. 핀 없는 PIN_ONLY. 키 없는 인증서 또는 인증서 없는 키. 인증서가 8개를 넘는다. 빈 ALPN 이름. 존재하지 않는 버전, 또는 최대보다 큰 최소. PROVEN_TLS_LEGACY_CBC 없이 TLS 1.2 미만인 최소. 존재하지 않는 legacy 비트
PROVEN_ERR_INVALID_FORMAT	파싱되지 않는 PEM, 또는 인증서나 키가 들어 있지 않은 PEM. 2048~4096비트 밖이거나, 부분들이 서로 맞지 않거나, 서명하지 못하는 RSA 키
PROVEN_ERR_UNSUPPORTED	다른 종류의 키 - 예컨대 P-384 - 또는 암호화된 키 파일
PROVEN_ERR_INVALID_STATE	키가 인증서의 키가 아니다
PROVEN_ERR_NOMEM	할당자가 거부했다

어느 키를 고를까. 세 종류 모두 연결의 어느 쪽에서든 쓸 수 있다. 비용은 같지 않다: RSA 키로는 이쪽이 서명하는 전체 핸드셰이크마다 P-256보다 오래 걸리고 - 2048비트로 약 다섯 배, 4096비트로 서른다섯 배 - Ed25519는 P-256보다 조금 싸다(수치는 9절에 있다). 서버에서는 서명이 핸드셰이크의 가장 큰 부분이다. 이미 가진 인증서가 요구하는 곳에서는 RSA를, 새로 만드는 것이라면 P-256이나 Ed25519를 쓴다. RSA 키는 설정을 만들 때 시험된다 - 서명 하나를 만들고 검사한다 - 그래서 손상된 키 파일은 첫 연결이 아니라 거기서 거절된다.

기관 없는 신원. 라이브러리는 신원을 읽을 뿐 만들지 않는다. 자체 서명 인증서와 그 키는 도구로 만든다 - OpenSSL로, 여러분에게 닿을 이름들에 대해:

```
openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \
    -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem
```

cert.pem은 certificate_pem에, key.pem(PKCS #8)은 private_key_pem에 들어간다. 아래 두 예제처럼 파일에서 읽는다. -newkey ec -pkeyopt ec_paramgen_curve:P-256이나 -newkey rsa:2048은 다른 두 종류를 만든다. v0.78.0 전에는 라이브러리가 이런 쌍을 스스로 만들었다(proven_tls_self_signed). 프로그램이 시작할 때마다 스스로 지어낸 신원은 배포할 만한 것이 거의 아니므로, 이 함수는 공개 API에서 빠졌다.

자체 서명 인증서는 아무도 보증하지 않는다. 상대는 그것을 이미 갖고 있어야만 믿는다. 인증서를 상대의 앵커 저장소에 넣거나, 상대에게 키의 핀과 PROVEN_TLS_VERIFY_PIN_ONLY를 준다(7절). 직접 운영하는 기계 사이에서, 그리고 개발과 테스트에서 옳은 방식이다. 기관만 믿는 브라우저를 쓰는 일반 사용자가 닿게 하는 방법은 아니다. 키는 여러분이 안전하게 보관하고 지워야 한다.

3. HTTPS와 wss: 두 줄

HTTP에서는 설정 말고 아무것도 바뀌지 않는다.

클라이언트는 이 라이브러리가 주는 랩 함수와, 그 컨텍스트로 쓸 설정을 받으면 https URL을 쓸 수 있다.

```
proven_err_t proven_tls_http_wrap(void *config, proven_transport_t plain, proven_u8str_view_t host,
                                  proven_net_deadline_t until, proven_transport_t *out);
```

11장의 proven_http_client_config_t에서 tls_wrap = proven_tls_http_wrap으로, tls_ctx를 여러분의 proven_tls_config_t *로 둔다. 그때부터 https URL 요청은 연결하고 - 프록시가 있으면 프록시 터널을 지나 - URL의 호스트를 상대로 핸드셰이크를 하고 진행한다. proven_ws_conn_connect에 준 wss URL도 같다. 공개 인터넷이라면 설정의 앵커는 시스템의 것이다(proven_cert_store_add_system). 검증할 수 없는 서버는 요청에서 PROVEN_ERR_UNTRUSTED, PROVEN_ERR_EXPIRED, PROVEN_ERR_NOT_YET_VALID 또는 PROVEN_ERR_NAME_MISMATCH가 되고, **요청의 어떤 부분도 보내지지 않았다**.

서버는 proven_http_server_config_t의 tls에 인증서를 가진 설정을 두면 TLS로 말한다. 그러면 받아들이는 모든 연결이 TLS다. 그 포트에 평문 HTTP를 보낸 클라이언트는 끊긴다. 핸들러는 바뀌지 않는다. 핸들러가 돌 때쯤 요청은 평문이고, 핸들러가 쓰는 것은 나가는 길에 암호화된다. 그런 핸들러에서 수락한 WebSocket은 wss다.

핸드셰이크는 클라이언트의 바이트가 도착하는 대로 서버 자신의 루프가 몬다. 연결 하나를 기다리는 일이 없으므로, 연결만 하고 아무 말도 하지 않는 클라이언트는 head_timeout_ms까지 자리 하나를 차지할 뿐 - 새 연결의 핸드셰이크는 이 시간을 첫 요청의 헤드와 함께 쓴다 - 누구도 붙잡지 않는다.

http와 https를 함께 서비스하려면 서버를 둘 돌린다. 서버 하나는 둘 중 하나다.

컴파일되고, 테스트 스위트가 스스로 만든 인증서와 키를 주어 실행한다 (tests/test_docs_tls_example_https.c):

```
#include <stdio.h>
#include <string.h>

/*
 * HTTPS, 양쪽 끝을 한 프로그램에: 11장의 HTTP 서버에 TLS 설정을, 11장의 HTTP 클라이언트에 TLS
 * 랩을 붙인 것. 그 밖에는 어느 쪽도 달라지지 않는다.
 *
 * 서버는 루프백 인터페이스에서 다른 스레드로 돌아가므로 예제에 네트워크가 필요 없다.
 */

/* 이 파일에는 인증서가 없다. 서버의 신원은 명령줄에서 이름을 준 PEM 파일 두 개에서 읽는다:
 * 127.0.0.1을 위한 인증서와 그 키. 아무것도 주지 않고 돌리면 OpenSSL로 자체 서명 쌍을 만드는
 * 법을 알려 준다. 클라이언트에는 그 인증서를 믿는 단 하나의 것으로 준다. */

/* 파일을 주지 않았을 때 실행할 것. */
static const char *const HOW_TO_MAKE =
    "  openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \\\n"
    "      -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem\n";

typedef struct {
```

```
    proven_http_server_t *server;
} ex_app_t;

static void handle(void *ctx, proven_http_exchange_t *x) {
    ex_app_t *app = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    /* 핸들러는 TLS가 없을 때와 같다: 핸들러가 돌 때쯤 요청은 평문이다. */
    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/quit"))) {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        proven_http_server_stop(app->server);
        return;
    }
    (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("hello over
TLS\n")));
}

static void serve(void *arg) {
    (void)proven_http_server_run(((ex_app_t *)arg)->server);
}

int main(int argc, char **argv) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- 신원 ------------------------------------------------------------------
    /* 이 예제가 서비스하는 주소를 위한 인증서와 그 키를 파일에서. 실제 서버라면 기관이 낸 인증서를
     * 받는다. 직접 운영하는 기계 한 쌍이라면 자체 서명 인증서나 핀을 쓸 수 있다(14장 7절). */
    if (argc < 3) {
        printf("usage: %s CERTIFICATE.pem KEY.pem\nA self-signed pair for 127.0.0.1:\n%s", argv[0],
HOW_TO_MAKE);
        return 0;
    }
    proven_result_mem_mut_t cert = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[1]), 64 *
1024);
    proven_result_mem_mut_t key = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[2]), 16 *
1024);
    EXAMPLE_REQUIRE(cert.err == PROVEN_OK && key.err == PROVEN_OK, "인증서와 키를 읽는다");
    if (cert.err != PROVEN_OK || key.err != PROVEN_OK) return 1;
    const proven_byte_t *cert_pem = cert.value.ptr, *key_pem = key.value.ptr;
    const proven_size_t cert_len = cert.value.size, key_len = key.value.size;

    // ---- 서버 ------------------------------------------------------------------
    /* 필드 하나가 서버를 TLS로 말하게 한다: 설정의 `tls`. TLS 설정은 서버가 내미는 인증서와,
     * 그것이 서버의 것임을 증명하는 키를 담는다. */
    proven_tls_options_t server_options = {
        .alloc = heap,
        .certificate_pem = { cert_pem, cert_len },
        .private_key_pem = { key_pem, key_len },
    };
    proven_tls_config_t *server_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&server_options, &server_tls) == PROVEN_OK, "서버의 TLS 설정");

    ex_app_t app = { 0 };
    proven_http_server_config_t server_config = { .alloc = heap, .handler = handle, .handler_ctx = &app, .tls
= server_tls };
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &app.server) == PROVEN_OK &&
                    proven_http_server_listen(app.server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0),
&at) == PROVEN_OK, "빈 포트의 HTTPS 서버");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
```

```
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &app) == PROVEN_OK, "루프가 다른 스레드에서 돈다");

    // ---- 클라이언트 ----------------------------------------------------------
    /* 클라이언트의 TLS 설정은 누구를 믿을지 말한다: 여기서는 방금 만든 인증서. 공개 인터넷이라면
     * 대신 proven_cert_store_add_system이다. */
    proven_cert_store_t *anchors = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK &&
                    proven_cert_store_add_pem(anchors, (proven_mem_view_t){ cert_pem, cert_len }, NULL) == PROVEN_OK,
                    "앵커");
    proven_tls_options_t client_options = { .alloc = heap, .anchors = anchors };
    proven_tls_config_t *client_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&client_options, &client_tls) == PROVEN_OK, "클라이언트의 TLS 설정");

    /* 필드 둘이 클라이언트를 HTTPS로 말하게 한다: 이 라이브러리가 주는 랩 함수와, 그 컨텍스트로
     * 쓰이는 TLS 설정. */
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 1, .tls_wrap = proven_tls_http_wrap, .tls_ctx = client_tls };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "HTTP 클라이언트");

    char url_text[64];
    int url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/", (unsigned)at.port);
    proven_u8str_view_t url = { (const proven_byte_t *)url_text, (proven_size_t)url_len };

    proven_http_client_response_t response;
    proven_u8str_t body = { 0 };
    EXAMPLE_REQUIRE(proven_http_client_get(client, url, &response) == PROVEN_OK && response.status == 200, "https로 GET: 200");
    EXAMPLE_REQUIRE(proven_http_client_read_all(&response, heap, &body, 4096) == PROVEN_OK &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("hello over TLS\n")), "그리고 본문, 선 위를 암호화된 채 건너왔다");
    proven_u8str_destroy(heap, &body);
    proven_http_client_finish(&response);

    // ---- 믿을 것이 없는 클라이언트 -------------------------------------------
    /* 아무것도 검증하지 않는 설정은 없다. 체인을 검사할 클라이언트에는 앵커를 줘야 하고, 빈
     * 저장소를 받은 클라이언트는 어떤 서버도 믿지 않는다. */
    proven_tls_options_t stranger_options = { .alloc = heap, .anchors = NULL };
    proven_tls_config_t *no_anchors = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&stranger_options, &no_anchors) == PROVEN_ERR_INVALID_ARG, "앵커가 아예 없으면 설정을 만들 때 거부된다");

    proven_cert_store_t *empty = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &empty) == PROVEN_OK, "빈 저장소");
    stranger_options.anchors = empty;
    EXAMPLE_REQUIRE(proven_tls_config_create(&stranger_options, &no_anchors) == PROVEN_OK, "아무도 믿지 않는 설정");
    proven_http_client_config_t wary_config = { .alloc = heap, .tls_wrap = proven_tls_http_wrap, .tls_ctx = no_anchors };
    proven_http_client_t *wary = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&wary_config, &wary) == PROVEN_OK, "HTTP 클라이언트");
    EXAMPLE_REQUIRE(proven_http_client_get(wary, url, &response) == PROVEN_ERR_UNTRUSTED, "같은 서버가 PROVEN_ERR_UNTRUSTED이고, 요청은 보내지지 않았다");
    proven_http_client_finish(&response);
    proven_http_client_destroy(wary);
```

```
    // ---- 끝 ------------------------------------------------------------------------
    url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/quit", (unsigned)at.port);
    EXAMPLE_REQUIRE(proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url_text,
(proven_size_t)url_len }, &response) == PROVEN_OK, "서버에 멈추라고 요청한다");
    proven_http_client_finish(&response);
    proven_job_group_wait(threads, &running);

    proven_http_client_destroy(client);
    proven_http_server_destroy(app.server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    proven_tls_config_destroy(client_tls);
    proven_tls_config_destroy(no_anchors);
    proven_tls_config_destroy(server_tls);
    proven_cert_store_destroy(anchors);
    proven_cert_store_destroy(empty);
    proven_mem_wipe(key.value);
    heap.free_fn(heap.ctx, key.value.ptr);
    heap.free_fn(heap.ctx, cert.value.ptr);
    return EXAMPLE_OK();
}
```

4. 어떤 연결이든: 전송 래퍼

```
proven_err_t proven_tls_transport_client(proven_transport_t plain, const proven_tls_config_t *config,
                                         proven_u8str_view_t server_name, proven_tls_session_t *session,
                                         proven_net_deadline_t until, proven_transport_t *out);
proven_err_t proven_tls_transport_server(proven_transport_t plain, const proven_tls_config_t *config,
                                         proven_net_deadline_t until, proven_transport_t *out);
proven_tls_conn_t *proven_tls_transport_conn(proven_transport_t tls);
```

`proven_transport_t`(9장)는 인터페이스로서의 연결이다: 읽기, 쓰기, 종료, 닫기. `proven_tls_transport_client`와 `proven_tls_transport_server`는 하나를 받아 `until` 안에 그 위에서 핸드셰이크를 하고 다른 하나를 돌려준다 - 3절의 HTTP 랩이 이것으로 만들어졌고, 여러분 자신의 프로토콜에 쓰는 것도 이것이다.

- **성공하면 새 전송이 옛 전송을 소유(owned)한다.** 새 전송을 닫으면 TLS 작별을 고하고 밑의 연결을 닫는다. `plain`을 다시 쓰지 마라.
- **실패하면 `plain`은 손대지 않은 채 여전히 여러분의 것**이고, 닫는 것도 여러분이다.
- `server_name`은 클라이언트가 닿으려던 이름이다. 서버에 보내지고, 서버의 인증서와 대조된다.
- 읽기가 `PROVEN_ERR_EOF`를 돌려주면 상대가 제대로 닫은 것이다. 다른 식으로 끝난 연결은 `PROVEN_ERR_RESET`이다. 받은 것을 완전하다고 여기지 마라.
- `proven_tls_transport_conn`은 안의 엔진을 준다 - 5절의 질문들(어느 스위트, 어느 프로토콜, 누구의 키)을 위해서. 이 둘이 만든 것이 아닌 전송에는 널을 돌려준다.

읽기와 쓰기는 모든 전송이 그렇듯 기한을 받는다. 읽기는 써야 할 때가 있고(프로토콜이 가끔 상대에게 메시지를 빚진다) 쓰기는 읽어야 할 일이 없다.

5. 엔진

래퍼 아래에는 I/O가 없는 상태 기계가 있다. 도착한 바이트를 주면, 보낼 바이트를 준다. 전송이 여러분의 것일 때 - 이벤트 루프, 직렬 선, 테스트 안의 메모리 버퍼 - 또는 운영체제가 아예 없을 때 직접 쓴다.

```
proven_err_t proven_tls_client_create(const proven_tls_config_t *config, proven_u8str_view_t server_name,
                                      proven_tls_session_t *session, proven_tls_conn_t **out);
proven_err_t proven_tls_server_create(const proven_tls_config_t *config, proven_tls_conn_t **out);
void proven_tls_conn_destroy(proven_tls_conn_t *conn);

proven_err_t proven_tls_feed(proven_tls_conn_t *conn, proven_mem_view_t in, proven_size_t *consumed);
proven_mem_view_t proven_tls_pending_output(proven_tls_conn_t *conn);
```

```
void proven_tls_output_sent(proven_tls_conn_t *conn, proven_size_t n);

bool proven_tls_is_established(const proven_tls_conn_t *conn);
proven_result_size_t proven_tls_read(proven_tls_conn_t *conn, proven_mem_mut_t dest);
proven_result_size_t proven_tls_write(proven_tls_conn_t *conn, proven_mem_view_t src);
proven_err_t proven_tls_close(proven_tls_conn_t *conn);
```

쓰는 법 전부가 루프 하나다.

1. **`proven_tls_pending_output`이 보여 주는 것은 무엇이든 보내고**, 얼마가 나갔는지 `proven_tls_output_sent`로 알린다. 클라이언트는 만들어지는 순간 출력이 있다.
2. **도착한 것은 무엇이든 `proven_tls_feed`에 준다.** 레코드 하나 전체일 필요도, 하나뿐일 필요도 없다. 미완성 레코드는 안에 보관된다.
3. **feed를 할 때마다 `proven_tls_read`를 `PROVEN_ERR_NEED_MORE`라고 할 때까지 부른다.**
4. `proven_tls_is_established`가 되면 `proven_tls_write`가 여러분의 데이터를 보류 출력 안으로 암호화한다. 1로 간다.

이 루프에서 틀리기 쉬운 규칙 둘:

- **`proven_tls_feed`는 준 것보다 적게 소비할 수 있다.** 여러분이 아직 읽지 않은 복호화된 애플리케이션 데이터가 있으면 멈추고, 가져간 양을 `*consumed`에 적는다. 읽은 다음 나머지를 준다. 소비되지 않은 부분을 버리는 호출자는 데이터를 잃는다.
- **`proven_tls_pending_output`의 뷰(view)는 그 연결에 대한 다음 호출까지만 유효하다.** 거기서 바로 보내거나 복사하라. feed를 넘겨 들고 있지 마라.

`proven_tls_read`는 상대가 제대로 닫았고 그 전의 것을 모두 읽었으면 `PROVEN_ERR_EOF`를 돌려준다. `proven_tls_close`는 이쪽의 작별을 보류 출력에 넣는다. 그 뒤로는 더 쓸 수 없고, 읽기는 상대도 닫을 때까지 계속된다. `proven_tls_write`는 보내지 않은 출력이 64 KiB를 넘게 쌓여 있지 않은 한 전부 받는다. 넘으면 들어가는 만큼만 - 어쩌면 0 - 받고, 여러분은 먼저 얼마간 보낸다.

연결은 한 번에 한 스레드가 쓴다. 설정과 그 밑의 앵커 저장소는 몇이든 함께 쓸 수 있다.

수립된 뒤, 무엇에 합의했는가:

```
proven_u16 proven_tls_version(const proven_tls_conn_t *conn);
proven_u16 proven_tls_cipher_suite(const proven_tls_conn_t *conn);
proven_u8str_view_t proven_tls_alpn(const proven_tls_conn_t *conn);
bool proven_tls_resumed(const proven_tls_conn_t *conn);
bool proven_tls_peer_key_sha256(const proven_tls_conn_t *conn, proven_byte_t out[32]);
proven_mem_view_t proven_tls_peer_certificate(const proven_tls_conn_t *conn);
proven_u8str_view_t proven_tls_server_name(const proven_tls_conn_t *conn);
proven_err_t proven_tls_key_update(proven_tls_conn_t *conn);
```

- `proven_tls_version`은 `PROVEN_TLS_VERSION_1_3` 또는 `PROVEN_TLS_VERSION_1_2`(또는, 허락한 설정에서는 두 레거시 버전 중 하나)이고, 상대의 hello가 그것을 정하기 전에는 0이다. 무언가를 TLS 1.2 위로 실어서는 안 되는 프로그램은 여기서 확인하거나 - 더 낫게는 - `min_version`을 두어 거기까지 가지 않는다.
- `proven_tls_cipher_suite`는 스위트의 번호(9절), `proven_tls_alpn`은 합의한 애플리케이션 프로토콜이며 없으면 비어 있다.
- `proven_tls_peer_key_sha256`은 상대 공개 키의 해시 - 핀이 담는 값 - 를 주고, 상대가 인증서를 내밀지 않았으면 `false`를 돌려준다. 인증서 자체는 설정이 `keep_peer_certificate`를 켜지 않은 한 핸드셰이크 뒤에 보관하지 않는다. 켰다면 `proven_tls_peer_certificate`가 `proven_cert_parse`에 넣을 수 있게 돌려준다.
- `proven_tls_server_name`은 서버 역할에서, 핸드셰이크 동안에만, 클라이언트가 요청한 이름이다.
- `proven_tls_key_update`는 이쪽의 송신 키를 다음 키로 바꾼다. 엔진은 키가 너무 많은 것을 보호하기 한참 전에 스스로 이렇게 한다. 정책이 더 일찍 원할 때 부른다. TLS 1.2에는 그런 메시지가 없다: 거기서는 `PROVEN_ERR_UNSUPPORTED`를 돌려주고, 레코드를 2^31개 보낸 연결은 더 쓰기를 거부한다(`PROVEN_ERR_OVERFLOW`) - 새 연결을 연다.

컴파일되고, 테스트 스위트가 스스로 만든 인증서와 키를 주어 실행한다 (tests/test_docs_tls_example_engine.c):

```
#include <string.h>

/*
 * 네트워크가 보이지 않는 TLS 엔진: 한 프로그램 안의 클라이언트 연결과 서버 연결, 그리고 한쪽에서
 * 다른 쪽으로 바이트를 나르는 이 파일. 네트워크가 둘에게 해 주는 일은 그것이 전부다.
 *
 * 전송 래퍼 아래의 층이다. 전송이 여러분의 것일 때 직접 쓴다: 이벤트 루프, 직렬 선, 테스트.
 */

/* 이 파일에는 인증서가 없다. 서버의 신원은 명령줄에서 이름을 준 PEM 파일 두 개에서 읽는다:
 * example.test를 위한 인증서와 그 키. 아무것도 주지 않고 돌리면 OpenSSL로 자체 서명 쌍을 만드는
 * 법을 알려 준다. 아무도 보증하지 않는 인증서는 이미 그것을 가진 상대만 믿으므로, 클라이언트에는
 * 그것을 앵커로 준다. */

/* 파일을 주지 않았을 때 실행할 것. */
static const char *const HOW_TO_MAKE =
    "  openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \\\n"
    "      -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem\n";

/* 한쪽이 보내려는 것을 모두 다른 쪽에 건넨다. 받는 쪽은 읽히기를 기다리는 애플리케이션
 * 데이터가 있으면 받은 것보다 적게 가져갈 수 있다. 가져가지 않은 것은 보류 상태로 남는다. */
static proven_err_t carry(proven_tls_conn_t *from, proven_tls_conn_t *to) {
    proven_mem_view_t out = proven_tls_pending_output(from);
    proven_size_t at = 0;
    while (at < out.size) {
        proven_size_t used = 0;
        proven_err_t e = proven_tls_feed(to, (proven_mem_view_t){ out.ptr + at, out.size - at }, &used);
        if (e != PROVEN_OK) return e;
        if (used == 0) break;                /* 애플리케이션 데이터를 먼저 읽어 주기를 원한다 */
        at += used;
    }
    proven_tls_output_sent(from, at);
    return PROVEN_OK;
}

/* `text`의 바이트가 그 순서대로 `data` 어딘가에 나타나면 참. */
static bool appears(proven_mem_view_t data, const char *text) {
    proven_size_t n = strlen(text);
    for (proven_size_t i = 0; i + n <= data.size; ++i) {
        if (memcmp(data.ptr + i, text, n) == 0) return true;
    }
    return false;
}

int main(int argc, char **argv) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- 신원 ------------------------------------------------------------
    /* 실제 서버는 기관이 낸 인증서를 받는다. 직접 운영하는 기계 한 쌍이라면 자체 서명 인증서를 쓸
     * 수 있다. 어느 쪽이든 파일로 온다. 키는 서버의 비밀이다: 읽고, 쓰고, 지운다. */
    if (argc < 3) {
        printf("usage: %s CERTIFICATE.pem KEY.pem\nA self-signed pair for example.test:\n%s", argv[0],
HOW_TO_MAKE);
        return 0;
    }
    proven_result_mem_mut_t cert = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[1]), 64 *
1024);
```

```
    proven_result_mem_mut_t key = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[2]), 16 *
1024);
    EXAMPLE_REQUIRE(cert.err == PROVEN_OK && key.err == PROVEN_OK, "인증서와 키를 읽는다");
    if (cert.err != PROVEN_OK || key.err != PROVEN_OK) return 1;
    const proven_byte_t *cert_pem = cert.value.ptr, *key_pem = key.value.ptr;
    const proven_size_t cert_len = cert.value.size, key_len = key.value.size;

    // ---- 설정 둘 ------------------------------------------------------------
    /* 설정은 누구를 믿을지, 여러분이 누구인지 말한다. 한 번 만들어 모든 연결이 함께 쓴다.
     * 클라이언트는 방금 만든 인증서를 믿고, 서버는 그 인증서와 키다. */
    proven_cert_store_t *anchors = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK &&
                    proven_cert_store_add_pem(anchors, (proven_mem_view_t){ cert_pem, cert_len }, NULL) ==
PROVEN_OK,
                    "앵커");

    proven_u8str_view_t protocols[1] = { PROVEN_LIT("example/1") };
    /* 엔진은 스스로 시계를 읽지 않는다: `now`가 건네준다. 여기처럼 빼면 시스템의 시계다 -
     * 인증서는 오늘에 대해 검사된다. */
    proven_tls_options_t client_options = { .alloc = heap, .anchors = anchors, .alpn = protocols, .alpn_count
= 1 };
    proven_tls_options_t server_options = {
        .alloc = heap, .alpn = protocols, .alpn_count = 1,
        .certificate_pem = { cert_pem, cert_len },
        .private_key_pem = { key_pem, key_len },
    };
    proven_tls_config_t *client_config = NULL, *server_config = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&client_options, &client_config) == PROVEN_OK, "클라이언트 설
정");
    EXAMPLE_REQUIRE(proven_tls_config_create(&server_options, &server_config) == PROVEN_OK, "서버 설정: 키가
인증서와 맞는다");

    // ---- 핸드셰이크 ---------------------------------------------------------
    /* 클라이언트 연결은 생기는 순간 첫 메시지를 준비해 둔다. 서버 연결은 기다린다. 세션은
     * 다음번을 위한 티켓이 놓일 자리다. */
    proven_tls_session_t session = { 0 };
    proven_tls_conn_t *client = NULL, *server = NULL;
    EXAMPLE_REQUIRE(proven_tls_client_create(client_config, PROVEN_LIT("example.test"), &session, &client) ==
PROVEN_OK, "클라이언트");
    EXAMPLE_REQUIRE(proven_tls_server_create(server_config, &server) == PROVEN_OK, "서버");
    EXAMPLE_REQUIRE(proven_tls_pending_output(client).size > 0 && proven_tls_pending_output(server).size == 0,
"클라이언트가 먼저 말한다");

    /* 둘 다 끝날 때까지 바이트를 오가게 한다. 왕복 두 번이면 된다. */
    for (int round = 0; round < 10 && !(proven_tls_is_established(client) &&
proven_tls_is_established(server)); ++round) {
        EXAMPLE_REQUIRE(carry(client, server) == PROVEN_OK, "클라이언트에서 서버로");
        EXAMPLE_REQUIRE(carry(server, client) == PROVEN_OK, "서버에서 클라이언트로");
    }
    EXAMPLE_REQUIRE(proven_tls_is_established(client) && proven_tls_is_established(server), "양쪽이 수립되었
다");
    EXAMPLE_REQUIRE(proven_tls_version(client) == PROVEN_TLS_VERSION_1_3 && proven_tls_version(server) ==
PROVEN_TLS_VERSION_1_3, "양쪽이 가진 가장 새 버전인 TLS 1.3을 말했다");
    EXAMPLE_REQUIRE(proven_tls_cipher_suite(client) == proven_tls_cipher_suite(server) &&
proven_tls_cipher_suite(client) != 0, "암호 스위트에 합의했다");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_tls_alpn(client), PROVEN_LIT("example/1")), "애플리케이션 프로
토콜에도");
    EXAMPLE_REQUIRE(!proven_tls_resumed(client), "이번은 전체 핸드셰이크였다");

    // ---- 애플리케이션 데이터 -------------------------------------------------
```

```
    /* 쓰기는 보류 출력 안으로 암호화하고, 읽기는 넣어 준 것을 복호화한 결과를 준다. */
    proven_result_size_t wrote = proven_tls_write(client, proven_mem_view_from_u8(PROVEN_LIT("hello,
server")));
    EXAMPLE_REQUIRE(wrote.err == PROVEN_OK && wrote.value == 13, "열세 바이트를 받았다");
    proven_mem_view_t wire = proven_tls_pending_output(client);
    EXAMPLE_REQUIRE(wire.size > 13 && !appears(wire, "hello, server"), "선 위로 가는 것은 레코드이고, 그 텍스
트가 아니다");
    EXAMPLE_REQUIRE(carry(client, server) == PROVEN_OK, "클라이언트에서 서버로");
    proven_byte_t buf[64];
    proven_result_size_t got = proven_tls_read(server, (proven_mem_mut_t){ buf, sizeof buf });
    EXAMPLE_REQUIRE(got.err == PROVEN_OK && got.value == 13 && memcmp(buf, "hello, server", 13) == 0, "서버가
클라이언트가 쓴 것을 읽는다");
    EXAMPLE_REQUIRE(proven_tls_read(server, (proven_mem_mut_t){ buf, sizeof buf }).err ==
PROVEN_ERR_NEED_MORE, "그 뒤로는 아직 아무것도 없다");

    // ---- 닫기 ------------------------------------------------------------------
    /* 닫기도 메시지다: 끝이 끊어진 선이 아니라 정말 끝이라는 것을 상대에게 알린다. */
    EXAMPLE_REQUIRE(proven_tls_close(client) == PROVEN_OK && carry(client, server) == PROVEN_OK, "닫기가 전달
된다");
    EXAMPLE_REQUIRE(proven_tls_read(server, (proven_mem_mut_t){ buf, sizeof buf }).err == PROVEN_ERR_EOF, "서
버의 읽기가 제대로 된 끝을 보고한다");
    proven_tls_conn_destroy(client);
    proven_tls_conn_destroy(server);

    // ---- 재개 ------------------------------------------------------------------
    /* 첫 연결이 세션에 티켓을 남겼다. 같은 세션을 받은 두 번째 연결은 그것을 내밀고,
     * 핸드셰이크는 인증서와 서명 하나를 건너뛴다. */
    EXAMPLE_REQUIRE(proven_tls_client_create(client_config, PROVEN_LIT("example.test"), &session, &client) ==
PROVEN_OK &&
                    proven_tls_server_create(server_config, &server) == PROVEN_OK, "두 번째 쌍");
    for (int round = 0; round < 10 && !(proven_tls_is_established(client) &&
proven_tls_is_established(server)); ++round) {
        EXAMPLE_REQUIRE(carry(client, server) == PROVEN_OK && carry(server, client) == PROVEN_OK, "클라이언트
에서 서버로");
    }
    EXAMPLE_REQUIRE(proven_tls_resumed(client) && proven_tls_resumed(server), "양쪽이 재개했다");
    proven_tls_conn_destroy(client);
    proven_tls_conn_destroy(server);

    // ---- 요청한 서버가 아닌 서버 ------------------------------------------------
    /* 같은 서버에, 인증서에 없는 이름으로 닿는다. 클라이언트는 거부하고 이유를 정확히 말한다. */
    proven_tls_conn_t *wrong = NULL;
    EXAMPLE_REQUIRE(proven_tls_client_create(client_config, PROVEN_LIT("elsewhere.test"), NULL, &wrong) ==
PROVEN_OK &&
                    proven_tls_server_create(server_config, &server) == PROVEN_OK, "두 번째 쌍");
    EXAMPLE_REQUIRE(carry(wrong, server) == PROVEN_OK, "클라이언트에서 서버로");
    EXAMPLE_REQUIRE(carry(server, wrong) == PROVEN_ERR_NAME_MISMATCH, "클라이언트의 feed가
PROVEN_ERR_NAME_MISMATCH를 돌려준다");
    EXAMPLE_REQUIRE(proven_tls_peer_fault(wrong) == PROVEN_CERT_FAULT_NAME_MISMATCH && !
proven_tls_is_established(wrong), "정확한 fault와 함께, 연결은 없다");
    /* 실패는 보류 출력에 경고를 남긴다. 보내라: 상대가 알게 되는 길이다. */
    EXAMPLE_REQUIRE(proven_tls_pending_output(wrong).size > 0 && carry(wrong, server) == PROVEN_ERR_PROTOCOL,
"서버가 통보받는다");
    EXAMPLE_REQUIRE(proven_tls_alert_received(server) == 42, "bad_certificate, 그 번호로");
    proven_tls_conn_destroy(wrong);
    proven_tls_conn_destroy(server);

    proven_mem_wipe((proven_mem_mut_t){ session.opaque, sizeof session.opaque });
    proven_mem_wipe(key.value);
    heap.free_fn(heap.ctx, key.value.ptr);
```

```
    heap.free_fn(heap.ctx, cert.value.ptr);
    proven_tls_config_destroy(client_config);
    proven_tls_config_destroy(server_config);
    proven_cert_store_destroy(anchors);
    return EXAMPLE_OK();
}
```

6. 재개

전체 핸드셰이크는 서명 둘과 인증서 체인 하나가 든다. 전에 서버와 대화한 적이 있는 클라이언트는 그것을 건너뛸 수 있다. 서버가 첫 핸드셰이크 끝에 **티켓**을 줬고, 그 티켓을 내미는 두 번째 연결은 양쪽이 이미 비밀을 공유한다는 것을 그것으로 증명한다. 새 키 교환은 여전히 일어나므로 새 연결의 키는 그 연결만의 것이다.

클라이언트에서는 구조체 하나다.

```
typedef struct { alignas(8) proven_byte_t opaque[PROVEN_TLS_SESSION_SIZE]; } proven_tls_session_t;   /* 2,048 바이트 */
```

`proven_tls_session_t`를 0으로 초기화하고, 같은 서버 이름으로 가는 연결마다 `proven_tls_client_create`(또는 `proven_tls_transport_client`)에 넘긴다. 서버가 티켓을 보내면 연결이 거기에 적어 넣고, 다음 연결이 그것을 내민다. 서버가 받아들였는지는 `proven_tls_resumed`가 말한다. 포인터가 없는 평범한 바이트 블록이므로 어디에든, 얼마 동안이든 두어도 되고, **비밀을 담고 있으므로** 다 쓰면 `proven_mem_wipe`로 지운다. 만들어진 그 이름에만 내밀고, 서버가 준 수명을 넘겨서는 절대 내밀지 않는다. 세션 하나는 한 번에 연결 하나를 섬긴다.

서버에서는 할 일이 없다. `no_resumption`이 켜져 있지 않으면 티켓을 발급하고, `ticket_lifetime_s` (기본 두 시간, 최대 일주일) 동안 인정한다. **서버는 클라이언트별로 아무것도 저장하지 않는다.** 티켓은 이 프로세스만 가진 키로 봉인한 세션 상태다. 그 키는 설정을 만들 때 생기고, 티켓 수명마다 교체되며, 나가 있는 티켓이 계속 열리도록 수명 하나만큼 더 보관되고, 어디에도 기록되지 않는다 - 그래서 티켓은 재시작을 넘기지 못하고, 두 프로세스는 서로의 티켓을 받지 않는다.

여기서 재개가 하지 않는 것:

- **조기 데이터는 그 주인이 요청할 때만**(다음 부분): 이 라이브러리의 HTTP 서버와 클라이언트가 `early_data`를 켰을 때. 엔진을 직접 다루거나 전송 래퍼를 서버로 쓰는 프로그램은 받지 않는다.
- **`proven_tls_http_wrap`을 통해서는 `early_data`가 있을 때만.** 그것을 켠 클라이언트 설정은 서버 이름마다 가장 새 세션을 간직하고(이름 여덟 개), 전송 래퍼는 세션을 받지 않으면 그것으로 재개한다. 없으면 HTTP 클라이언트의 연결은 재개하지 않고, 대신 열린 연결을 다시 쓴다.
- 재개된 연결의 상대는 세션이 만들어질 때 검증된 그 상대다. 인증서는 다시 보내지지 않으므로 다시 검사되지 않고, 그 사이에 지난 만료는 알아채지 못한다. 티켓 수명이 그것이 문제될 수 있는 기간을 제한한다.

TLS 1.2에서 재개된 연결은 새 연결보다 약하며, 어떻게 약한지 알아야 한다. 새 키 교환이 없다: 새 연결의 키는 첫 연결의 마스터 시크릿과 새 난수 둘에서 나온다. 그래서 나중에 서버의 티켓 키를 얻는 자는 그 키가 봉인한 티켓으로 재개된 모든 연결을, 그리고 첫 연결의 시크릿까지 읽을 수 있다 - 전체 핸드셰이크와 TLS 1.3의 재개는 이것을 허락하지 않는다. 여기서 티켓 키는 프로세스의 메모리에만 있고 티켓 수명마다 교체되므로 노출은 그만큼으로 제한된다. `no_resumption`은 그것을 없앤다. 한 버전에서 만든 세션은 다른 버전에 내놓지도, 다른 버전에서 받아들이지도 않는다: `proven_tls_session_t` 하나는 마지막 연결이 남긴 종류를 담고, 다른 버전으로 끝나는 핸드셰이크는 그저 전체 핸드셰이크가 된다.

조기 데이터(0-RTT)

재개도 첫 요청을 보내기 전에 핸드셰이크 왕복 하나를 기다린다. **조기 데이터**를 쓰면 클라이언트는 그 요청을 첫 패킷에 실어 티켓에서 만든 키로 보내고, 서버는 클라이언트의 Finished 전, 첫 응답에 답할 수 있다: 새 연결에서 왕복 하나가 준다(측정: 왕복 20 ms에서 HTTP/3 GET이 43 ms에서 22 ms로. TCP에서는 TCP 자신의 것, 핸드셰이크의 것, 요청의 것 셋 중 하나). HTTP/3(17장)에 있고, 0.84.0부터는 TCP 위의 HTTP/1.1과 HTTP/2 - 이벤트 기반 서버와 클라이언트(15장, 17장), 블로킹 서버와 클라이언트(11장) - 에도 있으며, 양쪽 설정이 모두 `early_data`를 켤 때만 쓴다(`no_resumption`과 함께 쓸 수 없다):

- `early_data`를 켠 서버는 그것을 허용하는 티켓을 발급하고 - TCP에서는 16,384바이트, QUIC에서는 RFC 9001대로 - 다음을 모두 갖췄을 때만 조기 데이터를 받는다: 티켓이 허용하고 같은 전송에서 만들어짐; 첫

번째 신원이고 binder가 맞음; cipher suite와 프로토콜(ALPN. TCP에서는 없음이면 없음)이 티켓의 것과 같음; QUIC에서는 이 서버가 약속하는 전송 한도가 티켓을 발급할 때 약속한 것과 같음; HelloRetryRequest가 없었음; 클라이언트가 알린 나이가 티켓 발급 뒤 흐른 시간과 10초 안에서 맞음; 그리고 **그 티켓으로 조기 데이터를 받은 적이 없음**. 그렇지 않으면 조기 데이터를 거절하고 핸드셰이크는 재개로 이어진다. TCP에서 티켓이 허용한 것보다 많은 조기 데이터는 연결을 끝내고(`unexpected_message`), 클라이언트의 `EndOfEarlyData`가 끝나는 곳을 알린다.

- `early_data`를 켠 클라이언트는 세션의 티켓이 허용하고 그 프로토콜을 지금도 내밀 때 조기 데이터를 보낸다. 거절되어도 잃는 것은 아끼려던 왕복 하나뿐이다 - 보낸 것은 핸드셰이크 뒤에 다시 간다. 조기 데이터를 실었던 티켓은 다시 그것에 내밀지 않는다: 서버가 거절할 것이고, 더 새 티켓이 그 자리를 잇는다.

조기 데이터는 재전송될 수 있다. 클라이언트의 첫 묶음을 본 사람은 그것을 다시 보낼 수 있고, 그것을 받는 서버는 요청을 다시 처리한다. 서버 설정마다 조기 데이터를 받은 티켓의 기록부(4,096개, 96 KiB, 설정과 함께 만듦)를 신선도 창이 그것을 들일 수 있는 동안 두고, 티켓마다 조기 데이터를 한 번만 받는다. 기록부가 차면 조기 데이터를 거절한다. 티켓은 그것을 봉인한 프로세스와 설정에서만 열리므로(위), 기록부 하나가 그 티켓을 받을 수 있는 모든 서버를 덮는다. 기록부로 막을 수 없는 것: 공격자가 클라이언트의 첫 묶음을 붙잡아 두고 사본을 먼저 보내면 서버는 사본을 처리한다. 원본의 조기 데이터는 거절되어 핸드셰이크 뒤에 다시 가므로, 요청이 두 번 처리된다. 그래서 HTTP 서버와 클라이언트는 다음 규칙을 따른다: 반복해도 안전한 요청 - 본문 없는 GET, HEAD, OPTIONS - 만 조기에 보내고, 핸들러는 `proven_http_stream_early`로 물어 **425 Too Early**로 답할 수 있으며, 그 밖의 것이 조기에 오면 서버가 스스로 425로 답한다.

TCP에서 누가 받는가. 주인이 요청한 연결만: HTTP 서버는 `tls` 설정에 `early_data`가 있으면 받고, 조기에 온 요청마다 표시하며(블로킹 서버의 `proven_http_exchange_early`, 이벤트 기반 서버의 `proven_http_stream_early`), 반복해도 안전하지 않은 것에는 스스로 425로 답하고, 나머지에는 클라이언트의 Finished 전에 답할 수 있다. HTTP 클라이언트는 반복해도 안전한 것만 조기에 보낸다. 블로킹 클라이언트의 전송은 조기에 보낸 사본을 간직했다가 서버가 거절하면 먼저 다시 보내고, 425를 받으면 요청을 새 연결에서 조기가 아니게 한 번 더 보낸다. 엔진이나 전송 래퍼를 서버로 직접 다루는 프로그램은 설정이 무엇이든 받지 않는다.

```
#include <stdio.h>
#include <string.h>

/*
 * HTTPS 위의 early data(0-RTT), 양쪽을 한 프로그램에: 11장의 서버와 11장의 클라이언트가 각자 TLS 설정에
 * `early_data`를 둔다. 첫 요청은 전체 핸드셰이크를 하고 티켓을 남기고, 두 번째 요청은 다음 연결의
 * ClientHello와 함께 가며, 핸들러는 그것을 알 수 있다.
 */

static const char *const HOW_TO_MAKE =
    "  openssl req -x509 -newkey ed25519 -nodes -days 30 -subj /CN=example.test \\\n"
    "      -addext subjectAltName=DNS:example.test,IP:127.0.0.1 -keyout key.pem -out cert.pem\n";

typedef struct {
    proven_http_server_t *server;
} ex_app_t;

static void handle(void *ctx, proven_http_exchange_t *x) {
    ex_app_t *app = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/quit"))) {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        proven_http_server_stop(app->server);
        return;
    }
    /* early data로 온 요청은 누군가 다시 보낸 사본일 수 있다. 본문 없는 GET, HEAD, OPTIONS만 early로
     * 여기까지 온다 - 그 밖의 것은 서버가 스스로 425로 답한다 - 그리고 무언가를 바꾸는 GET도 425로
     * 답해야 한다: 클라이언트는 핸드셰이크 뒤에 그것을 다시 보낸다. */
    const bool early = proven_http_exchange_early(x);
    (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(early ?
```

```
PROVEN_LIT("early\n") : PROVEN_LIT("after the handshake\n")));
}

static void serve(void *arg) {
    (void)proven_http_server_run(((ex_app_t *)arg)->server);
}

/* `url`을 GET하고 본문이 `want`인지 돌려준다. */
static bool get_is(proven_http_client_t *client, proven_allocator_t heap, proven_u8str_view_t url,
proven_u8str_view_t want) {
    proven_http_client_response_t response;
    proven_u8str_t body = { 0 };
    bool ok = proven_http_client_get(client, url, &response) == PROVEN_OK && response.status == 200 &&
              proven_http_client_read_all(&response, heap, &body, 4096) == PROVEN_OK &&
proven_u8str_view_eq(proven_u8str_as_view(&body), want);
    proven_u8str_destroy(heap, &body);
    proven_http_client_finish(&response);
    return ok;
}

int main(int argc, char **argv) {
    proven_allocator_t heap = proven_heap_allocator();
    if (argc < 3) {
        printf("usage: %s CERTIFICATE.pem KEY.pem\nA self-signed pair for 127.0.0.1:\n%s", argv[0],
HOW_TO_MAKE);
        return 0;
    }
    proven_result_mem_mut_t cert = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[1]), 64 *
1024);
    proven_result_mem_mut_t key = proven_fs_read_all_bounded(heap, proven_u8str_view_from_cstr(argv[2]), 16 *
1024);
    EXAMPLE_REQUIRE(cert.err == PROVEN_OK && key.err == PROVEN_OK, "the certificate and the key are read");
    if (cert.err != PROVEN_OK || key.err != PROVEN_OK) return 1;

    /* 서버: `early_data`가 있으면 티켓이 early data를 허락하고 서버가 그것을 받는다 - 티켓마다 한 번,
     * 티켓 나이의 10초 안에서. */
    proven_tls_options_t server_options = {
        .alloc = heap, .early_data = true,
        .certificate_pem = { cert.value.ptr, cert.value.size }, .private_key_pem = { key.value.ptr,
key.value.size },
    };
    proven_tls_config_t *server_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&server_options, &server_tls) == PROVEN_OK, "the server's TLS
configuration");
    ex_app_t app = { 0 };
    proven_http_server_config_t server_config = { .alloc = heap, .handler = handle, .handler_ctx = &app, .tls
= server_tls };
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_server_create(&server_config, &app.server) == PROVEN_OK &&
                    proven_http_server_listen(app.server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0),
&at) == PROVEN_OK, "an HTTPS server on a free port");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &app) == PROVEN_OK, "its loop runs on
another thread");

    /* 클라이언트: `early_data`가 있으면 설정이 서버마다 가장 새 세션을 간직하고, 반복해도 안전한
     * 요청을 다음 연결의 ClientHello와 함께 보낸다. */
```

```
    proven_cert_store_t *anchors = NULL;
    EXAMPLE_REQUIRE(proven_cert_store_create(heap, &anchors) == PROVEN_OK &&
                    proven_cert_store_add_pem(anchors, (proven_mem_view_t){ cert.value.ptr, cert.value.size },
NULL) == PROVEN_OK, "the anchors");
    proven_tls_options_t client_options = { .alloc = heap, .anchors = anchors, .early_data = true };
    proven_tls_config_t *client_tls = NULL;
    EXAMPLE_REQUIRE(proven_tls_config_create(&client_options, &client_tls) == PROVEN_OK, "the client's TLS
configuration");
    /* 연결을 간직하지 않아, 요청마다 새 연결을 만든다. */
    proven_http_client_config_t client_config = { .alloc = heap, .tls_wrap = proven_tls_http_wrap, .tls_ctx =
client_tls };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "an HTTP client");

    char url_text[64];
    int url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/", (unsigned)at.port);
    proven_u8str_view_t url = { (const proven_byte_t *)url_text, (proven_size_t)url_len };
    EXAMPLE_REQUIRE(get_is(client, heap, url, PROVEN_LIT("after the handshake\n")), "the first connection: a
full handshake, and a ticket");
    EXAMPLE_REQUIRE(get_is(client, heap, url, PROVEN_LIT("early\n")), "the second: the GET went with its
ClientHello");

    url_len = snprintf(url_text, sizeof url_text, "https://127.0.0.1:%u/quit", (unsigned)at.port);
    url.size = (proven_size_t)url_len;
    (void)get_is(client, heap, url, PROVEN_LIT("bye\n"));
    proven_job_group_wait(threads, &running);
    proven_http_client_destroy(client);
    proven_http_server_destroy(app.server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    proven_tls_config_destroy(client_tls);
    proven_tls_config_destroy(server_tls);
    proven_cert_store_destroy(anchors);
    proven_mem_wipe(key.value);
    heap.free_fn(heap.ctx, key.value.ptr);
    heap.free_fn(heap.ctx, cert.value.ptr);
    return EXAMPLE_OK();
}
```

서버가 받지 않는 조기 데이터. 클라이언트는 이 서버가 받지 않을 조기 데이터를 보낼 수 있다 - 열 수 없는 티켓, 조기 데이터를 받지 않는 주인의 서버, 다시 보낸 hello. 서버는 RFC 8446 4.2.10절이 조기 데이터를 거절하는 서버에게 요구하는 대로 그 레코드를 건너뛴다. HelloRetryRequest 뒤라면 두 번째 ClientHello까지 오는 application_data 레코드를 모두, 그렇지 않으면 핸드셰이크 키로 열리지 않는 레코드를 하나가 열릴 때까지 건너뛰고, 핸드셰이크는 그것들이 오지 않았던 것처럼 이어진다. 클라이언트는 서버의 답에서 조기 데이터가 받아들여지지 않았음을 알고 다시 보낸다. 건너뛰는 양에는 한도가 있다: 64 KiB이고, 레코드마다 5바이트 헤더도 센다. 그것을 넘으면 다음 레코드는 열리지 않는 여느 레코드처럼 연결을 끝낸다(bad_record_mac, HelloRetryRequest 뒤라면 unexpected_message).

7. 클라이언트 인증서와 핀

클라이언트 인증서. 서버는 클라이언트도 자신이 누구인지 증명하기를 요구할 수 있다 - 상호 TLS, 서비스 사이에서 흔한 선택. 서버에서 `client_auth`를 정하고 `anchors`를 준다.

client_auth	인증서 없는 클라이언트	검증되지 않는 인증서를 가진 클라이언트
PROVEN_TLS_CLIENT_AUTH_NONE	(요구하지 않음)	(요구하지 않음)

PROVEN_TLS_CLIENT_AUTH_REQUEST	들여보낸다. proven_tls_peer_key_sha256이 false를 돌려준다	거부
PROVEN_TLS_CLIENT_AUTH_REQUIRE	거부	거부

클라이언트에서는 certificate_pem과 private_key_pem을 정한다. 요구받으면 내밀고 아니면 내밀지 않는다. 서버가 알게 되는 것은 클라이언트가 어떤 기관이 보증한 키를 갖고 있다는 *사실*이다. 어느 클라이언트인지는 여러분이 정한다 - proven_tls_peer_key_sha256으로, 또는 인증서(keep_peer_certificate)와 그 이름으로 - 그리고 PROVEN_TLS_CLIENT_AUTH_REQUEST는 "증명했는지" 조차 여러분에게 남긴다. 요청에 따라 행동하기 전에 확인하라.

HTTP 서버에서는 엔진이 서버 안에 있고, 이 버전에서 핸들러는 거기에 닿을 호출이 없다. 거기서는 REQUIRE를 써서 핸들러가 보는 모든 요청이 검증된 클라이언트에게서 온 것이 되게 하라.

핀. 핀은 공개 키의 SHA-256이다(13장 7절). pins를 주면 상대의 키가 그 가운데 하나여야 한다.

- PROVEN_TLS_VERIFY_CHAIN에서는 체인에 **더해서**. 그러면 세상 어느 기관이 낸 유효한 인증서도 충분하지 않다. 기대한 키를 위한 것이어야 한다.
- PROVEN_TLS_VERIFY_PIN_ONLY에서는 체인 **대신**. 체인도, 날짜도, 이름도 없다 - 키가 신원이고 server_name은 비어 있어도 된다. 자체 서명 인증서로 직접 운영하는 상대를 위한 모드이며, 핀이 보안의 전부라는 뜻이다.

핀은 둘 - 쓰고 있는 키와 예비 - 실어 보내라. 그렇지 않으면 잃어버린 키 하나가 연결하지 못하는 기기 무리가 된다.

8. 실패할 때

연결은 한 번, 에러 하나로 실패하고, 실패한 채로 남는다.

에러	무슨 일이 있었나	흔한 원인
PROVEN_ERR_UNTRUSTED	상대의 인증서가 앵커로 이어지지 않거나, 핀에 없거나, 필수 클라이언트 인증서를 내밀지 않았다(상대에게는 TLS 1.3에서 certificate_required, 더 가까운 말이 없는 1.2에서는 handshake_failure로 알린다)	사설 또는 자체 서명 기관. 빠진 중간 인증서. Windows에서는 기계가 아직 받아 오지 않은 루트(13장)
PROVEN_ERR_EXPIRED, PROVEN_ERR_NOT_YET_VALID	경로 위의 인증서가 유효 기간 밖이다	갱신하지 않은 인증서 - 또는 이 기계의 시계
PROVEN_ERR_NAME_MISMATCH	인증서는 유효하고, 다른 이름을 위한 것이다	틀린 URL, 잘못 설정된 서버, 또는 가로채기
PROVEN_ERR_PROTOCOL	상대가 한 그 밖의 모든 것: 이쪽이 말하지 않는 버전이나 알고리즘, 형식이 틀렸거나 예상하지 못한 메시지, 인증되지 않는 레코드 - 또는 상대가 자기 경고로 핸드셰이크를 끝냈다	TLS 1.1 이하만 말하거나, TLS 1.2 가운데 이 라이브러리가 뺀 부분만 말하는 상대(9절). min_version이 1.3일 때의 TLS 1.2 상대. 공통된 암호 스위트나 그룹이 없다. 이쪽의 인증서를 거부한 상대. 손상
PROVEN_ERR_RESET (래퍼)	TLS 닫기 없이 연결이 끝났다	상대가 죽었거나 끊겼다. 받은 것을 완전하다고 믿지 마라
PROVEN_ERR_TIMEOUT (래퍼)	기한이 지났다	응답을 멈춘 상대

```
proven_cert_fault_t proven_tls_peer_fault(const proven_tls_conn_t *conn);
int proven_tls_alert_received(const proven_tls_conn_t *conn);
int proven_tls_alert_sent(const proven_tls_conn_t *conn);
```

로그 한 줄을 위해: proven_tls_peer_fault는 인증서가 거부된 정확한 이유다(13장의 목록에 PROVEN_CERT_FAULT_PIN_MISMATCH를 더한 것). proven_tls_alert_received는 상대가 보낸 경고를 RFC 8446 6절의 번호로 준다 - 답이 PROVEN_ERR_PROTOCOL이고 *상대쪽*이 무엇에 반대했는지 알고 싶을 때 여기를 본다(48 unknown_ca와 42 bad_certificate는 여러분의 인증서를 받아들이지 않았다는 뜻이고, 40 handshake_failure는 공통된 것이 없다는 뜻, 70 protocol_version은 다른 버전을 원한다는 뜻이다). proven_tls_alert_sent는 이쪽이 상대에게 말한 것이다. 둘 다 없었으면 -1이다.

엔진에서는 에러가 proven_tls_feed에서 한 번 돌아온다. 그때 상대에게 알리는 경고가 보류 출력에 있고 - 보내라 - 그 뒤로는 출력 쌍과 질문들을 뺀 모든 호출이 PROVEN_ERR_INVALID_STATE를 돌려준다.

에러는 일부러 적다. 인증에 실패한 상대는 실패했다는 것만 알게 된다. proven_tls_peer_fault나 PROVEN_ERR_PROTOCOL의 세부를 연결 반대편에 전달하지 마라. 그것은 여러분의 로그를 위한 것이다.

9. 무엇을 협상하고, 무엇을 붙들고, 무엇이 없는가

협상하는 것. TLS 1.3, 그리고 아직 방어할 수 있는 부분만 남기고 줄인 TLS 1.2. 양쪽이 모두 1.3을 말할 수 있으면 1.3을 말한다.

TLS 1.3에서:

무엇	내용
암호 스위트	TLS_AES_128_GCM_SHA256 (0x1301), TLS_AES_256_GCM_SHA384 (0x1302), TLS_CHACHA20_POLY1305_SHA256 (0x1303)
그 순서	AES 명령이 있는 프로세서(AES-NI가 있는 x86 - 64비트, 그리고 32비트도 - 와 암호 확장이 있는 AArch64)에서는 AES-128, AES-256, ChaCha20. **그 밖의 모든 곳에서는 ChaCha20 먼저**: 그 명령이 없으면 이 라이브러리의 AES는 상수 시간이되 ChaCha20보다 적어도 서른 배 느리다. ChaCha20은 x86에서 SSE2로 네 블록씩, AVX2로 여덟 블록씩 한 번에 처리한다(v0.85.0부터. 프로세서에 실행 시점에 묻는다)
키 교환	X25519. 상대가 고집하면 P-256(왕복 한 번 추가)
이쪽의 키	P-256의 ECDSA, Ed25519, 또는 2048~4096비트의 RSA(RSA-PSS와 SHA-256으로 서명)
상대의 키	그 둘, P-384의 ECDSA, RSA(2048~8192비트, RSA-PSS)
그 밖에	ALPN, 서버 이름 표시, 세션 티켓, 키 갱신, record_size_limit, 미들박스 호환 메시지

TLS 1.2에서:

무엇	내용
암호 스위트	ECDHE-ECDSA와 ECDHE-RSA, 각각 AES128-GCM-SHA256, AES256-GCM-SHA384, CHACHA20-POLY1305와 함께: 0xC02B, 0xC02C, 0xCCA9, 0xC02F, 0xC030, 0xCCA8. 암호의 순서는 위와 같다
키 교환	X25519 또는 P-256, 연결마다 새 키 쌍 - 그래서 언제나 순방향 비밀성이 있다
이쪽의 키	ECDHE-ECDSA 스위트에는 P-256의 ECDSA 또는 Ed25519. ECDHE-RSA에는 RSA이며, 상대가 내놓으면 RSA-PSS로, 아니면 SHA-256의 PKCS #1 v1.5로 서명한다
상대의 키	1.3과 같은 종류. 그 서명은 SHA-256, SHA-384 또는 SHA-512를 써야 한다
상대에게 요구하는 것	확장 마스터 시크릿(RFC 7627). 그것이 없는 상대는 handshake_failure로 거부된다
그 밖에	ALPN, 서버 이름 표시, 세션 티켓(RFC 5077. 6절), 클라이언트 인증서, RFC 8446 4.1.3절의 다운그레이드 표시

무엇을 뺐는가가 요점이다. 이름이 붙은 TLS 1.2 공격은 모두 이 목록에 없는 것을 거쳐 갔다: CBC 암호 스위트(패딩 오라클), RSA 키 교환(Bleichenbacher의 공격, 그리고 순방향 비밀성 없음), 약하거나 상대가 고른 그룹의 유한체 Diffie-Hellman, 재협상, 압축, 핸드셰이크에 묶이지 않은 마스터 시크릿, 핸드셰이크 서명의 SHA-1과 MD5. 그것들만 내놓는 상대는 handshake_failure를 받고, 프로그램은 PROVEN_ERR_PROTOCOL을 받는다.

그 가운데 일부는 다른 것이 없는 상대를 위해 하나씩 켤 수 있다: 어느 것인지, 각각 무엇을 치르는지는 10절에 있다.

- **재협상은 답하지 않고 사양한다.** 수립된 연결에서 새 핸드셰이크를 청하는 상대에게는 `no_renegotiation` 경고를 보내고 연결은 그대로 이어 간다. 그다음에 상대가 무엇을 할지는 상대의 선택이다: 계속하는 것도 있고 닫는 것도 있다. 계속 청하는 상대는 연달아 열여섯 번 뒤에 끊는다.
- **클라이언트는 1.2로 속아 내려가지 않는다.** 1.3을 말할 수 있었으면서 1.2로 답하는 서버는 자기 난수에 표시를 남긴다. 1.3을 내놓았던 이쪽 클라이언트가 그 표시를 보면 `illegal_parameter`로 핸드셰이크를 끝낸다. 중간의 누군가가 더 나은 제안을 지웠기 때문이다.
- **ChangeCipherSpec은 방향마다 한 곳에서만** - 키가 바뀌는 곳에서 - 받아들이고 다른 어디에서도 받지 않는다.
- **Ed25519 키는 TLS 1.3에서만 클라이언트를 인증한다.** 키가 Ed25519인 클라이언트는 1.2 서버에 인증서를 내밀지 않는다(없는 것처럼). 서버의 Ed25519 키는 양쪽 버전에서 쓰인다.

TLS 1.3만 말하려면 `min_version = PROVEN_TLS_VERSION_1_3`으로 둔다. 0.22.0 이전의 이 라이브러리가 하던 그대로다. 클라이언트를 직접 통제하는 서버, 그리고 6절의 더 약한 재개를 받아들일 수 없는 모든 곳에서 고려하라. 기본값이 1.2를 받는 까닭은, 더 새로운 것을 말하지 않는 서버에 닿지 못하는 클라이언트는 더 안전한 것이 아니라 끊긴 것일 뿐이기 때문이다 - 그리고 아주 많은 서버와 기기가 아직 더 새로운 것을 말하지 않는다.

연결마다 붙드는 것. 핸드셰이크 동안에는 몇 킬로바이트이고, 끝나면 해제된다. 그 뒤로 아무것도 오가지 않을 때는 연결마다 **약 1.25 KiB**(15장의 이벤트 구동 서버는 1초 조용한 연결에서 키의 펼친 형태 약 700바이트를 돌려주고 원래 키만 남겼다가 다음 레코드에 다시 만든다 - 15장 14절): 키(v0.77.0부터 AES-GCM의 한 번에 도는 코드가 보관하는 GHASH 키의 거듭제곱 포함)와 카운터. 레코드 버퍼는 레코드를 받는 동안, 읽히기를 기다리는 동안, 보내지기를 기다리는 동안에만 있고 끝나면 해제된다 - 그래서 한가한 TLS 연결 만 개는 16 KiB 버퍼 둘씩이 치를 삼백 메가바이트가 아니라 십이 메가바이트쯤을 붙든다. 그것은 이 층의 몫이다. 전송 래퍼는 수십 바이트를 더하고, HTTP 서버 자신의 연결별 요청 버퍼는 11장의 것이며 별개다. (TLS 1.2 연결도 같은 만큼을 붙든다.)

핸드셰이크 하나의 비용. 개발 기계의 x86-64 코어 하나에서 X25519와 P-256 인증서의 전체 핸드셰이크는 서버에서 프로세서 시간 약 0.31 ms, 그 인증서를 앵커로 가진 클라이언트에서는 약 0.41 ms가 든다. 클라이언트가 앵커까지 가는 길에 서명을 검사하는 인증서마다 - 앵커가 발급한 인증서라면 하나 - 약 0.25 ms가 더해진다. 서버라면 코어당 초당 새 연결 3,200개쯤. (GCC 14로 빌드한 것이다. v0.90.0부터 P-256에는 그 자신의 field 코드가 있고, GCC 16과 Clang은 그것을 일반 코드의 약 두 배 속도로, GCC 14는 약 1.15배로 컴파일한다.) Ed25519 인증서라면 두 수치는 0.21 ms와 0.25 ms, 인증서 하나가 더해질 때 0.08 ms이고, 코어 하나가 초당 새 연결 4,800개쯤을 받는다. 재개(6절)는 서명을 없애고, 연결을 열어 두는 것은 핸드셰이크를 없앤다.

여기의 공개 키 연산은 읽을 수 있도록, 그리고 비밀 - 개인 키, 논스, 공유 비밀 - 을 다루는 곳에서는 상수 시간이 되도록 쓰였다. **서명 검증은 상수 시간이 아니며**, 그럴 필요가 없다: 검증이 받는 키, 메시지, 서명은 모두 공개된 것이다. 검증은 두 곱셈을 가변 시간의 한 번의 진행으로 하고, 그래서 약 두 배 빠르다. 그렇게 하는 함수들은 검증에서만 닿을 수 있고, 다른 무엇이 그것을 부르면 빌드가 실패한다 (`scripts/check-vartime.py`). 같은 기계의 `openssl speed`(OpenSSL 3.5)와 비교하면, 여기의 Ed25519는 OpenSSL 시간의 약 4분의 3에 서명하고 약 3분의 2에 검증한다. X25519와 NIST 곡선은 1.2배에서 7배의 시간이 걸린다: P-384 검증이 가장 적게, P-256 키 합의가 가장 많이(v0.90.0부터 GCC 16이나 Clang으로 빌드하면 P-256은 2.1배에서 4배).

이 수치는 GCC나 Clang으로 만든 64비트 빌드의 것이다: 그때 P-256, RSA, Diffie-Hellman의 수는 64비트 limb에, Curve25519 체의 수는 51비트 limb 다섯 개에 담기고, 두 limb의 곱은 컴파일러의 128비트 정수에 들어간다. 32비트 빌드와 `-DPROVEN_CRYPTO_MP_LIMB_BITS=32`로 만든 빌드는 32비트 limb와 26·25비트 limb에 담고, 같은 연산에 두 배에서 네 배의 시간을 쓴다 - P-256 인증서의 핸드셰이크에 0.83 ms와 1.1 ms, Ed25519로는 0.59 ms와 0.75 ms, 아래 RSA 서명에 3.4, 11, 25 ms. 결과는 어느 쪽이든 같은 바이트다.

RSA 키에서는 서명이 대부분을 차지한다. 같은 코어에서 서명 하나는 2048비트 키로 약 1.25 ms, 3072비트로 3.7 ms, 4096비트로 8.4 ms가 걸린다(v0.88.0부터. 제곱을 곱셈의 4분의 3쯤 되는 일로 한다. 그 전에는 1.5, 4.7, 11 ms): 2048비트 RSA 키를 가진 서버는 코어당 초당 새 연결 650개쯤을 마치고, P-256으로는 3,000개다. 15장의 이벤트 구동 서버에서 그 시간은 루프의 스레드에서 쓰이고, 그동안 다른 것은 서비스되지 않는다. 서명은 두 소수를 법으로 하여 상수 시간 지수 연산으로 하고, 입력을 블라인딩하며, 결과를 내보내기 전에 공개 키로 검사한다. 작업용 수들은 스택에 둔다 - x86-64의 `-O2`에서 컴파일러 자신의 계산으로 약 20 KiB - 작은 타깃은 그것을 감안해야 한다. 상대는 `rsa_pss_rsae_sha256`을 받아들여야 한다. TLS 1.3이 모든 구현에 요구하는 것이며, 거기서 이쪽이 서명에 쓰는 단 하나의 방식이다.

상대에 대한 한계. 핸드셰이크 메시지는 최대 `max_handshake_bytes`이고, 레코드는 프로토콜이 허락하는 크기까지이며 헤더에서 거부된다. 아무것도 나르지 않는 레코드 - 빈 레코드, 반복되는 호환 메시지 - 는 연달아 열여섯 개까지 봐준다. 키 갱신 요청에는 보내지 않은 답이 64 KiB 쌓일 때까지 답한다. 어느 것이든 넘으면 연결은 `PROVEN_ERR_PROTOCOL`로 끝난다.

여기에 없는 것:

- **설정이 요구하지 않는 한(10절):** TLS 1.1과 1.0, CBC 암호 스위트, RSA 및 유한체 Diffie-Hellman 키 교환, 확장 마스터 시크릿이 없는 상대.
- **전혀 없는 것:** SSL 3.0과 그 이전. RC4, 3DES, 수출용, 익명, DSS 스위트. 재협상. 압축. 서버에 보관하는 세션 ID. TLS 1.2의 `record_size_limit`. 그중 하나가 필요한 상대는 `PROVEN_ERR_PROTOCOL`로 실패한다.
- 이쪽을 위한 **4096비트를 넘는 RSA 키**, 그리고 RSA-PSS 키(키가 PSS 전용으로 표시된 인증서).
- **엔진을 직접 다루는 프로그램에 오는 조기 데이터.** TCP에서는 이 라이브러리의 HTTP 서버와 클라이언트만 보내고 받는다(6절). 엔진이나 전송 래퍼 위에 직접 만든 서버는 클라이언트가 보낸 것을 64 KiB까지 건너뛴다.
- **폐기.** CRL도, OCSP도, 스테이플링도 없다. 13장을 보라.
- **클라이언트가 요청한 이름에 따라 인증서를 고르기.** 설정 하나에 인증서 하나.
- **암호화된 ClientHello, 핸드셰이크 뒤의 클라이언트 인증, 외부 사전 공유 키.**
- **레코드 패딩.** 사용 중인 TLS 대부분이 그렇듯 레코드 길이가 데이터 길이를 드러낸다.
- **프로세스 사이의 티켓 공유**, 또는 재시작을 넘긴 보관.

어떻게 시험했는가. 등록된 테스트는 RFC 8448의 예시 핸드셰이크 다섯을 키 스케줄과 레코드 계층에 값 하나하나 재현하고, 그 가운데 둘은 클라이언트에 레코드 하나하나 재현한다. 메모리 안에서 클라이언트와 서버를 서로 맞붙여 모든 스위트와 키 종류, 7절과 8절의 모든 거부, 재개, 그리고 핸드셰이크의 단일 비트 변경 스윕을 돌린다. 그리고 실제 소켓 위에서 HTTPS와 `wss`를 돌린다. 등록된 테스트 밖에서는, 양쪽 역할을 OpenSSL과, 서버를 GnuTLS와 대조해 모든 스위트와 그룹, 재시도, 재개, 클라이언트 인증서를 돌렸고, 클라이언트로 공개 서버의 페이지를 가져왔다. 이쪽의 RSA 키에 대해서는: 같은 두 구현이, OpenSSL이 만든 2048·3072·4096비트 키로 서명한 핸드셰이크를 받아들였다. 그 키들에 대한 이 라이브러리의 PKCS #1 v1.5 서명은 OpenSSL의 것과 같은 바이트였고, PSS 서명은 OpenSSL이 받아들였다. TLS 1.2에 대해서는: 그 키 유도를 PRF의 공개된 시험 벡터 둘과, 이 라이브러리 밖에서 계산한 값에 대조한다(1.3과 달리 1.2 핸드셰이크 전체의 공개된 기록은 없으므로, 핸드셰이크에 대해서는 다른 구현들이 증거다). 양쪽 역할을 OpenSSL과 대조해 여섯 스위트 모두와 세 종류의 키 모두, 티켓, 클라이언트 인증서, PKCS #1과 PSS 서명, 확장 마스터 시크릿이 없는 상대(거부), 뺀 것만 내놓는 상대(거부), 재협상을 청하는 상대(사양)를 돌렸고, GnuTLS의 클라이언트를 서버에 대조해 돌렸다. 레거시 묶음에 대해서는: 그 조각들을 이 라이브러리 밖에서 계산한 값과, 존재하는 공개 값(10절에 나열)에 대조한다. 양쪽 역할을 OpenSSL과, 서버를 GnuTLS의 클라이언트와 대조해 TLS 1.0, 1.1, 1.2에서 추가된 키 교환과 암호 각각, encrypt_then_mac이 있을 때와 없을 때, 클라이언트 인증서, 확장 마스터 시크릿이 없는 상대를 돌렸다. 그리고 한 걸음만 잘못 디뎌도 키를 내주는 두 곳 - CBC 레코드를 여는 것과 RSA 프리마스터를 되찾는 것(RSA 키 교환의 서버 쪽이 있던 동안) - 을, 복호화된 것을 비밀로 표시하고 아래와 같은 검사기 아래에서 돌렸으며, 아무것도 보고되지 않았다. 그리고 서명 경로와 두 버전의 키 유도 및 레코드 보호를, 비밀 - 소수, 비밀 지수, 블라인딩 값, 공유 키와 거기서 유도된 모든 것 - 에 의존하는 분기나 메모리 인덱스를 보고하는 검사기 아래에서 돌렸고, 한 컴파일러와 한 프로세서 계열의 두 최적화 수준에서 아무것도 보고되지 않았다. **하지 않은 것:** 커버리지 유도 퍼징 없음, 프로토콜 수준 퍼징 스

위트(tlsfuzzer, BoGo) 없음, 하드웨어에서의 시간 측정 없음, 외부 검토 없음. 밑의 기본 연산은 13장의 것이며 거기 적힌 한계가 그대로 있다.

10. 레거시 묶음: 설정 하나하나가 무엇에 노출시키는가

이 절의 모든 것은 **설정이 이름을 들지 않는 한 꺼져 있고**, 그 어느 것도 당신이 원하는 것이 아니다. 이것이 있는 까닭은, 프로그램이 때로는 더 나은 것이 없는 기계 - 아무도 갱신할 수 없는 장비, 남의 네트워크에 있는 서버 - 와 말해야 하고, 그 대안이 연결 없음이나 두 번째 TLS 라이브러리이기 때문이다. RFC 8996은 TLS 1.0과 1.1을 퇴역시켰고 RFC 9325는 나머지를 협상하지 말라고 한다. 양 끝을 모두 통제한다면 저쪽 끝을 바꿔라.

비트 하나가 결정 하나다. 옵션의 `legacy`에 뜻하는 것들을 둔다:

비트	가능해지는 것	포기하는 것
`PROVEN_TLS_LEGACY_CBC`	AES의 CBC 모드로 암호화하고 HMAC-SHA1, HMAC-SHA256 또는 HMAC-SHA384로 인증하는 암호 스위트. 1.2 미만의 모든 버전에 필요하다 - 거기에는 다른 것이 없다	이십 년의 패딩 공격 이력을 가진 구성
`PROVEN_TLS_LEGACY_RSA_KEY_EXCHANGE`	양쪽이 합의하는 대신 클라이언트가 세션의 시크릿을 서버의 RSA 키로 암호화한다	**순방향 비밀성**: 나중에 서버의 키를 얻는 자는 녹음된 모든 연결을 읽는다. 그리고 키 자체가 서버 쪽 실수 하나의 사정거리에 놓일 것이므로, 서버는 거부한다(아래)
`PROVEN_TLS_LEGACY_DHE`	RSA 인증서와 함께 하는, 유한체의 Diffie-Hellman 키 교환	원리상으로는 없다. 실제로는 시간 - 타원곡선 교환보다 몇 배 느리다 - 그리고 대개 다른 면에서도 오래된 상대에게만 있다
`PROVEN_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET`	RFC 7627이 없는 상대와의 연결이 진행된다	세션의 키를 그것을 만든 핸드셰이크에 묶는 것

그리고 그 버전들을 허락하려면 `min_version`을 `PROVEN_TLS_VERSION_1_1` 또는 `PROVEN_TLS_VERSION_1_0`으로 둔다 - 이것에도 CBC 비트가 필요하고, 없으면 거부된다. 이 가운데 아무것도 두지 않는 설정에는 달라지는 것이 없다: 이것들이 있기 전과 같은 hello를 보내고, 같은 답을 하고, 같은 메모리를 붙든다.

핸드셰이크 안에서도 마지막 수단이다. 레거시 스위트는 9절의 모든 스위트 뒤에 내놓고 뒤에 고른다. 더 오래된 버전은 상대에게 더 새 것이 없을 때에만 합의된다. 모든 것을 허락하는 양쪽도 결국 TLS 1.3에 이른다. 그리고 9절의 다운그레이드 검사는 오래된 버전도 덮는다: 1.2를 말할 수 있었으면서 1.0으로 답하게 된 서버는 클라이언트가 확인하는 방식으로 그것을 밝힌다.

CBC 스위트

`ECDHE-ECDSA-`와 `ECDHE-RSA-`에 `AES256-SHA384`, `AES128-SHA256`, `AES128-SHA`, `AES256-SHA`. 그리고 아래의 비트들과 함께라면 `DHE-RSA-` 뒤와 순수 RSA 뒤에 `AES128-SHA256`, `AES256-SHA256`, `AES128-SHA`, `AES256-SHA`. MAC이 SHA-1인 스위트가 TLS 1.0과 1.1이 가진 전부다.

약점은 연산의 순서다. 보내는 쪽은 평문을 인증하고, 패딩하고, 그다음에 암호화한다. 받는 쪽은 복호화하고, 패딩을 벗기고, 그제야 MAC을 확인해야 한다. 한 종류의 나쁜 레코드에 다른 종류보다 조금 빠르거나 조금 다르게 반응하는 수신자는 공격자에게 평문이 무엇이었는지 한 바이트씩 알려 준다(Vaudenay 2002, POODLE, Lucky Thirteen). 이 라이브러리가 하는 일:

- **먼저 해결책을 청한다.** `encrypt_then_mac`(RFC 7366)은 암호문을 인증하므로, 상대가 보내지 않은 것은 복호화되지 않는다. CBC를 내놓을 때마다 내놓고 상대가 동의할 때마다 쓴다. 아직 CBC가 필요한 상대 대부분은 이것이 없다.

- **그것이 없으면, 레코드를 여는 일은 안에 무엇이 있든 같은 일을 한다.** 복호화 뒤의 모든 것 - 패딩을 찾고, 확인하고, MAC을 찾고, 패딩에 따라 달라지는 길이에 대해 계산하고, 비교하는 것 - 은 마스크로 하며, 복호화된 바이트에 의존하는 분기도 메모리 인덱스도 없고, 계산하는 해시 블록의 수는 레코드의 길이에만 의존한다. 모든 실패는 하나의 경고 `bad_record_mac`이다.
- **복호화는 어느 쪽이든 데이터에 따라 달라지는 시간이 걸리지 않는다**: x86에서는 프로세서의 AES 명령으로 - 그러면 CBC 연결은 개발 기계에서 약 100 MiB/s로, 보내는 만큼 빠르게 받는다 - 그 밖의 모든 곳에서는 포터블한 비트슬라이스 암호로, 약 6 MiB/s로.
- **TLS 1.0에서는** 각 레코드의 IV가 앞 레코드의 마지막 블록이므로, 당신이 고른 데이터를 보내게 만들 수 있는 공격자는 IV를 미리 안다(BEAST). 그래서 애플리케이션 데이터는 한 바이트, 그다음 나머지로 보낸다 - 쓰기 한 번에 레코드 둘.

RSA에 의한 키 교환

`AES128-GCM-SHA256`과 `AES256-GCM-SHA384`, 그리고 CBC 비트와 함께 `AES128-SHA256`, `AES256-SHA256`, `AES128-SHA`, `AES256-SHA`. 서버의 키는 RSA여야 한다.

v0.78.0부터 클라이언트만 쓴다. `proven_tls_server_create`는 이 비트가 있는 설정을 거부하고 (`PROVEN_ERR_INVALID_ARG`), 그 위에 지은 모든 서버도 그렇다 - 이벤트 구동 서버는 만들 때. 클라이언트는 오래된 서버나 장비에 닿으려고 쓸 수 있다. 아래의 위험은 어느 것도 클라이언트의 것이 아니다: 클라이언트는 아무것도 복호화하지 않는다. 서버가 쓰지 못하는 까닭:

- **순방향 비밀성이 없다.** 모든 키가 유도되는 시크릿이 서버의 장기 키로 암호화되어 지나간다. 연결의 녹음은 그 키만큼만, 영원히, 안전하다.
- **서버가 실수 하나를 하면 복호화 오라클이 된다.** 서버는 클라이언트가 고른 값을 복호화한다. 그 뒤에 하는 어떤 일 - 경고, 지연 - 이 그 값이 제대로 된 형태로 복호화되었는지에 의존하면, 공격자는 그것을 써서, 한 번에 수천 연결로, 서버의 키로 무엇이든 복호화하거나 *서명*할 수 있다(Bleichenbacher 1998, ROBOT 2017). 이것은 이 키 교환에 한정되지 않는다: 이렇게 위조한 서명은 TLS 1.3에서도 똑같이 통한다. **이 키 교환을 하는 서버의 키는 그것이 쓰이는 모든 프로토콜에 대해 그 위험에 놓인다.** 그리고 그 서버가 받아 줄 클라이언트 - ECDHE도 DHE도 못 하는 것 - 는 아주 오래되었다. v0.78.0 전에는 서버도 켤 수 있었다. 그 코드는 한동안 이 라이브러리의 클라이언트를 시험하는 상대로서 자신의 테스트만 닿는 채 남아 있다가 지워졌다: 이 라이브러리에는 RSA 키 교환을 복호화하는 코드가 없다. 클라이언트 쪽은 다른 구현의 서버와 대조해 돌리고, 그 암호화는 등록된 테스트에서 개인 키로 열어 검사한다.

유한체 Diffie-Hellman

`DHE-RSA-`에 `AES128-GCM-SHA256`, `AES256-GCM-SHA384`, `CHACHA20-POLY1305`, 그리고 CBC 비트와 함께 `AES128-SHA256`, `AES256-SHA256`, `AES128-SHA`, `AES256-SHA`.

DHE에서 잘못된 것은 그룹이었다. 서버가 고르는데 클라이언트는 핸드셰이크가 걸리는 시간 안에 그것을 검사할 수 없다: 너무 작거나(Logjam), 소수가 아니거나, 새도록 만들어졌거나. 그래서 여기서는 어떤 그룹도 검사하지 않는다:

- **클라이언트는 소수가 바이트 하나까지 여섯 중 하나일 때에만 받는다** - RFC 7919와 RFC 3526의 2048, 3072, 4096비트 그룹 - 생성원은 2. 그 밖의 것은 `insufficient_security`로 핸드셰이크를 끝낸다: 스스로 만든 그룹을 쓰는 서버에는, 그 그룹이 아무리 좋아도, 닿을 수 없다.
- **서버는 하나를 내놓는다**: RFC 7919의 2048비트 그룹.
- 상대의 값이 0, 1, 또는 소수에서 1을 뺀 값이면 거부한다. 모든 연결이 새 지수를 쓰며, 그것이 키 유도의 알려진 약점(Raccoon, 2020)을 닿지 않게 한다.
- 개발 기계에서 서명에 더해 한쪽에 약 7 ms의 프로세서 시간이 든다: 2048비트 RSA 키를 가진 서버는 코어당 초당 그런 핸드셰이크 예순다섯 개쯤을 마친다. 그리고 작업용 수들을 스택에 둔다: 핸드셰이크의 가장 깊은 지점에서 약 16 KiB - x86-64의 `-O2`에서 컴파일러 자신의 계산으로.

TLS 1.0과 1.1

`min_version = PROVEN_TLS_VERSION_1_1` 또는 `PROVEN_TLS_VERSION_1_0`, CBC 비트와 함께.

- 핸드셰이크의 무결성이 MD5와 SHA-1 둘에 함께 기대고, 서명은 그것들 위에 만든다: RSA 키는 두 해시를, ECDSA 키는 SHA-1을 서명한다. 어느 해시도 이 라이브러리의 다른 어디에서도 이 용도로 받아들여지지 않는다.
- AEAD 암호가 없다: 모든 스위트가 HMAC-SHA1의 CBC이고, 위에서 말한 모든 것이 따른다.
- **Ed25519 키는 이 버전들을 서비스할 수 없다** - 그 서명을 실을 방법이 없다. 키가 Ed25519인 서버는 더 새것을 말하지 않는 클라이언트를 거부한다.
- 세션은 1.2에서처럼 동작한다(6절): 티켓 하나, **만들어진 그 버전에서만** 재개된다 - TLS 1.0에서 만든 세션은 1.2에 합의하는 핸드셰이크가 재개하지 않는다.
- 클라이언트 인증서는 P-256과 RSA 키로 동작한다.

확장 마스터 시크릿이 없는 상대
RFC 7627(2015)은 세션의 키를 그것을 만든 핸드셰이크에 묶는다. 그것이 없으면 중간의 누군가가 서로 다른 두 연결이 마스터 시크릿을 공유하도록 꾸미고, 한쪽에서 다른 쪽으로 클라이언트의 인증서 증명을 넘길 수 있다(트리플 핸드셰이크 공격). 기본 설정은 그것이 없는 상대를 거부한다. TLS 1.0이나 1.1만 말하는 상대 대부분은 그것이 없다.

`PROVEN_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET`이 있으면 그런 상대와 말하고, 그 연결에서는 공격에 필요한 일들을 하지 않는다: **티켓을 발급하지도 받지도 않고**, 클라이언트는 **인증서를 보이지 않으며**, 서버는 **요청하지 않는다** - 클라이언트 인증서를 요구하는 서버는 그런 클라이언트를 곧바로 거부한다. 그 확장이 있는 상대는 여전히 그것을 쓴다. 이 비트는 그것의 부재를 허락할 뿐이다.

이 가운데 아무것도 쓰지 않는 프로그램이 치르는 것
없다: CBC 스위트에 합의하지 않는 연결은 이 가운데 어느 것도 있기 전에 붙들던 만큼을 붙든다. 한가한 CBC 연결은 약 1.25 KiB 대신 약 1.4 KiB를 붙든다.

어떻게 확인했고, 무엇을 하지 않았는가
등록된 테스트는 모든 조각을 이 라이브러리 밖에서 계산한 값에 대조한다 - AES-CBC는 SP 800-38A에, HMAC은 RFC 2202와 RFC 4231에, 나머지(옛 PRF, 모든 버전과 두 MAC 순서의 CBC 레코드, Diffie-Hellman)는 독립된 구현에 - 그리고 모든 스위트를 그것이 존재하는 모든 버전에서, 양쪽 역할로, 위에서 말한 거부 각각과 함께 돌린다. Diffie-Hellman 소수 여섯은 추가할 때 다른 구현 둘과 비교했다. 등록된 테스트 밖에서는 양쪽 역할을 OpenSSL과, 서버를 GnuTLS와 대조해 돌렸다. 그리고 CBC 레코드를 여는 것과 (서버 쪽이 있던 동안) RSA 프리마스터를 되찾는 것을 각각, 복호화된 바이트를 비밀로 표시하고 검사기 아래에서 돌렸으며, 검사기가 잡아야 하는 일부러 틀린 판도 함께 돌렸다(잡았다).

그리고 시간을 쟀다. 개발 기계의 프로세서에서, 이런 누출을 찾는 방식대로: 같은 연산을 두 종류의 입력에 대해 무작위 순서로 수십만 번 돌리고, 한 번 한 번을 사이클 카운터로 재고, 두 시간 집합을 통계적으로 비교했다. CBC 레코드에 대해 - 세 MAC 각각, 프로세서의 AES와 포터블 AES로 - 패딩이 없는 레코드 대 255바이트인 레코드, 각 패딩 뒤의 틀린 MAC, 틀린 MAC 대 틀린 패딩, 그리고 불가능한 패딩 길이. RSA 프리마스터에 대해(그 코드가 있던 동안) 제대로 된 것 대 잡음, 대 틀린 버전, 대 틀린 길이. 두 번 했다. **되풀이되는 차이를 보인 쌍은 없었다.** 잰 그대로의 전체 실행에 대해서는 검정 통계량이 모든 쌍에서 3 미만에 머물렀다(통상의 기준선은 4.5). 끼어든 방해를 걸러 내려고 관례대로 가장 느린 십분의 일을 빼면, 열 쌍에서 한쪽 실행에서만 4.5에서 7.7 사이로 올랐고 두 실행 모두에서 오른 쌍은 없었다 - 잡음이 하는 일이고, 실제 차이는 하지 않는 일이다. 대조군으로 같은 방식으로 잰 일부러 순진한 레코드 검사는 수천의 통계량을 냈다. 레코드가 열렸는지 자체는, 답에서 그렇듯이, 시간에도 보인다.

그것이 보이지 않는 것. 프로세서 하나, 컴파일러 하나, 다른 일도 하고 있던 기계. 레코드는 쌍마다 400,000회, RSA는 6,000회 - 몇 사이클의 차이는 놓칠 것이다. 측정이지 증명이 아니며, 더 조용한 기계나 다른 프로세서는 이 기계가 보지 못한 것을 볼 수 있다. 이 경로들의 퍼징 없음. 외부 검토 없음.

15장: 이벤트 루프와 이벤트 구동 서버

5부 — 운영체제와 대화하기. 선행 조건: 소켓, 데드라인, 셀렉터를 다루는 9장. 이 장의 서버가 그 다른 형태인 HTTP 서버를 다루는 11장. TLS를 쓰려면 14장. 이 장을 마치면 타이머와 다른 스레드에서 돌아오는 일을 가진 루프를 돌리고, 기다리는 호출 하나 없이 그 위에서 HTTP를 서비스하고, 요청에 나중에 답하고, 프로그램보다 느린 클라이언트에게 응답을 스트리밍하고, 연결마다 스레드를 두지 않고 WebSocket 연결을 열어 두고, 한 스레드에서 많은 HTTP 요청을 한꺼번에 보내고, 서버를 여러 프로세서 코어에 펼치고, 아무 일도 하지 않는 연결 하나가 무엇을 차지하는지 - 그리고 무엇을 측정했는지 - 말할 수 있다.

이 장은 `loop.h`, `http_event.h`, `ws_event.h`, `http_event_client.h`를 다룬다. 모두 호스티드(hosted) 전용이고 `PROVEN_NO_NET`으로 빠진다.

목차

1. 서비스하는 두 가지 방식
2. 루프
3. 타이머
4. 다른 곳에서 하는 일
5. 여러분 자신의 소켓
6. 이벤트 구동 서버
7. 답하기
8. 요청 본문
9. 한도, 시간, 거절
10. 스레드, TLS, 종료
11. 루프 위의 WebSocket
12. 이벤트 구동 클라이언트
13. 여러 루프
14. 연결 하나가 붙드는 것, 측정한 것, 그리고 여기 없는 것

1. 서비스하는 두 가지 방식

11장의 서버는 핸들러에게 요청을 건네고, 기다리는 호출로 읽고 쓰게 한다. 이보다 쓰기 쉬운 것은 없다: 핸들러는 요청에서 응답까지 이어지는 한 줄기다. 그 값은, 처리 중인 요청 하나가 걸리는 시간 내내 - 클라이언트가 느린 시간까지 전부 - 스레드 하나를 차지한다는 것이다.

이 장의 서버는 그것을 뒤집는다. **그 안의 무엇도 기다리지 않는다.** 서버는 이벤트 루프 위에 앉는다. 루프는 어느 소켓에 할 말이 있는지 시스템에 묻고, 그것들을 처리하고, 다시 묻는 스레드 하나다. 여러분의 함수들은 무슨 일이 일어났는지 - 요청이 도착했다, 본문 한 조각이 도착했다, 더 쓸 자리가 생겼다 - 전해 듣고 곧바로 돌아온다. 그러면 아무 일도 하지 않는 연결은 작은 구조체 하나일 뿐 다른 것이 아니다: 스레드도, 스택도, 버퍼도 없다.

그 대신 떠맡는 것은 실제로 있고, 그것이 이 장의 전부다:

- **여러분의 함수는 기다려서는 안 된다** - 소켓도, 파일도, 누군가 오래 쥘 수 있는 락도. 시간이 걸리는 일은 다른 스레드로 가고 그 결과가 게시되어 돌아온다(4절).
- **쓰기는 거절될 수 있다.** 클라이언트가 보낸 것을 가져가지 않으면 서버는 여러분에게서 더 받기를 멈추고, 언제 계속할지 알려 준다(7절).
- **보여 주는 것은 수명이 짧다.** 요청 헤드와 본문 조각은 여러분의 함수가 돌아올 때까지만 유효한 뷰(view)다. 간직할 것은 복사한다.

어느 쪽을 고를까: 동시에 열려 있는 연결 수가 스레드 풀(pool)이 넉넉히 감당하는 정도이고 핸들러를 한 줄기로 쓰는 편이 쉬운 동안에는 블로킹 서버. 대부분의 연결이 대부분의 시간 동안 한가하고 그런 연결이 많을 때 - 롱 폴링, 이벤트 스트림, 느린 클라이언트, 아주 많은 유지 연결 - 는 이 서버. 둘은 10장의 같은 코드로 HTTP를 파싱하고 같은 잘못된 요청을 거절한다.

2. 루프

```
proven_err_t proven_loop_create(proven_allocator_t alloc, proven_loop_t **out);
proven_err_t proven_loop_create_ex(proven_allocator_t alloc, const proven_loop_options_t *options,
proven_loop_t **out);
proven_loop_backend_t proven_loop_backend(const proven_loop_t *loop);
void proven_loop_destroy(proven_loop_t *loop);
proven_err_t proven_loop_run(proven_loop_t *loop);
proven_err_t proven_loop_poll(proven_loop_t *loop, proven_net_deadline_t until);
void proven_loop_stop(proven_loop_t *loop);
```

`proven_loop_t`는 완료되는 소켓 작업들(5절), 타이머 묶음, 다른 스레드가 게시할 수 있는 큐, 그리고 임시 작업용(scratch) 버퍼다. 만들 때 할당자(allocator)를 하나 받는다.

호출	하는 일	반환
`proven_loop_create(alloc, &loop)`	루프를 만든다.	`proven_err_t`: `INVALID_ARG`. `NOMEM`. 셀렉터를 여는 호출이 돌려준 것.
`proven_loop_create_ex(alloc, &options, &loop)`	`options.backend`가 가리키는 백엔드(아래)로 루프를 만든다. `options`는 널이어도 되며, 그러면 `proven_loop_create`와 같다.	`proven_err_t`: `proven_loop_create`와 같고, 이 시스템에 없는 백엔드면 `UNSUPPORTED`.
`proven_loop_backend(loop)`	루프가 어느 백엔드로 만들어졌는지.	`proven_loop_backend_t`. `AUTO`는 돌려주지 않는다.
`proven_loop_run(loop)`	기다리고, 준비된 것을 부르고, 때가 된 것을 울리고, 반복한다 - `proven_loop_stop`까지.	멈춘 뒤 `PROVEN_OK`. 기다리는 일 자체가 실패하면 오류.
`proven_loop_poll(loop, until)`	한 바퀴: 무언가 준비되거나 `until`이 지날 때까지 기다리고 처리한다. `PROVEN_NET_DONT_WAIT`는 지금 준비된 것만 처리한다.	`proven_err_t`.
`proven_loop_stop(loop)`	`proven_loop_run`이 돌아오게 한다. 어느 스레드에서든, 콜백 안에서든 안전하다.	없음.
`proven_loop_destroy(loop)`	루프를 해제한다. 이미 끝난 것은 메모리를 놓을 수 있도록 지금 마지막 답을 받는다 - `proven_loop_close`에 준 소켓의 `closed`, 취소된 수락의 `PROVEN_ERR_CANCELLED`. 그 함수들은 해제만 한다. 그 밖에 아직 붙어 있거나, 진행 중이거나, 맞춰진 것은 잊히고 아무것도 불리지 않는다. 보내려고 기다리던 바이트는 해제된다. 널을 받는다.	없음.

`proven_loop_run`은 다른 일을 하지 않는 스레드를 위한 것이다. `proven_loop_poll`은 이미 자신의 메인 루프가 있어서 그 안에서 이 루프에 차례를 주는 프로그램을 위한 것이다.

백엔드. 루프가 일을 하는 방식은 만들 때 고르며, 그 밖의 것은 고른 것에 따라 바뀌지 않는다. 호출도 같고, 답이 오는 순서의 규칙도 같다.

`proven_loop_backend_t`	어디서	어떻게

PROVEN_LOOP_BACKEND_AUTO	어디서나	Windows에서는 완료 포트, 그 밖에서는 셀렉터 - proven_loop_create가 주는 것.
PROVEN_LOOP_BACKEND_SELECTOR	POSIX	epoll, kqueue, poll(9장): 소켓이 준비됐다는 말을 듣고 루프가 직접 호출한다.
PROVEN_LOOP_BACKEND_URING	Linux 5.19 이상	io_uring: 수신, 송신, accept, connect를 커널에 맡기고 커널이 끝났다고 알린다. 더 오래된 커널이나 시스템이 io_uring을 꺼 둔 곳(컨테이너의 seccomp 필터, io_uring_disabled)에서는 거절된다(UNSUPPORTED). 데이터그램은 다른 백엔드처럼 보통 호출로 보낸다.
PROVEN_LOOP_BACKEND_IOCP	Windows	I/O 완료 포트.

Linux에서 io_uring은 기본값이 아니다. 실제 기계에서 이 라이브러리의 서버가 그것으로 더 빠르다는 측정이 아직 없고, 더 자주 거절되는 백엔드이기 때문이다. io_uring으로 만든 루프는 커널이 아직 쥔 것을 Windows처럼 정리한다 - 루프를 없앨 때 취소하고 최대 2초까지 기다린다. Linux 6.0 이상에서는 proven_loop_read(5절)로 읽는 연결을 커널이 루프의 풀로 받아 둔다 - 기본은 16 KiB 버퍼 64개이고, 그런 읽기가 처음 있을 때 만든다. proven_loop_options_t의 ring_buffers와 ring_buffer_bytes로 바꿀 수 있다. 그래서 0바이트 수신 뒤의 읽기는 시스템 호출이 아니라 복사다. 버퍼는 데이터가 올 때만 쓰이므로 쉬는 연결은 여전히 하나도 쥐지 않는다. 아무도 읽지 않는 연결은 버퍼를 넷까지만 쥐고, 그다음은 다른 백엔드에서처럼 커널의 창이 상대를 붙잡는다.

스레드 규칙. 이 장에서 proven_loop_stop과 proven_loop_post를 뺀 모든 것은 루프 자신의 스레드 - proven_loop_run이나 proven_loop_poll 안에 있는 스레드 - 에서, 또는 루프가 시작하기 전에 불러야 한다. 서버와 뒤 절들의 모든 스트림 함수가 여기에 든다. 대신 검사해 주는 것은 없다.

3. 타이머

```
void proven_loop_timer_set(proven_loop_t *loop, proven_loop_timer_t *timer, proven_u32 ms, proven_loop_fn fn,
void *ctx);
void proven_loop_timer_cancel(proven_loop_t *loop, proven_loop_timer_t *timer);
bool proven_loop_timer_is_set(const proven_loop_timer_t *timer);
```

proven_loop_timer_t는 여러분의 것이다: 그것이 속한 구조체 안에 0으로 초기화해 두고, 맞춰져 있는 동안에는 복사하거나 옮기지 않는다. 맞추는 데 아무것도 할당하지 않는다.

- 타이머는 지금부터 ms 밀리초 뒤에 **한 번 울린다**. 일찍 울리지 않고, POSIX에서는 그 시각의 약 1 ms 안에 울린다. Windows에서는 기다림이 시스템 타이머 간격(약 16 ms)으로 끝난다(v0.70.0 전에는 모든 타이머가 루프의 16 ms 틱으로 올림되었다). 반복해야 하는 타이머는 자기 함수 안에서 스스로 다시 맞춘다.
- 이미 맞춰진 타이머를 맞추면 **옮겨진다**.
- proven_loop_timer_cancel은 어느 콜백 안에서든, 다른 타이머의 콜백 안에서도 안전하고, 맞춰져 있지 않은 타이머에는 해가 없다.
- 맞추기, 취소하기, 울리기는 상수 시간이고, 기다리는 타이머는 때가 될 때까지 들여다보지 않는다. 서버의 모든 연결이 저마다의 타임아웃을 가질 수 있는 것은 그 덕이다.

타이머는 5장의 단조 시계를 잰다. 벽시계를 맞춰도 움직이지 않는다.

4. 다른 곳에서 하는 일

```
proven_err_t proven_loop_post(proven_loop_t *loop, proven_loop_fn fn, void *ctx);
```

proven_loop_post는 루프에게 fn(ctx)를 루프 자신의 스레드에서 곧 불러 달라고 한다. **어느 스레드에서든 안전하고**, 결과가 돌아오는 길이 이것이다: 콜백은 느린 부분을 워커(6장의 job system)에게 넘기고 돌아오며, 워커는 루프의 데이터를 만져도 되는 곳에서 일을 마무리할 함수를 게시한다.

- 함수들은 게시된 순서대로 불린다.
- 큐 항목은 루프의 할당자에서 나오므로, 다른 스레드가 게시한다면 그 할당자는 여러 스레드에서 불러도 되는 것이어야 한다. 힙(heap) 할당자는 그렇다.

• 그 할당이 실패하면 PROVEN_ERR_NOMEM을, 그리고 PROVEN_ERR_INVALID_ARG를 돌려준다.

아래 프로그램은 타이머들과 일 하나만 올라가 있는 루프다. 테스트 스위트가 컴파일하고 실행한다:

```
#include <stdatomic.h>

/*
 * 이벤트 루프 그 자체: 타이머, 그리고 다른 스레드에서 하고 돌아오는 일.
 *
 * 루프의 스레드는 "다음은 무엇인가" 말고는 아무것도 기다리지 않는다. 시간이 걸리는 것 - 여기서는
 * 파일이나 데이터베이스를 대신해 잠을 자는 워커 - 은 다른 곳에서 일어나고, 그 결과는 루프에
 * 게시되어 다른 이벤트들 사이에서 실행된다.
 */

typedef struct {
    proven_loop_t *loop;
    proven_loop_timer_t tick, late;
    int ticks;
    int order[8];
    int count;
    atomic_int from_worker;
} ex_app_t;

static void note(ex_app_t *app, int what) { if (app->count < 8) app->order[app->count++] = what; }

/* 타이머는 한 번 울린다. 반복해야 하는 타이머는 스스로 다시 맞춘다. */
static void on_tick(void *ctx) {
    ex_app_t *app = ctx;
    note(app, 1);
    if (++app->ticks < 3) proven_loop_timer_set(app->loop, &app->tick, 20, on_tick, app);
}

static void on_late(void *ctx) { note(ctx, 9); }

/* 워커 스레드가 요청했지만, 이것은 루프의 스레드에서 실행된다. */
static void on_result(void *ctx) {
    ex_app_t *app = ctx;
    note(app, 5);
    proven_loop_timer_cancel(app->loop, &app->late);      /* 오 초 뒤에 울렸을 것이다. 이제는 영영 울리지 않는
다 */
    proven_loop_stop(app->loop);
}

/* 이것은 워커 스레드에서 실행된다. 얼마든지 오래 걸려도 된다: 루프는 기다리지 않는다. */
static void slow_work(void *ctx) {
    ex_app_t *app = ctx;
    proven_time_sleep(400);
    atomic_store(&app->from_worker, 1);
    if (proven_loop_post(app->loop, on_result, app) != PROVEN_OK) proven_loop_stop(app->loop);   /* 게시는 할
당 실패일 때만 거절된다. 그때는 그냥 실행을 끝낸다 */
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- 루프 ------------------------------------------------------------------
    /* 루프는 완료되는 소켓 작업들, 타이머 묶음, 그리고 다른 스레드가 게시할 수 있는 큐다.
     * 옵션은 루프가 일을 하는 방식을 고른다. AUTO는 proven_loop_create가 주는 것과 같다. */
    EXAMPLE_REQUIRE(proven_loop_create_ex(heap, &(proven_loop_options_t){ .backend =
```

```
PROVEN_LOOP_BACKEND_AUTO }, &app.loop) == PROVEN_OK, "루프");
    EXAMPLE_REQUIRE(proven_loop_backend(app.loop) != PROVEN_LOOP_BACKEND_AUTO, "어느 백엔드가 일하는지 말한
다");

    // ---- 타이머 ------------------------------------------------------------
    /* 타이머 구조체는 여러분의 것이다: 여러분의 상태 안에 살고, 맞추는 데 아무것도 할당하지
     * 않는다. 울리기를 기다리는 타이머도 비용이 없다 - 때가 될 때까지 들여다보지 않는다. */
    proven_loop_timer_set(app.loop, &app.tick, 20, on_tick, &app);
    proven_loop_timer_set(app.loop, &app.late, 5000, on_late, &app);
    EXAMPLE_REQUIRE(proven_loop_timer_is_set(&app.tick) && proven_loop_timer_is_set(&app.late), "타이머 둘이
맞춰졌다");

    // ---- 다른 곳에서 하는 일 ------------------------------------------------
    /* job system이 slow_work를 워커 스레드에서 돌린다. 끝나면 on_result를 루프로 게시한다. */
    proven_job_sys_t *workers = NULL;
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &workers) == PROVEN_OK && proven_job_submit(workers,
slow_work, &app), "워커가 느린 일을 맡았다");

    // ---- 실행 --------------------------------------------------------------
    /* proven_loop_run은 무언가가 proven_loop_stop을 부르면 돌아온다 - 여기서는 on_result가 부른다. */
    proven_time_t began = proven_time_monotonic_now();
    EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK, "루프가 돌았고 멈춰졌다");
    proven_time_t took = proven_time_monotonic_now() - began;

    EXAMPLE_REQUIRE(app.ticks == 3 && app.count == 4 && app.order[0] == 1 && app.order[1] == 1 && app.order[2]
== 1 && app.order[3] == 5,
                    "틱 세 번, 그다음 워커의 결과, 그 순서로");
    EXAMPLE_REQUIRE(atomic_load(&app.from_worker) == 1 && took < 2000000000, "결과는 워커에게서 왔고, 오 초 타
이머보다 훨씬 먼저 왔다");
    EXAMPLE_REQUIRE(!proven_loop_timer_is_set(&app.late), "취소된 타이머는 더는 맞춰져 있지 않다");

    proven_job_system_close(workers);
    proven_job_system_destroy(workers);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

5. 여러분 자신의 소켓

다음 절의 서버는 이것을 대신 해 준다. 이 절은 HTTP가 아닌 프로토콜을 위한 것이다.

루프는 소켓이 준비되었다고 알려 주고 나머지를 여러분에게 맡기지 않는다. 여러분이 **작업** - 수신, 수락, 연결 - 을 시작하면, 그것이 **끝났을 때** 그 함수가 불린다. 보낼 바이트를 루프에 주면 루프가 보낸다. Windows에서는 이것들이 I/O 완료 포트의 작업이다. Linux, BSD, macOS에서는 루프가 9장의 셀렉터(epoll, kqueue) 위에서 같은 일을 한다. 프로그램은 어디서나 같은 호출을 보고, 같은 순서로 답을 받는다.

```
proven_err_t proven_loop_attach(proven_loop_t *loop, proven_loop_sock_t *sock, const proven_net_conn_t *conn,
                                const proven_loop_sock_fns_t *fns, void *ctx);
proven_err_t proven_loop_connect(proven_loop_t *loop, proven_loop_sock_t *sock, proven_net_addr_t to,
proven_net_conn_t *out,
                                 const proven_loop_sock_fns_t *fns, void *ctx);
proven_err_t proven_loop_recv(proven_loop_t *loop, proven_loop_sock_t *sock, proven_mem_mut_t buf);
void proven_loop_recv_cancel(proven_loop_t *loop, proven_loop_sock_t *sock);
proven_result_size_t proven_loop_read(proven_loop_t *loop, proven_loop_sock_t *sock, proven_mem_mut_t buf);
proven_result_size_t proven_loop_send(proven_loop_t *loop, proven_loop_sock_t *sock, proven_mem_view_t bytes);
proven_size_t proven_loop_send_queued(const proven_loop_sock_t *sock);
proven_err_t proven_loop_send_wait(proven_loop_t *loop, proven_loop_sock_t *sock, proven_size_t at_most);
void proven_loop_close(proven_loop_t *loop, proven_loop_sock_t *sock);
proven_err_t proven_loop_accept(proven_loop_t *loop, proven_loop_accept_t *acc, proven_net_listener_t
*listener,
                                proven_loop_accept_fn fn, void *ctx);
```

```
void proven_loop_accept_cancel(proven_loop_t *loop, proven_loop_accept_t *acc);
proven_mem_mut_t proven_loop_scratch(proven_loop_t *loop);
proven_allocator_t proven_loop_allocator(const proven_loop_t *loop);
```

루프에 준 연결은 `proven_loop_sock_t` - 여러분의 것으로, 여러분의 연결 구조체 안에 두고, `proven_loop_attach`부터 그 `closed`까지 옮기지 않는다 - 와 함수 네 개의 표 `proven_loop_sock_fns_t`다. 표는 보통 같은 종류의 연결 모두가 함께 쓰는 `static const` 하나다:

함수	불리는 때
`received(ctx, err, n)`	수신이 끝났다: 데이터가 왔다(0바이트 수신: `PROVEN_OK`, `n`은 0), 또는 여러분의 버퍼에 `n`바이트가 있다. `PROVEN_ERR_EOF`, `PROVEN_ERR_RESET`. `PROVEN_ERR_CANCELLED`.
`writable(ctx, err)`	보내려고 기다리는 바이트가 `proven_loop_send_wait`가 물은 만큼으로 줄었다. 또는 보내기가 실패했다(이때는 대기를 묻지 않았어도). 또는 닫기 때문에 `PROVEN_ERR_CANCELLED`.
`connected(ctx, err)`	`proven_loop_connect`로 시작한 연결이 끝났다.
`closed(ctx)`	마지막: `proven_loop_close` 뒤, 그 소켓의 어떤 것도 진행 중이지 않을 때. 여기서 연결을 닫고 메모리를 해제한다.

호출	하는 일	반환
`proven_loop_attach(loop, &sock, &conn, &fns, ctx)`	연결을 루프에 준다. 아무것도 시작하지 않는다.	`proven_err_t`: `INVALID_ARG`.
`proven_loop_connect(loop, &sock, to, &conn, &fns, ctx)`	연결을 시작한다. 새 연결은 결과와 상관없이 붙고, 어떻게 끝났는지는 `connected`가 말한다.	`proven_err_t`: 곧바로 실패한 연결. 이때는 아무것도 붙지 않는다.
`proven_loop_recv(loop, &sock, buf)`	받는다. `buf.size`가 0이면 "읽을 것이 생기면 알려 달라"다: 아무것도 가져오지 않고, 기다리는 동안 버퍼를 쥐지 않는다. 버퍼를 주면 온 것이 거기 들어 있다.	`proven_err_t`: 수신이 진행 중이면 `BUSY`. 닫은 뒤나 연결이 끝나기 전이면 `INVALID_STATE`.
`proven_loop_recv_cancel(loop, &sock)`	진행 중인 수신을 멈춘다. `PROVEN_ERR_CANCELLED`로 답이 온다.	없음.
`proven_loop_read(loop, &sock, buf)`	온 것을 기다리지 않고 읽는다: 연결에 `PROVEN_NET_DONT_WAIT`로 `proven_net_read`를 하는 것과 같고, io_uring에서는 루프의 풀에서 복사한다(2절). 붙인 연결은 언제나 이것으로 읽는다.	`proven_result_size_t`: 읽은 바이트. `.err`는 지금 아무것도 없으면 `TIMEOUT`, `EOF`, `RESET`.
`proven_loop_send(loop, &sock, bytes)`	보낸다. 기다리는 것이 없으면 바이트는 곧장 소켓으로 간다. 소켓이 바로 받지 않은 것은 루프가 복사해 맡아 두며, 연결마다 64 KiB까지다. 호출이 돌아오면 여러분의 바이트는 다시 자유롭다.	`proven_result_size_t`: 받아 간 바이트 수. 나머지가 들어가지 않았으면 `.err`는 `AGAIN`. 보내기가 실패한 뒤에는 그 실패.
`proven_loop_send_queued(&sock)`	받아 간 바이트 가운데 아직 나가지 않은 수.	`proven_size_t`.
`proven_loop_send_wait(loop, &sock, at_most)`	기다리는 바이트가 `at_most` 이하가 되면 `writable`을 한 번 부르게 한다(0: 모두 나갔다).	`proven_err_t`.

`proven_loop_close(loop, &sock)`	끝: 진행 중인 것은 `PROVEN_ERR_CANCELLED`로 답하고, 아직 기다리던 바이트는 버리고, 그 다음 `closed`.	없음.
`proven_loop_accept(loop, &acc, &listener, fn, ctx)`	연결 하나를 받는다: `fn(ctx, err, &conn)`이 한 번, 연결 - 여러분의 것이니 복사해 가져간다 - 이나 연결이 없는 까닭과 함께 온다. `fn` 안에서 다시 건다. `acc`는 여러분의 것으로, 처음 쓰기 전에 0으로 초기화한다.	`proven_err_t`: 수락이 진행 중이면 `BUSY`.
`proven_loop_accept_cancel(loop, &acc)`	수락을 멈춘다: 진행 중인 것은 `PROVEN_ERR_CANCELLED`로 답하고, 리스너를 닫아도 된다.	없음.
`proven_loop_scratch(loop)`	루프의 임시 작업용 버퍼, 64 KiB.	`proven_mem_mut_t`.
`proven_loop_allocator(loop)`	루프를 만들 때 준 할당자.	`proven_allocator_t`.

이것을 지탱하는 규칙:

- **시작한 작업은 모두 정확히 한 번 답을 받는다.** 그 함수가 루프의 스레드에서 불린다 - 그리고 곧바로 끝날 수 있었더라도, 그것을 시작한 호출 안에서는 결코 불리지 않는다. 성공, 실패, 취소가 모두 이 길로 오므로, 여러분의 함수는 여러분이 루프를 부르는 호출 안에서 돌지 않는다.
- **소켓마다 수신은 한 번에 하나, 수락기마다 수락은 한 번에 하나.** 송신은 여러분의 작업이 아니다: 루프가 바이트를 받아 간다.
- **끝은 `proven_loop_close`, 그다음 `closed`다.** `closed`까지는 루프가 소켓의 메모리를 아직 쓸 수 있다. 그 뒤로는 결코 건드리지 않는다. 연결 자체는 `closed`에서 닫는다. 점잖게 끝내려면 - 모두 먼저 보내고 - `proven_loop_send_wait(loop, &sock, 0)`으로 기다렸다가 `writable`에서 닫는다.
- **0바이트 수신은 쉬는 연결 만 개가 읽기 버퍼를 하나도 쓰지 않는 방법이다.** 답이 오면 `proven_loop_read`로 임시 작업용 버퍼에 읽어 들이고, 온 것을 쓰고, 아직 쓸 수 없는 것만 간직한다. 함수가 돌아오거나, 그 버퍼를 쓸 수도 있는 다른 것을 부르고 나면 내용은 아무 뜻이 없다.
- **느린 상대를 위해 루프가 맡아 두는 것에 한도를 둔다.** `proven_loop_send_queued`가 많이 기다린다고 하면, 읽기를 멈추고 `proven_loop_send_wait`에 언제 다시 가면 되는지 묻는다 - 예제가 그렇게 한다.
- 연결을 붙인 뒤로는 바이트가 순서를 지키도록 `proven_loop_read`로 읽고 `proven_loop_send`로 보낸다. io_uring에서는 소켓에 더는 없는 바이트를 루프의 풀이 쥐고 있을 수 있다. 연결은 한 루프에, 한 번만 준다.

아래 프로그램은 루프 하나 위의 에코 서버와 그 클라이언트다. 테스트 묶음이 컴파일하고 실행한다:

```
#include <string.h>

/*
 * 루프 하나 위의 에코 서버와 그 클라이언트. 루프의 작업만 쓴다.
 *
 * 서버는 자기 함수 안에서 다시 거는 수락으로 연결을 받고, 연결마다 0바이트 수신으로 데이터를
 * 기다린다: 쉬는 연결은 읽기 버퍼를 쥐지 않는다. 데이터가 오면 루프의 scratch 버퍼로 읽어 되돌려
 * 보낸다. 소켓이 바로 받지 않은 것은 루프가 맡아 둔다. 클라이언트도 루프를 통해 연결하고, 타이머
 * 아래에서 자기 버퍼로 받는 수신으로 읽는다.
 */

#define MESSAGE "every operation is answered exactly once"
#define ROUNDS 3

typedef struct ex_app ex_app_t;
```

```
typedef struct {                /* 서버의 연결 하나 */
    ex_app_t *app;
    proven_loop_sock_t sock;
    proven_net_conn_t conn;
    bool ending, used;
} ex_peer_t;

struct ex_app {
    proven_loop_t *loop;
    proven_net_listener_t listener;
    proven_loop_accept_t acc;
    ex_peer_t peers[4];
    int peers_closed;
    /* 클라이언트 */
    proven_loop_sock_t sock;
    proven_net_conn_t conn;
    proven_loop_timer_t deadline;
    proven_byte_t buf[64];
    proven_size_t have;
    int rounds;
    bool echoed, client_closed, accepting_ended;
};

// ---- 서버의 연결 ------------------------------------------------------------

static void peer_end(ex_peer_t *p) {
    /* 클라이언트가 할 말을 다 했다: 보낼 것이 모두 나간 뒤에 닫는다. */
    p->ending = true;
    if (proven_loop_send_wait(p->app->loop, &p->sock, 0) != PROVEN_OK) proven_loop_close(p->app->loop, &p-
>sock);
}

/* 온 것을 읽어 되돌려 보낸다. 읽는 것보다 빨리 보내는 클라이언트는 루프가 맡아 둘 것을 끝없이
 * 늘리므로, 기다리는 것이 32 KiB를 넘으면 서버는 그것이 따라잡을 때까지 읽기를 멈춘다: 송신 대기를
 * 걸고, 읽기는 `writable`에서 다시 시작한다. */
static void peer_read(ex_peer_t *p) {
    proven_loop_t *loop = p->app->loop;
    proven_mem_mut_t scratch = proven_loop_scratch(loop);
    scratch.size = 16384;
    for (;;) {
        if (proven_loop_send_queued(&p->sock) > 32768) {
            if (proven_loop_send_wait(loop, &p->sock, 0) != PROVEN_OK) proven_loop_close(loop, &p->sock);
            return;
        }
        proven_result_size_t r = proven_loop_read(loop, &p->sock, scratch);
        if (r.err == PROVEN_ERR_TIMEOUT) break;                 /* 다 읽었다: 더 오기를 기다린다 */
        if (r.err != PROVEN_OK) { peer_end(p); return; }        /* PROVEN_ERR_EOF: 클라이언트가 끝냈다 */
        proven_result_size_t s = proven_loop_send(loop, &p->sock, (proven_mem_view_t){ scratch.ptr,
r.value });
        if (s.err != PROVEN_OK) { proven_loop_close(loop, &p->sock); return; }
    }
    /* 이 연결은 기다리는 동안 아무것도 쥐지 않는다: 0바이트 수신은 버퍼를 받지 않는다. */
    if (proven_loop_recv(loop, &p->sock, (proven_mem_mut_t){ 0 }) != PROVEN_OK) proven_loop_close(loop, &p-
>sock);
}

static void peer_received(void *ctx, proven_err_t err, proven_size_t n) {
    ex_peer_t *p = ctx;
    (void)n;                                                    /* 0바이트 수신은 아무것도 가져오지 않는다 */
```

```
    if (err != PROVEN_OK) return;                                   /* PROVEN_ERR_CANCELLED: 닫히는 중이다 */
    peer_read(p);
}

static void peer_writable(void *ctx, proven_err_t err) {
    ex_peer_t *p = ctx;
    if (err == PROVEN_ERR_CANCELLED) return;
    if (err != PROVEN_OK || p->ending) proven_loop_close(p->app->loop, &p->sock);
    else peer_read(p);
}

static void peer_closed(void *ctx) {
    ex_peer_t *p = ctx;
    (void)proven_net_close(&p->conn);                               /* 연결 자체는 여기서, 마지막으로 닫는다 */
    p->used = false;
    /* 이 프로그램의 단 하나뿐인 클라이언트가 갔다: 연결 받기를 멈춘다. */
    if (++p->app->peers_closed == 1) proven_loop_accept_cancel(p->app->loop, &p->app->acc);
}

static const proven_loop_sock_fns_t peer_fns = { .received = peer_received, .writable = peer_writable, .closed
= peer_closed };

static void on_accept(void *ctx, proven_err_t err, proven_net_conn_t *conn) {
    ex_app_t *app = ctx;
    if (err != PROVEN_OK) {                                         /* PROVEN_ERR_CANCELLED: 이제 리스너를 닫아도
된다 */
        (void)proven_net_listener_close(&app->listener);
        app->accepting_ended = true;
        return;
    }
    ex_peer_t *p = NULL;
    for (int i = 0; i < 4 && !p; ++i) if (!app->peers[i].used) p = &app->peers[i];
    if (!p) { (void)proven_net_close(conn); }
    else {
        *p = (ex_peer_t){ .app = app, .conn = *conn, .used = true };    /* 이제 이 연결은 우리 것이다 */
        if (proven_loop_attach(app->loop, &p->sock, &p->conn, &peer_fns, p) != PROVEN_OK ||
            proven_loop_recv(app->loop, &p->sock, (proven_mem_mut_t){ 0 }) != PROVEN_OK)
{ (void)proven_net_close(&p->conn); p->used = false; }
    }
    /* 안에서 다시 건다: 이미 기다리는 연결은 한 바퀴를 더 돌지 않고 받는다. */
    if (proven_loop_accept(app->loop, &app->acc, &app->listener, on_accept, app) != PROVEN_OK) app-
>accepting_ended = true;
}

// ---- 클라이언트 ------------------------------------------------------------------

static void client_deadline(void *ctx) {
    ex_app_t *app = ctx;
    proven_loop_recv_cancel(app->loop, &app->sock);             /* 그 `received`가 PROVEN_ERR_CANCELLED로 온다
*/
}

static void client_send(ex_app_t *app) {
    app->have = 0;
    proven_result_size_t s = proven_loop_send(app->loop, &app->sock, (proven_mem_view_t){ (const proven_byte_t
*)MESSAGE, strlen(MESSAGE) });
    /* 클라이언트 자신의 버퍼로 받는 수신. 버퍼는 `received`까지 루프의 것이다. */
    if (s.err != PROVEN_OK || proven_loop_recv(app->loop, &app->sock, (proven_mem_mut_t){ app->buf, sizeof
app->buf }) != PROVEN_OK) {
        proven_loop_close(app->loop, &app->sock);
```

```
        return;
    }
    proven_loop_timer_set(app->loop, &app->deadline, 5000, client_deadline, app);
}

static void client_received(void *ctx, proven_err_t err, proven_size_t n) {
    ex_app_t *app = ctx;
    if (err != PROVEN_OK) { proven_loop_close(app->loop, &app->sock); return; }    /* 기한이 지났거나 서버가
갔다 */
    app->have += n;
    if (app->have < strlen(MESSAGE)) {                          /* 스트림이다: 메아리가 조각으로 올 수 있다 */
        if (proven_loop_recv(app->loop, &app->sock, (proven_mem_mut_t){ app->buf + app->have, sizeof app->buf
- app->have }) != PROVEN_OK)
            proven_loop_close(app->loop, &app->sock);
        return;
    }
    proven_loop_timer_cancel(app->loop, &app->deadline);
    if (memcmp(app->buf, MESSAGE, strlen(MESSAGE)) != 0) { proven_loop_close(app->loop, &app->sock); return; }
    if (++app->rounds < ROUNDS) { client_send(app); return; }
    app->echoed = true;
    proven_loop_close(app->loop, &app->sock);                   /* 서버가 끝을 읽고 자기 쪽을 닫는다 */
}

static void client_connected(void *ctx, proven_err_t err) {
    ex_app_t *app = ctx;
    if (err != PROVEN_OK) { proven_loop_close(app->loop, &app->sock); return; }
    client_send(app);
}

static void client_closed(void *ctx) {
    ex_app_t *app = ctx;
    (void)proven_net_close(&app->conn);
    app->client_closed = true;
}

static const proven_loop_sock_fns_t client_fns = {
    .received = client_received, .connected = client_connected, .closed = client_closed
};

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static ex_app_t app;
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "루프");

    // ---- 서버 ------------------------------------------------------------------
    proven_net_addr_t bound;
    EXAMPLE_REQUIRE(proven_net_listen(proven_net_addr_ipv4(127, 0, 0, 1, 0), 16, &app.listener, &bound) ==
PROVEN_OK, "빈 포트의 리스너");
    EXAMPLE_REQUIRE(proven_loop_accept(app.loop, &app.acc, &app.listener, on_accept, &app) == PROVEN_OK, "수락
시작");

    // ---- 클라이언트 ------------------------------------------------------------
    EXAMPLE_REQUIRE(proven_loop_connect(app.loop, &app.sock, bound, &app.conn, &client_fns, &app) ==
PROVEN_OK, "연결 시작");

    // ---- 모든 것이 끝날 때까지 돈다 ---------------------------------------------
    proven_net_deadline_t end = proven_net_deadline_in(10000);
    while (!(app.client_closed && app.accepting_ended) && proven_time_monotonic_now() < end) {
        EXAMPLE_REQUIRE(proven_loop_poll(app.loop, proven_net_deadline_in(100)) == PROVEN_OK, "한 바퀴");
    }
```

```
    EXAMPLE_REQUIRE(app.echoed && app.rounds == ROUNDS, "메시지가 세 번 되돌아왔다");
    EXAMPLE_REQUIRE(app.peers_closed == 1 && app.accepting_ended && !
proven_net_listener_is_open(&app.listener),
                    "서버는 클라이언트 다음에 자기 쪽을 닫고 수락을 멈췄다");
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

6. 이벤트 구동 서버

```
proven_err_t proven_http_event_server_create(proven_loop_t *loop, const proven_http_event_server_config_t
*config,
                                             proven_http_event_server_t **out);
proven_err_t proven_http_event_server_listen(proven_http_event_server_t *server, proven_net_addr_t at,
proven_net_addr_t *bound);
void proven_http_event_server_destroy(proven_http_event_server_t *server);
proven_size_t proven_http_event_server_connections(const proven_http_event_server_t *server);
proven_err_t proven_http_event_server_set_max_streams(proven_http_event_server_t *server, proven_size_t n);
proven_err_t proven_http_event_server_set_window(proven_http_event_server_t *server, proven_size_t n);
void proven_http_event_server_stop_listening(proven_http_event_server_t *server);
typedef void (*proven_http_event_drained_fn)(void *ctx, proven_err_t why);
proven_err_t proven_http_event_server_shutdown(proven_http_event_server_t *server, proven_u32 grace_ms,
                                               proven_http_event_drained_fn on_drained, void *ctx);
```

서버는 루프 위에 만들어지고, 어디서 들을지 지시받고, 루프가 돌 때까지는 아무 일도 하지 않는다.

호출	하는 일	반환
`proven_http_event_server_create(loop, &config, &server)`	서버를 만든다.	`proven_err_t`: 루프가 없거나, `on_request`가 없거나, TLS 설정에 인증서가 없으면 `INVALID_ARG`. `NOMEM`.
`proven_http_event_server_listen(server, at, &bound)`	`at`에서 듣는다. 서버 하나에 주소 넷까지. `bound`(선택)는 쓰이는 주소를 받는다 - `at`이 포트 0을 청했다면 시스템이 고른 포트와 함께.	`proven_err_t`.
`proven_http_event_server_connections(server)`	지금 열려 있는 연결.	`proven_size_t`.
`proven_http_event_server_set_max_streams(server, n)`	서버가 도는 동안 `max_concurrent_streams`를 바꾼다: 1부터 설정값까지, 열린 연결과 앞으로 올 연결에 (17장 6절과 9절). 루프의 스레드에서.	`proven_err_t`: `INVALID_ARG`.
`proven_http_event_server_set_window(server, n)`	처음부터, 클라이언트가 처리기보다 앞서 보낼 수 있는 요청 본문의 천장 - 스트림당, 연결당, HTTP/2와 HTTP/3 - 을 열린 연결과 앞으로 올 연결에 둔다. 이미 준 것은 남고, 자라는 창은 거기서 멈춘다(17장 6절). 루프의 스레드에서.	`proven_err_t`: 0이나 2^31-1보다 크면 `INVALID_ARG`.
`proven_http_event_server_stop_listening(server)`	새 연결 받기를 멈춘다. 열린 연결은 계속되지만, 이제부터 HTTP/1.1 응답은 `Connection:`	없음.

	close를 말하고 HTTP/2는 작별한다.	
proven_http_event_server_shutdown(server, grace_ms, on_drained, ctx)	부드럽게 끝낸다(10절). 끝나면 on_drained(ctx, why).	proven_err_t: 이미 종료 중이면 INVALID_STATE.
proven_http_event_server_destroy(server)	모든 연결과 리스너를 닫고 서버를 해제한다. 진행 중인 교환은 PROVEN_ERR_RESET과 함께 on_done을 받는다. 루프는 멈추지도 해제되지도 않는다. 루프가 아직 답하지 않은 것 - 진행 중인 수락, 연결의 소켓 - 은 루프의 다음 바퀴나 proven_loop_destroy까지 메모리를 쥔다(5절 규칙 3). 널을 받는다.	없음.

함수 넷. 설정이 on 안에 싣고, ctx가 각각에 전달된다:

함수	언제	비고
on_request(ctx, stream, head)	요청 헤드가 도착해 서버의 검사를 통과했다.	필수. 여기서 답하거나, stream을 기억해 두고 나중에 답한다. head는 이 함수가 돌아올 때까지만 유효하다.
on_body(ctx, stream, piece, last)	요청 본문의 한 조각. 마지막 조각에서 last가 참이고, 그 조각은 비어 있을 수 있다.	선택: 없으면 본문은 읽혀서 버려진다. piece는 이 함수가 돌아올 때까지만 유효하다.
on_writable(ctx, stream)	보류되었던 출력이 나갔고, proven_http_stream_write가 더 받는다.	선택.
on_done(ctx, stream, why)	교환이 끝났다.	선택. **on_request마다 정확히 한 번** 불린다: 응답 전체가 연결에 넘겨졌으면 PROVEN_OK, 아니면 그러지 못한 이유.

스트림은 요청 하나와 그 응답이다. proven_http_stream_t *는 on_request부터 on_done이 돌아올 때까지 유효하고, 그보다 한순간도 더 길지 않다. 그러므로 on_done은 스트림에 붙여 둔 것을 해제하고, 아직 스트림을 쥐고 있는 타이머를 취소하거나 일을 잊게 할 단 하나의 자리다: 끝난 스트림을 들고 나중에 도착하는 함수에게는 부를 수 있는 유효한 것이 없다.

```
void proven_http_stream_set_user(proven_http_stream_t *stream, void *user);
void *proven_http_stream_user(const proven_http_stream_t *stream);
proven_net_addr_t proven_http_stream_peer(const proven_http_stream_t *stream);
```

proven_http_stream_set_user는 여러분의 포인터를 스트림에 붙이고 proven_http_stream_user가 그것을 돌려준다 - 아무것도 받지 않은 스트림은 널을 돌려주며, 해제하는 on_done은 그 경우를 받아들여야 한다. proven_http_stream_peer는 클라이언트의 주소다.

7. 답하기

```
proven_err_t proven_http_stream_respond(proven_http_stream_t *stream, proven_u16 status,
                                        const proven_http_header_t *headers, proven_size_t header_count,
                                        proven_mem_view_t body);
proven_err_t proven_http_stream_begin(proven_http_stream_t *stream, proven_u16 status,
                                      const proven_http_header_t *headers, proven_size_t header_count,
                                      proven_u64 content_length);
```

```
proven_result_size_t proven_http_stream_write(proven_http_stream_t *stream, proven_mem_view_t data);
proven_err_t proven_http_stream_end(proven_http_stream_t *stream);
void proven_http_stream_abort(proven_http_stream_t *stream);
proven_size_t proven_http_stream_buffered(const proven_http_stream_t *stream);
```

전부 한 번에. `proven_http_stream_respond`는 상태, 여러분의 헤더, 메모리에 든 본문을 보낸다. 본문은 크기가 얼마든 복사되므로 작은 본문을 위한 것이다. `Content-Length`, `Date`, `Connection`은 대신 써 주며 `headers`에 들어 있어서는 안 된다. `on_request` 안에서, 또는 스트림이 유효한 동안 그 뒤 어느 때든 루프의 스레드에서 부를 수 있다.

조각조각. `proven_http_stream_begin`은 헤드를 보내고 `content_length` 바이트의 본문을, 또는 - `PROVEN_HTTP_EVENT_LENGTH_UNKNOWN`을 주면 - 아직 아무도 길이를 모르는, chunked로 보내는 본문을 약속한다. 그다음 `proven_http_stream_write`가 바이트를 내놓고 `proven_http_stream_end`가 끝낸다.

`proven_http_stream_write`는 내놓은 것보다 적게 받을 수 있다 - `max_buffered_output` 아래에 들어가는 만큼, 어쩌면 하나도 - 그리고 몇 바이트를 받았는지 돌려준다. 이것은 오류가 아니다. 이미 보낸 것을 클라이언트가 가져가지 않았다는 뜻이고, 규칙은 이렇다: **나머지를 간직하고, 돌아오고, `on_writable`이 불리면 다시 내놓는다.** 그렇게 쓴 생산자는 가장 느린 클라이언트와 정확히 같은 속도로 가고, 그 연결에 쓰이는 서버의 메모리는 응답 크기와 무관하게 한도 아래에 머문다.

호출	반환
`proven_http_stream_respond`, `proven_http_stream_begin`	`proven_err_t`: 응답이 이미 시작되었거나 스트림이 끝났으면 `INVALID_STATE`. 상태가 200-999 밖이거나 서버가 직접 쓰는 헤더가 있으면 `INVALID_ARG`. 응답 헤드가 `max_head_bytes`에 들어가지 않으면 `OUT_OF_BOUNDS`. `NOMEM`.
`proven_http_stream_write`	`proven_result_size_t`: 받은 바이트 수. `begin` 전, `end` 후, 또는 스트림이 끝났으면 `INVALID_STATE`. 약속한 `Content-Length`를 넘으면 `OUT_OF_BOUNDS`.
`proven_http_stream_flush`	`proven_err_t`: 압축 중인 응답이 붙들고 있는 것을 내보낸다(16장 5절). 다른 응답에는 아무 일도 하지 않는다. `begin` 전, `end` 후, 또는 스트림이 끝났으면 `INVALID_STATE`. 그러다 연결이 끝났으면 `RESET`.
`proven_http_stream_end`	`proven_err_t`: 응답이 시작되지 않았거나 이미 끝났으면 `INVALID_STATE`. 약속보다 적게 썼으면 `INVALID_FORMAT` - 그때 연결은 닫힌다. 클라이언트가 모자란 본문을 온전한 것으로 받아들이지 못하게 하기 위해서다.
`proven_http_stream_abort`	없음. 연결은 곧바로 닫히고 `on_done`이 `PROVEN_ERR_RESET`과 함께 불린다.
`proven_http_stream_buffered`	`proven_size_t`: 이 연결을 위해 붙들고 있고 클라이언트가 아직 받지 않은 응답 바이트.
`proven_http_stream_compress`	없음. 클라이언트가 gzip을 받아들이면 이 응답을 압축해 달라고 요청한다. 응답을 시작하기 전에. 16장 6절 - 그 절의 주의할 점을 먼저 읽을 것.

프로토콜에서 따라 나오고 여러분에게 아무것도 요구하지 않는 세 가지: `HEAD`에 대한 응답과 상태 204, 304의 응답은 무엇을 넘기든 본문 없이 나간다. `Expect: 100-continue`라고 한 요청은 본문이 처음 필요해질 때 계속하라는 말을 듣는다. 그리고 닫기를 청한 요청이나 끝까지 읽을 수 없었던 요청에 응답한 뒤에는 연결을 유지하지 않고 닫는다.

모든 것이 한 번에 소켓에 들어갈 때, `on_done`은 `proven_http_stream_respond`나 `proven_http_stream_end` **안에서** 불릴 수 있다. 둘 중 하나가 돌아온 뒤에는 스트림도, `on_done`이 해제한 것도 건드리지 않는다.

아래 프로그램은 네 가지 방식으로 답한다 - 즉시, 나중에 타이머에서, 거절당하면 물러서는 펌프로 스트리밍해서, 그리고 조각조각 받은 본문 뒤에. 테스트 스위트가 컴파일하고 실행한다:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * 이벤트 루프 위의 HTTP 서버: 스레드 하나, 그리고 그 안에 기다리는 호출은 없다.
 *
 * 서버가 이 함수들에게 무슨 일이 일어났는지 알려 주고, 함수들은 곧바로 돌아온다. 답하는 방식
 * 네 가지를 보인다: 즉시. 나중에 타이머에서. 스트리밍하되 클라이언트가 느리면 물러서며. 그리고
 * 요청 본문이 조각조각 도착한 뒤에.
 *
 * 클라이언트는 11장의 블로킹 클라이언트이고, 예제의 메인 스레드에서 돈다.
 */

typedef struct {
    proven_loop_t *loop;
    int served;
    proven_http_event_server_t *server;
    proven_err_t ended;               /* on_drained가 들은 것 */
} ex_app_t;

/* 요청 하나에 답하는 동안 이 프로그램이 간직하는 것. */
typedef struct {
    ex_app_t *app;
    proven_http_stream_t *stream;
    proven_loop_timer_t timer;
    proven_size_t sent, total;        /* /count: how far the body has got */
    proven_size_t part;               /* /count: 지금 줄에서 이미 나간 만큼 */
    proven_size_t received;           /* /size: body bytes seen */
} ex_exchange_t;

/* 서버가 "지금은 그만"이라고 할 때까지 줄을 쓴다. 이미 보낸 것을 클라이언트가 가져가지
 * 않았을 때 그렇게 말한다. 나머지는 on_writable을 기다린다. */
static void count_pump(ex_exchange_t *x) {
    while (x->sent < x->total) {
        char line[32];
        int n = snprintf(line, sizeof line, "%u\n", (unsigned)x->sent);
        proven_result_size_t w = proven_http_stream_write(x->stream, (proven_mem_view_t){ (const proven_byte_t
*)line + x->part, (proven_size_t)n - x->part });
        if (w.err != PROVEN_OK) return;                  /* 클라이언트가 떠났다 */
        x->part += w.value;                              /* 쓰기는 받은 것의 일부만 가져갈 수 있다 */
        if (x->part < (proven_size_t)n) return;          /* 보류됨: 언제 되는지는 on_writable이 알려 준다 */
        x->part = 0;
        x->sent++;
    }
    (void)proven_http_stream_end(x->stream);
}

static void answer_later(void *ctx) {
    ex_exchange_t *x = ctx;
    (void)proven_http_stream_respond(x->stream, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("worth the
wait\n")));
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    ex_exchange_t *x = calloc(1, sizeof *x);
```

```
    if (!x) { proven_http_stream_abort(stream); return; }
    x->app = app;
    x->stream = stream;
    proven_http_stream_set_user(stream, x);

    /* `head`와 그 안의 모든 것은 이 함수가 돌아올 때까지만 유효하다. */
    if (proven_u8str_view_eq(head->target, PROVEN_LIT("/"))) {
        /* 가장 단순한 답: 전부, 지금. */
        (void)proven_http_stream_respond(stream, 200, NULL, 0,
proven_mem_view_from_u8(PROVEN_LIT("hello\n")));
    } else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/slow"))) {
        /* 지금은 아니다: 스트림을 기억해 두고 타이머에서 답한다. 그동안 루프는 계속 돈다. */
        proven_loop_timer_set(app->loop, &x->timer, 50, answer_later, x);
    } else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/count"))) {
        /* 만들어 가며 보내는 본문. 길이를 미리 알 수 없으므로 chunked로 보낸다. */
        x->total = 20000;
        if (proven_http_stream_begin(stream, 200, NULL, 0, PROVEN_HTTP_EVENT_LENGTH_UNKNOWN) == PROVEN_OK)
count_pump(x);
    } else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/size"))) {
        /* 아직 없다: 본문이 오는 중이고, 다 오면 on_body가 답한다. */
    } else {
        (void)proven_http_stream_respond(stream, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("not
here\n")));
    }
}

static void on_body(void *ctx, proven_http_stream_t *stream, proven_mem_view_t piece, bool last) {
    (void)ctx;
    ex_exchange_t *x = proven_http_stream_user(stream);
    x->received += piece.size;
    if (last) {
        char text[32];
        int n = snprintf(text, sizeof text, "%u bytes\n", (unsigned)x->received);
        (void)proven_http_stream_respond(stream, 200, NULL, 0, (proven_mem_view_t){ (const proven_byte_t
*)text, (proven_size_t)n });
    }
}

/* 보류되었던 출력이 나갔다: 쓰기가 멈춘 자리에서 이어 간다. */
static void on_writable(void *ctx, proven_http_stream_t *stream) {
    (void)ctx;
    ex_exchange_t *x = proven_http_stream_user(stream);
    if (x->total > 0) count_pump(x);
}

/* 무슨 일이 있었든 on_request마다 한 번 불린다. 붙여 둔 것을 해제할 자리. */
static void on_done(void *ctx, proven_http_stream_t *stream, proven_err_t why) {
    ex_app_t *app = ctx;
    ex_exchange_t *x = proven_http_stream_user(stream);
    if (!x) return;
    proven_loop_timer_cancel(app->loop, &x->timer);
    free(x);
    if (why == PROVEN_OK) app->served++;
}

/* 끝, 루프의 스레드에서: 열린 것이 끝나고, 그다음 루프를 멈춘다. */
static void on_drained(void *ctx, proven_err_t why) {
    ex_app_t *app = ctx;
    app->ended = why;
    proven_loop_stop(app->loop);
```

```
}
static void begin_end(void *ctx) {
    ex_app_t *app = ctx;
    if (proven_http_event_server_shutdown(app->server, 2000, on_drained, app) != PROVEN_OK)
proven_loop_stop(app->loop);
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

static char g_url[64];
static proven_u8str_view_t url_for(proven_u16 port, const char *path) {
    int n = snprintf(g_url, sizeof g_url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    return (proven_u8str_view_t){ (const proven_byte_t *)g_url, (proven_size_t)n };
}

static bool body_equals(proven_http_client_response_t *resp, proven_allocator_t alloc, const char *want) {
    proven_u8str_t body = { 0 };
    bool ok = proven_http_client_read_all(resp, alloc, &body, 1024 * 1024) == PROVEN_OK &&
              proven_u8str_view_eq(proven_u8str_as_view(&body), (proven_u8str_view_t){ (const proven_byte_t
*)want, strlen(want) });
    proven_u8str_destroy(alloc, &body);
    return ok;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- 서버 ----------------------------------------------------------------
    /* 서버는 루프 위에 만들어지고 듣는다. 루프가 돌 때까지는 아무 일도 하지 않는다. */
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "루프");
    proven_http_event_server_config_t config = {
        .on = { on_request, on_body, on_writable, on_done },
        .ctx = &app,
        .max_buffered_output = 16 * 1024,        /* 클라이언트가 가져가지 않은 것을 연결마다 최대 이만큼만 붙
든다 */
    };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "빈 포트의 이벤트 구동 서버");
    app.server = server;

    /* 이 예제는 메인 스레드가 클라이언트 노릇을 할 수 있도록 루프에 따로 스레드를 준다. */
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "루프가 다른 스레드에
서 돈다");

    // ---- 요청 넷 -------------------------------------------------------------
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 1 };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "클라이언트");
    proven_http_client_response_t resp;

    EXAMPLE_REQUIRE(proven_http_client_get(client, url_for(at.port, "/"), &resp) == PROVEN_OK &&
body_equals(&resp, heap, "hello\n"), "즉시 답했다");
```

```
    proven_http_client_finish(&resp);
    EXAMPLE_REQUIRE(proven_http_client_get(client, url_for(at.port, "/slow"), &resp) == PROVEN_OK &&
body_equals(&resp, heap, "worth the wait\n"), "50 ms 뒤에 타이머에서 답했다");
    proven_http_client_finish(&resp);

    /* 이만 줄. 서버가 보류할 때마다 핸들러는 멈추고 on_writable이 재개시킨다. */
    EXAMPLE_REQUIRE(proven_http_client_get(client, url_for(at.port, "/count"), &resp) == PROVEN_OK &&
resp.status == 200, "스트리밍 응답이 시작된다");
    proven_u8str_t counted = { 0 };
    EXAMPLE_REQUIRE(proven_http_client_read_all(&resp, heap, &counted, 1024 * 1024) == PROVEN_OK, "끝까지 읽힌
다");
    proven_u8str_view_t all = proven_u8str_as_view(&counted);
    EXAMPLE_REQUIRE(all.size == 108890 && memcmp(all.ptr, "0\n1\n2\n", 6) == 0 && memcmp(all.ptr + all.size -
6, "19999\n", 6) == 0, "모든 줄이 순서대로 도착했다");
    proven_u8str_destroy(heap, &counted);
    proven_http_client_finish(&resp);

    static proven_byte_t upload[50000];
    proven_http_client_request_t post = { .method = PROVEN_LIT("POST"), .url = url_for(at.port, "/
size"), .body = { upload, sizeof upload } };
    EXAMPLE_REQUIRE(proven_http_client_send(client, &post, &resp) == PROVEN_OK && body_equals(&resp, heap,
"50000 bytes\n"), "조각조각 센 본문");
    proven_http_client_finish(&resp);
    proven_http_client_destroy(client);

    // ---- 끝 ----------------------------------------------------------------------
    /* 루프의 스레드에서 부드럽게 끝낸다. on_drained가 루프를 멈춘다. 그다음 서버를 해체한다:
     * 그때는 다른 무엇도 서버를 건드리지 않는다. */
    app.ended = PROVEN_ERR_INVALID_STATE;
    EXAMPLE_REQUIRE(proven_loop_post(app.loop, begin_end, &app) == PROVEN_OK, "끝내기를 요청한다");
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.ended == PROVEN_OK && app.served == 4, "아무것도 끊기지 않고 끝났고, 교환 넷이
on_done(PROVEN_OK)로 끝났다");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

count_pump를 보라: 스트리밍되는 모든 응답이 갖는 모양이다. 시작할 때 on_request에서, 이어 갈 때 on_writable에서 불리고, 자기 위치를 교환 자신의 상태에 간직하며, 쓰기가 내놓은 것보다 적게 받는 순간 돌아온다. 어떤 실행에서 실제로 거절당하는지는 클라이언트가 얼마나 빨리 읽느냐에 달렸다. 함수는 어느 쪽이든 옳다.

8. 요청 본문

```
void proven_http_stream_pause(proven_http_stream_t *stream);
void proven_http_stream_resume(proven_http_stream_t *stream);
```

본문은 네트워크가 전달한 크기 그대로의 조각으로 on_body를 통해 도착하고, 프레이밍 - Content-Length나 청크 - 은 이미 벗겨져 있다. 본문이 없는 요청은 on_body 호출을 아예 받지 않는다. 본문이 있는 요청은 last가 참인 마지막 호출을 받는다.

핸들러는 본문이 도착하기 전에, 또는 본문을 읽지 않고 답해도 된다. 그때 본문의 나머지는 비용이 작으면 읽혀서 버려지고, 그렇지 않으면 응답 뒤에 연결이 닫힌다.

proven_http_stream_pause는 반대 방향의 backpressure다: proven_http_stream_resume까지 조각이 더 전달되지 않고 연결도 읽히지 않는다. 받은 것을 네트워크보다 느린 무언가에 쓰는 핸들러는 이 쌍을 써서, 본문이 메모

리에 쌓이는 대신 클라이언트가 느려지게 한다. 기다리던 조각들은 `proven_http_stream_resume` 호출 안에서 도착할 수 있다.

9. 한도, 시간, 거절

`proven_http_event_server_config_t`를 0으로 초기화하고, `on.on_request`를 정하고, 필요한 것을 정한다. 0인 필드마다 기본값이 있다:

필드	뜻	기본값
`alloc`	연결과 버퍼를 위한 할당자.	루프의 것
`max_connections`	동시에 열려 있는 연결. 그 이상은 받아들여지지 않은 채 시스템의 백로그에서 기다린다.	10,000
`max_head_bytes`	가장 큰 요청 헤드. 가장 큰 응답 헤드이기도 하다.	16 KiB
`max_headers`	요청 하나의 헤더 필드 최대 수.	64
`max_body_bytes`	가장 큰 요청 본문.	1 MiB
`max_buffered_output`	쓰기가 거절되기 전까지 연결 하나를 위해 붙드는 응답 바이트.	64 KiB
`head_timeout_ms`	요청의 첫 바이트부터 헤드의 끝까지. TLS에서는 핸드셰이크도 포함한다.	10초
`body_timeout_ms`	요청 본문의 조각과 조각 사이.	30초
`write_timeout_ms`	붙든 출력을 클라이언트가 하나도 받지 않는 시간.	30초
`idle_timeout_ms`	요청이 없는 열린 연결.	60초
`linger_ms`	응답 뒤에 닫는 중인 연결에서 계속 읽어 주는 시간. 응답이 리셋에 묻히지 않게 한다. 그때까지는 연결 하나로 센다.	1초
`tls`	인증서가 있는 TLS 설정, 또는 평문 HTTP를 뜻하는 널. 그 `early_data`가 있으면(v0.84.0부터) TLS 조기 데이터로 온 요청이 핸드셰이크가 끝나기 전에 핸들러에게 가고, `proven_http_stream_early`가 참이며, 바로 답할 수 있다. 본문 없는 GET, HEAD, OPTIONS만 거기까지 간다 - 그 밖의 것이 조기에 오면 서버가 스스로 425로 답한다(14장 6절).	널
`http2`	연결이 HTTP/2일 수 있게 한다 - TLS에서는 ALPN으로, 평문에서는 첫 바이트들로 - 같은 포트에서 HTTP/1.1과 나란히, 같은 처리기로: 17장 6절.	false
`max_concurrent_streams`	HTTP/2: 한 연결에서 진행 중인 요청 수.	100
`http2_window_bytes`	HTTP/2: 핸들러가 가져가기 전에 클라이언트가 한 스트림에, 그리고 연결 전체에 보낼 수 있는 요청 본문 바이트. 멈춘 요청이 연결에 쥐게 할 수 있는 양이며, 큰 업로드는 이만큼 보낼 때마다 자리를	65,535, 자람

	기다린다. 0이면 창이 거기서 시작해 본문을 읽는 동안 자란다 - 왕복 두 번 안에 절반이 읽히는 동안 두 배씩, 스트림당 16 MiB, 연결당 24 MiB까지(v0.75.0, HTTP/3은 v0.71.0부터); 읽지 않는 본문은 키우지 않는다. 정한 값은 고정.	
`http3`	HTTP/3도: 각 TCP 리스너 곁의 같은 포트 UDP 소켓에서 QUIC으로, 같은 처리기로, 그리고 TCP 응답에 `Alt-Svc`: 17장 9절. `h3`를 내는 `tls`가 필요하다.	false
`http3_retry`	HTTP/3: 클라이언트마다 첫 패킷에 Retry로 답해, 받는다는 것을 보이지 않은 주소에는 상태를 두지 않는다 - 연결마다 한 왕복이 더 들며, 부하가 걸린 서버를 위한 것.	false
`http3_window_bytes`	HTTP/3: 처리기가 가져가기 전에 클라이언트가 한 스트림에, 그리고 연결 전체에 보낼 수 있는 요청 본문 바이트(QUIC의 흐름 제어).	스트림 256 KiB, 연결 1 MiB에서 시작해 본문을 오는 대로 읽는 동안 16 MiB와 24 MiB까지 자람; 값을 주면 고정

서버가 서비스하지 않을 요청은 `on_request`에 닿지 않는다. 답을 받고 연결이 닫힌다:

상태	무엇에
400	HTTP가 아닌 헤드. `Host`가 정확히 하나가 아닌 HTTP/1.1 요청. 스스로 모순되는 본문 프레이밍. 청크가 아닌 청크.
408	시작되었으나 `head_timeout_ms` 안에 끝나지 않은 헤드.
413	`max_body_bytes`보다 긴 본문.
431	`max_head_bytes`보다 큰 헤드, 또는 `max_headers`보다 많은 필드.
501	`chunked`가 아닌 전송 코딩.
505	HTTP/1.0도 HTTP/1.1도 아닌 버전.

이런 일이 `on_request` 뒤에 일어나면 - 너무 긴 것으로 드러나는 본문, 도중에 잘못되는 본문 - 교환은 `on_done`과 그 이유로 끝나고, 응답이 아직 시작되지 않았다면 클라이언트는 그 상태를 받는다. `body_timeout_ms`, `write_timeout_ms`, `idle_timeout_ms`를 넘긴 연결은 닫힌다. 그 위에서 진행 중이던 교환은 `on_done(PROVEN_ERR_TIMEOUT)`으로 끝난다. 사라진 클라이언트는 자기 교환을 `PROVEN_ERR_RESET`으로 끝낸다.

기다리지 않고 한 연결에 잇달아 보낸 요청 - 파이프라이닝 - 은 순서대로, 한 번에 스트림 하나씩 서비스된다.

10. 스레드, TLS, 종료

스레드. 루프 하나는 스레드 하나이고, 서버는 그 스레드에 속한다. 계산하거나 무언가를 기다려야 하는 핸들러는 일을 제출하고, 돌아오고, 그 일이 답을 게시해 돌려보낸다:

1. `on_request`가 헤드에서 필요한 것을 복사하고, 스트림을 기억하고, 일을 제출한다.
2. 일은 다른 스레드에서, 서버의 것은 아무것도 건드리지 않고 돈다.
3. 일은 `proven_loop_post`로, 루프의 스레드에서 `proven_http_stream_respond`를 부를 함수를 게시한다.
4. `on_done`이 먼저 왔다면 - 클라이언트가 떠났다 - 그 함수는 그것을 알아채고 아무것도 하지 않아야 한다. 흔한 방법은 `on_done`이 교환 자신의 상태에 끝났다고 표시하고, 그 해제는 둘 중 나중에 도는 쪽에 맡기는 것이다.

TLS. 설정에 인증서가 있는 `proven_tls_config_t`(14장 2절)를 주면 모든 연결이 TLS다. 여러분의 코드에서 다른 것은 바뀌지 않는다. 핸드셰이크는 다른 읽기와 쓰기처럼 루프가 나르고 `head_timeout_ms`의 적용을 받는다.

설정은 서버보다 오래 살아야 한다. 핸드셰이크의 공개 키 연산은 루프의 스레드에서 이루어지고, 그동안 다른 것은 서비스되지 않는다: 하나의 비용은 14장 9절에 있다.

종료: `proven_http_event_server_shutdown(server, grace_ms, on_drained, ctx)`. 루프의 스레드에서. 호출 순간부터 새것은 받지 않는다. 요청이 없는 HTTP/1.1 연결은 닫힌다. 요청이 오기 시작했거나 응답 중인 연결은 `Connection: close`와 함께 그것을 끝낸다. HTTP/2 연결은 작별하고(GOAWAY) 열린 스트림을 끝낸다. WebSocket은 1001, "going away"로 닫히고 상대의 답을 기다린다. `grace_ms`가 지나면 남은 것은 끝난다 - 교환은 `on_done(PROVEN_ERR_RESET)`을, WebSocket은 `on_closed`를 받는다. `on_drained(ctx, why)`는 마지막 연결이 사라졌을 때 루프에서 한 번 불린다: `why`는 `PROVEN_OK`, 또는 기한이 무언가를 끊었으면 `PROVEN_ERR_TIMEOUT`. 그 안에서 `proven_loop_stop`을 불러도 된다. 서버와 루프는 루프가 돌아온 뒤에 해제한다. 종료는 마지막이다 - 서버는 더 이상 듣지도 넘겨받지도 않는다. 기한은 연결을 제한하지 핸들러를 제한하지 않는다: 핸들러가 다른 스레드로 보낸 일은 이미 사라진 스트림으로 돌아온다.

`proven_http_event_server_stop_listening`만 부르면 그 첫걸음만 하고 나머지는 하지 않는다: 새로운 누구도 없고, HTTP/1.1 응답은 `Connection: close`를 말하고, HTTP/2는 작별한다 - 그리고 연결은 클라이언트가 놓아줄 때, 또는 `idle_timeout_ms`에 끝난다. 서버는 루프의 스레드에서, 또는 루프가 멈춘 뒤에 해제한다 - 다른 스레드가 `proven_loop_run` 안에 있는 동안은 아니다. 7절의 예제가 종료로 끝난다.

11. 루프 위의 WebSocket

12장의 WebSocket 연결은 기다리는 호출을 가지며, 그래서 열려 있는 동안 스레드 하나를 차지한다. `ws_event.h`는 다른 형태다: 이 장의 서버로 온 요청이 `on_request` 안에서 WebSocket이 되고, 그때부터 루프가 여러분의 함수에게 무엇이 도착했는지 알려 준다. 조용한 연결은 자기 상태를 붙들 뿐 버퍼는 붙들지 않는다.

```
proven_err_t proven_ws_event_accept(proven_http_stream_t *stream, const proven_http_request_t *head,
                                    const proven_ws_event_config_t *config, proven_ws_stream_t **out);
proven_err_t proven_ws_stream_send(proven_ws_stream_t *ws, bool text, proven_mem_view_t data);
proven_err_t proven_ws_stream_send_piece(proven_ws_stream_t *ws, bool text, proven_mem_view_t data, bool
first, bool last);
proven_err_t proven_ws_stream_ping(proven_ws_stream_t *ws, proven_mem_view_t data);
void proven_ws_stream_close(proven_ws_stream_t *ws, proven_u16 code, proven_u8str_view_t reason);
void proven_ws_stream_abort(proven_ws_stream_t *ws);
void proven_ws_stream_pause(proven_ws_stream_t *ws);
void proven_ws_stream_resume(proven_ws_stream_t *ws);
void proven_ws_stream_set_user(proven_ws_stream_t *ws, void *user);
void *proven_ws_stream_user(const proven_ws_stream_t *ws);
proven_net_addr_t proven_ws_stream_peer(const proven_ws_stream_t *ws);
proven_size_t proven_ws_stream_buffered(const proven_ws_stream_t *ws);
```

받아들이기. `proven_ws_event_accept`는 `head`가 유효한 동안, `on_request` 안에서 부른다. 요청을 검사하고, 101로 답하고, 연결을 WebSocket으로 바꾼다. **HTTP 교환은 그 호출 안에서 끝난다**: 그 스트림에 대해 `on_done(PROVEN_OK)`가 불리고 스트림은 끝나며, `on_closed`가 돌아올 때까지 여러분이 쥐는 것은 `*out`이다.

반환	뜻	핸들러가 할 일
`PROVEN_OK`	연결이 WebSocket이 되었다.	`*out`에 상태를 붙이고 돌아온다.
`PROVEN_ERR_NOT_FOUND`	요청이 WebSocket을 청하지 않았다. 아무것도 보내지 않았다.	보통 요청으로서 답한다.
`PROVEN_ERR_UNSUPPORTED`	13이 아닌 버전의 업그레이드. 아무것도 보내지 않았다.	`Sec-WebSocket-Version: 13`과 함께 426으로 답한다.
`PROVEN_ERR_INVALID_FORMAT`	업그레이드이지만 형식이 잘못되었다. 아무것도 보내지 않았다.	400으로 답한다.
`PROVEN_ERR_INVALID_ARG`	`on_message`가 없거나, 토큰이 아닌 서브프로토콜.	프로그램의 결함이다.

PROVEN_ERR_INVALID_STATE	이 스트림의 on_request 안이 아니거나, 응답이 이미 시작되었다.	프로그램의 결함이다.
PROVEN_ERR_NOMEM		503으로 답하거나 abort한다.
PROVEN_ERR_RESET	바로 그 순간 클라이언트가 사라졌다. on_done은 이미 불렸다.	없음: 스트림은 끝났다.

proven_ws_event_config_t는 0으로 초기화하고, on.on_message를 주고, 그 밖에 필요한 것을 준다:

필드	뜻	기본값
on.on_message(ctx, ws, text, piece, first, last)	메시지의 한 조각. 필수.	
on.on_writable(ctx, ws)	거절되었던 보내기가 이제 받아들여진다.	없음
on.on_closed(ctx, ws, code, why)	연결이 끝났다. 정확히 한 번 불린다.	없음
ctx	모든 콜백에 전달된다.	
subprotocol	비어 있지 않으면 답에 적힌다. 클라이언트가 제안했을 때에만 적는다(proven_ws_request_offers).	없음
max_message_bytes	받아들이는 가장 큰 메시지, 그 조각들을 합쳐서.	1 MiB
max_buffered_output	보내기가 거절되기 전까지 붙드는 출력.	64 KiB
ping_interval_ms	클라이언트가 이만큼 조용하면 ping을 보낸다.	30초
pong_timeout_ms	그 뒤로도 이만큼 조용하면 연결을 끝낸다.	10초
close_timeout_ms	close에 대한 답을 기다리는 시간. 클라이언트가 마지막으로 출력을 가져간 때부터 센다.	5초
compress, compress_level, compress_window_bits, compress_no_takeover, compress_peer_no_takeover	클라이언트가 제안하면 압축된 메시지를 받아들인다: 12장 9절. compress_no_takeover를 켜거나 클라이언트가 막지 않는 한 양쪽이 메시지 사이에 윈도를 이어 간다(연결마다 약 64 KiB). 압축된 메시지는 풀린 채 최대 16 KiB 조각으로 도착하고, max_message_bytes는 풀린 크기를 센다.	꺼짐. 6. takeover에서 11, 아니면 15. 꺼짐

메시지는 조각으로 도착한다. on_message는 각 메시지를 네트워크가 전달한 조각대로 받고, first와 last가 그 양 끝을 표시한다: 어떤 크기의 메시지든 조립되거나 붙들리지 않고 지나가며, piece는 함수가 돌아올 때까지만 유효하다. 텍스트는 메시지 전체에 걸쳐 UTF-8인지 검사되고, 조각은 문자 중간에서 끝날 수 있다. 온전한 메시지를 원하는 핸들러는 스스로 정한 한도까지 조각을 직접 모은다 - 아래 프로그램이 그렇게 한다. max_message_bytes는 서버의 한도이지 버퍼가 아니다: 그것을 넘는 메시지는 close 코드 1009로 거절된다.

보내기는 통째로 받아들여지거나 전혀 받아들여지지 않는다. proven_ws_stream_send는 붙든 것이 max_buffered_output보다 적을 때 메시지 하나를 큐에 넣고, 그렇지 않으면 아무것도 받지 않고 PROVEN_ERR_AGAIN을 돌려준다: 메시지를 간직하고, on_writable이 불리면 다시 보낸다. 이것이 7절의 HTTP 본문과 다른 점이

다. 거기의 쓰기는 몇 바이트를 받았는지 말하지만, 프레임은 라이브러리가 여러분의 메시지를 어디서 자를지 정하지 않고서는 반만 보낼 수 없다. 그러므로 연결 하나가 붙드는 최대는 한도에 여러분이 고른 메시지 하나를 더한 것이다. 만들어 가며 보내는 큰 메시지는 `proven_ws_stream_send_piece`가 같은 규칙으로 한 번에 한 프래그먼트씩 보낸다. 되돌려 보내기만 하는 핸들러는 기다리는 동안 `proven_ws_stream_pause`로 읽기를 멈출 수 있고, 그러면 읽는 것보다 빨리 보내는 클라이언트는 버퍼에 쌓이는 대신 느려진다.

호출	반환
`proven_ws_stream_send`, `proven_ws_stream_send_piece`	`proven_err_t`: 출력 한도에 닿았으면 `AGAIN`, 아무것도 받지 않는다. close를 보낸 뒤이거나, 메시지 안에서의 `first`이거나, 메시지 밖에서 `first`가 아닌 조각이면 `INVALID_STATE`. 연결이 사라졌으면 `RESET`.
`proven_ws_stream_ping`	같다. 125바이트를 넘으면 `OUT_OF_BOUNDS`.
`proven_ws_stream_close`	없음. `code`와 `reason`으로 close를 보내고, 클라이언트가 답하거나 close 타임아웃이 지나면 연결을 끝낸다. `on_closed`는 나중에 따른다. 보낼 수 없는 코드는 1000이 된다.
`proven_ws_stream_abort`	없음. close 프레임 없이 곧바로 연결을 끝낸다. `on_closed(1006, PROVEN_ERR_RESET)`이 호출 안에서 불린다.
`proven_ws_stream_pause`, `proven_ws_stream_resume`	없음. 메시지 전달과 연결 읽기를 멈추고 다시 시작한다.
`proven_ws_stream_buffered`	`proven_size_t`: 클라이언트를 위해 큐에 있고 아직 받아 가지 않은 바이트.
`proven_ws_stream_compressed`	`bool`: `compress`를 켰고 클라이언트가 동의했는가.
`proven_ws_stream_set_compress`	없음. 압축에 합의한 연결에서, 이제부터 보내는 메시지를 압축할지(12장 9절).

어떻게 끝나는가. `on_closed`는 정확히 한 번 불리고, 받는 값이 경위를 말한다:

`code`	`why`	일어난 일
클라이언트의 코드 (close에 코드가 없었으면 1005)	`PROVEN_OK`	클라이언트가 닫았고, 답을 받았다.
이쪽이 보낸 코드	`PROVEN_OK`	이쪽이 닫았고, 클라이언트가 답했다.
이쪽이 보낸 코드	`PROVEN_ERR_TIMEOUT`	이쪽이 닫았고, 클라이언트는 끝내 답하지 않았다.
1002, 1007, 1009	`PROVEN_ERR_INVALID_FORMAT`, `PROVEN_ERR_INVALID_ENCODING`, `PROVEN_ERR_OUT_OF_BOUNDS`	클라이언트가 프로토콜을 어겼거나, UTF-8이 아닌 텍스트를 보냈거나, 한도를 넘는 메시지를 보냈다. 그 코드를 받았다.
1006	`PROVEN_ERR_TIMEOUT`	두 생존 한도 안에 데이터로도 ping에도 답하지 않았다.
1006	`PROVEN_ERR_RESET`	close 프레임 없이 사라졌다. 또는 `proven_ws_stream_abort`. 또는 서버가 해제되었다.

클라이언트의 ping에는 대신 답해 주고, 그 pong은 여러분에게 아무것도 요구하지 않는다. 생존은 받는 것으로 판단한다: 데이터를 받기만 하고 읽지 않는 클라이언트는 그것만으로는 발견되지 않는다 - 다만 그 클라이언트를 위해 출력 한도 이상은 큐에 쌓이지 않고, ping에 답하기를 멈추면 끊긴다.

테스트 스위트가 이 프로그램을 컴파일하고 실행한다:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * 이벤트 구동 서버 위의 WebSocket: 스레드를 쓰지 않는 연결.
 *
 * 요청은 on_request 안에서 WebSocket이 된다. 그때부터 루프가 이 함수들에게 무엇이 도착했는지
 * 알려 주고, 함수들은 곧바로 돌아온다. 이 서버는 되받아 외친다: 텍스트 메시지마다 대문자로
 * 답한다.
 *
 * 클라이언트는 12장의 블로킹 클라이언트이고, 예제의 메인 스레드에서 돈다.
 */

typedef struct {
    proven_loop_t *loop;
    int opened, closed;
    proven_u16 last_code;
} ex_app_t;

/* 연결 하나에 대해 이 프로그램이 간직하는 것: 모으고 있는 메시지. */
typedef struct {
    ex_app_t *app;
    char text[4096];
    proven_size_t len;
    bool waiting;                /* 서버가 아직 받아 주지 않은 답 */
} ex_peer_t;

/* 답을 보낸다. 클라이언트가 앞서 보낸 것을 가져가지 않고 있으면 서버가 거절한다:
 * 답을 간직하고, 이 클라이언트에게서 읽기를 멈추고, on_writable이 알려 줄 때 다시 시도한다. */
static void reply(proven_ws_stream_t *ws, ex_peer_t *peer) {
    proven_err_t err = proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)peer->text,
peer->len });
    if (err == PROVEN_ERR_AGAIN) {
        peer->waiting = true;
        proven_ws_stream_pause(ws);
        return;
    }
    peer->waiting = false;
    peer->len = 0;
}

/* 메시지는 네트워크가 전달한 조각대로 도착한다. 이 핸들러는 온전한 메시지를 원하므로
 * 직접 모은다 - 스스로 정한 한도까지. 손으로 하는 까닭이 바로 그 한도다. */
static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    (void)ctx;
    ex_peer_t *peer = proven_ws_stream_user(ws);
    if (first) peer->len = 0;
    if (!text || peer->len + piece.size > sizeof peer->text) {
        proven_ws_stream_close(ws, 1009, PROVEN_LIT("short text only"));       /* 1009: 너무 크다 */
        return;
    }
    for (proven_size_t i = 0; i < piece.size; ++i) {
        proven_byte_t ch = piece.ptr[i];
        peer->text[peer->len++] = (char)(ch >= 'a' && ch <= 'z' ? ch - 32 : ch);
    }
    if (last) reply(ws, peer);
```

```
}

/* 다시 자리가 생겼다. */
static void on_writable(void *ctx, proven_ws_stream_t *ws) {
    (void)ctx;
    ex_peer_t *peer = proven_ws_stream_user(ws);
    if (!peer->waiting) return;
    reply(ws, peer);
    if (!peer->waiting) proven_ws_stream_resume(ws);
}

/* 연결이 어떻게 끝났든 한 번 불린다. 붙여 둔 것을 해제할 자리. */
static void on_closed(void *ctx, proven_ws_stream_t *ws, proven_u16 code, proven_err_t why) {
    ex_app_t *app = ctx;
    (void)why;
    free(proven_ws_stream_user(ws));
    app->last_code = code;
    app->closed++;
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    ex_peer_t *peer = calloc(1, sizeof *peer);
    if (!peer) { proven_http_stream_abort(stream); return; }
    peer->app = app;

    proven_ws_event_config_t config = {
        .on = { on_message, on_writable, on_closed },
        .ctx = app,
        .max_message_bytes = sizeof peer->text,
    };
    proven_ws_stream_t *ws = NULL;
    /* 요청을 검사하고, 101로 답하고, 연결을 WebSocket으로 바꾼다. HTTP 교환은 이 호출
     * 안에서 끝난다(on_done이 있으면 불린다). */
    proven_err_t err = proven_ws_event_accept(stream, head, &config, &ws);
    if (err != PROVEN_OK) {
        free(peer);
        /* 아무것도 보내지 않았다: 요청은 여전히 보통 요청이고, 그렇게 답한다. */
        if (err == PROVEN_ERR_UNSUPPORTED) {
            proven_http_header_t version = { PROVEN_LIT("Sec-WebSocket-Version"), PROVEN_LIT("13") };
            (void)proven_http_stream_respond(stream, 426, &version, 1, (proven_mem_view_t){ 0 });
        } else if (err != PROVEN_ERR_RESET) {
            (void)proven_http_stream_respond(stream, err == PROVEN_ERR_NOT_FOUND ? 404 : 400, NULL, 0,
(proven_mem_view_t){ 0 });
        }
        return;
    }
    /* `stream`은 이제 끝났다. `ws`는 on_closed가 돌아올 때까지 유효하다. */
    proven_ws_stream_set_user(ws, peer);
    app->opened++;
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- 루프 위의 서버, 자기 스레드에서 ------------------------------------------
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "루프");
```

```
    /* HTTP/2도, 그리고 그 위의 WebSocket도(RFC 8441): HTTP/2를 말하는 클라이언트는 WebSocket을 그
     * 연결의 스트림으로 연다. 위의 on_request는 둘 다에 같다 - 아래 실행이 쓰는 것은 이 프로그램의
     * HTTP/1.1 클라이언트다. */
    proven_http_event_server_config_t config = { .on = { .on_request = on_request }, .ctx = &app, .http2 =
true, .http2_websocket = true };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "빈 포트의 이벤트 구동 서버");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "루프가 다른 스레드에
서 돈다");

    // ---- 클라이언트 ----------------------------------------------------------
    /* 반대쪽 끝을 보이는 가장 간단한 방법은 블로킹 클라이언트다. */
    proven_http_client_config_t client_config = { .alloc = heap };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "HTTP 클라이언트");
    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/shout", (unsigned)at.port);
    proven_ws_conn_config_t ws_config = { .alloc = heap };
    proven_ws_conn_t *conn = NULL;
    EXAMPLE_REQUIRE(proven_ws_conn_connect(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, NULL, 0, PROVEN_LIT(""),
                                           &ws_config, &conn, NULL) == PROVEN_OK, "서버가 WebSocket을 받아들인
다");

    proven_ws_message_t msg;
    EXAMPLE_REQUIRE(proven_ws_conn_send_text(conn, PROVEN_LIT("hello, loop")) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.text && msg.data.size == 11 && memcmp(msg.data.ptr, "HELLO, LOOP", 11) == 0, "메시지가
대문자로 돌아온다");
    /* 클라이언트가 메시지를 어떻게 자르든 핸들러는 메시지 하나를 본다. */
    EXAMPLE_REQUIRE(proven_ws_conn_send_part(conn, true, proven_mem_view_from_u8(PROVEN_LIT("in three ")),
false) == PROVEN_OK &&
                    proven_ws_conn_send_part(conn, true, proven_mem_view_from_u8(PROVEN_LIT("pieces, ")),
false) == PROVEN_OK &&
                    proven_ws_conn_send_part(conn, true, proven_mem_view_from_u8(PROVEN_LIT("one answer")),
true) == PROVEN_OK &&
                    proven_ws_conn_receive(conn, proven_net_deadline_in(5000), &msg) == PROVEN_OK &&
                    msg.data.size == 27 && memcmp(msg.data.ptr, "IN THREE PIECES, ONE ANSWER", 27) == 0, "조각
셋이 핸들러에게는 메시지 하나다");

    // ---- 닫기 ----------------------------------------------------------------
    EXAMPLE_REQUIRE(proven_ws_conn_close(conn, 1000, PROVEN_LIT("done")) == PROVEN_OK, "클라이언트의 close에
답이 온다");
    proven_ws_conn_destroy(conn);
    proven_http_client_destroy(client);

    /* 루프의 스레드가 쓴 것을 보기 전에 루프를 멈춘다. */
    proven_loop_stop(app.loop);
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.opened == 1 && app.closed == 1 && app.last_code == 1000, "연결 하나가 열렸고, 클라이언
트의 코드로 한 번 닫혔다");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
```

```
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

HTTP/2 위에서 (RFC 8441)

`http2_websocket`을 켜면(`http2`와 함께) 서버는 첫 HTTP/2 설정에서 그렇다고 말하고, RFC 8441을 아는 클라이언트는 WebSocket을 따로 연결을 만들지 않고 이미 가진 HTTP/2 연결의 스트림으로 연다. **핸들러는 같다**: 요청은 CONNECT로, 경로를 대상으로, `:protocol` 필드를 맨 앞에 두고 `on_request`에 오며, `proven_ws_check_request`가 그것을 알아보고(키는 비어 있다 - 없으므로), `proven_ws_event_accept`가 101 대신 200으로 답한다. 위의 프로그램은 한 줄로 그것을 켠다. 그 프로그램 자신의 클라이언트는 HTTP/1.1이고, 같은 서버가 그것도 계속 서비스한다.

- **그런 WebSocket은 각각 스트림 하나를 쥔다** - `max_concurrent_streams`(기본 100) 중 하나 - 열려 있는 동안. WebSocket으로 가득한 연결은 새 요청을 거절한다.
- 그 프레임은 DATA 프레임에 실려 가므로 HTTP/2의 흐름 제어가 WebSocket 자신의 것 아래에 있다: `http2_window_bytes`는 WebSocket이 가져가기 전에 클라이언트가 보낼 수 있는 양이고(0이면 여느 본문처럼 WebSocket이 읽는 동안 자라는 창), `on_writable`은 스트림의 출력이 움직였을 때 온다.
- 끝은 close 핸드셰이크 뒤 양쪽의 END_STREAM이다 - 클라이언트의 것이 올 때까지 스트림은 자리를 지킨다. 스트림을 리셋하는 상대는 1006과 `PROVEN_ERR_RESET`으로 닫힌 WebSocket이다. 종료는 그것을 1001로 닫는다. 요청을 거절하는 핸들러(2xx가 아닌 답)는 스트림을 요청의 것처럼 끝낸다.
- `http2_websocket`이 없으면 클라이언트는 시도해서는 안 되고(설정이 그렇게 말한다), 시도하는 스트림은 PROTOCOL_ERROR로 거절된다.
- **여기 없는 것**: 이 라이브러리 자신의 HTTP/2 위 WebSocket 클라이언트 - `proven_ws_event_connect`는 전처럼 자기 연결을 만든다. Node의 HTTP/2 클라이언트와 대조해 시험했고, 브라우저로는 해 보지 않았다.

이쪽에서 만드는 연결

```
proven_err_t proven_ws_event_connect(proven_loop_t *loop, const proven_ws_event_connect_options_t *options,
proven_ws_stream_t **out);
```

루프 위의 프로그램은 요청하는 쪽인 때도 그만큼 많다. `proven_ws_event_connect`는 서버로 가는 연결을 시작하고 곧바로 돌아온다. 돌려주는 것은 `proven_ws_stream_t`, **서버의 연결과 같은 타입**이다. 이 절의 모든 호출이 서버의 연결에서처럼 동작하고, 메시지는 같은 세 콜백으로 도착한다: 한쪽을 위해 쓴 처리기가 다른 쪽에도 쓰인다.

다른 것은 시작이다.

`url`	`ws://` 또는 `wss://`. 호스트가 `Host`에 들어가고, `wss`에서는 인증서가 가리켜야 하는 이름이다
`address`	연결할 곳. 호스트가 IP 주소가 아니면 필요하다: **루프 위에서는 이름을 풀지 않는다**(까닭과 방법은 12절)
`tls`	`wss`용. 없으면 `wss` URL은 거절된다 - 대신 평문으로 연결하는 일은 없다
`headers`, `protocols`	여러분의 필드(`Origin`, `Authorization`)와, 제안할 서브프로토콜
`connect_timeout_ms`	연결되고, 보안이 서고, 수락되기까지. 0: 10초
`on_open`	서버가 수락했다. 서버가 고른 서브프로토콜을 받는다
`ws`	`proven_ws_event_config_t` - 연결의 콜백, 한도, 핑, 압축. 서버가 `proven_ws_event_accept`에 주는 것과 같다. 여기서 `compress`는 확장을 제안한다는 뜻이다

서버가 수락하기 전에는 아무것도 보낼 수 없다: 보내는 호출은 `PROVEN_ERR_INVALID_STATE`로 답하고, `on_open`이 언제 수락되었는지 알려 준다(그 안에서 보내는 것은 괜찮다). 아직 진행 중인 시도는 `proven_ws_stream_abort`가 끝내고, `proven_ws_stream_close`도 그렇게 한다.

on_closed는 시작한 연결마다 정확히 한 번 불린다 - WebSocket이 되지 못한 연결에도. 그때 코드는 0이고 why가 무슨 일이 있었는지 말한다:

코드 0과 함께 오는 why	무슨 일이 있었나
PROVEN_ERR_REFUSED	거기서 아무도 듣지 않는다 - 또는 서버가 101이 아닌 것으로 답했다. 리다이렉트를 따르지 않고 인증 요구에 답하지 않는다
PROVEN_ERR_TIMEOUT	connect_timeout_ms 안에 연결·보안·수락이 끝나지 않았다
PROVEN_ERR_INVALID_FORMAT	답이 HTTP가 아니었거나, 이 핸드셰이크의 것이 아닌 101이었다: 틀린 Sec-WebSocket-Accept, 제안하지 않은 서브프로토콜이나 확장
PROVEN_ERR_RESET, PROVEN_ERR_UNREACHABLE, TLS 오류	연결을 잃었거나, 만들 수 없었거나, 보안을 세울 수 없었다

클라이언트의 프레임은 프로토콜이 요구하는 대로 마스킹된다. 키는 연결을 시작할 때 시스템에서 시드를 받은 생성기에서 나온다. 여기에 없는 것: 프록시, 재연결(다시 시작할 것), HTTP/2 위의 WebSocket.

이 프로그램은 양쪽 끝을 루프 하나에 둔다. 테스트 스위트가 컴파일하고 실행한다:

```
#include <stdio.h>
#include <string.h>

/*
 * 루프 위의 WebSocket 클라이언트: 이쪽에서도 스레드가 들지 않는 연결.
 *
 * proven_ws_event_connect는 연결을 시작하고 곧바로 돌아온다. 돌려주는 것은
 * proven_ws_stream_t - 서버의 연결과 같은 타입 - 이므로 호출과 콜백이 위의 서버가 쓰는
 * 것과 같다. 새로운 것은 시작이다: on_open이 서버가 수락했음을 알리고, 그때까지는 아무것도
 * 보낼 수 없다.
 *
 * 여기서는 루프 하나가 양쪽을 다 싣는다: 요청받는 서버와 요청하는 클라이언트.
 */

typedef struct {
    proven_loop_t *loop;
    int opened, answers, closed, refused;
    proven_u16 code;
    char last[64];
} ex_app_t;

// ---- 서버: 무엇을 받았는지 말해 준다 ----

static void served(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    (void)ctx; (void)first;
    if (!last) return;                      /* 이 예제의 메시지는 작다: 각각 한 조각 */
    char reply[48];
    int n = snprintf(reply, sizeof reply, "%s, %u bytes", text ? "text" : "binary", (unsigned)piece.size);
    (void)proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)reply,
(proven_size_t)n });
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    proven_ws_event_config_t config = { .on = { .on_message = served } };
    proven_ws_stream_t *ws = NULL;
```

```
    if (proven_ws_event_accept(stream, head, &config, &ws) != PROVEN_OK)
(void)proven_http_stream_respond(stream, 400, NULL, 0, (proven_mem_view_t){ 0 });
}

// ---- 클라이언트 ----

/* 서버가 수락했다. 여기서부터 - 이 호출 안에서도 - 메시지를 보낼 수 있다. */
static void on_open(void *ctx, proven_ws_stream_t *ws, proven_u8str_view_t protocol) {
    ex_app_t *app = ctx;
    (void)protocol;
    app->opened++;
    (void)proven_ws_stream_send(ws, true, proven_mem_view_from_u8(PROVEN_LIT("hello")));
}

/* 서버가 갖는 것과 같은 콜백. 이쪽은 두 번 묻고 작별한다. */
static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    ex_app_t *app = ctx;
    (void)text; (void)first;
    if (!last) return;
    proven_size_t n = piece.size < sizeof app->last - 1 ? piece.size : sizeof app->last - 1;
    memcpy(app->last, piece.ptr, n);
    app->last[n] = '\0';
    printf("the server says: %s\n", app->last);
    if (++app->answers == 1) {
        static const proven_byte_t bytes[300] = { 1, 2, 3 };
        (void)proven_ws_stream_send(ws, false, (proven_mem_view_t){ bytes, sizeof bytes });
    } else {
        proven_ws_stream_close(ws, 1000, PROVEN_LIT("thank you"));
    }
}

/* 시작한 연결마다 정확히 한 번 불린다 - 끝내 열리지 않은 연결에도:
 * 그때 코드는 0이고 `why`가 무엇이 잘못되었는지 말한다. */
static void on_closed(void *ctx, proven_ws_stream_t *ws, proven_u16 code, proven_err_t why) {
    ex_app_t *app = ctx;
    (void)ws;
    if (code == 0) {
        /* WebSocket이 되지 못했다: 아무도 듣지 않았거나, 서버가 다른 답을 했거나, 시간이 다 됐다. */
        printf("no connection: error %d\n", (int)why);
        app->refused++;
    } else {
        app->code = code;
        app->closed++;
    }
    /* 이 프로그램의 두 연결이 모두 끝났다: 루프가 할 일이 남지 않았다. */
    if (app->closed == 1 && app->refused == 1) proven_loop_stop(app->loop);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request } };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
```

```
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    /* 호스트가 주소이므로 따로 줄 것이 없다: 루프 위에서는 이름을 풀지 않는다. */
    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/chat", (unsigned)at.port);
    proven_ws_event_connect_options_t options = {
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .on_open = on_open,
        .ws = { .on = { .on_message = on_message, .on_closed = on_closed }, .ctx = &app },
    };
    proven_ws_stream_t *ws = NULL;
    EXAMPLE_REQUIRE(proven_ws_event_connect(app.loop, &options, &ws) == PROVEN_OK && ws != NULL && app.opened
== 0, "a connection is begun; nothing has happened yet");
    /* 호출은 돌아왔고 연결은 만들어지는 중이다: 말하기에는 아직 이르다. */
    EXAMPLE_REQUIRE(proven_ws_stream_send(ws, true, proven_mem_view_from_u8(PROVEN_LIT("too early"))) ==
PROVEN_ERR_INVALID_STATE, "nothing can be sent before the server has accepted");

    /* 그리고 실패할 연결 하나. 그것이 어떻게 전해지는지 보려고: 방금 닫은 포트. */
    proven_net_listener_t unused;
    proven_net_addr_t nobody;
    EXAMPLE_REQUIRE(proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 1, &unused,
&nobody) == PROVEN_OK && proven_net_listener_close(&unused) == PROVEN_OK, "a port nobody listens on");
    n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/chat", (unsigned)nobody.port);
    options.url.size = (proven_size_t)n;
    proven_ws_stream_t *lost = NULL;
    err = proven_ws_event_connect(app.loop, &options, &lost);
    EXAMPLE_REQUIRE(err == PROVEN_OK || err == PROVEN_ERR_REFUSED, "an attempt that cannot succeed: refused
now, or told later");
    if (err != PROVEN_OK) app.refused++;

    /* 나머지는 모두 여기서 일어난다. */
    if (!(app.closed == 1 && app.refused == 1)) EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK, "the
loop runs until both have ended");
    EXAMPLE_REQUIRE(app.opened == 1 && app.answers == 2 && strcmp(app.last, "binary, 300 bytes") == 0, "it
opened once, and two messages were answered");
    EXAMPLE_REQUIRE(app.closed == 1 && app.code == 1000 && app.refused == 1, "it closed once, with this side's
code - and the other attempt was reported once");

    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

두 스레드, 연결 하나. 한 스레드는 메시지를 기다리고 다른 스레드는 무엇을 보낼지 정하는 프로그램 - 채팅 클라이언트, 제어 채널이 있는 피드 - 은 블로킹 연결로는 그렇게 할 수 없다(12장이 이유를 말한다). 이 연결로는 된다: 루프는 자기 스레드 없이 메시지를 기다리고, 다른 스레드는 보낼 것을 `proven_loop_post`로 넘긴다. 보내기는 연결을 만지는 유일한 스레드인 루프의 스레드에서 돈다. 건너가는 것은 메시지의 사본이고, post된 호출이 그 메모리를 해제한다. 이 프로그램은 테스트 스위트가 컴파일하고 실행한다:

```
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
/*
 * 한 스레드는 메시지를 기다리고 다른 스레드는 무엇을 보낼지 정한다 - 채팅 클라이언트의 모양.
 *
 * 블로킹 연결(12장)로는 안 된다: 연결은 한 스레드의 것이고, proven_ws_conn_receive를 감싼 락은
 * 받기가 기다리는 동안 모든 보내기를 붙잡는다. 루프의 연결로는 된다. 그 연결은 자기 스레드 없이
 * 메시지를 기다리고, 다른 스레드는 보낼 것을 proven_loop_post로 넘긴다: 보내기 자체는 연결을
 * 만지는 유일한 스레드인 루프의 스레드에서 일어난다.
 *
 * 여기서는 루프 하나가 양 끝을 모두 나른다: 메시지마다 답하는 서버, 그리고 클라이언트.
 */

#define LINES 3

typedef struct {
    proven_loop_t *loop;
    proven_ws_stream_t *ws;          /* 클라이언트의 연결: 루프의 스레드만 */
    atomic_bool open;                /* 타이핑 스레드가 읽어도 되는 것 */
    int answers, closed;
} ex_app_t;

// ---- 서버: 메시지마다 답한다 ------------------------------------------------

static void served(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    (void)ctx; (void)text; (void)first;
    if (!last) return;                     /* 이 예제의 메시지는 작다: 각각 한 조각 */
    char reply[80];
    int n = snprintf(reply, sizeof reply, "got: %.*s", (int)piece.size, (const char *)piece.ptr);
    (void)proven_ws_stream_send(ws, true, (proven_mem_view_t){ (const proven_byte_t *)reply,
(proven_size_t)n });
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    proven_ws_event_config_t config = { .on = { .on_message = served } };
    proven_ws_stream_t *ws = NULL;
    if (proven_ws_event_accept(stream, head, &config, &ws) != PROVEN_OK)
(void)proven_http_stream_respond(stream, 400, NULL, 0, (proven_mem_view_t){ 0 });
}

// ---- 클라이언트, 루프의 스레드에서 ------------------------------------------

static void on_open(void *ctx, proven_ws_stream_t *ws, proven_u8str_view_t protocol) {
    ex_app_t *app = ctx;
    (void)protocol;
    app->ws = ws;
    atomic_store(&app->open, true);
}

static void on_message(void *ctx, proven_ws_stream_t *ws, bool text, proven_mem_view_t piece, bool first, bool
last) {
    ex_app_t *app = ctx;
    (void)text; (void)first;
    if (!last) return;
    printf("the server says: %.*s\n", (int)piece.size, (const char *)piece.ptr);
    if (++app->answers == LINES) proven_ws_stream_close(ws, 1000, PROVEN_LIT("bye"));
}

static void on_closed(void *ctx, proven_ws_stream_t *ws, proven_u16 code, proven_err_t why) {
    ex_app_t *app = ctx;
```

```
    (void)ws; (void)code; (void)why;
    app->ws = NULL;
    app->closed++;
    proven_loop_stop(app->loop);
}

// ---- 타이핑 스레드에서 건너오는 것 ------------------------------------------

/* 보낼 한 줄, 자기 메모리에: 그것이 보내질 때쯤 타이핑 스레드는 이미 다음으로 넘어가 있다. */
typedef struct { ex_app_t *app; proven_size_t len; char text[64]; } ex_line_t;

/* 루프의 스레드에서. */
static void send_here(void *ctx) {
    ex_line_t *line = ctx;
    if (line->app->ws) (void)proven_ws_stream_send(line->app->ws, true, (proven_mem_view_t){ (const
proven_byte_t *)line->text, line->len });
    free(line);
}

/* 타이핑 스레드. 연결은 절대 만지지 않는다: post한다. */
static void typist(void *arg) {
    ex_app_t *app = arg;
    static const char *const lines[LINES] = { "hello", "how are you", "goodbye" };
    while (!atomic_load(&app->open)) proven_time_sleep(1);
    for (int i = 0; i < LINES; ++i) {
        ex_line_t *line = malloc(sizeof *line);
        if (!line) return;
        line->app = app;
        line->len = strlen(lines[i]);
        memcpy(line->text, lines[i], line->len);
        if (proven_loop_post(app->loop, send_here, line) != PROVEN_OK) free(line);
        proven_time_sleep(5);               /* 누군가 타이핑하는 중 */
    }
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static ex_app_t app;
    atomic_init(&app.open, false);
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request } };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    char url[64];
    int n = snprintf(url, sizeof url, "ws://127.0.0.1:%u/chat", (unsigned)at.port);
    proven_ws_event_connect_options_t options = {
```

```
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .on_open = on_open,
        .ws = { .on = { .on_message = on_message, .on_closed = on_closed }, .ctx = &app },
    };
    proven_ws_stream_t *ws = NULL;
    EXAMPLE_REQUIRE(proven_ws_event_connect(app.loop, &options, &ws) == PROVEN_OK, "a connection is begun");

    // ---- 두 스레드: 루프, 그리고 타이핑하는 사람 --------------------------------
    proven_job_sys_t *threads = NULL;
    proven_job_group_t looping, typing;
    proven_job_group_init(&looping);
    proven_job_group_init(&typing);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 2, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &looping, run, &app) == PROVEN_OK &&
                    proven_job_group_submit(threads, &typing, typist, &app) == PROVEN_OK, "the loop on one
thread, the typist on another");

    /* 세 번째 답 뒤 연결이 닫히면 루프가 멈춘다. */
    proven_job_group_wait(threads, &typing);
    proven_job_group_wait(threads, &looping);
    EXAMPLE_REQUIRE(app.answers == LINES && app.closed == 1, "three lines sent from the typing thread, three
answers received on the loop's");

    /* post되고 아직 돌지 않은 것은 루프가 사라지기 전에 여기서 돈다. */
    EXAMPLE_REQUIRE(proven_loop_poll(app.loop, PROVEN_NET_DONT_WAIT) == PROVEN_OK, "a last round");
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

12. 이벤트 구동 클라이언트

```
proven_err_t proven_http_event_client_create(proven_loop_t *loop, const proven_http_event_client_config_t
*config,
                                             proven_http_event_client_t **out);
void proven_http_event_client_destroy(proven_http_event_client_t *client);
proven_size_t proven_http_event_client_requests(const proven_http_event_client_t *client);
proven_err_t proven_http_event_client_start(proven_http_event_client_t *client, const
proven_http_event_request_options_t *options,
                                            proven_http_event_request_t **out);
proven_result_size_t proven_http_event_request_write(proven_http_event_request_t *request, proven_mem_view_t
data);
proven_err_t proven_http_event_request_end(proven_http_event_request_t *request);
void proven_http_event_request_abort(proven_http_event_request_t *request);
void proven_http_event_request_pause(proven_http_event_request_t *request);
void proven_http_event_request_resume(proven_http_event_request_t *request);
void proven_http_event_request_set_user(proven_http_event_request_t *request, void *user);
void *proven_http_event_request_user(const proven_http_event_request_t *request);
```

11장의 클라이언트는 요청을 보내고 답을 기다린다. 이 클라이언트는 요청을 시작하고 돌아온다. 응답 헤드, 본문 조각, 끝은 루프의 스레드에서 콜백으로 도착하고, 요청은 몇 개든 동시에 진행된다.

의도적으로 좁다. 그리고 두 가지 한계는 다른 무엇보다 먼저 알아 둘 만하다.

- **요청은 교환 하나이고 그 둘레의 것은 없다.** 리다이렉트, 인증 요구에 대한 답, 쿠키, 프록시가 없다. 11장의 클라이언트에는 모두 있다. 진행 중인 요청마다 스레드 하나를 쓸 수 있는 곳에서는 그것을 쓴다.
- **이름을 해석하지 않는다.** 이름 조회는 몇 초가 걸릴 수 있고, 루프 위의 무엇도 기다려서는 안 된다. 요청은 `address`로 어디에 연결할지 말한다. URL의 호스트가 IP 주소일 때만 필요 없다. 이름이라면: 일(job)에서 `proven_net_resolve`를 부르고, 결과를 루프에 게시하고, 거기서 요청을 시작한다 - 이 절 끝의 프로그램이 그

렇게 한다. URL은 이름을 그대로 둔다 - `Host`에 들어가는 것이 그것이고, TLS에서는 서버의 인증서가 위해야 하는 이름이다.

요청은 `proven_http_event_request_options_t`로 기술한다. 그것이 가리키는 모든 것은 `proven_http_event_client_start`가 돌아오기 전에 복사되거나 쓰인다:

필드	뜻
`method`	비어 있으면 `GET`. `CONNECT`는 거절된다.
`url`	절대 URL, `http` 또는 `https`, 자격 증명이 들어 있지 않은 것.
`address`	연결할 곳. 그 포트를 그대로 쓴다. URL의 호스트가 IP 주소일 때에만 널.
`headers`, `header_count`	여러분의 헤더. `Host`, `Content-Length`, `Transfer-Encoding`, `Connection`은 대신 써 주며 여기 있어서는 안 된다.
`body`	메모리에 든 본문, 통째로 보낸다.
`body_length`	`body`가 없을 때: 0이면 본문 없음. 길이를 주면 나중에 쓰는 본문. `PROVEN_HTTP_EVENT_LENGTH_UNKNOWN`이면 chunked로 쓴다.
`on.on_response(ctx, request, head)`	응답 헤드. 선택. 중간 응답(100, 103)은 지나치고 알리지 않는다.
`on.on_body(ctx, request, piece, last)`	본문 한 조각. 마지막 조각에서 `last`이고 그 조각은 비어 있을 수 있다. 본문 없는 응답은 호출을 받지 않는다. 선택.
`on.on_writable(ctx, request)`	보류되었던 요청 본문 바이트가 나갔다. 선택.
`on.on_done(ctx, request, why)`	끝. **필수**이고, 시작된 요청마다 정확히 한 번 불린다.
`ctx`	모든 콜백에 전달된다.

연결은 보관되고, 세어진다. v0.36.0부터 HTTP/1.1 요청은 제 연결을 만들었다가 닫지 않는다. 클라이언트는 **오리진** - URL의 스킴·호스트·포트, 그리고 연결한 주소 - 마다 연결을 갖고 있고, 요청은 그 가운데 하나를 쓴다:

- **한가한 것**, 오리진에 있으면. 연결은 그 위의 교환이 깨끗하게 끝났을 때 보관된다: 응답이 `Connection: close` 없는 HTTP/1.1이었고, 본문에 제 끝(길이, 또는 청크)이 있었고 그 끝에 이르렀고, 요청이 통째로 나갔고, 응답 뒤에 아무것도 오지 않았다. 클라이언트 전체에서 많아야 `max_kept_connections`(8)개가 보관되며, 가장 오래 쉰 것이 새 것에게 자리를 내준다. 어느 것도 `idle_timeout_ms`(60초)보다 오래 보관되지 않는다;
- **또는 새 것**, 오리진의 연결이 바쁜 것과 한가한 것을 합쳐 `max_connections_per_origin`(6)보다 적은 동안;
- **또는 기다린다**, 시작한 순서대로, 제 오리진의 연결이 다음에 비기를. **알아 둘 대목은 이것이다: 한 오리진으로 가는 일곱째 요청은 여섯 가운데 하나가 끝나기 전에는 나가지 않는다.** 일부러 한 서버에 여섯보다 많은 요청을 열어 두는 프로그램 - 롱 폴링, 끝나지 않는 스트림 - 은 `max_connections_per_origin`을 올려야 한다. 그러지 않으면 일곱째 요청은 기다리다가 `connect_timeout_ms` 뒤에 `PROVEN_ERR_TIMEOUT`으로 끝난다: 기다림은 연결 시계 위에 있고, 그 시계는 요청과 함께 시작한다. 한 클라이언트에서 많아야 `max_waiting_requests`(1,000)개가 기다리고, 그다음 `start`는 `PROVEN_ERR_BUSY`다. 기다리는 요청은 제 머리와 본문을 갖고 있고, (`max_buffered_output`까지) 쓸 수 있고, 중단할 수 있다.

보관된 연결은 쓰려는 순간 이미 죽어 있을 수 있다: 서버는 한가한 연결을 묻지 않고 닫고, 그 닫음이 다음 요청과 선 위에서 엇갈릴 수 있다. 그래서 전에 요청을 실어 나른 연결 위에서 요청을 잃었는데 그 위로 아무것도 돌아오지 않았으면, 클라이언트는 서버가 그 요청을 보지 못한 것으로 받아들이고 **알리지 않고 새 연결로 한 번 더 보낸다** - `on_done`은 한 번, 그것을 끝낸 쪽의 결과로. 본문을 조각으로 쓰는 요청은 다시 보낼 수 없다. 클라이언트가 그 조각들을 갖고 있지 않기 때문이다. 그런 요청은 `PROVEN_ERR_AGAIN`으로 끝나고, 뜻은 HTTP/2에서와 같다: 처리되지 않았고, 다시 시작할지는 호출자의 몫이다. 응답의 바이트가 하나라도 온 뒤이거나, 그 요청을 위해 만든 연결이면, 잃는 것은 전과 같이 `PROVEN_ERR_RESET`이다. ("보지 못했다"는 판단은 틀릴 수 있다 - 서버가 요청을 받고 답하기 전에 죽을 수도 있다. 연결을 재사용하는 HTTP 클라이언트는 모두 이렇게 판단하고, 11장의 블로킹 클라이언트도 그렇다. 그것을 받아들일 수 없는 프로그램을 위해 `no_connection_reuse`가 있다.)

retry_unsent는 횟수이고 기본값은 0이다. 서버가 처리했을 리 없는 요청을 클라이언트가 스스로 다시 시작하게 한다. HTTP/1.1에서는 그런 경우가 하나뿐이고 좁다: 연결이 만들어진 뒤, 요청이 조금도 건네지기 전에 잃은 경우이며, 실제로는 TLS를 합의하는 도중에 사라지는 서버다. 그런 요청은 설정이 없으면 PROVEN_ERR_RESET으로 끝나고, 설정이 있으면 그 횟수만큼 새 연결에서 다시 시작되며 시도마다 connect_timeout_ms가 따로 주어진다. 조금이라도 보낸 요청은 다시 시작하지 않고, 본문을 조각으로 쓰는 요청도 그렇다. (위에서 말한, 보관한 연결에서 다시 보내는 것은 설정이 필요 없고 횟수에 들지 않는다. HTTP/2에서는 이 설정이 더 많은 경우를 다룬다: 17장.)

retry_delay_ms는 그런 시도마다 클라이언트가 그 전에 기다리는 시간이다. 두 프로토콜 모두에 해당하고, 기본값은 0이며, retry_delay_max_ms를 설정하지 않으면 시도마다 같다 - 설정하면 한 요청의 시도마다 기다림이 두 배가 되어 retry_delay_ms에서 그 한도까지 늘어난다. 무작위로 더하는 것은 없다: 함께 거절된 요청은 함께 다시 시도된다. 기다리는 동안 요청에는 연결이 없다 - 잃은 연결은 닫히고 그 자리는 기다리던 요청에게 갈 수 있다 - 그리고 그 시도의 connect_timeout_ms는 기다림이 끝날 때 시작한다. 기다리는 동안 요청을 abort하면 여느 abort처럼 끝난다. 보관한 연결에서 다시 보내는 것은 늦추지 않는다. (아래의 max_connections와 함께 쓸 때: 기다림이 끝나면 요청은 새 요청처럼 자리를 구하고, 없으면 자리를 기다린다. 기다릴 수 있는 만큼이 이미 기다리고 있으면 PROVEN_ERR_BUSY로 끝난다 - proven_http_event_client_start가 아니라 on_done이 그렇게 말하는 유일한 경우다.)

max_connections는 클라이언트 전체의 연결 수를 제한한다. 두 프로토콜 모두, 쓰는 중인 것과 쉬는 것을 함께 센다. 기본값 0은 제한 없음이다. 연결을 하나 더 만들어야 하는 요청은 먼저 쉬는 연결 하나를 닫게 한다 - HTTP/1.1 연결 가운데 가장 오래 쉰 것이 먼저이고, 그다음이 아무것도 실려 있지 않은 HTTP/2 연결이다. 쉬는 연결이 없으면 요청은 프로토콜과 상관없이 기다린다. HTTP/1.1 요청은 위에서 말한 대기열에서 기다리다가 다음에 비는 자리를 얻는다(닫히는 연결은 자기 origin으로 가는 요청을 먼저, 그다음에는 가장 오래 기다렸고 origin에 여유가 있는 요청을 받는다). origin에 연결이 없는 HTTP/2 요청은 어느 연결 위도 아닌 곳에서 시작한 순서대로 기다리다가, 자리가 나면 연결에 놓인다 - 그사이 다른 요청이 그 origin의 연결을 만들었으면 거기에. 어느 쪽 기다림이든 연결을 기다리는 것이다: connect_timeout_ms가 PROVEN_ERR_TIMEOUT으로 끝내고, max_waiting_requests보다 많이는 기다리지 못한다(다음 start는 PROVEN_ERR_BUSY). 두 프로토콜이 함께 기다리면 자리는 HTTP/1.1 요청에게 먼저 간다. max_kept_connections를 max_connections보다 크게 설정하면 PROVEN_ERR_INVALID_ARG다.

no_connection_reuse는 연결에 관한 옛 동작을 돌려준다 - 요청마다 Connection: close, 응답이 끝나면 연결을 닫는다 - 그러나 상한이 없던 것까지 돌려주지는 않는다: 한 오리진에 여전히 한 번에 여섯이다.

시작. proven_http_event_client_start가 오류를 돌려주면 요청은 시작되지 않았고 콜백은 지금도 나중에도 불리지 않는다. PROVEN_OK를 돌려주면 on_done이 정확히 한 번 불린다 - **그 호출 안에서는 결코 불리지 않으므로**, 요청을 시작하는 코드는 그 요청에 무슨 일이 일어나기 전에 하던 일을 마칠 수 있다. 요청은 시작부터 on_done이 돌아올 때까지 유효하다.

start의 반환	무엇에
PROVEN_ERR_INVALID_ARG	on_done이 없다. 절대 http/https가 아니거나 자격 증명을 실은 URL. TLS 설정이 없는 https - 거절되며 평문으로 보내지 않는다. address 없는 이름. 클라이언트가 직접 쓰는 헤더. 토큰이 아닌 메서드, 또는 CONNECT.
PROVEN_ERR_OUT_OF_BOUNDS	max_head_bytes에 들어가지 않는 요청 헤드.
PROVEN_ERR_NOMEM	
PROVEN_ERR_REFUSED, PROVEN_ERR_UNREACHABLE	연결이 곧바로 실패했다. 나중에 실패할 수도 있고, 그때는 on_done이 알린다.

어떻게 끝나는가. on_done이 말한다:

why	일어난 일

PROVEN_OK	응답 전체가 도착했다 - 상태가 무엇이든: 404도 응답이다.
PROVEN_ERR_REFUSED, PROVEN_ERR_UNREACHABLE	아무것도 듣고 있지 않았거나 경로가 없었다.
PROVEN_ERR_TIMEOUT	네 시간 한도 가운데 하나가 지났다.
PROVEN_ERR_RESET	응답이 온전해지기 전에 서버가 닫았거나 사라졌다 - 길이에 못 미쳐 잘린 본문은 결코 완전하다고 보고되지 않는다. 또는 proven_http_event_request_abort. 또는 클라이언트가 해제되었다.
PROVEN_ERR_INVALID_FORMAT	답이 HTTP가 아니었거나, 프레이밍이 스스로 모순되었거나, 중간 응답이 여덟 번 넘게 잇달아 왔다.
PROVEN_ERR_OUT_OF_BOUNDS	헤드가 max_head_bytes보다 컸거나 본문이 max_body_bytes보다 컸다.
PROVEN_ERR_UNSUPPORTED	서버가 101로 답했다: 이 클라이언트는 프로토콜 변경을 청하지 않는다.
PROVEN_ERR_UNTRUSTED, PROVEN_ERR_NAME_MISMATCH, PROVEN_ERR_EXPIRED, …	TLS가 서버를 거절했다(14장 8절). 요청은 보내지지 않았다.

만들어 가며 쓰는 요청 본문은 7절의 규칙을 따른다. proven_http_event_request_write는 max_buffered_output 아래에 들어가는 만큼, 어쩌면 하나도 받지 않고, 몇 바이트를 받았는지 말한다. 나머지는 on_writable이 불리면 다시 내놓는다. start 직후, 연결이 생기기 전에 불러도 된다: 받은 것은 기다린다.
proven_http_event_request_end가 본문을 끝낸다. 약속한 길이에 못 미쳐 끝내면 요청은 PROVEN_ERR_INVALID_FORMAT으로 끝난다.

호출	반환
proven_http_event_client_create(loop, &config, &client)	proven_err_t: 루프가 없으면 INVALID_ARG. NOMEM. config는 널이어도 된다.
proven_http_event_client_requests(client)	proven_size_t: 시작되어 아직 끝나지 않은 요청 수 - 연결을 기다리는 것 포함.
proven_http_event_client_destroy(client)	없음. 진행 중이든 기다리는 중이든 모든 요청이 on_done(PROVEN_ERR_RESET)을 받고, 보관된 연결은 닫힌다. 루프가 아직 소켓을 쓸 수 있는 연결은 루프의 다음 바퀴나 proven_loop_destroy까지 메모리를 쥔다(5절 규칙 3). 널을 받는다.
proven_http_event_request_write(request, data)	proven_result_size_t: 받은 바이트 수. 쓸 본문이 없는 요청이거나 끝낸 뒤면 INVALID_STATE. 약속한 길이를 넘으면 OUT_OF_BOUNDS.
proven_http_event_request_end(request)	proven_err_t: 끝낼 것이 없었으면 INVALID_STATE. 약속보다 적게 썼으면 INVALID_FORMAT - 그때 on_done이 호출 안에서 불린다.
proven_http_event_request_abort(request)	없음. on_done(PROVEN_ERR_RESET)이 호출 안에서 불린다.
proven_http_event_request_pause, proven_http_event_request_resume	없음. 응답 본문 전달을 멈추고 다시 시작한다.

설정의 필드. 0이면 각각 기본값이 쓰인다:

필드	뜻	기본값

`alloc`	요청과 버퍼를 위한 할당자.	루프의 것
`tls`	서버를 검증할 수 있는 TLS 설정, `https`용. 클라이언트보다 오래 살아야 한다. 출처로의 두 번째 연결은 지난 연결이 배운 TLS 세션을 재개한다 (v0.72.0부터, RFC-0020): 클라이언트는 TCP용 64개, QUIC용 64개 출처의 세션을 두고, 가장 오래 안 쓴 것을 버리며 지우고, TCP의 것을 QUIC으로 내밀지 않는다. 티켓을 거절하는 서버는 전체 핸드셰이크로 답한다. 설정의 `no_resumption`이 그것을 끈다 - 매번 인증서를 확인해야 하거나, 연결과 연결이 이어져선 안 되는 클라이언트용. 세션은 2,048바이트이므로(v0.73.0부터) 보관하는 세션은 최대 256 KiB다. 설정의 `early_data`(HTTP/3에서는 v0.73.0부터, RFC-0021. HTTP/1.1과 HTTP/2에서는 v0.84.0부터, RFC-0027)는 재개하는 연결에서 본문 없는 GET, HEAD, OPTIONS 요청을 조기 데이터로 보내고 - 왕복 하나 먼저 - 그렇게 거절되었거나 425를 받은 요청은 핸드셰이크 뒤 한 번 다시 보낸다: 14장 6절.	없음: `https`는 거절된다
`max_head_bytes`	가장 큰 응답 헤드, 그리고 요청 헤드.	16 KiB
`max_headers`	응답 하나의 헤더 필드 최대 수.	64
`max_body_bytes`	가장 큰 응답 본문.	한도 없음: 조각으로 전달되고 붙들지 않는다
`max_buffered_output`	쓰기가 거절되기 전까지 붙드는 요청 본문 바이트.	64 KiB
`connect_timeout_ms`	연결되기까지, TLS 핸드셰이크 포함 - 그리고 오리진의 연결이 허용된 만큼 차 있을 때 연결을 기다리는 시간까지. 요청의 시작부터 잰다. 다시 보내지는 요청은 새로 받는다.	10초
`response_timeout_ms`	요청을 다 보낸 때부터 응답 헤드의 끝까지.	30초
`body_timeout_ms`	응답 본문의 조각과 조각 사이.	30초
`write_timeout_ms`	붙든 출력을 서버가 하나도 받지 않는 시간.	30초
`decompress`	gzip을 요청하고 푼다: `on_body`는 풀린 조각을 받고 `max_body_bytes`는 그것을 센다. 16장 6절.	꺼짐
`http2`, `http2_cleartext`	합의하는 오리진(ALPN)에, 또는 모든 `http` 오리진에 HTTP/2로 말한다: 오	꺼짐

	리진마다 연결 하나를 간직해 그 요청들이 함께 쓴다. 17장 7절.	
idle_timeout_ms	요청이 없는 연결을 간직하는 시간, 두 프로토콜 모두.	60초
max_waiting_requests	기다릴 수 있는 요청 수: HTTP/2 연결은 그 연결 하나에서, HTTP/1.1 연결은 클라이언트 전체에서. 그다음 start는 PROVEN_ERR_BUSY.	1,000
max_connections_per_origin	HTTP/1.1: 한 오리진으로 동시에 둘 수 있는 연결 수, 바쁜 것과 한가한 것 합쳐. 넘는 요청은 기다린다.	6
max_kept_connections	HTTP/1.1: 클라이언트 전체에서 보관하는 한가한 연결 수. 이 표의 다른 0과 마찬가지로 0은 기본값이고, 하나도 보관하지 않게 하는 것은 no_connection_reuse다. (v0.36.0에서는 max_idle_connections였다. 그것은 11장 클라이언트의 필드 이름이고 거기서 0은 하나도 보관하지 않는다는 뜻이라, 이제 두 필드는 이름을 같이 쓰지 않는다.)	8
no_connection_reuse	HTTP/1.1: 요청마다 Connection: close, 연결을 보관하지 않는다.	꺼짐
retry_unsent	서버가 처리하지 않은 요청(HTTP/2: PROVEN_ERR_AGAIN, HTTP/1.1: 조금도 보내기 전에 잃은 것)을 클라이언트가 스스로, 알리지 않고 곧바로 다시 보내는 횟수. 본문을 조각으로 쓰는 요청과 on_response 뒤는 제외.	0: 하지 않음
retry_delay_ms	retry_unsent의 시도마다 그 전에 기다리는 시간. 두 프로토콜 모두. 그 시도의 connect_timeout_ms는 기다림이 끝날 때 시작한다.	0: 없음
retry_delay_max_ms	설정하면 한 요청의 시도마다 기다림이 두 배가 되어 retry_delay_ms에서 여기까지 늘어난다. retry_delay_ms보다 작거나 retry_delay_ms 없이 설정하면 INVALID_ARG.	0: 매번 같은 기다림
max_connections	클라이언트 전체의 연결 수. 두 프로토콜, 쓰는 중과 쉬는 것 모두. 한도에서는 새 요청을 위해 쉬는 연결 하나를 닫는다. 쉬는 것이 없으면 요청은 프로토콜과 상관없이 기다린다. 길어야 connect_timeout_ms.	0: 제한 없음
http2_window_bytes	HTTP/2: on_body가 가져가기 전에 서버가 한 스트림에, 그리고 연결 전체에 보낼 수 있는 응답 본문 바이트. 멈춘	65,535, 자람

	요청이 연결에 쥐게 할 수 있는 양이며, 큰 다운로드는 서버가 이만큼 보낼 때마다 자리를 기다린다(1 MiB 응답 16개 동시, v0.75.0 전 측정: 기본값에서 초당 568개, 1 MiB로 1,208개). 0이면 창이 거기서 시작해 본문을 읽는 동안 자란다 - 왕복 두 번 안에 절반이 읽히는 동안 두 배씩, 스트림당 16 MiB, 연결당 24 MiB까지(v0.75.0, HTTP/3은 v0.71.0부터); 읽지 않는 본문은 키우지 않는다. 정한 값은 고정. 한쪽 10 ms 지연 중계기 너머 64 MiB 내려받기 하나: 창을 65,535로 고정하면 16 Mbit/s, 자라면 1,310.	
`http3`	그것을 청하는 요청(옵션의 `http3`, `https` URL)은 HTTP/3으로, 그 오리진에 하나 간직하는 QUIC 연결 위로 간다: 17장 10절. 청하지 않은 요청은 결코, `Alt-Svc`를 따라서도 결코 가지 않는다. **대체 없음**: UDP로 답이 없으면 `connect_timeout_ms` 뒤 `PROVEN_ERR_TIMEOUT`. `h3`를 내는 `tls`가 필요하다.	false
`http3_window_bytes`	HTTP/3: `on_body`가 가져가기 전에 서버가 한 스트림에, 그리고 연결 전체에 보낼 수 있는 응답 본문 바이트(QUIC의 흐름 제어).	스트림 256 KiB, 연결 1 MiB에서 시작해 본문을 오는 대로 읽는 동안 16 MiB와 24 MiB까지 자람; 값을 주면 고정

테스트 스위트가 이 프로그램을 컴파일하고 실행한다:

```
#include <stdio.h>
#include <string.h>

/*
 * 이벤트 구동 HTTP 클라이언트: 한 스레드에서 동시에 진행되는 여러 요청.
 *
 * 요청을 시작하면 호출은 곧바로 돌아온다. 그 요청에 일어나는 일은 루프의 스레드에서 콜백으로
 * 도착한다. 여기의 서버는 이 장의 이벤트 구동 서버이고 같은 루프 위에 있다 - 이 프로그램
 * 전체가 스레드 하나, 그리고 기다리는 단 하나의 일을 위한 워커 하나다.
 *
 * 그 하나의 일은 이름을 찾는 것이다. 클라이언트는 이름을 해석하지 않는다: 조회는 몇 초가 걸릴
 * 수 있고, 루프 위의 무엇도 기다려서는 안 된다. 그래서 세 번째 요청이 그 패턴을 보인다 -
 * 워커에서 해석하고, 답을 게시해 돌려보내고, 요청을 시작한다.
 */

typedef struct app ex_app_t;

/* 요청 하나에 대해 이 프로그램이 간직하는 것. */
typedef struct {
    ex_app_t *app;
    const char *path;
    int status;
    char body[64];
    proven_size_t len;
```

```
    proven_err_t why;
    bool done;
} ex_fetch_t;

struct app {
    proven_loop_t *loop;
    proven_http_event_client_t *client;
    proven_job_sys_t *workers;
    proven_u16 port;
    ex_fetch_t fetch[3];
    int finished;
    /* 워커가 쓰고, 게시된 뒤 루프에서 읽는다 */
    proven_net_addr_t found;
    proven_err_t lookup;
};

// ---- 요청이 가는 서버 -----------------------------------------------------

static void serve(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    if (proven_u8str_view_eq(head->target, PROVEN_LIT("/a"))) (void)proven_http_stream_respond(stream, 200,
NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("first")));
    else if (proven_u8str_view_eq(head->target, PROVEN_LIT("/b"))) (void)proven_http_stream_respond(stream,
200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("second")));
    else (void)proven_http_stream_respond(stream, 404, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("nothing
here")));
}

// ---- 클라이언트의 콜백 -----------------------------------------------------

/* 헤드: 이 함수가 돌아올 때까지만 유효하므로, 필요한 것을 여기서 꺼낸다. */
static void on_response(void *ctx, proven_http_event_request_t *request, const proven_http_response_t *head) {
    (void)request;
    ((ex_fetch_t *)ctx)->status = head->status;
}

/* 본문, 한 번에 한 조각. 대신 모아 주지 않는다. 원하는 것을 직접 모은다. */
static void on_body(void *ctx, proven_http_event_request_t *request, proven_mem_view_t piece, bool last) {
    (void)request; (void)last;
    ex_fetch_t *f = ctx;
    for (proven_size_t i = 0; i < piece.size && f->len < sizeof f->body - 1; ++i) f->body[f->len++] =
(char)piece.ptr[i];
}

/* 시작된 요청마다 한 번, 무슨 일이 있었든. */
static void on_done(void *ctx, proven_http_event_request_t *request, proven_err_t why) {
    (void)request;
    ex_fetch_t *f = ctx;
    f->why = why;
    f->done = true;
    if (++f->app->finished == 3) proven_loop_stop(f->app->loop);
}

static proven_err_t fetch(ex_app_t *app, ex_fetch_t *f, const char *path, const proven_net_addr_t *address,
const char *host) {
    char url[96];
    int n = snprintf(url, sizeof url, "http://%s:%u%s", host, (unsigned)app->port, path);
    f->app = app;
    f->path = path;
    proven_http_event_request_options_t options = {
```

```
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .address = address,                             /* URL의 호스트가 IP 주소일 때에만 널을 쓸 수 있다
*/
        .on = { on_response, on_body, NULL, on_done },
        .ctx = f,
    };
    return proven_http_event_client_start(app->client, &options, NULL);     /* 곧바로 돌아온다 */
}

// ---- 다른 곳에서 해석한 이름 ------------------------------------------------

/* 다시 루프의 스레드: 이제 주소가 있으니 요청을 시작할 수 있다.
 * URL은 이름을 그대로 둔다 - Host에 들어가는 것이 그것이다 - 그리고 주소가 어디로 연결할지 말한다. */
static void resolved(void *ctx) {
    ex_app_t *app = ctx;
    if (app->lookup != PROVEN_OK || fetch(app, &app->fetch[2], "/missing", &app->found, "localhost") !=
PROVEN_OK) {
        app->fetch[2].why = PROVEN_ERR_NOT_FOUND;
        if (++app->finished == 3) proven_loop_stop(app->loop);
    }
}

/* 워커 스레드에서: 이 호출은 리졸버가 걸리는 만큼 기다려도 된다. */
static void resolve(void *ctx) {
    ex_app_t *app = ctx;
    proven_net_addr_t all[8];
    proven_size_t count = 0;
    app->lookup = proven_net_resolve(PROVEN_LIT("localhost"), app->port, all, 8, &count);
    /* 여기의 서버는 IPv4로만 듣는다. 실제 프로그램이라면 주소를 차례로 시도할 것이다. */
    bool have = false;
    for (proven_size_t i = 0; i < count && !have; ++i) {
        if (all[i].family == PROVEN_NET_FAMILY_IPV4) { app->found = all[i]; have = true; }
    }
    if (app->lookup == PROVEN_OK && !have) app->lookup = PROVEN_ERR_NOT_FOUND;
    if (proven_loop_post(app->loop, resolved, app) != PROVEN_OK) proven_loop_stop(app->loop);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };

    // ---- 루프 하나, 그 위의 서버와 클라이언트 ------------------------------------
    /* 아직 아무것도 돌지 않는다: 루프는 돌릴 때까지 아무 일도 하지 않는다. */
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "루프");
    proven_http_event_server_config_t server_config = { .on = { .on_request = serve } };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "물어볼 서버");
    app.port = at.port;
    EXAMPLE_REQUIRE(proven_http_event_client_create(app.loop, NULL, &app.client) == PROVEN_OK, "이벤트 구동 클
라이언트");
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &app.workers) == PROVEN_OK, "조회를 맡을 워커 하나");

    // ---- 요청 셋 ----------------------------------------------------------
    /* 둘은 주소로. 어느 것도 보내지기 전에 둘 다 시작된다. */
    EXAMPLE_REQUIRE(fetch(&app, &app.fetch[0], "/a", NULL, "127.0.0.1") == PROVEN_OK &&
                    fetch(&app, &app.fetch[1], "/b", NULL, "127.0.0.1") == PROVEN_OK, "요청 둘이 시작되었다");
    EXAMPLE_REQUIRE(proven_http_event_client_requests(app.client) == 2 && !app.fetch[0].done && !
```

```
app.fetch[1].done, "둘 다 진행 중이고 답은 없다: 아직 아무것도 돌지 않았다");

    /* 하나는 이름으로. 주소가 없으면 거절된다 - 그러니 먼저 워커에서 해석한다. */
    ex_fetch_t unresolved = { 0 };
    EXAMPLE_REQUIRE(fetch(&app, &unresolved, "/missing", NULL, "localhost") == PROVEN_ERR_INVALID_ARG, "주소
없는 이름은 거절된다");
    EXAMPLE_REQUIRE(proven_job_submit(app.workers, resolve, &app), "조회를 워커에 넘겼다");

    // ---- 실행 ------------------------------------------------------------------
    /* 세 번째 요청이 끝나면 on_done이 루프를 멈춘다. */
    EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK, "셋이 모두 끝날 때까지 루프가 돌았다");

    EXAMPLE_REQUIRE(app.fetch[0].why == PROVEN_OK && app.fetch[0].status == 200 && strcmp(app.fetch[0].body,
"first") == 0, "첫 번째 답");
    EXAMPLE_REQUIRE(app.fetch[1].why == PROVEN_OK && app.fetch[1].status == 200 && strcmp(app.fetch[1].body,
"second") == 0, "두 번째 답");
    EXAMPLE_REQUIRE(app.fetch[2].why == PROVEN_OK && app.fetch[2].status == 404 && strcmp(app.fetch[2].body,
"nothing here") == 0, "세 번째, 이름으로: 404는 오류가 아니라 응답이다");

    proven_job_system_close(app.workers);
    proven_job_system_destroy(app.workers);
    proven_http_event_client_destroy(app.client);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

13. 여러 루프

루프 하나는 스레드 하나이고, 연결은 평생 루프 하나에 속한다: 그 연결의 어떤 것도 다른 스레드가 건드리지 않으며, 이 장의 무엇도 락을 잡지 않는 까닭이 그것이다. 그러므로 프로세서 코어가 둘 이상 필요한 서버는 **각자 서버를 가진 여러 루프**이고, 들어오는 연결을 그들 사이에 나눠 줄 무언가다.

```
proven_err_t proven_http_event_server_adopt(proven_http_event_server_t *server, proven_net_conn_t *conn);
```

`proven_http_event_server_adopt`는 다른 곳에서 받아들인 연결을 서버에 준다. 그 연결은 서버가 직접 받아들인 것과 똑같이 서버의 것이 된다. TLS라면 핸드셰이크가 시작된다. 그래서 스레드 하나가 듣고 받아들이며, 연결마다 서버들에게 차례로 나눠 준다. 세 가지 규칙이 이것을 올바르게 만든다:

- **`adopt`는 서버의 루프 스레드에서 부른다** - 다른 모든 것처럼. 받아들이는 스레드는 그것을 부르지 않는다: 그 루프에 함수를 게시하고(`proven_loop_post`), 그 함수가 부른다.
- **연결은 자기 메모리에 담겨 간다.** `proven_net_conn_t`는 열려 있는 동안 복사해서는 안 되므로, 게시하는 것은 그것을 담은 작은 블록이고, adopt하는 함수가 해제한다.
- **해체는 순서대로:** 받아들이는 스레드가 먼저다. 그래야 더는 게시되지 않는다. 그다음 루프들. 그다음 - 게시되었으나 아직 돌지 않은 것을 위해 각 루프를 마지막으로 한 바퀴 돌린 뒤 - 서버들.

`adopt`의 반환	뜻
`PROVEN_OK`	서버가 연결을 소유한다. 호출자의 값은 더는 열려 있지 않다.
`PROVEN_ERR_BUSY`	서버가 `max_connections`에 닿았다. 연결은 여전히 호출자의 것이고, 호출자가 닫는다.
`PROVEN_ERR_INVALID_STATE`	연결이 열려 있지 않거나, 서버가 `proven_http_event_server_stop_listening`을 들었다. 여전히 호출자의 것.
`PROVEN_ERR_INVALID_ARG`, `PROVEN_ERR_NOMEM`	여전히 호출자의 것.

같은 목적에 이르는 다른 길은 호출이 전혀 필요 없다: 루프마다 서버에 다른 주소나 포트의 리스너를 따로 주고, 앞에 있는 무언가 - 로드 밸런서나 DNS - 가 클라이언트를 퍼뜨리게 한다. 다음 절의 부하 측정은 그 방식으로 했다.

루프를 늘리는 것이 도움이 되는지는 시간이 어디에 쓰이느냐에 달렸다. 루프는 핸들러가 계산할 수 있는 양을 곱해 준다. 기다리는 핸들러에게는 아무 도움이 되지 않으며, 그런 핸들러는 애초에 루프 위에 있어서는 안 된다(10절).

테스트 스위트가 이 프로그램을 컴파일하고 실행한다:

```
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * 포트 하나 뒤의 여러 루프: 스레드 하나가 받아들이고, 각자의 루프 위에 있는 서버들에게
 * 연결을 나눠 준다.
 *
 * 연결은 루프 하나에 속하고, 그 연결의 어떤 것도 다른 스레드가 건드리지 않는다. 그래서
 * 프로세서에 걸친 확장은 각자 서버를 가진 여러 루프다 - 그리고 들어오는 연결을 그들 사이에
 * 나눠 줄 무언가가 있어야 한다. 여기서는 받아들이고 차례로 나눠 주는 스레드가 그 일을 하며,
 * 이 방식은 라이브러리가 도는 곳이면 어디서나 된다.
 */

enum { LOOPS = 2 };

/* 루프 하나, 그 서버, 그리고 그 핸들러가 세는 것. 그 루프의 스레드만 건드린다. */
typedef struct {
    int index;
    proven_loop_t *loop;
    proven_http_event_server_t *server;
    int served;                       /* 루프가 멈춘 뒤에만 메인 스레드가 읽는다 */
} ex_worker_t;

static ex_worker_t g_workers[LOOPS];
static proven_net_listener_t g_listener;
static atomic_bool g_stop;

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_worker_t *me = ctx;
    (void)head;
    char text[16];
    int n = snprintf(text, sizeof text, "loop %d", me->index);
    me->served++;
    (void)proven_http_stream_respond(stream, 200, NULL, 0, (proven_mem_view_t){ (const proven_byte_t *)text,
(proven_size_t)n });
}

/* 받아들이는 스레드에서 루프로 건너가는 것. 열린 연결은 복사해서는 안 되므로
 * 자기 메모리에 담겨 간다. */
typedef struct { ex_worker_t *to; proven_net_conn_t conn; } ex_handoff_t;

/* 루프의 스레드에서 - 서버에 연결을 줄 수 있는 유일한 곳. */
static void adopt_here(void *ctx) {
    ex_handoff_t *h = ctx;
    if (proven_http_event_server_adopt(h->to->server, &h->conn) != PROVEN_OK) (void)proven_net_close(&h-
>conn);   /* 거절됨: 여전히 우리 것이니 닫는다 */
    free(h);
}
```

```
/* 받아들이는 스레드. 어떤 서버도 건드리지 않는다: 받아들이고, 고르고, 게시한다. */
static void acceptor(void *arg) {
    (void)arg;
    int next = 0;
    while (!atomic_load(&g_stop)) {
        ex_handoff_t *h = malloc(sizeof *h);
        if (!h) break;
        /* 한 번에 잠깐씩만 기다려서, 멈추라는 말을 스레드가 알아채게 한다. */
        if (proven_net_accept(&g_listener, proven_net_deadline_in(20), &h->conn, NULL) != PROVEN_OK)
{ free(h); continue; }
        h->to = &g_workers[next];
        next = (next + 1) % LOOPS;
        if (proven_loop_post(h->to->loop, adopt_here, h) != PROVEN_OK) { (void)proven_net_close(&h->conn);
free(h); }
    }
}

static void run(void *arg) { (void)proven_loop_run(((ex_worker_t *)arg)->loop); }

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    // ---- 루프 둘, 각각 리스너 없는 서버 하나 ------------------------------------
    /* 리스너 없는 서버는 주어진 것만 서비스한다. */
    proven_job_sys_t *threads = NULL;
    proven_job_group_t loops, accepting;
    proven_job_group_init(&loops);
    proven_job_group_init(&accepting);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, LOOPS + 1, 8, &threads) == PROVEN_OK, "스레드 셋");
    for (int i = 0; i < LOOPS; ++i) {
        ex_worker_t *w = &g_workers[i];
        w->index = i;
        proven_http_event_server_config_t config = { .on = { .on_request = on_request }, .ctx = w };
        EXAMPLE_REQUIRE(proven_loop_create(heap, &w->loop) == PROVEN_OK &&
                        proven_http_event_server_create(w->loop, &config, &w->server) == PROVEN_OK &&
                        proven_job_group_submit(threads, &loops, run, w) == PROVEN_OK, "자기 스레드의 루프와
서버");
    }

    // ---- 리스너 하나, 그리고 나눠 주는 스레드 ------------------------------------
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_net_listen(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), 64, &g_listener,
&at) == PROVEN_OK &&
                    proven_job_group_submit(threads, &accepting, acceptor, NULL) == PROVEN_OK, "리스너와 받아
들이는 스레드");

    // ---- 연결 여덟 ---------------------------------------------------------
    /* 연결을 열어 두지 않는 클라이언트: 요청 여덟은 연결 여덟이다. */
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 0 };
    proven_http_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK, "클라이언트");
    char url[64];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u/", (unsigned)at.port);
    int answers[LOOPS] = { 0 };
    for (int i = 0; i < 8; ++i) {
        proven_http_client_response_t response;
        proven_u8str_t body = { 0 };
        EXAMPLE_REQUIRE(proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, &response) == PROVEN_OK &&
```

```
                    proven_http_client_read_all(&response, heap, &body, 64) == PROVEN_OK, "요청에 답이 온
다");
        proven_u8str_view_t text = proven_u8str_as_view(&body);
        if (text.size == 6 && memcmp(text.ptr, "loop ", 5) == 0 && text.ptr[5] - '0' < LOOPS)
answers[text.ptr[5] - '0']++;
        proven_u8str_destroy(heap, &body);
        proven_http_client_finish(&response);
    }
    proven_http_client_destroy(client);
    EXAMPLE_REQUIRE(answers[0] == 4 && answers[1] == 4, "차례로 나눠 줬다: 루프마다 연결 넷을 서비스했다");

    // ---- 해체, 이 순서로 ----------------------------------------------------------
    /* 받아들이는 쪽이 먼저다. 그래야 더는 게시되지 않는다. 그다음 루프들. 그다음 서버들. */
    atomic_store(&g_stop, true);
    proven_job_group_wait(threads, &accepting);
    (void)proven_net_listener_close(&g_listener);
    for (int i = 0; i < LOOPS; ++i) proven_loop_stop(g_workers[i].loop);
    proven_job_group_wait(threads, &loops);
    int served = 0;
    for (int i = 0; i < LOOPS; ++i) {
        /* 게시되었으나 아직 돌지 않은 것은 여기서, 서버가 사라지기 전에 돈다. */
        EXAMPLE_REQUIRE(proven_loop_poll(g_workers[i].loop, PROVEN_NET_DONT_WAIT) == PROVEN_OK, "마지막 한 바
퀴");
        served += g_workers[i].served;
        proven_http_event_server_destroy(g_workers[i].server);
        proven_loop_destroy(g_workers[i].loop);
    }
    EXAMPLE_REQUIRE(served == 8, "모두 여덟 요청이 서비스되었다");
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

14. 연결 하나가 붙드는 것, 측정한 것, 그리고 여기 없는 것

연결 하나가 붙드는 것. 요청이 없는 연결은 자기 구조체를 붙들 뿐 버퍼는 붙들지 않는다: 요청은 루프의 임시 작업용 버퍼로 읽히고, 바이트는 요청이 아직 완전하지 않을 때에만 따로 복사된다. x86-64 Linux에서 등록된 테스트는 한가한 평문 HTTP 연결 하나에 힙 480바이트를 재고 512를 넘으면 실패하며, 한가한 평문 WebSocket 연결 하나에 976바이트를 재고 1 KiB를 넘으면 실패한다. TLS 연결은 엔진의 상태를 더한다. 한가할 때 약 1.25 KiB다(14장 9절). 요청 사이에서 1초 조용하면 서버는 레코드 키의 펼친 형태를 (지워서) 돌려주고 원래 키만 남긴다. 다음 요청이 그것을 다시 만드는데, AES 명령이 있는 프로세서에서 방향마다 약 0.1마이크로초 걸린다. 1초 조용한 TLS 연결은 모두 합쳐 1,088바이트를 재고, 테스트는 2 KiB에서 실패한다 - 평문 연결의 목표가 1 KiB인 것과 달리 TLS의 목표다. **HTTP/2 연결은 크기가 다르다**: 작은 요청 하나 뒤 한가할 때 2,360바이트를 재고 3 KiB를 넘으면 실패하며, 기억하게 된 헤더 필드만큼 자라 두 압축 테이블이 가득 차면 약 12.8 KiB가 된다 (17장 6절). 아래 수치는 HTTP/1.1의 것이다. 같은 부하 아래 HTTP/2의 수치는 17장 8절에 있다.

측정한 것. 테스트 스위트에 들어 있지 않은 부하 프로그램을, 기계 한 대에서, 한 번 돌린 결과다: 3.5 GHz의 16스레드 x86-64 데스크톱 프로세서, 62 GB, Linux 6.12, GCC `-O2`. 서버와 클라이언트는 그 기계의 서로 다른 프로세스이고 루프백 인터페이스로, 평문 HTTP로 말한다. 모든 연결이 요청 하나를 보낸 뒤 열린 채 남는다. 그다음 정해진 비율의 연결이 10초 동안, 답을 기다렸다가 곧바로 다음 요청을 보내는 식으로 요청을 잇는다. 핸들러는 두 바이트로 답한다. **루프백은 네트워크가 아니다** - 여기에는 손실도 느린 상대도 없다 - 그리고 실제 일을 하는 핸들러는 "ok"라고 답하는 핸들러보다 느릴 것이다.

루프 하나:

유지한 연결	요청을 보내는 연결	초당 요청	왕복 시간: 중앙값, 99번째 백분위	연결당 서버 상주 메모리
1,000	10	169,000	0.05 ms, 0.14 ms	479 B
50,000	10	175,000	0.04 ms, 0.11 ms	481 B
10,000	100	184,000	0.5 ms, 0.9 ms	486 B
50,000	500	171,000	2.8 ms, 5.2 ms	481 B
100,000	1,000	155,000	6.4 ms, 9.7 ms	480 B

표가 말하는 것: **아무 일도 하지 않는 연결은 바쁜 연결에게 아무 비용도 지우지 않는다** - 연결 열 개는 오만 개 사이에서도 천 개 사이에서와 같은 빠르기로 서비스된다 - 그리고 메모리는 비례해서 든다. 서버에서 하나에 약 480바이트다. 루프 하나는 초당 정해진 양의 일을 하므로, 더 많은 연결이 한꺼번에 물으면 각자 더 오래 기다린다: 왕복 시간은 유지한 수가 아니라 묻는 수에 따라 늘어난다. 어느 행에서도 연결은 하나도 잃지 않았다. 100,000개를 여는 데 96초가 걸렸고 50,000개는 1.7초였다. 그것은 포트 범위가 바닥나면서 클라이언트 쪽 시스템이 빈 포트를 찾느라 쓴 시간으로 보이며 서버의 시간이 아니다: 블로킹 서버도 같은 단계에 그만큼 또는 그 이상 걸렸다.

소켓에 대한 시스템 자신의 비용은 별개이고 더 크다: 같은 실행들에서 커널의 slab 메모리는 연결 하나에, 그 양 끝을 합쳐, 약 7.5 KiB씩 늘었다.

여러 루프. 클라이언트 프로세스 넷이 각각 연결 12,500개를 유지하고 그 1%가 요청을 보낸다:

루프	초당 요청, 모든 클라이언트 합계	왕복 시간: 중앙값
1	167,000	2.9 ms
2	337,000	1.4 ms
4	514,000	1.0 ms

비교를 위한 11장의 블로킹 서버. 같은 프로그램에서, 핸들러를 자기 루프의 스레드에 둔 채: 같은 100,000개의 연결을 유지했고 조금 더 빨리 답했다 - 50,000개를 유지할 때 초당 186,000 요청, 이쪽은 171,000. 그 값은 메모리다: 연결당 39 KiB를 할당하고 그중 10.9 KiB가 상주한다. 이쪽은 480바이트다. 그리고 그것은 핸들러가 기다리는 스레드 하나다. 이 장의 서버를 고를 이유는 1절의 것들이지, 사소한 핸들러에서의 순수한 속도가 아니다.

v0.80.0까지 Windows에서는 한계가 낮았다. 거기서 루프는 `WSAPoll`로 기다렸는데, 그것은 호출마다 모든 소켓을 들여다본다. 같은 프로그램의 양쪽 끝을 Windows 11 가상 머신 하나에 두고, 연결 열 개가 요청을 보낼 때:

유지한 연결	초당 요청	왕복 시간: 중앙값
100	40,000	0.2 ms
1,000	8,100	1.0 ms
2,000	2,700	3.2 ms
5,000	480	18 ms
10,000	150	66 ms

클라이언트도 같은 루프 구현 위에 있었으므로 각 수치는 그 비용을 두 번 싣는다. 20,000개의 연결을 청했을 때 그 기계는 포트가 바닥나기 전에 16,369개를 열었다. v0.81.0부터 그곳의 루프는 I/O 완료 포트 위에 지어졌다(5절). 완료 포트는 끝난 작업만 알려 주고 한가한 소켓을 들여다보지 않는다. 같은 기계에서 같은 프로그램을 연달아 두 번 돌린 결과다(칸마다 두 수치. 클라이언트도 완료 포트 위에 있다):

유지한 연결	요청하는 연결	초당 요청	왕복 시간: 중앙값, 99번째 백분위	서버의 연결당 전용 메모리

100	10	42,000 / 46,000	0.18 / 0.16 ms, 0.8 / 0.4 ms	-
1,000	10	50,000 / 50,000	0.15 ms, 0.4 ms	1.2 KB
2,000	10	55,000 / 51,000	0.15 ms, 0.3 / 0.4 ms	1,065 B
5,000	10	48,000 / 36,000	0.16 / 0.26 ms, 0.4 ms	1,049 B
10,000	10	53,000 / 51,000	0.15 ms, 0.4 ms	1,040 B
10,000	100	47,000 / 48,000	2.0 / 1.9 ms, 3.9 / 4.2 ms	1,045 B
16,372 (20,000 요청)	9	53,000 / 54,000	0.13 ms, 0.3 ms	1,047 B

연결 열 개는 천 개 사이에서와 만 육천 개 사이에서 똑같이 빠르게 처리된다: 10,000개를 유지할 때 `WSAPoll`보다 요청은 약 350배, 왕복 시간은 400분의 1이다. 서버의 할당자가 쥔 양은 모든 행에서 한가한 연결 하나에 536바이트다(등록된 시험의 수치와 같다). 힙 자체의 장부까지 더한 프로세스의 전용 메모리는 연결 하나에 약 1 KB다. 공유 호스트 위의 가상 머신이라 한 실행의 한 행이 다른 실행보다 4분의 1쯤 느릴 수 있다(5,000개). 포트는 여전히 약 16,370개에서 바닥났다. selector도 완료 포트 위에 있는 11장의 블로킹 서버는 1,000개를 유지할 때 초당 50,000 / 54,000, 10,000개에서 53,000 / 52,000 요청, 중앙값 0.15 ms를 냈고, 연결 하나에 전용 메모리 40 KB를 썼다.

여기 없는 것.

- **이름값을 하는 부하 시험.** 위의 수치는 기계 한 대가 자기 자신과 말한 것이다: 네트워크도, 느리거나 적대적인 클라이언트도, 부하 속의 TLS도 없고, 아무 일도 하지 않는 핸들러 하나뿐이다.
- 이 서버 위의 WebSocket 아닌 업그레이드. 프록시를 거치거나 HTTP/2 위에 놓인 WebSocket 클라이언트.
- **이벤트 구동 클라이언트에서:** 리다이렉트, 인증, 쿠키, 프록시, 이름 해석(대신 무엇을 할지는 12절에 있다). 파이프라이닝, 그리고 기다리는 요청들 사이의 시작 순서 말고 다른 순서.
- **루프의 스레드 밖에서 하는 TLS 핸드셰이크.** 하나마다 루프는 약 2 ms 동안 다른 아무것도 서비스하지 못한다(14장 9절).
- **파일을 복사 없이 보내기**, HTTP/2. (압축된 응답은 16장 6절에 있다: 서버에서는 `proven_http_stream_compress` - Zstandard는 설정의 `compress_zstd` - , 클라이언트 설정에서는 `decompress` - Zstandard는 `decompress_zstd`, Brotli는 `decompress_brotli` - 다.)
- 루프로 **파일, 시그널, 자식 프로세스 지켜보기**. 루프는 소켓을 지켜본다.

어떻게 시험했나. 등록된 테스트는 루프의 타이머와 게시를 한 스레드에서, 그리고 여러 스레드에서 몬다 - 해당 콜백 안에서의 취소를 포함해서. 5절의 모든 작업을 시스템의 셀렉터에서, 그리고 poll 종류에서 다시 몬다 - 각각 한 번 답을 받고, 그것을 시작한 호출 안에서는 결코 받지 않음을, 취소, 모든 것이 진행 중일 때의 닫기, 실패하는 송신, 느린 독자에게 가는 3 MiB를 거쳐 확인한다. 그리고 서버를 루프백 위에서, 평문과 TLS로, 7절과 9절 표의 모든 행에 걸쳐 돌린다: 즉시 하는 답, 타이머에서 하는 답, 다른 스레드에서 하는 답, pause와 resume이 있는 업로드, 쓰기가 한 번 거절될 때까지 읽지 않는 클라이언트로의 16 MB 다운로드, 파이프라이닝, 각 거절, 각 타임아웃, 사라지는 클라이언트, 답하지 않은 요청을 둔 채 해제되는 서버 - `on_request`마다 `on_done`이 한 번 불렸는지 확인하면서. WebSocket은 12장의 블로킹 클라이언트와 날 소켓을 상대로 11절 표들의 행에 걸쳐 돌린다 - `PROVEN_ERR_NOMEM`, 그리고 `proven_ws_event_accept`와 보내기 호출의 `PROVEN_ERR_RESET`만 빼고. 이것들은 일으키지 않는다. 클라이언트는 같은 루프 위의 이 장의 서버와, 테스트가 직접 써 주는 답을 상대로 12절 표들의 행에 걸쳐 돌린다 - `PROVEN_ERR_UNREACHABLE`, `PROVEN_ERR_NOMEM`, 만료된 인증서, 그리고 네 시간 한도 가운데 셋만 빼고: 지나가게 만드는 것은 응답 타임아웃뿐이다. 이 모두를 기계에 부하를 건 채 주소 검사와 미정의 동작 검사 아래에서 수백 번, 그리고 스레드 검사 아래에서 돌렸다. 여러 루프는 받아들이는 스레드 하나 뒤의 세 스레드 위 세 서버로, 평문과 TLS로, 스레드 검사 아래에서 돌린다. 루프의 휠 한 바퀴(16.4초)보다 먼 타이머는 테스트가 움직이는 시계로 시험한다. **하지 않은 것:** 나열한 경우를

넘어서는 적대적 상대와의 실행, 공개된 WebSocket 적합성 스위트를 이 서버에 돌리는 것, 그리고 위에 적은 한 번의 실행을 넘어서는 부하.

16장: 압축

3부 — 자료 구조. 선행 조건: 압축기를 만들 때 쓰는 할당자(allocator) - 예제에서는 힙(heap) 할당자 - 는 2장, 체크섬은 4장. 이 장을 마치면 어떤 길이의 버퍼나 스트림도 압축하고 풀 수 있고, `gzip`과 `zlib` 포장을 읽고 쓸 수 있으며, Zstandard를 읽고 쓸 수 있고, Brotli를 읽을 수 있고, 낯선 이의 데이터가 얼마까지 부풀면 - 그리고 메모리를 얼마나 쓰게 하면 - 거부할지 정할 수 있다. 그리고 무엇을 시험했고 얼마나 빠른지 말할 수 있다.

이 장은 `deflate.h`를, 7절에서는 `zstd.h`를, 8절에서는 `brotli.h`를 다룬다. 모두 순수한 계산이다 - 할당은 여러분이 준 할당자로만 하고, 운영체제에서는 아무것도 받지 않는다 - 그래서 프리스탠딩(freestanding) 빌드에서 쓸 수 있다. 입력과 출력은 여러분의 메모리를 가리키는 뷰(view)로 주고받는다. 6절은 예외다: 11장과 15장의 서버와 클라이언트에서 다루는 압축된 HTTP 응답 이야기다.

목차

1. DEFLATE가 하는 일, 한 쪽으로
2. 버퍼 전체
3. 스트림
4. 레벨, 윈도, 그리고 그 비용
5. 남이 보낸 데이터
6. HTTP 위에서
7. Zstandard
8. Brotli
9. 무엇을 받아들이고, 무엇이 없으며, 어떻게 시험했는가

1. DEFLATE가 하는 일, 한 쪽으로

DEFLATE(RFC 1951)는 `.gz` 파일, HTTP의 `Content-Encoding: gzip`, PNG 이미지, ZIP 압축 파일, WebSocket의 `permessage-deflate` 안에 있는 압축이다. 서른 살이고, 가장 좋은 것은 아니며, 모두가 읽는다.

차례로 두 가지를 한다:

- **되풀이를 참조로 바꾼다.** 다음 몇 바이트가 지난 32 KiB 안에 나온 적이 있으면, 그 바이트 대신 "거리 바이트 전과 같은 것을 길이 바이트만큼"이라고 쓴다. 로그 파일, 웹 페이지, 소스 코드는 대부분 되풀이다.
- **남은 것을 길이가 다른 코드로 쓴다**: 흔한 것은 짧게, 드문 것은 길게.

그래서 텍스트는 삼분의 일이나 사분의 일로 줄고, 숫자 표는 흔히 더 줄며, **이미 압축되었거나 암호화된 데이터는 전혀 줄지 않는다** - 몇 바이트 늘어난다.

압축된 데이터가 파일이나 메시지가 되려면 둘레에 무언가가 필요하다. 선택은 셋이고, **양 끝이 같은 것을 골라야 한다**:

형식	데이터의 둘레	쓰이는 곳
`PROVEN_DEFLATE_GZIP`	열 바이트 헤더, 그리고 끝에 데이터의 CRC-32와 길이(RFC 1952)	`.gz` 파일. HTTP의 `gzip`
`PROVEN_DEFLATE_ZLIB`	두 바이트 헤더와 데이터의 Adler-32(RFC 1950)	PNG. HTTP의 `deflate`
`PROVEN_DEFLATE_RAW`	없음	자기 포장이 있는 다른 형식의 안: ZIP 항목, `permessage-deflate`

체크섬이 있는 둘은 손상을 알아챈다. raw는 그렇지 않다: 바뀐 비트는 다른 데이터나 오류를 내고, 어느 쪽인지 아무것도 말해 주지 않는다.

2. 버퍼 전체

```
proven_size_t proven_deflate_bound(proven_deflate_format_t format, proven_size_t input_size);
proven_err_t proven_deflate_all(const proven_deflate_options_t *options, proven_mem_view_t in,
```

```
                                proven_mem_mut_t out, proven_size_t *written);
proven_err_t proven_inflate_all(proven_allocator_t alloc, proven_deflate_format_t format, proven_mem_view_t
in,
                                proven_mem_mut_t out, proven_size_t *written, proven_size_t *consumed);
```

데이터가 메모리에 있고 결과를 둘 자리도 그렇다면 이 두 호출이 전부다. 둘 다 여러분의 버퍼에 쓴다.

- **압축**은 확실히 들어가려면 `proven_deflate_bound` 바이트의 버퍼가 필요하다 - 입력의 크기에 조금 더. 압축되지 않는 데이터는 약간 커지기 때문이다. 더 작으면 `PROVEN_ERR_OUT_OF_BOUNDS`로 실패할 수 있다.
- **풀기**는 결과가 얼마나 커도 되는지를 여러분이 정해야 한다. 압축된 데이터가 정하게 두지 않기 때문이다. **`out`의 크기가 한계다**: 더 많이 낼 스트림은, 그 자체가 아무리 작아도, `PROVEN_ERR_OUT_OF_BOUNDS`로 거부된다. 버퍼를 대신 키워 주는 판이 왜 없는지는 5절에 있다.

테스트 스위트가 컴파일하고 실행한다:

```
#include <string.h>

/*
 * 버퍼를 압축하고, 되찾고, 약속한 것보다 많이 건네받기를 거부한다: 버퍼 전체를 다루는 두 호출과,
 * 남이 보낸 것을 풀 때의 한 가지 규칙.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    /* 압축되는 것: 텍스트는 스스로를 되풀이한다. */
    static proven_byte_t text[4000];
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];

    /* 버퍼가 한계값이 말하는 만큼 크면 압축은 자리가 없어 실패하지 않는다. */
    static proven_byte_t packed[4200];
    proven_size_t packed_len = 0;
    proven_deflate_options_t options = { .alloc = heap, .format = PROVEN_DEFLATE_GZIP };
    EXAMPLE_REQUIRE(proven_deflate_bound(PROVEN_DEFLATE_GZIP, sizeof text) <= sizeof packed, "4,000바이트의 한
계값은 4,000을 조금 넘는다");
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ packed, sizeof packed }, &packed_len) == PROVEN_OK,
                    "로그 4,000바이트가 압축된다");
    EXAMPLE_REQUIRE(packed_len < 200, "200 미만으로: 같은 줄이 되풀이되므로");
    EXAMPLE_REQUIRE(packed[0] == 0x1f && packed[1] == 0x8b, "나온 것은 gzip 멤버다 - .gz 파일, 또는 Content-
Encoding: gzip인 HTTP 본문");

    /* 풀기: 출력 버퍼가 곧 한계다. 여기서는 데이터와 정확히 같은 크기다. */
    static proven_byte_t back[4000];
    proven_size_t back_len = 0, used = 0;
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len },
(proven_mem_mut_t){ back, sizeof back }, &back_len, &used) == PROVEN_OK,
                    "풀린다");
    EXAMPLE_REQUIRE(back_len == sizeof text && memcmp(back, text, sizeof text) == 0 && used == packed_len, "넣
은 그대로, 멤버의 모든 바이트를 써서");

    /* 같은 멤버를 더 작은 버퍼에 풀면 거부된다. 이것이 "압축 폭탄"에 대한 방어의 전부다: 얼마나
     * 받을지를 당신이 말하고, 자리는 그만큼뿐이다. 몇백 바이트가 이 프로그램에게 기가바이트를
     * 할당하게 만들 수 없다. */
    static proven_byte_t small[1000];
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len },
(proven_mem_mut_t){ small, sizeof small }, &back_len, NULL) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "4,000바이트는 1,000에 들어가지 않는다: PROVEN_ERR_OUT_OF_BOUNDS");

    /* 손상은 알아챈다. gzip은 데이터의 CRC-32를 싣는다. 압축된 스트림 어디서든 비트 하나가
```

```
     * 바뀌면 데이터가 바뀌거나 형식이 깨진다. */
    packed[packed_len / 2] ^= 0x04;
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len },
(proven_mem_mut_t){ back, sizeof back }, &back_len, NULL) == PROVEN_ERR_INVALID_FORMAT,
                    "바뀐 비트 하나: PROVEN_ERR_INVALID_FORMAT");
    packed[packed_len / 2] ^= 0x04;
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ packed, packed_len -
1 }, (proven_mem_mut_t){ back, sizeof back }, &back_len, NULL) == PROVEN_ERR_INVALID_FORMAT,
                    "잘린 멤버는 멤버가 아니다");

    /* 포장은 양 끝에서 함께 정하는 선택이다. 같은 데이터가 zlib로는 열두 바이트 작고(더 짧은
     * 헤더, 더 짧은 체크섬), raw는 더 작다 - 그리고 체크섬이 아예 없어서 손상이 눈에 띄지 않을 수
     * 있다. */
    proven_size_t zlib_len = 0, raw_len = 0;
    static proven_byte_t other[4200];
    options.format = PROVEN_DEFLATE_ZLIB;
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ other, sizeof other }, &zlib_len) == PROVEN_OK && zlib_len == packed_len - 12,
                    "zlib 포장: gzip보다 12바이트 적다");
    EXAMPLE_REQUIRE(proven_inflate_all(heap, PROVEN_DEFLATE_GZIP, (proven_mem_view_t){ other, zlib_len },
(proven_mem_mut_t){ back, sizeof back }, &back_len, NULL) == PROVEN_ERR_INVALID_FORMAT,
                    "그리고 gzip이 아니다: 푸는 쪽은 무엇을 읽는지 들어야 한다");
    options.format = PROVEN_DEFLATE_RAW;
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t)
{ other, sizeof other }, &raw_len) == PROVEN_OK && raw_len == zlib_len - 6,
                    "raw: 헤더도 체크섬도 없이 다시 6바이트 적다");

    /* 압축되지 않는 것은 조금 커진다 - 최악의 경우 한계값까지. */
    static proven_byte_t noise[4000], noisy[4200];
    proven_u32 x = 2463534242u;
    for (proven_size_t i = 0; i < sizeof noise; ++i) { x ^= x << 13; x ^= x >> 17; x ^= x << 5; noise[i] =
(proven_byte_t)x; }
    proven_size_t noisy_len = 0;
    options.format = PROVEN_DEFLATE_GZIP;
    EXAMPLE_REQUIRE(proven_deflate_all(&options, (proven_mem_view_t){ noise, sizeof noise },
(proven_mem_mut_t){ noisy, sizeof noisy }, &noisy_len) == PROVEN_OK &&
                    noisy_len > sizeof noise && noisy_len <= proven_deflate_bound(PROVEN_DEFLATE_GZIP, sizeof
noise),
                    "무작위 바이트는 조금 더 크게 나오고, 한계값을 넘지는 않는다");

    return EXAMPLE_OK();
}
```

3. 스트림

```
proven_err_t proven_deflate_create(const proven_deflate_options_t *options, proven_deflate_t **out);
proven_err_t proven_deflate(proven_deflate_t *z, proven_mem_view_t in, proven_size_t *consumed,
                            proven_mem_mut_t out, proven_size_t *produced, proven_deflate_flush_t flush, bool
*done);
void proven_deflate_reset(proven_deflate_t *z);
void proven_deflate_destroy(proven_deflate_t *z);

proven_err_t proven_inflate_create(proven_allocator_t alloc, proven_deflate_format_t format, proven_inflate_t
**out);
proven_err_t proven_inflate(proven_inflate_t *z, proven_mem_view_t in, proven_size_t *consumed,
                            proven_mem_mut_t out, proven_size_t *produced, bool *done);
void proven_inflate_reset(proven_inflate_t *z);
void proven_inflate_restart(proven_inflate_t *z);
void proven_inflate_destroy(proven_inflate_t *z);
```

데이터가 메모리보다 큰 파일이거나, 아직 만들어지고 있는 응답이거나, 연결이라면, 두 방향 모두 같은 방식으로 동작한다: 상태를 쥔 객체 하나와, **입력을 조금 받아 출력을 조금 채우는 단계 함수 하나**.

단계의 계약은 짧고, 한 번은 읽어 둘 만하다:

- `in`에서 할 수 있는 만큼 소비하고 `out`에 할 수 있는 만큼 생산하며, 각각 얼마인지 알려 준다. 할당하지 않고, `out`을 넘어 쓰지 않는다. (`out` 안에서는 해제기가 보고한 바이트 뒤로 일곱 바이트까지 쓸 수 있다: 비어 있는 자리를 넘길 것.)
- **아무것도 소비하지 않고 아무것도 생산하지 않은 `PROVEN_OK`는 오류가 아니다.** 단계가 둘 중 하나를 더 원한다는 뜻이다: 입력을 더, 또는 자리를 더. 생기면 다시 부른다. 그냥 도착하기를 멈춘 스트림도 꼭 이렇게 보이므로, 더 올 입력이 없음을 아는 호출자는 이것을 잘린 스트림으로 다뤄야 한다.
- `*done`은 스트림이 끝났고 모든 것이 생산되었을 때 참이 된다 - 데이터가 준 자리를 정확히 채웠을 때에도. 끝 뒤의 입력은 소비되지 않는다: `*consumed`가 스트림이 어디서 끝났는지 알려 준다.

압축할 때 `flush`는 출력이 얼마나 급한지를 말한다.

`flush`	뜻	쓸 때
`PROVEN_DEFLATE_FLUSH_NONE`	더 온다. 출력은 압축기가 블록 하나 만큼 모았을 때 나온다 - 오래 걸릴 수 있다	아래 두 경우가 아닌 모든 때
`PROVEN_DEFLATE_FLUSH_SYNC`	지금까지 준 모든 것을 지금 풀 수 있어야 한다. 스트림은 이어진다	쓴 것을 누군가 기다릴 때: 연결 위의 메시지, 실시간 로그의 한 줄. 매번 몇 바이트와 압축률 얼마를 치른다
`PROVEN_DEFLATE_FLUSH_FINISH`	그것이 마지막 입력이었다: 스트림을 끝낸다	끝에서 한 번. `*done`이 될 때까지 이것과 함께, 자리를 더 주며 계속 부른다

sync flush는 바이트 경계에서 끝나고, 그것이 내는 마지막 네 바이트는 언제나 `00 00 ff ff`다. WebSocket의 `permessage-deflate`는 메시지마다 그 네 바이트를 뗀 sync flush로 보내고, 받는 쪽이 그것을 다시 붙인다 (12장 9절: WebSocket 드라이버는 그 네 바이트를 떼고 다시 붙이며, context takeover에서는 연결 내내 압축기 하나와 해제기 하나를 쓴다). raw 스트림이 끝나지 않아도 되는 까닭도 그것이다: 해제기는 조각조각 받고, `*done`은 참이 되지 않으며, 앞선 조각에 대해 기억하는 것이 뒤의 조각을 작게 만든다.

`proven_deflate_reset`과 `proven_inflate_reset`은 같은 객체에서 새 스트림을 시작한다. 하나 더 만드는 것보다 싸다. `*done` 뒤에, 첫 gzip 멤버에 이어지는 둘째를 읽어야 하는 해제기는 reset하고 첫째가 끝난 곳부터 받는다. `proven_inflate_restart`도 새 스트림을 시작하지만 마지막 32 KiB를 지킨다. 그래서 새 스트림의 일치가 옛 스트림 안을 가리킬 수 있다. context takeover를 쓰는 `permessage-deflate`를 위한 것이다: 보내는 쪽이 메시지를 마지막 블록으로 끝내도, 다음 메시지는 여전히 앞의 메시지들을 가리킨다.

테스트 스위트가 컴파일하고 실행한다:

```
#include <string.h>

/*
 * 스트림으로서의 압축: 조각으로 들어와 조각으로 나가는 데이터를, 가진 크기 그대로의 버퍼로.
 * 같은 루프가 어떤 길이의 파일에도, 아직 만들어지고 있는 응답에도, 연결에도 통한다.
 */

/* 압축된 바이트가 가는 곳. 실제 프로그램이라면 파일이나 소켓에 쓴다. */
static proven_byte_t g_wire[8192];
static proven_size_t g_wire_len;

/* 압축기에 입력을 주고 가진 출력을 64바이트씩 받는다. 입력을 모두 가져갈 때까지 - 그리고
 * `flush`가 그 이상을 청하면 그것까지 모두 내놓을 때까지. */
static bool feed(proven_deflate_t *z, const char *text, proven_deflate_flush_t flush, bool *done) {
    proven_mem_view_t in = { (const proven_byte_t *)text, strlen(text) };
    for (;;) {
```

```
        proven_byte_t out[64];
        proven_size_t consumed = 0, produced = 0;
        if (proven_deflate(z, in, &consumed, (proven_mem_mut_t){ out, sizeof out }, &produced, flush, done) !=
PROVEN_OK) return false;
        if (g_wire_len + produced > sizeof g_wire) return false;
        memcpy(g_wire + g_wire_len, out, produced);
        g_wire_len += produced;
        in.ptr += consumed; in.size -= consumed;
        /* 더 줄 것이 없고, 내놓은 것이 버퍼 하나에 못 미쳤다: 밀린 것이 없다. */
        if (in.size == 0 && produced < sizeof out) return true;
    }
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();

    /* 압축기는 한 번 만든다. 옵션의 0은 기본값을 뜻한다: 레벨 6, 32 KiB 윈도. 레벨 1은 더
     * 빠르고 조금 덜 압축하며, 9는 더 느리고 조금 더 압축한다. */
    proven_deflate_options_t options = { .alloc = heap, .format = PROVEN_DEFLATE_ZLIB, .level = 6 };
    proven_deflate_t *z = NULL;
    EXAMPLE_REQUIRE(proven_deflate_create(&options, &z) == PROVEN_OK, "압축기");

    /* 조각은 오는 대로 들어간다. FLUSH_NONE이면 압축기는 블록 하나만큼 모일 때까지 받은 것을
     * 쥐고 있다 - 오랫동안 아무것도 나오지 않을 수 있고, 그래서 잘 압축한다. */
    bool done = false;
    EXAMPLE_REQUIRE(feed(z, "first piece of the stream; ", PROVEN_DEFLATE_FLUSH_NONE, &done), "조각 하나가 들
어간다");
    const proven_size_t after_first = g_wire_len;
    EXAMPLE_REQUIRE(after_first <= 2 && !done, "아직 두 바이트 헤더 말고는 나온 것이 없다");

    /* sync flush는 말한다: 상대가 여기까지를 지금 읽을 수 있어야 한다. 매번 몇 바이트가 든다 -
     * 누군가 기다릴 때 쓰고, 쓸 때마다 쓰지는 말 것. */
    EXAMPLE_REQUIRE(feed(z, "second piece of the stream; ", PROVEN_DEFLATE_FLUSH_SYNC, &done) && g_wire_len >
after_first + 20 && !done,
                    "sync flush 뒤에는 두 조각 모두 선 위에 있다");
    const proven_size_t at_flush = g_wire_len;

    /* FINISH는 스트림을 끝낸다: 마지막 블록, 그리고 zlib와 gzip이라면 체크섬. */
    EXAMPLE_REQUIRE(feed(z, "third and last piece.", PROVEN_DEFLATE_FLUSH_FINISH, &done) && done, "마지막 조
각, 그리고 스트림이 끝났다");

    /* 받는 쪽: 10바이트씩 받아 16바이트 버퍼에 푸는 해제기. */
    proven_inflate_t *u = NULL;
    EXAMPLE_REQUIRE(proven_inflate_create(heap, PROVEN_DEFLATE_ZLIB, &u) == PROVEN_OK, "해제기");
    char text[200];
    proven_size_t text_len = 0, at = 0, seen_at_flush = 0;
    bool ended = false;
    /* 먼저 sync flush가 돌아왔을 때 쓰여 있던 것만, 그다음 나머지. */
    for (proven_size_t limit = at_flush; !ended; limit = g_wire_len) {
        for (;;) {
            proven_byte_t out[16];
            proven_size_t consumed = 0, produced = 0;
            const proven_size_t give = limit - at < 10 ? limit - at : 10;
            if (proven_inflate(u, (proven_mem_view_t){ g_wire + at, give }, &consumed, (proven_mem_mut_t)
{ out, sizeof out }, &produced, &ended) != PROVEN_OK) break;
            if (text_len + produced > sizeof text) break;
            memcpy(text + text_len, out, produced);
            text_len += produced;
            at += consumed;
            /* 소비도 생산도 없는 PROVEN_OK: 받은 것을 다 쓰고 더 원한다. 오류가 아니다 -
```

```
             * 스트림은 그렇게 기다린다. */
            if (ended || (consumed == 0 && produced == 0)) break;
        }
        if (limit == at_flush) seen_at_flush = text_len;
        else break;
    }
    static const char whole[] = "first piece of the stream; second piece of the stream; third and last
piece.";
    EXAMPLE_REQUIRE(ended && text_len == sizeof whole - 1 && memcmp(text, whole, text_len) == 0, "세 조각이 하
나의 텍스트로 나온다");
    EXAMPLE_REQUIRE(at == g_wire_len, "스트림은 압축기가 쓰기를 멈춘 바로 그곳에서 끝났다");
    EXAMPLE_REQUIRE(seen_at_flush >= 55, "flush 지점에서 받는 쪽은 이미 앞의 두 조각을 모두 갖고 있었다");

    /* 두 객체는 다시 만들지 않고 다음 스트림에 다시 쓴다. */
    proven_deflate_reset(z);
    proven_inflate_reset(u);
    g_wire_len = 0;
    EXAMPLE_REQUIRE(feed(z, "another stream", PROVEN_DEFLATE_FLUSH_FINISH, &done) && done, "같은 압축기, 새 스
트림");
    proven_byte_t again[32];
    proven_size_t consumed = 0, produced = 0;
    EXAMPLE_REQUIRE(proven_inflate(u, (proven_mem_view_t){ g_wire, g_wire_len }, &consumed, (proven_mem_mut_t)
{ again, sizeof again }, &produced, &ended) == PROVEN_OK &&
                    ended && produced == 14 && memcmp(again, "another stream", 14) == 0, "같은 해제기가 그것을
읽는다");

    proven_deflate_destroy(z);
    proven_inflate_destroy(u);
    return EXAMPLE_OK();
}
```

4. 레벨, 윈도, 그리고 그 비용

압축기는 `proven_deflate_options_t`로 만든다. 0으로 초기화하고, `alloc`을 두고, 바꾸고 싶은 것을 둔다:

필드	뜻
`alloc`	필수. 압축기는 할당 하나이며 여기서 만들고 `proven_deflate_destroy`가 해제한다
`format`	1절의 포장. 0은 `PROVEN_DEFLATE_RAW`
`level`	1~9: 되풀이를 얼마나 열심히 찾는가. **0은 기본값 6이다.** -1은 전혀 압축하지 않는다 - 데이터를 감싸기만 한다
`window_bits`	되풀이가 얼마나 멀리 있어도 되는가, 2의 거듭제곱으로: 9(512바이트)~15(32 KiB). **0은 기본값 15다**

레벨은 시간과 크기를 맞바꾸고, 그 교환은 한쪽으로 기운다. 개발 기계에서 이 라이브러리 자신의 소스와 매뉴얼(텍스트 3.2 MiB)을 압축하면:

레벨	압축 결과	압축	풀기
1	29.4%	34 MiB/s	330 MiB/s
6(기본값)	25.2%	16 MiB/s	375 MiB/s
9	25.0%	9 MiB/s	380 MiB/s

1에서 6으로 가면 속도의 절반으로 크기의 칠분의 일을 산다. 6에서 9로 가면 또 속도의 절반 가까이로 거의 아무것도 사지 못한다. 풀기는 어떤 레벨이 쓰였는지 상관하지 않는다.

모두가 쓰는 구현인 zlib는 같은 데이터를 같은 크기로 압축하고 - 레벨 6과 9에서 사백분의 일 이내, 레벨 1에서는 zlib 쪽이 조금 더 크다 - 압축은 두 배쯤, 레벨 1에서는 세 배 가까이 빠르다. 풀기는 둘이 비슷하다: 같

은 데이터를 Python에서 부른 zlib 1.3.1은 기계의 부하에 따라 190에서 310 MiB/s를 냈다. (v0.42.0 전에는 이 라이브러리의 해제기가 한 비트씩 읽었고 75에서 90 MiB/s였다.) 압축기는 읽히도록 쓰였다. 그 수치가 어떤 프로그램에 너무 느리다면, 그 프로그램에 필요한 것은 zlib다.

윈도는 대체로 메모리의 문제다. 압축기는 윈도의 두 배와 그 옆의 표들을 쥔다. 해제기는 언제나 32 KiB를 쥔다. 상대가 어떤 윈도를 썼는지 알 수 없기 때문이다:

	메모리
해제기	약 35 KiB
압축기, `window_bits` 15(기본값)	약 325 KiB
압축기, `window_bits` 12	약 53 KiB
압축기, `window_bits` 9	약 11 KiB

윈도가 작으면 덜 압축된다 - 윈도보다 멀리 떨어진 되풀이는 찾지 못한다 - 그리고 속도에는 계획할 만한 차이가 없다. 압축기가 한꺼번에 많을 때(연결마다 하나)나 메모리가 적을 때 쓴다.

5. 남이 보낸 데이터

프로그램 밖에서 들어오는 압축 데이터가 위험한 경우이고, 까닭은 하나다:

보기보다 훨씬 클 수 있다. DEFLATE는 "마지막 바이트를 다시, 258번"을 두 바이트로, 거듭거듭 말할 수 있다. 이 라이브러리의 테스트에 있는 117바이트는 25,801바이트로 부푼다. 메가바이트 하나가 기가바이트가 될 수 있다. 받은 것을 무엇이든 필요한 만큼 자라는 버퍼에 푸는 프로그램은, 보내는 모든 이에게 자기 메모리를 바닥낼 능력을 준 것이다.

그래서 여기서는 아무것도 자라지 않는다. `proven_inflate`는 넘겨받은 공간에만 쓰고, 그것이 차면 멈추고 그렇다고 말한다. `proven_inflate_all`은 한계를 출력의 크기로 받는다. 읽고 있는 것 - 설정 파일, 요청 본문, 메시지 - 에 말이 되는 가장 큰 결과를 정하고, 그것을 넘는 것은 거부하라. 해제기 자신의 메모리는 무엇을 받든 약 35 KiB로 고정이다.

그리고 그냥 틀렸을 수 있다, 사고로든 일부러든:

반환	**언제**
`PROVEN_ERR_INVALID_FORMAT`	바이트가 해제기를 만들 때 정한 형식의 유효한 스트림이 아니다. 또는 - gzip과 zlib에서 - 끝의 체크섬이 데이터와 맞지 않는다. 또는 `proven_inflate_all`에서, 스트림이 끝나기 전에 입력이 끝났다
`PROVEN_ERR_UNSUPPORTED`	미리 정한 사전이 필요한 zlib 스트림
`PROVEN_ERR_OUT_OF_BOUNDS`	`proven_inflate_all`에서: 데이터가 준 버퍼에 들어가지 않는다

오류 뒤에 해제기는 reset할 때까지 모든 호출에서 그 오류를 돌려준다. 오류 전에 생산한 데이터를 완전한 것으로 믿어서는 안 된다: gzip과 zlib에서 체크섬이 맞는지는 맨 끝에서야 알 수 있다.

알아 둘 만한 것 둘 더.

- **압축은 압축하는 것을 새게 할 수 있다.** 비밀과 공격자가 고른 텍스트를 함께 압축하고 공격자가 결과의 크기를 볼 수 있으면, 그 크기는 고른 텍스트가 비밀의 일부를 되풀이했는지를 말해 주고, 비밀은 한 글자씩 추측될 수 있다. 이것은 실제로 HTTPS를 깼다(CRIME, BREACH). 지킬 규칙은 애플리케이션의 몫이다: 비밀을 낯선 이가 통제하는 것과 함께 압축하지 말 것.
- **raw DEFLATE에는 체크섬이 없다.** 바이트가 오는 길에 손상되었을 수 있고 둘레의 형식이 검사하지 않는다면 zlib나 gzip을 쓴다.

6. HTTP 위에서

```
void proven_http_exchange_compress(proven_http_exchange_t *exchange);      /* 11장의 서버 */
void proven_http_stream_compress(proven_http_stream_t *stream);            /* 15장의 서버 */
```

```
bool proven_http_accepts_coding(const proven_http_header_t *headers, proven_size_t count, proven_http_coding_t
coding);
unsigned proven_http_accept_weight(const proven_http_header_t *headers, proven_size_t count,
proven_http_coding_t coding);
proven_http_coding_t proven_http_content_coding(const proven_http_header_t *headers, proven_size_t count);
```

HTTP는 이것을 *콘텐츠 코딩*이라고 부른다: 서버가 본문을 압축하고 `Content-Encoding`으로 그 사실을 알리는데, `Accept-Encoding`으로 허락한 클라이언트에게만 그렇게 할 수 있다. 11장과 15장의 HTTP 드라이버 넷은 이 두 가지를 모두 한다 - 그리고 **요청하기 전에는 어느 것도 하지 않는다**. 아무것도 설정하지 않은 프로그램은 이전과 같은 바이트를 보낸다.

이 장의 나머지와 달리 이 부분은 소켓이 필요하고 프리스탠딩 빌드에는 없다 - 헬퍼 함수 셋은 예외로, `http.h`에 있는 헤더 계산일 뿐이다.

서버는 핸들러가 그 응답에 대해 요청할 때 응답을 압축한다. 응답을 시작하기 전에 호출 하나:

	일어나는 일
요청이 gzip을 허용하고, 응답에 자기 본문이 있다	`Content-Encoding: gzip`으로 보낸다 - 또는 `zstd`로, 아래
요청이 어느 쪽도 허용하지 않는다(`Accept-Encoding` 없음, `gzip;q=0`, 다른 코딩만)	그대로 보낸다
상태 204, 206, 304. 또는 여러분의 헤더에 이미 `Content-Encoding`이나 `Content-Range`가 있다	그대로 보낸다
한 번에 보내는 본문이 256바이트 미만이거나, 작아지지 않는다	그대로 보낸다
압축기에 줄 메모리가 없다	그대로 보낸다
이 모든 경우에	`Vary: Accept-Encoding`이 붙는다. 여러분의 `Vary`가 이미 그것을 말하면 붙지 않는다

`Vary`가 붙는 까닭은 응답이 이제 요청 헤더에 따라 달라지기 때문이다. 중간의 캐시가 gzip을 읽지 못하는 클라이언트에게 gzip을 건네서는 안 된다.

서버 설정에 `compress_zstd`가 있으면 Zstandard로도 압축한다. 압축을 요청한 응답은 요청의 `Accept-Encoding`이 zstd에 gzip 이상의 가중치를 줄 때 `zstd`로, 그 밖에는 이전처럼 gzip으로 나간다. curl과, zstd를 받는 브라우저는 둘을 같은 가중치로 보낸다 - `gzip, deflate, br, zstd` - 그래서 zstd를 받는다. `gzip;q=1, zstd;q=0.5`라고 쓰는 클라이언트는 gzip을 받고, zstd를 적지 않는 클라이언트는 전에 받던 것을 받는다. 같을 때 zstd로 가는 까닭은 둘 가운데 만드는 비용이 적기 때문이다(7절: 레벨 1에서 gzip의 여섯 배 속도, 크기는 gzip 1 이하). 레벨과 창은 `compress_zstd_level`(1부터 9, 0은 1)과 `compress_zstd_window_log`(10부터 23, 0은 17, 128 KiB)다. 한 번에 보내는 본문은 창과 상관없이 본문에 맞춘 표로 압축하고, 스트림으로 보내는 본문은 이 기본값에서 끝날 때까지 약 0.8 MiB를 쥔다. 레벨 6은 gzip 6의 크기 이하를 그 속도의 약 세 배로 낸다 - gzip 6의 크기를 원하는 서버를 위한 것이다. 그 표 때문에 스트림 응답 하나가 약 1.8 MiB를 쥔다.
`proven_http_accept_weight`는 같은 가중치를 돌려준다. 직접 고를 때 쓴다.

한 번에 보내는 본문은 먼저 압축한 다음 새 `Content-Length`와 함께 보낸다. 조각조각 쓰는 본문은 쓰는 대로 압축해서, 알려 준 길이와 상관없이 청크로 보낸다 - 그 길이는 여전히 여러분이 써야 할 길이다.

클라이언트는 설정에 `decompress`가 있을 때 요청하고 푼다. 그러면 `Accept-Encoding: gzip`을 보내고, `gzip`이나 `deflate`로 돌아온 본문을 여러분에게 가는 길에 푼다: `proven_http_client_read`는 여러분의 버퍼를 풀린 바이트로 채우고, 이벤트 구동 클라이언트의 `on_body`는 풀린 조각을 최대 16 KiB씩 받는다.

`decompress_zstd`도 켜면 Zstandard도 요청한다 - `Accept-Encoding: zstd, gzip` - 그리고 `zstd` 본문을 7절의 디코더와, RFC 9659가 HTTP에 정한 8 MiB 창 상한으로 같은 방식으로 푼다. 따로 켜는 까닭은 그 창 때문이다: gzip은 35 KiB면 되는데, 서버는 응답 하나에 클라이언트가 최대 8 MiB를 들고 있게 할 수 있다. `decompress` 없이 `decompress_zstd`만 켜면 `PROVEN_ERR_INVALID_ARG`다.

`decompress_brotli`를 켜면 Brotli도 요청한다 - 목록에 `br`이 들어가고, 두 스위치를 다 켜면 `Accept-Encoding: zstd, br, gzip` - 그리고 `br` 본문을 8절의 디코더로 푼다. 이 디코더는 RFC 7932가 허용하는 모든 창, 최대 16 MiB를 받아들인다. RFC 9659가 Zstandard에 대해 하듯 더 낮은 창을 정한 HTTP 명세는 없다. 따로 켜는 까닭은 Zstandard와 같다: 응답 하나에 그중 얼마를 쓰게 할지 서버가 정한다. `decompress` 없이 켜면 `PROVEN_ERR_INVALID_ARG`, `PROVEN_NO_BROTLI`로 만든 빌드(8절)에서는 `PROVEN_ERR_UNSUPPORTED`다. 서버는 `br`을 보내지 않는다: 이 라이브러리에는 Brotli 인코더가 없다.

예제: 페이지는 압축하고, 계정 페이지는 하지 않는다

```
#include <stdio.h>
#include <string.h>

/*
 * HTTP로 보내는 압축된 응답: 핸들러가 요청하고 클라이언트가 받아들이면 압축하는 서버와,
 * 압축을 요청하고 풀어 읽는 클라이언트.
 *
 * 프로그램이 말하기 전에는 둘 다 꺼져 있다. 서버 쪽은 핸들러에서 응답마다 부르는 호출 하나이고,
 * 클라이언트 쪽은 설정의 플래그 하나다.
 */

static proven_byte_t g_page[20000];        /* 제공할 페이지: 텍스트라서 압축이 잘 된다 */

static void fill_page(void) {
    proven_size_t n = 0;
    for (unsigned k = 0; n < sizeof g_page; ++k) {
        char line[48];
        int len = snprintf(line, sizeof line, "row %u of a table that says much the same\n", k);
        for (int i = 0; i < len && n < sizeof g_page; ++i) g_page[n++] = (proven_byte_t)line[i];
    }
}

static void handle(void *ctx, proven_http_exchange_t *x) {
    proven_http_server_t **server = ctx;
    const proven_http_request_t *req = proven_http_exchange_request(x);
    proven_mem_view_t page = { g_page, sizeof g_page };

    if (proven_u8str_view_eq(req->target, PROVEN_LIT("/page"))) {
        /* 응답을 시작하기 전에 호출 하나. 나머지는 서버가 한다: 요청의 Accept-Encoding을
         * 읽고, gzip이 허용되면 압축하고, Content-Encoding과 Vary와 새 Content-Length를
         * 쓴다. */
        proven_http_exchange_compress(x);
        proven_http_header_t type = { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain") };
        (void)proven_http_exchange_respond(x, 200, &type, 1, page);

    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/account"))) {
        /* 여기서는 일부러 요청하지 않는다: 이 응답은 비밀을 클라이언트가 고른 글자 옆에
         * 싣게 되는데, 압축된 본문의 크기는 그 비밀을 한 바이트씩 새어 나가게 한다.
         * 압축을 응답마다 요청하게 한 것은 핸들러만이 이를 알기 때문이다. */
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("token=... you
searched for: ...\n")));

    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/accepts"))) {
        /* 서버가 묻는 것과 같은 질문을, 스스로 고르고 싶은 핸들러가 묻는다 -
         * 예컨대 파일과 미리 압축해 둔 그 짝 가운데 하나를. */
        bool gzip = proven_http_accepts_coding(req->headers, req->header_count, PROVEN_HTTP_CODING_GZIP);
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(gzip ?
PROVEN_LIT("gzip") : PROVEN_LIT("identity")));

    } else if (proven_u8str_view_eq(req->target, PROVEN_LIT("/log"))) {
        /* 조각으로 쓰는 압축 응답은 받은 것을 붙들고 있다. flush가 그것을 내보낸다:
```

```
         * 클라이언트는 일이 끝났을 때가 아니라 지금 "started"를 읽는다. */
        proven_http_exchange_compress(x);
        if (proven_http_exchange_begin(x, 200, NULL, 0, PROVEN_HTTP_LENGTH_UNKNOWN) == PROVEN_OK &&
            proven_http_exchange_write(x, proven_mem_view_from_u8(PROVEN_LIT("started\n"))) == PROVEN_OK &&
            proven_http_exchange_flush(x) == PROVEN_OK) {
            (void)proven_http_exchange_write(x, proven_mem_view_from_u8(PROVEN_LIT("finished\n")));
        }

    } else {
        (void)proven_http_exchange_respond(x, 200, NULL, 0, proven_mem_view_from_u8(PROVEN_LIT("bye\n")));
        proven_http_server_stop(*server);
    }
}

static void serve(void *arg) { (void)proven_http_server_run(*(proven_http_server_t **)arg); }

/* `path`를 GET한다. 읽은 본문의 크기, Content-Length, 헤더가 말하는 코딩을 알려 준다. */
static bool get(proven_http_client_t *client, proven_u16 port, const char *path,
                proven_size_t *read_size, proven_u64 *content_length, proven_http_coding_t *coding,
proven_u8str_t *body) {
    char url[96];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u%s", (unsigned)port, path);
    proven_http_client_response_t resp;
    bool ok = proven_http_client_get(client, (proven_u8str_view_t){ (const proven_byte_t *)url,
(proven_size_t)n }, &resp) == PROVEN_OK && resp.status == 200;
    if (ok) {
        /* 헤더는 서버가 보낸 그대로다: 무엇이 전송됐는지는 이렇게 본다. */
        *coding = proven_http_content_coding(resp.headers, resp.header_count);
        proven_u8str_view_t length = { 0 };
        *content_length = 0;
        if (proven_http_header_find(resp.headers, resp.header_count, PROVEN_LIT("Content-Length"), &length)) {
            for (proven_size_t i = 0; i < length.size; ++i) *content_length = *content_length * 10 +
(proven_u64)(length.ptr[i] - '0');
        }
        /* 한도는 여기에 도착하는 것에 걸린다 - 클라이언트가 푼다면 풀린 바이트에. */
        (void)proven_u8str_reset(body);
        ok = proven_http_client_read_all(&resp, proven_heap_allocator(), body, 1024 * 1024) == PROVEN_OK;
        *read_size = proven_u8str_as_view(body).size;
    }
    proven_http_client_finish(&resp);
    return ok;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    fill_page();

    static proven_http_server_t *server;
    proven_http_server_config_t config = { .alloc = heap, .handler = handle, .handler_ctx = &server };
    config.compress_level = 6;              /* 1이 가장 빠르고 9가 가장 작다. 0은 이 값을 뜻한다 */
    EXAMPLE_REQUIRE(proven_http_server_create(&config, &server) == PROVEN_OK, "a server");
    proven_net_addr_t at;
    proven_err_t err = proven_http_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0),
&at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_server_destroy(server);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");
```

```
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, serve, &server) == PROVEN_OK, "the server loop
is started");

    /* 클라이언트 둘: 하나는 늘 하던 대로, 하나는 압축된 응답을 요청한다. */
    proven_http_client_config_t plain_config = { .alloc = heap, .max_idle_connections = 2 };
    proven_http_client_config_t decoding_config = { .alloc = heap, .max_idle_connections = 2, .decompress =
true };
    proven_http_client_t *plain = NULL, *decoding = NULL;
    EXAMPLE_REQUIRE(proven_http_client_create(&plain_config, &plain) == PROVEN_OK &&
                    proven_http_client_create(&decoding_config, &decoding) == PROVEN_OK, "two clients");

    proven_u8str_t body = { 0 };
    proven_size_t read_size = 0;
    proven_u64 sent = 0;
    proven_http_coding_t coding = PROVEN_HTTP_CODING_IDENTITY;

    /* 아무것도 설정하지 않은 클라이언트는 Accept-Encoding을 보내지 않고, 달라지는 것이 없다. */
    EXAMPLE_REQUIRE(get(plain, at.port, "/page", &read_size, &sent, &coding, &body), "the page, to a client
that did not ask");
    EXAMPLE_REQUIRE(coding == PROVEN_HTTP_CODING_IDENTITY && sent == sizeof g_page && read_size == sizeof
g_page, "arrives as it is");

    /* 요청한 쪽은 같은 바이트를 그 일부만으로 받는다. */
    EXAMPLE_REQUIRE(get(decoding, at.port, "/page", &read_size, &sent, &coding, &body), "the page, to a client
that asked");
    EXAMPLE_REQUIRE(coding == PROVEN_HTTP_CODING_GZIP && read_size == sizeof g_page &&
                    memcmp(proven_u8str_as_view(&body).ptr, g_page, sizeof g_page) == 0, "was sent as gzip and
read as the page");
    EXAMPLE_REQUIRE(sent < sizeof g_page / 5, "in less than a fifth of the bytes");
    printf("the page: %u bytes, sent as %u\n", (unsigned)sizeof g_page, (unsigned)sent);

    /* 핸들러가 요청하지 않은 응답은, 누가 달라고 하든 압축되지 않는다. */
    EXAMPLE_REQUIRE(get(decoding, at.port, "/account", &read_size, &sent, &coding, &body) && coding ==
PROVEN_HTTP_CODING_IDENTITY, "the account page is sent as it is");

    EXAMPLE_REQUIRE(get(decoding, at.port, "/accepts", &read_size, &sent, &coding, &body) &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("gzip")), "the handler can
see that this client accepts gzip");
    EXAMPLE_REQUIRE(get(plain, at.port, "/accepts", &read_size, &sent, &coding, &body) &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("identity")), "and that the
other does not");

    /* 중간에 flush한 응답도 여전히 gzip 스트림 하나다. */
    EXAMPLE_REQUIRE(get(decoding, at.port, "/log", &read_size, &sent, &coding, &body) && coding ==
PROVEN_HTTP_CODING_GZIP &&
                    proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("started\nfinished\n")), "a
log sent in two parts, the first flushed, reads as one");

    EXAMPLE_REQUIRE(get(plain, at.port, "/quit", &read_size, &sent, &coding, &body), "stop");
    proven_job_group_wait(threads, &running);
    proven_u8str_destroy(heap, &body);
    proven_http_client_destroy(plain);
    proven_http_client_destroy(decoding);
    proven_http_server_destroy(server);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
```

```
    return EXAMPLE_OK();
}
```

예제: 같은 일을 루프 위에서

```
#include <stdio.h>
#include <string.h>

/*
 * 같은 일을 루프 위에서: 조각조각 쓰는 응답을 압축하는 이벤트 구동 서버와, 도착하는 대로
 * 풀어 읽는 이벤트 구동 클라이언트.
 *
 * 어느 쪽도 본문을 쥐고 있지 않는다. 서버는 받은 것을 압축해 출력 큐에 넣되 큐의 한도를
 * 지키고, 클라이언트는 풀린 조각을 최대 16 KiB씩 넘겨준다.
 */

#define REPORT_BYTES ((proven_size_t)300000)

typedef struct {
    proven_loop_t *loop;
    proven_http_stream_t *stream;      /* 쓰고 있는 응답 하나 */
    proven_size_t written;
    proven_size_t received, pieces, largest;
    unsigned long sum_sent, sum_received;
    bool gzip, intact, done;
    proven_err_t why;
} ex_app_t;

/* 보고서는 쓰면서 만들어진다: 전체를 담은 버퍼는 없다. */
static proven_byte_t report_byte(proven_size_t i) { return (proven_byte_t)("measured value \n"[i % 16]); }

// ---- 서버 ----------------------------------------------------------------

/* 서버가 더 받지 않을 때까지 쓴다. 자리가 나면 on_writable이 이것을 다시 부른다. */
static void pump(ex_app_t *app) {
    proven_byte_t piece[4096];
    while (app->written < REPORT_BYTES) {
        proven_size_t n = REPORT_BYTES - app->written < sizeof piece ? REPORT_BYTES - app->written : sizeof
piece;
        for (proven_size_t i = 0; i < n; ++i) piece[i] = report_byte(app->written + i);
        proven_result_size_t took = proven_http_stream_write(app->stream, (proven_mem_view_t){ piece, n });
        if (took.err != PROVEN_OK) return;
        for (proven_size_t i = 0; i < took.value; ++i) app->sum_sent += piece[i];
        app->written += took.value;
        /* 첫 조각은 곧바로 내보낸다: 나머지를 아직 만드는 동안 읽는 쪽은 보고서가 시작되는 것을
         * 본다. 이것이 없으면 압축기 안에서 기다렸을 것이다. */
        if (app->written == sizeof piece) (void)proven_http_stream_flush(app->stream);
        if (took.value < n) return;        /* 압축된 출력이 한도에 닿았다: 기다린다 */
    }
    (void)proven_http_stream_end(app->stream);
}

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)head;
    ex_app_t *app = ctx;
    app->stream = stream;
    /* 응답을 시작하기 전에. 여기서부터 본문은 이것이 없을 때와 똑같이 쓴다:
     * 클라이언트에게는 gzip 청크가 가지만, 알려 준 길이는 여전히 써야 할 길이다.
     * 쓴 것은 뒤따르는 것이 올 때까지 압축기 안에서 기다릴 수 있다. */
    proven_http_stream_compress(stream);
    if (proven_http_stream_begin(stream, 200, NULL, 0, REPORT_BYTES) == PROVEN_OK) pump(app);
```

```
}
static void on_writable(void *ctx, proven_http_stream_t *stream) { (void)stream; pump(ctx); }

// ---- 클라이언트 ------------------------------------------------------------

static void on_response(void *ctx, proven_http_event_request_t *request, const proven_http_response_t *head) {
    (void)request;
    ex_app_t *app = ctx;
    app->gzip = proven_http_content_coding(head->headers, head->header_count) == PROVEN_HTTP_CODING_GZIP;
}

/* 풀린 조각들. 조각은 이 함수가 돌아갈 때까지만 유효한 뷰다. */
static void on_body(void *ctx, proven_http_event_request_t *request, proven_mem_view_t piece, bool last) {
    (void)request; (void)last;
    ex_app_t *app = ctx;
    for (proven_size_t i = 0; i < piece.size; ++i) {
        if (piece.ptr[i] != report_byte(app->received + i)) app->intact = false;
        app->sum_received += piece.ptr[i];
    }
    app->received += piece.size;
    app->pieces++;
    if (piece.size > app->largest) app->largest = piece.size;
}

static void on_done(void *ctx, proven_http_event_request_t *request, proven_err_t why) {
    (void)request;
    ex_app_t *app = ctx;
    app->why = why;
    app->done = true;
    proven_loop_stop(app->loop);
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { .intact = true };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");

    /* 창 크기가 압축 응답 하나가 쓰이는 동안 쥐는 메모리를 정한다: 12비트에서 약 53 KiB,
     * 최대인 15비트에서 325 KiB. 한꺼번에 많이 다룬다면 골라야 할 숫자가 이것이다. */
    proven_http_event_server_config_t server_config = { .on = { .on_request = on_request, .on_writable =
on_writable }, .ctx = &app, .compress_window_bits = 12 };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &server_config, &server) == PROVEN_OK, "a
server");
    proven_err_t err = proven_http_event_server_listen(server,
proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, 0), &at);
    if (err == PROVEN_ERR_PERMISSION || err == PROVEN_ERR_UNSUPPORTED) {
        printf("no sockets in this environment; nothing to show\n");
        proven_http_event_server_destroy(server);
        proven_loop_destroy(app.loop);
        return EXAMPLE_OK();
    }
    EXAMPLE_REQUIRE(err == PROVEN_OK, "it listens");

    /* `decompress`는 gzip을 요청하고 풀어 준다. 그러면 `max_body_bytes`는 풀린 크기에도
     * 걸리므로, 작은 본문이 거대하게 펼쳐지더라도 한도에서 끊긴다. */
    proven_http_event_client_config_t client_config = { .decompress = true, .max_body_bytes = 1024 * 1024 };
    proven_http_event_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_event_client_create(app.loop, &client_config, &client) == PROVEN_OK, "a client
```

```
that decodes");

    char url[64];
    int n = snprintf(url, sizeof url, "http://127.0.0.1:%u/report", (unsigned)at.port);
    proven_http_event_request_options_t options = {
        .url = { (const proven_byte_t *)url, (proven_size_t)n },
        .on = { on_response, on_body, NULL, on_done },
        .ctx = &app,
    };
    EXAMPLE_REQUIRE(proven_http_event_client_start(client, &options, NULL) == PROVEN_OK, "the request is
started");
    EXAMPLE_REQUIRE(proven_loop_run(app.loop) == PROVEN_OK && app.done, "the loop ran until it was done");

    EXAMPLE_REQUIRE(app.why == PROVEN_OK && app.gzip, "the response came as gzip");
    EXAMPLE_REQUIRE(app.received == REPORT_BYTES && app.intact && app.sum_received == app.sum_sent, "and was
delivered as the report that was written");
    EXAMPLE_REQUIRE(app.largest <= 16384 && app.pieces >= REPORT_BYTES / 16384, "in pieces of at most 16
KiB");
    printf("%u bytes written, compressed, sent, decoded and received\n", (unsigned)app.received);

    proven_http_event_client_destroy(client);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    return EXAMPLE_OK();
}
```

레퍼런스

API	의도	반환
`proven_http_exchange_compress(x)`	이 응답을 압축해 달라고 요청한다. `respond`나 `begin` 전에. 그 뒤에는 아무 일도 하지 않는다.	없음.
`proven_http_stream_compress(stream)`	이벤트 구동 서버에서 같은 일: `on_request` 안에서나 그 뒤에, 응답을 시작하기 전에.	없음.
`proven_http_exchange_flush(x)`, `proven_http_stream_flush(stream)`	압축 중인 응답이 붙들고 있는 것을 내보낸다. 응답은 계속된다. 압축하지 않는 응답에는 아무 일도 하지 않고 성공한다.	`proven_err_t`: 응답이 시작되기 전이나 끝난 뒤면 `INVALID_STATE`. 그 밖에는 쓰기와 같다.
`compress_level` (두 서버 설정)	1(가장 빠름)부터 9(가장 작음). -1은 압축 없이 저장만 한다. 0은 6. 측정값은 4절에 있다.	범위를 벗어나면 `create`가 `INVALID_ARG`.
`compress_window_bits` (둘 다)	9부터 15: 스트림으로 쓰는 응답이 쓰이는 동안 쥐는 메모리(4절의 표). 0은 15.	위와 같다.
`compress_zstd` (두 서버 설정)	zstd에 gzip 이상의 가중치를 주는 요청에는 `zstd`로, 그 밖에는 gzip으로 압축한다. false: gzip만.	-
`compress_zstd_level`, `compress_zstd_window_log` (둘 다)	7절 인코더의 레벨(1부터 9, 0은 1)과 창(10부터 23, 0은 17).	범위를 벗어나면 `create`가 `INVALID_ARG`.
`decompress` (두 클라이언트 설정)	`Accept-Encoding: gzip`을 보내고 `gzip`과 `deflate` 본문을 푼다.	-

`decompress_zstd` (두 클라이언트 설정)	`decompress`와 함께: `Accept-Encoding: zstd, gzip`을 보내고 `zstd` 본문도 푼다.	`decompress` 없이 켜면 `create`가 `INVALID_ARG`.
`decompress_brotli` (두 클라이언트 설정)	`decompress`와 함께: `br`도 요청하고 `br` 본문도 푼다(v0.60.0부터).	`decompress` 없이 켜면 `create`가 `INVALID_ARG`. `PROVEN_NO_BROTLI` 빌드에서는 `UNSUPPORTED`.
`proven_http_accepts_coding(headers, count, coding)`	요청의 `Accept-Encoding`이 어떤 코딩을 허용하는가: 0보다 큰 가중치로 적혀 있거나, `*`에 포함된다.	`bool`. 필드가 없으면 identity만 허용된다.
`proven_http_accept_weight(headers, count, coding)`	얼마나: 그 코딩의 가중치, 없으면 `*`의 가중치, 천분율로.	0부터 1000. `accepts_coding`이 아니라고 할 때 꼭 0. 필드가 없으면 identity는 1000, 나머지는 0.
`proven_http_content_coding(headers, count)`	`Content-Encoding`이 말하는 코딩.	`PROVEN_HTTP_CODING_IDENTITY`, `_GZIP`(`x-gzip`도), `_DEFLATE`, `_ZSTD`(v0.57.0부터. 그 전에는 `_OTHER`), `br`에는 `_BROTLI`(v0.60.0부터. 그 전에는 `_OTHER`), 그 밖의 것과 둘 이상에는 `_OTHER`.

주의할 점과, 잘못되는 경우

- **비밀과 클라이언트가 고른 글자가 섞인 응답에는 요청하지 말 것.** 5절의 누출이 가장 널리 알려진 모습으로 나타난 것이다(BREACH): 세션 토큰을 싣고 검색어도 그대로 되돌려 주는 페이지를 압축하면, 암호화된 연결을 지켜보는 이에게 - 길이만으로 - 검색어가 토큰의 일부와 맞았는지를 알려 준다. TLS는 돕지 못한다. TLS는 바이트를 가리지 크기를 가리지 않는다. 서버에 "전부 압축" 설정이 없는 까닭이 이것이다: 판단이 응답마다 달라서 호출도 응답마다 한다.
- **스트림으로 보내는 압축 응답은 flush하지 않으면 조각조각 도착하지 않는다.** 쓴 것은 더 쓰거나 응답이 끝날 때까지 압축기 안에 머물 수 있다. 이벤트 스트림(10장 13절)처럼 클라이언트가 일어나는 대로 보아야 하는 것이라면 조각마다 `proven_http_exchange_flush`나 `proven_http_stream_flush`를 부를 것: 그때까지 쓴 것이 클라이언트가 곧바로 읽을 수 있는 형태로 나간다. flush 한 번에는 몇 바이트와 압축률 일부가 든다. 읽는 쪽이 기다리는 자리에서만 부르고 쓸 때마다 부르지는 말 것 - 아니면 그런 응답은 아예 압축하지 말 것.
- **크기가 다른 압축 응답에는 다른 검증자가 필요하다.** `ETag`를 보낸다면 압축된 응답과 그대로인 응답은 서로 다른 표현이다. 다른 태그를 줄 것. 그러지 않으면 캐시가 한쪽을 다른 쪽과 맞출 수 있다. 서버는 여러분의 태그를 고쳐 쓰지 않는다.
- **클라이언트가 보는 헤더는 서버가 보낸 것이다.** `decompress`를 켜도 `Content-Encoding: gzip`은 그대로 있고 `Content-Length`는 여전히 압축된 바이트를 센다 - 여러분이 읽는 바이트가 아니다. 읽은 것을 세거나, 어느 쪽이었는지는 `proven_http_content_coding`으로 본다.
- **서버가 보낼 수 있는 양의 한도는 풀린 바이트로 센다.** `read_all`의 `max_bytes`와 이벤트 구동 클라이언트의 `max_body_bytes`는 해제기에서 나오는 것을 센다: 2메가바이트로 펼쳐지는 2킬로바이트는 상대의 한도가 아니라 여러분의 한도에서 끊긴다. 그냥 `proven_http_client_read`는 늘 그렇듯 여러분이 준 버퍼로 묶인다.
- **손상된 압축 본문은 `PROVEN_ERR_INVALID_FORMAT`이다.** 짧은 성공이 되는 일은 없다: 중간에 끊긴 스트림, 체크섬이 틀린 스트림, 또 하나의 gzip 멤버(또는 Zstandard 프레임)가 아닌 바이트가 뒤따르는 스트림. 잇달아 오는 gzip 멤버나 Zstandard 프레임 여럿은 본문 하나로 읽는다. Brotli 스트림 뒤에는 아무것도 없다.
- **`deflate`는 실제로 두 가지를 뜻한다.** 표준은 zlib 스트림이라고 하는데 raw DEFLATE를 보내는 서버가 있다. 클라이언트는 첫 두 바이트를 보고 어느 쪽이든 읽는다. `deflate`를 요청하는 일은 없고 - `gzip`과, `decompress_zstd`를 켰다면 `zstd`, `decompress_brotli`를 켰다면 `br`만 요청한다 - 서버가 그것을 보내는 일도 없다.
- **풀지 않는 것:** `206 Partial Content`(압축된 본문의 일부 범위는 압축된 본문이 아니다 - 그리고 `decompress`를 켰을 때 `Range`가 실린 요청은 `Accept-Encoding` 없이 나간다). `decompress_zstd` 없이 온 `zstd`. `decompress_brotli` 없이 온 `br`. 그 밖의 코딩(`compress`). 겹쳐 적용한 코딩 둘. 그런 본문은 헤더와 함께, 보낸 그대로 도착한다.

- **메모리.** 풀리는 중인 응답은 열려 있는 동안 약 35 KiB(이벤트 구동 클라이언트에서는 52 KiB)를 쓴다 - `zstd` 응답은 그 프레임의 창만큼(최대 8 MiB)과 약 400 KiB를 더, `br` 응답은 그 스트림의 창만큼(최대 16 MiB)과 그 코드들(8절)을 더, 각각 출력이 자라는 대로 할당해 쓴다. 스트림으로 압축되는 응답은 끝날 때까지 4절의 표가 윈도에 대해 말하는 만큼 - 기본값으로 325 KiB - 을 쓴다. 한 번에 보내는 응답은 그 호출 동안만, 본문보다 크지 않은 윈도로 쓴다. 기본 윈도로 스트림 응답 만 개면 3 GiB다: `compress_window_bits`를 낮추거나, 덜 압축한다. `zstd`로 스트림을 보내는 응답은 기본값(레벨 1, 128 KiB 창)에서 약 0.8 MiB - gzip의 325 KiB보다 많다 - 를, 레벨 3에 512 KiB 창이면 2 MiB까지 쥔다. 한 번에 보내는 응답에는 본문에 맞춘 표가 필요할 뿐이고, 짧은 본문이면 몇 킬로바이트다.

7. Zstandard

```
#include "proven/zstd.h"

proven_err_t proven_zstd_decoder_create(proven_allocator_t alloc, proven_u64 window_max, proven_zstd_decoder_t
**out);
void proven_zstd_decoder_destroy(proven_zstd_decoder_t *z);
void proven_zstd_decoder_reset(proven_zstd_decoder_t *z);
proven_err_t proven_zstd_decode(proven_zstd_decoder_t *z, proven_mem_view_t in, proven_size_t *consumed,
                                proven_mem_mut_t out, proven_size_t *produced, bool *done);
proven_err_t proven_zstd_decode_all(proven_allocator_t alloc, proven_mem_view_t in, proven_mem_mut_t out,
proven_size_t *written);
```

Zstandard(RFC 8878)는 DEFLATE보다 새 형식이다: `.zst` 파일, HTTP의 `Content-Encoding: zstd`(RFC 9659), 일부 패키지 형식. 압축기는 gzip의 가장 빠른 것보다 빠른 데서부터 gzip의 가장 작은 것보다 작은 데까지 고를 수 있고, 무엇으로 압축했든 빨리 풀린다. 이 라이브러리는 이것을 읽고, v0.58.0부터는 쓴다 - v0.59.0부터는 레벨 아홉으로(아래 "Zstandard 쓰기").

디코더는 3절의 해제기처럼 쓴다 - 상태 하나와, 있는 입력을 가져가고 있는 출력 자리를 채우는 한 걸음 - 다만 형식 때문에 두 가지가 다르다:

- **한 걸음이 할당할 수 있다.** Zstandard는 블록의 일부를 끝에서부터 읽으므로, 블록 하나(압축된 상태로 최대 128 KiB)를 통째로 쥔 다음에 풀고, 뒤 블록들이 가리키는 기록 옆에 통째로 푼다. 이 버퍼들은 프레임의 출력이 자라는 대로 자라고 - 300바이트라고 밝힌 프레임은 8 MiB가 아니라 300바이트의 기록만 쓴다 - 다음 프레임을 위해 남겨 둔다. 그래서 `proven_zstd_decode`가 `PROVEN_ERR_NOMEM`을 돌려줄 수 있다.
- **출력이 디코더 안에서 기다릴 수 있다.** 여러분이 준 자리가 모자라는 동안, 최대 블록 하나만큼.

창이 곧 메모리이고, 상한은 여러분이 정한다. 모든 프레임은 얼마나 뒤까지 참조할 수 있는지 - 창 - 를 밝히고, 디코더는 자기가 낸 것을 그만큼 들고 있어야 한다. 프레임은 테라바이트를 주장할 수도 있다. `window_max`는 여러분이 받아들이는 최대치다: 0이면 RFC 9659가 HTTP에 정한 8 MiB. 그보다 많이 필요한 프레임은 그것을 위해 아무것도 할당하기 전에 `PROVEN_ERR_OUT_OF_BOUNDS`다.

	메모리
만든 디코더	표에 약 20 KiB
프레임을 푸는 동안	그 창(최대 여러분의 상한)과, 풀고 있는 블록에 최대 384 KiB. `proven_zstd_decode_all`로 풀 때 크기를 밝히고 들어가는 프레임은 창을 잡지 않는다 - 그 자리에서 풀린다

스트림은 프레임의 연속이다. `*done`은 프레임이 끝나는 곳에서 참이 되고, 그 호출은 거기서 멈춘다: 가져간 것이 그 프레임이다. 나머지로 다시 부르면 다음 프레임이 시작된다 - reset은 필요 없다. 건너뛸 수 있는 프레임(매직 넘버 `0x184D2A50`부터 `0x184D2A5F`)은 아무것도 내지 않고 읽고 지나간다. 사전이 필요한 프레임은 `PROVEN_ERR_UNSUPPORTED`다. HTTP로 보내는 콘텐츠는 사전을 쓰지 않는다.

예제: 버퍼 하나, 프레임 둘, 너무 큰 창

```
#include <string.h>

/*
 * Zstandard: 버퍼 전체, 이어서 두 프레임으로 된 스트림을 조각조각 읽기, 그리고 남의 프레임이
 * 이 프로그램이 쓸 메모리를 정하지 못하게 하는 창 상한.
```

```
 */

/* 로그 줄 4,000바이트를 libzstd가 레벨 19, 체크섬 포함으로 쓴 것: 77바이트. */
static const proven_byte_t FRAME[] = {
    0x28, 0xb5, 0x2f, 0xfd, 0x64, 0xa0, 0x0e, 0xfd, 0x01, 0x00, 0x62, 0xc3,
    0x0c, 0x11, 0x90, 0x7d, 0x00, 0x04, 0x1c, 0xf9, 0xc4, 0x4b, 0x1e, 0x4d,
    0x80, 0xa0, 0x8f, 0x99, 0x3a, 0xe6, 0x0c, 0xe0, 0x84, 0xb9, 0x79, 0xda,
    0x52, 0xe7, 0x47, 0xe1, 0xa3, 0xb1, 0xbb, 0xdc, 0x1f, 0x9e, 0x10, 0xf3,
    0xe1, 0x8b, 0x28, 0xa3, 0x39, 0xc7, 0xde, 0xde, 0x4d, 0x45, 0x97, 0x11,
    0xd7, 0xf1, 0x4e, 0xad, 0x02, 0x00, 0xc6, 0xde, 0x95, 0x2f, 0x3b, 0x3b,
    0x9f, 0xe8, 0x22, 0xa1, 0x2a,
};

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    static proven_byte_t text[4000];
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];

    /* 버퍼 전체: gzip과 같이 출력 버퍼가 한계다. */
    static proven_byte_t out[8000];
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ FRAME, sizeof FRAME },
(proven_mem_mut_t){ out, sizeof out }, &n) == PROVEN_OK &&
                    n == sizeof text && memcmp(out, text, n) == 0, "Zstandard 77바이트가 libzstd에 주어진
4,000바이트로 풀린다");
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ FRAME, sizeof FRAME },
(proven_mem_mut_t){ out, 3999 }, &n) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "그리고 3,999바이트에는 들어가지 않는다: PROVEN_ERR_OUT_OF_BOUNDS");

    /* 스트림: 같은 프레임 두 개를 10바이트씩, 자리는 500바이트씩 준다. 각 호출은 할 수 있는
     * 만큼 가져가고, `done`은 프레임 하나가 막 끝났다는 뜻이며, 다음 호출은 그대로 뒤따르는
     * 프레임을 이어 읽는다. */
    static proven_byte_t two[2 * sizeof FRAME];
    memcpy(two, FRAME, sizeof FRAME);
    memcpy(two + sizeof FRAME, FRAME, sizeof FRAME);
    proven_zstd_decoder_t *z = NULL;
    EXAMPLE_REQUIRE(proven_zstd_decoder_create(heap, 0, &z) == PROVEN_OK, "기본 창 상한(8 MiB)의 디코더");
    proven_size_t in_at = 0, out_at = 0;
    int frames = 0;
    while (in_at < sizeof two || out_at < sizeof out) {
        proven_size_t give = sizeof two - in_at < 10 ? sizeof two - in_at : 10, room = sizeof out - out_at <
500 ? sizeof out - out_at : 500;
        proven_size_t used = 0, made = 0;
        bool done = false;
        EXAMPLE_REQUIRE(proven_zstd_decode(z, (proven_mem_view_t){ two + in_at, give }, &used,
(proven_mem_mut_t){ out + out_at, room }, &made, &done) == PROVEN_OK, "한 걸음");
        in_at += used;
        out_at += made;
        if (done && used + made > 0) frames++;
        if (used == 0 && made == 0) break;              /* 줄 것이 더 없거나, 둘 곳이 없다 */
    }
    EXAMPLE_REQUIRE(frames == 2 && out_at == 2 * sizeof text && memcmp(out + sizeof text, text, sizeof text)
== 0, "프레임 둘이 차례로, 8,000바이트");

    /* 창 상한. 모든 프레임은 얼마나 뒤까지 참조할 수 있는지 밝히고, 디코더는 그만큼을 들고
     * 있어야 한다. 손으로 만든 이 헤더는 1 MiB를 요구한다. 64 KiB로 제한한 디코더는 그것을 위해
     * 아무것도 할당하기 전에 거부한다. reset이 디코더를 다시 쓸 수 있게 한다. */
    static const proven_byte_t wide[] = { 0x28, 0xb5, 0x2f, 0xfd, 0x00, (20 - 10) << 3, 0x19, 0x00, 0x00, 'a',
'b', 'c' };
```

```
    proven_zstd_decoder_t *small = NULL;
    proven_size_t used = 0, made = 0;
    bool done = false;
    EXAMPLE_REQUIRE(proven_zstd_decoder_create(heap, 64 * 1024, &small) == PROVEN_OK, "64 KiB로 제한한 디코
더");
    EXAMPLE_REQUIRE(proven_zstd_decode(small, (proven_mem_view_t){ wide, sizeof wide }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_ERR_OUT_OF_BOUNDS,
                    "1 MiB 창이 필요한 프레임: PROVEN_ERR_OUT_OF_BOUNDS");
    proven_zstd_decoder_reset(small);
    EXAMPLE_REQUIRE(proven_zstd_decode(small, (proven_mem_view_t){ FRAME, sizeof FRAME }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_OK &&
                    done && made == sizeof text, "reset 뒤에는 상한 안의 프레임이 풀린다");
    EXAMPLE_REQUIRE(proven_zstd_decode(z, (proven_mem_view_t){ wide, sizeof wide }, &used, (proven_mem_mut_t)
{ out, sizeof out }, &made, &done) == PROVEN_OK &&
                    done && made == 3, "기본 상한은 그것을 받아들인다");

    proven_zstd_decoder_destroy(small);
    proven_zstd_decoder_destroy(z);
    return EXAMPLE_OK();
}
```

레퍼런스

API	의도	반환
`proven_zstd_decoder_create(alloc, window_max, &z)`	디코더. `window_max`는 1 KiB부터 2 GiB. 0은 8 MiB.	`INVALID_ARG`, `NOMEM`.
`proven_zstd_decode(z, in, &consumed, out, &produced, &done)`	`in`의 일부를 가져가고 `out`의 일부를 채운다. 프레임 끝에서 멈춘다. 낸다고 알린 바이트만 쓴다.	`INVALID_FORMAT`(손상, 맞지 않는 체크섬이나 밝힌 크기), `OUT_OF_BOUNDS`(상한을 넘는 창), `UNSUPPORTED`(사전), `NOMEM`, `INVALID_ARG`. 오류 뒤에는 reset까지 같은 오류.
`proven_zstd_decoder_reset(z)`	진행 중인 스트림을 잊는다. 버퍼는 남긴다.	없음.
`proven_zstd_decoder_destroy(z)`	해제한다. NULL도 된다.	없음.
`proven_zstd_decode_all(alloc, in, out, &written)`	`in`의 모든 프레임을 `out`으로, 기본 상한으로. `out`의 크기가 출력의 한도다. 크기를 밝히고 들어가는 프레임은 `out`에 바로 풀며 자기 창을 따로 잡지 않는다(v0.86.0부터). 그때 `out`의 `*written` 뒤 바이트가 쓰일 수 있다.	들어가지 않으면 `OUT_OF_BOUNDS`. `in`이 비었거나 프레임 중간에서 끝나도 `INVALID_FORMAT`.

주의할 점과, 잘못되는 경우

- **메모리는 데이터의 크기가 아니라 창이 정한다.** 서버가 - 또는 여러분이 프레임을 풀어 주는 누구든 - 여러분의 상한 안에서 그것을 고른다. 응답 천 개를 동시에, 각각 8 MiB 창으로 풀면 8 GiB다. 할 수 있는 곳에서는 `window_max`를 낮추고(자기 `.zst` 파일을 푸는 디코더는 그 파일이 쓰는 창을 안다), 디코더 수를 연결 수처럼 셀 것.
- **Zstandard에서 체크섬은 선택이다.** 체크섬 없이 쓴 프레임은 손상되고도 틀린 바이트로 풀릴 수 있다. 디코더는 해석되지 않는 것만 거부할 수 있다. 중요한 곳에서는 푼 것을 직접 확인할 것.
- **gzip 스트림이 아니다.** `proven_zstd_decode`는 Zstandard만 읽는다. HTTP에서는 클라이언트가 `Content-Encoding`을 보고 디코더를 고른다. 파일이라면 첫 네 바이트(`28 b5 2f fd`)를 본다.

libzstd가 거부하는 것은 하나도 받아들이지 않고, libzstd가 읽어 넘기는 손상 일부는 거부한다. RFC 8878이 디코더에 맡긴 경우는 libzstd 1.5.7이 정하는 대로 - 12비트까지의 Huffman 코드를 읽는다 - 또는 더 엄격하게 정했다: 모든 Huffman 스트림과 시퀀스 스트림은 첫 비트까지 정확히 읽혀야 한다(libzstd의 빠른 Huffman 디코더는 스트림 넷에 대해 이것을 확인하지 않는다). 바이트가 없는 압축 블록은 RFC 8878대로 거부한다(libzstd의 스트림 디코더는 빈 블록으로 받고, 한 번 호출 디코더는 거부한다). 등록된 테스트 밖에서, libzstd가 만든 스트림 92개를 자르고 바꾼 사본 80,232개를 양쪽에 주었다: libzstd가 거부한 것을 여기서 받아들인 것은 없었고, 둘 다 받아들인 것이 다르게 나온 것도 없었으며, libzstd가 받아들인 1,985개를 여기서는 거부했다: 1,948개는 네 Huffman 스트림 중 하나가 정확히 읽히지 않아서, 25개는 프레임 뒤의 바이트 때문에(libzstd를 부른 방식인 Node는 그것을 무시한다), 8개는 바이트가 없는 압축 블록이어서, 4개는 리터럴을 담기에 너무 짧은 블록이어서다.

속도. 개발 기계에서, 이 라이브러리의 C 소스(2.9 MiB)를 libzstd가 레벨 1, 3, 19로 압축한 것은 `proven_zstd_decode_all`로 각각 약 650, 650, 750 MiB/s, 디코더에 한 번에 다 주면 약 590, 570, 730 MiB/s로 풀린다. 같은 프로세스의 libzstd는 약 1,020, 940, 1,030 MiB/s - 이 디코더는 libzstd 속도의 0.6-0.7이다(v0.86.0. 그 전에는 절반쯤). v0.86.0 전에, 같은 소스 2.2 MiB를 gzip으로 해서 3절의 해제기로 풀면 약 335 MiB/s였다.

Zstandard 쓰기

```
proven_err_t proven_zstd_encoder_create(const proven_zstd_encoder_options_t *options, proven_zstd_encoder_t
**out);
void proven_zstd_encoder_destroy(proven_zstd_encoder_t *z);
void proven_zstd_encoder_reset(proven_zstd_encoder_t *z);
proven_err_t proven_zstd_encode(proven_zstd_encoder_t *z, proven_mem_view_t in, proven_size_t *consumed,
                                proven_mem_mut_t out, proven_size_t *produced, proven_zstd_flush_t flush, bool
*done);
proven_size_t proven_zstd_encode_bound(proven_size_t input_size);
proven_err_t proven_zstd_encode_all(const proven_zstd_encoder_options_t *options, proven_mem_view_t in,
proven_mem_mut_t out, proven_size_t *written);
```

인코더는 3절의 압축기처럼 쓴다: 옵션(`alloc`, `level`, `window_log`, `checksum`), 상태, 그리고 flush가 있는 한 걸음 - 더 올 때는 `PROVEN_ZSTD_FLUSH_NONE`, 프레임은 이어 가면서 지금까지 준 것을 모두 풀 수 있게 하려면 `_BLOCK`, 프레임을 끝내려면 `_FINISH`. 만드는 프레임은 어떤 Zstandard 디코더든 읽는다. 같은 입력에 대해 libzstd가 썼을 바이트와 같지는 않다.

레벨 아홉, libzstd의 처음 아홉처럼. 레벨 1은 해시 표 하나로, 레벨 2와 3은 둘로 일치를 찾는다(libzstd의 "fast"와 "double fast"). 레벨 4부터 9는 모든 위치를 해시 체인에 두고 그 앞쪽 4에서 32개를 보며, 찾은 가장 좋은 일치를 고르거나(레벨 4와 5), 그 전에 한 바이트(6) 또는 두 바이트(7부터 9) 뒤에 더 좋은 것이 있는지 본다. 블록마다 리터럴에는 Huffman 코드를, 시퀀스에는 FSE 표를 붙이는데, 표마다 그 블록에 드는 비용을 보고 고른다. 4절의 세 파일을 개발 머신의 코어 하나에서, 릴리스 플래그로 잰 값:

	HTML 페이지	Markdown	C 소스
이 라이브러리 gzip 레벨 1	26.25 %, 45 MiB/s	36.77 %, 33	27.24 %, 43
이 라이브러리 gzip 레벨 6	22.62 %, 25	32.89 %, 16	22.88 %, 21
이 라이브러리 Zstandard 레벨 1	24.93 %, 263	36.19 %, 191	26.71 %, 234
이 라이브러리 Zstandard 레벨 3	23.83 %, 193	34.58 %, 132	25.50 %, 172
이 라이브러리 Zstandard 레벨 6	21.66 %, 67	32.68 %, 49	22.79 %, 64
이 라이브러리 Zstandard 레벨 9	21.22 %, 37	32.11 %, 28	22.21 %, 36
libzstd 1.5.7 레벨 1	25.27 %, 312	36.68 %, 231	26.91 %, 273
libzstd 1.5.7 레벨 3	23.76 %, 232	34.42 %, 161	25.42 %, 213
libzstd 1.5.7 레벨 6	21.93 %, 74	32.65 %, 54	22.97 %, 67
libzstd 1.5.7 레벨 9	21.25 %, 48	32.12 %, 36	22.22 %, 43

레벨 1은 이 라이브러리의 gzip 레벨 1보다 다섯에서 여섯 배 빠르고 더 작다. 레벨 6은 gzip 레벨 6보다 크지 않고 2.6에서 3.2배 빠르다. libzstd에 견주면 레벨 1~3과 5~8은 속도 0.8에서 1.0배, 레벨 9는 약 0.8배, 크기는 0.3포인트 안이다. 레벨 6은 libzstd의 레벨 6이 보는 8개 대신 12개의 후보를 보는데, 그래서 세 파일 모두에서 gzip 6 이하에 머문다(libzstd 6은 C 소스에서 gzip 6보다 조금 크다). 레벨 4는 libzstd가 그 레벨에서 쓰는 방법이 아니다: 약 1포인트 더 작고, 속도는 절반쯤이다. 메가바이트 단위 입력에서는 libzstd의 높은 레벨이 더 큰 창과 표도 써서 0.3에서 0.5포인트 더 작다.

표는 v0.89.0 전에 잰 것이다. 그 뒤로 BMI2(플래그 없이 레지스터만큼 시프트)가 있는 x86 프로세서에서는 인코더가 그것에 맞춰 컴파일한 블록 코더의 두 번째 빌드를 쓴다 - 같은 C, 같은 출력이고, 프로세서에 처음 물을 때 고른다. 0.4~3.9 MB의 다른 입력 넷에서 레벨 1~3은 libzstd 속도의 0.76~0.88에서 0.82~0.96이 되었다.

메모리. `proven_zstd_encode_all`은 호출자의 버퍼를 그 자리에서 압축하고, 표와 블록 버퍼를 그 크기에 맞춘다: 짧은 입력이면 몇 킬로바이트다. 스트림은 창을 자기 버퍼에 두고, 많아야 다음을 쥔다:

	메모리
해시 표(인코더를 만들거나 reset할 때 0으로 채운다)	레벨 1에서 64 KiB, 2에서 96 KiB, 3과 4에서 768 KiB, 5부터 9에서 1 MiB(통째로 주는 버퍼가 짧으면 더 적다)
블록 하나의 일: 시퀀스, 리터럴, 출력	약 500 KiB
스트림의 입력 버퍼	창과 창의 절반, 또는 그보다 많으면 창과 128 KiB: 기본 512 KiB에서 768 KiB

창(`window_log`, 10부터 23, 0은 19)은 디코더가 그 프레임을 읽으려면 들고 있어야 하는 양이라서, 읽는 쪽의 메모리이기도 하다: 23(8 MiB)이 RFC 9659가 HTTP 클라이언트에게 받아들이라고 하는 최대다. 창보다 크지 않은 버퍼를 통째로 주면 그 크기를 창으로 밝힌다.

예제: 통째로, flush가 있는 스트림으로, 그리고 요청의 가중치

```
#include <string.h>

/*
 * Zstandard 쓰기: 버퍼 전체, flush에서 가진 것을 넘기는 스트림, 그리고 요청의 Accept-Encoding을
 * 보고 서버가 zstd와 gzip 가운데 고르는 일.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    static proven_byte_t text[40000];
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];
    static proven_byte_t frame[48000], back[40000];

    /* 버퍼 전체. encode_bound면 언제나 충분하다. 프레임이 입력의 크기를 밝히므로 디코더는 그보다
     * 큰 창을 둘 필요가 없다. */
    proven_zstd_encoder_options_t o = { .alloc = heap, .level = 1 };
    proven_size_t n = 0, got = 0;
    EXAMPLE_REQUIRE(proven_zstd_encode_bound(sizeof text) <= sizeof frame, "40,000바이트의 상한이 버퍼에 들어간다");
    EXAMPLE_REQUIRE(proven_zstd_encode_all(&o, (proven_mem_view_t){ text, sizeof text }, (proven_mem_mut_t){ frame, sizeof frame }, &n) == PROVEN_OK && n < 200,
                    "되풀이되는 로그 줄 40,000바이트가 200바이트 미만으로 압축된다");
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ frame, n }, (proven_mem_mut_t){ back, sizeof back }, &got) == PROVEN_OK &&
                    got == sizeof text && memcmp(back, text, got) == 0, "그리고 한 바이트도 빠짐없이 풀린다");

    /* 스트림: 레벨 6, 64 KiB 창, 체크섬 포함으로 1,000바이트씩. 처음 5,000바이트 뒤의 블록 flush는 그때까지를
```

```
     * 바로 풀 수 있게 한다 - 서버가 더 보낼 것을 기다리기 전에 하는 일이다. FINISH가 프레임을 끝낸다. */
    proven_zstd_encoder_t *z = NULL;
    o = (proven_zstd_encoder_options_t){ .alloc = heap, .level = 6, .window_log = 16, .checksum = true };
    EXAMPLE_REQUIRE(proven_zstd_encoder_create(&o, &z) == PROVEN_OK, "레벨 6 - gzip 6 정도의 크기 - , 64 KiB
창의 인코더");
    proven_size_t in_at = 0, out_at = 0;
    bool done = false;
    while (!done) {
        const proven_size_t give = sizeof text - in_at < 1000 ? sizeof text - in_at : 1000;
        const proven_zstd_flush_t flush = in_at + give == sizeof text ? PROVEN_ZSTD_FLUSH_FINISH
                                        : in_at + give == 5000 ? PROVEN_ZSTD_FLUSH_BLOCK :
PROVEN_ZSTD_FLUSH_NONE;
        proven_size_t used = 0, made = 0;
        EXAMPLE_REQUIRE(proven_zstd_encode(z, (proven_mem_view_t){ text + in_at, give }, &used,
(proven_mem_mut_t){ frame + out_at, sizeof frame - out_at }, &made, flush, &done) == PROVEN_OK,
                        "한 걸음");
        in_at += used;
        out_at += made;
        if (flush == PROVEN_ZSTD_FLUSH_BLOCK) {
            /* 지금까지 나온 것만으로 지금까지의 전부가 풀린다. 프레임은 아직 끝나지 않았다. */
            proven_zstd_decoder_t *d = NULL;
            proven_size_t dused = 0, dmade = 0;
            bool dend = true;
            EXAMPLE_REQUIRE(proven_zstd_decoder_create(heap, 0, &d) == PROVEN_OK &&
                            proven_zstd_decode(d, (proven_mem_view_t){ frame, out_at }, &dused,
(proven_mem_mut_t){ back, sizeof back }, &dmade, &dend) == PROVEN_OK &&
                            dmade == 5000 && !dend, "flush 뒤 디코더가 5,000바이트를 모두 돌려주고 나머지를 기
다린다");
            proven_zstd_decoder_destroy(d);
        }
    }
    EXAMPLE_REQUIRE(proven_zstd_decode_all(heap, (proven_mem_view_t){ frame, out_at }, (proven_mem_mut_t)
{ back, sizeof back }, &got) == PROVEN_OK &&
                    got == sizeof text && memcmp(back, text, got) == 0, "끝난 프레임은 체크섬까지 맞고 40,000
바이트로 풀린다");
    proven_zstd_encoder_reset(z);                       /* 같은 인코더, 새 프레임 */
    proven_size_t used = 0, made = 0;
    EXAMPLE_REQUIRE(proven_zstd_encode(z, (proven_mem_view_t){ text, 300 }, &used, (proven_mem_mut_t){ frame,
sizeof frame }, &made, PROVEN_ZSTD_FLUSH_FINISH, &done) == PROVEN_OK &&
                    done && used == 300, "reset 뒤 두 번째 프레임");
    proven_zstd_encoder_destroy(z);

    /* 요청이 어느 코딩을 가장 원하는가: 천분율의 가중치. curl과 브라우저는 zstd와 gzip을 같은
     * 가중치로 보낸다. zstd를 제공하는 서버(compress_zstd)는 그때 zstd를 고른다. */
    const proven_http_header_t asks[] = { { PROVEN_LIT("Accept-Encoding"), PROVEN_LIT("gzip, deflate, br,
zstd") } };
    const proven_http_header_t prefers_gzip[] = { { PROVEN_LIT("Accept-Encoding"), PROVEN_LIT("gzip;q=1.0,
zstd;q=0.5") } };
    EXAMPLE_REQUIRE(proven_http_accept_weight(asks, 1, PROVEN_HTTP_CODING_ZSTD) == 1000 &&
proven_http_accept_weight(asks, 1, PROVEN_HTTP_CODING_GZIP) == 1000,
                    "브라우저의 목록: zstd도 gzip도 1000");
    EXAMPLE_REQUIRE(proven_http_accept_weight(prefers_gzip, 1, PROVEN_HTTP_CODING_ZSTD) == 500 &&
proven_http_accept_weight(prefers_gzip, 1, PROVEN_HTTP_CODING_GZIP) == 1000,
                    "gzip을 더 원하는 클라이언트: zstd 500, gzip 1000");
    return EXAMPLE_OK();
}
```

레퍼런스: 인코더

API	의도	반환

`proven_zstd_encoder_create(&options, &z)`	인코더: `level` 1부터 9(0은 libzstd의 기본값인 3), `window_log` 10부터 23(0은 19), `checksum`. 해시 표를 할당하고, 버퍼는 입력이 오는 대로.	`INVALID_ARG`, `NOMEM`.
`proven_zstd_encode(z, in, &consumed, out, &produced, flush, &done)`	`in`의 일부를 받고 `out`의 일부를 채운다. flush가 있으면 입력을 다 받고 더 나오는 것이 없을 때까지(FINISH면 `done`까지) 다시 부른다.	FINISH 뒤의 입력은 `INVALID_STATE`(먼저 reset). `NOMEM` - 그 뒤로는 reset할 때까지 모든 호출이 그렇게 말한다. `INVALID_ARG`.
`proven_zstd_encoder_reset(z)`	같은 옵션으로 새 프레임을 시작한다. 버퍼는 둔다.	없음.
`proven_zstd_encoder_destroy(z)`	해제한다. NULL 허용.	없음.
`proven_zstd_encode_bound(n)`	`n`바이트의 FINISH나 `encode_all`이 쓸 수 있는 최대.	크기. 상한이 크기에 들어가지 않으면 가장 큰 크기.
`proven_zstd_encode_all(&options, in, out, &written)`	`in`을 자기 크기를 밝힌 프레임 하나로 `out`에.	`out`이 작으면 `OUT_OF_BOUNDS`.

쓸 때 주의할 점

스트림은 자기 크기를 밝히지 않으므로 디코더는 창 전체를 준비해야 한다: `window_log`는 압축률만이 아니라 읽는 쪽을 보고 고를 것. 체크섬은 요청하지 않으면 없다: 체크섬이 없으면 손상된 프레임이 틀린 바이트로 풀릴 수 있다. 블록 flush 한 번에는 블록 헤더와 짧은 블록의 표가 든다 - 읽는 쪽이 기다리는 자리에서 flush하고, 쓸 때마다 하지는 말 것. 압축은 gzip보다 더 비밀스럽지 않다: 클라이언트가 고른 글자 옆의 비밀에 대한 6절의 주의는 gzip에서와 같이 `zstd`에도 해당한다.

8. Brotli

```
#include "proven/brotli.h"

proven_err_t proven_brotli_decoder_create(proven_allocator_t alloc, proven_u64 window_max,
proven_brotli_decoder_t **out);
void proven_brotli_decoder_destroy(proven_brotli_decoder_t *b);
void proven_brotli_decoder_reset(proven_brotli_decoder_t *b);
proven_err_t proven_brotli_decode(proven_brotli_decoder_t *b, proven_mem_view_t in, proven_size_t *consumed,
                                  proven_mem_mut_t out, proven_size_t *produced, bool *done);
proven_err_t proven_brotli_decode_all(proven_allocator_t alloc, proven_mem_view_t in, proven_mem_mut_t out,
proven_size_t *written);
```

Brotli(RFC 7932)는 HTTP의 `Content-Encoding: br`이다. 서버는 대개 미리 한 번, 가장 높은 품질로 압축해 두는 파일에 쓴다. 그 품질에서는 gzip과 Zstandard보다 작게 나온다. 그 품질로 압축하는 일은 느리고, 푸는 일은 품질과 상관없이 빠르다. 이 라이브러리는 v0.60.0부터 이것을 읽고, 쓰지는 않는다: 여기의 서버는 gzip이나 Zstandard를 보낸다.

디코더는 7절의 것처럼 쓴다 - 상태 하나와, 있는 만큼의 입력을 받아 있는 만큼의 출력 공간을 채우는 한 걸음. 그리고 어디서든 멈출 수 있다: 입력이 거기서 끝나면 코드의 두 비트 사이에서, 공간이 거기서 끝나면 복사 한가운데서. 두 가지가 7절과 같다:

- **한 걸음이 할당할 수 있다.** 뒤의 바이트가 참조하는 히스토리는 스트림의 출력과 함께 자라며, 스트림이 밝힌 창을 넘지 않는다. 그리고 메타블록(Brotli의 단위, 출력 16 MiB까지)마다 접두 부호를 따로 가져오고, 그것들은 개수에 따라 자라는 표 하나에 만들어진다. 둘 다 다음 스트림을 위해 남겨 둔다. 그래서 `proven_brotli_decode`는 `PROVEN_ERR_NOMEM`을 돌려줄 수 있다.
- **출력이 디코더 안에서 기다릴 수 있다.** 여러분이 준 공간이 모자라는 동안 그 히스토리 안에서.

창이 곧 메모리이고, 상한은 여러분의 것이다. 스트림의 처음 몇 비트가 얼마나 멀리 거슬러 참조할지를 말한다: 2^WBITS - 16바이트, WBITS는 10부터 24, 그러니 최대 16 MiB. `window_max`는 여러분이 허락하는 최대다: 0은 16 MiB로, RFC 7932가 허용하는 모든 스트림을 받아들인다. 그보다 많이 필요한 스트림은 그것을 위해 아무것도 할당하기 전에 `PROVEN_ERR_OUT_OF_BOUNDS`다. RFC 7932가 아닌 libbrotli의 "큰 창" 변형은 잘못된 형식으로 거절한다.

	메모리
만든 디코더	약 22 KiB
스트림을 푸는 동안	히스토리 - 출력이 닿은 만큼의 창, 최대 2^WBITS바이트와 32바이트 - 와 코드들: 인코더가 쓰는 것은 수십 KiB, 메타블록이 가질 수 있는 최대(세 종류마다 256개, 각각 코드 표가 될 수 있는 가장 큰 크기)는 1,343,024바이트

스트림 하나, 그 뒤에는 아무것도 없다. Brotli 스트림은 gzip 멤버나 Zstandard 프레임처럼 잇달아 오지 않는다. `*done`은 스트림이 끝나는 곳에서 참이 되고, 그 호출은 거기서 멈춘다 - 소비한 것이 곧 그 스트림이다 - 그 뒤의 호출은 아무것도 더 소비하지 않는다. 다른 스트림을 읽으려면 reset한다. `proven_brotli_decode_all`은 끝 뒤의 바이트를 거절하고, HTTP 클라이언트도 그렇다.

정적 사전. Brotli 스트림은 모든 디코더가 지니는 사전에서 단어를 복사할 수 있다: 여러 언어의 단어 122,784바이트와, 단어마다 바꾸는 방식 121가지(앞에 공백, 첫 글자를 대문자로, `ing ` 같은 접미사). 이것이 이 디코더가 프로그램에 더하는 것의 대부분이다: 이것이 있으면 라이브러리가 162,351바이트 커진다 (x86-64에서 `-O2`로, 코드와 데이터 998,985바이트 대 836,634바이트). `-DPROVEN_NO_BROTLI`로 만든 빌드는 디코더와 사전을 뺀다. 그때 `proven.h`는 `brotli.h`를 포함하지 않고, Brotli를 풀라는 HTTP 클라이언트는 `PROVEN_ERR_UNSUPPORTED`다.

예제: 버퍼, 뒤에 무언가 붙은 스트림, 그리고 너무 큰 창

```
#include <string.h>

/*
 * Brotli: 버퍼 전체, 이어서 뒤에 다른 바이트가 붙은 같은 스트림을 조각조각 읽기, 그리고 남의
 * 스트림이 이 프로그램이 쓸 메모리를 정하지 못하게 하는 창 상한.
 */

/* 로그 줄 4,000바이트를 libbrotli가 품질 11, 64 KiB 창으로 쓴 것: 58바이트. */
static const proven_byte_t STREAM[] = {
    0xe2, 0xf3, 0x01, 0xb1, 0xe3, 0x30, 0x76, 0xb3, 0x54, 0x22, 0x7f, 0x6e,
    0x31, 0x35, 0x23, 0x54, 0x7a, 0x83, 0xbc, 0xb8, 0x1c, 0x8d, 0x96, 0x8c,
    0x29, 0xa2, 0x82, 0x0d, 0x38, 0x60, 0x0f, 0x30, 0xc2, 0x45, 0x17, 0x5a,
    0x8c, 0xce, 0x18, 0x20, 0xa3, 0x8c, 0xc1, 0x41, 0x9a, 0xb7, 0x0a, 0x2d,
    0x23, 0xe3, 0x31, 0x41, 0x6b, 0xd0, 0x07, 0x39, 0x00, 0x40,
};

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    static const char line[] = "GET /index.html HTTP/1.1 - 200 - text/html; charset=utf-8\n";
    static proven_byte_t text[4000];
    for (proven_size_t i = 0; i < sizeof text; ++i) text[i] = (proven_byte_t)line[i % (sizeof line - 1)];

    /* 버퍼 전체: gzip처럼 출력 버퍼가 상한이다. */
    static proven_byte_t out[8000];
    proven_size_t n = 0;
    EXAMPLE_REQUIRE(proven_brotli_decode_all(heap, (proven_mem_view_t){ STREAM, sizeof STREAM },
(proven_mem_mut_t){ out, sizeof out }, &n) == PROVEN_OK &&
                    n == sizeof text && memcmp(out, text, n) == 0, "Brotli 58바이트가 libbrotli가 받은 4,000바
이트로 풀린다");
    EXAMPLE_REQUIRE(proven_brotli_decode_all(heap, (proven_mem_view_t){ STREAM, sizeof STREAM },
(proven_mem_mut_t){ out, 3999 }, &n) == PROVEN_ERR_OUT_OF_BOUNDS,
```

```
                "3,999바이트에는 들어가지 않는다: PROVEN_ERR_OUT_OF_BOUNDS");

    /* 뒤에 다른 5바이트가 붙은 스트림을 7바이트씩, 500바이트의 자리로 준다. `done`은 스트림이
     * 끝나는 곳에서 오고, 그 뒤의 바이트는 소비되지 않는다. Brotli 스트림 뒤에는 이어질
     * 프레임이 없으니, 그 바이트가 무엇인지는 호출한 쪽이 정할 일이다. */
    static proven_byte_t more[sizeof STREAM + 5];
    memcpy(more, STREAM, sizeof STREAM);
    memcpy(more + sizeof STREAM, "after", 5);
    proven_brotli_decoder_t *b = NULL;
    EXAMPLE_REQUIRE(proven_brotli_decoder_create(heap, 0, &b) == PROVEN_OK, "기본 창 상한(16 MiB)의 디코더");
    proven_size_t in_at = 0, out_at = 0;
    bool done = false;
    while (!done) {
        proven_size_t give = sizeof more - in_at < 7 ? sizeof more - in_at : 7, room = sizeof out - out_at <
500 ? sizeof out - out_at : 500;
        proven_size_t used = 0, made = 0;
        EXAMPLE_REQUIRE(proven_brotli_decode(b, (proven_mem_view_t){ more + in_at, give }, &used,
(proven_mem_mut_t){ out + out_at, room }, &made, &done) == PROVEN_OK, "한 걸음");
        in_at += used;
        out_at += made;
        if (used == 0 && made == 0 && !done) break;     /* 줄 것이 없거나 둘 자리가 없다 */
    }
    EXAMPLE_REQUIRE(done && in_at == sizeof STREAM && out_at == sizeof text && memcmp(out, text, out_at) == 0,
                "스트림의 58바이트 뒤에서 done, 그 뒤의 5바이트는 그대로");

    /* 창 상한. 스트림의 처음 몇 비트가 얼마나 멀리 거슬러 참조할지를 말하고, 디코더는 그만큼을
     * 쥐고 있어야 한다. libbrotli의 가장 큰 창으로 만든 이 스트림은 16 MiB를 요구한다. 64 KiB로
     * 제한한 디코더는 아무것도 할당하기 전에 거절하고, reset하면 다시 쓸 수 있다. */
    static const proven_byte_t wide[] = { 0x0f, 0x01, 0x80, 0x61, 0x62, 0x63, 0x03 };
    proven_brotli_decoder_t *small = NULL;
    proven_size_t used = 0, made = 0;
    EXAMPLE_REQUIRE(proven_brotli_decoder_create(heap, 64 * 1024, &small) == PROVEN_OK, "64 KiB로 제한한 디코
더");
    EXAMPLE_REQUIRE(proven_brotli_decode(small, (proven_mem_view_t){ wide, sizeof wide }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_ERR_OUT_OF_BOUNDS,
                "16 MiB 창의 스트림: PROVEN_ERR_OUT_OF_BOUNDS");
    proven_brotli_decoder_reset(small);
    EXAMPLE_REQUIRE(proven_brotli_decode(small, (proven_mem_view_t){ STREAM, sizeof STREAM }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_OK &&
                done && made == sizeof text, "reset 뒤에는 상한 안의 스트림이 풀린다");
    proven_brotli_decoder_reset(b);
    EXAMPLE_REQUIRE(proven_brotli_decode(b, (proven_mem_view_t){ wide, sizeof wide }, &used,
(proven_mem_mut_t){ out, sizeof out }, &made, &done) == PROVEN_OK &&
                done && made == 3 && memcmp(out, "abc", 3) == 0, "기본 상한은 받아들인다 - 히스토리는 16
MiB가 아니라 몇 KiB");

    proven_brotli_decoder_destroy(small);
    proven_brotli_decoder_destroy(b);
    return EXAMPLE_OK();
}
```

레퍼런스

API	의도	반환
`proven_brotli_decoder_create(alloc, window_max, &b)`	디코더. `window_max`는 1 KiB부터 2 GiB. 0은 16 MiB.	`INVALID_ARG`, `NOMEM`.

`proven_brotli_decode(b, in, &consumed, out, &produced, &done)`	`in`의 일부를 받아 `out`의 일부를 채운다. 스트림의 끝에서 멈춘다. 만들었다고 보고한 바이트만 쓴다.	`INVALID_FORMAT`(Brotli가 아니거나 손상됨), `OUT_OF_BOUNDS`(창이 상한 위), `NOMEM`, `INVALID_ARG`. 오류 뒤에는 reset까지 같은 오류.
`proven_brotli_decoder_reset(b)`	진행 중인 스트림을 잊는다. 버퍼는 남긴다.	없음.
`proven_brotli_decoder_destroy(b)`	해제한다. NULL도 된다.	없음.
`proven_brotli_decode_all(alloc, in, out, &written)`	`in`의 스트림 하나를 기본 상한으로 `out`에. `out`의 크기가 출력의 상한이다.	들어가지 않으면 `OUT_OF_BOUNDS`. `in`이 비었거나, 스트림 안에서 끝나거나, 스트림 뒤에 더 있으면 `INVALID_FORMAT`.

주의할 점과, 잘못되는 경우

- **창이 메모리를 정한다.** 7절과 같다: 스트림을 쓴 쪽이 여러분의 상한 안에서 고른다. 16 MiB 창을 쓰고 그 창을 채울 만큼 긴 응답 천 개를 한꺼번에 받으면 16 GiB다. 스트림을 아는 곳(알려진 창으로 직접 만든 파일)에서는 `window_max`를 낮추고, 디코더 수를 연결 수처럼 셀 것.
- **체크섬이 없다.** 여전히 유효한 스트림을 이루는 손상은 다른 바이트로 풀린다. 디코더는 해석되지 않는 것만 거부할 수 있다. 중요한 곳에서는 푼 것을 직접 확인할 것.
- **첫 바이트로 알아볼 수 없다.** Brotli에는 매직 넘버가 없다: gzip이나 텍스트를 받은 디코더는 곧바로가 아니라 안쪽 어딘가에서 실패하고, 다른 종류의 아주 짧은 입력이 우연히 유효한 스트림일 수도 있다. HTTP에서는 `Content-Encoding`이 본문이 무엇인지 말한다. 파일이라면 이름(`.br`)이나 자기 헤더를 붙여 둘 것.

libbrotli가 거부하는 것은 하나도 받아들이지 않는다. 등록된 테스트 밖에서, 스트림 176개를 자르거나 바꾼 사본 1,437,490개를 이 디코더와 libbrotli 1.1.0의 한 번 호출 디코더에 함께 주었다: libbrotli가 거부한 것을 여기서 받아들인 것은 없고, 둘 다 받아들인 것 가운데 다르게 나온 것도 없으며, libbrotli가 받아들이고 여기서 거부한 것이 11,242개다 - 모두 입력보다 먼저 끝나 같은 출력을 내는 스트림으로, 남은 바이트를 libbrotli의 한 번 호출 디코더는 무시하고 `proven_brotli_decode_all`은 (libbrotli의 Python 바인딩처럼) 무시하지 않는다.

속도. 개발 기계에서, 이 라이브러리의 C 소스(2.9 MiB)를 libbrotli가 품질 1, 5, 11로 압축한 것은 각각 약 195, 265, 290 MiB/s로 풀린다 - 같은 프로세스에서 잰 libbrotli 자신의 디코더(약 270, 410, 435 MiB/s)의 0.65~0.73배(v0.87.0. 그 전에는 0.57~0.65). 비압축 메타블록이나 긴 반복은 libbrotli의 속도로 복사한다.

9. 무엇을 받아들이고, 무엇이 없으며, 어떻게 시험했는가

해제기가 받아들이는 것은 zlib가 받아들이는 것이다. RFC 1951이 모퉁이를 열어 둔 곳마다, 경우 하나하나를 먼저 zlib에 시험해 보고 그 판정을 따랐다. 실제 생산자가 zlib가 봐주는 것을 낼 수 있기 때문이다. 그래서: 코드 하나만 쓰는 블록은 받아들이고, 마지막 앞 코드의 가장 긴 형태로 쓴 길이도 그렇다. 존재할 수 있는 것보다 많은 기호를 기술하는 코드, 형식 밖의 길이와 거리, 첫 바이트보다 앞을 가리키는 거리, 코드 표의 합이 맞지 않는 스트림은 모두 거부한다. gzip 헤더는 추가 필드, 파일 이름, 주석, 자기 체크섬을 실을 수 있고, 모두 읽고 지나간다(체크섬은 검사한다). 이름과 시간은 알려 주지 않는다.

압축기가 쓰는 것은 어떤 해제기든 읽는 유효한 DEFLATE다. 같은 입력에 대해 zlib가 썼을 것과 바이트 하나까지 같지는 않고, 그럴 필요도 없다.

여기에 없는 것:

- **미리 정한 사전**(zlib의 `FDICT`), 어느 방향이든.
- **ZIP 압축 파일, tar 파일**: 그것들은 그릇이다. 이것은 그 안의 코덱이다.
- **Deflate64**, 그리고 다른 모든 압축 형식: LZ4, bzip2, xz. Zstandard의 느린 레벨(libzstd의 10부터 22: 이진 트리와 optimal 파싱), 사전, long-distance 모드. Brotli 압축. Brotli의 큰 창 변형과 공유 사전.
- **gzip 헤더의 필드** - 이름, 시간, 주석 - 는 두거나 읽을 수 없다.

- **여러 gzip 멤버를 하나의 스트림으로** 자동으로 읽기: 멤버마다 reset한다(3절). (6절의 HTTP 클라이언트는 응답 본문에 대해 이것을 해 준다.)
- **HTTP 위에서:** 어느 방향이든 압축된 요청 본문. `br` 보내기. 스트림 응답이 지금까지 쓴 것을 밀어내는 호출. 미리 압축해 둔 파일 제공(핸들러가 직접 할 수 있다: `proven_http_accepts_coding`으로 물은 다음 자기 `Content-Encoding`을 쓴다).

어떻게 시험했는가. 등록된 테스트는 zlib 1.3.1이 만든 스트림 93개 - 모든 레벨과 전략, 작은 윈도, 중간의 flush, 세 형식 모두 - 를 한 바이트씩, 그리고 한 번의 호출로 풀어 zlib가 받았던 데이터를 얻는다. 드물지만 zlib가 받아들이는, 손으로 만든 스트림 열 개를 받아들인다. zlib가 거부하는, 손으로 만든 스트림 서른 개 - 스트림이 틀릴 수 있는 방식마다 하나 - 를 거부한다. 짧은 스트림들의 모든 진부분 접두에 대해 "입력을 더"를 답하고, 그것들의 바뀐 비트 하나하나에 대해 어떤 식으로든 답을 낸다 - 버퍼 밖에 쓰는 일 없이. 그리고 열한 종류의 데이터를 모든 레벨과 세 윈도 크기로 압축해 되찾는다. 등록된 테스트 밖에서는: 이 압축기가 더 큰 말뭉치에 대해 낸 모든 것 - 모든 레벨, 윈도, 형식, flush와 작은 조각으로 - 을 zlib가 풀었다(스트림 2,856개). 이 해제기를 5,838개에 대해 더 zlib와 비교했다. 그리고 무작위로 손상시킨 스트림 육백만 개를, 버퍼 밖의 읽기나 쓰기를 보고하는 도구 아래에서 풀었고, 보고된 것은 없었으며 해제기가 진전 없이 일하게 만든 스트림도 없었다.

6절은 zlib이 만든 본문 - gzip, zlib, raw DEFLATE, 멤버 둘, 선택 헤더 필드 전부, 그리고 끊기거나 손상되는 각 방식 - 을 두 클라이언트에 통째로 그리고 한 바이트짜리 청크로 통과시켜 시험했고, 시험의 표를 만들 때 각각을 zlib 자신의 판정과 대조했다. 두 서버는 일곱 가지 크기를, 한 번에 그리고 스트림으로, 평문과 TLS 위에서 압축했다. 등록된 시험 밖에서는 curl과 Python이 두 서버가 보내는 것을 읽었고, 두 클라이언트가 Python 서버가 보내는 것을 읽었다.

7절은 Node를 거쳐 libzstd 1.5.7에 대고 시험했다: 등록된 테스트는 libzstd가 일곱 종류의 데이터로 만든 스트림 91개 - 레벨 1, 3, 9, 19, 체크섬과 콘텐츠 크기가 있고 없는 것, 블록을 넘어 가리키는 일치, 창보다 긴 프레임 - 를 한 번의 호출로, 한 바이트씩, 그리고 엇갈린 걸음으로 풀어 libzstd가 받았던 데이터를 얻는다. 드물지만 libzstd가 받아들이는, 손으로 만든 프레임 열세 개(그중에는 모든 종류의 반복 오프셋이 있다)를 받아들이고, 스무 개를 그 이유(손상, 사전, 8 MiB를 넘는 창, 위의 빈 압축 블록)대로 거부한다. 짧은 스트림들의 모든 진부분 접두를 거부하고, 그것들의 바뀐 비트 하나하나에 대해 통째로든 걸음으로든 같은 판정을 낸다. 그리고 창 상한이 어떤 할당보다 먼저 거부하는 것, 들고 있는 메모리, 실패한 할당마다 `PROVEN_ERR_NOMEM`으로 답하는 것을 보인다. 등록된 테스트 밖에서는: libzstd가 만든 스트림 130개를 더 - 2.2 MiB까지, 프레임 여럿, flush한 스트림 - 1바이트부터 70,000바이트까지의 걸음으로 바이트 하나까지 같게 풀었다. 그중 일곱의 잘림과 바뀐 비트 약 7만 개를 버퍼 밖의 읽기나 쓰기를 보고하는 도구 아래에서 풀었고, 보고된 것은 없었다. HTTP에서는 블로킹, 이벤트 구동, HTTP/2 클라이언트가 Node 서버가 `zstd`로 보낸 것(레벨 1, 3, 19, 통째로, flush하며, 프레임 여럿으로, 3 MB까지)을 풀었고, 끊기거나 뒤에 쓰레기가 붙은 것은 거부했으며, 요청하지 않은 클라이언트에게는 gzip이 왔다. curl은 이 라이브러리 서버에서 핸들러가 직접 보낸 Zstandard를 읽었다.

8절은 Python 바인딩(1.1.0)을 거쳐 libbrotli에 대고 시험했다: 등록된 테스트는 libbrotli가 열 종류의 데이터로 만든 스트림 110개 - 품질 0, 1, 5, 9, 11, 1 KiB부터 16 MiB까지의 창, 텍스트와 일반 모드 - 를 한 번의 호출로, 한 바이트씩, 그리고 고르지 않은 걸음으로 풀어 libbrotli가 받았던 데이터를 얻는다. libbrotli가 푸는, 손으로 만든 스트림 66개를 같은 결과로 푼다 - 메타데이터, 비압축 메타블록, 모든 창 크기, 121가지 변환 모두를 거친 사전 단어(2바이트와 3바이트 글자의 대문자 변환 포함), 모든 문맥 모드, 반복과 move-to-front를 쓴 문맥 맵, 모든 짧은 거리 코드, 모든 종류의 블록 전환, 접두 부호를 쓰는 모든 방식. libbrotli가 거부하는 25개를, 각기 다른 방식으로 틀린 것들을 거부한다. 짧은 스트림들의 모든 진부분 접두를 거부하고, 그 모든 1비트 변경(72,376개)에 통째와 걸음 단위에서 같은 판정을 내린다. 창 상한이 할당 전에 거부하는 것, 쥐는 메모리, 실패한 할당마다 `PROVEN_ERR_NOMEM`이 오는 것도 보인다. 사전과 변환은 RFC 7932 본문에서 가져와, RFC가 밝힌 CRC-32와 libbrotli 자신의 사본 모두와 대조했다. 등록된 테스트 밖에서는: Python의 libbrotli가 만든 스트림 358개와 Node의 libbrotli(따로 들어 있는 1.2.0)가 만든 144개 - 이 저장소의 파일들, 한국어 매뉴얼, 소스 2.2 MB - 를 1바이트부터 70,000바이트까지의 걸음으로, 버퍼 밖의 읽기나 쓰기를 보고하는 도구 아래에서 바이트 하나까지 같게 풀었다. 위의 손상된 사본 1,437,490개. 그리고 알파벳마다 만들 수 있는 가장 큰 코드 표를 계산한 다음 실제로 만들어 보았다. HTTP에서는 블로킹, 이벤트 구동,

HTTP/2 클라이언트가 Node 서버가 `br`로 보낸 것(품질 1, 5, 11, 1 KiB부터 16 MiB까지의 창, 통째로와 flush하며, 3 MB까지)을 풀었고, 끊기거나 뒤에 쓰레기가 붙거나 두 번 이어진 것은 거부했으며, 요청하지 않은 클라이언트에게는 gzip이 왔다.

7절의 인코더는 자기 디코더와 libzstd에 대고 시험했다. 등록된 테스트는 열한 종류의 데이터(빈 것, 한 바이트, 같은 바이트의 연속, 번호 붙은 줄, 먼 뒤를 복사하는 잡음, 무작위 바이트, 128개가 넘는 바이트 값 등)를 아홉 레벨 모두와 세 창으로 통째로, 그리고 방식마다 한 레벨씩 스트림으로 - 조각마다 블록 flush, 출력은 한 바이트씩 - 압축하고, 모든 프레임을 이 라이브러리의 디코더로 푼다. 이 디코더는 오프셋마다 프레임이 밝힌 창 안인지 검사한다. 모든 레벨에서 스트림의 블록이 같은 입력을 통째로 압축한 블록과 바이트까지 같은지도 본다. 프레임 헤더, 입력 8,000개에 대한 상한, flush, 끝과 reset, 압축 블록 안의 한 값 리터럴, 쥐는 메모리, 실패한 할당을 검사하고, libzstd가 푼 프레임 열셋 - 그중 다섯은 reset 뒤에 버퍼를 밀며 스트림으로 만든 것 - 의 크기와 CRC를 고정해 둔다. 등록된 테스트 밖에서는 인코더가 말뭉치 - 이 저장소의 파일들, 한국어 매뉴얼, 바이너리, 소스 4 MB, 4,096바이트까지의 모든 크기와 블록 및 헤더 경계의 입력, 한 값·두 값·네 값의 리터럴, 블록의 시퀀스를 채우는 주기 - 로 만든 모든 프레임을 모든 레벨과 1 KiB부터 8 MiB까지의 창으로, 통째로 그리고 한 바이트부터의 조각으로, 버퍼 밖의 읽기나 쓰기를 보고하는 도구 아래에서 libzstd가 풀었다: 프레임 4,105개, 모두 바이트 하나까지 같다. HTTP에서는 Node의 libzstd와 curl이 두 서버가 `zstd`로 보낸 것을, 통째로와 스트림으로, 레벨 1, 2, 3, 6, 9와 1 KiB부터 8 MiB까지의 창으로 풀었고, gzip을 더 원하는 클라이언트에게는 가중치가 gzip을 골랐다.

하지 않은 것: 커버리지 기반 퍼징 없음. zlib 말고 두 번째 압축기의 출력과의 비교 없음. 외부 검토 없음. Zstandard에 대해서는: libzstd 말고 다른 인코더의 프레임 없음. 브라우저 없음. 인코더의 프레임을 푼 디코더는 libzstd와 이 라이브러리의 것뿐. Brotli에 대해서는: libbrotli의 두 사본 말고 다른 인코더의 스트림 없음. 브라우저 없음.

17장: HTTP/2 - 프레임과 헤더 압축

5부 — 운영체제와 대화하기. 선행 조건: 요청과 응답이 무엇인지는 10장, 압축 테이블을 만들 때 쓰는 할당자(allocator) - 예제에서는 힙(heap) 할당자 - 는 2장. 이 장을 마치면 HTTP/2가 무엇을 바꾸고 무엇을 바꾸지 않는지 말할 수 있고, 헤더 블록을 인코딩하고 디코딩하며, 프레임을 읽고 쓰고, 잘 만들어진 요청과 전달해서는 안 되는 요청을 가려내고, 직접 실어 나르는 바이트 위에서 연결 하나 - 스트림, 윈도, 리셋, 작별 - 를 몰 수 있고, 처리기를 바꾸지 않고 15장의 이벤트 구동 서버에서 HTTP/2를 - 그리고 HTTP/3을 - 켤 수 있다. 그리고 이 가운데 이 라이브러리가 지금 하는 것이 어디까지인지 말할 수 있다.

이 장은 `hpack.h`와 `h2.h`를 다룬다. 둘 다 순수한 계산이고 프리스탠딩(freestanding) 빌드에서 쓸 수 있다: HPACK은 인코더에 테이블 하나, 디코더에 테이블 하나를 할당하고, 프레임 함수는 아무것도 할당하지 않으며, 연결은 설정이 정한 한도 안에서 할당한다. 입력과 출력은 여러분의 메모리를 가리키는 뷰(view)로 주고받는다.

여기 있는 것을, 아래에서 위로. 2절부터 4절까지는 코덱이다: HTTP/2 연결 위를 오가는 것을 읽고 쓴다. 5절은 연결 그 자체다 - 어떤 스트림이 열려 있는지, 얼마나 보낼 수 있는지, 무엇에 무엇으로 답하는지 - 바이트를 받고 바이트를 돌려주는 객체로서. 6절과 7절은 그 객체가 일하는 모습이다: 이벤트 구동 서버는 요청받으면 같은 포트에서 HTTP/1.1과 나란히 HTTP/2로 말하고, 이벤트 구동 클라이언트는 합의한 오리진마다 HTTP/2 연결 하나를 간직한다. 11장의 블로킹 서버와 블로킹 클라이언트는 HTTP/1.1로만 말한다. 그 선이 정확히 어디인지는 8절에 있다. 9절은 같은 서버가 UDP 위 QUIC으로, 같은 처리기에게 HTTP/3을 말하는 모습이고, 10절은 요청이 청할 때 클라이언트가 그것을 말하는 모습이다.

목차

1. HTTP/2가 바꾸는 것, 한 쪽으로
2. 헤더 압축
3. 프레임
4. 메시지로서의 헤더 목록
5. 연결
6. 이벤트 구동 서버의 HTTP/2
7. 이벤트 구동 클라이언트의 HTTP/2
8. 무엇을 받아들이고, 무엇이 없으며, 어떻게 시험했는가
9. 이벤트 구동 서버의 HTTP/3
10. 이벤트 구동 클라이언트의 HTTP/3

1. HTTP/2가 바꾸는 것, 한 쪽으로

HTTP/2(RFC 9113)는 HTTP/1.1과 같은 요청과 응답 - 같은 메서드, 상태 코드, 헤더 필드 - 을 다른 모양으로 실어 나른다. 세 가지가 다르다:

- **모든 것이 프레임이다.** 헤더 아홉 바이트 - 길이, 종류, 플래그, 스트림 번호 - 와 페이로드. 파싱할 텍스트도 끝낼 줄도 없다: 메시지는 HEADERS 프레임과 DATA 프레임이다.
- **여러 교환이 한 연결을 나눠 쓴다.** 요청 하나와 그 응답이 번호가 붙은 *스트림*이다. 서로 다른 스트림의 프레임이 섞여 오가므로, 느린 응답이 뒤의 것들을 붙잡지 않는다. 스트림마다, 그리고 연결 전체에 *윈도*가 있다: 상대가 받겠다고 말한 DATA 바이트 수다.
- **헤더 필드는 압축된다.** 헤더를 위해 만든 방식(HPACK, RFC 7541)으로. 같은 필드가 요청마다 다시 전송되기 때문이다.

HTTP/1.1이 첫 줄에서 말하던 것은 이름이 콜론으로 시작하는 *유사 헤더 필드*가 말한다: 요청에는 `:method`, `:scheme`, `:authority`, `:path`, 응답에는 `:status`. `Host`는 없고(`:authority`가 그것이다) `Connection`도 `Transfer-Encoding`도 없다: 그것들이 하던 일은 프레이밍이 한다.

연결은 클라이언트가 보내는 고정된 24바이트(`PROVEN_H2_PREFACE`)로 시작하고, 이어서 양쪽이 SETTINGS 프레임을 하나씩 보낸다.

2. 헤더 압축

```
proven_err_t proven_hpack_encoder_create(proven_allocator_t alloc, proven_size_t max_table_bytes,
proven_hpack_encoder_t **out);
proven_err_t proven_hpack_encode(proven_hpack_encoder_t *encoder, const proven_http_header_t *headers,
proven_size_t count,
                                 const bool *sensitive, proven_mem_mut_t out, proven_size_t *len);

proven_err_t proven_hpack_decoder_create(proven_allocator_t alloc, proven_size_t max_table_bytes,
proven_hpack_decoder_t **out);
proven_err_t proven_hpack_decode(proven_hpack_decoder_t *decoder, proven_mem_view_t block,
                                 proven_http_header_t *headers, proven_size_t header_cap,
                                 proven_mem_mut_t scratch, proven_size_t max_list_bytes, proven_size_t
*count);
```

HPACK은 필드를 세 가지 방법 가운데 하나로 보낸다: 흔한 필드 61개가 담긴 고정 테이블의 항목을 가리키는 번호로(`:method: GET`은 2번 항목이다), 이 연결이 전에 보낸 필드를 담은 *동적 테이블*의 항목을 가리키는 번호로, 또는 리터럴로 - 리터럴은 보내면서 동적 테이블에 넣을 수 있고, 그 글자는 허프만 코딩할 수 있다. 처음에 65바이트인 요청이 두 번째에는 22바이트다.

양쪽은 같은 테이블을 유지해야 한다. 모든 블록은 보낸 순서대로, 그 연결 그 방향의 디코더 하나가 디코딩한다. 블록 하나를 건너뛰거나 두 번 디코딩하면 그 뒤의 모든 블록이 다른 뜻이 된다. 잘못된 블록을 만난 디코더가 그 뒤의 것을 모두 거부하는 까닭, 그리고 HTTP/2가 그 일로 연결을 끝내는 까닭이 이것이다.

테이블 크기는 여러분이 정하며, 그것은 비용이 아니라 상한이다: 기본 4,096바이트(`PROVEN_HPACK_DEFAULT_TABLE_BYTES`). 그 뒤의 메모리는 항목이 추가되는 만큼 자란다 - 짧은 필드 몇 개를 보내는 상대라면 수백 바이트 - 테이블이 가득 차면 크기의 한 배 반쯤이 된다. 0으로 만든 인코더는 테이블을 두지 않아 상태가 없고, 0으로 만든 디코더는 상대에게 테이블을 두지 말라고 알린다.

예제: 두 번 보낸 요청, 테이블에 넣지 않은 비밀, 그리고 한도

```
#include <stdio.h>
#include <string.h>

/*
 * HPACK: HTTP/2가 헤더 필드를 보내는 방식.
 *
 * 인코더는 필드 목록을 바이트 블록으로 바꾸고, 반대편의 디코더는 블록을 다시 목록으로 바꾼다.
 * 양쪽 모두 앞서 지나간 필드의 테이블을 유지하며, 같은 필드를 두 번째로 보낼 때는 한 바이트다.
 * 두 테이블은 같은 상태여야 한다 - 그래서 연결에는 방향마다 인코더 하나와 디코더 하나가 있고,
 * 순서대로 쓴다.
 */

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_hpack_encoder_t *encoder = NULL;
    proven_hpack_decoder_t *decoder = NULL;
    /* 테이블 4096바이트가 프로토콜의 기본값이다. 양쪽이 다른 크기에 합의할 수 있다. */
    EXAMPLE_REQUIRE(proven_hpack_encoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &encoder) == PROVEN_OK
&&
                    proven_hpack_decoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &decoder) ==
PROVEN_OK, "an encoder here, a decoder there");

    // ---- 요청 하나를 두 번 ------------------------------------------------------
    /* HTTP/2에서 이름은 소문자이고, HTTP/1이 첫 줄에 적던 것은 여기서 이름이 콜론으로
     * 시작하는 필드가 된다. */
    proven_http_header_t request[] = {
        { PROVEN_LIT(":method"), PROVEN_LIT("GET") },
        { PROVEN_LIT(":scheme"), PROVEN_LIT("https") },
        { PROVEN_LIT(":path"), PROVEN_LIT("/reports/2026/october") },
        { PROVEN_LIT(":authority"), PROVEN_LIT("www.example.com") },
```

```
        { PROVEN_LIT("user-agent"), PROVEN_LIT("example-client/1.0") },
        { PROVEN_LIT("authorization"), PROVEN_LIT("Bearer 5f2c1e9a7b3d") },
    };
    proven_byte_t block[512];
    EXAMPLE_REQUIRE(proven_hpack_encode_bound(request, 6) <= sizeof block, "the block will fit");
    proven_size_t first = 0;
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, request, 6, NULL, (proven_mem_mut_t){ block, sizeof block },
&first) == PROVEN_OK, "the request, encoded");

    /* 디코더는 필드를 뷰로 돌려준다: 블록 안, 고정 테이블 안, 또는 빌려준 스크래치 안을
     * 가리킨다. */
    proven_http_header_t fields[16];
    proven_byte_t scratch[1024];
    proven_size_t count = 0;
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ block, first }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 6,
                    "and decoded");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(fields[2].name, PROVEN_LIT(":path")) &&
proven_u8str_view_eq(fields[2].value, PROVEN_LIT("/reports/2026/october")), "field for field");

    /* 같은 요청을 다시 보내면 비용은 일부분이다: 양쪽이 기억하고 있다. */
    proven_size_t second = 0;
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, request, 6, NULL, (proven_mem_mut_t){ block, sizeof block },
&second) == PROVEN_OK, "the same request again");
    printf("the first time: %u bytes; the second: %u\n", (unsigned)first, (unsigned)second);
    EXAMPLE_REQUIRE(second < first / 2, "less than half");
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ block, second }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 6 &&
                    proven_u8str_view_eq(fields[4].value, PROVEN_LIT("example-client/1.0")), "and the decoder
still gives the whole list");

    // ---- 기억하지 않는 것 ------------------------------------------------------------
    /* 테이블은 압축 문맥이고, 그 안의 비밀은 뒤따르는 블록의 크기로 추측될 수 있다.
     * 그래서 authorization 값은 테이블에 들어간 적이 없다 - 어느 쪽에서도: */
    proven_size_t entries = proven_hpack_decoder_table_count(decoder);
    bool secret_kept = false;
    for (proven_size_t i = 0; i < entries; ++i) {
        proven_u8str_view_t name, value;
        EXAMPLE_REQUIRE(proven_hpack_decoder_table_entry(decoder, i, &name, &value), "an entry of the table");
        if (proven_u8str_view_eq(name, PROVEN_LIT("authorization"))) secret_kept = true;
    }
    printf("the table holds %u entries, %u bytes by the protocol's count\n", (unsigned)entries,
(unsigned)proven_hpack_decoder_table_bytes(decoder));
    EXAMPLE_REQUIRE(entries == 3 && !secret_kept, "the path, the authority and the user agent - not the
secret");
    /* 어떤 필드든 같은 취급을 받게 할 수 있다. */
    proven_http_header_t more[] = { { PROVEN_LIT("x-api-key"), PROVEN_LIT("k-0123456789") } };
    bool sensitive[] = { true };
    bool marked[16] = { false };
    proven_size_t n = 0;
    /* 디코더는 어느 필드가 표시와 함께 왔는지 알려 줄 수 있다 - 필드를 넘기는 쪽에 필요한 것. */
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, more, 1, sensitive, (proven_mem_mut_t){ block, sizeof
block }, &n) == PROVEN_OK &&
                    proven_hpack_decode_marked(decoder, (proven_mem_view_t){ block, n }, fields, marked, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK &&
                    proven_hpack_decoder_table_count(decoder) == 3 && marked[0], "a field marked sensitive
adds nothing to the table, and arrives marked");

    // ---- 한도, 그리고 그것의 쓸모 ----------------------------------------------------
    /* 블록의 크기는 그것이 말하는 내용의 크기가 아니다. 이 블록은 다섯 바이트이고, 기억된
```

```
     * 같은 필드를 다섯 번 가리킨다: */
    static const proven_byte_t repeat[] = { 0xbe, 0xbe, 0xbe, 0xbe, 0xbe };       /* "62번 항목" - 테이블에서
가장 새것 */
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, sizeof repeat }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 5,
                    "five bytes, five fields");
    /* 그래서 한도는 블록이 아니라 풀린 목록 - 이름, 값, 그리고 필드마다 32 - 에 걸린다.
     * 넘으면 필드는 내주지 않지만, 디코더는 블록을 끝까지 읽었고 여전히 쓸 수 있다:
     * 그 요청 하나만 거절하고 연결은 유지한다. */
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, sizeof repeat }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 100, &count) == PROVEN_ERR_OUT_OF_BOUNDS && count == 0,
                    "a list limit of 100 bytes refuses it");
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, 1 }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 100, &count) == PROVEN_OK && count == 1, "and the next block is
decoded as if nothing had happened");

    /* 아예 HPACK이 아닌 블록은 다른 문제다: 두 테이블이 일치한다고 더는 믿을 수 없고,
     * 디코더는 그때부터 그렇게 말한다. HTTP/2에서는 연결이 끝난다. */
    static const proven_byte_t nonsense[] = { 0x80 };                             /* "0번 항목": 그런 것은 없다
*/
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ nonsense, 1 }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_ERR_INVALID_FORMAT &&
                    proven_hpack_decode(decoder, (proven_mem_view_t){ repeat, 1 }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_ERR_INVALID_FORMAT,
                    "a malformed block is refused, and so is everything after it");

    // ---- 테이블 크기는 협상된다 ------------------------------------------------
    /* 상대가 더 작은 테이블을 알리면 인코더에게 전하고, 인코더는 다음 블록의 맨 앞에서
     * 따랐다고 말한다. 자기 한도가 낮아진 디코더는 그 말을 요구한다. */
    proven_hpack_decoder_destroy(decoder);
    EXAMPLE_REQUIRE(proven_hpack_decoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &decoder) ==
PROVEN_OK, "a fresh decoder");
    proven_hpack_decoder_set_max_table_bytes(decoder, 0);          /* "이쪽은 테이블을 두지 않겠다" ... */
    proven_hpack_encoder_set_peer_table_bytes(encoder, 0);         /* ... 인코더가 그것을 들었다 */
    n = 0;
    EXAMPLE_REQUIRE(proven_hpack_encode(encoder, request, 6, NULL, (proven_mem_mut_t){ block, sizeof block },
&n) == PROVEN_OK && block[0] == 0x20, "the next block opens by saying: table size 0");
    EXAMPLE_REQUIRE(proven_hpack_decode(decoder, (proven_mem_view_t){ block, n }, fields, 16,
(proven_mem_mut_t){ scratch, sizeof scratch }, 4096, &count) == PROVEN_OK && count == 6 &&
                    proven_hpack_decoder_table_count(decoder) == 0, "and nothing is remembered on either
side");

    // ---- 허프만 코딩만 따로 ----------------------------------------------------
    /* 리터럴 문자열은 흔한 문자에 더 적은 비트를 주는 코드로 보낸다. */
    proven_u8str_view_t text = PROVEN_LIT("www.example.com");
    proven_byte_t coded[32], back[32];
    proven_size_t coded_len = 0, back_len = 0;
    EXAMPLE_REQUIRE(proven_hpack_huffman_size(text) == 12 && proven_hpack_huffman_encode(text,
(proven_mem_mut_t){ coded, sizeof coded }, &coded_len) == PROVEN_OK && coded_len == 12, "fifteen characters in
twelve bytes");
    EXAMPLE_REQUIRE(proven_hpack_huffman_decode((proven_mem_view_t){ coded, coded_len }, (proven_mem_mut_t)
{ back, sizeof back }, &back_len) == PROVEN_OK && back_len == 15 && memcmp(back, "www.example.com", 15) == 0,
"and back");

    proven_hpack_encoder_destroy(encoder);
    proven_hpack_decoder_destroy(decoder);
    return EXAMPLE_OK();
}
```

참조

API	의도	반환
`proven_hpack_encoder_create(alloc, max_table_bytes, &enc)`	테이블을 많아야 이만큼 쓰는 인코더(0: 없음).	`proven_err_t`: `INVALID_ARG`, `NOMEM`.
`proven_hpack_encode(enc, headers, count, sensitive, out, &len)`	이 필드들의 헤더 블록 하나를 덧붙인다. `sensitive`는 선택 사항.	`proven_err_t`: `out`의 자리가 bound보다 적으면 `OUT_OF_BOUNDS` - 아무것도 쓰지 않고 아무것도 바뀌지 않는다.
`proven_hpack_encode_bound(headers, count)`	언제나 충분한 자리.	`proven_size_t`.
`proven_hpack_encoder_set_peer_table_bytes(enc, n)`	상대의 한도가 바뀌었다. 다음 블록에서 알린다.	없음.
`proven_hpack_decoder_create(alloc, max_table_bytes, &dec)`	상대에게 이만큼의 테이블을 허용하는 디코더.	`proven_err_t`.
`proven_hpack_decode(dec, block, headers, cap, scratch, max_list_bytes, &count)`	블록 하나를 통째로 디코딩한다.	`proven_err_t`: `OUT_OF_BOUNDS` - 목록이 너무 크거나, 필드가 너무 많거나, 스크래치가 너무 작다. 디코더는 여전히 쓸 수 있다. `INVALID_FORMAT` - HPACK이 아니다. 디코더는 끝났다.
`proven_hpack_decode_marked(dec, block, headers, never_indexed, cap, scratch, max_list_bytes, &count)`	같은 일을 하고, `never_indexed[i]`가 필드 `i`가 절대 인덱싱하지 말라는 형태로 왔는지 알려 준다.	`proven_hpack_decode`와 같다.
`proven_hpack_decoder_set_max_table_bytes(dec, n)`	이쪽의 한도가 바뀌었다. 낮췄다면 상대의 다음 블록이 따랐다고 말해야 한다.	없음.
`proven_hpack_decoder_table_count`, `_table_bytes`, `_table_entry`	디코더의 테이블을 들여다본다.	개수. `bool`.
`proven_hpack_huffman_size`, `_huffman_encode`, `_huffman_decode`	문자열 코드만 따로.	크기. `proven_err_t`.
`proven_hpack_encoder_destroy`, `proven_hpack_decoder_destroy`	해제한다. 널을 받는다.	없음.

주의사항, 그리고 무엇이 잘못되는가

- **블록의 크기는 그것이 말하는 내용의 크기가 아니다.** 다섯 바이트가 기억된 큰 필드를 다섯 번 가리킬 수 있고, 오천 바이트는 오천 번 가리킬 수 있다. `max_list_bytes`를 늘 줄 것: 풀린 목록 - RFC가 세는 대로 이름, 값, 그리고 필드마다 32바이트 - 을 묶고, 그 크기의 `scratch`면 언제나 충분하다.
- **한도 초과는 연결의 끝이 아니고, 잘못된 형식은 끝이다.** `OUT_OF_BOUNDS`는 블록을 끝까지 읽었고 테이블이 맞아 있으며 이 메시지만 거절한다는 뜻이다. `INVALID_FORMAT`은 두 테이블이 달라졌을 수 있다는 뜻이고, 디코더는 그 뒤로 계속 같은 답을 한다.
- **뷰, 세 가지.** 디코딩된 이름이나 값은 블록 안, 라이브러리의 고정 테이블 안, 또는 여러분이 빌려준 임시 작업용(scratch) 버퍼 - 아래에서는 스크래치 - 안을 가리킨다. 셋 모두 필드를 쓰는 동안 살아 있어야 한다. 간직할 것은 복사한다.
- **비밀은 테이블에 넣지 않는다.** 테이블은 압축 문맥이고, 그 안의 것은 공격자가 영향을 주는 뒤따르는 블록의 크기로 한 바이트씩 추측될 수 있다. 인코더는 `authorization`과 `proxy-authorization`의 값을 기억하지 않고, 어느 중간 지점도 기억하지 않도록 표시해서 보낸다. `cookie`와 `set-cookie`도 기억하지 않는다. `sensitive`는 여

러분이 지목한 필드까지 그렇게 다룬다. 필드를 받아서 넘기는 쪽 - 프록시 - 은 어느 필드가 그 표시와 함께 왔는지 `proven_hpack_decode_marked`로 알아내어, 그 표시를 인코더의 `sensitive`로 넘긴다.

- **두 번 같기 어려운 것도 넣지 않는다.** `content-length`, `content-range`, `etag`, `age`, `if-none-match`, `if-modified-since`는 이름의 인덱스로 보내고 기억하지 않는다. 새 값마다 테이블에서 되풀이되는 필드의 자리를 빼앗기 때문이다. (`content-length`가 매번 다른 응답 헤드: 값을 기억하면 26바이트에 초당 430,000개, 기억하지 않으면 13바이트에 초당 1,020,000개 - 한 기계에서, 인코딩과 디코딩.)
- **이름은 인코더에 닿기 전에 소문자여야 한다.** 인코더는 받은 대로 인코딩한다. 목록이 보내도 되는 것인지 검사하는 일은 4절의 것이다.
- **인코더 하나, 디코더 하나, 방향 하나, 순서대로.** 요청마다가 아니고 스트림마다가 아니다.

3. 프레임

```
proven_err_t proven_h2_frame_parse(proven_mem_view_t data, proven_h2_frame_t *out);
proven_err_t proven_h2_frame_check(const proven_h2_frame_t *frame, proven_u32 max_frame_size,
proven_h2_error_t *code, bool *stream_error);
proven_err_t proven_h2_frame_write(proven_mem_mut_t out, proven_size_t *len, const proven_h2_frame_t *frame);
```

프레임은 아홉 바이트와 페이로드다:

필드	비트	뜻
길이	24	페이로드의 바이트 수. 모든 끝점은 16,384를 받는다. 더 큰 것은 그렇게 말한 경우에만
종류	8	열 가지 가운데 하나. 그 밖의 것은 건너뛴다
플래그	8	뜻은 종류에 따라 다르다
스트림	31(그리고 예약 비트 하나)	0은 연결 자체. 클라이언트는 홀수 스트림을 연다

`proven_h2_frame_check`는 페이로드를 보기 전에 종류마다 있는 규칙을 적용한다:

종류	스트림	크기	싣는 것
`DATA`	필요	아무 크기	본문의 한 조각. 패딩될 수 있다. `END_STREAM`은 보낸 쪽을 끝낸다
`HEADERS`	필요	아무 크기	헤더 블록, 또는 그 시작. 패딩될 수 있고, 우선순위 다섯 바이트를 실을 수 있는데 그것은 건너뛴다
`CONTINUATION`	필요	아무 크기	`END_HEADERS`가 나올 때까지 헤더 블록의 나머지
`RST_STREAM`	필요	4	오류 코드: 이 스트림은 끝났다
`SETTINGS`	0이어야 한다	6의 배수. `ACK`면 0	식별자와 값의 쌍
`PING`	0이어야 한다	8	`ACK`와 함께 돌려보낼 여덟 바이트
`GOAWAY`	0이어야 한다	8 이상	처리한 마지막 스트림, 오류 코드, 사람이 읽을 글
`WINDOW_UPDATE`	어느 쪽이든	4	몇 바이트를 더 보내도 되는가
`PRIORITY`	필요	5	폐기된 것. 검사하고 무시한다
`PUSH_PROMISE`	필요	아무 크기	서버 푸시. 이 라이브러리는 언제나 사양한다

오류에는 두 가지 크기가 있다. *연결 오류*는 연결을 끝낸다: 코드를 실은 GOAWAY를 보내고 닫는다. *스트림 오류*는 스트림 하나를 끝낸다: RST_STREAM을 보내고 계속한다. 검사는 코드와 어느 쪽인지를 알려 준다.

예제: 서버가 읽듯이 읽은, 연결의 첫 바이트들

```
#include <stdio.h>
#include <string.h>

/*
 * HTTP/2 프레임과 그 안의 헤더 목록 - 서버가 연결의 첫 바이트들을 읽듯이, 여기서는 버퍼에서
```

```
 * 읽는다.
 *
 * h2.h에는 연결을 하나로 묶어 주는 것이 없다. 프레임을 읽고 쓰고, 프레임 종류마다 있는 규칙을
 * 적용하고, 풀린 헤더 목록이 요청인지 검사한다. 어떤 스트림이 열려 있고 얼마나 보낼 수 있는지는
 * 이것들을 쓰는 쪽의 일이다.
 */

/* 클라이언트가 연결을 열고 페이지 하나를 요청할 때 보낼 바이트. */
static proven_size_t client_bytes(proven_allocator_t heap, proven_byte_t *wire, proven_size_t cap) {
    proven_mem_mut_t out = { wire, cap };
    proven_size_t len = 0;
    memcpy(wire, PROVEN_H2_PREFACE, PROVEN_H2_PREFACE_SIZE);       /* 고정된 24바이트가 먼저 온다 */
    len = PROVEN_H2_PREFACE_SIZE;

    /* 다음은 클라이언트의 설정: 여기서는 "푸시하지 말 것"과 더 큰 윈도. */
    proven_h2_setting_t settings[] = {
        { PROVEN_H2_SETTINGS_ENABLE_PUSH, 0 },
        { PROVEN_H2_SETTINGS_INITIAL_WINDOW_SIZE, 1048576 },
    };
    if (proven_h2_write_settings(out, &len, settings, 2) != PROVEN_OK) return 0;

    /* 요청은 새 스트림 - 클라이언트의 스트림은 홀수 - 위의 HEADERS 프레임이고, HPACK
     * 블록을 싣는다. END_HEADERS: 블록이 온전하다. END_STREAM: 본문이 뒤따르지 않는다. */
    proven_http_header_t request[] = {
        { PROVEN_LIT(":method"), PROVEN_LIT("GET") },
        { PROVEN_LIT(":scheme"), PROVEN_LIT("https") },
        { PROVEN_LIT(":path"), PROVEN_LIT("/") },
        { PROVEN_LIT(":authority"), PROVEN_LIT("www.example.com") },
        { PROVEN_LIT("accept"), PROVEN_LIT("text/html") },
    };
    proven_hpack_encoder_t *encoder = NULL;
    proven_byte_t block[256];
    proven_size_t block_len = 0;
    if (proven_hpack_encoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &encoder) != PROVEN_OK) return 0;
    proven_err_t err = proven_hpack_encode(encoder, request, 5, NULL, (proven_mem_mut_t){ block, sizeof
block }, &block_len);
    proven_hpack_encoder_destroy(encoder);
    if (err != PROVEN_OK) return 0;
    proven_h2_frame_t headers = { .length = (proven_u32)block_len, .type = PROVEN_H2_HEADERS,
                                  .flags = PROVEN_H2_FLAG_END_HEADERS | PROVEN_H2_FLAG_END_STREAM, .stream_id
= 1 };
    if (proven_h2_frame_write(out, &len, &headers) != PROVEN_OK) return 0;
    memcpy(wire + len, block, block_len);
    len += block_len;

    /* 그리고 예제를 위해, 규칙을 어기는 프레임 하나: PING은 여덟 바이트여야 한다. */
    proven_h2_frame_t bad_ping = { .length = 4, .type = PROVEN_H2_PING };
    if (proven_h2_frame_write(out, &len, &bad_ping) != PROVEN_OK) return 0;
    memset(wire + len, 0, 4);
    return len + 4;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_byte_t wire[1024];
    proven_size_t size = client_bytes(heap, wire, sizeof wire);
    EXAMPLE_REQUIRE(size > 0, "the client's bytes");

    // ---- 서버 쪽 ----------------------------------------------------------------
    EXAMPLE_REQUIRE(size >= PROVEN_H2_PREFACE_SIZE && memcmp(wire, PROVEN_H2_PREFACE, PROVEN_H2_PREFACE_SIZE)
```

```
== 0, "the preface: this is HTTP/2");
    proven_size_t at = PROVEN_H2_PREFACE_SIZE;

    proven_hpack_decoder_t *decoder = NULL;
    EXAMPLE_REQUIRE(proven_hpack_decoder_create(heap, PROVEN_HPACK_DEFAULT_TABLE_BYTES, &decoder) ==
PROVEN_OK, "the connection's decoder");
    int frames = 0, requests = 0;
    proven_h2_error_t refused_with = PROVEN_H2_NO_ERROR;

    while (at < size) {
        /* 헤더 아홉 바이트가 프레임의 길이, 종류, 그리고 어느 스트림의 것인지를 말한다. */
        proven_h2_frame_t frame;
        proven_err_t err = proven_h2_frame_parse((proven_mem_view_t){ wire + at, size - at }, &frame);
        if (err == PROVEN_ERR_NEED_MORE) break;                    /* 실제 서버라면 더 읽을 것이다 */
        EXAMPLE_REQUIRE(err == PROVEN_OK, "a frame header");

        /* 페이로드를 보기 전에: 이 프레임은 자기가 말하는 그것이어도 되는가? 크기 한도는
         * 이쪽이 알린 값이다(여기서는 프로토콜의 최솟값). */
        proven_h2_error_t code;
        bool stream_error;
        if (proven_h2_frame_check(&frame, PROVEN_H2_MIN_MAX_FRAME_SIZE, &code, &stream_error) != PROVEN_OK) {
            /* 실제 서버는 GOAWAY로 답한다(스트림만 잘못됐으면 RST_STREAM). */
            refused_with = code;
            EXAMPLE_REQUIRE(!stream_error, "a PING of the wrong size ends the connection");
            break;
        }
        proven_mem_view_t payload = { wire + at + PROVEN_H2_FRAME_HEADER_SIZE, frame.length };
        EXAMPLE_REQUIRE(PROVEN_H2_FRAME_HEADER_SIZE + frame.length <= size - at, "the whole frame is here");
        frames++;

        if (frame.type == PROVEN_H2_SETTINGS) {
            proven_size_t pos = 0;
            proven_h2_setting_t setting;
            while ((err = proven_h2_settings_next(payload, &pos, &setting, &code)) == PROVEN_OK) {
                if (setting.id == PROVEN_H2_SETTINGS_INITIAL_WINDOW_SIZE) printf("the client's window: %u
bytes a stream\n", (unsigned)setting.value);
                /* 이쪽이 모르는 설정은 그저 따르지 않는다. */
            }
            EXAMPLE_REQUIRE(err == PROVEN_ERR_EOF, "every setting was in range");
        } else if (frame.type == PROVEN_H2_HEADERS) {
            /* 헤더 블록은 패딩과 옛 우선순위 필드를 걷어 낸 나머지다. (END_HEADERS가 꺼져
             * 있으면 CONTINUATION 프레임들이 나머지를 싣는다.) */
            proven_mem_view_t block;
            EXAMPLE_REQUIRE(proven_h2_frame_content(&frame, payload, &block) == PROVEN_OK && (frame.flags &
PROVEN_H2_FLAG_END_HEADERS) != 0, "a whole header block");
            proven_http_header_t fields[32];
            proven_byte_t scratch[2048];
            proven_size_t count = 0;
            EXAMPLE_REQUIRE(proven_hpack_decode(decoder, block, fields, 32, (proven_mem_mut_t){ scratch,
sizeof scratch }, 16384, &count) == PROVEN_OK, "decoded");

            /* 필드 목록이 아직 요청인 것은 아니다. 대문자가 든 이름, 값 안의 줄바꿈, 빠진
             * :path, "connection" 헤더가 여기서 걸린다. */
            proven_h2_request_head_t head;
            EXAMPLE_REQUIRE(proven_h2_request_head(fields, count, &head) == PROVEN_OK, "a well-formed
request");
            printf("stream %u: %.*s %.*s for %.*s, %u more fields\n", (unsigned)frame.stream_id,
(int)head.method.size, (const char *)head.method.ptr,
                   (int)head.path.size, (const char *)head.path.ptr, (int)head.authority.size, (const char
*)head.authority.ptr, (unsigned)head.field_count);
```

```
            EXAMPLE_REQUIRE(proven_u8str_view_eq(head.path, PROVEN_LIT("/")) && head.field_count == 1 &&
proven_u8str_view_eq(head.fields[0].name, PROVEN_LIT("accept")), "its parts");
            requests++;
        }
        at += PROVEN_H2_FRAME_HEADER_SIZE + frame.length;
    }
    EXAMPLE_REQUIRE(frames == 2 && requests == 1 && refused_with == PROVEN_H2_FRAME_SIZE_ERROR, "two frames
taken, one request, and the third frame refused with FRAME_SIZE_ERROR");
    proven_hpack_decoder_destroy(decoder);

    // ---- 서버가 써 보내는 것 ------------------------------------------------------
    proven_byte_t reply[256];
    proven_mem_mut_t out = { reply, sizeof reply };
    proven_size_t len = 0;
    static const proven_byte_t opaque[8] = { 'p', 'r', 'o', 'v', 'e', 'n', '!', '!' };
    EXAMPLE_REQUIRE(proven_h2_write_settings(out, &len, NULL, 0) == PROVEN_OK &&       /* 자기 설정: 모두 기본
값 */
                    proven_h2_write_settings_ack(out, &len) == PROVEN_OK &&            /* "그쪽 설정이 적용되
었다" */
                    proven_h2_write_window_update(out, &len, 0, 1 << 20) == PROVEN_OK &&   /* 연결에 1메가바이
트를 더 보내도 된다 */
                    proven_h2_write_ping(out, &len, opaque, false) == PROVEN_OK &&     /* 거기 있는가? */
                    proven_h2_write_rst_stream(out, &len, 3, PROVEN_H2_REFUSED_STREAM) == PROVEN_OK &&   /* 그
스트림은 지금은 안 된다 */
                    proven_h2_write_goaway(out, &len, 1, PROVEN_H2_FRAME_SIZE_ERROR,
proven_mem_view_from_u8(PROVEN_LIT("a PING is eight bytes"))) == PROVEN_OK,
                    "six frames a server might send");

    /* 클라이언트가 하듯이 다시 읽는다. */
    at = 0;
    int seen = 0;
    while (at < len) {
        proven_h2_frame_t frame;
        proven_h2_error_t code;
        bool stream_error;
        EXAMPLE_REQUIRE(proven_h2_frame_parse((proven_mem_view_t){ reply + at, len - at }, &frame) ==
PROVEN_OK &&
                        proven_h2_frame_check(&frame, PROVEN_H2_MIN_MAX_FRAME_SIZE, &code, &stream_error) ==
PROVEN_OK, "each passes its own rule");
        proven_mem_view_t payload = { reply + at + PROVEN_H2_FRAME_HEADER_SIZE, frame.length };
        proven_u32 a = 0, b = 0;
        proven_mem_view_t debug;
        if (frame.type == PROVEN_H2_WINDOW_UPDATE) EXAMPLE_REQUIRE(proven_h2_read_window_update(payload, &a)
== PROVEN_OK && a == 1 << 20, "the increment");
        if (frame.type == PROVEN_H2_RST_STREAM) EXAMPLE_REQUIRE(proven_h2_read_rst_stream(payload, &a) ==
PROVEN_OK && a == PROVEN_H2_REFUSED_STREAM && frame.stream_id == 3, "the reset");
        if (frame.type == PROVEN_H2_GOAWAY) EXAMPLE_REQUIRE(proven_h2_read_goaway(payload, &a, &b, &debug) ==
PROVEN_OK && a == 1 && b == PROVEN_H2_FRAME_SIZE_ERROR && debug.size == 21, "the goodbye");
        seen++;
        at += PROVEN_H2_FRAME_HEADER_SIZE + frame.length;
    }
    EXAMPLE_REQUIRE(seen == 6, "six frames read back");

    // ---- 응답과 트레일러 ----------------------------------------------------------
    proven_http_header_t response[] = { { PROVEN_LIT(":status"), PROVEN_LIT("200") }, { PROVEN_LIT("content-
type"), PROVEN_LIT("text/html") } };
    proven_u16 status = 0;
    const proven_http_header_t *rest = NULL;
    proven_size_t rest_count = 0;
    EXAMPLE_REQUIRE(proven_h2_response_head(response, 2, &status, &rest, &rest_count) == PROVEN_OK && status
```

```
== 200 && rest_count == 1, "a response has one pseudo-header field: its status");
    proven_http_header_t smuggle[] = { { PROVEN_LIT(":status"), PROVEN_LIT("200") }, { PROVEN_LIT("x-note"),
PROVEN_LIT("fine\r\ntransfer-encoding: chunked") } };
    EXAMPLE_REQUIRE(proven_h2_response_head(smuggle, 2, &status, &rest, &rest_count) ==
PROVEN_ERR_INVALID_FORMAT, "a value with a line break in it is no response at all");
    proven_http_header_t trailers[] = { { PROVEN_LIT("x-checksum"), PROVEN_LIT("9f86d081") } };
    EXAMPLE_REQUIRE(proven_h2_trailers_check(trailers, 1) == PROVEN_OK, "trailers are ordinary fields");
    return EXAMPLE_OK();
}
```

참조

API	의도	반환
`proven_h2_frame_parse(data, &frame)`	프레임 헤더 아홉 바이트를 읽는다.	`proven_err_t`: 아홉 바이트 미만이면 `NEED_MORE`.
`proven_h2_frame_write(out, &len, &frame)`	프레임 헤더를 덧붙인다.	`proven_err_t`: `INVALID_ARG`(길이가 2^24-1 초과, 스트림이 2^31-1 초과), `OUT_OF_BOUNDS`.
`proven_h2_frame_check(&frame, max_frame_size, &code, &stream_error)`	위의 표를 적용한다.	`proven_err_t`: `INVALID_FORMAT`과 함께 보낼 코드, 그리고 스트림이 끝나는지 연결이 끝나는지.
`proven_h2_frame_content(&frame, payload, &content)`	패딩과 우선순위를 뺀 DATA나 HEADERS의 내용.	`proven_err_t`: 맞지 않는 패딩은 `INVALID_FORMAT`(연결 오류, `PROTOCOL_ERROR`).
`proven_h2_settings_next(payload, &pos, &setting, &code)`	다음 설정을 읽는다.	`proven_err_t`: 끝에서 `EOF`. 범위를 벗어난 값은 코드와 함께 `INVALID_FORMAT`.
`proven_h2_write_settings`, `_settings_ack`, `_ping`, `_window_update`, `_rst_stream`, `_goaway`	프레임 하나를 통째로 덧붙인다.	`proven_err_t`: 아무것도 쓰지 않은 채 `OUT_OF_BOUNDS`.
`proven_h2_read_window_update`, `_rst_stream`, `_goaway`	검사를 통과한 프레임의 페이로드를 읽는다.	`proven_err_t`. 윈도 증가분 0은 `INVALID_FORMAT`.

주의사항, 그리고 무엇이 잘못되는가

- **읽기 전에 검사한다.** `proven_h2_frame_parse`는 아홉 바이트에서 실패할 수 없다. 그 바이트들이 있어도 되는 프레임을 말하는지는 검사가 답한다. 여러분이 알린 크기보다 긴 프레임은 무엇이라고 주장하든 연결 오류다.
- **흐름 제어는 페이로드 전체를 센다.** `proven_h2_frame_content`는 패딩된 DATA 프레임의 내용이 어디 있는지 알려 준다. 윈도에서는 패딩까지 차감된다.
- **헤더 블록은 여러 프레임에 걸칠 수 있고**, 그 사이에 다른 것은 올 수 없다: `END_HEADERS`가 없는 HEADERS 뒤에는 같은 스트림의 CONTINUATION 프레임이, 그 플래그가 나올 때까지 이어진다. 모으는 양에 한도를 둘 것 - 헤더 목록 한도의 두 배면 정직한 블록에는 충분하다 - 그러지 않으면 상대가 보내고 싶은 만큼을 쥐게 된다.
- **모르는 것은 틀린 것이 아니다.** 이 라이브러리가 모르는 프레임 종류나 설정은, 프로토콜이 요구하는 대로 검사를 통과한다. 건너뛴다.
- **설정에는 범위가 있고**, 벗어난 값은 연결 오류다: `ENABLE_PUSH`와 `ENABLE_CONNECT_PROTOCOL`은 0 또는 1, 초기 윈도는 많아야 2^31-1, 프레임 크기는 16,384에서 2^24-1까지. `ENABLE_CONNECT_PROTOCOL`은 한 번 1이 되면 0이 될 수 없다.

4. 메시지로서의 헤더 목록

```
proven_err_t proven_h2_request_head(const proven_http_header_t *headers, proven_size_t count,
proven_h2_request_head_t *out);
proven_err_t proven_h2_response_head(const proven_http_header_t *headers, proven_size_t count, proven_u16
*status,
                                     const proven_http_header_t **fields, proven_size_t *field_count);
proven_err_t proven_h2_trailers_check(const proven_http_header_t *headers, proven_size_t count);
```

디코딩된 헤더 목록은 바이트 문자열의 목록이다. 그것이 요청인가는 별개의 질문이고, 그 답은 HTTP/2가 HTTP/1.1과 만나는 곳에서 가장 중요하다: 줄바꿈이 든 값이 텍스트로 전달되면 헤더 두 줄이 된다 - 받는 서버가 도착하는 것을 본 적 없는 요청이다. 그래서 다음과 같은 목록은 *잘못된 형식*이며, 고치거나 전달하지 말고 거절해야 한다:

규칙	까닭
필드 이름에 대문자, 공백, 제어 바이트, `0x7e`보다 큰 바이트가 있거나, 맨 앞이 아닌 곳에 콜론이 있다	이름은 소문자 토큰이다. 콜론은 유사 헤더 필드를 시작할 뿐이다
값에 NUL, CR, LF가 있거나, 값이 공백이나 탭으로 시작하거나 끝난다	이것들은 HTTP/1.1의 구분자다
유사 헤더 필드가 보통 필드 뒤에 오거나, 두 번 나오거나, 아무도 정의하지 않은 것이다	그것들은 메시지의 첫 줄이다: 한 번, 맨 먼저
요청에 `:method`, `:scheme`, `:path`가 없다. `http`나 `https`인데 `:path`가 비었다. 요청에 `:status`가 있다	첫 줄은 온전해야 한다. (`CONNECT`에는 `:method`와 `:authority`만 있다)
응답에 `:status`가 없거나 세 자리 숫자가 아니다	위와 같다
`connection`, `proxy-connection`, `keep-alive`, `transfer-encoding`, `upgrade`가 있다. `te`가 `trailers`가 아닌 것을 말한다	이것들은 HTTP/1.1 연결을 기술한다. 여기서는 남의 연결을 기술하려는 시도다
트레일러에 유사 헤더 필드가 있다	트레일러는 보통 필드다

`proven_h2_request_head`는 이 규칙들을 적용하고 부분들을 돌려준다: 메서드, 스킴, authority, 경로, 그리고 뒤따르는 보통 필드들 - 그리고 확장 CONNECT(RFC 8441)이면 `protocol`: `:protocol`은 CONNECT에만, 그리고 그때는 스킴, authority, 경로가 모두 있어야 한다. 그것이 허락되는지는 연결이 정한다: 설정에 `enable_connect_protocol`이 있는 서버는 `SETTINGS_ENABLE_CONNECT_PROTOCOL`을 말하고 그것을 받는다. 다른 서버는 그런 스트림을 `PROTOCOL_ERROR`로 거절하고, 받아들인 것의 자기 쪽은 리셋이 아니라 END_STREAM으로 끝낸다 - 터널은 양쪽에서 끝난다. 잘못된 형식의 메시지는 `PROTOCOL_ERROR` 종류의 스트림 오류다: 응답을 아직 보낼 수 있으면 `400`으로 답하고, 아니면 스트림을 리셋한다.

5. 연결

```
proven_err_t proven_h2_conn_create(const proven_h2_config_t *config, proven_h2_conn_t **out);
void proven_h2_conn_destroy(proven_h2_conn_t *conn);

proven_err_t proven_h2_conn_feed(proven_h2_conn_t *conn, proven_mem_view_t in, proven_size_t *consumed,
proven_h2_event_t *event);
proven_mem_view_t proven_h2_conn_pending_output(const proven_h2_conn_t *conn);
void proven_h2_conn_output_sent(proven_h2_conn_t *conn, proven_size_t n);
```

`proven_h2_conn_t`는 어느 쪽 끝이든 될 수 있는 HTTP/2 연결 하나다. 2절부터 4절까지가 여러분에게 남겨 두었던 것을 모두 쥐고 있다: 양쪽의 설정, 열려 있는 스트림, 양방향의 윈도, 압축 테이블 둘, 그리고 프레임마다 프로토콜이 요구하는 답. **입출력을 하지 않고 시계도 읽지 않는다.** 모양은 14장의 TLS 엔진과 같다:

```
상대가 보낸 바이트  -->  proven_h2_conn_feed  -->  사건 하나: 요청, 본문의 한 조각, 리셋, ...
여러분의 요청, 응답, 본문  -->  proven_h2_conn_submit_* / _send_data
proven_h2_conn_pending_output  -->  상대에게 갈 바이트  -->  proven_h2_conn_output_sent
```

소켓, TLS, 모든 타임아웃은 호출자의 것이다. 그래서 객체 하나가 블로킹 프로그램에도, 이벤트 루프에도, 네트워크가 전혀 없는 시험에도 쓰인다 - 아래 예제는 서로 버퍼를 건네는 연결 둘이고, 6절은 소켓 뒤에 놓인 같은 객체다.

사건

`proven_h2_conn_feed`는 알릴 사건이 **하나** 생길 때까지 `in`의 바이트를 쓰고, 몇 바이트를 썼는지 알려 준다. 나머지를 가지고 다시 호출한다. `PROVEN_H2_EVENT_NONE`은 모두 썼다는 뜻이다.

사건	뜻	필드
`PROVEN_H2_EVENT_HEADERS`	서버: 요청이 시작되었다. 이미 4절의 규칙으로 검사되고 나뉘어 있다. 클라이언트: 응답의 머리(`status`가 200 미만이면 중간 응답이고, 더 온다)	`stream_id`, `request` 또는 `status`와 `fields`, `end_stream`, `user`
`PROVEN_H2_EVENT_DATA`	본문의 한 조각. `in` 안을 가리키는 뷰다	`stream_id`, `data`, `end_stream`, `user`
`PROVEN_H2_EVENT_TRAILERS`	트레일러 필드: 상대 쪽의 스트림은 끝났다	`stream_id`, `fields`, `user`
`PROVEN_H2_EVENT_STREAM_RESET`	스트림이 끝나기 전에 끝났다: 상대가 리셋했거나, 그 스트림에서 규칙을 어겼거나, 처리하지 않은 채 작별했다	`stream_id`, `code`, `user`; 클라이언트에서는 그래도 응답이 완전할 때 `end_stream`(아래); 상대의 헤더 목록이 `max_header_list_bytes`를 넘어 이 쪽이 리셋했을 때 `too_large`
`PROVEN_H2_EVENT_STREAM_WRITABLE`	잘렸던 전송을 이어 갈 수 있다	`stream_id`, `user`
`PROVEN_H2_EVENT_PONG`	`proven_h2_conn_ping`의 답	`data`: 그 여덟 바이트
`PROVEN_H2_EVENT_GOAWAY`	상대가 새 스트림을 받지 않는다	`code`, `last_stream_id`, `data` (함께 보낸 텍스트: 처음 256바이트, 로그용)
`PROVEN_H2_EVENT_CLOSED`	연결이 끝났고, 모든 스트림도 함께 끝났다. 남은 출력을 보내고 닫는다	`code`

이 표에서 세 가지는 다시 볼 만하다.

본문은 복사되지 않는다. DATA 사건은 여러분이 넣어 준 버퍼 안을 가리키므로, 어떤 크기의 본문이든 여러분의 읽기 한 번보다 크지 않은 조각으로 지나간다. 세 번의 읽기에 걸쳐 도착한 프레임은 사건 셋이고, `end_stream`은 마지막 것에 붙는다.

상대의 잘못은 오류가 아니라 사건이다. 상대가 프로토콜을 어겨도 `proven_h2_conn_feed`는 `PROVEN_OK`를 돌려준다: 연결은 코드를 담은 GOAWAY 프레임을 쌓아 두고 `PROVEN_H2_EVENT_CLOSED`를 알린다. 스트림 하나의 잘못은 그 스트림의 `PROVEN_H2_EVENT_STREAM_RESET`일 뿐이다. 상대가 한 일 때문에 feed가 돌려주는 오류는 하나, `PROVEN_ERR_INVALID_FORMAT`이다: 서버인데 클라이언트가 24바이트 서문으로 시작하지 않았을 때. **그것은 HTTP/2가 아니다.** 상대에게 아무 말도 하지 않으며, 달라지는 첫 바이트에서 알린다 - 그래서 그 바이트들은 여전히 여러분의 것이고 HTTP/1.1 파서에 넘길 수 있다.

입력이 필요 없는 사건이 있다. 스트림이 다시 쓸 수 있게 되는 것은 무언가 도착했을 때가 아니라 출력이 나갔을 때다. 그러니 `proven_h2_conn_output_sent` 뒤에는 `proven_h2_conn_feed`를 빈 뷰로 한 번 더 불러 생긴 사건을 받는다.

보내기

```
proven_err_t proven_h2_conn_submit_request(proven_h2_conn_t *conn, const proven_http_header_t *headers,
proven_size_t count,
                                           bool end_stream, proven_u32 *stream_id);
proven_err_t proven_h2_conn_submit_response(proven_h2_conn_t *conn, proven_u32 stream_id, const
```

```
proven_http_header_t *headers, proven_size_t count,
                                        bool end_stream);
proven_result_size_t proven_h2_conn_send_data(proven_h2_conn_t *conn, proven_u32 stream_id, proven_mem_view_t
data, bool end_stream);
proven_err_t proven_h2_conn_send_trailers(proven_h2_conn_t *conn, proven_u32 stream_id, const
proven_http_header_t *headers, proven_size_t count);
proven_err_t proven_h2_conn_reset(proven_h2_conn_t *conn, proven_u32 stream_id, proven_h2_error_t code);
```

클라이언트는 의사 헤더 필드 넷으로 시작하는 헤더 목록으로 스트림을 열고, 서버는 :status로 시작하는 목록으로 스트림에 답한다. 둘 다 한 바이트라도 쌓이기 전에 4절의 규칙으로 검사되므로, 이 라이브러리는 자기가 거부할 것을 보내지 않는다.

proven_h2_conn_send_data는 **내민 것보다 적게 가져갈 수 있고** 전혀 가져가지 않을 수도 있으며, 그것은 오류가 아니다. 가져가는 양은 세 가지 가운데 가장 작은 것이다: 스트림의 윈도, 연결의 윈도, max_buffered_output 아래에 남은 자리. 나머지는 쥐고 있다가 PROVEN_H2_EVENT_STREAM_WRITABLE이 그 스트림을 가리키면 다시 내민다. end_stream은 내민 것이 모두 받아들여졌을 때 효력이 생긴다.

여러 스트림이 연결의 윈도를 기다리고 있으면 **차례대로** 알린다. 알림을 받은 스트림은 윈도에 있는 만큼을 가져가고, 자리가 다시 생기면 그 다음 스트림부터 찾는다. 그래서 긴 본문 하나가 같은 연결의 다른 스트림을 붙잡아 두지 않는다. (v0.37.1 전에는 그럴 수 있었다: 찾기가 매번 같은 스트림에서 시작했다.)

받는 쪽의 흐름 제어

```
void proven_h2_conn_consumed(proven_h2_conn_t *conn, proven_u32 stream_id, proven_size_t n);
```

상대는 이쪽이 받겠다고 한 만큼만 보낼 수 있다: 스트림마다 initial_window 바이트, 연결 전체로도 그만큼. **자리는 바이트가 도착했을 때가 아니라 처리했다고 말할 때 돌아온다** - proven_h2_conn_consumed로. 뒤처져서 이 호출을 멈춘 처리기는 많아야 윈도 하나만큼을 버퍼에 둔 채 상대를 멈춘다. 이것이 HTTP/2의 역압(backpressure)이고, 이 호출이 말 그대로일 때만 통한다: 바이트를 건네받았을 때가 아니라 가야 할 곳으로 보냈을 때 부른다.

아무도 소비하지 않는 것은 연결이 스스로 돌려준다: 패딩, 리셋된 스트림으로 온 데이터, 그리고 스트림이 끝날 때 아직 소비되지 않았던 것.

상대가 연결에 쥐게 하거나 하게 할 수 있는 것

모든 양에는 설정이 있고, 모든 설정에는 기본값이 있다:

설정	기본값	경계에서
max_concurrent_streams	100	넘는 스트림은 거절되고(REFUSED_STREAM) 알려지지 않는다
initial_window	65,535	스트림의 윈도를 넘는 DATA는 그 스트림의 FLOW_CONTROL_ERROR, 연결의 윈도를 넘으면 연결의 오류
max_frame_size	16,384	더 큰 프레임은 연결의 FRAME_SIZE_ERROR
max_header_list_bytes	16 KiB	더 큰 목록은 그 스트림의 PROTOCOL_ERROR. 이 값의 두 배를 넘는 헤더 블록은 연결을 끝낸다
header_table_bytes	4,096	방향마다의 HPACK 테이블. set_header_table과 0이면 없음
max_buffered_output	64 KiB	얼마가 나갈 때까지 본문을 더 받지 않고 submit 호출은 PROVEN_ERR_BUSY로 답한다. 따로 센다: 읽지 않는 상대에게 프로토콜이 빚진 답이 이만큼을 넘으면 연결이 끝난다
max_reset_debt	100	리셋된 스트림 - 상대가 했거나 상대의 잘못 때문에 - 이 제대로 끝난 스트림보다 이만큼 많은 뒤: 하나 더 오면 ENHANCE_YOUR_CALM으로 연결이 끝난다
enable_connect_protocol	false	서버: 확장 CONNECT(RFC 8441)를 받고 첫 설정에서 그렇다고 말한다. 이벤트 구동 서버는 http2_websocket(15장)으로 이것을 정한다

마지막 줄은, 서버가 일을 시작하게 해 놓고 버리게 하려고 스트림을 열자마자 리셋하기를 되도록 빨리 되풀이하는 클라이언트에 대한 답이다. 시계는 쓰지 않는다: 리셋 하나가 하나를 더하고, 제대로 끝난 스트림 하나가 하나를 덜며, 시작한 것을 대체로 끝내는 상대는 한도 근처에도 가지 않는다.

막 열린 연결은 많아야 400바이트인 블록 하나를 쥔다. 요청을 실어 나른 뒤 다시 한가해진 연결은 그것과, 마지막 스트림들이 어떻게 끝났는지의 기록과, 압축 테이블 둘을 쥔다. **테이블은 쓰이는 만큼 자란다**: 작은 요청 백 개 뒤에 3 KiB 미만, 둘 다 기본 크기 4,096바이트로 가득 찼을 때 약 12.5 KiB, 테이블 없이는 1,280바이트 미만. 스트림이 열려 있는 동안에는 스트림마다 48바이트와, 헤더 목록 하나를 읽는 데 쓰는 버퍼 - `max_header_list_bytes`의 다섯 배, 기본값에서 80 KiB - 를 더 쥔다. **이 마지막 것은 연결마다 가질 필요가 없다**: 설정에 `proven_h2_header_buffer_size` 바이트짜리 `header_buffer`를 주면 한 스레드가 모는 모든 연결이 같은 메모리에 읽을 수 있다 - 거기 읽힌 것이 곧 사건이 알리는 것이고, 사건은 다음 호출까지만 유효하다. 6절의 서버가 이렇게 한다.

연결 뒤에 바꾸는 설정

```
proven_err_t proven_h2_conn_set_max_streams(proven_h2_conn_t *conn, proven_u32 n);
proven_err_t proven_h2_conn_set_initial_window(proven_h2_conn_t *conn, proven_u32 n);
```

설정 가운데 둘은 연결이 도는 동안 바꿀 수 있다(v0.79.0). 하나는 상대가 한꺼번에 열 수 있는 스트림 수이고, 다른 하나는 스트림마다 처음 주는 창이다. 바쁜 서버는 둘을 줄였다가 나중에 다시 올린다. 호출할 때마다 SETTINGS 프레임이 하나 쌓인다. RFC 9113에 따르면 설정은 상대가 확인한 뒤부터 유효하고, 연결도 그대로 따른다.

- **만들 때 준 값이 상한이다.** 연결이 쥘 수 있는 메모리는 만들 때 그 값으로 정해진다. 그래서 값을 낮췄다가 다시 올릴 수는 있어도 넘을 수는 없다. 넘으면 `PROVEN_ERR_INVALID_ARG`이다.
- **확인 전에는 큰 쪽이 유효하다.** 상대는 변경을 읽기 전의 값으로 스트림을 열거나 본문을 보낼 수 있고, 연결은 그것을 받는다. 확인 뒤에는 새 수를 넘는 스트림을 `REFUSED_STREAM`으로 거절하고(클라이언트는 다시 보낼 수 있다), 스트림 창을 넘는 DATA는 그 스트림의 `FLOW_CONTROL_ERROR`이다.
- **열린 것은 자르지 않는다.** 이미 열린 스트림은 몇 개든 계속된다.
  - 창을 바꾸면 열린 스트림마다 창이 그 차이만큼 움직인다(RFC 9113 6.9.2).
  - 낮추면 어떤 스트림은 남은 창이 0보다 작아질 수 있다. 그 스트림은 충분히 소비될 때까지 기다린다.
  - 올리면 바로 유효하다.
  - 연결 전체의 창은 설정이 아니어서 바뀌지 않는다.
- **진행 중인 변경은 하나다.** 확인을 기다리는 동안 들어온 호출은 기억해 두었다가, 확인이 오면 마지막 값들을 프레임 하나로 보낸다. 몇 번을 불러도 프레임은 하나다.
- **클라이언트에서는** `max_streams`가 이쪽이 여는 스트림 수의 상한도 되고, 그 상한은 바로 유효하다.

아래 예제는 서버에서 둘을 낮추고, 클라이언트가 새 수를 지키는 것을 보인다.

작별

```
void proven_h2_conn_shutdown(proven_h2_conn_t *conn, proven_h2_error_t code);
proven_err_t proven_h2_conn_ping(proven_h2_conn_t *conn, const proven_byte_t opaque[8]);
bool proven_h2_conn_is_closed(const proven_h2_conn_t *conn);
```

`PROVEN_H2_NO_ERROR`로 부른 `proven_h2_conn_shutdown`은 GOAWAY 프레임을 쌓아 두고 열려 있는 스트림이 끝나게 둔다: 새 스트림은 받지도 시작하지도 않으며, 마지막 스트림이 끝나면 `PROVEN_H2_EVENT_CLOSED`가 따라온다. 상대의 GOAWAY도 같은 방식으로 처리되며, 클라이언트는 서버가 처리하지 않은 스트림이 어느 것인지 듣는다 - `PROVEN_H2_REFUSED_STREAM`이 담긴 `PROVEN_H2_EVENT_STREAM_RESET` - 그것들은 다른 연결에서 다시 보내도 안전하기 때문이다.

예제: 사이에 소켓이 없는, 연결의 양쪽 끝

```
#include <stdio.h>
#include <string.h>

/*
 * 소켓 없이 양쪽 끝을 모두 가진 HTTP/2 연결: 한쪽이 쌓아 둔 것을 마치 선을 건너온 것처럼
 * 다른 쪽에 넘겨준다.
```

```
 *
 * proven_h2_conn_t는 입출력을 하지 않는다. 상대가 보낸 바이트를 넣어 주면 사건을 한 번에
 * 하나씩 알려 주고, 보내 달라고 하면 프레임을 쌓아 두어 가져가게 한다. 실제 프로그램은
 * carry()가 있는 자리에 소켓(과 TLS)을 둔다.
 */

typedef struct {
    proven_h2_conn_t *conn;
    const char *name;
    /* 이쪽 끝이 알게 된 것. 아래의 검사에 쓴다. */
    int requests, responses, trailers, resets, writable, pongs, goaways, closed;
    proven_size_t body_bytes;
    proven_u32 last_stream, reset_code, close_code;
    bool hold;                          /* DATA를 소비했다고 알리지 않는다 */
} ex_end_t;

/* `in`의 바이트에 든 사건을 모두 받는다. 한 번 호출에 사건 하나다: 사건도 없고 남은 것도 없을
 * 때까지 나머지를 가지고 다시 호출한다. */
static bool take_events(ex_end_t *e, proven_mem_view_t in) {
    proven_size_t pos = 0;
    for (;;) {
        proven_size_t used = 0;
        proven_h2_event_t ev;
        if (proven_h2_conn_feed(e->conn, (proven_mem_view_t){ in.ptr + pos, in.size - pos }, &used, &ev) !=
PROVEN_OK) return false;
        pos += used;
        switch (ev.kind) {
            case PROVEN_H2_EVENT_NONE:
                if (pos == in.size) return true;          /* 다 썼다: 더 읽어 온다 */
                break;
            case PROVEN_H2_EVENT_HEADERS:
                e->last_stream = ev.stream_id;
                if (ev.status == 0) {                     /* 서버: 이미 검사되고 나뉜 요청 */
                    e->requests++;
                    printf("%s: stream %u asks %.*s %.*s\n", e->name, (unsigned)ev.stream_id,
(int)ev.request.method.size, (const char *)ev.request.method.ptr,
                           (int)ev.request.path.size, (const char *)ev.request.path.ptr);
                } else {                                  /* 클라이언트: 응답의 머리 */
                    e->responses++;
                    printf("%s: stream %u answers %u%s\n", e->name, (unsigned)ev.stream_id,
(unsigned)ev.status, ev.user ? " (the stream we marked)" : "");
                }
                break;
            case PROVEN_H2_EVENT_DATA:
                /* ev.data는 `in` 안을 가리킨다: 복사된 것이 없다. 소비했다고 알려야 상대가
                 * 더 보낼 수 있다. */
                e->body_bytes += ev.data.size;
                if (!e->hold) proven_h2_conn_consumed(e->conn, ev.stream_id, ev.data.size);
                break;
            case PROVEN_H2_EVENT_TRAILERS: e->trailers++; break;
            case PROVEN_H2_EVENT_STREAM_RESET: e->resets++; e->reset_code = ev.code; break;
            case PROVEN_H2_EVENT_STREAM_WRITABLE: e->writable++; break;
            case PROVEN_H2_EVENT_PONG: e->pongs++; break;
            case PROVEN_H2_EVENT_GOAWAY: e->goaways++; break;
            case PROVEN_H2_EVENT_CLOSED: e->closed++; e->close_code = ev.code; return true;
        }
    }
}

/* 선: `from`이 쌓아 둔 것이 `to`로 간다. 그런 다음 `from`에게 빈 입력으로 한 번 더 묻는다 -
```

```
 * 보낸 덕분에 생긴 사건, 곧 다시 보낼 수 있게 된 스트림이 있는지. */
static bool carry(ex_end_t *from, ex_end_t *to) {
    static proven_byte_t wire[200000];
    proven_mem_view_t out = proven_h2_conn_pending_output(from->conn);
    if (out.size > sizeof wire) return false;
    if (out.size > 0) memcpy(wire, out.ptr, out.size);
    proven_size_t n = out.size;
    proven_h2_conn_output_sent(from->conn, n);            /* 소켓이라면 몇 바이트를 받았는지 알려 줄 것이다
*/
    if (!to->closed && !take_events(to, (proven_mem_view_t){ wire, n })) return false;
    return from->closed || take_events(from, (proven_mem_view_t){ wire, 0 });
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_end_t client = { .name = "client" }, server = { .name = "server" };

    /* 0은 모든 한계에서 기본값을 뜻한다. 꼭 말해야 하는 것은 `server` 하나다. */
    proven_h2_config_t ccfg = { .alloc = heap };
    proven_h2_config_t scfg = { .alloc = heap, .server = true, .max_concurrent_streams = 10 };
    EXAMPLE_REQUIRE(proven_h2_conn_create(&ccfg, &client.conn) == PROVEN_OK && proven_h2_conn_create(&scfg,
&server.conn) == PROVEN_OK, "two ends");

    /* 여는 말은 이미 쌓여 있다: 클라이언트의 서문과 양쪽의 설정. */
    EXAMPLE_REQUIRE(carry(&client, &server) && carry(&server, &client) && carry(&client, &server), "settings
cross and are acknowledged");
    /* 첫 요청 전에 이것을 기다리는 클라이언트는 서버가 받는 것보다 많은 스트림을 요청하는 일이 없다. */
    EXAMPLE_REQUIRE(proven_h2_conn_has_peer_settings(client.conn) &&
proven_h2_conn_has_peer_settings(server.conn), "each end has the other's settings");

    // ---- 본문이 있는 요청과 그 응답 ------------------------------------------------------------
    proven_http_header_t request[] = {
        { PROVEN_LIT(":method"), PROVEN_LIT("POST") }, { PROVEN_LIT(":scheme"), PROVEN_LIT("https") },
        { PROVEN_LIT(":authority"), PROVEN_LIT("www.example.com") }, { PROVEN_LIT(":path"), PROVEN_LIT("/
upload") },
        { PROVEN_LIT("content-type"), PROVEN_LIT("text/plain") },
    };
    proven_u32 stream = 0;
    int mark = 0;
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, false, &stream) == PROVEN_OK &&
stream == 1, "a client's first stream is 1");
    proven_h2_conn_stream_set_user(client.conn, stream, &mark);       /* 이 스트림의 사건에 실려 온다 */
    EXAMPLE_REQUIRE(proven_h2_conn_stream_user(client.conn, stream) == &mark, "and it can be asked for");
    proven_result_size_t sent = proven_h2_conn_send_data(client.conn, stream,
proven_mem_view_from_u8(PROVEN_LIT("hello, ")), false);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == 7, "seven bytes of body");
    sent = proven_h2_conn_send_data(client.conn, stream, proven_mem_view_from_u8(PROVEN_LIT("world")),
true);     /* true: 본문의 끝 */
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == 5 && carry(&client, &server), "and five more, the
last");
    EXAMPLE_REQUIRE(server.requests == 1 && server.body_bytes == 12 &&
proven_h2_conn_stream_count(server.conn) == 1, "the server has the request and its twelve bytes");

    proven_http_header_t response[] = { { PROVEN_LIT(":status"), PROVEN_LIT("200") }, { PROVEN_LIT("content-
type"), PROVEN_LIT("text/plain") } };
    proven_http_header_t trailers[] = { { PROVEN_LIT("x-checksum"), PROVEN_LIT("9f86d081") } };
    EXAMPLE_REQUIRE(proven_h2_conn_submit_response(server.conn, stream, response, 2, false) == PROVEN_OK, "a
response head");
    sent = proven_h2_conn_send_data(server.conn, stream, proven_mem_view_from_u8(PROVEN_LIT("thanks")),
false);
```

```
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == 6 && proven_h2_conn_send_trailers(server.conn,
stream, trailers, 1) == PROVEN_OK, "a body, and trailers to end it");
    EXAMPLE_REQUIRE(carry(&server, &client), "back to the client");
    EXAMPLE_REQUIRE(client.responses == 1 && client.body_bytes == 6 && client.trailers == 1, "the client has
all three parts");
    EXAMPLE_REQUIRE(proven_h2_conn_stream_count(client.conn) == 0 && proven_h2_conn_stream_count(server.conn)
== 0, "and the stream is over at both ends");

    // ---- 흐름 제어: 뒤처진 수신자는 송신자를 멈춘다 ------------------------------------------
    static proven_byte_t big[100000];
    server.hold = true;                                  /* 서버의 처리기가 바쁘다: 아무것도 소비하지 않는다
*/
    server.body_bytes = 0;
    /* 어느 중간 지점도 기억하면 안 되는 필드는 바로 다음 목록 하나에 대해 표시한다: 여기서는 마지막 필드를,
키인 셈 치고. */
    static const bool keep_out[5] = { false, false, false, false, true };
    proven_h2_conn_next_sensitive(client.conn, keep_out);
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, false, &stream) == PROVEN_OK &&
stream == 3 && carry(&client, &server), "a second request");
    sent = proven_h2_conn_send_data(client.conn, stream, (proven_mem_view_t){ big, sizeof big }, true);
    const proven_size_t taken = sent.value;
    printf("client: offered %u bytes, %u were taken\n", (unsigned)sizeof big, (unsigned)taken);
    /* 연결의 창은 기본 65,535바이트이고, 첫 요청의 12바이트는 아직 돌려받지 못했다: 창은
     * 바이트마다가 아니라 묶어서 다시 열린다. */
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && taken == 65535 - 12, "what the windows allow and no more: taking
less than was offered is not an error");
    EXAMPLE_REQUIRE(proven_h2_conn_send_room(client.conn, stream) == 0 && carry(&client, &server) &&
carry(&server, &client) && client.writable == 0, "nothing more may go until the server says so");
    proven_h2_conn_consumed(server.conn, stream, taken);   /* 처리기가 따라잡았다 */
    server.hold = false;
    EXAMPLE_REQUIRE(carry(&server, &client) && client.writable == 1, "and now the client is told:
PROVEN_H2_EVENT_STREAM_WRITABLE");
    sent = proven_h2_conn_send_data(client.conn, stream, (proven_mem_view_t){ big + taken, sizeof big -
taken }, true);
    EXAMPLE_REQUIRE(sent.err == PROVEN_OK && sent.value == sizeof big - taken && carry(&client, &server) &&
server.body_bytes == sizeof big, "the rest follows");

    /* 서버는 이 요청에 답하는 대신 포기한다. */
    EXAMPLE_REQUIRE(proven_h2_conn_reset(server.conn, stream, PROVEN_H2_CANCEL) == PROVEN_OK && carry(&server,
&client), "a reset");
    EXAMPLE_REQUIRE(client.resets == 1 && client.reset_code == PROVEN_H2_CANCEL &&
proven_h2_conn_stream_count(client.conn) == 0, "which the client hears of, with the code");

    // ---- 거기 누구 있는가? ------------------------------------------------------------
    static const proven_byte_t opaque[8] = { 'p', 'r', 'o', 'v', 'e', 'n', '!', '!' };
    EXAMPLE_REQUIRE(proven_h2_conn_ping(client.conn, opaque) == PROVEN_OK && carry(&client, &server) &&
carry(&server, &client) && client.pongs == 1, "a ping is answered by the connection itself");

    // ---- 연결 뒤에 바꾸는 설정 --------------------------------------------------------
    /* 바쁜 서버가 동시 스트림 수와 창을 줄인다. SETTINGS 프레임 하나이고, 클라이언트가 확인한
     * 뒤부터 유효하다. 만들 때 준 값이 상한이고, 이미 열린 스트림은 건드리지 않는다. */
    EXAMPLE_REQUIRE(proven_h2_conn_set_max_streams(server.conn, 1) == PROVEN_OK &&
proven_h2_conn_set_initial_window(server.conn, 16384) == PROVEN_OK, "the server lowers both");
    EXAMPLE_REQUIRE(carry(&server, &client) && carry(&client, &server), "the change crosses, and is
acknowledged");
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, true, &stream) == PROVEN_OK &&
carry(&client, &server), "one request");
    proven_u32 second = 0;
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, true, &second) == PROVEN_ERR_BUSY,
"and not a second at the same time: the client holds itself to the new number");
```

```
    EXAMPLE_REQUIRE(proven_h2_conn_set_max_streams(server.conn, 11) == PROVEN_ERR_INVALID_ARG, "above the
number given at creation: refused");
    EXAMPLE_REQUIRE(proven_h2_conn_reset(server.conn, stream, PROVEN_H2_CANCEL) == PROVEN_OK && carry(&server,
&client), "the server lets that one go");

    // ---- 규칙을 어기는 상대 ------------------------------------------------------------------
    /* 아무것도 더하지 않는 WINDOW_UPDATE는 프로토콜 오류다. 연결은 상대에게 그렇게 말하고
     * (GOAWAY), 끝났다고 알리고, 더는 아무것도 받지 않는다. */
    ex_end_t strict = { .name = "strict" };
    EXAMPLE_REQUIRE(proven_h2_conn_create(&scfg, &strict.conn) == PROVEN_OK, "another server");
    proven_byte_t bad[128];
    proven_size_t len = PROVEN_H2_PREFACE_SIZE;
    memcpy(bad, PROVEN_H2_PREFACE, PROVEN_H2_PREFACE_SIZE);
    EXAMPLE_REQUIRE(proven_h2_write_settings((proven_mem_mut_t){ bad, sizeof bad }, &len, NULL, 0) ==
PROVEN_OK, "a client that opens properly");
    static const proven_byte_t zero_increment[13] = { 0, 0, 4, PROVEN_H2_WINDOW_UPDATE, 0, 0, 0, 0, 0, 0, 0,
0, 0 };
    memcpy(bad + len, zero_increment, sizeof zero_increment);
    len += sizeof zero_increment;
    EXAMPLE_REQUIRE(take_events(&strict, (proven_mem_view_t){ bad, len }), "the feed itself succeeds: a peer's
fault is an event");
    printf("strict: closed with code %u\n", (unsigned)strict.close_code);
    EXAMPLE_REQUIRE(strict.closed == 1 && strict.close_code == PROVEN_H2_PROTOCOL_ERROR &&
proven_h2_conn_is_closed(strict.conn), "PROTOCOL_ERROR, and the connection is over");
    EXAMPLE_REQUIRE(proven_h2_conn_pending_output(strict.conn).size > 0, "with a GOAWAY still to be sent
before the socket is closed");
    proven_h2_conn_destroy(strict.conn);

    /* 그리고 HTTP/2가 전혀 아닌 바이트는 그 첫 바이트에서 호출의 오류가 된다. */
    EXAMPLE_REQUIRE(proven_h2_conn_create(&scfg, &strict.conn) == PROVEN_OK, "one more");
    proven_size_t used = 0;
    proven_h2_event_t ev;
    EXAMPLE_REQUIRE(proven_h2_conn_feed(strict.conn, proven_mem_view_from_u8(PROVEN_LIT("GET /
HTTP/1.1\r\n\r\n")), &used, &ev) == PROVEN_ERR_INVALID_FORMAT && used == 0,
                    "HTTP/1.1 is PROVEN_ERR_INVALID_FORMAT with nothing taken: the bytes are yours to give to
something else");
    proven_h2_conn_destroy(strict.conn);

    // ---- 작별 ------------------------------------------------------------------------------
    proven_h2_conn_shutdown(server.conn, PROVEN_H2_NO_ERROR);          /* 새 스트림은 없다. 열려 있던 것은 먼
저 끝난다 */
    EXAMPLE_REQUIRE(carry(&server, &client) && carry(&client, &server), "the goodbye crosses");
    EXAMPLE_REQUIRE(client.goaways == 1 && client.closed == 1 && server.closed == 1 && client.close_code ==
PROVEN_H2_NO_ERROR, "with no stream open, both ends report the end at once");
    EXAMPLE_REQUIRE(proven_h2_conn_submit_request(client.conn, request, 5, true, &stream) ==
PROVEN_ERR_INVALID_STATE, "and nothing more is begun");

    proven_h2_conn_destroy(client.conn);
    proven_h2_conn_destroy(server.conn);
    return EXAMPLE_OK();
}
```

참조

API	의도	반환
`proven_h2_conn_create(&config, &conn)`	연결을 만든다. 여는 말이 곧바로 쌓인다.	`proven_err_t`: `INVALID_ARG`(범위를 벗어난 설정), `NOMEM`.
`proven_h2_header_buffer_size(max_header_list_bytes)`	`config.header_buffer`를 직접 줄 때 필요한 크기.	`proven_size_t`.

proven_h2_conn_destroy(conn)	해제한다.	void.
proven_h2_conn_feed(conn, in, &consumed, &event)	상대의 바이트를 받아 사건 하나를 알린다.	proven_err_t: INVALID_FORMAT(서버: HTTP/2가 아님), 끝을 알린 뒤에는 INVALID_STATE, NOMEM.
proven_h2_conn_pending_output(conn)	보낼 바이트.	proven_mem_view_t.
proven_h2_conn_output_sent(conn, n)	몇 바이트가 나갔는지 알린다.	void.
proven_h2_conn_submit_request(conn, headers, count, end_stream, &stream_id)	클라이언트: 요청을 시작한다.	proven_err_t: INVALID_ARG, BUSY(스트림 한도, 또는 기다리는 출력), INVALID_STATE(작별 뒤), OUT_OF_BOUNDS(상대가 그만큼 큰 목록을 받지 않음), NOMEM.
proven_h2_conn_submit_response(conn, stream_id, headers, count, end_stream)	서버: 답한다.	proven_err_t: INVALID_ARG, NOT_FOUND, INVALID_STATE(이미 답함), BUSY, OUT_OF_BOUNDS, NOMEM.
proven_h2_conn_send_data(conn, stream_id, data, end_stream)	본문 바이트를 보낸다: 갈 수 있는 만큼.	proven_result_size_t: 가져간 바이트 수. NOT_FOUND, INVALID_STATE.
proven_h2_conn_send_trailers(conn, stream_id, headers, count)	트레일러 필드로 이쪽을 끝낸다.	proven_err_t, 응답과 같다.
proven_h2_conn_next_sensitive(conn, sensitive)	연결에 다음으로 넘기는 헤더 목록의 필드 - 필드마다 플래그 하나 - 를 절대 인덱싱하지 않도록 표시한다. 표시는 그 목록 하나에만 쓰인다. 보내졌든 거절되었든.	void.
proven_h2_conn_reset(conn, stream_id, code)	스트림을 포기한다.	proven_err_t: NOT_FOUND.
proven_h2_conn_consumed(conn, stream_id, n)	DATA를 처리했다고 알린다.	void.
proven_h2_conn_ping(conn, opaque)	상대가 있는지 묻는다.	proven_err_t: 끝난 뒤에는 INVALID_STATE.
proven_h2_conn_set_max_streams(conn, n)	상대가 한꺼번에 열 수 있는 스트림 수를 바꾼다. 0부터 만들 때의 값까지. 상대가 확인하면 유효하고, 열린 스트림은 건드리지 않는다.	proven_err_t: INVALID_ARG(상한 초과), INVALID_STATE(작별 뒤), NOMEM.
proven_h2_conn_set_initial_window(conn, n)	스트림마다의 창을 바꾼다. 0부터 만들 때의 값까지. 열린 스트림은 차이만큼 움직인다. 올리면 바로, 낮추면 확인 뒤 유효하다.	proven_err_t, 위와 같다.
proven_h2_conn_shutdown(conn, code)	작별한다.	void.
proven_h2_conn_is_closed(conn), _stream_count(conn), _send_room(conn, stream_id)	끝났는가, 열린 스트림 수, 지금 갈 수 있는 본문 바이트 수.	bool, proven_size_t.
proven_h2_conn_stream_set_user(conn, stream_id, user), _stream_user(conn, stream_id)	열린 스트림에 붙이는 여러분의 포인터.	void, void *.

주의사항, 그리고 무엇이 잘못되는가

- **호출할 때마다, 쌓인 것을 보낸다.** 연결은 스스로 답한다 - 승인, 윈도 갱신, 리셋, 마지막 GOAWAY - 그리고 그 답은 여러분이 무엇을 요청했든 아니든 `proven_h2_conn_pending_output`에 있다.
- **보낸 뒤에는, 빈 입력을 넣는다.** 빈 뷰로 부른 `proven_h2_conn_feed`가 전송 덕분에 생긴 사건을 거둔다. 그러지 않고 소켓을 기다리는 드라이버는 자기를 기다리고 있는 상대를 기다리게 된다.
- **`proven_h2_conn_send_data`가 0을 돌려주는 것은 세상의 끝도, 스트림의 끝도 아니다.** "지금은 아니다"다. 예외는 `end_stream`과 함께 준 빈 `data`다: 그것은 언제나 나가고, 그때의 0은 "끝났다"는 뜻이다.
- **뷰는 다음 호출에서 죽는다.** 사건의 필드와 쌓인 출력은 연결을 받는 다음 호출까지만 유효하다. 간직할 것은 복사한다.
- **포인터는 사건과 함께 온다.** 스트림이 끝나면 `proven_h2_conn_stream_user`는 더는 그것을 찾지 못한다: 그 스트림의 마지막 사건이 `user`에 포인터를 싣고 오며, 가리키는 것을 해제할 곳은 거기다.
- **`PROVEN_H2_EVENT_CLOSED`는 모든 스트림을 하나하나 말하지 않고 끝낸다.** 열려 있던 것의 목록은 여러분의 것을 따라간다.
- **요청보다 먼저 끝난 응답은 클라이언트에게 그만 보내라고 말한다.** 서버 쪽은 RFC 9113이 정한 대로 `NO_ERROR`를 담은 `RST_STREAM`을 보내고 스트림은 끝난다. 클라이언트는 응답 뒤에 `PROVEN_H2_NO_ERROR`가 담긴 `PROVEN_H2_EVENT_STREAM_RESET`을 본다: 실패가 아니고, 이벤트의 `end_stream`이 참이다. **어떤 서버는 스트림을 끝내는 대신 `NO_ERROR`로 리셋한다**, 응답의 `content-length`가 선언한 바이트를 모두 보낸 뒤에. 길이가 빠진 것이 없음을 보여 주므로 그때도 `end_stream`이 참이다. 한 바이트라도 모자라거나, 길이가 없거나(본문이 없는 길이 - HEAD, 204, 304 - 도 마찬가지), 다른 코드라면 거짓이다: 응답이 잘렸을 수 있다.
- **연결 하나는 한 번에 한 스레드의 것이다.**
- **시계가 없으니 타임아웃도 없다.** 연결을 열고 아무 말도 하지 않는 상대, 윈도를 끝내 열지 않는 상대를 포기하는 것은 호출자의 일이다.

6. 이벤트 구동 서버의 HTTP/2

```
typedef struct {
    /* ... 15장이 설명하는 모든 것 ... */
    bool http2;
    proven_size_t max_concurrent_streams;
    proven_size_t http2_window_bytes;
} proven_http_event_server_config_t;

int proven_http_stream_protocol(const proven_http_stream_t *stream);
```

15장의 이벤트 구동 서버는 설정이 `http2`라고 하면 HTTP/2로 말한다. **처리기에게는 그 밖에 달라지는 것이 없다**: 같은 콜백 넷, 같은 `proven_http_stream_*` 호출, 같은 요청 타입. 연결 위의 교환 하나였던 것이 이제 여러 스트림 가운데 하나이고, 옮겨 주는 일은 서버가 한다:

처리기가 하는 일	HTTP/2에서는
`head->method`, `head->target`, 필드를 읽는다	의사 헤더 필드와 목록. 이미 4절의 규칙으로 검사되었다. `Host`가 있다 - 클라이언트가 `host` 필드를 보내지 않았으면 `:authority`로 만든다. `version_minor`는 1이다
`Content-Length`나 `Transfer-Encoding`으로 본문이 따라옴을 안다	똑같다: HTTP/2는 둘 다 없이 본문을 보낼 수 있고, 그런 요청에는 `Transfer-Encoding: chunked`를 붙여 준다. 뜻은 그대로다 - 길이를 알리지 않은 본문
`proven_http_stream_respond`, `_begin`, `_write`, `_end`	응답 머리, 클라이언트의 윈도와 출력 한계가 허락하는 만큼의 DATA 프레임, 스트림의 끝. 필드 이름은 소문자로 나간다. `Keep-Alive`, `Proxy-Connection`, `Upgrade`는 보내지 않는다

`_write`에서 거절당하고 `on_writable`을 기다린다	HTTP/1.1과 꼭 같다 - 이유도 같은 둘인데 처리기는 둘을 구별할 수 없다: 서버가 `max_buffered_output`만큼 쥐고 있거나, 클라이언트가 자리를 내주지 않는다
`proven_http_stream_pause`	HTTP/1.1처럼, 재개할 때까지 `on_body`가 없다. 이미 도착한 것은 쥐고 있다가 - 많아야 윈도 하나, 클라이언트는 그것이 소비되었다는 말을 듣지 못해 더 보낼 수 없으므로 - 재개할 때 한 조각으로 전달한다
`proven_http_stream_compress`	그대로다: 압축기는 프레이밍 위에 있다
`proven_http_stream_abort`	스트림이 리셋된다. 연결과 그 밖의 스트림은 계속된다
`on_done(why)`를 받는다	전과 같이 `on_request`마다 한 번: 응답 전체가 넘겨졌으면 `PROVEN_OK`, 클라이언트가 스트림을 리셋했거나 연결이 끝났으면 `PROVEN_ERR_RESET`, `PROVEN_ERR_TIMEOUT`, 한도를 넘는 본문은 `PROVEN_ERR_OUT_OF_BOUNDS`

`proven_http_stream_protocol`은 로그 한 줄을 위해 1이나 2(HTTP/3이면 3, 9절)를 답한다. 처리기가 물을 다른 이유는 드물다.

연결이 어느 프로토콜로 말하는가

- **TLS에서는 핸드셰이크가 말한다**(ALPN). `http2`가 켜져 있으면 TLS 설정이 `h2`를 - 그리고 더 새로운 것이 없는 클라이언트를 위해 그 뒤에 `http/1.1`을 - 내놓아야 하고, 꺼져 있으면 `h2`를 내놓아서는 안 된다: 핸드셰이크에서 합의하고는 말하지 않을 수 있는 조합을 서버는 생성 때 거부한다. 1.2보다 오래된 TLS에서, 또는 순방향 비밀성과 인증된 암호화가 없는 1.2 스위트에서 `h2`를 협상한 연결 - 14장의 구형 옵션을 켰을 때만 가능하다 - 은 RFC 9113이 요구하는 대로 `INADEQUATE_SECURITY`로 끝낸다.
- **TLS 설정에 `early_data`가 있으면 0-RTT**(14장 6절, v0.84.0). HTTP/1.1, HTTP/3과 같다: HEADERS가 클라이언트의 조기 데이터로 온 요청은 핸드셰이크가 끝나기 전에 핸들러에게 가고, `proven_http_stream_early`가 참이며, 바로 답할 수 있다. 본문 없는 GET, HEAD, OPTIONS만 거기까지 간다 - 그 밖의 것이 조기에 오면 서버가 그 스트림에 425로 답한다.
- **평문에서는 첫 바이트들이 말한다.** HTTP/2를 원한다는 것을 아는 클라이언트는 24바이트 서문으로 시작한다(curl의 `--http2-prior-knowledge`). 그 밖의 것은 HTTP/1.1이다. 서버는 달라지는 첫 바이트에서 정하므로, 서문과 첫 글자가 같은 `POST`는 둘째 바이트에서 HTTP/1.1이 된다. `Upgrade: h2c`는 제공하지 않는다: RFC 9113이 없앴다.

두 프로토콜은 한 포트에서, 한 처리기로, 동시에 서비스된다.

한계와 시간

설정	HTTP/2에서
`max_connections`	전과 같이 연결 수: HTTP/2 연결은 스트림을 몇 개 나르든 하나다
`max_concurrent_streams` (0: 100)	한 연결에서 진행 중인 요청 수. 클라이언트에게 그 수를 알리고, 넘는 스트림은 거절한다
`http2_window_bytes` (0: 65,535, 자람)	핸들러가 가져가기 전에 클라이언트가 한 스트림에, 그리고 연결에 보낼 수 있는 요청 본문 바이트: 멈춘 요청이 연결에 쥐게 할 수 있는 양. 0이면 창이 거기서 시작해 본문을 읽는 동안 자란다 - 왕복 두 번 안에 절반이 읽히는 동안 두 배씩, 스트림당 16 MiB, 연결당 24 MiB까지(v0.75.0, HTTP/3은 v0.71.0부

	터); 읽지 않는 본문은 키우지 않는다. 정한 값은 고정.
`max_head_bytes`, `max_headers`	HPACK이 세는 방식의 요청 헤더 목록 크기. `max_headers`보다 필드가 많으면 `431`
`max_body_bytes`	요청마다: 본문이 아직 도착하는 중에 `413`을 보내며, 그것이 클라이언트에게 그만 보내라고 말한다
`max_buffered_output`	읽지 않는 클라이언트를 위해 연결이 쥐는 양. **그 연결의 모든 스트림을 합쳐서**
`head_timeout_ms`	연결의 첫 바이트부터 첫 요청까지
`body_timeout_ms`	본문을 기다리는 요청 하나의 침묵: 그 스트림이 리셋되고, 연결과 다른 스트림은 계속된다
`http2_header_table_bytes` (0: 4,096), `http2_no_header_table`	방향마다의 헤더 압축 테이블: 한가한 연결이 쥐게 될 수 있는 양과, 되풀이되는 필드를 보내는 비용 사이의 선택
`write_timeout_ms`	쥐고 있는 출력이 하나도 받아들여지지 않음: 연결이 끝난다
`idle_timeout_ms`	열린 스트림이 없음: 서버가 작별하고(GOAWAY) 닫는다

```
proven_err_t proven_http_event_server_set_max_streams(proven_http_event_server_t *server, proven_size_t n);
proven_err_t proven_http_event_server_set_window(proven_http_event_server_t *server, proven_size_t n);
```

`max_concurrent_streams`는 서버가 도는 동안 바꿀 수 있다(v0.79.0).

- 루프의 스레드에서 부르고, 값은 1부터 설정값까지다. 설정값이 상한으로 남는다.
- HTTP/2 연결마다 클라이언트에게 새 수를 알린다(5절의 `proven_h2_conn_set_max_streams`). 나중에 생기는 연결은 새 수로 시작한다.
- 진행 중인 요청은 계속된다. 클라이언트가 확인하기 전에는 이전 수 안에서 요청을 더 시작할 수 있다.
- HTTP/3에는 따로 규칙이 있다(9절).
- 새 요청을 아예 받지 않으려면 서버를 종료한다. 0은 `PROVEN_ERR_INVALID_ARG`이다.

`proven_http_event_server_set_window`(v0.82.0)는 요청 본문에 같은 일을 한다.

- 루프의 스레드에서 부르면 `n`이, 클라이언트가 처리기보다 앞서 보낼 수 있는 양의 천장이 된다. 스트림당, 연결당, HTTP/2와 HTTP/3, 열린 연결과 앞으로 올 연결에.
- 돌려주는 room은 그 천장까지다 - 클라이언트가 아직 보낼 수 있는 양과 서버가 읽지 않고 쥔 양을 합쳐서. 자라는 창(`http2_window_bytes`를 0으로 둔 경우)은 거기서 멈추고, 올리면 다시 자란다. 설정한 창은 그대로 프로토콜이 주는 최대다.
- 이미 준 것은 남는다: 더 낮게 알린 HTTP/2 창은 클라이언트가 확인한 뒤에 효력이 있고, HTTP/3 credit은 되돌릴 수 없다. 그래서 낮춘 천장은 남은 것이 쓰이면서 드러난다.
- HTTP/2 연결마다 `min(n, 그 연결의 창)`을 초기 창으로 알린다(5절의 `proven_h2_conn_set_initial_window`). 지금 열린 HTTP/3 연결에서 클라이언트가 나중에 여는 스트림은 핸드셰이크에서 약속한 창을 여전히 쓸 수 있고, 나중에 생기는 연결에는 많아야 `n`을 약속한다.

`proven_http_event_server_stop_listening`이 우아하게 끝내는 길이다: HTTP/2 연결마다 GOAWAY를 보내고, 가진 스트림을 끝내고, 닫는다. `proven_http_event_server_destroy`는 모든 것을 즉시 끝내고, 열려 있던 스트림의 처리기는 모두 `on_done(PROVEN_ERR_RESET)`을 받는다.

예제: 포트 하나, 처리기 하나, 프로토콜 둘

```
#include <stdio.h>
#include <string.h>

/*
```

```
 * 서버 하나, 포트 하나, 프로토콜 둘: `http2`를 켠 이벤트 구동 HTTP 서버.
 *
 * 아래 처리기는 평범한 처리기다 - 같은 콜백, 같은 스트림 호출 - 묻지 않는 한 요청이 어느
 * 프로토콜로 왔는지 모른다. 그런 다음 HTTP/1.1 클라이언트와 HTTP/2 클라이언트가 같은 포트로
 * 말을 건다. 여기의 HTTP/2 클라이언트는 이 장 5절의 연결 엔진에 소켓을 통과시킨 것으로, 그것이
 * 무엇으로 이루어지는지 보여 준다: 마흔 줄이다. (연결을 간직하고 함께 쓰는 라이브러리 자체의
 * 클라이언트는 7절에 있다.)
 */

typedef struct { proven_loop_t *loop; int served, over_http2; } ex_app_t;

static void on_request(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    ex_app_t *app = ctx;
    /* 요청은 어느 쪽이든 같아 보인다: 메서드, 대상, 필드 - 그 가운데 Host도 있다. HTTP/2는
     * 그것을 `:authority`로 싣고, 서버는 익숙한 이름으로 넘겨준다. */
    /* TLS 설정에 `early_data`가 있으면 요청은 조기 데이터로 올 수 있고(14장 6절),
     * 공격자가 재전송한 사본일 수 있다. 이 답은 두 번 해도 괜찮다. 그렇지 않은 요청이라면 425로
     * 답하고, 클라이언트는 나중에 다시 보낸다. */
    if (proven_http_stream_early(stream) && head->method != PROVEN_HTTP_GET) {
        (void)proven_http_stream_respond(stream, 425, NULL, 0, (proven_mem_view_t){ 0 });
        return;
    }
    const int protocol = proven_http_stream_protocol(stream);
    char text[96];
    int n = snprintf(text, sizeof text, "%.*s over HTTP/%d\n", (int)head->target.size, (const char *)head-
>target.ptr, protocol);
    proven_http_header_t fields[] = { { PROVEN_LIT("Content-Type"), PROVEN_LIT("text/plain") } };
    if (proven_http_stream_respond(stream, 200, fields, 1, (proven_mem_view_t){ (const proven_byte_t *)text,
(proven_size_t)n }) != PROVEN_OK) return;
    if (protocol == 2) app->over_http2++;
}

static void on_done(void *ctx, proven_http_stream_t *stream, proven_err_t why) {
    (void)stream;
    if (why == PROVEN_OK) ((ex_app_t *)ctx)->served++;
}

static void run(void *arg) { (void)proven_loop_run(((ex_app_t *)arg)->loop); }

/* 작은 HTTP/2 클라이언트: 엔진, 소켓, 그리고 요청한 스트림이 모두 끝날 때까지 둘 사이로
 * 바이트를 옮기는 루프. 200으로 답을 받은 수를 돌려준다. */
static int ask_three(proven_u16 port, char answers[3][64]) {
    proven_net_conn_t sock;
    if (proven_net_connect(proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4, port),
proven_net_deadline_in(5000), &sock) != PROVEN_OK) return -1;

    /* 엔진은 스트림이 열려 있는 동안 자기가 할당한 메모리에 헤더 목록을 읽는다 - 또는 여러분의
     * 메모리에. 한 스레드의 여러 연결이 그것을 함께 쓸 수 있다. */
    static proven_u64 header_room[16384];
    proven_h2_config_t config = { .alloc = proven_heap_allocator(), .header_buffer = { (proven_byte_t
*)header_room, sizeof header_room } };
    proven_h2_conn_t *conn = NULL;
    if (proven_h2_header_buffer_size(0) > sizeof header_room || proven_h2_conn_create(&config, &conn) !=
PROVEN_OK) { (void)proven_net_close(&sock); return -1; }

    /* 한 연결에서 요청 셋을 한꺼번에: 스트림 1, 3, 5. */
    static const char *const paths[3] = { "/first", "/second", "/third" };
    for (int i = 0; i < 3; i++) {
        proven_http_header_t request[] = {
            { PROVEN_LIT(":method"), PROVEN_LIT("GET") }, { PROVEN_LIT(":scheme"), PROVEN_LIT("http") },
```

```
            { PROVEN_LIT(":authority"), PROVEN_LIT("localhost") }, { PROVEN_LIT(":path"),
proven_u8str_view_from_cstr(paths[i]) },
        };
        proven_u32 stream = 0;
        if (proven_h2_conn_submit_request(conn, request, 4, true, &stream) != PROVEN_OK)
{ proven_h2_conn_destroy(conn); (void)proven_net_close(&sock); return -1; }
    }

    int ended = 0, ok = 0;
    static proven_byte_t in[16384];
    while (ended < 3) {
        /* 엔진이 쌓아 둔 것 - 서문과 설정, 요청, 나중에는 승인 - 이 먼저 소켓으로 간다. */
        proven_mem_view_t out = proven_h2_conn_pending_output(conn);
        if (out.size > 0) {
            if (proven_net_write_all(&sock, out, proven_net_deadline_in(5000)).err != PROVEN_OK) break;
            proven_h2_conn_output_sent(conn, out.size);
        }
        proven_result_size_t got = proven_net_read(&sock, (proven_mem_mut_t){ in, sizeof in },
proven_net_deadline_in(5000));
        if (got.err != PROVEN_OK) break;
        proven_size_t pos = 0;
        for (;;) {
            proven_size_t used = 0;
            proven_h2_event_t ev;
            if (proven_h2_conn_feed(conn, (proven_mem_view_t){ in + pos, got.value - pos }, &used, &ev) !=
PROVEN_OK) { ended = 3; break; }
            pos += used;
            const proven_u32 k = ev.stream_id ? (ev.stream_id - 1) / 2 : 0;     /* 스트림 1, 3, 5 -> 0, 1, 2
*/
            if (ev.kind == PROVEN_H2_EVENT_HEADERS && ev.status == 200 && k < 3) ok++;
            if (ev.kind == PROVEN_H2_EVENT_DATA && k < 3) {
                snprintf(answers[k], 64, "%.*s", (int)(ev.data.size < 63 ? ev.data.size : 63), (const char
*)ev.data.ptr);
                proven_h2_conn_consumed(conn, ev.stream_id, ev.data.size);
            }
            if ((ev.kind == PROVEN_H2_EVENT_DATA || ev.kind == PROVEN_H2_EVENT_HEADERS) && ev.end_stream)
ended++;
            if (ev.kind == PROVEN_H2_EVENT_STREAM_RESET || ev.kind == PROVEN_H2_EVENT_CLOSED) { ended = 3;
break; }
            if (ev.kind == PROVEN_H2_EVENT_NONE && pos == got.value) break;
        }
    }
    proven_h2_conn_destroy(conn);
    (void)proven_net_close(&sock);
    return ok;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    ex_app_t app = { 0 };
    EXAMPLE_REQUIRE(proven_loop_create(heap, &app.loop) == PROVEN_OK, "a loop");

    /* `http2`가 전부다. 평문에서는 HTTP/2 서문으로 시작하는 클라이언트가 HTTP/2이고 나머지는
     * 모두 HTTP/1.1이다. TLS에서는 핸드셰이크가 정하며(ALPN), 그때는 TLS 설정이 "h2"를
     * 내놓아야 한다. */
    proven_http_event_server_config_t config = { .on = { .on_request = on_request, .on_done = on_done }, .ctx
= &app, .http2 = true };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(app.loop, &config, &server) == PROVEN_OK &&
```

```
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "a server that speaks both");
    /* 바쁜 서버는 연결마다 동시에 받는 요청 수를 줄였다가, 나중에 설정값(여기서는 기본값 100)까지
     * 다시 올릴 수 있다. 루프의 스레드에서, 또는 여기처럼 루프가 돌기 전에 부른다. */
    EXAMPLE_REQUIRE(proven_http_event_server_set_max_streams(server, 8) == PROVEN_OK &&
proven_http_event_server_set_max_streams(server, 0) == PROVEN_ERR_INVALID_ARG,
                    "eight requests at once from now on, and never none");
    /* 핸들러보다 앞서 클라이언트를 위해 쥐는 요청 본문도 줄일 수 있다: 지금부터 스트림당, 연결당
     * 최대 64 KiB - 주는 것의 천장이고, 이미 준 것을 거두지는 않는다. */
    EXAMPLE_REQUIRE(proven_http_event_server_set_window(server, 64 * 1024) == PROVEN_OK, "64 KiB of body at
most ahead of the handler");
    proven_job_sys_t *threads = NULL;
    proven_job_group_t running;
    proven_job_group_init(&running);
    EXAMPLE_REQUIRE(proven_job_system_init(heap, 1, 4, &threads) == PROVEN_OK &&
                    proven_job_group_submit(threads, &running, run, &app) == PROVEN_OK, "the loop runs on
another thread");

    // ---- HTTP/1.1 클라이언트 ------------------------------------------------------------------
    char url[64];
    snprintf(url, sizeof url, "http://127.0.0.1:%u/hello", (unsigned)at.port);
    proven_http_client_config_t client_config = { .alloc = heap, .max_idle_connections = 1 };
    proven_http_client_t *client = NULL;
    proven_http_client_response_t resp;
    proven_u8str_t body = { 0 };
    EXAMPLE_REQUIRE(proven_http_client_create(&client_config, &client) == PROVEN_OK &&
                    proven_http_client_get(client, proven_u8str_view_from_cstr(url), &resp) == PROVEN_OK &&
resp.status == 200 &&
                    proven_http_client_read_all(&resp, heap, &body, 4096) == PROVEN_OK, "the blocking client
of chapter 11 is answered");
    printf("HTTP/1.1 client: %.*s", (int)proven_u8str_as_view(&body).size, (const char
*)proven_u8str_as_view(&body).ptr);
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&body), PROVEN_LIT("/hello over HTTP/1\n")),
"over HTTP/1.1");
    proven_u8str_destroy(heap, &body);
    proven_http_client_finish(&resp);
    proven_http_client_destroy(client);

    // ---- 같은 포트의 HTTP/2 클라이언트 --------------------------------------------------------
    char answers[3][64] = { "", "", "" };
    EXAMPLE_REQUIRE(ask_three(at.port, answers) == 3, "three requests on one HTTP/2 connection are each
answered");
    for (int i = 0; i < 3; i++) printf("HTTP/2 client:   %s", answers[i]);
    EXAMPLE_REQUIRE(strcmp(answers[0], "/first over HTTP/2\n") == 0 && strcmp(answers[1], "/second over
HTTP/2\n") == 0 && strcmp(answers[2], "/third over HTTP/2\n") == 0,
                    "by the same handler, which was told the protocol was 2");

    // ---- 끝 ----------------------------------------------------------------------------------
    /* 루프를 멈춘 다음 처리기가 센 것을 보고 서버를 해체한다: 그때는 다른 무엇도 둘을 건드리지
     * 않는다. */
    proven_loop_stop(app.loop);
    proven_job_group_wait(threads, &running);
    EXAMPLE_REQUIRE(app.served == 4 && app.over_http2 == 3, "four exchanges ended with on_done(PROVEN_OK),
three of them over HTTP/2");
    proven_http_event_server_destroy(server);
    proven_loop_destroy(app.loop);
    proven_job_system_close(threads);
    proven_job_system_destroy(threads);
    return EXAMPLE_OK();
}
```

주의사항, 그리고 무엇이 잘못되는가

- **`on_writable`을 넘어 자기 위치를 기억하는 처리기는 전과 같이 안전하다.** `on_writable`은 처리기가 직접 한 호출 안에서는 결코 불리지 않는다 - 같은 처리기로 되돌아 들어오는 쓰기는 같은 바이트를 두 번 보내게 된다. 이것은 두 프로토콜 모두에 대한 서버의 약속이며, HTTP/2에서는 결함 하나를 겪고서야 찾았다 (8절).
- **읽지 않는 클라이언트에게서는 읽지 않는다.** 응답의 헤드는 언제나 큐에 넣는다 - 서버에는 그것을 달리 둘 곳이 없다. 그래서 묻기만 하고 읽지 않는 클라이언트를 제한하는 것은 이것이다: `max_buffered_output`의 두 배가 그 클라이언트를 기다리는 동안 서버는 그 연결에서 더 받지 않는다. 뒤따르는 요청은 소켓에 남는다. HTTP/1.1에서 응답이 밀려 있는 동안 파이프라인된 요청이 남는 것과 같다. 그때 연결이 출력으로 쥐는 최대는 `max_buffered_output`의 두 배와, 열려 있던 스트림마다 헤드 하나다 - 많아야 `max_concurrent_streams`개이고, 각각 처리기가 만든 크기다. 따라서 다른 응답을 읽지 않으면서 업로드하는 클라이언트는 읽히지 않게 되고, 읽으면 다시 읽힌다.
- **`max_buffered_output`은 스트림이 아니라 연결마다다.** 스트림 백 개를 가진 느린 클라이언트 하나는 HTTP/1.1 클라이언트 하나만큼을 쥐게 하지, 그 백 배가 아니다.
- **한가한 HTTP/2 연결은 HTTP/1.1 연결보다 크다. 받은 것만큼.** HTTP/1.1은 480바이트다. HTTP/2는 작은 요청 하나 뒤에 2,360바이트이고, 클라이언트가 기억하게 한 헤더 필드만큼 자라 두 압축 테이블이 가득 차면 약 12.8 KiB가 된다: 한가한 연결 만 개는 22에서 128 MiB 사이다. `http2_header_table_bytes`는 그 상한을 낮추고 `http2_no_header_table`은 없앤다(언제나 2 KiB 미만). 대가는 모든 필드를 매번 온전히 보내는 것이다. 헤더 목록을 읽는 버퍼 - 80 KiB - 는 연결마다가 아니라 서버 전체에 하나다. 진행 중인 요청 하나는 약 210바이트를 더한다: 서버에 160, 연결에 48.
- **요청이 끝나기 전에 답한 처리기는 그 요청과 끝난 것이다.** 서버는 클라이언트에게 그만 보내라고 말하고(5절), 이미 오던 것은 읽지 않고 버린다 - 오류였을 프레임까지도. 본문이 중요하다면 본문이 도착한 뒤에 답한다.
- **WebSocket에는 HTTP/1.1이 필요하다.** HTTP/2 스트림에서의 업그레이드는 거부된다 (`PROVEN_ERR_UNSUPPORTED`): WebSocket을 원하는 브라우저는 그것을 위해 HTTP/1.1 연결을 열고, 같은 포트가 그것을 받는다.
- **트레일러는 전달되지 않는다.** 어느 방향이든, HTTP/1.1에서도 그렇듯이: 요청의 트레일러는 본문을 끝내고 버려진다.

7. 이벤트 구동 클라이언트의 HTTP/2

```
typedef struct {
    /* ... 15장이 설명하는 모든 것 ... */
    bool http2;
    bool http2_cleartext;
    proven_u32 idle_timeout_ms;
    proven_size_t max_waiting_requests;
} proven_http_event_client_config_t;

int proven_http_event_request_protocol(const proven_http_event_request_t *request);
```

15장의 이벤트 구동 클라이언트는 HTTP/1.1에서 요청 하나에 연결 하나다. `http2`를 켜면 **HTTP/2에 합의한 오리진마다 연결 하나를 간직하고, 그 오리진으로 가는 모든 요청을 거기에 올린다**: 한 서버로 가는 요청 백 개가 핸드셰이크 백 번이 아니라 한 번이다. 콜백과 `proven_http_event_request_*` 호출은 같다. `start`가 약속하는 것도 같다 - 옵션이 가리키는 것은 모두 반환 전에 복사되거나 쓰이고, `on_done`은 정확히 한 번, `start` 안에서는 결코 오지 않는다.

- **TLS에서는** `http2`와, `h2`를 (그리고 그 뒤에 `http/1.1`을) 내놓는 TLS 설정: 오리진마다 핸드셰이크가 정한다. HTTP/1.1을 고른 오리진은 그렇게 기억되고, 전처럼 요청 하나에 연결 하나로 서비스된다. (그런 오리진은 마지막 64개까지, 하나에 약 600바이트로 기억한다. 잊힌 오리진은 핸드셰이크 한 번을 들여 다시 알아낸다.) TLS 설정에 `early_data`가 있고 세션이 `h2`로 만들어졌으면, 새 연결은 첫 비행 안에서 HTTP/2를 시작한다: 서문, 설정, 그리고 본문 없는 GET, HEAD, OPTIONS 요청 하나가 조기 데이터로 가고(14장 6절, v0.84.0) 나머지는 핸드셰이크 뒤에 간다. 서버가 거절한 것은 그대로 다시 가고, 425를 받으면 이제 열린 같은 연결에서 요청을 한 번 더 보낸다.

- **평문에서는** `http2_cleartext`: 그 클라이언트의 모든 `http` 요청이 사전 지식 방식의 HTTP/2다. 평문에서는 서버에 물어볼 방법이 없으니, 아는 서버에 대해서만 이렇게 말한다. 평문으로 두 종류와 모두 말하는 클라이언트는 클라이언트 둘이다. HTTP/2를 말하지 않는 서버에 대해 이렇게 말하면 모든 요청이 `PROVEN_ERR_UNSUPPORTED`로 끝난다: 연결은 맺어졌고, 서버의 설정이 오기 전에 - HTTP/2가 아닌 바이트를 받고, 또는 아무것도 받지 못하고 - 끝났다. 평문에서는 서버의 설정이 그 서버가 HTTP/2 서버라는 첫 신호다. 설정보다 앞서 보내는 것은 없으므로 그런 요청은 처리된 적이 없다. 그러나 다시 시작해도 똑같이 끝날 것이고, 그래서 답이 `PROVEN_ERR_AGAIN`이 아니다. (v0.35.0 전에는 그랬다.) 마침 그 순간 재시작 중이던 서버도 같은 답을 받는다: 그 서버가 HTTP/2를 말한다는 것을 안다면 요청을 다시 시작하라.

HTTP/1.1 요청은 결코 그러지 않았지만 **요청은 기다릴 수 있다**: 연결이 만들어지는 동안 그 연결을, 스트림을 몇 개 허락하는지 말해 주는 서버의 첫 마디를 - 그보다 앞서 보내는 것이 없으므로 그 때문에 거절되는 것도 없다 - 그리고 서버의 한도에 닿았을 때 빈 스트림을. 기다리는 요청은 시작할 때 받은 것의 사본을 쥔다. 연결 하나에 많아야 `max_waiting_requests`(1,000)개가 기다리고, 그다음 `start`는 `PROVEN_ERR_BUSY`다. 그리고 `max_connections`를 설정했고 모든 연결이 쓰이는 중이면, 연결이 없는 origin으로 가는 요청은 클라이언트에 연결 하나를 더 둘 자리가 날 때까지 기다린다 - 어느 연결 위도 아닌 곳에서, 많아야 그만큼 더, 길어야 `connect_timeout_ms`(15장).

`on_done`이 말하는 것은 전과 같되 답이 둘 늘었다. `PROVEN_ERR_AGAIN`은 HTTP/2 서버가 처리하지 않은 요청이다 - 서버가 먼저 작별했거나, 스트림을 거절했거나, 요청을 보내기 전에 연결을 잃었다. 그런 요청은 메서드가 무엇이든 다시 시작해도 된다. 클라이언트가 알아서 그러지는 않는다 - `retry_unsent`로 몇 번까지 그래도 되는지 정해 주지 않는 한(아래). `PROVEN_ERR_UNSUPPORTED`는 평문에서 HTTP/2를 말하지 않는 것으로 드러난 서버로 간 요청이다(위의 `http2_cleartext`): 역시 처리되지 않았고, 다시 시작할 가치가 없다. `PROVEN_ERR_RESET`은 여전히 "처리되었을 수도 있다"는 뜻이다 - 다만 응답이 선언한 `content-length`의 바이트가 모두 도착한 뒤 스트림을 `NO_ERROR`로 리셋한 서버는 끝까지 답한 것이고, 그것은 본문 전체와 함께 `PROVEN_OK`다(어떤 리셋이 그런지는 5절이 말한다). TLS 위에서는 `PROVEN_ERR_UNSUPPORTED`가 없다: 핸드셰이크가 프로토콜을 정하고, HTTP/1.1에 동의한 서버와는 HTTP/1.1로 말한다.

설정	HTTP/2에서
`connect_timeout_ms`	오리진의 연결 만들기 - 핸드셰이크, 그리고 그것을 HTTP/2 연결로 만드는 서버의 SETTINGS까지 - 그것을 기다리던 요청은 모두 그와 함께 끝난다(`PROVEN_ERR_TIMEOUT`)
`response_timeout_ms`, `body_timeout_ms`	요청 각각의 것: 답을 받지 못한 요청은 끝나고, 연결과 다른 요청은 계속된다
`write_timeout_ms`	연결의 것: 서버가 출력을 하나도 받지 않으면 연결이 끝난다
`idle_timeout_ms` (0: 60초)	요청이 없는 연결을 다음 요청을 위해 이만큼 간직했다가 닫는다
`max_buffered_output`	그 연결의 요청들을 합쳐서
`decompress`	그대로다. 정지된 요청은 도착한 것을 압축된 채로 쥐었다가 재개할 때 푼다
`decompress_zstd`	`zstd`에 대해 같다: `accept-encoding`으로 요청하고, 스트림에서 푼다
`decompress_brotli`	`br`에 대해 같다

예제: 요청 셋, 연결 하나

```
#include <stdio.h>
#include <string.h>

/*
 * HTTP/2로 말하는 이벤트 구동 클라이언트: 요청 셋, 연결 하나.
 *
 * 여기서는 클라이언트와 서버가 한 루프 위에 있고 손으로 돌리므로 스레드가 필요 없다.
 * 클라이언트의 콜백은 HTTP/1.1 프로그램의 것과 같다. 다른 것은 그 아래다 - 클라이언트가
 * 오리진마다 연결 하나를 간직하고, 그 오리진으로 가는 모든 요청을 그 연결에 올린다.
```

```
 */

typedef struct { int answered, over_http2; char text[3][32]; bool done[3]; } ex_seen_t;
typedef struct { ex_seen_t *seen; int which; } ex_call_t;

static void serve(void *ctx, proven_http_stream_t *stream, const proven_http_request_t *head) {
    (void)ctx;
    (void)proven_http_stream_respond(stream, 200, NULL, 0, (proven_mem_view_t){ head->target.ptr, head-
>target.size });     /* 자기 경로를 그대로 돌려준다 */
}

static void on_response(void *ctx, proven_http_event_request_t *request, const proven_http_response_t *head) {
    ex_call_t *call = ctx;
    /* 여기서는 2다. 아직 연결을 기다리는 요청이나, 서버가 HTTP/1.1만 말하는 것으로 드러난
     * 요청은 1이다. */
    if (head->status == 200 && proven_http_event_request_protocol(request) == 2) call->seen->over_http2++;
}

static void on_body(void *ctx, proven_http_event_request_t *request, proven_mem_view_t piece, bool last) {
    (void)request; (void)last;
    ex_call_t *call = ctx;
    snprintf(call->seen->text[call->which], sizeof call->seen->text[0], "%.*s", (int)piece.size, (const char
*)piece.ptr);
}

static void on_done(void *ctx, proven_http_event_request_t *request, proven_err_t why) {
    (void)request;
    ex_call_t *call = ctx;
    call->seen->done[call->which] = true;
    if (why == PROVEN_OK) call->seen->answered++;
}

int main(void) {
    proven_allocator_t heap = proven_heap_allocator();
    proven_loop_t *loop = NULL;
    EXAMPLE_REQUIRE(proven_loop_create(heap, &loop) == PROVEN_OK, "a loop");

    /* 두 프로토콜을 모두 말하는 서버(6절). */
    proven_http_event_server_config_t server_config = { .on = { .on_request = serve }, .http2 = true };
    proven_http_event_server_t *server = NULL;
    proven_net_addr_t at;
    EXAMPLE_REQUIRE(proven_http_event_server_create(loop, &server_config, &server) == PROVEN_OK &&
                    proven_http_event_server_listen(server, proven_net_addr_loopback(PROVEN_NET_FAMILY_IPV4,
0), &at) == PROVEN_OK, "a server");

    /* 평문에는 물어볼 핸드셰이크가 없으므로 클라이언트에게 알려 준다: 이 클라이언트의 모든
     * `http` 요청은 HTTP/2다. TLS에서는 대신 `http2`이고, "h2"를 내놓는 TLS 설정과 함께 쓰며,
     * 오리진마다 핸드셰이크가 정한다. */
    proven_http_event_client_config_t client_config = { .alloc = heap, .http2_cleartext = true };
    proven_http_event_client_t *client = NULL;
    EXAMPLE_REQUIRE(proven_http_event_client_create(loop, &client_config, &client) == PROVEN_OK, "a client");

    ex_seen_t seen = { 0 };
    ex_call_t calls[3] = { { &seen, 0 }, { &seen, 1 }, { &seen, 2 } };
    static const char *const paths[3] = { "/first", "/second", "/third" };
    for (int i = 0; i < 3; i++) {
        char url[64];
        snprintf(url, sizeof url, "http://127.0.0.1:%u%s", (unsigned)at.port, paths[i]);
        proven_http_event_request_options_t request = {
            .url = proven_u8str_view_from_cstr(url),
```

```
            .on = { .on_response = on_response, .on_body = on_body, .on_done = on_done }, .ctx = &calls[i],
        };
        /* 연결이 생기기 전에 한꺼번에 시작한다: 첫 요청이 연결을 만들고 나머지는 그것을
         * 기다리며, 서버가 무엇을 허락하는지 말한 뒤 셋이 함께 나간다. */
        EXAMPLE_REQUIRE(proven_http_event_client_start(client, &request, NULL) == PROVEN_OK, "a request is
started");
    }
    EXAMPLE_REQUIRE(proven_http_event_client_requests(client) == 3, "three are under way");

    proven_net_deadline_t until = proven_net_deadline_in(5000);
    while (!(seen.done[0] && seen.done[1] && seen.done[2]) && proven_time_monotonic_now() < until)
        EXAMPLE_REQUIRE(proven_loop_poll(loop, proven_net_deadline_in(10)) == PROVEN_OK, "a round of the
loop");

    for (int i = 0; i < 3; i++) printf("%s\n", seen.text[i]);
    EXAMPLE_REQUIRE(seen.answered == 3 && seen.over_http2 == 3, "all three were answered, over HTTP/2");
    EXAMPLE_REQUIRE(strcmp(seen.text[0], "/first") == 0 && strcmp(seen.text[1], "/second") == 0 &&
strcmp(seen.text[2], "/third") == 0, "each with its own answer");
    EXAMPLE_REQUIRE(proven_http_event_server_connections(server) == 1, "and the server saw one connection, not
three");

    proven_http_event_client_destroy(client);
    proven_http_event_server_destroy(server);
    proven_loop_destroy(loop);
    return EXAMPLE_OK();
}
```

주의사항, 그리고 무엇이 잘못되는가

- **요청의 객체는 여러분 손에서 바뀌지 않는다.** 오리진을 모르는 동안 시작되었고 그 오리진이 HTTP/1.1로 말하는 것으로 드러난 요청도 여전히 `start`가 준 그 포인터다. 그때 `proven_http_event_request_protocol`은 1을 답한다.
- **기다리는 요청은 보내지기 전에 타임아웃될 수 있다**: `connect_timeout_ms`가 그것이 기다리는 연결을 덮는다. 빈 스트림을 기다리는 요청에게는 자기 시계가 없다 - 재어지는 것은 그 앞의 요청들이다.
- **정지는 많아야 윈도 하나를 쥔다.** 서버에서와 같다: 클라이언트가 서버에게 소비했다는 말을 멈추고, 서버는 보내기를 멈춘다.
- **`PROVEN_ERR_AGAIN`은 요청하지 않는 한 대신 재시도해 주지 않는다.** `retry_unsent`는 횟수다: 그렇게 끝났을 요청 - 작별 뒤에 남은 것, 스트림이 거절된 것, 잃어버린 연결에서 기다리던 것 - 을 클라이언트가 스스로 서버에 몇 번까지 다시 보낼지를 정한다. 스트림 하나만 거절되고 연결이 계속되면 같은 연결로, 그렇지 않으면 같은 오리진의 다른 연결로 가며, 없으면 새로 만든다. 함께 옮겨지는 요청들은 순서를 유지한다. 실패한 시도에 대해서는 아무것도 알려 주지 않고, `on_done`은 마지막 시도의 결과로 한 번 온다. 기본값 0은 재시도하지 않는다는 뜻이다: 호출자가 정한다. 이 설정이 다루지 않는 것과 그 비용:
  - **본문을 조각으로 쓰는 요청은 아니다** - 클라이언트가 다시 보낼 조각을 갖고 있지 않다 - **그리고 `on_response` 뒤에도 아니다**: 답이 시작되는 것을 이미 보았다.
  - **`PROVEN_ERR_UNSUPPORTED`는 아니다.** 다시 해도 똑같이 끝난다. **`PROVEN_ERR_RESET`도 아니다.** 서버가 처리했을 수 있다. 시간 제한도 아니다.
  - **지연 없이 곧바로 한다.** 바빠서 스트림을 거절한 서버에게 같은 순간에 다시 묻는다. 작은 횟수는 몇 순간일 뿐이다. 시도 사이에 기다리고 싶은 프로그램은 설정을 0으로 두고 직접 다시 시작한다.
  - **시간 제한은 시도마다 따로다**: 요청 하나가 시도 횟수만큼 `connect_timeout_ms`나 `response_timeout_ms`를 쓸 수 있다.
  - **메모리에 든 본문은 응답이 시작될 때까지 메모리에 남는다.** 다 보낼 때까지만이 아니다.
  - **다른 이유로 실패한 시도는 그 이유로 요청을 끝낸다.** 새 연결을 만들 수 없으면 `on_done`은 `PROVEN_ERR_AGAIN`이 아니라 `PROVEN_ERR_REFUSED`를 말한다.

- **연결 하나는 실패 지점 하나다.** 그것이 끝나면 그 위의 모든 요청이 함께 끝난다 - 아직 보내지 않은 것은 `PROVEN_ERR_AGAIN`, 나머지는 `PROVEN_ERR_RESET`(평문에서 한 번도 HTTP/2가 아니었던 연결이면 모두 `PROVEN_ERR_UNSUPPORTED`).

8. 무엇을 받아들이고, 무엇이 없으며, 어떻게 시험했는가

디코더가 받아들이는 것은 RFC 7541이 정의한 HPACK이다. 블록 맨 앞의 테이블 크기 갱신은 몇 개든(RFC는 많아야 둘이라고 약속한다), 정수는 연속 바이트 다섯 개까지, 리터럴의 허프만 패딩은 1 비트 일곱 개까지이고 그 밖의 것은 안 된다.

인코더가 쓰는 것은 흔한 선택을 하는 유효한 HPACK이다: 테이블에 있으면 참조로, 허프만 코딩은 더 짧을 때만(길이가 같으면 하지 않는다), 새 필드는 비밀이거나 테이블보다 크지 않으면 테이블에 넣는다.

연결이 받아들이는 것은 RFC 9113이다. RFC가 선택을 남겨 둔 곳에서는 이렇게 한다: 폐기된 우선순위 필드는 무시하되, 자기 자신에게 의존한다는 스트림은 RFC 7540이 그랬듯 그 스트림의 오류다. 이쪽이 리셋한 스트림으로 온 프레임은, 그 리셋이 기억하고 있는 마지막 `max_concurrent_streams`개의 종료 안에 있는 동안 말없이 버린다. 한도를 넘는 헤더 목록은 연결이 아니라 그 스트림의 오류다. 본문과 맞지 않는 `content-length`는 스트림의 오류다. `HEAD`에 대한 응답과 상태 204, 304인 응답은 싣지 않은 길이를 알릴 수 있다.

여기에 없는 것:

- 11장 **블로킹 서버와 블로킹 클라이언트의 HTTP/2**: 연결 하나에 스레드 하나는 한 번에 한 교환에 어울리고, 그것이 HTTP/1.1이다.
- **지연을 둔 재시도, 그리고 서버가 처리하지 않은 것 말고 다른 것의 재시도**: 이벤트 구동 클라이언트는 그런 요청을 알리고(`PROVEN_ERR_AGAIN`), `retry_unsent`가 정해져 있으면 그 횟수만큼 곧바로 다시 보낸다. 시도 사이에 기다리는 것과, 처리되었을 수도 있는 것을 다시 시도하는 것은 호출자의 몫이다.
- **이벤트 구동 서버의 트레일러**, 어느 프로토콜이든.
- **확장 CONNECT**(RFC 8441, HTTP/2 위의 WebSocket)와 거기에 필요한 `SETTINGS`.
- **서버 푸시**: `PUSH_PROMISE`는 거절할 수 있도록 알아보기만 한다.
- **우선순위**(RFC 9113이 폐기한 RFC 7540의 방식): 건너뛴다.
- RFC 9113이 없앤 `Upgrade: h2c`. (HTTP/3은 9절과 10절.)
- **디코딩된 필드의 never-indexed 표시**: 프록시라면 그것을 전달하겠지만, 여기에는 그럴 것이 없다.

어떻게 시험했는가. 정적 테이블과 허프만 코드는 손으로 입력한 것이 아니다: 비공개 프로그램이 RFC 7541의 공개된 본문에서 읽어 오며, 항목이 61개이고 코드 257개가 서로 접두사가 아니며 코드 공간을 정확히 채우지 않으면 멈춘다. 등록된 시험은 링크된 테이블에 대해 같은 것을 검사한다. 그 시험은 이어서 RFC 부록 C의 예제 열여섯 개를 모두 디코더에 통과시키며 단계마다 동적 테이블을 RFC의 것과 비교하고, 인코더가 같은 선택을 하는 예제를 바이트 단위로 재현하게 하고, 잘못된 블록 열세 종류를 거부하고, 각 한도를 지키고, RFC 블록의 모든 앞부분과 모든 한 비트 변경을 디코딩한다. 두 번째 시험은 프레임 종류마다의 규칙을 경계의 양쪽에서, 그리고 잘못된 요청 서른한 개를 시험한다. 등록된 시험 밖에서는: 이 디코더가 다른 구현 열네 개가 같은 헤더 목록에 대해 만든 블록 47,142개를 읽었고(nghttp2, Go, Node, Python, Haskell, Swift 인코더. 테이블 크기가 바뀌는 순서 포함), Node의 HTTP/2 서버 - nghttp2다 - 가 이 인코더가 공유 연결 위의 요청 1,290개에 대해 쓴 블록을 모두 디코딩했으며, 손상된 블록 오백만 개를 버퍼 밖의 읽기나 쓰기를 보고하는 도구 아래에서 디코딩했고 보고된 것은 없었다.

연결에는 따로 등록된 시험이 있다. 연결 둘을 메모리에서 맞물려 아홉 가지 크기의 본문을 세 가지 윈도 크기로, 한 바이트씩 그리고 한꺼번에 실어 나르고, 스트림 쉰 개를 동시에 나르고, 프레임 셋으로 나가는 40,000바이트짜리 머리를 나른다. 대본 백한 개가 저마다 RFC 9113의 규칙 하나를 어기거나 바로 그 안쪽에 서고, 연결은 RFC가 정한 판정 - 무시, 코드가 붙은 스트림의 오류, 코드가 붙은 연결의 오류 - 을 지켜야 한다. 한 번은 바이트를 한 덩어리로, 한 번은 한 바이트씩 준다. 5절의 한계는 저마다 양쪽에서 그 경계까지 몬다. 녹음한 대화를 각 역할의 새 연결에 12,214번 다시 들려준다 - 앞쪽에서는 모든 지점에서, 그 뒤로는 간격을 두고 끊거나, 한 비트를 바꿔서. 그리고 대화의 모든 할당을 한 번씩 실패시킨다. 등록된 시험 밖에서는, 소켓을 연결에 통과시키는 비공개 프로그램을 통해: Node의 HTTP/2 클라이언트와 서버(nghttp2), 그리고 curl이 양쪽 역할로 요청, 5 MB까지의 본문, 20,000바이트짜리 머리, 트레일러, 중간 응답, 취소, 설정 변경을 열두 가지 설정 조합 아래에서 주고받았다 - 검사 7,896개, 실패 없음. 그리고 무작위로 행동하는 애플리케이션과

함께 연결 140,108개에 걸친 무작위 프레임 5,000,230개가 같은 도구 아래에서 연결의 불변식을 하나도 깨지 못했다.

6절의 서버에는 따로 등록된 시험이 있다: HTTP/1.1 서버의 처리기를 그대로 두고, 이 라이브러리의 연결로 만든 클라이언트가 실제 소켓으로 - 평문, TLS 1.3, TLS 1.2로 - 말을 건다. 양방향 본문, 동시 스트림 예순 개, 압축, 일시 정지, 양쪽의 리셋, 보내는 쪽을 닫고도 답을 받는 클라이언트, 한계, 타임아웃 각각, 작별, 그리고 내내 같은 포트의 HTTP/1.1 클라이언트. 등록된 시험 밖에서는, 비공개 프로그램이 한 포트에서 두 프로토콜을 Node의 http2, http, https 클라이언트와 다섯 가지 방식의 curl에게 서비스했다: 검사 7,392개, 마지막에는 실패 없음. **처음에는 하나가 실패했다**: TLS 위에서 Node에게 나란히 보내던 큰 응답 둘이 멈췄다. 서버가 처리기 자신의 쓰기 안에서 `on_writable`을 불렀고, 다시 들어온 처리기가 자기 위치를 잃었기 때문이다. 이 라이브러리 자체의 클라이언트는 그것을 일으키지 못했었다. 이제 등록된 시험이 스트림 둘을 경쟁시키며 처리기가 "결코 재진입되지 않음"을 지키게 한다.

HTTP/2 서버의 적합성 시험 모음인 **h2spec**(버전 2.6.0, 사례 146개)을 그 서버에 돌렸다: TLS에서 146개 가운데 146개, 평문에서 146개 가운데 145개. 그 하나는 "잘못된 연결 서문은 연결을 끝내야 한다"이다: 평문에서 서문이 아닌 바이트는 여기서 HTTP/2가 아니라 HTTP/1.1이고, 그렇게 답한다. 첫 실행은 두 가지를 찾았다: 길이를 알리지 않은 본문이 온다는 것을 처리기가 알 길이 없었고(이제 `Transfer-Encoding: chunked`를 본다), 자기 자신에게 의존하는 스트림을 거부하지 않았다.

7절의 클라이언트에는 6절의 서버를 한 루프에서 상대하는 등록된 시험이 있다 - 평문과 TLS로: 한 연결의 요청 마흔 개, 양방향 본문, 압축, 일시 정지, 중단, 서버의 스트림 한도, 타임아웃 각각, 놓아 주는 한가한 연결, 그리고 HTTP/1.1만 말하는 것으로 드러나는 오리진 - 기다리던 요청들은 호출자가 이미 쥐고 있던 객체 그대로 HTTP/1.1로 서비스된다. 등록된 시험 밖에서는, 비공개 프로그램이 그 클라이언트를 TLS와 평문의 Node HTTP/2 서버에 서버 설정 네 벌로, 그리고 HTTP/2가 없는 HTTPS 서버에 돌렸다: 검사 1,156개, 마지막에는 실패 없음. **처음에는 열다섯이 실패했다**: 스트림 넷을 허락하는 서버를 상대로, 새 연결에 몰아 보낸 요청들이 서버가 그렇게 말하기 전에 나갔고, 넘친 것이 거절되어 돌아왔다. 이제 클라이언트는 서버의 설정보다 앞서 아무것도 보내지 않는다.

등록된 시험이 찾지 못한 것, 한 번 더. 서버가 한 번의 읽기를 처리하는 동안 출력을 붙들어 두게 처음 만들었을 때(아래 표가 그것으로 얻은 것이다), 본문을 통째로 주는 응답이 몰리자 연결의 출력이 가득 찼고 그 뒤에 시작한 응답들은 자리가 없어 리셋되었다. 등록된 시험은 모두 통과했다. 비공개 검증의 Node 클라이언트는 7,392개 가운데 945개가 실패했다. 이제 붙들어 두기는 `max_buffered_output`의 4분의 1이 쌓이면 끝나고, 통째로 준 본문은 머리가 쓸 자리를 남기며, 등록된 시험 하나가 한 번의 쓰기로 요청 백 개를 그 머리들보다 한도가 작은 서버에 보낸다. 그 뒤 비공개 검증을 전부 다시 돌렸다 - Node와 curl을 상대로 7,896개, 7,392개, 1,156개, h2spec은 전과 같이 - 실패는 없었다.

속도, 소켓 없이 한 번 잰 것: 3.5 GHz x86-64 프로세서의 코어 하나에서 메모리 안의 연결 둘, GCC `-O2`. 필드 열두 개짜리 요청 머리(리터럴로 299바이트)를 초당 680,000번, 204 MB/s로 허프만 디코딩한다. 초당 385,000개의 요청이 보내지고, 디코딩되고, 머리로 답을 받고, 다시 읽힌다(백 개가 동시에). 본문은 2.4 GB/s로 건넌다. 이것은 프로토콜만의 비용이다 - 소켓, TLS, 처리기는 모두 더 든다.

소켓 위의 속도, 한 번 잰 것: 15장 14절의 부하를 같은 기계의 같은 서버에 HTTP/2로 다시 걸었다 - 테스트 스위트에 속하지 않는 부하 프로그램, 루프백 인터페이스 위의 두 프로세스인 클라이언트와 서버, 평문, 루프 하나, 2바이트를 답하는 처리기, 행마다 10초. 모든 연결은 요청 하나를 하고 열린 채 남는다. 그 뒤 그 가운데 일부가 요청 몇 개씩을 동시에 띄워 둔다. HTTP/1.1 행은 비교를 위해 같은 날 같은 서버 프로세스를 상대로 쟀다.

프로토콜	붙잡은 연결	요청하는 연결	연결마다 동시에	초당 요청	왕복: 중앙값, 99번째 백분위
HTTP/1.1	1,000	10	1	176,000	0.04 ms, 0.12 ms
HTTP/2	1,000	10	1	153,000	0.05 ms, 0.16 ms
HTTP/2	1,000	10	10	821,000	0.1 ms, 0.2 ms
HTTP/2	1,000	10	100	1,368,000	0.6 ms, 1.4 ms
HTTP/1.1	10,000	100	1	188,000	0.5 ms, 0.9 ms

HTTP/2	10,000	100	1	165,000	0.6 ms, 0.9 ms
HTTP/2	10,000	100	10	820,000	1.2 ms, 2.0 ms

표가 말하는 것. **한 번에 요청 하나씩이면 HTTP/2가 8분의 1쯤 느리다**: 요청마다 여전히 읽기 한 번과 쓰기 한 번이고, 그 사이에 할 일이 더 많다. **요청이 함께 떠 있으면 몇 배 빠르다**. 서버가 한 번의 읽기가 가져온 것을 모두 처리한 뒤에 쓰기 때문이다: 함께 도착한 요청 백 개는 한 번의 쓰기로 답을 받고, 서버의 10초 가운데 시스템의 몫이 7.3초에서 0.8초로 떨어진다. 왕복 시간은 동시에 뜬 수와 함께 늘어난다. 그럴 수밖에 없다: 요청 백 개가 루프 하나에서 차례를 기다린다. 어느 행에서도 연결이나 요청을 잃지 않았다. 서버에서, 요청 하나를 마친 한가한 HTTP/2 연결은 2,272바이트를 쥐었고 (`http2_no_header_table`이면 1,920), HTTP/1.1은 448바이트였다. 헤더 목록을 읽는 버퍼 82 KiB는 서버 전체에 하나다.

같은 서버를 상대한 라이브러리 자신의 클라이언트(7절): 연결 하나에 백 개를 띄워 세 번 재어 초당 471,000~486,000개 - 거기서는 두 프로세스가 번갈아 일하는데 클라이언트 차례가 더 길고, 서버는 10초 가운데 4초쯤 바빴다 - 그리고 하나씩이면 초당 42,000개, 중앙값으로 하나에 21마이크로초. (백 개를 띄운 수치는 403,000이었다. 연결이 스트림을 테이블을 훑어 찾지 않고 번호로 찾게 되고, 클라이언트가 요청의 URL을 두 번이 아니라 한 번 읽게 되기 전의 값이다.) 같은 클라이언트의 HTTP/1.1은 요청마다 연결을 만들던 때 초당 22,500개였다. v0.36.0부터는 연결을 보관하고(15장 12절), 여섯 연결 위에 백 개를 띄워 초당 129,000개, 한 연결에서 하나씩이면 초당 44,000개다. **루프백은 네트워크가 아니며**, 실제 일을 하는 처리기는 "ok"를 답하는 처리기보다 느리다.

하지 않은 것: 커버리지 기반 퍼징 없음. 실제 네트워크 위, TLS, 부하 아래의 큰 본문에 대한 측정 없음. 브라우저가 이 서버와 말해 본 적 없음. 외부 검토 없음. 연결은 스트림을 번호에서 자리로 가는 지도로 한 걸음에 찾는다. 지도는 스트림 테이블의 자리마다 칸 둘을 가진다. 와 보니 제 칸이 이미 찬 스트림 - 이웃들보다 훨씬 오래 열려 있는 스트림이 그렇게 만든다 - 은 테이블을 훑어 찾으며, 상대가 일부러 그렇게 만들면 연결은 지도가 있기 전처럼 테이블을 훑게 되고, 그보다 나쁜 일은 없다.

9. 이벤트 구동 서버의 HTTP/3

```
typedef struct {
    /* ... 15장이 설명하는 모든 것, 그리고 6절의 필드 ... */
    bool http3;
    bool http3_retry;
    proven_size_t http3_window_bytes;
} proven_http_event_server_config_t;
```

HTTP/3(RFC 9114)는 QUIC(RFC 9000) 위의 HTTP다. QUIC은 UDP 위의 전송으로, 스트림, 흐름 제어, 손실 복구와 혼잡 제어를 스스로 하며 TLS 1.3을 아래가 아니라 안에 품는다. 헤더 압축은 HPACK의 후속인 QPACK(RFC 9204)이다. HTTP/2에 견주어 주는 것은 잃은 패킷이 제 스트림만 붙잡는다는 것, 그리고 전송과 TLS의 핸드셰이크가 함께 한 왕복이라는 것이다. 모두 이 라이브러리의 코드다.

`http3`를 켜면 6절의 서버가 그것도 말한다. **처리기는 여전히 바뀌지 않는다**: 6절의 표는 HTTP/2에 대해 쓴 그대로 HTTP/3에도 맞고, 클라이언트의 창이라고 한 곳이 QUIC의 흐름 제어이며, `proven_http_stream_protocol`은 3을 답한다. 다른 것은 아래에 있다:

- **listen마다 UDP 소켓도 연다.** TCP 리스너와 같은 주소와 포트에. UDP 소켓을 얻지 못한 listen은 실패하고 TCP 리스너는 다시 닫힌다. 포트 0일 때 TCP가 받은 번호의 UDP 포트가 이미 쓰이고 있으면 실패가 아니라 다른 번호를 시도한다(모두 17개까지). HTTP/3은 TLS 위에서만: TLS 설정이 `h3`를(`h2`, `http/1.1` 곁에) 내야 하고, `http3`가 꺼져 있으면 내지 말아야 한다. QUIC에서 서버는 목록이 TCP에서 무엇을 어떤 순서로 내든 `h3`만 합의한다(RFC 9114 3.1절). `h3`를 내지 않는 QUIC 클라이언트는 `no_application_protocol`로 끝난다. 0.73.2 전에는 서버가 클라이언트가 낸 것 중 자기 첫 선택을 골랐으므로, 이 라이브러리의 클라이언트와 서버는 QUIC에서 `h2`를 합의하고 그 아래에서 HTTP/3을 말했다.
- **클라이언트가 찾는 법.** HTTP/1.1과 HTTP/2 응답에 `Alt-Svc: h3=":PORT"` - 연결이 들어온 포트 - 가 붙는다. 처리기가 제 `Alt-Svc`를 썼으면 그것이 대신 간다. 브라우저나 `curl --alt-svc`는 다음 요청에서 HTTP/3을 시도하고, `curl --http3-only`는 곧바로 그것을 청한다.

- **소켓 하나, 연결 여럿.** QUIC 연결은 소켓이 아니라 연결 ID로 가린다. 데이터그램은 그것이 실은 ID의 연결로 가고, 클라이언트의 첫 패킷이 연결을 만들며, 아무도 쥐지 않은 ID로 온 짧은 패킷에는 상태 없는 리셋으로 답한다.
- **쓰기를 묶는 것**은 `max_buffered_output`이 아니라 QUIC의 보내기 버퍼 - 보냈고 아직 확인되지 않은 데이터가 스트림당 256 KiB, 연결당 1 MiB, 또는 클라이언트가 허락한 범위 안에서 혼잡 창의 두 배, 16 MiB와 24 MiB까지 - 다. 거절된 쓰기는 다른 둘에서처럼 `on_writable`을 듣는다.
- **멈춘 요청**은 읽히지 않게 되고, QUIC의 흐름 제어가 클라이언트를 `http3_window_bytes`에서 멈춘다. 기본은 스트림당 256 KiB, 연결당 1 MiB에서 시작해, 본문을 오는 대로 읽는 동안 두 배씩 16 MiB와 24 MiB까지 자란다. 읽히지 않는 본문은 자람을 멈추므로 멈춘 요청은 가졌던 창보다 더 쥐지 않는다(v0.71.0). 보내기 버퍼는 상대가 허락한 범위 안에서 경로가 나르는 만큼(혼잡 창의 두 배) 같은 상한까지 자란다.

설정	HTTP/3에서
`max_connections`	TCP 연결과 함께 QUIC 연결을 센다. 한도에서는 클라이언트의 첫 패킷에 답하지 않는다
`max_concurrent_streams`	클라이언트가 한 연결에서 한꺼번에 열 수 있는 요청 스트림. 서버가 도는 동안 바꿀 수 있다 (`proven_http_event_server_set_max_streams`, 6절). 클라이언트에게 이미 허락한 것은 거둘 수 없어서(RFC 9000 4.6), 낮춘 수는 요청이 충분히 끝난 뒤에 드러난다. 올린 수는 바로 준다
`max_head_bytes`, `max_headers`	QPACK이 세는 가장 큰 필드 구역으로, SETTINGS로 클라이언트에 알린다. `max_headers`보다 많은 필드는 `431`
`max_body_bytes`	요청마다: HTTP/2에서처럼 `413`
`http3_window_bytes` (0: 스트림 256 KiB, 연결 1 MiB에서 본문을 읽는 동안 16 MiB와 24 MiB까지 자람)	처리기가 가져가기 전에 클라이언트가 보낼 수 있는 요청 본문 바이트. 값을 주면 고정. 최대 2^31-1
`http3_retry`	클라이언트마다 첫 패킷에 Retry: 주의할 것을 보라
`head_timeout_ms`	연결의 첫 패킷부터 - 핸드셰이크를 포함해 - 첫 요청까지
`body_timeout_ms`	본문이 남은 요청 하나의 침묵: 그 스트림이 리셋되고, 나머지는 계속된다
`write_timeout_ms`	움직이지 않는 출력: 연결이 닫힌다
`idle_timeout_ms`	열린 요청이 없음: 서버가 작별하고 닫는다. 아무것도 오지 않는 연결 - 사라진 클라이언트 - 은 이것과 `head_timeout_ms` 중 큰 것보다 1초 뒤에 놓는다(QUIC의 유휴 시간 초과)

작별은 한 왕복을 사이에 둔 GOAWAY 둘이다(RFC 9114 5.2절). 첫째는 클라이언트가 아직 보낼 수 있는 가장 큰 요청을 알려, 이미 오는 중인 요청은 거절되지 않고 답을 받는다. 둘째는 처리하지 않은 첫 요청을 알리고, 받은 것에 다 답하면 연결은 H3_NO_ERROR로 닫힌다. `proven_http_event_server_stop_listening`이 이렇게 한다. UDP 소켓은 연결이 쓰는 동안 열려 있고, 같은 포트에 다시 listen하면 그것을 이어 쓴다. `proven_http_event_server_shutdown`과 `_destroy`는 HTTP/2 연결을 끝내듯 HTTP/3 연결을 끝낸다.

주의할 것, 그리고 잘못되는 것

- **UDP는 핸드셰이크 전에 서버에 닿는다.** 누구든 어떤 주소에서나 클라이언트의 첫 패킷을 보낼 수 있고, `http3_retry`가 없으면 하나하나가 연결을 만든다 - 핸드셰이크를 기다리는 동안 약 40 KiB, 답하지 않는 주소 천 곳에서 잰 값 - 그리고 `head_timeout_ms`가 그것을 놓을 때까지 간다. 그런 주소에는 그 주소가 보낸 것의 세 배보다 많이 보내지 않으므로(RFC 9000 8절) 서버를 남을 향한 범람에 쓸 수는 없지만, 범람은

max_connections만큼을 붙잡을 수 있다. http3_retry는 주소가 받는다는 것을 보일 때까지 아무것도 두지 않으며, 연결마다 한 왕복을 치른다.

- **Retry 토큰과 상태 없는 리셋의 키는 서버를 만들 때 뽑는다.** 다시 시작한 서버는 클라이언트의 옛 연결을 리셋할 수 없고, 그 클라이언트는 유휴 시간 초과를 기다린다.
- **타이머는 제 시각에 울린다**(POSIX에서는 약 1 ms 안, Windows에서는 시스템 타이머 간격; v0.70.0): 조용한 연결의 QUIC 타이머 - 프로브, 늦춘 확인 - 는 더 이상 다음 16 ms 경계를 기다리지 않고, 바쁜 연결은 라운드마다 제 일을 한다.
- **HTTP/3이 얼마나 빠른지는 플랫폼에 달렸다.** Linux에서는 한 라운드의 데이터그램이 커널에 함께 간다 - 1,452바이트 45개까지 호출 하나에(UDP_SEGMENT), 수신 하나가 여러 개를 돌려줄 수도 있다(UDP_GRO). Windows에서도 USO와 URO로 같다(UDP_SEND_MSG_SIZE, UDP_RECV_MAX_COALESCED_SIZE; v0.74.0). 다른 곳에서는 데이터그램 하나가 시스템 호출 하나다. 어느 쪽이든 하나하나 따로 암호화되고 보호된다. Windows 11 시험 VM에서 이 서버와 이 클라이언트를 한 프로세스에 두고 루프백으로 256 MiB: HTTP/3 2,545 Mbit/s, 데이터그램마다 호출 하나일 때 1,021, HTTP/2 1,919(x86-64. i686은 1,807, 845, 1,627). Linux 루프백에서 이 서버로부터 curl로 100 MiB 하나를 내려받는 데 HTTP/3은 369 MiB/s, HTTP/2는 382, HTTP/1.1은 499였다(모두 TLS 위; v0.68.0. v0.67.0에서는 데이터그램마다 호출 하나로 219, v0.65.0에서는 174).
- **데이터그램은 1,200바이트에서 시작해 탐침으로 커진다**(RFC 9000 14.3절). 경로와 상대가 실어 나르면 1,452까지: 1,452를 나르는 경로에서는 핸드셰이크 뒤 탐침 하나, 찾은 뒤에는 없음. 경로가 그 크기를 더 나르지 않으면 손실이나 탐침 타임아웃으로 알아채고, 크기를 1,200으로 되돌린 뒤 다시 찾는다. 설정할 것은 없다. 모든 데이터그램에 조각 금지 비트가 서고 이쪽에서는 아무것도 조각내지 않으므로(Linux IP_PMTUDISC_PROBE, Windows IP_DONTFRAGMENT; v0.69.0), 통과한 탐침은 통째로 통과한 것이다.
- **TLS 설정에 early_data가 있으면 0-RTT**(14장 6절; v0.73.0). 재개하는 클라이언트는 첫 요청들을 첫 패킷에 실어 보낼 수 있고, 서버는 첫 응답에서 답한다: 왕복 둘이 하나가 된다. 그렇게 온 요청은 핸드셰이크가 끝나기 전에 핸들러에게 가며, 그 요청에서 proven_http_stream_early는 참이다: **공격자가 재전송한 사본일 수 있다**. 핸들러에게 조기에 가는 것은 본문 없는 GET, HEAD, OPTIONS뿐이고, 그 밖의 것이 조기에 오면 서버가 스스로 425(Too Early)로 답한다. 사본에 따라 행동하면 안 되는 핸들러도 425로 답하면 된다 - 클라이언트는 핸드셰이크가 끝난 뒤 요청을 다시 보낸다. 기본은 꺼짐.
- **클라이언트의 다음 연결은 주소 확인을 건너뛴다.** 핸드셰이크마다 서버는 클라이언트에게 IP 주소에 묶이고 하루 동안 유효한 토큰(NEW_TOKEN, RFC 9000 8.1.3절)을 준다. 그 토큰을 가져온 연결은 곧바로 확인된 것으로 다뤄지므로
  - 보낸 양의 3배라는 한도도, http3_retry의 Retry도 없다 - 약 3.6 KB보다 큰 0-RTT 응답도 왕복 하나를 기다리지 않는다. 확인되지 않는 토큰은 없는 것으로 다룬다.
- **HTTP/3 위에서 없는 것:** WebSocket(proven_ws_event_accept는 그 스트림을 거절한다), 트레일러, 서버 푸시, 이쪽 스스로의 이주(클라이언트의 새 주소는 검증한다), 1이 아닌 QUIC 버전.
  proven_http_event_server_adopt는 TCP 연결만 받으므로, 여러 루프가 UDP 포트 하나를 나눠 쓸 수 없다.
- **이벤트 구동 클라이언트는 요청이 청하면 HTTP/3을 말한다**(10절). 이 서버의 Alt-Svc를 스스로 따르지는 않는다.

어떻게 시험했는가. 등록된 시험이 루프백 UDP 위에서 이 라이브러리의 QUIC과 HTTP/3 엔진으로 만든 클라이언트를 이 서버에 붙인다 - WebSocket이 아닌 HTTP/2 시험의 모든 경로, 한도, 각 시간 초과, 작별, Retry. 등록된 시험 밖에서: curl --http3-only(OpenSSL의 QUIC)와 aioquic의 클라이언트를 이 서버에, 모든 응답을 바이트 단위로 확인. h3spec 0.1.14, 77개 중 76개 통과 - 통과하지 않은 하나는 반복된 모르는 설정으로, RFC 9114가 서버에 받아도 된다고 하는 것이다. 답하지 않는 주소에서 클라이언트의 첫 패킷 천 개, 주소마다 보낸 것의 세 배 이하만 보냈고, 모두 head_timeout_ms 뒤에 놓았으며, http3_retry로는 하나도 두지 않았다.

10. 이벤트 구동 클라이언트의 HTTP/3

```
typedef struct {
    /* ... 15장이 설명하는 모든 것, 그리고 7절의 필드 ... */
    bool http3;
    proven_size_t http3_window_bytes;
```

```
} proven_http_event_client_config_t;

typedef struct {
    /* ... */
    bool http3;            /* 이 요청을 HTTP/3으로 */
} proven_http_event_request_options_t;
```

HTTP/2는 TLS 핸드셰이크가 고른다. HTTP/3은 그럴 수 없다 - UDP 위의 다른 전송이고, 첫 패킷 전에는 물을 것이 없다. 그래서 **호출자가 고른다**: `http3`를 켠 클라이언트는 옵션에 `http3`를 적은 요청을, 각각 `https` URL로, HTTP/3으로 보내고, 다른 요청은 전처럼 보낸다. 서버의 `Alt-Svc`를 스스로 따르지 않는다. 콜백과 요청 호출은 세 프로토콜에 같고, `proven_http_event_request_protocol`은 처음부터 3을 답한다.

아래에서 일어나는 일은 QUIC 위의 7절이다: 오리진(스킴, 호스트, 포트, 주소)마다 연결 하나를 제 UDP 소켓에 두고 요청들이 나눠 쓴다. 서버의 스트림 한도를 넘는 요청은 그 위에서 기다린다. 쉬는 연결은 `idle_timeout_ms` 뒤에 작별하고 닫는다. `max_connections`는 TCP 연결과 함께 센다. `retry_unsent`는 서버가 처리하지 않고 거절한 요청(H3_REQUEST_REJECTED, 또는 GOAWAY 뒤)을 다시 보낸다. `decompress`는 다른 것에서처럼 푼다. 쓰기는 QUIC의 보내기 버퍼 - 스트림 256 KiB, 연결 1 MiB, 또는 서버가 허락한 범위 안에서 혼잡 창의 두 배, 16 MiB와 24 MiB까지 - 로 묶이고, 멈춘 응답은 읽히지 않아 서버는 가졌던 창에서 멈춘다. 오는 대로 읽으면 `http3_window_bytes`가 고정하지 않는 한 창이 스트림 16 MiB, 연결 24 MiB까지 자란다(v0.71.0).

대체는 없다. HTTP/3으로 청한 요청의 서버가 UDP로 답하지 않으면 `connect_timeout_ms`가 지난 뒤 `PROVEN_ERR_TIMEOUT`으로 - 연결하지 않은 UDP 소켓은 거절을 듣지 못한다 - , `h3`를 거절하는 서버면 `PROVEN_ERR_UNSUPPORTED`로, 믿을 수 없는 서버면 `PROVEN_ERR_UNTRUSTED`로 끝난다. QUIC에서 클라이언트는 TLS 설정이 TCP에서 무엇을 내든 `h3`만 내고 그 밖의 것은 받지 않는다(0.73.2부터). TCP 연결은 시도하지 않는다: 그것을 원하는 호출자는 `http3` 없이 요청을 다시 시작한다.

주의할 것, 그리고 잘못되는 것

- **그것을 말한다고 알려진 서버에만 청하라.** 어느 서버가 그런지 여기서는 배우지 않는다. 서버의 `Alt-Svc`가 말해 주며, 그것을 읽는 것은 호출자의 몫이다.
- **얼마나 빠른지는 서버처럼 플랫폼에 달렸다**: Linux에서는 한 라운드의 데이터그램이 커널에 함께 가고 수신 하나가 여러 개를 돌려줄 수 있으며, 다른 곳에서는 하나에 시스템 호출 하나다. Linux 루프백에서 이 라이브러리의 서버로부터 이 클라이언트로 100 MiB 하나를 내려받는 데 HTTP/3은 393 MiB/s, HTTP/2는 340이었다(v0.68.0; v0.67.0의 HTTP/3은 233, v0.65.0은 171). 데이터그램은 서버처럼 탐침으로 1,200에서 1,452바이트까지 커진다.
- **TLS 세션은 출처마다 보관한다**(15장; v0.72.0): 새 연결은 그것을 재개하고, 서버의 NEW_TOKEN 토큰을 다음 연결을 위해 보관하므로 다음 연결은 주소 확인을 건너뛴다.
- **TLS 설정에 `early_data`가 있으면 0-RTT**(14장 6절; v0.73.0). 세션이 허용하는 새 연결에서는 본문 없는 GET, HEAD, OPTIONS 요청을 첫 패킷에 실어 보내 왕복 하나 먼저 답을 받는다. 그 밖의 요청은 핸드셰이크를 기다린다. 조기 데이터로 거절되거나 425를 받은 요청은 핸드셰이크 뒤 한 번 다시 보내며, 두 번째 답만 전달한다. 기본은 꺼짐.
- **여기 없는 것:** 이쪽 스스로의 연결 이주, HTTP/3 위 WebSocket.

어떻게 시험했는가. 등록된 시험이 이 클라이언트를 루프백 UDP 위 이 라이브러리의 서버에 붙인다 - 서버가 답할 수 있는 7절 시험의 모든 사례, 클라이언트의 거절, 대체 없음, 믿을 수 없는 서버, 시간 초과. 등록된 시험 밖에서: aioquic의 서버(50 MB까지 바이트 단위 다운로드, 업로드, 트레일러, 건너뛴 중간 응답, 요청 400개)와 공개 서버 둘, cloudflare-quic.com과 www.google.com.

6장: 실행, 별칭, PAL, Freestanding, 크로스 빌드

6부 — 더 나아가기. 선행 조건: 2–5부. 이 장을 마치면 메모리 모델에 놀라는 일 없이 둘 이상의 스레드에서 작업을 실행할 수 있고, 루프처럼 읽히는 상태 기계를 작성할 수 있으며, 운영체제가 없는 대상용으로 빌드할 수 있다.

이 장은 `coro.h`, `job.h`, `alias_xcv.h`, 스레드 안전성과 포인터 프로버넌스, PAL 계약, 프리스탠딩(freestanding) 모드, 플랫폼 빌드 관련 사항을 다룬다. 이것은 이 매뉴얼에서 가장 어려운 내용이며, 의도적으로 맨 마지막에 놓았다 — 1–5부의 어떤 것도 여기에 의존하지 않는다.

목차

1. 스택리스 코루틴
2. Job system
3. 스레드, allocator, 포인터 프로버넌스
4. 별칭 계층
5. PAL 계약
6. Freestanding 부분집합
7. 크로스 컴파일
8. 예제와 오용 사례

1. 스택리스 코루틴

코루틴 매크로는 switch 안에서 `__LINE__` 라벨을 사용하여 스택리스 상태 기계를 구현한다. 코루틴 함수는 `proven_i32`를 반환해야 한다: `0`은 yield 또는 대기 중을 의미하고, `1`은 완료를 의미한다.

구조체

```
typedef struct {
proven_i32 state;
} proven_coro_t;
```

매크로

매크로	의도
`PROVEN_CORO_INIT(c)`	첫 사용 전에 state를 0으로 설정한다.
`PROVEN_CORO_BEGIN(c)`	코루틴 switch 블록을 시작한다.
`PROVEN_CORO_YIELD(c)`	재개 지점을 저장하고 0을 반환한다.
`PROVEN_CORO_AWAIT(c, cond)`	`cond`가 참이 될 때까지 0을 반환하고, 참이 되면 계속 진행한다.
`PROVEN_CORO_END(c)`	완료로 표시하고 1을 반환한다.
`PROVEN_CORO_IS_DONE(c)`	state가 -1인지 확인한다.

어떻게 전개되는가 (그리고 유일하게 중요한 규칙 하나)

이 매크로들은 고전적인 Protothreads/Duff's-device 기법이다. `BEGIN`은 `switch (state)`를 열고, 각 `YIELD`/`AWAIT`는 `__LINE__`을 재개 라벨로 기록한 뒤 `return`하며, `END`는 switch를 닫는다. 개념적으로는 다음과 같다:

```
proven_i32 f(Ctx *c) {
    switch (c->coro.state) {       /* PROVEN_CORO_BEGIN */
    case 0:
        /* ... code before the first yield ... */
        c->coro.state = 42; return 0; case 42:;   /* PROVEN_CORO_YIELD on line 42 */
        /* ... code after the yield ... */
    }
    c->coro.state = -1; return 1;  /* PROVEN_CORO_END */
}
```

함수는 각 yield 지점에서 **반환**되고, **맨 위에서 다시 진입**하여 재개 `case`로 곧장 점프한다. 그 결과로 기억해야 할 단 하나의 규칙:

지역(스택) 변수는 yield를 넘어 살아남지 못한다. YIELD/AWAIT를 넘어 값이 유지되어야 하는 것은 코루틴 자신의 구조체 안에(또는 static으로) 있어야 한다. 지역 변수는 호출할 때마다 다시 생성되기 때문이다.

전개에서 따라 나오는 다른 제약들: 두 코루틴 매크로는 같은 소스 라인을 공유해서는 안 되고(__LINE__이 충돌한다), YIELD/AWAIT는 여러분 자신의 switch 안에 있으면 안 된다(잘못된 case에 착지하게 된다). 코루틴은 스택이 없으므로 자신이 호출하는 헬퍼 함수에서는 yield할 수 없다 — 오직 자기 본문에서만 가능하다.

반례 — yield를 넘는 지역 변수

```
static proven_i32 bad(Ctx *c) {
    PROVEN_CORO_BEGIN(&c->coro);
    int i = 0;                      /* WRONG: a local */
    while (i < 3) {
        i += 1;
        PROVEN_CORO_YIELD(&c->coro); /* on re-entry `i` is re-initialized to 0 */
    }
    PROVEN_CORO_END(&c->coro);      /* loop never ends: infinite yields */
}
/* RIGHT: put `i` in Ctx (like `value` in the Counter example below). */
```

예제(스케치: 코루틴은 함수 전체이므로 문장 조각으로는 보여줄 수 없다 — 이 형태를 컴파일한 버전은 8절의 다루어진 예제, manual/examples/ex_06_coro.c이다):

```
typedef struct Counter {
proven_coro_t coro;
int value;
} Counter;

static proven_i32 counter_next(Counter *c) {
PROVEN_CORO_BEGIN(&c->coro);
c->value = 0;
while (c->value < 3) {
c->value += 1;
PROVEN_CORO_YIELD(&c->coro);
}
PROVEN_CORO_END(&c->coro);
}

Counter c = {0};
PROVEN_CORO_INIT(&c.coro);
while (!counter_next(&c)) {
use_value(c.value);
}
```

2. Job 시스템

작업(job) 시스템은 워커 스레드와 경계가 있는 MPMC 방식 큐를 소유한다. 생산자는 routine(arg) 작업 항목을 제출한다. 워커는 시스템이 닫히고 비워질 때까지 큐에 들어온 작업을 실행한다.

구조체

```
typedef struct {
void (*routine)(void *arg);
void *arg;
} proven_job_t;

typedef struct proven_job_sys proven_job_sys_t;
```

proven_job_sys_t는 불투명(opaque) 타입이다.

함수

API	의도	반환

`proven_job_system_init(alloc, num_workers, max_queue_capacity, out_sys)`	job system을 할당하고 시작한다. 큐 용량은 2의 거듭제곱이어야 한다.	`proven_err_t`.
`proven_job_system_close(sys)`	새 작업 수락을 중단한다.	void.
`proven_job_system_destroy(sys)`	필요하면 닫고, 큐를 비우고, 워커를 join하고, 자원을 해제한다.	void.
`proven_job_submit(sys, routine, arg)`	작업 하나를 제출한다. 다른 제출자와 스레드 안전하다.	큐에 들어가면 true, 가득 차거나 닫혀 있으면 false.
`proven_job_execute_one(sys)`	호출 스레드가 사용 가능한 작업 하나를 실행하게 한다.	작업이 실행되었으면 true.
`proven_job_submit_ex(sys, routine, arg)`	제출하되, 거절되면 이유를 말한다.	`PROVEN_OK`. 가득 차면 `PROVEN_ERR_AGAIN`(재시도하거나 거든다), 닫혔으면 `PROVEN_ERR_INVALID_STATE`(멈춘다).
`proven_job_group_init(&group)`	작업 묶음을 세기 시작한다.	void.
`proven_job_group_submit(sys, &group, routine, arg)`	그룹이 세는 작업을 제출한다. 거절된 작업은 세지 않는다.	`submit_ex`와 같다.
`proven_job_group_pending(&group)`	그룹에서 아직 끝나지 않은 작업 수.	`proven_size_t`.
`proven_job_group_wait(sys, &group)`	센 작업이 모두 실행되면 돌아온다. 기다리는 동안 호출자가 큐의 작업을 직접 실행한다. 작업이 쓴 내용은 돌아온 뒤에 보인다.	void.

```
/* Render 16 tiles on the pool and wait for all of them. render_tile is the
 * program's own job routine; the wait runs queued tiles itself while it waits. */
void render_tile(void *tile);
static int tiles[16];
proven_job_sys_t *pool = NULL;
if (proven_is_ok(proven_job_system_init(alloc, 4, 64, &pool))) {
    proven_job_group_t frame;
    proven_job_group_init(&frame);
    for (int i = 0; i < 16; ++i) {
        proven_err_t e = proven_job_group_submit(pool, &frame, render_tile, &tiles[i]);
        if (e == PROVEN_ERR_AGAIN) { (void)proven_job_execute_one(pool); --i; continue; }   /* full: help,
retry */
        if (e != PROVEN_OK) break;                                                          /* closed */
    }
    proven_job_group_wait(pool, &frame);
    proven_println("{} tiles still pending", PROVEN_ARG(proven_job_group_pending(&frame)));
    proven_job_system_close(pool);
    if (proven_job_submit_ex(pool, render_tile, &tiles[0]) == PROVEN_ERR_INVALID_STATE) {
        proven_println("closed: no more work is taken");   /* not "full - try again" */
    }
    proven_job_system_destroy(pool);
}
```

중요한 제약:

- `proven_job_system_close()`는 제출자와 경쟁할 수 있다. close 전에 진입을 허가받은 제출은 close가 반환되기 전에 작업을 commit하거나 `false`를 반환하며, 그 이후의 제출은 거부된다.
- `proven_job_system_destroy()`는 여전히 `proven_job_submit()`을 호출하는 생산자 스레드와 경쟁(race)해서는 안 된다.

- close를 정지 신호로 사용하려면 close를 호출하고, 모든 생산자를 join한 뒤에만 destroy해야 한다.
- 큐 시퀀스 카운터는 하나의 job-system 수명 동안 부호 있는 포인터 차이(signed pointer-difference) 범위를 넘어 랩(wrap)하지 않는다고 가정한다.

동시성 모델

큐는 `max_queue_capacity`개의 슬롯을 가진 고정 크기 링 버퍼이다(2의 거듭제곱이어야 하며, 그래서 슬롯 인덱스는 `seq & (capacity - 1)`이다). 생산자와 소비자는 큐 전체를 잠그지 않고 **원자적 시퀀스 카운터**로 협조한다:

- 생산자(`proven_job_submit`)는 다음 enqueue 위치를 원자적으로 확보한다. 확보가 아직 소비되지 않은 가장 느린 슬롯을 덮어쓰게 되는 경우, 큐가 가득 찬 것이며 submit은 용량이 생기기를 기다리거나 대기 중인 작업을 덮어쓰는 대신 `false`를 반환한다. 워커 wake를 post하는 과정에서는 플랫폼 동기화에 잠시 진입할 수 있다.
- 소비자(워커 스레드, 또는 `proven_job_execute_one`을 통한 여러분 자신의 스레드)는 다음 dequeue 위치를 원자적으로 확보하고, `routine`/`arg`를 읽고, 슬롯을 재사용 가능으로 표시한 뒤, 확보 구간 바깥에서 routine을 실행한다.

유휴 워커는 계속 폴링하고 yield하는 대신 플랫폼 카운팅 세마포어에서 블록한다. 성공한 submit은 슬롯을 먼저 공개한 뒤 permit 하나를 post한다. 대기 중인 워커가 없어도 permit은 남으므로 빈 큐를 확인한 시점과 대기에 들어가는 시점 사이에서 wake가 유실되지 않는다. `close`는 먼저 새 진입을 막고 이미 허가된 submit이 끝나기를 기다린 다음, 워커마다 최종 permit 하나를 post한다. 외부 `proven_job_execute_one`이 워커보다 먼저 작업을 가져가면 남은 permit은 단지 stale 상태가 된다. 워커는 빈 큐를 다시 확인한 뒤 다시 대기하거나, close 뒤라면 종료한다.

submit과 execute가 MPMC-안전하므로, 여러 스레드가 동시에 제출할 수 있고 여러 스레드가 동시에 소비할 수 있다. 라이브러리가 여러분을 위해 **하지 않는** 것: 그것은 *작업이 건드리는 데이터*를 동기화하지 않는다. 같은 변수를 쓰는 두 작업은 여전히 자체 잠금/원자 연산이 필요하다 — 큐는 인계(handoff)만 순서 짓고 작업 자체는 아니다. 그 작업이 사용하는 할당자(allocator)의 스레드 안전성, 할당을 공유할 때의 포인터 프로버넌스 위험, 그리고 락프리 도구 상자(CAS, ABA, 태그 포인터, hazard pointer, 에폭 기반 회수)에 대해서는 2장 §7 "Allocator 스레드 안전성과 프로버넌스"를 참조하라.

수명 주기 상태 기계(이 순서로 구동하라):

```
init  --->  running  --(close)-->  closed  --(destroy: drain + join)-->  freed
```

- `init`은 `num_workers`개의 OS 스레드와 슬롯 버퍼를 미리 예약한다.
- `close`는 플래그를 뒤집어 이후의 모든 `submit`이 `false`를 반환하게 한다. 이미 허가된 submit은 close가 반환되기 전에 정리되며, 수락된 모든 작업은 여전히 실행된다.
- `destroy`는 필요하면 닫은 다음, **큐가 비고 모든 워커가 join될 때까지 호출 스레드를 블록**한 뒤, 모든 것을 해제한다.

메모리 가시성. `destroy` 내부에서 워커가 join되는 것은 동기화 지점이다: 모든 작업의 모든 메모리 효과는 `destroy`가 반환된 후 `destroy`를 호출한 스레드에 가시적이다. 따라서 결과를 수집하는 안전한 패턴은 "모두 submit → close → destroy → 결과 읽기"이다. 그 join 이전에 다른 스레드에서 작업의 출력을 읽으려면 여러분 자신의 동기화가 필요하다.

반례

```
/* WRONG: capacity must be a power of two. */
proven_job_system_init(alloc, 4, 100, &sys);   /* 100 is not 2^n */

/* WRONG: reading a result before the join makes it visible. */
int total = 0;
(void)proven_job_submit(sys, accumulate, &total);
proven_println("total = {}", PROVEN_ARG(total));  /* data race: job may still be running */
/* RIGHT: */
proven_job_system_close(sys);
proven_job_system_destroy(sys);                   /* joins workers -> writes now visible */
proven_println("total = {}", PROVEN_ARG(total));
```

```
/* WRONG: ignoring the submit result drops work silently when the ring is full. */
proven_job_submit(sys, work, arg);   /* [[nodiscard]]: a false return means NOT queued */
```

예제(스케치인 이유는 job routine이 자체 함수여야 하기 때문이다. 공유 카운터가 실제로 필요로 하는 원자 연산을 포함한 컴파일된 버전은 8절의 다루어진 예제, `manual/examples/ex_06_job.c`이다):

```
static void increment(void *arg) {
int *p = arg;
*p += 1;
}

proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);
if (!proven_is_ok(e)) return e;

int value = 0;
if (!proven_job_submit(sys, increment, &value)) {
proven_job_system_close(sys);
proven_job_system_destroy(sys);
return PROVEN_ERR_BUSY;
}

proven_job_system_close(sys);
proven_job_system_destroy(sys);
```

routine이 전혀 보이지 않는, 수명 주기 그 자체는 평범한 문장 코드이다:

```
proven_job_sys_t *sys = NULL;
proven_err_t e = proven_job_system_init(alloc, 2, 64, &sys);  /* capacity: power of two */
if (proven_is_ok(e)) {
    /* ... submit work here, from this thread or from producers you own ... */
    proven_job_system_close(sys);     /* every later submit now returns false */
    proven_job_system_destroy(sys);   /* drains the queue, joins the workers */
}
```

3. 스레드, allocator, 포인터 프로버넌스

이 절은 2장에서 여기로 옮겨 왔다. 매뉴얼에서 가장 어려운 내용이 독자가 가장 먼저 여는 장 중 하나에 놓여 있었고, 1–5장의 어떤 것도 여기에 의존하지 않는다. `proven` 객체를 스레드 간에 공유하고 있거나 그 위에 락프리 구조를 만들고 있다면 이 절이 필요한 절이다. 그렇지 않다면 통째로 건너뛰어도 되고, 이후에 아무 지장도 없다.

`proven_allocator_t` 트레이트는 `ctx` 하나에 함수 포인터 셋일 뿐이며, **자체적인 동기화를 전혀 담고 있지 않다.** 동시 사용이 안전한지는 전적으로 트레이트 뒤에 있는 구체 allocator에, 그리고 allocator가 만들어 낸 *객체*를 어떻게 공유하는지에 달려 있다. 이 절은 먼저 보장 사항을 밝히고, 그다음 더 깊은 포인터 프로버넌스 위험과 그 위에 동시성 구조를 만들 때 필요한 락프리 개념들을 설명한다.

3.1 무엇이 스레드 안전하고 무엇이 아닌가

Allocator	한 인스턴스의 동시 사용	이유
`proven_heap_allocator()`	동시 `alloc`/`realloc`/`free`에 **안전**	트레이트는 상태가 없다(`ctx == NULL`). 플랫폼의 `aligned_alloc`/`posix_memalign`/`_aligned_malloc`+`free`로 넘기며, 이들은 C11 이래 스레드 안전하다.
`proven_arena_t`	**안전하지 않음**	`proven_arena_alloc_*`는 `arena->offset`을 비원자적으로 읽고-수정하고-쓴다.
`proven_pool_t`	**안전하지 않음**	`proven_pool_*`는 프리리스트(`bin`, `bin_len`)를 비원자적으로 pop/push한다.

여기서 두 가지 규칙이 따라 나온다:

1. **"allocator 안전"은 "객체 안전"이 아니다.** 힙 allocator를 쓰더라도 그 위에 세워진 컨테이너들(`proven_u8str_t`, `proven_array_t`, `proven_map_t`, …)은 내부 잠금을 전혀 추가하지 않는다. 한 스레드가 append하는 동안 다른 스레드가 읽는 `proven_u8str_t`는 allocator가 얼마나 스레드 안전하든 데이터 경쟁이다. 공유 가변 객체에는 언제나 여러분의 동기화가 필요하다.
2. **Arena와 풀(pool)은 공유해서는 안 된다.** 동시 `proven_arena_alloc`은 `offset`을 찢어 놓아 *같은 바이트*를 두 스레드에 넘기거나 할당 하나를 통째로 잃어버릴 수 있고, 동시 pool pop/push는 같은 슬롯을 두 번 내주거나 `bin_len`을 언더플로시킬 수 있다. 데이터 경쟁은 그 자체로 미정의 동작이다 — 그 결과가 왜 "숫자가 하나 틀린 것"보다 나쁜지는 §7.3을 보라.

3.2 한 문단으로 보는 포인터 프로버넌스

C의 추상 기계에서 모든 포인터는 **프로버넌스**를 지닌다: 그 포인터가 유래한 저장소 인스턴스의 정체성이다. 여기서 중요한 규칙은 둘이다: (a) 포인터를 그 프로버넌스 객체의 수명이나 경계 바깥에서 사용하는 것은 미정의이며, (b) 최적화기는 **프로버넌스가 다른 포인터끼리는 앨리어싱하지 않는다**고 가정하고 그 근거로 메모리 접근을 재정렬하거나 제거해도 된다. 할당, 재할당, 해제는 모두 프로버넌스를 만들고 없앤다 — 바로 그래서 동기화되지 않은 공유와 나쁘게 상호작용한다.

3.3 스레드에서 할당 + 프로버넌스가 물어뜯는 지점

- **여기서 `realloc`은 항상 재배치한다.** 플랫폼의 `realloc`은 *새로 할당 + 복사 + 옛것 해제*이므로(정렬된 블록은 이식성 있게 제자리에서 크기를 바꿀 수 없다), 성공한 성장은 **항상** 새 프로버넌스를 가진 새 객체를 반환하고 옛 객체를 끝낸다. 옛 포인터를 남겨 둔 복사본은 무엇이든 — 캐시해 둔 원소 포인터, 빌려온 `proven_mem_view_t`/`proven_u8str_view_t` — 이제 댕글링이다. 단일 스레드에서는 성장을 넘어 뷰를 들고 있지 않음으로써 이를 피하지만, 스레드가 여럿이면 다른 스레드가 아무 순간에나 공유 컨테이너를 성장시킬 수 있고, 여러분의 뷰는 해제된 저장소를 가리키게 된다(규칙 (a): UB, 게다가 데이터 경쟁).
- **주소 재사용 / ABA.** `free` 후 `alloc`(특히 pool의 프리리스트 재활용)은 *같은 주소*를 다른 객체에 새 프로버넌스로 돌려줄 수 있다. 옛 포인터를 여전히 들고 있는 스레드는 옛 프로버넌스를 가정한다. 규칙 (b)에 따라 컴파일러는 바이트가 일치하더라도 둘을 앨리어싱하지 않는 것으로 취급할 수 있고, happens-before 간선이 없으면 새 객체에 대한 쓰기가 가시적일 필요도 없다. 같은 주소, 다른 프로버넌스 — 여전히 UB다.
- **`uintptr_t` 왕복 + 찢어진 읽기.** 아레나(arena)는 정렬 계산을 위해 `backing.ptr`을 정수로 변환한다. arena는 정수로부터 포인터를 날조하는 대신 원래 `backing.ptr`을 오프셋하여 결과 포인터를 유도하도록(그 프로버넌스를 보존하도록) 조심한다 — 프로버넌스 관점에서 올바른 기법이다. 하지만 `offset`이 경쟁 상황에서 찢어진 채로 읽히면, 계산된 포인터가 backing 객체 *바깥*에 착지할 수 있다. 즉 그 위치에 대한 유효한 프로버넌스가 없는 접근이 된다. 경쟁은 단지 틀린 오프셋이 아니라 그곳을 가리킬 권리가 없는 포인터를 만들어 낸다.
- **데이터 경쟁 × 프로버넌스 추론 = 잘못된 값이 아니라 미스컴파일.** 컴파일러는 각 스레드 안에서 단일 스레드적, 프로버넌스 기반의 비앨리어싱 추론을 적용하므로, 경쟁하는 allocator는 최적화기가 런타임에는 성립하지 않는 비앨리어싱을 "증명"하게 만들 수 있다 — 찢어진 포인터, 컴파일러가 겹칠 수 없다고 믿는 이중 할당, 사라진 저장이 그 결과다. 실패 양상은 어쩌다 나오는 값이 아니라 구조적이다.

3.4 락프리 도구 상자 (그 위에 동시성을 만든다면, 개념 정리)

`proven`은 락프리 allocator나 안전 메모리 회수(safe-memory-reclamation) 기법을 **전혀** 구현하지 않는다. 이 allocator들 위에 동시 자료구조를 만든다면 다음은 여러분이 직접 공급한다(`<stdatomic.h>`는 사용 가능하다 — 6장의 job system이 큐 인덱스에 이를 쓴다). 아래는 표준적인 조각들과 그것들이 위의 프로버넌스 위험과 어떻게 관계되는지다.

- **CAS (compare-and-swap).** 거의 모든 락프리 코드의 바탕이 되는 원자 기본 연산이다: `atomic_compare_exchange_strong(&p, &expected, desired)`는 `p == expected`일 때만 원자적으로 `p = desired`로 설정하고, 아니면 현재 값을 보고한다. 다른 스레드가 그 워드를 먼저 바꾸지 않았을 때만 한 스레드가 변경을 공표하게 해 준다.

```
text /* Treiber-stack style push (sketch): ex_node_t and n are illustrative, so this is a listing rather than a compiled block. */ _Atomic(ex_node_t *) head; ex_node_t *old = atomic_load(&head); do { n->next = old; } while (!atomic_compare_exchange_weak(&head, &old, n));
```

- **ABA 문제.** CAS는 *값*을 비교하지 이력을 비교하지 않는다. 락프리 pop이 `head == A`를 읽고 `A->next`를 설치할 계획을 세운 뒤 CAS를 한다. 그 사이에 다른 스레드들이 `A`를 pop하고, 그 후속을 pop하고, 그것들을 해제한 뒤, `A`를 다시 push하면(주소가 재사용되어), CAS는 여전히 `head == A`를 보고 성공한다 — 해제된 메모리를 가리키는 포인터를 설치하면서. 값은 일치했지만(A→B→A) 세상은 바뀌었다. pool의 프리리스트는 순진하게 락프리로 만들면 교과서적인 ABA 후보다.
- **태그 포인터 / 버전 카운터.** 단조 증가하는 태그를 포인터 옆에 담아 함께 CAS한다(더블 폭 CAS를 쓰거나, 낮은 정렬 비트나 상위 비트를 훔쳐서). 성공한 갱신마다 태그가 올라가므로 A→B→A 시퀀스는 다른 태그로 돌아오고 CAS는 실패한다 — ABA가 탐지된다. 비용/한계: 비트 훔치기는 정렬이 보장되어야 하고 쓸 수 있는 주소 범위를 줄인다. 전체 폭 태그는 하드웨어 더블 워드 CAS(예: `cmpxchg16b`)를 필요로 한다. 태그는 원리상 랩할 수 있다. **결정적으로, 태그는 공유 워드에서의 ABA *탐지*를 해결할 뿐 프로버넌스를 복원하지는 않는다.** 재사용된 주소는 여전히 새 객체이며, 태그는 낡은 관점에 따라 행동하는 것을 막아 줄 뿐이다.
- **Hazard pointer (안전 메모리 회수).** 각 스레드가 단일 기록자/다중 독자 "hazard" 슬롯을 몇 개 소유한다. 공유 포인터를 역참조하기 전에 그 포인터를 슬롯에 공표하고 다시 검증한다. 객체를 해제하려는 스레드는 먼저 모든 hazard 슬롯을 훑고, 누군가 보호 중이면 해제를 **미룬다**(retire 목록). 이는 메모리를 유계로 유지하고 use-after-free와 회수-ABA를 막아 주며, 대가는 보호되는 접근마다 저장 한 번 + 펜스 하나, 그리고 스레드당 동시에 보호할 수 있는 포인터 개수가 고정된다는 점이다.
- **에폭 기반 회수(EBR).** 전역 에폭 카운터를 둔다. 스레드는 공유 구조를 건드리는 동안 현재 에폭을 "핀"하고, 은퇴시킨 메모리에는 은퇴한 에폭을 태그한다. 에폭 *e*에서 은퇴한 메모리는 모든 스레드가 *e*를 지난 것이 관찰된 뒤에야 해제된다(에폭 두어 개 분량의 유예 기간). 연산당 비용은 hazard pointer보다 싸지만(핀 플래그 하나뿐), 핀한 채로 멈춰 버린 스레드는 모든 회수를 막는다 — 무한한 메모리 증가. 변종: 정지 상태 기반(QSBR)과 구간 기반 회수.
- **이것들이 프로버넌스와 어떻게 이어지는가.** 회수 기법(hazard pointer, EBR)은 정확히 해제된 객체의 저장소를 살려 두기 위해 — 그 프로버넌스를 유효하게 유지하기 위해 — 존재한다. 어떤 스레드도 더는 참조할 수 없을 때까지 말이다. CAS + 태그는 공유 *워드*를 일관되게 유지하지만 그것이 가리키는 대상의 수명에 대해서는 아무 말도 하지 않는다. 그래서 올바른 락프리 스택은 보통 **둘 다** 필요하다: 태그(head 워드의 ABA용) 그리고 SMR 기법(pop된 노드의 안전한 회수용). `proven`의 pool이나 arena는 둘 중 아무것도 주지 않으므로, 동시성은 그것들 위에 층으로 쌓아야지 가정해서는 안 된다.

3.5 `proven`에서의 안전한 패턴

- **스레드별 arena/pool.** 각 스레드에 자신의 `proven_arena_t` / `proven_pool_t`를 준다. 공유가 없으면 경쟁도 없고 스레드 간 프로버넌스도 없다 — 가장 단순하게 올바른 설계이며, 보통 가장 빠르기도 하다.
- **스레드 간 alloc/free에는 힙을 쓰되, 객체는 동기화하라.** 스레드 A가 할당하고 스레드 B가 `proven_heap_allocator()`로 해제하는 것은 괜찮다. 하지만 둘이 잠금 없이 같은 `proven_array_t`/`proven_u8str_t`를 변경하는 것은 괜찮지 않다.
- **happens-before 간선과 함께 소유권을 넘겨라.** 포인터를 다른 스레드로 넘길 때는 뮤텍스, release/acquire 순서를 가진 원자 연산, 또는 (job system 같은) 큐를 통해서 넘겨, 한 번에 한 스레드만 그것을 관찰하게 하라. 이렇게 하면 각 할당의 프로버넌스가 한 스레드 안에 갇히고, 생산자의 쓰기가 소비자에게 가시화된다.
- **빌려온 뷰를 스레드 간에 넘기지 말라.** 빌림이 지속되는 내내 소유자가 성장/이동/해제 하지 않는다고 보장되지 않는 한 그렇다 — 그리고 여기서 성장은 항상 재배치임을 기억하라.
- **꼭 arena/pool을 공유해야 한다면 감싸라.** 여러분 자신의 뮤텍스로 감싸거나(또는 §7.4의 도구로 진짜 락프리 allocator를 만들거나) 하라. 내장된 것들은 한 번에 한 소유자를 가정한다.

4. 별칭 계층

왜 두 번째 이름 집합이 존재하는가

`proven_u8str_view_slice`는 26자다. 그런 함수를 스무 번 호출하는 파일에서 접두사는 한 줄의 3분의 1을 차지하고, 정작 달라지는 부분 — 여러분이 실제로 읽는 부분 — 은 남은 자리에 눌려 들어간다.

C에서 접두사는 장식이 아니다. 이 언어에는 함수를 위한 전역 이름 공간이 하나뿐이므로, 슬라이스(slice)를 뜻하는 `slice`나 `create`를 내보내는 라이브러리는 언젠가 프로그램이 링크하는 다른 무언가와 충돌하게 된다. 접두사는 C 라이브러리를 다른 것들과 안전하게 조합할 수 있게 해 주는 장치이며, 진지한 C 라이브러리마다 접두사가 있는 이유다.

`alias_xcv.h`는 그 타협안이다: 모든 공개 심벌에 대한 더 짧은 `xcv_` 표기를, 여러분이 선택해서 포함하는 헤더에 담았다. 포함하면 `xcv_u8str_view_slice`가 동작하고, 포함하지 않으면 짧은 이름은 아예 존재하지 않으므로 무엇과도 충돌하지 않는다.

정규(canonical) 이름은 여전히 진실의 원천으로 남는다 — ABI에 대해서도, 이 매뉴얼에 대해서도, 테스트에 대해서도 그렇다. 별칭 계층은 표기이지 API가 아니다: 긴 이름으로 내보내지 않은 것은 짧은 이름으로도 내보내지 않으며, 완전성 게이트(`tests/test_docs_alias_completeness.c`)는 공개 함수가 별칭 없이 추가되면 빌드를 실패시킨다. 구멍 난 별칭 계층은 아예 없느니만 못하기 때문이다 — 그것을 채택한 호출자는 컴파일 오류를 하나씩 만나며 그 구멍들을 발견하게 된다.

잘못됨 — 헤더를 포함하지 않고 별칭을 기대하기:

```
#include "proven.h"
xcv_u8str_view_t v;   /* wrong: proven.h does not pull in the alias layer */
```

잘못됨 — 한 파일에서 같은 개념에 두 표기를 섞기:

```
proven_result_u8str_t s = xcv_u8str_create(alloc, 32);   /* wrong: pick one and stay with it */
```

둘 다 컴파일된다. 둘 다 좋은 생각은 아니다: 이제 독자는 함수 하나를 따라가기 위해 두 어휘를 모두 알아야 한다.

정규 헤더 다음에 포함하라:

```
#include "proven.h"
#include "proven/alias_xcv.h"

xcv_allocator_t heap = xcv_heap_allocator();
(void)heap;
xcv_println("{}", XCV_ARG(123));
```

별칭 설계 규칙:

- 별칭은 전처리기 편의 기능이지 별도의 ABI가 아니다.
- 정규 API가 진실의 원천으로 남는다.
- 프로젝트가 자체적인 더 짧은 로컬 접두사를 원한다면 `alias_xcv.h`를 템플릿으로 취급하라.
- 공개 API 이름이 추가되거나 제거될 때 별칭 테스트를 동기화 상태로 유지하라.

별칭 매크로 계열:

- 포매팅: `XCV_ARG`, `XCV_ARG_CSTR`, `XCV_ARG_CSTR_N`, `XCV_FMT_IS_OK`.
- 컨테이너: `XCV_ARRAY_*`, `XCV_RING_*`, `XCV_MAP_*`, `XCV_LIST_*`.
- 오류 및 산술: `XCV_ERR_*`, `XCV_IS_OK`, `XCV_CKD_*`.
- 코루틴: `XCV_CORO_*`.
- 파일시스템 및 sysio: `xcv_fs_*`, `xcv_print*`, `xcv_scan*` 별칭.
- 타입 및 allocator: `xcv_*_t` 타입 별칭과 allocator 별칭.

소스에 근거한 완전한 별칭 테이블은 7장: 별칭 인덱스를 참조하라.

5. PAL 계약

왜 syscall을 격리했는가

운영체제를 건드리는 모든 라이브러리는 OS별 코드를 어디에 둘지 결정해야 한다. 흔한 답은 "필요한 곳마다"이고, 그러면 소스 곳곳에 `#ifdef _WIN32`가 흩뿌려진다. 그것도 동작은 하지만, 누적되는 두 가지 비용이 있다: 이 라이브러리가 OS에 실제로 무엇을 요구하는지 아무도 알 수 없고, 이식은 모든 파일을 감사한다는 뜻이 된다.

이 라이브러리는 그 전부를 한 디렉터리에 넣는다. **`platform/`은 syscall을 하는 유일한 장소다.** `src/proven/` 안의 모든 것은 PAL을 통해 호출하는 이식 가능한 C이며, 그 결과:

- **요구 사항 목록이 곧 파일 목록이다.** 이 라이브러리가 운영체제에 요구하는 것은 정확히 `proven_sys_*` 함수들의 집합이다 — 숨은 것도 없고, 뒤늦게 발견될 것도 없다.
- **이식의 범위가 유계다.** 새 대상은 `platform/`을 다시 구현한다. `src/proven/` 안의 것은 아무것도 바뀌지 않고, 이식 가능한 절반을 검사하는 테스트들은 계속 통과한다.
- **Freestanding은 특수 사례가 아니라 같은 메커니즘이다.** 호스티드 PAL 파일 없이 빌드하면 남는 것이 이식 가능한 코어다. freestanding 가이드가 별도의 이식 작업이 아니라 어떤 파일을 빼는지의 목록인 이유가 그것이다.

PAL은 **내부**다. `proven_sys_*` 함수는 공개 API가 아니며 시그니처가 바뀔 수 있다. 여러분이 호출할 것은 `fs.h`, `sysio.h`, `time.h`, `random.h`의 공개 래퍼다. 심벌 게이트도 이를 알고 있어서 PAL을 "문서화되어야 한다"는 요구에서 면제한다. 바로 그것이 여러분이 대상으로 프로그래밍할 표면이 아니기 때문이다.

잘못됨 — 애플리케이션 코드에서 PAL을 직접 호출하기:

```
proven_sys_fs_open(path, flags);   /* wrong: internal. Use proven_fs_open. */
```

오류 변환, 인자 검증, 그리고 다음 릴리스에서도 그 호출이 계속 동작한다는 보장을 모두 잃는다.

PAL 영역:

- `proven_sys_mem`: 힙 할당 백엔드.
- `proven_sys_fs`: 파일, 디렉터리, 경로, 링크, 권한, 잠금.
- `proven_sys_time`: 벽시계, 단조 시계, 슬립, 시간 분해.
- `proven_sys_net`: 소켓, 준비 상태, 이름 해석.
- `proven_sys_env`: 환경 변수.
- `proven_sys_thread`: job system을 위한 스레드와 동기화.
- `proven_sys_io`: 표준 스트림 I/O.
- `proven_sys_math`: 필요한 경우의 수학 헬퍼.

공개 애플리케이션 코드는 고수준 API를 선호해야 한다:

- 파일시스템 연산에는 `proven_fs_*`.
- 콘솔 I/O에는 `proven_sysio_*`와 print/scan 매크로.
- 시간에는 `proven_time_*`.
- 스레드 작업에는 `proven_job_*`.

PAL 내부 예외: 스레드 수명 주기 코드는 불투명한 OS 메타데이터를 진입점을 넘어 살려 두기 위해 내부 플랫폼 할당이 필요할 수 있다. 이 예외는 PAL 코드 안에 머물러야 하며 핵심 컨테이너 API로 새어 나가서는 안 된다.

6. Freestanding 부분집합

`PROVEN_FREESTANDING`는 OS가 없거나 libc가 최소인 대상을 위한 축소된 부분집합을 빌드한다. 현재 freestanding 빌드는 또한 다음을 정의한다:

```
-DPROVEN_FREESTANDING -DPROVEN_FMT_NO_FLOAT -DPROVEN_NO_U16STR -ffreestanding
```

현재 freestanding 구성에서 사용 가능한 모듈:

- `types`
- `error`
- `memory`
- `align`
- `allocator`
- `arena`
- `pool`
- `buffer`

- array
- list
- ring
- map
- algorithm
- u8str
- coro
- scan
- 부동소수점 포매팅이 없는 fmt
- panic — 패닉(panic) 처리

제외되거나 스텁 처리된 모듈:

- heap — 힙(heap): 컴파일되지만, proven_heap_allocator()는 유효하지 않은 0 allocator를 반환한다.
- u16str: 현재 freestanding 빌드에서 PROVEN_NO_U16STR에 의해 제외됨.
- time: 부분적. src/proven/time.c는 컴파일되므로 proven_time_breakdown()과 datetime 포매터는 사용 가능하지만, 클럭 백엔드 (platform/proven_sys_time.c)는 포함되지 않는다 — 따라서 proven_time_now(), proven_time_now_datetime(), proven_time_sleep()은 링크할 구현이 없다.
- fs, sysio, mmap, job, net: 호스티드/PAL 서비스로, 현재 freestanding 부분집합에서 제외됨. 호스티드 빌드에서도 -DPROVEN_NO_NET으로 net을 뺄 수 있다.

정확한 소스 목록과 명령 예제는 manual-freestanding-ko.md를 참조하라.

7. 크로스 컴파일

왜 실행할 수도 없는 대상용으로 빌드하는가
이식 가능하다고 주장하는 라이브러리는 조용히 썩어 가는 주장을 하고 있는 것이다. 포인터가 64비트라고, char가 부호 있다고, 정렬되지 않은 로드가 괜찮다고 가정한 코드는 그것이 작성된 기계에서는 완벽하게 컴파일되고 여섯 달 뒤 ARM 보드에서 깨진다 — 그리고 그것을 깨뜨린 커밋은 무해해 보였다.

./nob cross는 아무것도 실행하지 않고 매트릭스의 모든 대상에 대해 라이브러리를 컴파일한다. 설정된 MinGW 경로는 smoke 실행 파일을 추가로 링크한다. 약해 보이지만 그렇지 않다: **대부분의 이식성 실패는 컴파일 타임 실패다.** 가정한 폭이 아닌 타입, 없는 인트린식, 대상이 실제로 강제하는 정렬 요구, 거기에는 존재하지 않는 헤더 — 전부 컴파일러에서 실패한다. 이 기계에서, 몇 초 만에, 그것을 도입한 커밋에서.

컴파일 전용 검사가 잡을 수 없는 것은 동작이다: 엔디안 버그, 런타임에만 일어나는 정렬 폴트, 그리고 타이밍에 의존하는 모든 것. 그것들에는 실제 하드웨어나 에뮬레이터가 필요하고, 이 매트릭스는 그런 척하지 않는다. 이것은 이식성 테스트의 값싼 절반이며, 비싼 절반을 결코 하지 않는 대신 모든 빌드에서 돌아간다.

크로스 빌드는 또한 freestanding 프로파일이 실제 대상 — 그중에도 Cortex-M, RISC-V, wasm32 — 에 대해 실전으로 검증되는 곳이다. 그래서 "운영체제 없음"이라는 주장은 가이드의 한 문단이 아니라 컴파일러가 검사한다.

잘못됨 — 크로스 매트릭스가 초록색이라고 해서 라이브러리가 그 대상에서 동작한다는 증거로 취급하기:

```
/* wrong: ./nob cross compiles and links objects. It does not execute anything.
   Endianness, alignment faults and real timing still need the hardware. */
```

빌드 드라이버 명령은 다음과 같다:

```
./nob cross -build-root build-out/proven_c_lib
```

현재 대상 범주:

- 네이티브 GCC 호스티드.
- 사용 가능한 경우 네이티브 Clang 호스티드.
- Linux AArch64 호스티드 컴파일 전용.
- Linux ARM 하드플로트 호스티드 컴파일 전용.
- i686-linux-gnu-gcc 또는 gcc -m32를 통한 Linux i686 호스티드 컴파일 전용.

- MinGW를 통한 Windows x86_64 및 i686 WinAPI 컴파일·링크 smoke 검사.
- ARM Cortex-M freestanding 컴파일 전용.
- RISC-V ELF freestanding 컴파일 전용.

규칙:

- 없는 선택적 컴파일러는 건너뛴다.
- 실제 컴파일 오류는 명령을 실패시킨다.
- 크로스 빌드는 설정된 MinGW 컴파일·링크 smoke를 제외하면 컴파일 전용이며, 어느 경로도 런타임 검증은 아니다.
- 런타임 동작은 여전히 대상 러너, 에뮬레이터, 장치, 또는 OS 환경이 필요하다.

8. 예제와 오용 사례

코루틴 매크로는 하나의 소스 라인을 공유해서는 안 된다

잘못됨:

```
PROVEN_CORO_YIELD(&c->coro); PROVEN_CORO_YIELD(&c->coro);
/* wrong: both use the same __LINE__ value */
```

올바름:

```
PROVEN_CORO_YIELD(&c->coro);
PROVEN_CORO_YIELD(&c->coro);
```

코루틴 지역 변수는 평범한 지역 변수이다

스택리스 코루틴은 호출자에게 반환된다. yield를 넘어 값이 유지되어야 하는 지역 변수는 코루틴 함수 안의 자동 지역 변수가 아니라 코루틴 상태 객체 안에 있어야 한다.

잘못됨:

```
static proven_i32 next(Task *t) {
int temporary = 0;
PROVEN_CORO_BEGIN(&t->coro);
temporary = 42;
PROVEN_CORO_YIELD(&t->coro);
use_int(temporary); /* wrong assumption: ordinary local lifetime/state is not persistent */
PROVEN_CORO_END(&t->coro);
}
```

올바름:

```
typedef struct Task {
proven_coro_t coro;
int temporary;
} Task;
```

Job destroy는 제출자와 경쟁해서는 안 된다

잘못됨:

```
/* thread A */
proven_job_system_destroy(sys);

/* thread B at the same time */
proven_job_submit(sys, work, arg); /* wrong: external synchronization missing */
```

올바름:

```
proven_job_system_close(sys); /* submit과 경쟁할 수 있으며 submit을 false로 만든다 */
join_producer_threads();      /* 이 지점 뒤에는 어떤 생산자도 sys에 접근하지 않는다 */
proven_job_system_destroy(sys);
```

Job 인자 수명

잘못됨:

```
void submit_bad(proven_job_sys_t *sys) {
int value = 10;
proven_job_submit(sys, work, &value);
} /* wrong: value may be dead before work runs */
```

올바름:

```
JobData *data = allocate_job_data();
proven_job_submit(sys, work_and_free_data, data);
```

별칭 계층은 정규 문서를 가려서는 안 된다

잘못됨:

```
/* Document only xcv_map_t and forget proven_map_t. */
```

올바름: 정규 `proven_` API를 먼저 문서화한 다음, 별칭을 선택적 로컬 표기로 언급하라.

다루어진 예제: 경계가 있는 job system

테스트 스위트가 컴파일하고 실행한다. 계약이 요구하는 순서에 주목하라: submit, 그다음 close, 그다음 destroy. `proven_job_system_destroy`는 `proven_job_submit`과 경쟁해서는 안 된다.

```
#include <stdatomic.h>

/*
 * 작업 시스템: 일꾼 스레드와 경계가 있는 원자적 MPMC 큐. 놀고 있는 일꾼은 플랫폼
 * 동기화에 세워 둔다. 이것이 순서를 정해 주는 것은 일의 *넘김*이지, 그 일이 건드리는
 * 자료가 아니다. 아래 계수기가 그냥 int 가 아니라 atomic 인 이유가 그것이다 - 두 작업이
 * 같은 변수를 올리는 것은 부르는 쪽이 달리 말하지 않는 한 자료 경합이다.
 *
 * 수명은 곧은 한 줄이고, 선택 사항이 아니다.
 *
 *     init -> submit... -> close -> destroy
 *
 * destroy 는 submit 과 경합해서는 안 된다. 라이브러리 안의 어느 것도 그것을 강제하지
 * 않는다. 부르는 쪽이 먼저 생산자를 멈춰야 한다. 여기서는 생산자가 이 스레드 하나뿐이라,
 * "생산자를 멈춘다" 는 것은 "닫기 전에 submit 반복문을 끝낸다" 는 뜻이다.
 */

#define JOB_COUNT 64

static void increment(void *arg) {
    atomic_int *counter = arg;
    /* relaxed 로 충분하다. 우리는 합계가 맞기만 하면 되지, 그것에 맞춰 무엇의 순서를
     * 정할 필요가 없다. 결과를 우리에게 공표하는 것은 destroy 안의 join 이다. */
    atomic_fetch_add_explicit(counter, 1, memory_order_relaxed);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    proven_job_sys_t *sys = NULL;
    /* 큐 용량은 2의 거듭제곱이어야 한다 - 고리가 마스크로 순번을 칸에 대응시킨다.
     * JOB_COUNT 보다 크게 잡아, 일꾼이 모두 아직 뜨는 중이어도 submit 이 고리가 가득 찬
     * 것을 만나지 않게 했다. */
    proven_err_t err = proven_job_system_init(alloc, 4, 128, &sys);
    EXAMPLE_REQUIRE(proven_is_ok(err), "starting a job system with 4 workers should succeed");
    if (!proven_is_ok(err)) return 1;

    /* destroy 뒤까지 산다. 작업의 인자는 작업보다 오래 살아야 하고, 작업은 destroy 가
     * 큐를 비울 때까지 돈다. */
    atomic_int counter = 0;
```

```
    proven_size_t submitted = 0;
    for (proven_size_t i = 0; i < JOB_COUNT; ++i) {
        /* submit 은 고리가 가득 찼거나 시스템이 닫혔을 때 false 를 돌려준다. 큐에 자리가
         * 나기를 기다리는 일도, 일을 조용히 버리는 일도 없다. 깨우는 경로에서 잠깐 플랫폼
         * 동기화에 들어갈 수는 있다. 그 답을 무시하는 것이 작업을 잃는 방법이고, 그래서
         * [[nodiscard]] 가 붙어 있다. */
        if (!proven_job_submit(sys, increment, &counter)) {
            /* 실제 부르는 쪽이라면 물러났다 다시 시도하거나 proven_job_execute_one 으로
             * 그 자리에서 돌릴 것이다. 여기서 고리가 가득 찼다는 것은 위의 크기 잡기가
             * 틀렸다는 뜻이므로, 덮어 두지 말고 그렇다고 말한다. */
            EXAMPLE_REQUIRE(false, "the queue was sized to hold every job");
            break;
        }
        ++submitted;
    }

    /* 이 스레드가 유일한 생산자이고 submit 을 마쳤다 - 그러니 닫아도 안전하다. close 는
     * 그 뒤의 submit 을 모두 실패시킨다. 이미 줄에 선 작업은 그대로 돈다. */
    proven_job_system_close(sys);

    /* destroy 는 큐가 비고 모든 일꾼이 join 될 때까지 막는다. 그 join 이 동기화 지점이다.
     * destroy 가 돌아온 뒤에는 모든 작업의 모든 기억 효과가 이 스레드에 보인다. 이 줄
     * 앞에서 `counter` 를 읽는 것은 일꾼들이 아직 쓰고 있는 값을 읽는 것이다. */
    proven_job_system_destroy(sys);

    int ran = atomic_load(&counter);
    EXAMPLE_REQUIRE(submitted == JOB_COUNT, "every job should have been accepted");
    EXAMPLE_REQUIRE(ran == (int)submitted, "every submitted job should have run exactly once");

    printf("submitted %zu jobs, %d ran\n", (size_t)submitted, ran);

    return EXAMPLE_OK();
}
```

다루어진 예제: 스택리스 코루틴

테스트 스위트가 컴파일하고 실행한다. 모두를 잡는 규칙: 지역 변수는 yield를 넘어 살아남지 못하므로, 코루틴이 서스펜션을 넘어 필요로 하는 모든 상태는 자신의 구조체 안에 있어야 한다.

```
/*
 * 스택 없는 코루틴은 사실 switch 문이다. BEGIN 이 저장된 상태를 두고 switch 를 열고,
 * YIELD 마다 __LINE__ 을 재개 표지로 적고 *돌아간다*. 다음 호출은 함수를 위에서부터 다시
 * 들어와 그 표지로 곧장 뛴다.
 *
 * 뒤따르는 모든 것이 그 사실 하나에서 나온다.
 *
 *   - 지역 변수는 yield 를 넘겨 살아남지 *못한다*. 함수가 돌아갔고 그 스택 프레임은
 *     사라졌다. 남아 있어야 할 것은 코루틴 자신의 구조체에 산다 - 아래의 `value` 와
 *     `remaining` 이 하는 일이 그것이다.
 *   - 코루틴 매크로 둘이 같은 소스 줄에 있으면 안 된다(__LINE__ 이 부딪친다).
 *   - 자기가 부른 도우미 안에서 yield 할 수 없다. 멈춰 둘 스택이 없기 때문이다.
 *
 * 그 대가로, 멈춰 있는 코루틴은 정확히 자기 구조체만큼의 값이 든다 - 상태 4바이트에
 * 여러분이 곁에 둔 것 - 그리고 스레드도, 스택도, 문맥 전환도 없다.
 */

typedef struct {
    proven_coro_t coro;
    /* 생성기의 상태. 보통의 반복문이라면 `int i` 같은 지역 변수였을 것들이다. 여기서는
     * 필드여야 한다. 아니면 재개할 때마다 처음 값으로 되돌아가 반복문이 끝나지 않는다. */
    int value;
```

```
    int remaining;
} ex_squares_t;

/* 코루틴은 proven_i32 를 돌려준다. 0 = 멈췄음(다시 불러 달라), 1 = 끝. */
static proven_i32 squares_next(ex_squares_t *g) {
    PROVEN_CORO_BEGIN(&g->coro);

    g->remaining = 5;
    g->value = 1;

    while (g->remaining > 0) {
        g->value = g->value * g->value;
        PROVEN_CORO_YIELD(&g->coro);      /* 부르는 쪽이 여기서 g->value 를 읽는다 */
        g->value = g->value + 1;          /* 정확히 이 줄에서 재개한다 */
        g->remaining -= 1;
    }

    PROVEN_CORO_END(&g->coro);
}

int main(void) {
    proven_allocator_t alloc = proven_heap_allocator();

    /* 코루틴은 기억을 소유하지 않으므로 지울 것이 없다 - 다만 그것이 내놓는 값들은
     * 어딘가로 가야 하고, 그 문자열에는 소유자가 있다. */
    proven_result_u8str_t out = proven_u8str_create(alloc, 32);
    EXAMPLE_REQUIRE(proven_is_ok(out.err), "creating the output string should succeed");
    if (!proven_is_ok(out.err)) return 1;

    ex_squares_t gen = {0};
    PROVEN_CORO_INIT(&gen.coro);   /* 조건 없이, 첫 호출 전에, 정확히 한 번 */

    int produced = 0;
    int last = 0;

    /* 끝까지 굴린다. squares_next 는 본문의 끝을 지나 달리는 호출에서 1 을 돌려준다 -
     * 그 호출은 값을 내놓지 않으므로, 반복문 본문은 0 을 돌려준 동안에만 돈다. */
    while (!squares_next(&gen)) {
        proven_fmt_result_t r = proven_u8str_append_fmt_grow(alloc, &out.value, "{} ",
                                                             PROVEN_ARG(gen.value));
        EXAMPLE_REQUIRE(PROVEN_FMT_IS_OK(r), "appending a generated value should succeed");
        last = gen.value;
        ++produced;
    }

    /* 끝난 상태는 들러붙는다. 상태는 -1 이고 그대로 있다. 다시 불러도 본문을 다시 돌리지
     * 않고 1 만 돌려준다. */
    EXAMPLE_REQUIRE(PROVEN_CORO_IS_DONE(&gen.coro), "the generator should have finished");
    EXAMPLE_REQUIRE(squares_next(&gen) == 1, "a finished coroutine stays finished");

    /* 1, 그다음 (1+1)^2 = 4, 그다음 (4+1)^2 = 25, 그다음 676, 그다음 458329. */
    EXAMPLE_REQUIRE(produced == 5, "the generator yields once per iteration");
    EXAMPLE_REQUIRE(last == 458329, "the state carried across every yield");
    EXAMPLE_REQUIRE(proven_u8str_view_eq(proven_u8str_as_view(&out.value),
                                         PROVEN_LIT("1 4 25 676 458329 ")),
                    "the generated sequence should be exactly this");

    printf("squares: %s\n", proven_u8str_as_cstr(&out.value));

    proven_u8str_destroy(alloc, &out.value);
```

```
    return EXAMPLE_OK();
}
```

Proven Freestanding 모드 (v0.92.0)

6부 — 더 나아가기. 선행 조건: 2–5부, 그리고 6장. 이 가이드를 마치면 운영체제도 libc도 없는 대상용으로 이 라이브러리를 빌드할 수 있고, 그 상황에서 어떤 모듈이 살아남고 어떤 모듈이 살아남지 못하는지 정확히 알게 된다.

이 가이드는 `nob.c`와 공개 헤더가 구현한 현재의 `PROVEN_FREESTANDING` 구성을 설명한다.

이 가이드의 생김새에 대한 한마디. 다른 장들과 달리 이 가이드의 대략 절반은 절차다 — 플래그 목록, 파일 목록, 그리고 컴파일 명령 두 개. 그 절들은 의도적으로 짧다: 길게 설명된 컴파일 명령은 아무도 읽지 않는 컴파일 명령이다. *결정*을 담은 절들 — 이 모드가 존재하는 이유(§0), "제외됨"이 정말 무슨 뜻인지(§3), 여러분의 패닉(panic) 핸들러가 무엇을 해야 하는지(§5), 부동소수점을 뺌으로써 무엇을 잃는지(§8), 호스티드 호출이 왜 링크 타임에 실패하는지(§9), 수명 규칙이 왜 여기서 더 중요한지(§10), 그리고 이 가이드의 주장들이 어떻게 검증되는지(§11) — 은 온전히 풀어 썼다.

0. 이 모드가 존재하는 이유, 그리고 라이브러리 전체가 이 제약에 의해 빚어진 이유

Freestanding 모드는 펌웨어, 커널, 부트로더, 하이퍼바이저, 그리고 그 밑에 운영체제가 없는 그 밖의 모든 곳을 위한 것이다 — `malloc`도, `open`도, `printf`도 없고, 흔히 `libc` 자체가 없다.

C 표준에는 이를 가리키는 이름이 있다. **호스티드(hosted)** 구현은 표준 라이브러리 전체를 주고 프로그램을 `main`에서 시작한다. **Freestanding** 구현은 소수의 헤더만을 보장한다 — `<stddef.h>`, `<stdint.h>`, `<limits.h>`와 몇 가지 더 — 그리고 이것이 베어메탈을 대상으로 하는 컴파일러가 주는 것이다. `-ffreestanding`은 컴파일러에게 정확히 그렇게 가정하라고 알린다.

대부분의 C 라이브러리는 여기서 아예 쓸 수 없으며, 그 이유가 거창한 경우는 드물다. 세 단계 아래 호출에 묻혀 있는 `malloc` 하나, 오류 경로의 `printf` 하나, 또는 결코 실행되지 않는 무언가에 의해 초기화되는 전역 하나다. 호스트에 대한 숨은 의존성 하나면 마이크로컨트롤러에서 라이브러리를 쓸 수 없게 만들기에 충분하다.

이것이 이 라이브러리의 설계 결정 대부분을 낳은 제약이며, 그 연결을 보아 둘 가치가 있다. 그렇지 않으면 취향처럼 보이는 선택들을 설명해 주기 때문이다:

설계 결정	앞선 장에서는 이렇게 읽힌다	그리고 진짜 이유는
Allocator는 매개변수다 (2장)	"전략을 갈아 끼울 수 있도록"	여기에는 `malloc`이 없다. `static` 배열 위의 arena가 유일한 allocator이며, allocator를 매개변수로 받던 코드는 이미 그대로 동작한다.
오류는 반환되는 값이다 (1장)	"무시할 수 없도록"	여기에는 `errno`가 없고 풀어낼 스택도 없다. 반환값이 존재하는 유일한 메커니즘이다.
Panic은 훅을 거친다 (1장 §6)	"오버라이드할 수 있도록"	여기에는 `abort()`도 `stderr`도 없다. 기본 동작이 trap인 것은 이식성 있게 할 수 있는 전부가 그것이기 때문이다.
Syscall은 `platform/`에만 산다	"관심사의 분리"	새 대상을 위해 교체해야 하는 유일한 디렉터리다. `src/proven/` 안의 모든 것은 구성상 이식 가능하다.
C 문자열 대신 뷰 (3장)	"길이를 잃어버리지 않도록"	`strlen`은 libc다. 뷰는 자기 길이를 함께 가지고 다니며 아무것도 필요로 하지 않는다.

달리 말하면: 프리스탠딩(freestanding) 지원은 옆에 덧붙인 기능이 아니다. 이 매뉴얼의 나머지는 여기서 살아남도록 지어진 라이브러리를 계속 설명해 온 것이고, 이 가이드는 무엇이 살아남고 무엇이 살아남지 못하는지 알아내는 곳이다.

무엇을 포기하는가
운영체제를 필요로 하는 모든 것, 그리고 그 밖에는 아무것도 아니다:

- **힙 없음.** `proven_heap_allocator`는 존재하지 않는다. 여러분이 메모리를 공급하고 — 보통은 `static` 배열이다 — 그 위에 아레나(arena)나 풀(pool)을 얹는다.
- **파일시스템 없음, 표준 스트림 없음, 클럭 없음, OS 난수 없음.** `fs.h`, `sysio.h`, `mmap.h`, `time.h`, 그리고 OS 엔트로피 소스는 모두 호스티드 서비스다. `proven_chacha_rng_t`는 여러분이 직접 시드를 주면 여전히 동작하며, `proven_random_set_source`가 그것을 위한 것이다.
- **기본적으로 부동소수점 포매팅 없음.** `PROVEN_FMT_NO_FLOAT`가 그것을 뺀다. 부동소수점 경로는 포매터에서 가장 큰 코드 덩어리이고 대부분의 펌웨어는 `double`을 출력할 일이 없기 때문이다. 다시 켜는 방법은 8절이 설명한다.
- **UTF-16 문자열 없음.** `PROVEN_NO_U16STR`가 켜져 있다. 여기에는 대화할 Windows API가 없다.

여러분이 유지하는 것은 이식 가능한 코어 전체다: 오류, 뷰, 검사된 산술, 정렬, arena, pool, 버퍼, 문자열, 배열, 맵, 리스트, 링, 정렬, 검색, 해싱, 인코딩, 코루틴, 그리고 정수 포매팅과 스캐닝.

여전히 물어뜯는 것
대상이 작아졌다고 해서 수명 규칙이 느슨해지지는 않는다 — 오히려 더 중요해진다. 실수를 알아챌 운영체제도 없고, 여러분이 아직 들고 있는 메모리를 재사용하지 않으려 해 줄 할당자(allocator)도 없기 때문이다. 10절은 짧고, 사람들이 건너뛰는 절이다.

1. 빌드 프로파일

빌드 드라이버는 다음 freestanding 플래그를 사용한다:

```
-std=c23
-ffreestanding
-DPROVEN_FREESTANDING
-DPROVEN_FMT_NO_FLOAT
-DPROVEN_NO_U16STR
```

드라이버는 여전히 `-std=c23`을 먼저 탐지하고, 컴파일러가 과도기 표기를 사용하는 경우 `-std=c2x`로 폴백한다. 빌드 래퍼가 두 플래그 표기 중 어느 쪽도 받아들이더라도 freestanding 소스 계약은 C23 우선으로 유지된다.

호스티드 `./nob freestanding` 명령은 또한 로컬 freestanding 테스트 실행 파일을 빌드 호스트에서 정적으로 링크한다. `./nob cross`의 크로스 freestanding 대상은 컴파일 전용 오브젝트 검사를 사용한다.

프로젝트 freestanding 검사를 실행하라:

```
cc nob.c -o nob
./nob freestanding -build-root build-out/proven_c_lib
```

크로스 빌드 매트릭스를 실행하라:

```
./nob cross -build-root build-out/proven_c_lib
```

2. 사용 가능한 소스 파일

현재 freestanding 빌드에서 `nob.c`는 호스티드 전용 모듈을 제외한 이식 가능한 소스 파일을 컴파일한다.

포함된 핵심 모듈:

```
src/proven/memory.c
src/proven/arena.c
src/proven/pool.c
src/proven/buffer.c
src/proven/heap.c
src/proven/u8str.c
src/proven/array.c
src/proven/ring.c
src/proven/map.c
```

```
src/proven/algorithm.c
src/proven/hash.c
src/proven/hash_legacy.c
src/proven/hmac.c
src/proven/utf.c
src/proven/url.c
src/proven/http.c
src/proven/http_auth.c
src/proven/http_cookie.c
src/proven/sse.c
src/proven/ws.c
src/proven/encode.c
src/proven/random.c
src/proven/float_decimal.c
src/proven/float_parse.c
src/proven/float_format.c
src/proven/time.c
src/proven/fmt.c
src/proven/scan.c
src/proven/panic.c
platform/proven_sys_math.c
```

제외된 호스티드 모듈:

```
src/proven/u16str.c
src/proven/fs.c
src/proven/stream.c
src/proven/sysio.c
src/proven/mmap.c
src/proven/job.c
platform/proven_sys_fs.c
platform/proven_sys_thread.c
platform/proven_sys_io.c
platform/proven_sys_env.c
platform/proven_sys_time.c
platform/proven_sys_random.c
platform/proven_sys_mem.c
```

대상에 대한 실제 플랫폼 백엔드를 함께 제공하지 않는 한, 제외된 호스티드 모듈을 베어메탈 빌드에 추가하지 말라.

3. 모듈 사용 가능성

이 표를 읽는 법, 그리고 "제외됨"의 의미
여기서 모듈이 제외되는 이유는 정확히 하나다: 운영체제만이 제공할 수 있는 무언가를 필요로 하기 때문이다. 코드가 작은 대상에서 검증되지 않았다거나 누군가 아직 손을 못 댔다는 뜻이 아니다 — `fs.h`는 파일시스템이, `sysio.h`는 표준 스트림이, `mmap.h`는 가상 메모리가, `job.h`는 스레드가 필요하다. 이식할 것이 없다.

흥미로운 행은 사용 가능하지도 제외되지도 않은 두 개다:

- **`heap.h`는 스텁이다.** `proven_heap_allocator()`는 여전히 존재하고 여전히 컴파일되며, 함수 포인터가 널인 allocator를 반환한다 — `proven_alloc_is_valid`가 유효하지 않다고 보고하는 그것이다. 조용히 다른 어딘가에서 할당하지도 않고 링크에 실패하지도 않는다. 이 선택은 중요하다: allocator 트레이트를 대상으로 작성된 코드는 베어메탈 대상에서도 계속 컴파일되고, 깨지는 지점은 힙을 요구한 바로 그 한 줄이며, 런타임에, 여러분이 직접 쓸 수 있는 검사와 함께 드러난다. 대신 무엇을 넘길지는 4절이 보여 준다.
- **`time.h`는 제한적이다.** datetime 포매팅은 여러분이 공급한 숫자에 대한 순수 산술이므로 컴파일된다. 없는 것은 그 숫자다: 클럭을 읽는 것은 syscall이므로 여기서 `proven_time_now`에는 백엔드가 없다. 여러분 자신의 하드웨어 타이머에서 얻은 타임스탬프를 포매팅하라.

"사용 가능"으로 표시된 모든 것은 이식 가능한 코어이며, "사용 가능"이란 호스티드 빌드에서 도는 바로 그 테스트들이 여기에 대해서도 돈다는 뜻이다 — freestanding 프로파일은 이 표에서 주장되는 것이 아니라 릴리스마다 `./nob freestanding`이 빌드하고 검사한다.

모듈	현재 freestanding 프로파일에서의 상태	비고
`types.h`, `error.h`, `align.h`, `memory.h`	사용 가능	고정 폭 정수와 `uintptr_t` 지원이 필요하다.
`allocator.h`	사용 가능	트레이트만 제공. 호출자가 뒷받침 allocator를 제공한다.
`alloc_check.h`	사용 가능	호출자가 준 기록 위의 순수한 장부 관리; `proven_panic`으로 보고한다.
`arena.h`	사용 가능	정적 메모리 영역을 위한 주 allocator.
`pool.h`	사용 가능	호출자가 제공하는 기반 allocator(흔히 arena)를 사용한다.
`buffer.h`	사용 가능	고정 용량 바이트 버퍼.
`u8str.h`	사용 가능	U8 문자열과 뷰.
`u16str.h`	제외됨	현재 프로파일은 `PROVEN_NO_U16STR`를 정의한다.
`array.h`, `list.h`, `ring.h`, `map.h`	사용 가능	숨겨진 OS 의존성 없음.
`algorithm.h`	사용 가능	배열용 정렬/검색 헬퍼.
`hash.h`	사용 가능	FNV-1a, SipHash-2-4, CRC-32, SHA-256 — 바이트 단위로 정확, OS 의존성 없음. CRC-32는 기본으로 256항목 표 여덟 개(읽기 전용 데이터 8 KiB)를 쓴다. 라이브러리를 `-DPROVEN_CRC32_SMALL=1`로 빌드하면 1 KiB 표 하나만 남는다. 긴 입력에서 약 5배 느리고 결과는 같다.
`hash_legacy.h`	사용 가능	SHA-1과 MD5. 그것을 못 박은 형식(WebSocket accept 키, 오래된 체크섬 파일)을 위한 것이다. SHA-256과 같은 모양이고, MD5의 상수 64개보다 큰 표는 없다.
`hmac.h`	사용 가능	SHA-256, SHA-384, SHA-512(뒤의 둘은 `hash.h`에 있다) 위의 HMAC과 HKDF. 순수한 산술이다. 함께 쓰는 `proven_mem_equal_ct`와 `proven_mem_wipe`는 `memory.h`에 있다.
`deflate.h`	사용 가능	DEFLATE, zlib, gzip을 양방향으로. 순수한 산술이다. 압축기(기본 윈도에서 약 325 KiB, 가장 작

		은 윈도에서 11 KiB)나 해제기(약 35 KiB)마다 할당 하나를, 넘겨준 할당자로 한다.
zstd.h	사용 가능	Zstandard 양방향. 순수한 산술이다. 디코더는 약 16 KiB, 프레임을 푸는 동안에는 그 창(여러분이 정한 상한까지, 기본 8 MiB)과 최대 384 KiB를 출력이 자라는 대로, 인코더는 해시 표(레벨에 따라 64 KiB부터 1 MiB), 블록 하나에 약 500 KiB, 스트림이면 창의 한 배 반인 입력 버퍼를 - 모두 넘겨준 할당자로 할당한다. 디코더와 인코더는 다른 오브젝트 파일이다: 읽기만 하는 프로그램에는 인코더가 링크되지 않는다.
brotli.h	사용 가능	Brotli 풀기. 순수한 산술이다. 디코더는 약 22 KiB, 스트림을 푸는 동안에는 히스토리(출력이 닿은 만큼의 창, 최대 16 MiB)와 접두부호(최대 1.3 MiB)를 필요한 만큼 넘겨준 할당자로 할당한다. 122,784바이트의 정적 사전이 함께 들어 있고, -DPROVEN_NO_BROTLI는 둘 다 뺀다.
hpack.h, h2.h	사용 가능	HTTP/2의 헤더 압축, 프레임, 연결. 순수한 산술이고 소켓도 시계도 없다. HPACK은 인코더에 테이블 하나, 디코더에 테이블 하나를 할당하고(각각 쓰이는 만큼 자라며, 상한은 기본으로 항목 4,096바이트) 프레임 함수는 아무것도 할당하지 않으며, 연결(proven_h2_conn_t)은 받은 할당자로 설정이 정한 한도 안에서 할당한다.
cert.h	사용 가능, proven_cert_store_add_system 제외	X.509 인증서: 읽기, 이름, PEM, 신뢰 앵커 저장소(여러분의 할당자로), 체인 검증. 시각과 앵커는 여러분이 공급한다. 운영체제의 루트를 읽는 것만 운영체제가 필요하다.
tls.h	엔진: 사용 가능. 전송 래퍼와 proven_tls_http_wrap: 호스티드	I/O 없는 상태 기계로서의 TLS(1.3과 줄인 1.2): 도착한 바이트를 주고, 내주는 바이트를 보낸다. 프리스탠딩 빌드는 설정에 난수원과 시계를 공급해야 한다(random, now).

url.h, http.h	사용 가능	URL 파싱과 퍼센트 코딩, 그리고 HTTP/1.1 메시지 코덱. 순수한 텍스트 처리다: 소켓도 할당도 없다. 전송은 여러분이 제공한다 - 소켓이 없는 타깃(wasm32, 베어메탈)에서도 코덱은 메시지를 파싱하고 쓴다. proven_http_date_format은 여러분이 주는 시각을 받는다. 프리스탠딩 시계는 0을 돌려준다.
http_auth.h, http_cookie.h, sse.h	사용 가능	Basic과 Digest 인증 값, 쿠키 보관함, 서버 전송 이벤트 파서. 코덱과 마찬가지로 텍스트 처리다. 쿠키 보관함은 여러분이 넘긴 할당자에서 할당하고, 시계를 읽는 대신 현재 시각을 인자로 받는다. proven_http_multipart_boundary는 여러분이 준 무작위 바이트를 받으므로 엔트로피 원천이 필요 없다.
ws.h	사용 가능	WebSocket 코덱: 핸드셰이크 값과 확인, 프레임 헤더, 마스킹, close 페이로드, 메시지 디코더. 라이브러리의 SHA-1과 Base64를 쓰고, 키와 마스크의 무작위 바이트는 인자로 받는다. wasm32 모듈을 돌리는 브라우저는 자기 WebSocket을 갖고 있다. 거기서 이 코덱의 쓰임은 브라우저가 아닌 호스트이거나 핸드셰이크 계산뿐이다.
encode.h	사용 가능	Hex와 Base64 — 순수 계산, OS 없음.
utf.h	사용 가능	엄격한 UTF-8 <-> UTF-16 변환 — 순수 계산, OS 없음. proven_utf8_append_to_u16str는 PROVEN_NO_U16STR와 함께 제외되고, 나머지는 proven_u16 배열 위에서 동작한다.
fmt.h	부동소수점 없이 사용 가능	현재 프로파일은 PROVEN_FMT_NO_FLOAT를 정의한다.
scan.h	사용 가능	메모리 뷰용 스캐너.
float_parse.h	사용 가능	proven_strtod, proven_parse_double_ascii, proven_parse_f64_ascii가 모두 여기서 컴파일된다. 십진수→binary64 엔진은 정수 연산만

		쓰므로 libc가 필요 없다. 유일한 차이는 freestanding 빌드가 오버플로/언더플로에서 `errno`를 설정하지 않는다는 점이며, 대신 반환된 `proven_err_t`가 그 정보를 담는다 — `errno` 자체가 없는 타깃에서 확인해야 할 값이다. 이는 이 프로파일이 실제로 컴파일에서 제외하는 `fmt.h`의 부동소수점 **출력**과는 별개다.
`float_format.h`	`fmt.h` 연동 없이 사용 가능	binary64 포매터는 정수 연산만 쓰므로 컴파일되지만, 현재 프로파일이 `PROVEN_FMT_NO_FLOAT`를 정의하므로 `{}`는 부동소수점을 렌더링하지 않는다. 코드 크기를 감수할 가치가 있다고 판단한 타깃에서 자릿수 출력이 필요하다면 `float_format.h`의 진입점을 직접 호출할 것.
`time.h`	제한적	핵심 datetime 포매팅은 컴파일 가능하지만, 실제 PAL 시간은 제외된다.
`heap.h`	스텁	`proven_heap_allocator()`는 유효하지 않은 allocator를 반환한다.
`fs.h`, `stream.h`, `net.h`, `http_client.h`, `http_server.h`, `loop.h`, `http_event.h`, `ws_event.h`, `http_event_client.h`, `ws_conn.h`, `mmap.h`, `sysio.h`, `job.h`	제외됨	호스티드 PAL 서비스가 필요하다.
`random.h`	사용 가능	생성기와 헬퍼는 순수 산술이다. `proven_random_bytes`도 여기서 동작하지만, `proven_random_set_source`로 엔트로피 소스 — 보드의 TRNG, 링 오실레이터, 또는 ADC 잡음 바닥 — 를 설치한 후에만 가능하다. 아무것도 설치되지 않았을 때는 클럭으로 시드된 PRNG로 폴백하지 않고 **false**를 반환한다. 폴백은 성공처럼 보이면서 아무것도 보고하지 않는 보안 구멍이 될 것이기 때문이다. 그런 다음 `proven_chacha_rng_seed_from_entropy`가 보드의 엔트로피를 끝없는 암호학적 스트림으로 바꾼다. 소스가 없으면 `proven_random_u64`는 말없이 0을 돌려주므로, 토큰이나

		키에는 false를 돌려주는 `proven_random_u64_checked`를 쓴다.
`coro.h`	사용 가능	매크로 전용 스택리스 코루틴 지원.
`panic.h`	사용 가능	대상별 trap/reset 동작을 위한 오버라이드.

4. 최소 정적 arena 설정

힙이 없으므로 메모리는 정적 블록에서 나오고 그 위에 arena를 얹는다. arena는 그 블록을 소유하지 않는다: `alloc`은 오프셋을 증가시키고, `free`는 no-op이며, `reset`은 전체를 한 번에 되감는다.

이것이 라이브러리 전체가 가능하게 하려고 빚어진 패턴이며, 왜 그것이 통하는지 볼 가치가 있다. 할당할 수 있는 모든 함수는 `proven_allocator_t`를 매개변수로 받으므로, 호스티드 빌드에서 `proven_heap_allocator()`를 대상으로 작성된 코드는 arena를 대신 건네주기만 하면 여기서 그대로 돈다. `u8str.h`, `array.h`, `map.h`의 어떤 것도 메모리가 `malloc`이 아니라 `.bss`의 `static` 배열에서 왔다는 것을 알지도, 상관하지도 않는다.

그 블록의 크기를 정할 때 내려야 할 세 가지 결정이 있고, 어느 것도 라이브러리가 대신 내려 줄 수 없다:

- **얼마나 크게.** arena는 성장할 수 없으므로 그 크기는 여러분이 컴파일 타임에 고르는 단단한 한계다. 너무 작으면 할당이 `PROVEN_ERR_NOMEM`을 반환하기 시작하고, 너무 크면 펌웨어의 나머지가 필요로 하던 RAM을 써 버린 것이다. 이것은 몇 달 뒤 힙 단편화로 발견하는 대신 명시적으로 하라고 요구되는 거래다.
- **언제 reset할지.** arena의 이점 전부는 한 번에 모든 것을 해제하는 데 있다. 자연스러운 지점은 루프 한 번, 수신한 패킷 하나, 명령 하나 — 작업 묶음에 분명한 끝이 있는 곳이면 어디든이다. 그 묶음 동안 할당된 모든 것은 reset에서 죽으므로, **그것보다 오래 살아남아도 되는 것은 없다**.
- **뒷받침 배열의 정렬.** 아래처럼 선언에 `alignas(max_align_t)`를 붙인다. 그것이 없으면 배열이 `double`을 담을 수 없는 주소에서 시작할 수 있고, arena에는 그것을 고칠 수단이 없다.

잘못됨 — reset보다 오래 사는, arena 안에 만들어진 뷰:

```
proven_u8str_view_t label = build_label(arena_alloc);   /* lives in the arena */
proven_arena_reset(&arena);
send(label);   /* wrong: those bytes are free space now, and the next allocation takes them */
```

호스티드 빌드에서는 이것이 테스트에서 괜찮아 보일 만큼은 자주 살아남는다. 여기서는 다음 할당이 곧바로 같은 바이트를 가져가므로 그렇지 않다.

```
#include "proven/types.h"
#include "proven/memory.h"
#include "proven/arena.h"
#include "proven/array.h"
#include "proven/panic.h"

/* static storage: no OS, no heap, known at link time */
static alignas(PROVEN_MAX_ALIGN) proven_byte_t storage[4096];

proven_arena_t arena = proven_arena_create((proven_mem_mut_t){
    .ptr = storage,
    .size = sizeof storage,
});
proven_allocator_t arena_alloc = proven_arena_as_allocator(&arena);

proven_result_array_t r = PROVEN_ARRAY_INIT(arena_alloc, proven_i32, 16);
if (proven_is_ok(r.err)) {
    proven_array_t values = r.value;
    proven_err_t e = PROVEN_ARRAY_PUSH(&values, proven_i32, 42);
    (void)e;
    PROVEN_ARRAY_DESTROY(&values); /* arena free is a no-op */
```

```
}

proven_arena_destroy(&arena);      /* also a no-op: the caller owns `storage` */
```

흔한 실수:

```
proven_allocator_t heap = proven_heap_allocator();
heap.alloc_fn(heap.ctx, 64, 8); /* wrong: heap is invalid in PROVEN_FREESTANDING */
```

올바름:

```
proven_allocator_t heap = proven_heap_allocator();
if (!proven_alloc_is_valid(heap)) {
    /* use an arena, pool, or target-provided allocator instead */
    proven_panic("no heap on this target");
}
```

5. Panic 핸들러 오버라이드

여기서는 왜 핸들러 설치가 선택 사항이 아닌가
호스티드 시스템에서 trap하는 panic은 견딜 만하다: 프로세스가 죽고, 운영체제가 뒷정리를 하고, 무언가가 그것을 다시 띄운다. 베어메탈에는 그 밑에 아무것도 없다. trap은 코어를 멈추고, 장치가 하던 일이 무엇이든 — 모터를 어떤 속도로 유지하는 것, 무선 링크를 유지하는 것, 히터를 구동하는 것 — CPU가 멈춘 뒤에도 여전히 그러고 있다.

그래서 기본 핸들러는 여러분이 반드시 작성해야 할 핸들러의 자리 표시자이며, 여러분의 핸들러가 무엇을 하는지는 프로그래밍 결정이라기보다 제품 결정이다. 흔한 형태들:

- **하드웨어를 안전 상태로 만든 다음 멈춘다.** 모터 끄기, 출력을 알려진 수준으로, 그다음 회전 대기하거나 디버거를 기다린다. 물리적 액추에이터가 있는 것이라면 무엇에든 올바르다.
- **기록하고 리셋한다.** 리셋을 넘어 살아남는 레지스터나 예약된 RAM 영역에 사유 코드를 쓴 다음, 워치독을 발동시킨다. 다음 부팅이 지난 부팅이 왜 끝났는지 보고한다.
- **요란하게 멈춘다.** 알아볼 수 있는 패턴으로 LED를 깜빡인다. 콘솔이 없는 보드에서는 이것이 진단 채널의 전부이며, 들리는 것보다 값어치가 크다.

1장 §6의 규칙이 여기서는 더 강하게 적용된다: **핸들러는 반환해서는 안 된다.** 반환하면 `proven_arena_alloc_or_panic`은 애초에 할당되지 않은 블록을 가지고 진행하고, 실패는 "장치가 멈췄다"에서 "장치가 널 포인터를 통해 쓰고 있다"로 옮겨 간다.

잘못됨 — 로그를 남기고 반환하는 핸들러:

```
static void my_panic(const char *msg) {
    uart_write(msg);   /* wrong: this returns, and the caller uses a block it never got */
}
```

`proven_arena_alloc_or_panic()`과 관련 panic 경로는 `proven_panic()`을 호출하며, 이는 `proven_set_panic_handler()`로 설치된 핸들러로 디스패치한다. 기본 핸들러는 trap한다.

핸들러는 함수 전체이므로, 이것은 조각이 아니라 리스팅이다:

```
#include "proven/panic.h"

static void my_panic(const char *msg) {
(void)msg;
/* optional: write msg to UART or crash log */
for (;;) {
/* or reset the MCU */
}
}

/* install once during startup; pass NULL to restore the default */
proven_set_panic_handler(my_panic);
```

프로덕션 panic 핸들러는 반환해서는 안 된다. 반환하면 panic을 유발한 할당 결과가 유효하다고 보장되지 않는다.

6. Cortex-M 컴파일 명령 예제

```
arm-none-eabi-gcc \
-std=c23 \
-mcpu=cortex-m4 -mthumb \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/memory.c -o memory.o
```

실제 펌웨어 빌드는 위에 나열된 포함 freestanding 모듈을 컴파일하고, 대상 startup 코드를 컴파일하고, 여러분의 linker script로 링크해야 한다.

7. RISC-V ELF 컴파일 명령 예제

```
riscv64-elf-gcc \
-std=c23 \
-ffreestanding -nostdlib \
-DPROVEN_FREESTANDING \
-DPROVEN_FMT_NO_FLOAT \
-DPROVEN_NO_U16STR \
-Iinclude -Iplatform \
-c src/proven/arena.c -o arena.o
```

여러분의 툴체인이 `riscv64-unknown-elf-gcc` 이름을 사용하면 그 컴파일러를 대신 사용하라. 프로젝트 크로스 매트릭스는 사용 가능한 경우 두 이름을 모두 검사한다.

8. Freestanding 모드에서의 포매팅

무엇이 남는가, 그리고 왜 대개 그것으로 충분한가
포매터는 이식 가능한 계산이다 — 여러분이 공급한 목적지에 바이트를 만들어 넣는다 — 그래서 거의 전부가 여기서 살아남는다. 사라지는 것은 자리 표시자 타입 하나: `double`이다.

큰 손실처럼 들리지만 대개 그렇지 않다. 올바른 부동소수점 포매팅이란 서브노멀을 포함한 모든 입력에 대해 같은 값으로 되읽히는 가장 짧은 십진수를 내보내는 것을 뜻하고, 그러려면 큰 정수 산술과 룩업 테이블이 필요하다. 그것이 포매터에서 단일로 가장 큰 코드 덩어리다. 대부분의 펌웨어는 정수, 문자열 뷰, 문자, 포인터를 포매팅한다 — 센서 읽은 값은 스케일된 정수이고, 상태 줄은 텍스트다 — 그래서 이 프로파일은 기본적으로 부동소수점 경로를 빼고, 그 덕분에 바이너리가 더 작다.

남는 것: 모든 폭의 정수, 뷰, 문자, 불리언, 포인터에 대한 `{}`. 스펙 문법 전체(폭, 채움, 정렬, 진법, 부호). 여러분 자신의 타입을 위한 `PROVEN_ARG_OF`. 그리고 스캐너 전체 — 같은 무게를 지지 않는 부동소수점 *파싱*까지 포함해서.

작은 대상에서 정말로 부동소수점이 필요하다면 스위치는 `PROVEN_FMT_NO_FLOAT`이고, 그와 함께 가는 용량 조절 손잡이는 §8a가 다룬다. 결정하기 전에 크기 변화를 측정하라 — 플래시가 32 KB인 부품에서 그것은 반올림 오차가 아니다.

다른 귀결에도 주의하라: **여기에는 `proven_println`이 없다.** 포매팅은 `proven_u8str_t`나 버퍼에 덧붙이며, 그 바이트를 UART나 RTT 채널로 보내는 것은 여러분의 플랫폼 코드다. freestanding 빌드에는 가정할 수 있는 표준 출력이 없기 때문이다.

부동소수점 포매팅은 `PROVEN_FMT_NO_FLOAT`에 의해 비활성화된다. 정수와 문자열 뷰 포매팅은 계속 사용 가능하다.

올바름(여기서 `alloc`은 4절에서처럼 arena allocator이다 — 힙이 없다):

```
proven_result_u8str_t r = proven_u8str_create(alloc, 32);
if (proven_is_ok(r.err)) {
    proven_fmt_result_t f = proven_u8str_append_fmt_grow(alloc, &r.value,
                                                         "value={}", PROVEN_ARG(123));
    (void)f;
    proven_u8str_destroy(alloc, &r.value);
}
```

잘못됨:

```
proven_u8str_append_fmt_grow(alloc, &s, "{}", PROVEN_ARG(3.14));
/* wrong in the current freestanding profile: float args are excluded, so
   PROVEN_ARG has no _Generic association for double and this fails to compile */
```

8a. Float 큰 정수 용량 조정

정확한 십진수-to-binary64 폴백과 큰 정수 나눗셈 헬퍼는 용량이 `PROVEN_FLOAT_BIGINT_LIMBS`(기본값 160, `proven/float_config.h`에 있음)로 설정되는 고정 크기 큰 정수를 사용한다. 이것이 그들의 스택 사용량을 지배하는 요인이다: 각 큰 정수는 `8 * PROVEN_FLOAT_BIGINT_LIMBS` 바이트이며, 나눗셈은 여러 개에 더해 32비트 스크래치를 사용한다.

스택이 빠듯한 대상은 이를 낮출 수 있다. 예를 들면:

```
-DPROVEN_FLOAT_BIGINT_LIMBS=48
```

유지되는 가수 상한(`PROVEN_FLOAT_MAX_SIGNIFICAND_DIGITS`)은 이 용량에서 파생되므로, 더 작은 값이 절대 오버플로하지 않는다: 파서는 파생된 유효 자릿수까지의 입력에 대해 여전히 올바르게 반올림하고, 더 긴 입력에 대해서는 1 ULP 이내에 머문다. Clinger와 Eisel-Lemire 빠른 경로는 큰 정수를 사용하지 않으므로, 일반적인 짧은 입력은 영향을 받지 않는다. binary64 반올림 경계는 최대 767개의 유효 자릿수를 필요로 할 수 있으므로, 병적으로 긴 십진수의 정확한 반올림이 중요할 때는 기본값(160)을 유지하라.

9. 호스티드 API 피하기

왜 이것이 런타임의 뜻밖이 아니라 링크 오류인가

호스티드 모듈은 freestanding 빌드에 아예 컴파일되어 들어가지 않는다. 따라서 그중 하나를 호출하면 **링크 오류**가 된다 — 빌드 타임에, 쓰지 말았어야 할 함수의 이름을 대는 미정의 심벌로.

그것이 설계가 작동하는 모습이다. 많은 임베디드 라이브러리가 택하는 대안은 런타임에 오류를 반환하는 스텁을 제공하는 것이다. 그러면 마이크로컨트롤러에서 `proven_fs_open` 호출이 컴파일되고, 링크되고, 출하되고, 아무도 밟아 보지 않은 경로에서 현장에서 실패한다. 여기서는 빌드 밖으로 나갈 수가 없다.

여러분 자신의 코드에 대한 실질적인 귀결: 어떤 모듈이 호스티드 도구와 펌웨어 양쪽에 공유된다면, 호스티드 전용 호출은 `#ifndef PROVEN_FREESTANDING` 뒤에 두어야 하고, 링크 오류가 어떤 것을 놓쳤는지 정확히 알려 준다.

현재 freestanding 프로파일에서 다음을 호출하지 말라:

```
proven_fs_open(...);
proven_println(...);
proven_env_get(...);
proven_mmap_create(...);
proven_job_system_init(...);
```

이들은 의도적으로 제외된 호스티드 PAL 파일을 필요로 한다.

환경이 제공해야 하는 것

GCC나 Clang으로 컴파일한 모든 프리스탠딩 C 프로그램이 필요로 하는 것, 그 이상은 없다: **`memcpy`, `memmove`, `memset`, `memcmp`** - 소스가 무엇이라 하든 컴파일러가 복사와 0 채우기에 부를 수 있다 - 그리고 컴파일러 자신의 지원 라이브러리(`libgcc`: Cortex-M에서는 `__aeabi_*` 나눗셈과 소프트 부동소수점 헬퍼). newlib, picolibc, 벤더 HAL에는 이미 이 넷이 있고, 없다면 각각 몇 줄이다. `strlen`도, `malloc`도, stdio도, 시작 파일도 필요 없다.

C 라이브러리의 **헤더**도 필요 없다. 라이브러리의 프리스탠딩 소스는 프리스탠딩 컴파일러가 스스로 제공하는 헤더(<stddef.h>, <stdint.h>, <limits.h> 같은 것)만 포함하고, memcpy를 부르는 곳에서는 그것을 직접 선언한다. v0.9.0 전에는 그렇지 않았다. 세 파일이 <string.h>나 <math.h>를 포함했는데, 임베디드 툴체인에는 그것이 마침 들어 있고 맨 Clang에는 없다. 아래의 wasm32 타깃이 그것을 찾아냈고, 이제는 다시 생기지 않게 지킨다.

바란 것이 아니라 검사한 것이다: ./nob cross는 Cortex-M4와 RISC-V에 대해 라이브러리의 모든 프리스탠딩 오브젝트를 그 네 함수만 제공하는 프로그램과 -nostdlib -nostartfiles -static -lgcc로 링크한다(tests/test_portability_freestanding_nocrt_link.c). 정적 링크는 해결되지 않은 심볼이 하나라도 있으면 실패하므로, 깨끗한 링크가 곧 증거다. 실행하지는 않는다 - 여기에는 보드가 없다 - 그래서 타깃에서의 동작은 여전히 당신이 시험해야 한다.

wasm32도 Clang과 wasm-ld로 같은 링크를 거치며, wasm-ld 역시 정의되지 않은 심볼을 거부한다. 지원 라이브러리는 libgcc가 아니라 compiler-rt의 builtins이고, 라이브러리가 거기서 필요로 하는 도우미는 128비트 곱셈인 __multi3 하나다. 빌드 호스트에 wasm32용 builtins가 설치돼 있지 않을 수 있으므로 링크 프로그램이 그 함수 하나를 직접 제공한다. 여러분의 wasm32 빌드는 builtins를 링크한다. 여기서 wasm32에는 소켓도 파일 시스템도 없다. 호스티드 포트가 아니라 다른 타깃과 같은 프리스탠딩 부분집합이다.

10. 수명 규칙은 여전히 적용된다

사람들이 건너뛰는 절, 그리고 여기서 그 대가가 더 큰 이유
Freestanding는 0장 §5의 소유권 규칙을 하나도 완화하지 않는다. 오히려 날을 세운다. 모두 같은 방향을 가리키는 세 가지 이유가 있다:

- **여러분과 메모리 사이에 allocator가 없다.** 힙 블록을 가리키는 댕글링 포인터는 allocator가 그 블록을 아직 다시 내주지 않았다는 이유로 한동안 살아남는 일이 흔하다. arena는 reset 직후 바로 다음 할당에서 같은 바이트를 내주므로, 낡은 뷰는 즉시, 그리고 결정론적으로 덮어써진다.
- **여러분을 붙잡아 줄 것이 없다.** 작은 대상에는 MMU 트랩도 없고, 프로세스를 죽여 줄 운영체제도 없고, 현장에는 새니타이저도 없다. use-after-reset은 크래시하지 않는다 — 남의 데이터를 읽고 그냥 계속 가며, 증상은 전혀 관계없는 곳에서 나타난다.
- **결과가 물리적이다.** 여기서 잘못된 바이트 하나는 모터 명령이거나 무선 패킷일 수 있다.

규칙은 여러분이 이미 아는 그것들이다: 뷰는 소유자가 살아 있는 동안에만 유효하고, 소유된 객체는 그것을 만든 allocator로 정확히 한 번 파괴되며, 호출자 소유 상태 구조체는 복사해서는 안 된다. 달라지는 것은 오차 허용 범위이고, 그것은 0이다.

Freestanding는 컨테이너 규칙을 완화하지 않는다.

잘못됨:

```
int *p = PROVEN_ARRAY_GET_MUT(&arr, int, 0);
PROVEN_ARRAY_PUSH(&arr, int, 9);
*p = 1; /* wrong: push may have moved the array */
```

잘못됨:

```
proven_u8str_view_t key = make_stack_view();
PROVEN_MAP_SET_U8_BORROWED(&map, key, int, 1);
return; /* wrong if map survives after key bytes go out of scope */
```

11. 검증

왜 이 가이드는 믿는 것이 아니라 검사되는가
위의 모든 것은 운영체제 없이 무엇이 컴파일되고 링크되는지에 대한 주장이며, 그런 주장은 조용히 썩는다. 관계없는 커밋에서 이식 가능한 소스 파일에 #include <stdio.h> 하나가 추가되면 freestanding 프로파일은 깨진다 — 호스트 빌드에서는 아무도 알아채지 못한다. stdio.h는 바로 거기에 있으니까.

그래서 이 프로파일은 서술되는 대신 릴리스마다 빌드된다. ./nob freestanding은 이식 가능한 코어를 -ffreestanding과 이 프로파일의 정의들로 컴파일하고, 아래의 검사들을 빌드 호스트에서 **정적으로** 링크한 뒤

실행한다. 그런 다음 `./nob cross`가 같은 프로파일을 실제 임베디드 대상 — 그중에도 Cortex-M, RISC-V, wasm32 — 에 대해 컴파일 전용 매트릭스로 컴파일한다.

두 검사는 무엇을 잡아낼지를 기준으로 골랐다:

- **힙(heap) 스텁** 검사는 `proven_heap_allocator()`가 여전히 존재하고 `proven_alloc_is_valid`가 거부하는 무언가를 반환함을 증명한다. 누군가 그것을 대신 링크 실패로 만들었다면 트레이트 기반 코드가 베어메탈용으로 컴파일되지 않게 된다 — §3이 설명하는, 이 설계가 피하는 그 실패다.
- **컴파일 검사**는 이 프로파일에 대해 대표적인 프로그램을 빌드하며, 이것이 호스티드 헤더가 이식 가능한 파일에 몰래 들어오는 것을 잡아낸다 - 툴체인에 그 헤더가 없을 때에 한해서다. 호스트와 임베디드 툴체인에는 `<string.h>`가 언제나 있으므로, 이 검사를 완전하게 만드는 것은 C 라이브러리 헤더가 아예 없는 wasm32 타깃이다.

어느 쪽도 실제 하드웨어에서 돌지 않으며, 이 가이드는 그렇다고 주장하지 않는다: 정렬 폴트, 엔디안, 타이밍에는 보드가 필요하다. 검증되는 것은 검증될 수 있는 부분이고, 그것이 모든 빌드에서 돈다. 전부를 아무 빌드에서도 하지 않는 대신에.

프로젝트 freestanding 명령은 빌드 호스트에서 다음 로컬 검사를 빌드하고 실행한다:

```
tests/test_portability_freestanding_heap_stub
tests/test_portability_compile_freestanding
tests/test_portability_compile_nofloat
tests/test_portability_compile_nou16str
tests/test_portability_freestanding
```

크로스 명령은 사용 가능한 임베디드 컴파일러에 대해 컴파일 전용 검사를 수행한다:

```
freestanding-arm-cortex-m4        arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb
freestanding-riscv64-elf          riscv64-elf-gcc
freestanding-riscv64-unknown-elf  riscv64-unknown-elf-gcc
freestanding-wasm32               clang --target=wasm32
```

모든 타깃은 PASS, FAIL, SKIP으로 보고된다. 없는 툴체인은 SKIP이다 - 통과가 아니라 증거의 빈자리다. 프리스탠딩 타깃은 건너뛸 수 있지만, 네이티브 호스트 컴파일러와 두 Windows 타깃은 건너뛸 수 없다. 실제 컴파일이나 링크 실패는 명령을 실패시킨다. 런타임 동작은 여전히 대상 또는 에뮬레이터에서의 검증이 필요하다.

부록 A: alias_xcv.h Alias 인덱스 (7장)

> 영문 manual/manual-07-alias-xcv-index.md의 한국어 번역본. 아래 매핑 표는 심볼 대응이라 원문 그대로 둔다.

이것은 조회표이지 읽을거리가 아니다. 매뉴얼에서 훑어 나갈 산문이 없는 유일한 파일이다: 모든 xcv_ 짧은 이름을 정본 proven_ 이름으로 대응시키는 500여 행. 처음 읽을 때는 건너뛰고, 알아보지 못하는 xcv_ 철자를 만났을 때, 또는 자기 코드에 짧은 이름을 도입할지 정할 때 돌아오라.

이 파일이 manual-07 번호를 유지하는 것은 그 번호가 링크와 빌드가 쓰는 안정적인 식별자이기 때문이다. 읽기 순서에서의 위치는 "부록"이며, 이 제목이 말하는 바가 그것이다.

이 부록은 include/proven/alias_xcv.h에서 생성된다. alias 계층은 선택 사항이다. 정본 proven_·PROVEN_ 이름이 ABI, 문서, 테스트의 기준으로 유지된다.

사용 규칙

alias는 일반 헤더 뒤에 include하라:

```
#include "proven.h"
#include "proven/alias_xcv.h"
```

alias 문서와 정본 API 문서를 섞지 마라. 이 인덱스는 철자 맵으로만 사용하라.

Alias 표

alias 1333개: 소문자 xcv_ 이름 982개와 대문자 XCV_ 이름 351개. 표는 include/proven/alias_xcv.h에서 생성된다. 공개 함수에 alias가 없으면 tests/test_docs_alias_completeness가 빌드를 실패시킨다. 다만 그 게이트는 헤더끼리 비교할 뿐 이 부록과는 비교하지 않는다 — 재생성 전에 표는 헤더보다 61행 뒤처져 있었으므로, 행을 손으로 고치지 말고 헤더에서 재생성할 것.

행 번호 열은 의도적으로 두지 않았다. 위에 alias가 삽입될 때마다 틀려졌고, 그건 열이 아예 없느니만 못했다.

Alias macro	Expands to
XCV_ARG	PROVEN_ARG
XCV_ARG_CSTR	proven_arg_cstr
XCV_ARG_DATETIME	proven_arg_datetime
XCV_ARG_F64	proven_arg_f64
XCV_ARG_FN	PROVEN_ARG_FN(f)
XCV_ARG_I32	proven_arg_i32
XCV_ARG_I64	proven_arg_i64
XCV_ARG_NONE	proven_arg_none
XCV_ARG_PTR	proven_arg_ptr
XCV_ARG_STR_VIEW	proven_arg_str_view
XCV_ARG_U16	proven_arg_u16
XCV_ARG_U32	proven_arg_u32
XCV_ARG_U64	proven_arg_u64
XCV_ARRAY_DESTROY	PROVEN_ARRAY_DESTROY
XCV_ARRAY_GET	PROVEN_ARRAY_GET
XCV_ARRAY_GET_MUT	PROVEN_ARRAY_GET_MUT
XCV_ARRAY_INIT	PROVEN_ARRAY_INIT
XCV_ARRAY_IS_VALID	proven_array_is_valid

XCV_ARRAY_POP	PROVEN_ARRAY_POP
XCV_ARRAY_PUSH	PROVEN_ARRAY_PUSH
XCV_BROTLI_WINDOW_DEFAULT	PROVEN_BROTLI_WINDOW_DEFAULT
XCV_CERT_EKU_	PROVEN_CERT_EKU_
XCV_CERT_EKU_ANY	PROVEN_CERT_EKU_ANY
XCV_CERT_EKU_CLIENT_AUTH	PROVEN_CERT_EKU_CLIENT_AUTH
XCV_CERT_EKU_OTHER	PROVEN_CERT_EKU_OTHER
XCV_CERT_EKU_SERVER_AUTH	PROVEN_CERT_EKU_SERVER_AUTH
XCV_CERT_FAULT_ALGORITHM	PROVEN_CERT_FAULT_ALGORITHM
XCV_CERT_FAULT_BAD_SIGNATURE	PROVEN_CERT_FAULT_BAD_SIGNATURE
XCV_CERT_FAULT_CRITICAL_EXTENSION	PROVEN_CERT_FAULT_CRITICAL_EXTENSION
XCV_CERT_FAULT_EXPIRED	PROVEN_CERT_FAULT_EXPIRED
XCV_CERT_FAULT_MALFORMED	PROVEN_CERT_FAULT_MALFORMED
XCV_CERT_FAULT_NAME_CONSTRAINT	PROVEN_CERT_FAULT_NAME_CONSTRAINT
XCV_CERT_FAULT_NAME_MISMATCH	PROVEN_CERT_FAULT_NAME_MISMATCH
XCV_CERT_FAULT_NONE	PROVEN_CERT_FAULT_NONE
XCV_CERT_FAULT_NOT_A_CA	PROVEN_CERT_FAULT_NOT_A_CA
XCV_CERT_FAULT_NOT_YET_VALID	PROVEN_CERT_FAULT_NOT_YET_VALID
XCV_CERT_FAULT_NO_ISSUER	PROVEN_CERT_FAULT_NO_ISSUER
XCV_CERT_FAULT_PATH_LENGTH	PROVEN_CERT_FAULT_PATH_LENGTH
XCV_CERT_FAULT_PIN_MISMATCH	PROVEN_CERT_FAULT_PIN_MISMATCH
XCV_CERT_FAULT_TOO_DEEP	PROVEN_CERT_FAULT_TOO_DEEP
XCV_CERT_FAULT_USAGE	PROVEN_CERT_FAULT_USAGE
XCV_CERT_KEY_EC_P256	PROVEN_CERT_KEY_EC_P256
XCV_CERT_KEY_EC_P384	PROVEN_CERT_KEY_EC_P384
XCV_CERT_KEY_ED25519	PROVEN_CERT_KEY_ED25519
XCV_CERT_KEY_RSA	PROVEN_CERT_KEY_RSA
XCV_CERT_KEY_UNKNOWN	PROVEN_CERT_KEY_UNKNOWN
XCV_CERT_KU_	PROVEN_CERT_KU_
XCV_CERT_KU_CRL_SIGN	PROVEN_CERT_KU_CRL_SIGN
XCV_CERT_KU_DIGITAL_SIGNATURE	PROVEN_CERT_KU_DIGITAL_SIGNATURE
XCV_CERT_KU_KEY_AGREEMENT	PROVEN_CERT_KU_KEY_AGREEMENT
XCV_CERT_KU_KEY_CERT_SIGN	PROVEN_CERT_KU_KEY_CERT_SIGN
XCV_CERT_KU_KEY_ENCIPHERMENT	PROVEN_CERT_KU_KEY_ENCIPHERMENT
XCV_CERT_MAX_DEPTH	PROVEN_CERT_MAX_DEPTH
XCV_CERT_NAME_DNS	PROVEN_CERT_NAME_DNS
XCV_CERT_NAME_IP	PROVEN_CERT_NAME_IP
XCV_CERT_NAME_OTHER	PROVEN_CERT_NAME_OTHER
XCV_CERT_SIG_ECDSA	PROVEN_CERT_SIG_ECDSA
XCV_CERT_SIG_ED25519	PROVEN_CERT_SIG_ED25519
XCV_CERT_SIG_RSA_PKCS1	PROVEN_CERT_SIG_RSA_PKCS1
XCV_CERT_SIG_RSA_PSS	PROVEN_CERT_SIG_RSA_PSS
XCV_CERT_SIG_UNKNOWN	PROVEN_CERT_SIG_UNKNOWN
XCV_CERT_USE_CLIENT	PROVEN_CERT_USE_CLIENT
XCV_CERT_USE_SERVER	PROVEN_CERT_USE_SERVER

XCV_CKD_ADD	PROVEN_CKD_ADD
XCV_CKD_MUL	PROVEN_CKD_MUL
XCV_CKD_SUB	PROVEN_CKD_SUB
XCV_CONTAINER_OF	PROVEN_CONTAINER_OF
XCV_CORO_AWAIT	PROVEN_CORO_AWAIT
XCV_CORO_BEGIN	PROVEN_CORO_BEGIN
XCV_CORO_END	PROVEN_CORO_END
XCV_CORO_INIT	PROVEN_CORO_INIT
XCV_CORO_IS_DONE	PROVEN_CORO_IS_DONE
XCV_CORO_YIELD	PROVEN_CORO_YIELD
XCV_DEFAULT_ALIGNMENT	PROVEN_DEFAULT_ALIGNMENT
XCV_DEFLATE_FLUSH_FINISH	PROVEN_DEFLATE_FLUSH_FINISH
XCV_DEFLATE_FLUSH_NONE	PROVEN_DEFLATE_FLUSH_NONE
XCV_DEFLATE_FLUSH_SYNC	PROVEN_DEFLATE_FLUSH_SYNC
XCV_DEFLATE_GZIP	PROVEN_DEFLATE_GZIP
XCV_DEFLATE_RAW	PROVEN_DEFLATE_RAW
XCV_DEFLATE_ZLIB	PROVEN_DEFLATE_ZLIB
XCV_ERR_AGAIN	PROVEN_ERR_AGAIN
XCV_ERR_BUSY	PROVEN_ERR_BUSY
XCV_ERR_CANCELLED	PROVEN_ERR_CANCELLED
XCV_ERR_EOF	PROVEN_ERR_EOF
XCV_ERR_EXISTS	PROVEN_ERR_EXISTS
XCV_ERR_EXPIRED	PROVEN_ERR_EXPIRED
XCV_ERR_INVALID_ARG	PROVEN_ERR_INVALID_ARG
XCV_ERR_INVALID_ENCODING	PROVEN_ERR_INVALID_ENCODING
XCV_ERR_INVALID_FORMAT	PROVEN_ERR_INVALID_FORMAT
XCV_ERR_INVALID_STATE	PROVEN_ERR_INVALID_STATE
XCV_ERR_IO	PROVEN_ERR_IO
XCV_ERR_LAST	PROVEN_ERR_LAST
XCV_ERR_NAME_MISMATCH	PROVEN_ERR_NAME_MISMATCH
XCV_ERR_NEED_MORE	PROVEN_ERR_NEED_MORE
XCV_ERR_NOMEM	PROVEN_ERR_NOMEM
XCV_ERR_NOT_FOUND	PROVEN_ERR_NOT_FOUND
XCV_ERR_NOT_YET_VALID	PROVEN_ERR_NOT_YET_VALID
XCV_ERR_OUT_OF_BOUNDS	PROVEN_ERR_OUT_OF_BOUNDS
XCV_ERR_OVERFLOW	PROVEN_ERR_OVERFLOW
XCV_ERR_PERMISSION	PROVEN_ERR_PERMISSION
XCV_ERR_PROTOCOL	PROVEN_ERR_PROTOCOL
XCV_ERR_REFUSED	PROVEN_ERR_REFUSED
XCV_ERR_RESERVED_END	PROVEN_ERR_RESERVED_END
XCV_ERR_RESET	PROVEN_ERR_RESET
XCV_ERR_TIMEOUT	PROVEN_ERR_TIMEOUT
XCV_ERR_UNREACHABLE	PROVEN_ERR_UNREACHABLE
XCV_ERR_UNSUPPORTED	PROVEN_ERR_UNSUPPORTED
XCV_ERR_UNTRUSTED	PROVEN_ERR_UNTRUSTED

XCV_FS_APPEND	PROVEN_FS_APPEND
XCV_FS_CREATE	PROVEN_FS_CREATE
XCV_FS_LOCK_EXCLUSIVE	PROVEN_FS_LOCK_EXCLUSIVE
XCV_FS_LOCK_SHARED	PROVEN_FS_LOCK_SHARED
XCV_FS_LOCK_UNLOCK	PROVEN_FS_LOCK_UNLOCK
XCV_FS_PATH_SEP	PROVEN_FS_PATH_SEP
XCV_FS_PERM_DEFAULT	PROVEN_FS_PERM_DEFAULT
XCV_FS_PERM_GROUP_R	PROVEN_FS_PERM_GROUP_R
XCV_FS_PERM_GROUP_W	PROVEN_FS_PERM_GROUP_W
XCV_FS_PERM_GROUP_X	PROVEN_FS_PERM_GROUP_X
XCV_FS_PERM_OTHER_R	PROVEN_FS_PERM_OTHER_R
XCV_FS_PERM_OTHER_W	PROVEN_FS_PERM_OTHER_W
XCV_FS_PERM_OTHER_X	PROVEN_FS_PERM_OTHER_X
XCV_FS_PERM_OWNER_R	PROVEN_FS_PERM_OWNER_R
XCV_FS_PERM_OWNER_W	PROVEN_FS_PERM_OWNER_W
XCV_FS_PERM_OWNER_X	PROVEN_FS_PERM_OWNER_X
XCV_FS_READ	PROVEN_FS_READ
XCV_FS_TRUNC	PROVEN_FS_TRUNC
XCV_FS_TYPE_DIR	PROVEN_FS_TYPE_DIR
XCV_FS_TYPE_FILE	PROVEN_FS_TYPE_FILE
XCV_FS_TYPE_OTHER	PROVEN_FS_TYPE_OTHER
XCV_FS_WRITE	PROVEN_FS_WRITE
XCV_H2_CANCEL	PROVEN_H2_CANCEL
XCV_H2_COMPRESSION_ERROR	PROVEN_H2_COMPRESSION_ERROR
XCV_H2_CONNECT_ERROR	PROVEN_H2_CONNECT_ERROR
XCV_H2_CONTINUATION	PROVEN_H2_CONTINUATION
XCV_H2_DATA	PROVEN_H2_DATA
XCV_H2_DEFAULT_WINDOW	PROVEN_H2_DEFAULT_WINDOW
XCV_H2_ENHANCE_YOUR_CALM	PROVEN_H2_ENHANCE_YOUR_CALM
XCV_H2_EVENT_CLOSED	PROVEN_H2_EVENT_CLOSED
XCV_H2_EVENT_DATA	PROVEN_H2_EVENT_DATA
XCV_H2_EVENT_GOAWAY	PROVEN_H2_EVENT_GOAWAY
XCV_H2_EVENT_HEADERS	PROVEN_H2_EVENT_HEADERS
XCV_H2_EVENT_NONE	PROVEN_H2_EVENT_NONE
XCV_H2_EVENT_PONG	PROVEN_H2_EVENT_PONG
XCV_H2_EVENT_STREAM_RESET	PROVEN_H2_EVENT_STREAM_RESET
XCV_H2_EVENT_STREAM_WRITABLE	PROVEN_H2_EVENT_STREAM_WRITABLE
XCV_H2_EVENT_TRAILERS	PROVEN_H2_EVENT_TRAILERS
XCV_H2_FLAG_ACK	PROVEN_H2_FLAG_ACK
XCV_H2_FLAG_END_HEADERS	PROVEN_H2_FLAG_END_HEADERS
XCV_H2_FLAG_END_STREAM	PROVEN_H2_FLAG_END_STREAM
XCV_H2_FLAG_PADDED	PROVEN_H2_FLAG_PADDED
XCV_H2_FLAG_PRIORITY	PROVEN_H2_FLAG_PRIORITY
XCV_H2_FLOW_CONTROL_ERROR	PROVEN_H2_FLOW_CONTROL_ERROR
XCV_H2_FRAME_HEADER_SIZE	PROVEN_H2_FRAME_HEADER_SIZE

XCV_H2_FRAME_SIZE_ERROR	PROVEN_H2_FRAME_SIZE_ERROR
XCV_H2_GOAWAY	PROVEN_H2_GOAWAY
XCV_H2_HEADERS	PROVEN_H2_HEADERS
XCV_H2_HTTP_1_1_REQUIRED	PROVEN_H2_HTTP_1_1_REQUIRED
XCV_H2_INADEQUATE_SECURITY	PROVEN_H2_INADEQUATE_SECURITY
XCV_H2_INTERNAL_ERROR	PROVEN_H2_INTERNAL_ERROR
XCV_H2_MAX_MAX_FRAME_SIZE	PROVEN_H2_MAX_MAX_FRAME_SIZE
XCV_H2_MAX_WINDOW	PROVEN_H2_MAX_WINDOW
XCV_H2_MIN_MAX_FRAME_SIZE	PROVEN_H2_MIN_MAX_FRAME_SIZE
XCV_H2_NO_ERROR	PROVEN_H2_NO_ERROR
XCV_H2_PING	PROVEN_H2_PING
XCV_H2_PREFACE	PROVEN_H2_PREFACE
XCV_H2_PREFACE_SIZE	PROVEN_H2_PREFACE_SIZE
XCV_H2_PRIORITY	PROVEN_H2_PRIORITY
XCV_H2_PROTOCOL_ERROR	PROVEN_H2_PROTOCOL_ERROR
XCV_H2_PUSH_PROMISE	PROVEN_H2_PUSH_PROMISE
XCV_H2_REFUSED_STREAM	PROVEN_H2_REFUSED_STREAM
XCV_H2_RST_STREAM	PROVEN_H2_RST_STREAM
XCV_H2_SETTINGS	PROVEN_H2_SETTINGS
XCV_H2_SETTINGS_ENABLE_CONNECT_PROTOCOL	PROVEN_H2_SETTINGS_ENABLE_CONNECT_PROTOCOL
XCV_H2_SETTINGS_ENABLE_PUSH	PROVEN_H2_SETTINGS_ENABLE_PUSH
XCV_H2_SETTINGS_HEADER_TABLE_SIZE	PROVEN_H2_SETTINGS_HEADER_TABLE_SIZE
XCV_H2_SETTINGS_INITIAL_WINDOW_SIZE	PROVEN_H2_SETTINGS_INITIAL_WINDOW_SIZE
XCV_H2_SETTINGS_MAX_CONCURRENT_STREAMS	PROVEN_H2_SETTINGS_MAX_CONCURRENT_STREAMS
XCV_H2_SETTINGS_MAX_FRAME_SIZE	PROVEN_H2_SETTINGS_MAX_FRAME_SIZE
XCV_H2_SETTINGS_MAX_HEADER_LIST_SIZE	PROVEN_H2_SETTINGS_MAX_HEADER_LIST_SIZE
XCV_H2_SETTINGS_TIMEOUT	PROVEN_H2_SETTINGS_TIMEOUT
XCV_H2_STREAM_CLOSED	PROVEN_H2_STREAM_CLOSED
XCV_H2_WINDOW_UPDATE	PROVEN_H2_WINDOW_UPDATE
XCV_HMAC_MAX_SIZE	PROVEN_HMAC_MAX_SIZE
XCV_HMAC_SHA256	PROVEN_HMAC_SHA256
XCV_HMAC_SHA384	PROVEN_HMAC_SHA384
XCV_HMAC_SHA512	PROVEN_HMAC_SHA512
XCV_HPACK_DEFAULT_TABLE_BYTES	PROVEN_HPACK_DEFAULT_TABLE_BYTES
XCV_HPACK_ENTRY_OVERHEAD	PROVEN_HPACK_ENTRY_OVERHEAD
XCV_HTTP_BODY_CHUNKED	PROVEN_HTTP_BODY_CHUNKED
XCV_HTTP_BODY_LENGTH	PROVEN_HTTP_BODY_LENGTH
XCV_HTTP_BODY_NONE	PROVEN_HTTP_BODY_NONE
XCV_HTTP_BODY_UNTIL_CLOSE	PROVEN_HTTP_BODY_UNTIL_CLOSE
XCV_HTTP_BOUNDARY_SIZE	PROVEN_HTTP_BOUNDARY_SIZE
XCV_HTTP_CODING_BROTLI	PROVEN_HTTP_CODING_BROTLI
XCV_HTTP_CODING_DEFLATE	PROVEN_HTTP_CODING_DEFLATE
XCV_HTTP_CODING_GZIP	PROVEN_HTTP_CODING_GZIP
XCV_HTTP_CODING_IDENTITY	PROVEN_HTTP_CODING_IDENTITY
XCV_HTTP_CODING_OTHER	PROVEN_HTTP_CODING_OTHER

XCV_HTTP_CODING_ZSTD	PROVEN_HTTP_CODING_ZSTD
XCV_HTTP_CONNECT	PROVEN_HTTP_CONNECT
XCV_HTTP_COOKIE_MAX_SIZE	PROVEN_HTTP_COOKIE_MAX_SIZE
XCV_HTTP_DATE_SIZE	PROVEN_HTTP_DATE_SIZE
XCV_HTTP_DEFAULT_MAX_HEAD	PROVEN_HTTP_DEFAULT_MAX_HEAD
XCV_HTTP_DELETE	PROVEN_HTTP_DELETE
XCV_HTTP_DIGEST_MD5	PROVEN_HTTP_DIGEST_MD5
XCV_HTTP_DIGEST_MD5_SESS	PROVEN_HTTP_DIGEST_MD5_SESS
XCV_HTTP_DIGEST_SHA256	PROVEN_HTTP_DIGEST_SHA256
XCV_HTTP_DIGEST_SHA256_SESS	PROVEN_HTTP_DIGEST_SHA256_SESS
XCV_HTTP_EVENT_LENGTH_UNKNOWN	PROVEN_HTTP_EVENT_LENGTH_UNKNOWN
XCV_HTTP_GET	PROVEN_HTTP_GET
XCV_HTTP_HEAD	PROVEN_HTTP_HEAD
XCV_HTTP_LENGTH_UNKNOWN	PROVEN_HTTP_LENGTH_UNKNOWN
XCV_HTTP_MAX_CHUNK_LINE	PROVEN_HTTP_MAX_CHUNK_LINE
XCV_HTTP_MAX_TRAILER_BYTES	PROVEN_HTTP_MAX_TRAILER_BYTES
XCV_HTTP_METHOD_OTHER	PROVEN_HTTP_METHOD_OTHER
XCV_HTTP_OPTIONS	PROVEN_HTTP_OPTIONS
XCV_HTTP_PATCH	PROVEN_HTTP_PATCH
XCV_HTTP_POST	PROVEN_HTTP_POST
XCV_HTTP_PUT	PROVEN_HTTP_PUT
XCV_HTTP_RANGE_TO_END	PROVEN_HTTP_RANGE_TO_END
XCV_HTTP_TRACE	PROVEN_HTTP_TRACE
XCV_INDEX_NOT_FOUND	PROVEN_INDEX_NOT_FOUND
XCV_IS_OK	PROVEN_IS_OK
XCV_KEY_TYPE_INT	PROVEN_KEY_TYPE_INT
XCV_KEY_TYPE_U8_BORROWED	PROVEN_KEY_TYPE_U8_BORROWED
XCV_KEY_TYPE_U8_OWNED	PROVEN_KEY_TYPE_U8_OWNED
XCV_LIST_ENTRY	PROVEN_LIST_ENTRY
XCV_LIST_FOR_EACH	PROVEN_LIST_FOR_EACH
XCV_LIST_FOR_EACH_SAFE	PROVEN_LIST_FOR_EACH_SAFE
XCV_LIT	PROVEN_LIT
XCV_LOOP_BACKEND_AUTO	PROVEN_LOOP_BACKEND_AUTO
XCV_LOOP_BACKEND_IOCP	PROVEN_LOOP_BACKEND_IOCP
XCV_LOOP_BACKEND_SELECTOR	PROVEN_LOOP_BACKEND_SELECTOR
XCV_LOOP_BACKEND_URING	PROVEN_LOOP_BACKEND_URING
XCV_MAP_DESTROY	PROVEN_MAP_DESTROY
XCV_MAP_GET_INT	PROVEN_MAP_GET_INT
XCV_MAP_GET_MUT_INT	PROVEN_MAP_GET_MUT_INT
XCV_MAP_GET_MUT_U8_BORROWED	PROVEN_MAP_GET_MUT_U8_BORROWED
XCV_MAP_GET_U8_BORROWED	PROVEN_MAP_GET_U8_BORROWED
XCV_MAP_GET_U8_OWNED	PROVEN_MAP_GET_U8_OWNED
XCV_MAP_INIT_INT	PROVEN_MAP_INIT_INT
XCV_MAP_INIT_U8_BORROWED	PROVEN_MAP_INIT_U8_BORROWED
XCV_MAP_INIT_U8_OWNED	PROVEN_MAP_INIT_U8_OWNED

XCV_MAP_IS_VALID	proven_map_is_valid
XCV_MAP_REMOVE_INT	PROVEN_MAP_REMOVE_INT
XCV_MAP_REMOVE_U8_BORROWED	PROVEN_MAP_REMOVE_U8_BORROWED
XCV_MAP_REMOVE_U8_OWNED	PROVEN_MAP_REMOVE_U8_OWNED
XCV_MAP_SET_INT	PROVEN_MAP_SET_INT
XCV_MAP_SET_U8_BORROWED	PROVEN_MAP_SET_U8_BORROWED
XCV_MAP_SET_U8_OWNED	PROVEN_MAP_SET_U8_OWNED
XCV_MAP_SET_WITH_SCRATCH_INT	PROVEN_MAP_SET_WITH_SCRATCH_INT
XCV_MAP_SET_WITH_SCRATCH_U8_BORROWED	PROVEN_MAP_SET_WITH_SCRATCH_U8_BORROWED
XCV_MAX_ALIGN	PROVEN_MAX_ALIGN
XCV_MMAP_EXEC	PROVEN_MMAP_EXEC
XCV_MMAP_PRIVATE	PROVEN_MMAP_PRIVATE
XCV_MMAP_READ	PROVEN_MMAP_READ
XCV_MMAP_SHARED	PROVEN_MMAP_SHARED
XCV_MMAP_WRITE	PROVEN_MMAP_WRITE
XCV_NET_ADDR_TEXT_MAX	PROVEN_NET_ADDR_TEXT_MAX
XCV_NET_DONT_WAIT	PROVEN_NET_DONT_WAIT
XCV_NET_FAILED	PROVEN_NET_FAILED
XCV_NET_FAMILY_IPV4	PROVEN_NET_FAMILY_IPV4
XCV_NET_FAMILY_IPV6	PROVEN_NET_FAMILY_IPV6
XCV_NET_FAMILY_NONE	PROVEN_NET_FAMILY_NONE
XCV_NET_FAMILY_UNIX	PROVEN_NET_FAMILY_UNIX
XCV_NET_NO_DEADLINE	PROVEN_NET_NO_DEADLINE
XCV_NET_POLL_INLINE_MAX	PROVEN_NET_POLL_INLINE_MAX
XCV_NET_READABLE	PROVEN_NET_READABLE
XCV_NET_SELECTOR_EPOLL	PROVEN_NET_SELECTOR_EPOLL
XCV_NET_SELECTOR_KQUEUE	PROVEN_NET_SELECTOR_KQUEUE
XCV_NET_SELECTOR_IOCP	PROVEN_NET_SELECTOR_IOCP
XCV_NET_SELECTOR_POLL	PROVEN_NET_SELECTOR_POLL
XCV_NET_UNIX_PATH_MAX	PROVEN_NET_UNIX_PATH_MAX
XCV_NET_WRITABLE	PROVEN_NET_WRITABLE
XCV_OK	PROVEN_OK
XCV_RING_DESTROY	PROVEN_RING_DESTROY
XCV_RING_INIT	PROVEN_RING_INIT
XCV_RING_IS_VALID	proven_ring_is_valid
XCV_RING_POP	PROVEN_RING_POP
XCV_RING_PUSH	PROVEN_RING_PUSH
XCV_SCAN_ARG	PROVEN_SCAN_ARG(x)
XCV_SCAN_ARG_LONG	PROVEN_SCAN_ARG_LONG(ptr)
XCV_SCAN_ARG_TYPE_F64	PROVEN_SCAN_ARG_TYPE_F64
XCV_SCAN_ARG_TYPE_I32	PROVEN_SCAN_ARG_TYPE_I32
XCV_SCAN_ARG_TYPE_I64	PROVEN_SCAN_ARG_TYPE_I64
XCV_SCAN_ARG_TYPE_INT	PROVEN_SCAN_ARG_TYPE_INT
XCV_SCAN_ARG_TYPE_LLONG	PROVEN_SCAN_ARG_TYPE_LLONG
XCV_SCAN_ARG_TYPE_LONG	PROVEN_SCAN_ARG_TYPE_LONG

XCV_SCAN_ARG_TYPE_NONE	PROVEN_SCAN_ARG_TYPE_NONE
XCV_SCAN_ARG_TYPE_SHORT	PROVEN_SCAN_ARG_TYPE_SHORT
XCV_SCAN_ARG_TYPE_STR_VIEW	PROVEN_SCAN_ARG_TYPE_STR_VIEW
XCV_SCAN_ARG_TYPE_U32	PROVEN_SCAN_ARG_TYPE_U32
XCV_SCAN_ARG_TYPE_U64	PROVEN_SCAN_ARG_TYPE_U64
XCV_SCAN_ARG_TYPE_UINT	PROVEN_SCAN_ARG_TYPE_UINT
XCV_SCAN_ARG_TYPE_ULLONG	PROVEN_SCAN_ARG_TYPE_ULLONG
XCV_SCAN_ARG_TYPE_ULONG	PROVEN_SCAN_ARG_TYPE_ULONG
XCV_SCAN_ARG_TYPE_USHORT	PROVEN_SCAN_ARG_TYPE_USHORT
XCV_SCAN_ARG_ULONG	PROVEN_SCAN_ARG_ULONG(ptr)
XCV_SHA256_SIZE	PROVEN_SHA256_SIZE
XCV_SHA384_SIZE	PROVEN_SHA384_SIZE
XCV_SHA512_SIZE	PROVEN_SHA512_SIZE
XCV_TLS_CLIENT_AUTH_NONE	PROVEN_TLS_CLIENT_AUTH_NONE
XCV_TLS_CLIENT_AUTH_REQUEST	PROVEN_TLS_CLIENT_AUTH_REQUEST
XCV_TLS_CLIENT_AUTH_REQUIRE	PROVEN_TLS_CLIENT_AUTH_REQUIRE
XCV_TLS_LEGACY_CBC	PROVEN_TLS_LEGACY_CBC
XCV_TLS_LEGACY_DHE	PROVEN_TLS_LEGACY_DHE
XCV_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET	PROVEN_TLS_LEGACY_NO_EXTENDED_MASTER_SECRET
XCV_TLS_LEGACY_RSA_KEY_EXCHANGE	PROVEN_TLS_LEGACY_RSA_KEY_EXCHANGE
XCV_TLS_SESSION_SIZE	PROVEN_TLS_SESSION_SIZE
XCV_TLS_VERIFY_CHAIN	PROVEN_TLS_VERIFY_CHAIN
XCV_TLS_VERIFY_PIN_ONLY	PROVEN_TLS_VERIFY_PIN_ONLY
XCV_TLS_VERSION_1_0	PROVEN_TLS_VERSION_1_0
XCV_TLS_VERSION_1_1	PROVEN_TLS_VERSION_1_1
XCV_TLS_VERSION_1_2	PROVEN_TLS_VERSION_1_2
XCV_TLS_VERSION_1_3	PROVEN_TLS_VERSION_1_3
XCV_U16_LIT	PROVEN_U16_LIT
XCV_VERSION_ENCODE	minor, patch) PROVEN_VERSION_ENCODE(major, minor, patch)
XCV_VERSION_MAJOR	PROVEN_VERSION_MAJOR
XCV_VERSION_MINOR	PROVEN_VERSION_MINOR
XCV_VERSION_NUM	PROVEN_VERSION_NUM
XCV_VERSION_PATCH	PROVEN_VERSION_PATCH
XCV_VERSION_STRING	PROVEN_VERSION_STRING
XCV_WS_ACCEPT_SIZE	PROVEN_WS_ACCEPT_SIZE
XCV_WS_BINARY	PROVEN_WS_BINARY
XCV_WS_CLOSE	PROVEN_WS_CLOSE
XCV_WS_CLOSE_ABNORMAL	PROVEN_WS_CLOSE_ABNORMAL
XCV_WS_CLOSE_GOING_AWAY	PROVEN_WS_CLOSE_GOING_AWAY
XCV_WS_CLOSE_INTERNAL_ERROR	PROVEN_WS_CLOSE_INTERNAL_ERROR
XCV_WS_CLOSE_INVALID_DATA	PROVEN_WS_CLOSE_INVALID_DATA
XCV_WS_CLOSE_NORMAL	PROVEN_WS_CLOSE_NORMAL
XCV_WS_CLOSE_NO_STATUS	PROVEN_WS_CLOSE_NO_STATUS
XCV_WS_CLOSE_POLICY	PROVEN_WS_CLOSE_POLICY
XCV_WS_CLOSE_PROTOCOL_ERROR	PROVEN_WS_CLOSE_PROTOCOL_ERROR

XCV_WS_CLOSE_TOO_BIG	PROVEN_WS_CLOSE_TOO_BIG
XCV_WS_CLOSE_UNSUPPORTED_DATA	PROVEN_WS_CLOSE_UNSUPPORTED_DATA
XCV_WS_CONTINUATION	PROVEN_WS_CONTINUATION
XCV_WS_DEFLATE_ANSWER_MAX	PROVEN_WS_DEFLATE_ANSWER_MAX
XCV_WS_DEFLATE_OFFER	PROVEN_WS_DEFLATE_OFFER
XCV_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER	PROVEN_WS_DEFLATE_OFFER_PEER_NO_TAKEOVER
XCV_WS_DEFLATE_OFFER_TAKEOVER	PROVEN_WS_DEFLATE_OFFER_TAKEOVER
XCV_WS_EVENT_CLOSE	PROVEN_WS_EVENT_CLOSE
XCV_WS_EVENT_DATA	PROVEN_WS_EVENT_DATA
XCV_WS_EVENT_NONE	PROVEN_WS_EVENT_NONE
XCV_WS_EVENT_PING	PROVEN_WS_EVENT_PING
XCV_WS_EVENT_PONG	PROVEN_WS_EVENT_PONG
XCV_WS_KEY_SIZE	PROVEN_WS_KEY_SIZE
XCV_WS_MAX_CONTROL	PROVEN_WS_MAX_CONTROL
XCV_WS_MAX_FRAME_HEADER	PROVEN_WS_MAX_FRAME_HEADER
XCV_WS_PING	PROVEN_WS_PING
XCV_WS_PONG	PROVEN_WS_PONG
XCV_WS_TEXT	PROVEN_WS_TEXT
XCV_ZSTD_FLUSH_BLOCK	PROVEN_ZSTD_FLUSH_BLOCK
XCV_ZSTD_FLUSH_FINISH	PROVEN_ZSTD_FLUSH_FINISH
XCV_ZSTD_FLUSH_NONE	PROVEN_ZSTD_FLUSH_NONE
XCV_ZSTD_WINDOW_DEFAULT	PROVEN_ZSTD_WINDOW_DEFAULT
xcv_alloc_check_entry_t	proven_alloc_check_entry_t
xcv_alloc_check_live	proven_alloc_check_live
xcv_alloc_check_owns	proven_alloc_check_owns
xcv_alloc_check_t	proven_alloc_check_t
xcv_alloc_check_wrap	proven_alloc_check_wrap
xcv_alloc_checked	proven_alloc_checked
xcv_alloc_fn_t	proven_alloc_fn_t
xcv_alloc_is_valid	proven_alloc_is_valid
xcv_allocator_t	proven_allocator_t
xcv_arena_alloc	proven_arena_alloc
xcv_arena_alloc_aligned	proven_arena_alloc_aligned
xcv_arena_alloc_aligned_or_panic	proven_arena_alloc_aligned_or_panic
xcv_arena_alloc_or_panic	proven_arena_alloc_or_panic
xcv_arena_alloc_trait	proven_arena_alloc_trait
xcv_arena_as_allocator	proven_arena_as_allocator
xcv_arena_create	proven_arena_create
xcv_arena_destroy	proven_arena_destroy
xcv_arena_free_trait	proven_arena_free_trait
xcv_arena_realloc_aligned	proven_arena_realloc_aligned
xcv_arena_realloc_trait	proven_arena_realloc_trait
xcv_arena_reset	proven_arena_reset
xcv_arena_t	proven_arena_t
xcv_arg_bool	proven_arg_bool

xcv_arg_char	proven_arg_char
xcv_arg_cstr	proven_arg_cstr
xcv_arg_cstr_n	proven_arg_cstr_n
xcv_arg_custom	proven_arg_custom
xcv_arg_datetime	proven_arg_datetime
xcv_arg_f64	proven_arg_f64
xcv_arg_fn	proven_arg_fn
xcv_arg_i32	proven_arg_i32
xcv_arg_i64	proven_arg_i64
xcv_arg_identity	proven_arg_identity
xcv_arg_none	proven_arg_none
xcv_arg_ptr	proven_arg_ptr
xcv_arg_str_view	proven_arg_str_view
xcv_arg_t	proven_arg_t
xcv_arg_type_t	proven_arg_type_t
xcv_arg_u16	proven_arg_u16
xcv_arg_u32	proven_arg_u32
xcv_arg_u64	proven_arg_u64
xcv_arg_ucstr	proven_arg_ucstr
xcv_array_binary_search	proven_array_binary_search
xcv_array_clear	proven_array_clear
xcv_array_create	proven_array_create
xcv_array_destroy	proven_array_destroy
xcv_array_extend	proven_array_extend
xcv_array_get	proven_array_get
xcv_array_get_mut	proven_array_get_mut
xcv_array_insert	proven_array_insert
xcv_array_is_valid	proven_array_is_valid
xcv_array_linear_search	proven_array_linear_search
xcv_array_pop	proven_array_pop
xcv_array_push	proven_array_push
xcv_array_remove_at	proven_array_remove_at
xcv_array_reserve	proven_array_reserve
xcv_array_sort	proven_array_sort
xcv_array_swap_remove	proven_array_swap_remove
xcv_array_t	proven_array_t
xcv_array_truncate	proven_array_truncate
xcv_base64_decode	proven_base64_decode
xcv_base64_decoded_size	proven_base64_decoded_size
xcv_base64_encode	proven_base64_encode
xcv_base64_encoded_size	proven_base64_encoded_size
xcv_base64url_encode	proven_base64url_encode
xcv_brotli_decode	proven_brotli_decode
xcv_brotli_decode_all	proven_brotli_decode_all
xcv_brotli_decoder_create	proven_brotli_decoder_create

xcv_brotli_decoder_destroy	proven_brotli_decoder_destroy
xcv_brotli_decoder_reset	proven_brotli_decoder_reset
xcv_brotli_decoder_t	proven_brotli_decoder_t
xcv_buf_append	proven_buf_append
xcv_buf_create	proven_buf_create
xcv_buf_destroy	proven_buf_destroy
xcv_buf_t	proven_buf_t
xcv_byte_t	proven_byte_t
xcv_c_lib	proven_c_lib
xcv_cert_alt_name_next	proven_cert_alt_name_next
xcv_cert_fault_t	proven_cert_fault_t
xcv_cert_key_kind_t	proven_cert_key_kind_t
xcv_cert_key_sha256	proven_cert_key_sha256
xcv_cert_matches_host	proven_cert_matches_host
xcv_cert_name_kind_t	proven_cert_name_kind_t
xcv_cert_parse	proven_cert_parse
xcv_cert_sig_kind_t	proven_cert_sig_kind_t
xcv_cert_store	proven_cert_store
xcv_cert_store_add_der	proven_cert_store_add_der
xcv_cert_store_add_pem	proven_cert_store_add_pem
xcv_cert_store_add_system	proven_cert_store_add_system
xcv_cert_store_count	proven_cert_store_count
xcv_cert_store_create	proven_cert_store_create
xcv_cert_store_destroy	proven_cert_store_destroy
xcv_cert_store_t	proven_cert_store_t
xcv_cert_t	proven_cert_t
xcv_cert_use_t	proven_cert_use_t
xcv_cert_verify	proven_cert_verify
xcv_cert_verify_options_t	proven_cert_verify_options_t
xcv_cert_verify_result_t	proven_cert_verify_result_t
xcv_chacha_rng	proven_chacha_rng
xcv_chacha_rng_fill	proven_chacha_rng_fill
xcv_chacha_rng_next	proven_chacha_rng_next
xcv_chacha_rng_seed	proven_chacha_rng_seed
xcv_chacha_rng_seed_from_entropy	proven_chacha_rng_seed_from_entropy
xcv_compare_fn_t	proven_compare_fn_t
xcv_coro_t	proven_coro_t
xcv_crc32	proven_crc32
xcv_crc32_update	proven_crc32_update
xcv_cstr_len	proven_cstr_len
xcv_datetime_t	proven_datetime_t
xcv_deflate	proven_deflate
xcv_deflate_all	proven_deflate_all
xcv_deflate_bound	proven_deflate_bound
xcv_deflate_create	proven_deflate_create

xcv_deflate_destroy	proven_deflate_destroy
xcv_deflate_flush_t	proven_deflate_flush_t
xcv_deflate_format_t	proven_deflate_format_t
xcv_deflate_options_t	proven_deflate_options_t
xcv_deflate_reset	proven_deflate_reset
xcv_deflate_t	proven_deflate_t
xcv_entropy_fn	proven_entropy_fn
xcv_env_get	proven_env_get
xcv_eprint	proven_eprint
xcv_eprintln	proven_eprintln
xcv_err_t	proven_err_t
xcv_file_t	proven_file_t
xcv_float_format_f32_policy	proven_float_format_f32_policy
xcv_float_format_f64_policy	proven_float_format_f64_policy
xcv_float_format_options_fixed_default	proven_float_format_options_fixed_default
xcv_float_format_options_scientific	proven_float_format_options_scientific
xcv_float_format_options_shortest	proven_float_format_options_shortest
xcv_fmt_put	proven_fmt_put
xcv_fmt_to_writer_impl	proven_fmt_to_writer_impl
xcv_fprint	proven_fprint
xcv_free_fn_t	proven_free_fn_t
xcv_fs_chmod	proven_fs_chmod
xcv_fs_close	proven_fs_close
xcv_fs_copy	proven_fs_copy
xcv_fs_dir_close	proven_fs_dir_close
xcv_fs_dir_next	proven_fs_dir_next
xcv_fs_dir_open	proven_fs_dir_open
xcv_fs_entry_t	proven_fs_entry_t
xcv_fs_handle_t	proven_fs_handle_t
xcv_fs_is_absolute	proven_fs_is_absolute
xcv_fs_is_staging_name	proven_fs_is_staging_name
xcv_fs_link	proven_fs_link
xcv_fs_list	proven_fs_list
xcv_fs_list_destroy	proven_fs_list_destroy
xcv_fs_lock	proven_fs_lock
xcv_fs_lock_type_t	proven_fs_lock_type_t
xcv_fs_mkdir	proven_fs_mkdir
xcv_fs_mkdir_all	proven_fs_mkdir_all
xcv_fs_mode_t	proven_fs_mode_t
xcv_fs_open	proven_fs_open
xcv_fs_perms_t	proven_fs_perms_t
xcv_fs_pread	proven_fs_pread
xcv_fs_pwrite	proven_fs_pwrite
xcv_fs_read	proven_fs_read
xcv_fs_read_all	proven_fs_read_all

xcv_fs_read_all_bounded	proven_fs_read_all_bounded
xcv_fs_read_all_u8str	proven_fs_read_all_u8str
xcv_fs_remove	proven_fs_remove
xcv_fs_rename	proven_fs_rename
xcv_fs_rmdir	proven_fs_rmdir
xcv_fs_seek	proven_fs_seek
xcv_fs_size	proven_fs_size
xcv_fs_stat	proven_fs_stat
xcv_fs_stat_t	proven_fs_stat_t
xcv_fs_symlink	proven_fs_symlink
xcv_fs_sync	proven_fs_sync
xcv_fs_sync_dir	proven_fs_sync_dir
xcv_fs_tell	proven_fs_tell
xcv_fs_truncate	proven_fs_truncate
xcv_fs_type_t	proven_fs_type_t
xcv_fs_walk_close	proven_fs_walk_close
xcv_fs_walk_next	proven_fs_walk_next
xcv_fs_walk_open	proven_fs_walk_open
xcv_fs_write	proven_fs_write
xcv_fs_write_all	proven_fs_write_all
xcv_fs_write_file	proven_fs_write_file
xcv_fs_write_file_atomic	proven_fs_write_file_atomic
xcv_fs_write_file_durable	proven_fs_write_file_durable
xcv_h2_config_t	proven_h2_config_t
xcv_h2_conn_consumed	proven_h2_conn_consumed
xcv_h2_conn_create	proven_h2_conn_create
xcv_h2_conn_destroy	proven_h2_conn_destroy
xcv_h2_conn_feed	proven_h2_conn_feed
xcv_h2_conn_has_peer_settings	proven_h2_conn_has_peer_settings
xcv_h2_conn_is_closed	proven_h2_conn_is_closed
xcv_h2_conn_next_sensitive	proven_h2_conn_next_sensitive
xcv_h2_conn_output_sent	proven_h2_conn_output_sent
xcv_h2_conn_pending_output	proven_h2_conn_pending_output
xcv_h2_conn_ping	proven_h2_conn_ping
xcv_h2_conn_reset	proven_h2_conn_reset
xcv_h2_conn_send_data	proven_h2_conn_send_data
xcv_h2_conn_send_room	proven_h2_conn_send_room
xcv_h2_conn_send_trailers	proven_h2_conn_send_trailers
xcv_h2_conn_set_initial_window	proven_h2_conn_set_initial_window
xcv_h2_conn_set_max_streams	proven_h2_conn_set_max_streams
xcv_h2_conn_shutdown	proven_h2_conn_shutdown
xcv_h2_conn_stream_count	proven_h2_conn_stream_count
xcv_h2_conn_stream_set_user	proven_h2_conn_stream_set_user
xcv_h2_conn_stream_user	proven_h2_conn_stream_user
xcv_h2_conn_submit_request	proven_h2_conn_submit_request

xcv_h2_conn_submit_response	proven_h2_conn_submit_response
xcv_h2_conn_t	proven_h2_conn_t
xcv_h2_error_t	proven_h2_error_t
xcv_h2_event_kind_t	proven_h2_event_kind_t
xcv_h2_event_t	proven_h2_event_t
xcv_h2_frame_check	proven_h2_frame_check
xcv_h2_frame_content	proven_h2_frame_content
xcv_h2_frame_parse	proven_h2_frame_parse
xcv_h2_frame_t	proven_h2_frame_t
xcv_h2_frame_type_t	proven_h2_frame_type_t
xcv_h2_frame_write	proven_h2_frame_write
xcv_h2_header_buffer_size	proven_h2_header_buffer_size
xcv_h2_read_goaway	proven_h2_read_goaway
xcv_h2_read_rst_stream	proven_h2_read_rst_stream
xcv_h2_read_window_update	proven_h2_read_window_update
xcv_h2_request_head	proven_h2_request_head
xcv_h2_request_head_t	proven_h2_request_head_t
xcv_h2_response_head	proven_h2_response_head
xcv_h2_setting_id_t	proven_h2_setting_id_t
xcv_h2_setting_t	proven_h2_setting_t
xcv_h2_settings_next	proven_h2_settings_next
xcv_h2_trailers_check	proven_h2_trailers_check
xcv_h2_write_goaway	proven_h2_write_goaway
xcv_h2_write_ping	proven_h2_write_ping
xcv_h2_write_rst_stream	proven_h2_write_rst_stream
xcv_h2_write_settings	proven_h2_write_settings
xcv_h2_write_settings_ack	proven_h2_write_settings_ack
xcv_h2_write_window_update	proven_h2_write_window_update
xcv_hash_bytes	proven_hash_bytes
xcv_hash_keyed	proven_hash_keyed
xcv_heap_allocator	proven_heap_allocator
xcv_hex_decode	proven_hex_decode
xcv_hex_decoded_size	proven_hex_decoded_size
xcv_hex_encode	proven_hex_encode
xcv_hex_encoded_size	proven_hex_encoded_size
xcv_hkdf	proven_hkdf
xcv_hkdf_expand	proven_hkdf_expand
xcv_hkdf_extract	proven_hkdf_extract
xcv_hmac	proven_hmac
xcv_hmac_final	proven_hmac_final
xcv_hmac_hash_t	proven_hmac_hash_t
xcv_hmac_init	proven_hmac_init
xcv_hmac_size	proven_hmac_size
xcv_hmac_t	proven_hmac_t
xcv_hmac_update	proven_hmac_update

xcv_hpack_decode	proven_hpack_decode
xcv_hpack_decode_marked	proven_hpack_decode_marked
xcv_hpack_decoder	proven_hpack_decoder
xcv_hpack_decoder_create	proven_hpack_decoder_create
xcv_hpack_decoder_destroy	proven_hpack_decoder_destroy
xcv_hpack_decoder_set_max_table_bytes	proven_hpack_decoder_set_max_table_bytes
xcv_hpack_decoder_t	proven_hpack_decoder_t
xcv_hpack_decoder_table_bytes	proven_hpack_decoder_table_bytes
xcv_hpack_decoder_table_count	proven_hpack_decoder_table_count
xcv_hpack_decoder_table_entry	proven_hpack_decoder_table_entry
xcv_hpack_encode	proven_hpack_encode
xcv_hpack_encode_bound	proven_hpack_encode_bound
xcv_hpack_encoder	proven_hpack_encoder
xcv_hpack_encoder_create	proven_hpack_encoder_create
xcv_hpack_encoder_destroy	proven_hpack_encoder_destroy
xcv_hpack_encoder_set_peer_table_bytes	proven_hpack_encoder_set_peer_table_bytes
xcv_hpack_encoder_t	proven_hpack_encoder_t
xcv_hpack_huffman_decode	proven_hpack_huffman_decode
xcv_hpack_huffman_encode	proven_hpack_huffman_encode
xcv_hpack_huffman_size	proven_hpack_huffman_size
xcv_http_accept_weight	proven_http_accept_weight
xcv_http_accepts_coding	proven_http_accepts_coding
xcv_http_auth_offers	proven_http_auth_offers
xcv_http_basic_auth	proven_http_basic_auth
xcv_http_body_end	proven_http_body_end
xcv_http_body_feed	proven_http_body_feed
xcv_http_body_init	proven_http_body_init
xcv_http_body_kind_t	proven_http_body_kind_t
xcv_http_body_received	proven_http_body_received
xcv_http_body_t	proven_http_body_t
xcv_http_client_config_t	proven_http_client_config_t
xcv_http_client_create	proven_http_client_create
xcv_http_client_destroy	proven_http_client_destroy
xcv_http_client_finish	proven_http_client_finish
xcv_http_client_get	proven_http_client_get
xcv_http_client_read	proven_http_client_read
xcv_http_client_read_all	proven_http_client_read_all
xcv_http_client_request_t	proven_http_client_request_t
xcv_http_client_response_t	proven_http_client_response_t
xcv_http_client_send	proven_http_client_send
xcv_http_client_t	proven_http_client_t
xcv_http_client_upgrade	proven_http_client_upgrade
xcv_http_coding_t	proven_http_coding_t
xcv_http_content_coding	proven_http_content_coding
xcv_http_content_range_parse	proven_http_content_range_parse

xcv_http_cookie_jar_clear	proven_http_cookie_jar_clear
xcv_http_cookie_jar_count	proven_http_cookie_jar_count
xcv_http_cookie_jar_destroy	proven_http_cookie_jar_destroy
xcv_http_cookie_jar_header	proven_http_cookie_jar_header
xcv_http_cookie_jar_init	proven_http_cookie_jar_init
xcv_http_cookie_jar_store	proven_http_cookie_jar_store
xcv_http_cookie_jar_t	proven_http_cookie_jar_t
xcv_http_date_format	proven_http_date_format
xcv_http_date_parse	proven_http_date_parse
xcv_http_digest_algorithm_t	proven_http_digest_algorithm_t
xcv_http_digest_auth	proven_http_digest_auth
xcv_http_digest_challenge_parse	proven_http_digest_challenge_parse
xcv_http_digest_challenge_t	proven_http_digest_challenge_t
xcv_http_event_callbacks_t	proven_http_event_callbacks_t
xcv_http_event_client_callbacks_t	proven_http_event_client_callbacks_t
xcv_http_event_client_config_t	proven_http_event_client_config_t
xcv_http_event_client_create	proven_http_event_client_create
xcv_http_event_client_destroy	proven_http_event_client_destroy
xcv_http_event_client_requests	proven_http_event_client_requests
xcv_http_event_client_start	proven_http_event_client_start
xcv_http_event_client_t	proven_http_event_client_t
xcv_http_event_drained_fn	proven_http_event_drained_fn
xcv_http_event_request_abort	proven_http_event_request_abort
xcv_http_event_request_end	proven_http_event_request_end
xcv_http_event_request_options_t	proven_http_event_request_options_t
xcv_http_event_request_pause	proven_http_event_request_pause
xcv_http_event_request_protocol	proven_http_event_request_protocol
xcv_http_event_request_resume	proven_http_event_request_resume
xcv_http_event_request_set_user	proven_http_event_request_set_user
xcv_http_event_request_t	proven_http_event_request_t
xcv_http_event_request_user	proven_http_event_request_user
xcv_http_event_request_write	proven_http_event_request_write
xcv_http_event_server_adopt	proven_http_event_server_adopt
xcv_http_event_server_config_t	proven_http_event_server_config_t
xcv_http_event_server_connections	proven_http_event_server_connections
xcv_http_event_server_create	proven_http_event_server_create
xcv_http_event_server_destroy	proven_http_event_server_destroy
xcv_http_event_server_listen	proven_http_event_server_listen
xcv_http_event_server_set_max_streams	proven_http_event_server_set_max_streams
xcv_http_event_server_set_window	proven_http_event_server_set_window
xcv_http_event_server_shutdown	proven_http_event_server_shutdown
xcv_http_event_server_stop_listening	proven_http_event_server_stop_listening
xcv_http_event_server_t	proven_http_event_server_t
xcv_http_exchange_begin	proven_http_exchange_begin
xcv_http_exchange_compress	proven_http_exchange_compress

xcv_http_exchange_early	proven_http_exchange_early
xcv_http_exchange_end	proven_http_exchange_end
xcv_http_exchange_flush	proven_http_exchange_flush
xcv_http_exchange_peer	proven_http_exchange_peer
xcv_http_exchange_read	proven_http_exchange_read
xcv_http_exchange_request	proven_http_exchange_request
xcv_http_exchange_respond	proven_http_exchange_respond
xcv_http_exchange_t	proven_http_exchange_t
xcv_http_exchange_upgrade	proven_http_exchange_upgrade
xcv_http_exchange_write	proven_http_exchange_write
xcv_http_framing_t	proven_http_framing_t
xcv_http_handler_fn	proven_http_handler_fn
xcv_http_header_count	proven_http_header_count
xcv_http_header_find	proven_http_header_find
xcv_http_header_has_token	proven_http_header_has_token
xcv_http_header_t	proven_http_header_t
xcv_http_method_from_text	proven_http_method_from_text
xcv_http_method_t	proven_http_method_t
xcv_http_method_text	proven_http_method_text
xcv_http_multipart_boundary	proven_http_multipart_boundary
xcv_http_multipart_write_content_type	proven_http_multipart_write_content_type
xcv_http_multipart_write_end	proven_http_multipart_write_end
xcv_http_multipart_write_part	proven_http_multipart_write_part
xcv_http_multipart_write_part_end	proven_http_multipart_write_part_end
xcv_http_parse_request	proven_http_parse_request
xcv_http_parse_response	proven_http_parse_response
xcv_http_range_parse	proven_http_range_parse
xcv_http_reason_phrase	proven_http_reason_phrase
xcv_http_request_framing	proven_http_request_framing
xcv_http_request_keep_alive	proven_http_request_keep_alive
xcv_http_request_t	proven_http_request_t
xcv_http_response_framing	proven_http_response_framing
xcv_http_response_keep_alive	proven_http_response_keep_alive
xcv_http_response_t	proven_http_response_t
xcv_http_server_config_t	proven_http_server_config_t
xcv_http_server_connection_count	proven_http_server_connection_count
xcv_http_server_create	proven_http_server_create
xcv_http_server_destroy	proven_http_server_destroy
xcv_http_server_drained	proven_http_server_drained
xcv_http_server_listen	proven_http_server_listen
xcv_http_server_poll	proven_http_server_poll
xcv_http_server_run	proven_http_server_run
xcv_http_server_shutdown	proven_http_server_shutdown
xcv_http_server_shutting_down	proven_http_server_shutting_down
xcv_http_server_stop	proven_http_server_stop

xcv_http_server_t	proven_http_server_t
xcv_http_stream_abort	proven_http_stream_abort
xcv_http_stream_begin	proven_http_stream_begin
xcv_http_stream_buffered	proven_http_stream_buffered
xcv_http_stream_compress	proven_http_stream_compress
xcv_http_stream_early	proven_http_stream_early
xcv_http_stream_end	proven_http_stream_end
xcv_http_stream_flush	proven_http_stream_flush
xcv_http_stream_pause	proven_http_stream_pause
xcv_http_stream_peer	proven_http_stream_peer
xcv_http_stream_protocol	proven_http_stream_protocol
xcv_http_stream_respond	proven_http_stream_respond
xcv_http_stream_resume	proven_http_stream_resume
xcv_http_stream_set_user	proven_http_stream_set_user
xcv_http_stream_t	proven_http_stream_t
xcv_http_stream_user	proven_http_stream_user
xcv_http_stream_write	proven_http_stream_write
xcv_http_tls_wrap_fn	proven_http_tls_wrap_fn
xcv_http_write_chunk_begin	proven_http_write_chunk_begin
xcv_http_write_chunk_end	proven_http_write_chunk_end
xcv_http_write_content_range	proven_http_write_content_range
xcv_http_write_head_end	proven_http_write_head_end
xcv_http_write_header	proven_http_write_header
xcv_http_write_header_u64	proven_http_write_header_u64
xcv_http_write_last_chunk	proven_http_write_last_chunk
xcv_http_write_range	proven_http_write_range
xcv_http_write_request_line	proven_http_write_request_line
xcv_http_write_status_line	proven_http_write_status_line
xcv_i16	proven_i16
xcv_i32	proven_i32
xcv_i64	proven_i64
xcv_i8	proven_i8
xcv_inflate	proven_inflate
xcv_inflate_all	proven_inflate_all
xcv_inflate_create	proven_inflate_create
xcv_inflate_destroy	proven_inflate_destroy
xcv_inflate_reset	proven_inflate_reset
xcv_inflate_restart	proven_inflate_restart
xcv_inflate_t	proven_inflate_t
xcv_intptr_t	proven_intptr_t
xcv_is_ok	proven_is_ok
xcv_is_pow2	proven_is_pow2
xcv_job_execute_one	proven_job_execute_one
xcv_job_group_init	proven_job_group_init
xcv_job_group_pending	proven_job_group_pending

xcv_job_group_submit	proven_job_group_submit
xcv_job_group_t	proven_job_group_t
xcv_job_group_wait	proven_job_group_wait
xcv_job_submit	proven_job_submit
xcv_job_submit_ex	proven_job_submit_ex
xcv_job_sys	proven_job_sys
xcv_job_sys_t	proven_job_sys_t
xcv_job_system_close	proven_job_system_close
xcv_job_system_destroy	proven_job_system_destroy
xcv_job_system_init	proven_job_system_init
xcv_job_t	proven_job_t
xcv_key_type_t	proven_key_type_t
xcv_list_init	proven_list_init
xcv_list_insert_after	proven_list_insert_after
xcv_list_is_empty	proven_list_is_empty
xcv_list_node_t	proven_list_node_t
xcv_list_push_back	proven_list_push_back
xcv_list_remove	proven_list_remove
xcv_list_t	proven_list_t
xcv_loop_accept_cancel	proven_loop_accept_cancel
xcv_loop_accept_fn	proven_loop_accept_fn
xcv_loop_accept_t	proven_loop_accept_t
xcv_loop_accept	proven_loop_accept
xcv_loop_allocator	proven_loop_allocator
xcv_loop_attach	proven_loop_attach
xcv_loop_backend_t	proven_loop_backend_t
xcv_loop_backend	proven_loop_backend
xcv_loop_close	proven_loop_close
xcv_loop_connect	proven_loop_connect
xcv_loop_create_ex	proven_loop_create_ex
xcv_loop_create	proven_loop_create
xcv_loop_destroy	proven_loop_destroy
xcv_loop_fn	proven_loop_fn
xcv_loop_options_t	proven_loop_options_t
xcv_loop_poll	proven_loop_poll
xcv_loop_post	proven_loop_post
xcv_loop_recv_cancel	proven_loop_recv_cancel
xcv_loop_read	proven_loop_read
xcv_loop_recv	proven_loop_recv
xcv_loop_run	proven_loop_run
xcv_loop_scratch	proven_loop_scratch
xcv_loop_send_queued	proven_loop_send_queued
xcv_loop_send_wait	proven_loop_send_wait
xcv_loop_send	proven_loop_send
xcv_loop_sock_fns_t	proven_loop_sock_fns_t

xcv_loop_sock_t	proven_loop_sock_t
xcv_loop_stop	proven_loop_stop
xcv_loop_t	proven_loop_t
xcv_loop_timer_cancel	proven_loop_timer_cancel
xcv_loop_timer_is_set	proven_loop_timer_is_set
xcv_loop_timer_set	proven_loop_timer_set
xcv_loop_timer_t	proven_loop_timer_t
xcv_map_create	proven_map_create
xcv_map_create_trusted	proven_map_create_trusted
xcv_map_create_with_capacity	proven_map_create_with_capacity
xcv_map_destroy	proven_map_destroy
xcv_map_get	proven_map_get
xcv_map_get_mut	proven_map_get_mut
xcv_map_hash	proven_map_hash
xcv_map_is_valid	proven_map_is_valid
xcv_map_iter_init	proven_map_iter_init
xcv_map_iter_next	proven_map_iter_next
xcv_map_iter_t	proven_map_iter_t
xcv_map_key_t	proven_map_key_t
xcv_map_len	proven_map_len
xcv_map_remove	proven_map_remove
xcv_map_reserve	proven_map_reserve
xcv_map_set	proven_map_set
xcv_map_set_u8_owned	proven_map_set_u8_owned
xcv_map_set_with_scratch	proven_map_set_with_scratch
xcv_map_t	proven_map_t
xcv_md5	proven_md5
xcv_md5_final	proven_md5_final
xcv_md5_init	proven_md5_init
xcv_md5_to_hex	proven_md5_to_hex
xcv_md5_update	proven_md5_update
xcv_mem_align_up	proven_mem_align_up
xcv_mem_copy	proven_mem_copy
xcv_mem_equal_ct	proven_mem_equal_ct
xcv_mem_move	proven_mem_move
xcv_mem_mut_from_owned	proven_mem_mut_from_owned
xcv_mem_mut_slice_checked	proven_mem_mut_slice_checked
xcv_mem_mut_slice_unchecked	proven_mem_mut_slice_unchecked
xcv_mem_mut_t	proven_mem_mut_t
xcv_mem_t	proven_mem_t
xcv_mem_view_from_owned	proven_mem_view_from_owned
xcv_mem_view_from_u8	proven_mem_view_from_u8
xcv_mem_view_slice_checked	proven_mem_view_slice_checked
xcv_mem_view_slice_unchecked	proven_mem_view_slice_unchecked
xcv_mem_view_t	proven_mem_view_t

xcv_mem_wipe	proven_mem_wipe
xcv_memcmp	proven_memcmp
xcv_mmap_as_view	proven_mmap_as_view
xcv_mmap_create	proven_mmap_create
xcv_mmap_destroy	proven_mmap_destroy
xcv_mmap_flags_t	proven_mmap_flags_t
xcv_mmap_prot_t	proven_mmap_prot_t
xcv_mmap_sync	proven_mmap_sync
xcv_mmap_t	proven_mmap_t
xcv_net_accept	proven_net_accept
xcv_net_addr_any	proven_net_addr_any
xcv_net_addr_eq	proven_net_addr_eq
xcv_net_addr_format	proven_net_addr_format
xcv_net_addr_ipv4	proven_net_addr_ipv4
xcv_net_addr_ipv6	proven_net_addr_ipv6
xcv_net_addr_loopback	proven_net_addr_loopback
xcv_net_addr_parse	proven_net_addr_parse
xcv_net_addr_t	proven_net_addr_t
xcv_net_addr_unix	proven_net_addr_unix
xcv_net_close	proven_net_close
xcv_net_conn_handle	proven_net_conn_handle
xcv_net_conn_is_open	proven_net_conn_is_open
xcv_net_conn_local_addr	proven_net_conn_local_addr
xcv_net_conn_peer_addr	proven_net_conn_peer_addr
xcv_net_conn_set_nodelay	proven_net_conn_set_nodelay
xcv_net_conn_t	proven_net_conn_t
xcv_net_conn_transport	proven_net_conn_transport
xcv_net_connect	proven_net_connect
xcv_net_connect_finish	proven_net_connect_finish
xcv_net_connect_start	proven_net_connect_start
xcv_net_deadline_in	proven_net_deadline_in
xcv_net_deadline_t	proven_net_deadline_t
xcv_net_event_t	proven_net_event_t
xcv_net_family_t	proven_net_family_t
xcv_net_handle_t	proven_net_handle_t
xcv_net_listen	proven_net_listen
xcv_net_listener_addr	proven_net_listener_addr
xcv_net_listener_close	proven_net_listener_close
xcv_net_listener_handle	proven_net_listener_handle
xcv_net_listener_is_open	proven_net_listener_is_open
xcv_net_listener_t	proven_net_listener_t
xcv_net_pair	proven_net_pair
xcv_net_poll	proven_net_poll
xcv_net_poll_item_t	proven_net_poll_item_t
xcv_net_poll_scratch_size	proven_net_poll_scratch_size

xcv_net_poll_with	proven_net_poll_with
xcv_net_read	proven_net_read
xcv_net_ready_t	proven_net_ready_t
xcv_net_resolve	proven_net_resolve
xcv_net_selector_add	proven_net_selector_add
xcv_net_selector_count	proven_net_selector_count
xcv_net_selector_create	proven_net_selector_create
xcv_net_selector_create_poll	proven_net_selector_create_poll
xcv_net_selector_destroy	proven_net_selector_destroy
xcv_net_selector_kind	proven_net_selector_kind
xcv_net_selector_kind_t	proven_net_selector_kind_t
xcv_net_selector_modify	proven_net_selector_modify
xcv_net_selector_remove	proven_net_selector_remove
xcv_net_selector_t	proven_net_selector_t
xcv_net_selector_wait	proven_net_selector_wait
xcv_net_shutdown_write	proven_net_shutdown_write
xcv_net_udp_addr	proven_net_udp_addr
xcv_net_udp_close	proven_net_udp_close
xcv_net_udp_handle	proven_net_udp_handle
xcv_net_udp_is_open	proven_net_udp_is_open
xcv_net_udp_open	proven_net_udp_open
xcv_net_udp_recv_from	proven_net_udp_recv_from
xcv_net_udp_send_to	proven_net_udp_send_to
xcv_net_udp_t	proven_net_udp_t
xcv_net_waker_close	proven_net_waker_close
xcv_net_waker_drain	proven_net_waker_drain
xcv_net_waker_handle	proven_net_waker_handle
xcv_net_waker_open	proven_net_waker_open
xcv_net_waker_t	proven_net_waker_t
xcv_net_waker_wake	proven_net_waker_wake
xcv_net_write	proven_net_write
xcv_net_write_all	proven_net_write_all
xcv_panic	proven_panic
xcv_parse_double_ascii	proven_parse_double_ascii
xcv_parse_f64_ascii	proven_parse_f64_ascii
xcv_pem_next	proven_pem_next
xcv_pool_as_allocator	proven_pool_as_allocator
xcv_pool_destroy	proven_pool_destroy
xcv_pool_init	proven_pool_init
xcv_pool_t	proven_pool_t
xcv_print	proven_print
xcv_println	proven_println
xcv_ptrdiff_t	proven_ptrdiff_t
xcv_random_bytes	proven_random_bytes
xcv_random_set_source	proven_random_set_source

xcv_random_u64	proven_random_u64
xcv_random_u64_checked	proven_random_u64_checked
xcv_range_contains_ptr	proven_range_contains_ptr
xcv_reader_buffered	proven_reader_buffered
xcv_reader_from_file	proven_reader_from_file
xcv_reader_from_view	proven_reader_from_view
xcv_reader_is_valid	proven_reader_is_valid
xcv_reader_read	proven_reader_read
xcv_reader_read_line	proven_reader_read_line
xcv_realloc_fn_t	proven_realloc_fn_t
xcv_result_array_t	proven_result_array_t
xcv_result_buf_t	proven_result_buf_t
xcv_result_cstr_t	proven_result_cstr_t
xcv_result_f64_t	proven_result_f64_t
xcv_result_file_t	proven_result_file_t
xcv_result_i64_t	proven_result_i64_t
xcv_result_map_t	proven_result_map_t
xcv_result_mem_mut_t	proven_result_mem_mut_t
xcv_result_mem_view_t	proven_result_mem_view_t
xcv_result_mmap_t	proven_result_mmap_t
xcv_result_ring_t	proven_result_ring_t
xcv_result_size_t	proven_result_size_t
xcv_result_u16str_t	proven_result_u16str_t
xcv_result_u16str_view_t	proven_result_u16str_view_t
xcv_result_u64_t	proven_result_u64_t
xcv_result_u8str_t	proven_result_u8str_t
xcv_result_u8str_view_t	proven_result_u8str_view_t
xcv_ring_create	proven_ring_create
xcv_ring_destroy	proven_ring_destroy
xcv_ring_is_valid	proven_ring_is_valid
xcv_ring_pop	proven_ring_pop
xcv_ring_push	proven_ring_push
xcv_ring_t	proven_ring_t
xcv_rng_below	proven_rng_below
xcv_rng_f64	proven_rng_f64
xcv_rng_fill	proven_rng_fill
xcv_rng_is_valid	proven_rng_is_valid
xcv_rng_range	proven_rng_range
xcv_rng_shuffle	proven_rng_shuffle
xcv_rng_u64	proven_rng_u64
xcv_scan_arg_f64	proven_scan_arg_f64
xcv_scan_arg_i32	proven_scan_arg_i32
xcv_scan_arg_i64	proven_scan_arg_i64
xcv_scan_arg_identity	proven_scan_arg_identity
xcv_scan_arg_int	proven_scan_arg_int

xcv_scan_arg_llong	proven_scan_arg_llong
xcv_scan_arg_long	proven_scan_arg_long
xcv_scan_arg_none	proven_scan_arg_none
xcv_scan_arg_short	proven_scan_arg_short
xcv_scan_arg_str_view	proven_scan_arg_str_view
xcv_scan_arg_t	proven_scan_arg_t
xcv_scan_arg_type_t	proven_scan_arg_type_t
xcv_scan_arg_u32	proven_scan_arg_u32
xcv_scan_arg_u64	proven_scan_arg_u64
xcv_scan_arg_uint	proven_scan_arg_uint
xcv_scan_arg_ullong	proven_scan_arg_ullong
xcv_scan_arg_ulong	proven_scan_arg_ulong
xcv_scan_arg_ushort	proven_scan_arg_ushort
xcv_scan_f64	proven_scan_f64
xcv_scan_fmt	proven_scan_fmt
xcv_scan_fmt_cursor	proven_scan_fmt_cursor
xcv_scan_fmt_from_file	proven_scan_fmt_from_file
xcv_scan_fmt_from_stdin	proven_scan_fmt_from_stdin
xcv_scan_fmt_internal	proven_scan_fmt_internal
xcv_scan_fmt_internal_view	proven_scan_fmt_internal_view
xcv_scan_i64	proven_scan_i64
xcv_scan_i64_hex	proven_scan_i64_hex
xcv_scan_init	proven_scan_init
xcv_scan_skip_until	proven_scan_skip_until
xcv_scan_skip_until_number	proven_scan_skip_until_number
xcv_scan_skip_whitespace	proven_scan_skip_whitespace
xcv_scan_str	proven_scan_str
xcv_scan_t	proven_scan_t
xcv_scan_u64	proven_scan_u64
xcv_scan_u64_hex	proven_scan_u64_hex
xcv_set_panic_handler	proven_set_panic_handler
xcv_sha1	proven_sha1
xcv_sha1_final	proven_sha1_final
xcv_sha1_init	proven_sha1_init
xcv_sha1_to_hex	proven_sha1_to_hex
xcv_sha1_update	proven_sha1_update
xcv_sha256	proven_sha256
xcv_sha256_final	proven_sha256_final
xcv_sha256_init	proven_sha256_init
xcv_sha256_t	proven_sha256_t
xcv_sha256_to_hex	proven_sha256_to_hex
xcv_sha256_update	proven_sha256_update
xcv_sha384	proven_sha384
xcv_sha384_final	proven_sha384_final
xcv_sha384_init	proven_sha384_init

xcv_sha384_t	proven_sha384_t
xcv_sha384_update	proven_sha384_update
xcv_sha512	proven_sha512
xcv_sha512_final	proven_sha512_final
xcv_sha512_init	proven_sha512_init
xcv_sha512_t	proven_sha512_t
xcv_sha512_update	proven_sha512_update
xcv_size_t	proven_size_t
xcv_sse_event_t	proven_sse_event_t
xcv_sse_feed	proven_sse_feed
xcv_sse_init	proven_sse_init
xcv_sse_last_id	proven_sse_last_id
xcv_sse_t	proven_sse_t
xcv_strtod	proven_strtod
xcv_sysio_file_buffered	proven_sysio_file_buffered
xcv_sysio_lines_open	proven_sysio_lines_open
xcv_sysio_lines_t	proven_sysio_lines_t
xcv_sysio_out_t	proven_sysio_out_t
xcv_sysio_print_impl	proven_sysio_print_impl
xcv_sysio_read_line	proven_sysio_read_line
xcv_sysio_read_u16_line	proven_sysio_read_u16_line
xcv_sysio_scan_chunk_impl	proven_sysio_scan_chunk_impl
xcv_sysio_scanner_deinit	proven_sysio_scanner_deinit
xcv_sysio_scanner_init	proven_sysio_scanner_init
xcv_sysio_scanner_scan_impl	proven_sysio_scanner_scan_impl
xcv_sysio_std_t	proven_sysio_std_t
xcv_sysio_stderr	proven_sysio_stderr
xcv_sysio_stderr_writer	proven_sysio_stderr_writer
xcv_sysio_stdin	proven_sysio_stdin
xcv_sysio_stdin_lines	proven_sysio_stdin_lines
xcv_sysio_stdin_reader	proven_sysio_stdin_reader
xcv_sysio_stdin_u16_lines	proven_sysio_stdin_u16_lines
xcv_sysio_stdout	proven_sysio_stdout
xcv_sysio_stdout_buffered	proven_sysio_stdout_buffered
xcv_sysio_stdout_writer	proven_sysio_stdout_writer
xcv_sysio_u16_lines_open	proven_sysio_u16_lines_open
xcv_sysio_u16_lines_t	proven_sysio_u16_lines_t
xcv_text_encoding_t	proven_text_encoding_t
xcv_time_breakdown	proven_time_breakdown
xcv_time_locale_en	proven_time_locale_en
xcv_time_locale_t	proven_time_locale_t
xcv_time_monotonic_now	proven_time_monotonic_now
xcv_time_now	proven_time_now
xcv_time_now_datetime	proven_time_now_datetime
xcv_time_sleep	proven_time_sleep

xcv_time_t	proven_time_t
xcv_time_u16_fmt	proven_time_u16_fmt
xcv_time_u8_fmt	proven_time_u8_fmt
xcv_tls_alert_received	proven_tls_alert_received
xcv_tls_alert_sent	proven_tls_alert_sent
xcv_tls_alpn	proven_tls_alpn
xcv_tls_cipher_suite	proven_tls_cipher_suite
xcv_tls_client_auth_t	proven_tls_client_auth_t
xcv_tls_client_create	proven_tls_client_create
xcv_tls_close	proven_tls_close
xcv_tls_config	proven_tls_config
xcv_tls_config_create	proven_tls_config_create
xcv_tls_config_destroy	proven_tls_config_destroy
xcv_tls_config_t	proven_tls_config_t
xcv_tls_conn	proven_tls_conn
xcv_tls_conn_destroy	proven_tls_conn_destroy
xcv_tls_conn_t	proven_tls_conn_t
xcv_tls_feed	proven_tls_feed
xcv_tls_http_wrap	proven_tls_http_wrap
xcv_tls_is_established	proven_tls_is_established
xcv_tls_key_update	proven_tls_key_update
xcv_tls_now_fn	proven_tls_now_fn
xcv_tls_options_t	proven_tls_options_t
xcv_tls_output_sent	proven_tls_output_sent
xcv_tls_peer_certificate	proven_tls_peer_certificate
xcv_tls_peer_fault	proven_tls_peer_fault
xcv_tls_peer_key_sha256	proven_tls_peer_key_sha256
xcv_tls_pending_output	proven_tls_pending_output
xcv_tls_random_fn	proven_tls_random_fn
xcv_tls_read	proven_tls_read
xcv_tls_resumed	proven_tls_resumed
xcv_tls_server_create	proven_tls_server_create
xcv_tls_server_name	proven_tls_server_name
xcv_tls_session_t	proven_tls_session_t
xcv_tls_transport_client	proven_tls_transport_client
xcv_tls_transport_conn	proven_tls_transport_conn
xcv_tls_transport_server	proven_tls_transport_server
xcv_tls_verify_t	proven_tls_verify_t
xcv_tls_version	proven_tls_version
xcv_tls_write	proven_tls_write
xcv_transport_close	proven_transport_close
xcv_transport_is_valid	proven_transport_is_valid
xcv_transport_read	proven_transport_read
xcv_transport_reader	proven_transport_reader
xcv_transport_shutdown	proven_transport_shutdown

xcv_transport_stream_t	proven_transport_stream_t
xcv_transport_t	proven_transport_t
xcv_transport_write	proven_transport_write
xcv_transport_write_all	proven_transport_write_all
xcv_transport_writer	proven_transport_writer
xcv_u16	proven_u16
xcv_u16_reader_init	proven_u16_reader_init
xcv_u16_reader_read	proven_u16_reader_read
xcv_u16_reader_read_line	proven_u16_reader_read_line
xcv_u16_reader_t	proven_u16_reader_t
xcv_u16str_append	proven_u16str_append
xcv_u16str_append_grow	proven_u16str_append_grow
xcv_u16str_append_partial	proven_u16str_append_partial
xcv_u16str_as_ptr	proven_u16str_as_ptr
xcv_u16str_create	proven_u16str_create
xcv_u16str_create_from_view	proven_u16str_create_from_view
xcv_u16str_destroy	proven_u16str_destroy
xcv_u16str_len	proven_u16str_len
xcv_u16str_t	proven_u16str_t
xcv_u16str_view_t	proven_u16str_view_t
xcv_u32	proven_u32
xcv_u64	proven_u64
xcv_u8	proven_u8
xcv_u8str_append	proven_u8str_append
xcv_u8str_append_byte	proven_u8str_append_byte
xcv_u8str_append_fmt	proven_u8str_append_fmt
xcv_u8str_append_fmt_grow	proven_u8str_append_fmt_grow
xcv_u8str_append_fmt_trunc	proven_u8str_append_fmt_trunc
xcv_u8str_append_fmt_with_scratch	proven_u8str_append_fmt_with_scratch
xcv_u8str_append_grow	proven_u8str_append_grow
xcv_u8str_append_partial	proven_u8str_append_partial
xcv_u8str_as_cstr	proven_u8str_as_cstr
xcv_u8str_as_view	proven_u8str_as_view
xcv_u8str_borrow	proven_u8str_borrow
xcv_u8str_create	proven_u8str_create
xcv_u8str_create_from_view	proven_u8str_create_from_view
xcv_u8str_destroy	proven_u8str_destroy
xcv_u8str_fmt_internal	proven_u8str_fmt_internal
xcv_u8str_insert	proven_u8str_insert
xcv_u8str_insert_grow	proven_u8str_insert_grow
xcv_u8str_is_valid	proven_u8str_is_valid
xcv_u8str_mut_t	proven_u8str_mut_t
xcv_u8str_remove	proven_u8str_remove
xcv_u8str_replace_at	proven_u8str_replace_at
xcv_u8str_replace_at_grow	proven_u8str_replace_at_grow

xcv_u8str_replace_first	proven_u8str_replace_first
xcv_u8str_reserve	proven_u8str_reserve
xcv_u8str_reset	proven_u8str_reset
xcv_u8str_t	proven_u8str_t
xcv_u8str_view_cmp	proven_u8str_view_cmp
xcv_u8str_view_cmp_ptr	proven_u8str_view_cmp_ptr
xcv_u8str_view_contains	proven_u8str_view_contains
xcv_u8str_view_ends_with	proven_u8str_view_ends_with
xcv_u8str_view_eq	proven_u8str_view_eq
xcv_u8str_view_eq_ascii_ci	proven_u8str_view_eq_ascii_ci
xcv_u8str_view_find	proven_u8str_view_find
xcv_u8str_view_find_last	proven_u8str_view_find_last
xcv_u8str_view_from_bytes	proven_u8str_view_from_bytes
xcv_u8str_view_from_cstr	proven_u8str_view_from_cstr
xcv_u8str_view_is_well_formed	proven_u8str_view_is_well_formed
xcv_u8str_view_remove_prefix	proven_u8str_view_remove_prefix
xcv_u8str_view_remove_suffix	proven_u8str_view_remove_suffix
xcv_u8str_view_slice	proven_u8str_view_slice
xcv_u8str_view_split	proven_u8str_view_split
xcv_u8str_view_split_next	proven_u8str_view_split_next
xcv_u8str_view_split_t	proven_u8str_view_split_t
xcv_u8str_view_starts_with	proven_u8str_view_starts_with
xcv_u8str_view_starts_with_ascii_ci	proven_u8str_view_starts_with_ascii_ci
xcv_u8str_view_t	proven_u8str_view_t
xcv_u8str_view_to_cstr	proven_u8str_view_to_cstr
xcv_u8str_view_trim	proven_u8str_view_trim
xcv_u8str_view_trim_end	proven_u8str_view_trim_end
xcv_u8str_view_trim_start	proven_u8str_view_trim_start
xcv_uintptr_align_up	proven_uintptr_align_up
xcv_uintptr_t	proven_uintptr_t
xcv_url_default_port	proven_url_default_port
xcv_url_effective_port	proven_url_effective_port
xcv_url_encode_component	proven_url_encode_component
xcv_url_encode_path	proven_url_encode_path
xcv_url_encoded_size_max	proven_url_encoded_size_max
xcv_url_form_append	proven_url_form_append
xcv_url_form_decode	proven_url_form_decode
xcv_url_form_encode	proven_url_form_encode
xcv_url_parse	proven_url_parse
xcv_url_path_resolve	proven_url_path_resolve
xcv_url_percent_decode	proven_url_percent_decode
xcv_url_query_iter	proven_url_query_iter
xcv_url_query_iter_t	proven_url_query_iter_t
xcv_url_query_next	proven_url_query_next
xcv_url_resolve	proven_url_resolve

xcv_url_scheme_is	proven_url_scheme_is
xcv_url_split_target	proven_url_split_target
xcv_url_t	proven_url_t
xcv_utf16_append_to_u8str	proven_utf16_append_to_u8str
xcv_utf16_to_utf8	proven_utf16_to_utf8
xcv_utf16_to_utf8_partial	proven_utf16_to_utf8_partial
xcv_utf16_to_utf8_size	proven_utf16_to_utf8_size
xcv_utf8_append_to_u16str	proven_utf8_append_to_u16str
xcv_utf8_char_t	proven_utf8_char_t
xcv_utf8_decode_next	proven_utf8_decode_next
xcv_utf8_encode_one	proven_utf8_encode_one
xcv_utf8_validate	proven_utf8_validate
xcv_utf8_to_utf16	proven_utf8_to_utf16
xcv_utf8_to_utf16_partial	proven_utf8_to_utf16_partial
xcv_utf8_to_utf16_size	proven_utf8_to_utf16_size
xcv_utf_step_t	proven_utf_step_t
xcv_writer_buffered	proven_writer_buffered
xcv_writer_flush	proven_writer_flush
xcv_writer_from_buffer	proven_writer_from_buffer
xcv_writer_from_file	proven_writer_from_file
xcv_writer_from_u8str	proven_writer_from_u8str
xcv_writer_is_valid	proven_writer_is_valid
xcv_writer_write	proven_writer_write
xcv_writer_write_bom	proven_writer_write_bom
xcv_writer_write_partial	proven_writer_write_partial
xcv_writer_write_str	proven_writer_write_str
xcv_writer_write_u16	proven_writer_write_u16
xcv_ws_accept_key	proven_ws_accept_key
xcv_ws_check_request	proven_ws_check_request
xcv_ws_check_response	proven_ws_check_response
xcv_ws_check_response_deflate	proven_ws_check_response_deflate
xcv_ws_check_response_peer_no_takeover	proven_ws_check_response_peer_no_takeover
xcv_ws_check_response_takeover	proven_ws_check_response_takeover
xcv_ws_close_code_for	proven_ws_close_code_for
xcv_ws_close_code_is_valid	proven_ws_close_code_is_valid
xcv_ws_close_parse	proven_ws_close_parse
xcv_ws_close_write	proven_ws_close_write
xcv_ws_conn_accept	proven_ws_conn_accept
xcv_ws_conn_close	proven_ws_conn_close
xcv_ws_conn_close_code	proven_ws_conn_close_code
xcv_ws_conn_close_reason	proven_ws_conn_close_reason
xcv_ws_conn_compressed	proven_ws_conn_compressed
xcv_ws_conn_config_t	proven_ws_conn_config_t
xcv_ws_conn_connect	proven_ws_conn_connect
xcv_ws_conn_destroy	proven_ws_conn_destroy

xcv_ws_conn_open	proven_ws_conn_open
xcv_ws_conn_ping	proven_ws_conn_ping
xcv_ws_conn_pong_count	proven_ws_conn_pong_count
xcv_ws_conn_protocol	proven_ws_conn_protocol
xcv_ws_conn_receive	proven_ws_conn_receive
xcv_ws_conn_send_binary	proven_ws_conn_send_binary
xcv_ws_conn_send_part	proven_ws_conn_send_part
xcv_ws_conn_send_text	proven_ws_conn_send_text
xcv_ws_conn_set_compress	proven_ws_conn_set_compress
xcv_ws_conn_t	proven_ws_conn_t
xcv_ws_decoder_allow_compressed	proven_ws_decoder_allow_compressed
xcv_ws_decoder_feed	proven_ws_decoder_feed
xcv_ws_decoder_init	proven_ws_decoder_init
xcv_ws_decoder_t	proven_ws_decoder_t
xcv_ws_deflate_accept	proven_ws_deflate_accept
xcv_ws_deflate_accept_peer_no_takeover	proven_ws_deflate_accept_peer_no_takeover
xcv_ws_deflate_accept_takeover	proven_ws_deflate_accept_takeover
xcv_ws_deflate_terms_t	proven_ws_deflate_terms_t
xcv_ws_event_accept	proven_ws_event_accept
xcv_ws_event_callbacks_t	proven_ws_event_callbacks_t
xcv_ws_event_config_t	proven_ws_event_config_t
xcv_ws_event_connect	proven_ws_event_connect
xcv_ws_event_kind_t	proven_ws_event_kind_t
xcv_ws_event_t	proven_ws_event_t
xcv_ws_frame_parse	proven_ws_frame_parse
xcv_ws_frame_t	proven_ws_frame_t
xcv_ws_frame_write	proven_ws_frame_write
xcv_ws_make_key	proven_ws_make_key
xcv_ws_mask	proven_ws_mask
xcv_ws_message_t	proven_ws_message_t
xcv_ws_opcode_t	proven_ws_opcode_t
xcv_ws_check_origin	proven_ws_check_origin
xcv_ws_request_offers	proven_ws_request_offers
xcv_ws_stream_abort	proven_ws_stream_abort
xcv_ws_stream_buffered	proven_ws_stream_buffered
xcv_ws_stream_close	proven_ws_stream_close
xcv_ws_stream_compressed	proven_ws_stream_compressed
xcv_ws_stream_pause	proven_ws_stream_pause
xcv_ws_stream_peer	proven_ws_stream_peer
xcv_ws_stream_ping	proven_ws_stream_ping
xcv_ws_stream_resume	proven_ws_stream_resume
xcv_ws_stream_send	proven_ws_stream_send
xcv_ws_stream_send_piece	proven_ws_stream_send_piece
xcv_ws_stream_set_compress	proven_ws_stream_set_compress
xcv_ws_stream_set_user	proven_ws_stream_set_user

xcv_ws_stream_t	proven_ws_stream_t
xcv_ws_stream_user	proven_ws_stream_user
xcv_xoshiro256ss_next	proven_xoshiro256ss_next
xcv_xoshiro256ss_rng	proven_xoshiro256ss_rng
xcv_xoshiro256ss_seed	proven_xoshiro256ss_seed
xcv_zstd_decode	proven_zstd_decode
xcv_zstd_decode_all	proven_zstd_decode_all
xcv_zstd_decoder_create	proven_zstd_decoder_create
xcv_zstd_decoder_destroy	proven_zstd_decoder_destroy
xcv_zstd_decoder_reset	proven_zstd_decoder_reset
xcv_zstd_decoder_t	proven_zstd_decoder_t
xcv_zstd_encode	proven_zstd_encode
xcv_zstd_encode_all	proven_zstd_encode_all
xcv_zstd_encode_bound	proven_zstd_encode_bound
xcv_zstd_encoder_create	proven_zstd_encoder_create
xcv_zstd_encoder_destroy	proven_zstd_encoder_destroy
xcv_zstd_encoder_options_t	proven_zstd_encoder_options_t
xcv_zstd_encoder_reset	proven_zstd_encoder_reset
xcv_zstd_encoder_t	proven_zstd_encoder_t
xcv_zstd_flush_t	proven_zstd_flush_t